

# **ORBITER**

## **Classification of Combinatorial Objects**

User's Guide  
Build Number 3189

Anton Betten

March 9, 2024



# Contents

|          |                                                 |            |
|----------|-------------------------------------------------|------------|
| <b>1</b> | <b>Introduction</b>                             | <b>7</b>   |
| 1.1      | What is Orbiter . . . . .                       | 7          |
| 1.2      | Acknowledgements . . . . .                      | 8          |
| 1.3      | Installing Orbiter for the First Time . . . . . | 9          |
| 1.4      | Updating Orbiter . . . . .                      | 12         |
| 1.5      | The Work Directory . . . . .                    | 13         |
| <b>2</b> | <b>The Orbiter Language</b>                     | <b>15</b>  |
| 2.1      | The Orbiter Session . . . . .                   | 15         |
| 2.2      | Objects and Activities . . . . .                | 18         |
| 2.3      | Mathematical Data . . . . .                     | 23         |
| 2.4      | Set Builder . . . . .                           | 25         |
| 2.5      | Vector Builders . . . . .                       | 26         |
| 2.6      | Vector of Group Elements . . . . .              | 31         |
| 2.7      | Symbolic Objects . . . . .                      | 33         |
| <b>3</b> | <b>Finite Fields</b>                            | <b>53</b>  |
| 3.1      | Prime Fields . . . . .                          | 53         |
| 3.2      | Extension Fields . . . . .                      | 59         |
| 3.3      | Linear Algebra Over Finite Fields . . . . .     | 71         |
| 3.4      | Advanced Topics in Finite Fields . . . . .      | 77         |
| 3.5      | Basic Ring Theory . . . . .                     | 85         |
| <b>4</b> | <b>Geometry</b>                                 | <b>87</b>  |
| 4.1      | Finite Projective Spaces . . . . .              | 87         |
| 4.2      | Indexing Points and Lines . . . . .             | 93         |
| 4.3      | Incidence Matrices . . . . .                    | 97         |
| 4.4      | The Grassmannian . . . . .                      | 103        |
| 4.5      | Algebraic Sets . . . . .                        | 106        |
| 4.6      | The Klein Quadric and the Plücker Map . . . . . | 109        |
| 4.7      | Orthogonal Spaces . . . . .                     | 110        |
| 4.8      | Hermitian Varieties . . . . .                   | 118        |
| 4.9      | Advanced Topics . . . . .                       | 120        |
| 4.10     | Geometric Objects . . . . .                     | 131        |
| <b>5</b> | <b>Ring Theory</b>                              | <b>139</b> |
| 5.1      | Polynomials Over Finite Fields . . . . .        | 139        |
| 5.2      | Multivariate Polynomial Rings . . . . .         | 147        |
| 5.3      | Algebraic Geometry . . . . .                    | 158        |

|           |                                                           |            |
|-----------|-----------------------------------------------------------|------------|
| <b>6</b>  | <b>Group Theory</b>                                       | <b>163</b> |
| 6.1       | Generic Permutation Groups . . . . .                      | 163        |
| 6.2       | Matrix Groups . . . . .                                   | 173        |
| 6.3       | Subgroups . . . . .                                       | 193        |
| 6.4       | New Actions from Old . . . . .                            | 206        |
| 6.5       | Group Theoretic Activities . . . . .                      | 216        |
| 6.6       | Group Theoretic Activities Based on Magma . . . . .       | 226        |
| 6.7       | Group Theoretic Activities Based on GAP . . . . .         | 235        |
| 6.8       | Linear Groups, Advanced Topics . . . . .                  | 241        |
| <b>7</b>  | <b>Orbit Algorithms</b>                                   | <b>245</b> |
| 7.1       | Schreier Trees . . . . .                                  | 245        |
| 7.2       | Poset Classification . . . . .                            | 251        |
| 7.3       | Orbits on Subsets . . . . .                               | 256        |
| 7.4       | Orbits on Subspaces . . . . .                             | 260        |
| 7.5       | Orbits on Set-Partitions . . . . .                        | 263        |
| 7.6       | Arcs and Caps in Projective Spaces . . . . .              | 265        |
| 7.7       | Cubic Curves . . . . .                                    | 269        |
| <b>8</b>  | <b>Cubic Surfaces and Quartic Curves</b>                  | <b>275</b> |
| 8.1       | Generalities . . . . .                                    | 275        |
| 8.2       | Creating Cubic Surfaces . . . . .                         | 278        |
| 8.3       | Cubic Surface Activities . . . . .                        | 284        |
| 8.4       | Classification of Cubic Surfaces . . . . .                | 289        |
| 8.5       | Quartic Curves . . . . .                                  | 304        |
| 8.6       | Classification of Quartic Curves . . . . .                | 316        |
| 8.7       | Interface to GAP . . . . .                                | 320        |
| <b>9</b>  | <b>Applications</b>                                       | <b>323</b> |
| 9.1       | Number Theory . . . . .                                   | 323        |
| 9.2       | Representation Theory . . . . .                           | 335        |
| 9.3       | Cryptography . . . . .                                    | 337        |
| <b>10</b> | <b>Coding Theory</b>                                      | <b>347</b> |
| 10.1      | Introduction . . . . .                                    | 347        |
| 10.2      | Linear Codes . . . . .                                    | 353        |
| 10.3      | Golay Codes . . . . .                                     | 364        |
| 10.4      | CRC Codes . . . . .                                       | 365        |
| 10.5      | Reed-Muller Codes . . . . .                               | 370        |
| 10.6      | BCH Codes . . . . .                                       | 377        |
| 10.7      | Reed-Solomon Codes . . . . .                              | 384        |
| 10.8      | Twisted Tensor Product Codes . . . . .                    | 389        |
| 10.9      | Bounds . . . . .                                          | 391        |
| 10.10     | Classification of Optimal Linear Codes . . . . .          | 395        |
| <b>11</b> | <b>Combinatorics</b>                                      | <b>397</b> |
| 11.1      | Introduction . . . . .                                    | 397        |
| 11.2      | Combinatorial Objects . . . . .                           | 404        |
| 11.3      | Combinatorial Linear Spaces . . . . .                     | 412        |
| 11.4      | Classification of Configurations and Geometries . . . . . | 415        |
| 11.5      | Tactical Decompositions . . . . .                         | 424        |



|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>12 Graph Theory</b>                                        | <b>427</b> |
| 12.1 Creating Graphs . . . . .                                | 427        |
| 12.2 Graph Theoretic Activities . . . . .                     | 435        |
| 12.3 Classification of Graphs and Tournaments . . . . .       | 440        |
| <b>13 Design Theory</b>                                       | <b>445</b> |
| 13.1 Creating Designs . . . . .                               | 445        |
| 13.2 Assuming Symmetry . . . . .                              | 453        |
| 13.3 Design Theory – Large Sets . . . . .                     | 458        |
| 13.4 Design Theory – Delandtsheer-Doyen . . . . .             | 461        |
| <b>14 Finite Geometry</b>                                     | <b>465</b> |
| 14.1 Spreads . . . . .                                        | 465        |
| 14.2 Translation Planes . . . . .                             | 480        |
| 14.3 Packings . . . . .                                       | 485        |
| 14.4 BLT-Sets . . . . .                                       | 490        |
| <b>15 Computer Science Primitives</b>                         | <b>499</b> |
| 15.1 Clique Finding . . . . .                                 | 499        |
| 15.2 Rainbow Cliques . . . . .                                | 503        |
| 15.3 Diophantine Systems . . . . .                            | 506        |
| <b>16 Canonical Forms with Nauty</b>                          | <b>511</b> |
| 16.1 Overview of Canonical Forms . . . . .                    | 511        |
| 16.2 Canonical Forms of Objects in Projective Space . . . . . | 513        |
| 16.3 Canonical Forms of Incidence Geometries . . . . .        | 522        |
| 16.4 Canonical Forms of Objects from Design Theory . . . . .  | 537        |
| 16.5 Canonical Forms of Linear Codes . . . . .                | 541        |
| 16.6 Canonical Forms of General Codes . . . . .               | 548        |
| 16.7 Canonical Forms of Graphs . . . . .                      | 549        |
| 16.8 Canonical Forms of Quartic Curves . . . . .              | 554        |
| <b>17 Interfaces</b>                                          | <b>557</b> |
| 17.1 Graphical Output . . . . .                               | 557        |
| 17.2 The Povray Interface . . . . .                           | 561        |
| 17.3 Creating Animations . . . . .                            | 574        |
| 17.4 Continuous Function Plotter . . . . .                    | 575        |
| 17.5 The Gnuplot Interface . . . . .                          | 579        |
| <b>18 Mathematical Data in Orbiter</b>                        | <b>583</b> |
| 18.1 Cubic Surfaces . . . . .                                 | 583        |
| 18.2 BLT-Sets . . . . .                                       | 585        |
| <b>19 Miscellaneous</b>                                       | <b>587</b> |
| 19.1 Miscellaneous . . . . .                                  | 587        |
| 19.2 Limitations . . . . .                                    | 591        |
| <b>20 Orbiter on Windows</b>                                  | <b>595</b> |
| 20.1 Using Windows Subsystem Linux . . . . .                  | 595        |
| <b>21 Templates</b>                                           | <b>605</b> |
| 21.1 Templates . . . . .                                      | 605        |
| <b>A Data on Cubic Surfaces</b>                               | <b>609</b> |
| A.1 Data on Cubic Surfaces . . . . .                          | 609        |

|          |                                  |            |
|----------|----------------------------------|------------|
| <b>B</b> | <b>The Class Library</b>         | <b>631</b> |
| B.1      | Classes and Namespaces . . . . . | 631        |
| <b>C</b> | <b>The Makefile</b>              | <b>643</b> |
| C.1      | The Makefile . . . . .           | 643        |
|          | References . . . . .             | 1046       |

# Chapter 1

## Introduction

### 1.1 What is Orbiter

This is the user's guide of the computer algebra system Orbiter. For the most recent version of this guide, please visit

[https://www.math.colostate.edu/~betten/orbiter/users\\_guide.pdf](https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf)

For the github repository, see

<https://github.com/abetten/orbiter>

Orbiter is a computer algebra system for the classification of combinatorial objects. Orbiter contributes to the knowledge base of combinatorial structures, and to provide useful tools to investigate structures from various points of view, including their symmetry properties. For background on Orbiter, see [10].

Orbiter is open source, and can be downloaded and installed from github. A C++ compiler environment similar to gnu gcc and make are required. Compiler tools like bison and flex are required also. Latex is recommended.

## 1.2 Acknowledgements

The following people contributed to Orbiter:

1. Abdullah Alazemi
2. Sajeeb Ray Chowdhuri
3. Fatma Karaoglu

The following software packages found their way into Orbiter:

1. Nauty [52] (Brendan McKay)
2. Possolve (Brendan McKay)
3. Eigen (Benoît Jacob and Gaël Guennebaud)
4. Pugixml (Arseny Kapoulkine, Kristen Wegner)
5. EasyBMP (Paul Macklin)
6. Part (Jeffrey Leon)

## 1.3 Installing Orbiter for the First Time

To install Orbiter, we first check if git is installed. Type

```
git
```

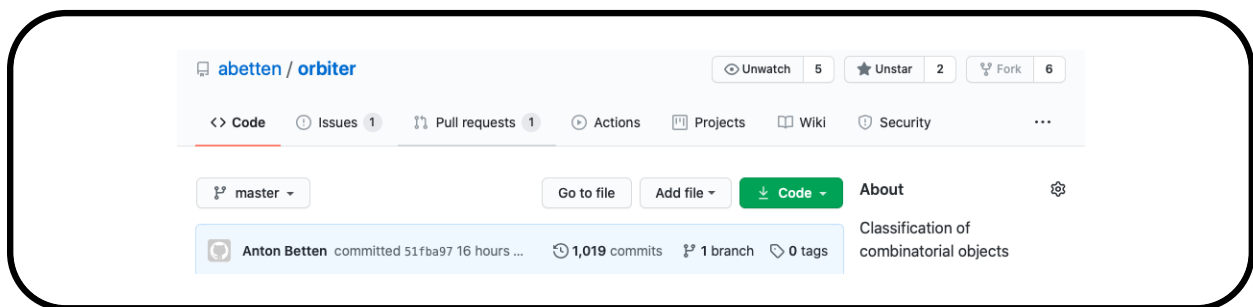
If you get an error message, please install git. If you get a long output with commands, you are fine. Git is installed.

The steps to install Orbiter differ slightly, depending on your operating system. For Windows, see Section 20.1. On Linux/unix based systems, including Macintosh, you will utilize a terminal window. You will also need the development tools. You need the C++ compiler and library tools. On Macintosh, you need Xcode and the command-line tools. You can get them for free on the App-store. You will also need the compiler generator tools flex and bison. They are open source projects. On Macintosh, you can use the package manager brew. On Linux, simply use your package manager to install these tools.

To install Orbiter, you first need to download the source code from github. So, navigate your web browser to

<https://github.com/abetten/orbiter>

Click on the green box that says “Code” and copy the address into the clipboard by clicking the clipboard symbol, see below:



Click on “Code” and copy paste the address. Then type

```
git clone XXX
```

Where XXX is to be replaced with the address that you copied (simply use CTRL-P to paste).

Once the command finishes, you need to check that the prerequisites are satisfied, which are

1. flex
2. bison

These are compiler generator tools, and they are required for compiling Orbiter. Unfortunately, many OS versions come with outdated versions of bison and flex, so we need to check the version that is installed (or install it in the first place). To check the version of bison, type

```
bison --version
```

If you get a message `bison not found` it means that you don't have bison. In this case you need to install bison. Otherwise, check out the version number. Orbiter requires anything above 3.8.2. If your version number is below that, you need to upgrade your bison software. Regarding flex, type

```
flex --version
```

If you get a message `flex not found` it means that you don't have flex. In this case you need to install flex. Otherwise, check out the version number. Orbiter requires anything above 2.6.4. If your version number is below that, you need to upgrade your flex software.

Next, we will show how to install bison and flex, and possibly update the version that is installed. On a Macintosh system, the package manager brew can be used. On Linux, different package managers exist. In either case, we have to also update certain shell variables, so that the correct version of flex and bison is used during the Orbiter build process.

On a Macintosh, using cshell (csh), you can install bison using "brew." Type

```
brew install bison
```

The install will go to

```
/opt/homebrew/Cellar
```

Go there and check whether a directory bison exists. If so, do

```
cd bison
```

and

```
ls
```

and note the version number. You may still have an old version of bison somewhere else. In order to activate the new bison and not the old, you will have to change the path in your shell. For csh, make sure you add the line

```
set path = ( /opt/homebrew/Cellar/bison/3.8.2/bin $path )
```

Here, the 3.8.2. part may have to be changed depending on your version of bison.

You can install flex using brew. Type

```
brew install flex
```

The install will go to

```
/opt/homebrew/Cellar
```

Go there and check whether a directory flex exists. If so, do

```
cd flex
```

and

```
ls
```

and note the version number. Like with bison earlier, you may still have an old version of flex somewhere else. In order to activate the new flex and not the old, you will have to change the path in your shell. For csh, make sure you add the line

```
set path = ( /opt/homebrew/Cellar/bison/2.6.4_2/bin $path )
```

Here, the 2.6.4\_2 part may have to be changed depending on your version of bison.

Finally, you have to add

```
set LDFLAGS = ( -L/usr/local/Cellar/flex/2.6.4_2/lib -L/usr/local/Cellar/bison/3.8.2/lib )
```

to your `~/.cshrc`. Here, the version numbers may have to be changed to match those that you used before.

After these changes, you need to read the `~/.cshrc` file again and rehash. Type

```
read ~/.cshrc
```

and

```
rehash
```

If you type `bison --version`, you should get the correct version number of bison. Likewise, if you type `flex --version`, you should get the correct version number of flex.

You are now ready to compile Orbiter. For this purpose, you need to enter the orbiter home directory. Assuming that you are still in the directory where you did git clone, type

```
cd orbiter
```

and then

```
make
```

If your computer CPU has multiple cores, you can try

```
make -j 8
```

The number 8 can be adjusted to the number of cores that you have. This multitasking reduces the time it takes to compile Orbiter. Once done, you should have an orbiter executable in the directory

```
orbiter/src/apps/orbiter
```

The executable is called

```
orbiter.out
```

Note that you are already in orbiter, so you could do

```
ls src/apps/orbiter
```

to check if `orbiter.out` appears. In either case, you can do

```
cd ..
```

to leave the orbiter branch. You should only go back in the Orbiter branch if you want to update Orbiter. Otherwise, you should not be there. In particular, you should not change any files in this branch, as this would cause problems in the update.

Finally, if you have to update bison or flex, you can type

```
brew upgrade bison
```

or

```
brew upgrade flex
```

## 1.4 Updating Orbiter

To update Orbiter, we first check if git is installed. Type

```
git
```

If you get an error message, please install git. If you get a long output with commands, you are fine. Git is installed.

Next, we enter the `orbiter` directory. This is the directory that was created by `git clone`.

```
cd orbiter
```

Now type

```
git pull
```

to update the Orbiter sources. If you get an error message to the effect that you should stash something first, it means that you have made changes inside the orbiter directory. Most likely, you edited a file in the Orbiter directory tree by chance. In this case, it is best to let go of these changes by issuing

```
git reset --hard
```

and then doing

```
git pull
```

again. Then type

```
make clean
```

and

```
make
```

or

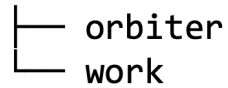
```
make -j 8
```

for parallel compile (if necessary, change the 8). You should get a new Orbiter executable.



## 1.5 The Work Directory

The orbiter directory is under the control of git, whose purpose it is to keep the sources up-to-date. The user should not make any changes in this directory, as it might cause trouble with updating Orbiter through git. It is recommended to create a separate directory for work, *outside the orbiter directory*. For the following, we assume that a parallel directory work has been created, so that the directory tree structure looks like this:



There is a large makefile in

`orbiter/examples/users_guide`

containing all commands from this user's guide (listed in Appendix C). It would be good to copy this makefile into the work directory. Then, all commands in this user's guide can be executed immediately from within the work directory. This way, the orbiter directory stays clean, so github will not complain during any future updates of Orbiter.



## Chapter 2

# The Orbiter Language

### 2.1 The Orbiter Session

Orbiter is a command line program. There is no graphical user interface. The Orbiter executable is `orbiter.out`. It needs to be called from a terminal window, through the command shell. Orbiter takes commands from the command line for execution. The language is called Orbiter dash code. This refers to the fact that keywords are indicated with a dash, just like in the unix / linux command syntax. This user's guide will describe the orbiter dash code language.

Orbiter dash code programs can be quite long, and command names are typically lengthy, in order to make the code more readable. It is recommended to store the command sequence in either a makefile or in a shell script. Makefiles have the advantage that they can hold many command sequences, each with a different call name. This reduces clutter, as all these command sequences are stored in one file. On the other hand, makefiles come with some very stringent syntax requirements, so there is a learning curve.

Let us take a closer look at an Orbiter session. As a test, we wish to run orbiter just for fun, without any arguments. To run Orbiter, we can use a makefile entry like this:

#### Example 1

```
test_orbiter_session:  
▷ $(ORBITER)
```

Once started, the Orbiter session will produce a short welcome message:

```
Welcome to Orbiter! Your build number is 1081.  
A user's guide is available here:  
https://www.math.colostate.edu/~betten/orbiter/users\_guide.pdf  
The sources are available here:  
https://github.com/abetten/orbiter  
An example makefile with many commands from the user's guide is here:  
https://github.com/abetten/orbiter/tree/master/examples/users\_guide/makefile  
Orbiter session finished.  
User time: 0:00
```

The build number is the version number of the Orbiter software, as defined by the number of submits to the Git repository. Higher numbers mean more recent versions. After this message, Orbiter will start parsing

the command line arguments. Once this is done, the session will execute these commands. At the end of the session, a short message is given that specifies the processor time used up by the session.

Several commands are global, and must appear right at the beginning of the orbiter dash code. These are the session commands. They are listed in Table 2.1.

| Orbiter session commands              |             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                               | Arguments   | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| -v                                    | $v$         | Set verbosity to $v$ . Larger values of $v$ lead to more text output. $v = 0$ gives minimal to no output.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| -draw_options                         | options     | Set options for drawing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| -draw_incidence_structure_description | options     | Set options for drawing of incidence structures.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| -list_arguments                       |             | Prints the command line arguments.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| -seed                                 | $s$         | Seed the pseudo random number generator with the integer value $s$ .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| -memory_debug                         |             | Turn on dynamic memory debugging.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| -override_polynomial                  | poly        | Set the override polynomial for finite fields to poly.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| -orbiter_path                         | $p$         | Set the orbiter path to $p$ . This is useful in case the Orbiter session has to clone or fork new Orbiter sessions. In most cases, the orbiter path will end with a forward slash “/.”                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| -magma_path                           | $p$         | Set the magma path to $p$ . This is useful in case the Orbiter session has to create a magma process.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| -fork                                 | $L M s u d$ | Fork new Orbiter sessions in parallel. The new sessions will be indexed by the values $i = s + kd$ , where $k = 0, 1, \dots$ subject to $i < u$ . Every occurrence of the string $L$ in the argument list is replaced by the resulting value of the loop variable $i$ . The forked process will write to a file. The filename asis from $M$ using by replacing %d with the value of $i$ (similar to a printf command in C programming). All of the command line arguments after the fork command are passed through to the new Orbiter session, with all arguments $L$ replaced by the integer value of the loop counter. The number of Orbiter sessions forked is $(u - s)/d$ . The orbiter path is considered when starting the forked sessions. |

Table 2.1: Orbiter session commands

## 2.2 Objects and Activities

The majority of work in Orbiter is done by means of objects and activities. This follows the object oriented paradigm of programming, realized in the C++ programming language, which is the language in which Orbiter is written. Objects hold data and can perform tasks, which in Orbiter are called activities. This leaves two questions:

1. How are objects created?
2. What activities exist?

There are many different types of objects, and each has specific requirements. Also, the types of activities depend on the types of objects. This user's guide will answer the two questions one-by-one, by going over the different types of objects that exist.

The syntax to create an object is

`-define LABEL KEYWORD PARAMETERS -end`

Here, *LABEL* is any label under which the object is stored in the symbol table. Any object with the same label already in the symbol table will be overwritten. The *KEYWORD* can be any of the commands in Tables 2.2-2.3. The *PARAMETERS* depend on the type of object created. the command `-end` is necessary to terminate the definition command. For more details on the objects that exist, see the appropriate section listed in the table.

For instance, the command

### Example 2

```
object_F_2:
▷ $(ORBITER) -v 3 -define F -finite_field -q 2 -end
```

creates a finite field object  $F$  for the field  $\mathbb{F}_2$ , the field with two elements. For more details, see Section 3.1. The command

### Example 3

```
object_PG_3_2:
▷ $(ORBITER) \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end
```

creates the same finite field  $\mathbb{F}_2$  and uses it to construct  $\text{PG}(3, 2)$ . The `-projective_space` command requires additional options to set the dimension  $n$  and the field  $\mathbb{F}_q$  in  $\text{PG}(n, q)$ . The `-n` command sets the dimension  $n$ . The `-field` command can be used to specify a particular field. The `-q` command can be used to create a generic field  $\mathbb{F}_q$ .

In order to do something with an object, we need to invoke an *activity*. To select an object for an activity, the

`-with LABEL -do DESCRIPTION -end`

command sequence is used. Here, *LABEL* is the name under which the object is registered in the symbol table. *DESCRIPTION* is the activity that should be applied. Some activities require more than one object, in which case the syntax

`-with LABEL1 -and LABEL2 -do DESCRIPTION -end`

| Orbiter Objects (Part I)          |                      |                                                                 |
|-----------------------------------|----------------------|-----------------------------------------------------------------|
| Command                           | Arguments            | Purpose                                                         |
| -finite_field                     | Sections 3.1 and 3.2 | A finite field $\mathbb{F}_q$ .                                 |
| -polynomial_ring                  | Section 5.2          | A multivariate polynomial ring.                                 |
| -projective_space                 | Section 4.1          | A projective space $\text{PG}(n, q)$ .                          |
| -orthogonal_space                 | Section 4.7          | A non-degenerate orthogonal space $O^\epsilon(n, q)$ .          |
| -BLT_set_classifier               | Section 14.4         | An object to classify BLT-sets.                                 |
| -spread_classifier                | Section 14.1         | An object to classify spreads.                                  |
| -linear_group                     | Section 6.2          | A linear group.                                                 |
| -permutation_group                | Section 6.1          | A generic permutation group.                                    |
| -modified_group                   | Section 6.4          | A new group from an old.                                        |
| -geometric_object                 | Section 4.10         | A geometric object.                                             |
| -collection                       |                      | A collection of objects.                                        |
| -graph                            | Section 12.1         | A graph.                                                        |
| -code                             | Section 10.2         | A code.                                                         |
| -spread                           | Section 14.1         | A spread.                                                       |
| -cubic_surface                    | Section 8.2          | A cubic surface.                                                |
| -quartic_curve                    | Section 8.5          | A quartic curve.                                                |
| -BLT-set                          | Section 14.4         | A BLT-set.                                                      |
| -translation_plane                | Section 14.2         | A translation plane.                                            |
| -spread_table                     | Section 14.3         | A table of spreads.                                             |
| -packing_with_symmetry_assumption | Section 14.3         | A generator for packings with assumed symmetry.                 |
| -packing_choose_fixed_points      | Section 14.3         | A selection of fixed orbits for packings with assumed symmetry. |

Table 2.2: Orbiter Objects (Part I)

| Orbiter Objects (Part II)           |                |                                                                                                                                                            |
|-------------------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                             | Arguments      | Purpose                                                                                                                                                    |
| -packing_long_orbits                | Section 14.3   | A search for long orbits for packings with assumed symmetry.                                                                                               |
| -graph_classification               | Section 12.3   | An object which allows classifying graphs and tournaments.                                                                                                 |
| -diophant                           | Section 15.3   | A diophantine system, i.e., a system of positive integer equations.                                                                                        |
| -design                             | Section 13.1   | A combinatorial design.                                                                                                                                    |
| -design_table                       | Section 13.1   | A table of designs, possibly isomorphic. It can be used to construct large sets of designs. A large set is a set of designs satisfying certain properties. |
| -large_set_with_symmetry_assumption | Section 13.1   | An object to create a large set of designs.                                                                                                                |
| -set                                | Section 2.4    | A set of integers.                                                                                                                                         |
| -vector                             | Section 2.5    | A vector of elements of a finite field.                                                                                                                    |
| -symbolic_object                    | Section 2.7    | A symbolic algebraic expression.                                                                                                                           |
| -combinatorial_object               | Section 11.2   | A set of combinatorial objects.                                                                                                                            |
| -geometry_builder                   | Section 11.4   | An object to classify incidence geometries.                                                                                                                |
| -vector_ge                          | Section 6.3    | A vector of group elements.                                                                                                                                |
| -action_on_forms                    |                | The action of a projective group on homogeneous polynomials of a given degree.                                                                             |
| -orbits                             | Section 7.1    | An object to compute orbits of a group.                                                                                                                    |
| -poset_classification_control       | Section 7.2    | An object to set options for the poset classification algorithm.                                                                                           |
| -arc_generator_control              | See Table 7.6. | An object to set options for the arc classification algorithm.                                                                                             |
| -poset_classification_activity      | Section 14.4   | An object to define an activity for a poset of orbits.                                                                                                     |
| -crc_code                           | Section 10.6   | An object to define CRC-code.                                                                                                                              |
| -mapping                            | See Table 5.6  | An object to define a mapping.                                                                                                                             |

Table 2.3: Orbiter Objects (Part II)



| Orbiter Activities (Part 1)    |                      |                                         |
|--------------------------------|----------------------|-----------------------------------------|
| Command                        | Arguments            | Purpose                                 |
| -finite_field_activity         | Sections 3.1 and 3.2 | An activity for finite fields.          |
| -polynomial_ring_activity      | Table 5.3            | An activity for a polynomial ring.      |
| -projective_space_activity     | Section 4.1          | An activity for a projective space.     |
| -orthogonal_space_activity     | Section 4.7          | An activity for an orthogonal space.    |
| -group_theoretic_activity      | Section 6.5          | An activity for a group.                |
| -coding_theoretic_activity     | Tables 10.3-10.5     | An activity for a code.                 |
| -cubic_surface_activity        | Section 8.2          | An activity for a cubic surface.        |
| -quartic_curve_activity        | Section 8.5          | An activity for a quartic curve.        |
| -blt_set_activity              | Table 14.16          | An activity for a BLT-set.              |
| -combinatorial_object_activity | Section 4.5          | An activity for a combinatorial object. |

Table 2.4: Orbiter Activities (Part 1)

is used. Here, *LABEL1* and *LABEL2* are the objects for which the activity is invoked. For an example of an activity requiring two objects, see Sections 14.1 and 14.2.

Tables 2.4-2.5 list the possible activities for Orbiter objects. More details about each of these activities will be given in the sections below.

| Orbiter Activities (Part 2)                                      |              |                                                                                    |
|------------------------------------------------------------------|--------------|------------------------------------------------------------------------------------|
| Command                                                          | Arguments    | Purpose                                                                            |
| -graph_theoretic_activity                                        | Section 12.1 | An activity for a graph.                                                           |
| -classification_of_cubic_surfaces_<br>with_double_sixes_activity | Section 8.2  | An activity for a cubic surface.                                                   |
| -spread_table_activity                                           | Section 14.3 | An activity associated with a table of spreads.                                    |
| -packing_with_symmetry_<br>assumption_activity                   | Section 14.3 | An activity related to creating packings with assumed symmetry group.              |
| -packing_fixed_points_activity                                   | Section 14.3 | An activity related to creating packings with assumed symmetry group.              |
| -graph_classification_activity                                   | Section 12.3 | An activity for a classification of graphs problem.                                |
| -diophant_activity                                               | Section 15.3 | An activity for a diophantine system.                                              |
| -design_activity                                                 | Section 13.1 | An activity for a combinatorial design.                                            |
| -large_set_with_symmetry_<br>assumption_activity                 | Section 13.3 | An activity related to creating large sets of designs with assumed symmetry group. |
| -symbolic_object_activity                                        | Table 2.12   | An activity related to a symbolic object.                                          |
| -BLT_set_classify_activity                                       | Table 14.18  | An activity related to a classification of BLT-sets.                               |
| -spread_classify_activity                                        | Table 14.4   | An activity related to a classification of spreads.                                |
| -spread_activity                                                 | Table 14.2   | An activity related to a spread.                                                   |
| -translation_plane_activity                                      | Table 14.9   | An activity related to a translation plane.                                        |
| -orbits_activity                                                 | Table 7.2    | An activity related to orbits.                                                     |

Table 2.5: Orbiter Activities (Part 2)

| Mathematical Data in Orbiter |                                                                                                                                                                                                                                                                                           |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Object                       | Description                                                                                                                                                                                                                                                                               |
| BLT-sets                     | BLT sets of $Q(4, q)$ exist for all odd prime powers. The classification of BLT-sets of $Q(4, q)$ is known to Orbiter for all $q \leq 73$ .                                                                                                                                               |
| Cubic Surfaces               | Cubic surfaces with 27 lines exist for all finite fields apart from $\mathbb{F}_2, \mathbb{F}_3, \mathbb{F}_5$ . Orbiter knows the classification of cubic surfaces with 27 lines for all fields $\mathbb{F}_q$ of order $q \leq 128$ .                                                   |
| Quartic curves               | Orbiter knows the classification of smooth quartic curves with 28 bitangents in projective planes over all fields $\mathbb{F}_q$ for $q = 9, 13, 19, 25, 27, 29, 31$ .                                                                                                                    |
| Spreads                      | A spread is a set of $q^k + 1$ pairwise non-intersecting $k$ -dimensional subspaces of $\mathbb{F}_q^{2k}$ . Spreads are related to translation planes of order $q^k$ . Orbiter knows the classification of spreads for $(q, k) \in \{(2, 2), (3, 2), (2, 4), (4, 2), (5, 2), (3, 3)\}$ . |
| Hyperovals                   | A hyperoval in $\text{PG}(2, 2^e)$ is a set of $2^e + 2$ points, no three collinear. Orbiter knows the classification of hyperovals for $e = 3, 4, 5$ .                                                                                                                                   |
| Dual hyperovals              | A $k$ -dimensional dual hyperoval in an ambient space $\mathbb{F}_2^n$ is called a $\text{DH}(k, n)$ . Orbiter knows the classification of dual hyperovals $\text{DH}(4, 7)$ and $\text{DH}(4, 8)$ .                                                                                      |
| Packings                     | Orbiter knows the classification of packings of $\text{PG}(3, 3)$ .                                                                                                                                                                                                                       |

Table 2.6: Mathematical Data in Orbiter

## 2.3 Mathematical Data

Orbiter serves as a repository for mathematical data. The knowledge base is concerned with classifications of geometric and combinatorial objects for small parameters. The types of objects for which a classification is available in Orbiter are listed in Table 2.6.

The mathematical objects are stored in a catalogue, together with generators for their automorphism groups. The objects are indexed by a zero-based integer, called the *Orbiter Catalogue Number* (OCN). It is possible to access any object in the catalogue. Let us consider some examples:

The command

### Example 4

```
create.BLT.5.1:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 5 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -define BLT -BLT_set \
> > > -space 0 -catalogue 1 \
> > -end \
> > -with BLT -do -blt_set_activity \
> > > -report \
> > -end
```

```

> pdflatex BLT_catalogue_q5_iso1.tex
> $(OPEN) BLT_catalogue_q5_iso1.pdf

```

recalls the BLT-set with Orbiter Catalogue Number 1 in  $Q(4, 5)$ . A latex report `catalogue_q5_iso1.tex` is written. For more details about BLT-sets, see Section 14.4.

The command

#### Example 5

```

create_surface_4_0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define S4_0 -cubic_surface -space P -catalogue 0 -end \
> > -with S4_0 -do \
> > > -cubic_surface_activity \
> > > -report \
> > -end
> pdflatex surface_catalogue_q4_iso0_report.tex
> $(OPEN) surface_catalogue_q4_iso0_report.pdf

```

recalls the cubic surface with Orbiter Catalogue Number 0 in  $PG(3, 4)$ . A latex report `surface_catalogue_q4_iso0_report.tex` is written. For more details about cubic surfaces, see Section 8.2.

| Set Builder Commands        |           |                                                                                               |
|-----------------------------|-----------|-----------------------------------------------------------------------------------------------|
| Command                     | Arguments | Purpose                                                                                       |
| -index_set_loop             | a b c     | creates the set $\{k \mid k = a + ic, i \in \mathbb{Z}, a \leq k < c\}$                       |
| -set_builder                | set       | Use a set as input to a construction.                                                         |
| -clone_with_affine_function | a b       | Apply the affine function $ax + b$ to the elements $x$ of a set specified using -set_builder. |
| -affine_function            | a b       | Apply the affine function $ax + b$ to the elements $x$ of a set specified using -set_builder. |
| -here                       | elements  | Create the set from the given list of elements. The elements of the list are comma separated. |
| -file                       | fname     | Create a set from the elements listed in the given file.                                      |
| -file_orbiter_format        | fname     | Create a set from the elements listed in the given file.                                      |

Table 2.7: Set Builder Commands

## 2.4 Set Builder

Table 2.7 shows Orbiter commands to create sets.

Here is an example. We create the set  $S$  of the first six prime numbers  $\{2, 3, 5, 7, 11, 13\}$ :

### Example 6

```
set_of_primes:
▷ $(ORBITER) -v 2 \
▷ ▷ -define S -set -here "2,3,5,7,11,13" -end \
▷ ▷ -print.symbols
```

The next command creates the integer interval  $[0, 63]$ . We use the -loop command to save us from typing out all elements of the set. The -loop command has three arguments: the start value  $s$ , the upper bound  $u$ , and the increment  $d$ . The numbers created have the form  $s + kd$ ,  $k = 0, 1, \dots$  as long as they are less than  $u$ .

### Example 7

```
set_interval:
▷ $(ORBITER) -v 2 -define S -set -loop 0 64 1 -end \
▷ ▷ -print.symbols
```

| Vector Builder Commands |             |                                                                                                                                                                                                          |
|-------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                 | Arguments   | Purpose                                                                                                                                                                                                  |
| -field                  | $F$         | Assume that the vector entries are in the range $[0, q - 1]$ , where $q$ is the order of the field $F$ . An error is raised if not.                                                                      |
| -allow_negatives        |             | Allow negative numbers when using finite fields. A negative number will be considered modulo the characteristic of the field. For instance, in the field $\mathbb{F}_5$ , a -1 is considered equal to 4. |
| -dense                  |             | Assume that the vector entries are given as a comma separated list (dense coding).                                                                                                                       |
| -compact                |             | Assume that the vector entries are given as a continuous list without commas or other any kind of whitespace.                                                                                            |
| -repeat                 | pattern $n$ | Repeat the given pattern $n$ times.                                                                                                                                                                      |
| -format                 | $r$         | Assume that the data is a matrix with $r$ rows.                                                                                                                                                          |
| -file                   | file        | Read the data from the given csv-file.                                                                                                                                                                   |
| -load_csv_no_border     | file        | Read the data from the given file, assuming that there are no headers.                                                                                                                                   |
| -load_csv_data_column   | file $c$    | Read the data from column $c$ in the given csv-file.                                                                                                                                                     |
| -sparse                 | data        | Provide data in sparse form, i.e. as pairs value and position.                                                                                                                                           |
| -concatenate            | list        | Concatenate the given vectors. List is a comma separated list of vectors that have been created before.                                                                                                  |
| -loop                   | $s \ u \ d$ | Create vector entries of the form $s + id < u$ , $i \in \mathbb{Z}$ . The upper bound $u$ itself will not be part of the sequence.                                                                       |

Table 2.8: Vector Builder Commands

## 2.5 Vector Builders

Table 2.8 shows Orbiter commands to create vector and matrix objects. Matrices are vectors with an additional format information. By specifying the number of rows, a vector turns into a matrix. So, to create a matrix, one creates a vector, and uses the `-format` command. Examples will be shown below.

Vectors can be defined in two different formats, called dense and sparse, respectively. In the dense format, the coefficients are listed in order from the lowest to the highest term. The `-dense` command creates the vector from a list of coefficients. The sparse format can be useful for coefficient vectors with few nonzero entries. It is a list of coefficient pairs, each of which describing one entry in the vector. One pair consists of the coefficient and the index of the term. The pairs are listed in sequence. The `-sparse` command creates the vectors from a given list of coefficient pairs.

If the option `-field` is given, Orbiter will assume that the vector entries lie in the interval  $[0, q - 1]$ , where  $q$  is the order of the finite field. If entries lie outside that interval, an error is raised. There are limits on the size of  $q$ , see Section 19.2. It is safe to assume that any number  $q$  less than  $2^{31} - 1$  is fine. On 64 bit systems (with Orbiter compiled in 64 bit mode) we can have  $q \leq 2^{63} - 1$ .

Here is an example. We first create the field  $\mathbb{F}_5$ , and then create the vector  $v = (0, 1, 2, 3, 4)$ . The `-field`

option refers to the finite field created previously. The `-dense` option allows to enter the vector coefficients on the command line.

#### Example 8

```
vector_example1:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 5 -end \
> > -define v -vector -field F -dense "0,1,2,3,4" -end \
> > -print_symbols
```

Vectors can also be read from file. The `-file` option can be used to name a csv file.

A vector can also serve as a matrix. The `-format  $k$`  option can be used to specify the number  $k$  of rows. The number of columns is determined as  $n/k$ , where  $n$  is the length of the vector given. For instance, the next example creates a  $2 \times 3$  matrix over  $\mathbb{F}_5$ :

#### Example 9

```
vector_example2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 5 -end \
> > -define v -vector -field F -format 2 -dense "0,1,2,3,4,0" -end \
> > -print_symbols
```

Orbiter has a command to create vectors whose entries repeat. For instance, the following code creates a vector of length 11 whose entries repeat over the sequence 0, 1, 2, 3. It is not necessary that the vector length is an integer multiple of the length of the repeating sequence.

#### Example 10

```
vector_example_repeat:
> $(ORBITER) -v 2 \
> > -define v -vector -repeat "0,1,2,3" 11 -end \
> > -print_symbols
```

The sequence 0,1,2,3 is repeated sufficiently often to make a vector of length 11. This creates the vector

$$(0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2).$$

In order to create a constant vector, the `-repeat` command can be used as well. Simply use a repeat sequence consisting of a single number. For instance, the following example creates the all-one vector of length 11:

#### Example 11

```
vector_example_all_one_11:
> $(ORBITER) -v 2 \
> > -define v -vector -repeat 1 11 -end \
```

```
▷ ▷ -print_symbols
```

This code will create the all-one vector of length 11:

$$(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1).$$

A loop command can create sequences of numbers  $a_0, a_1, \dots$ . It takes three arguments:  $s$ ,  $u$ , and  $d$ . The values of the sequence are

$$a_i = s + id, \dots k = 0, 1, 2, \dots, \quad a_i < u.$$

So, the values of the sequence increases in steps of  $d$ , and are forced to be less than the upper bound  $u$ . For instance, in order to create all integers from 0 to 7, we use the command:

#### Example 12

```
vector_loop_8:
▷ $(ORBITER) -v 2 \
▷ ▷ -define v -vector -loop 0 8 1 -end \
```

If we wanted to create the odd integers less than or equal to 16, we could do this:

#### Example 13

```
vector_loop_odd_16:
▷ $(ORBITER) -v 2 \
▷ ▷ -define v -vector -loop 1 16 2 -end \
```

This command creates the vector

$$(1, 3, 5, 7, 9, 11, 13, 15).$$

Two vectors can be concatenated, as demonstrated in the following example:

#### Example 14

```
vector_concatenate:
▷ $(ORBITER) -v 2 \
▷ ▷ -define a -vector -compact "1,2,3" -end \
▷ ▷ -define b -vector -compact "4,5,6" -end \
▷ ▷ -define c -vector -compact "7,8,9" -end \
▷ ▷ -define abc -vector -concatenate a,b,c -end
```

The vectors (1, 2, 3), (4, 5, 6) and (7, 8, 9) are created. After that, the concatenation command produces

$$(1, 2, 3, 4, 5, 6, 7, 8, 9).$$

The vector command can also be used to create matrices. A matrix is a vector with additional formatting information. The vector is the coefficient vector, using row-major ordering. For instance, the following command creates the vector

$$(1, 2, 3, 4, 5, 6),$$



which by means of the `-format 2` command turns into the matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Here is the command:

#### Example 15

```
matrix_example1:
▷ $(ORBITER) -v 2 \
▷ ▷ -define v -vector -format 2 \
▷ ▷ ▷ -dense "1,2,3,4,5,6" -end \
▷ ▷ -print_symbols
```

For large vectors, the `sparse` command can be used to enter non-zero coefficients as a list of pairs. For instance,

#### Example 16

```
vector_example_sparse:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define v -vector -field F -format 4 -sparse 20 "1,0,1,19" -end \
▷ ▷ -print_symbols
```

creates a vector of length 20 and sets the 0-th and the 19-th coefficient to 1. So, the vector is

$$(1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1).$$

Finally, the vector is displayed as a four-rowed matrix:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Makefile variables can be used to hold the coefficients in a separate place. This helps reduce clutter. For instance, the following command creates the generator matrix of the Hamming code:

#### Example 17

```
HAMMING_CODE_GENERATOR="\
1,0,0,0,0,1,1, \
0,1,0,0,1,0,1, \
0,0,1,0,1,1,0, \
0,0,0,1,1,1,1"
```

## Example 18

```
matrix_example2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F -format 4 \
> > > -dense $(HAMMING_CODE_GENERATOR) -end \
> > -print_symbols
```

For matrices over small fields, the `-compact` option can be given. This works similar as the `-dense` option, but the entries are not separated by commas. This is only good for single digit data, as it arises with very small fields. For instance, the following code creates a  $22 \times 22$  matrix over the binary field:

## Example 19

```
CONWAY_GEN1="\
1101110001000001010000\
1111010111110100001011\
0000001000000100010101\
1111100110110001001110\
0101010000000010011101\
0000010000000100010101\
0010000000000100010101\
0001000011000000111111\
1110100100110100010011\
0000000000000110010101\
0000000000100100010101\
011011111010011101111\
0000000000001100010101\
0000000000000100000101\
0000000001000100010101\
0000000000000100011101\
0001000110000010011010\
0000000000000000010101\
0000000000000101010101\
0000000000000100010100\
0000000000000100010111\
0000000000000100010001"
```

## Example 20

```
matrix_example_co_1:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F -format 22 \
> > > -compact $(CONWAY_GEN1) -end \
> > -print_symbols
```

| Vector of Group Elements Builder Commands |           |                                                 |
|-------------------------------------------|-----------|-------------------------------------------------|
| Command                                   | Arguments | Purpose                                         |
| -action                                   | $A$       | Set the action type for the group elements.     |
| -read_csv                                 | fname     | Read group elements from the given csv file.    |
| -vector_data                              | $v$       | Read group elements from the given vector data. |

Table 2.9: Vector of Group Elements Builder Commands

## 2.6 Vector of Group Elements

Orbiter can create vectors of group elements. Table 2.9 shows the Orbiter commands that can be used to create a vector of group elements.

Let us consider an example. Our goal is to describe the action of the elements of class 2A in the stabilizer of the Hirschfeld surface. The group element is a Baer collineation:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1$$

The class representative is stored in a makefile variable:

### Example 21

```
CLASS_2A_REP="1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1"
```

The first command creates the conjugacy class 2A. The elements will be written to a csv file:

### Example 22

```
Hirschfeld.Class_2A:
▷ $(ORBITER) -v 9 \
▷ ▷ -orbiter_path $(ORBITER_PATH) \
▷ ▷ -define G -linear_group -PGGL 4 4 \
▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
▷ ▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -conjugacy_class_of "2A" $(CLASS_2A_REP) \
▷ ▷ -end
```

The filename is stored in a makefile variable:

### Example 23

```
FILE_NAME_CLASS_2A="PGGL_4_4.Subgroup.Hirschfeld.Stab.51840.class_of_2A.csv"
```

The next command reads the elements of class 2A from this file. It then prints the action of these elements on the various objects associated with the cubic surface:

#### Example 24

```
surface_4_0_export_action:
> $(ORBITER) -v 3 \
> > -define Class2A -vector -file $(FILE_NAME_CLASS_2A) -end \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define S -cubic_surface -space P -catalogue 0 -end \
> > -define G -linear_group -PGGL 4 4 -end \
> > -define gens_ge -vector_ge \
> > > -action G \
> > > -vector_data Class2A \
> > -end \
> > -with S -do \
> > -cubic_surface_activity \
> > > -export_something_with_group_element "action_on_tritangent_planes" gens_ge
\
> > -end \
> > -with S -do \
> > -cubic_surface_activity \
> > > -export_something_with_group_element "action_on_double_sixes" gens_ge \
> > -end \
> > -with S -do \
> > -cubic_surface_activity \
> > > -export_something_with_group_element "action_on_lines" gens_ge \
> > -end \
```

| Creating Symbolic Objects I |                           |                                                                                                                                                                                           |
|-----------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                     | Arguments                 | Purpose                                                                                                                                                                                   |
| -managed_variables          | list                      | A comma-separated list of managed variables may be given. The manage variables can later be matched to the variables of a polynomial ring. The other variables can be used as parameters. |
| -text                       | text                      | The well-formed sequence of expressions in ascii text format, separated by commas.                                                                                                        |
| -field                      | $F$                       | The finite field of coefficients in the expression.                                                                                                                                       |
| -matrix                     | $r$                       | The expression vector is a matrix with $r$ rows.                                                                                                                                          |
| -determinant                | $s$                       | Compute the symbolic determinant of the source object $s$ . The object $s$ must be a square matrix.                                                                                       |
| -characteristic_polynomial  | $s \text{ } \lambda$      | Compute the characteristic polynomial of the source object $s$ using the variable $\lambda$ . The object $s$ must be a square matrix.                                                     |
| -substitute                 | $v \text{ } t \text{ } s$ | Substitute the variables $v$ in the expression $t$ by the expressions given in $s$ . $v$ and $s$ must be vectors of the same length.                                                      |
| -simplify                   | $s$                       | Simplify the object $s$ .                                                                                                                                                                 |
| -expand                     | $s$                       | Expand the object $s$ .                                                                                                                                                                   |
| -right_nullspace            | $s$                       | Compute the right nullspace of the object $s$ , which must be a fully numerical matrix.                                                                                                   |
| -minor                      | $s \text{ } i \text{ } j$ | Compute the $(i, j)$ -minor of the matrix object $s$ . $s$ must be a square matrix.                                                                                                       |
| -symbolic_nullspace         | $s$                       | Compute the right nullspace of the object $s$ , which must be a symbolic matrix.                                                                                                          |
| -stack_matrices_vertically  | list                      | Stack the given matrices in the list vertically. The matrices must have the same number of columns.                                                                                       |

Table 2.10: Creating Symbolic Objects I

## 2.7 Symbolic Objects

Symbolic objects represent algebraic expressions, commonly known as formulae. They consist of the elementary arithmetic operations, numbers, variables and exponents. In Orbiter, algebraic expressions can be parsed from an ascii text representation. This is different from for instance Maple, where an algebraic expression composed inside the graphical user interface. Internally, an algebraic expression is represented as an abstract syntax tree. The syntax tree is an efficient way to code the expression, and allows many algebraic operations to be performed easily. The process of creating the abstract syntax tree from the ascii representation is called parsing. Orbiter can deal with vectors and matrices of symbolic expressions.

Tables 2.10-2.11 show Orbiter commands to create symbolic objects.

Table 2.12 show Orbiter activities for symbolic objects.

Let us consider some examples. The examples are over finite field  $\mathbb{F}_q$ . For more information about these fields, see Chapter 3. For now, it suffices to know that the elements of the finite field  $\mathbb{F}_q$  are the integers

| Creating Symbolic Objects II             |                  |                                                                                                              |
|------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------|
| Command                                  | Arguments        | Purpose                                                                                                      |
| <code>-multiply_2x2_from_the_left</code> | $s \ A2 \ i \ j$ | Multiply the $2 \times 2$ matrix $A2$ from the left to $s$ , applying to rows $i$ and $j$ .                  |
| <code>-matrix_entry</code>               | $s \ i \ j$      | Select the $(i, j)$ -entry of the matrix $s$ .                                                               |
| <code>-vector_entry</code>               | $s \ i$          | Select the $j$ -th entry of the vector $s$ . Using row-major ordering, a matrix is considered a vector also. |
| <code>-collect</code>                    | $s \ t$          | Collect by coefficients of $t$ in the expression $s$ .                                                       |
| <code>-do_not_simplify</code>            |                  | Do not simplify the output object.                                                                           |
| <code>-write_trees_during_expand</code>  |                  | Export trees during the expand command.                                                                      |

Table 2.11: Creating Symbolic Objects II

| Activities for Symbolic Objects |           |                                                                                       |
|---------------------------------|-----------|---------------------------------------------------------------------------------------|
| Command                         | Arguments | Purpose                                                                               |
| <code>-export</code>            |           | Export the symbolic object.                                                           |
| <code>-print</code>             | $F$       | Print the symbolic object, assuming that coefficients are from the finite field $F$ . |

Table 2.12: Activities for Symbolic Objects

from 0 to  $q - 1$ . It is also true that 0 represents zero and 1 represents one. Numbers outside the interval 0 to  $q - 1$  are not allowed. In particular, we do not allow negative numbers. So, for instance,  $-1$  is not allowed.

The first example creates the polynomial  $1 - X - X^2$  over  $\mathbb{F}_7$ :

#### Example 25

```

symbolic_poly:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "-X^2-X+1"\
▷ ▷ -end \

```

The output is

$$M = 1 + (6X) + (6X^2)$$

The coefficients of  $X$  and  $X^2$  are 6, because  $6 \equiv -1 \pmod{7}$  and Orbiter chooses field elements of  $\mathbb{F}_q$  to lie in the interval  $[0, q - 1]$ .

The second example creates the polynomial CRC32, using the makefile variable

**Example 26**

```
CRC32_ETHERNET_POLY="X^32+X^26+X^23+X^22+X^16+\
X^12+X^11+X^10+X^8+X^7+X^5+X^4+X^2+X^1+1"
```

The command

**Example 27**

```
symbolic_CRC32:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(CRC32_ETHERNET_POLY) \
▷ ▷ -end \
```

produces the following output:

$$M = 1 + X + X^2 + X^4 + X^5 + X^7 + X^8 + X^{10} + X^{11} + X^{12} + X^{16} + X^{22} + X^{23} + X^{26} + X^{32}$$

The second example creates the BCH polynomial Alfa, defined over the field  $\mathbb{F}_{256}$ , using the makefile variable

**Example 28**

```
BCH_POLYNOMIAL_ALFA_F256="X^2 + 167*X + 214"
```

The command

**Example 29**

```
symbolic_CRC_alfa:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 256 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(BCH_POLYNOMIAL_ALFA_F256) \
▷ ▷ -end \
```

produces the following output:

$$M = 214 + X^2 + (167X)$$

The next example creates the CRC polynomial bravo over the field  $\mathbb{F}_{256}$ :

**Example 30**

```
symbolic_CRC_bravo:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 256 -end \
```

```

▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(BCH_POLYNOMIAL_BRAVO_F256) \
▷ ▷ -end \

```

The output is

$$M = 1 + X^4 + (27X^2) + (23X) + (213X^3)$$

Let us consider the formula  $(a + b)^3$  and expand:

#### Example 31

```

symbolic_test_5_expand:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "(a+b)^3" \
▷ ▷ -end \
▷ ▷ -define M1 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -expand M \
▷ ▷ -end
▷ pdflatex M.tex
▷ $(OPEN) M.pdf
▷ pdflatex M1.tex
▷ $(OPEN) M1.pdf

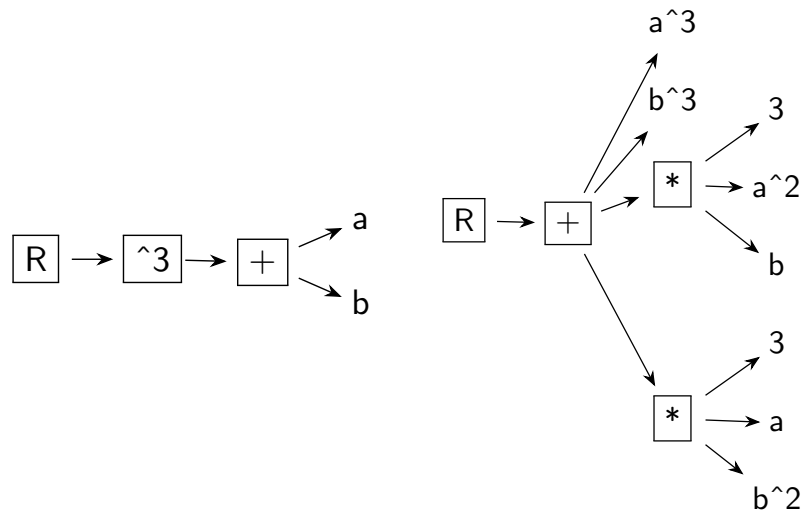
```

The output is

$$M1 = \left[ a^3 + b^3 + (3a^2b) + (3ab^2) \right]$$

It is also possible to inspect the representation of the expressions as abstract syntax trees. The node labeled R signifies the root node of the tree. The tree on the left is the syntax tree before expansion, whereas the tree on the right is the syntax tree after expansion:





The command

#### Example 32

```
test_expand_1:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Fq -finite_field -q 31 -end \
▷ ▷ -define f -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -text "a*(x+y)" \
▷ ▷ -end \
▷ ▷ -define fe -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -expand f \
▷ ▷ ▷ -write_trees_during_expand \
▷ ▷ -end
▷ pdflatex fe_0_efsxsfsfscsc.tex
▷ $(OPEN) fe_0_efsxsfsfscsc.pdf
```

tests that

$$a(x + y) = ax + ay.$$

The command

#### Example 33

```
test_expand_2:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Fq -finite_field -q 31 -end \
▷ ▷ -define f -symbolic_object \
▷ ▷ ▷ -field Fq \
```

```

▷ ▷ ▷ -text "a*(-(x-y))" \
▷ ▷ -end \
▷ ▷ -define fe -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -expand f \
▷ ▷ ▷ -write_trees_during_expand \
▷ ▷ -end
▷ pdflatex fe_0_efsxsfcsc.tex
▷ $(OPEN) fe_0_efsxsfcsc.pdf

```

tests that

$$a(-(x-y)) = -ax + ay.$$

The command

#### Example 34

```

test_expand_3:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Fq -finite_field -q 31 -end \
▷ ▷ -define f -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -text "a*(-(x-y)+(x+y))" \
▷ ▷ -end \
▷ ▷ -define fe -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -expand f \
▷ ▷ ▷ -write_trees_during_expand \
▷ ▷ -end
▷ pdflatex fe_0_efs.tex
▷ $(OPEN) fe_0_efs.pdf
▷ pdflatex fe_0_efsx.tex
▷ $(OPEN) fe_0_efsx.pdf
▷ pdflatex fe_0_efsxsfcsc.tex
▷ $(OPEN) fe_0_efsxsfcsc.pdf
▷ pdflatex fe_0_efsxsfcscsefsx.tex
▷ $(OPEN) fe_0_efsxsfcscsefsx.pdf
▷ pdflatex fe_0_efsxsfcscsefsxsfcsc.tex
▷ $(OPEN) fe_0_efsxsfcscsefsxsfcsc.pdf

```

tests that

$$a(-(x-y) + (x+y)) = 2ay.$$

The output is

$$fe = (2ay)$$

The command shows the abstract syntax trees for the intermediate steps in the algebraic simplification.

The command

### Example 35

```
test_expand4:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Fq -finite_field -q 31 -end \
▷ ▷ -define f -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -text "x*(a+(x*(a+(x*(a*x*(a*x*(a*x*(a*x)))))))" \
▷ ▷ -end \
▷ ▷ -define fx -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -expand f \
▷ ▷ ▷ -write_trees_during_expand \
▷ ▷ -end
▷ pdflatex fx_0_efxsxsfscsefsxsfscsefsxsfscsefsxsfscsefsxsfscsc.tex
▷ $(OPEN) fx_0_efxsxsfscsefsxsfscsefsxsfscsefsxsfscsefsxsfscsc.pdf
```

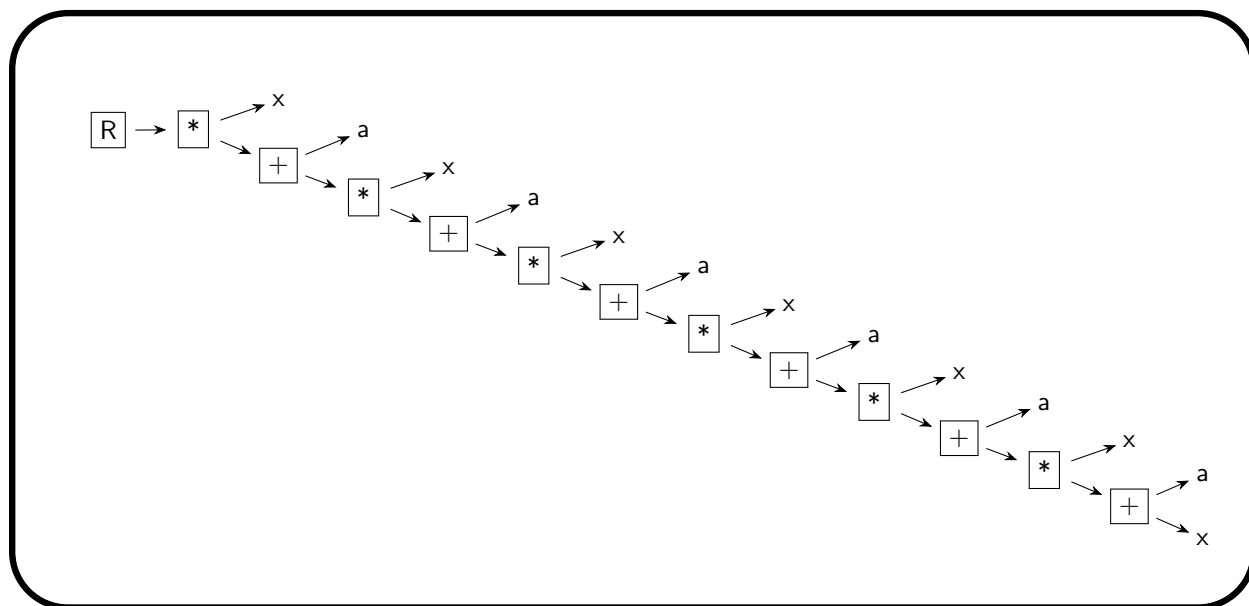
tests that

$$x * (a + (x * (a + (x * (a + x * (a + x * (a + x * (a + x)))))))) = ax + ax^2 + ax^3 + ax^4 + ax^5 + ax^6 + x^7.$$

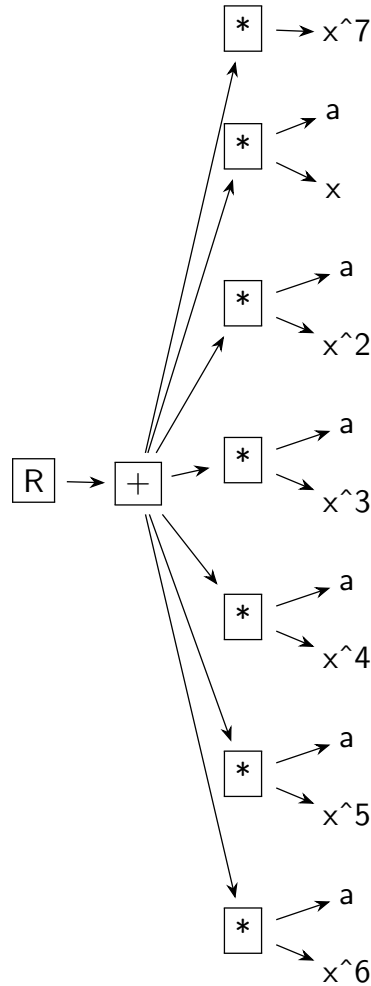
The output is

$$fx = x^7 + (ax^2) + (ax^3) + (ax^4) + (ax^5) + (ax^6) + (ax)$$

The abstract syntax tree before expansion is:



The abstract syntax tree after expansion is:



The abstract syntax trees for the intermediate steps during expansion are available also.

Objects are stored in the Orbiter symbol table. There is only one table. There can be only one object with any given name. Symbolic objects can recycle objects defined earlier using a process called implicit substitution. In this process, the object being referenced is taken from the symbol table and substituted immediately. For instance, in the next example we create an object  $a = x + y$  and then another object  $b = a + z$  which immediately expands into  $b = x + y + z$ . This is because the object  $a$  already exists. Implicit substitution is non-recursive, so the substituted syntax tree is not subjected to implicit substitution itself. This prevents unlimited recursion which would result in crashes.

#### Example 36

```

symbolic_objects.building_on_earlier_objects:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 2 -end \
> > -define a -symbolic_object \
> > > -field F \
> > > -text "x+y"\
> > -end \

```

```

▷ ▷ -define b -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "a+z"\
▷ ▷ -end \

```

The output of the command is:

$$b = x + y + z$$

Self-referential objects are allowed. The following command is legal:

#### Example 37

```

symbolic_objects_self_referential_1:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define x -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "x"\
▷ ▷ -end \

```

The output is

$$x = x$$

Likewise, the following command is ok:

#### Example 38

```

symbolic_objects_self_referential_2:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define x -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "x"\
▷ ▷ -end \
▷ ▷ -define a -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "x+a"\
▷ ▷ -end \

```

The output is

$$a = a + x$$

The implicit substitution process is limited to scalar quantities. Matrices and vectors cannot be substituted.

Let us create a Vandermonde matrix. We use the makefile variable

#### Example 39

```

VANDERMONDE_4X4_FORMULA="1,x0,x0^2,x0^3,1,x1,x1^2,x1^3,\
1,x2,x2^2,x2^3,1,x3,x3^2,x3^3"

```

The following command creates the matrix:

#### Example 40

```
symbolic_matrix1:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -matrix 4 \
> > > -text $(VANDERMONDE_4X4_FORMULA) \
> > -end
```

The output is:

$$\begin{bmatrix} 1 & x0 & x0^2 & x0^3 \\ 1 & x1 & x1^2 & x1^3 \\ 1 & x2 & x2^2 & x2^3 \\ 1 & x3 & x3^2 & x3^3 \end{bmatrix}$$

Let us create a fully symbolic  $2 \times 2$  matrix:

#### Example 41

```
symbolic_matrix_2x2:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -matrix 2 \
> > > -text "a,b,c,d" \
> > -end
```

The output is:

$$M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Let us create the determinant of a symbolic  $2 \times 2$  matrix:

#### Example 42

```
symbolic_determinant_2x2:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -matrix 2 \
> > > -text "a,b,c,d" \
> > -end \
> > -define det -symbolic_object \
> > > -field F \
```

```

> > > -determinant M \
> > -end \
> > -define det1 -symbolic_object \
> > > -field F \
> > > -expand det \
> > -end
> pdflatex det1.tex
> $(OPEN) det1.pdf

```

The output is

$$\det1 = (ad) + (12bc)$$

Let us create the determinant of a symbolic  $3 \times 3$  matrix:

#### Example 43

```

symbolic_determinant_3x3:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -matrix 3 \
> > > -text "a,b,c,d,e,f,g,h,i" \
> > -end \
> > -define det -symbolic_object \
> > > -field F \
> > > -determinant M \
> > -end \
> > -define det1 -symbolic_object \
> > > -field F \
> > > -expand det \
> > -end
> pdflatex det1.tex
> $(OPEN) det1.pdf

```

The output is

$$\det1 = (cdh) + (bfg) + (12bdi) + (12afh) + (aei) + (12ceg)$$

Let us create the determinant of the Vandermonde matrix from above:

#### Example 44

```

symbolic_determinant_vandermonde:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -matrix 4 \
> > > -text $(VANDERMONDE_4X4_FORMULA) \
> > -end \

```

```

▷ ▷ -define det -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -determinant M \
▷ ▷ -end \
▷ ▷ -define det1 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -expand det \
▷ ▷ -end

```

The output is lengthy, so it is given in text notation:

```

0 :
(12 * x1 * x2 ^ 3 * x3 ^ 2) + (12 * x1 ^ 2 * x2 * x3 ^ 3) + (x1 ^ 2 * x2 ^ 3
* x3) + (x1 ^ 3 * x2 * x3 ^ 2) + (12 * x1 ^ 3 * x2 ^ 2 * x3) + (12 * x0 * x
2 ^ 2 * x3 ^ 3) + (x0 * x2 ^ 3 * x3 ^ 2) + (x0 * x1 ^ 2 * x3 ^ 3) + (12 * x0
* x1 ^ 2 * x2 ^ 3) + (12 * x0 * x1 ^ 3 * x3 ^ 2) + (x0 * x1 ^ 3 * x2 ^ 2) +
(x0 ^ 2 * x2 * x3 ^ 3) + (12 * x0 ^ 2 * x2 ^ 3 * x3) + (12 * x0 ^ 2 * x1 *
x3 ^ 3) + (x0 ^ 2 * x1 * x2 ^ 3) + (x0 ^ 2 * x1 ^ 3 * x3) + (12 * x0 ^ 2 * x
1 ^ 3 * x2) + (12 * x0 ^ 3 * x2 * x3 ^ 2) + (x0 ^ 3 * x2 ^ 2 * x3) + (x0 ^ 3
* x1 * x3 ^ 2) + (12 * x0 ^ 3 * x1 * x2 ^ 2) + (12 * x0 ^ 3 * x1 ^ 2 * x3)
+ (x0 ^ 3 * x1 ^ 2 * x2) + (x1 * x2 ^ 2 * x3 ^ 3)

```

The next command computes the characteristic polynomial of a symbolic  $2 \times 2$  matrix. When all is done, the terms of the polynomial are collected to form a coefficient vector:

#### Example 45

```

symbolic_characteristic_polynomial_2x2:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -matrix 2 \
▷ ▷ ▷ -text "a,b,c,d" \
▷ ▷ -end \
▷ ▷ -define p -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -characteristic_polynomial lambda M \
▷ ▷ -end \
▷ ▷ -define p1 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -expand p \
▷ ▷ ▷ -write_trees_during_expand \
▷ ▷ -end \
▷ ▷ -define coeffs -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -collect p1 lambda \
▷ ▷ -end

```



The output is:

$$\text{coeffs} = \begin{bmatrix} (ad) + (12bc) \\ (12d) + (12a) \\ 1 \end{bmatrix}$$

Let us do the characteristic polynomial of a symbolic  $3 \times 3$  matrix:

#### Example 46

```
symbolic_characteristic_polynomial_3x3:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -matrix 3 \
> > > -text "a,b,c,d,e,f,g,h,i" \
> > -end \
> > -define p -symbolic_object \
> > > -field F \
> > > -characteristic_polynomial lambda M \
> > -end \
> > -define p1 -symbolic_object \
> > > -field F \
> > > -expand p \
> > -end \
> > -define coeffs -symbolic_object \
> > > -field F \
> > > -collect p1 lambda \
> > -end
> #pdflatex p1.tex
> #$(OPEN) p1.pdf
```

The output is:

$$\text{coeffs} = \begin{bmatrix} (cdh) + (bfg) + (12bdi) + (12afh) + (aei) + (12ceg) \\ (12ei) + (cg) + (bd) + (12ai) + (12ae) + (fh) \\ a + e + i \\ 12 \end{bmatrix}$$

Let us compare Orbiter and Maple with an example. We create an object and then substitute values over the field  $\mathbb{F}_{13}$ . Here is the Orbiter code:

#### Example 47

```
symbolic_object1:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
> > -end
```

The output is:

$$M = a + (10((3d) + (12a^2b^2c^2) + (2c^4)^3)) + (2b^2(x + (12y)))$$

Let us substitute some values and evaluate the formula:

#### Example 48

```
symbolic_object1.evaluate:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Fq -finite_field -q 13 -end \
▷ ▷ -define f -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
▷ ▷ -end \
▷ ▷ -define assignment -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -text "1,2,3,4,5,6" \
▷ ▷ -end \
▷ ▷ -define eval -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -substitute "a,b,c,d,x,y" f assignment \
▷ ▷ -end \
```

The value obtained after substitution is:

$$eval = \begin{bmatrix} 4 \end{bmatrix}$$

Let us perform the same task in Maple:

$$f := a + 2 * b^2 * (x - y) + 3 * (-(2 * c^4 + 3 * d - (a * b * c)^2)^3)$$

$$f := a + 2b^2(x - y) - 3(-a^2b^2c^2 + 2c^4 + 3d)^3 \quad (2.1)$$

$$subs(a = 1, b = 2, c = 3, d = 4, x = 5, y = 6, f) \bmod 13;$$

$$4 \quad (2.2)$$

Let us create the general equation of the Eckardt surface using a makefile variable:

#### Example 49

```
ECKARDT_SURFACE_EQN="a*X3^3-a*b^2*(X0^2+X1^2+X2^2)*X3+b^3*(a^2+1)*X0*X1*X2"
```

The following command parses the equation:

#### Example 50

```
symbolic_Eckardt_surface:
▷ $(ORBITER) -v 3 \
```

```

▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \
▷ ▷ -end

```

The output is:

$$M = (12X3ab^2(X0^2 + X1^2 + X2^2)) + (X0X1X2b^3(1 + a^2)) + (X3^3a)$$

We specify  $a = 3$  and  $b = 1$  by issuing the following command:

#### Example 51

```

symbolic_Eckardt_surface_q13_a3_b1:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \
▷ ▷ -end \
▷ ▷ -define L -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "3,1" \
▷ ▷ -end \
▷ ▷ -define M1 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -substitute "a,b" M L \
▷ ▷ -end \
▷ ▷ -define M2 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -expand M1 \
▷ ▷ ▷ -write_trees_during_expand \
▷ ▷ -end
▷ pdflatex M2.tex
▷ $(OPEN) M2.pdf

```

The output is:

$$M2 = \left[ (10X2^2X3) + (10X1^2X3) + (10X0X1X2) + (10X0^2X3) + (3X3^3) \right]$$

Let us consider the 4 parameter normal form of a cubic surface from [14]. A makefile variable can be used to define the equation in text form:

#### Example 52

```

F_abcd_eqn="-(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(b - d)*X0^2*X2 \
+ (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(a + b - c - d)*X0*X1*X2 \

```

```

+ (a^2*c - a^2*d - a*c^2 + b*c^2 + a*d - b*c)*(b - d)*X0*X1*X3 \
- (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X0*X2^2 \
- (a^2*c*d - a*b*c^2 - a^2*d + a*b*d + b*c^2 - b*c*d)*(b - d)*X0*X2*X3 \
- (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X2 \
- (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X3 \
+ (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1*X2^2 \
+ ((1+1)*a^2*b*c*d - a^2*b*d^2 - (1+1)*a^2*c*d^2 \
- (1+1)*a*b^2*c^2 + a*b^2*c*d + (1+1)*a*b*c^2*d + a*b*c*d^2 \
- b^2*c^2*d - a^2*b*c + a^2*c*d + a^2*d^2 + a*b^2*c + a*b*c^2 \
- (1+1+1+1)*a*b*c*d - a*c^2*d + a*c*d^2 + b^2*c^2)*X1*X2*X3 \
+ c*a*(a*d - b*c - a + b + c - d)*(b - d)*X1*X3^2"

```

The following command parses the equation.

### Example 53

```

symbolic_Fabcd:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define M -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
▷ ▷ ▷ -text $(F_abcd_eqn) \
▷ ▷ -end

```

The output in text form is: 0 :

```

(X0 * X1 * X2 * (a + b + (12 * c) + (12 * d)) * ((12 * a * b * d) + (12 * a
* c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c))) + (X0 * X1 *
X3 * (b + (12 * d)) * ((12 * a ^ 2 * d) + (12 * a * c ^ 2) + (b * c ^ 2) +
(a * d) + (12 * b * c) + (a ^ 2 * c))) + (12 * X0 * X2 ^ 2 * ((12 * a * b *
d) + (12 * a * c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c))
* ((12 * b * c) + (a * d))) + (12 * X0 * X2 * X3 * (b + (12 * d)) * ((12 * a
* b * c ^ 2) + (12 * a ^ 2 * d) + (a * b * d) + (b * c ^ 2) + (12 * b * c *
d) + (a ^ 2 * c * d))) + (12 * X1 ^ 2 * X2 * ((12 * a * b * d) + (12 * a *
c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c)) * (a + (12 * c)
)) + (12 * X1 ^ 2 * X3 * ((12 * a * b * d) + (12 * a * c * d) + (b * c * d)
+ (a * d) + (12 * b * c) + (a * b * c)) * (a + (12 * c))) + (X1 * X2 ^ 2 * (
(12 * a * b * d) + (12 * a * c * d) + (b * c * d) + (a * d) + (12 * b * c) +
(a * b * c)) * ((12 * b * c) + (a * d))) + (X1 * X2 * X3 * ((12 * a ^ 2 * b
* d ^ 2) + (11 * a ^ 2 * c * d ^ 2) + (11 * a * b ^ 2 * c ^ 2) + (a * b ^ 2
* c * d) + (2 * a * b * c ^ 2 * d) + (a * b * c * d ^ 2) + (12 * b ^ 2 * c
^ 2 * d) + (12 * a ^ 2 * b * c) + (a ^ 2 * c * d) + (a ^ 2 * d ^ 2) + (a * b
^ 2 * c) + (a * b * c ^ 2) + (9 * a * b * c * d) + (12 * a * c ^ 2 * d) + (
a * c * d ^ 2) + (b ^ 2 * c ^ 2) + (2 * a ^ 2 * b * c * d))) + (X1 * X3 ^ 2
* a * c * (b + c + (12 * a) + (12 * b * c) + (12 * d) + (a * d)) * (b + (12
* d))) + (12 * X0 ^ 2 * X2 * (b + (12 * d)) * ((12 * a * b * d) + (12 * a *
c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c)))

```

The next command expands the equation. The domain of coefficients is the finite field  $\mathbb{F}_{31}$ .

**Example 54**

```

symbolic_Fabcd_expand:
> $(ORBITER) -v 3 \
> > -define Fq -finite_field -q 31 -end \
> > -define Fabcd -symbolic_object \
> > > -field Fq \
> > > -managed_variables "X0,X1,X2,X3" \
> > > -text $(F_abcd_eqn) \
> > -end \
> > -define Fabcd_e -symbolic_object \
> > > -field Fq \
> > > -managed_variables "X0,X1,X2,X3" \
> > > -expand Fabcd \
> > > -write_trees_during_expand \
> > -end

```

The expanded equation has 127 terms.

Over the field  $\mathbb{F}_{31}$ , let us substitute  $(a, b, c, d) = (25, 5, 5, 25)$  in the general equation. The following command does the job:

**Example 55**

```

symbolic_Fabcd_q31_E18:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 31 -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -managed_variables "X0,X1,X2,X3" \
> > > -text $(F_abcd_eqn) \
> > -end \
> > -define L -symbolic_object \
> > > -field F \
> > > -text "25,5,5,25" \
> > -end \
> > -define M1 -symbolic_object \
> > > -field F \
> > > -substitute "a,b,c,d" M L \
> > -end \
> > -define M2 -symbolic_object \
> > > -field F \
> > > -expand M1 \
> > > -write_trees_during_expand \
> > -end
> pdflatex M2.tex
> $(OPEN) M2.pdf

```

The output is:

$$M2 = \left[ (13X_1X_2X_3) + (22X_1X_2^2) + (22X_1^2X_3) + (22X_1^2X_2) + (9X_0X_2^2) + (9X_0^2X_2) + (22X_1X_3^2) \right]$$

It is also possible to use implicit substitution to achieve the same effect, as shown in the following command:

#### Example 56

symbolic\_Fabcd.q31.E18\_with\_implicit\_substitution:

```

> $(ORBITER) -v 3 \
> > -define F -finite_field -q 31 -end \
> > -define a -symbolic_object \
> > > -field F \
> > > -text "25" \
> > -end \
> > -define b -symbolic_object \
> > > -field F \
> > > -text "5" \
> > -end \
> > -define c -symbolic_object \
> > > -field F \
> > > -text "5" \
> > -end \
> > -define d -symbolic_object \
> > > -field F \
> > > -text "25" \
> > -end \
> > -define M -symbolic_object \
> > > -field F \
> > > -managed_variables "X0,X1,X2,X3" \
> > > -text $(F_abcd.eqn) \
> > -end \
> > -define M2 -symbolic_object \
> > > -field F \
> > > -expand M \
> > -end
> pdflatex M2.tex
> $(OPEN) M2.pdf

```

The output is again

$$M2 = (13X1X2X3) + (22X1X2^2) + (22X1^2X3) + (22X1^2X2) + (9X0X2^2) + (9X0^2X2) + (22X1X3^2)$$

We can use the option `-do_not_simplify` to see the syntax tree exactly after the moment of substitution, before the nodes are evaluated: So, the command

#### Example 57

symbolic\_Fabcd.q31.E18\_with\_implicit\_substitution\_do\_not\_simplify:

```

> $(ORBITER) -v 3 \
> > -define F -finite_field -q 31 -end \
> > -define a -symbolic_object \
> > > -field F \
> > > -text "25" \
> > -end \
> > -define b -symbolic_object \
> > > -field F \

```

```

▷ ▷ ▷ -text "5" \
▷ ▷ -end \
▷ ▷ -define c -symbolic-object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "5" \
▷ ▷ -end \
▷ ▷ -define d -symbolic-object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "25" \
▷ ▷ -end \
▷ ▷ -define M -symbolic-object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
▷ ▷ ▷ -text $(F_abcd_eqn) \
▷ ▷ ▷ -do_not_simplify \
▷ ▷ -end

```

produces the output

0 :

$$\begin{aligned}
& ((30 * ((25 * 5 * 5) + (30 * (25 * 5 * 25))) + (30 * (25 * 5 * 25)) + (5 * 5 \\
& * 25) + (25 * 25) + (30 * (5 * 5))) * (5 + (30 * 25)) * X0^2 * X2) + (((2 \\
& 5 * 5 * 5) + (30 * (25 * 5 * 25))) + (30 * (25 * 5 * 25)) + (5 * 5 * 25) + (2 \\
& 5 * 25) + (30 * (5 * 5))) * (25 + 5 + (30 * 5) + (30 * 25)) * X0 * X1 * X2 \\
& + (((25^2 * 5) + (30 * (25^2 * 25))) + (30 * (25 * 5^2)) + (5 * 5^2) \\
& + (25 * 25) + (30 * (5 * 5))) * (5 + (30 * 25)) * X0 * X1 * X3) + (30 * (((2 \\
& 5 * 25) + (30 * (5 * 5))) * ((25 * 5 * 5) + (30 * (25 * 5 * 25)) + (30 * (25 \\
& * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X0 * X2^2) + ( \\
& 30 * (((25^2 * 5 * 25) + (30 * (25 * 5 * 5^2)) + (30 * (25^2 * 25)) + \\
& (25 * 5 * 25) + (5 * 5^2) + (30 * (5 * 5 * 25))) * (5 + (30 * 25)) * X0 * \\
& X2 * X3)) + (30 * ((25 + (30 * 5)) * ((25 * 5 * 5) + (30 * (25 * 5 * 25)) + \\
& (30 * (25 * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X1^2 * \\
& X2)) + (30 * ((25 + (30 * 5)) * ((25 * 5 * 5) + (30 * (25 * 5 * 25)) + (30 \\
& * (25 * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X1^2 * X3) \\
& ) + (((25 * 25) + (30 * (5 * 5))) * ((25 * 5 * 5) + (30 * (25 * 5 * 25)) + ( \\
& 30 * (25 * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X1 * X2^2 \\
& + (((1 + 1) * 25^2 * 5 * 5 * 25) + (30 * (25^2 * 5 * 25^2)) + (30 \\
& * ((1 + 1) * 25^2 * 5 * 25^2)) + (30 * ((1 + 1) * 25 * 5^2 * 5^2)) \\
& + (25 * 5^2 * 5 * 25) + ((1 + 1) * 25 * 5 * 5^2 * 25) + (25 * 5 * 5 * 25 \\
& ^2) + (30 * (5^2 * 5^2 * 25)) + (30 * (25^2 * 5 * 5)) + (25^2 * 5 \\
& * 25) + (25^2 * 25^2) + (25 * 5^2 * 5) + (25 * 5 * 5^2) + (30 * ((1 \\
& + 1 + 1) * 25 * 5 * 5 * 25)) + (30 * (25 * 5^2 * 25)) + (25 * 5 * 25^2 \\
& + (5^2 * 5^2)) * X1 * X2 * X3) + (5 * 25 * ((25 * 25) + (30 * (5 * 5 \\
& )) + (30 * 25) + 5 + 5 + (30 * 25)) * (5 + (30 * 25)) * X1 * X3^2)
\end{aligned}$$





## Chapter 3

# Finite Fields

### 3.1 Prime Fields

Let  $\mathbb{F}_q$  denote the finite field with  $q$  elements. Up to isomorphism, there is only one field of order  $q$ . In Orbiter, the elements of  $\mathbb{F}_q$  are represented by the integers in the interval  $[0, q - 1]$ .

Table 3.1 lists Orbiter commands for creating a finite field. For instance,

#### Example 58

```
F_2:  
▷ $(ORBITER) -v 3 -list_arguments \  
▷ ▷ -define F -finite_field -q 2 -end \  
▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end  
▷ pdflatex GF_2.tex  
▷ $(OPEN) GF_2.pdf
```

creates the finite field  $\mathbb{F}_2$  and produces a report for it.

Table 3.2 lists basic Orbiter activities for finite fields. More activities will follow in Section 3.2. The command

#### Example 59

```
F_7:  
▷ $(ORBITER) -v 3 \  
▷ ▷ -define F -finite_field -q 7 -end \  
▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end  
▷ pdflatex GF_7.tex  
▷ $(OPEN) GF_7.pdf
```

creates a cheat sheet for  $\mathbb{F}_7$  as shown below. The element  $\alpha$  is a primitive element.

$$Z_i = \log_{\alpha}(1 + \alpha^i)$$

| Creating Finite Fields               |           |                                                                                                                                                 |
|--------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                              | Arguments | Purpose                                                                                                                                         |
| <code>-q</code>                      | $q$       | Specify the order of the field. Here, $q = p^k$ for some prime $p$ and some positive integer $k$ .                                              |
| <code>-override_polynomial</code>    | $n$       | Specify the polynomial used to create the finite field. The polynomial is given as integer, using the base $p$ representation. See Section 3.2. |
| <code>-without_tables</code>         |           | Create the field without precomputing the tables.                                                                                               |
| <code>-compute_related_fields</code> |           | Compute information about all subfields.                                                                                                        |
| <code>-symbol</code>                 | $s$       | Use $s$ as a symbol for the primitive element during printing.                                                                                  |

Table 3.1: Creating Finite Fields

| $i$ | $\gamma_i$     | $-\gamma_i$ | $\gamma_i^{-1}$ | $\log_\alpha(\gamma_i)$ | $\alpha^i$ | $Z_i$ |
|-----|----------------|-------------|-----------------|-------------------------|------------|-------|
| 0   | $0 = 0$        | 0           | DNE             | DNE                     | 1          | 2     |
| 1   | $1 = 1$        | 6           | 1               | 0                       | 3          | 4     |
| 2   | $2 = \alpha^2$ | 5           | 4               | 2                       | 2          | 1     |
| 3   | $3 = \alpha$   | 4           | 5               | 1                       | 6          | DNE   |
| 4   | $4 = \alpha^4$ | 3           | 2               | 4                       | 4          | 5     |
| 5   | $5 = \alpha^5$ | 2           | 3               | 5                       | 5          | 3     |
| 6   | $6 = \alpha^3$ | 1           | 6               | 3                       | 1          | 2     |

| + | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| 1 | 1 | 2 | 3 | 4 | 5 | 6 | 0 |
| 2 | 2 | 3 | 4 | 5 | 6 | 0 | 1 |
| 3 | 3 | 4 | 5 | 6 | 0 | 1 | 2 |
| 4 | 4 | 5 | 6 | 0 | 1 | 2 | 3 |
| 5 | 5 | 6 | 0 | 1 | 2 | 3 | 4 |
| 6 | 6 | 0 | 1 | 2 | 3 | 4 | 5 |

| · | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 1 | 2 | 3 | 4 | 5 | 6 |
| 2 | 2 | 4 | 6 | 1 | 3 | 5 |
| 3 | 3 | 6 | 2 | 5 | 1 | 4 |
| 4 | 4 | 1 | 5 | 2 | 6 | 3 |
| 5 | 5 | 3 | 1 | 6 | 4 | 2 |
| 6 | 6 | 5 | 4 | 3 | 2 | 1 |

| Finite Field Activities       |                                                 |                                                                                                                         |
|-------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Command                       | Arguments                                       | Purpose                                                                                                                 |
| -cheat_sheet_GF               |                                                 | Produce a cheat sheet in latex which shows information about the field, including addition and multiplication tables.   |
| -export_tables                |                                                 | Export addition and multiplication table to a csv-file.                                                                 |
| -polynomial_division          | $A \ B$                                         | Perform polynomial division: Divide $B$ into $A$ .                                                                      |
| -extended_gcd_for_polynomials | $A \ B$                                         | Compute the GCD of $G$ of two polynomials $A$ and $B$ . Determine two polynomials $U$ and $V$ such that $G = UA + VB$ . |
| -polynomial_mult_mod          | $A \ B \ M$                                     | Perform modular polynomial multiplication. Compute $A \cdot B \pmod{M}$ .                                               |
| -polynomial_power_mod         | $A \ n \ M$                                     | Perform modular polynomial exponentiation. Compute $A^n \pmod{M}$ .                                                     |
| -Berlekamp_matrix             | $A$                                             | Compute the Berlekamp matrix associated with the polynomial $A$ .                                                       |
| -polynomial_find_roots        | $A$                                             | Computes the roots of the polynomial $A$ .                                                                              |
| -product_of                   | $v$                                             | Compute the product of all field elements in the vector $v$ .                                                           |
| -sum_of                       | $v$                                             | Compute the sum of all field elements in the vector $v$ .                                                               |
| -negate                       | $v$                                             | Negate each field element in the vector $v$ .                                                                           |
| -inverse                      | $v$                                             | Compute the multiplicative inverse of each field element in the vector $v$ .                                            |
| -power_map                    | $k \ v$                                         | Compute the $k$ -th power of each field element in the vector $v$ .                                                     |
| -parse_and_evaluate           | label<br>managed-<br>vars formula<br>parameters | Parse the given formula with the given managed variables and evaluate at the given values of the parameters.            |
| -evaluate                     | label param-<br>eters                           | Parse the given formula and evaluate at the given values of the parameters.                                             |

Table 3.2: Finite Field Activities

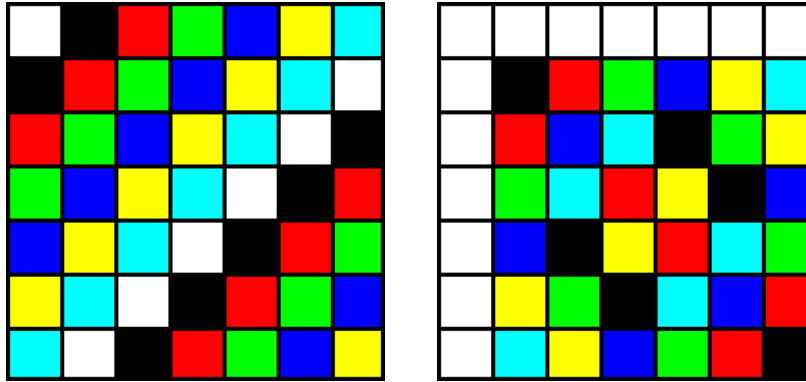
|                |                |
|----------------|----------------|
| $3^0 \equiv 1$ | $3^4 \equiv 4$ |
| $3^1 \equiv 3$ | $3^5 \equiv 5$ |
| $3^2 \equiv 2$ | $3^6 \equiv 1$ |
| $3^3 \equiv 6$ |                |

The command

#### Example 60

```
F_7_tables:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -with F -do -finite_field_activity -export_tables -end
▷ $(ORBITER) -v 3 \
▷ ▷ -define all_one -vector -repeat 1 7 -end \
▷ ▷ -draw_matrix -input_csv_file GF_q7_table_add.csv \
▷ ▷ ▷ -box_width 40 -bit_depth 24 \
▷ ▷ ▷ -partition 3 all_one all_one \
▷ ▷ -end \
▷ ▷ -draw_matrix -input_csv_file GF_q7_table_mul.csv \
▷ ▷ ▷ -box_width 40 -bit_depth 24 \
▷ ▷ ▷ -partition 3 all_one all_one \
▷ ▷ -end
▷ convert GF_q7_table_add.draw.bmp GF_q7_table_add.draw.png
▷ convert GF_q7_table_mul.draw.bmp GF_q7_table_mul.draw.png
```

creates the addition and multiplication tables for the field, shown below



Suppose we want to check Wilson's theorem:

$$\prod_{a \in \mathbb{F}_q^\times} a = -1.$$

The following command verifies this statement for  $q = 11$ . We first create a vector of all nonzero field elements, which we take as the integers from 1 to 10. After that, we use the `-product_of` finite field activity. This computes the product of the elements in the given set.

**Example 61**

```

F_11_product_of_all_nonzero_elements:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 11 -end \
> > -define S -vector -field F -loop 1 11 1 -end \
> > -with F -do -finite_field_activity -product_of S -end

```

The answer is 10, which is congruent to  $-1 \pmod{11}$  :

Suppose we want to create the Vandermonde matrix whose entries are  $x_i^j$ . Here  $x_0, \dots, x_{q-1}$  are the elements of the field  $\mathbb{F}_q$  and  $j$  ranges from 0 to  $q-1$ . The following command does so for  $q = 7$ . The command also computes the inverse of the Vandermonde matrix.

**Example 62**

```

F_7_vandermonde:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 7 -end \
> > -with F -do -finite_field_activity \
> > > -Vandermonde_matrix \
> > -end

```

The output is shown below. The first matrix is  $V = (x_i^j)$ . The second matrix is  $V^{-1}$ :

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 3 & 2 & 5 & 4 & 1 \\ 0 & 6 & 5 & 3 & 3 & 5 & 6 \\ 0 & 6 & 6 & 1 & 6 & 1 & 1 \\ 0 & 6 & 3 & 5 & 5 & 3 & 6 \\ 0 & 6 & 5 & 4 & 3 & 2 & 1 \\ 6 & 6 & 6 & 6 & 6 & 6 & 6 \end{bmatrix}$$

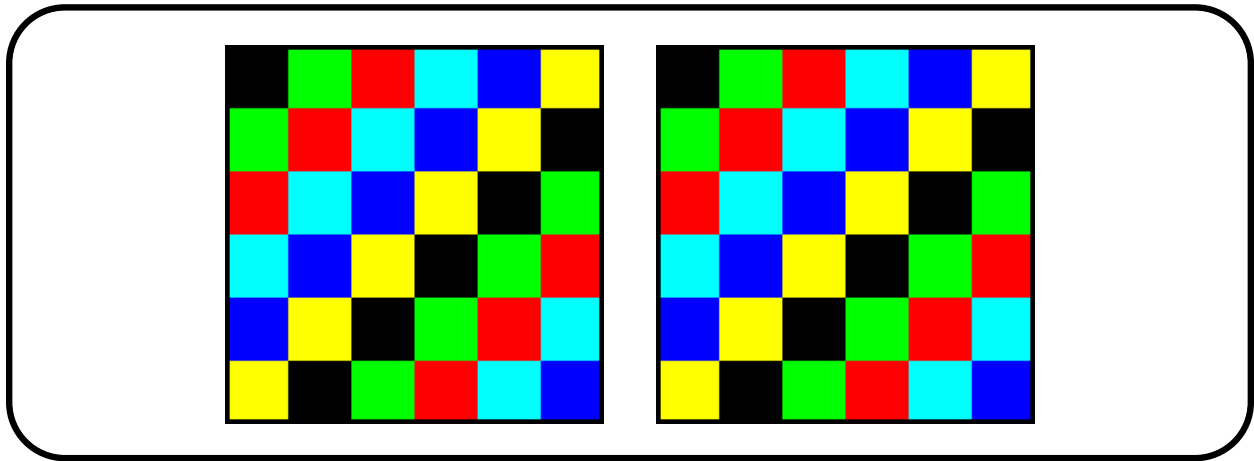
There is a second ordering of the elements which is used occasionally. In this labeling, every non-zero element is written as a power of a fixed primitive element. So, if  $\alpha$  is a primitive element, we arrange the elements of  $\mathbb{F}_p$  as

$$0, 1, \alpha, \alpha^2, \dots, \alpha^{q-2}.$$

The cheat sheet contains this list of field elements at the very end. It is sometimes convenient to arrange the elements in the order of powers of a primitive element. In  $\mathbb{F}_7$ , 3 is a primitive element. The powers of 3 are

$$0, 3^0, 3^1, 3^2, \dots, 3^6 = 0, 1, 3, 2, 6, 4, 5, 1 \pmod{7}.$$

With respect to this ordering of elements, the addition and multiplication tables become



For small field orders, the Orbiter employs precomputed tables for the arithmetic operations such as addition and multiplication and computing inverses. Precomputing these tables can be time-consuming. The option `-without_tables` can be given to avoid precomputing tables. Of course, not by not precomputing the tables, arithmetic operations are somewhat slower than before. Here is an example. We create the field  $\mathbb{F}_{101}$  without precomputed tables:

#### Example 63

```
F_101_wo:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 101 -without_tables -end \
▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
▷ pdflatex GF_101.tex
▷ $(OPEN) GF_101.pdf
```

See Section 19.2 for a list of limitations of Orbiter. For instance, the possible range of  $q$  is restricted to the machine integers.

### 3.2 Extension Fields

Let  $E$  and  $F$  be fields. Recall that  $E$  is an extension of  $F$  if  $F$  is a subfield of  $E$ . In this case,  $E$  is a vector space over  $F$ , which we denote as  $E/F$ . The degree of  $E/F$  is the dimension of  $E/F$ . An example of a field extension is a field of the form  $E = F(\alpha)$ , where  $\alpha$  is any element over  $F$ . Here,  $F(\alpha)$  is the smallest field which contains  $F$  and  $\alpha$ . If  $\gamma \in E$  satisfies a polynomial equation with coefficients in  $F$ , then  $\gamma$  is called algebraic over  $F$ . The minimum polynomial of an element  $\gamma$  in  $E$  over  $F$  is the monic polynomial in  $F[X]$  of lowest degree which has  $\gamma$  as a root. A field extension  $E/F$  is algebraic if every element in  $E$  is algebraic over  $F$ . In particular,  $F(\alpha)$  is algebraic over  $F$  if  $\alpha$  is. The degree of  $F(\alpha)/F$  is the degree of the minimum polynomial of  $\alpha$  over  $F$ .

In this section, we will consider algebraic extensions of finite fields. If  $F = \mathbb{F}_q$  is a field of order  $q$ , then any algebraic extension  $E$  of  $F$  has order  $q^e$  where  $e$  is the degree of  $E$  over  $F$ . If  $E = F(\alpha)$  is algebraic, the degree of  $E$  over  $F$  is the degree of the minimum polynomial of  $E$  over  $F$ . If  $F = \mathbb{F}_q$  and  $E = F(\alpha)$  is algebraic of degree  $e$ , then  $|E| = q^e$ . Every finite field  $E$  is of this form, where  $F = \mathbb{F}_p$  and  $p$  is the characteristic of  $E$ . So, any algebraic extension  $E$  of  $F$  can be constructed as a polynomial factorring of the ring  $\mathbb{F}_p[X]$ . For a polynomial  $m(X)$  we consider the ideal

$$\mathbf{I}(m) = m(X)\mathbb{F}_p[X] = \{m(X)k(X) \mid k(X) \in \mathbb{F}_p[X]\}$$

of all polynomial multiples of  $m(X)$ . Under the assumption that  $m(X)$  has degree  $e > 1$  and is irreducible, the residue class ring

$$\mathbb{F}_p[X]/\mathbf{I}(m)$$

is a field with  $q = p^e$  elements. Each residue class has a canonical representative. The canonical representative is the unique element in the residue class modulo  $\mathbf{I}(m)$  which has degree less than  $e$  and leading coefficient one.

So, for instance, for  $q = 4 = 2^2$ , we can pick the irreducible polynomial  $m(X) = X^2 + X + 1$  over  $\mathbb{F}_2$ . We have four standard representatives modulo  $\mathbf{I}(m)$ , namely

$$\begin{aligned} &0, \\ &1, \\ &X, \\ &X + 1. \end{aligned}$$

Together, these make up a complete set of representatives of the residue classes modulo  $\mathbf{I}(m)$ , and hence can be identified with the elements of  $\mathbb{F}_4$ :

$$\mathbb{F}_4 = \{0, 1, X, X + 1\}.$$

The addition of polynomials is as in  $\mathbb{F}_2[X]$ , so

|         | 0       | 1       | $X$     | $X + 1$ |
|---------|---------|---------|---------|---------|
| 0       | 0       | 1       | $X$     | $X + 1$ |
| 1       | 1       | 0       | $X + 1$ | $X$     |
| $X$     | $X$     | $X + 1$ | 0       | 1       |
| $X + 1$ | $X + 1$ | $X$     | 1       | 0       |

To compute the multiplication table for the field  $\mathbb{F}_4$ . We can use polynomial arithmetic modulo  $m(X)$ : It is clear how multiplication by 0 or 1 works, so we need to focus on the polynomials  $X$  and  $X + 1$ :

$$\begin{aligned} X \cdot X &= X^2 &\equiv X + 1 &\pmod{X^2 + X + 1}, \\ X \cdot (X + 1) &= X^2 + X &\equiv 1 &\pmod{X^2 + X + 1}, \\ (X + 1) \cdot X &= X^2 + X &\equiv 1 &\pmod{X^2 + X + 1}, \\ (X + 1) \cdot (X + 1) &= X^2 + 1 &\equiv X &\pmod{X^2 + X + 1}, \end{aligned}$$

| More Finite Field Activities |                       |                                                                                                                                      |
|------------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Command                      | Arguments             | Purpose                                                                                                                              |
| -trace                       |                       | Computes the partition of the field elements according to the value of their absolute trace.                                         |
| -norm                        |                       | Computes the partition of the field elements according to the value of their absolute norm.                                          |
| -normal_basis                | $d$                   | Computes a normal basis for $\mathbb{F}_{q^d}$ over $\mathbb{F}_q$ .                                                                 |
| -nth_roots                   | $n$                   | Compute the minimum polynomials of all $n$ -th roots over $F$ .                                                                      |
| -field_reduction             | label $q$ $m$ $n$ $v$ | Compute the field reduction from $F$ to the subfield $\mathbb{F}_q$ of the $m \times n$ matrix whose coefficients are given in $v$ . |

Table 3.3: More Finite Field Activities

The multiplication table of  $\mathbb{F}_4$  is

|         | 0 | 1       | $X$     | $X + 1$ |
|---------|---|---------|---------|---------|
| 0       | 0 | 0       | 0       | 0       |
| 1       | 0 | 1       | $X$     | $X + 1$ |
| $X$     | 0 | $X$     | $X + 1$ | 1       |
| $X + 1$ | 0 | $X + 1$ | 1       | $X$     |

Table 3.3 lists Orbiter activities for finite fields. This extends Table 3.2 in Section 3.2.

The isomorphism type of the resulting field only depends on the order  $q$  of the field, and not on the choice of the polynomial. However, for practical computations, the choice of the polynomial matters. For instance, results can only be shared between different computer algebra systems if the same polynomials are used. Orbiter has a large collection of polynomials built in. Besides these, a polynomial can be specified. The polynomials that Orbiter offers are in fact primitive, which means that the root  $\alpha$  is a primitive element for the field  $\mathbb{F}_q$ . This just means that it is a generator of the multiplicative group. So, any non-zero element in  $\mathbb{F}_q$  is a suitable power of  $\alpha$ .

If  $\mathbb{F}_q$  is an extension of the prime field  $\mathbb{F}_p$ , we use a different labeling. This time, we exploit the fact that  $\mathbb{F}_q$  is a vector space over  $\mathbb{F}_p$ . Let  $\alpha$  be a root of the irreducible polynomial  $m(X) \in \mathbb{F}_p[X]$  used to create the field. Suppose that  $q = p^e$ , i.e., the degree of  $m(X)$  is  $e$ . An  $\mathbb{F}_p$ -basis for the vector space  $\mathbb{F}_q$  over  $\mathbb{F}_p$  is given by the powers  $\alpha^i$ , for  $0 \leq i < e$ . Therefore, any element  $\gamma$  of  $\mathbb{F}_q$  has a unique expression of the form

$$\gamma = \sum_{h=0}^{e-1} a_i \alpha^i, \quad 0 \leq a_i < p \text{ for all } i.$$

The associated integer rank of  $\gamma$  is obtained by replacing  $\alpha$  by  $p$  in this expression and evaluating the expression over the integers. So, the rank of  $\gamma$  is

$$\sum_{h=0}^{e-1} a_i p^i.$$

As  $\gamma$  ranges over all field element in  $\mathbb{F}_q$ , the rank values take on every value in the interval  $[0, q - 1]$ . The ordering of elements of  $\mathbb{F}_q$  according to these ranks is called the lexicographical ordering. The numerical rank of zero is 0 and the numerical rank of one is 1. The numerical rank of  $\alpha$ , the primitive element, is  $p$ . The numerical ranks of the elements of the prime subfield are exactly the elements of  $[0, p - 1]$ .



| Orbiter Primitive Polynomials |                             |           |                           |
|-------------------------------|-----------------------------|-----------|---------------------------|
| $q$                           | Polynomial                  | Numerical | Relation                  |
| 4                             | $X^2 + X + 1$               | 7         | $\omega^2 = \omega + 1$   |
| 8                             | $X^3 + X^2 + 1$             | 13        | $\gamma^3 = \gamma^2 + 1$ |
| 9                             | $X^2 + X + 2$               | 14        |                           |
| 16                            | $X^4 + X^3 + 1$             | 25        | $\delta^4 = \delta^3 + 1$ |
| 25                            | $X^2 + X + 2$               | 22        |                           |
| 27                            | $X^3 + 2X + 1$              | 34        |                           |
| 32                            | $X^5 + X^2 + 1$             | 37        | $\eta^5 = \eta^2 + 1$     |
| 49                            | $X^2 + X + 3$               | 59        |                           |
| 64                            | $X^6 + X^5 + 1$             | 97        |                           |
| 81                            | $X^4 + X^3 + 2$             | 110       |                           |
| 121                           | $X^2 + 4X + 2$              | 167       |                           |
| 125                           | $X^3 + X^2 + X + 2$         | 86        |                           |
| 128                           | $X^7 + X^6 + 1$             | 193       | $\zeta^7 = \zeta^6 + 1$   |
| 169                           | $X^2 + X + 2$               | 184       |                           |
| 243                           | $X^5 + 2X + 1$              | 250       |                           |
| 256                           | $X^8 + X^4 + X^3 + X^2 + 1$ | 285       |                           |
| 289                           | $X^2 + X + 3$               | 309       |                           |
| 343                           | $X^3 + 3X + 2$              | 366       |                           |
| 361                           | $X^2 + X + 2$               | 382       |                           |
| 512                           | $X^9 + X^4 + 1$             | 529       |                           |
| 529                           | $X^2 + 2X + 5$              | 580       |                           |
| 625                           | $X^4 + X^3 + X + 2$         | 326       |                           |
| 729                           | $X^6 + X^5 + 2$             | 974       |                           |
| 841                           | $X^2 + 5X + 2$              | 988       |                           |
| 961                           | $X^2 + 2X + 3$              | 1026      |                           |
| 1024                          | $X^{10} + X^3 + 1$          | 1033      |                           |

Table 3.4: Orbiter Primitive Polynomials

The primitive polynomials used by Orbiter to create small finite fields are listed in Table 3.4. The relation is given using the Greek letter that is used in orbiter cheat sheets for that particular field.

Let us look at a few examples. The command

#### Example 64

```
F_4:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -with F -do -finite_field_activity -cheat_sheet_GF -end
> pdflatex GF_4.tex
> $(OPEN) GF_4.pdf
```

creates a report for the field  $\mathbb{F}_4$ .

The command

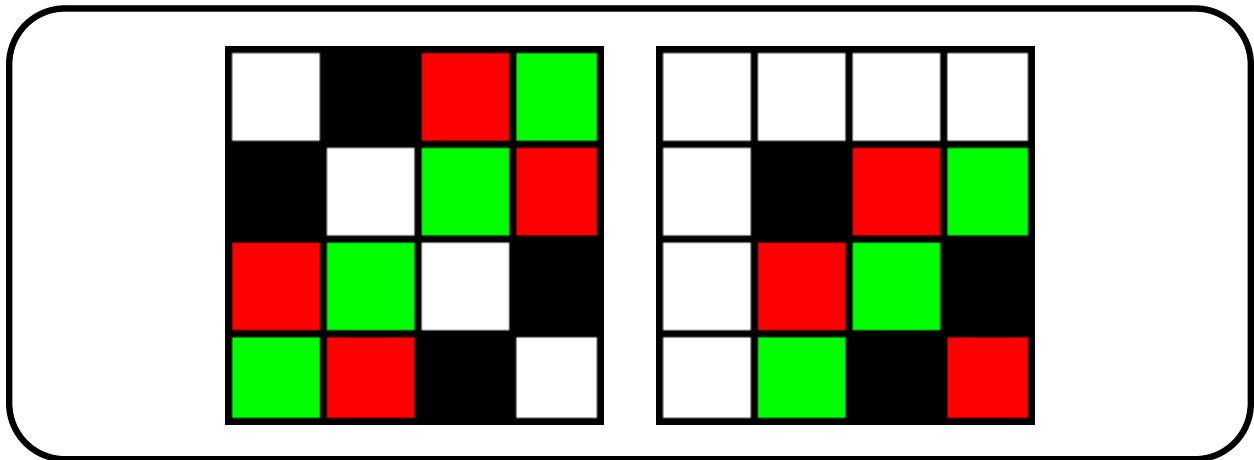
**Example 65**

```

F_4_tables:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -with F -do -finite_field_activity -export_tables -end
> $(ORBITER) -v 3 \
> > -define all_one -vector -repeat 1 4 -end \
> > -draw_matrix -input_csv_file GF_q4_table_add.csv \
> > > -box_width 40 -bit_depth 24 \
> > > -partition 3 all_one all_one \
> > -end \
> > -draw_matrix -input_csv_file GF_q4_table_mul.csv \
> > > -box_width 40 -bit_depth 24 \
> > > -partition 3 all_one all_one \
> > -end
> convert GF_q4_table_add.draw.bmp GF_q4_table_add.draw.png
> convert GF_q4_table_mul.draw.bmp GF_q4_table_mul.draw.png

```

creates the addition and multiplication tables for the field:



The command

**Example 66**

```

F_16:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 16 -compute_related_fields -end \
> > -with F -do -finite_field_activity -cheat_sheet_GF -end
> pdflatex GF_16.tex
> $(OPEN) GF_16.pdf

```

creates a cheat sheet for  $\mathbb{F}_{16}$ .

polynomial:  $X^4 + X^3 + 1 = 25$

$Z_i = \log_{\alpha}(1 + \alpha^i)$

Subfields:

| Subfield       | Polynomial    | Numerical Rank |
|----------------|---------------|----------------|
| $\mathbb{F}_4$ | $X^2 + X + 1$ | 7              |

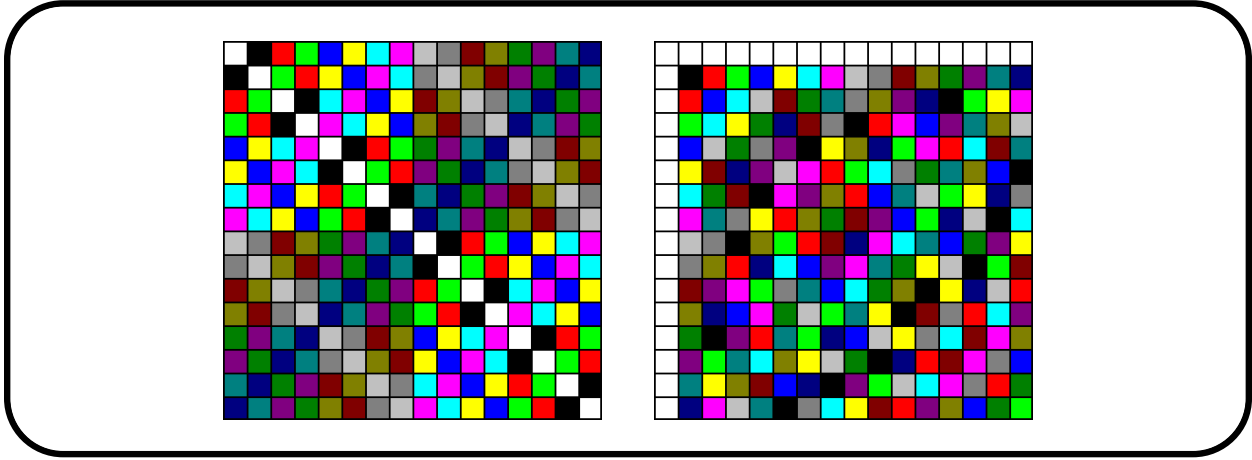
| $i$ | $\gamma_i$                                    | $-\gamma_i$ | $\gamma_i^{-1}$ | $\log_{\alpha}(\gamma_i)$ | $\alpha^i$ | $Z_i$ | $\phi(\gamma_i)$ | $T(\gamma_i)$ | $N(\gamma_i)$ | $T_2(\gamma_i)$ | $N_2(\gamma_i)$ |
|-----|-----------------------------------------------|-------------|-----------------|---------------------------|------------|-------|------------------|---------------|---------------|-----------------|-----------------|
| 0   | $0 = 0$                                       | 0           | DNE             | DNE                       | 1          | DNE   | 0                | 0             | 0             | 0               | 0               |
| 1   | $1 = 1$                                       | 1           | 1               | 15                        | 2          | 12    | 1                | 0             | 1             | 0               | 1               |
| 2   | $\alpha = \delta$                             | 2           | 12              | 1                         | 4          | 9     | 4                | 1             | 1             | 11              | 11              |
| 3   | $\alpha + 1 = \delta^{12}$                    | 3           | 8               | 12                        | 8          | 4     | 5                | 1             | 1             | 11              | 1               |
| 4   | $\alpha^2 = \delta^2$                         | 4           | 6               | 2                         | 9          | 3     | 9                | 1             | 1             | 10              | 10              |
| 5   | $\alpha^2 + 1 = \delta^9$                     | 5           | 15              | 9                         | 11         | 10    | 8                | 1             | 1             | 10              | 1               |
| 6   | $\alpha^2 + \alpha = \delta^{13}$             | 6           | 4               | 13                        | 15         | 8     | 13               | 0             | 1             | 1               | 11              |
| 7   | $\alpha^2 + \alpha + 1 = \delta^7$            | 7           | 14              | 7                         | 7          | 13    | 12               | 0             | 1             | 1               | 11              |
| 8   | $\alpha^3 = \delta^3$                         | 8           | 3               | 3                         | 14         | 6     | 15               | 1             | 1             | 11              | 1               |
| 9   | $\alpha^3 + 1 = \delta^4$                     | 9           | 13              | 4                         | 5          | 2     | 14               | 1             | 1             | 11              | 11              |
| 10  | $\alpha^3 + \alpha = \delta^{10}$             | 10          | 11              | 10                        | 10         | 5     | 11               | 0             | 1             | 0               | 11              |
| 11  | $\alpha^3 + \alpha + 1 = \delta^5$            | 11          | 10              | 5                         | 13         | 14    | 10               | 0             | 1             | 0               | 10              |
| 12  | $\alpha^3 + \alpha^2 = \delta^{14}$           | 12          | 2               | 14                        | 3          | 1     | 6                | 0             | 1             | 1               | 10              |
| 13  | $\alpha^3 + \alpha^2 + 1 = \delta^{11}$       | 13          | 9               | 11                        | 6          | 7     | 7                | 0             | 1             | 1               | 10              |
| 14  | $\alpha^3 + \alpha^2 + \alpha = \delta^8$     | 14          | 7               | 8                         | 12         | 11    | 2                | 1             | 1             | 10              | 10              |
| 15  | $\alpha^3 + \alpha^2 + \alpha + 1 = \delta^6$ | 15          | 5               | 6                         | 1          | DNE   | 3                | 1             | 1             | 10              | 1               |

The command

#### Example 67

```
F_16_tables:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 16 -end \
> > -with F -do -finite_field_activity -export_tables -end
> $(ORBITER) -v 3 \
> > -define all_one -vector -repeat 1 16 -end \
> > -draw_matrix -input_csv_file GF_q16_table_add.csv \
> > > -box_width 40 -bit_depth 24 \
> > > -partition 3 all_one all_one \
> > -end \
> > -draw_matrix -input_csv_file GF_q16_table_mul.csv \
> > > -box_width 40 -bit_depth 24 \
> > > -partition 3 all_one all_one \
> > -end
> convert GF_q16_table_add_draw.bmp GF_q16_table_add_draw.png
> convert GF_q16_table_mul_draw.bmp GF_q16_table_mul_draw.png
```

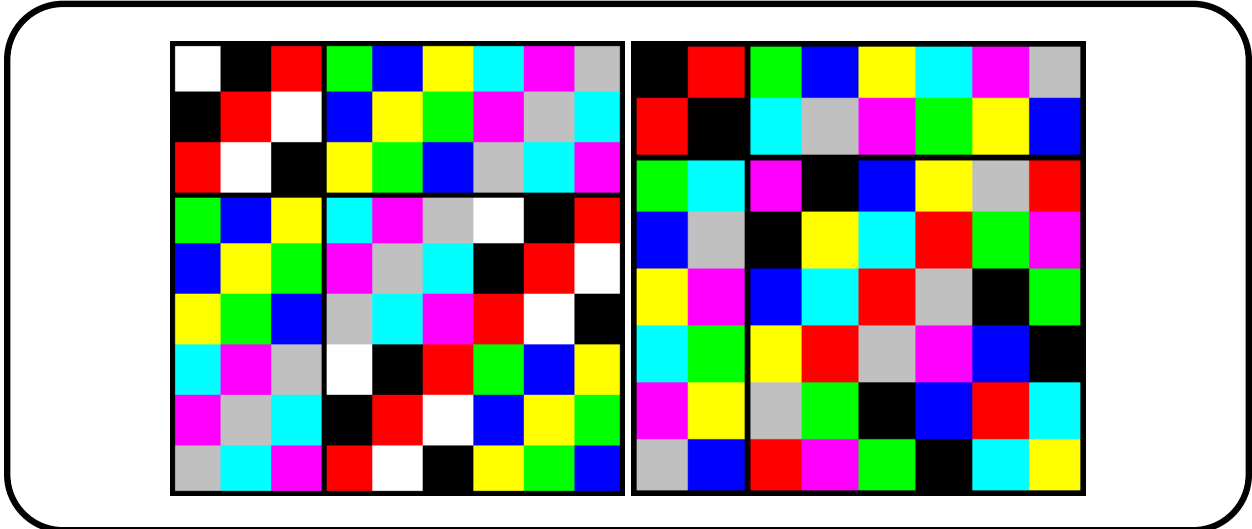
creates the addition and multiplication tables for the field:



Unlike other GAP [28] and Magma [16], Orbiter does not use Conway polynomials to create field extensions. Instead, it provides the option to override the polynomial used to create the finite field. For subfield relationships, the cheat sheet will indicate the irreducible polynomials of all subfields for a given field. The table below is taken from a cheat sheet for the field  $\mathbb{F}_{64}$  generated by the polynomial  $X^6 + X^5 + 1$ , whose numerical rank is 97:

| Subfield       | Polynomial    | Numerical rank |
|----------------|---------------|----------------|
| $\mathbb{F}_4$ | $X^2 + X + 1$ | 7              |
| $\mathbb{F}_8$ | $X^3 + X + 1$ | 11             |

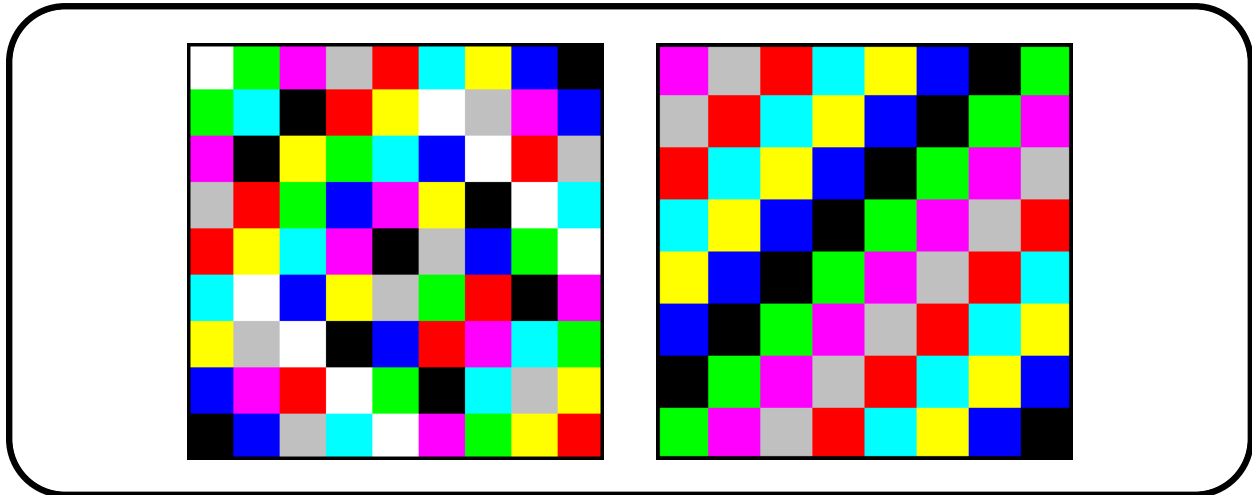
The lexicographic ordering has an interesting side-effect for the ordering of elements in extension fields. The elements of the prime subfield are always listed before any other elements in the extension field. For this reason, the addition and multiplication tables of the extension field contain the respective table of the prime field in the upper left corner, as shown below for the field  $\mathbb{F}_9$ :



Here, we omit the zero element in the multiplication table.

Orbiter uses primitive polynomials for creating extension fields. Because of this, the element  $\alpha$  is always primitive. Since the numerical rank of  $\alpha$  is  $p$ , this means that the rank  $p$  always represents a primitive

element in an extension field. For the addition and multiplication tables of  $\mathbb{F}_9$  arranged with respect to powers of a primitive element, see below:



The command

#### Example 68

```
nth_roots.63:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 2 -end \
> > -with F -do -finite_field_activity \
> > > -nth_roots 63 \
> > -end
> pdflatex Nth_roots.q2_n63.tex
> $(OPEN) Nth_roots.q2_n63.pdf
```

creates the minimum polynomials of all elements in  $\mathbb{F}_{64}^\times$ . This corresponds to all irreducible polynomials over  $\mathbb{F}_2$  of degree dividing 6, apart from the polynomial  $X$ .

| $i$ | $r_i$ | $\text{Cyc}(r_i)$        | $m_{\beta^{r_i}}(X)$                                                                              | $m_{\beta^{r_i}}(X)$        | rank |
|-----|-------|--------------------------|---------------------------------------------------------------------------------------------------|-----------------------------|------|
| 0   | 0     | (0)                      | $(100000)X^0 + (100000)X^1$                                                                       | $X + 1$                     | 3    |
| 1   | 1     | (1, 2, 4, 8, 16, 32)     | $(100000)X^0 + (000000)X^1 + (000000)X^2 + (000000)X^3 + (000000)X^4 + (100000)X^5 + (100000)X^6$ | $X^6 + X^5 + 1$             | 97   |
| 2   | 3     | (3, 6, 12, 24, 48, 33)   | $(100000)X^0 + (000000)X^1 + (100000)X^2 + (000000)X^3 + (100000)X^4 + (100000)X^5 + (100000)X^6$ | $X^6 + X^5 + X^4 + X^2 + 1$ | 117  |
| 3   | 5     | (5, 10, 20, 40, 17, 34)  | $(100000)X^0 + (100000)X^1 + (000000)X^2 + (000000)X^3 + (100000)X^4 + (100000)X^5 + (100000)X^6$ | $X^6 + X^5 + X^4 + X + 1$   | 115  |
| 4   | 7     | (7, 14, 28, 56, 49, 35)  | $(100000)X^0 + (000000)X^1 + (000000)X^2 + (100000)X^3 + (000000)X^4 + (000000)X^5 + (100000)X^6$ | $X^6 + X^3 + 1$             | 73   |
| 5   | 9     | (9, 18, 36)              | $(100000)X^0 + (100000)X^1 + (000000)X^2 + (100000)X^3$                                           | $X^3 + X + 1$               | 11   |
| 6   | 11    | (11, 22, 44, 25, 50, 37) | $(100000)X^0 + (100000)X^1 + (000000)X^2 + (100000)X^3 + (100000)X^4 + (000000)X^5 + (100000)X^6$ | $X^6 + X^4 + X^3 + X + 1$   | 91   |
| 7   | 13    | (13, 26, 52, 41, 19, 38) | $(100000)X^0 + (000000)X^1 + (100000)X^2 + (100000)X^3 + (000000)X^4 + (100000)X^5 + (100000)X^6$ | $X^6 + X^5 + X^3 + X^2 + 1$ | 109  |
| 8   | 15    | (15, 30, 60, 57, 51, 39) | $(100000)X^0 + (100000)X^1 + (100000)X^2 + (000000)X^3 + (100000)X^4 + (000000)X^5 + (100000)X^6$ | $X^6 + X^4 + X^2 + X + 1$   | 87   |
| 9   | 21    | (21, 42)                 | $(100000)X^0 + (100000)X^1 + (100000)X^2$                                                         | $X^2 + X + 1$               | 7    |
| 10  | 23    | (23, 46, 29, 58, 53, 43) | $(100000)X^0 + (100000)X^1 + (100000)X^2 + (000000)X^3 + (000000)X^4 + (100000)X^5 + (100000)X^6$ | $X^6 + X^5 + X^2 + X + 1$   | 103  |
| 11  | 27    | (27, 54, 45)             | $(100000)X^0 + (000000)X^1 + (100000)X^2 + (100000)X^3$                                           | $X^3 + X^2 + 1$             | 13   |
| 12  | 31    | (31, 62, 61, 59, 55, 47) | $(100000)X^0 + (100000)X^1 + (000000)X^2 + (000000)X^3 + (000000)X^4 + (000000)X^5 + (100000)X^6$ | $X^6 + X + 1$               | 67   |

The command

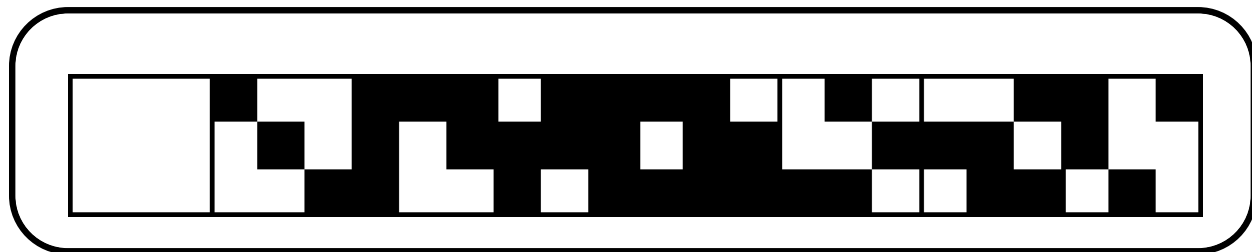
#### Example 69

```

F8_F2_field_reduction:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define v -vector -loop 0 8 1 -end \
▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -field_reduction "F8_to_F2" \
▷ ▷ ▷ 2 1 8 v \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix -input_csv_file F8_to_F2.csv \
▷ ▷ -box_width 40 -bit_depth 24 \
▷ ▷ -partition 4 "3" "3,3,3,3,3,3,3" -end
▷ convert F8_to_F2_draw.bmp F8_to_F2_draw.png
▷ $(OPEN) F8_to_F2_draw.png

```

performs field reduction on the elements  $0, \dots, 7$  in  $\mathbb{F}_8$  down to  $\mathbb{F}_2$ . Here is a graphical representation of the matrices arising this way:



Let us discuss field embeddings in Orbiter. This is important for working in field extensions, as there is a restriction on the polynomials used to create the fields. Suppose we have fields  $\mathbb{F}_q$  and  $\mathbb{F}_Q$  with

$$\mathbb{F}_q \leq \mathbb{F}_Q,$$

i.e.

$$Q = q^e$$

for some positive integer  $e$ . Suppose  $\alpha$  is a primitive element in  $\mathbb{F}_Q$  and  $\beta$  is a primitive element in  $\mathbb{F}_q$ . Then Orbiter assumes that

$$\beta = \alpha^{\frac{Q-1}{q-1}},$$

where

$$\frac{Q-1}{q-1}$$

is the index of  $\mathbb{F}_q^\times$  in the multiplicative group  $\mathbb{F}_Q^\times$ . With respect to this embedding, the minimum polynomial of  $\beta$  is determined. Let us consider an example. The command

#### Example 70

```
F_4096:
> $(ORBITER) -v 4 \
> > -define F -finite_field -q 4096 \
> > > -compute_related_fields \
> > -end \
> > -with F -do -finite_field_activity -cheat_sheet_GF -end
> pdflatex GF_4096.tex
> $(OPEN) GF_4096.pdf
```

creates the field  $\mathbb{F}_{2^{12}}$ . Orbiter chooses the polynomial

$$X^{12} + X^6 + X^4 + X + 1$$

to create the field. This field has non-trivial subfields  $\mathbb{F}_4$ ,  $\mathbb{F}_8$ ,  $\mathbb{F}_{16}$  and  $\mathbb{F}_{64}$ . The report shows the polynomials for the primitive elements of the subfields:

| Subfield          | Polynomial      | Numerical Rank |
|-------------------|-----------------|----------------|
| $\mathbb{F}_4$    | $X^2 + X + 1$   | 7              |
| $\mathbb{F}_8$    | $X^3 + X + 1$   | 11             |
| $\mathbb{F}_{16}$ | $X^4 + X + 1$   | 19             |
| $\mathbb{F}_{64}$ | $X^6 + X^5 + 1$ | 97             |

For each subfield, including the prime field, Orbiter computes a subfield basis and the embedding of the elements in the subfield. We use the convention that  $d = \frac{Q-1}{q-1}$ . Here is the output from the report:

Subfield  $\mathbb{F}_2$  generated by polynomial 3:

Subfield of Order 2:

polynomial:

Field basis:

(1, 83, 342, 598, 1143, 518, 1492, 938, 1881, 3381, 1159, 227)

The power table shows  $(\alpha^d)^i$  for  $i = 0, 1, \dots, e$  where  $q = p^e$ :

| $i$ | $i \cdot d$ | $\alpha^{id}$ | vector                               |
|-----|-------------|---------------|--------------------------------------|
| 0   | 0           | 1             | (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1) |
| 1   | 4095        | 1             | (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1) |

Subfield  $\mathbb{F}_4$  generated by polynomial 7:

Subfield of Order 4:

polynomial: 7

Field basis:

(1, 64, 83, 1171, 342, 1183)

The power table shows  $(\alpha^d)^i$  for  $i = 0, 1, \dots, e$  where  $q = p^e$ :

| $i$ | $i \cdot d$ | $\alpha^{id}$ | vector                                  |
|-----|-------------|---------------|-----------------------------------------|
| 0   | 0           | 1             | (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)    |
| 1   | 1365        | 70            | (0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0) |
| 2   | 2730        | 71            | (0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1) |

Subfield  $\mathbb{F}_8$  generated by polynomial 11:

Subfield of Order 8:

polynomial: 11

Field basis:

(1, 16, 256, 83)



The power table shows  $(\alpha^d)^i$  for  $i = 0, 1, \dots, e$  where  $q = p^e$ :

| $i$ | $i \cdot d$ | $\alpha^{id}$ | vector                               |
|-----|-------------|---------------|--------------------------------------|
| 0   | 0           | 1             | (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1) |
| 1   | 585         | 937           | (0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1) |
| 2   | 1170        | 1198          | (0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0) |
| 3   | 1755        | 936           | (0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0) |

Subfield  $\mathbb{F}_{16}$  generated by polynomial 19:

Subfield of Order 16:

polynomial: 19

Field basis:

$$(1, 8, 64)$$

The power table shows  $(\alpha^d)^i$  for  $i = 0, 1, \dots, e$  where  $q = p^e$ :

| $i$ | $i \cdot d$ | $\alpha^{id}$ | vector                               |
|-----|-------------|---------------|--------------------------------------|
| 0   | 0           | 1             | (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1) |
| 1   | 273         | 1971          | (0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1) |
| 2   | 546         | 2037          | (0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1) |
| 3   | 819         | 3214          | (1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0) |
| 4   | 1092        | 1970          | (0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0) |

Subfield  $\mathbb{F}_{64}$  generated by polynomial 97:

Subfield of Order 64:

polynomial: 97

Field basis:

$$(1, 4)$$

The power table shows  $(\alpha^d)^i$  for  $i = 0, 1, \dots, e$  where  $q = p^e$ :

| $i$ | $i \cdot d$ | $\alpha^{id}$ | vector                               |
|-----|-------------|---------------|--------------------------------------|
| 0   | 0           | 1             | (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1) |
| 1   | 65          | 396           | (0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0) |
| 2   | 130         | 1068          | (0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0) |
| 3   | 195         | 1315          | (0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1) |
| 4   | 260         | 1615          | (0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1) |
| 5   | 325         | 811           | (0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1) |
| 6   | 390         | 810           | (0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0) |

The command

**Example 71**

```
F_4489:  
▷ $(ORBITER) -v 4 \  
▷ ▷ -define F -finite_field -q 4489 -without_tables -end \  

```

creates the field  $\mathbb{F}_{67^2}$ . For the sake of speed, the field tables are not computed.

| Linear Algebra Activities |             |                                                                                                                                                                                                                                                                                         |
|---------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                   | Arguments   | Purpose                                                                                                                                                                                                                                                                                 |
| -nullspace                | $m \ n \ L$ | Compute a basis for the right nullspace of the $m \times n$ matrix $L$                                                                                                                                                                                                                  |
| -RREF                     | $m \ n \ L$ | Compute the RREF of the $m \times n$ matrix $L$ over $\mathbb{F}_q$                                                                                                                                                                                                                     |
| -normalize_from_the_left  |             | Compute left normalized RREF or nullspace.                                                                                                                                                                                                                                              |
| -normalize_from_the_right |             | Compute right normalized RREF or nullspace.                                                                                                                                                                                                                                             |
| -RREF_random_matrix       | $m \ n$     | Produce a random $m \times n$ matrix over $F$ and perform RREF.                                                                                                                                                                                                                         |
| -Walsh_matrix             | $n$         | Create the Walsh matrix of order $n$ .                                                                                                                                                                                                                                                  |
| -Vandermonde_matrix       |             | Create the Vandermonde matrix of order $q$ and compute its inverse. The $(i, j)$ entry is $\gamma_i^j$ ( $i, j = 0, \dots, q-1$ ). Here, $\gamma_0, \dots, \gamma_{q-1}$ is the list of elements in $\mathbb{F}_q$ in Orbiter ordering. The matrix and its inverse are written to file. |

Table 3.5: Linear Algebra Activities

### 3.3 Linear Algebra Over Finite Fields

In Table 3.5, some finite field activities regarding linear algebra are shown. For instance, the command

#### Example 72

```

RREF:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F -format 2 \
> > > -dense "1,1,1,1,0,1,1,0,0,1" \
> > -end \
> > -with F -do -finite_field_activity \
> > > -RREF v \
> > -end
> pdflatex v_rref.tex
> $(OPEN) v_rref.pdf

```

computes the RREF form of the matrix

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

over  $\mathbb{F}_2$ . The output is the matrix

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

The `-RREF` command produces a latex log of the steps. This can be used to follow the algorithm along. For a somewhat longer example, consider the Vandermonde matrix over the field  $\mathbb{F}_7$ . Suppose we want to compute the inverse matrix directly. We can use the following command to do so. Notice how we first create the matrix and an identity matrix next to it. After that we apply the `-RREF` command:

**Example 73**

```
V7_VANDERMONDE_EXTENDED="\
1,0,0,0,0,0,0,1,0,0,0,0,0,0, \
1,1,1,1,1,1,1,0,1,0,0,0,0,0, \
1,2,4,1,2,4,1,0,0,1,0,0,0,0, \
1,3,2,6,4,5,1,0,0,0,1,0,0,0, \
1,4,2,1,4,2,1,0,0,0,0,1,0,0, \
1,5,4,6,2,3,1,0,0,0,0,0,1,0, \
1,6,1,6,1,6,1,0,0,0,0,0,0,1"
```

**Example 74**

```
RREF_V7:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -define V7 -vector -format 7 \
> > > -dense $(V7_VANDERMONDE_EXTENDED) \
> > -end \
> > -with F -do -finite_field_activity \
> > > -RREF V7 \
> > -end
> pdflatex V7_rref.tex
> $(OPEN) V7_rref.pdf
```

The following (shortened) output is produced. Observe how the inverse matrix appears in the second half once the `-RREF` algorithm is finished:

A matrix over the field  $\mathbb{F}_7$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Position  $(i, j) = (0, 0)$ , found pivot in column 0

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

After making pivot 1:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

output truncated

After elimination above pivot 0 in position (0,0):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 3 & 2 & 5 & 4 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 6 & 5 & 3 & 3 & 5 & 6 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 6 & 6 & 1 & 6 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 6 & 3 & 5 & 5 & 3 & 6 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 6 & 5 & 4 & 3 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 6 & 6 & 6 & 6 & 6 & 6 & 6 \end{bmatrix}$$

The inverse matrix agrees with the output obtained in Section 3.1.

Another task is computing the nullspace of a matrix. The command

#### Example 75

```
nullspace:
> $(ORBITER) -v 2 \
> > -define F2 -finite_field -q 2 -end \
> > -define v -vector -field F2 -format 2 \
> > > -dense "1,1,1,1,0,1,1,0,0,1" \
> > -end \
> > -with F2 -do \
> > -finite_field_activity \
> > > -nullspace v \
> > -end
> pdflatex v_nullspace.tex
> $(OPEN) v_nullspace.pdf
```

computes the right nullspace of the matrix from the first example. The output is the matrix

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}.$$

Orbiter can compute eigenvalues and eigenvectors of matrices over finite fields. For instance, the command

#### Example 76

```
eigenstuff:
> $(ORBITER) -v 6 \
> > -define F -finite_field -q 5 -end \
> > -eigenstuff F 4 "0,1,0,2,0,1,2,1,4,2,3,1,2,0,4,3"
```

computes all eigenvectors and eigenvalues of the matrix

$$\begin{bmatrix} 0 & 1 & 0 & 2 \\ 0 & 1 & 2 & 1 \\ 4 & 2 & 3 & 1 \\ 2 & 0 & 4 & 3 \end{bmatrix}$$

over  $\mathbb{F}_5$ .

Orbiter can produce a list of all conjugacy classes of endomorphisms of  $\mathbb{F}_q^d$  by means of their rational normal forms. For instance

#### Example 77

```
classes_GL_3_2:
> $(ORBITER) -v 7 \
> > -define F -finite_field -q 2 -end \
> > -all_rational_normal_forms F 3
> pdflatex Class_reps_GL_3_2.tex
> $(OPEN) Class_reps_GL_3_2.pdf
```

produces a list of all conjugacy classes of  $\text{GL}(3, 2)$ . There are 6 of them. The report includes the order of the centralizer and the order of the conjugacy class. The order of the centralizer is computed using Kung's formula [42]. This command relies on the Orbiter catalogue of irreducible polynomials. For an introduction to the rational normal form of endomorphisms, see [48].

## Conjugacy Classes of $GL(3, 2)$

The number of conjugacy classes of  $GL(3, 2)$  is 6:

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Class 0 / 6

3, 1, 0

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

centralizer order 7

class size 24

Class 1 / 6

2, 1, 0

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

centralizer order 7

class size 24

Class 2 / 6

0, 1, 0; 1, 1, 0

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

centralizer order 3

class size 56

Class 3 / 6

0, 3, 0

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

centralizer order 4

class size 42

Class 4 / 6

0, 3, 1

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

centralizer order 8  
class size 21  
Class 5 / 6  
0, 3, 2

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

centralizer order 168  
class size 1



| Finite Field Activities (Part I)           |               |                                                                                                 |
|--------------------------------------------|---------------|-------------------------------------------------------------------------------------------------|
| Command                                    | Arguments     | Purpose                                                                                         |
| -write_code_for_division                   | fname $A$ $B$ | Write C++ source code for the polynomial division of $A$ by $B$ . See Section 10.4.             |
| -weight_enumerator                         | $A$           | Computes the weight enumerator of the code whose generator matrix is $A$ .                      |
| -Walsh_Hadamard_transform                  | fname $n$     | Computes the Walsh-Hadamard transform for the $n$ -variable boolean function in the given file. |
| -algebraic_normal_form_of_boolean_function | fname $n$     | Computes the algebraic normal form for the $n$ -variable boolean function in the given file.    |
| -algebraic_normal_form                     | $n$ input     | Computes the algebraic normal form for the $n$ -variable function in the given file.            |
| -apply_trace_function                      | fname         | Applies the absolute trace function to the function in the given file.                          |
| -apply_power_function                      | fname $d$     | Applies the raise-to-the-power- $d$ function to the function in the given file.                 |
| -identity_function                         | fname.csv     | Creates the identity function and stores in the given csv file.                                 |

Table 3.6: Finite Field Activities (Part I)

### 3.4 Advanced Topics in Finite Fields

Let us now look at some advanced topics in the theory of finite fields.

First, in Tables 3.6-3.7, a summary of finite field activities is shown.

A normal basis for a field extension  $\mathbb{F}_{q^d}$  over  $\mathbb{F}_q$  is a basis of  $\mathbb{F}_{q^d}$  as vector space over  $\mathbb{F}_q$  which consists of one cycle of the Frobenius automorphism of  $\mathbb{F}_{q^d}$  over  $\mathbb{F}_q$ . For instance, the command

#### Example 78

```
normal_basis_2.3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -with F -do -finite_field_activity \
▷ ▷ -normal_basis 3 -end
▷ pdflatex normal_basis_q2_d3.tex
▷ $(OPEN) normal_basis_q2_d3.pdf
```

computes a normal basis of  $\mathbb{F}_8$  over  $\mathbb{F}_2$ . Using the polynomial  $X^3 + X^2 + 1$ , the normal basis in terms of the standard polynomial basis  $1, X, X^2, \dots$  is given by the columns of the matrix

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Reading the columns as coefficient vectors with respect to the standard basis, the normal basis is

$$b_1 = 1 + X + X^2, \quad b_2 = X, \quad b_3 = X^2.$$

| Finite Field Activities (Part II)  |             |                                                                                                                                                                 |
|------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                            | Arguments   | Purpose                                                                                                                                                         |
| -transversal                       | $L1\ L2\ P$ | Computes the unique transversal to the lines $L1$ and $L2$ through the point $P$ in $PG(3, q)$ . The lines are given by a basis consisting of 8 field elements. |
| -intersection_of_two_lines         | $L1\ L2$    | Computes the intersection of two lines in $PG(3, q)$ . The lines are given by a basis consisting of 8 field elements.                                           |
| -rank_point_in_PG                  | $P$         | Computes the orbiter point rank corresponding to the point $P$ in $PG(n, q)$ . $P$ is a label of a vector, which is the coefficient vector.                     |
| -unrank_point_in_PG                | $r$         | Computes the coordinate vector of the Orbiter point in $PG(n, q)$ corresponding to the Orbiter rank value $r$ .                                                 |
| -inverse_isomorphism_klein_quadric | $L36$       |                                                                                                                                                                 |
| -NTT                               | $k\ n$      | Computes the Number-theoretic transform for $n = 2^k$ , which must divide $q - 1$ .                                                                             |

Table 3.7: Finite Field Activities (Part II)

Let us apply the Frobenius mapping  $\Phi$  to the elements of the normal bases:

$$\begin{aligned}
 b_1^\Phi &= (1 + X + X^2)^2 = 1 + X^2 + X^4 = 1 + X^2 + X^3 + X = 1 + X + X^2 + X^2 + 1 = X = b_2, \\
 b_2^\Phi &= X^2 = b_3, \\
 b_3^\Phi &= X^4 = X^3 + X = X^2 + X + 1 = b_1.
 \end{aligned}$$

Thus,

$$b_1 \mapsto b_2 \mapsto b_3 \mapsto b_1$$

under  $\Phi$ , as required.

A field is a vector space over any of its subfields. Using a field basis, the elements of the large field can be identified with invertible matrices. So, for  $\mathbb{F}_{q^r}$  over  $\mathbb{F}_q$ , and for  $a \in \mathbb{F}_{q^r}$ , we consider the  $\mathbb{F}_q$ -linear map

$$\mathbb{F}_{q^r} \rightarrow \mathbb{F}_{q^r}, x \mapsto ax.$$

The following code computes the field reduction from  $\mathbb{F}_{64}$  to  $\mathbb{F}_8$ . Elements in the small field are represented as colors. The  $(i, j)$ -th block is the matrix of  $a = i8 + j$  in the chosen basis.

#### Example 79

```

F_64.over_F8.field_reduction:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 64 -end \
> > -define elts -vector -field F -loop 0 64 1 -end \
> > -with F -do \
> > -finite_field_activity -field_reduction "F64_over_F8" 8 8 8 \
> > > elts -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file F64_over_F8.csv \
> > -box_width 40 -bit_depth 24 \

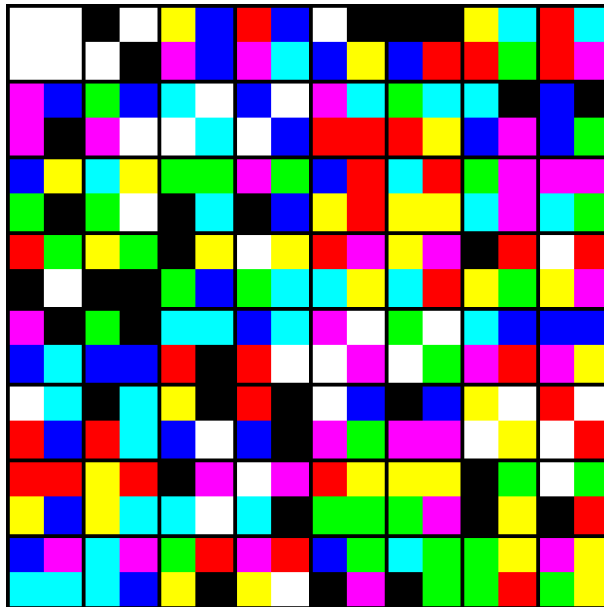
```

```

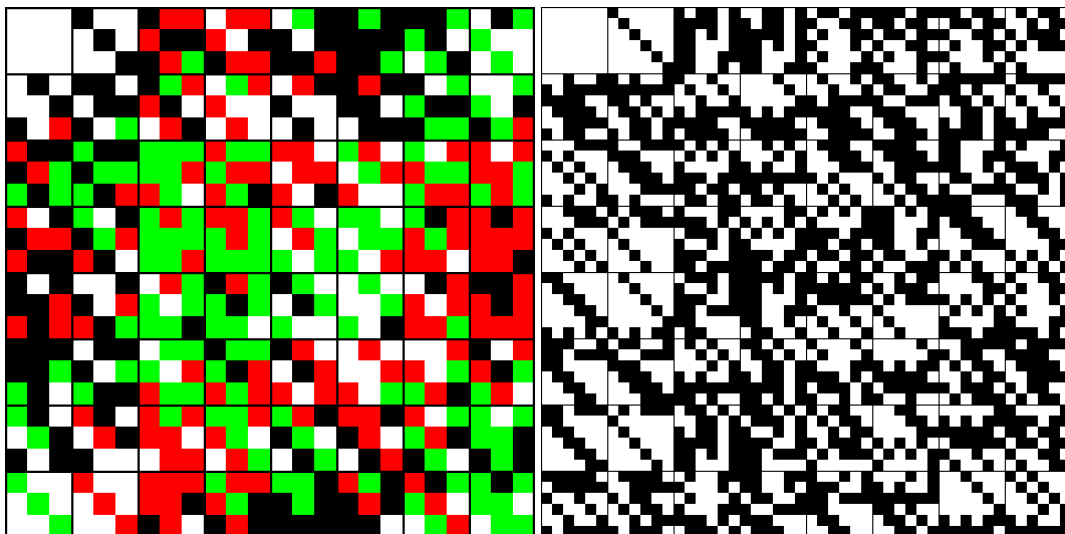
▷ ▷ -partition 4 "2,2,2,2,2,2,2,2" "2,2,2,2,2,2,2,2" -end
▷ $(OPEN) F64_over_F8_draw.bmp
▷ #pdflatex field_reduction_Q64.q8_8_8.tex
▷ #$(OPEN) field_reduction_Q64.q8_8_8.pdf

```

The output is shown below:



Note that the dimension of the vector space is 2, so the block matrices are  $2 \times 2$ . Observe that  $\mathbb{F}_{64}$  has many subfields. The reduction from  $\mathbb{F}_{64}$  to  $\mathbb{F}_4$  and to  $\mathbb{F}_2$  is also possible:



Here, the block matrices have size  $3 \times 3$  when reducing to  $\mathbb{F}_4$  (left picture) and  $6 \times 6$  when reducing to  $\mathbb{F}_2$ .

The minimum polynomials associated with the  $n$ -th roots over  $\mathbb{F}_q$  can be computed using the `-nth_roots` command, which is a finite field activity. The activity is applied to the field  $\mathbb{F}_q$  over which the  $n$ -th roots are defined. The command constructs the field extension  $\mathbb{F}_{q^m}$  where  $m$  is the order of  $q$  modulo  $n$ . This field extension contains the  $n$ -th roots of unity. Let  $\alpha$  be a primitive element of  $\mathbb{F}_{q^m}$  and let  $\beta$  be a generator of the subgroup of  $n$ -th roots. Also, let  $\gamma$  be the generator of the subgroup of  $q - 1$  th roots, which are the elements of the multiplicative group of  $\mathbb{F}_q$ . The output lists the  $n$ -th roots first, generated by  $\beta$ . After that, the  $q - 1$ th roots are shown, generated by  $\gamma$ . Finally, a table is produced which shows the irreducible polynomials over  $\mathbb{F}_q$  associated with the  $n$ -th roots of unity. For instance, the following command computes the minimum polynomials of all 21st roots of unity over  $\mathbb{F}_8$ :

#### Example 80

```
F_8_Nth_roots_21:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 8 -override_polynomial 11 -end \
> > -with F -do -finite_field_activity \
> > > -nth_roots 21 \
> > -end
> pdflatex Nth_roots.q8_n21.tex
> $(OPEN) Nth_roots.q8_n21.pdf
```

The output is:

Let  $\alpha$  be a primitive element of  $\text{GF}(64)$ . Let  $\beta$  be a primitive 21-th root in  $\text{GF}(64)$ , so  $\beta = \alpha^3$ .

$$\begin{aligned}
\beta^0 &= 100000 = 1 \\
\beta^1 &= 000100 = \alpha^3 \\
\beta^2 &= 100001 = \alpha^5 + 1 \\
\beta^3 &= 111101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha + 1 \\
\beta^4 &= 011111 = \alpha^5 + \alpha^4 + \alpha^3 + \alpha^2 + \alpha \\
\beta^5 &= 101010 = \alpha^4 + \alpha^2 + 1 \\
\beta^6 &= 110100 = \alpha^3 + \alpha + 1 \\
\beta^7 &= 100111 = \alpha^5 + \alpha^4 + \alpha^3 + 1 \\
\beta^8 &= 101101 = \alpha^5 + \alpha^3 + \alpha^2 + 1 \\
\beta^9 &= 011101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha \\
\beta^{10} &= 011011 = \alpha^5 + \alpha^4 + \alpha^2 + \alpha \\
\beta^{11} &= 001011 = \alpha^5 + \alpha^4 + \alpha^2 \\
\beta^{12} &= 001001 = \alpha^5 + \alpha^2 \\
\beta^{13} &= 111000 = \alpha^2 + \alpha + 1 \\
\beta^{14} &= 000111 = \alpha^5 + \alpha^4 + \alpha^3 \\
\beta^{15} &= 101001 = \alpha^5 + \alpha^2 + 1 \\
\beta^{16} &= 111100 = \alpha^3 + \alpha^2 + \alpha + 1 \\
\beta^{17} &= 100110 = \alpha^4 + \alpha^3 + 1 \\
\beta^{18} &= 010100 = \alpha^3 + \alpha \\
\beta^{19} &= 100011 = \alpha^5 + \alpha^4 + 1 \\
\beta^{20} &= 001100 = \alpha^3 + \alpha^2
\end{aligned}$$

Let  $\gamma$  be a primitive 7-th root in  $\text{GF}(64)$ , so  $\gamma = \alpha^9$ .

$$\begin{aligned}
\gamma^0 &= 100000 = 1 \\
\gamma^1 &= 111101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha + 1
\end{aligned}$$

$$\begin{aligned}
\gamma^2 &= 110100 = \alpha^3 + \alpha + 1 \\
\gamma^3 &= 011101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha \\
\gamma^4 &= 001001 = \alpha^5 + \alpha^2 \\
\gamma^5 &= 101001 = \alpha^5 + \alpha^2 + 1 \\
\gamma^6 &= 010100 = \alpha^3 + \alpha
\end{aligned}$$

The  $q$ -cyclotomic set for  $q = 8$  are:

$$\begin{aligned}
&\{ 0 \} \\
&\{ 1, 8 \} \\
&\{ 2, 16 \} \\
&\{ 3 \} \\
&\{ 4, 11 \} \\
&\{ 5, 19 \} \\
&\{ 6 \} \\
&\{ 7, 14 \} \\
&\{ 9 \} \\
&\{ 10, 17 \} \\
&\{ 12 \} \\
&\{ 13, 20 \} \\
&\{ 15 \} \\
&\{ 18 \}
\end{aligned}$$

Subfield basis, a basis for GF(8) inside GF(64):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

The irreducible polynomials associated with the 21-th roots over GF(8) are:

| $i$ | $r_i$ | $\text{Cyc}(r_i)$ | $m_{\beta^{r_i}}(X)$                      | $m_{\beta^{r_i}}(X)$ |
|-----|-------|-------------------|-------------------------------------------|----------------------|
| 0   | 0     | (0)               | $(100000)X^0 + (100000)X^1$               | $X + 1$              |
| 1   | 1     | (1, 8)            | $(011101)X^0 + (101001)X^1 + (100000)X^2$ | $X^2 + 7X + 3$       |
| 2   | 2     | (2, 16)           | $(010100)X^0 + (011101)X^1 + (100000)X^2$ | $X^2 + 3X + 5$       |
| 3   | 3     | (3)               | $(111101)X^0 + (100000)X^1$               | $X + 2$              |
| 4   | 4     | (4, 11)           | $(101001)X^0 + (010100)X^1 + (100000)X^2$ | $X^2 + 5X + 7$       |
| 5   | 5     | (5, 19)           | $(111101)X^0 + (001001)X^1 + (100000)X^2$ | $X^2 + 6X + 2$       |
| 6   | 6     | (6)               | $(110100)X^0 + (100000)X^1$               | $X + 4$              |
| 7   | 7     | (7, 14)           | $(100000)X^0 + (100000)X^1 + (100000)X^2$ | $X^2 + X + 1$        |
| 8   | 9     | (9)               | $(011101)X^0 + (100000)X^1$               | $X + 3$              |
| 9   | 10    | (10, 17)          | $(110100)X^0 + (111101)X^1 + (100000)X^2$ | $X^2 + 2X + 4$       |
| 10  | 12    | (12)              | $(001001)X^0 + (100000)X^1$               | $X + 6$              |
| 11  | 13    | (13, 20)          | $(001001)X^0 + (110100)X^1 + (100000)X^2$ | $X^2 + 4X + 6$       |
| 12  | 15    | (15)              | $(101001)X^0 + (100000)X^1$               | $X + 7$              |
| 13  | 18    | (18)              | $(010100)X^0 + (100000)X^1$               | $X + 5$              |

In Section 3.1, we have considered the Vandermonde matrix over  $\mathbb{F}_7$ . Let us do the same for the field  $\mathbb{F}_8$  instead. We use the following command:

#### Example 81

```
F_8_vandermonde:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 8 -end \
> > -with F -do -finite_field_activity \
> > > -Vandermonde_matrix \
> > -end
```

The output is shown below. Again, the first matrix is  $V = (x_i^j)$ . The second matrix is  $V^{-1}$ :

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 5 & 7 & 3 & 6 & 1 \\ 1 & 3 & 5 & 2 & 6 & 7 & 4 & 1 \\ 1 & 4 & 7 & 6 & 2 & 5 & 3 & 1 \\ 1 & 5 & 6 & 4 & 3 & 2 & 7 & 1 \\ 1 & 6 & 3 & 7 & 5 & 4 & 2 & 1 \\ 1 & 7 & 2 & 3 & 4 & 6 & 5 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 6 & 4 & 3 & 7 & 2 & 5 \\ 0 & 1 & 3 & 7 & 5 & 2 & 4 & 6 \\ 0 & 1 & 7 & 6 & 2 & 3 & 5 & 4 \\ 0 & 1 & 5 & 2 & 6 & 4 & 7 & 3 \\ 0 & 1 & 4 & 5 & 7 & 6 & 3 & 2 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Let us now do a somewhat larger example of the same problem. The next command computes the Vandermonde matrix and its inverse over the field  $\mathbb{F}_{1024}$ :

**Example 82**

```
F_1024_vandermonde:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 1024 -end \
> > -with F -do -finite_field_activity \
> > > -Vandermonde_matrix \
> > -end
> #rm Vandermonde_1024.csv
> #rm Vandermonde_inv_1024.csv
```

This command takes a bit of time to execute. The matrix is not shown. It would be too big to be printed. In order to save disc space, we delete the output files, using the `rm` command.

Orbiter can create code for the number theoretic transform. This is the discrete Fourier transform performed over finite fields. The generated code can be compiled with the Orbiter library. Compiling code requires additional makefile options are necessary. Because of this, we define the following makefile variables at the top of the makefile.

**Example 83**

```
SRC=$(ORBITER_PATH)/src
MY_CPP = g++
MY_CC = gcc
CPPFLAGS = -Wall -I../DEV.23/orbiter/src/lib -std=c++14
LIB = $(SRC)/lib/liborbiter.a -lpthread
LFLAGS = -lm -Wl,-rpath -Wl,/usr/local/gcc-8.2.0/lib64
```

Suppose we want to create the number theoretic transform for the 16th roots of unity inside the field  $\mathbb{F}_{17}$ . Here is the command to generate the Orbiter source code:

**Example 84**

```
NTT_k4_q17.cpp:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 17 -end \
> > -with F -do -coding_theoretic_activity \
> > > -NTT 4 17 \
> > -end
```

This produces a C++ file `NTT_k4_q17.cpp`. This file should be compiled and linked against the Orbiter library. The command

**Example 85**

```
F_17_NTT_compile: NTT_k4_q17.cpp
> $(MY_CPP) NTT_k4_q17.cpp $(CPPFLAGS) \
```

```
▷ ▷ $(LIB) $(LFLAGS) -o NTT_k4_q17.out  
▷ ./NTT_k4_q17.out
```

can be used to compile the code and run it. Note the dependency on the file `NTT_k4_q17.cpp`. This means that `make` would automatically invoke the first command if only the second one was issued.



## 3.5 Basic Ring Theory

Orbiter can deal with multivariate polynomial rings with coefficients over finite fields. Orbiter creates the homogeneous components only (so it is technically not a ring).

The following command creates the homogeneous component of degree 3 in a polynomial ring in 4 variables. The variables are named. They are  $x_0, x_1, x_2, x_3$ . Note that two sets of names are defined using the `-variables` command. The first is the labels for regular text output. The second is the set of names for latex output. Here is the command:

### Example 86

```
Polynomial_ring:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -define R -polynomial_ring -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 3 \
> > > -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
> > -end
```

For more on rings, see Chapter 5.



# Chapter 4

## Geometry

### 4.1 Finite Projective Spaces

Orbiter commands to create a finite projective space  $\text{PG}(n, q)$  are listed in Table 4.1. The command also creates the group of the space, which is  $\text{PTL}(n + 1, q)$ . Let us look at a very simple example. Suppose we want to create  $\text{PG}(3, 2)$ . The following command sequence creates the finite field  $\mathbb{F}_2$  and stores the object in the symbol table, using the label  $F$ . After that, the projective space  $\text{PG}(3, F)$  is created, using the field  $F$ . The projective space is stored in the symbol table under the label  $P$ .

#### Example 87

```
PG_3_2_easy:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define P -projective_space -n 3 -field F -end
```

Orbiter offers indexing for points and for subspaces of all  $\text{PG}(n, q)$  of a fixed dimension. The incidence matrix between points and lines with respect to this ordering can be computed. The indexing is used to establish the permutation representations of the projective group, as will be described in Section 6.2. The indexing of points is not the lexicographic ordering. It emphasizes the role of frames in the geometry by assigning the

| Creating a Projective Space |           |                                                                                                                                      |
|-----------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------|
| Command                     | Arguments | Purpose                                                                                                                              |
| -n                          | $n$       | Specify the projective dimension $n$ .                                                                                               |
| -q                          | $q$       | Specify the order $q$ of the underlying finite field $\mathbb{F}_q$ . A default version of the field $\mathbb{F}_q$ will be created. |
| -field                      | $F$       | Specify the underlying field $F$ .                                                                                                   |
| -use_projectivity_subgroup  |           | Create the projectivity group instead of the collineation group.                                                                     |
| -v                          | $k$       | Set verbosity to $k$ during creation of the projective space. Higher values of $k$ give more output. $k = 0$ is silent mode.         |

Table 4.1: Options for Creating a Projective Space

smallest rank values to the members of the standard frame. After that, the other points are listed.

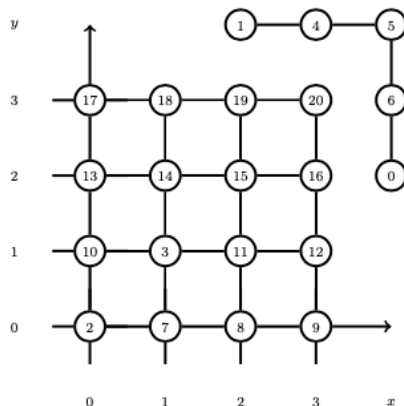
Orbiter can create a cheat sheet for  $\text{PG}(n, q)$ . The cheat sheet lists the objects in the space, and basic properties. Here are two examples. The first is a projective plane. The second is a projective 3-space.

The following command creates a cheat sheet for  $\text{PG}(2, 4)$ :

#### Example 88

```
PG_2_4:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do -projective_space_activity \
> > > -cheat_sheet \
> > -end
> pdflatex PG_2_4.tex
> $(OPEN) PG_2_4.pdf
```

The cheat sheet contains a drawing of the plane:



The affine plane is shown in the cartesian plane, while the line at infinity is wrapped around the top right corner. The cheat sheet continues by listing the points, including the canonical Baer subgeometry  $\text{PG}(2, 2)$ . After that, the points are listed again, but with left-normalized vectors. Finally, the lines are shown.

$\text{PG}(2, 4)$  has 21 points:

$$\begin{aligned}
P_0 &= (1, 0, 0) = (1, 0, 0) \\
P_1 &= (0, 1, 0) = (0, 1, 0) \\
P_2 &= (0, 0, 1) = (0, 0, 1) \\
P_3 &= (1, 1, 1) = (1, 1, 1) \\
P_4 &= (1, 1, 0) = (1, 1, 0) \\
P_5 &= (2, 1, 0) = (\alpha, 1, 0) \\
P_6 &= (3, 1, 0) = (\alpha^2, 1, 0) \\
P_7 &= (1, 0, 1) = (1, 0, 1) \\
P_8 &= (2, 0, 1) = (\alpha, 0, 1) \\
P_9 &= (3, 0, 1) = (\alpha^2, 0, 1) \\
P_{10} &= (0, 1, 1) = (0, 1, 1)
\end{aligned}$$

$$\begin{aligned}
P_{11} &= (2, 1, 1) = (\alpha, 1, 1) \\
P_{12} &= (3, 1, 1) = (\alpha^2, 1, 1) \\
P_{13} &= (0, 2, 1) = (0, \alpha, 1) \\
P_{14} &= (1, 2, 1) = (1, \alpha, 1) \\
P_{15} &= (2, 2, 1) = (\alpha, \alpha, 1) \\
P_{16} &= (3, 2, 1) = (\alpha^2, \alpha, 1) \\
P_{17} &= (0, 3, 1) = (0, \alpha^2, 1) \\
P_{18} &= (1, 3, 1) = (1, \alpha^2, 1) \\
P_{19} &= (2, 3, 1) = (\alpha, \alpha^2, 1) \\
P_{20} &= (3, 3, 1) = (\alpha^2, \alpha^2, 1)
\end{aligned}$$

Baer subgeometry:

$$\begin{array}{llll}
P_0 = (1, 0, 0) & P_2 = (0, 0, 1) & P_4 = (1, 1, 0) & P_{10} = (0, 1, 1) \\
P_1 = (0, 1, 0) & P_3 = (1, 1, 1) & P_7 = (1, 0, 1) &
\end{array}$$

There are 7 elements in the Baer subgeometry.  
Normalized from the left:

$$\begin{array}{llll}
P_0 = (1, 0, 0) & P_6 = (1, 2, 0) & P_{12} = (1, 2, 2) & P_{18} = (1, 3, 1) \\
P_1 = (0, 1, 0) & P_7 = (1, 0, 1) & P_{13} = (0, 1, 3) & P_{19} = (1, 2, 3) \\
P_2 = (0, 0, 1) & P_8 = (1, 0, 3) & P_{14} = (1, 2, 1) & P_{20} = (1, 1, 2) \\
P_3 = (1, 1, 1) & P_9 = (1, 0, 2) & P_{15} = (1, 1, 3) & \\
P_4 = (1, 1, 0) & P_{10} = (0, 1, 1) & P_{16} = (1, 3, 2) & \\
P_5 = (1, 3, 0) & P_{11} = (1, 3, 3) & P_{17} = (0, 1, 2) &
\end{array}$$

The Lines of  $\text{PG}(2, 4)$ .  $\text{PG}(2, 4)$  has 21 1-subspaces:

$$\begin{array}{lll}
L_0 = \begin{bmatrix} 100 \\ 010 \end{bmatrix} & L_7 = \begin{bmatrix} 101 \\ 012 \end{bmatrix} & L_{14} = \begin{bmatrix} 120 \\ 001 \end{bmatrix} \\
L_1 = \begin{bmatrix} 100 \\ 011 \end{bmatrix} & L_8 = \begin{bmatrix} 101 \\ 013 \end{bmatrix} & L_{15} = \begin{bmatrix} 103 \\ 010 \end{bmatrix} \\
L_2 = \begin{bmatrix} 100 \\ 012 \end{bmatrix} & L_9 = \begin{bmatrix} 110 \\ 001 \end{bmatrix} & L_{16} = \begin{bmatrix} 103 \\ 011 \end{bmatrix} \\
L_3 = \begin{bmatrix} 100 \\ 013 \end{bmatrix} & L_{10} = \begin{bmatrix} 102 \\ 010 \end{bmatrix} & L_{17} = \begin{bmatrix} 103 \\ 012 \end{bmatrix} \\
L_4 = \begin{bmatrix} 100 \\ 001 \end{bmatrix} & L_{11} = \begin{bmatrix} 102 \\ 011 \end{bmatrix} & L_{18} = \begin{bmatrix} 103 \\ 013 \end{bmatrix} \\
L_5 = \begin{bmatrix} 101 \\ 010 \end{bmatrix} & L_{12} = \begin{bmatrix} 102 \\ 012 \end{bmatrix} & L_{19} = \begin{bmatrix} 130 \\ 001 \end{bmatrix} \\
L_6 = \begin{bmatrix} 101 \\ 011 \end{bmatrix} & L_{13} = \begin{bmatrix} 102 \\ 013 \end{bmatrix} & L_{20} = \begin{bmatrix} 010 \\ 001 \end{bmatrix}
\end{array}$$

The following command creates a cheat sheet for  $\text{PG}(3, 2)$ .

#### Example 89

```
PG_3_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do -projective_space_activity \
> > > -cheat_sheet \
> > -end
> pdflatex PG_3_2.tex
> $(OPEN) PG_3_2.pdf
```

The cheat sheet shows points, lines and planes. The lines are shown together with their Plücker coordinates. The lines whose Plücker coordinates are unit vectors are shown separately.

### The projective space $\text{PG}(3, 2)$

$q = 2$   
 $p = 2$   
 $e = 1$   
 $n = 3$   
 Number of points = 15  
 Number of lines = 35  
 Number of lines on a point = 7  
 Number of points on a line = 3

### The points of $\text{PG}(3, 2)$

$\text{PG}(3, 2)$  has 15 points:

|                      |                      |                         |                         |
|----------------------|----------------------|-------------------------|-------------------------|
| $P_0 = (1, 0, 0, 0)$ | $P_4 = (1, 1, 1, 1)$ | $P_8 = (1, 1, 1, 0)$    | $P_{12} = (0, 0, 1, 1)$ |
| $P_1 = (0, 1, 0, 0)$ | $P_5 = (1, 1, 0, 0)$ | $P_9 = (1, 0, 0, 1)$    | $P_{13} = (1, 0, 1, 1)$ |
| $P_2 = (0, 0, 1, 0)$ | $P_6 = (1, 0, 1, 0)$ | $P_{10} = (0, 1, 0, 1)$ | $P_{14} = (0, 1, 1, 1)$ |
| $P_3 = (0, 0, 0, 1)$ | $P_7 = (0, 1, 1, 0)$ | $P_{11} = (1, 1, 0, 1)$ |                         |

Normalized from the left:

|                      |                      |                         |                         |
|----------------------|----------------------|-------------------------|-------------------------|
| $P_0 = (1, 0, 0, 0)$ | $P_4 = (1, 1, 1, 1)$ | $P_8 = (1, 1, 1, 0)$    | $P_{12} = (0, 0, 1, 1)$ |
| $P_1 = (0, 1, 0, 0)$ | $P_5 = (1, 1, 0, 0)$ | $P_9 = (1, 0, 0, 1)$    | $P_{13} = (1, 0, 1, 1)$ |
| $P_2 = (0, 0, 1, 0)$ | $P_6 = (1, 0, 1, 0)$ | $P_{10} = (0, 1, 0, 1)$ | $P_{14} = (0, 1, 1, 1)$ |
| $P_3 = (0, 0, 0, 1)$ | $P_7 = (0, 1, 1, 0)$ | $P_{11} = (1, 1, 0, 1)$ |                         |

**The lines of  $\text{PG}(3, 2)$** 

$\text{PG}(3, 2)$  has 35 1-subspaces:

$$L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 0, 0)$$

$$L_1 = \begin{bmatrix} 1000 \\ 0110 \end{bmatrix} = \mathbf{Pl}(1, 0, 1, 0, 0, 0)$$

$$L_2 = \begin{bmatrix} 1000 \\ 0101 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 1, 0)$$

$$L_3 = \begin{bmatrix} 1000 \\ 0111 \end{bmatrix} = \mathbf{Pl}(1, 0, 1, 0, 1, 0)$$

$$L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix} = \mathbf{Pl}(0, 0, 1, 0, 0, 0)$$

$$L_5 = \begin{bmatrix} 1000 \\ 0011 \end{bmatrix} = \mathbf{Pl}(0, 0, 1, 0, 1, 0)$$

$\vdots$

$$L_{34} = \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)$$

Lines sorted by Pluecker coordinates

$$0 = \mathbf{Pl}(1, 0, 0, 0, 0, 0) = L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix}$$

$$1 = \mathbf{Pl}(0, 1, 0, 0, 0, 0) = L_{34} = \begin{bmatrix} 0010 \\ 0001 \end{bmatrix}$$

$$2 = \mathbf{Pl}(0, 0, 1, 0, 0, 0) = L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix}$$

$$3 = \mathbf{Pl}(0, 0, 0, 1, 0, 0) = L_{30} = \begin{bmatrix} 0100 \\ 0001 \end{bmatrix}$$

$$4 = \mathbf{Pl}(0, 0, 0, 0, 1, 0) = L_6 = \begin{bmatrix} 1000 \\ 0001 \end{bmatrix}$$

$$5 = \mathbf{Pl}(0, 0, 0, 0, 0, 1) = L_{28} = \begin{bmatrix} 0100 \\ 0010 \end{bmatrix}$$

$\vdots$

$$34 = \mathbf{Pl}(0, 1, 1, 1, 1, 1) = L_{26} = \begin{bmatrix} 1101 \\ 0011 \end{bmatrix}$$

$\text{PG}(3, 2)$  has the following low weight Pluecker lines:

$$L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 0, 0)$$

$$L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix} = \mathbf{Pl}(0, 0, 1, 0, 0, 0)$$

$$L_6 = \begin{bmatrix} 1000 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 0, 0, 0, 1, 0)$$

$$\begin{aligned}
L_{28} &= \begin{bmatrix} 0100 \\ 0010 \end{bmatrix} = \mathbf{Pl}(0, 0, 0, 0, 0, 1) \\
L_{30} &= \begin{bmatrix} 0100 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 0, 0, 1, 0, 0) \\
L_{34} &= \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)
\end{aligned}$$

### The planes of $\text{PG}(3, 2)$

$\text{PG}(3, 2)$  has 15 2-subspaces:

$$\begin{aligned}
L_0 &= \begin{bmatrix} 1000 \\ 0100 \\ 0010 \end{bmatrix} \\
L_1 &= \begin{bmatrix} 1000 \\ 0100 \\ 0011 \end{bmatrix} \\
&\vdots \\
L_{14} &= \begin{bmatrix} 0100 \\ 0010 \\ 0001 \end{bmatrix}
\end{aligned}$$

### The polynomial rings associated with $\text{PG}(3, 2)$

| $h$ | monomial | vector         |
|-----|----------|----------------|
| 0   | $X_0$    | $(1, 0, 0, 0)$ |
| 1   | $X_1$    | $(0, 1, 0, 0)$ |
| 2   | $X_2$    | $(0, 0, 1, 0)$ |
| 3   | $X_3$    | $(0, 0, 0, 1)$ |



| Ranking Points in Small Projective Spaces                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PG(2, 2)                                                                                                                                                                                                              | PG(3, 2)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | PG(2, 3)                                                                                                                                                                                                                                                                                                                                                                                                                 |
| $P_0 = \mathbf{P}(1, 0, 0)$<br>$P_1 = \mathbf{P}(0, 1, 0)$<br>$P_2 = \mathbf{P}(0, 0, 1)$<br>$P_3 = \mathbf{P}(1, 1, 1)$<br>$P_4 = \mathbf{P}(1, 1, 0)$<br>$P_5 = \mathbf{P}(1, 0, 1)$<br>$P_6 = \mathbf{P}(0, 1, 1)$ | $P_0 = \mathbf{P}(1, 0, 0, 0)$<br>$P_1 = \mathbf{P}(0, 1, 0, 0)$<br>$P_2 = \mathbf{P}(0, 0, 1, 0)$<br>$P_3 = \mathbf{P}(0, 0, 0, 1)$<br>$P_4 = \mathbf{P}(1, 1, 1, 1)$<br>$P_5 = \mathbf{P}(1, 1, 0, 0)$<br>$P_6 = \mathbf{P}(1, 0, 1, 0)$<br>$P_7 = \mathbf{P}(0, 1, 1, 0)$<br>$P_8 = \mathbf{P}(1, 1, 1, 0)$<br>$P_9 = \mathbf{P}(1, 0, 0, 1)$<br>$P_{10} = \mathbf{P}(0, 1, 0, 1)$<br>$P_{11} = \mathbf{P}(1, 1, 0, 1)$<br>$P_{12} = \mathbf{P}(0, 0, 1, 1)$<br>$P_{13} = \mathbf{P}(1, 0, 1, 1)$<br>$P_{14} = \mathbf{P}(0, 1, 1, 1)$ | $P_0 = \mathbf{P}(1, 0, 0)$<br>$P_1 = \mathbf{P}(0, 1, 0)$<br>$P_2 = \mathbf{P}(0, 0, 1)$<br>$P_3 = \mathbf{P}(1, 1, 1)$<br>$P_4 = \mathbf{P}(1, 1, 0)$<br>$P_5 = \mathbf{P}(2, 1, 0)$<br>$P_6 = \mathbf{P}(1, 0, 1)$<br>$P_7 = \mathbf{P}(2, 0, 1)$<br>$P_8 = \mathbf{P}(0, 1, 1)$<br>$P_9 = \mathbf{P}(2, 1, 1)$<br>$P_{10} = \mathbf{P}(0, 2, 1)$<br>$P_{11} = \mathbf{P}(1, 2, 1)$<br>$P_{12} = \mathbf{P}(2, 2, 1)$ |

Table 4.2: Ranking Points in Small Projective Spaces

## 4.2 Indexing Points and Lines

The enumerator for points establishes a bijection between the set of points and the integers on the interval  $[0, \theta_n(q) - 1]$ , where

$$\theta_n(q) = \frac{q^{n+1} - 1}{q - 1}.$$

In order to facilitate the bijection, Orbiter enumerates representative vectors for the one-dimensional subspaces. The conditions on the vectors are summarized below:

1. The vector is not the zero vector.
2. The rightmost nonzero entry in the vector is one. If it is not, we normalize the vector so that the rightmost nonzero vector is indeed one. This operation does not change the projective point which is associated with the vector.

The second condition ensures that we list each projective point exactly once. We require two functions, RANK and UNRANK. The function RANK takes a vector  $\mathbf{x} \in \mathbb{F}_q^n$ , not zero, and maps it to the element in  $\mathbb{Z}_N$  representing the projective point  $\mathbf{P}(\mathbf{x})$ . A frame in  $\text{PG}(n, q)$  is a set of  $n + 2$  points, no  $n + 1$  in a hyperplane. We assume that the coordinates of a vector are indexed by the elements of  $\mathbb{Z}_n$ . Also, we let  $\mathbf{e}_i$  be the  $i$ -th unit vector. A frame for  $\text{PG}(n, q)$  is

$$\mathbf{e}_0, \dots, \mathbf{e}_{n-1}, \mathbf{e}_0 + \dots + \mathbf{e}_{n-1}.$$

This is the *standard frame*. We start the labeling of points with the standard frame. After these  $n + 2$  points, we list the remaining points in lexicographic ordering (utilizing right-normalized representative). The points in the small spaces  $\text{PG}(2, 2)$ ,  $\text{PG}(3, 2)$  and  $\text{PG}(2, 3)$  are listed by their ranks in Table 4.2.

It is possible to request ranks of specific points given by means of homogeneous coordinate vectors. Conversely, given a point rank, the coordinate vector can be produced. The following command computes the rank of

$$\mathbf{P}(3, 3, 1) = \mathbf{P}(\omega + 1, \omega + 1, 1)$$

in  $\text{PG}(2, 4)$ :

#### Example 90

```
PG_2.4_rank_point:
> $(ORBITER) -v 2 \
> > -define v -vector -dense "3,3,1" -format 1 -end \
> > -define F -finite_field -q 4 -end \
> > -with F -do -finite_field_activity \
> > > -rank_point_in_PG v -end
```

The rank turns out to be 20. Conversely, runnig

#### Example 91

```
PG_2.4_unrank_point:
> $(ORBITER) -v 2 \
> > -define v -vector -dense "20" -end \
> > -define F -finite_field -q 4 -end \
> > -with F -do -finite_field_activity \
> > > -unrank_point_in_PG 2 v -end
```

shows that the point with rank 20 is  $\mathbf{P}(3, 3, 1)$ .

Line ranking will be discussed next. The lines in the small spaces  $\text{PG}(2, 2)$  and  $\text{PG}(2, 3)$  are listed by their ranks in Table 4.3.

The command

#### Example 92

```
PG_2.2_rank_lines:
> $(ORBITER) -v 2 \
> > -define v -vector -format 4 \
> > -dense "1,0,0, 0,1,0, 1,0,0, 0,0,1, 1,1,1, 0,1,0, 1,1,1, 0,0,1" \
> > -end \
> > -define F -finite_field -q 2 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -rank_lines_in_PG v \
> > -end \
```

computes the Orbiter ranks of 4 lines in  $\text{PG}(2, 2)$ . The lines are given by two points each. The data is formatted as a matrix with 4 rows of size 6. Each row represents one line, with the coordinates of the two points written one after the other.

| Ranking Lines in Small Projective Spaces                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PG(2,2)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | PG(2,3)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| $L_0 = \mathbf{L} \begin{bmatrix} 100 \\ 010 \end{bmatrix} \{0, 1, 4\}$ $L_1 = \mathbf{L} \begin{bmatrix} 100 \\ 011 \end{bmatrix} \{0, 3, 6\}$ $L_2 = \mathbf{L} \begin{bmatrix} 100 \\ 001 \end{bmatrix} \{0, 2, 5\}$ $L_3 = \mathbf{L} \begin{bmatrix} 101 \\ 010 \end{bmatrix} \{1, 3, 5\}$ $L_4 = \mathbf{L} \begin{bmatrix} 101 \\ 011 \end{bmatrix} \{4, 5, 6\}$ $L_5 = \mathbf{L} \begin{bmatrix} 110 \\ 001 \end{bmatrix} \{2, 3, 4\}$ $L_6 = \mathbf{L} \begin{bmatrix} 010 \\ 001 \end{bmatrix} \{1, 2, 6\}$ | $L_0 = \mathbf{L} \begin{bmatrix} 100 \\ 010 \end{bmatrix} \{0, 1, 4, 5\}$ $L_1 = \mathbf{L} \begin{bmatrix} 100 \\ 011 \end{bmatrix} \{0, 3, 8, 9\}$ $L_2 = \mathbf{L} \begin{bmatrix} 100 \\ 012 \end{bmatrix} \{0, 10, 11, 12\}$ $L_3 = \mathbf{L} \begin{bmatrix} 100 \\ 001 \end{bmatrix} \{0, 2, 6, 7\}$ $L_4 = \mathbf{L} \begin{bmatrix} 101 \\ 010 \end{bmatrix} \{1, 3, 6, 11\}$ $L_5 = \mathbf{L} \begin{bmatrix} 101 \\ 011 \end{bmatrix} \{5, 6, 8, 12\}$ $L_6 = \mathbf{L} \begin{bmatrix} 101 \\ 012 \end{bmatrix} \{4, 6, 9, 10\}$ $L_7 = \mathbf{L} \begin{bmatrix} 110 \\ 001 \end{bmatrix} \{2, 3, 4, 12\}$ $L_8 = \mathbf{L} \begin{bmatrix} 102 \\ 010 \end{bmatrix} \{1, 7, 9, 12\}$ $L_9 = \mathbf{L} \begin{bmatrix} 102 \\ 011 \end{bmatrix} \{4, 7, 8, 11\}$ $L_{10} = \mathbf{L} \begin{bmatrix} 102 \\ 012 \end{bmatrix} \{3, 5, 7, 10\}$ $L_{11} = \mathbf{L} \begin{bmatrix} 120 \\ 001 \end{bmatrix} \{2, 5, 9, 11\}$ $L_{12} = \mathbf{L} \begin{bmatrix} 010 \\ 001 \end{bmatrix} \{1, 2, 8, 10\}$ |

Table 4.3: Ranking Lines in Small Projective Spaces

The command

**Example 93**

```
PG_2_2_unrank_lines:
> $(ORBITER) -v 2 \
> > -define v -vector -format 4 \
> > -dense "0,1,2,3,4,5,6" \
> > -end \
> > -define F -finite_field -q 2 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -unrank_lines_in_PG v \
> > -end \
```

lists the 7 lines of  $\text{PG}(2, 2)$  by their Orbiter ranks from 0 to 6.

### 4.3 Incidence Matrices

The projective spaces  $\text{PG}(2, q)$  are examples of a more general structure called projective planes. The  $\text{PG}(2, F)$ ,  $F$  a field, are distinguished in the class of projective planes by the fact that the theorem of Desargues always holds. They are called the desarguesian projective planes. Other classes of projective planes are the translation planes, see Section 14.2.

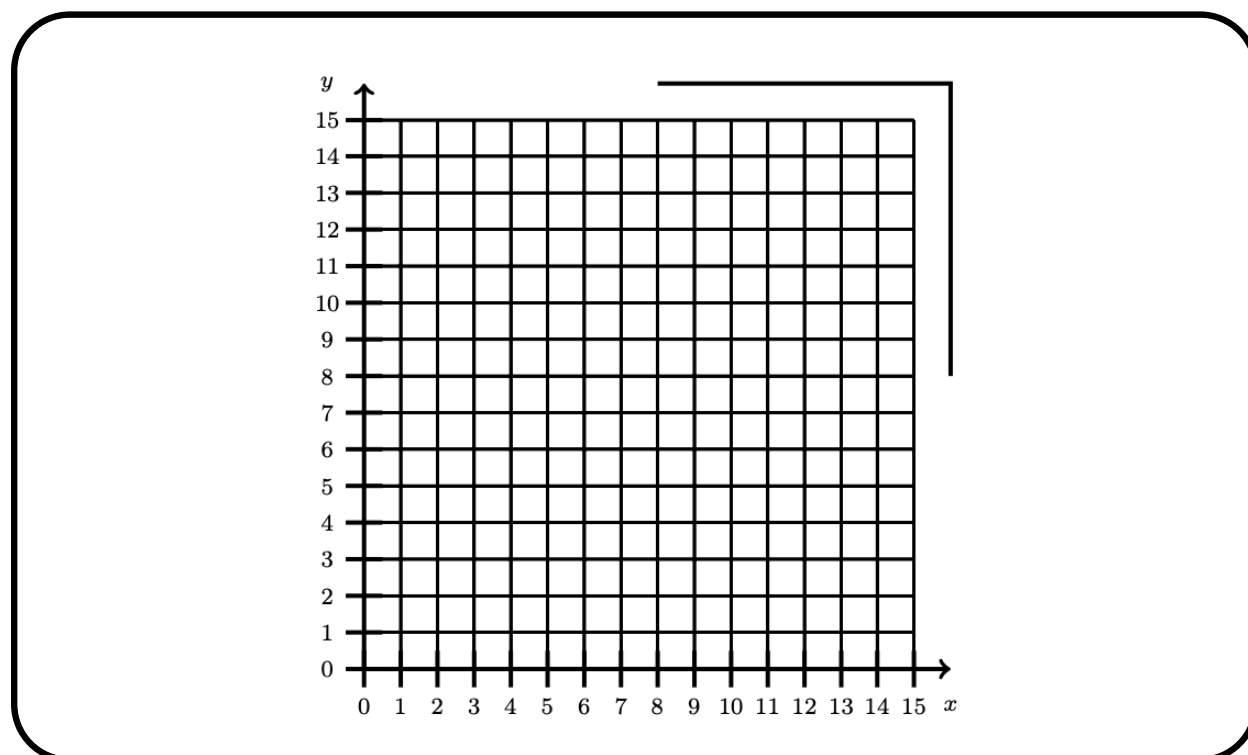
The points in the dearguesian projective plane  $\text{PG}(2, q)$  have the coordinates  $\mathbf{P}(x, y, z)$ , with  $x, y, z \in \mathbb{F}_q$ , not all three zero. We distinguish one line called the line at infinity. The points not on this line form an affine plane  $\text{AG}(2, q)$ .

The command

#### Example 94

```
PG_2_16:
> $(ORBITER) -v 2 \
> > -draw_options -xin 20000 -yin 20000 \
> > -radius 200 -line_width 0.3 -nodes_empty -end \
> > -define F -finite_field -q 16 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do -projective_space_activity \
> > > -cheat_sheet \
> > -end
> pdflatex PG_2_16.tex
> $(OPEN) PG_2_16.pdf
```

produces the drawing of  $\text{PG}(2, 16)$  shown below



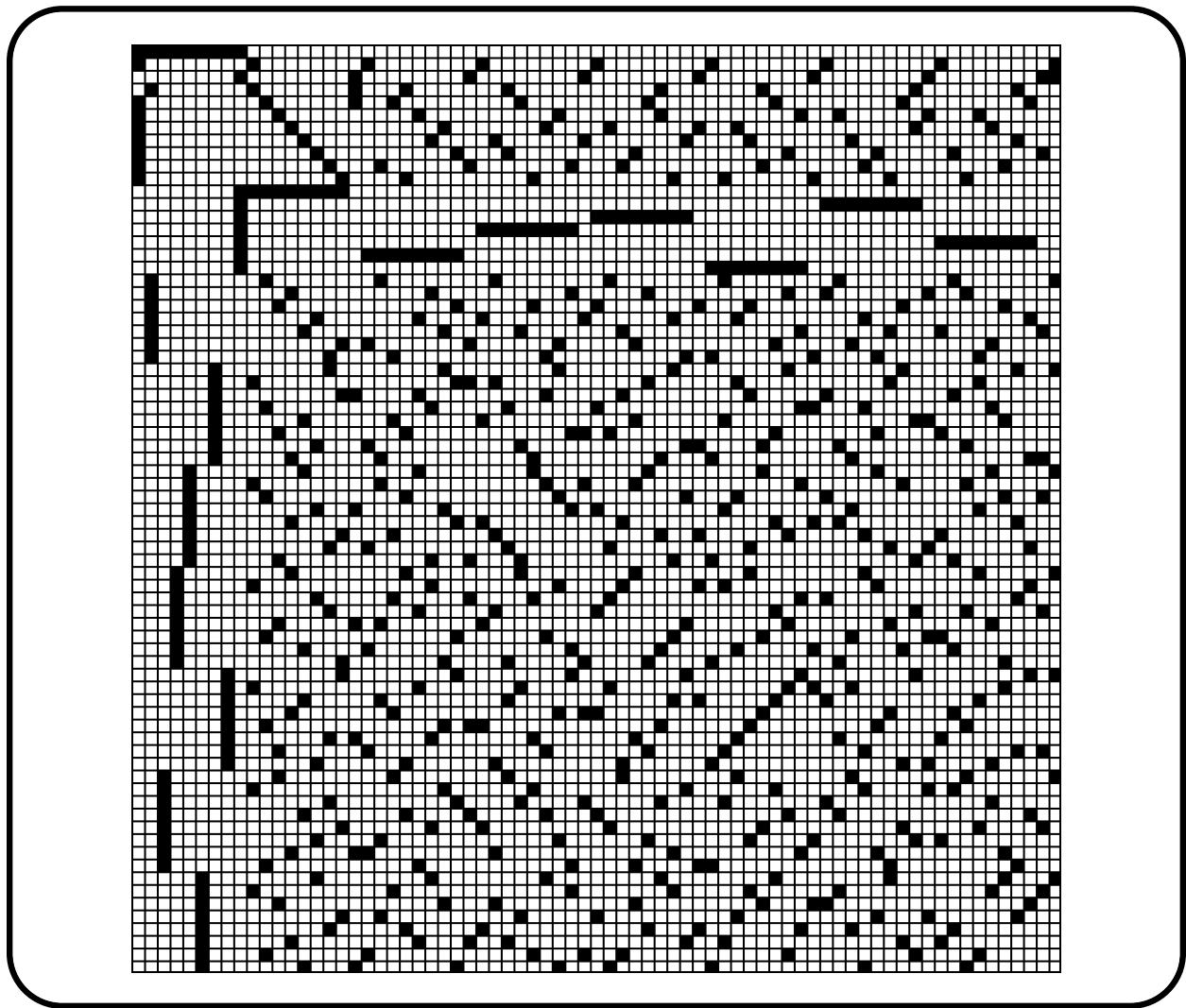
The `-nodes_empty` command is used to suppress the drawing of the nodes. The `-xin 20000` and `-yin 20000` options double the input coordinate system (recall from Table 17.1 that the default values are 10000), which has the effect that the text appears smaller relative to the grid.

It is possible to export the incidence matrix of a projective space to a file. The following example creates  $PG(2,8)$  and exports the incidence matrix to a csv file. After that, a graphical representation is produced.

#### Example 95

```
PG_2_8_incidence_matrix:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 8 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do -projective_space_activity \
> > > -export_point_line_incidence_matrix \
> > -end
> $(ORBITER) -v 2 \
> > -define all_one -vector -repeat 1 73 -end \
> > -draw_matrix \
> > > -input_csv_file PG_n2_q8_incidence_matrix.csv \
> > > -box_width 20 -bit_depth 8 \
> > > -partition 3 \
> > > > all_one all_one \
> > -end
> $(OPEN) PG_n2_q8_incidence_matrix_draw.bmp
```

The incidence matrix is shown below



The rows and columns correspond to points and lines, respectively, and are ordered according to their respective Orbiter ranks.

Projective spaces admit a cyclic group action on points and hyperplanes. Such a group is often called a Singer cycle. It is generated from a projectivity defined by the companion matrix of an irreducible polynomial. Let us look at an example. The following command creates a Singer cycle of  $PG(2, 4)$

#### Example 96

```
PG_2.4_with_decomposition:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do -projective_space_activity \
> > > -decomposition_by_element_PG \
> > > 1 "0,1,0, 0,0,1, 2,1,1, 0" \
> > > "PG_2.4_singer" \
> > -end
> pdflatex PG_2.4_singer.tex
> $(OPEN) PG_2.4_singer.pdf
```

The output is shown below:

Considering the cyclic group generated by

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ \omega & 1 & 1 \end{bmatrix}_0 = \begin{bmatrix} 010 \\ 001 \\ 211 \end{bmatrix}_0$$

The group is transitive on points and on lines.

Orbits on points:

There are 1 orbits, the orbit lengths are 21

Orbits on lines:

There are 1 orbits, the orbit lengths are 21

Fixed points:

Fixed lines:

Row scheme:

$$\begin{array}{c|c} \rightarrow & 21 \\ \hline 21 & 5 \end{array}$$

Column scheme:

$$\begin{array}{c|c} \downarrow & 21 \\ \hline 21 & 5 \end{array}$$

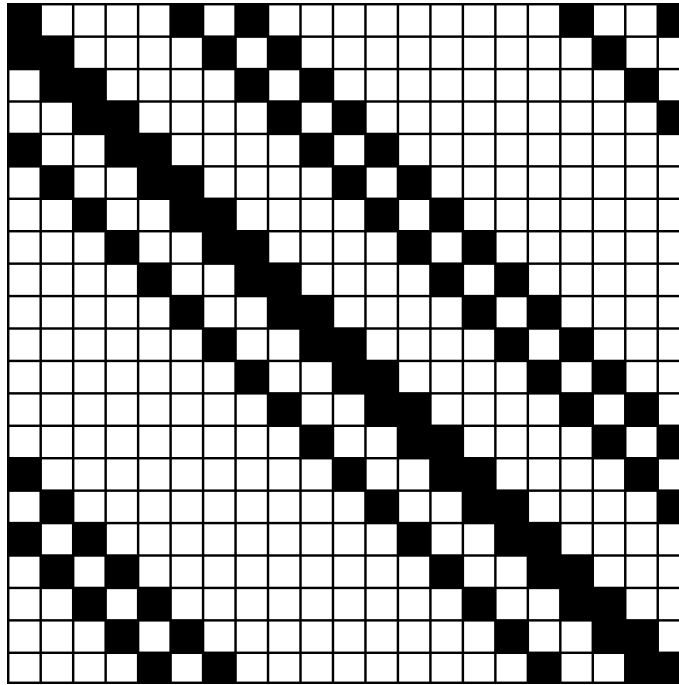
The command produces a csv file containing the cyclic incidence matrix, which can be drawn using the following command:

#### Example 97

```
PG_2.4_incma_cyclic:
> $(ORBITER) -v 2 \
> > -list_arguments \
> > -define R -vector -repeat 1 21 -end \
> > -define C -vector -repeat 1 21 -end \
> > -draw_matrix \
> > -input_csv_file PG_2.4_singer_incma_cyclic.csv \
> > -box_width 40 -bit_depth 24 \
> > -partition 3 R C \
> > -end
> $(OPEN) PG_2.4_singer_incma_cyclic_draw.bmp
```

The cyclic incidence matrix is



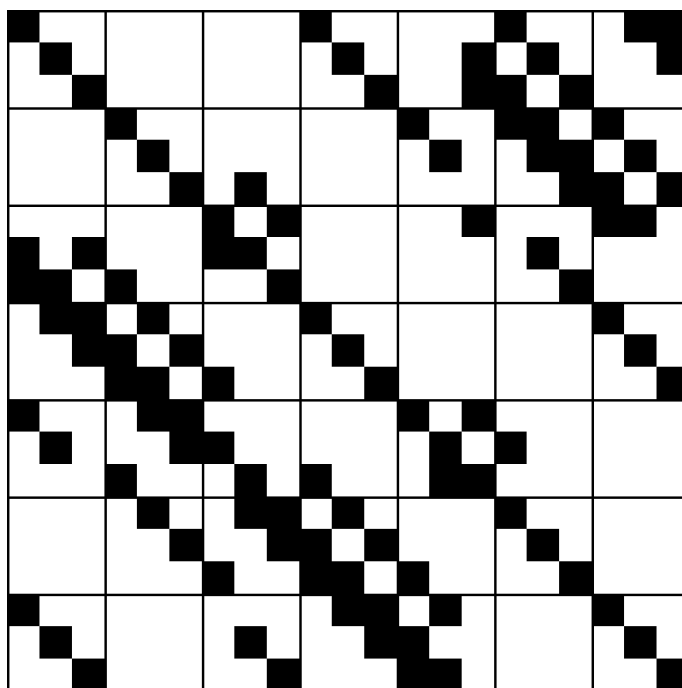


If the number of points is not a prime power, the group acts imprimitively. By considering various subgroups of the Singer group, tactical decompositions are created. For instance, for  $PG(2,4)$ , with 21 points, we can consider a subgroup the Singer cycle of order 3, which induces a partition with 7 classes of size 3 on both points and lines:

#### Example 98

```
PG_2_4_incma_singer_sub_3:
> $(ORBITER) -v 2 \
> > -list_arguments \
> > -define R -vector -repeat 3 7 -end \
> > -define C -vector -repeat 3 7 -end \
> > -draw_matrix \
> > -input_csv_file PG_2_4_singer_incma_subgroup_index_3.csv \
> > -box_width 40 -bit_depth 24 \
> > -partition 3 R C \
> > -end
> $(OPEN) PG_2_4_singer_incma_subgroup_index_3_draw.bmp
```

The tactical decomposition of the incidence matrix is



## 4.4 The Grassmannian

Let  $V$  be a finite dimensional vector space and let  $\mathfrak{Gr}_k(V)$  be the Grassmannian of  $k$ -dimensional subspaces of  $V$ . If  $\dim(V) = n$ , the notation  $\mathfrak{Gr}_{n,k}$  is used for  $\mathfrak{Gr}_k(V)$ . If  $V = \mathbb{F}_q^n$ , the notation  $\mathfrak{Gr}_{n,k,q}$  is used for  $\mathfrak{Gr}_k(V)$ . The order of the set  $\mathfrak{Gr}_{n,k,q}$  can be computed as

$$\begin{bmatrix} n \\ k \end{bmatrix}_q = \prod_{i=0}^{k-1} \frac{q^{n-i} - 1}{q^{k-i} - 1},$$

using the  $q$ -binomial coefficient.

Orbiter has an enumerator for the Grassmannian. The purpose of this enumerator is to establish a bijection between the Grassmannian and the integers in the interval  $[0, N - 1]$ , where  $N = \begin{bmatrix} n \\ k \end{bmatrix}_q$ . In order to do so, Orbiter picks a basis for each subspace. By writing the elements of the basis in the rows of a matrix, a  $k \times n$  matrix is obtained. In order to make the matrix unique, we assume it to be in RREF. In coding theory, such a matrix is called a generator matrix.

The Orbiter cheat sheets for  $\text{PG}(n, q)$  (see Section 4.1) contain lists of all Grassmannians, provided they are not too big. It is also possible to create cheat sheets specifically for one Grassmannian. For instance, the command

### Example 99

```
GR_3.2.2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -report_Grassmannian 2 \
> > -end
> pdflatex Gr_3.2.2.tex
> $(OPEN) Gr_3.2.2.pdf
```

produces a list of 2-dimensional subspaces of  $\mathbb{F}_2^3$ , i.e. the lines of  $\text{PG}(2, 2)$  :

$$\begin{array}{lll} L_0 = \begin{bmatrix} 100 \\ 010 \end{bmatrix} & L_3 = \begin{bmatrix} 101 \\ 010 \end{bmatrix} & L_6 = \begin{bmatrix} 010 \\ 001 \end{bmatrix} \\ L_1 = \begin{bmatrix} 100 \\ 011 \end{bmatrix} & L_4 = \begin{bmatrix} 101 \\ 011 \end{bmatrix} & \\ L_2 = \begin{bmatrix} 100 \\ 001 \end{bmatrix} & L_5 = \begin{bmatrix} 110 \\ 001 \end{bmatrix} & \end{array}$$

The following command illustrates how to rank lines. In the example, we consider lines in  $\text{PG}(3, 3)$ . The lines are given as vectors of length 8. Three lines are given in `v1` and three lines are given in `v2`.

**Example 100**

```

PG_3.3_rank_lines:
> $(ORBITER) -v 2 \
> > -define v1 -vector -format 3 \
> > -dense "1,0,2,2,0,1,1,2, 1,0,2,0,0,1,1,2, 1,0,2,2,0,1,2,1" \
> > -end \
> > -define v2 -vector -format 3 \
> > -dense "1,0,0,0,0,1,0,0, 1,0,0,0,0,0,0,1, 0,1,0,0,0,0,2,1" \
> > -end \
> > -define F -finite_field -q 3 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -rank_lines_in_PG v1 \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -rank_lines_in_PG v2 \
> > -end

```

In the next example, we unrank six lines in  $PG(3, 5)$ .

**Example 101**

```

PG_3.5_unrank_lines:
> $(ORBITER) -v 2 \
> > -define v -vector \
> > > -dense "0,36,72,108,144,805" \
> > -end \
> > -define F -finite_field -q 5 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -unrank_lines_in_PG v \
> > -end

```

The following command produces a list of planes through a line. In the example, the line is 0. The projective space is  $PG(3, 8)$

**Example 102**

```

planes_in_pencil:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 8 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -planes_through_line 0 \

```

▷ ▷ -end

## 4.5 Algebraic Sets

A set of points  $V$  in  $\text{PG}(n, q)$  is algebraic if there is a set of homogeneous polynomials  $p_1, \dots, p_r$  whose roots over  $\mathbb{F}_q$  are the given set. In this case, we write  $V = \mathbf{v}(p_1, \dots, p_r)$ . The set  $V$  is often called the variety of  $p_1, \dots, p_r$ .

Conversely, given a set of points  $V$  in  $\text{PG}(n, q)$ , the ideal  $I(V)$  is the set of homogeneous polynomials in  $\mathbb{F}_q[X_0, \dots, X_n]$  which vanish on all of  $V$ . This set is an ideal in the polynomial ring. In  $\text{PG}(n, q)$ , every set is algebraic of degree at most  $(n+1)(q-1)$  [29]. The associated polynomial is unique and known as the algebraic normal form of the set.

In order to work with algebraic sets, multivariate polynomial rings are required. For details, see Section 5.2.

Suppose we are interested in  $\mathbb{F}_{11}$ -rational points of the elliptic curve  $y^2 = x^3 + x + 3$ . We write  $x^3 + 3 - y^2 + x = 0$ . Homogenizing yields  $X^3 + 3Z^3 - Y^2Z + XZ = 0$ . Using  $X_0, X_1, X_2$  instead of  $X, Y, Z$  yields

$$X_0^3 + 3X_2^3 + 10X_1^2X_2 + X_0X_2^2 = 0.$$

Using the indexing of monomials from Table 5.4, we record the coefficient vector of the equation as sequence

$$(1, 0, 3, 0, 0, 0, 10, 1, 0, 0).$$

The Orbiter command

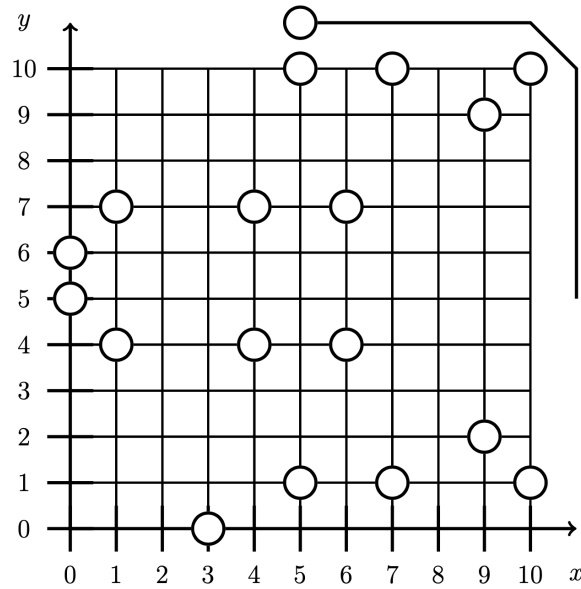
### Example 103

```
EC_11_EQUATION="1,0,3,0,0,0,10,1,0,0"
```

### Example 104

```
EC_11.txt:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 11 -end \
> > -define R -polynomial_ring -field F \
> > > -number_of_variables 3 \
> > > -homogeneous_of_degree 3 \
> > > -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -define EC -geometric_object P \
> > > -projective_variety R \
> > > > "EC_11" "EC_11" \
> > > > $(EC_11_EQUATION) \
> > > -end \
> > -with EC -do -combinatorial_object_activity -save \
> > -end
```

creates the algebraic set associated to the cubic curve  $y^2 = x^3 + x + 3$  in  $\text{PG}(2, 11)$ . It turns out that there are exactly 18 points over  $\mathbb{F}_{11}$ . A picture is shown below:



Suppose we want to create the Hirschfeld surface with equation

$$X_0^2 X_3 + X_1^2 X_2 + X_1 X_2^2 + X_0 X_3^2 = 0.$$

Based on the partition ordering of Figure 5.5, the equation is coded as coefficient vector

$$(0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0).$$

The following command can be used to create the variety over  $\mathbb{F}_4$ :

#### Example 105

```
HIRSCHFELD_SURFACE_EQUATION="0,0,0,0,0,0,1,0,1,0, 0,1,0,1,0,0,0,0,0,0"
```

#### Example 106

Hirschfeld\_surface.q4.txt:

```
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -define R -polynomial_ring -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 3 \
> > > -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define H4 -geometric_object P \
> > > -projective_variety R \
> > > > "Hirschfeld_surface.q4" \
> > > > "Hirschfeld\_surface\_q4" \
> > > > $(HIRSCHFELD_SURFACE_EQUATION) \
> > > -end \
> > -with H4 -do -combinatorial_object_activity -save \
```

```
▷ ▷ -end
```

A file called `Hirschfeld_surface_q4.txt` is created. The file contains the Orbiter ranks of the 45 points on the surface.



## 4.6 The Klein Quadric and the Plücker Map

Orbiter can work with the Grassmannian over a finite field. In particular, Orbiter offers indexing for subspaces. For the special case of the Grassmannian  $\mathfrak{Gr}_{4,2}(V)$ , Plücker coordinates can be used to identify  $\mathfrak{Gr}_{4,2}(V)$  with the  $Q^+(5, q)$  (Klein) quadric. Here is an example. The command

### Example 107

```
GR_4.2.2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -report_Grassmannian 2 \
> > -end
> pdflatex Gr_4.2.2.tex
> $(OPEN) Gr_4.2.2.pdf
```

creates the elements of  $\mathfrak{Gr}_{4,2,2}$  and lists them together with their Plücker coordinates. The following list is produced (output shortened):

There are 35 lines:

$$\begin{aligned} L_0 &= \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 0, 0) & L_2 &= \begin{bmatrix} 1000 \\ 0101 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 1, 0) \\ L_1 &= \begin{bmatrix} 1000 \\ 0110 \end{bmatrix} = \mathbf{Pl}(1, 0, 1, 0, 0, 0) & \vdots & \\ & & L_{34} &= \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 1, 0, 0, 0, 0) \end{aligned}$$

The Plücker coordinates satisfy

$$p_{12}p_{34} - p_{13}p_{24} + p_{14}p_{23} = 0$$

and hence belong to the Klein quadric  $Q^+(5, q)$ . Orthogonal spaces and quadrics will be discussed in Section 4.7.

The Orbiter labeling of points of the  $Q^+(5, q)$  quadric (see Section 4.7) can then be used to enumerate the lines of  $\text{PG}(3, q)$  in a second, different way. In the example of  $\text{PG}(3, 2)$ , this yields the following list (output shortened):

$$\begin{aligned} 0 &= \mathbf{Pl}(1, 0, 0, 0, 0, 0) = L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} & 2 &= \mathbf{Pl}(0, 0, 1, 0, 0, 0) = L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix} \\ 1 &= \mathbf{Pl}(0, 1, 0, 0, 0, 0) = L_{34} = \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} & \vdots & \\ & & 34 &= \mathbf{Pl}(0, 1, 1, 1, 1, 1) = L_{26} = \begin{bmatrix} 1101 \\ 0011 \end{bmatrix} \end{aligned}$$

| Types of Quadrics                       |                                                                  |                                                    |
|-----------------------------------------|------------------------------------------------------------------|----------------------------------------------------|
| Type                                    | Quadratic Form                                                   | # Points                                           |
| $Q^+(n, q)$<br>Hyperbolic ( $n$ is odd) | $\sum_{i=0}^{\frac{n-1}{2}-1} X_{2i} X_{2i+1}$                   | $\frac{(q^{(n+1)/2} - 1)(q^{(n-1)/2} + 1)}{q - 1}$ |
| $Q^-(n, q)$<br>Elliptic ( $n$ is odd)   | $p(X_{n-1}, X_n) + \sum_{i=0}^{\frac{n-1}{2}-1} X_{2i} X_{2i+1}$ | $\frac{(q^{(n+1)/2} + 1)(q^{(n-1)/2} - 1)}{q - 1}$ |
| $Q(n, q)$<br>Parabolic ( $n$ is even)   | $X_0^2 + \sum_{i=0}^{\frac{n}{2}-1} X_{2i+1} X_{2i+2}$           | $\frac{q^n - 1}{q - 1}$                            |

Table 4.4: Types of Quadrics

| Creating an Orthogonal Space |           |                                                                                                                                               |
|------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Command                      | Arguments | Purpose                                                                                                                                       |
| -label_txt                   | L         | Set the ascii-label of the space. The label is used for things like file names etc. A default label will be used if this option is not given. |
| -label_tex                   | L         | Set the tex-label of the space. The label is used within latex reports. A default label will be used if this option is not given.             |
| -without_group               |           | Do not create the orthogonal group.                                                                                                           |

Table 4.5: Creating an Orthogonal Space

## 4.7 Orthogonal Spaces

Orbiter can create and work with orthogonal spaces and their groups. An orthogonal space is created by a quadratic form. We assume that the form is nondegenerate. There are three types of nondegenerate quadratic forms in  $PG(n, q)$ . Two when  $n$  is odd (hyperbolic and elliptic) and one if  $n$  is even (parabolic). Basic information about these quadrics and their representative quadratic forms in Orbiter is given in Table 4.4. Here,  $p(X, Y) = c_1 X^2 + c_2 XY + c_3 Y^2 \in \mathbb{F}_q[X, Y]$  is irreducible over  $\mathbb{F}_q$ .

To create an orthogonal space, the

-orthogonal\_space  $\epsilon$   $d$   $q$  -end

command can be used. Here,  $d = n + 1$ ,  $q$  is the order of the finite field, and

$$\epsilon = \begin{cases} 1 & \text{hyperbolic type } Q^+(d-1, q), & d \text{ even} \\ 0 & \text{elliptic type } Q(d-1, q), & d \text{ odd} \\ -1 & \text{hyperbolic type } Q^-(d-1, q), & d \text{ even} \end{cases}$$

In Table 4.5, Orbiter command options for creating orthogonal spaces are shown.

For instance, the following command creates  $Q(3, 2)$  together with its group  $PGO^+(4, 2)$ :

### Example 108

```
Op_4.2:
▷ $(ORBITER) -v 2 \
```

```

▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define O -orthogonal_space 1 4 F -without_group -end \
▷ ▷ -with O -do -orthogonal_space_activity \
▷ ▷ ▷ -cheat_sheet_orthogonal -end
▷ pdflatex 0_1_4_2_report.tex
▷ $(OPEN) 0_1_4_2_report.pdf

```

The next command creates  $Q(4, 2)$  together with its group  $PGO(5, 2)$ . There are 15 points and 15 lines. The geometry is a configuration  $15_3$  which is also known as the Cremona-Richmond configuration.

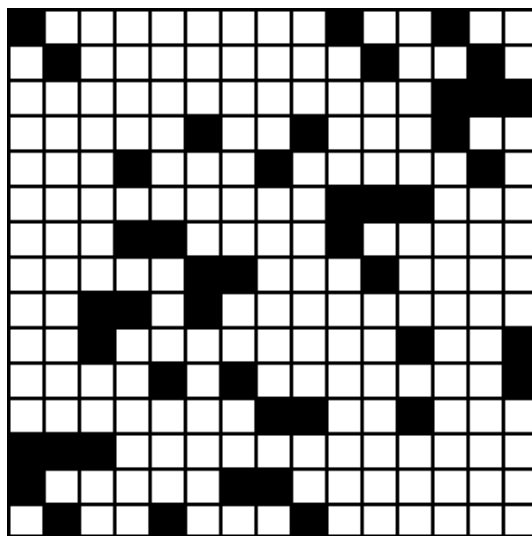
#### Example 109

```

0_5_2_incidence_matrix.csv:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
▷ ▷ -with O -do -orthogonal_space_activity \
▷ ▷ ▷ -export_point_line_incidence_matrix \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -define all_one_r -vector -repeat 1 15 -end \
▷ ▷ -define all_one_c -vector -repeat 1 15 -end \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file 0_5_2_incidence_matrix.csv \
▷ ▷ ▷ -box_width 20 -bit_depth 8 \
▷ ▷ ▷ -partition 2 \
▷ ▷ ▷ ▷ all_one_r all_one_c \
▷ ▷ -end
▷ $(OPEN) 0_5_2_incidence_matrix_draw.bmp

```

The command also creates a bitmap drawing of the incidence matrix between points and lines of  $Q(4, 2)$ .



The Orbiter indexing for points and lines of quadrics is used to address rows and columns.

By default, the orthogonal space is created together with the orthogonal group  $\text{PGO}(n+1, q)$ . When  $q$  is prime, the group  $\text{PGO}(n+1, q)$  is created instead (the groups are isomorphic in this case, and  $\text{PGO}(n+1, q)$  is a bit more efficient). For large orthogonal spaces, creating the group is expensive in terms of time and memory. The a command `-without_group` can be used to prevent the group from being created. For instance

```
-define 0 -orthogonal_space 1 6 2 -end
```

creates an object  $O$  of type  $Q^+(5, 2)$ . In Table 4.6, Orbiter activities for orthogonal spaces are shown.

The command

#### Example 110

```
Op_6.2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define 0 -orthogonal_space 1 6 F -without_group -end \
> > -with 0 -do -orthogonal_space_activity \
> > > -cheat_sheet.orthogonal -end
> pdflatex 0_1.6.2_report.tex
> $(OPEN) 0_1.6.2_report.pdf
```

produces a cheat sheet for the quadric  $Q^+(5, 2)$ . This is the Klein quadric from Section 4.6. Orbiter produces the following output. At the top is the tactical decomposition of the incidence matrix between points and lines with respect to a hyperbolic pair. After that, the points and lines are listed (output shortened):

| Orthogonal Space Activities         |           |                                                                                                              |
|-------------------------------------|-----------|--------------------------------------------------------------------------------------------------------------|
| Command                             | Arguments | Purpose                                                                                                      |
| -create_BLT_set                     | descr     | Creates a BLT-set of $Q(4, q)$ . See Section 14.4.                                                           |
| -cheat_sheet_orthogonal             |           | Create a cheat sheet.                                                                                        |
| -print_points                       | $v$       | Print the points whose ranks are given in the vector $v$ .                                                   |
| -print_lines                        | $v$       | Print the lines whose ranks are given in the vector $v$ .                                                    |
| -unrank_line_through_two_points     | $p1\ p2$  | Determine the rank of the line through the points whose ranks are $p1$ and $p2$ .                            |
| -lines_on_point                     | $p$       | Create the ranks of all lines through the point whose rank is $p$ .                                          |
| -perp                               | $L$       | Determine the common perp of a set of points. The point ranks are given in the list $L$ .                    |
| -export_point_line_incidence_matrix |           | Create a csv file with the point line incidence matrix of the space.                                         |
| -intersect_with_subspace            | $M$       | Find the points in the intersection of the quadric with the subspace whose generating matrix has label $M$ . |

Table 4.6: Orthogonal Space Activities

| $\rightarrow$ | 9 | 36 | 18 | 18 | 6 | 9 | 9 |
|---------------|---|----|----|----|---|---|---|
| 6             | 3 | 6  | 0  | 0  | 0 | 0 | 0 |
| 9             | 0 | 4  | 4  | 0  | 0 | 1 | 0 |
| 9             | 0 | 4  | 0  | 4  | 0 | 0 | 1 |
| 9             | 1 | 0  | 2  | 2  | 2 | 1 | 1 |
| 1             | 0 | 0  | 0  | 0  | 0 | 9 | 0 |
| 1             | 0 | 0  | 0  | 0  | 0 | 0 | 9 |
| $\downarrow$  | 9 | 36 | 18 | 18 | 6 | 9 | 9 |
| 6             | 2 | 1  | 0  | 0  | 0 | 0 | 0 |
| 9             | 0 | 1  | 2  | 0  | 0 | 1 | 0 |
| 9             | 0 | 1  | 0  | 2  | 0 | 0 | 1 |
| 9             | 1 | 0  | 1  | 1  | 3 | 1 | 1 |
| 1             | 0 | 0  | 0  | 0  | 0 | 1 | 0 |
| 1             | 0 | 0  | 0  | 0  | 0 | 0 | 1 |

The number of points is 35  
points:

$$P_0 = (1, 0, 0, 0, 0, 0)$$

$$P_1 = (0, 1, 0, 0, 0, 0)$$

$$P_2 = (0, 0, 1, 0, 0, 0)$$

$$P_3 = (1, 0, 1, 0, 0, 0)$$

$$P_4 = (0, 1, 1, 0, 0, 0)$$

$$\begin{aligned}
P_5 &= (0, 0, 0, 1, 0, 0) \\
P_6 &= (1, 0, 0, 1, 0, 0) \\
P_7 &= (0, 1, 0, 1, 0, 0) \\
P_8 &= (1, 1, 1, 1, 0, 0) \\
P_9 &= (0, 0, 0, 0, 1, 0) \\
P_{10} &= (1, 0, 0, 0, 1, 0) \\
P_{11} &= (0, 1, 0, 0, 1, 0) \\
P_{12} &= (0, 0, 1, 0, 1, 0) \\
P_{13} &= (1, 0, 1, 0, 1, 0) \\
P_{14} &= (0, 1, 1, 0, 1, 0) \\
P_{15} &= (0, 0, 0, 1, 1, 0) \\
P_{16} &= (1, 0, 0, 1, 1, 0) \\
P_{17} &= (0, 1, 0, 1, 1, 0) \\
P_{18} &= (1, 1, 1, 1, 1, 0) \\
P_{19} &= (0, 0, 0, 0, 0, 1) \\
P_{20} &= (1, 0, 0, 0, 0, 1) \\
P_{21} &= (0, 1, 0, 0, 0, 1) \\
P_{22} &= (0, 0, 1, 0, 0, 1) \\
P_{23} &= (1, 0, 1, 0, 0, 1) \\
P_{24} &= (0, 1, 1, 0, 0, 1) \\
P_{25} &= (0, 0, 0, 1, 0, 1) \\
P_{26} &= (1, 0, 0, 1, 0, 1) \\
P_{27} &= (0, 1, 0, 1, 0, 1) \\
P_{28} &= (1, 1, 1, 1, 0, 1) \\
P_{29} &= (1, 1, 0, 0, 1, 1) \\
P_{30} &= (1, 1, 1, 0, 1, 1) \\
P_{31} &= (1, 1, 0, 1, 1, 1) \\
P_{32} &= (0, 0, 1, 1, 1, 1) \\
P_{33} &= (1, 0, 1, 1, 1, 1) \\
P_{34} &= (0, 1, 1, 1, 1, 1)
\end{aligned}$$

The number of lines is 105

$$\begin{aligned}
L_0 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \{P_0, P_{32}, P_{33}\} \\
L_1 &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \{P_1, P_{32}, P_{34}\} \\
L_{104} &= \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \{P_8, P_9, P_{18}\}
\end{aligned}$$

Orbiter has enumerators for points and lines in orthogonal spaces. For small spaces, the cheat sheet lists points and lines in the Orbiter ordering. Creating the groups can be expensive. For large spaces, it may be necessary to disable the group using the `-without_group` option. The command

#### Example 111

```

Op_6.64.line_rank:
> $(ORBITER) -v 4 \
> > -define F -finite_field -q 64 -end \
> > -define O -orthogonal_space 1 6 F -without_group -end \
> > -with O -do -orthogonal_space_activity \
> > > -unrank_line_through_two_points 15447347 15225451 \
> > -end

```

computes the Orbiter rank of the line through the points with rank 15447347 and 15225451, respectively. The rank of the line is 16767254. These ranks refer to the orthogonal geometry. They are different from the ranks of points and lines in projective spaces.

It is possible to create reports for orthogonal spaces without group. In this case, the group information will be skipped. For instance, the following command creates a report for  $Q(5, 64)$ :

#### Example 112

```
Op_6_64_report:
▷ $(ORBITER) -v 4 \
▷ ▷ -define F -finite_field -q 64 -end \
▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
▷ ▷ -with O -do -orthogonal_space_activity \
▷ ▷ ▷ -cheat_sheet_orthogonal \
▷ ▷ -end
▷ pdflatex O_1_6_64_report.tex
▷ $(OPEN) O_1_6_64_report.pdf
```

The report does not show information about the group. However, it still contains the tactical decomposition with respect to a hyperbolic pair. The printing of points is restricted to small spaces only.

The group is not available.  
The quadratic form is:

$$X_0X_1 + X_2X_3 + X_4X_5 = 0$$

| →        | 16769025 | 1090252800 | 532350 | 532350 | 130 | 4225 | 4225 |
|----------|----------|------------|--------|--------|-----|------|------|
| 16511040 | 65       | 4160       | 0      | 0      | 0   | 0    | 0    |
| 266175   | 0        | 4096       | 128    | 0      | 0   | 1    | 0    |
| 266175   | 0        | 4096       | 0      | 128    | 0   | 0    | 1    |
| 4225     | 3969     | 0          | 126    | 126    | 2   | 1    | 1    |
| 1        | 0        | 0          | 0      | 0      | 0   | 4225 | 0    |
| 1        | 0        | 0          | 0      | 0      | 0   | 0    | 4225 |
| ↓        | 16769025 | 1090252800 | 532350 | 532350 | 130 | 4225 | 4225 |
| 16511040 | 64       | 63         | 0      | 0      | 0   | 0    | 0    |
| 266175   | 0        | 1          | 64     | 0      | 0   | 63   | 0    |
| 266175   | 0        | 1          | 0      | 64     | 0   | 0    | 63   |
| 4225     | 1        | 0          | 1      | 1      | 65  | 1    | 1    |
| 1        | 0        | 0          | 0      | 0      | 0   | 1    | 0    |
| 1        | 0        | 0          | 0      | 0      | 0   | 0    | 1    |

The number of points is 17047617  
Too many points to print.  
The number of lines is 1108095105

To study BLT-sets in  $Q(4, q)$ , see Section 14.4.

According to Table 4.4, Orbiter uses the equation

$$X_0X_1 + X_2X_3 + X_4X_5 = 0$$

to define the Klein quadric. An elliptic quadric is an ovoid of the Klein quadric that is obtained by intersecting the quadric with a suitable solid. In  $\text{PG}(5, 5)$ , the subspace generated by the rows of the matrix

$$\begin{bmatrix} 1 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

is such a subspace. The ordering of columns corresponds to the natural ordering of the variables as  $X_0, X_1, X_2, X_3, X_4, X_5$ . The following command produces a list of points of an elliptic quadric ovoid in  $Q^+(5, 5)$ .

#### Example 113

```
elliptic_quadric_subspace:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 5 -end \
> > -define v -vector -format 4 \
> > > -dense "1,3,0,0,0,0, 0,0,1,0,0,0, 0,0,0,1,0,0, 0,0,0,0,1,1" \
> > -end \
> > -define O -orthogonal_space 1 6 F -end \
> > -with O -do -orthogonal_space_activity \
> > > -intersect_with_subspace v \
> > -end
```

The elliptic quadric has 26 points.

The coding of points and line in orthogonal spaces is different from the coding of points in projective spaces. We will create and print a set called BLT set (after [4]). This is a set of  $q + 1$  points on the  $Q(4, q)$  quadric satisfying a special geometric property. According to Table 4.4, Orbiter uses the equation

$$X_0^2 + X_1X_2 + X_3X_4 = 0$$

to define the  $Q(4, q)$  quadric. The following example creates the BLT-set with Orbiter catalogue number #1 in  $Q(4, 7)$ :

#### Example 114

```
BLT_database_7_1:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -define P -projective_space -n 4 -field F -v 0 -end \
> > -define S -geometric_object P \
> > > -BLT_database 1 \
> > -end \
> > -with S -do -combinatorial_object_activity -save \
> > -end
```

The set is stored in a file. The next command reads the file and prints the elements of the set:



**Example 115**

```
BLT_database_7_1_print:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -define O -orthogonal_space 0 5 F -without_group -end \
> > -define S -set -file_orbiter_format BLT_7_1.txt -end \
> > -with O -do -orthogonal_space_activity \
> > > -print_points S -end
> pdflatex S_set_report.tex
> $(OPEN) S_set_report.pdf
```

The command produces the following list of points, comprising the second BLT-set over  $\mathbb{F}_7$ .

A set of points of size 8  
The Points:  
0 :  $P_0 = (0, 1, 0, 0, 0)$   
1 :  $P_1 = (0, 0, 1, 0, 0)$   
2 :  $P_{40} = (0, 1, 2, 6, 2)$   
3 :  $P_{41} = (0, 1, 4, 3, 1)$   
4 :  $P_{225} = (1, 6, 6, 5, 1)$   
5 :  $P_{270} = (1, 5, 5, 4, 4)$   
6 :  $P_{241} = (1, 2, 2, 2, 1)$   
7 :  $P_{340} = (1, 1, 1, 4, 3)$

More on BLT-sets can be found in Section 14.4.

The next command prints points and lines of the  $W(2)$ , also known as the Doily. It is an example of a generalized quadrangle.

**Example 116**

```
Doily_W2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define O -orthogonal_space 0 5 F -without_group -end \
> > -define W2_points -set -loop 0 15 1 -end \
> > -define W2_lines -set -loop 0 15 1 -end \
> > -with O -do \
> > > -orthogonal_space_activity \
> > > -print_points W2_points \
> > -end \
> > -with O -do \
> > > -orthogonal_space_activity \
> > > -print_lines W2_lines \
> > -end
> pdflatex W2_points_set_report.tex
> $(OPEN) W2_points_set_report.pdf
> pdflatex W2_lines_set_of_lines_report.tex
> $(OPEN) W2_lines_set_of_lines_report.pdf
```

## 4.8 Hermitian Varieties

Orbiter has enumerators for points of the hermitian variety  $H(k, Q)$ . Here,  $Q$  is a square, and so  $q = \sqrt{Q}$  is an integer. The equation of the variety is

$$\sum_{i=0}^k X_i^{q+1} = 0.$$

The command

### Example 117

```
H_2_4:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -with F -do -finite_field_activity \
> > > -cheat_sheet_hermitian 2 -end
> pdflatex H_2_4.tex
> $(OPEN) H_2_4.pdf
```

produces a cheat sheet for the variety  $H(2, 4)$ :

The Hermitian variety  $H(2, 4)$  contains 9 points:

$$P_0 = (1, 1, 0) = 4$$

$$P_1 = (2, 1, 0) = 5$$

$$P_2 = (3, 1, 0) = 6$$

$$P_3 = (1, 0, 1) = 7$$

$$P_4 = (2, 0, 1) = 8$$

$$P_5 = (3, 0, 1) = 9$$

$$P_6 = (0, 1, 1) = 10$$

$$P_7 = (0, 2, 1) = 13$$

$$P_8 = (0, 3, 1) = 17$$

All points: ( 4, 5, 6, 7, 8, 9, 10, 13, 17 )

The command

### Example 118

```
H_3_4:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -with F -do -finite_field_activity \
> > > -cheat_sheet_hermitian 3 -end
> pdflatex H_3_4.tex
> $(OPEN) H_3_4.pdf
```

produces a cheat sheet for the variety  $H(3, 4)$ .

The Hermitian variety  $H(3, 4)$  contains 45 points:

|                              |                              |
|------------------------------|------------------------------|
| $P_0 = (1, 1, 0, 0) = 5$     | $P_{23} = (3, 3, 1, 1) = 52$ |
| $P_1 = (2, 1, 0, 0) = 6$     | $P_{24} = (0, 0, 1, 1) = 38$ |
| $P_2 = (3, 1, 0, 0) = 7$     | $P_{25} = (1, 1, 2, 1) = 58$ |
| $P_3 = (1, 0, 1, 0) = 8$     | $P_{26} = (2, 1, 2, 1) = 59$ |
| $P_4 = (2, 0, 1, 0) = 9$     | $P_{27} = (3, 1, 2, 1) = 60$ |
| $P_5 = (3, 0, 1, 0) = 10$    | $P_{28} = (1, 2, 2, 1) = 62$ |
| $P_6 = (0, 1, 1, 0) = 11$    | $P_{29} = (2, 2, 2, 1) = 63$ |
| $P_7 = (0, 2, 1, 0) = 15$    | $P_{30} = (3, 2, 2, 1) = 64$ |
| $P_8 = (0, 3, 1, 0) = 19$    | $P_{31} = (1, 3, 2, 1) = 66$ |
| $P_9 = (1, 0, 0, 1) = 23$    | $P_{32} = (2, 3, 2, 1) = 67$ |
| $P_{10} = (2, 0, 0, 1) = 24$ | $P_{33} = (3, 3, 2, 1) = 68$ |
| $P_{11} = (3, 0, 0, 1) = 25$ | $P_{34} = (0, 0, 2, 1) = 53$ |
| $P_{12} = (0, 1, 0, 1) = 26$ | $P_{35} = (1, 1, 3, 1) = 74$ |
| $P_{13} = (0, 2, 0, 1) = 30$ | $P_{36} = (2, 1, 3, 1) = 75$ |
| $P_{14} = (0, 3, 0, 1) = 34$ | $P_{37} = (3, 1, 3, 1) = 76$ |
| $P_{15} = (1, 1, 1, 1) = 4$  | $P_{38} = (1, 2, 3, 1) = 78$ |
| $P_{16} = (2, 1, 1, 1) = 43$ | $P_{39} = (2, 2, 3, 1) = 79$ |
| $P_{17} = (3, 1, 1, 1) = 44$ | $P_{40} = (3, 2, 3, 1) = 80$ |
| $P_{18} = (1, 2, 1, 1) = 46$ | $P_{41} = (1, 3, 3, 1) = 82$ |
| $P_{19} = (2, 2, 1, 1) = 47$ | $P_{42} = (2, 3, 3, 1) = 83$ |
| $P_{20} = (3, 2, 1, 1) = 48$ | $P_{43} = (3, 3, 3, 1) = 84$ |
| $P_{21} = (1, 3, 1, 1) = 50$ | $P_{44} = (0, 0, 3, 1) = 69$ |
| $P_{22} = (2, 3, 1, 1) = 51$ |                              |

All points: ( 5, 6, 7, 8, 9, 10, 11, 15, 19, 23, 24, 25, 26, 30, 34, 4, 43, 44, 46, 47, 48, 50, 51, 52, 38, 58, 59, 60, 62, 63, 64, 66, 67, 68, 53, 74, 75, 76, 78, 79, 80, 82, 83, 84, 69 )

Coincidentally, this Hermitian variety is the Hirschfeld cubic surface over  $\mathbb{F}_4$ .

## 4.9 Advanced Topics

The Orbiter commands associated with projective space objects are summarized in Tables 4.7-4.10.

Table 4.11 lists Orbiter global commands related to projective geometries. These commands do not need an object of type projective space in order to be invoked.

Suppose we want to study the fix structure of a collineation in projective space. Suppose we want to do so for the Baer collineation

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1.$$

It fixes a subgeometry PG(3, 2). The command

### Example 119

```
fixed_objects_2A:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -report_fixed_objects \
> > > > "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1, 1" \
> > > > fix_structure_2A.tex \
> > -end
> pdflatex fix_structure_2A.tex
> $(OPEN) fix_structure_2A.pdf
```

can be used. The output is shown below. There are 15 fixed points, 35 fixed lines, and 15 fixed planes. These objects form a Baer subgeometry.

The element

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1$$

has the following fixed objects:

There are 15 / 85 fixed points, they are:

```
0 / 15 = 0 : ( 1, 0, 0, 0 )
1 / 15 = 1 : ( 0, 1, 0, 0 )
2 / 15 = 2 : ( 0, 0, 1, 0 )
3 / 15 = 3 : ( 0, 0, 0, 1 )
4 / 15 = 4 : ( 1, 1, 1, 1 )
5 / 15 = 5 : ( 1, 1, 0, 0 )
6 / 15 = 8 : ( 1, 0, 1, 0 )
7 / 15 = 11 : ( 0, 1, 1, 0 )
8 / 15 = 12 : ( 1, 1, 1, 0 )
```

| Projective Space Activities (Part 1)                         |                        |                                                                                                                                                                                                                     |
|--------------------------------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                                      | Arguments              | Purpose                                                                                                                                                                                                             |
| -export_point_line_incidence_matrix                          |                        | Create a csv file of the point line incidence matrix.                                                                                                                                                               |
| -export_restricted_point_line_incidence_matrix               | points lines           | Create a csv file of the incidence matrix between the given set of points (rows) and lines (columns).                                                                                                               |
| -export_cubic_surface_line_vs_line_incidence_matrix          |                        |                                                                                                                                                                                                                     |
| -export_cubic_surface_line_tritangent_plane_incidence_matrix |                        |                                                                                                                                                                                                                     |
| -export_double_sixes                                         |                        |                                                                                                                                                                                                                     |
| -table_of_cubic_surfaces_compute_properties                  | fname $q_0$ col-offset | Computes the properties of cubic surfaces over a different field.                                                                                                                                                   |
| -cubic_surface_properties_analyze                            | fname $q_0$            | Explores the properties of cubic surfaces over a different field.                                                                                                                                                   |
| -canonical_form_of_code                                      | label $m$ $n$ matrix   | Compute the automorphism group of a linear code using Nauty. See Section 10.                                                                                                                                        |
| -map                                                         | formula parameters     | Evaluate a formula using the given parameters                                                                                                                                                                       |
| -affine_map                                                  | R formula parameters   | Evaluate an affine map given by a formula using the given parameters over the ring R.                                                                                                                               |
| -analyze_del_Pezzo_surface                                   | label parameters       |                                                                                                                                                                                                                     |
| -decomposition_by_element_PG                                 | power elt<br>fname     | Analyze the orbit structure of the cyclic group generated by the given element in the action on $PG(n, q)$ . The command also produces a latex report.                                                              |
| -decomposition_by_subgroup                                   | label descr            | Analyze the orbit structure of the subgroup $H$ in the action on $PG(n, q)$ . The subgroup must be a linear group, and the description of $H$ must come from the commands from Section 6.2. Produce a latex report. |
| -table_of_quartic_curves                                     |                        | Export the classification of quartic curves to a csv file.                                                                                                                                                          |
| -table_of_cubic_surfaces                                     |                        | Export the classification of cubic surfaces to a csv file.                                                                                                                                                          |

Table 4.7: Projective Space Activities (Part 1)

| Projective Space Activities (Part 2)                    |                                                  |                                                                                                           |
|---------------------------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Command                                                 | Arguments                                        | Purpose                                                                                                   |
| -sweep                                                  | fname                                            |                                                                                                           |
| -sweep_4_15_lines                                       | fname<br>surface-descr                           |                                                                                                           |
| -sweep_F_beta_9_lines                                   | fname<br>surface-descr                           |                                                                                                           |
| -sweep_6_9_lines                                        | fname<br>surface-descr                           |                                                                                                           |
| -sweep_4_27                                             | fname<br>surface-descr                           |                                                                                                           |
| -sweep_4_L9_E4                                          | fname<br>surface-descr                           |                                                                                                           |
| -cheat_sheet                                            |                                                  | Produce a cheat sheet for $\text{PG}(n, q)$                                                               |
| -set_stabilizer                                         | $k$ fname-<br>mask $N$<br>col-label<br>fname-out | Compute canonical form of sets using the substructure algorithm.                                          |
| -conic_type                                             | $t$ set                                          | Compute the conic type of the given set (given by its label). Record intersections of size $\geq t$ only. |
| -arc_with_given_set_as_<br>s_lines_after_dualizing      | $sz$ $d$ $d_{\min}$ $s$                          | Finds arcs with the given set as $s$ -lines.                                                              |
| -arc_with_two_given_sets_<br>of_lines_after_dualizing   | $sz$ $d$ $d_{\min}$ $s$ $t$<br>$T$               | Finds arcs with the two given sets as $s$ -lines and $t$ -lines, respectively.                            |
| -arc_with_three_given_sets_<br>of_lines_after_dualizing | $sz$ $d$ $d_{\min}$ $s$ $t$<br>$T$ $u$ $U$       | Finds arcs with the three given sets as $s$ -lines and $t$ -lines and $u$ -lines, respectively.           |
| -dualize_hyperplanes_to_points                          |                                                  | Turns ranks of hyperplanes into ranks of points.                                                          |
| -dualize_points_to_hyperplanes                          |                                                  | Turns ranks of points into ranks of hyperplanes.                                                          |
| -dualize_rank_k_subspaces                               | $k$                                              | Turns ranks of $k$ -subspaces into ranks of $n-k$ subspaces.                                              |

Table 4.8: Projective Space Activities (Part 2)

| Projective Space Activities (Part 3)          |                         |                                                                                                                                                                                                                      |
|-----------------------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                       | Arguments               | Purpose                                                                                                                                                                                                              |
| -lines_on_point_but_within_a_plane            | pt-rk plane-rk          | Compute the lines through a given point contained in a given plane.                                                                                                                                                  |
| -rank_lines_in_PG                             | $M$                     | Rank lines. The generating systems for the lines are given in rows of the matrix $M$ .                                                                                                                               |
| -unrank_lines_in_PG                           | $v$                     | Unrank lines. The ranks are given in the vector $v$ .                                                                                                                                                                |
| -move_two_lines_in_hyperplane_stabilizer      | $l1\ l2\ m1\ m2$        | Find the unique transvection fixing the hyperplane at infinity moving $l1$ and $l2$ to $m1$ and $m2$ .                                                                                                               |
| -move_two_lines_in_hyperplane_stabilizer_text | $l1\ l2\ m1\ m2$        | Find the unique transvection fixing the hyperplane at infinity moving $l1$ and $l2$ to $m1$ and $m2$ .                                                                                                               |
| -planes_through_line                          | $l$                     | Find all planes through the line $l$ .                                                                                                                                                                               |
| -restricted_incidence_matrix                  | $t1\ t2\ R\ C$<br>fname | Compute the restricted incidence matrix between objects $R$ of type $t1$ and objects $C$ of type $t2$ . Write output to the given file in csv format.                                                                |
| -plane_intersection_type_of_klein_image       | $t$ input               | Compute the plane intersection type of the Klein image of the lines given in input. Use a threshold of $t$ to filter planes. Only planes which intersect in at least $t$ points on the Klein quadric are considered. |
| -report_Grassmannian                          | $k$                     | Produce a report on the Grassmannian of $k$ -dimensional subspaces.                                                                                                                                                  |
| -report_fixed_objects                         | data fname              | Produce a report about the fixed points of the group element given by data in the action on objects of various types of the projective space. Writes a latex report using the given file name.                       |

Table 4.9: Projective Space Activities (Part 3)

| Projective Space Activities (Part 4)                |                                 |                                                                                                                                      |
|-----------------------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Command                                             | Arguments                       | Purpose                                                                                                                              |
| -classify_surfaces_with_double_sixes                | label control                   | Classify cubic surfaces using the approach of double sixes. See Section 8.4.                                                         |
| -classify_surfaces_through_arcs_and_two_lines       |                                 | Classify cubic surfaces using the approach of six-arcs and two skew lines. See Section 8.4.                                          |
| -classify_surfaces_through_arcs_and_trihedral_pairs |                                 | See Section 8.4.                                                                                                                     |
| -test_nb_Eckardt_points                             | $e$                             | Restrict to $e$ Eckardt points. See Section 8.4. To be used in conjunction with -classify_surfaces_through_arcs_and_trihedral_pairs. |
| -trihedra1_control                                  | poset-control                   | For -classify_surfaces_through_arcs_and_trihedral_pairs                                                                              |
| -trihedra2_control                                  | poset-control                   | For -classify_surfaces_through_arcs_and_trihedral_pairs                                                                              |
| -control_six_arcs                                   | poset-control                   | For -classify_surfaces_through_arcs_and_trihedral_pairs                                                                              |
| -six_arcs_not_on_conic                              |                                 | Classify six-arcs not on a conic in a plane.                                                                                         |
| -filter_by_nb_Eckardt_points                        | $e$                             | Filter for the number of Eckardt points to be equal to $e$ . Used in conjunction with -six_arcs_not_on_conic.                        |
| -classify_quartic_curves_nauty                      | fname-mask<br>$N$ fname         | Classify quartic curves using Nauty.                                                                                                 |
| -classify_quartic_curves_with_substructure          | fname-mask<br>$N$ $k$ $d$ fname | Classify quartic curves using substructure algorithm.                                                                                |
| -classify_arcs                                      | descr                           | Classify arcs.                                                                                                                       |
| -classify_semifields                                | descr                           | Classify semifields.                                                                                                                 |
| -classify_bent_functions                            | $n$                             | Classify bent functions in $n$ variables.                                                                                            |

Table 4.10: Projective Space Activities (Part 4)



| Global Commands for Projective Geometry |                                                            |                                                                                                                                                                                                                                                       |
|-----------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                 | Arguments                                                  | Purpose                                                                                                                                                                                                                                               |
| -create_points_on_quartic               | $\epsilon$                                                 | Create a table of points on a specific quartic curve. Consecutive points are no more than $\epsilon$ apart.                                                                                                                                           |
| -create_points_on_parabola              | $\epsilon \ a \ b \ c$                                     | Create a table of points on the parabola $y = ax^2 + bx + c$ . Consecutive points are no more than $\epsilon$ apart.                                                                                                                                  |
| -smooth_curve                           | $\epsilon \ N \ b \ t_{\min} \ t_{\max} \ \text{function}$ | Create at least $N$ points on a continuous curve given by "function". Consecutive points are no more than $\epsilon$ apart. The function must be in terms of a parameter $t$ . The values of $t$ are taken from the interval $[t_{\min}, t_{\max}]$ . |
| -make_table_of_surfaces                 |                                                            | Produce a latex table summarizing the surfaces in the Orbiter catalogue.                                                                                                                                                                              |
| -create_surface_reports                 | field-orders                                               | Produce reports for all surfaces in the Orbiter catalogue over the give field orders.                                                                                                                                                                 |
| -create_surface_atlas                   | $q_{\max}$                                                 | Produce reports for all surfaces in the Orbiter catalogue for field orders $q \leq q_{\max}$ .                                                                                                                                                        |
| -create_dickson_atlas                   |                                                            | Produce reports of Dickson surfaces.                                                                                                                                                                                                                  |

Table 4.11: Global Commands for Projective Geometry

$$9 / 15 = 23 : (1, 0, 0, 1)$$

$$10 / 15 = 26 : (0, 1, 0, 1)$$

$$11 / 15 = 27 : (1, 1, 0, 1)$$

$$12 / 15 = 38 : (0, 0, 1, 1)$$

$$13 / 15 = 39 : (1, 0, 1, 1)$$

$$14 / 15 = 42 : (0, 1, 1, 1)$$

There are 35 / 357 fixed subspaces of rank 2, they are:

$$0 / 35 = 0 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$1 / 35 = 1 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$2 / 35 = 4 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$3 / 35 = 5 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}$$

$$4 / 35 = 16 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$5 / 35 = 17 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$$6 / 35 = 20 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[illegible]

$$\begin{aligned}
27 / 35 = 125 : & \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
28 / 35 = 336 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
29 / 35 = 337 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
30 / 35 = 340 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
31 / 35 = 341 : & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
32 / 35 = 342 : & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
33 / 35 = 345 : & \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
34 / 35 = 356 : & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned}$$

There are 15 / 85 fixed subspaces of rank 3, they are:

$$\begin{aligned}
0 / 15 = 0 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
1 / 15 = 1 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
2 / 15 = 4 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
3 / 15 = 5 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
4 / 15 = 6 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
5 / 15 = 9 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
6 / 15 = 20 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
7 / 15 = 21 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}
\end{aligned}$$

$$\begin{array}{lcl}
8 / 15 = 22 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} & = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
9 / 15 = 25 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
10 / 15 = 26 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} & = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
11 / 15 = 27 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} & = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
12 / 15 = 30 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
13 / 15 = 41 : & \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
14 / 15 = 84 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{array}$$

Suppose we want to study the fix structure of a collineation group in projective space. Suppose we want to do so for the elementary abelian group of order 4 generated by the two elements

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_0, \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_0.$$

It fixes no points, 5 lines, and no plane. The following command sequence can be used. It uses two makefile variables: First, we set a label for the group:

#### Example 120

```
H1_LABEL="H1"
```

Now we define the subgroup:

#### Example 121

```
H1_GENs=-PGGL 4 4 \
> -subgroup_by_generators "H1" 4 2 \
> "1,0,0,0,1,1,0,0,0,0,1,0,0,0,1,1,0, \
> 1,0,0,0,0,1,0,0,1,0,1,0,0,1,0,1,0" \
> -end
```

And finally the command itself:

#### Example 122

```
fix_structure_G1:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with P -do \
▷ ▷ -projective_space_activity \
▷ ▷ ▷ -decomposition_by_subgroup \
▷ ▷ ▷ ▷ $(H1_LABEL) $(H1_GENS) \
▷ ▷ -end
▷ pdflatex PGGL_4_4.Subgroup_$(H1_LABEL)_4.decomp.tex
▷ $(OPEN) PGGL_4_4.Subgroup_$(H1_LABEL)_4.decomp.pdf
```

The decomposition based on orbit lengths is shown below:

Decomposition based on orbit lengths:  
Row scheme:

| →  | 3 | 2 | 48 | 16 | 16 | 144 | 128 |
|----|---|---|----|----|----|-----|-----|
| 1  | 3 | 2 | 0  | 16 | 0  | 0   | 0   |
| 12 | 1 | 0 | 4  | 0  | 4  | 12  | 0   |
| 8  | 0 | 1 | 0  | 0  | 4  | 0   | 16  |
| 64 | 0 | 0 | 3  | 1  | 0  | 9   | 8   |

Column scheme:

| ↓  | 3 | 2 | 48 | 16 | 16 | 144 | 128 |
|----|---|---|----|----|----|-----|-----|
| 1  | 1 | 1 | 0  | 1  | 0  | 0   | 0   |
| 12 | 4 | 0 | 1  | 0  | 3  | 1   | 0   |
| 8  | 0 | 4 | 0  | 0  | 2  | 0   | 1   |
| 64 | 0 | 0 | 4  | 4  | 0  | 4   | 4   |

The decomposition of orbits is shown as well, but it is too large to be shown.

Suppose we are looking for a projectivity of  $\text{PG}(3, 16)$  fixing the plane  $v(X_3)$  pointwise and mapping a pair of skew lines not in that plane to another pair of skew lines not in that plane. For instance, suppose we want to map

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mapsto N_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_2 = \begin{bmatrix} 1 & 1 & 0 & \delta \\ 0 & 0 & 1 & 0 \end{bmatrix} \mapsto N_2 = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The command

**Example 123**

```

trans:
> $(ORBITER) -v 5 \
> > -define F -finite_field -q 16 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -move_two_lines_in_hyperplane_stabilizer_text \
> > > > "1,0,0,0, 0,0,0,1" "1,1,0,2, 0,0,1,0" \
> > > > "1,0,0,0, 0,0,0,1" "0,1,0,1, 0,0,1,0" \
> > -end

```

computes a projectivity (transvection) to do so:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \delta^{14} & 0 & 0 & \delta^{14} \end{bmatrix}$$

Here,  $\delta$  is the primitive element in the built-in field  $\mathbb{F}_{16}$ , satisfying  $\delta^4 = \delta^3 + 1$ .

## 4.10 Geometric Objects

Orbiter can create objects in projective space. To do so, define an object of type `-geometric_object`. The definition of a geometric object requires a projective geometry object. For this reason, the definition requires an extra argument, which is the label of a previously created projective geometry object. After that, one of the commands shown in Tables 4.12 and 4.13 can be issued.

The following command creates an elliptic quadric ovoid on  $PG(3,8)$ :

### Example 124

```
elliptic_quadric_ovoid_q8:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define O -geometric_object P \
▷ ▷ ▷ -elliptic_quadric_ovoid \
▷ ▷ -end \
▷ ▷ -with O -do -combinatorial_object_activity -save \
▷ ▷ -end
```

The next command creates the Suzuki-Tits ovoid in  $PG(3,8)$ :

### Example 125

```
ovoid_ST_q8:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define O -geometric_object P \
▷ ▷ ▷ -ovoid_ST \
▷ ▷ -end \
▷ ▷ -with O -do -combinatorial_object_activity -save \
▷ ▷ -end
```

The following command creates the Baer subplane of  $PG(2,4)$ :

### Example 126

```
Baer_PG_2_4:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define O -geometric_object P \
▷ ▷ ▷ -Baer_substructure \
▷ ▷ -end \
▷ ▷ -with O -do -combinatorial_object_activity -save \
▷ ▷ -end
```

The next command creates the Baer subgeometry of  $PG(3,4)$ :

| Geometric Objects (Part 1)    |            |                                                                                |
|-------------------------------|------------|--------------------------------------------------------------------------------|
| Command                       | Arguments  | Purpose                                                                        |
| -hyperoval                    |            | To create a hyperoval                                                          |
| -subiaco_oval                 | f_short    | Create the Subiaco oval                                                        |
| -subiaco_hyperoval            |            | Create the Subiaco hyperoval                                                   |
| -adelaide_hyperoval           |            | Create the Adelaide hyperoval                                                  |
| -translation                  | $i$        | Create the translation hyperoval with exponent $i$                             |
| -Segre                        |            | Create the Segre hyperoval                                                     |
| -Payne                        |            | Create the Payne hyperoval                                                     |
| -Cherowitzo                   |            | Create the Cherowitzo hyperoval                                                |
| -O'Keefe_Penttila             |            | Create the O'Keefe, Penttila hyperoval                                         |
| -BLT_database                 | $k$        | Create the $k$ th BLT-set of order $q$ from the database ( $k = 0, 1, \dots$ ) |
| -elliptic_quadric_ovoid       |            | Create an elliptic quadric ovoid in $\text{PG}(3, q)$ .                        |
| -ovoid_ST                     |            | Create the Suzuki Tits ovoid in $\text{PG}(3, q)$ . Here, $q = 2^{2r+1}$ .     |
| -Baer                         |            | Create the (standard) Baer subgeometry                                         |
| -orthogonal                   | $\epsilon$ | Create the $Q^\epsilon(n, q)$ quadric                                          |
| -hermitian                    |            | Create the Hermitian variety given by $\sum_{i=0}^n X_i^{\sqrt{q}+1} = 0$      |
| -cuspidal_cubic               |            | Create the cuspidal cubic $(s^3, ts^2, t^3)$ in $\text{PG}(2, q)$              |
| -twisted_cubic                |            | Create a twisted cubic $(s^3, s^2t, st^2, t^3)$ in $\text{PG}(3, q)$           |
| -elliptic_curve               | $a \ b$    | Create the elliptic curve $y^2 = x^3 + ax + b$                                 |
| -ttp_construction_A           |            | Create the twisted tensor product code of type $A$ [8]                         |
| -ttp_construction_A_hyperoval |            | Create the twisted tensor product code of type $A$ [8]                         |
| -ttp_construction_B           |            | Create the twisted tensor product code of type $B$ [8]                         |

Table 4.12: Geometric Objects (Part 1)



| Geometric Objects (Part 2)          |                                    |                                                                                                                                                                                                                                                                 |
|-------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                             | Arguments                          | Purpose                                                                                                                                                                                                                                                         |
| -unital_XXq_YZq_ZYq                 |                                    | Create the unital with equation $XX^q + YZ^q + ZY^q = 0$                                                                                                                                                                                                        |
| -desarguesian_line_spread_in_PG_3_q |                                    | Create the desarguesian line spread in $PG(3, q)$ as a set of 2-subspaces                                                                                                                                                                                       |
| -Buekenhout_Metz                    |                                    | Create the Buekenhout Metz unital                                                                                                                                                                                                                               |
| -Uab                                | $a \ b$                            | Create the Buekenhout Metz unital in the form of Barwick and Ebert [6]                                                                                                                                                                                          |
| -whole_space                        |                                    | Create the whole space                                                                                                                                                                                                                                          |
| -hyperplane                         | pt                                 | Create the hyperplane given by dual coordinates associated with the given point                                                                                                                                                                                 |
| -segre_variety                      | $a \ b$                            | Create the Segre variety                                                                                                                                                                                                                                        |
| -arc1_BCKM                          |                                    | Create arc from family 1 in [12].                                                                                                                                                                                                                               |
| -arc2_BCKM                          |                                    | Create arc from family 2 in [12].                                                                                                                                                                                                                               |
| -Maruta_Hamada_arc                  |                                    | Create the Maruta Hamada arc                                                                                                                                                                                                                                    |
| -projective_variety                 | lab_ascii<br>lab_tex $d$<br>coeffs | Create a projective variety of degree $d$ from an equation. By default, the coefficients of the equation are listed in the partition ordering. A different ordering can be specified. A label for the variety in ascii and in tex is required. See Section 4.5. |
| -intersection_of_zariski_open_sets  | $l \ d \ n \ C_1 \dots C_n$        | Create the intersection of the Zariski open sets given by equations $C_1, \dots, C_n$ of degree $d$ with label $l$ , see Section 4.5.                                                                                                                           |
| -projective_curve                   | $l \ r \ d \ C$                    | Create the projective curve of degree $d$ with label $l$ , with coefficient vector $C$ in $r$ variables                                                                                                                                                         |

Table 4.13: Geometric Objects (Part 2)

**Example 127**

```

Baer_PG_3.4:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define O -geometric_object P \
> > > -Baer_substructure \
> > -end \
> > -with O -do -combinatorial_object_activity -save \
> > -end

```

Let us consider BLT-sets. The following command recalls the unique BLT-set of order 5 from the database:

**Example 128**

```

BLT_database_5.0:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 5 -end \
> > -define P -projective_space -n 4 -field F -v 0 -end \
> > -define O -geometric_object P \
> > > -BLT_database 0 \
> > -end \
> > -with O -do -combinatorial_object_activity -save \
> > -end

```

Let us pull the first BLT-set of order 7 and print it to latex:

**Example 129**

```

BLT_database_7.0_print:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -define P -projective_space -n 4 -field F -v 0 -end \
> > -define Obj -geometric_object P \
> > > -BLT_database 0 \
> > -end \
> > -with Obj -do -combinatorial_object_activity -save \
> > -end \
> > -define O -orthogonal_space 0 5 F -without_group -end \
> > -define BLT_7_0 -set -file_orbiter_format BLT_7_0.txt -end \
> > -with O -do -orthogonal_space_activity \
> > > -print_points BLT_7_0 -end
> pdflatex BLT_7_0_set_report.tex
> $(OPEN) BLT_7_0_set_report.pdf

```

The output is shown below:

A set of points of size 8

The Points:

- 0 :  $P_0 = (0, 1, 0, 0, 0)$
- 1 :  $P_1 = (0, 0, 1, 0, 0)$
- 2 :  $P_{40} = (0, 1, 2, 6, 2)$
- 3 :  $P_{41} = (0, 1, 4, 3, 1)$
- 4 :  $P_{42} = (0, 1, 1, 2, 3)$
- 5 :  $P_{43} = (0, 1, 1, 5, 4)$
- 6 :  $P_{44} = (0, 1, 4, 4, 6)$
- 7 :  $P_{45} = (0, 1, 2, 1, 5)$

In Section 4.7, we have created the generalized quadrangle  $W(2)$  (the Doily). We can search for disjoint sets of size 5 in the collineation graph:

#### Example 130

```
Doily_disjoint_sets_graph_cliques_5:
> echo $(DOILY) >doily.csv
> $(ORBITER) -v 2 \
> > -define G -graph -disjoint_sets_graph \
> > > doily.csv \
> > -end \
> > -with G -do \
> > > -graph_theoretic_activity \
> > > -find_cliques \
> > > > -target_size 5 \
> > > > -output_file doily_cliques_5 \
> > > -end \
> > -end \
> > -print_symbols
```

There are exactly 6 cliques.

The Edge curve is given by the equation

$$X^4 - Y^4 - Z^4 + 2f^2Y^2Z^2 + 4fX^2YZ = 0$$

where  $f$  is a primitive element of  $\mathbb{F}_q$ . Let us pick  $q = 17$ . The next example creates the Edge curve in  $\text{PG}(2, 17)$  and saves it to file. The equation is encoded using the ordering of quartic monomials from Table 5.4.

#### Example 131

```
EDGE_CURVE_Q17_EQUATION="1, 16, 16, 0, 0, 0, 0, 0, 0, 0, 0, 1, 12, 0, 0"
```

#### Example 132

```
EDGE_CURVE_Q17_AS_POINTS="4, 7, 16, 19, 20, 23, 32, 35, 89, 100, 244, 251"
```

**Example 133**

```
FILE_Q17="orbit,curve,pts_on_curve,bitangents,go\
\n0,\"$(EDGE_CURVE_Q17_EQUATION)\",\"$(EDGE_CURVE_Q17_AS.POINTS)\",\"\",-1\
\nEND"
```

**Example 134**

```
Edge_curve_17:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 17 -end \
▷ ▷ -define R -polynomial_ring -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 4 \
▷ ▷ ▷ -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define C -geometric_object P \
▷ ▷ ▷ -projective_variety R \
▷ ▷ ▷ ▷ "Edge.q17" "Edge\q17" \
▷ ▷ ▷ ▷ $(EDGE_CURVE_Q17_EQUATION) \
▷ ▷ -end \
▷ ▷ -with C -do -combinatorial_object_activity -save \
▷ ▷ -end
```

The following command computes the line type of the Edge curve:

**Example 135**

```
Edge_curve_17.line.type:
▷ echo $(FILE_Q17) >edge_q17.csv
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 17 -end \
▷ ▷ -define R -polynomial_ring -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 4 \
▷ ▷ ▷ -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define C -geometric_object P \
▷ ▷ ▷ -projective_variety R \
▷ ▷ ▷ ▷ "Edge.q17" "Edge\q17" \
▷ ▷ ▷ ▷ $(EDGE_CURVE_Q17_EQUATION) \
▷ ▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -line_type_old \
▷ ▷ -end \
▷ ▷ -print_symbols
```

The line type is

$$(4^6, 2^{30}, 1^{132}, 0^{139})$$

This means that there are 6 4-secants, 30 2-secants, 132 tangent lines, and 139 external lines to the curve.



## Chapter 5

# Ring Theory

### 5.1 Polynomials Over Finite Fields

For  $p$  prime, the finite field  $\mathbb{F}_p$  of order  $p$  can be constructed as factorring of the integers modulo  $p$ . In this section, we will consider polynomials over  $\mathbb{F}_p$ . The ring of polynomials in one variable with coefficients in  $\mathbb{F}_p$  is denoted as  $\mathbb{F}_p[X]$ . Table 5.1 lists Orbiter finite field activities related to polynomials. For instance, the command

#### Example 136

```
poly_division:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_division "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" -end
```

computes the polynomial long division of  $A(X)$  by  $B(X)$  over  $\mathbb{F}_2$  where

$$A(X) = X^{10} + 1, \quad B(X) = X^3 + X^2 + 1.$$

The result is  $Q(X)$  and  $R(X)$  with

$$A(X) = Q(X) \cdot B(X) + R(X)$$

with

$$Q(X) = X^7 + X^6 + X^5 + X^3 + 1, \quad R(X) = X^2.$$

The coefficient lists in the arguments are from the lowest term up.

It is perhaps more convenient to create the polynomials as vectors, as in Section 2.5. The following example uses vectors named  $A$  and  $B$ . After that, the division command is called.

#### Example 137

```
poly_division2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define A -vector -field F -sparse 11 "1,0,1,10" -end \
> > -define B -vector -field F -dense "1,0,1,1" -end \
```

| Finite Field Activities Related to Polynomials |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                        | Arguments             | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| -polynomial_division                           | $A(X) \ B(X)$         | Polynomial division of $A(X)$ by $B(X)$ over $\mathbb{F}_q$ . $A(X)$ and $B(X)$ are given as coefficient list, starting from the lowest coefficient.                                                                                                                                                                                                                                                                                                                                                  |
| -extended_gcd_for_polynomials                  | $A(X) \ B(X)$         | Extended gcd for polynomials $A(X)$ and $B(X)$ over $\mathbb{F}_q$ . $A(X)$ and $B(X)$ are given as coefficient list, starting from the lowest coefficient.                                                                                                                                                                                                                                                                                                                                           |
| -polynomial_mult_mod                           | $A(X) \ B(X) \ M(X)$  | Multiply the polynomials $A(X)$ and $B(X)$ modulo $M(X)$ in $\mathbb{F}_q[X]$ .                                                                                                                                                                                                                                                                                                                                                                                                                       |
| -Berlekamp_matrix                              | $A(X)$                | Computes the rank of the Berlekamp matrix associated to the polynomial $A(X)$ over $\mathbb{F}_q$ . The polynomial $A(X)$ is irreducible over $\mathbb{F}_q$ if the Berlekamp matrix has rank $d-1$ , where $d$ is the degree of $A(X)$ . The Berlekamp matrix is $F - I$ where $F$ is the Frobenius matrix and $I$ is the identity matrix. The Frobenius matrix is the matrix of the Frobenius endomorphism with respect to the standard basis of the polynomial ring: $1, X, X^2, \dots, X^{d-1}$ . |
| -polynomial_find_roots                         | $A(X)$                | Find the roots of $A(X)$ over $\mathbb{F}_q$ .                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| -get_primitive_polynomial                      | $d$                   | Get a primitive polynomial of degree $d$ over $\mathbb{F}_q$ .                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| -get_primitive_polynomial_in_range             | $d_{\min} \ d_{\max}$ | Get a primitive polynomial of degree $d$ over $\mathbb{F}_q$ for all $d \in [d_{\min}, d_{\max}]$ .                                                                                                                                                                                                                                                                                                                                                                                                   |
| -make_table_of_irreducible_polynomials         | $d$                   | Produces a list of all irreducible polynomials of degree $d$ over $\mathbb{F}_q$ .                                                                                                                                                                                                                                                                                                                                                                                                                    |

Table 5.1: Finite Field Activities Related to Polynomials



```

> > -with F -do \
> > -finite_field_activity \
> > -polynomial_division A B -end

```

The command `-extended_gcd_for_polynomials` takes two polynomials  $A(X)$  and  $B(X)$  and computes polynomials  $U(X)$  and  $V(X)$  and  $G(X)$  such that  $G(X)$  is the greatest common divisor of  $A(X)$  and  $B(X)$  and

$$G(X) = U(X) \cdot A(X) + V(X) \cdot B(X).$$

For instance,

#### Example 138

```

poly_gcd:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -extended_gcd_for_polynomials "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" -end

```

computes

$$U(X) = X + 1, \quad V(X) = X^8 + X^5 + X^4 + X^3 + X, \quad G(X) = 1.$$

The next command computes

$$(3X^2 + 2X + 1) \cdot (5X^2 + 4X + 3) \pmod{(X^3 + 6)} \pmod{7}.$$

#### Example 139

```

poly_mult_mod1:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_mult_mod "1,2,3" "3,4,5" "6,0,0,1" -end

```

which has a result of

$$X^2 + 4X + 4.$$

The coefficients are given from the lowest to the highest term. For the opposite order, the following command computes

$$(2X^2 + X + 3) \cdot (4X^2 + 3X + 5) \pmod{(X^3 + 6)} \pmod{7}.$$

#### Example 140

```

poly_mult_mod2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -with F -do \

```

```

> > -finite_field_activity \
> > -polynomial_mult_mod "3,1,2" "5,3,4" "6,0,0,1" -end

```

The result is

$$4X^2 + X + 4.$$

The finite field  $\mathbb{F}_4$  can be defined by using polynomial arithmetic over  $\mathbb{F}_2$  modulo  $X^2 + X + 1$ . Here is a command that computes the three non-trivial products of polynomials:

#### Example 141

```

poly_mult_mod.F4:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_mult_mod "1,1" "1,1" "1,1,1" -end
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_mult_mod "0,1" "1,1" "1,1,1" -end
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_mult_mod "0,1" "0,1" "1,1,1" -end

```

It is possible to use numerical values for polynomials, using the representation in radix  $q$ . The following command computes the product of the polynomials 5 and 7 over  $\mathbb{F}_2$ :

#### Example 142

```

mult_polynomials.2.5.7:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity -mult_polynomials 5 7 -end
> pdflatex polynomial_mult.5.7.tex
> $(OPEN) polynomial_mult.5.7.pdf

```

The next command performs polynomial long division based on numerical polynomials:

#### Example 143

```

polynomial_division_ranked.2.27.13:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \

```

```

> > -with F -do \
> > -finite_field_activity \
> > > -polynomial_division_ranked 27 13 \
> > -end
> pdflatex polynomial_division_27_13.tex
> $(OPEN) polynomial_division_27_13.pdf

```

Here is a somewhat larger example for numerical arguments. We wish to multiply 999 by 997 modulo 1033. The first command performs multiplication:

#### Example 144

```

mult_polynomials_1024_999_997:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > > -mult_polynomials 999 997 \
> > -end
> pdflatex polynomial_mult_999_997.tex
> $(OPEN) polynomial_mult_999_997.pdf

```

The next command performs division with remainder:

#### Example 145

```

polynomial_division_ranked_2_349147_1033:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > > -polynomial_division_ranked 349147 1033 \
> > -end
> pdflatex polynomial_division_349147_1033.tex
> $(OPEN) polynomial_division_349147_1033.pdf

```

Orbiter allows polynomial arithmetic modulo a factor polynomial. The coefficient vector of the polynomial can be created as a vector. Here is an example which performs arithmetic modulo the CRC32 polynomial. The goal is to compute the multiplicative inverse of  $X$ . In order to do so, we use the fact that the CRC32 polynomial is irreducible, and hence the factor ring is a finite field of order  $2^{32}$ . The inverse of a polynomial can be computed by raising to the power of  $2^{32} - 2$ :

#### Example 146

```

CRC32_SPARSE="1,32,1,26,1,23,1,22,1,16,1,12,1,11,\
1,10,1,8,1,7,1,5,1,4,1,2,1,1,1,0"

```

**Example 147**

```
TWO_TO_THE_32_MINUS_2=4294967294
```

**Example 148**

```
power_mod_inverse:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
> > -define A -vector -field F -sparse 2 "1,1" -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_power_mod A $(TWO_TO_THE_32_MINUS_2) M \
> > -end
```

This command produces the polynomial

$$B(X) = X^{31} + X^{25} + X^{22} + X^{21} + X^{15} + X^{11} + X^{10} + X^9 + X^7 + X^6 + X^4 + X^3 + X + 1$$

In order to test that this polynomial really is the multiplicative inverse of  $X$  modulo CRC32, we perform the following command:

**Example 149**

```
INVERSE_SPARSE="1,31,1,25,1,22,1,21,1,15,\
1,11,1,10,1,9,1,7,1,6,1,4,1,3,1,1,1,0"
```

**Example 150**

```
mult_mod_to_get_one:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
> > -define A -vector -field F -sparse 2 "1,1" -end \
> > -define B -vector -field F -sparse 33 $(INVERSE_SPARSE) -end \
> > -with F -do \
> > -finite_field_activity \
> > > -polynomial_mult_mod A B M \
> > -end
```

The product is indeed 1.

The Berlekamp matrix can be used to test if a polynomial is irreducible over a given finite field. The polynomial is irreducible if and only if the rank of the Berlekamp matrix is  $d - 1$ , where  $d$  is the degree of the polynomial. For instance, the command

**Example 151**

```

Berlekamp_matrix_2_3:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F -dense "1,1,0,1" -end \
> > -with F -do \
> > -finite_field_activity \
> > -Berlekamp_matrix v -end
> pdflatex Berlekamp_matrix.q2.d3.tex
> $(OPEN) Berlekamp_matrix.q2.d3.pdf

```

computes the Berlekamp matrix associated with the polynomial  $X^3 + X + 1$  over  $\mathbb{F}_2$ . The matrix is

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Since the matrix has rank 2, the polynomial is irreducible.

Orbiter can compute irreducible and primitive polynomials. The following command creates a list of all irreducible polynomials of degree 3 over  $\mathbb{F}_4$ :

**Example 152**

```

irred_3_4:
> $(ORBITER) -v 6 \
> > -define F -finite_field -q 4 -end \
> > -with F -do \
> > -finite_field_activity \
> > -make_table_of_irreducible_polynomials 3 -end
> pdflatex Irred.q4.d3.tex
> $(OPEN) Irred.q4.d3.pdf

```

The output is:

There are 20 irreducible polynomials of degree 3 over the field F4:

```

0 : 1123 : 91
1 : 1031 : 77
2 : 1213 : 103
3 : 1323 : 123
4 : 1322 : 122
5 : 1222 : 106
6 : 1021 : 73
7 : 1101 : 81
8 : 1333 : 127
9 : 1232 : 110
10 : 1113 : 87
11 : 1233 : 111
12 : 1301 : 113
13 : 1003 : 67

```

```

14 : 1112 : 86
15 : 1002 : 66
16 : 1312 : 118
17 : 1011 : 69
18 : 1132 : 94
19 : 1201 : 97

```

The following command creates a primitive polynomial of degree 10 over  $\mathbb{F}_2$ :

#### Example 153

```

F2_get_primitive_poly_of_deg_10:
> $(ORBITER) -v 6 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -get_primitive_polynomial 10 -end

```

The following command creates one primitive polynomial over  $\mathbb{F}_2$  for each degree in the range  $[2, 10]$ :

#### Example 154

```

F2_get_primitive_poly_range_2_10:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -with F -do \
> > -finite_field_activity \
> > -get_primitive_polynomial_in_range 2 10 -end

```

The unix command `grep` is used to filter the output for lines containing the given pattern “//”. This yields the list

```

"7", // X^{2} + X + 1
"13", // X^{3} + X^{2} + 1
"25", // X^{4} + X^{3} + 1
"37", // X^{5} + X^{2} + 1
"97", // X^{6} + X^{5} + 1
"193", // X^{7} + X^{6} + 1
"285", // X^{8} + X^{4} + X^{3} + X^{2} + 1
"529", // X^{9} + X^{4} + 1
"1033", // X^{10} + X^{3} + 1

```

Primitive polynomials over the base field  $\mathbb{F}_s$  are converted into integers, using the base- $s$  representation of integers. For instance, the polynomial  $X^2 + X + 1$  is read as binary string 111, which in turn translates to the integer 7 (we use  $s = 2$ ).

| Creating Multivariate Polynomial Rings |                        |                                                  |
|----------------------------------------|------------------------|--------------------------------------------------|
| Command                                | Arguments              | Purpose                                          |
| -field                                 | label                  | Specify the field of coefficients.               |
| -homogeneous_of_degree                 | $d$                    | Specify the degree $d$ of polynomials.           |
| -number_of_variables                   | $n$                    | Specify the number $n$ of variables.             |
| -monomial_ordering_partition           |                        | Set monomial ordering to partition ordering.     |
| -monomial_ordering_lex                 |                        | Set monomial ordering to lexicographic ordering. |
| -variables                             | label-txt<br>label-tex | Specify variable labels in ascii and in latex.   |

Table 5.2: Creating Multivariate Polynomial Rings

| Multivariate Polynomial Ring Activities |                                     |                                                                            |
|-----------------------------------------|-------------------------------------|----------------------------------------------------------------------------|
| Command                                 | Arguments                           | Purpose                                                                    |
| -cheat_sheet                            |                                     | Create a report about the polynomial ring.                                 |
| -ideal                                  | label-txt<br>label-tex<br>label-set | Compute the ideal of the set of points.                                    |
| -apply_transformation                   | eqn elt                             | Apply the transformation given by the group element to the given equation. |
| -set_variable_names                     | txt tex                             | Choose new variable names in text and tex format.                          |
| -print_equation                         | equation                            | Print the given equation.                                                  |

Table 5.3: Multivariate Polynomial Ring Activities

## 5.2 Multivariate Polynomial Rings

Orbiter can work with multivariate polynomial rings. Table 5.2 lists the commands for creating a multivariate polynomial ring.

Table 5.3 lists the activities for a multivariate polynomial ring.

There are two orderings of the monomials which can be chosen:

1. The partition ordering is grouping terms according to the partition that results from the degrees of the variables first, and then applies the lexicographic ordering as a tie breaker.
2. The lexicographic ordering orders the monomials lexicographically.

By default, the partition ordering is used. Table 5.4 shows the monomials in the partition ordering for degrees 1, 2, 3 and 4 in a plane.

Table 5.5 shows the partition ordering monomials of degree at most 3 in  $\text{PG}(3, q)$ .

The following example shows how a Cremona map can be defined. At first, we define 4 polynomials as makefile variables. After that, we invoke Orbiter to create a polynomial ring and to evaluate the map.

| Monomial Ordering of Partition Type in the Plane |               |           |
|--------------------------------------------------|---------------|-----------|
| Degrees 1 and 2                                  |               |           |
| $h$                                              | mon           | vector    |
| 0                                                | $X_0$         | (1, 0, 0) |
| 1                                                | $X_1$         | (0, 1, 0) |
| 2                                                | $X_2$         | (0, 0, 1) |
| $h$                                              | mon           | vector    |
| 0                                                | $X_0^2$       | (2, 0, 0) |
| 1                                                | $X_1^2$       | (0, 2, 0) |
| 2                                                | $X_2^2$       | (0, 0, 2) |
| 3                                                | $X_0X_1$      | (1, 1, 0) |
| 4                                                | $X_0X_2$      | (1, 0, 1) |
| 5                                                | $X_1X_2$      | (0, 1, 1) |
| Degree 3                                         |               |           |
| $h$                                              | mon           | vector    |
| 0                                                | $X_0^3$       | (3, 0, 0) |
| 1                                                | $X_1^3$       | (0, 3, 0) |
| 2                                                | $X_2^3$       | (0, 0, 3) |
| 3                                                | $X_0^2X_1$    | (2, 1, 0) |
| 4                                                | $X_0^2X_2$    | (2, 0, 1) |
| 5                                                | $X_0X_1^2$    | (1, 2, 0) |
| 6                                                | $X_1^2X_2$    | (0, 2, 1) |
| 7                                                | $X_0X_2^2$    | (1, 0, 2) |
| 8                                                | $X_1X_2^2$    | (0, 1, 2) |
| 9                                                | $X_0X_1X_2$   | (1, 1, 1) |
| Degree 4                                         |               |           |
| $h$                                              | mon           | vector    |
| 0                                                | $X_0^4$       | (4, 0, 0) |
| 1                                                | $X_1^4$       | (0, 4, 0) |
| 2                                                | $X_2^4$       | (0, 0, 4) |
| 3                                                | $X_0^3X_1$    | (3, 1, 0) |
| 4                                                | $X_0^3X_2$    | (3, 0, 1) |
| 5                                                | $X_0X_1^3$    | (1, 3, 0) |
| 6                                                | $X_1^3X_2$    | (0, 3, 1) |
| 7                                                | $X_0X_2^3$    | (1, 0, 3) |
| 8                                                | $X_1X_2^3$    | (0, 1, 3) |
| 9                                                | $X_0^2X_1^2$  | (2, 2, 0) |
| 10                                               | $X_0^2X_2^2$  | (2, 0, 2) |
| 11                                               | $X_1^2X_2^2$  | (0, 2, 2) |
| 12                                               | $X_0^2X_1X_2$ | (2, 1, 1) |
| 13                                               | $X_0X_1^2X_2$ | (1, 2, 1) |
| 14                                               | $X_0X_1X_2^2$ | (1, 1, 2) |

Table 5.4: Monomial Ordering of Partition Type in the Plane



| Monomial Ordering of Partition Type in Three-Space |             |              |
|----------------------------------------------------|-------------|--------------|
| Degree 1                                           |             |              |
| $h$                                                | mon         | vector       |
| 0                                                  | $X_0$       | (1, 0, 0, 0) |
| 1                                                  | $X_1$       | (0, 1, 0, 0) |
| 2                                                  | $X_2$       | (0, 0, 1, 0) |
| 3                                                  | $X_3$       | (0, 0, 0, 1) |
| Degree 2                                           |             |              |
| $h$                                                | mon         | vector       |
| 0                                                  | $X_0^2$     | (2, 0, 0, 0) |
| 1                                                  | $X_1^2$     | (0, 2, 0, 0) |
| 2                                                  | $X_2^2$     | (0, 0, 2, 0) |
| 3                                                  | $X_3^2$     | (0, 0, 0, 2) |
| 4                                                  | $X_0X_1$    | (1, 1, 0, 0) |
| 5                                                  | $X_0X_2$    | (1, 0, 1, 0) |
| 6                                                  | $X_0X_3$    | (1, 0, 0, 1) |
| 7                                                  | $X_1X_2$    | (0, 1, 1, 0) |
| 8                                                  | $X_1X_3$    | (0, 1, 0, 1) |
| 9                                                  | $X_2X_3$    | (0, 0, 1, 1) |
| Degree 3                                           |             |              |
| $h$                                                | mon         | vector       |
| 0                                                  | $X_0^3$     | (3, 0, 0, 0) |
| 1                                                  | $X_1^3$     | (0, 3, 0, 0) |
| 2                                                  | $X_2^3$     | (0, 0, 3, 0) |
| 3                                                  | $X_3^3$     | (0, 0, 0, 3) |
| 4                                                  | $X_0^2X_1$  | (2, 1, 0, 0) |
| 5                                                  | $X_0^2X_2$  | (2, 0, 1, 0) |
| 6                                                  | $X_0^2X_3$  | (2, 0, 0, 1) |
| 7                                                  | $X_0X_1^2$  | (1, 2, 0, 0) |
| 8                                                  | $X_1^2X_2$  | (0, 2, 1, 0) |
| 9                                                  | $X_1^2X_3$  | (0, 2, 0, 1) |
| 10                                                 | $X_0X_2^2$  | (1, 0, 2, 0) |
| 11                                                 | $X_1X_2^2$  | (0, 1, 2, 0) |
| 12                                                 | $X_2^2X_3$  | (0, 0, 2, 1) |
| 13                                                 | $X_0X_3^2$  | (1, 0, 0, 2) |
| 14                                                 | $X_1X_3^2$  | (0, 1, 0, 2) |
| 15                                                 | $X_2X_3^2$  | (0, 0, 1, 2) |
| 16                                                 | $X_0X_1X_2$ | (1, 1, 1, 0) |
| 17                                                 | $X_0X_1X_3$ | (1, 1, 0, 1) |
| 18                                                 | $X_0X_2X_3$ | (1, 0, 1, 1) |
| 19                                                 | $X_1X_2X_3$ | (0, 1, 1, 1) |

Table 5.5: Monomial Ordering of Partition Type in Three-Space

**Example 155**

```
CREMONA_MAP_Y0="3*y0^5*y2+4*y0^3*y1^2*y2\
+2*y0^3*y2^3+y0*y1^4*y2\
+6*y0*y1^2*y2^3+9*y0*y2^5"
```

**Example 156**

```
CREMONA_MAP_Y1="y0^5*y1+5*y0^3*y1^3\
+12*y0^3*y1*y2^2+3*y0*y1^5\
+5*y0*y1^3*y2^2+y0*y1*y2^4"
```

**Example 157**

```
CREMONA_MAP_Y2="10*y0^6+11*y0^4*y1^2\
+11*y0^4*y2^2+4*y0^2*y1^4\
+9*y0^2*y1^2*y2^2+4*y0^2*y2^4"
```

**Example 158**

```
CREMONA_MAP_Y3="0"
```

**Example 159**

```
Cremona_map:
▷ $(ORBITER) -v 6 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 6 \
▷ ▷ ▷ -monomial_ordering_lex \
▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
▷ ▷ ▷ -end \
▷ ▷ -define Y0 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(CREMONA_MAP_Y0) \
▷ ▷ ▷ -end \
▷ ▷ -define Y1 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(CREMONA_MAP_Y1) \
▷ ▷ ▷ -end \
▷ ▷ -define Y2 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(CREMONA_MAP_Y2) \
▷ ▷ ▷ -end \
```

```

> > -define Cremona -symbolic_object \
> > > -field F \
> > > -text "Y0,Y1,Y2" \
> > > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -map R Cremona "" \
> > -end

```

Next, we will consider ideals. As an application, we will classify arcs in a projective plane and see which conics we get. The next command classifies the  $(5, 2)$ -arcs in  $\text{PG}(2, 11)$ :

#### Example 160

```

arcs_5.2.q11:
> $(ORBITER) -v 4 \
> > -define F -finite_field -q 11 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -define Control -poset_classification_control \
> > > -problem_label arcs_5.2.q11 \
> > > -W -depth 5 \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -classify_arcs \
> > > > -control Control \
> > > > -target_size 5 \
> > > > -d 2 \
> > > -end \
> > -end
> #pdflatex arcs_5.2.q11.poset.tex
> #$(OPEN) arcs_5.2.q11.poset.pdf

```

It finds exactly two isomorphism types of arcs. The representative sets are

$$\{0, 1, 2, 3, 37\}, \quad \{0, 1, 2, 3, 49\}.$$

They are stored in the file `arcs_5.2.q11_lv1_5`. Let us now create the ideal in the quadratic component of the polynomial ring in three variables over  $\mathbb{F}_{11}$ :

#### Example 161

```

arcs_5.2.q11_ideal:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 11 -end \
> > -define R -polynomial_ring \
> > > -field F \
> > > -number_of_variables 3 \
> > > -homogeneous_of_degree 2 \
> > > -monomial_ordering_lex \

```

```

> > > -variables "x0,x1,x2" "x_0,x_1,x_2" \
> > > -end \
> > > -define C -combinatorial_object \
> > > -label arcs_5.2_q11_lv1_5 arcs\5\2\q11\lv1\5 \
> > > -file_of_points arcs_5.2_q11_lv1_5 \
> > > -end \
> > > -with C -do \
> > > -combinatorial_object_activity \
> > > -ideal R \
> > > -end

```

The ideals are generated by

$$7*x_0*x_1 + 5*x_0*x_2 + 10*x_1*x_2$$

and

$$4*x_0*x_1 + 8*x_0*x_2 + 10*x_1*x_2,$$

respectively.

Let us consider a smooth cubic surface with 9 lines and 4 Eckardt points. Suppose we have the set of points and we wish to determine the equation of the object. To do so, we first define the object from the given set of points.

#### Example 162

```

PTS_OF_SURFACE_ORBIT211_Q3_L9_E4=" \
0,1,2,5,7,8,10,14,9,12, \
15,3,16,37,31,34,20,19,17,32,36,33"

```

Then, we create a ring and compute the ideal:

#### Example 163

```

surface_9lines_4E_ideal:
> $(ORBITER) -v 2 \
> > > -define Pts -vector -dense \
> > > $(PTS_OF_SURFACE_ORBIT211_Q3_L9_E4) \
> > > -end \
> > > -define F -finite_field -q 3 -end \
> > > -define R -polynomial_ring \
> > > -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 3 \
> > > -monomial_ordering_lex \
> > > -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
> > > -end \
> > > -with R -do \
> > > -ring_theoretic_activity \
> > > -ideal "surf_eqn" "surf\_eqn" Pts \
> > > -end

```

We find a two-dimensional ideal. Generators are:

$$x_0x_0x_1 + 2x_0x_1x_1 + 2x_0x_1x_3 \quad \text{and} \quad 2x_2x_2x_3 + 2x_2x_3x_3.$$

Let us take the sum of the two polynomials and create the cubic surface:

#### Example 164

```
SURFACE_F_9="x0*x0*x1 - x0*x1*x1 -x0*x1*x3 -x2*x2*x3 - x2*x3*x3"
```

#### Example 165

```
F_9.q7:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 7 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define R -polynomial_ring \
> > > -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 3 \
> > > -monomial_ordering_partition \
> > > -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
> > > -end \
> > -define F_9 -cubic_surface -space P \
> > > -by_equation R "F_9" \
> > > "\DF_9\D" "x0,x1,x2,x3" \
> > > $(SURFACE_F_9) \
> > > "" \
> > > "\Dno parameters\D" "" \
> > -end \
> > -with F_9 -do \
> > -cubic_surface_activity \
> > > -report \
> > -end
> pdflatex surface_equation_F_9.q7.report.tex
> $(OPEN) surface_equation_F_9.q7.report.pdf
```

In the next example, we wish to explore the relationship between conics and  $(5,2)$ -arcs. We consider the plane  $\text{PG}(2, 11)$ . Instead of classification, we will try random generation this time. Since there are 133 points, we create a number of 5-subsets of a set of size 133. In this case, we create 20 sets at random:

#### Example 166

```
random_k_subsets_PG_2_11:
> $(ORBITER) -v 4 \
> > -create_random_k_subsets 133 5 20
```

The sets are stored in the file `random_k_subsets_n133_k5_nb20.csv`. Now, let's compute the line type of these subsets, to see which ones are arcs:

**Example 167**

```

line_type_in_PG_2_11:
▷ $(ORBITER) -v 3 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define F -finite_field -q 11 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label random_sets random_sets \
▷ ▷ ▷ -file_of_points random_k_subsets.n133.k5.nb20.csv \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ line_type_old \
▷ ▷ -end

```

It turns out that the second set is an arc. It is the set  $\{3, 33, 40, 83, 102\}$ . We create the conic through these 5 points:

**Example 168**

```

random_arc_5_2_q11_ideal:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 11 -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 2 \
▷ ▷ ▷ -monomial_ordering_lex \
▷ ▷ ▷ -variables "x0,x1,x2" "x_0,x_1,x_2" \
▷ ▷ ▷ -end \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label random_arc random\_arc \
▷ ▷ ▷ -set_of_points "3,33,40,83,102" \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -ideal R \
▷ ▷ -end

```

The ideal is generated by

$$10x_0x_1 + 3x_0x_2 + 8x_1x_2 + 2x_1^2 + 10x_2^2.$$

The conic contains the following 12 points:

$$\{3, 15, 19, 33, 40, 42, 46, 50, 83, 88, 102, 108\}.$$

The next command creates the Endrass surface over  $\mathbb{F}_7$ . The surface is defined as a makefile variable in sparse form.

**Example 169**

```

ENDRASS_SPARSE="\
6,0,4,4,2,7,5,9,6,20,6,23,1,25,3,30,1,32,3,34,4,56,6,59,1,61,6,66, \
2,68,6,70,3,77,2,79,6,83,6,120,2,123,5,125,3,130,1,132,3,134,3,141, \
2,143,6,147,3,156"

```

**Example 170**

```

Endrass_F7.txt:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 7 -end \
> > -define R -polynomial_ring -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 8 \
> > > -end \
> > -define eqn -vector -field F -sparse 165 \
> > > $(ENDRASS_SPARSE) -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define Endrass_F7 -geometric_object P \
> > > -projective_variety R \
> > > > "Endrass_F7" \
> > > > "Endrass\_F7" \
> > > > eqn \
> > -end \
> > -with Endrass_F7 -do \
> > -combinatorial_object_activity -save \
> > -end

```

Suppose we want to create the monomials of degree 8 in 4 variables. We use an diophantine system to do so. The following command creates the system and solves it. After that, it applies the unix sort command to sort the monomials:

**Example 171**

```

octic.prepare:
> $(ORBITER) -v 4 \
> > -define A -vector -format 1 -dense "1,1,1,1" -end \
> > -define D -diophant \
> > > -label octic_monomials \
> > > -coefficient_matrix A \
> > > -RHS "mult=1,EQ=8" \
> > > -x_min_global 0 -x_max_global 8 \
> > -end \
> > -with D -do \
> > > -diophant_activity -solve_mckay \
> > -end
> sort -r octic_monomials_sol.csv >octic_monomials_sorted.txt

```

There are 165 monomials. They are listed in the file `octic_monomials_sorted.txt`.

The following command creates the permutation polynomial associated with the map

$$\sigma : \mathbb{F}_5 \rightarrow \mathbb{F}_5, \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 3 & 0 & 4 & 1 & 2 \end{bmatrix}$$

#### Example 172

```
permutation_polynomial_F5:
▷ $(ORBITER) -v 6 \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 2 \
▷ ▷ ▷ -homogeneous_of_degree 4 \
▷ ▷ ▷ -monomial_ordering_lex \
▷ ▷ ▷ -variables "x,y" "x,y" \
▷ ▷ ▷ -end \
▷ ▷ -define Y0 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "3*(1-x^4)+4*(1-(x-2)^4)+1*(1-(x-3)^4)+2*(1-(x-4)^4)" \
▷ ▷ ▷ -end \
▷ ▷ -define Y -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "Y0" \
▷ ▷ ▷ -end \
▷ ▷ -with P -do \
▷ ▷ -projective_space_activity \
▷ ▷ ▷ -affine_map R Y "" \
▷ ▷ -end \
▷ ▷ -define Yx -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -expand Y \
▷ ▷ ▷ -end \
```

The code utilizes the basis functions

$$\delta_{x,i} = 1 - (x - i)^{q-1}$$

of  $\mathbb{F}_5$ , which are one if  $x = i$  and zero otherwise. Because Orbiter offers homogeneous polynomials only, a polynomial ring in two-variables is utilized. The polynomials are homogeneous of degree  $q - 1 = 4$ . We can write

$$\begin{aligned} \sigma &= \sum_{i=0}^4 \sigma(i) \delta_{x,i} \\ &= \sigma(0) \delta_{x,0} + \sigma(1) \delta_{x,1} + \sigma(2) \delta_{x,2} + \sigma(3) \delta_{x,3} + \sigma(4) \delta_{x,4} \\ &= 3 \delta_{x,0} + 0 \delta_{x,1} + 4 \delta_{x,2} + 1 \delta_{x,3} + 2 \delta_{x,4}, \\ &= 3(1 - x^4) + 4(1 - (x - 2)^4) + (1 - (x - 3)^4) + 2(1 - (x - 4)^4), \end{aligned}$$

because

$$\sigma(0) = 3, \sigma(1) = 0, \sigma(2) = 4, \sigma(3) = 1, \sigma(4) = 2.$$



After expansion, the permutation polynomial associated with  $\sigma$  turns out to be

$$3 + x^3 + (3x^2) + (3x).$$

| Mappings Between Projective Spaces |           |                                                                                      |
|------------------------------------|-----------|--------------------------------------------------------------------------------------|
| Command                            | Arguments | Purpose                                                                              |
| -domain                            | D         | Set the domain to D (a projective space).                                            |
| -codomain                          | C         | Set the codomain to C (a projective space).                                          |
| -ring                              | R         | Set the coordinate ring for D.                                                       |
| -formula                           | f         | Use the symbolic object f to define the mapping.                                     |
| -substitute                        | s         | Substitute s into the equation before doing the mapping.                             |
| -affine                            |           | Treat the domain as an affine space.                                                 |
| -object_in_codomain                | S         | Set the codomain to the cubic surface object S and map to the points on the surface. |

Table 5.6: Mappings Between Projective Spaces

### 5.3 Algebraic Geometry

Table 5.6 lists Orbiter commands for mappings between projective spaces. The first example creates the Veronese variety of  $PG(1, 7)$  in  $PG(3, 7)$ :

#### Example 173

```

Veronese_1_3_q7:
▷ $(ORBITER) -v 5 \
▷ ▷ -define Fq -finite_field -q 7 -end \
▷ ▷ -define C -symbolic_object \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -managed_variables "y0,y1" \
▷ ▷ ▷ -text "y1^3,y0*y1^2,y0^2*y1,y0^3" \
▷ ▷ -end \
▷ ▷ -define P1 -projective_space -n 1 -field Fq -v 0 -end \
▷ ▷ -define P3 -projective_space -n 3 -field Fq -v 0 -end \
▷ ▷ -define R2 -polynomial_ring \
▷ ▷ ▷ -field Fq \
▷ ▷ ▷ -number_of_variables 2 \
▷ ▷ ▷ -homogeneous_of_degree 3 \
▷ ▷ ▷ -monomial_ordering_partition \
▷ ▷ ▷ -variables "y0,y1" "y_0,y_1" \
▷ ▷ ▷ -end \
▷ ▷ -define map -mapping \
▷ ▷ ▷ -domain P1 \
▷ ▷ ▷ -codomain P3 \
▷ ▷ ▷ -ring R2 \
▷ ▷ ▷ -formula C \
▷ ▷ ▷ -substitute "" \
▷ ▷ -end

```

The image is written as a set of points in Orbiter ranks:

$$(3, 0, 4, 268, 336, 184, 224, 364)$$

For a deeper discussion of the combinatorial and group-theoretic properties of this object, see Section 16.3.

Our next example is the Hermitian curve, with equation

$$X^4 + Y^4 + Z^4$$

in  $\text{PG}(2, 9)$ . Orbiter can compute the variety over a specific field, as well as some of its properties, like automorphism group and tactical decomposition. We first need a makefile variable which holds the equation in algebraic form:

#### Example 174

```
FILE_HERMITIAN_CURVE="Line,equation\n0,\"X^4+Y^4+Z^4\"\\nEND\\n\\n"
```

The following command copies the makefile variable into a file and then performs the relevant Orbiter computations to create the curve and to investigate it.

#### Example 175

```
variety_hermitian_curve:
▷ echo $(FILE_HERMITIAN_CURVE) >input.csv
▷ $(ORBITER) -v 8 \
▷ ▷ -define F -finite_field -q 9 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 4 \
▷ ▷ ▷ -monomial_ordering_partition \
▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
▷ ▷ -end \
▷ ▷ -define O -orbits -classification_by_canonical_form \
▷ ▷ ▷ -space P \
▷ ▷ ▷ -ring R \
▷ ▷ ▷ -input_fname_mask input.csv \
▷ ▷ ▷ -nb_files 1 \
▷ ▷ ▷ -output_fname hermitian_curve_q9 \
▷ ▷ ▷ -label_equation "equation" \
▷ ▷ ▷ -algorithm_nauty \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ -with O -do -orbits_activity \
▷ ▷ -report \
▷ ▷ -report_options \
▷ ▷ ▷ -fname hermitian_curve_q9 \
▷ ▷ -end \
▷ -end
▷ pdflatex hermitian_curve_q9_orbits.tex
▷ $(OPEN) hermitian_curve_q9_orbits.pdf
```

The command produces a latex report, shown below:

Classification

$q = 9$

Number of isomorphism classes: 1

Automorphism group order statistic: 12096

The isomorphism classes are:

Isomorphism class 0 / 1 is input 0:

Automorphism group order 12096

Number of points 28

Equation  $X^4+Y^4+Z^4$

Equation ( 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 )

Points:

0: 3 = (1, 1, 1), 1: 6 = (3, 1, 0), 2: 7 = (4, 1, 0), 3: 9 = (6, 1, 0), 4: 11 = (8, 1, 0), 5: 14 = (3, 0, 1),  
6: 15 = (4, 0, 1), 7: 17 = (6, 0, 1), 8: 19 = (8, 0, 1), 9: 21 = (2, 1, 1), 10: 24 = (5, 1, 1), 11:  
26 = (7, 1, 1), 12: 29 = (1, 2, 1), 13: 30 = (2, 2, 1), 14: 33 = (5, 2, 1), 15: 35 = (7, 2, 1), 16:  
37 = (0, 3, 1), 17: 46 = (0, 4, 1), 18: 56 = (1, 5, 1), 19: 57 = (2, 5, 1), 20: 60 = (5, 5, 1), 21:  
62 = (7, 5, 1), 22: 64 = (0, 6, 1), 23: 74 = (1, 7, 1), 24: 75 = (2, 7, 1), 25: 78 = (5, 7, 1), 26:  
80 = (7, 7, 1), 27: 82 = (0, 8, 1)

Number of equations with the same set of points 1

Automorphism group:

Strong generators for a group of order 12096:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 7 & 0 & 0 \\ 0 & 7 & 0 \\ 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}_0, \begin{bmatrix} 8 & 0 & 0 \\ 0 & 5 & 7 \\ 0 & 1 & 1 \end{bmatrix}_1, \begin{bmatrix} 6 & 4 & 0 \\ 4 & 6 & 0 \\ 0 & 0 & 1 \end{bmatrix}_1,$$

$$, \begin{bmatrix} 2 & 8 & 2 \\ 1 & 5 & 3 \\ 3 & 5 & 1 \end{bmatrix}_1$$

1,0,0,0,1,0,0,0,1,1,

1,0,0,0,1,0,0,0,5,1,

1,0,0,0,5,0,0,0,1,1,

1,0,0,0,0,1,0,1,0,0,

1,0,0,0,8,4,0,6,6,1,

1,7,0,7,1,0,0,0,8,1,

1,4,1,2,7,6,6,7,2,1,

The TDO decomposition is

| $\downarrow$ | $63_1$ | $28_3$ |
|--------------|--------|--------|
| $28_0$       | 4      | 1      |
| $63_2$       | 6      | 9      |

| $\rightarrow$ | $63_1$ | $28_3$ |
|---------------|--------|--------|
| $28_0$        | 9      | 1      |
| $63_2$        | 6      | 4      |

Point classes:

Set 0 has size 28 :  $\{ 3, 6, 7, 9, 11, 14, 15, 17, 19, 21, 24, 26, 29, 30, 33, 35, 37, 46, 56, 57, 60, 62, 64, 74, 75, 78, 80, 82 \}$

Set 1 has size 63 :  $\{ 0, 1, 2, 4, 5, 8, 10, 12, 13, 16, 18, 20, 22, 23, 25, 27, 28, 31, 32, 34, 36, 38, 39, 40, 41, 42, 43, 44, 45, 47, 48, 49, 50, 51, 52, 53, 54, 55, 58, 59, 61, 63, 65, 66, 67, 68, 69, 70, 71, 72, 73, 76, 77, 79, 81, 83, 84, 85, 86, 87, 88, 89, 90 \}$

Block classes:

Set 0 has size 63 :  $\{ 0, 1, 2, 5, 7, 9, 10, 13, 14, 16, 18, 19, 20, 23, 24, 26, 28, 29, 31, 32, 33, 34, 35, 36, 37, 38, 41, 42, 43, 44, 45, 46, 47, 48, 50, 53, 54, 56, 58, 59, 61, 62, 63, 64, 65, 66, 67, 68, 70, 73, 74, 76, 78, 79, 81, 82, 83, 84, 85, 86, 87, 88, 90 \}$

Set 1 has size 28 :  $\{ 3, 4, 6, 8, 11, 12, 15, 17, 21, 22, 25, 27, 30, 39, 40, 49, 51, 52, 55, 57, 60, 69, 71, 72, 75, 77, 80, 89 \}$

The TDA decomposition is

| $\downarrow$ | $63_1$ | $28_3$ |
|--------------|--------|--------|
| $28_0$       | 4      | 1      |
| $63_2$       | 6      | 9      |

| $\rightarrow$ | $63_1$ | $28_3$ |
|---------------|--------|--------|
| $28_0$        | 9      | 1      |
| $63_2$        | 6      | 4      |

Point classes:

Set 0 has size 28 :  $\{ 3, 6, 7, 9, 11, 14, 15, 17, 19, 21, 24, 26, 29, 30, 33, 35, 37, 46, 56, 57, 60, 62, 64, 74, 75, 78, 80, 82 \}$

Set 1 has size 63 :  $\{ 0, 1, 2, 4, 5, 8, 10, 12, 13, 16, 18, 20, 22, 23, 25, 27, 28, 31, 32, 34, 36, 38, 39, 40, 41, 42, 43, 44, 45, 47, 48, 49, 50, 51, 52, 53, 54, 55, 58, 59, 61, 63, 65, 66, 67, 68, 69, 70, 71, 72, 73, 76, 77, 79, 81, 83, 84, 85, 86, 87, 88, 89, 90 \}$

Block classes:

Set 0 has size 63 :  $\{ 0, 1, 2, 5, 7, 9, 10, 13, 14, 16, 18, 19, 20, 23, 24, 26, 28, 29, 31, 32, 33, 34, 35, 36, 37, 38, 41, 42, 43, 44, 45, 46, 47, 48, 50, 53, 54, 56, 58, 59, 61, 62, 63, 64, 65, 66, 67, 68, 70, 73, 74, 76, 78, 79, 81, 82, 83, 84, 85, 86, 87, 88, 90 \}$

Set 1 has size 28 :  $\{ 3, 4, 6, 8, 11, 12, 15, 17, 21, 22, 25, 27, 30, 39, 40, 49, 51, 52, 55, 57, 60, 69, 71, 72, 75, 77, 80, 89 \}$



## Chapter 6

# Group Theory

### 6.1 Generic Permutation Groups

Orbiter distinguishes two types of permutation groups: generic permutation groups and matrix groups. Both groups come with a permutation action. The difference is how a group element is stored. In a generic permutation group, a group element is stored as a permutation of the given degree. For a matrix group, a group element is stored as a matrix. Matrix group representations are often more efficient than the corresponding generic representation as a permutation group. However, not every group has a matrix group representation of reasonably small dimension.

In this section, we will talk about generic permutation groups in Orbiter. In Section 6.2, we will discuss matrix groups over finite fields.

In order to establish the permutation representation of a group, the technique of indexing is used. Indexing sets up a fixed bijection between the permutation domain (the set we act on) and the integer interval  $[0, n - 1]$  for some  $n$ . The integer associated to an element in the permutation domain is called the rank. Conversely, given an integer in  $[0, n - 1]$ , the element in the permutation domain associated with it is obtained by the unrank function. The process of converting integers to elements of the permutation domain and vice-versa is indexing. We have seen indexing for projective points in Section 4.1.

The enumerators for projective points from Section 4.1 are used to realize the permutation domain. This enumerator relies on an enumerator for finite fields, as discussed in Sections 3.1 and 3.2. For extension fields, the enumerator for finite fields in turn depends on the choice of the irreducible polynomial which is used to create the field. For affine groups, a different enumerator is used to describe the permutation domain. This enumerator uses the base- $q$  representation of integers, which associates a vector over  $\mathbb{F}_q$  of length  $n$  with an integer in  $[0, q^n - 1]$ .

Permutation groups and matrix groups can be represented on a computer using the technique of stabilizer chains, or Sims chains (cf. [36, 62]). The stabilizer chain is defined with respect to a sequence of points in the permutation domain called a base. A set of generators which allows to generate each group along the chain is called a strong generating set. Many algorithms for permutation groups rely on knowing a base and strong generating set. In Orbiter, permutation groups can be created from a base and strong generating set. Many types of groups come with their own built-in base and strong generating set. On the other hand, it is also possible to create groups from generating sets which are either not strong or for which a base is not known. For efficiency purposes, small basic orbits are desired.

Table 6.1 lists Orbiter commands to create a generic permutation group.

Let us start with a cyclic group. The following command creates a cyclic group of order 6:

| Creating a Generic Permutation Group |                                         |                                                                                                                                                                                                                   |
|--------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                              | Arguments                               | Purpose                                                                                                                                                                                                           |
| <code>-symmetric_group</code>        | $n$                                     | Symmetric group of degree $n$ and order $n!$ .                                                                                                                                                                    |
| <code>-cyclic_group</code>           | $n$                                     | Cyclic group of degree $n$ and order $n$ .                                                                                                                                                                        |
| <code>-identity_group</code>         | $n$                                     | Identity group of degree $n$ and order 1.                                                                                                                                                                         |
| <code>-dihedral_group</code>         | $n$                                     | Dihedral group of degree $n$ and order $2n$ .                                                                                                                                                                     |
| <code>-bsgs</code>                   | lab1 lab2 $n$<br>order base $k$<br>gens | Group with a given base and a given set of generators. Here, lab1 and lab2 are labels in ascii and latex, respectively, $k$ is the number of generators, gens is the generators as an Orbiter vector of integers. |
| <code>-subgroup_by_generators</code> | lab order $k$<br>gens                   | Subgroup given by generators. Here, $k$ is the number of generators, gens is an Orbiter vector of integers.                                                                                                       |

Table 6.1: Creating a Generic Permutation Group

**Example 176**

```

Cyclic_6:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
▷ ▷ -with G -do \
▷ ▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex C_6_report.tex
▷ $(OPEN) C_6_report.pdf

```

The following command

**Example 177**

```

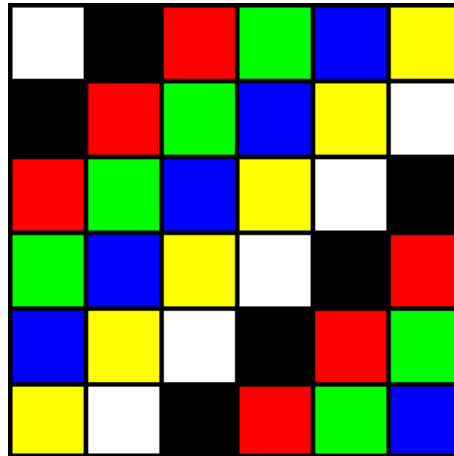
Cyclic_6_group_table:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
▷ ▷ -with G -do \
▷ ▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -export_group_table \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file C_6_group_table.csv \
▷ ▷ ▷ -box_width 50 -bit_depth 24 \
▷ ▷ ▷ -partition 3 all_one_r all_one_c \
▷ ▷ -end

```



```
▷ $(OPEN) C_6_group_table_draw.bmp
```

produces a graphical representation of the group table of the cyclic group  $C_6$ , shown below



Next, let us consider the symmetric group  $\text{Sym}(n)$ . The following command does so:

#### Example 178

```
Symmetric_3:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex Sym_3_report.tex
▷ $(OPEN) Sym_3_report.pdf
```

The report is shown below:

#### Stabilizer chain

| Level | Base pt | Orbit length | Subgroup order |
|-------|---------|--------------|----------------|
| 0     | 0       | 3            | 6              |
| 1     | 1       | 2            | 2              |

**Basic Orbit 0**

0

1

2

Basic orbit 0 has size 3  
0, 1, 2

**Basic Orbit 1**

1

2

Basic orbit 1 has size 2  
1, 2

The following command

**Example 179**

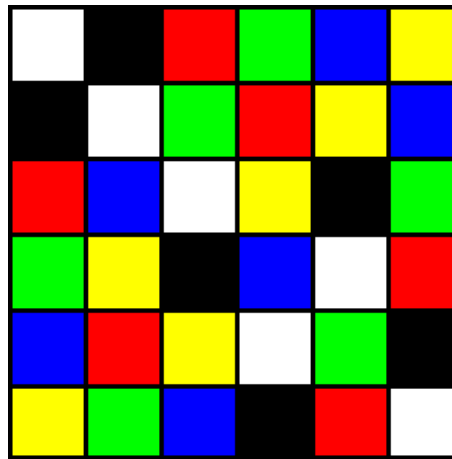
```
Symmetric_3.group_table:
> $(ORBITER) -v 3 \
> > -define G -permutation_group -symmetric_group 3 -end \
> > -with G -do \
> > -group.theoretic.activity \
> > > -export_group_table \
> > -end
> $(ORBITER) -v 2 \
> > -define all_one_r -vector -repeat 1 6 -end \
> > -define all_one_c -vector -repeat 1 6 -end \
> > -draw_matrix \
```

```

> > > -input_csv_file Sym_3_group_table.csv \
> > > -box_width 50 -bit_depth 24 \
> > > -partition 3 all_one_r all_one_c \
> > -end
> $(OPEN) Sym_3_group_table_draw.bmp

```

produces a graphical representation of the group table of the symmetric group  $\text{Sym}(3)$ , shown below



The next command

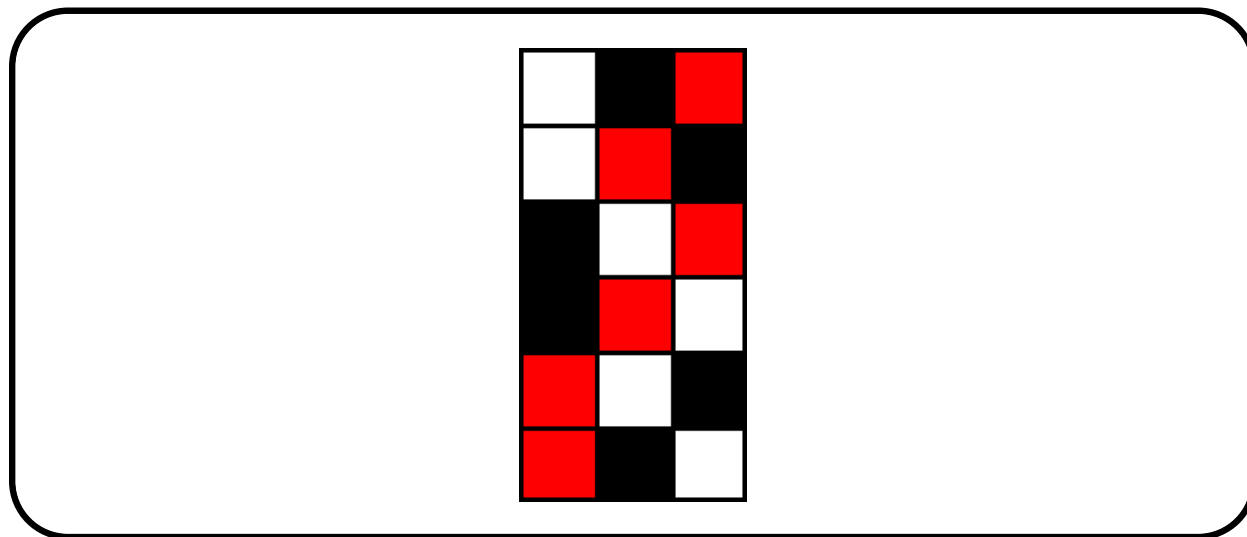
#### Example 180

```

Symmetric_3_elements:
> $(ORBITER) -v 3 \
> > -define G -permutation_group -symmetric_group 3 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -save_elements_csv "Symmetric3_elts.csv" \
> > -end
> $(ORBITER) -v 2 \
> > -define Sym3_elts -vector -load_csv_data_column \
> > > Symmetric3_elts.csv 1 -end \
> > -save_matrix_csv Sym3_elts
> $(ORBITER) -v 2 \
> > -define all_one_r -vector -repeat 1 6 -end \
> > -define all_one_c -vector -repeat 1 3 -end \
> > -draw_matrix \
> > > -input_csv_file Sym3_elts_matrix.csv \
> > > -box_width 50 -bit_depth 8 \
> > > -partition 3 \
> > > > all_one_r all_one_c \
> > -end
> $(OPEN) Sym3_elts_matrix_draw.bmp

```

produces a graphical representation of the elements of the symmetric group  $\text{Sym}(3)$ , shown below



Let us create the groups of order 8. Up to isomorphism, there are 5 groups. Three are abelian ( $C_8$ ,  $C_4 \times C_2$  and  $C_2 \times C_2 \times C_2$ ) and two are not (Dihedral group of order 8 and the quaternion group). The following commands can be used to create these five groups one-by-one:

#### Example 181

```
Group_8_1:
> $(ORBITER) -v 2 \
> > -define gens -vector -dense \
> > > "1,2,3,4,5,6,7,0" \
> > -end \
> > -define G -permutation_group -symmetric_group 8 \
> > > -subgroup_by_generators G1 8 1 gens -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -export_group_table \
> > -end
> $(ORBITER) -v 2 \
> > -define all_one_r -vector -repeat 1 8 -end \
> > -define all_one_c -vector -repeat 1 8 -end \
> > -draw_matrix \
> > > -input_csv_file \
> > > > Sym_8_Subgroup_G1_8_group_table.csv \
> > > -box_width 50 -bit_depth 24 \
> > > -partition 3 all_one_r all_one_c \
> > -end
> convert Sym_8_Subgroup_G1_8_group_table_draw.bmp \
> > Sym_8_Subgroup_G1_8_group_table_draw.png
> $(OPEN) Sym_8_Subgroup_G1_8_group_table_draw.png
```

## Example 182

```

Group.8_2:
> $(ORBITER) -v 2 \
> > -define gens -vector -dense \
> > > "1,2,3,0,4,5, 0,1,2,3,5,4" \
> > -end \
> > -define G -permutation_group -symmetric_group 6 \
> > > -subgroup_by_generators G2 8 2 gens -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -export_group_table \
> > -end
> $(ORBITER) -v 2 \
> > -define all_one_r -vector -repeat 1 8 -end \
> > -define all_one_c -vector -repeat 1 8 -end \
> > -draw_matrix \
> > > -input_csv_file \
> > > > Sym_6.Subgroup.G2.8.group_table.csv \
> > > -box_width 50 -bit_depth 24 \
> > > -partition 3 all_one_r all_one_c \
> > -end
> convert Sym_6.Subgroup.G2.8.group_table_draw.bmp \
> > Sym_6.Subgroup.G2.8.group_table_draw.png
> $(OPEN) Sym_6.Subgroup.G2.8.group_table_draw.png

```

## Example 183

```

Group.8_3:
> $(ORBITER) -v 2 \
> > -define gens -vector -dense \
> > > "1,0,2,3,4,5, 0,1,3,2,4,5, 0,1,2,3,5,4" \
> > -end \
> > -define G -permutation_group -symmetric_group 6 \
> > > -subgroup_by_generators G3 8 3 gens -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -export_group_table \
> > -end
> $(ORBITER) -v 2 \
> > -define all_one_r -vector -repeat 1 8 -end \
> > -define all_one_c -vector -repeat 1 8 -end \
> > -draw_matrix \
> > > -input_csv_file \
> > > > Sym_6.Subgroup.G3.8.group_table.csv \
> > > -box_width 50 -bit_depth 24 \
> > > -partition 3 all_one_r all_one_c \
> > -end
> convert Sym_6.Subgroup.G3.8.group_table_draw.bmp \
> > Sym_6.Subgroup.G3.8.group_table_draw.png
> $(OPEN) Sym_6.Subgroup.G3.8.group_table_draw.png

```

## Example 184

```

Group.8_4:
▷ $(ORBITER) -v 2 \
▷ ▷ -define gens -vector -dense \
▷ ▷ ▷ "1,2,3,0, 0,3,2,1" \
▷ ▷ -end \
▷ ▷ -define G -permutation.group -symmetric.group 4 \
▷ ▷ ▷ -subgroup.by.generators G4 8 2 gens -end \
▷ ▷ -with G -do \
▷ ▷ -group.theoretic.activity \
▷ ▷ ▷ -export.group.table \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
▷ ▷ -draw.matrix \
▷ ▷ ▷ -input.csv.file \
▷ ▷ ▷ ▷ Sym_4.Subgroup.G4.8.group.table.csv \
▷ ▷ ▷ -box.width 50 -bit.depth 24 \
▷ ▷ ▷ -partition 3 all_one_r all_one_c \
▷ ▷ -end
▷ convert Sym_4.Subgroup.G4.8.group.table.draw.bmp \
▷ ▷ Sym_4.Subgroup.G4.8.group.table.draw.png
▷ $(OPEN) Sym_4.Subgroup.G4.8.group.table.draw.png

```

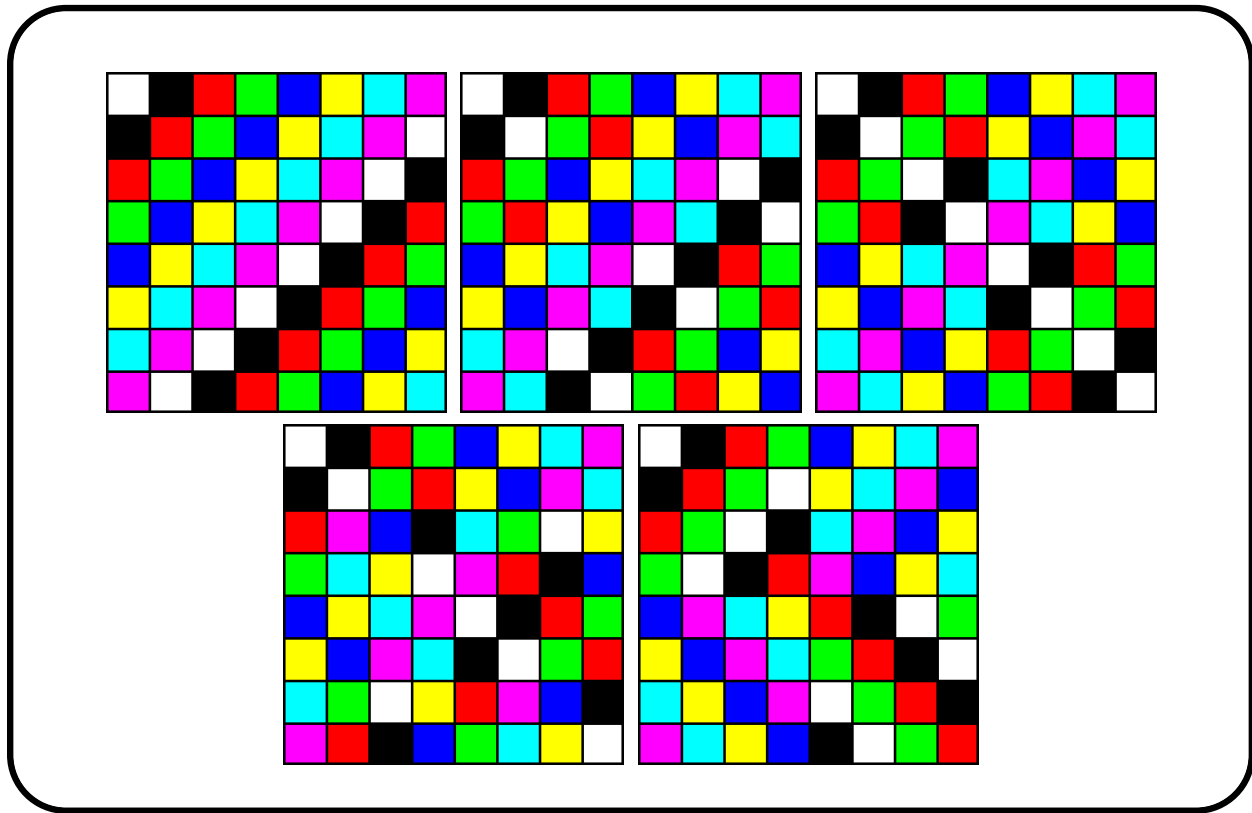
## Example 185

```

Group.8_5:
▷ $(ORBITER) -v 2 \
▷ ▷ -define gens -vector -dense \
▷ ▷ ▷ "2,3,1,0,6,7,5,4, 4,5,7,6,1,0,2,3" \
▷ ▷ -end \
▷ ▷ -define G -permutation.group -symmetric.group 8 \
▷ ▷ ▷ -subgroup.by.generators G5 8 2 gens -end \
▷ ▷ -with G -do \
▷ ▷ -group.theoretic.activity \
▷ ▷ ▷ -export.group.table \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
▷ ▷ -draw.matrix \
▷ ▷ ▷ -input.csv.file \
▷ ▷ ▷ ▷ Sym_8.Subgroup.G5.8.group.table.csv \
▷ ▷ ▷ -box.width 50 -bit.depth 24 \
▷ ▷ ▷ -partition 3 all_one_r all_one_c \
▷ ▷ -end
▷ convert Sym_8.Subgroup.G5.8.group.table.draw.bmp \
▷ ▷ Sym_8.Subgroup.G5.8.group.table.draw.png
▷ $(OPEN) Sym_8.Subgroup.G5.8.group.table.draw.png

```

The Cayley tables are shown below



The Mathieu group  $M_{12}$  of order 95040 is generated by the permutations

$$(0, 3)(2, 9)(4, 10)(5, 11), \quad (0, 7, 8)(1, 2, 3)(4, 11, 10)(5, 9, 6),$$

taken from [70]. The generators can be written in list notation, which is the second row in the arrays

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 3 & 1 & 9 & 0 & 10 & 11 & 6 & 7 & 8 & 2 & 4 & 5 \end{bmatrix} \text{ and } \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 7 & 2 & 3 & 1 & 11 & 9 & 5 & 8 & 0 & 6 & 4 & 10 \end{bmatrix}.$$

We use a makefile variable to store these generators, written one after another:

#### Example 186

```
GENERATORS_M12="3,1,9,0,10,11,6,7,8,2,4,5, 7,2,3,1,11,9,5,8,0,6,4,10"
```

The following command creates the group and produces a report.

#### Example 187

```
Group_M12:
▷ $(ORBITER) -v 2 \
▷ ▷ -define gens -vector -dense $(GENERATORS_M12) -end \
▷ ▷ -define G -permutation_group -symmetric_group 12 \
▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \
```

```
▷ ▷ -with G -do \  
▷ ▷ -group_theoretic_activity \  
▷ ▷ ▷ -report \  
▷ ▷ -end  
▷ pdflatex Sym_12_Subgroup_M12_95040_report.tex  
▷ $(OPEN) Sym_12_Subgroup_M12_95040_report.pdf
```



| Creating a Matrix Group (Part 1) |           |                                                                                                      |
|----------------------------------|-----------|------------------------------------------------------------------------------------------------------|
| Command                          | Arguments | Purpose                                                                                              |
| -GL                              | $n F$     | $GL(n, q)$ . Here (and in all of below), $F$ is a Orbiter object for a finite field $\mathbb{F}_q$ . |
| -GGL                             | $n F$     | $\Gamma L(n, q)$ .                                                                                   |
| -SL                              | $n F$     | $SL(n, q)$ .                                                                                         |
| -SSL                             | $n F$     | $\Sigma L(n, q)$ .                                                                                   |
| -PGL                             | $n F$     | $PGL(n, q)$ .                                                                                        |
| -PGGL                            | $n F$     | $PTL(n, q)$ .                                                                                        |
| -PSL                             | $n F$     | $PSL(n, q)$ .                                                                                        |
| -PSSL                            | $n F$     | $P\Sigma L(n, q)$ .                                                                                  |
| -AGL                             | $n F$     | $AGL(n, q)$ .                                                                                        |
| -AGGL                            | $n F$     | $A\Gamma L(n, q)$ .                                                                                  |
| -ASL                             | $n F$     | $ASL(n, q)$ .                                                                                        |
| -ASSL                            | $n F$     | $A\Sigma L(n, q)$ .                                                                                  |
| -GL_d_q_wr_Sym_n                 | $d F n$   | $GL(d, q)$ wreath product $Sym(n)$ .                                                                 |
| -PGO                             | $n F$     | $PGO(n, q)$ .                                                                                        |
| -PGOp                            | $n F$     | $PGO^+(n, q)$ .                                                                                      |
| -PGOm                            | $n F$     | $PGO^-(n, q)$ .                                                                                      |
| -PGGO                            | $n F$     | $P\Gamma O(n, q)$ .                                                                                  |
| -PGGOp                           | $n F$     | $P\Gamma O^+(n, q)$ .                                                                                |
| -PGGOm                           | $n F$     | $P\Gamma O^-(n, q)$ .                                                                                |

Table 6.2: Creating a Matrix Group (Part 1)

## 6.2 Matrix Groups

Tables 6.2-6.3 shows the Orbiter commands to create a matrix group. A matrix group always comes with a permutation group representation, mostly called the “natural” representation. In some commands, the permutation action is replaced by something else. For background information about the classical groups of matrices over finite fields, see cf. [67].

Let  $V \simeq \mathbb{F}_q^n$  be a finite dimensional vector space over  $\mathbb{F}_q$ . The set of subspaces of  $V$  form the projective geometry  $PG(n-1, q)$ .

Let  $\pi$  be a projective space. A collineation of a projective space  $\pi$  is a bijective mapping from the points of  $\pi$  to themselves which preserves collinearity. That is, a collineation  $\varphi$  maps any three collinear points  $P, Q, R$  to another collinear triple  $\varphi(P), \varphi(Q), \varphi(R)$ . The collineations form a group with respect to composition, the collineation group. If  $M$  is the matrix of an endomorphism, then  $\Psi_M$  is the induced map on projective space. By considering the homomorphism  $M \mapsto \Psi_M$ , the group  $GL(n+1, q)$  of invertible endomorphisms becomes a subgroup of the group of collineations of  $PG(n, q)$ . This is the projectivity group  $PGL(n+1, q)$ . It is isomorphic to  $GL(n+1, q)/\mathbb{F}_q^\times$ . Another source of collineations is this: Let  $\Phi \in \text{Aut}(\mathbb{F}_q)$  be a field automorphism. Then  $\Phi$  acts on projective space by sending  $P(\mathbf{x})$  to  $P(\mathbf{x}\Phi)$ . This map is another type of collineation, called automorphic collineation. This way,  $\text{Aut}(\mathbb{F}_q)$  gives rise to a group of collineations. If  $q = p^h$  for some prime  $p$  and some integer  $h$  then

$$\Phi_0 : \mathbb{F}_q \rightarrow \mathbb{F}_q, x \mapsto x^p$$

is a generator for the cyclic group  $C_h \simeq \text{Aut}(\mathbb{F}_q)$ . The collineation group of  $PG(n, q)$  ( $n \geq 2$ ) is isomorphic to the semidirect product of the projectivity group and the automorphism group of the field. The collineation

| Creating a Matrix Group (Part 2) |                         |                                                                                                                           |
|----------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Command                          | Arguments               | Purpose                                                                                                                   |
| -wedge                           |                         | Create the group in the action on the wedge product.                                                                      |
| -wedge_detached                  |                         | Create the group in the action on the wedge product.                                                                      |
| -PGL20nConic                     |                         | Create the action of $\text{PGL}(2, q)$ on the conic.                                                                     |
| -monomial                        |                         | Create the group in the monomial action.                                                                                  |
| -diagonal                        |                         | Create the group in the diagonal action.                                                                                  |
| -null_polarity_group             |                         | Create the subgroup corresponding to a null polarity.                                                                     |
| -symplectic_group                |                         | Create the subgroup of type symplectic group.                                                                             |
| -singer                          |                         | Create the Singer subgroup.                                                                                               |
| -singer_and_frobenius            | $k$                     | Create the Singer subgroup extended by a Frobenius automorphism. Here, $k$ is the power of the Frobenius endomorphism.    |
| -subfield_structure_action       | $s$                     | Create the action on the field reduction w.r.t. the subfield $\mathbb{F}_r$ , where $r^s = q$ .                           |
| -subgroup_from_file              | fname label             | Create the subgroup from the given file and assign the given label.                                                       |
| -borel_upper                     |                         | Create the upper Borel subgroup.                                                                                          |
| -borel_lower                     |                         | Create the lower Borel subgroup.                                                                                          |
| -identity_group                  |                         | Create the identity subgroup.                                                                                             |
| -orthogonal                      | epsilon                 | Create the orthogonal group of type epsilon as subgroup, where $\epsilon \in \{0, 1, -1\}$ .                              |
| -on_tensors                      |                         | Create the tensor action.                                                                                                 |
| -on_rank_one_tensors             |                         | Create the action on rank-one tensors.                                                                                    |
| -subgroup_by_generators          | label order $k$<br>gens | Create a subgroup with a known order from a set of generators. There are $k$ generators. gens is a orbiter vector object. |
| -Janko1                          |                         | Create the Janko group $J_1$ as a subgroup.                                                                               |
| -export_magma                    |                         | Export the group to Magma.                                                                                                |
| -import_group_of_plane           | plane                   | Import the group of a translation plane. Here, plane must be an Orbiter object of type translation plane.                 |
| -lex_least_base                  |                         | Create the lex-least base. This will make indexing of group elements deterministic.                                       |

Table 6.3: Creating a Matrix Group (Part 2)

group is  $\text{PTL}(n+1, q) = \text{PGL}(n+1, q) \ltimes \text{Aut}(\mathbb{F}_q)$ . We use the following notation for elements of  $\text{PTL}(n+1, q)$ . Let  $\Phi_0$  be a generator for  $\text{Aut}(\mathbb{F}_q)$  and let  $M \in \text{GL}(n+1, q)$ . The map

$$(\Psi_M, \Phi_0^k) : \text{PG}(n, q) \rightarrow \text{PG}(n, q), \quad P(\mathbf{x}) \mapsto P(\mathbf{y}), \quad \mathbf{y} = (\mathbf{x} \cdot M)^{\Phi_0^k}$$

is denoted as

$$M_k. \tag{6.1}$$

The identity element is  $I_0$ , where  $I$  is the identity matrix and 0 is the residue class modulo  $h$ . The rules for multiplication and inversion in the collineation group are given as

$$M_k \cdot N_l = \left( M \cdot N^{\Phi^{-k}} \right)_{k+l}, \tag{6.2}$$

$$\left( M_k \right)^{-1} = \left( \left( M^{-1} \right)^{\Phi^k} \right)_{-k}. \tag{6.3}$$

The affine group  $\text{AGL}(n, q)$  is the semidirect product of  $\text{GL}(n, q)$  with  $\mathbb{F}_q^n$ . The affine semilinear group  $\text{ATL}(n, q)$  is the semidirect product of  $\text{AGL}(n, q)$  with  $\text{Aut}(\mathbb{F}_q)$ . The elements of  $\text{ATL}(n, q)$  are triples

$$M_{\mathbf{a}, k} := (M, \mathbf{a}, k) \in \text{GL}(n, q) \times \mathbb{F}_q^n \times \text{Aut}(\mathbb{F}_q),$$

which act on  $\mathbb{F}_q^n$ :

$$\left( \mathbf{x}, (M, \mathbf{a}, k) \right) \mapsto (\mathbf{x} \cdot M + \mathbf{a})^{\Phi^k}.$$

The multiplication in  $\text{ATL}(n, q)$  is

$$M_{\mathbf{a}, k} \cdot N_{\mathbf{b}, l} = (MN)_{\mathbf{a}N^{\Phi^{-k}} + \mathbf{b}^{\Phi^{-k}}, k+l}.$$

The inverse of an element is

$$\left( M_{\mathbf{a}, k} \right)^{-1} = \left( M^{-1} \right)_{\mathbf{a}^{\Phi^k} M^{-1}, -k}.$$

A correlation is a one-to-one mapping between the set of points and the set of hyperplanes which reverses incidence. So, if  $\rho$  is a correlation and  $P$  is a point and  $\ell$  is a hyperplane then  $P^\rho$  is a hyperplane and  $\ell^\rho$  is a point and

$$\ell^\rho \in P^\rho \iff P \in \ell.$$

A correlation of order two is called polarity. The standard polarity is the map

$$\rho : \mathcal{P} \leftrightarrow \mathcal{L}, \quad P(\mathbf{x}) \leftrightarrow [\mathbf{x}].$$

A group  $G$  can act on  $V$  in one of the types listed in Table 6.4.

One can create a matrix group over a finite field  $\mathbb{F}_q$  is created as described in in two steps. In the first step, the field  $\mathbb{F}_q$  is created as described in Sections 3.1 and 3.2. The field is stored in the symbol table. Then, the group is created using the symbolic label for the field.

For instance,

#### Example 188

```
PGL_4.2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 4 F -lex_least_base -end \
> > -with G -do \
> > -group_theoretic_activity \
```

| Basic Actions                               |                                                             |                                          |
|---------------------------------------------|-------------------------------------------------------------|------------------------------------------|
| Type                                        | Perm. Do-<br>main                                           | Degree                                   |
| General linear $GL(n, q)$                   | all vectors of $V$                                          | $q^n$                                    |
| Affine $AGL(n, q)$                          | all vectors of $V$                                          | $q^n$                                    |
| Projective $PGL(n, q)$                      | $\mathfrak{S}r_1(V)$                                        | $\frac{q^n-1}{q-1}$                      |
| Wreath product $GL(d, q) \wr \text{Sym}(n)$ | $\mathfrak{S}r_1((\mathbb{F}_q^d)^{\otimes n})$<br>extended | $n + nq^d + \frac{q^{dn}-1}{q-1}$        |
| Orthogonal $PGO(n, q)$                      | $Q(V)$                                                      | $\frac{q^{n-1}-1}{q-1}$                  |
| Orthogonal $PGO^+(n, q)$                    | $Q^+(V)$                                                    | $\frac{(q^{n/2}-1)(q^{(n-2)/2}+1)}{q-1}$ |
| Orthogonal $PGO^-(n, q)$                    | $Q^-(V)$                                                    | $\frac{(q^{n/2}+1)(q^{(n-2)/2}-1)}{q-1}$ |

Table 6.4: Basic actions

```

> > > -report \
> > -end
> pdflatex PGL_4.2.report.tex
> $(OPEN) PGL_4.2.report.pdf

```

creates the group  $PGL(4, 2)$  acting on the 15 elements of  $\mathfrak{S}r_1(\mathbb{F}_2^4)$ . At first, the field  $\mathbb{F}_2$  is created. Secondly, the group  $G = PGL(3, 2)$  is created using the previously created field  $\mathbb{F}_2$ , and a report is generated. The report gives information about the permutation group action, including the underlying field and the projective geometry.

## The Group $PGL(4, 2)$

The order of the group  $PGL(4, 2)$  is 20160

The group acts on a set of size 15

Strong generators for a group of order 20160:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \\
 \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,  
 1,0,0,0,0,1,0,0,0,0,1,0,0,1,0,1,  
 1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,  
 1,0,0,0,0,1,0,0,0,0,0,1,0,0,1,0,  
 1,0,0,0,0,0,1,0,0,1,0,0,0,0,0,1,  
 0,1,0,0,1,0,0,0,0,0,1,0,0,0,0,1,

## The Action

Group action  $\text{PGL}(4, 2)$  of degree 15

We act on the following set:

|                    |                     |
|--------------------|---------------------|
| $0 = (1, 0, 0, 0)$ | $8 = (1, 1, 1, 0)$  |
| $1 = (0, 1, 0, 0)$ | $9 = (1, 0, 0, 1)$  |
| $2 = (0, 0, 1, 0)$ | $10 = (0, 1, 0, 1)$ |
| $3 = (0, 0, 0, 1)$ | $11 = (1, 1, 0, 1)$ |
| $4 = (1, 1, 1, 1)$ | $12 = (0, 0, 1, 1)$ |
| $5 = (1, 1, 0, 0)$ | $13 = (1, 0, 1, 1)$ |
| $6 = (1, 0, 1, 0)$ | $14 = (0, 1, 1, 1)$ |
| $7 = (0, 1, 1, 0)$ |                     |

The group is a matrix group.

The group acts on projective space  $\text{PG}(3, 2)$

$q = 2$

$p = 2$

$e = 1$

$n = 3$

Number of points = 15

Number of lines = 35

Number of lines on a point = 7

Number of points on a line = 3

## The finite field $\mathbb{F}_2$

$Z_i = \log_\alpha(1 + \alpha^i)$

| $i$ | $\gamma_i$ | $-\gamma_i$ | $\gamma_i^{-1}$ | $\log_\alpha(\gamma_i)$ | $\alpha^i$ | $Z_i$ |
|-----|------------|-------------|-----------------|-------------------------|------------|-------|
| 0   | $0 = 0$    | 0           | DNE             | DNE                     | 1          | DNE   |
| 1   | $1 = 1$    | 1           | 1               | 0                       | 1          | DNE   |

|   |   |   |
|---|---|---|
| + | 0 | 1 |
| 0 | 0 | 1 |
| 1 | 1 | 0 |

|   |   |
|---|---|
| · | 1 |
| 1 | 1 |

$1^0 \equiv 1$

$1^1 \equiv 1$

## Base and Stabilizer Chain

Group order 20160

tl=15, 14, 12, 8,

Base: (0, 1, 2, 3)

Strong generators for a group of order 20160:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix},$$

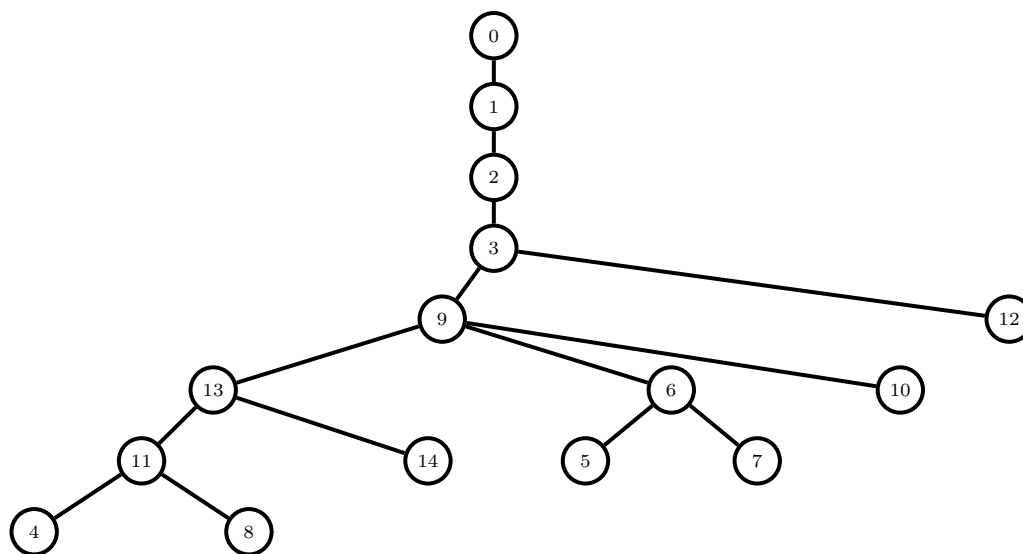
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,  
1,0,0,0,0,1,0,0,0,0,1,0,0,1,0,1,  
1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,  
1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,0,  
1,0,0,0,0,1,0,0,0,0,1,0,0,1,0,  
1,0,0,0,0,0,1,0,0,1,0,0,0,0,0,1,  
0,1,0,0,1,0,0,0,0,0,1,0,0,0,0,1,

## Stabilizer chain

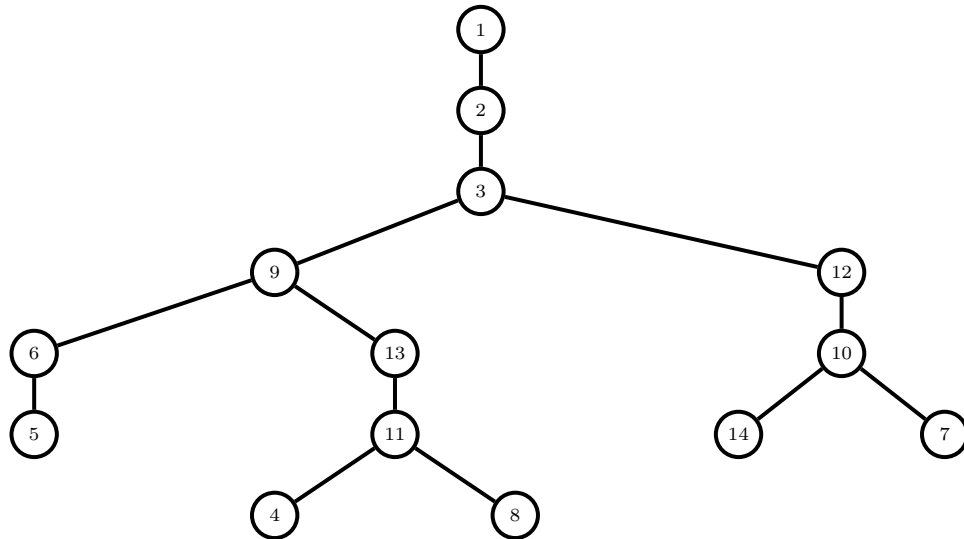
| Level | Base pt | Orbit length | Subgroup order |
|-------|---------|--------------|----------------|
| 0     | 0       | 15           | 20160          |
| 1     | 1       | 14           | 1344           |
| 2     | 2       | 12           | 96             |
| 3     | 3       | 8            | 8              |

## Basic Orbit 0

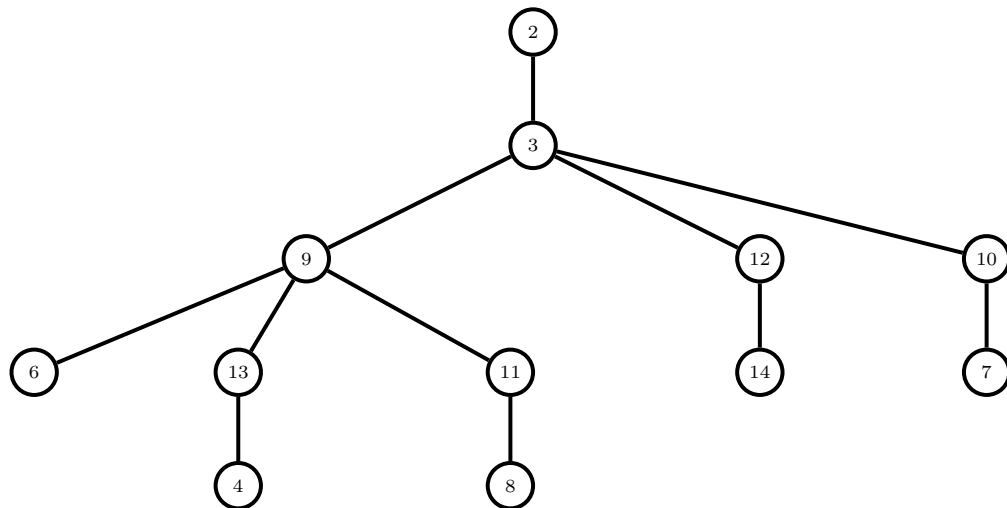


Basic orbit 0 has size 15

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

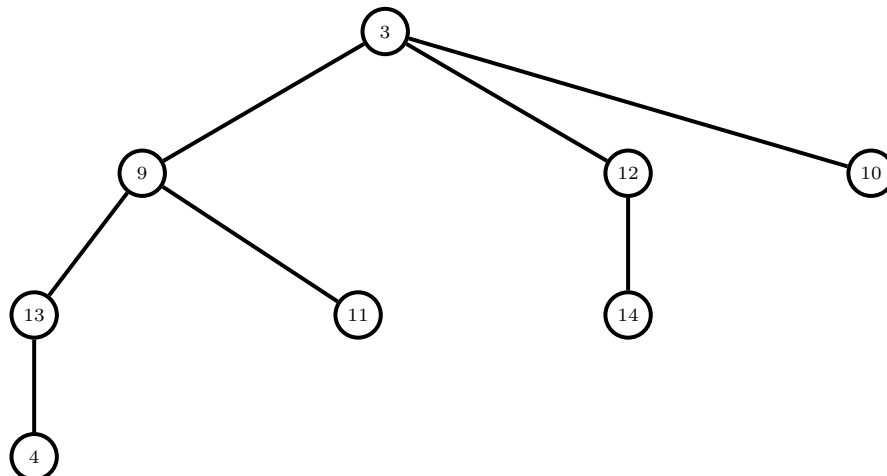
**Basic Orbit 1**

Basic orbit 1 has size 14  
 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

**Basic Orbit 2**

Basic orbit 2 has size 12  
 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14

### Basic Orbit 3



Basic orbit 3 has size 8  
 3, 4, 9, 10, 11, 12, 13, 14  
 GAP export:

Generators in GAP format are:  
`G := Group([(4, 10)(5, 15)(11, 12)(13, 14),  
 (4, 11)(5, 14)(10, 12)(13, 15),  
 (4, 13)(5, 12)(10, 14)(11, 15),  
 (3, 4)(7, 10)(8, 11)(9, 12),  
 (2, 3)(6, 7)(11, 13)(12, 14),  
 (1, 2)(7, 8)(10, 11)(14, 15)]);`

Magma export:

```

G := GeneralLinearGroup(4, GF(2));
H := sub< G | [1,0,0,0, 0,1,0,0, 0,0,1,0, 1,0,0,1],
[1,0,0,0, 0,1,0,0, 0,0,1,0, 0,1,0,1],
[1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,1,1],
[1,0,0,0, 0,1,0,0, 0,0,0,1, 0,0,1,0],
[1,0,0,0, 0,0,1,0, 0,1,0,0, 0,0,0,1],
[0,1,0,0, 1,0,0,0, 0,0,1,0, 0,0,0,1] >;

```

Compact form:

Generators in compact permutation form are:  
 6 15  
 0 1 2 9 14 5 6 7 8 3 11 10 13 12 4  
 0 1 2 10 13 5 6 7 8 11 3 9 14 4 12  
 0 1 2 12 11 5 6 7 8 13 14 4 3 9 10  
 0 1 3 2 4 5 9 10 11 6 7 8 12 13 14  
 0 2 1 3 4 6 5 7 8 9 12 13 10 11 14  
 1 0 2 3 4 5 7 6 8 10 9 11 12 14 13  
 -1

The base has length 4  
 The basic orbits are:  
 Basic orbit 0 is orbit of 0 of length 15



Basic orbit 1 is orbit of 1 of length 14  
 Basic orbit 2 is orbit of 2 of length 12  
 Basic orbit 3 is orbit of 3 of length 8

We use the following Orbiter command creates  $\text{PGL}(4, 2)$  again. The command invokes two activities. The first creates a latex report for the group in the file `PGL_4_2_report.tex`. The second activity exports the permutation representation in Orbiter makefile format.

#### Example 189

```
PGL_4_2_export:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -export_orbiter \
▷ ▷ -end
▷ pdflatex PGL_4_2_report.tex
▷ $(OPEN) PGL_4_2_report.pdf
```

The file `PGL_4_2.makefile` is created:

#### Example 190

```
PGL_4_2_generated:
▷ $(ORBITER) -v 2 \
▷ ▷ -define gens -vector -file PGL_4_2_gens.csv -end \
▷ ▷ -define G -permutation_group \
▷ ▷ -bsgs PGL_4_2 "{\rm PGL}(4,2)" 15 20160 "0,1,2,3" 6 gens -end \
```

This command can be used to recreate the group as permutation group directly. This group will be considered again in Section 6.2 below. The permutation representation itself is stored in the file `PGL_4_2_gens.csv`:

```
Row,C0,C1,C2,C3,C4,C5,C6,C7,C8,C9,C10,C11,C12,C13,C14
0,0,1,2,9,14,5,6,7,8,3,11,10,13,12,4
1,0,1,2,10,13,5,6,7,8,11,3,9,14,4,12
2,0,1,2,12,11,5,6,7,8,13,14,4,3,9,10
3,0,1,3,2,4,5,9,10,11,6,7,8,12,13,14
4,0,2,1,3,4,6,5,7,8,9,12,13,10,11,14
5,1,0,2,3,4,5,7,6,8,10,9,11,12,14,13
END
```

The command

**Example 191**

```

L_5.3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define G -linear_group -PSL 5 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PSL_5.3_report.tex
▷ $(OPEN) PSL_5.3_report.pdf

```

creates  $\text{PSL}(5, 3)$  of order 237783237120 in the action on the 121 points of  $\text{PG}(4, 3)$ .

The command

**Example 192**

```

L_4.5:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define G -linear_group -PSL 4 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PSL_4.5_report.tex
▷ $(OPEN) PSL_4.5_report.pdf

```

creates  $\text{PSL}(4, 5)$  of order 7254000000 in the action on the 156 points of  $\text{PG}(3, 5)$ .

The command

**Example 193**

```

PGL_4.5:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_4.5_report.tex
▷ $(OPEN) PGL_4.5_report.pdf

```

creates  $\text{PGL}(4, 5)$  of order 29016000000 in the action on the 156 points of  $\text{PG}(3, 5)$ . The group  $\text{PSL}(4, 5)$  is a subgroup of  $\text{PGL}(4, 5)$  of index 4.

The command

**Example 194**

```

PGGL_3_4:
> $(ORBITER) -v 2 \
> > -define G -linear_group -PGGL 3 4 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > > -report_sylow \
> > > -report_classes \
> > -end
> pdflatex PGGL_3_4_report.tex
> $(OPEN) PGGL_3_4_report.pdf

```

creates  $\text{PTL}(3, 4)$  of order 120960 in the action on the 21 points of  $\text{PG}(2, 4)$ . The report contains one  $p$ -Sylow subgroup for each of the primes  $p$  dividing the order of the group.

The command

**Example 195**

```

PGGL_3_8:
> $(ORBITER) -v 2 \
> > -define G -linear_group -PGGL 3 8 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > > -report_sylow \
> > > -report_classes \
> > -end
> pdflatex PGGL_3_8_report.tex
> $(OPEN) PGGL_3_8_report.pdf

```

creates  $\text{PTL}(3, 8)$  of order 49448448 in the action on the 73 points of  $\text{PG}(2, 8)$ . The report contains one  $p$ -Sylow subgroup for each of the primes  $p$  dividing the order of the group.

The command

**Example 196**

```

AGL_1_27:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 27 -end \
> > -define G -linear_group -AGL 1 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex AGL_1_27_report.tex
> $(OPEN) AGL_1_27_report.pdf

```

creates  $\text{AGL}(1, 27)$  of order 702 in the action on the 27 points of  $\text{AG}(1, 27)$ .

The command

#### Example 197

```
AGGL_2_27:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 27 -end \
> > -define G -linear_group -AGGL 2 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex AGGL_2_27_report.tex
> $(OPEN) AGGL_2_27_report.pdf
```

creates  $\text{AGL}(2, 27)$  of order 1117679472 in the action on the 729 points of  $\text{AG}(2, 27)$ .

The command

#### Example 198

```
SP_4_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -GL 4 F \
> > > -symplectic_group \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex GL_4_2_Sp_4_2_report.tex
> $(OPEN) GL_4_2_Sp_4_2_report.pdf
```

creates the symplectic group  $\text{Sp}(4, 2)$  of order 720 acting on the 16 points of  $\text{AG}(4, 2)$ .

The command

#### Example 199

```
PSP_4_4:
> $(ORBITER) -v 5 \
> > -define F -finite_field -q 4 -end \
> > -define G -linear_group -PGL 4 F \
> > > -symplectic_group \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
```

```

> pdflatex PGL_4_4_Sp_4_4_report.tex
> $(OPEN) PGL_4_4_Sp_4_4_report.pdf

```

creates the symplectic group  $\mathrm{PSp}(4, 4)$  of order 979200 acting on the 85 points of  $\mathrm{PG}(3, 4)$ .

The command

#### Example 200

```

PGO_5_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGO 5 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGO_5_2_report.tex
> $(OPEN) PGO_5_2_report.pdf

```

creates the group  $\mathrm{PGO}(5, 2)$  acting on the 15 points of the  $Q(4, 2)$  quadric. The following latex report is produced:

### The Group $\mathrm{PGO}(5, 2)$

The order of the group  $\mathrm{PGO}(5, 2)$  is 720

The group acts on a set of size 15

Strong generators for a group of order 720:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \\
 \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

```

1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,1,0,1,1,
1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,1,0,0,0,1,
1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,1,0,0,0,1,
1,0,0,0,0,0,1,0,0,0,0,0,1,0,1,1,1,0,1,0,1,0,
1,0,0,0,0,1,1,1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,

```

1,0,0,0,0,0,1,0,0,1,0,0,1,0,0,0,0,1,1,0,0,0,0,0,1,

## The Action

Group action  $\text{PGO}(5, 2)$  of degree 15

We act on the following set:

$$0 = (0, 1, 0, 0, 0)$$

$$1 = (0, 0, 1, 0, 0)$$

$$2 = (0, 0, 0, 1, 0)$$

$$3 = (0, 1, 0, 1, 0)$$

$$4 = (0, 0, 1, 1, 0)$$

$$5 = (0, 0, 0, 0, 1)$$

$$6 = (0, 1, 0, 0, 1)$$

$$7 = (0, 0, 1, 0, 1)$$

$$8 = (0, 1, 1, 1, 1)$$

$$9 = (1, 1, 1, 0, 0)$$

$$10 = (1, 1, 1, 1, 0)$$

$$11 = (1, 1, 1, 0, 1)$$

$$12 = (1, 0, 0, 1, 1)$$

$$13 = (1, 1, 0, 1, 1)$$

$$14 = (1, 0, 1, 1, 1)$$

The group is a matrix group.

The base action is on projective space  $\text{PG}(4, 2)$

$$q = 2$$

$$p = 2$$

$$e = 1$$

$$n = 4$$

Number of points = 31

Number of lines = 155

Number of lines on a point = 15

Number of points on a line = 3

## The finite field $\mathbb{F}_2$

$$Z_i = \log_\alpha(1 + \alpha^i)$$

| $i$ | $\gamma_i$ | $-\gamma_i$ | $\gamma_i^{-1}$ | $\log_\alpha(\gamma_i)$ | $\alpha^i$ | $Z_i$ |
|-----|------------|-------------|-----------------|-------------------------|------------|-------|
| 0   | $0 = 0$    | 0           | DNE             | DNE                     | 1          | DNE   |
| 1   | $1 = 1$    | 1           | 1               | 0                       | 1          | DNE   |

|   |   |   |
|---|---|---|
| + | 0 | 1 |
| 0 | 0 | 1 |
| 1 | 1 | 0 |

|   |   |
|---|---|
| · | 1 |
| 1 | 1 |

$$1^0 \equiv 1$$

$$1^1 \equiv 1$$

### Base and Stabilizer Chain

Group order 720

tl=15, 8, 3, 1, 1, 2,

Base: (0, 1, 2, 3, 4, 5)

Strong generators for a group of order 720:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

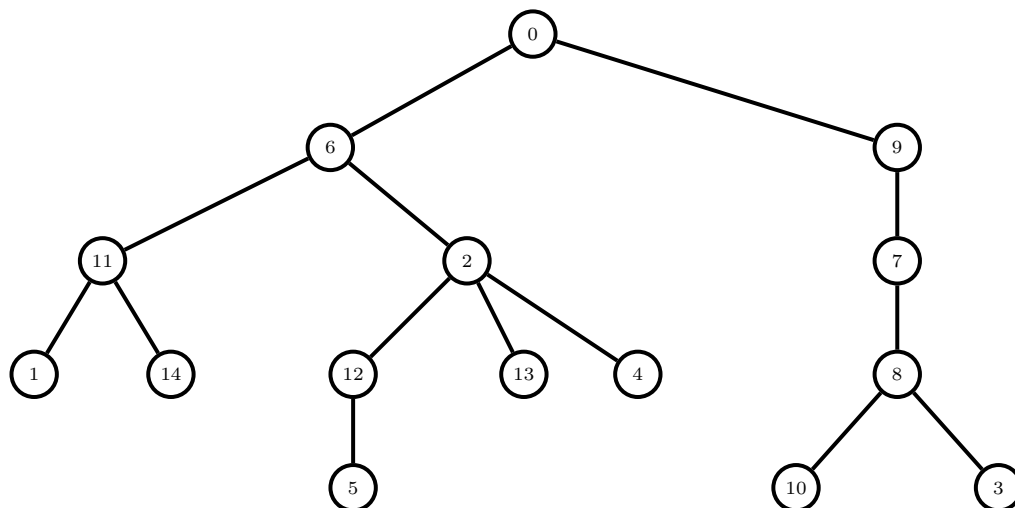
$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,1,  
 1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,1,0,0,1,1,0,0,0,0,1,  
 1,0,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,1,0,1,1,0,0,0,0,1,  
 1,0,0,0,0,0,1,0,0,0,0,0,1,0,1,1,1,0,1,1,0,1,0,1,0,  
 1,0,0,0,0,1,1,1,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1,  
 1,0,0,0,0,0,1,0,0,1,0,0,1,0,0,0,0,1,1,0,0,0,0,1,

### Stabilizer chain

| Level | Base pt | Orbit length | Subgroup order |
|-------|---------|--------------|----------------|
| 0     | 0       | 15           | 720            |
| 1     | 1       | 8            | 48             |
| 2     | 2       | 3            | 6              |
| 3     | 3       | 1            | 2              |
| 4     | 4       | 1            | 2              |
| 5     | 5       | 2            | 2              |

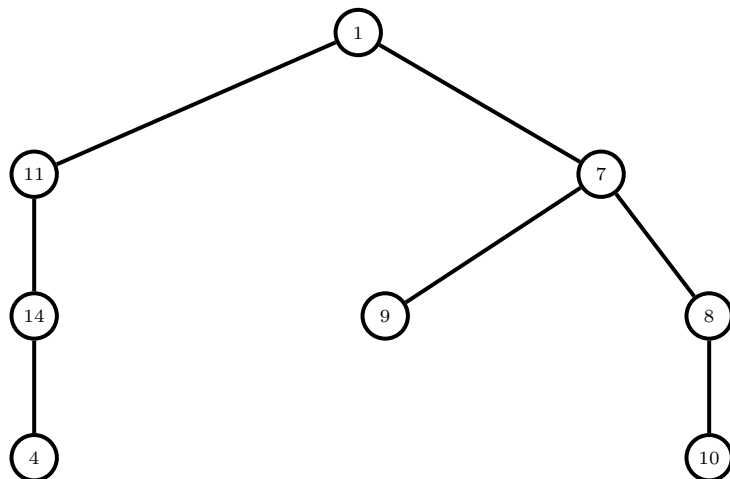
### Basic Orbit 0



Basic orbit 0 has size 15

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

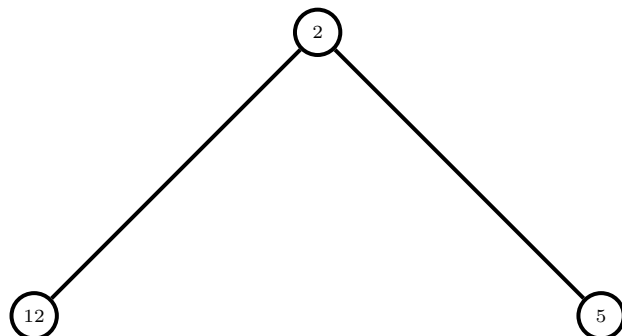
### Basic Orbit 1



Basic orbit 1 has size 8

1, 4, 7, 8, 9, 10, 11, 14

### Basic Orbit 2



Basic orbit 2 has size 3

2, 5, 12

### Basic Orbit 3



Basic orbit 3 has size 1

3



**Basic Orbit 4**

Basic orbit 4 has size 1

4

**Basic Orbit 5**

Basic orbit 5 has size 2

5, 12

GAP export:

Generators in GAP format are:

```
G := Group([(6, 13)(7, 14)(8, 15)(9, 12),
(3, 13)(4, 14)(5, 15)(9, 11),
(2, 12)(3, 14)(4, 13)(8, 10),
(2, 8, 9, 10, 12, 15)(3, 14, 7)(4, 13, 6)(5, 11),
(1, 10)(4, 11)(7, 12)(9, 14),
(1, 7)(3, 5)(4, 9)(10, 12)(11, 14)(13, 15)]);
```

Magma export:

Compact form:

Generators in compact permutation form are:

```
6 15
0 1 2 3 4 12 13 14 11 9 10 8 5 6 7
0 1 12 13 14 5 6 7 10 9 8 11 2 3 4
0 11 13 12 4 5 6 9 8 7 10 1 3 2 14
0 7 13 12 10 3 2 8 9 11 4 14 5 6 1
9 1 2 10 4 5 11 7 13 0 3 6 12 8 14
6 1 4 8 2 5 0 7 3 11 13 9 14 10 12
-1
```

The base has length 6

The basic orbits are:

Basic orbit 0 is orbit of 0 of length 15

Basic orbit 1 is orbit of 1 of length 8

Basic orbit 2 is orbit of 2 of length 3

Basic orbit 3 is orbit of 3 of length 1  
 Basic orbit 4 is orbit of 4 of length 1  
 Basic orbit 5 is orbit of 5 of length 2

The command

#### Example 201

```
PGG0_5_4:
> $(ORBITER) -v 2 \
> > -define F4 -finite_field -q 4 -end \
> > -define G -linear_group -PGG0 5 F4 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGG0_5_4_report.tex
> $(OPEN) PGG0_5_4_report.pdf
```

creates the group  $\text{PTO}(5, 4)$  of order 1958400 acting on the 85 points of the  $Q(4, 4)$  quadric.

The command

#### Example 202

```
PGOp_6_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOp 6 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGOp_6_2_report.tex
> $(OPEN) PGOp_6_2_report.pdf
```

creates the group  $\text{PTO}^+(6, 2)$  of order 40320 acting on the 35 points of the  $Q^+(5, 2)$  quadric.

The command

#### Example 203

```
PGOm_6_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOm 6 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
```

```

> pdflatex PG0m_6_2_report.tex
> $(OPEN) PG0m_6_2_report.pdf

```

creates the group  $\text{PTO}^-(6, 2)$  of order 51840 acting on the 27 points of the  $Q^-(5, 2)$  quadric.

The symplectic group  $\text{PSp}(6, 2)$  can be created using the following command:

#### Example 204

```

PSP_6_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 6 F \
> > > -symplectic_group \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_6_2_Sp_6_2_report.tex
> $(OPEN) PGL_6_2_Sp_6_2_report.pdf

```

The group  $\text{PGO}(7, 2)$ , isomorphic to  $\text{PSp}(6, 2)$ , can be created using the following command:

#### Example 205

```

PGO_7_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGO 7 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGO_7_2_report.tex
> $(OPEN) PGO_7_2_report.pdf

```

The command

#### Example 206

```

NullPolarity_6_2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 6 F \
> > > -null_polarity_group \
> > -end \
> > -with G -do \

```

```
▷ ▷ -group_theoretic_activity \  
▷ ▷ ▷ -report \  
▷ ▷ -end  
▷ pdflatex PGL_6_2_NullPolarity_6_2_report.tex  
▷ $(OPEN) PGL_6_2_NullPolarity_6_2_report.pdf
```

creates the null polarity group of  $\text{PG}(6, 2)$  acting on the 63 points of  $\text{PG}(5, 2)$ . This is the group preserving the dot product

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i.$$

| Creating Subgroups      |                          |                                                                                                                                                                                                                                |
|-------------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                 | Arguments                | Purpose                                                                                                                                                                                                                        |
| -Janko1                 |                          | first Janko group, needs $\text{PGL}(7, 11)$                                                                                                                                                                                   |
| -monomial               |                          | subgroup of monomial matrices                                                                                                                                                                                                  |
| -diagonal               |                          | subgroup of diagonal matrices                                                                                                                                                                                                  |
| -null_polarity_group    |                          | null polarity group                                                                                                                                                                                                            |
| -symplectic_group       |                          | symplectic group                                                                                                                                                                                                               |
| -singer                 | $k$                      | subgroup of index $k$ in the Singer cycle                                                                                                                                                                                      |
| -singer_and_frobenius   | $k$                      | subgroup of index $k$ in the Singer cycle, extended by the Frobenius automorphism of $\mathbb{F}_{q^n}$ over $\mathbb{F}_q$                                                                                                    |
| -borel_upper            |                          | Borel subgroup of upper triangular matrices                                                                                                                                                                                    |
| -borel_lower            |                          | Borel subgroup of lower triangular matrices                                                                                                                                                                                    |
| -identity_group         |                          | identity subgroup                                                                                                                                                                                                              |
| -subgroup_from_file     | $f\ l$                   | read subgroup from file $f$ and give it the label $l$                                                                                                                                                                          |
| -orthogonal             | $\epsilon$               | orthogonal group $O^\epsilon(n, q)$ , with $\epsilon \in \{\pm 1\}$ when $n$ is even                                                                                                                                           |
| -subgroup_by_generators | $l\ o\ n\ s_1 \dots s_n$ | Generate a subgroup from generators. The label "l" is used to denote the subgroup; $o$ is the order of the subgroup; $n$ is the number of generators and $s_1, \dots, s_n$ are the generators for the subgroup in vector form. |

Table 6.5: Creating Subgroups

## 6.3 Subgroups

There are many ways to create subgroups of a group. Table 6.5 lists some commands to do so.

We start with an example of an explicit permutation group using a known base and strong generating set, using the `bsgs` command. Here is the cyclic group of order 13 acting on the permutation domain  $[0, 12]$ . The base is  $(0)$ . When creating a group, we supply a label in ascii text and in tex. Then we specify the degree of the action, and the group order. After that, we specify the number of generators and the generators themselves. The labels will be used in reports about the group, for instance.

### Example 207

```
GEN_C13="1,2,3,4,5,6,7,8,9,10,11,12,0"
# (0,1,2,3,4,5,6,7,8,9,10,11,12)
```

### Example 208

```
C13:
> $(ORBITER) -v 2 \
> > -define gens -vector -dense $(GEN_C13) -end \
> > -define G -permutation_group \
> > > -bsgs C13 C_{13} 13 13 0 1 \
```

```

▷ ▷ ▷ ▷ gens \
▷ ▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -export_orbiter \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -export_group_table \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
▷ ▷ -end
▷ pdflatex C13_report.tex
▷ $(OPEN) C13_report.pdf

```

The makefile variable GEN\_C13 is used to define the generator of the group, which is the cycle

$$(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12).$$

The command creates the group from the known base 0. After that, several activities are invoked. Specifically, these are group theoretic activities. They will be discussed in more detail in Section 6.5.

Let us take a closer look at the three activities performed in this example. The `-export_orbiter` command exports the group in Orbiter makefile format. The file `C13.makefile` is generated, which can be used to recreate the permutation group in an Orbiter makefile. Here is the content of the file:

#### Example 209

```

C13_generated:
▷ $(ORBITER) -v 2 \
▷ ▷ -define gens -vector -file C13_gens.csv -end \
▷ ▷ -define G -permutation_group \
▷ ▷ -bsgs C13 "C_{13}" 13 13 "0" 1 gens -end \

```

The activity `-report` produces a report for the cyclic group, shown below:

#### Stabilizer chain

| Level | Base pt | Orbit length | Subgroup order |
|-------|---------|--------------|----------------|
| 0     | 0       | 13           | 13             |

**Basic Orbit 0**

Basic orbit 0 has size 13

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

The command `-save_elements_csv` creates a csv file containing all group elements. Each group element is listed one-by-one, using the list notation of permutations. The csv file `C13_elts.csv` has the following content:

```
Row,Element
0,"0,1,2,3,4,5,6,7,8,9,10,11,12"
1,"1,2,3,4,5,6,7,8,9,10,11,12,0"
2,"2,3,4,5,6,7,8,9,10,11,12,0,1"
3,"3,4,5,6,7,8,9,10,11,12,0,1,2"
4,"4,5,6,7,8,9,10,11,12,0,1,2,3"
5,"5,6,7,8,9,10,11,12,0,1,2,3,4"
6,"6,7,8,9,10,11,12,0,1,2,3,4,5"
7,"7,8,9,10,11,12,0,1,2,3,4,5,6"
8,"8,9,10,11,12,0,1,2,3,4,5,6,7"
9,"9,10,11,12,0,1,2,3,4,5,6,7,8"
10,"10,11,12,0,1,2,3,4,5,6,7,8,9"
11,"11,12,0,1,2,3,4,5,6,7,8,9,10"
12,"12,0,1,2,3,4,5,6,7,8,9,10,11"
END
```

It is possible to create a permutation group as a subgroup of the symmetric group, using the known base for the symmetric group. Because the base of the symmetric group is large, this way of creating the group is less efficient than creating the group with a known (small) base. Here is an example. We create  $C_{13}$  as a subgroup of  $\text{Sym}(13)$ .

**Example 210**

```

C13_as_subgroup:
> $(ORBITER) -v 2 \
> > -define G -permutation_group -symmetric_group 13 \
> > > -subgroup_by_generators C13 13 1 $(GEN_C13) -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -export_orbiter \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -save_elements_csv "C13.elts.csv" \
> > -end
> #pdflatex Perm13_Subgroup_C13_13_report.tex
> #$(OPEN) Perm13_Subgroup_C13_13_report.pdf

```

For instance, the command

**Example 211**

```

J1:
> $(ORBITER) -v 2 \
> > -define G -linear_group -PGL 7 11 -Janko1 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex Janko1_report.tex
> $(OPEN) Janko1_report.pdf

```

creates the first Janko group as a subgroup of  $\text{PGL}(7, 11)$ .

The command

**Example 212**

```

PGL_3_11_singer:
> $(ORBITER) -v 2 \
> > -define G -linear_group -PGL 3 11 -singer 19 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_3_11_Singer_3_11_19_report.tex
> $(OPEN) PGL_3_11_Singer_3_11_19_report.pdf

```



creates a subgroup of the Singer cycle of order 7. The Singer cycle in  $GL(d, q)$  is a generator for a subgroup of order  $q^d - 1$ . It induces an element of order  $\frac{q^d - 1}{q - 1}$  on the associated projective geometry  $PG(d - 1, q)$ . The additional integer parameter  $k$  after the `-singer` command is used to create the subgroup of index  $k$  of the Singer cycle.

The command

#### Example 213

```
PGL_3_11_singer_and_frobenius:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -linear_group -PGL 3 11 -singer_and_frobenius 19 -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ ▷ pdflatex PGL_3_11_Singer_and_Frob3_11_19_report.tex
▷ ▷ $(OPEN) PGL_3_11_Singer_and_Frob3_11_19_report.pdf
```

creates a subgroup of index 19 of the Singer cycle of  $PG(2, 11)$ , extended by a group of order 3 that arises from the field extension  $\mathbb{F}_{11}^3$  over  $\mathbb{F}_{11}$ . The group created by this command has order 21.

The quaternion group is a group of order 8 generated by the following matrices over  $\mathbb{R}$ :

$$i = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad j = \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}, \quad k = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

It is isomorphic to a subgroup of  $SL(2, 3)$ . The Orbiter command

#### Example 214

```
quaternion:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -linear_group -SL 2 3 \
▷ ▷ -subgroup_by_generators "quaternion" "8" 3 \
▷ ▷ ▷ "1,1,1,2, 2,1,1,1, 0,2,1,0" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -print_elements_tex \
▷ ▷ ▷ -report_group_table \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex SL_2_3_Subgroup_quaternion_8_report.tex
▷ $(OPEN) SL_2_3_Subgroup_quaternion_8_report.pdf
```

creates the group. The command produces the list of group elements shown below.

Element 0 / 8 of order 1:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$(0)(1)(2)(3)(4)(5)(6)(7)(8)$$

Element 1 / 8 of order 4:

$$\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

$$(0)(1, 5, 2, 7)(3, 4, 6, 8)$$

Element 2 / 8 of order 2:

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

$$(0)(1, 2)(3, 6)(4, 8)(5, 7)$$

Element 3 / 8 of order 4:

$$\begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix}$$

$$(0)(1, 7, 2, 5)(3, 8, 6, 4)$$

Element 4 / 8 of order 4:

$$\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$(0)(1, 4, 2, 8)(3, 7, 6, 5)$$

Element 5 / 8 of order 4:

$$\begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}$$

$$(0)(1, 3, 2, 6)(4, 5, 8, 7)$$

Element 6 / 8 of order 4:

$$\begin{bmatrix} 2 & 2 \\ 2 & 1 \end{bmatrix}$$

$$(0)(1, 8, 2, 4)(3, 5, 6, 7)$$

Element 7 / 8 of order 4:

$$\begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}$$

$$(0)(1, 6, 2, 3)(4, 7, 8, 5)$$

The group table is created as csv file:

```
Row,C0,C1,C2,C3,C4,C5,C6,C7
0,0,1,2,3,4,5,6,7
1,1,2,3,0,5,6,7,4
2,2,3,0,1,6,7,4,5
3,3,0,1,2,7,4,5,6
4,4,7,6,5,2,1,0,3
5,5,4,7,6,3,2,1,0
```

```

6,6,5,4,7,0,3,2,1
7,7,6,5,4,1,0,3,2
END

```

The group of the cube can be created over the field  $\mathbb{F}_3$ :

#### Example 215

```

cube_group:
▷ $(ORBITER) -v 2 \
▷ ▷ -define gens -vector -dense \
▷ ▷ ▷ "0,1,0,2,0,0,0,0,1, \
▷ ▷ ▷ 0,0,1,0,1,0,2,0,0, \
▷ ▷ ▷ 2,0,0,0,1,0,0,0,1" \
▷ ▷ -end \
▷ ▷ -define G -linear_group -GL 3 3 \
▷ ▷ -subgroup_by_generators "cube" "48" 3 \
▷ ▷ ▷ gens \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -print_elements_tex \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex GL_3.3_Subgroup_cube_48_report.tex
▷ $(OPEN) GL_3.3_Subgroup_cube_48_report.pdf

```

The tetrahedral subgroup can be created as well:

#### Example 216

```

tetra_group:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -linear_group -GL 3 3 \
▷ ▷ -subgroup_by_generators "tetra" "12" 2 \
▷ ▷ ▷ "0,1,0,0,0,1,1,0,0, 0,0,1,2,0,0,0,2,0" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -print_elements_tex \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex GL_3.3_Subgroup_tetra_12_report.tex
▷ $(OPEN) GL_3.3_Subgroup_tetra_12_report.pdf

```

The Hesse group of order 216 extended by the automorphism group of the field can be created in PG(3, 4)

**Example 217**

```

GENERATORS_HESSE_GROUP="\
3000300030 \
2000201230 \
1000100111 \
1000220200 \
1002312010 \
0331003211 \
2200011331"

```

**Example 218**

```

Hesse_group:
▷ $(ORBITER) -v 3 \
▷ ▷ -define gens -vector -compact \
▷ ▷ ▷ $(GENERATORS_HESSE_GROUP) \
▷ ▷ -end \
▷ ▷ -define G -linear_group -PGGL 3 4 \
▷ ▷ -subgroup_by_generators "Hesse" "432" 7 gens \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -print_elements_tex \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGGL_3_4.Subgroup_Hesse_432_report.tex
▷ $(OPEN) PGGL_3_4.Subgroup_Hesse_432_report.pdf

```

The group has order 432.

The Weyl group of type  $E_8$  can be generated as a subgroup of  $GL(8, 3)$  using the following command:

**Example 219**

```

GENERATORS_WEYL_GROUP_E8="\
-1,-1,-1,-1,0,0,0,0, \
0,0,0,1,0,0,0,0, \
1,0,0,0,0,0,0,0, \
0,0,1,0,0,0,0,0, \
0,1,0,1,1,0,0,0, \
0,0,0,0,0,1,0,0, \
0,0,0,0,0,0,1,0, \
0,0,0,0,0,0,0,1, \
-1,0,-1,-1,-1,-1,-1,-1, \
0,1,0,1,1,1,1,1, \
1,0,0,0,0,0,0,0, \
0,0,1,0,0,0,0,0, \
0,0,0,1,0,0,0,0, \
0,0,0,0,1,0,0,0, \
0,0,0,0,0,1,0,0, \

```

```
0,0,0,0,0,0,1,0"
```

**Example 220**

```
Weyl_E8:
> $(ORBITER) -v 3 \
> > -define gens -vector -dense \
> > > $(GENERATORS.WEYL.GROUP.E8) \
> > -end \
> > -define G -linear_group -GL 8 3 \
> > -subgroup_by_generators \
> > > "Weyl_E8" "696729600" 2 gens \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex GL_8_3_Subgroup_Weyl_E8_696729600_report.tex
> $(OPEN) GL_8_3_Subgroup_Weyl_E8_696729600_report.pdf
```

A latex report is generated in the file `GL_8_3_Subgroup_Weyl_E8_696729600_report.tex`. This command uses generators found by Gabi Nebe:

<http://www.math.rwth-aachen.de/~Gabriele.Nebe/LATTICES/E8.b.html>

We can test if a group is a subgroup of another. In the following example, we test whether  $\text{PGO}^+(6,2)$  is a subgroup of  $\text{PSp}(6,2)$ . The fact that it is depends on the choice of forms associated with the groups and on the fact that the characteristic is two.

**Example 221**

```
test_subgroup:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G1 -linear_group -PGOp 6 F -end \
> > -define G2 -linear_group -PGL 6 F \
> > > -symplectic_group \
> > -end \
> > -with G1 -and G2 -do \
> > -group_theoretic_activity \
> > > -is_subgroup_of \
> > -end
```

Since the subgroup index is small (36), we create a set of coset representatives using the following command:

**Example 222**

```
coset_reps:
> $(ORBITER) -v 2 \
```

```

▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
▷ ▷ -define G2 -linear_group -PGL 6 F \
▷ ▷ ▷ -symplectic_group \
▷ ▷ -end \
▷ ▷ -with G1 -and G2 -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -coset_reps \
▷ ▷ -end
▷ pdflatex PGOp_6_2_coset_reps.tex
▷ $(OPEN) PGOp_6_2_coset_reps.pdf

```

The coset representatives are written to a csv file. The (shortened) list of coset representatives in latex is:

coset 0 / 36:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 100000 \\ 010000 \\ 001000 \\ 000100 \\ 000010 \\ 000001 \end{bmatrix}$$

⋮

coset 35 / 36:

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 101110 \\ 101000 \\ 011101 \\ 011111 \\ 100010 \\ 110100 \end{bmatrix}$$

The following command reads the vector of coset representatives from the file just created.

#### Example 223

```

coset_reps_read:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
▷ ▷ -define G2 -linear_group -PGL 6 F \
▷ ▷ ▷ -symplectic_group \
▷ ▷ -end \
▷ ▷ -define CR -vector_ge -action G2 \
▷ ▷ ▷ -read_csv \
▷ ▷ ▷ PGOp_6_2_coset_reps.csv Element \
▷ ▷ -end

```

The following command creates the group  $\mathrm{PSp}(6, 2)$  and computes a point stabilizer subgroup.

#### Example 224

```
SP_6_2.point_stab.subgroup:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 6 F \
> > > -symplectic_group \
> > -end \
> > -define G0 -modified_group -from G \
> > > -point_stabilizer 0 \
> > -end \
> > -with G0 -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_6_2.Sp_6_2.Stab0.report.tex
> $(OPEN) PGL_6_2.Sp_6_2.Stab0.report.pdf
```

The next command creates the group  $\mathrm{PGO}^+(6, 2)$  and produces a report.

#### Example 225

```
PGOp_6_2.report:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOp 6 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGOp_6_2.report.tex
> $(OPEN) PGOp_6_2.report.pdf
```

The next command creates the group  $\mathrm{PGO}^-(6, 2)$  and produces a report.

#### Example 226

```
PGOm_6_2.report:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOm 6 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGOm_6_2.report.tex
> $(OPEN) PGOm_6_2.report.pdf
```

The next command creates the group  $\text{PGO}^-(6, 2)$  and exports the group to Magma.

#### Example 227

```
PGOm_6_2.export_magma:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOm 6 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -export_magma \
> > -end
```

The next command creates the group  $\text{PGO}^+(6, 2)$  and computes a point stabilizer subgroup.

#### Example 228

```
PGOp_6_2.point_stab_subgroup:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOp 6 F -end \
> > -define G0 -modified_group -from G \
> > > -point_stabilizer 0 \
> > -end \
> > -with G0 -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGOp_6_2_report.tex
> $(OPEN) PGOp_6_2_report.pdf
```

We can save the generators of  $\text{PGO}^+(6, 2)$  from the report to a makefile variable:

#### Example 229

```
PGOp_6_2.GENS="\
1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1,0, \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0, \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,1,0, \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,1,0,0,1,0,0,1,0,0,0,1,0,0, \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1,0,1,0,0,0,0,0,1,0,0,0,0,1,0,1, \
1,0,0,0,0,0,0,1,1,1,1,1,1,0,0,0,1,0,1,0,0,0,0,1,1,0,1,0,0,0,1,0,0,1,0,0, \
1,0,0,0,0,0,0,1,0,1,0,0,1,0,1,0,0,1,0,0,0,1,0,0,0,0,0,1,1,0,0,0,0,0,0,1, \
0,1,0,1,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,1,0,1,0,0,0,0,0,1,0,0,1,1,0,0,1, \
0,0,1,1,1,1,1,0,1,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,1,0,1,0,1,0,1,0,1,1,0, \
0,0,0,0,1,0,1,0,1,1,1,1,1,0,1,0,1,0,1,0,0,1,1,0,1,0,0,0,0,0,1,1,1,1,1,0,"
```

Using this variable, we can use the following command to recreate the group as a subgroup of  $\text{PGL}(6, 2)$  and write a report:



**Example 230**

```

PGOp_6_2_linear:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PGL 6 F \
▷ ▷ ▷ -subgroup_by_generators PGOp_6_2 \
▷ ▷ ▷ ▷ 40320 10 $(PGOp_6_2_GENS) \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_6_2.Subgroup_PGOp_6_2.40320.report.tex
▷ $(OPEN) PGL_6_2.Subgroup_PGOp_6_2.40320.report.pdf

```

In the following command, we compute a point stabilizer subgroup:

**Example 231**

```

PGOp_6_2_linear_stab6:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PGL 6 F \
▷ ▷ ▷ -subgroup_by_generators PGOp_6_2 \
▷ ▷ ▷ ▷ 40320 10 $(PGOp_6_2_GENS) \
▷ ▷ -end \
▷ ▷ -define G6 -modified_group -from G \
▷ ▷ ▷ -point_stabilizer 6 \
▷ ▷ -end \
▷ ▷ -with G6 -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_6_2.Subgroup_PGOp_6_2.40320.Stab6.report.tex
▷ $(OPEN) PGL_6_2.Subgroup_PGOp_6_2.40320.Stab6.report.pdf

```

| Group Modifications                     |                 |                                                                                                                                                             |
|-----------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                 | Arguments       | Purpose                                                                                                                                                     |
| -restricted_action                      | $S$ label-tex   | restricted action on the set $S$ using the given tex-label.                                                                                                 |
| -on_k_subspaces                         | $k$             | induced action on $k$ dimensional subspaces                                                                                                                 |
| -on_k_subsets                           | $k$             | induced action on $k$ subsets.                                                                                                                              |
| -on_wedge_product                       |                 | action on the exterior square                                                                                                                               |
| -create_special_subgroup                |                 | Create the subgroup of special matrices.                                                                                                                    |
| -point_stabilizer                       | pt              | Create the subgroup which stabilizes pt.                                                                                                                    |
| -projectivity_subgroup                  |                 | Create the projectivity subgroup of a semilinear group.                                                                                                     |
| -subfield_subgroup                      | $k$             | Create the subfield subgroup w.r.t. $\mathbb{F}_s$ . Here $k = \frac{q-1}{s-1}$ .                                                                           |
| -action_on_self_by_right_multiplication |                 | Create the action on the group itself by right multiplication.                                                                                              |
| -direct_product                         | list order gens | Create the direct product action of the two actions given in list (comma separated). Then create the subgroup of the given order from the given generators. |
| -polarity_extension                     | G0 P            | Create the polarity extension of G0. P is the underlying projective space on which G0 acts.                                                                 |
| -from                                   | label           | Specify the input group action (this option can be repeated).                                                                                               |

Table 6.6: Group Modifications

## 6.4 New Actions from Old

It is possible to create new groups and group actions from old. This is done using the command `-modified_group`. Table 6.6 lists Orbiter commands that are available. Let us create the action of  $\text{Sym}(4)$  on pairs:

### Example 232

```
Symmetric_4_on_pairs:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
▷ ▷ -define G_on_2subsets -modified_group -from G \
▷ ▷ ▷ -on_k_subsets 2 \
▷ ▷ -end \
▷ ▷ -with G_on_2subsets -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex Sym_4_OnSubsets_2_report.tex
▷ $(OPEN) Sym_4_OnSubsets_2_report.pdf
```

The next example creates a group in product action. Let  $C_n$  be the cyclic group of order  $n$ . The group we wish to create is  $C_{21}$ . Because of  $C_{21} \simeq C_3 \times C_7$ , there is a product action on a grid of type  $3 \times 7$ . To create

the cyclic group, we use the translation complement in the respective one-dimensional affine groups. This works because the numbers are prime. So, we create the group as a subgroup of  $\text{AGL}(1, 3) \times \text{AGL}(1, 7)$ . Recall the encoding of affine elements: The affine map

$$f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$$

with

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

is encoded in row-major ordering, with the entries of  $\mathbf{b}$  following the entries of  $A$

$$(a_{11}, a_{12}, \dots, a_{nn}, b_1, \dots, b_n).$$

The cyclic group is generated by the mapping

$$f : \mathbb{F}_3 \times \mathbb{F}_7 \rightarrow \mathbb{F}_3 \times \mathbb{F}_7, f(x, y) = (x + 1, y + 1),$$

which encodes as

$$(1, 1, 1, 1)$$

The command to create the product action and the subgroup of order 21 is:

#### Example 233

```
direct_product_AGL:
▷ $(ORBITER) -v 4 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define G1 -linear_group -AGL 1 3 \
▷ ▷ -end \
▷ ▷ -define G2 -linear_group -AGL 1 7 \
▷ ▷ -end \
▷ ▷ -define G -modified_group -direct_product "G1,G2" \
▷ ▷ ▷ "21" "1,1,1,1" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex product_AGL_1.3_AGL_1.7_report.tex
▷ $(OPEN) product_AGL_1.3_AGL_1.7_report.pdf
```

The polarity group  $\text{PFL}^*(n, q)$  arises from  $\text{PFL}(n, q)$  by means of extending by the polarity map. The degree of extension is two. The next command creates the group  $\text{PFL}^*(3, 4)$  of order 241920:

#### Example 234

```
PGGLstar_3.4:
▷ $(ORBITER) -v 6 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
```

```

> > -define G0 -linear_group -PGGL 3 F -end \
> > -define G -modified_group -polarity_extension \
> > > G0 P \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGGL_3_4.polarity_ext.report.tex
> $(OPEN) PGGL_3_4.polarity_ext.report.pdf

```

Let us create a point-stabilizer subgroup. Continuing an example from Section 6.1, we consider the Mathieu group  $M_{12}$ , with its point stabilizer subgroup  $M_{11}$  of order 7920. To form the point stabilizer, we utilize a group modification. The following command also produces a report for the subgroup.

#### Example 235

```

Group.M11:
> $(ORBITER) -v 2 \
> > -define gens -vector -dense $(GENERATORS.M12) -end \
> > -define G -permutation_group -symmetric_group 12 \
> > > -subgroup_by_generators M12 95040 2 gens -end \
> > -define G0 -modified_group -from G \
> > > -point_stabilizer 0 \
> > -end \
> > -with G0 -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex Sym_12_Subgroup_M12_95040_Stab0_report.tex
> $(OPEN) Sym_12_Subgroup_M12_95040_Stab0_report.pdf

```

In the following example, we will demonstrate two types of induced actions. One is the action induced on  $k$ -dimensional subspaces. The second is the restricted action on an invariant subset. The example we show is related to cubic surfaces. At first, we create the Eckardt surface in  $PG(3, 13)$  from the arc

$$\{0, 1, 2, 3, 43, 113\}.$$

Then we export the set of 45 tritangent planes to file and we produce a report about the surface and its automorphism group. The next command creates the stabilizer of the surface from the generators given in the report, creates the induced action on planes, and restricts the action to the 45 tritangent planes stored in the file. The command sequence includes a makefile variable for the generators of the stabilizer of the surface:

#### Example 236

```

SURFACE_Q13_STAB_GEN="1,0,0,0,9,12,0,0,10,0,12,0,9,0,0,12, \
1,0,0,0,0,12,12,6,6,0,0,7,1,0,2,0, \
0,1,1,7,3,9,9,11,2,10,10,3,9,9,1,11"

```

**Example 237**

```

surface_q13.Eckardt_on_tritangent_planes:
▷ $(ORBITER) -v 2 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -draw_options -embedded -end \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define gens -vector -field F \
▷ ▷ ▷ -dense $(SURFACE_Q13_STAB_GENs) \
▷ ▷ -end \
▷ ▷ -define TriP -set -file \
▷ ▷ ▷ surface_arc_lifting_triherdral_q13_arc_0_1_2_3_43_113.tritangent_planes.csv
\
▷ ▷ -end \
▷ ▷ -define G -linear_group -PGL 4 F \
▷ ▷ ▷ -subgroup_by_generators "stab" \
▷ ▷ ▷ ▷ 24 3 gens \
▷ ▷ ▷ -end \
▷ ▷ -define G_on_planes -modified_group -from G \
▷ ▷ ▷ -on_k_subspaces 3 \
▷ ▷ -end \
▷ ▷ -define Gr -modified_group -from G_on_planes \
▷ ▷ ▷ -restricted_action TriP TriP \
▷ ▷ -end \
▷ ▷ -with Gr -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -define Orb -orbits -group Gr \
▷ ▷ ▷ -on_points \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -draw_tree 0 \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -draw_tree 1 \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -stabilizer 36 \
▷ ▷ -end
▷ pdflatex PGL_4_13_Gr_4_3_res_TriP_orbits_report.tex
▷ $(OPEN) PGL_4_13_Gr_4_3_res_TriP_orbits_report.pdf

```

Let us create the stabilizer of the Hirschfeld surface, isomorphic to the Weyl group of type  $E_6$ , of order 51840. The group is created as a subgroup of  $\text{PGL}(4, 4)$ . The following command computes the projectivity subgroup, of order 25920, and produces a report.

**Example 238**

```

Hirschfeld_stab_projectivity_group:
> $(ORBITER) -v 4 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define G -linear_group -PGGL 4 4 \
> > > -subgroup_by_generators "Hirschfeld.Stab" \
> > > 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
> > > -end \
> > -define G0 -modified_group -from G \
> > > -projectivity_subgroup \
> > -end \
> > -with G0 -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGGL_4.4.Subgroup.Hirschfeld.Stab.51840.ProjectivitySubgroup.report.te
x
> $(OPEN) PGGL_4.4.Subgroup.Hirschfeld.Stab.51840.ProjectivitySubgroup.report.pdf

```

In the next example, we consider the action of

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

in  $\text{PGL}(4, 8)$  on the set of planes through a fixed line 0. The Orbiter ranks of the set of planes through the line have been computed in Section 4.4. They are

$$0, 8, 1, 6, 4, 3, 7, 2, 5,$$

and are given as the first argument to the `apply` command. The second argument is the coding of the group element.

**Example 239**

```

on_planes:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 8 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define G -linear_group -PGL 4 F -end \
> > -define G_on_planes -modified_group -from G \
> > > -on_k_subspaces 3 \
> > -end \
> > -with G_on_planes -do \
> > -group_theoretic_activity \
> > > -apply "0,8,1,6,4,3,7,2,5" \
> > > "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
> > -end
> pdflatex PGL_4.8.Gr_4.3.apply.tex

```

```
▷ $(OPEN) PGL_4.8_Gr_4.3_apply.pdf
```

The apply command computes the permutation representation of the group element. In order to create the group acting on the planes, we use a restricted action. At first, we create the cyclic group of order 9 as a subgroup of  $\text{PGL}(4, 8)$ . Then we create the action on planes and restrict the action to the 9 planes through the line that we fix. Here is the command:

#### Example 240

```
on_planes_restricted:
▷ $(ORBITER) -v 2 \
▷ ▷ -define planes -vector -dense "0,8,1,6,4,3,7,2,5" -end \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define G -linear_group -PGL 4 F \
▷ ▷ ▷ -subgroup_by_generators "cyclic" \
▷ ▷ ▷ 9 1 "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
▷ ▷ ▷ -end \
▷ ▷ -define G_on_planes -modified_group -from G \
▷ ▷ ▷ -on_k_subspaces 3 \
▷ ▷ -end \
▷ ▷ -define Gr -modified_group -from G_on_planes \
▷ ▷ ▷ -restricted_action planes planes \
▷ ▷ -end \
▷ ▷ -with Gr -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_4.8_Gr_4.3_res_planes_report.tex
▷ $(OPEN) PGL_4.8_Gr_4.3_res_planes_report.pdf
```

Table 6.7 lists specific actions that can be requested when creating a linear group. Let us consider an example of a tensor product action. For instance, the command

#### Example 241

```
T3_on_tensors:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G \
▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
▷ ▷ ▷ -on_tensors -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex GL_2.2_wreath_Sym3_report.tex
▷ $(OPEN) GL_2.2_wreath_Sym3_report.pdf
▷
```

| Creating Special Actions   |           |                                                                                                                                                                                                     |
|----------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                    | Arguments | Purpose                                                                                                                                                                                             |
| -wedge_detached            |           | action on the exterior square. Unlike -wedge, this command does not establish the homomorphism to the original group. Instead, the group is created as subgroup of the larger general linear group. |
| -PGL20nConic               |           | induced action of $\text{PGL}(2, q)$ on the conic in the plane $\text{PG}(2, q)$                                                                                                                    |
| -subfield_structure_action | $s$       | action by field reduction to the subfield of index $s$                                                                                                                                              |
| -on_tensors                |           | induced action of $\text{GL}(d, q) \wr \text{Sym}(n)$ on the tensor space                                                                                                                           |
| -on_rank_one_tensors       |           | induced action of $\text{GL}(d, q) \wr \text{Sym}(n)$ on the tensor space                                                                                                                           |

Table 6.7: Creating Special Actions

creates the group  $\text{GL}(2, 2) \wr \text{Sym}(3)$  acting on the 255 elements of  $\text{PG}(7, 2)$  which are identified with the tensors of type  $(2, 2, 2)$  over  $\mathbb{F}_2$ . Elements of this group are denoted in the notation of the semidirect product. A vector of elements in the linear group is followed by a permutation of the components.

## The Group $\text{GL}(2, 2) \wr \text{Sym}(3)$

The order of the group  $\text{GL}(2, 2) \wr \text{Sym}(3)$  is 1296

The group acts on a set of size 255

## The Action

Group action  $\text{GL}(2, 2) \wr \text{Sym}(3)$ res255 of degree 255

## Base and Stabilizer Chain

Group order 1296

tl=3, 2, 1, 3, 2, 3, 2, 3, 2,

Strong generators for a group of order 1296.

$$\left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}; id \right), \left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; id \right), \left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right),$$

$$\left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \left( \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \left( \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right),$$

$$\left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; (1, 2) \right), \left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; (0, 1) \right)$$

0,1,2,1,0,0,1,1,0,0,1,1,0,1,1,

0,1,2,1,0,0,1,1,0,0,1,0,1,1,0,



```

0,1,2,1,0,0,1,1,0,1,1,0,0,1,
0,1,2,1,0,0,1,0,1,1,0,1,0,0,1,
0,1,2,1,0,1,1,1,0,0,1,1,0,0,1,
0,1,2,0,1,1,0,1,0,0,1,1,0,0,1,
0,2,1,1,0,0,1,1,0,0,1,1,0,0,1,
1,0,2,1,0,0,1,1,0,0,1,1,0,0,1,

```

### Stabilizer chain

| Level | Base pt | Orbit length | Subgroup order |
|-------|---------|--------------|----------------|
| 0     | 0       | 3            | 1296           |
| 1     | 1       | 2            | 432            |
| 2     | 2       | 1            | 216            |
| 3     | 4       | 3            | 216            |
| 4     | 5       | 2            | 72             |
| 5     | 8       | 3            | 36             |
| 6     | 9       | 2            | 12             |
| 7     | 12      | 3            | 6              |
| 8     | 13      | 2            | 2              |

It is also possible to restrict the action on all rank-one tensors, as the following example shows:

#### Example 242

```

T3r1:
> $(ORBITER) -v 4 \
> > -define G \
> > -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
> > > -on_rank_one_tensors -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex GL_2.2_wreath.Sym3_report.tex
> $(OPEN) GL_2.2_wreath.Sym3_report.pdf

```

This creates an action of degree 27. The Orbiter report shows all points in the permutation domain.

### The Group $GL(2, 2) \wr \text{Sym}(3)$

The order of the group  $GL(2, 2) \wr \text{Sym}(3)$  is 1296  
The group acts on a set of size 27

## The Action

Group action  $GL(2, 2) \wr \text{Sym}(3) \text{res} 27$  of degree 27

We act on the following set:

$$\begin{aligned}
 0 &= (1, 0, 0, 0, 0, 0, 0, 0) & 14 &= (0, 0, 0, 0, 0, 0, 1, 1) \\
 1 &= (0, 1, 0, 0, 0, 0, 0, 0) & 15 &= (0, 0, 0, 0, 1, 0, 1, 0) \\
 2 &= (1, 1, 0, 0, 0, 0, 0, 0) & 16 &= (0, 0, 0, 0, 0, 1, 0, 1) \\
 3 &= (0, 0, 1, 0, 0, 0, 0, 0) & 17 &= (0, 0, 0, 0, 1, 1, 1, 1) \\
 4 &= (0, 0, 0, 1, 0, 0, 0, 0) & 18 &= (1, 0, 0, 0, 1, 0, 0, 0) \\
 5 &= (0, 0, 1, 1, 0, 0, 0, 0) & 19 &= (0, 1, 0, 0, 0, 1, 0, 0) \\
 6 &= (1, 0, 1, 0, 0, 0, 0, 0) & 20 &= (1, 1, 0, 0, 1, 1, 0, 0) \\
 7 &= (0, 1, 0, 1, 0, 0, 0, 0) & 21 &= (0, 0, 1, 0, 0, 0, 1, 0) \\
 8 &= (1, 1, 1, 1, 0, 0, 0, 0) & 22 &= (0, 0, 0, 1, 0, 0, 0, 1) \\
 9 &= (0, 0, 0, 0, 1, 0, 0, 0) & 23 &= (0, 0, 1, 1, 0, 0, 1, 1) \\
 10 &= (0, 0, 0, 0, 0, 1, 0, 0) & 24 &= (1, 0, 1, 0, 1, 0, 1, 0) \\
 11 &= (0, 0, 0, 0, 1, 1, 0, 0) & 25 &= (0, 1, 0, 1, 0, 1, 0, 1) \\
 12 &= (0, 0, 0, 0, 0, 0, 1, 0) & 26 &= (1, 1, 1, 1, 1, 1, 1, 1) \\
 13 &= (0, 0, 0, 0, 0, 0, 0, 1)
 \end{aligned}$$

## Base and Stabilizer Chain

Group order 1296

tl=3, 2, 1, 3, 2, 3, 2, 3, 2,

Strong generators for a group of order 1296:

$$\begin{aligned}
 &\left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} ; id \right), \left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} ; id \right), \left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} ; id \right), \\
 &\left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} ; id \right), \left( \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} ; id \right), \left( \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} ; id \right), \\
 &\left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} ; (1, 2) \right), \left( \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} ; (0, 1) \right)
 \end{aligned}$$

0,1,2,1,0,0,1,1,0,0,1,1,0,1,1,  
 0,1,2,1,0,0,1,1,0,0,1,0,1,1,0,  
 0,1,2,1,0,0,1,1,0,1,1,1,0,0,1,  
 0,1,2,1,0,0,1,0,1,1,0,1,0,0,1,  
 0,1,2,1,0,1,1,1,0,0,1,1,0,0,1,  
 0,1,2,0,1,1,0,1,0,0,1,1,0,0,1,  
 0,2,1,1,0,0,1,1,0,0,1,1,0,0,1,  
 1,0,2,1,0,0,1,1,0,0,1,1,0,0,1,

Orbiter supports the induced action on the wedge product by means of a modified group command. The following command shows how the induced action of  $PGL(4, 2)$  on the wedge product is created:

### Example 243

```

PGL_4_2_wedge:
> $(ORBITER) -v 3 \

```

```

> > -define G -linear_group -PGL 4 2 -end \
> > -define G2 -modified_group -from G \
> > > -on_wedge_product \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_4.2.report.tex
> $(OPEN) PGL_4.2.report.pdf

```

This creates a matrix group of order 20160 and dimension 4. The representation is only evaluated when needed, for instance when acting on elements of the wedge product.

A different way of creating wedge actions is by applying an attribute during the creation of the group. This has the effect that the group is created in the wedge representation right away. The following example creates the representation of  $\text{PGL}(4, 2)$  in the wedge product action.

#### Example 244

```

PGL_4.2.wd:
> $(ORBITER) -v 3 \
> > -define G -linear_group -PGL 4 2 -wedge_detached -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_6.2.Wedge_4.2.detached_report.tex
> $(OPEN) PGL_6.2.Wedge_4.2.detached_report.pdf

```

This creates a matrix group of order 20160 and dimension 10.

## 6.5 Group Theoretic Activities

Once a group has been created as in Sections 6.1 and 6.2, a group theoretic activity can be performed. For this purpose, Orbiter provides the `-group_theoretic_activity` option. Tables 6.8- 6.10 lists the commands that are available. Table 6.11 lists commands for element processing.

The command

### Example 245

```
PGL_3_2_elements:
▷ $(ORBITER) -v 5 \
▷ ▷ -define G -linear_group -PGL 3 2 -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -save_elements_csv "PGL_3_2_elements.csv" \
▷ ▷ -end
```

creates all elements of  $\text{PGL}(3, 2)$  and writes them into the file `PGL_3_2_elements.csv`.

The command

### Example 246

```
Sym_3_elements:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -print_elements_tex \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_options \
▷ ▷ ▷ -nodes \
▷ ▷ ▷ -embedded -radius 250 \
▷ ▷ ▷ -xin 10000 -yin 10000 \
▷ ▷ ▷ -xout 1000000 -yout 600000 \
▷ ▷ ▷ -scale 0.3 -line-width 1.0 \
▷ ▷ -end \
▷ ▷ -tree_draw -file Sym_3_elements_tree.txt -end
▷ pdflatex Sym_3_elements_tree_draw.tex
▷ $(OPEN) Sym_3_elements_tree_draw.pdf
```

creates a tree of the elements of  $\text{Sym}(3)$ :

| Group theoretic activities (Part 1) |            |                                                                                                                                   |
|-------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Command                             | Arguments  | Purpose                                                                                                                           |
| -apply                              | $a\ s$     | Applies the group element given by the coded vector $s$ to the element $a$ .                                                      |
| -element_processing                 | options    | Perform an activity for a sequence of group elements. For options, see Table 6.11.                                                |
| -multiply                           | $s_1\ s_2$ | Multiplies group elements $s_1$ and $s_2$ , assuming the elements are given in coded form. Produces a latex report.               |
| -inverse                            | $s$        | Computes the inverse of $s$ , which is given in coded form. Produces a latex report.                                              |
| -consecutive_powers                 | $s\ n$     | Computes all powers $s^i$ for $i = 1, \dots, n$ . $s$ is given in coded form. Produces a latex report.                            |
| -raise_to_the_power                 | $s\ n$     | Computes the $n$ -th power of $s$ , which is given in coded form. Produces a latex report.                                        |
| -export_orbiter                     |            | Exports the group to Orbiter.                                                                                                     |
| -export_gap                         |            | Exports the group to GAP [28].                                                                                                    |
| -export_magma                       |            | Exports the group to Magma [16].                                                                                                  |
| -canonical_image_GAP                | $S$        | Compute the canonical image of the set $S$ using GAP.                                                                             |
| -canonical_image                    | $S$        | Compute the canonical image of the set $S$ using Orbiter.                                                                         |
| -search_element_of_order            | $i$        | Finds all elements of order $i$ in the group ( $i \in \mathbb{N}$ ).                                                              |
| -find_standard_generators           | $a\ b\ c$  | Finds all pairs of elements $x, y$ in $G$ such that $ x =a$ , $ y =b$ , and $ xy =c$ . Such pairs are called standard generators. |
| -random_element                     | label      | Creates a random element. Writes the element to file in coded form.                                                               |
| -element_rank                       | $s$        | Determines the rank of the group element $s$ in the given group. $s$ is given in coded form.                                      |
| -element_unrank                     | $r$        | Produces the group element whose rank is $r$ .                                                                                    |
| -find_singer_cycle                  |            | Finds all Singer cycles whose matrix is a companion matrix.                                                                       |
| -classes_based_on_normal_form       |            |                                                                                                                                   |
| -normalizer                         |            | See Section 6.6.                                                                                                                  |
| -centralizer_of_element             |            | See Section 6.6.                                                                                                                  |

Table 6.8: Group theoretic activities (Part 1)

| Group theoretic activities (Part 2)         |            |                                                                                                                                                           |
|---------------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                     | Arguments  | Purpose                                                                                                                                                   |
| -permutation_representation_of_element      | $g$        | Compute the permutation representation of $g$                                                                                                             |
| -orbits_on_group_elements_under_conjugation |            |                                                                                                                                                           |
| -normalizer_of_cyclic_subgroup              |            | See Section 6.6.                                                                                                                                          |
| -classes                                    |            | See Section 6.6.                                                                                                                                          |
| -report                                     |            | Produce a latex report for the group.                                                                                                                     |
| -report_sylow                               |            | Include Sylow subgroups in the report (requires -report).                                                                                                 |
| -report_group_table                         |            | Include the group table in the report (requires -report).                                                                                                 |
| -report_classes                             |            | Include the conjugacy classes in the report (requires -report).                                                                                           |
| -export_group_table                         |            | Exports the group table as csv-file.                                                                                                                      |
| -conjugacy_class_of                         | $g$        | Compute conjugacy class of $g$ .                                                                                                                          |
| -isomorphism_Klein_quadric                  | fname      |                                                                                                                                                           |
| -print_elements                             |            | Produces a printout of all group elements.                                                                                                                |
| -print_elements_tex                         |            | Produces a latex report of all group elements.                                                                                                            |
| -print_given_elements_tex                   | label file | Produces a latex report of all group elements in file. The latex file will be named label.tex. The group elements are given in coded (make-element) form. |
| -save_elements_csv                          | fname      | Writes the elements of the group to the given csv file.                                                                                                   |
| -export_inversion_graphs                    | fname      | Exports the inversion graphs associated to the group elements to the given file.                                                                          |

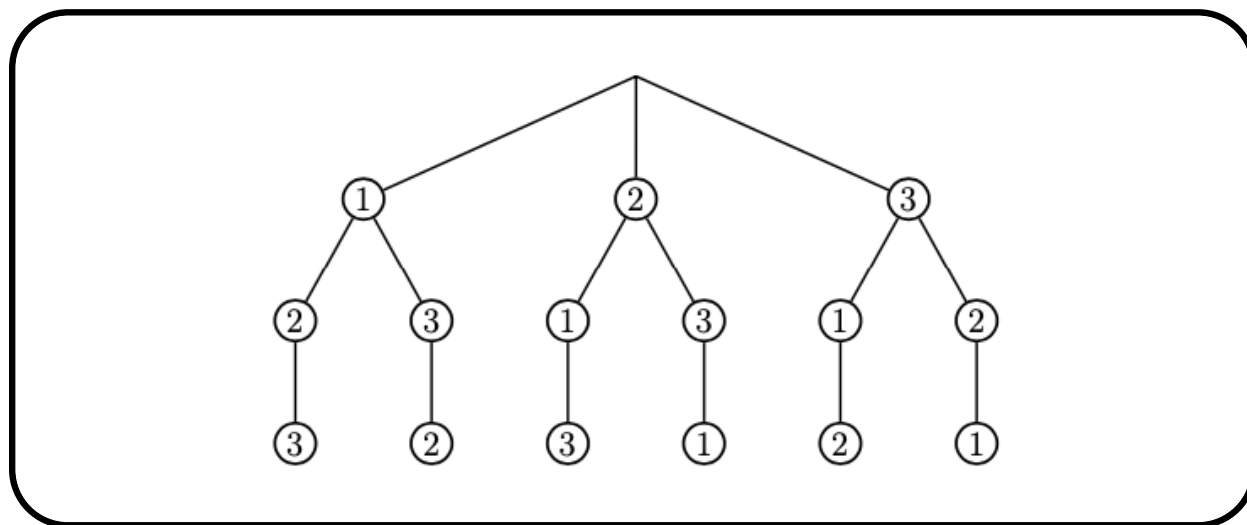
Table 6.9: Group theoretic activities (Part 2)

| Group theoretic activities (Part 3)          |                           |                                                                                                                                                                                                                                             |
|----------------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                      | Arguments                 | Purpose                                                                                                                                                                                                                                     |
| -multiply_elements_csv_column_major_ordering | f1 f2 f3                  |                                                                                                                                                                                                                                             |
| -multiply_elements_csv_row_major_ordering    | f1 f2 f3                  |                                                                                                                                                                                                                                             |
| -apply_elements_csv_to_set                   | f1 f2 S                   |                                                                                                                                                                                                                                             |
| -order_of_products                           | $g_1 \dots g_n$           | Creates a table of the orders of all products $g_i g_j$ , $1 \leq i, j \leq n$ .                                                                                                                                                            |
| -reverse_isomorphism_exterior_square         |                           | Given a set of generators of a subgroup of $\text{PGO}^+(6, q)$ as $6 \times 6$ matrixes, compute the inverse image of the generators in $\text{PGL}(4, q)$ (if possible).                                                                  |
| -is_subgroup_of                              |                           |                                                                                                                                                                                                                                             |
| -coset_reps                                  |                           |                                                                                                                                                                                                                                             |
| -evaluate_word                               | word gens                 |                                                                                                                                                                                                                                             |
| -multiply_all_elements_in_lex_order          |                           | Multiplies all group elements together. The main purpose is for speed-testing.                                                                                                                                                              |
| -stats                                       | prefix                    | Report statistics on group operations.                                                                                                                                                                                                      |
| -move_a_b                                    | a b                       | Determine a group element mapping a to b. Computes the stabilizer of b in the given group $G$ (the group for which the activity is applied).                                                                                                |
| -orbit_of                                    | pt                        |                                                                                                                                                                                                                                             |
| -orbits_on_set_system_from_file              | f c n                     |                                                                                                                                                                                                                                             |
| -orbit_of_set_from_file                      | fname                     |                                                                                                                                                                                                                                             |
| -linear_codes                                | control $d$<br>$n_{\max}$ | Classify linear codes with prescribed minimum distance $d$ . Assumes that the group is $\text{PGL}(r, q)$ or $\text{PTL}(r, q)$ . For each $n \leq n_{\max}$ , the $[n, k, \geq d]$ codes are classified with $n - k = r$ . See Section 10. |
| -tensor_permutations                         |                           | Computes the permutation representation of generators of the wreath product.                                                                                                                                                                |
| -classify_ovoids                             | descr                     |                                                                                                                                                                                                                                             |
| -representation_on_polynomials               | R                         | Compute the representation in the action on homogeneous polynomials as defined in the ring R.                                                                                                                                               |

Table 6.10: Group theoretic activities (Part 3)

| Element Processing Options            |           |                                                              |
|---------------------------------------|-----------|--------------------------------------------------------------|
| Command                               | Arguments | Purpose                                                      |
| -input                                | label     | Specify the input source for a sequence of group elements.   |
| -print                                |           | Print each group element.                                    |
| -apply_isomorphism_wedge_product_4to6 |           | Applies a specific isomorphism from $GL(4, q)$ to $GL(6, q)$ |
| -with_permutation                     |           | Print with permutation representation.                       |
| -with_fix_structure                   |           | Print with fix structure on the underlying projective space. |
| -order_of_products_of_pairs           |           | Make a table of the order of all pairwise products.          |
| -conjugate                            | data      | Conjugate each group element by the element in data.         |
| -print_action_on_surface              | surface   | Print the action on a cubic surface object.                  |

Table 6.11: Element Processing Options



It is possible to compute all powers of a fixed element, as in the following command:

#### Example 247

```

Cycle_12_power:
> $(ORBITER) -v 5 \
> > -define G -permutation_group -symmetric_group 12 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -consecutive_powers \
> > > "1,2,3,4,5,6,7,8,9,10,11,0" 12 \
> > -end
> pdflatex Sym_12_all_powers.tex
> $(OPEN) Sym_12_all_powers.pdf
>

```



We create the 12 powers of the cycle

$$(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11).$$

The output is

| $i$ | $(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)^i$ |
|-----|--------------------------------------------|
| 1   | (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)     |
| 2   | (0, 2, 4, 6, 8, 10)(1, 3, 5, 7, 9, 11)     |
| 3   | (0, 3, 6, 9)(1, 4, 7, 10)(2, 5, 8, 11)     |
| 4   | (0, 4, 8)(1, 5, 9)(2, 6, 10)(3, 7, 11)     |
| 5   | (0, 5, 10, 3, 8, 1, 6, 11, 4, 9, 2, 7)     |
| 6   | (0, 6)(1, 7)(2, 8)(3, 9)(4, 10)(5, 11)     |
| 7   | (0, 7, 2, 9, 4, 11, 6, 1, 8, 3, 10, 5)     |
| 8   | (0, 8, 4)(1, 9, 5)(2, 10, 6)(3, 11, 7)     |
| 9   | (0, 9, 6, 3)(1, 10, 7, 4)(2, 11, 8, 5)     |
| 10  | (0, 10, 8, 6, 4, 2)(1, 11, 9, 7, 5, 3)     |
| 11  | (0, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1)     |
| 12  | id                                         |

The command

#### Example 248

```
PGL_3_4_singer:
> $(ORBITER) -v 5 \
> > -define G -linear_group -PGL 3 4 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -find_singer_cycle \
> > -end
```

finds all Singer cycles in  $\text{PGL}(3, 4)$  whose matrix is the companion matrix of a polynomial. The first one found is

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 2 & 3 & 2 \end{bmatrix}$$

whose projective order is 21. Here, we are using the numeric form of field elements, so 2 is  $\omega$  and 3 is  $\omega + 1$ .

Suppose we want to multiply two elements in a group. The following command shows an example in  $\text{GL}(2, 8)$ . We multiply the elements coded by 0,1,2,3 and 4,5,6,7:

**Example 249**

```

GL_2.8_multiply:
> $(ORBITER) -v 5 \
> > -define G -linear_group -GL 2 8 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -multiply "0,1,2,3" "4,5,6,7" \
> > -end
> pdflatex GL_2.8_mult.tex
> $(OPEN) GL_2.8_mult.pdf

```

The output is

$$\begin{bmatrix} 0 & 1 \\ \gamma & \gamma^5 \end{bmatrix} \cdot \begin{bmatrix} \gamma^2 & \gamma^3 \\ \gamma^6 & \gamma^4 \end{bmatrix} = \begin{bmatrix} \gamma^6 & \gamma^4 \\ \gamma & \gamma^5 \end{bmatrix}$$

0,1,2,3,  
4,5,6,7,  
6,7,2,3,

Note that the output shows the codings of the three group elements. This way, the result of this computation can be processed further easily. The same example over  $\mathbb{F}_7$ , noting that  $7 \equiv 0 \pmod{7}$  is:

**Example 250**

```

GL_2.7_multiply:
> $(ORBITER) -v 5 \
> > -define G -linear_group -GL 2 7 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -multiply "0,1,2,3" "4,5,6,0" \
> > -end
> pdflatex GL_2.7_mult.tex
> $(OPEN) GL_2.7_mult.pdf

```

The output is

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 4 & 5 \\ 6 & 0 \end{bmatrix} = \begin{bmatrix} 6 & 0 \\ 5 & 3 \end{bmatrix}$$

0,1,2,3,  
4,5,6,0,  
6,0,5,3,

We can compute the inverse of a group element:

**Example 251**

```

GL_2_7_inv:
> $(ORBITER) -v 5 \
> > -define G -linear_group -GL 2 7 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -inverse "0,1,2,3" \
> > -end
> pdflatex GL_2_7_inv.tex
> $(OPEN) GL_2_7_inv.pdf

```

The output is

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}^{-1} = \begin{bmatrix} 2 & 4 \\ 1 & 0 \end{bmatrix}$$

0,1,2,3,  
2,4,1,0,

We can raise a group element to a power:

**Example 252**

```

GL_2_7_power:
> $(ORBITER) -v 5 \
> > -define G -linear_group -GL 2 7 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -raise_to_the_power "0,1,2,3" 2 \
> > -end
> pdflatex GL_2_7_power.tex
> $(OPEN) GL_2_7_power.pdf

```

The output is

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}^2 = \begin{bmatrix} 2 & 3 \\ 6 & 4 \end{bmatrix}$$

0,1,2,3,  
2,3,6,4,

The next example computes the action of a specific group element on the set of planes through a line. The planes have been computed in Section 4.4.

**Example 253**

```

on_planes:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 8 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define G -linear_group -PGL 4 F -end \
> > -define G_on_planes -modified_group -from G \
> > > -on_k_subspaces 3 \
> > -end \
> > -with G_on_planes -do \
> > -group_theoretic_activity \
> > > -apply "0,8,1,6,4,3,7,2,5" \
> > > "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
> > -end
> pdflatex PGL_4.8_Gr_4.3_apply.tex
> $(OPEN) PGL_4.8_Gr_4.3_apply.pdf

```

The output is

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \gamma \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1000 \\ 0100 \\ 0002 \\ 0011 \end{bmatrix}$$

1,0,0,0,0,1,0,0,0,0,2,0,0,1,1,

maps:

$0 \mapsto 8$

$8 \mapsto 1$

$1 \mapsto 3$

$6 \mapsto 5$

$4 \mapsto 7$

$3 \mapsto 4$

$7 \mapsto 6$

$2 \mapsto 0$

$5 \mapsto 2$

The command `-random_element` can be used to create a random element in a group. Here is an example. We create the group  $\text{PGL}(4, 2)$  and then ask for a random element:

**Example 254**

```

PGL_4.2_random_element:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 4 F -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -random_element r1 \

```

```
▷ ▷ -end \  
▷ -print_symbols
```

| Group Theoretic Activities Based on Magma |              |                                                                                 |
|-------------------------------------------|--------------|---------------------------------------------------------------------------------|
| Command                                   | Arguments    | Purpose                                                                         |
| -classes                                  |              | Compute a report of the conjugacy classes of elements.                          |
| -centralizer_of_element                   | label coding | Compute the centralizer of the group element, using label to create file names. |
| -normalizer_of_cyclic_subgroup            | label $s$    | Compute the normalizer of the cyclic subgroup generated by the element $s$ .    |
| -normalizer                               |              | Compute the normalizer of a subgroup in the larger group.                       |

Table 6.12: Group Theoretic Activities Based on Magma

## 6.6 Group Theoretic Activities Based on Magma

Orbiter has an interface to Magma [16]. This allows Orbiter commands to utilize Magma for specialized (and often difficult) computations in Group theory. Applications are computing conjugacy classes in groups, normalizers and other things. By using these commands through Orbiter, the tasks can be performed for and kind of Orbiter group, including matrix groups. This adds functionality that does not exist in Magma. We will see examples below.

Table 6.12 lists the group theoretic commands in Orbiter that rely on Magma. The actual computation of conjugacy classes happens inside Magma, using a permutation representation of the group. The class representatives and centralizer groups are presented in Orbiter as matrix groups, using the latex interface to display the result. The communication between Magma and Orbiter is based on files. Orbiter writes a file that is used as Magma input. Magma produces a file that is then read by Orbiter.

For instance, the three-step command sequence

### Example 255

```
PGGL_2_4.classes:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G \
▷ ▷ -linear_group -PGGL 2 4 \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group.theoretic.activity \
▷ ▷ ▷ -classes \
▷ ▷ -end
▷ $(MAGMA_PATH)magma PGGL_2_4.classes.magma
▷ $(ORBITER) -v 3 \
▷ ▷ -define G \
▷ ▷ -linear_group -PGGL 2 4 \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group.theoretic.activity \
▷ ▷ ▷ -classes \
▷ ▷ -end
▷ pdflatex PGGL_2_4.classes_out.tex
▷ $(OPEN) PGGL_2_4.classes_out.pdf
```

```
▷ $(OPEN) PGGL_2_4_classes_out.csv
```

computes the classes of elements in  $\text{PGL}(2,4)$ . The first Orbiter command produces the file `PGGL_2_4_classes.magma`. The magma command reads this file and produces the file `PGGL_2_4_classes_out.txt`. The second Orbiter command reads the file `PGGL_2_4_classes_out.txt` and produces the latex report `PGGL_2_4_classes_out.tex`.

The report produced by Orbiter is too long to be reproduced here fully. Let us look at just one conjugacy class. Here is the output for class 1 / 7 (numbering starts from 0, so this is the second class):

Order of element = 2  
 Class size = 10  
 Centralizer order = 12  
 Normalizer order = 12  
 Representing element is

$$c_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}_1$$

of order 2 and with 3 fixed points. 0, 1, 1, 0, 1,

The normalizer is generated by:

Strong generators for a group of order 12:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}_1, \begin{bmatrix} \omega^2 & 0 \\ 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}_1$$

1,0,0,1,1,  
 1,0,0,2,1,  
 0,1,1,0,1,

The command sequence

#### Example 256

```
PGGL_2_4.cent.2A:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G \
▷ ▷ -linear_group -PGGL 2 4 -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ $(MAGMA_PATH)magma element_2A.centralizer.magma
▷ $(ORBITER) -v 6 \
▷ ▷ -define G \
▷ ▷ -linear_group -PGGL 2 4 -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
```

```

> > > -centralizer_of_element "2A" "1,0, 0,1, 1" \
> > > -report \
> > -end
> pdflatex PGGL_2_4_elt_2A_centralizer.tex
> $(OPEN) PGGL_2_4_elt_2A_centralizer.pdf
>

```

computes the centralizer of the Baer involution

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1.$$

The centralizer is a group of order 40320, isomorphic to  $\mathrm{PGL}(4,2).Z_2$ . Orbiter produces a list of strong generators, shown below:

Strong generators for a group of order 40320:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,  
1,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,1,  
1,0,0,0,0,1,0,0,0,0,1,0,1,1,1,0,  
1,0,0,0,0,1,0,0,0,0,1,0,1,1,0,1,0,  
1,0,0,0,0,1,0,0,1,1,0,1,1,1,0,0,  
1,0,0,0,0,1,0,0,0,0,1,1,1,1,0,0,  
1,0,0,0,0,1,0,0,1,1,1,0,0,0,1,0,  
1,0,0,0,1,1,1,1,0,1,0,0,1,0,0,0,  
0,1,0,1,1,1,1,1,1,0,0,1,0,1,1,0,

The end of the report has a list of generators in coded form. This list can be used to create the centralizer in Orbiter.

The command



**Example 257**

```

Hirschfeld_stab_classes:
> $(ORBITER) -v 9 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define G -linear_group -PGGL 4 4 \
> > > -subgroup_by_generators "Hirschfeld.Stab" \
> > > 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
> > > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -classes \
> > -end
> $(ORBITER) -v 9 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define G -linear_group -PGGL 4 4 \
> > > -subgroup_by_generators "Hirschfeld.Stab" \
> > > 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
> > > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -classes \
> > -end
> pdflatex PGGL_4_4_Subgroup_Hirschfeld.Stab_51840_classes_out.tex
> $(OPEN) PGGL_4_4_Subgroup_Hirschfeld.Stab_51840_classes_out.pdf

```

computes the conjugacy classes of the Weyl group of type  $E_6$ . The group is the stabilizer of the Hirschfeld surface (see Section 8.2). The generators have been stored in a makefile variable. They have been computed with the commands described in Section 16.2.

Orbiter can compute the normalizer of a subgroup. The group must be constructed as a subgroup  $H$  of a larger group  $G$  containing  $H$ . Typically, the group  $G$  is the group in which  $H$  is generated as a subgroup (either the full linear group of the full symmetric group). Then, the normalizer of  $H$  in  $G$  is computed. Here is an example in a symmetric group. We first create a subgroup of order 5, using a makefile variable. The generator is

$$(0, 1, 2, 3, 4)(5, 6, 7, 8, 9)(10)(11)(12).$$

We store it in a makefile variable:

**Example 258**

```

GENERATORS_H5="1,2,3,4,0,6,7,8,9,5,10,11,12"
# (0, 1, 2, 3, 4)(5, 6, 7, 8, 9)

```

The command

**Example 259**

```

Normalizer_of_H5:
> $(ORBITER) -v 2 \
> > -define G -permutation_group -symmetric_group 13 \
> > > -subgroup_by_generators H5 5 1 \

```

```

▷ ▷ ▷ ▷ $(GENERATORS_H5) -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -normalizer \
▷ ▷ -end
▷ pdflatex Sym_13_Subgroup_H5_5_normalizer.tex
▷ $(OPEN) Sym_13_Subgroup_H5_5_normalizer.pdf

```

computes the normalizer of  $H$  inside  $\text{Sym}(13)$ . The normalizer is a group of order 1200. Because of the way in which Orbiter and Magma collaborate, the command has to be executed twice. After the first execution, a magma session is started. The magma session has to be terminated by typing

quit;

The Orbiter command has to be run one more time after that. The following report is produced:

The group *Perm13SubgroupH5order5* of order 5 is:

Strong generators for a group of order 5:

$$(0, 1, 2, 3, 4)(5, 6, 7, 8, 9)$$

1, 2, 3, 4, 0, 6, 7, 8, 9, 5, 10, 11, 12,

Inside the group of order 6227020800, the normalizer has order 1200:

Strong generators for a group of order 1200:

$$(11, 12),$$

$$(10, 11),$$

$$(5, 9, 8, 7, 6),$$

$$(1, 2, 4, 3)(6, 7, 9, 8),$$

$$(0, 5)(1, 9)(2, 8)(3, 7)(4, 6)$$

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 11,

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 10, 12,

0, 1, 2, 3, 4, 9, 5, 6, 7, 8, 10, 11, 12,

0, 2, 4, 1, 3, 5, 7, 9, 6, 8, 10, 11, 12,

5, 9, 8, 7, 6, 0, 4, 3, 2, 1, 10, 11, 12,

Consider this example of a subgroup which is not cyclic: The group

$$H = \left\langle \begin{bmatrix} \alpha^4 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \right\rangle \simeq C_2 \times C_2$$

is a subgroup of  $G = \text{PGL}(2, 9)$ . To compute the normalizer of  $H$  in  $G$ , the following command sequence can be used:

**Example 260**

Normalizer\_of\_Z22\_in\_PGL\_2\_9:

```

> $(ORBITER) -v 2 \
> > -define G -linear_group -PGL 2 9 \
> > -subgroup_by_generators Z22 4 2 \
> > > "2,0,0,1, 0,1,1,0" -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -normalizer \
> > -end
> pdflatex PGL_2_9_Subgroup_Z22_4_normalizer.tex
> $(OPEN) PGL_2_9_Subgroup_Z22_4_normalizer.pdf

```

It produces a report showing that the normalizer is a group of order 24 (it is isomorphic to  $\text{Sym}(4)$ , though the report does not tell us this fact directly):

The group  $\text{PGL}(2, 9)\text{SubgroupZ22order4}$  of order 4 is:

Strong generators for a group of order 4:

$$\begin{bmatrix} \alpha^4 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

1,0,0,2,

0,1,1,0,

Inside the group of order 720, the normalizer has order 24:

Strong generators for a group of order 24:

$$\begin{bmatrix} \alpha^4 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} \alpha^2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} \alpha^4 & \alpha^4 \\ \alpha^4 & 1 \end{bmatrix},$$

$$\begin{bmatrix} \alpha^4 & \alpha^6 \\ \alpha^2 & 1 \end{bmatrix}$$

1,0,0,2,

1,0,0,5,

1,1,1,2,

1,7,5,2,

The command

**Example 261**

PGOm\_6\_2\_classes:

```

> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGOm 6 F -end \
> > -with G -do \

```

```

▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -classes \
▷ ▷ -end
▷ pdflatex PG0m_6_2_classes_out.tex
▷ $(OPEN) PG0m_6_2_classes_out.pdf

```

computes the conjugacy classes of  $\text{PGO}^-(6, 2)$ . The command has to be executed twice. Let us look at the class 2A of size 36. We store the representative as a makefile variable:

#### Example 262

```

PGOM_6_2_CLASS_2A_REP="1,0,0,0,0,0,0,1,0,0,\
0,0,0,0,1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0"

```

To create the whole conjugacy class, we use the following command:

#### Example 263

```

PG0m_6_2_Class_2A:
▷ $(ORBITER) -v 9 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PG0m 6 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -conjugacy_class_of "2A" $(PGOM_6_2_CLASS_2A_REP) \
▷ ▷ -end

```

The elements are stored in a csv file. We use a makefile variable to store the file name:

#### Example 264

```

FILE_NAME_PGOM_6_2_CLASS_2A="PG0m_6_2_class_of_2A.csv"

```

The following command prints all group elements in the class:

#### Example 265

```

PG0m_6_2_Class_2A.print:
▷ $(ORBITER) -v 9 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define PG0m_6_2_Class2A -vector \
▷ ▷ ▷ -file $(FILE_NAME_PGOM_6_2_CLASS_2A) -end \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PG0m 6 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -element_processing \

```

```

▷ ▷ ▷ ▷ -input PG0m_6_2_Class2A \
▷ ▷ ▷ ▷ -print \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex PG0m_6_2_Class2A_elements.tex
▷ $(OPEN) PG0m_6_2_Class2A_elements.pdf

```

We decide to pick group element 8 as our favorite class representative. We store the element in a makefile variable and compute the centralizer:

#### Example 266

```

PGOM_6_2_CLASS_2A_REP_CLEAN="1,0,0,0,0,0,0,1,0,0,\
0,0,0,0,0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1"

```

#### Example 267

```

PG0m_6_2_Class_2A.centralizer:
▷ $(ORBITER) -v 9 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PG0m 6 F -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -centralizer_of_element \
▷ ▷ ▷ ▷ "2Arep" $(PGOM_6_2_CLASS_2A_REP_CLEAN) \
▷ ▷ -end
▷ pdflatex PG0m_6_2_elt_2Arep_centralizer.tex
▷ $(OPEN) PG0m_6_2_elt_2Arep_centralizer.pdf

```

We store the generators for the centralizer in a makefile variable.

#### Example 268

```

PGOM_6_2_CLASS_2A_CENTRALIZER_GEN=" \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1,1, \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0, \
1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1, \
1,0,0,0,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0,0,1,1,0,0,0,0,0,1, \
1,0,0,0,0,0,0,1,1,0,0,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,1,0,0,0,1,1, \
1,0,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1, \
0,1,1,1,1,0,1,0,1,1,1,0,1,1,0,1,1,0,1,1,1,0,1,0,0,0,0,0,1,0,1,1,1,1,1,1, \
1,1,0,0,0,1,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,1,0,0,1,1,0,0,0,0,0,1, \
0,1,0,0,0,0,1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1"

```

We recompute the centralizer as a subgroup of  $\text{PGL}(6, 2)$ :

**Example 269**

```
PGOm_6_2.Class_2A.centralizer_recover:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define Class2A_cent_gens -vector \
> > > -dense $(PGOM_6_2.CLASS_2A.CENTRALIZER_GENS) -end \
> > -define G -linear_group -PGL 6 2 \
> > > -subgroup_by_generators "2ACent" \
> > > 1440 9 Class2A_cent_gens \
> > > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_6_2.Subgroup_2ACent_1440_report.tex
> $(OPEN) PGL_6_2.Subgroup_2ACent_1440_report.pdf
```

| Group Theoretic Activities Based on GAP |           |                                                                    |
|-----------------------------------------|-----------|--------------------------------------------------------------------|
| Command                                 | Arguments | Purpose                                                            |
| -canonical_image                        | set       | Compute the canonical image of the given set under the group $G$ . |
| -export_gap                             |           | Export the group $G$ to GAP [28] / Fining [5].                     |

Table 6.13: Group Theoretic Activities Based on GAP

## 6.7 Group Theoretic Activities Based on GAP

Orbiter has an interface to GAP [28]. This allows Orbiter commands to utilize GAP specific functionality. One application is the computation of canonical images of sets in the context of permutation groups. This is facilitated by means of the GAP package images [37].

Table 6.13 lists the group theoretic commands in Orbiter that rely on GAP.

How can we export a group from Orbiter to GAP? For semilinear matrix groups, we need to export to the GAP package Fining [5]. This is because semilinear groups are not available in GAP. Here is an example. We create the stabilizer of the Hirschfeld surface in  $\text{PG}(3, 4)$  and export the group to GAP/Finning as a group of collineations.

### Example 270

```
Hirschfeld_stab.export_gap:
▷ $(ORBITER) -v 9 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define G -linear_group -PGGL 4 4 \
▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
▷ ▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -export_gap \
▷ ▷ -end
```

The command creates a GAP/Finning file shown below:

```
LoadPackage("fining");
mat1 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^0,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^0,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^0]];
frob1 := FrobeniusAutomorphism(GF(4))^1;
psi1 := ProjectiveSemilinearMap(mat1, frob1,GF(4));
mat2 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^0]];
frob2 := FrobeniusAutomorphism(GF(4))^0;
psi2 := ProjectiveSemilinearMap(mat2, frob2,GF(4));
```

```

mat3 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), Z(4)^2, 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), Z(4)^2, 0*Z(4)],
[Z(4)^0, 0*Z(4), 0*Z(4), Z(4)^0]];
frob3 := FrobeniusAutomorphism(GF(4))^0;
psi3 := ProjectiveSemilinearMap(mat3, frob3, GF(4));
mat4 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[Z(4)^0, Z(4)^0, Z(4)^0, 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), Z(4)^0]];
frob4 := FrobeniusAutomorphism(GF(4))^1;
psi4 := ProjectiveSemilinearMap(mat4, frob4, GF(4));
mat5 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), Z(4)^0, 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), 0*Z(4), Z(4)^0]];
frob5 := FrobeniusAutomorphism(GF(4))^0;
psi5 := ProjectiveSemilinearMap(mat5, frob5, GF(4));
mat6 := [[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), 0*Z(4), Z(4)^0],
[0*Z(4), 0*Z(4), Z(4)^0, 0*Z(4)]];
frob6 := FrobeniusAutomorphism(GF(4))^0;
psi6 := ProjectiveSemilinearMap(mat6, frob6, GF(4));
gens := [psi1, psi2, psi3, psi4, psi5, psi6];
G := Group(gens);
Size(G);

```

When we load this file into GAP, we see the following output (for printing purposes, it has been shortened and reformatted slightly):

```

gap> LoadPackage("fining");
true
gap> mat1 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
> [0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
> [0*Z(4), 0*Z(4), Z(4)^0, 0*Z(4)],
> [0*Z(4), 0*Z(4), 0*Z(4), Z(4)^0]];
[ [ Z(2)^0, 0*Z(2), 0*Z(2), 0*Z(2) ],
[ 0*Z(2), Z(2)^0, 0*Z(2), 0*Z(2) ],
[ 0*Z(2), 0*Z(2), Z(2)^0, 0*Z(2) ],
[ 0*Z(2), 0*Z(2), 0*Z(2), Z(2)^0 ] ]
gap> frob1 := FrobeniusAutomorphism(GF(4))^1;
FrobeniusAutomorphism( GF(2^2) )
gap> psi1 := ProjectiveSemilinearMap(mat1, frob1, GF(4));
< a collineation: <cmat 4x4 over GF(2,2)>, F^2>
...
gap> gens := [psi1, psi2, psi3, psi4, psi5, psi6];
[ < a collineation: <cmat 4x4 over GF(2,2)>, F^2>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^0>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^0>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^2>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^0>,

```



```

< a collineation: <cmat 4x4 over GF(2,2)>, F^0> ]
gap> G := Group(gens);
<projective collineation group with 6 generators>
gap> Size(G);
51840
gap>

```

Suppose we want to compute the least image of the set  $\{3, 5\}$  in the permutation group  $C_6$ . The following command sets up the group  $C_6$  and invokes the Orbiter `-canonical_image` command.

**Example 271**

Cyclic\_6\_canonical\_image:

```

> $(ORBITER) -v 3 \
> > -define G -permutation_group -cyclic_group 6 -end \
> > -with G -do -group_theoretic_activity \
> > > -canonical_image_GAP 3,5 \
> > -end

```

The command creates a GAP file shown below:

```

G := Group([(1, 2, 3, 4, 5, 6)]);
LoadPackage("images");
MinimalImage(G, [4,6], OnSets);

```

Running this command in GAP yields:

```

gap> MinimalImage(G, [4,6], OnSets);
[ 1, 3 ]

```

Note that the permutation representations in GAP are 1-based, while Orbiter uses 0-based representations. So, the least image of  $\{1, 3\}$  in GAP really corresponds to the set  $\{0, 2\}$  in Orbiter.

Let us try a larger example. We want to compute the canonical form of the Edge curve by means of their rational points over  $\mathbb{F}_{17}$ . We use the following makefile variable to specify the set of rational points:

**Example 272**

```
EDGE_CURVE_Q17_AS_POINTS="4, 7, 16, 19, 20, 23, 32, 35, 89, 100, 244, 251"
```

The next command creates  $\text{PGL}(3, 17)$  and invokes the `-canonical_image` image command.

**Example 273**

Edge\_curve.q17\_canonical\_image:

```

> $(ORBITER) -v 3 \
> > -define G -linear_group -PGL 3 17 -end \

```

```

> > -with G -do -group-theoretic-activity \
> > > -canonical_image_GAP $(EDGE_CURVE_Q17_AS_POINTS) \
> > -end

```

The command creates a GAP input file called PGL\_3\_17\_canonical\_image.gap.

```

G := Group([(4, 39, 45, 46, 49, 41, 51, 47, 52, 50, 44, 43, 40, 48, 38, 42)(5,
7, 13, 14, 17, 9, 19, 15, 20, 18, 12, 11, 8, 16, 6, 10)(21, 23, 29, 30, 33,
25, 35, 31, 36, 34, 28, 27, 24, 32, 22, 26)(54, 56, 62, 63, 66, 58, 68, 64,
69, 67, 61, 60, 57, 65, 55, 59)(71, 73, 79, 80, 83, 75, 85, 81, 86, 84, 78,
77, 74, 82, 72, 76)(88, 90, 96, 97, 100, 92, 102, 98, 103, 101, 95, 94, 91,
99, 89, 93)(105, 107, 113, 114, 117, 109, 119, 115, 120, 118, 112, 111, 108,
116, 106, 110)(122, 124, 130, 131, 134, 126, 136, 132, 137, 135, 129, 128,
125, 133, 123, 127)(139, 141, 147, 148, 151, 143, 153, 149, 154, 152, 146,
145, 142, 150, 140, 144)(156, 158, 164, 165, 168, 160, 170, 166, 171, 169,
163, 162, 159, 167, 157, 161)(173, 175, 181, 182, 185, 177, 187, 183, 188,
186, 180, 179, 176, 184, 174, 178)(190, 192, 198, 199, 202, 194, 204, 200,
205, 203, 197, 196, 193, 201, 191, 195)(207, 209, 215, 216, 219, 211, 221,
217, 222, 220, 214, 213, 210, 218, 208, 212)(224, 226, 232, 233, 236, 228,
238, 234, 239, 237, 231, 230, 227, 235, 225, 229)(241, 243, 249, 250, 253,
245, 255, 251, 256, 254, 248, 247, 244, 252, 242, 246)(258, 260, 266, 267,
270, 262, 272, 268, 273, 271, 265, 264, 261, 269, 259, 263)(275, 277, 283,
284, 287, 279, 289, 285, 290, 288, 282, 281, 278, 286, 276, 280)(292, 294,
300, 301, 304, 296, 306, 302, 307, 305, 299, 298, 295, 303, 293, 297),
(4, 71, 173, 190, 241, 105, 275, 207, 292, 258, 156, 139, 88, 224, 54, 122)
(5, 10, 6, 16, 8, 11, 12, 18, 20, 15, 19, 9, 17, 14, 13, 7)(37, 70, 172,
189, 240, 104, 274, 206, 291, 257, 155, 138, 87, 223, 53, 121)(38, 72, 174,
191, 242, 106, 276, 208, 293, 259, 157, 140, 89, 225, 55, 123)(39, 73, 175,
192, 243, 107, 277, 209, 294, 260, 158, 141, 90, 226, 56, 124)(40, 74, 176,
193, 244, 108, 278, 210, 295, 261, 159, 142, 91, 227, 57, 125)(41, 75, 177,
194, 245, 109, 279, 211, 296, 262, 160, 143, 92, 228, 58, 126)(42, 76, 178,
195, 246, 110, 280, 212, 297, 263, 161, 144, 93, 229, 59, 127)(43, 77, 179,
196, 247, 111, 281, 213, 298, 264, 162, 145, 94, 230, 60, 128)(44, 78, 180,
197, 248, 112, 282, 214, 299, 265, 163, 146, 95, 231, 61, 129)(45, 79, 181,
198, 249, 113, 283, 215, 300, 266, 164, 147, 96, 232, 62, 130)(46, 80, 182,
199, 250, 114, 284, 216, 301, 267, 165, 148, 97, 233, 63, 131)(47, 81, 183,
200, 251, 115, 285, 217, 302, 268, 166, 149, 98, 234, 64, 132)(48, 82, 184,
201, 252, 116, 286, 218, 303, 269, 167, 150, 99, 235, 65, 133)(49, 83, 185,
202, 253, 117, 287, 219, 304, 270, 168, 151, 100, 236, 66, 134)(50, 84, 186,
203, 254, 118, 288, 220, 305, 271, 169, 152, 101, 237, 67, 135)(51, 85, 187,
204, 255, 119, 289, 221, 306, 272, 170, 153, 102, 238, 68, 136)(52, 86, 188,
205, 256, 120, 290, 222, 307, 273, 171, 154, 103, 239, 69, 137),
(3, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36)(4, 38,
39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 37)(53, 54, 55, 56,
57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69)(70, 71, 72, 73, 74, 75,
76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86)(87, 88, 89, 90, 91, 92, 93, 94,
95, 96, 97, 98, 99, 100, 101, 102, 103)(104, 105, 106, 107, 108, 109, 110,
111, 112, 113, 114, 115, 116, 117, 118, 119, 120)(121, 122, 123, 124, 125,
126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137)(138, 139, 140,
141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154)(155,
156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170,

```

171)(172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185,  
 186, 187, 188)(189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200,  
 201, 202, 203, 204, 205)(206, 207, 208, 209, 210, 211, 212, 213, 214, 215,  
 216, 217, 218, 219, 220, 221, 222)(223, 224, 225, 226, 227, 228, 229, 230,  
 231, 232, 233, 234, 235, 236, 237, 238, 239)(240, 241, 242, 243, 244, 245,  
 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256)(257, 258, 259, 260,  
 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273)(274, 275,  
 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290)  
 (291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305,  
 306, 307),  
 (3, 37, 53, 70, 87, 104, 121, 138, 155, 172, 189, 206, 223, 240, 257, 274,  
 291)(4, 54, 71, 88, 105, 122, 139, 156, 173, 190, 207, 224, 241, 258, 275,  
 292, 21)(22, 38, 55, 72, 89, 106, 123, 140, 157, 174, 191, 208, 225, 242,  
 259, 276, 293)(23, 39, 56, 73, 90, 107, 124, 141, 158, 175, 192, 209, 226,  
 243, 260, 277, 294)(24, 40, 57, 74, 91, 108, 125, 142, 159, 176, 193, 210,  
 227, 244, 261, 278, 295)(25, 41, 58, 75, 92, 109, 126, 143, 160, 177, 194,  
 211, 228, 245, 262, 279, 296)(26, 42, 59, 76, 93, 110, 127, 144, 161, 178,  
 195, 212, 229, 246, 263, 280, 297)(27, 43, 60, 77, 94, 111, 128, 145, 162,  
 179, 196, 213, 230, 247, 264, 281, 298)(28, 44, 61, 78, 95, 112, 129, 146,  
 163, 180, 197, 214, 231, 248, 265, 282, 299)(29, 45, 62, 79, 96, 113, 130,  
 147, 164, 181, 198, 215, 232, 249, 266, 283, 300)(30, 46, 63, 80, 97, 114,  
 131, 148, 165, 182, 199, 216, 233, 250, 267, 284, 301)(31, 47, 64, 81, 98,  
 115, 132, 149, 166, 183, 200, 217, 234, 251, 268, 285, 302)(32, 48, 65, 82,  
 99, 116, 133, 150, 167, 184, 201, 218, 235, 252, 269, 286, 303)(33, 49, 66,  
 83, 100, 117, 134, 151, 168, 185, 202, 219, 236, 253, 270, 287, 304)(34,  
 50, 67, 84, 101, 118, 135, 152, 169, 186, 203, 220, 237, 254, 271, 288,  
 305)(35, 51, 68, 85, 102, 119, 136, 153, 170, 187, 204, 221, 238, 255,  
 272, 289, 306)(36, 52, 69, 86, 103, 120, 137, 154, 171, 188, 205, 222,  
 239, 256, 273, 290, 307),  
 (2, 3)(5, 21)(6, 22)(7, 23)(8, 24)(9, 25)(10, 26)(11, 27)(12, 28)(13, 29)  
 (14, 30)(15, 31)(16, 32)(17, 33)(18, 34)(19, 35)(20, 36)(53, 172)(54, 181)  
 (55, 173)(56, 182)(57, 174)(58, 183)(59, 175)(60, 184)(61, 176)(62, 185)  
 (63, 177)(64, 186)(65, 178)(66, 187)(67, 179)(68, 188)(69, 180)(70, 121)  
 (71, 127)(72, 133)(73, 122)(74, 128)(75, 134)(76, 123)(77, 129)(78, 135)  
 (79, 124)(80, 130)(81, 136)(82, 125)(83, 131)(84, 137)(85, 126)(86, 132)  
 (87, 240)(88, 253)(89, 249)(90, 245)(91, 241)(92, 254)(93, 250)(94, 246)  
 (95, 242)(96, 255)(97, 251)(98, 247)(99, 243)(100, 256)(101, 252)(102,  
 248)(103, 244)(104, 138)(105, 145)(106, 152)(107, 142)(108, 149)(109,  
 139)(110, 146)(111, 153)(112, 143)(113, 150)(114, 140)(115, 147)(116,  
 154)(117, 144)(118, 151)(119, 141)(120, 148)(155, 274)(156, 289)(157,  
 287)(158, 285)(159, 283)(160, 281)(161, 279)(162, 277)(163, 275)(164,  
 290)(165, 288)(166, 286)(167, 284)(168, 282)(169, 280)(170, 278)(171,  
 276)(189, 223)(190, 235)(191, 230)(192, 225)(193, 237)(194, 232)(195,  
 227)(196, 239)(197, 234)(198, 229)(199, 224)(200, 236)(201, 231)(202,  
 226)(203, 238)(204, 233)(205, 228)(206, 257)(207, 271)(208, 268)(209,  
 265)(210, 262)(211, 259)(212, 273)(213, 270)(214, 267)(215, 264)(216,  
 261)(217, 258)(218, 272)(219, 269)(220, 266)(221, 263)(222, 260)(292,  
 307)(293, 306)(294, 305)(295, 304)(296, 303)(297, 302)(298, 301)(299,  
 300),  
 (1, 2)(6, 13)(7, 10)(8, 17)(9, 11)(12, 19)(14, 16)(15, 18)(21, 37)(22,  
 53)(23, 70)(24, 87)(25, 104)(26, 121)(27, 138)(28, 155)(29, 172)(30,  
 189)(31, 206)(32, 223)(33, 240)(34, 257)(35, 274)(36, 291)(38, 54)(39,  
 71)(40, 88)(41, 105)(42, 122)(43, 139)(44, 156)(45, 173)(46, 190)(47,

```

207)(48, 224)(49, 241)(50, 258)(51, 275)(52, 292)(56, 72)(57, 89)(58,
106)(59, 123)(60, 140)(61, 157)(62, 174)(63, 191)(64, 208)(65, 225)(66,
242)(67, 259)(68, 276)(69, 293)(74, 90)(75, 107)(76, 124)(77, 141)(78,
158)(79, 175)(80, 192)(81, 209)(82, 226)(83, 243)(84, 260)(85, 277)(86,
294)(92, 108)(93, 125)(94, 142)(95, 159)(96, 176)(97, 193)(98, 210)(99,
227)(100, 244)(101, 261)(102, 278)(103, 295)(110, 126)(111, 143)(112,
160)(113, 177)(114, 194)(115, 211)(116, 228)(117, 245)(118, 262)(119,
279)(120, 296)(128, 144)(129, 161)(130, 178)(131, 195)(132, 212)(133,
229)(134, 246)(135, 263)(136, 280)(137, 297)(146, 162)(147, 179)(148,
196)(149, 213)(150, 230)(151, 247)(152, 264)(153, 281)(154, 298)(164,
180)(165, 197)(166, 214)(167, 231)(168, 248)(169, 265)(170, 282)(171,
299)(182, 198)(183, 215)(184, 232)(185, 249)(186, 266)(187, 283)(188,
300)(200, 216)(201, 233)(202, 250)(203, 267)(204, 284)(205, 301)(218,
234)(219, 251)(220, 268)(221, 285)(222, 302)(236, 252)(237, 269)(238,
286)(239, 303)(254, 270)(255, 287)(256, 304)(272, 288)(273, 305)(290,
306)];
LoadPackage("images");
MinimalImage(G, [5,8,17,20,21,24,33,36,90,101,245,252], OnSets);

```

Running this file through GAP yields the canonical form:

```

gap> MinimalImage(G, [5,8,17,20,21,24,33,36,90,101,245,252], OnSets);
[ 1, 2, 3, 4, 5, 6, 55, 104, 108, 112, 206, 228 ]

```

Keep in mind that the permutation representation in Gap is 1-based, so the numbers have been incremented by one.

## 6.8 Linear Groups, Advanced Topics

It is sometimes necessary to control the finite field that is used in the construction of a matrix group. For prime fields, this is not an issue. For extension fields, the choice of polynomial does matter, as the generators depend on specific choices made for the finite field. Magma and GAP use Conway polynomials, which are difficult to compute. Orbiter has a built-in table of primitive polynomials. As explained in Section 3.2, Orbiter allows to specify the polynomial that should be used to create the finite field. The next example shows an instance where choosing the polynomial is important. We are recreating a group from the electronic Atlas on finite simple groups [70].

The electronic Atlas of finite simple groups [70] lists generators for  $U_3(3)$  as  $3 \times 3$  matrices over the field  $\mathbb{F}_9$  using the following short Magma [16] program:

```
F<w>:=GF(9);
x:=CambridgeMatrix(1,F,3,[
"164",
"506",
"851"]);
y:=CambridgeMatrix(1,F,3,[
"621",
"784",
"066"]);
G<x,y>:=MatrixGroup<3,F|x,y>;
```

The generators are given using the Magma command `CambridgeMatrix`, which allows for more efficient coding of field elements. The field elements are coded as base-3 integers (like in Orbiter) with respect to the Magma version of  $\mathbb{F}_9$ . The polynomial for  $\mathbb{F}_9$  can be determined using the following Magma command, which can be typed into Magma (or the free Magma online calculator at [65]):

```
F<w>:=GF(9);
print DefiningPolynomial(F);
```

It results in

```
$.1^2 + 2*$.1 + 2
```

which is the Magma way of printing the polynomial  $X^2 + 2X + 2$ . If  $\alpha$  is a root of the polynomial over  $\mathbb{F}_3$ , then

$$\alpha^2 = \alpha + 1.$$

The coefficient vector of the polynomial is  $(1, 2, 2)$ . As an integer written in base-3, we obtain

$$1 \cdot 3^2 + 2 \cdot 3 + 2 = 17.$$

The desired subgroup can now be created using the command

### Example 274

```
U_3_3:
▷ $(ORBITER) -v 3 \
```

```

▷ ▷ -define F -finite_field -q 9 \
▷ ▷ ▷ -override_polynomial "17" \
▷ ▷ -end \
▷ ▷ -define G -linear_group -PGL 3 F \
▷ ▷ ▷ -subgroup_by_generators \
▷ ▷ ▷ "U_3_3" "6048" 2 \
▷ ▷ ▷ "1,6,4, 5,0,6, 8,5,1, \
▷ ▷ ▷ 6,2,1, 7,8,4, 0,6,6" \
▷ ▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_3_9_Subgroup_U_3_3_6048_report.tex
▷ #$(OPEN) PGL_3_9_Subgroup_U_3_3_6048_report.pdf

```

Group theoretic activities will be discussed in Section 6.5.

As an example of a large group, consider the Conway group  $\text{Co}_3$ . Following [64], the group can be generated using two matrices of dimension 22 over  $\mathbb{F}_2$ . We use the makefile variables to give each generator in compact form. Then we define vectors for each of the generators. We concatenate the two generators to form one long vector, which is passed to the `-subgroup_by_generators` command. Finally, we create a report for the group.

#### Example 275

```

CONWAY_GEN1="\
1101110001000001010000\
1111010111110100001011\
0000001000000100010101\
1111100110110001001110\
010101000000010011101\
0000010000000100010101\
001000000000100010101\
0001000011000000111111\
1110100100110100010011\
0000000000000110010101\
0000000000100100010101\
0110111111010011101111\
0000000000001100010101\
0000000000000100000101\
0000000001000100010101\
0000000000000100011101\
0001000110000010011010\
0000000000000000010101\
0000000000000101010101\
0000000000000100010100\
0000000000000100010111\
0000000000000100010001"

```

## Example 276

```

CONWAY_GEN2="\
0101000010111010111111\
0110010100011110110000\
001101000011111010111\
0001101110001011010011\
1010010000100001011110\
1101000000001010100011\
1100101010001111010101\
1000110100110101010101\
0100110001010000000111\
1100000010100101010010\
0101110110011100000101\
010111101010011111001\
1000010101010101010001\
0001010000111100100111\
0011010010111011001111\
0100110010110011111010\
1101011001111101100011\
0100101001001000100001\
1100101100001001110011\
0101110110010100000001\
0000001101111000101110\
1101101010101110000101"

```

## Example 277

```

Co3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define g1 -vector -field F -format 22 -compact $(CONWAY_GEN1) -end \
▷ ▷ -define g2 -vector -field F -format 22 -compact $(CONWAY_GEN2) -end \
▷ ▷ -define gens -vector -concatenate g1,g2 -end \
▷ ▷ -define G -linear_group -PGL 22 2 \
▷ ▷ ▷ -subgroup_by_generators "Co3" "495766656000" 2 gens \
▷ ▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_22_2.Subgroup_Co3_495766656000_report.tex
▷ #$(OPEN) PGL_22_2.Subgroup_Co3_495766656000_report.pdf

```

The next example creates the Ree group in 7 dimensions over the field  $\mathbb{F}_{27}$ . Again, we use makefile variables to specify the two generators as  $7 \times 7$  matrices over  $\mathbb{F}_{27}$  and concatenate them, before passing them to the `-subgroup_by_generators` command.

**Example 278**

```
Ree_gen1="21,5,1,6,17,1,1, 3,13,5,21,6,6,18, 21,3,21,21,22,6,14, \
14,18,1,5,13,6,7, 3,3,2,1,24,16,3, 17,3,22,10,16,24,26, \
21,21,6,18,20,2,5"
```

**Example 279**

```
Ree_gen2="16,3,11,5,16,22,20, 24,6,18,24,7,1,26, 9,23,17,18,23,20,13, \
9,7,2,15,17,5,11, 3,3,6,21,4,24,16, 25,8,6,24,21,12,7, \
24,15,2,13,11,14,24"
```

**Example 280**

```
Ree_27:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 27 \
▷ ▷ ▷ -override_polynomial "34" \
▷ ▷ -end \
▷ ▷ -define g1 -vector -field F -format 7 -dense $(Ree_gen1) -end \
▷ ▷ -define g2 -vector -field F -format 7 -dense $(Ree_gen2) -end \
▷ ▷ -define gens -vector -concatenate g1,g2 -end \
▷ ▷ -define G -linear_group -PGL 7 F \
▷ ▷ ▷ -subgroup_by_generators "Ree_27" "10073444472" 2 gens \
▷ ▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
```



# Chapter 7

## Orbit Algorithms

### 7.1 Schreier Trees

Orbiter provides three different orbit algorithms: Schreier trees, poset classification and orderly generation using canonical forms. All algorithms are invoked by means of an orbit data structure. For background on Schreier trees, see [19, 36, 62]. Poset based algorithms will be discussed in Section 7.2. Orderly generation using canonical forms is discussed in Section 16.2.

Table 7.1 lists the Orbiter commands that can be used to compute orbits. An object of type orbit is created. Table 7.2 lists activities for an object of type orbit.

Consider the group  $\text{PGL}(4, 2)$  in the natural action on the set of points of  $\text{PG}(3, 2)$ . The degree of the action is 15. The action is transitive. The following example computes the Schreier tree for the action:

#### Example 281

```
orbits_PGL_4_2_on_points_draw_tree:
▷ $(ORBITER) -v 4 \
▷ ▷ -draw_options -embedded -end \
▷ ▷ -define G -linear_group -PGL 4 2 -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_points \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -export_something "orbit" 0 \
▷ ▷ -end
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -draw_tree 0 \
▷ ▷ -end
▷ pdflatex PGL_4_2_orbits_report.tex
▷ $(OPEN) PGL_4_2_orbits_report.pdf
```

To draw the tree, the following command is used:

| Computing Orbits           |               |                                                                                                                                                                                       |
|----------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                    | Arguments     | Purpose                                                                                                                                                                               |
| -group                     | $G$           | Specify the group that acts.                                                                                                                                                          |
| -on_points                 |               | Perform Schreier algorithm to compute the orbits of $G$ in the natural action and under the group included in the action.                                                             |
| -on_points_with_generators | gens          | Perform Schreier algorithm to compute the orbits of $G$ in the natural action and under the group given by a set of generators.                                                       |
| -on_subsets                | $k$ control   | Consider the induced action on subsets of size $k$ . This option will invoke the poset algorithm.                                                                                     |
| -of_one_subset             | $G$ set-label | Compute the orbit of the given set under the group $G$ . This algorithm will build up the Schreier tree. The orbit will be saved to a csv file.                                       |
| -on_subspaces              | $k$ control   | Consider the induced action on subspaces of size $k$ . We assume that the group action is linear. This option will invoke the poset algorithm.                                        |
| -on_tensors                | $d$ control   | Consider the orbits on the $d$ -fold tensor product.                                                                                                                                  |
| -on_partition              | $k$ control   | Consider the action on partitions of type $k + k + k$ . In this case, $3k$ must equal the degree of the defining action of $G$ .                                                      |
| -on_polynomials            | $R$           | Consider the action of $G$ on homogenous polynomials as defined in ring $R$ . The action must be linear and the number of variables in $R$ must match the degree of the matrix group. |
| -on_one_polynomial         | $R$ equation  | Consider the action of $G$ on polynomials of degree $d$ in $n$ variables. $G$ must be a matrix group. Here, the action must be linear on an $n$ -dimensional space.                   |
| -on_cubic_curves           | arc-generator | Classify cubic curves by means of $(9,3)$ -arcs. Options are of arc-generator type, see Tab. 7.6.                                                                                     |
| -on_cubic_surfaces         | $P$ control   | Classify cubic surfaces in the projective space $P$ , using the given poset classification control options.                                                                           |

Table 7.1: Computing Orbits

| Orbit Based Activities   |           |                                                                                                                         |
|--------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------|
| Command                  | Arguments | Purpose                                                                                                                 |
| -report                  |           | Create a report on the orbits.                                                                                          |
| -export_something        | type data | Export orbit data of the given type.                                                                                    |
| -export_trees            |           | Create drawings of the Schreier trees.                                                                                  |
| -draw_tree               | $i$       | Create a drawing of the $i$ -th orbit as a tree.                                                                        |
| -stabilizer              | $i$       | Compute the stabilizer of the given point $i$ .                                                                         |
| -stabilizer_of_orbit_rep | $i$       | Compute the stabilizer of the orbit representative of orbit $i$ .                                                       |
| -Kramer_Mesner_matrix    | $t$ $k$   | Compute the Kramer Mesner matrix of orbits on $t$ -subsets and $k$ -subsets. This option requires a poset type problem. |
| -recognize               | $S$       | Recognize $S$ in the orbit data structure.                                                                              |
| -report_options          | descr     | Set additional parameters for the report, see Table 7.5.                                                                |

Table 7.2: Orbit Based Activities

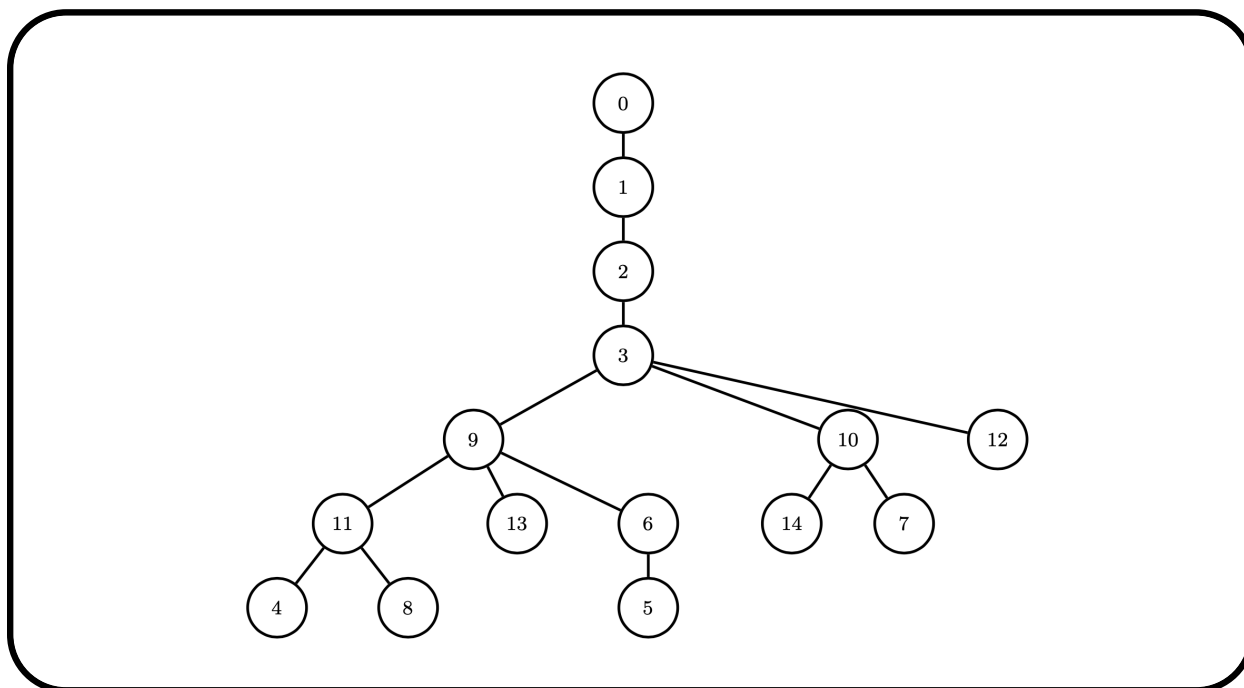
**Example 282**

```

orbits_PGL_4_2_on_points_export_trees:
▷ $(ORBITER) -v 4 \
▷ ▷ -draw_options -embedded -end \
▷ ▷ -define G -linear_group -PGL 4 2 -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_points \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -export_trees \
▷ ▷ -end
▷ $(ORBITER) -v 3 \
▷ ▷ -draw_layered_graph \
▷ ▷ ▷ orbit_PGL_4_2_0.layered_graph \
▷ ▷ -radius 500 -spanning_tree -embedded \
▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
▷ ▷ -end
▷ pdflatex orbit_PGL_4_2_0.draw.tex
▷ $(OPEN) orbit_PGL_4_2_0.draw.pdf

```

The Schreier tree is shown below

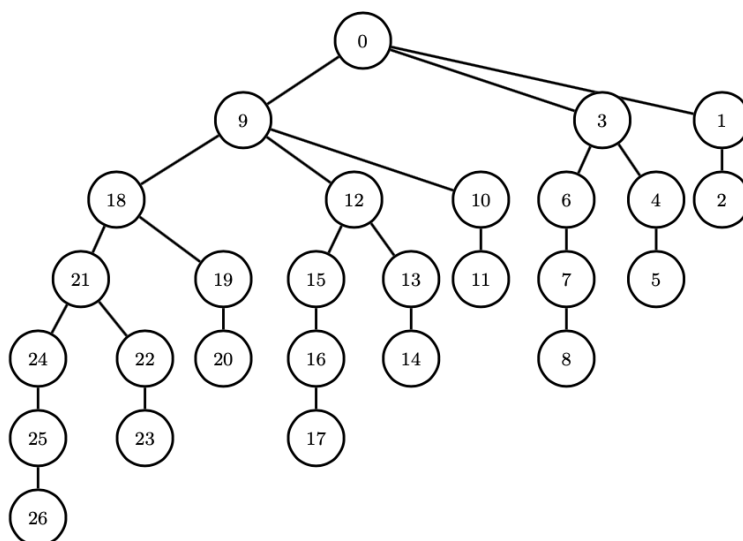


The next command creates the projective plane of order 4 using a line transitive group. The design is the orbit of a certain set of size 5. The group is created in product action. The set results from a search for a special class of linear spaces as described in Section 13.4.

#### Example 283

```
DD_PP4_orbit:
▷ $(ORBITER) -v 4 \
▷ ▷ -define G1 -linear_group -AGL 1 3 \
▷ ▷ ▷ -end \
▷ ▷ -define G2 -linear_group -AGL 1 7 \
▷ ▷ ▷ -end \
▷ ▷ -define G -modified_group -direct_product "G1,G2" \
▷ ▷ ▷ "21" "1,1,1,1" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -define S -vector -dense "0,1,3,13,20" -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -of_one_subset S \
▷ ▷ -end
```

Consider the wreath product acting on rank-one tensors from Section 6.4. The following command sequence computes the orbits, exports the Schreier tree, and produces the drawing

**Example 284**

```

T3r1_orbits:
> $(ORBITER) -v 4 \
> > -draw_options -embedded -end \
> > -define G \
> > -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
> > > -on_rank_one_tensors -end \
> > -define Orb -orbits -group G \
> > > -on_points \
> > > -end \
> > -with Orb -do -orbits_activity \
> > > -report \
> > > -end \
> > -with Orb -do -orbits_activity \
> > > -draw_tree 0 \
> > -end
> pdflatex GL_2.2.wreath_Sym3_orbit_0_tree.tex
> $(OPEN) GL_2.2.wreath_Sym3_orbit_0_tree.pdf

```

In the next example, we compute the orbits of the linear group  $\mathrm{PGL}(3, 2)$  on plane cubics:

**Example 285**

```

orbits_cubic_curves_q2:
> $(ORBITER) -v 4 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 3 F -end \
> > -define R -polynomial_ring \
> > > -field F \

```

```

> > > -number_of_variables 3 \
> > > -homogeneous_of_degree 3 \
> > > -monomial_ordering_partition \
> > > -variables "X,Y,Z" "X,Y,Z" \
> > -end \
> > -define Orb -orbits -group G \
> > > -on_polynomials R \
> > -end
> #pdflatex poly_orbits_d3_n3.q2.tex
> #$(OPEN) poly_orbits_d3_n3.q2.pdf

```

In the next example, we compute the orbits of the linear group  $\mathrm{PGL}(4, 2)$  on homogeneous polynomials of degree 3 in 4 variables:

#### Example 286

```

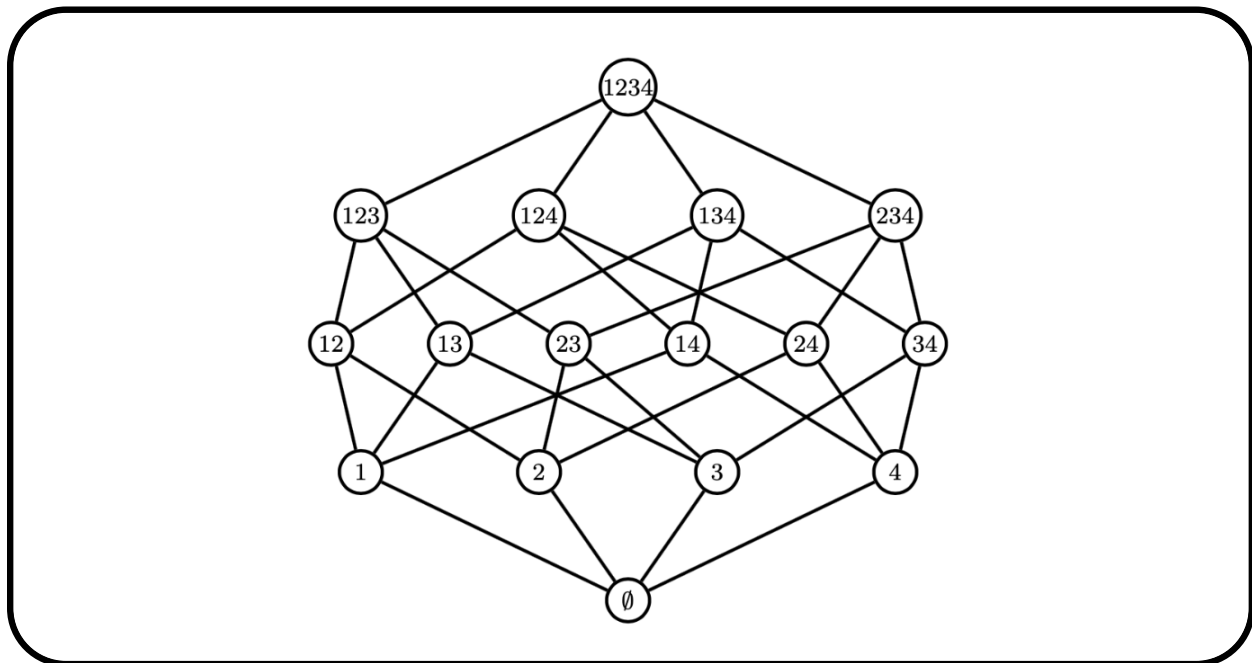
poly_orbits_d3_n3.q2.csv:
> $(ORBITER) -v 4 \
> > -draw_options -yout 500000 -radius 15 -nodes_empty \
> > > -line_width 0.5 -y_stretch 0.25 -embedded -end \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 4 F -end \
> > -define R -polynomial_ring \
> > > -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 3 \
> > > -monomial_ordering_partition \
> > > -variables "X,Y,Z,W" "X,Y,Z,W" \
> > -end \
> > -define Orb -orbits -group G \
> > > -on_polynomials R \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -report \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -draw_tree 6 \
> > -end
> pdflatex PGL_4.2_orbit_6_tree.tex
> $(OPEN) PGL_4.2_orbit_6_tree.pdf

```

This command computes the orbits of all cubic forms in 4 variables over the field  $\mathbb{F}_2$ , confirming the work of Dickson [25] and an enumerative result of Cooley [22].

## 7.2 Poset Classification

A partially ordered set (poset) is a set together with a partial order. For instance, the set of subsets of a fixed set form an order structure with respect to set-inclusion. The Hasse diagram is a diagram whose nodes represent the element. Nodes are arranged from top to bottom, and relations are indicated by lines. Consider for example set of subsets of a 4-element set, say  $\{1, 2, 3, 4\}$ :



Transitivity is implied by following paths in the same direction. For instance,  $\{1\}$  is a subset of  $\{1, 2\}$ , which is a subset of  $\{1, 2, 3\}$ . There is no line from  $\{1\}$  to  $\{1, 2, 3\}$  as the relation can be inferred.

Partial orders can be useful in the problem of classifying combinatorial objects by means of substructures. A poset action is group action on a poset. Let  $G$  be a group and let  $\mathcal{P}$  be a poset. For a poset action, we require that

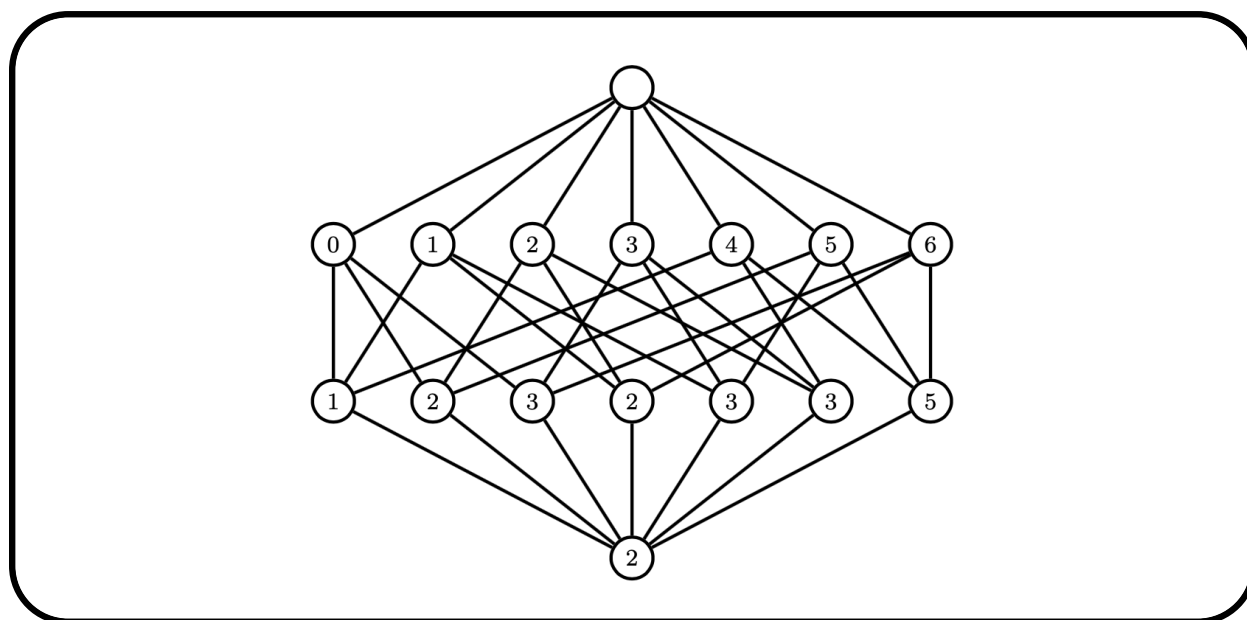
$$x \leq y \Rightarrow xg \leq yg \quad \text{for all } x, y \in \mathcal{P}, g \in G.$$

For background on poset actions, see Plesken [58]. The orbits of  $G$  on  $\mathcal{P}$  form another poset, the poset of orbits. A ranked poset can be decomposed into layers according to the rank function. It is then possible to build the poset by considering adjacent levels inductively. In each step, the next layer is constructed from the previous layer and all the layers below. This allows one to reduce a classification problem to a sequence of smaller, and in many cases easier steps.

Orbiter uses the algorithm of Schmalz [61] to perform poset classification. Two versions are available: one for subset-type posets and one for subspace-type posets. The subspace lattice of  $V(3, 2) = \mathbb{F}_2^3$  is shown below:

| Options for Computing Posets |           |                                                                                               |
|------------------------------|-----------|-----------------------------------------------------------------------------------------------|
| Command                      | Arguments | Purpose                                                                                       |
| -problem_label               | str       | Use str as a prefix for files that are created.                                               |
| -path                        | $p$       | Use path $p$ for files that are created.                                                      |
| -depth                       | $d$       | Set search depth to $d$ .                                                                     |
| -v                           | $v$       | Set verbosity to $v$ . Larger numbers mean more output.                                       |
| -gv                          | $v$       | Set verbosity for group theoretic operations to $v$ . Larger numbers mean more output.        |
| -recover                     | fname     | Recover from the given file.                                                                  |
| -lex                         |           | Use the lexicographic ordering to speed up the search.                                        |
| -w                           |           | Save orbits at level $d$ only.                                                                |
| -W                           |           | Save orbits at all levels.                                                                    |
| -write_data_files            |           | Save data to files.                                                                           |
| -t                           |           | Write a file containing the search tree at level $d$ .                                        |
| -T                           |           | Write a file containing the search tree at all levels.                                        |
| -draw_options                | options   | Drawing options according to Table 17.1.                                                      |
| -preferred_choice            | $n a b$   | At node $n$ , choose $b$ instead of $a$ as orbit representative. This option can be repeated. |
| -clique_test                 | graph     | Classify cliques in the given graph.                                                          |

Table 7.3: Options for Computing Posets



The basis elements are listed, using the enumerator for elements of the projective geometry  $\text{PG}(2, 2)$  explained in Section 4.1.

The commands shown in Table 7.3 can be used to control the poset classification algorithm. By default,



Orbiter will choose the lexicographically least orbit representatives. It is possible to direct Orbiter to choose different orbit representatives. To this end, the nodes in the orbit tree are labeled. The node number is the zero-based number of a given node in the tree, using the breadth first ordering.

For posets whose orbits have been computed, there are activities as shown in Table 7.4.

For reporting purposes, the options in Table 7.5 can be used.

Suppose that orbiter chooses element  $a$  at node  $n$ . Suppose we are interested in choosing element  $b$  instead. The command

`-preferred_choice  $n$   $a$   $b$`

can be used to force Orbiter to choose  $b$  instead of  $a$  at node  $n$ .

| Activities for Posets of Orbits          |           |                                                                                                     |
|------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------|
| Command                                  | Arguments | Purpose                                                                                             |
| -report                                  | options   | Produce a latex report. For options, see Table 7.5. Requires -orbiter_path option from Section 2.1. |
| -export_level_to_cpp                     | i         | Export level i to cpp file.                                                                         |
| -export_history_to_cpp                   | i         | Export history up to level i to cpp file.                                                           |
| -write_tree                              |           | Create the depth first tree from the poset of orbits. A tree file is created.                       |
| -find_node_by_stabilizer_order           | k         | Search for all nodes whose stabilizer has order k.                                                  |
| -draw_poset                              |           | Draw the poset of orbits.                                                                           |
| -draw_full_poset                         |           | Draw the poset of orbits in full.                                                                   |
| -plesken                                 |           | Compute the Plesken matrix of the poset of orbits. See [58].                                        |
| -print_data_structure                    |           | Print the data structure of the poset of orbits.                                                    |
| -list                                    |           | List the nodes in the poset of orbits.                                                              |
| -list_all                                |           | List all nodes in the poset of orbits.                                                              |
| -table_of_nodes                          |           | Produce a table of all nodes in the poset of orbits.                                                |
| -make_relations_with_flag_orbits         |           | Produce a bitmap drawing of the neighboring relations in the poset with flag orbits.                |
| -level_summary_csv                       |           | Write a summary of number of orbits at each level to a csv file.                                    |
| -orbit_reps_csv                          |           | Write orbit representatives to a csv file.                                                          |
| -node_label_is_group_order               |           | When drawing the poset of orbits, display the group order in the orbit nodes.                       |
| -node_label_is_element                   |           | When drawing the poset of orbits, display the element rank in the orbit nodes.                      |
| -show_orbit_decomposition                |           | Show the orbits of the stabilizers on the orbit representatives (in the induced action on the set). |
| -show_stab                               |           | Show the stabilizers.                                                                               |
| -save_stab                               |           | Save the stabilizer generators.                                                                     |
| -show_whole_orbits                       |           | Show the whole orbits.                                                                              |
| -export_schreier_trees                   |           | Export all Schreier trees.                                                                          |
| -draw_schreier_trees                     |           | Draw all Schreier trees.                                                                            |
| -test_multi_edge_in_decomposition_matrix |           |                                                                                                     |
| -recognize                               | set       | Recognizes the given set in the poset of orbits. This option can be repeated.                       |

Table 7.4: Activities for Posets of Orbits

| Poset Classification Report Options            |           |                                                                                                                                                                                             |
|------------------------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                        | Arguments | Purpose                                                                                                                                                                                     |
| -select_orbits_by_level                        | level     | Select the levels which should be drawn.                                                                                                                                                    |
| -select_orbits_by_stabilizer_order             | m         | Select the orbits whose stabilizer order is equal to $m$ .                                                                                                                                  |
| -select_orbits_by_stabilizer_order_multiple_of | m         | Select the orbits whose stabilizer order is a multiple of $m$ .                                                                                                                             |
| -include_projective_stabilizer                 |           | Include the projective stabilizer in the report. This option assumes that the acting group is a collineation group. The projective stabilizer is a subgroup of the collineation stabilizer. |
| -draw_poset                                    |           | Include a drawing of the poset of orbits in the report. This option requires one of the following four options to set the type of drawing.                                                  |
| -type_aux                                      |           | Poset drawing of auxiliary type. The auxiliary poset includes the flag orbits between layers.                                                                                               |
| -type_ordinary                                 |           | Poset drawing of ordinary type.                                                                                                                                                             |
| -type_tree                                     |           | Poset drawing of type spanning tree.                                                                                                                                                        |
| -type_detailed                                 |           | Poset drawing of type detailed. The detailed drawing includes the flag orbits between layers.                                                                                               |
| -fname                                         | fname     | Set the file name.                                                                                                                                                                          |

Table 7.5: Poset Classification Report Options

### 7.3 Orbits on Subsets

The lattice of subsets of a set  $X$  is  $\mathfrak{P}(X)$ , the set of all subsets of  $X$ , ordered with respect to inclusion. Assume that a group  $G$  acts on  $X$ , and hence on the lattice by means of the induced action on subsets. The orbits of  $G$  on subsets form a new poset, the poset of orbits. Poset classification is the process of computing the poset of orbits. Orbiter has an algorithm to perform poset classification. In many cases, we are not interested in the full lattice of subsets  $\mathfrak{P}(X)$  but rather in a subposet of it. We require that the subposet is closed under the group action and that the following property holds:

$$x, y \in \mathfrak{P}(X) \text{ and } x \leq y \Rightarrow (y \in \mathcal{P} \rightarrow x \in \mathcal{P}).$$

The join of two subsets in the poset may or may not belong to the poset. Let us consider the action of the Singer cycle on  $\text{PG}(3, 2)$ . The following command computes the orbits of the group  $G$  generated by a Singer cycle in  $\text{PG}(3, 2)$ :

#### Example 287

```
PGL_3_2_singer:
> $(ORBITER) -v 3 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define Control -poset_classification_control \
> > > -problem_label PGL_3_2_singer_1 \
> > > -W -depth 7 \
> > > -draw_options \
> > > > -radius 200 -embedded \
> > > -end \
> > -end \
> > -define G -linear_group -PGL 3 2 -singer 1 -end \
> > -define Orb -orbits -group G \
> > > -on_subsets 7 Control \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -report \
> > > -report_options \
> > > > -draw_poset -type_ordinary \
> > > -end \
> > -end
> pdflatex PGL_3_2_singer_1_poset.tex
> $(OPEN) PGL_3_2_singer_1_poset.pdf
```

The next command computes the orbits of the projective group  $\text{PGL}(4, 2)$  acting on all subsets of  $\text{PG}(3, 2)$ :

#### Example 288

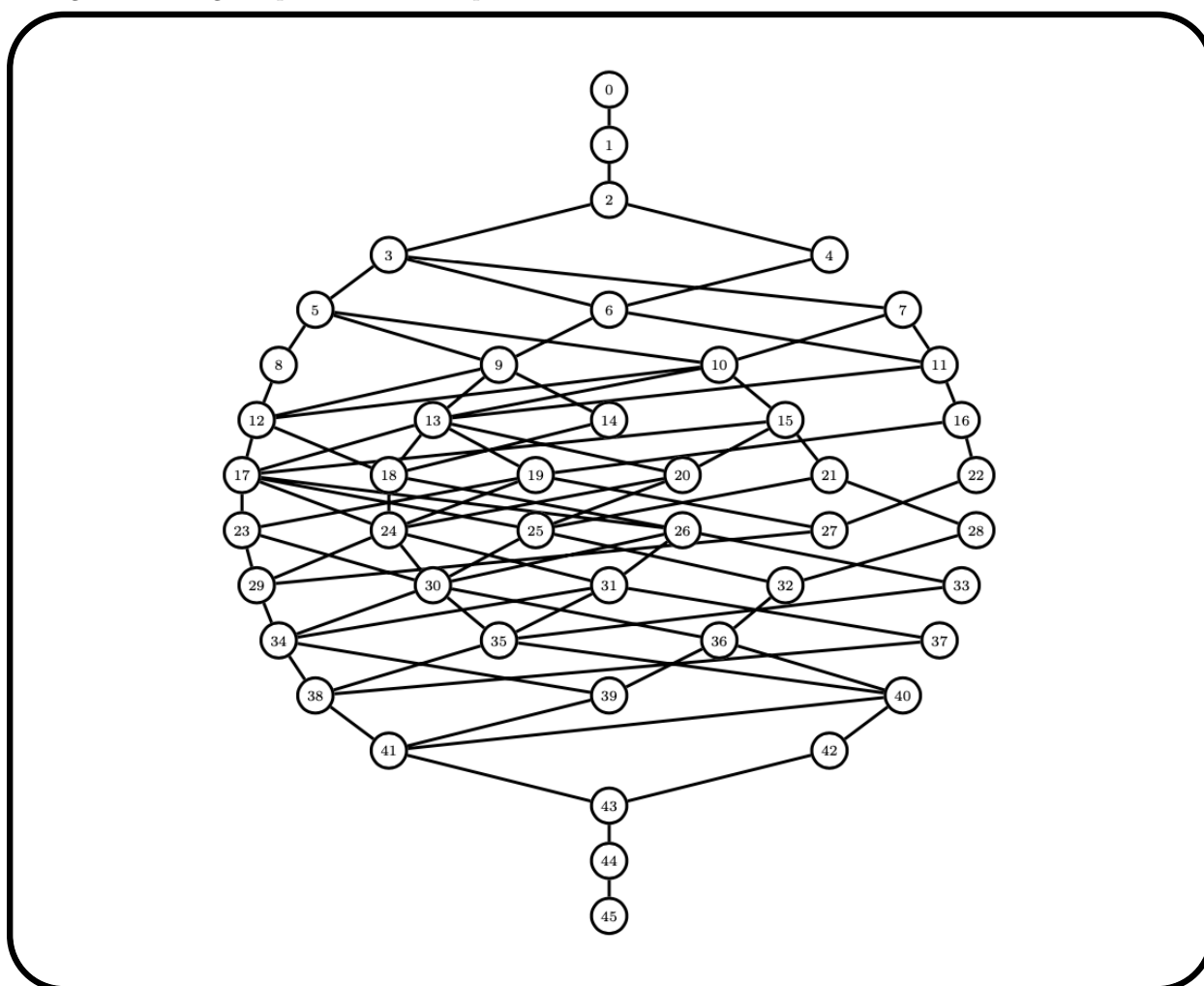
```
PG_3_2_subsets:
> $(ORBITER) -v 3 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define Control -poset_classification_control \
> > > -problem_label PGL_4_2 \
> > > -depth 15 \
> > > -draw_options \
> > > > -radius 200 -embedded \
```

```

▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_subsets 15 Control \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -report_options \
▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex PGL_4.2_poset.tex
▷ $(OPEN) PGL_4.2_poset.pdf

```

A diagram showing the poset of orbits is produced:



Orbiter can compute orbits of groups acting in various different actions. The following example computes the orbits of  $\text{PGL}(3, 2)$  on the subsets of lines of  $\text{PG}(2, 2)$ .

**Example 289**

```

PGL_3_2.on_lines:
▷ $(ORBITER) -v 3 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label PGL_3_2.lines \
▷ ▷ ▷ -W -depth 7 \
▷ ▷ -end \
▷ ▷ -define G -linear_group -PGL 3 2 -end \
▷ ▷ -define G.on_lines -modified_group -from G \
▷ ▷ ▷ -on_k_subspaces 2 \
▷ ▷ -end \
▷ ▷ -define Orb -orbits -group G.on_lines \
▷ ▷ ▷ -on_subsets 7 Control \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -report_options \
▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex PGL_3_2.lines_poset.tex
▷ $(OPEN) PGL_3_2.lines_poset.pdf

```

The following example computes the orbits of  $\text{PGO}(5, 2)$  on the power set lattice of points of  $Q(4, 2)$ :

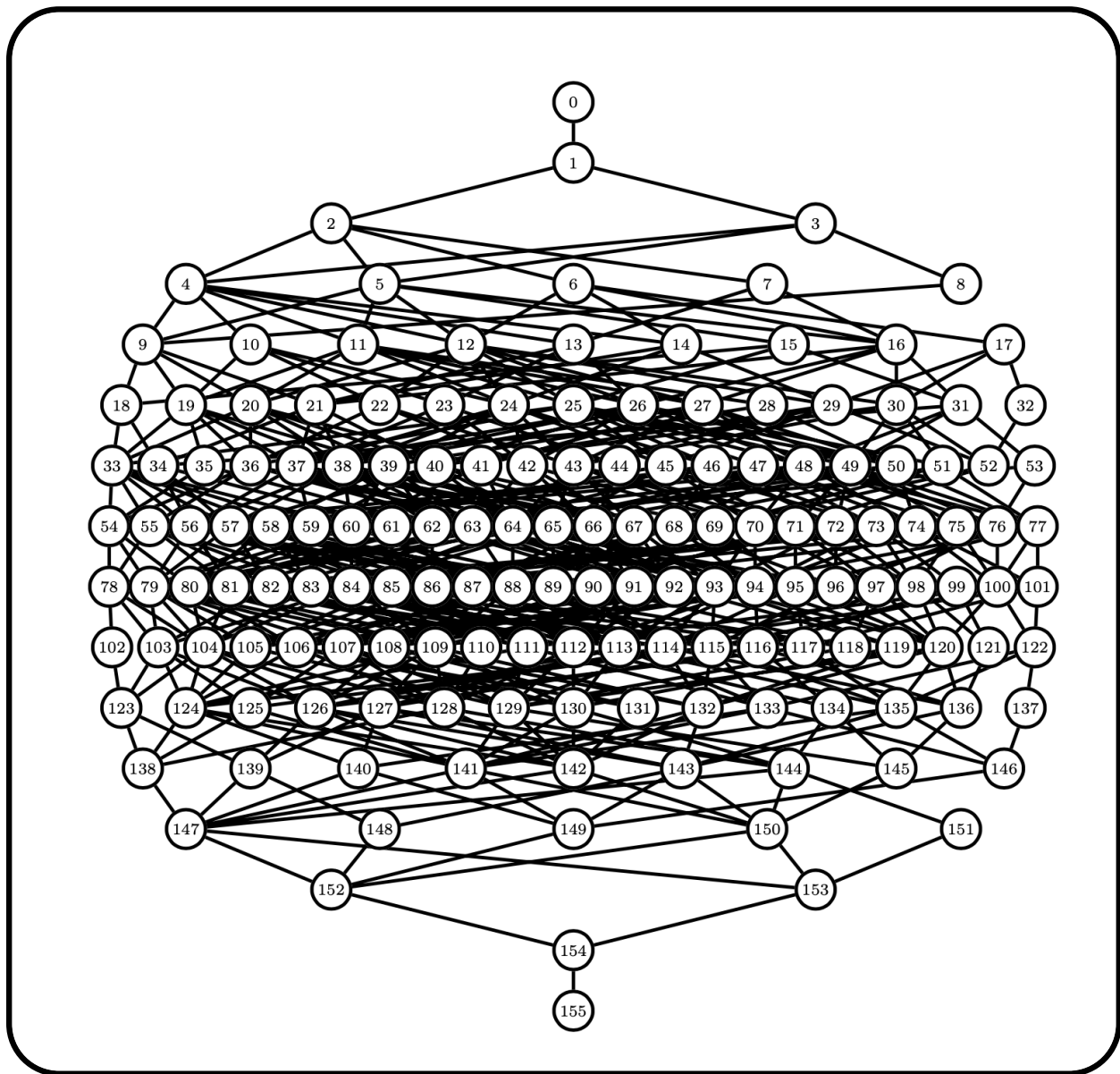
**Example 290**

```

PGO_5_2.on_subsets:
▷ $(ORBITER) -v 3 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label PGO_5_2 \
▷ ▷ ▷ -depth 15 \
▷ ▷ ▷ -w \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define G -linear_group -PGO 5 F -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_subsets 15 Control \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -report_options \
▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex PGO_5_2.poset.tex
▷ $(OPEN) PGO_5_2.poset.pdf

```

The poset of orbits is shown below:



## 7.4 Orbits on Subspaces

Orbiter can compute the orbits of a group on the lattice of subspaces of a finite vector space.

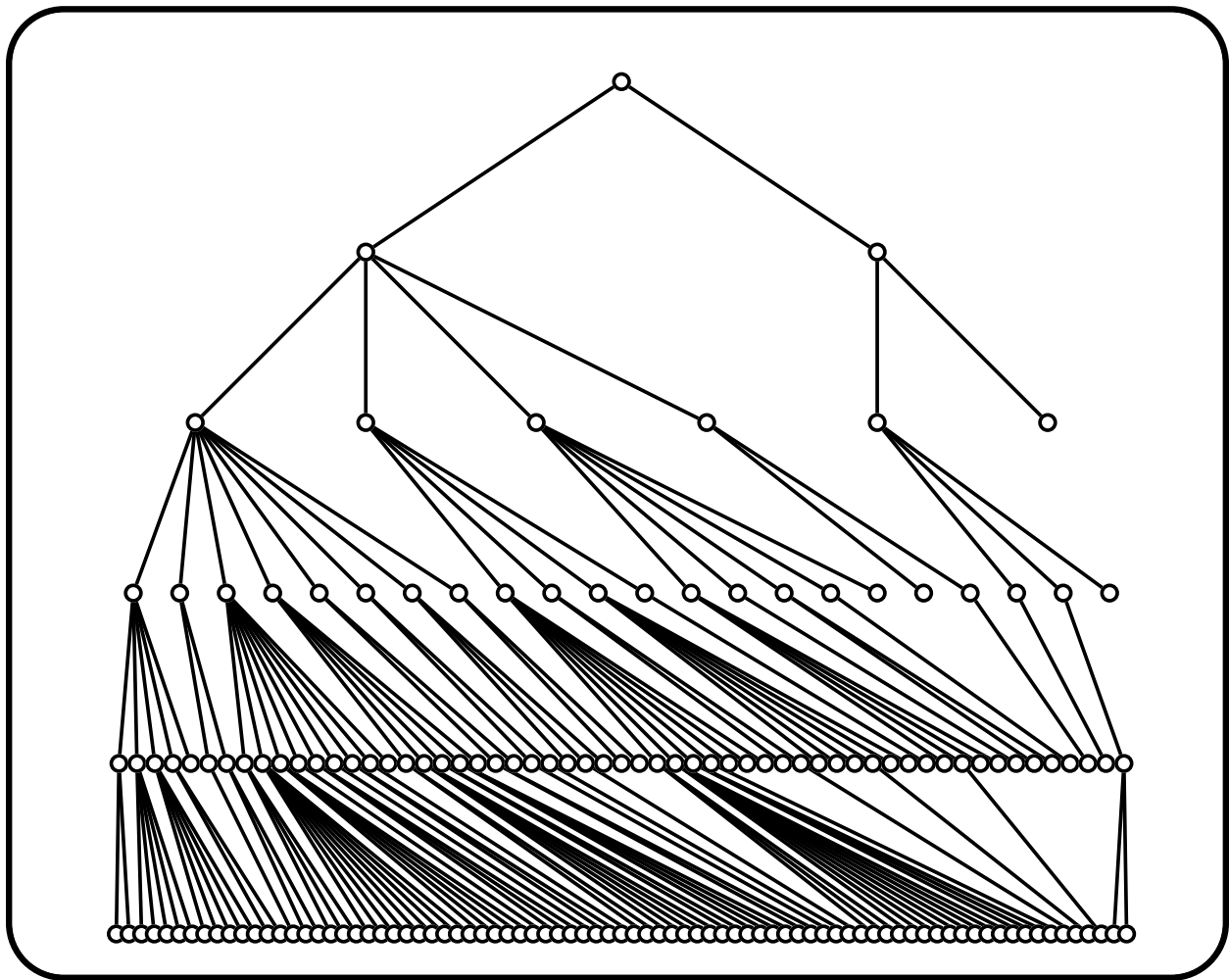
Suppose we want to compute the orbits of the projective group on the subspaces of the exterior algebra. The following command creates  $\text{PGL}(5, 3)$  in the wedge action as in Section 6.4. It then computes the orbits of subspaces of the exterior algebra of dimension 5. Because of the nature of the poset classification algorithm, all orbits of dimension  $i$  with  $0 \leq i \leq 5$  are computed.

### Example 291

```
PGL_5_3_wedge_subspace_orbits:
> $(ORBITER) -v 3 \
> > -draw_options -radius 75 -embedded -nodes_empty -end \
> > -define G -linear_group -PGL 5 3 -end \
> > -define G2 -modified_group -from G \
> > > -on_wedge_product \
> > -end \
> > -define Control -poset_classification_control \
> > > -draw_options -radius 200 -embedded -end \
> > > -problem_label PGL_5_3_wedge -W -depth 5 \
> > -end \
> > -define Orb -orbits -group G2 \
> > > -on_subspaces 5 Control \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -draw_tree 0 \
> > -end
> pdflatex PGL_5_3_OnWedge_tree_lvl_5.tex
> $(OPEN) PGL_5_3_OnWedge_tree_lvl_5.pdf
```

The computation finds 81 orbits at level 5. The lex-least spanning tree for the orbits is shown below:





The orthogonal group is the stabilizer of a non-degenerate quadric. Suppose we want to classify the subspaces in  $\text{PG}(3,2)$  under the action of the orthogonal group. In  $\text{PG}(3,2)$  there are two distinct nondegenerate quadrics,  $Q^+(3,2)$  and  $Q^-(3,2)$ . The  $Q^+(3,2)$  quadric is given by the equation

$$x_0x_1 + x_2x_3 = 0.$$

The command

#### Example 292

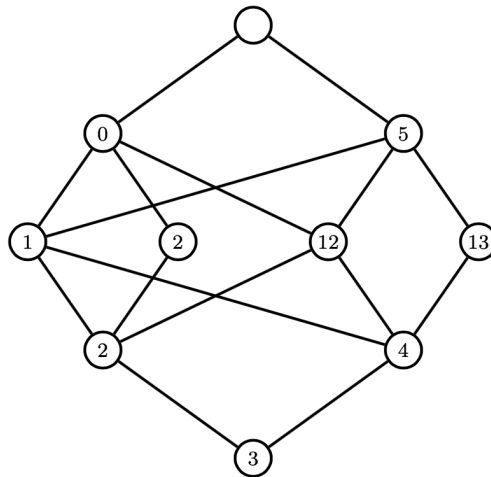
```
subspaces_Op_4.2:
> $(ORBITER) -v 5 \
> > -orbiter_path $(ORBITER.EXE_PATH) \
> > -define Control -poset_classification_control \
> > > -draw_options -radius 200 -end \
> > > -problem_label Op_4.2 -W -depth 4 \
> > -end \
> > -define G -linear_group -PGL 4 2 -orthogonal 1 -end \
> > -define Orb -orbits -group G \
> > > -on_subspaces 4 Control \
> > -end \
```

```

▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -report_options \
▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex PGL_4_2_Orthogonal_plus_4_2_poset.tex
▷ $(OPEN) PGL_4_2_Orthogonal_plus_4_2_poset.pdf

```

produces a classification of all subspaces of  $\text{PG}(3, 2)$  under  $\text{PGO}^+(4, 2)$ , shown below:



The nodes show the ranks of points in  $\text{PG}(3, 2)$  as described in Section 4.1.

## 7.5 Orbits on Set-Partitions

Orbiter can compute the orbits of a group on set-partitions. The set-partition needs to have three parts of equal size.

The command

### Example 293

```
C6_on_partition:
▷ $(ORBITER) -v 5 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label C6 \
▷ ▷ ▷ -depth 2 \
▷ ▷ ▷ -W \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 200 -embedded \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_partition 2 Control \
▷ ▷ -end
```

computes the orbits of the cyclic group  $C_6$  on set-partitions of type  $2 + 2 + 2$ . There are 15 set-partitions, and they fall into 5 orbits, with stabilizer orders

$$3, 1, 2, 2, 6.$$

The orbit count gives

$$6\left(\frac{1}{3} + \frac{1}{1} + \frac{1}{2} + \frac{1}{2} + \frac{1}{6}\right) = 15.$$

The command

### Example 294

```
PGL_2_17_on_partition:
▷ $(ORBITER) -v 5 \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label PGL_2_17 \
▷ ▷ ▷ -depth 6 \
▷ ▷ ▷ -W \
▷ ▷ ▷ -end \
▷ ▷ -define G -linear_group -PGL 2 17 -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_partition 6 Control \
▷ ▷ -end
```

computes the orbits of the group  $\text{PGL}(2, 17)$  on set-partitions of type  $6 + 6 + 6$ . The number of set-partitions is

$$\frac{\binom{18}{6} \cdot \binom{12}{6}}{3!} = 2858856$$

There are 720 orbits. The orbit stabilizer statistic is

$$(1^{480}, 2^{184}, 3^{11}, 4^{20}, 6^{15}, 8, 12^6, 18, 24, 36).$$

The orbit-stabilizer count confirms that

$$4896 \left( \frac{480}{1} + \frac{184}{2} + \frac{11}{3} + \frac{20}{4} + \frac{15}{6} + \frac{1}{8} + \frac{6}{12} + \frac{1}{18} + \frac{1}{24} + \frac{1}{36} + \right) = 2858856.$$

| Classifying Arcs        |           |                                                                                                                                                                                    |
|-------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                 | Arguments | Purpose                                                                                                                                                                            |
| -control                | label     | Set the label of the poset classification control object.                                                                                                                          |
| -projective_space       | P         | Set the projective space to P.                                                                                                                                                     |
| -d                      | $d$       | Require that no more than $d$ points lie on a line.                                                                                                                                |
| -target_size            | $t$       | Specify the size of the arc to be $t$ .                                                                                                                                            |
| -conic_test             |           | Require that no 6 points of the arc lie on a conic.                                                                                                                                |
| -test_nb_Eckardt_points | $n$       | Require exactly $n$ Eckardt points.                                                                                                                                                |
| -affine                 |           | Classify arcs in the affine geometry, assuming that $x_0 = 0$ is the hyperplane at infinity. The condition that no more that $d$ point lie on a line applies to affine lines only. |
| -no_arc_testing         |           | Do not test the “at most $d$ points per line” condition.                                                                                                                           |
| -forbidden_point_set    | set       | The arc must not contain any of the given points.                                                                                                                                  |
| -override_group         | label     | Override the group under which we classify.                                                                                                                                        |

Table 7.6: Classifying Arcs

## 7.6 Arcs and Caps in Projective Spaces

In  $\text{PG}(n, q)$ , an arc is a set of points, no  $n+1$  in a hyperplane. A cap is set of points, no three collinear. Here, we restrict our attention to arcs in  $\text{PG}(2, q)$ . Arcs in higher dimensional projective spaces are equivalent to MDS codes and will be treated in Section 10. Our main examples will be the construction of the Lunelli-Sce hyperoval in  $\text{PG}(2, 16)$  (cf. [49]) and the Pellegrino cap in  $\text{AG}(4, 3)$ . The uniqueness of this cap was proven by Hill [31].

A  $(k, d)$ -arc in a projective plane  $\pi$  is a set  $S$  of  $k$  points such that any line intersects  $S$  in at most  $d$  points. Two arcs  $S_1$  and  $S_2$  are equivalent if there is a projectivity  $\Phi$  such that  $\Phi(A) = B$ . The problem of classifying arcs is the problem of determining the orbits of the projectivity group on arcs. At times, we consider the larger group of collineations. In that case, the problem of classifying arcs is the problem of determining the orbits of the collineation group on arcs. Table 7.6 list the commands available to classify arcs. Here is an example. A hyperoval in a finite plane  $\text{PG}(2, 2^e)$  is a  $(2^e + 2, 2)$ -arc. It is an interesting but difficult problem to classify the hyperovals in finite projective planes under the appropriate group action. For small planes, computer search with Orbiter can help. The command

### Example 295

```
hyperoval_16_classify:
> $(ORBITER) -v 4 \
> > -orbiter_path $(ORBITER.EXE_PATH) \
> > -define F -finite_field -q 16 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -define Control -poset_classification_control \
> > > -problem_label hyperoval.q16 \
> > > -W -depth 18 \
```

```

> > > -draw_options \
> > > > -radius 200 \
> > > -end \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -classify_arcs \
> > > > -control Control \
> > > > -target_size 18 \
> > > > -d 2 \
> > > -end \
> > -end

```

performs the classification of hyperovals in  $PG(2, 16)$ . There are exactly two hyperovals in this plane. Orbiter also finds the stabilizers of these arcs. They have orders 16320 and 144, respectively. The two hyperovals are the regular hyperoval and the Lunelli-Sce hyperoval. Here is the relevant output from the Orbiter report (in the output, the Lunelli-Sce hyperoval is orbit 0, and the regular hyperoval is orbit 1):

### Orbits at Level 18

There are 2 orbits at level 18.

### Orbit 0 / 2 at Level 18

Node number: 4212

$\{0, 1, 2, 3, 52, 67, 89, 106, 126, 141, 159, 176, 184, 199, 220, 235, 245, 262\}_{144}$

|                        |                          |
|------------------------|--------------------------|
| 0 : 0 = ( 1, 0, 0 )    | 10 : 159 = ( 14, 8, 1 )  |
| 1 : 1 = ( 0, 1, 0 )    | 11 : 176 = ( 15, 9, 1 )  |
| 2 : 2 = ( 0, 0, 1 )    | 12 : 184 = ( 7, 10, 1 )  |
| 3 : 3 = ( 1, 1, 1 )    | 13 : 199 = ( 6, 11, 1 )  |
| 4 : 52 = ( 3, 2, 1 )   | 14 : 220 = ( 11, 12, 1 ) |
| 5 : 67 = ( 2, 3, 1 )   | 15 : 235 = ( 10, 13, 1 ) |
| 6 : 89 = ( 8, 4, 1 )   | 16 : 245 = ( 4, 14, 1 )  |
| 7 : 106 = ( 9, 5, 1 )  | 17 : 262 = ( 5, 15, 1 )  |
| 8 : 126 = ( 13, 6, 1 ) |                          |
| 9 : 141 = ( 12, 7, 1 ) |                          |

Strong generators for a group of order 144:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \delta^4 & \delta^9 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & \delta^7 & \delta^{13} \\ \delta^8 & \delta^9 & \delta^{10} \\ \delta & \delta^6 & 1 \end{bmatrix}_3, \begin{bmatrix} \delta^5 & \delta^5 & \delta^5 \\ \delta^5 & \delta^2 & \delta^{11} \\ \delta^5 & \delta^{14} & 1 \end{bmatrix}_0$$

1,0,0,0,1,0,9,5,1,1,  
1,7,6,14,5,10,2,15,1,3,  
1,1,1,1,3,15,1,5,10,0,

There are 0 extensions  
Number of generators 3

### Orbit 1 / 2 at Level 18

Node number: 4213

$$\{0, 1, 2, 3, 52, 70, 83, 109, 127, 139, 156, 174, 186, 199, 217, 229, 256, 264\}_{16320}$$

$$\begin{array}{ll} 0 : 0 = (1, 0, 0) & 10 : 156 = (11, 8, 1) \\ 1 : 1 = (0, 1, 0) & 11 : 174 = (13, 9, 1) \\ 2 : 2 = (0, 0, 1) & 12 : 186 = (9, 10, 1) \\ 3 : 3 = (1, 1, 1) & 13 : 199 = (6, 11, 1) \\ 4 : 52 = (3, 2, 1) & 14 : 217 = (8, 12, 1) \\ 5 : 70 = (5, 3, 1) & 15 : 229 = (4, 13, 1) \\ 6 : 83 = (2, 4, 1) & 16 : 256 = (15, 14, 1) \\ 7 : 109 = (12, 5, 1) & 17 : 264 = (7, 15, 1) \\ 8 : 127 = (14, 6, 1) & \\ 9 : 139 = (10, 7, 1) & \end{array}$$

Strong generators for a group of order 16320:

$$\begin{aligned} & \begin{bmatrix} \delta^6 & 0 & 0 \\ 0 & \delta^3 & 0 \\ 0 & 0 & 1 \end{bmatrix}_2, \begin{bmatrix} \delta^9 & 0 & 0 \\ 0 & \delta^7 & 0 \\ 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} \delta^2 & 0 & 0 \\ 0 & \delta^{11} & 0 \\ \delta^4 & \delta^7 & 1 \end{bmatrix}_3, \\ & \begin{bmatrix} \delta^{10} & 0 & 0 \\ 0 & \delta^3 & 0 \\ \delta & \delta^{11} & 1 \end{bmatrix}_3, \begin{bmatrix} \delta & 0 & 0 \\ \delta^{12} & \delta^2 & \delta^5 \\ \delta^{14} & \delta^{10} & 1 \end{bmatrix}_1, \begin{bmatrix} \delta^5 & 0 & 0 \\ \delta & \delta & \delta \\ \delta^6 & \delta^8 & 1 \end{bmatrix}_0, \\ & \begin{bmatrix} \delta^{12} & 1 & \delta^2 \\ \delta^4 & \delta^3 & \delta^7 \\ \delta^6 & \delta^3 & 1 \end{bmatrix}_2, \begin{bmatrix} \delta^5 & \delta^3 & \delta^6 \\ \delta^{11} & \delta^6 & \delta^{10} \\ \delta^{10} & \delta^6 & 1 \end{bmatrix}_3 \end{aligned}$$

1,0,0,0,3,0,0,0,5,2,  
1,0,0,0,6,0,0,0,15,1,  
1,0,0,0,5,0,4,11,6,3,  
1,0,0,0,14,0,15,2,11,3,  
1,0,0,13,2,9,6,5,12,1,  
1,0,0,13,13,13,2,8,10,0,  
1,8,11,7,15,10,5,15,8,2,  
1,6,2,15,2,11,11,2,10,3,  
There are 0 extensions  
Number of generators 8

In the theory of cubic surfaces, we are interested in non-conical arcs of size 6. Here, non-conical means that the set of points does not lie on a conic. Cubic surfaces are associated with non-conical arcs of size 6 (in a many-to-one relationship when considering isomorphism classes). The following example demonstrates how non-conical 6-arcs can be classified in Orbiter:

**Example 296**

```

nc_arcs_16:
> $(ORBITER) -v 4 \
> > -define F -finite_field -q 16 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -define Control -poset_classification_control \
> > > -problem_label nc_arcs.q16_d2 \
> > > -W -depth 6 \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -classify_arcs \
> > > > -control Control \
> > > > -target_size 6 \
> > > > -d 2 \
> > > > -conic_test \
> > > -end \
> > -end
> #pdflatex nc_arcs.q16_d2_poset.tex
> #$(OPEN) nc_arcs.q16_d2_poset.pdf

```

The number of Eckardt points of the surface can be recovered from properties of the arc. For this reason, it is interesting to classify arcs so that the associated cubic surface has a fixed number of Eckardt points.



## 7.7 Cubic Curves

Orbiter can classify cubic curves in  $\text{PG}(2, q)$ . To this end, the  $(9, 3)$ -arcs in  $\text{PG}(2, q)$  are classified first. From this classification, the classification of curves is computed. This classification only works for arcs which contain a  $(9, 3)$  arc. For very small fields, this is not always the case. Here is an example. The command

### Example 297

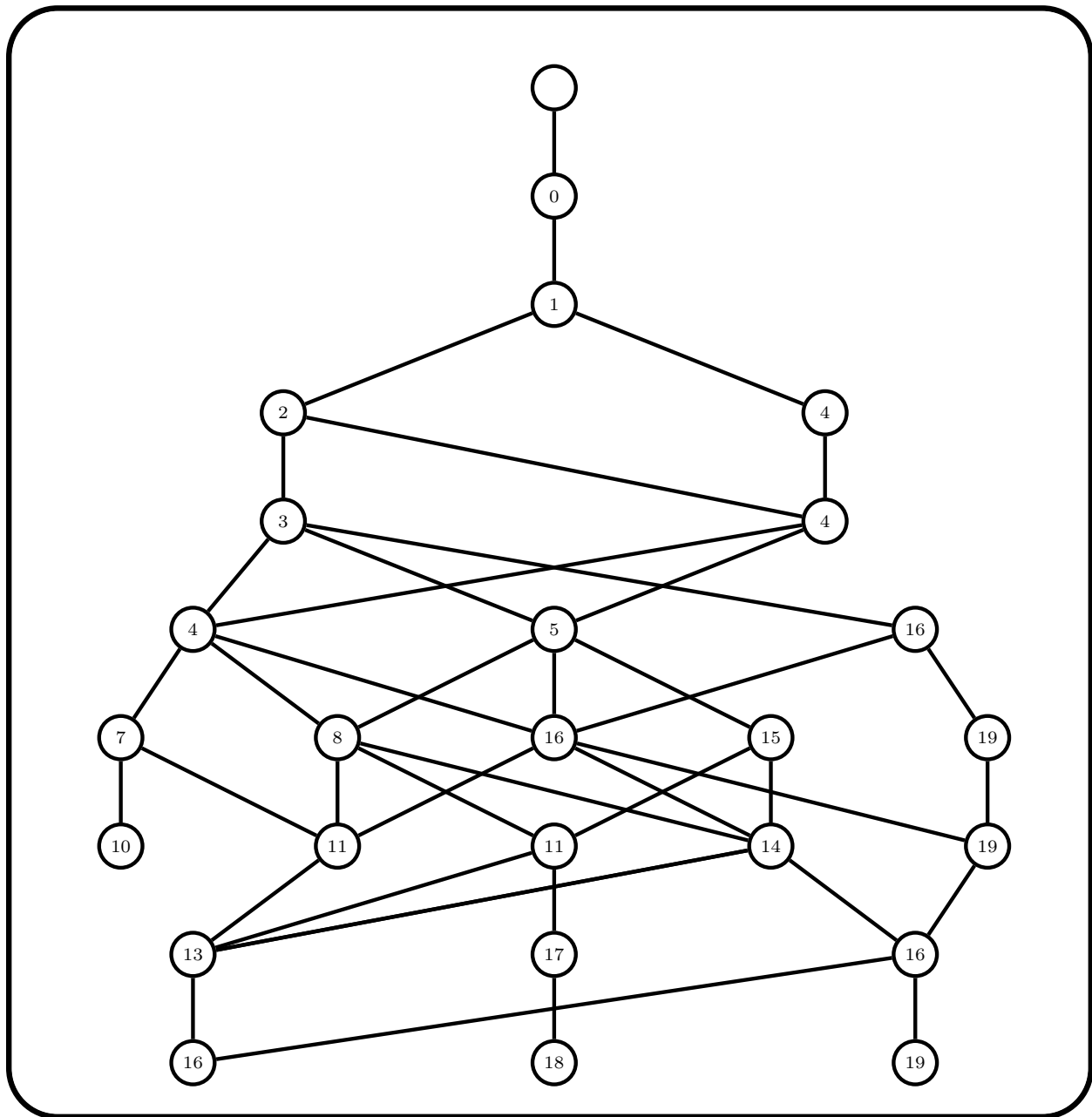
```
cubic_curves_PG_2_4:
> $(ORBITER) -v 3 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -draw_options -radius 200 -embedded -end \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -define Control -poset_classification_control \
> > > -problem_label cc_4 -W -depth 9 \
> > > -draw_options -radius 200 -embedded -end \
> > -end \
> > -define Arc_control -arc_generator_control \
> > > -projective_space P \
> > > -control Control \
> > > -target_size 9 \
> > > -d 3 \
> > -end \
> > -define Orb -orbits \
> > > -on_cubic_curves Arc_control \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -report \
> > -end
> > -with Orb -do -orbits_activity \
> > > -draw_tree 0 \
> > -end
> pdflatex cc_4_poset_lvl_9.tex
> $(OPEN) cc_4_poset_lvl_9.pdf
> pdflatex Cubic_curves_q4.tex
> $(OPEN) Cubic_curves_q4.pdf
```

classifies the cubic curves in  $\text{PG}(2, 4)$  containing at least 9 points. There are none. The command

### Example 298

```
cubic_curves_PG_2_4_draw:
> $(ORBITER) -v 3 \
> > -draw_layered_graph cc_4_poset_lvl_9.layered_graph \
> > > -radius 300 -embedded -line_width 1.1 \
> > > -y_stretch 0.9 -scale 0.25 \
> > -end
> pdflatex cc_4_poset_lvl_9_draw.tex
> $(OPEN) cc_4_poset_lvl_9_draw.pdf
```

draws the poset of orbits of the classification of 9-arcs, shown below:



The command

#### Example 299

```
poly_orbits.q4:
> $(ORBITER) -v 4 \
> > -draw.options -yout 500000 -radius 15 -nodes.empty \
> > > -line_width 0.5 -y_stretch 0.25 -embedded -end \
> > -define F -finite_field -q 4 -print_numerically -end \
> > -define R -polynomial_ring \
> > > -field F \
> > > -number_of_variables 3 \
```

```

> > > -homogeneous_of_degree 3 \
> > > -monomial_ordering_partition \
> > > -variables "X0,X1,X2" "X_0,X_1,X_2" \
> > > -end \
> > -define G -linear_group -PGGL 3 F -end \
> > -define Orb -orbits -group G \
> > > -on_polynomials R \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -report \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -export_something "orbit" 0 \
> > -end
> pdflatex poly_orbits_d3.n2.q4.tex
> $(OPEN) poly_orbits_d3.n2.q4.pdf

```

uses the basic Schreier algorithm to classify all cubic curves in  $PG(2, 4)$ . This is done using the action of the group  $PTL(3, 4)$  on all cubic forms over  $\mathbb{F}_4$ . The output is shown below:

## The Varieties of degree 3 in $PG(2, 4)$ , summary

### Objects with 13 Points

There are 1 objects with 13 Points:

orbit : rep : go : poly : Pts

4 : ( 11 ) : 576 :  $11=X_0^3 + X_1^3$  : 2,3,4,5,6,11,12,14,15,16,18,19,20

### Objects with 12 Points

There are 1 objects with 12 Points:

orbit : rep : go : poly : Pts

2 : ( 9 ) : 108 :  $9=X_0X_1X_2$  : 0,1,2,4,5,6,7,8,9,10,13,17

### Objects with 10 Points

There are 1 objects with 10 Points:

orbit : rep : go : poly : Pts

25 : ( 87743 ) : 20 :  $87743=X_0^3+2X_1^3+2X_2^3+X_0^2X_1+X_0^2X_2+X_0X_1X_2$  : 3,5,8,10,13,14,15,17,18,20

### Objects with 9 Points

There are 4 objects with 9 Points:

orbit : rep : go : poly : Pts

1 : ( 3 ) : 288 :  $3=X_0^2X_1$  : 0,1,2,7,8,9,10,13,17

6 : ( 18 ) : 432 :  $18=X_0^3 + X_1^3 + X_2^3$  : 4,5,6,7,8,9,10,13,17

8 : ( 24 ) : 54 :  $24=3X_0^3 + 2X_1^3 + X_2^3$  : 3,11,12,14,15,16,18,19,20

13 : ( 175 ) : 24 :  $175 = X_1^3 + X_2^3 + X_0^2 X_1 + X_0^2 X_2$  : 0,3,4,7,10,11,12,13,17

### Objects with 8 Points

There are 2 objects with 8 Points:

orbit : rep : go : poly : Pts

16 : ( 87383 ) : 12 :  $87383 = X_0^3 + X_0 X_1 X_2$  : 1,2,3,10,13,16,17,19

20 : ( 87466 ) : 4 :  $87466 = X_1^3 + X_2^3 + X_0^2 X_1 + X_0 X_1 X_2$  : 0,3,4,10,13,14,17,18

### Objects with 7 Points

There are 1 objects with 7 Points:

orbit : rep : go : poly : Pts

11 : ( 50 ) : 6 :  $50 = 2X_0^3 + X_1^3 + X_2^3 + X_0^2 X_1$  : 5,10,12,13,14,17,19

### Objects with 6 Points

There are 4 objects with 6 Points:

orbit : rep : go : poly : Pts

18 : ( 87404 ) : 6 :  $87404 = 2X_0^3 + X_1^3 + X_2^3 + X_0 X_1 X_2$  : 10,11,13,14,17,20

21 : ( 87468 ) : 3 :  $87468 = 2X_0^3 + X_1^3 + X_2^3 + X_0^2 X_1 + X_0 X_1 X_2$  : 5,10,13,15,16,17

23 : ( 87471 ) : 4 :  $87471 = X_0^3 + 2X_1^3 + X_2^3 + X_0^2 X_1 + X_0 X_1 X_2$  : 5,7,8,9,15,16

26 : ( 91990 ) : 60 :  $91990 = 2X_0^2 X_2 + X_1^2 X_2 + X_0 X_1 X_2$  : 0,1,2,4,5,6

### Objects with 5 Points

There are 4 objects with 5 Points:

orbit : rep : go : poly : Pts

0 : ( 0 ) : 5760 :  $0 = X_0^3$  : 1,2,10,13,17

9 : ( 34 ) : 192 :  $34 = 2X_0^3 + X_1^3 + X_0^2 X_1$  : 2,5,11,16,18

10 : ( 44 ) : 6 :  $44 = X_2^3 + X_0^2 X_1$  : 0,1,3,15,20

14 : ( 176 ) : 8 :  $176 = X_0^3 + X_1^3 + X_2^3 + X_0^2 X_1 + X_0^2 X_2$  : 10,13,16,17,19

### Objects with 4 Points

There are 3 objects with 4 Points:

orbit : rep : go : poly : Pts

3 : ( 10 ) : 12 :  $10 = X_0^3 + X_1^3 + X_2^3 + X_0^2 X_1 + X_0^2 X_2 + X_0 X_1^2 + X_1^2 X_2 + X_0 X_2^2 + X_1 X_2^2 + X_0 X_1 X_2$  : 3,4,7,10

17 : ( 87388 ) : 6 :  $87388 = 2X_0^3 + X_1^3 + X_0 X_1 X_2$  : 2,12,15,18

22 : ( 87470 ) : 2 :  $87470 = 2X_1^3 + X_2^3 + X_0^2 X_1 + X_0 X_1 X_2$  : 0,6,19,20

### Objects with 3 Points

There are 2 objects with 3 Points:

orbit : rep : go : poly : Pts

7 : ( 19 ) : 18 :  $19 = 2X_0^3 + X_1^3 + X_2^3$  : 10,13,17

15 : ( 180 ) : 9 : 180 =  $X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1 + X_0^2X_2$  : 5,15,18

### Objects with 2 Points

There are 1 objects with 2 Points:

orbit : rep : go : poly : Pts

24 : ( 87472 ) : 4 : 87472 =  $2X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1 + X_0X_1X_2$  : 11,12

### Objects with 1 Points

There are 2 objects with 1 Points:

orbit : rep : go : poly : Pts

5 : ( 12 ) : 288 : 12 =  $2X_0^3 + X_1^3$  : 2

12 : ( 55 ) : 48 : 55 =  $3X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1$  : 4

### Objects with 0 Points

There are 1 objects with 0 Points:

orbit : rep : go : poly : Pts

19 : ( 87409 ) : 126 : 87409 =  $3X_0^3 + 2X_1^3 + X_2^3 + X_0X_1X_2$  :

This classification involves curves that are degenerate or singular, or contain lines.



## Chapter 8

# Cubic Surfaces and Quartic Curves

### 8.1 Generalities

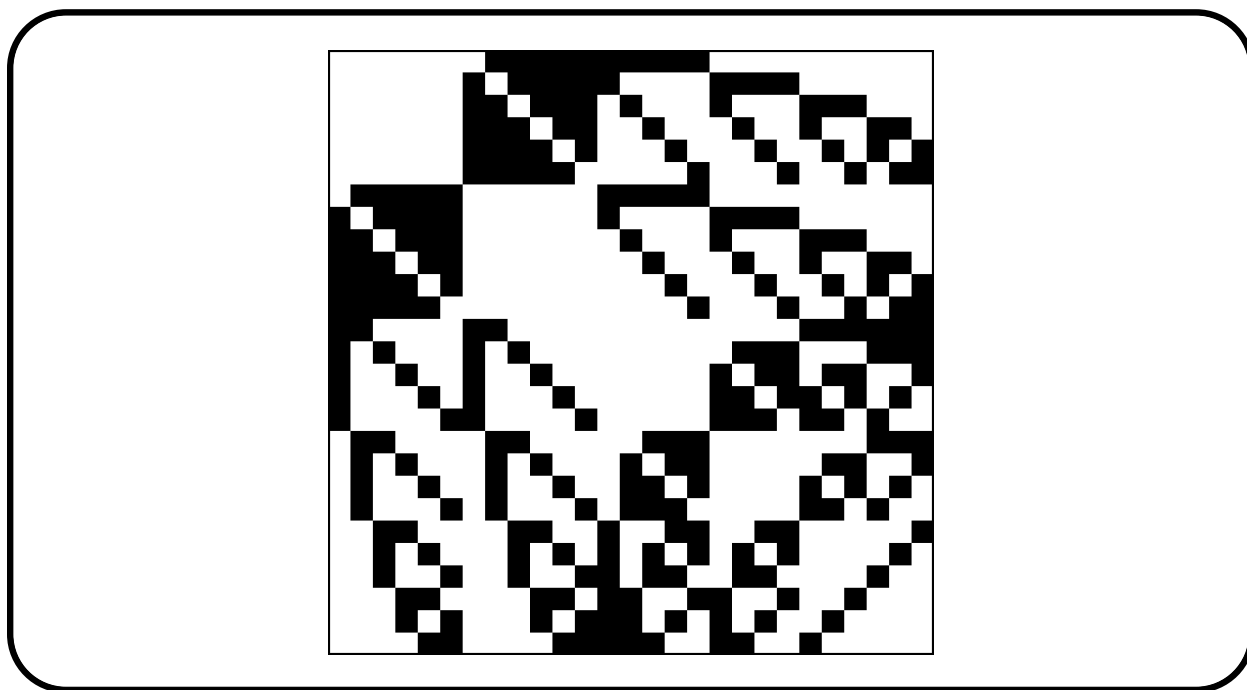
Orbiter can create, classify and investigate cubic surfaces over small finite fields. In this section, some global commands associated with cubic surfaces will be shown. Section 8.2 demonstrates how cubic surfaces can be created in Orbiter. Section 8.5 shows Orbiter commands related to quartic curves. Section 8.4 discusses how cubic surfaces can be classified. Section 8.7 covers the GAP interface for cubic surfaces.

The command

#### Example 300

```
surface_4.0_lines_vs_lines:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -end \
> > -with P -do \
> > -projective_space_activity \
> > > -export_cubic_surface_line_vs_line_incidence_matrix \
> > -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file PG_3.4_lines_vs_lines_incma.csv \
> > -box_width 32 -bit_depth 24 \
> > -partition 3 27 27 \
> > -end
> convert PG_3.4_lines_vs_lines_incma.draw.bmp PG_3.4_lines_vs_lines_incma.draw.png
```

exports the adjacency matrix of the graph of incident lines of a cubic surface. The lines are ordered w.r.t. the Schläfli labeling. The command invokes a projective space activity, and hence creates a projective space first. The order of the field is irrelevant. The command also produces a drawing of the adjacency matrix, shown below:



Using Schläefli labels, the lines are ordered

$$a_1, \dots, a_6, b_1, \dots, b_6, c_{12}, c_{13}, \dots, c_{56}.$$

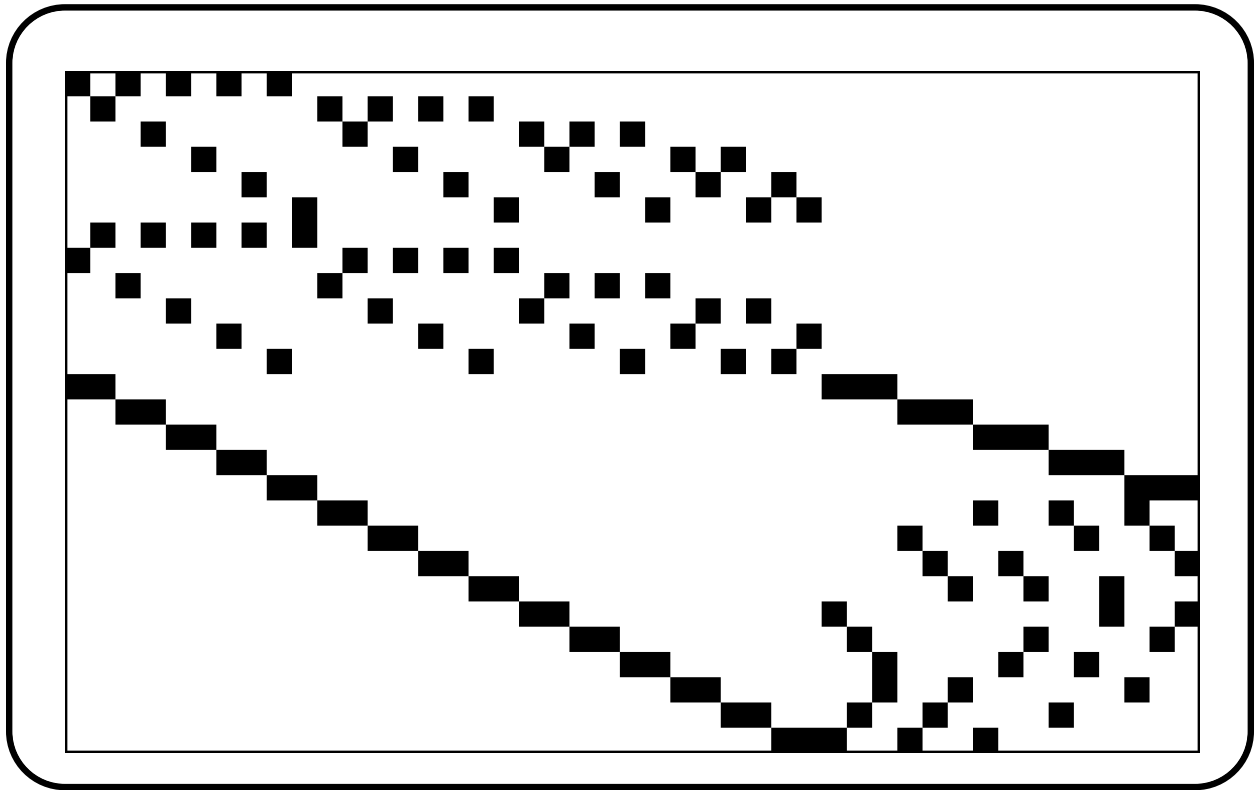
The command

#### Example 301

```
surface_4_0_lines_vs_tritangent_planes:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -end \
> > -with P -do \
> > -projective_space_activity \
> > > -export_cubic_surface_line_tritangent_plane_incidence_matrix \
> > -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file PG_3_4_lines_tritplanes_incma.csv \
> > -box_width 32 -bit_depth 24 \
> > -partition 3 27 45 \
> > -end
> convert PG_3_4_lines_tritplanes_incma_draw.bmp PG_3_4_lines_tritplanes_incma_dr
aw.png
```

exports the incidence matrix between lines and tritangent planes of a cubic surface. The lines are ordered w.r.t. the Schläefli labeling. The tritangent planes are ordered w.r.t. the Schläefli labeling. The command invokes a projective space activity, and hence creates a projective space first. The order of the field is irrelevant. The command also produces a drawing of the incidence matrix, shown below:





| Creating a Cubic Surface (Part 1) |                     |                                                                                                                                                                                                                                                                      |
|-----------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                           | Arguments           | Purpose                                                                                                                                                                                                                                                              |
| -space                            | P                   | Specify the 3-dimensional projective space $P = PG(3, q)$ .                                                                                                                                                                                                          |
| -label_txt                        | label               | Override the ascii label of the curve.                                                                                                                                                                                                                               |
| -label_tex                        | label               | Override the latex label of the curve.                                                                                                                                                                                                                               |
| -label_for_summary                | label               | Override the ascii label of the curve, to be used in summary commands.                                                                                                                                                                                               |
| -catalogue                        | $i$                 | Create the $i$ -th surface in the Orbiter catalogue. Here, $i$ is an index variable used to index all surfaces in $PG(3, q)$ . The index $i$ is zero-based. The automorphism group is created as well.                                                               |
| -by_coefficients                  | list-of-coeff-pairs | Create a surface from a list of coefficient-monomial pairs. The automorphism group is not created.                                                                                                                                                                   |
| -by_rank                          | rank $q_0$          | Create a surface from the numerical rank of the equation over the subfield $\mathbb{F}_{q_0}$ . Here, we think of the equation as a point in $PG(19, q_0)$ and use the Orbiter point rank.                                                                           |
| -family_Eckardt                   | $a$ $b$             | Create the Eckardt surface with parameters $(a, b)$ as in see [13] (where it is called the Hilbert, Cohn-Vossen surface). The equation is $X_3^3 - b^2(X_0^2 + X_1^2 + X_2^2)X_3 + \frac{b^3}{a}(a^2 + 1)X_0X_1X_2 = 0$ . The automorphism group is created as well. |

Table 8.1: Creating a Cubic Surface (Part 1)

## 8.2 Creating Cubic Surfaces

In this section, we describe ways in which surfaces can be created in Orbiter.

Orbiter contains a built-in catalogue of cubic surfaces with 27 lines for small finite fields  $\mathbb{F}_q$ . The surfaces in the catalogue all come with their automorphism group. It is also possible to create surfaces from known families, or to create surfaces from associated objects like 6-arcs. Some of these constructions only create the surface, not the automorphism group.

Tables 8.1-8.3 summarize the Orbiter commands that can be used to create cubic surfaces. The commands require a projective space object. Not all of the surfaces created may have 27 lines, and some of the constructions may yield degenerate surfaces.

Let us look at some examples. The next command creates the unique surface with 27 lines over the field  $\mathbb{F}_4$ . It is the Hirschfeld surface. In this command, the surface is pulled from Orbiter's built-in catalogue of cubic surfaces. The surface has Orbiter Catalogue Number (OCN) equal to 0.

### Example 302

```
surface_4.0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -end \
```

| Creating a Cubic Surface (Part 2) |              |                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                           | Arguments    | Purpose                                                                                                                                                                                                                                                                                                                       |
| -family_G13                       | $a$          | Create a member of the $G_{13}$ family with parameter $a$ . The surface has 13 Eckardt points.                                                                                                                                                                                                                                |
| -family_F13                       | $a$          | Create a member of the $F_{13}$ family with parameter $a$ . The surface has 13 Eckardt points.                                                                                                                                                                                                                                |
| -family_bes                       | $a\ c$       | Create a member of the “bes”-family with parameter $a$ . The surface has 5 Eckardt points.                                                                                                                                                                                                                                    |
| -family_general_abcd              | $a\ b\ c\ d$ | Create a member of the general family with parameters $a, b, c, d$ .                                                                                                                                                                                                                                                          |
| -arc_lifting                      | $A$          | Create the surface associated with the arc $A = a_1, \dots, a_6$ in $\text{PG}(2, q)$ by means of the Clebsch map. Each of the $a_i$ is the rank of a point in $\text{PG}(2, q)$ . Use the trihedral pair algorithm. Here, $A$ is a comma-separated string containing the numerical ranks of the $P_i$ in $\text{PG}(2, q)$ . |
| -arc_lifting_with_two_lines       | $A\ L$       | Create the surface associated with the arc $A = a_1, \dots, a_6$ in $\text{PG}(2, q)$ by means of the Clebsch map, defined by the lines $\ell_1$ and $\ell_2$ whose ranks are given in $L$ . If both of the lines are given as 0, the program will pick a suitable set of lines automatically.                                |

Table 8.2: Creating a Cubic Surface (Part 2)

| Creating a Cubic Surface (Part 3) |                                                               |                                                                                                                                                                                                                                                                                               |
|-----------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                           | Arguments                                                     | Purpose                                                                                                                                                                                                                                                                                       |
| -Cayley_form                      | $k\ l\ m\ n$                                                  | Create the surface from Cayley's normal form, using the parameters $k, l, m, n$ .                                                                                                                                                                                                             |
| -by_equation                      | R label label_tex vars<br>eqn para<br>para_tex<br>para_values | Create the surface from an equation.                                                                                                                                                                                                                                                          |
| -by_symbolic_object               | R formula                                                     | Create the surface from an equation. Here, R is a ring and formula is the equation of the surface.                                                                                                                                                                                            |
| -by_double_six                    | label label_tex $D$                                           | Create the surface from a double six $D$ .                                                                                                                                                                                                                                                    |
| -by_skew_hexagon                  | label label                                                   | Create the surface from a skew hexagon.                                                                                                                                                                                                                                                       |
| -select_double_six                | $L$                                                           | Relabel the lines by choosing the 12 lines in $L$ as new double six. The entries in $L$ are line indices with respect to the old double six. They are integers in the interval $[0, 26]$ . This command can be repeated. In each application, the double six refers to the previous labeling. |
| -override_group                   | descr                                                         | Override the automorphism group of the surface by the given group.                                                                                                                                                                                                                            |
| -random                           |                                                               | Create a random cubic surface with 27 lines over the given field.                                                                                                                                                                                                                             |
| -transform                        | elt                                                           | Apply the transformation given by the group element.                                                                                                                                                                                                                                          |
| -transform_inverse                | elt                                                           | Apply the inverse transformation given by the group element.                                                                                                                                                                                                                                  |

Table 8.3: Creating a Cubic Surface (Part 3)

```
▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
```

There is a 4-parameter normal form  $\mathcal{F}_{abcd}$  for cubic surfaces with 27 lines. Here,  $a, b, c, d$  are field elements in  $\mathbb{F}_q$ . The following command creates the surface  $\mathcal{F}_{abcd}$  with  $(a, b, c, d) = (2, 3, 3, 4)$  over the field  $\mathbb{F}_7$ :

#### Example 303

```
Family_general_F7:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define S7_abcd_2_3_3_4 -cubic_surface \
▷ ▷ ▷ -space P -family_general_abcd 2 3 3 4 \
▷ ▷ -end
```

This command uses a built-in command to realize the surface of type  $\mathcal{F}_{abcd}$ . We will later see that it is possible to create surfaces symbolically from an equation.

Another way of creating a cubic surface is as member of a known infinite family. However, different choices of the parameters may lead to isomorphic surfaces. For instance,

#### Example 304

```
surface_eckardt_13_4_12:
▷ $(ORBITER) -v 6 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define Eckardt_4_12 -cubic_surface \
▷ ▷ ▷ -space P -family_Eckardt 4 12 \
▷ ▷ -end
```

creates the member of the Eckardt family described in [13] with parameters  $(a, b) = (4, 12)$  over the field  $\mathbb{F}_{13}$ . On the other hand, the command

#### Example 305

```
Eckardt_13:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define Eckardt_3_1 -cubic_surface \
▷ ▷ ▷ -space P -family_Eckardt 3 1 \
▷ ▷ -end
```

creates the Eckardt surface over the field  $\mathbb{F}_{13}$  from the parameter set  $(a, b) = (3, 1)$ . The two surfaces are isomorphic.

It is possible to create a random cubic surface of order  $q$ . The following example creates a random cubic surface over the field  $\mathbb{F}_{11}$ :

**Example 306**

```

surface_11_random:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 11 -end \
> > -define P -projective_space -n 3 -field F -v 6 -end \
> > -define S -cubic_surface -space P -random -end

```

The only restriction is that  $q$  is within the range for which the classification of cubic surfaces is available in Orbiter.

A surface can be created symbolically, starting from an equation. For example, consider the surface of type  $\mathcal{F}_{abcd}$  from above. We encode the formula in algebraic notation as a makefile variable. Here it is important to note that we use notation such as  $1 + 1 + \dots + 1$  ( $k$  times) for the integer  $k$ . This makes the formula valid over very small fields such as  $\mathbb{F}_3$  and  $\mathbb{F}_4$  where the Orbiter conventions for field elements prohibit using 3 or 4. There is no automatic reduction modulo the characteristic when specifying field elements.

**Example 307**

```

F_abcd_eqn="(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(b - d)*X0^2*X2 \
+ (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(a + b - c - d)*X0*X1*X2 \
+ (a^2*c - a^2*d - a*c^2 + b*c^2 + a*d - b*c)*(b - d)*X0*X1*X3 \
- (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X0*X2^2 \
- (a^2*c*d - a*b*c^2 - a^2*d + a*b*d + b*c^2 - b*c*d)*(b - d)*X0*X2*X3 \
- (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X2 \
- (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X3 \
+ (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1*X2^2 \
+ ((1+1)*a^2*b*c*d - a^2*b*d^2 - (1+1)*a^2*c*d^2 \
- (1+1)*a*b^2*c^2 + a*b^2*c*d + (1+1)*a*b*c^2*d + a*b*c*d^2 \
- b^2*c^2*d - a^2*b*c + a^2*c*d + a^2*d^2 + a*b^2*c + a*b*c^2 \
- (1+1+1+1)*a*b*c*d - a*c^2*d + a*c*d^2 + b^2*c^2)*X1*X2*X3 \
+ c*a*(a*d - b*c - a + b + c - d)*(b - d)*X1*X3^2"

```

The following command chooses  $(a, b, c, d) = (2, 30, 30, 2)$  and creates the surface over  $\mathbb{F}_{31}$ .

**Example 308**

```

surface_F_abcd.Eckardt.q31_by_equation:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 31 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define R -polynomial_ring \
> > > -field F \
> > > -number_of_variables 4 \
> > > -homogeneous_of_degree 3 \
> > > -monomial_ordering_partition \
> > > -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
> > > -end \
> > -define F_abcd -cubic_surface -space P \
> > > -by_equation \
> > > R \

```

```

> > > "F_abcd" \
> > > "\DF_{a,b,c,d}\D" "X0,X1,X2,X3" \
> > > $(F_abcd_eqn) \
> > > "a,b,c,d" \
> > > "\D(a,b,c,d)=(2,30,30,2)\D" \
> > > "2,30,30,2" \
> > -end

```

The surface is of Eckardt type.

A cubic surface is determined uniquely up to isomorphism by a non-conical 6-arc in a plane. The points of the arc are given in Orbiter ranks. The following command chooses the arc 0, 1, 2, 3, 43, 113 and creates the associated cubic surface over  $\mathbb{F}_{13}$ .

#### Example 309

```

surface_q13_Eckardt_by_arc_lifting:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define S -cubic_surface -space P -arc_lifting "0,1,2,3,43,113" -end \
> > -with S -do \
> > -cubic_surface_activity \
> > > -report \
> > -end

```

The surface is of Eckardt type.

| Cubic Surface Activities             |                          |                                                                                                                                           |
|--------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Command                              | Arguments                | Purpose                                                                                                                                   |
| -report                              |                          | Produce a latex report about the cubic surface.                                                                                           |
| -report_group_elements               | csv-file head-<br>ing    | Report properties of elements of the automorphism group. The group elements must be given in make-element-form in a column in a csv-file. |
| -export_something                    | type                     | Exports data about the surface to a file. The type of data to be exported must be specified according to Table 8.5.                       |
| -export_gap                          |                          | Export a cubic surface to GAP. See Section 8.7.                                                                                           |
| -all_quartic_curves                  |                          | Creates all quartic curves with 28 bitangents from the surface by projecting along the tangent cone of a point not on any line.           |
| -export_all_quartic_curves           |                          | Export all quartic curves arising from the orbits of points not on lines of the surface. The curves are written to a csv file.            |
| -export_something_with_group_element | what label               | Exports data about the surface to a file. The type of data to be exported must be specified according to Table 8.5.                       |
| -action_on_module                    | type basis<br>generators |                                                                                                                                           |
| -Clebsch_map_up                      | line1 line2              | Define the birational Clebsch map from a plane to the surface. The skew lines line1 and line2 are used to define the map.                 |
| -Clebsch_map_up_single_point         | pt line1 line2           | Map a single point to the surface under the Clebsch map defined by the pair of lines line1 and line2.                                     |

Table 8.4: Cubic Surface Activities

### 8.3 Cubic Surface Activities

Table 8.4 lists activities for a cubic surface object.

Table 8.5 lists options for exporting data.

The next command creates the Hirschfeld surface over  $\mathbb{F}_4$  and produces a report for it:

#### Example 310

```

surface_4.0_report:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -end \
▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -report \
▷ ▷ -end

```



| Options for Exporting Data: Cubic Surfaces |                                                                                         |
|--------------------------------------------|-----------------------------------------------------------------------------------------|
| Command                                    | Purpose                                                                                 |
| points                                     | The points on the surface.                                                              |
| points_off                                 | The points off the surface.                                                             |
| Eckardt_points                             | The Eckardt points of the surface.                                                      |
| double_points                              | The double points of the surface.                                                       |
| single_points                              | The single points of the surface.                                                       |
| zero_points                                | The zero points of the surface. A point is a zero point if it does not lie on any line. |
| singular_points                            | The singular points of the surface.                                                     |
| lines                                      | The set of lines of the surface.                                                        |
| tritangent_planes                          | The set of tritangent planes of the surface.                                            |
| Hesse_planes                               | The set of Hesse planes of the surface.                                                 |

Table 8.5: Options for Exporting Data: Cubic Surfaces

```

▷ pdflatex surface_catalogue_q4_iso0_report.tex
▷ $(OPEN) surface_catalogue_q4_iso0_report.pdf

```

The following command creates the Hirschfeld surface over  $\mathbb{F}_4$  and exports some data:

**Example 311**

```

surface_4_0_export:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -end \
▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -export_something "points" \
▷ ▷ -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -export_something "points_off" \
▷ ▷ -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -export_something "lines" \
▷ ▷ -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -export_something "Hesse_planes" \
▷ ▷ -end

```

The following data is exported: the set of points on and off the surface, the set of lines, and the set of Hesse planes.

The following command creates the cubic surface over  $\mathbb{F}_7$  with catalogue number 0. It also writes a report and exports the set of lines:

#### Example 312

```
surface_7_0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 7 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define S -cubic_surface -space P -catalogue 0 -end \
> > -with S -do \
> > -cubic_surface_activity \
> > > -report \
> > -end \
> > -with S -do \
> > -cubic_surface_activity \
> > > -export_something "lines" \
> > -end
> pdflatex surface_catalogue_q7_iso0_report.tex
> $(OPEN) surface_catalogue_q7_iso0_report.pdf
```

It is often desired to produce a nice equation of a cubic surface. This can be done by applying a change of coordinates. In the following example, a “nice” equation for the cubic surface over  $\mathbb{F}_{16}$  with catalogue number 0 is produced:

#### Example 313

```
surface_16_0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 16 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define S16_0 -cubic_surface -space P -catalogue 0 \
> > > -transform "1,0,0,0,0,1,0,12,0,0,1,12,0,0,0,1,0" \
> > > -transform "15,11,4,0,0,0,12,0,0,12,0,0,0,0,0,1,3" \
> > -end \
> > -with S16_0 -do \
> > -cubic_surface_activity \
> > > -report \
> > -end
```

The following command parses the formula and creates the surface with  $(a, b, c, d) = (2, 30, 30, 2)$  over  $\mathbb{F}_{31}$ :

#### Example 314

```
surface_F_abcd.Eckardt.q31:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 31 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define R -polynomial_ring \
> > > -field F \
```

```

▷ ▷ ▷ -number_of_variables 4 \
▷ ▷ ▷ -homogeneous_of_degree 3 \
▷ ▷ ▷ -monomial_ordering_partition \
▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
▷ ▷ ▷ -end \
▷ ▷ -define F_abcd -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text $(F_abcd_eqn) \
▷ ▷ ▷ -end \
▷ ▷ -define abcd_values -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "2,30,30,2" \
▷ ▷ ▷ -end \
▷ ▷ -define F_abcd_sub -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
▷ ▷ ▷ -substitute "a,b,c,d" F_abcd abcd_values \
▷ ▷ -end \
▷ ▷ -define S -cubic_surface -space P \
▷ ▷ ▷ -by_symbolic_object \
▷ ▷ ▷ R \
▷ ▷ ▷ F_abcd_sub \
▷ ▷ -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex surface_equation.F_abcd_sub.q31_report.tex
▷ $(OPEN) surface_equation.F_abcd_sub.q31_report.pdf

```

The command creates a report which shows information about the surface. The surface has 6 Eckardt points.

In this case, the following surface is created:

The equation of the surface is :

$$\begin{aligned}
 &X_0^3 + 2X_1^3 + 4X_2^3 + 7X_3^3 + 7X_0^2X_1 + 2X_0^2X_2 + 2X_0^2X_3 + 9X_1^2X_2 + 9X_1^2X_3 + 6X_0X_2^2 \\
 &+ 9X_1X_2^2 + 4X_2^2X_3 + 10X_0X_3^2 + 9X_1X_3^2 + 3X_2X_3^2 + 8X_0X_1X_2 + 4X_0X_1X_3 = 0
 \end{aligned}$$

( 1, 2, 4, 7, 7, 2, 2, 0, 9, 9, 6, 9, 4, 10, 9, 3, 8, 4, 0, 0 )

**Summary**

| Object                  | Number                    | Orbit type             |
|-------------------------|---------------------------|------------------------|
| Lines                   | 27                        | 12, 15                 |
| Points on surface       | 199                       | 10, 15, 24, 30, $60^2$ |
| Singular points         | 0                         |                        |
| Eckardt points          | 10                        | 10                     |
| Double points           | 105                       | 15, 30, 60             |
| Single points           | 84                        | 24, 60                 |
| Points off lines        | 0                         |                        |
| Hesse planes            | 0                         |                        |
| Axes                    | 10                        | 10                     |
| Single sixes            | 72                        | 2, $20^2$ , 30         |
| Double sixes            | 36                        | 1, $10^2$ , 15         |
| Tritangent planes       | 45                        | 5, 10, 30              |
| Trihedral pairs         | 120                       | 10, $15^2$ , 20, 60    |
| Type of points on lines | $12^{27}$                 |                        |
| Type of lines on points | $3^{10}, 2^{105}, 1^{84}$ |                        |

| Activity for Cubic Surfaces After Classification |             |                                                                                                         |
|--------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------|
| Command                                          | Arguments   | Purpose                                                                                                 |
| -report                                          | options     | Create a latex report.                                                                                  |
| -stats                                           | prefix      | Report statistics on group operations.                                                                  |
| -identify_Eckardt                                |             |                                                                                                         |
| -identify_F13                                    |             |                                                                                                         |
| -identify_Bes                                    |             |                                                                                                         |
| -identify_general_abcd                           |             |                                                                                                         |
| -isomorphism_testing                             | surf1 surf2 | Compute an isomorphism from surface 1 to surface 2. surf1 and surf2 must be surface objects.            |
| -recognize                                       | surf        | Recognizes the isomorphism type of the given surface object in the Orbiter Catalogue of cubic surfaces. |
| -create_source_code                              |             | Create C++ source code for the cubic surfaces of the given order.                                       |
| -sweep_Cayley                                    |             | Identify the isomorphism type of all surfaces of Cayley type.                                           |

Table 8.6: Activity for Cubic Surfaces After Classification

## 8.4 Classification of Cubic Surfaces

There are several different approaches to classify cubic surfaces with 27 lines over finite fields  $\mathbb{F}_q$  in Orbiter. Classification means to determine the non-equivalent surfaces under the action of the collineation group  $\text{P}\Gamma\text{L}(4, q)$  of  $\text{PG}(3, q)$ . The approach described in [13] relies on Schlaefli's notion of a double six as a substructure [60]. The approach described in [38] utilizes the relation to non-conical six-arcs in a plane. A third approach is described in [39]. All three approaches are available in Orbiter.

Table 8.6 lists the Orbiter commands that can be performed once the classification is completed.

In  $\text{PG}(3, 4)$ , there is only one type of cubic surfaces with 27 lines. It is a member of the Hirschfeld family, described in [33]. The following Orbiter command can be used to construct this surface and to prove its uniqueness for  $\mathbb{F}_4$ . The following command utilizes the algorithm of [13] to do so:

### Example 315

```

surface_classify_q4:
▷ $(ORBITER) -v 5 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define Control -poset_classification_control -W \
▷ ▷ ▷ -problem_label "surf_q4" \
▷ ▷ -end \
▷ ▷ -define report_options -poset_classification_report_options \
▷ ▷ -end \
▷ ▷ -define Orb -orbits \
▷ ▷ ▷ -on_cubic_surfaces P Control \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \

```

```

▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex Surfaces_q4.tex
▷ $(OPEN) Surfaces_q4.pdf

```

The `-report` option creates a latex report. After some redactions, the report contains the following elements.

## The semilinear group

### The Action

Group action  $\text{PTL}(4, 4)$  of degree 85

The group is a matrix group.

⋮

The base action is on projective space  $\text{PG}(3, 4)$

$q = 4$

$p = 2$

$e = 2$

$n = 3$

Number of points = 85

Number of lines = 357

Number of lines on a point = 21

Number of points on a line = 5

⋮

## The orthogonal group

### The Action

Group action  $\text{PTL}(4, 4)\text{OnWedge}$  of degree 1365

The group is a matrix group.

The base action is on projective space  $\text{PG}(3, 4)$

$q = 4$

$p = 2$

$e = 2$

$n = 3$

Number of points = 85

Number of lines = 357

Number of lines on a point = 21

Number of points on a line = 5

⋮

## The group stabilizing the fixed line

### The Action

Group action PTL(4, 4)OnWedgeres100 of degree 100

⋮

Strong generators for a group of order 5529600: ⋮

## The classification of five-plus-ones

Poset classification up to depth 5

### The Orbits

#### Number of Orbits By Level

| Depth | Nb of orbits |
|-------|--------------|
| 0     | 1            |
| 1     | 1            |
| 2     | 1            |
| 3     | 1            |
| 4     | 3            |
| 5     | 4            |

### Summary of Orbit Representatives

N = node

D = depth or level

O = orbit with a level

Rep = orbit representative

(S,O) = (order of stabilizer, orbit length)

L = number of live points

F = number of flags

Gen = number of generators for the stabilizer of the orbit rep.

Table 8.7: Orbit Representatives

| N | D | O | Rep              | (S,O)        | L   | F | Gen |
|---|---|---|------------------|--------------|-----|---|-----|
| 0 | 0 | 0 | { }              | (5529600, 1) | 100 | 1 | 12  |
| 1 | 1 | 0 | { 0 }            | (55296, 100) | 64  | 1 | 9   |
| 2 | 2 | 0 | { 0, 3 }         | (1728, 3200) | 36  | 1 | 7   |
| 3 | 3 | 0 | { 0, 3, 56 }     | (144, 38400) | 16  | 3 | 6   |
| 4 | 4 | 0 | { 0, 3, 56, 76 } | (288, 19200) | 4   | 2 | 7   |
| 5 | 4 | 1 | { 0, 3, 56, 77 } | (96, 57600)  | 4   | 1 | 6   |

|    |   |   |                      |              |   |   |    |
|----|---|---|----------------------|--------------|---|---|----|
| 6  | 4 | 2 | { 0, 3, 56, 80 }     | (72, 76800)  | 4 | 2 | 6  |
| 7  | 5 | 0 | { 0, 3, 56, 76, 96 } | (1440, 3840) |   |   | 10 |
| 8  | 5 | 1 | { 0, 3, 56, 76, 97 } | (96, 57600)  |   |   | 5  |
| 9  | 5 | 2 | { 0, 3, 56, 80, 92 } | (360, 15360) |   |   | 7  |
| 10 | 5 | 3 | { 0, 3, 56, 80, 93 } | (120, 46080) |   |   | 7  |



## Poset of Orbits in Detail

⋮

### Classification of 5 + 1 Configurations in PG(3, 4)

The order of the group is 1974067200

The group has 4 orbits on five plus one configurations in PG(3, 4).

Of these, 1 impose 19 conditions.

Of these, 1 are associated with double sixes. They are:

0/1 is orbit 3/4  $\{0, 3, 56, 80, 93\}_{120}$  orbit length 46080

The overall number of five plus one configurations associated with double sixes in PG(3, 4) is: 46080

### Flag orbits for double sixes

The number of primary orbits below is 4

The number of primary orbits above is 1

The number of flag orbits is 1

The flag orbits are:

- (1) Flag orbit 0 / 1 down=(3,0) up=(0,-1) is ( 0, 3, 56, 80, 93, 16, 340, 38, 61, 156, 0, 16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0 ) with a stabilizer of order 120

Strong generators for a group of order 120:

$$\begin{aligned} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ \omega^2 & 0 & \omega & 0 \\ 0 & \omega^2 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & \omega^2 & 1 \end{bmatrix}_1, \\ & \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & 0 & 0 \\ \omega & 0 & \omega & 0 \\ \omega^2 & 1 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 1 & 0 & 0 \\ \omega^2 & 0 & 0 & 0 \\ 0 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & 0 \end{bmatrix}_1, \\ & \begin{bmatrix} 1 & \omega & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \omega & \omega & 1 & \omega^2 \\ 0 & \omega^2 & 0 & 1 \end{bmatrix}_1 \end{aligned}$$

1,0,0,0,0,3,0,0,0,0,3,0,0,0,1,1,  
 1,0,0,0,0,2,0,0,3,0,2,0,0,3,0,1,1,  
 1,0,0,0,3,2,0,0,0,0,2,0,0,0,3,1,1,  
 1,0,0,0,3,3,0,0,0,0,3,0,0,0,1,1,0,  
 1,0,0,0,3,2,0,0,2,0,2,0,3,1,3,1,0,  
 1,1,0,0,3,0,0,0,0,0,3,3,0,0,1,0,1,

1,2,0,0,0,1,0,0,2,2,1,3,0,3,0,1,1,  
nb received = 0

### Double Sixes

The order of the group is 1974067200

The group has 1 orbits:

The orbits are:

- (1) 0/1 {16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0}<sub>1440</sub> orbit length 1370880  
Strong generators for a group of order 1440:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ \omega^2 & 0 & \omega^2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega^2 & \omega^2 & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega & 0 & \omega & 0 \\ 1 & \omega & 0 & 0 \\ \omega & 1 & \omega & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 1 & \omega^2 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega^2 & \omega & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 1 & 0 & 0 \\ \omega^2 & 0 & 0 & 0 \\ 0 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & 0 \end{bmatrix}_1$$

1,0,0,0,0,3,0,0,0,0,3,0,0,0,0,1,1,  
1,0,0,0,0,3,0,0,3,0,3,0,0,1,0,1,0,  
1,0,0,0,3,0,2,0,2,2,0,0,3,3,1,1,0,  
1,0,0,0,2,0,2,0,1,2,0,0,2,1,2,1,1,  
0,0,1,0,0,0,2,1,1,0,3,0,3,1,3,2,0,  
1,1,0,0,3,0,0,0,0,0,3,3,0,0,1,0,1,

The overall number of objects is: 1370880

### Flag orbits for surfaces

The number of primary orbits below is 1

The number of primary orbits above is 1

The number of flag orbits is 1

The flag orbits are:

- (1) Flag orbit 0 / 1 down=(0,0) up=(0,-1) is ( 16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0, 71, 55, 26, 100, 1, 138, 109, 345, 84, 85, 122, 110, 145, 139, 81 ) with a stabilizer of order 1440  
Strong generators for a group of order 1440:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ \omega^2 & 0 & \omega^2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega^2 & \omega^2 & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega & 0 & \omega & 0 \\ 1 & \omega & 0 & 0 \\ \omega & 1 & \omega & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 1 & \omega^2 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega^2 & \omega & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 1 & 0 & 0 \\ \omega^2 & 0 & 0 & 0 \\ 0 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & 0 \end{bmatrix}_1$$

1,0,0,0,0,3,0,0,0,0,3,0,0,0,1,1,  
 1,0,0,0,0,3,0,0,3,0,3,0,0,1,0,1,0,  
 1,0,0,0,3,0,2,0,2,2,0,0,3,3,1,1,0,  
 1,0,0,0,2,0,2,0,1,2,0,0,2,1,2,1,1,  
 0,0,1,0,0,0,2,1,1,0,3,0,3,1,3,2,0,  
 1,1,0,0,3,0,0,0,0,0,3,3,0,0,1,0,1,  
 nb received = 0

## Surfaces

The order of the group is 1974067200

The group has 1 orbits:

The orbits are:

- (1) 0/1 {16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0, 71, 55, 26, 100, 1, 138, 109, 345, 84, 85, 122, 110, 145, 139, 81}<sub>51840</sub> orbit length 38080  
 Strong generators for a group of order 51840:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & \omega & 0 \\ 0 & 0 & \omega & 0 \\ 1 & 0 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega & \omega & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega^2 & \omega & \omega^2 & 1 \\ \omega^2 & 0 & 1 & 0 \\ \omega & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} \omega & \omega & 1 & 1 \\ 0 & 1 & 0 & \omega \\ \omega & \omega^2 & 0 & \omega \\ 0 & 1 & 0 & 0 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,  
 1,0,0,0,0,2,0,0,0,0,2,0,0,0,1,0,  
 1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0,  
 1,0,0,0,0,1,0,0,1,1,1,0,1,1,0,1,0,  
 1,0,0,0,3,2,2,0,0,0,2,0,1,0,3,1,0,  
 1,0,0,0,1,0,2,0,2,2,0,0,2,2,1,1,0,  
 1,3,1,2,1,0,2,0,3,2,0,0,2,0,0,0,0,  
 1,1,3,3,0,3,0,1,1,2,0,1,0,3,0,0,0,

The overall number of objects is: 38080

**The Group PGL(4, 4)**

The order of the group is 1974067200

**Cubic Surfaces with 27 Lines in PG(3, 4)**

The order of the group is 1974067200

The group has 1 orbits:

The orbits are:

- (1) 0/1 {16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0, 71, 55, 26, 100, 1, 138, 109, 345, 84, 85, 122, 110, 145, 139, 81}<sub>51840</sub> orbit length 38080

Strong generators for a group of order 51840:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & \omega & 0 \\ 0 & 0 & \omega & 0 \\ 1 & 0 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega & \omega & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega^2 & \omega & \omega^2 & 1 \\ \omega^2 & 0 & 1 & 0 \\ \omega & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} \omega & \omega & 1 & 1 \\ 0 & 1 & 0 & \omega \\ \omega & \omega^2 & 0 & \omega \\ 0 & 1 & 0 & 0 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1,  
 1,0,0,0,0,2,0,0,0,0,2,0,0,0,0,1,0,  
 1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0,  
 1,0,0,0,0,1,0,0,1,1,1,0,1,1,0,1,0,  
 1,0,0,0,3,2,2,0,0,0,2,0,1,0,3,1,0,  
 1,0,0,0,1,0,2,0,2,2,0,0,2,2,1,1,0,  
 1,3,1,2,1,0,2,0,3,2,0,0,2,0,0,0,0,  
 1,1,3,3,0,3,0,1,1,2,0,1,0,3,0,0,0,

The overall number of objects is: 38080

**Surface 4#0****The equation**

The equation of the surface is :

$$X_0^2 X_3 + X_1^2 X_2 + X_1 X_2^2 + X_0 X_3^2 = 0$$

( 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0 )

Number of points on the surface 45

The automorphism group of the surface has order 51840

The automorphism group is the following group

Strong generators for a group of order 51840:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & \omega & 0 \\ 0 & 0 & \omega & 0 \\ 1 & 0 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega & \omega & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega^2 & \omega & \omega^2 & 1 \\ \omega^2 & 0 & 1 & 0 \\ \omega & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} \omega & \omega & 1 & 1 \\ 0 & 1 & 0 & \omega \\ \omega & \omega^2 & 0 & \omega \\ 0 & 1 & 0 & 0 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,  
 1,0,0,0,0,2,0,0,0,0,2,0,0,0,1,0,  
 1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0,  
 1,0,0,0,0,1,0,0,1,1,1,0,1,1,0,1,0,  
 1,0,0,0,3,2,2,0,0,0,2,0,1,0,3,1,0,  
 1,0,0,0,1,0,2,0,2,2,0,0,2,2,1,1,0,  
 1,3,1,2,1,0,2,0,3,2,0,0,2,0,0,0,0,  
 1,1,3,3,0,3,0,1,1,2,0,1,0,3,0,0,0,

## General information

Points on lines:

$$5^{27}$$

Lines on points:

$$3^{45}$$

## The 27 Lines

$$\ell_0 = a_1 = \begin{bmatrix} 1 & 0 & \omega^2 & 0 \\ 0 & 1 & 1 & \omega \end{bmatrix}_{72} = \begin{bmatrix} 1 & 0 & 3 & 0 \\ 0 & 1 & 1 & 2 \end{bmatrix}_{72} = \mathbf{Pl}(3, 2, 3, 0, 3, 1)_{308}$$

$$\ell_1 = a_2 = \begin{bmatrix} 1 & 0 & \omega & 0 \\ 0 & 1 & 0 & \omega^2 \end{bmatrix}_{54} = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 3 \end{bmatrix}_{54} = \mathbf{Pl}(2, 3, 0, 0, 2, 1)_{238}$$

$$\ell_2 = a_3 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{25} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{25} = \mathbf{Pl}(1, 1, 0, 0, 1, 1)_{177}$$

$$\ell_3 = a_4 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{356} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{356} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)_1$$

$$\begin{aligned}
\ell_4 = a_5 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0 = \mathbf{Pl}(1, 0, 0, 0, 0, 0)_0 \\
\ell_5 = a_6 &= \begin{bmatrix} 1 & 0 & \omega^2 & 1 \\ 0 & 1 & 0 & \omega \end{bmatrix}_{155} = \begin{bmatrix} 1 & 0 & 3 & 1 \\ 0 & 1 & 0 & 2 \end{bmatrix}_{155} = \mathbf{Pl}(3, 2, 0, 2, 3, 1)_{314} \\
\ell_6 = b_1 &= \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{340} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{340} = \mathbf{Pl}(0, 0, 0, 1, 0, 0)_9 \\
\ell_7 = b_2 &= \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{38} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{38} = \mathbf{Pl}(0, 0, 1, 1, 1, 1)_{198} \\
\ell_8 = b_3 &= \begin{bmatrix} 1 & \omega & 0 & 0 \\ 0 & 0 & 1 & \omega^2 \end{bmatrix}_{61} = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 3 \end{bmatrix}_{61} = \mathbf{Pl}(0, 0, 2, 3, 2, 1)_{265} \\
\ell_9 = b_4 &= \begin{bmatrix} 1 & 0 & \omega^2 & 1 \\ 0 & 1 & 1 & \omega \end{bmatrix}_{156} = \begin{bmatrix} 1 & 0 & 3 & 1 \\ 0 & 1 & 1 & 2 \end{bmatrix}_{156} = \mathbf{Pl}(3, 0, 3, 2, 3, 1)_{335} \\
\ell_{10} = b_5 &= \begin{bmatrix} 1 & \omega^2 & 0 & 1 \\ 0 & 0 & 1 & \omega \end{bmatrix}_{165} = \begin{bmatrix} 1 & 3 & 0 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}_{165} = \mathbf{Pl}(0, 2, 3, 2, 3, 1)_{337} \\
\ell_{11} = b_6 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{16} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{16} = \mathbf{Pl}(0, 0, 1, 0, 0, 0)_2 \\
\ell_{12} = c_{12} &= \begin{bmatrix} 1 & 0 & \omega & 1 \\ 0 & 1 & 0 & \omega^2 \end{bmatrix}_{138} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 0 & 3 \end{bmatrix}_{138} = \mathbf{Pl}(2, 3, 0, 3, 2, 1)_{256} \\
\ell_{13} = c_{13} &= \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{109} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{109} = \mathbf{Pl}(1, 1, 0, 1, 1, 1)_{189} \\
\ell_{14} = c_{14} &= \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{345} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{345} = \mathbf{Pl}(0, 1, 0, 1, 0, 0)_{13} \\
\ell_{15} = c_{15} &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}_{84} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}_{84} = \mathbf{Pl}(1, 0, 0, 1, 0, 0)_{10} \\
\ell_{16} = c_{16} &= \begin{bmatrix} 1 & 0 & \omega^2 & 0 \\ 0 & 1 & 0 & \omega \end{bmatrix}_{71} = \begin{bmatrix} 1 & 0 & 3 & 0 \\ 0 & 1 & 0 & 2 \end{bmatrix}_{71} = \mathbf{Pl}(3, 2, 0, 0, 3, 1)_{299} \\
\ell_{17} = c_{23} &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}_{85} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}_{85} = \mathbf{Pl}(1, 1, 1, 1, 0, 0)_{16} \\
\ell_{18} = c_{24} &= \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{122} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{122} = \mathbf{Pl}(0, 1, 1, 1, 1, 1)_{202} \\
\ell_{19} = c_{25} &= \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{110} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{110} = \mathbf{Pl}(1, 0, 1, 1, 1, 1)_{199} \\
\ell_{20} = c_{26} &= \begin{bmatrix} 1 & 0 & \omega & 0 \\ 0 & 1 & 1 & \omega^2 \end{bmatrix}_{55} = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 1 & 3 \end{bmatrix}_{55} = \mathbf{Pl}(2, 3, 2, 0, 2, 1)_{244}
\end{aligned}$$

$$\begin{aligned}
\ell_{21} = c_{34} &= \begin{bmatrix} 1 & \omega & 0 & 1 \\ 0 & 0 & 1 & \omega^2 \end{bmatrix}_{145} = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}_{145} = \mathbf{Pl}(0, 3, 2, 3, 2, 1)_{271} \\
\ell_{22} = c_{35} &= \begin{bmatrix} 1 & 0 & \omega & 1 \\ 0 & 1 & 1 & \omega^2 \end{bmatrix}_{139} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 3 \end{bmatrix}_{139} = \mathbf{Pl}(2, 0, 2, 3, 2, 1)_{267} \\
\ell_{23} = c_{36} &= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{26} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{26} = \mathbf{Pl}(1, 1, 1, 0, 1, 1)_{180} \\
\ell_{24} = c_{45} &= \begin{bmatrix} 1 & \omega^2 & 0 & 0 \\ 0 & 0 & 1 & \omega \end{bmatrix}_{81} = \begin{bmatrix} 1 & 3 & 0 & 0 \\ 0 & 0 & 1 & 2 \end{bmatrix}_{81} = \mathbf{Pl}(0, 0, 3, 2, 3, 1)_{332} \\
\ell_{25} = c_{46} &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{100} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{100} = \mathbf{Pl}(0, 1, 1, 0, 0, 0)_6 \\
\ell_{26} = c_{56} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}_1 = \mathbf{Pl}(1, 0, 1, 0, 0, 0)_3
\end{aligned}$$

Rank of lines: ( 72, 54, 25, 356, 0, 155, 340, 38, 61, 156, 165, 16, 138, 109, 345, 84, 71, 85, 122, 110, 55, 145, 139, 26, 81, 100, 1 )

Rank of points on Klein quadric: ( 308, 238, 177, 1, 0, 314, 9, 198, 265, 335, 337, 2, 256, 189, 13, 10, 299, 16, 202, 199, 244, 271, 267, 180, 332, 6, 3 )

### All Points on surface

The surface has 45 points

### Eckardt Points

The surface has 45 Eckardt points:

- 0 :  $E_{56} = a_5 \cap b_6 \cap c_{56} = P_0 = P_0 = \mathbf{P}(1, 0, 0, 0) = \mathbf{P}(1, 0, 0, 0), T = 0$
- 1 :  $E_{51} = a_5 \cap b_1 \cap c_{15} = P_1 = P_1 = \mathbf{P}(0, 1, 0, 0) = \mathbf{P}(0, 1, 0, 0), T = 4$
- 2 :  $E_{46} = a_4 \cap b_6 \cap c_{46} = P_2 = P_2 = \mathbf{P}(0, 0, 1, 0) = \mathbf{P}(0, 0, 1, 0), T = 20$
- 3 :  $E_{41} = a_4 \cap b_1 \cap c_{14} = P_3 = P_3 = \mathbf{P}(0, 0, 0, 1) = \mathbf{P}(0, 0, 0, 1), T = 84$
- 4 :  $E_{32} = a_3 \cap b_2 \cap c_{23} = P_4 = P_4 = \mathbf{P}(1, 1, 1, 1) = \mathbf{P}(1, 1, 1, 1), T = 27$
- 5 :  $E_{52} = a_5 \cap b_2 \cap c_{25} = P_5 = P_5 = \mathbf{P}(1, 1, 0, 0) = \mathbf{P}(1, 1, 0, 0), T = 1$
- 6 :  $E_{54} = a_5 \cap b_4 \cap c_{45} = P_6 = P_6 = \mathbf{P}(\omega, 1, 0, 0) = \mathbf{P}(2, 1, 0, 0), T = 2$
- 7 :  $E_{53} = a_5 \cap b_3 \cap c_{35} = P_7 = P_7 = \mathbf{P}(\omega^2, 1, 0, 0) = \mathbf{P}(3, 1, 0, 0), T = 3$
- 8 :  $E_{36} = a_3 \cap b_6 \cap c_{36} = P_8 = P_8 = \mathbf{P}(1, 0, 1, 0) = \mathbf{P}(1, 0, 1, 0), T = 5$
- 9 :  $E_{16} = a_1 \cap b_6 \cap c_{16} = P_9 = P_9 = \mathbf{P}(\omega, 0, 1, 0) = \mathbf{P}(2, 0, 1, 0), T = 10$
- 10 :  $E_{26} = a_2 \cap b_6 \cap c_{26} = P_{10} = P_{10} = \mathbf{P}(\omega^2, 0, 1, 0) = \mathbf{P}(3, 0, 1, 0), T = 15$
- 11 :  $E_{14,23,56} = c_{14} \cap c_{23} \cap c_{56} = P_{11} = P_{11} = \mathbf{P}(0, 1, 1, 0) = \mathbf{P}(0, 1, 1, 0), T = 9$
- 12 :  $E_{13,24,56} = c_{13} \cap c_{24} \cap c_{56} = P_{12} = P_{12} = \mathbf{P}(1, 1, 1, 0) = \mathbf{P}(1, 1, 1, 0), T = 6$
- 13 :  $E_{65} = a_6 \cap b_5 \cap c_{56} = P_{13} = P_{13} = \mathbf{P}(\omega, 1, 1, 0) = \mathbf{P}(2, 1, 1, 0), T = 12$
- 14 :  $E_{12,34,56} = c_{12} \cap c_{34} \cap c_{56} = P_{14} = P_{14} = \mathbf{P}(\omega^2, 1, 1, 0) = \mathbf{P}(3, 1, 1, 0), T = 18$
- 15 :  $E_{15,23,46} = c_{15} \cap c_{23} \cap c_{46} = P_{15} = P_{23} = \mathbf{P}(1, 0, 0, 1) = \mathbf{P}(1, 0, 0, 1), T = 21$
- 16 :  $E_{31} = a_3 \cap b_1 \cap c_{13} = P_{16} = P_{26} = \mathbf{P}(0, 1, 0, 1) = \mathbf{P}(0, 1, 0, 1), T = 25$
- 17 :  $E_{15,24,36} = c_{15} \cap c_{24} \cap c_{36} = P_{17} = P_{27} = \mathbf{P}(1, 1, 0, 1) = \mathbf{P}(1, 1, 0, 1), T = 22$
- 18 :  $E_{21} = a_2 \cap b_1 \cap c_{12} = P_{18} = P_{30} = \mathbf{P}(0, \omega, 0, 1) = \mathbf{P}(0, 2, 0, 1), T = 46$
- 19 :  $E_{15,26,34} = c_{15} \cap c_{26} \cap c_{34} = P_{19} = P_{31} = \mathbf{P}(1, \omega, 0, 1) = \mathbf{P}(1, 2, 0, 1), T = 24$
- 20 :  $E_{61} = a_6 \cap b_1 \cap c_{16} = P_{20} = P_{34} = \mathbf{P}(0, \omega^2, 0, 1) = \mathbf{P}(0, 3, 0, 1), T = 67$

21 :  $E_{15} = a_1 \cap b_5 \cap c_{15} = P_{21} = P_{35} = \mathbf{P}(1, \omega^2, 0, 1) = \mathbf{P}(1, 3, 0, 1)$ ,  $T = 23$   
 22 :  $E_{42} = a_4 \cap b_2 \cap c_{24} = P_{22} = P_{38} = \mathbf{P}(0, 0, 1, 1) = \mathbf{P}(0, 0, 1, 1)$ ,  $T = 41$   
 23 :  $E_{13,25,46} = c_{13} \cap c_{25} \cap c_{46} = P_{23} = P_{39} = \mathbf{P}(1, 0, 1, 1) = \mathbf{P}(1, 0, 1, 1)$ ,  $T = 26$   
 24 :  $E_{14,25,36} = c_{14} \cap c_{25} \cap c_{36} = P_{24} = P_{42} = \mathbf{P}(0, 1, 1, 1) = \mathbf{P}(0, 1, 1, 1)$ ,  $T = 30$   
 25 :  $E_{62} = a_6 \cap b_2 \cap c_{26} = P_{25} = P_{47} = \mathbf{P}(\omega, \omega, 1, 1) = \mathbf{P}(2, 2, 1, 1)$ ,  $T = 53$   
 26 :  $E_{25} = a_2 \cap b_5 \cap c_{25} = P_{26} = P_{48} = \mathbf{P}(\omega^2, \omega, 1, 1) = \mathbf{P}(3, 2, 1, 1)$ ,  $T = 80$   
 27 :  $E_{16,25,34} = c_{16} \cap c_{25} \cap c_{34} = P_{27} = P_{51} = \mathbf{P}(\omega, \omega^2, 1, 1) = \mathbf{P}(2, 3, 1, 1)$ ,  $T = 55$   
 28 :  $E_{12} = a_1 \cap b_2 \cap c_{12} = P_{28} = P_{52} = \mathbf{P}(\omega^2, \omega^2, 1, 1) = \mathbf{P}(3, 3, 1, 1)$ ,  $T = 79$   
 29 :  $E_{43} = a_4 \cap b_3 \cap c_{34} = P_{29} = P_{53} = \mathbf{P}(0, 0, \omega, 1) = \mathbf{P}(0, 0, 2, 1)$ ,  $T = 62$   
 30 :  $E_{12,35,46} = c_{12} \cap c_{35} \cap c_{46} = P_{30} = P_{54} = \mathbf{P}(1, 0, \omega, 1) = \mathbf{P}(1, 0, 2, 1)$ ,  $T = 36$   
 31 :  $E_{35} = a_3 \cap b_5 \cap c_{35} = P_{31} = P_{59} = \mathbf{P}(\omega, 1, \omega, 1) = \mathbf{P}(2, 1, 2, 1)$ ,  $T = 49$   
 32 :  $E_{63} = a_6 \cap b_3 \cap c_{36} = P_{32} = P_{60} = \mathbf{P}(\omega^2, 1, \omega, 1) = \mathbf{P}(3, 1, 2, 1)$ ,  $T = 76$   
 33 :  $E_{14,26,35} = c_{14} \cap c_{26} \cap c_{35} = P_{33} = P_{61} = \mathbf{P}(0, \omega, \omega, 1) = \mathbf{P}(0, 2, 2, 1)$ ,  $T = 51$   
 34 :  $E_{23} = a_2 \cap b_3 \cap c_{23} = P_{34} = P_{62} = \mathbf{P}(1, \omega, \omega, 1) = \mathbf{P}(1, 2, 2, 1)$ ,  $T = 39$   
 35 :  $E_{13} = a_1 \cap b_3 \cap c_{13} = P_{35} = P_{67} = \mathbf{P}(\omega, \omega^2, \omega, 1) = \mathbf{P}(2, 3, 2, 1)$ ,  $T = 50$   
 36 :  $E_{16,24,35} = c_{16} \cap c_{24} \cap c_{35} = P_{36} = P_{68} = \mathbf{P}(\omega^2, \omega^2, \omega, 1) = \mathbf{P}(3, 3, 2, 1)$ ,  $T = 74$   
 37 :  $E_{45} = a_4 \cap b_5 \cap c_{45} = P_{37} = P_{69} = \mathbf{P}(0, 0, \omega^2, 1) = \mathbf{P}(0, 0, 3, 1)$ ,  $T = 83$   
 38 :  $E_{64} = a_6 \cap b_4 \cap c_{46} = P_{38} = P_{70} = \mathbf{P}(1, 0, \omega^2, 1) = \mathbf{P}(1, 0, 3, 1)$ ,  $T = 31$   
 39 :  $E_{12,36,45} = c_{12} \cap c_{36} \cap c_{45} = P_{39} = P_{75} = \mathbf{P}(\omega, 1, \omega^2, 1) = \mathbf{P}(2, 1, 3, 1)$ ,  $T = 59$   
 40 :  $E_{34} = a_3 \cap b_4 \cap c_{34} = P_{40} = P_{76} = \mathbf{P}(\omega^2, 1, \omega^2, 1) = \mathbf{P}(3, 1, 3, 1)$ ,  $T = 71$   
 41 :  $E_{24} = a_2 \cap b_4 \cap c_{24} = P_{41} = P_{79} = \mathbf{P}(\omega, \omega, \omega^2, 1) = \mathbf{P}(2, 2, 3, 1)$ ,  $T = 58$   
 42 :  $E_{13,26,45} = c_{13} \cap c_{26} \cap c_{45} = P_{42} = P_{80} = \mathbf{P}(\omega^2, \omega, \omega^2, 1) = \mathbf{P}(3, 2, 3, 1)$ ,  $T = 70$   
 43 :  $E_{14} = a_1 \cap b_4 \cap c_{14} = P_{43} = P_{81} = \mathbf{P}(0, \omega^2, \omega^2, 1) = \mathbf{P}(0, 3, 3, 1)$ ,  $T = 72$   
 44 :  $E_{16,23,45} = c_{16} \cap c_{23} \cap c_{45} = P_{44} = P_{82} = \mathbf{P}(1, \omega^2, \omega^2, 1) = \mathbf{P}(1, 3, 3, 1)$ ,  $T = 33$   
 Set of tangent planes: ( 0, 4, 20, 84, 27, 1, 2, 3, 5, 10, 15, 9, 6, 12, 18, 21, 25, 22, 46, 24, 67, 23, 41, 26, 30, 53, 80, 55, 79, 62, 36, 49, 76, 51, 39, 50, 74, 83, 31, 59, 71, 58, 70, 72, 33 )  
 Line type of Eckardt points:  $5^{27}, 3^{240}, 1^{90}$   
 Plane type of Eckardt points:  $13^{45}, 9^{40}$

## Hesse planes

Number of Hesse planes: 40

Set of Hesse planes: ( 7, 8, 11, 13, 14, 16, 17, 19, 28, 29, 32, 34, 35, 37, 38, 40, 42, 43, 44, 45, 47, 48, 52, 54, 56, 57, 60, 61, 63, 64, 65, 66, 68, 69, 73, 75, 77, 78, 81, 82 )  
 subspace 0 / 40 is 7:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & \omega \end{bmatrix}$$

⋮

subspace 39 / 40 is 82:

$$\begin{bmatrix} 1 & 0 & \omega^2 & 0 \\ 0 & 1 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

0 : 7 :  $E_{56}, E_{31}, E_{15,24,36}, E_{16,25,34}, E_{12}, E_{14,26,35}, E_{23}, E_{45}, E_{64}$

⋮

39 : 82 :  $E_{41}, E_{52}, E_{16}, E_{12,34,56}, E_{15,24,36}, E_{35}, E_{23}, E_{64}, E_{13,26,45}$



**Axes**

Number of axes: 240

Axes:

$$0 : 0 = 0, 0 = E_{23}, E_{31}, E_{12}$$

$\vdots$

$$239 : 239 = 119, 1 = E_{12,36,45}, E_{14,26,35}, E_{13,25,46}$$

$\vdots$

**Tritangent planes**

The 45 tritangent planes are:

$$\begin{aligned} \pi_{12} = \pi_0 = 79 &= \begin{bmatrix} 1 & 0 & 0 & \omega^2 \\ 0 & 1 & 0 & \omega^2 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\ &= V(\omega^2 X_0 + \omega^2 X_1 + X_2 + X_3) = V(3X_0 + 3X_1 + X_2 + X_3) \end{aligned}$$

dual pt rank = 52 = (3, 3, 1, 1).

$\vdots$

$$\begin{aligned} \pi_{16,25,34} = \pi_{44} = 55 &= \begin{bmatrix} 1 & 0 & 0 & \omega \\ 0 & 1 & 0 & \omega \\ 0 & 0 & 1 & \omega^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix} \\ &= V(\omega X_0 + \omega X_1 + \omega^2 X_2 + X_3) = V(2X_0 + 2X_1 + 3X_2 + X_3) \end{aligned}$$

dual pt rank = 79 = (2, 2, 3, 1).

Karaoglu [38] describes a different algorithm, based on non-conical six-arcs and Steiner trihedral pairs. The command

**Example 316**

```
surface_classify_q4_arc_lifting_two_lines:
▷ $(ORBITER) -v 10 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label sixarcs_q4 -W \
▷ ▷ -end \
▷ ▷ -with P -do \
▷ ▷ -projective_space_activity \
▷ ▷ ▷ -control_six_arcs Control \
▷ ▷ ▷ -classify_surfaces_through_arcs_and_two_lines \
▷ ▷ -end
▷ pdflatex surfaces_arc_lifting_4.tex
▷ $(OPEN) surfaces_arc_lifting_4.pdf
```

classifies all cubic surfaces with 27 lines over the field  $\mathbb{F}_4$  using the algorithm of Karaoglu. The result agrees with the previous algorithm. In  $\text{PG}(3, 4)$ , the only surface with 27 lines is the Hirschfeld surface.

The classification of cubic surfaces with double sixes can be used to perform isomorphism testing and recognition of surfaces. Suppose we want to create an isomorphism between the surface  $F_{a,b,c,d}$  with  $(a, b, c, d) = (25, 5, 5, 25)$  over  $\mathbb{F}_{31}$  and the appropriate surface chosen to be part of the transversal of orbit representatives by the classification algorithm. We can use the following command to create the surface, to classify all surfaces with 27 lines over  $\mathbb{F}_{31}$ , and to establish an isomorphism between the given surface and the standard model in the classified list:

#### Example 317

```
Family_general.F31_25_5_5_25_recognize:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 31 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define Surf -cubic_surface \
> > > -space P -family_general.abcd 25 5 5 25 \
> > -end \
> > -with Surf -do \
> > -cubic_surface_activity \
> > > -report \
> > -end \
> > -define Control -poset_classification_control -W \
> > > -problem_label "surf_q31" \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -classify_surfaces_with_double_sixes AllSurf Control \
> > -end \
> > -with AllSurf -do \
> > -classification_of_cubic_surfaces_with_double_sixes_activity \
> > > -recognize Surf \
> > -end
```

The following command creates all surfaces in the form  $F_{abcd}$  over  $\mathbb{F}_{11}$  and identifies the isomorphism type of each surface. The result is written to a csv file.

#### Example 318

```
surface_F_abcd_identify_q11:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 11 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define Control -poset_classification_control -W \
> > > -problem_label "surf_q11" \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -classify_surfaces_with_double_sixes AllSurf Control \
> > -end \
> > -with AllSurf -do \
```

```

> > -classification_of_cubic_surfaces_with_double_sixes_activity \
> > > -identify_general_abcd \
> > -end

```

The next example illustrates isomorphism testing for smooth cubic surfaces with 27 lines. In order to perform isomorphism testing, the classification of cubic surfaces of that order is required. Let us consider an example over the field  $\mathbb{F}_{11}$ . From the previous command, we know that the two surfaces of type  $F_{abcd}$  with  $(a, b, c, d) = (2, 4, 4, 2)$  and  $(a, b, c, d) = (5, 10, 7, 5)$ , over  $\mathbb{F}_{11}$  are isomorphic. Both surfaces have 10 Eckardt points. The following command sequence creates both surfaces, classifies all surfaces over  $\mathbb{F}_{11}$ , and then performs an isomorphism test for the two given surfaces. The surfaces are found to be isomorphic. A transformation mapping one surface to the other is computed.

#### Example 319

```

Family_general.F11.iso_test:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 11 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define Surf1 -cubic_surface \
> > > -space P -family_general_abcd 2 4 4 2 \
> > -end \
> > -define Surf2 -cubic_surface \
> > > -space P -family_general_abcd 5 10 7 5 \
> > -end \
> > -define Control -poset_classification_control -W \
> > > -problem_label "surf_q11" \
> > -end \
> > -with P -do \
> > -projective_space_activity \
> > > -classify_surfaces_with_double_sixes AllSurf Control \
> > -end \
> > -with AllSurf -do \
> > -classification_of_cubic_surfaces_with_double_sixes_activity \
> > > -isomorphism_testing Surf1 Surf2 \
> > -end

```

| Creating Quartic Curves |            |                                                                                                                                      |
|-------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Command                 | Arguments  | Purpose                                                                                                                              |
| -space                  | P          | Specify the 2-dimensional projective space $P = PG(2, q)$ .                                                                          |
| -label_txt              | label      | Override the ascii label of the curve.                                                                                               |
| -label_tex              | label      | Override the latex label of the curve.                                                                                               |
| -label_for_summary      | label      | Override the ascii label of the curve, to be used in summary commands.                                                               |
| -catalogue              | OCN        | Create the quartic curve from the Orbiter catalogue with the given Orbiter catalogue number.                                         |
| -by_coefficients        | coeffs     | Create a quartic curve given the coefficients of the equation.                                                                       |
| -by_equation            | eqn        | Create a quartic curve from an equation.                                                                                             |
| -by_symbolic_object     | R equation | Create a quartic curve from an equation using the polynomial ring $R$ . The equation is given as a symbolic object.                  |
| -from_cubic_surface     | $S$ $i$    | Create the quartic curve from $i$ th orbit of the automorphism group of the surface $S$ on points not on lines ( $i$ is zero based). |
| -override_group         | descr      | Override the automorphism group of the curve by the given group.                                                                     |
| -transform              | elt        | Apply the transformation given by the group element.                                                                                 |
| -transform_inverse      | elt        | Apply the inverse transformation given by the group element.                                                                         |

Table 8.8: Creating Quartic Curves

## 8.5 Quartic Curves

Cubic surfaces with 27 lines are associated with quartic curves with 28 bitangents (see [32]), which in turn are associated with del Pezzo surfaces. Orbiter can classify quartic curves based on a known classification of cubic surfaces. Orbiter also has a catalogue of quartic curves for small field sizes.

Table 8.8 lists options to create a quartic curve object.

Table 8.9 lists activities for a quartic curve object.

Table 8.10 lists options for exporting data.

Suppose we want to create a quartic curve. We can utilize the Orbiter catalogue of isomorphism types of quartic curves. The following command creates the unique curve over the field  $\mathbb{F}_9$ .

### Example 320

```
quartic_curve_9.0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 9 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -define C -quartic_curve -space P -catalogue 0 -end
```

| Quartic Curve Activities                               |           |                                                                                                                                                                     |
|--------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                                | Arguments | Purpose                                                                                                                                                             |
| -report                                                |           | Produce a latex report about the curve.                                                                                                                             |
| -export_something                                      | type      | Exports data of the quartic curve to a file. The type of data to be exported must be specified according to Table 8.10.                                             |
| -create_surface                                        |           | Create a cubic surface from the curve.                                                                                                                              |
| -extract_orbit_on_bitangents_by_length                 | $l$       | Extract the bitangents in the unique orbit of length $l$ . If there is no orbit of length $l$ , or if there are multiple orbits of length $l$ , an error is raised. |
| -extract_specific_orbit_on_bitangents_by_length        | $l\ i$    | Extract the $i$ th orbit on bitangents of length $l$ .                                                                                                              |
| -extract_specific_orbit_on_kovalevski_points_by_length | $l\ i$    | Extract the $i$ th orbit on Kovalevski points of length $l$ .                                                                                                       |

Table 8.9: Quartic Curve Activities

| Options for Exporting Data: Cubic Surfaces |                                                                          |
|--------------------------------------------|--------------------------------------------------------------------------|
| Command                                    | Purpose                                                                  |
| points                                     | The points on the quartic curve.                                         |
| equation                                   | The equation of the quartic curves as a coefficient vector of length 15. |
| bitangents                                 | The 28 bitangents.                                                       |
| Kovalevski_points                          | The Kovalevski points (points off the curve where 4 bitangents meet).    |
| singular_points                            | The singular points of the curve.                                        |

Table 8.10: Options for Exporting Data: Quartic Curves

The Edge curve over  $\mathbb{F}_q$  has an equation of the form

$$X^4 - Y^4 - Z^4 + 2f^2Y^2Z^2 + 4fX^2YZ = 0,$$

with  $f$  a primitive element of  $\mathbb{F}_q$ . Let us create the Edge curve over  $\mathbb{F}_9$ . Since we are in characteristic 3, we write 4 as 1. The default field polynomial chosen by Orbiter is always primitive, so we can pick  $\alpha = f$ , which encodes to 3. The following command creates the Edge curve:

#### Example 321

```
quartic_curve_edge_q9:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 9 -end \
▷ ▷ -define P2 -projective_space -n 2 -field F -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 4 \
▷ ▷ ▷ -monomial_ordering_partition \
▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
▷ ▷ -end \
▷ ▷ -define Edge -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X,Y,Z" \
▷ ▷ ▷ -text "X^4-Y^4-Z^4+2*f^2*Y^2*Z^2+1*f*X^2*Y*Z" \
▷ ▷ -end \
▷ ▷ -define f -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -text "3" \
▷ ▷ -end \
▷ ▷ -define Edge_f3 -symbolic_object \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -managed_variables "X,Y,Z" \
▷ ▷ ▷ -substitute "f" Edge f \
▷ ▷ -end \
▷ ▷ -define C -quartic_curve \
▷ ▷ ▷ -space P2 \
▷ ▷ ▷ -by_symbolic_object R Edge_f3 -end \
▷ ▷ -with C -do \
▷ ▷ ▷ -quartic_curve_activity \
▷ ▷ ▷ ▷ -report \
▷ ▷ ▷ -end
▷ pdflatex quartic_curve_equation_Edge_f3_q9_report.tex
▷ $(OPEN) quartic_curve_equation_Edge_f3_q9_report.pdf
```

The report shows that the curve is non-singular and has 8 rational points.

We can create quartic curves from cubic surfaces. The following command creates all quartic curves arising from points on no line of the cubic surface over  $\mathbb{F}_7$ :

**Example 322**

```

surface_7_0_make_quartic_curves:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -all_quartic_curves \
▷ ▷ -end \
▷ #pdflatex surface_catalogue_q7_iso0_report.tex
▷ #$(OPEN) surface_catalogue_q7_iso0_report.pdf

```

The next command creates all cubic surfaces arising from the second isomorphism class of cubic surfaces over the field  $\mathbb{F}_9$ :

**Example 323**

```

surface_9_1_make_quartic_curves:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 9 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -all_quartic_curves \
▷ ▷ -end \
▷ #pdflatex surface_catalogue_q9_iso1_report.tex
▷ #$(OPEN) surface_catalogue_q9_iso1_report.pdf

```

Suppose we want to study the (unique) quartic curve for  $q = 9$ . The following command creates the curve whose catalogue number is 0, and produces a report:

**Example 324**

```

quartic_curve_9_0_report:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 9 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end \
▷ ▷ -with C -do \
▷ ▷ ▷ -quartic_curve_activity \
▷ ▷ ▷ ▷ -report \
▷ ▷ ▷ -end \
▷ pdflatex quartic_curve_catalogue_q9_iso0_report.tex
▷ $(OPEN) quartic_curve_catalogue_q9_iso0_report.pdf

```

The report contains the following information:

### The equation

The equation of the quartic curve is :

$$\alpha^3 X_0^3 X_1 + \alpha^4 X_0^3 X_2 + \alpha^7 X_0 X_1^3 + \alpha^6 X_1^3 X_2 + \alpha^2 X_0 X_2^3 + X_1 X_2^3$$

$$(0, 0, 0, 8, 2, 4, 5, 7, 1, 0, 0, 0, 0, 0, 0)$$

### The gradient

The gradient of the quartic curve is :

$$\alpha^7 X_1^3 + \alpha^2 X_2^3$$

$$(0, 4, 7, 0, 0, 0, 0, 0, 0, 0)$$

$$\alpha^3 X_0^3 + X_2^3$$

$$(8, 0, 1, 0, 0, 0, 0, 0, 0, 0)$$

$$\alpha^4 X_0^3 + \alpha^6 X_1^3$$

$$(2, 5, 0, 0, 0, 0, 0, 0, 0, 0)$$

### General information

|                                       |              |
|---------------------------------------|--------------|
| Number of bitangents                  | 28           |
| Number of points                      | 28           |
| Fullness                              | is full      |
| Number of Kovalevski points           | 63           |
| Bitangent line type $(a_0, a_1, a_2)$ | $(0, 28, 0)$ |
| Number of singular points             | 0            |

### All points on the curve

The surface has 28 points:

The points on the quartic curve are:

|                          |                           |                           |
|--------------------------|---------------------------|---------------------------|
| 0 : $P_0 = (1, 0, 0)$    | 10 : $P_{30} = (2, 2, 1)$ | 20 : $P_{58} = (3, 5, 1)$ |
| 1 : $P_1 = (0, 1, 0)$    | 11 : $P_{32} = (4, 2, 1)$ | 21 : $P_{62} = (7, 5, 1)$ |
| 2 : $P_2 = (0, 0, 1)$    | 12 : $P_{34} = (6, 2, 1)$ | 22 : $P_{76} = (3, 7, 1)$ |
| 3 : $P_3 = (1, 1, 1)$    | 13 : $P_{38} = (1, 3, 1)$ | 23 : $P_{77} = (4, 7, 1)$ |
| 4 : $P_4 = (1, 1, 0)$    | 14 : $P_{41} = (4, 3, 1)$ | 24 : $P_{78} = (5, 7, 1)$ |
| 5 : $P_5 = (2, 1, 0)$    | 15 : $P_{44} = (7, 3, 1)$ | 25 : $P_{82} = (0, 8, 1)$ |
| 6 : $P_{14} = (3, 0, 1)$ | 16 : $P_{46} = (0, 4, 1)$ | 26 : $P_{83} = (1, 8, 1)$ |
| 7 : $P_{17} = (6, 0, 1)$ | 17 : $P_{51} = (5, 4, 1)$ | 27 : $P_{84} = (2, 8, 1)$ |
| 8 : $P_{24} = (5, 1, 1)$ | 18 : $P_{53} = (7, 4, 1)$ |                           |
| 9 : $P_{25} = (6, 1, 1)$ | 19 : $P_{57} = (2, 5, 1)$ |                           |



The points by rank are: ( 0, 1, 2, 3, 4, 5, 14, 17, 24, 25, 30, 32, 34, 38, 41, 44, 46, 51, 53, 57, 58, 62, 76, 77, 78, 82, 83, 84 )

The Kovalevski points are:

- 0 :  $P_7 = (4, 1, 0) = c_{13} \cap c_{14} \cap c_{36} \cap c_{46}$
- 1 :  $P_8 = (5, 1, 0) = a_2 \cap a_4 \cap c_{25} \cap c_{45}$
- 2 :  $P_9 = (6, 1, 0) = b_1 \cap b_6 \cap c_{12} \cap c_{26}$
- 3 :  $P_{10} = (7, 1, 0) = a_3 \cap b_5 \cap c_{35} \cap d$
- 4 :  $P_{11} = (8, 1, 0) = b_2 \cap b_3 \cap c_{24} \cap c_{34}$
- 5 :  $P_{12} = (1, 0, 1) = a_3 \cap a_4 \cap c_{23} \cap c_{24}$
- 6 :  $P_{13} = (2, 0, 1) = c_{34} \cap c_{36} \cap c_{45} \cap c_{56}$
- 7 :  $P_{15} = (4, 0, 1) = b_3 \cap b_6 \cap c_{13} \cap c_{16}$
- 8 :  $P_{16} = (5, 0, 1) = a_5 \cap a_6 \cap c_{25} \cap c_{26}$
- 9 :  $P_{18} = (7, 0, 1) = a_2 \cap b_1 \cap c_{35} \cap c_{46}$
- 10 :  $P_{19} = (8, 0, 1) = b_4 \cap b_5 \cap c_{14} \cap c_{15}$
- 11 :  $P_{20} = (0, 1, 1) = a_2 \cap b_3 \cap c_{14} \cap c_{56}$
- 12 :  $P_{21} = (2, 1, 1) = b_2 \cap b_4 \cap c_{26} \cap c_{46}$
- 13 :  $P_{22} = (3, 1, 1) = a_4 \cap b_5 \cap c_{12} \cap c_{36}$
- 14 :  $P_{23} = (4, 1, 1) = a_6 \cap b_1 \cap c_{23} \cap c_{45}$
- 15 :  $P_{26} = (7, 1, 1) = c_{16} \cap c_{25} \cap c_{34} \cap d$
- 16 :  $P_{27} = (8, 1, 1) = a_3 \cap a_5 \cap c_{13} \cap c_{15}$
- 17 :  $P_{28} = (0, 2, 1) = c_{12} \cap c_{13} \cap c_{25} \cap c_{35}$
- 18 :  $P_{29} = (1, 2, 1) = b_1 \cap b_5 \cap c_{16} \cap c_{56}$
- 19 :  $P_{31} = (3, 2, 1) = a_3 \cap a_6 \cap c_{34} \cap c_{46}$
- 20 :  $P_{33} = (5, 2, 1) = a_2 \cap b_4 \cap c_{24} \cap d$
- 21 :  $P_{35} = (7, 2, 1) = b_2 \cap b_6 \cap c_{23} \cap c_{36}$
- 22 :  $P_{36} = (8, 2, 1) = a_4 \cap b_3 \cap c_{15} \cap c_{26}$
- 23 :  $P_{37} = (0, 3, 1) = a_5 \cap b_1 \cap c_{24} \cap c_{36}$
- 24 :  $P_{39} = (2, 3, 1) = a_2 \cap a_6 \cap c_{12} \cap c_{16}$
- 25 :  $P_{40} = (3, 3, 1) = b_3 \cap b_4 \cap c_{35} \cap c_{45}$
- 26 :  $P_{42} = (5, 3, 1) = a_4 \cap b_2 \cap c_{13} \cap c_{56}$
- 27 :  $P_{43} = (6, 3, 1) = a_3 \cap b_6 \cap c_{14} \cap c_{25}$
- 28 :  $P_{45} = (8, 3, 1) = c_{15} \cap c_{23} \cap c_{46} \cap d$
- 29 :  $P_{47} = (1, 4, 1) = a_6 \cap b_2 \cap c_{14} \cap c_{35}$
- 30 :  $P_{48} = (2, 4, 1) = b_3 \cap b_5 \cap c_{23} \cap c_{25}$
- 31 :  $P_{49} = (3, 4, 1) = a_5 \cap b_6 \cap c_{56} \cap d$
- 32 :  $P_{50} = (4, 4, 1) = a_2 \cap a_3 \cap c_{26} \cap c_{36}$
- 33 :  $P_{52} = (6, 4, 1) = b_1 \cap b_4 \cap c_{13} \cap c_{34}$
- 34 :  $P_{54} = (8, 4, 1) = c_{12} \cap c_{15} \cap c_{24} \cap c_{45}$
- 35 :  $P_{55} = (0, 5, 1) = a_4 \cap a_6 \cap b_4 \cap b_6$
- 36 :  $P_{56} = (1, 5, 1) = c_{13} \cap c_{26} \cap c_{45} \cap d$
- 37 :  $P_{59} = (4, 5, 1) = c_{24} \cap c_{25} \cap c_{46} \cap c_{56}$
- 38 :  $P_{60} = (5, 5, 1) = c_{12} \cap c_{14} \cap c_{23} \cap c_{34}$
- 39 :  $P_{61} = (6, 5, 1) = a_2 \cap a_5 \cap b_2 \cap b_5$
- 40 :  $P_{63} = (8, 5, 1) = c_{15} \cap c_{16} \cap c_{35} \cap c_{36}$
- 41 :  $P_{64} = (0, 6, 1) = a_1 \cap b_5 \cap c_{26} \cap c_{34}$
- 42 :  $P_{65} = (1, 6, 1) = a_1 \cap b_4 \cap c_{25} \cap c_{36}$
- 43 :  $P_{66} = (2, 6, 1) = a_1 \cap b_6 \cap c_{24} \cap c_{35}$
- 44 :  $P_{67} = (3, 6, 1) = a_1 \cap a_2 \cap c_{13} \cap c_{23}$
- 45 :  $P_{68} = (4, 6, 1) = a_1 \cap b_2 \cap c_{12} \cap d$
- 46 :  $P_{69} = (5, 6, 1) = a_1 \cap a_3 \cap b_1 \cap b_3$
- 47 :  $P_{70} = (6, 6, 1) = a_1 \cap a_4 \cap c_{16} \cap c_{46}$
- 48 :  $P_{71} = (7, 6, 1) = a_1 \cap a_5 \cap c_{14} \cap c_{45}$
- 49 :  $P_{72} = (8, 6, 1) = a_1 \cap a_6 \cap c_{15} \cap c_{56}$

$$\begin{aligned}
50 : P_{73} &= (0, 7, 1) = a_3 \cap b_2 \cap c_{16} \cap c_{45} \\
51 : P_{74} &= (1, 7, 1) = a_5 \cap b_3 \cap c_{12} \cap c_{46} \\
52 : P_{75} &= (2, 7, 1) = a_4 \cap b_1 \cap c_{14} \cap d \\
53 : P_{79} &= (6, 7, 1) = c_{23} \cap c_{26} \cap c_{35} \cap c_{56} \\
54 : P_{80} &= (7, 7, 1) = a_6 \cap b_5 \cap c_{13} \cap c_{24} \\
55 : P_{81} &= (8, 7, 1) = a_2 \cap b_6 \cap c_{15} \cap c_{34} \\
56 : P_{85} &= (3, 8, 1) = c_{14} \cap c_{16} \cap c_{24} \cap c_{26} \\
57 : P_{86} &= (4, 8, 1) = a_4 \cap a_5 \cap c_{34} \cap c_{35} \\
58 : P_{87} &= (5, 8, 1) = b_5 \cap b_6 \cap c_{45} \cap c_{46} \\
59 : P_{88} &= (6, 8, 1) = a_6 \cap b_3 \cap c_{36} \cap d \\
60 : P_{89} &= (7, 8, 1) = a_3 \cap b_4 \cap c_{12} \cap c_{56} \\
61 : P_{90} &= (8, 8, 1) = b_1 \cap b_2 \cap c_{15} \cap c_{25} \\
62 : P_6 &= (3, 1, 0) = a_5 \cap b_4 \cap c_{16} \cap c_{23}
\end{aligned}$$

The Kovalevski points by rank are: ( 7, 8, 9, 10, 11, 12, 13, 15, 16, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 31, 33, 35, 36, 37, 39, 40, 42, 43, 45, 47, 48, 49, 50, 52, 54, 55, 56, 59, 60, 61, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 79, 80, 81, 85, 86, 87, 88, 89, 90, 6 )

The points off the curve are:

|                           |                           |                           |
|---------------------------|---------------------------|---------------------------|
| 0 : $P_6 = (3, 1, 0)$     | 22 : $P_{35} = (7, 2, 1)$ | 44 : $P_{66} = (2, 6, 1)$ |
| 1 : $P_7 = (4, 1, 0)$     | 23 : $P_{36} = (8, 2, 1)$ | 45 : $P_{67} = (3, 6, 1)$ |
| 2 : $P_8 = (5, 1, 0)$     | 24 : $P_{37} = (0, 3, 1)$ | 46 : $P_{68} = (4, 6, 1)$ |
| 3 : $P_9 = (6, 1, 0)$     | 25 : $P_{39} = (2, 3, 1)$ | 47 : $P_{69} = (5, 6, 1)$ |
| 4 : $P_{10} = (7, 1, 0)$  | 26 : $P_{40} = (3, 3, 1)$ | 48 : $P_{70} = (6, 6, 1)$ |
| 5 : $P_{11} = (8, 1, 0)$  | 27 : $P_{42} = (5, 3, 1)$ | 49 : $P_{71} = (7, 6, 1)$ |
| 6 : $P_{12} = (1, 0, 1)$  | 28 : $P_{43} = (6, 3, 1)$ | 50 : $P_{72} = (8, 6, 1)$ |
| 7 : $P_{13} = (2, 0, 1)$  | 29 : $P_{45} = (8, 3, 1)$ | 51 : $P_{73} = (0, 7, 1)$ |
| 8 : $P_{15} = (4, 0, 1)$  | 30 : $P_{47} = (1, 4, 1)$ | 52 : $P_{74} = (1, 7, 1)$ |
| 9 : $P_{16} = (5, 0, 1)$  | 31 : $P_{48} = (2, 4, 1)$ | 53 : $P_{75} = (2, 7, 1)$ |
| 10 : $P_{18} = (7, 0, 1)$ | 32 : $P_{49} = (3, 4, 1)$ | 54 : $P_{79} = (6, 7, 1)$ |
| 11 : $P_{19} = (8, 0, 1)$ | 33 : $P_{50} = (4, 4, 1)$ | 55 : $P_{80} = (7, 7, 1)$ |
| 12 : $P_{20} = (0, 1, 1)$ | 34 : $P_{52} = (6, 4, 1)$ | 56 : $P_{81} = (8, 7, 1)$ |
| 13 : $P_{21} = (2, 1, 1)$ | 35 : $P_{54} = (8, 4, 1)$ | 57 : $P_{85} = (3, 8, 1)$ |
| 14 : $P_{22} = (3, 1, 1)$ | 36 : $P_{55} = (0, 5, 1)$ | 58 : $P_{86} = (4, 8, 1)$ |
| 15 : $P_{23} = (4, 1, 1)$ | 37 : $P_{56} = (1, 5, 1)$ | 59 : $P_{87} = (5, 8, 1)$ |
| 16 : $P_{26} = (7, 1, 1)$ | 38 : $P_{59} = (4, 5, 1)$ | 60 : $P_{88} = (6, 8, 1)$ |
| 17 : $P_{27} = (8, 1, 1)$ | 39 : $P_{60} = (5, 5, 1)$ | 61 : $P_{89} = (7, 8, 1)$ |
| 18 : $P_{28} = (0, 2, 1)$ | 40 : $P_{61} = (6, 5, 1)$ | 62 : $P_{90} = (8, 8, 1)$ |
| 19 : $P_{29} = (1, 2, 1)$ | 41 : $P_{63} = (8, 5, 1)$ |                           |
| 20 : $P_{31} = (3, 2, 1)$ | 42 : $P_{64} = (0, 6, 1)$ |                           |
| 21 : $P_{33} = (5, 2, 1)$ | 43 : $P_{65} = (1, 6, 1)$ |                           |

( 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 31, 33, 35, 36, 37, 39, 40, 42, 43, 45, 47, 48, 49, 50, 52, 54, 55, 56, 59, 60, 61, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 79, 80, 81, 85, 86, 87, 88, 89, 90 )

The lines and their points of contact are:

$$\begin{aligned}
a_1 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \alpha^3 \end{bmatrix}_8 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 8 \end{bmatrix}_8 \quad P_0 = \mathbf{P}(1, 0, 0) \ 4\times \\
a_2 &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & 1 \end{bmatrix}_{51} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 1 \end{bmatrix}_{51} \quad P_{83} = \mathbf{P}(1, 8, 1) \ 4\times
\end{aligned}$$

$$\begin{aligned}
a_3 &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^6 \end{bmatrix}^{15} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 5 \end{bmatrix}^{15} & P_{57} = \mathbf{P}(2, 5, 1) \ 4\times \\
a_4 &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^2 \end{bmatrix}^{17} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 7 \end{bmatrix}^{17} & P_{53} = \mathbf{P}(7, 4, 1) \ 4\times \\
a_5 &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^7 \end{bmatrix}^{74} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 4 \end{bmatrix}^{74} & P_{30} = \mathbf{P}(2, 2, 1) \ 4\times \\
a_6 &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^2 \end{bmatrix}^{77} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 7 \end{bmatrix}^{77} & P_5 = \mathbf{P}(2, 1, 0) \ 4\times \\
b_1 &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & \alpha^7 \end{bmatrix}^{54} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 4 \end{bmatrix}^{54} & P_{58} = \mathbf{P}(3, 5, 1) \ 4\times \\
b_2 &= \begin{bmatrix} 1 & 0 & \alpha^7 \\ 0 & 1 & \alpha^6 \end{bmatrix}^{45} = \begin{bmatrix} 1 & 0 & 4 \\ 0 & 1 & 5 \end{bmatrix}^{45} & P_{14} = \mathbf{P}(3, 0, 1) \ 4\times \\
b_3 &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & 1 \end{bmatrix}^{31} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \end{bmatrix}^{31} & P_{62} = \mathbf{P}(7, 5, 1) \ 4\times \\
b_4 &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & \alpha^2 \end{bmatrix}^{67} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 7 \end{bmatrix}^{67} & P_{77} = \mathbf{P}(4, 7, 1) \ 4\times \\
b_5 &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & \alpha^3 \end{bmatrix}^{68} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 8 \end{bmatrix}^{68} & P_{41} = \mathbf{P}(4, 3, 1) \ 4\times \\
b_6 &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & \alpha^2 \end{bmatrix}^{37} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 7 \end{bmatrix}^{37} & P_3 = \mathbf{P}(1, 1, 1) \ 4\times \\
c_{12} &= \begin{bmatrix} 1 & 0 & \alpha^3 \\ 0 & 1 & \alpha^4 \end{bmatrix}^{82} = \begin{bmatrix} 1 & 0 & 8 \\ 0 & 1 & 2 \end{bmatrix}^{82} & P_{17} = \mathbf{P}(6, 0, 1) \ 4\times \\
c_{13} &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & \alpha^4 \end{bmatrix}^{32} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 2 \end{bmatrix}^{32} & P_{84} = \mathbf{P}(2, 8, 1) \ 4\times \\
c_{14} &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & 1 \end{bmatrix}^{61} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 1 \end{bmatrix}^{61} & P_{32} = \mathbf{P}(4, 2, 1) \ 4\times \\
c_{15} &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & 0 \end{bmatrix}^{60} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 0 \end{bmatrix}^{60} & P_1 = \mathbf{P}(0, 1, 0) \ 4\times \\
c_{16} &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & \alpha^6 \end{bmatrix}^{35} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 5 \end{bmatrix}^{35} & P_{51} = \mathbf{P}(5, 4, 1) \ 4\times \\
c_{23} &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^5 \end{bmatrix}^{16} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 6 \end{bmatrix}^{16} & P_{82} = \mathbf{P}(0, 8, 1) \ 4\times \\
c_{24} &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^7 \end{bmatrix}^{14} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 4 \end{bmatrix}^{14} & P_{25} = \mathbf{P}(6, 1, 1) \ 4\times \\
c_{25} &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^4 \end{bmatrix}^{72} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 2 \end{bmatrix}^{72} & P_{76} = \mathbf{P}(3, 7, 1) \ 4\times \\
c_{26} &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^3 \end{bmatrix}^{78} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 8 \end{bmatrix}^{78} & P_{44} = \mathbf{P}(7, 3, 1) \ 4\times \\
c_{34} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & \alpha^3 \end{bmatrix}^{28} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 8 \end{bmatrix}^{28} & P_{38} = \mathbf{P}(1, 3, 1) \ 4\times
\end{aligned}$$

$$\begin{aligned}
c_{35} &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & \alpha^4 \end{bmatrix}_{52} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 2 \end{bmatrix}_{52} & P_{24} = \mathbf{P}(5, 1, 1) \ 4\times \\
c_{36} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & \alpha^7 \end{bmatrix}_{24} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 4 \end{bmatrix}_{24} & P_{78} = \mathbf{P}(5, 7, 1) \ 4\times \\
c_{45} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & \alpha^6 \end{bmatrix}_{25} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 5 \end{bmatrix}_{25} & P_{34} = \mathbf{P}(6, 2, 1) \ 4\times \\
c_{46} &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & \alpha \end{bmatrix}_{53} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 3 \end{bmatrix}_{53} & P_{46} = \mathbf{P}(0, 4, 1) \ 4\times \\
c_{56} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & 1 \end{bmatrix}_{21} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}_{21} & P_4 = \mathbf{P}(1, 1, 0) \ 4\times \\
d &= \begin{bmatrix} 1 & \alpha^6 & 0 \\ 0 & 0 & 1 \end{bmatrix}_{59} = \begin{bmatrix} 1 & 5 & 0 \\ 0 & 0 & 1 \end{bmatrix}_{59} & P_2 = \mathbf{P}(0, 0, 1) \ 4\times
\end{aligned}$$

Rank of lines: ( 8, 51, 15, 17, 74, 77, 54, 45, 31, 67, 68, 37, 82, 32, 61, 60, 35, 16, 14, 72, 78, 28, 52, 24, 25, 53, 21, 59 )

Line type:  $1^{28}$

|    |   |                                         |
|----|---|-----------------------------------------|
|    |   | 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,          |
| 28 | 1 | 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, |
|    |   | 21, 22, 23, 24, 25, 26, 27, 0           |

point types:  $1^{28}$

|    |   |                                         |
|----|---|-----------------------------------------|
|    |   | 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,          |
| 28 | 1 | 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, |
|    |   | 21, 22, 23, 24, 25, 26, 27, 0           |

point types for points off the curve:  $4^{63}$

|    |   |                                         |
|----|---|-----------------------------------------|
|    |   | 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,          |
|    |   | 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, |
|    |   | 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, |
| 63 | 4 | 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, |
|    |   | 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, |
|    |   | 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, |
|    |   | 61, 62, 0                               |

Lines on points off the curve:

Off point 0 =  $P_6 = (3, 1, 0)$  lies on 4 bisecants : { 4, 9, 16, 17 }

Off point 1 =  $P_7 = (4, 1, 0)$  lies on 4 bisecants : { 13, 14, 23, 25 }

Off point 2 =  $P_8 = (5, 1, 0)$  lies on 4 bisecants : { 1, 3, 19, 24 }

Off point 3 =  $P_9 = (6, 1, 0)$  lies on 4 bisecants : { 6, 11, 12, 20 }

Off point 4 =  $P_{10} = (7, 1, 0)$  lies on 4 bisecants : { 2, 10, 22, 27 }

Off point 5 =  $P_{11} = (8, 1, 0)$  lies on 4 bisecants : { 7, 8, 18, 21 }

Off point 6 =  $P_{12} = (1, 0, 1)$  lies on 4 bisecants : { 2, 3, 17, 18 }

Off point 7 =  $P_{13} = (2, 0, 1)$  lies on 4 bisecants : { 21, 23, 24, 26 }

Off point 8 =  $P_{15} = (4, 0, 1)$  lies on 4 bisecants : { 8, 11, 13, 16 }

Off point 9 =  $P_{16} = (5, 0, 1)$  lies on 4 bisecants : { 4, 5, 19, 20 }

Off point 10 =  $P_{18} = (7, 0, 1)$  lies on 4 bisecants : { 1, 6, 22, 25 }

Off point 11 =  $P_{19} = (8, 0, 1)$  lies on 4 bisecants : { 9, 10, 14, 15 }

Off point 12 =  $P_{20} = (0, 1, 1)$  lies on 4 bisecants : { 1, 8, 14, 26 }  
 Off point 13 =  $P_{21} = (2, 1, 1)$  lies on 4 bisecants : { 7, 9, 20, 25 }  
 Off point 14 =  $P_{22} = (3, 1, 1)$  lies on 4 bisecants : { 3, 10, 12, 23 }  
 Off point 15 =  $P_{23} = (4, 1, 1)$  lies on 4 bisecants : { 5, 6, 17, 24 }  
 Off point 16 =  $P_{26} = (7, 1, 1)$  lies on 4 bisecants : { 16, 19, 21, 27 }  
 Off point 17 =  $P_{27} = (8, 1, 1)$  lies on 4 bisecants : { 2, 4, 13, 15 }  
 Off point 18 =  $P_{28} = (0, 2, 1)$  lies on 4 bisecants : { 12, 13, 19, 22 }  
 Off point 19 =  $P_{29} = (1, 2, 1)$  lies on 4 bisecants : { 6, 10, 16, 26 }  
 Off point 20 =  $P_{31} = (3, 2, 1)$  lies on 4 bisecants : { 2, 5, 21, 25 }  
 Off point 21 =  $P_{33} = (5, 2, 1)$  lies on 4 bisecants : { 1, 9, 18, 27 }  
 Off point 22 =  $P_{35} = (7, 2, 1)$  lies on 4 bisecants : { 7, 11, 17, 23 }  
 Off point 23 =  $P_{36} = (8, 2, 1)$  lies on 4 bisecants : { 3, 8, 15, 20 }  
 Off point 24 =  $P_{37} = (0, 3, 1)$  lies on 4 bisecants : { 4, 6, 18, 23 }  
 Off point 25 =  $P_{39} = (2, 3, 1)$  lies on 4 bisecants : { 1, 5, 12, 16 }  
 Off point 26 =  $P_{40} = (3, 3, 1)$  lies on 4 bisecants : { 8, 9, 22, 24 }  
 Off point 27 =  $P_{42} = (5, 3, 1)$  lies on 4 bisecants : { 3, 7, 13, 26 }  
 Off point 28 =  $P_{43} = (6, 3, 1)$  lies on 4 bisecants : { 2, 11, 14, 19 }  
 Off point 29 =  $P_{45} = (8, 3, 1)$  lies on 4 bisecants : { 15, 17, 25, 27 }  
 Off point 30 =  $P_{47} = (1, 4, 1)$  lies on 4 bisecants : { 5, 7, 14, 22 }  
 Off point 31 =  $P_{48} = (2, 4, 1)$  lies on 4 bisecants : { 8, 10, 17, 19 }  
 Off point 32 =  $P_{49} = (3, 4, 1)$  lies on 4 bisecants : { 4, 11, 26, 27 }  
 Off point 33 =  $P_{50} = (4, 4, 1)$  lies on 4 bisecants : { 1, 2, 20, 23 }  
 Off point 34 =  $P_{52} = (6, 4, 1)$  lies on 4 bisecants : { 6, 9, 13, 21 }  
 Off point 35 =  $P_{54} = (8, 4, 1)$  lies on 4 bisecants : { 12, 15, 18, 24 }  
 Off point 36 =  $P_{55} = (0, 5, 1)$  lies on 4 bisecants : { 3, 5, 9, 11 }  
 Off point 37 =  $P_{56} = (1, 5, 1)$  lies on 4 bisecants : { 13, 20, 24, 27 }  
 Off point 38 =  $P_{59} = (4, 5, 1)$  lies on 4 bisecants : { 18, 19, 25, 26 }  
 Off point 39 =  $P_{60} = (5, 5, 1)$  lies on 4 bisecants : { 12, 14, 17, 21 }  
 Off point 40 =  $P_{61} = (6, 5, 1)$  lies on 4 bisecants : { 1, 4, 7, 10 }  
 Off point 41 =  $P_{63} = (8, 5, 1)$  lies on 4 bisecants : { 15, 16, 22, 23 }  
 Off point 42 =  $P_{64} = (0, 6, 1)$  lies on 4 bisecants : { 0, 10, 20, 21 }  
 Off point 43 =  $P_{65} = (1, 6, 1)$  lies on 4 bisecants : { 0, 9, 19, 23 }  
 Off point 44 =  $P_{66} = (2, 6, 1)$  lies on 4 bisecants : { 0, 11, 18, 22 }  
 Off point 45 =  $P_{67} = (3, 6, 1)$  lies on 4 bisecants : { 0, 1, 13, 17 }  
 Off point 46 =  $P_{68} = (4, 6, 1)$  lies on 4 bisecants : { 0, 7, 12, 27 }  
 Off point 47 =  $P_{69} = (5, 6, 1)$  lies on 4 bisecants : { 0, 2, 6, 8 }  
 Off point 48 =  $P_{70} = (6, 6, 1)$  lies on 4 bisecants : { 0, 3, 16, 25 }  
 Off point 49 =  $P_{71} = (7, 6, 1)$  lies on 4 bisecants : { 0, 4, 14, 24 }  
 Off point 50 =  $P_{72} = (8, 6, 1)$  lies on 4 bisecants : { 0, 5, 15, 26 }  
 Off point 51 =  $P_{73} = (0, 7, 1)$  lies on 4 bisecants : { 2, 7, 16, 24 }  
 Off point 52 =  $P_{74} = (1, 7, 1)$  lies on 4 bisecants : { 4, 8, 12, 25 }  
 Off point 53 =  $P_{75} = (2, 7, 1)$  lies on 4 bisecants : { 3, 6, 14, 27 }  
 Off point 54 =  $P_{79} = (6, 7, 1)$  lies on 4 bisecants : { 17, 20, 22, 26 }  
 Off point 55 =  $P_{80} = (7, 7, 1)$  lies on 4 bisecants : { 5, 10, 13, 18 }  
 Off point 56 =  $P_{81} = (8, 7, 1)$  lies on 4 bisecants : { 1, 11, 15, 21 }  
 Off point 57 =  $P_{85} = (3, 8, 1)$  lies on 4 bisecants : { 14, 16, 18, 20 }  
 Off point 58 =  $P_{86} = (4, 8, 1)$  lies on 4 bisecants : { 3, 4, 21, 22 }  
 Off point 59 =  $P_{87} = (5, 8, 1)$  lies on 4 bisecants : { 10, 11, 24, 25 }  
 Off point 60 =  $P_{88} = (6, 8, 1)$  lies on 4 bisecants : { 5, 8, 23, 27 }  
 Off point 61 =  $P_{89} = (7, 8, 1)$  lies on 4 bisecants : { 2, 9, 12, 26 }  
 Off point 62 =  $P_{90} = (8, 8, 1)$  lies on 4 bisecants : { 6, 7, 15, 19 }

The following command creates the curve over  $\mathbb{F}_{13}$  with catalogue number 0 and produces a report. The command also exports the orbit of bitangents of length 4:

**Example 325**

```
quartic_curve_13_0_report:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define C -quartic_curve -space P -catalogue 0 \
▷ ▷ ▷ -transform "10,4,1,11,5,11,4,1,1" \
▷ ▷ ▷ -transform_inverse "9,1,0,12,9,0,2,10,11" \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ ▷ -quartic_curve_activity \
▷ ▷ ▷ ▷ -report \
▷ ▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ ▷ -quartic_curve_activity \
▷ ▷ ▷ ▷ -extract_orbit_on_bitangents_by_length 4 \
▷ ▷ ▷ -end
▷ #pdflatex quartic_curve_catalogue_q13_iso0_report.tex
▷ #$(OPEN) quartic_curve_catalogue_q13_iso0_report.pdf
```

Suppose we want to create reports on all quartic curves for a given value of  $q$ , utilizing the Orbiter knowledge base of quartic curves. Here is an example, considering the quartic curves over  $\mathbb{F}_{19}$ . We use a makefile variable to set the number of curves:

**Example 326**

```
NB_QUARTIC_CURVES_Q19=14
```

Now, we create each curve one-by-one and produce a report. To do so, we utilize the loop command. Once done, we translate the latex files:

**Example 327**

```
quartic_curves_19_report:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 19 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -loop L 0 $(NB_QUARTIC_CURVES_Q19) 1 \
▷ ▷ -define C -quartic_curve -space P -catalogue %L -end \
▷ ▷ -with C -do \
▷ ▷ ▷ -quartic_curve_activity \
▷ ▷ ▷ ▷ -report \
▷ ▷ ▷ -end \
▷ ▷ -end_loop L
▷ ▷ pdflatex quartic_curve_catalogue_q19_iso0_report.tex
▷ ▷ pdflatex quartic_curve_catalogue_q19_iso1_report.tex
```

```

> > pdflatex quartic_curve_catalogue.q19.iso2.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso3.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso4.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso5.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso6.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso7.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso8.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso9.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso10.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso11.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso12.report.tex
> > pdflatex quartic_curve_catalogue.q19.iso13.report.tex

```

In the next example, we will study the generation of a quartic curve from a cubic surface. The example is the quartic curve over  $\mathbb{F}_9$ . It is interesting, as it contains the maximum number of Kovalevski points, which is 63 (a Kovalevski point is a point where 4 bitangents meet). There are two cubic surfaces with 27 lines over  $\mathbb{F}_9$ . The first one has 10 Eckardt points, whereas the second one has 9. The quartic curve arises from the surface with 9 Eckardt points (OCN=1). The surface has a unique point not on any line. For this reason, we set the orbit number on points not on lines to zero. Here is the command:

#### Example 328

```

quartic_curve_q9_1:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 9 -end \
> > -define P2 -projective_space -n 2 -field F -end \
> > -define P3 -projective_space -n 3 -field F -end \
> > -define S9_1 -cubic_surface -space P3 -catalogue 1 -end \
> > -define C -quartic_curve -space P2 -from_cubic_surface S9_1 0 -end \
> > -with C -do \
> > > -quartic_curve_activity \
> > > > -report \
> > > -end
> pdflatex quartic_curve_surface.surface_pt_orb_0.report.tex
> $(OPEN) quartic_curve_surface.surface_pt_orb_0.report.pdf

```

## 8.6 Classification of Quartic Curves

Let us now address the problem of classification of quartic curves. For a fixed field  $\mathbb{F}_q$ ,  $q$  odd, we can exploit the previously classified list of cubic surfaces as a start. For each surface, and for each orbit on points not on lines, we perform the projection along the tangent cone to create one quartic curve. This guarantees that each isomorphism type of quartic curve with 28 bitangents will be created. Afterwards, we apply isomorphism testing based on canonical forms. This is based on substructures and using canonical forms for graphs using the built-in package Nauty.

Let us look at some examples of this algorithm. We try  $q = 7$  and then  $q = 13$ . We will set a makefile variable to the number of (isomorphism types of) cubic surfaces with 27 lines over  $\mathbb{F}_q$ . For  $q = 7$ , there is exactly one isomorphism type, so we put

### Example 329

```
NB_CUBIC_SURFACES_Q7=1
```

The next command creates a list of quartic curves using a projection construction. For each isomorphism type of cubic surface, and for each point not on any line, we consider the intersection of the tangent cone with the surface and project onto a plane not containing the point. Because of symmetry, it suffices to perform this projection only for a set of representatives of the orbits on points not on lines. Here is the Orbiter command for  $q = 7$ :

### Example 330

```
quartic_curves_q7:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define P -projective_space -n 3 -field F -end \
▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
▷ ▷ -end_loop L \
▷ ▷ -print_symbols \
▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
▷ ▷ ▷ -with S_%L -do \
▷ ▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ ▷ -export_all_quartic_curves \
▷ ▷ ▷ -end \
▷ ▷ -end_loop L \
▷ ▷ -print_symbols
```

The resulting curves are written to file. Unfortunately, in this example, there is no point which does not lie on any line of the surface. This means that no quartic curve with 28 lines exists over  $\mathbb{F}_7$ .

We move on to the next example, which is  $q = 13$ . Again, we use a makefile variable to set the number of isomorphism types of cubic surfaces with 27 lines over  $\mathbb{F}_{13}$ . There are exactly 4 types:

### Example 331

```
NB_CUBIC_SURFACES_Q13=4
```



Just like before, we create all quartic curves arising from projections:

#### Example 332

```
quartic_curves_q13:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define P -projective_space -n 3 -field F -end \
▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
▷ ▷ -end_loop L \
▷ ▷ -print_symbols \
▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
▷ ▷ ▷ -with S_%L -do \
▷ ▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ ▷ -export_all_quartic_curves \
▷ ▷ ▷ -end \
▷ ▷ -end_loop L \
▷ ▷ -print_symbols
```

Every cubic surface creates a number of quartic curves. Namely, each orbit of the automorphism group on points not on lines of the cubic surface creates one quartic curve. The curves arising this way are written to file. There is one file for each surface. To prepare for the classification, we first combine these output files into one:

#### Example 333

```
quartic_curves_q13_combine:
▷ $(ORBITER) -v 3 \
▷ ▷ -csv_file_concatenate_from_mask $(NB_CUBIC_SURFACES_Q13) \
▷ ▷ ▷ surface_catalogue_q13_iso%ld_quartics.csv \
▷ ▷ ▷ quartics.q13.csv
```

The following command performs the classification of quartic curves that are in the combined file. The result of this computation is the classification of quartic curves with 28 bitangents over the given field (here,  $\mathbb{F}_{13}$ ):

#### Example 334

```
quartic_curves_q13_classify:
▷ $(ORBITER) -v 30 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 3 \
▷ ▷ ▷ -homogeneous_of_degree 4 \
▷ ▷ ▷ -monomial_ordering_partition \
▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
▷ ▷ -end \
▷ ▷ -define O -orbits -classification_by_canonical_form \
▷ ▷ ▷ -space P \
```

```

> > > -ring R \
> > > -input_fname_mask quartics_q13.csv \
> > > -nb_files 1 \
> > > -output_fname quartic_curves.q13_classified \
> > > -label_po_go "PO_GO" \
> > > -label_po_index "PO_INDEX" \
> > > -label_po "PO" \
> > > -label_so "orbit" \
> > > -label_equation "curve" \
> > > -label_points "pts.on.curve" \
> > > -label_lines "bitangents" \
> > > -carry_through "NB.E" \
> > > -carry_through "NB.DOUBLE" \
> > > -carry_through "NB.SINGLE" \
> > > -carry_through "NB.ZERO" \
> > > -algorithm_substructure \
> > > -substructure_size 4 \
> > > -end \
> > -end \
> -with 0 -do -orbits.activity \
> > -report \
> > -report_options \
> > > -fname quartics_q13 \
> > -end \
> -end
> pdflatex quartics_q13_orbits.tex
> $(OPEN) quartics_q13_orbits.pdf

```

Over the given field, we find exactly two isomorphism types. The output is shown below, showing the canonical equation of each curve, the automorphism group by means of generators and group order, and the number of rational points of the curve over the given field.

Classification

$q = 13$

Number of isomorphism classes: 2

Automorphism group order statistic: 48, 24

The isomorphism classes are:

Isomorphism class 0 / 2:

Number of rational points over  $\mathbb{F}_{13}$ : 8

Canonical equation:

$$\begin{aligned}
 &5X_0^3X_1 + 4X_0^3X_2 + 7X_0X_1^3 + 11X_1^3X_2 + 10X_0X_2^3 + 6X_1X_2^3 + 8X_0^2X_1^2 \\
 &\quad + 7X_0^2X_2^2 + 7X_1^2X_2^2 + 5X_0^2X_1X_2 + 7X_0X_1^2X_2 + X_0X_1X_2^2
 \end{aligned}$$

Stabilizer order: 24

The stabilizer:

Strong generators for a group of order 24:

$$\begin{bmatrix} 10 & 0 & 0 \\ 4 & 2 & 6 \\ 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 11 & 2 & 4 \\ 0 & 0 & 4 \\ 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 12 & 12 & 12 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

1,0,0,3,8,11,4,4,4,

1,12,11,0,0,11,0,6,0,

1,1,1,0,12,0,12,0,0,

Isomorphism class 1 / 2:

Number of rational points over  $\mathbb{F}_{13}$ : 4

Canonical equation:

$$5X_2^4 + 5X_0^3X_1 + 7X_0X_1^3 + X_0^2X_1^2$$

Stabilizer order: 48

The stabilizer:

Strong generators for a group of order 48:

$$\begin{bmatrix} 12 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 8 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 4 & 0 & 0 \\ 9 & 12 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 10 & 0 \\ 3 & 3 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,1,0,0,0,12,

1,0,0,0,1,0,0,0,5,

1,0,0,12,3,0,0,0,10,

0,1,0,12,12,0,0,0,4,

The classification also produces a cpp file. This is C++ code for the classification. The file contains the list of curves from the classification, together with extra information about the automorphism group. It can be used to build the knowledge about these quartic curves into Orbiter, by means of Orbiter's knowledge base of combinatorial objects. This way, the data that is computed by Orbiter can feed back into a new version of Orbiter.

## 8.7 Interface to GAP

We wish to export a cubic surface to GAP. The following command creates the Hirschfeld surface and exports it to GAP

### Example 335

```
surface_4.0-export_gap:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -end \
▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
▷ ▷ -with S -do \
▷ ▷ -cubic_surface_activity \
▷ ▷ ▷ -export_gap \
▷ ▷ -end
```

The following GAP file is written:

```
LoadPackage("fining");
# Cubic surface catalogue_q4_iso0
# Group:
mat1 := [[Z(4)^1,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^1]];
frob1 := FrobeniusAutomorphism(GF(4))^1;
psi1 := ProjectiveSemilinearMap(mat1, frob1,GF(4));
mat2 := [[Z(4)^2,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^2]];
frob2 := FrobeniusAutomorphism(GF(4))^1;
psi2 := ProjectiveSemilinearMap(mat2, frob2,GF(4));
mat3 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^2,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^0]];
frob3 := FrobeniusAutomorphism(GF(4))^0;
psi3 := ProjectiveSemilinearMap(mat3, frob3,GF(4));
mat4 := [[Z(4)^2,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^2,0*Z(4)],
[Z(4)^2,0*Z(4),0*Z(4),Z(4)^2]];
frob4 := FrobeniusAutomorphism(GF(4))^0;
psi4 := ProjectiveSemilinearMap(mat4, frob4,GF(4));
mat5 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[Z(4)^1,0*Z(4),Z(4)^2,0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),Z(4)^0]];
frob5 := FrobeniusAutomorphism(GF(4))^1;
psi5 := ProjectiveSemilinearMap(mat5, frob5,GF(4));
```

```

mat6 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[Z(4)^1,Z(4)^2,Z(4)^2,0*Z(4)],
[Z(4)^0,Z(4)^1,0*Z(4),Z(4)^0]];
frob6 := FrobeniusAutomorphism(GF(4))^1;
psi6 := ProjectiveSemilinearMap(mat6, frob6,GF(4));
mat7 := [[Z(4)^1,0*Z(4),0*Z(4),0*Z(4)],
[Z(4)^1,Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,Z(4)^1]];
frob7 := FrobeniusAutomorphism(GF(4))^1;
psi7 := ProjectiveSemilinearMap(mat7, frob7,GF(4));
mat8 := [[Z(4)^2,0*Z(4),0*Z(4),0*Z(4)],
[Z(4)^2,Z(4)^0,Z(4)^0,0*Z(4)],
[0*Z(4),Z(4)^0,0*Z(4),0*Z(4)],
[Z(4)^2,Z(4)^0,0*Z(4),Z(4)^2]];
frob8 := FrobeniusAutomorphism(GF(4))^1;
psi8 := ProjectiveSemilinearMap(mat8, frob8,GF(4));
mat9 := [[Z(4)^0,Z(4)^2,Z(4)^0,Z(4)^2],
[0*Z(4),Z(4)^2,Z(4)^2,Z(4)^1],
[Z(4)^2,Z(4)^0,Z(4)^2,Z(4)^0],
[Z(4)^0,Z(4)^1,0*Z(4),Z(4)^0]];
frob9 := FrobeniusAutomorphism(GF(4))^0;
psi9 := ProjectiveSemilinearMap(mat9, frob9,GF(4));
gens := [psi1, psi2, psi3, psi4, psi5, psi6, psi7, psi8, psi9];
G := Group(gens);
Size(G);
r := PolynomialRing(GF(4),["X0","X1","X2","X3"]);
Eqn := (r.1)^2*(r.4)+(r.2)^2*(r.3)+(r.2)*(r.3)^2+(r.1)*(r.4)^2;

```



# Chapter 9

## Applications

### 9.1 Number Theory

In Table 9.1, global Orbiter commands for Algebra are summarized.

Table 9.2 shows Orbiter commands for basic number theory, including integer factor rings and the Euclidean algorithm.

In Table 9.3, some number theoretic commands are shown.

To compute primitive roots, there exists a randomized fast algorithm. On the other hand, there is also an algorithm that finds the smallest primitive root. For instance,

#### Example 336

```
PR101:  
▷ $(ORBITER) -v 5 -primitive_root 101
```

finds a primitive root modulo 101 using the randomized algorithm. The command finds the root 75. On the other hand, the command

#### Example 337

```
PR101_smallest:  
▷ $(ORBITER) -v 5 -smallest_primitive_root 101
```

finds the smallest primitive root modulo 101, which turns out to be 2.

#### Example 338

```
PR29:  
▷ $(ORBITER) -v 1 -smallest_primitive_root 29
```

computes a primitive root modulo 29. The answer in this case is 2.

It is possible to compute primitive roots for all prime numbers in a given interval. The command

| Global Algebra Commands             |                                         |                                                                                                                                                                                                      |
|-------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                             | Arguments                               | Purpose                                                                                                                                                                                              |
| -primitive_root                     | $p$                                     | Computes a primitive root modulo $p$ .                                                                                                                                                               |
| -smallest_primitive_root            | $p$                                     | Computes the smallest primitive root modulo $p$ .                                                                                                                                                    |
| -smallest_primitive_root_interval   | $m1\ m2$                                | Computes the smallest primitive root modulo $m$ for all $m$ with $m1 \leq m \leq m2$ .                                                                                                               |
| -number_of_primitive_roots_interval | $m1\ m2$                                | Counts the number of primitive roots modulo $m$ for all $m$ with $m1 \leq m \leq m2$ .                                                                                                               |
| -inverse_mod                        | $a\ p$                                  | Computes the modular inverse of $a$ modulo $p$ , i.e. an integer $b$ with $ab \equiv 1 \pmod{p}$ .                                                                                                   |
| -extended_gcd                       | $a\ b$                                  | Computes integers $g, u$ , and $v$ such that $g = \gcd(a, b) = ua + vb$ .                                                                                                                            |
| -power_mod                          | $a\ k\ n$                               | Compute the power $a^k \pmod{n}$ .                                                                                                                                                                   |
| -discrete_log                       | $y\ a\ m$                               | Computes $x$ such that $a^x \equiv y \pmod{m}$ . Employs a brute force algorithm.                                                                                                                    |
| -square_root                        | $n$                                     | computes $\lfloor \sqrt{a} \rfloor$ over the integers.                                                                                                                                               |
| -square_root_mod                    | $a\ p$                                  | Computes an integer $b$ such that $b^2 \equiv a \pmod{p}$ . Here, $p$ must be a prime number.                                                                                                        |
| -all_square_roots_mod_n             | $a\ n$                                  | Computes all integers $b$ such that $b^2 \equiv a \pmod{n}$ .                                                                                                                                        |
| -count_subprimitive                 | $Q\_max$<br>$H\_max$                    | Count subprimitive polynomials for $q \leq Q\_max$                                                                                                                                                   |
| -character_table_symmetric_group    | $n$                                     | Compute the character table of the symmetric group of degree $n$ .                                                                                                                                   |
| -make_A5_in_PSL_2_q                 | $q$                                     | Compute generators for a subgroup of $\text{PSL}(2, q)$ isomorphic to $\text{Alt}(5)$ .                                                                                                              |
| -order_of_q_mod_n                   | $q\ n_{\min}\ n_{\max}$                 | Computes the order $\text{ord}(q, n)$ of $q$ modulo $n$ for all $n$ with $n_{\min} \leq n \leq n_{\max}$ for which $\gcd(n, q) = 1$ . Also computes $\varphi(n)$ and $\varphi(n)/\text{ord}(q, n)$ . |
| -eulerfunction_interval             | $n1\ n2$                                | Compute Euler's totient function $\varphi(n)$ for all $n$ with $n1 \leq n \leq n2$ .                                                                                                                 |
| -young_symmetrizer                  | $n$                                     | Compute the Young symmetrizer in the symmetric group of degree $n$ .                                                                                                                                 |
| -young_symmetrizer_sym_4            |                                         | Compute the Young symmetrizer in the symmetric group of degree 4.                                                                                                                                    |
| -draw_mod_n                         | $\text{descr}$                          | Draw a diagram with $n$ vertices equally spaced out on a circle.                                                                                                                                     |
| -power_function_mod_n               | $k\ n$                                  |                                                                                                                                                                                                      |
| -all_rational_normal_forms          | $F\ d$                                  | Compute all rational normal forms of elements in $\text{GL}(d, F)$ , where $F$ is a field.                                                                                                           |
| -eigenstuff                         | $F\ n\ \text{coeffs}$<br>$\text{fname}$ | Compute the eigenvalues and eigenvectors of a matrix.                                                                                                                                                |
| -jacobi                             | $a\ p$                                  | Computes the Jacobi symbol $\left(\frac{a}{p}\right)$                                                                                                                                                |
| -smith_normal_form                  | $M$                                     | Compute the Smith Normal Form of the matrix $M$ .                                                                                                                                                    |

Table 9.1: Global Algebra Commands



| Basic Number Theory Commands |           |                                                                                                                |
|------------------------------|-----------|----------------------------------------------------------------------------------------------------------------|
| Command                      | Arguments | Purpose                                                                                                        |
| -draw_mod_n                  | descr     | Draws the integers modulo $n$ on a circle.                                                                     |
| -Chinese_remainders          | R M       | Solves a system of congruences with remainders R and moduli M. R and M must be vectors whose labels are given. |

Table 9.2: Basic Number Theory Commands

| Number Theoretic Commands |                |                                                                                                                                        |
|---------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Command                   | Arguments      | Purpose                                                                                                                                |
| -sift_smooth              | $a$ $n$ primes | Computes all smooth numbers in the interval $[a, a + n - 1]$ . Smooth means that they factor completely over the list of primes given. |
| -random                   | $n$ fname      | Creates $n$ random numbers and writes them to the csv file fname                                                                       |
| -random_last              | $n$            | Creates $n$ random numbers prints the last one                                                                                         |
| -affine_sequence          | $a$ $b$ $p$    | Splits the interval $[0, p - 1]$ into affine sequences of the form $x_{n+1} = ax_n + b \pmod{p}$                                       |

Table 9.3: Number Theoretic Commands

**Example 339**

```
PR_2_1000:
▷ $(ORBITER) -v 1 -smallest_primitive_root_interval 2 1000
▷ $(ORBITER) -v 1 -csv_file_latex 1 \
▷ > primitive_element_table_2_1000.csv
▷ pdflatex primitive_element_table_2_1000.tex
▷ $(OPEN) primitive_element_table_2_1000.pdf
```

finds all primitive roots for all primes between 2 and 1000.

The command

**Example 340**

```
PE_number_2_1000:
▷ $(ORBITER) -v 1 -number_of_primitive_roots_interval 2 1000
▷ $(ORBITER) -v 1 -csv_file_latex 1 table_number_of_pe_2_1000.csv
▷ pdflatex table_number_of_pe_2_1000.tex
▷ $(OPEN) table_number_of_pe_2_1000.pdf
```

determines the number of primitive elements for all primes between 2 and 1000.

The command

**Example 341**

```
PR_915839:
▷ $(ORBITER) -v 5 -primitive_root 915839
```

computes a primitive root modulo 915839. The answer is 43085. Let us check this result. The command

**Example 342**

```
PR_915839_check:
▷ $(ORBITER) -v 5 -power_mod 43085 49842 915839
```

computes

$$43085^{49842} \bmod 915839$$

which is 487320.

The command `-discrete_log` can be used to compute the discrete logarithm of  $a$  modulo  $p$  with respect to  $b$ . This means, a number  $k$  is computed such that

$$b^k \equiv a \pmod{p}.$$

For instance, the discrete log of 487320 with respect to the base 43085 modulo 915839 is 49842, based on the previous example. We can compute the discrete logarithm using the command

**Example 343**

```
DL_915839:
▷ $(ORBITER) -v 5 -discrete_log 487320 43085 915839
```

This command can be quite expensive.

Computing inverses modulo a prime  $p$  is possible using the `-inverse_mod` command. The command

**Example 344**

```
IM:
▷ $(ORBITER) -v 5 -inverse_mod 1865025205 2147483647
```

computes the inverse of 1865025205 modulo 2147483647 which is 579785381.

A different way of computing the inverse is by using the 1-trick. This approach computes the gcd of two numbers  $a$  and  $b$ , say, and writes

$$\gcd(a, b) = ua + vb$$

for some  $u, v \in \mathbb{Z}$ . The `-extended_gcd` command can be used. For instance, the following command computes the gcd of  $a = 2147483647$  and  $b = 1865025205$ .

**Example 345**

```
IM_gcd:
▷ $(ORBITER) -v 5 -extended_gcd 1865025205 2147483647
```

The output is

$$1 = -503526232 * 2147483647 + 579785381 * 1865025205,$$

from which we see that  $\gcd(a, b) = 1$  and  $u = -503526232$  and  $v = 579785381$ , which is the gcd written as a lattice combination of the input arguments. The inverse of  $1865025205 \bmod 2147483647$  is  $v = 579785381$ .

In order to compute the modular power

$$a^e \bmod n,$$

the `-power_mod` command can be used. For instance,

**Example 346**

```
PM3a:
▷ $(ORBITER) -v 5 -power_mod 16807 1073741823 2147483647
```

computes 16807 raised to the power 1073741823 modulo 2147483647, which is 2147483646.

The modular square root of  $a$  modulo  $p$  is any  $x$  such that

$$x^2 \equiv a \bmod p.$$

The command `-square_root_mod` can be used to compute modular square roots using the algorithm of Tonelli and Shanks (cf. [21]). For instance,

**Example 347**

```
sqrt_mod:
▷ $(ORBITER) -v 2 -square_root_mod 33 41
```

finds that the square root of  $33 \bmod 41$  is 19, i.e.

$$19^2 \equiv 33 \bmod 41.$$

The command `order_of_q_mod_n` computes  $\text{ord}(q, n)$ , the order of  $q$  modulo  $n$ , for all  $n$  with  $n_{\min} \leq n \leq n_{\max}$  and  $\gcd(n, q) = 1$ . It also computes Euler's totient function  $\varphi(n)$  and the cofactor  $\varphi(n)/\text{ord}(q, n)$ . For instance,

**Example 348**

```
order_of_2_mod_n:
▷ $(ORBITER) -v 3 -order_of_q_mod_n 2 3 51
▷ $(ORBITER) -v 1 -csv_file_latex 1 \
▷ ▷ order_of_q_mod_n.q2_3_51.csv
▷ pdflatex order_of_q_mod_n.q2_3_51.tex
▷ $(OPEN) order_of_q_mod_n.q2_3_51.pdf
```

produces the output shown below

| N  | ORD | PHI | COF | N   | ORD | PHI | COF | N   | ORD | PHI | COF |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 3  | 2   | 2   | 1   | 53  | 52  | 52  | 1   | 103 | 51  | 102 | 2   |
| 5  | 4   | 4   | 1   | 55  | 20  | 40  | 2   | 105 | 12  | 48  | 4   |
| 7  | 3   | 6   | 2   | 57  | 18  | 36  | 2   | 107 | 106 | 106 | 1   |
| 9  | 6   | 6   | 1   | 59  | 58  | 58  | 1   | 109 | 36  | 108 | 3   |
| 11 | 10  | 10  | 1   | 61  | 60  | 60  | 1   | 111 | 36  | 72  | 2   |
| 13 | 12  | 12  | 1   | 63  | 6   | 36  | 6   | 113 | 28  | 112 | 4   |
| 15 | 4   | 8   | 2   | 65  | 12  | 48  | 4   | 115 | 44  | 88  | 2   |
| 17 | 8   | 16  | 2   | 67  | 66  | 66  | 1   | 117 | 12  | 72  | 6   |
| 19 | 18  | 18  | 1   | 69  | 22  | 44  | 2   | 119 | 24  | 96  | 4   |
| 21 | 6   | 12  | 2   | 71  | 35  | 70  | 2   | 121 | 110 | 110 | 1   |
| 23 | 11  | 22  | 2   | 73  | 9   | 72  | 8   | 123 | 20  | 80  | 4   |
| 25 | 20  | 20  | 1   | 75  | 20  | 40  | 2   | 125 | 100 | 100 | 1   |
| 27 | 18  | 18  | 1   | 77  | 30  | 60  | 2   | 127 | 7   | 126 | 18  |
| 29 | 28  | 28  | 1   | 79  | 39  | 78  | 2   | 129 | 14  | 84  | 6   |
| 31 | 5   | 30  | 6   | 81  | 54  | 54  | 1   | 131 | 130 | 130 | 1   |
| 33 | 10  | 20  | 2   | 83  | 82  | 82  | 1   | 133 | 18  | 108 | 6   |
| 35 | 12  | 24  | 2   | 85  | 8   | 64  | 8   | 135 | 36  | 72  | 2   |
| 37 | 36  | 36  | 1   | 87  | 28  | 56  | 2   | 137 | 68  | 136 | 2   |
| 39 | 12  | 24  | 2   | 89  | 11  | 88  | 8   | 139 | 138 | 138 | 1   |
| 41 | 20  | 40  | 2   | 91  | 12  | 72  | 6   | 141 | 46  | 92  | 2   |
| 43 | 14  | 42  | 3   | 93  | 10  | 60  | 6   | 143 | 60  | 120 | 2   |
| 45 | 12  | 24  | 2   | 95  | 36  | 72  | 2   | 145 | 28  | 112 | 4   |
| 47 | 23  | 46  | 2   | 97  | 48  | 96  | 2   | 147 | 42  | 84  | 2   |
| 49 | 21  | 42  | 2   | 99  | 30  | 60  | 2   | 149 | 148 | 148 | 1   |
| 51 | 8   | 32  | 4   | 101 | 100 | 100 | 1   | 151 | 15  | 150 | 10  |

The command

#### Example 349

```
Eulerfunction_150:
> $(ORBITER) -v 1 -eulerfunction.interval 1 150
> $(ORBITER) -v 1 -csv_file.latex 1 \
> > table_eulerfunction_1_150.csv
> pdflatex table_eulerfunction_1_150.tex
> $(OPEN) table_eulerfunction_1_150.pdf
```

computes Euler's totient function for all integers  $n$  with  $1 \leq n \leq 150$ . The result is shown below:

| N  | PHI | N  | PHI | N  | PHI | N   | PHI | N   | PHI | N   | PHI |
|----|-----|----|-----|----|-----|-----|-----|-----|-----|-----|-----|
| 1  | 1   | 26 | 12  | 51 | 32  | 76  | 36  | 101 | 100 | 126 | 36  |
| 2  | 1   | 27 | 18  | 52 | 24  | 77  | 60  | 102 | 32  | 127 | 126 |
| 3  | 2   | 28 | 12  | 53 | 52  | 78  | 24  | 103 | 102 | 128 | 64  |
| 4  | 2   | 29 | 28  | 54 | 18  | 79  | 78  | 104 | 48  | 129 | 84  |
| 5  | 4   | 30 | 8   | 55 | 40  | 80  | 32  | 105 | 48  | 130 | 48  |
| 6  | 2   | 31 | 30  | 56 | 24  | 81  | 54  | 106 | 52  | 131 | 130 |
| 7  | 6   | 32 | 16  | 57 | 36  | 82  | 40  | 107 | 106 | 132 | 40  |
| 8  | 4   | 33 | 20  | 58 | 28  | 83  | 82  | 108 | 36  | 133 | 108 |
| 9  | 6   | 34 | 16  | 59 | 58  | 84  | 24  | 109 | 108 | 134 | 66  |
| 10 | 4   | 35 | 24  | 60 | 16  | 85  | 64  | 110 | 40  | 135 | 72  |
| 11 | 10  | 36 | 12  | 61 | 60  | 86  | 42  | 111 | 72  | 136 | 64  |
| 12 | 4   | 37 | 36  | 62 | 30  | 87  | 56  | 112 | 48  | 137 | 136 |
| 13 | 12  | 38 | 18  | 63 | 36  | 88  | 40  | 113 | 112 | 138 | 44  |
| 14 | 6   | 39 | 24  | 64 | 32  | 89  | 88  | 114 | 36  | 139 | 138 |
| 15 | 8   | 40 | 16  | 65 | 48  | 90  | 24  | 115 | 88  | 140 | 48  |
| 16 | 8   | 41 | 40  | 66 | 20  | 91  | 72  | 116 | 56  | 141 | 92  |
| 17 | 16  | 42 | 12  | 67 | 66  | 92  | 44  | 117 | 72  | 142 | 70  |
| 18 | 6   | 43 | 42  | 68 | 32  | 93  | 60  | 118 | 58  | 143 | 120 |
| 19 | 18  | 44 | 20  | 69 | 44  | 94  | 46  | 119 | 96  | 144 | 48  |
| 20 | 8   | 45 | 24  | 70 | 24  | 95  | 72  | 120 | 32  | 145 | 112 |
| 21 | 12  | 46 | 22  | 71 | 70  | 96  | 32  | 121 | 110 | 146 | 72  |
| 22 | 10  | 47 | 46  | 72 | 24  | 97  | 96  | 122 | 60  | 147 | 84  |
| 23 | 22  | 48 | 16  | 73 | 72  | 98  | 42  | 123 | 80  | 148 | 72  |
| 24 | 8   | 49 | 42  | 74 | 36  | 99  | 60  | 124 | 60  | 149 | 148 |
| 25 | 20  | 50 | 20  | 75 | 40  | 100 | 40  | 125 | 100 | 150 | 40  |

A power map sends  $a$  to  $a^k$  for some fixed  $k$ . Orbiter can compute power maps modulo  $p$ . For instance, the following command computes the function  $a \mapsto a^k \pmod{11}$ :

#### Example 350

```
power_function_2_mod_11:
> $(ORBITER) -v 5 -power_function_mod_n 2 11
> $(ORBITER) -v 1 -csv_file_latex 1 power_function_k2_n11.csv
> pdflatex power_function_k2_n11.tex
> $(OPEN) power_function_k2_n11.pdf
```

The result is shown below:

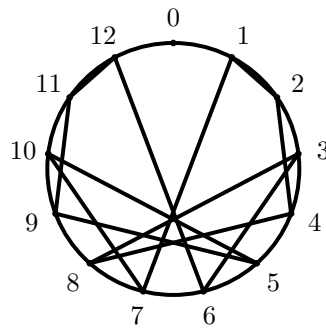
| A  | APOWK |
|----|-------|
| 0  | 0     |
| 1  | 1     |
| 2  | 4     |
| 3  | 9     |
| 4  | 5     |
| 5  | 3     |
| 6  | 3     |
| 7  | 5     |
| 8  | 9     |
| 9  | 4     |
| 10 | 1     |

It is sometimes helpful to draw the elements modulo  $n$  on a circle, using the vertices of an  $n$ -gon to represent the field elements. For instance, for the command

#### Example 351

```
draw_mod_13:
> $(ORBITER) -v 2 \
> > -draw_options -embedded -end \
> > -draw_mod_n \
> > > -n 13 \
> > > -file mod_13 \
> > > -power_cycle 2 \
> > -end
> pdflatex mod_13_draw.tex
> $(OPEN) mod_13_draw.pdf
```

uses a 13-gon to represent the elements modulo 13 as shown below



Consecutive powers of 2 mod 13 are connected.

The command

**Example 352**

```
inverse_mod_a:
> $(ORBITER) -v 2 -inverse_mod 18059241 58014043
```

computes the inverse of 18059241 modulo 58014043.

Orbiter can compute the Smith-Normal-Form of matrices with integer entries. The command

**Example 353**

```
smith_normal_form_1:
> $(ORBITER) -v 3 \
> > -define M -vector \
> > > -dense "1,2,3,4,5,6,7,8,9" \
> > > -format 3 \
> > -end \
> > -smith_normal_form M
```

computes the Smith-Normal-Form of the matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

The Legendre symbol evaluates if a number  $a$  is a square modulo an odd prime  $p$ . By definition,

$$\left(\frac{a}{p}\right) = \begin{cases} 1 & \text{if there exists } r \text{ s.t. } r^2 \equiv a \pmod{p} \\ -1 & \text{if there does not exist } r \text{ s.t. } r^2 \equiv a \pmod{p} \\ 0 & \text{if } p \text{ divides } a. \end{cases}$$

The Jacobi symbol generalizes the Legendre symbol to allow non-prime bottom arguments. By definition,

$$\left(\frac{a}{b}\right) = \prod_{i=1}^k \left(\frac{a}{r_i}\right)^{e_i},$$

where

$$b = \prod_{i=1}^k r_i^{e_i}$$

is the prime factorization of  $b$  with pairwise distinct primes  $r_i$ . The Jacobi symbol agrees with the Legendre symbol whenever the bottom argument  $b$  is an odd prime. Because there is no ambiguity, the same notation is used for the Jacobi symbol as for the Legendre symbol. Orbiter can compute Jacobi symbols. For instance, the command

**Example 354**

```
jacobi_a:
> $(ORBITER) -v 5 -jacobi 2221 7817
```

computes the Jacobi symbol

$$\left(\frac{2221}{7817}\right).$$

In the Jacobi symbol, the denominator  $p$  has to be a positive odd integer. This command creates the file `jacobi_2221_7817.tex` which contains a detailed step-by-step description of the computation. After reformatting, the description looks like this:

$$\begin{aligned}
& \left(\frac{2221}{7817}\right) \\
&= \left(\frac{7817}{2221}\right) \cdot (-1)^{\frac{2221-1}{2} \cdot \frac{7817-1}{2}} \\
&= \left(\frac{7817}{2221}\right) \\
&= \left(\frac{1154}{2221}\right) \\
&= \left(\frac{2}{2221}\right) \cdot \left(\frac{577}{2221}\right) \\
&= (-1)^{\frac{2221^2-1}{8}} \cdot \left(\frac{577}{2221}\right) \\
&= (-1) \cdot \left(\frac{577}{2221}\right) \\
&= (-1) \cdot \left(\frac{2221}{577}\right) \cdot (-1)^{\frac{577-1}{2} \cdot \frac{2221-1}{2}} \\
&= (-1) \cdot \left(\frac{2221}{577}\right) \\
&= (-1) \cdot \left(\frac{490}{577}\right) \\
&= (-1) \cdot \left(\frac{2}{577}\right) \cdot \left(\frac{245}{577}\right) \\
&= (-1) \cdot (-1)^{\frac{577^2-1}{8}} \cdot \left(\frac{245}{577}\right) \\
&= (-1) \cdot \left(\frac{245}{577}\right) \\
&= (-1) \cdot \left(\frac{577}{245}\right) \cdot (-1)^{\frac{245-1}{2} \cdot \frac{577-1}{2}} \\
&= (-1) \cdot \left(\frac{577}{245}\right) \\
&= (-1) \cdot \left(\frac{87}{245}\right) \\
&= (-1) \cdot \left(\frac{245}{87}\right) \cdot (-1)^{\frac{87-1}{2} \cdot \frac{245-1}{2}} \\
&= (-1) \cdot \left(\frac{245}{87}\right) \\
&= (-1) \cdot \left(\frac{71}{87}\right) \\
&= (-1) \cdot \left(\frac{87}{71}\right) \cdot (-1)^{\frac{71-1}{2} \cdot \frac{87-1}{2}} \\
&= \left(\frac{87}{71}\right) \\
&= \left(\frac{16}{71}\right) \\
&= \left(\frac{2}{71}\right)^4 \cdot \left(\frac{1}{71}\right) \\
&= \left((-1)^{\frac{71^2-1}{8}}\right)^4 \cdot \left(\frac{1}{71}\right) \\
&= \left(\frac{1}{71}\right) \\
&= 1
\end{aligned}$$

The answer 1 tells us that 2221 is a square modulo 7817. Because 7817 is prime, the Jacobi symbol and the Legendre symbol agree on this input pair. We can use the `square_root_mod` command to compute a square root of 2221 modulo 7817 and verify this fact. The command



**Example 355**

```
sqrt_mod_7817:
▷ ▷ $(ORBITER) -v 2 -square_root_mod 2221 7817
```

yields that 7634 is a square root. Indeed,

$$7634^2 \equiv 2221 \pmod{7817}.$$

The next command illustrates how to solve a system of congruences with coprime moduli using the Chinese remainder theorem. Suppose we want to find the integer  $x$  such that

$$\begin{aligned} x &\equiv 2 \pmod{5} \\ x &\equiv 2 \pmod{6} \\ x &\equiv 5 \pmod{7} \end{aligned}$$

The following command creates one vector for the remainders and one for the moduli and then invokes the `-Chinese_remainders` command.

**Example 356**

```
Chinese_remainders_A:
▷ $(ORBITER) -v 2 \
▷ ▷ -define R -vector -dense "2,2,5" -end \
▷ ▷ -define M -vector -dense "5,6,7" -end \
▷ ▷ -Chinese_remainders R M
```

The answer is  $x \equiv 152$  modulo 210.

The next example shows that the Chinese remainder algorithm is safe for large numbers. We pick two 10 digit prime numbers as moduli and solve

$$\begin{aligned} x &\equiv 2 \pmod{2147483647} \\ x &\equiv 3 \pmod{5915587277} \end{aligned}$$

using the command

**Example 357**

```
Chinese_remainders_C2:
▷ $(ORBITER) -v 2 \
▷ ▷ -define R -vector -dense "2,3" -end \
▷ ▷ -define M -vector -dense "2147483647,5915587277" -end \
▷ ▷ -Chinese_remainders R M
```

The answer is

$$x \equiv 5684294357108828365 \pmod{12703626939758759219}.$$

A quick check with Maple shows that

$$\begin{aligned}5684294357108828365 \bmod 2147483647 &\equiv 2 \\5684294357108828365 \bmod 5915587277 &\equiv 3\end{aligned}$$

as required.

| Representation Theory Activities            |                |                                                                                                                                                                          |
|---------------------------------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                     | Arguments      | Purpose                                                                                                                                                                  |
| <code>-representation_on_polynomials</code> | <code>R</code> | Computes the representation of the given group on homogeneous polynomials from the ring <code>R</code> . This is a group theoretic activity as described in Section 6.5. |

Table 9.4: Representation Theory Activities

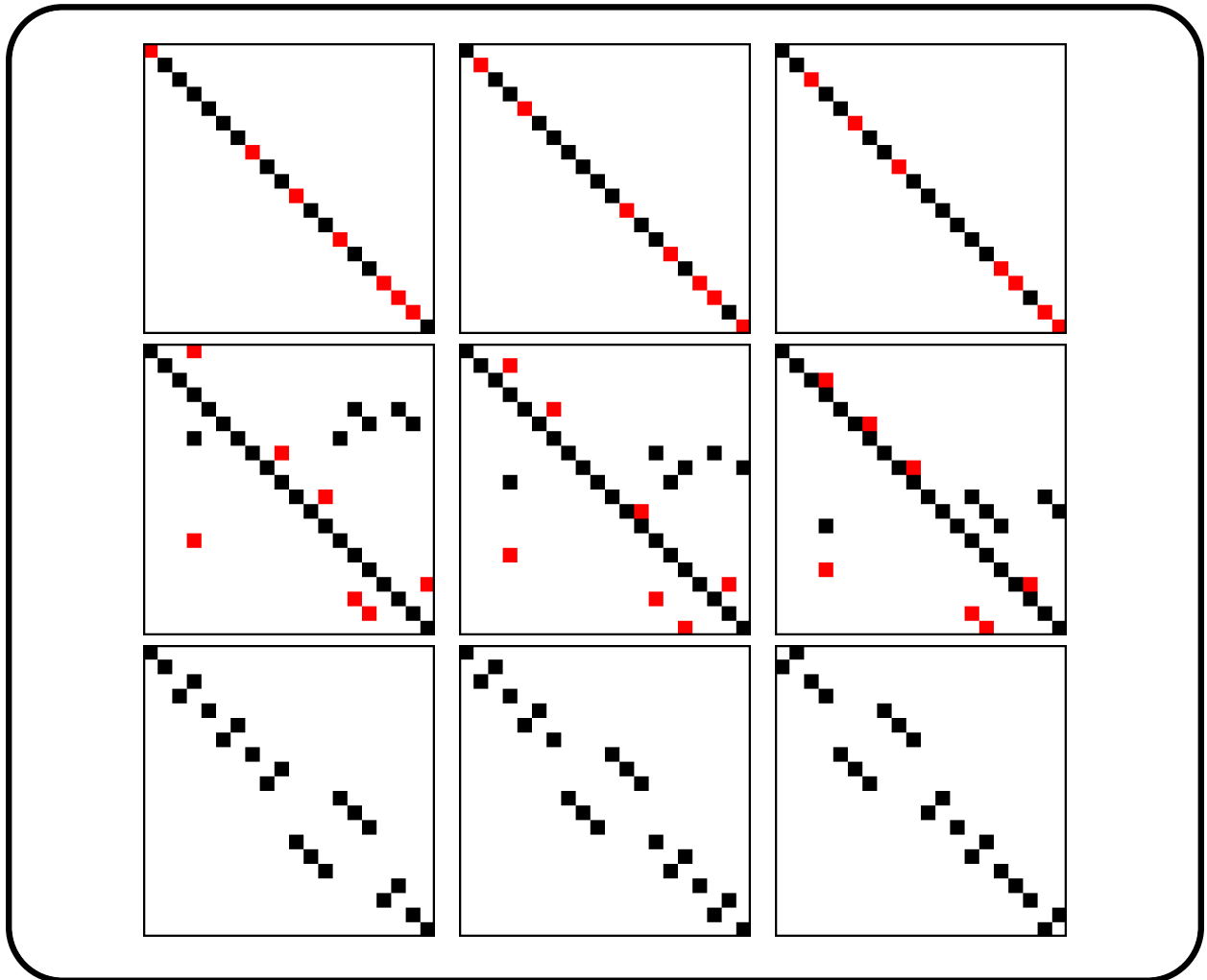
## 9.2 Representation Theory

Table 9.4 list the Orbiter commands for group representations. The command

### Example 358

```
representation_on_polynomials_of_degree_3:
▷ $(ORBITER) -v 4 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -define R -polynomial_ring \
▷ ▷ ▷ -field F \
▷ ▷ ▷ -number_of_variables 4 \
▷ ▷ ▷ -homogeneous_of_degree 3 \
▷ ▷ ▷ -monomial_ordering_partition \
▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -representation_on_polynomials R \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -loop L 0 9 1 -draw_matrix \
▷ ▷ ▷ -input_csv_file PGL_4_3_rep_3.%L.csv \
▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 20 20 -end \
▷ ▷ -end_loop L
```

creates  $G = \text{PGL}(4, 3)$  and computes the representation on polynomials of degree 3 in 4 variables. The representation has degree 20. The second command produces bitmap drawings for the representing matrices associated with a generating set of the group.



| Cryptographic Commands   |                          |                                                                                                                  |
|--------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------|
| Command                  | Arguments                | Purpose                                                                                                          |
| -solovay_strassen        | $a$ $n$                  | Performs $n$ Solovay / Strassen tests on the number $a$                                                          |
| -miller_rabin            | $a$ $n$                  | Performs $n$ Miller / Rabin tests on the number $a$                                                              |
| -fermat                  | $a$ $n$                  | Performs $n$ Fermat tests on the number $a$                                                                      |
| -find_pseudoprime        | $a$ $n_1$ $n_2$ $n_3$    | Computes a pseudoprime which survives $n_1$ Fermat tests, $n_2$ Miller Rabin tests, $n_3$ Solovay Strassen tests |
| -find_strong_pseudoprime | $a$ $n_1$ $n_2$          | Computes a pseudoprime which survives $n_1$ Fermat tests and $n_2$ Miller Rabin tests                            |
| -RSA_encrypt_text        | $d$ $n$ $b$ text         | Using blocks of $b$ letters at a time, encrypt "text" using RSA with exponent $d$ modulo $n$                     |
| -RSA                     | $d$ $n$ list-of-integers | encrypt the given sequence of integers using RSA with exponent $d$ modulo $n$                                    |

Table 9.5: Cryptographic Commands

## 9.3 Cryptography

In Table 9.5, some global cryptographic commands are shown. Some of these commands require a finite field and appear as a finite field activity, see Table 9.6. For instance,

### Example 359

```
EC_add:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 11 -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -EC_add 1 3 "1,4" "1,4" -end
```

adds the point  $(1, 4)$  on the curve  $y^2 = x^3 + x + 3 \pmod{11}$  to itself. The command

### Example 360

```
EC_cyclic_subgroup:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 11 -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -EC_cyclic_subgroup 1 3 "1,4" -end
```

computes the cyclic subgroup generated by the point  $(1, 4)$  on the curve  $y^2 = x^3 + x + 3 \pmod{11}$ . The command

| Finite Field Activities for Cryptography |                                        |                                                                                                                                                                                |
|------------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                  | Arguments                              | Purpose                                                                                                                                                                        |
| -EC_add                                  | $a \ b \ i_1 \ i_2$                    | On the elliptic curve $y^2 \equiv x^3 + ax + b$ in $\mathbb{F}_q$ , add the points with indices $i_1$ and $i_2$ , each given as a pair $x, y$                                  |
| -EC_points                               | $a \ b$                                | Computes all points of the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$                                                                                        |
| -EC_multiple_of                          | $a \ b \ pt \ n$                       | Computes the $n$ fold multiple of the given point $pt$ on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$                                                     |
| -EC_cyclic_subgroup                      | $a \ b \ pt$                           | Computes the cyclic subgroup generated by the given point $pt$ on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$                                             |
| -EC_Koblitz_encoding                     | $a \ b \ s \ pt \ plain$               | Computes the Koblitz encoding of "plain" (all caps) on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$ using the base point $pt$ and the secret exponent $s$  |
| -EC_bsgs                                 | $a \ b \ pt \ n \ cipher$              | Prepare the baby-step giant-step tables for the ciphertext "cipher" on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$ using the base point $pt$ of order $n$ |
| -EC_bsgs_decode                          | $a \ b \ pt \ n \ cipher \ round-keys$ | Decodes the ciphertext "cipher" on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$ using the base point $pt$ of order $n$ and the round keys "keys"           |
| -EC_discrete_log                         | $a \ b \ pt \ base-pt$                 | Computes the elliptic curve discrete log analogue of $pt$ with respect to $base-pt$ on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$                        |
| -NTRU_encrypt                            | $N \ p \ H \ R \ M$                    | NTRU encryption for the message $M(X)$ using the public key $H(X)$ and one-time-key $R(X)$ .                                                                                   |
| -polynomial_center_lift                  | $A(X)$                                 | Compute the center lift mod $q$ for the coefficients of $A$                                                                                                                    |
| -polynomial_reduce_mod_p                 | $p \ A(X)$                             | Reduce the coefficients of the polynomial $A$ modulo $p$                                                                                                                       |

Table 9.6: Finite Field Activities for Cryptography

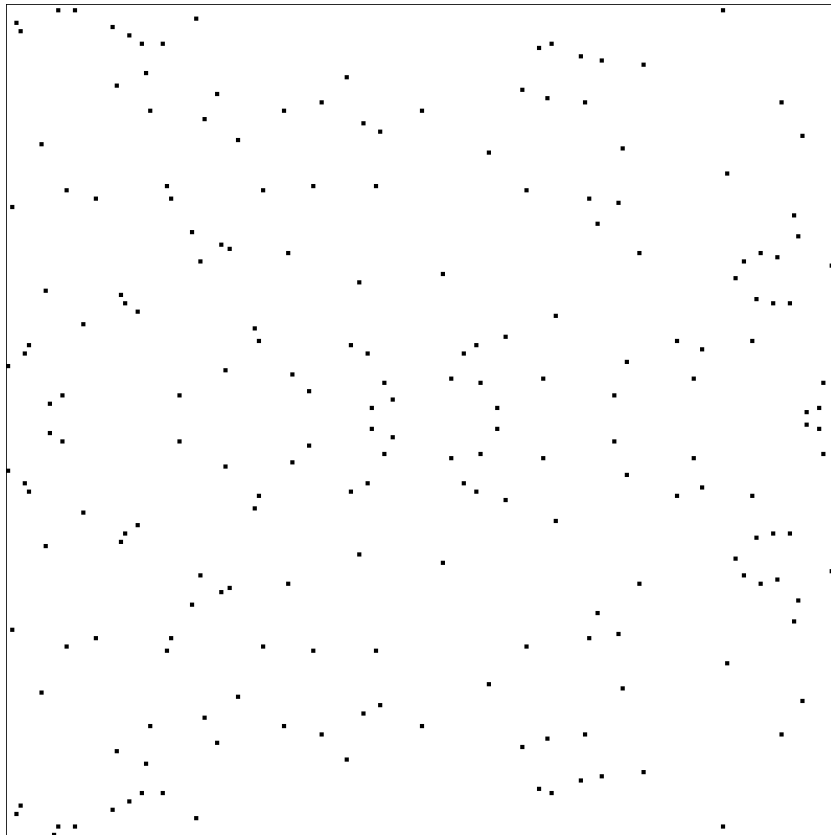
**Example 361**

```

EC_points_199:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 199 -end \
> > -with F -do \
> > -finite_field_activity \
> > -EC_points "EC_5_7_q199" 5 7 -end
> $(ORBITER) -v 2 \
> > -draw_matrix -input_csv_file EC_5_7_q199_points.xy.csv \
> > -box_width 10 -bit_depth 24 \
> > -partition 2 199 199 -end

```

computes all points on the curve  $y^2 = x^3 + 5x + 7 \pmod{199}$  and produces a bitmap drawing of the points in the affine plane:



Both the  $x$ -axis and the  $y$ -axis are indexed by the field elements from 0 to 198.

The command

**Example 362**

```

EC_Koblitz_encoding:
> $(ORBITER) -v 6 -seed 17 \

```

```

> > -define F -finite_field -q 199 -end \
> > -with F -do \
> > -finite_field_activity \
> > -EC_Koblitz_encoding 5 7 67 "147,164" "DEADBEEF" \
> > -end

```

encode the message “DEADBEEF” on the curve  $y^2 = x^3 + 5x + 7 \pmod{199}$  using the base point  $(147, 164)$  and the secret key 67. The  $i$ th input character is encoded as two points  $(R_i, T_i)$  on the curve using the Elgamal scheme. A random round key is generated for each plaintext symbol. As seen in this example, the `-seed` command can be used to seed the random number generator with an arbitrary integer (here 17).

The command

#### Example 363

```

EC_bsgs:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 199 -end \
> > -with F -do \
> > -finite_field_activity \
> > -EC_bsgs 5 7 "147,164" 212 \
> > "172,158,45,195,50,22,10,103,55,33,50,22,\
> > 145,105,31,74,73,155,67,60,25,6" \
> > -end

```

performs a baby-step-giant-step brute force attack on the ciphertext sequence

$$R_i = (172, 158), (45, 195), (50, 22), (10, 103), (55, 33), \\ (50, 22), (145, 105), (31, 74), (73, 155), (67, 60), (25, 6),$$

using the base point  $(147, 164)$  on the curve  $y^2 = x^3 + 5x + 7 \pmod{199}$ , assuming a group order of 212. The command

#### Example 364

```

EC_bsgs_decode:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 199 -end \
> > -with F -do \
> > -finite_field_activity \
> > -EC_bsgs_decode 5 7 "129,176" 212 \
> > "127,188,51,141,85,29,106,90,41,105,179,71,\
> > 171,2,16,197,183,72,27,129,37,10" \
> > "50,179,169,13,153,169,115,116,188,110,176" \
> > -end

```

decodes the ciphertext sequence

$$T_i = (127, 188), (51, 141), (85, 29), (106, 90), (41, 105), (179, 71), \\ (171, 2), (16, 197), (183, 72), (27, 129), (37, 10),$$



assuming round keys

$$k_i = 50, 179, 169, 13, 153, 169, 115, 116, 188, 110, 176,$$

using the base point  $(147, 164)$  on the curve  $y^2 = x^3 + 5x + 7 \pmod{199}$ , and assuming a group order of 212.

The next sequence of examples discusses the NTRU cryptosystem (cf. Example 7.53 in [35]). In the example, we choose the parameters of the cryptosystem to be  $(N, p, q, d) = (7, 41, 3, 2)$ . Orbiter uses the following convention for polynomials over a finite field  $\mathbb{F}_q$ : The coefficients of  $A(X) = a_0 + a_1X + \cdots + a_dX^d$  are listed as a sequence, starting with the constant term and ending with the leading coefficient. The cryptosystem requires coefficients  $a_i$  in the range  $-\frac{p}{2} \leq a_i \leq \frac{p}{2}$ . So, in an extension to the conventions for field elements in  $\mathbb{F}_q$ , Orbiter allows negative coefficients as well. The assumption is that  $q$  is prime and negative coefficients are considered modulo  $q$ . In the example, Alice picks the private polynomials  $f(x) = x^6 - x^4 + x^3 + x^2 - 1$  (with  $d+1$  coefficients equal to plus one and  $d$  coefficients equal to minus one) and  $g(x) = x^6 + x^4 - x^2 - x$  with  $d$  coefficients plus one and  $d$  coefficients minus one. We also need the polynomial  $x^N - 1$ . The makefile commands

Example 365

NTRU\_N=7

Example 366

NTRU\_P=3

Example 367

NTRU\_Q=41

Example 368

NTRU\_D=2

Example 369

NTRU\_XN1="-1,0,0,0,0,0,0,1,"

Example 370

ALICE\_PRIVATE\_F="-1,0,1,1,-1,0,1"

Example 371

ALICE\_PRIVATE\_G="0,-1,-1,0,1,0,1"

are used to set up the appropriate variables according to these choices.

Regarding the NTRU set-up, Alice needs to compute her private keys  $F_p(x)$  and  $F_q(x)$ . These two polynomials are defined as follows:

1.  $F_p(x)$  is the inverse of  $f(x)$  in  $\mathbb{F}_p[x]/(x^n - 1)$ ,
2.  $F_q(x)$  the inverse of  $f(x)$  in  $\mathbb{F}_q[x]/(x^n - 1)$ .

To this end, we can use the `extended_gcd_for_polynomials` command from Table 9.3. The following two makefile commands do the job:

#### Example 372

```
NTRU_Alice1:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -extended_gcd_for_polynomials \
▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
▷ ▷ -end
```

#### Example 373

```
ALICE_PRIVATE_FQ="37,2,40,21,31,26,8"
```

#### Example 374

```
NTRU_Alice2:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -extended_gcd_for_polynomials \
▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
▷ ▷ -end
```

The resulting polynomials (indicated as comments by means of the `#` symbol) are again encoded as makefile variables.

#### Example 375

```
ALICE_PRIVATE_FP="1,1,1,1,0,2,1"
```

There is a chance that the polynomial  $f(x)$  does not have an inverse in either  $\mathbb{F}_p[x]$  or in  $\mathbb{F}_q[x]$ . In that case, Alice simply chooses a different polynomial  $f(x)$  and tries again. Alice can now compute her public key:

**Example 376**

```

NTRU_Alice_public_key:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
▷ ▷ ▷ $(ALICE_PRIVATE_G) $(NTRU_XN1) \
▷ ▷ -end

```

**Example 377**

```

ALICE_PUBLIC_KEY="30,26,8,38,2,40,20"

```

The public key is assigned to the makefile variable `ALICE_PUBLIC_KEY`. Now, Bob chooses his message to Alice and his one-time-key. The message must be the center lift of a polynomial in  $\mathbb{F}_p[x]$ . The round-key must have exactly  $d$  coefficients one and  $d$  coefficients  $-1$  (rest zeroes).

**Example 378**

```

BOB_MESSAGE="1,-1,1,1,0,-1"

```

**Example 379**

```

BOB_ONE_TIME_KEY="-1,1,0,0,0,-1,1"

```

The encryption proceeds using the `NTRU_encrypt` command, and the result is stored in the makefile variable `BOB_ENCRYPT`:

**Example 380**

```

NTRU_encrypt:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -NTRU_encrypt \
▷ ▷ $(NTRU_N) $(NTRU_P) $(ALICE_PUBLIC_KEY) \
▷ ▷ $(BOB_ONE_TIME_KEY) $(BOB_MESSAGE) \
▷ ▷ -end

```

**Example 381**

```

BOB_ENCRYPT="25,3,40,2,4,19,31"

```

Decryption is done in five steps.

#### Example 382

```
NTRU_decrypt1:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
▷ ▷ ▷ $(BOB_ENCRYPT) $(NTRU_XN1) \
▷ ▷ -end
```

#### Example 383

```
ALICE_C1="40,1,40,40,33,10,1"
```

#### Example 384

```
NTRU_decrypt2:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -polynomial_center_lift $(ALICE_C1) -end
```

#### Example 385

```
ALICE_C2="-1,1,-1,-1,-8,10,1"
```

#### Example 386

```
NTRU_decrypt3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -polynomial_reduce_mod_p $(ALICE_C2) -end
```

#### Example 387

```
ALICE_C3="2,1,2,2,1,1,1"
```

**Example 388**

```
NTRU_decrypt4:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q $(NTRU_Q) -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_mult_mod $(ALICE_PRIVATE_FP) \
> > > $(ALICE_C3) $(NTRU_XN1) \
> > -end
```

**Example 389**

```
ALICE_C4="1,2,1,1,0,2"
```

**Example 390**

```
NTRU_decrypt5:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q $(NTRU_P) -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_center_lift $(ALICE_C4) -end
```

Decryption produces Bob's message to Alice.



# Chapter 10

## Coding Theory

### 10.1 Introduction

Table 10.1 lists global Orbiter commands related to coding theory.

The command

#### Example 391

```
Allen.Gates_noise_1_percent:  
▷ cp $(MY_PATH)/examples/users_guide/allen.Gates.bmp .  
▷ $(ORBITER) -v 3 \  
▷ ▷ -random_noise_in_bitmap_file \  
▷ ▷ allen.Gates.bmp \  
▷ ▷ allen.Gates_1.bmp \  
▷ ▷ 1 100  
▷ $(OPEN) allen.Gates_1.bmp
```

simulates random noise at the 1 percent level applied to the file `allen_Gates.bmp`, see below

| Global Coding Theoretic Commands           |                           |                                                                                                                                |
|--------------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Command                                    | Arguments                 | Purpose                                                                                                                        |
| -make_macwilliams_system                   | $q\ n\ k$                 | Create the MacWilliams equations for the weight enumerator of the dual code.                                                   |
| -table_of_bounds                           | $n_{\max}\ q$             | Make a table of bounds for $q$ -ary linear codes for all $k \leq n \leq n_{\max}$                                              |
| -make_bounds_for_d_given_n_and_k_and_q     | $n\ k\ q$                 | Make bounds for the minimum distance of an $[n, k]_q$ code                                                                     |
| -introduce_errors                          | CRC-options               | Introduce errors to a file. See Table 10.8.                                                                                    |
| -check_errors                              | CRC-options               | Find errors in a CRC coded file. See Table 10.8.                                                                               |
| -extract_block                             | CRC-options               | Extract a block from a CRC coded file. See Table 10.8.                                                                         |
| -random_noise_in_bitmap_file               | input output<br>$n\ d$    | Apply random noise at the $d/n$ level to the bitmap file "input" and write to "output."                                        |
| -random_noise_of_burst_type_in_bitmap_file | input output<br>$n\ d\ b$ | Apply random noise of type burst with max burst length $b$ at the $d/n$ level to the bitmap file "input" and write to "output" |

Table 10.1: Global Coding Theoretic Commands





The original is on the top. The effect of noise can be seen in the picture below. The picture shows Paul Allen and Bill Gates in the early 1970s.

The command

#### Example 392

```
Hamming_space_4_2_distance_matrix:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -with F -do -coding_theoretic_activity \
▷ ▷ ▷ -Hamming_space_distance_matrix 4 \
▷ ▷ -end
```

creates the distance matrix of the Hamming graph  $H(n, q)$ . The data is written to the file `Hamming_n4_q2.csv`. The command

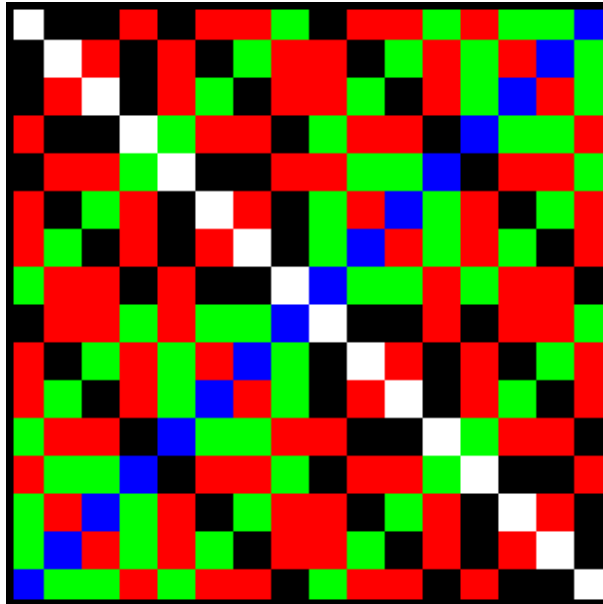
**Example 393**

```

Hamming_space_4_2_distance_matrix.draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file Hamming_n4_q2.csv \
▷ ▷ ▷ -box_width 20 -bit_depth 24 \
▷ ▷ ▷ -partition 4 16 16 \
▷ ▷ -end
▷ $(OPEN) Hamming_n4_q2_draw.bmp

```

produces the bitmap graphic Hamming\_n4\_q2\_draw.bmp shown below:



The command

**Example 394**

```

Hamming_code_macwilliams:
▷ $(ORBITER) -v 2 \
▷ ▷ -make_macwilliams_system 7 4 2
▷ pdflatex MacWilliams_n7_k4_q2.tex
▷ $(OPEN) MacWilliams_n7_k4_q2.pdf

```

creates the coefficient matrix of the MacWilliams system for the  $[7, 4, 2]$  Hamming code:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 7 & 5 & 3 & 1 & -1 & -3 & -5 & -7 \\ 21 & 9 & 1 & -3 & -3 & 1 & 9 & 21 \\ 35 & 5 & -5 & -3 & 3 & 5 & -5 & -35 \\ 35 & -5 & -5 & 3 & 3 & -5 & -5 & 35 \\ 21 & -9 & 1 & 3 & -3 & -1 & 9 & -21 \\ 7 & -5 & 3 & -1 & -1 & 3 & -5 & 7 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix}$$

For examples concerning the bounds, see Section 10.9.

Consider the  $[5, 2]_2$  code whose codewords are  $\{0, 7, 25, 30\}$ . We start with a makefile variable:

#### Example 395

```
CODE_5_2_3_CODEWORDS="0,7,25,30"
```

The following command creates the code and draws the code diagram in Hamming space:

#### Example 396

```
code_5_2_3_diagram:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -with F -do -coding_theoretic_activity \
▷ ▷ ▷ -general_code_binary 5 "code_5_2_3" \
▷ ▷ ▷ $(CODE_5_2_3_CODEWORDS) \
▷ ▷ ▷ -metric_balls 1 \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file code_5_2_3_char_func_5.4.csv \
▷ ▷ ▷ -box_width 25 -bit_depth 24 \
▷ ▷ ▷ -partition 4 8 4 \
▷ ▷ -end
```

The Hamming graph  $H(5, 2)$  can be created with the following command:

#### Example 397

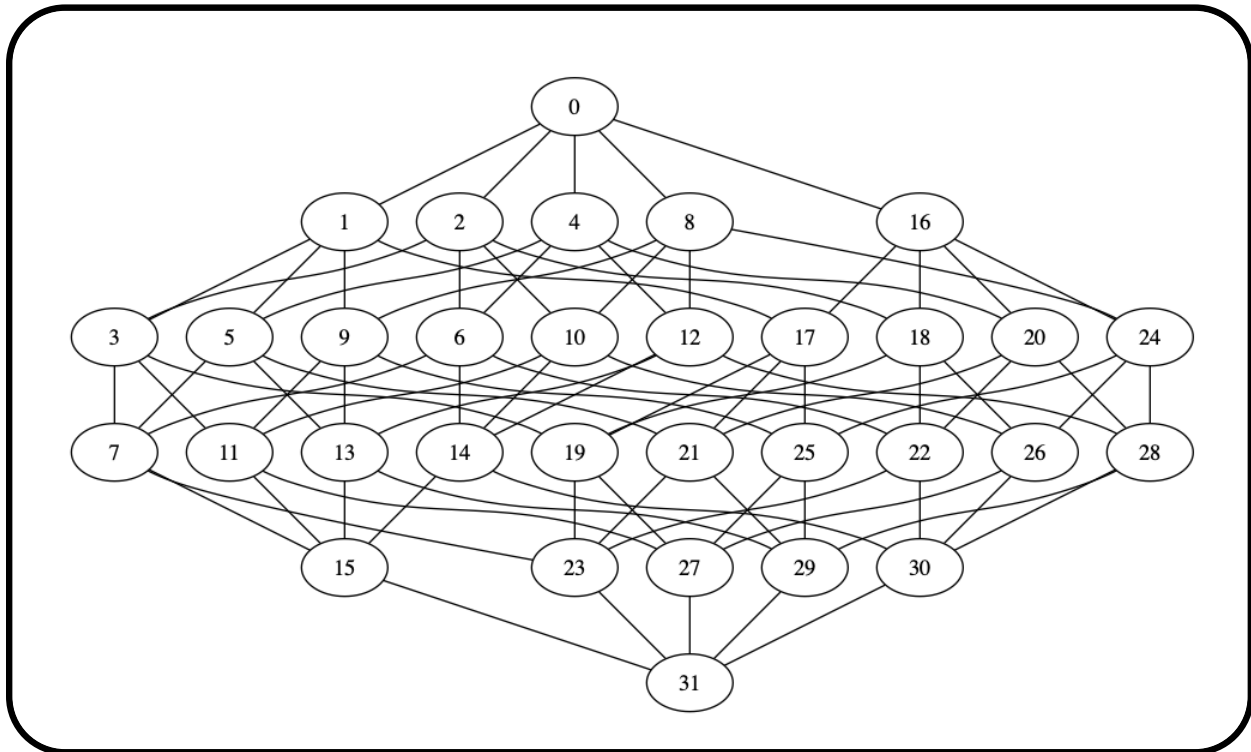
```
Hamming_5_2_graph:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -graph -Hamming 5 2 -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity -export_csv -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity -export_graphviz -end \
```

```

> > -with G -do \
> > -graph_theoretic_activity -save -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file Hamming_5.2.csv \
> > -box_width 8 -bit_depth 24 -partition 4 32 32 -end
> dot -Tpng Hamming_5.2.gv >Hamming_5.2.png

```

Using the unix dot program, this command sequence creates the drawing of  $H(5,2)$



The command

#### Example 398

```

Sylvester_Hadamard_code_8:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 3 -end \
> > -with F -do -coding_theoretic_activity \
> > > -Sylvester_Hadamard_code 8 \
> > -end

```

creates the code arising from a Sylvester-type Hadamard matrix of order 8.

## 10.2 Linear Codes

Table 10.2 lists Orbiter commands to create codes. Tables 10.3-10.5 list coding theoretic activities in Orbiter. Table 10.6 shows ways to create new codes from old.

The following command creates the first order Reed-Muller code in three variables:

### Example 399

```
RM_3_1:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -Reed_Muller 3 \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma RM_3_1.magma \
▷ ▷ -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex RM_3_code_n8_k4_q2.tex
▷ $(OPEN) RM_3_code_n8_k4_q2.pdf
```

Consider the linear code with generator matrix

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}.$$

The row vectors correspond to the integers 60, 50, 41 in binary. We can use the following command to create the code:

### Example 400

```
code_6:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -basis 6 "60,50,41" \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -make_diagram \
▷ ▷ -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file by_basis_n6_k3_char_func_6_8.csv \
▷ ▷ -box_width 20 -bit_depth 24 \
▷ ▷ -partition 2 "1,1,1,1,1,1,1" "1,1,1,1,1,1,1" \
▷ ▷ -end
```

| Creating Codes               |           |                                                                                                                                                                                                                                                                       |
|------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                      | Arguments | Purpose                                                                                                                                                                                                                                                               |
| -field                       | F         | The field $F = \mathbb{F}_q$ over which the code is defined.                                                                                                                                                                                                          |
| -generator_matrix            | M         | A generator matrix over F defining the code as Orbiter object.                                                                                                                                                                                                        |
| -basis                       | n basis   | A basis for the code as Orbiter ranks. The rows of the generator matrix are coded as integers using the binary representation.                                                                                                                                        |
| -long_code                   | n genma   | Create a code over $\mathbb{F}_q$ defined by a generator matrix. The rows of the generator matrix are given as a list of strings in "words". Each string describes one row, by listing the columns of the generator matrix which have a one in the corresponding row. |
| -projective_set              | nmk       | A projective set defining the columns of a check matrix of a code. The projective set is in $PG(n - k - 1, q)$ where $nmk = n - k$ is given.                                                                                                                          |
| -columns_of_generator_matrix | k cols    | The columns of a check matrix defining the code are given, using the ranking of elements of $AG(k, q)$ .                                                                                                                                                              |
| -Reed_Muller                 | m         | The first order Reed Muller code with $m$ variables.                                                                                                                                                                                                                  |
| -BCH                         | n d       | BCH-code over $\mathbb{F}_q$ of length $n$ with designed distance $d$ . The minimum distance will be at least the designed distance.                                                                                                                                  |
| -Reed_Solomon                | n d       | A Reed-Solomon code of length $n$ with designed distance $d$ . This requires that $n$ divides $q - 1$ .                                                                                                                                                               |
| -Gilbert_Varshamov           | n k d     | A Gilbert-Varshamov code with length $n$ , dimension $k$ and minimum distance $d$ .                                                                                                                                                                                   |
| -ttpA                        | FQ        | Twisted tensor product code of type A using the extension field $FQ$ . The code arises from a hyperoval in $\mathbb{F}_{q^2}$ (Theorem 1 in [8])                                                                                                                      |
| -ttpB                        | FQ        | Twisted tensor product code of type B using the extension field $FQ$ . The code arises from the projective line over $\mathbb{F}_{q^3}$ (Theorem 2 in [8]).                                                                                                           |
| -dual                        |           | Create the dual code instead.                                                                                                                                                                                                                                         |

Table 10.2: Creating Codes

| Coding Theoretic Activities (Part 1) |                                  |                                                                                                                                                                                                                                                                                   |
|--------------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                              | Arguments                        | Purpose                                                                                                                                                                                                                                                                           |
| -report                              |                                  | Produce a latex report.                                                                                                                                                                                                                                                           |
| -general_code_binary                 | $n$ label code-words             | Produces a plot of the codewords in the Hamming space. The code is given by a set of codewords in binary. The code has length $n$ . Produces several csv files. If -metric_balls $r$ is given, the metric balls of radius $r$ centered around codewords are included in the plot. |
| -encode_text_5bits                   | input fname                      |                                                                                                                                                                                                                                                                                   |
| -field_induction                     | fname-in<br>fname-out<br>nb-bits |                                                                                                                                                                                                                                                                                   |
| -weight_enumerator                   | matrix                           | Compute the complete weight enumerator of the code $C$ .                                                                                                                                                                                                                          |
| -minimum_distance                    | code-object-label                | Compute the minimum distance of the linear code object.                                                                                                                                                                                                                           |
| -generator_matrix_cyclic_code        | $n$ poly                         |                                                                                                                                                                                                                                                                                   |
| -Sylvester_Hadamard_code             | $n$                              | Create the codewords of a Sylvester type Hadamard matrix of order $n$ . Here, $n$ must be a power of 2.                                                                                                                                                                           |
| -fixed_code                          | perm                             | Create the subcode which is fix under the group generated by perm, which acts on the generator matrix by column permutations.                                                                                                                                                     |
| -export_magma                        | fname                            |                                                                                                                                                                                                                                                                                   |
| -export_codewords                    | fname                            | Export a list of codewords using coding as binary numbers.                                                                                                                                                                                                                        |
| -export_codewords_long               | fname                            | Export a table of codewords.                                                                                                                                                                                                                                                      |
| -export_codewords_by_weight          | fname                            |                                                                                                                                                                                                                                                                                   |
| -export_genma                        | fname                            |                                                                                                                                                                                                                                                                                   |
| -export_checkma                      | fname                            |                                                                                                                                                                                                                                                                                   |
| -f_export_checkma_as_projective_set  | fname                            |                                                                                                                                                                                                                                                                                   |

Table 10.3: Coding Theoretic Activities (Part 1)

| Coding Theoretic Activities (Part 2) |           |                                                                                                                                                                                                                              |
|--------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                              | Arguments | Purpose                                                                                                                                                                                                                      |
| -make_diagram                        |           | Creates a diagram of the code in Hamming space. Uses -embellish $r$ to emphasize codewords by drawing a circle of radius $r$ . If -metric_balls $r$ is given, the metric balls around codewords of radius $r$ are indicated. |
| -boolean_function_of_code            |           | Computes the boolean function of the code $C$ .                                                                                                                                                                              |
| -embellish                           | $r$       | See -make_diagram.                                                                                                                                                                                                           |
| -metric_balls                        | $r$       | See -make_diagram.                                                                                                                                                                                                           |
| -Hamming_space_distance_matrix       | $n$       | Make the distance matrix of the Hamming graph $H(n, q)$ .                                                                                                                                                                    |

Table 10.4: Coding Theoretic Activities (Part 2)

```

▷ pdflatex by_basis_n6_k3_code_n6_k3.q2.tex
▷ $(OPEN) by_basis_n6_k3_code_n6_k3.q2.pdf
▷ $(OPEN) by_basis_n6_k3_char_func_6_8.draw.bmp

```

The command also draws the code in the Hamming space.

Let us create the Hamming code. The dual of the Hamming code is the simplex code, so we create the simplex code first. The following makefile variable is defined to hold the generator matrix of the simplex code:

#### Example 401

```

SIMPLEX_CODE_GENERATOR="\
1,0,1,0,1,0,1, \
0,1,1,0,0,1,1, \
0,0,0,1,1,1,1"

```

The following command computes the nullspace of this matrix, which is the Hamming code:

#### Example 402

```

simplex_code:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -field F -format 3 \
▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
▷ ▷ -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -generator_matrix v \
▷ ▷ -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -report \

```



| Coding Theoretic Activities (Part 3)                    |                                                                   |         |
|---------------------------------------------------------|-------------------------------------------------------------------|---------|
| Command                                                 | Arguments                                                         | Purpose |
| -crc32                                                  | text                                                              |         |
| -crc32_hexdata                                          | hexdata                                                           |         |
| -crc32_test                                             | block-length                                                      |         |
| -crc256_test                                            | message-length $R$<br>$k$                                         |         |
| -crc32_remainders                                       | msg-length                                                        |         |
| -crc_encode_file_based                                  | fname-in<br>fname-out<br>block-length<br>crc_type<br>block_length |         |
| -crc_compare                                            | fname code1<br>code2 error-weight<br>nb tests-per-block           |         |
| -crc_compare_read_output_file                           | fname nb-lines code1<br>code2                                     |         |
| -all_errors_of_a_given_weight                           | fname block-idx code1<br>code2 max-weight                         |         |
| -weight_enumerator_bottom_up                            | code max-weight                                                   |         |
| -convert_data_to_polynomials                            | fname-in<br>fname-out<br>block-length<br>symbol-size              |         |
| -find_CRC_polynomials                                   | nb-errors<br>info-bits<br>check-bits                              |         |
| -write_code_for_division                                | fname $A$ $B$                                                     |         |
| -polynomial_division_from_file                          | fname $r1$                                                        |         |
| -polynomial_division_from_file_all_k_bit_error_patterns | fname $r1$ $k$                                                    |         |

Table 10.5: Coding Theoretic Activities (Part 3)

| Code Modifications |           |                       |
|--------------------|-----------|-----------------------|
| Command            | Arguments | Purpose               |
| -dual              |           | Create the dual code. |

Table 10.6: Code Modifications

```

> > -end
> pdflatex by_genma_n7_k3_code_n7_k3_q2.tex
> $(OPEN) by_genma_n7_k3_code_n7_k3_q2.pdf

```

The following latex output is produced:

Input matrix:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

RREF:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Basis for Perp:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

It is possible to create the Hamming code by taking the dual of the simplex code. The following command does so:

#### Example 403

Hamming\_code:

```

> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F -format 3 \
> > > -dense $(SIMPLEX_CODE_GENERATOR) \
> > -end \
> > -define C -code -field F \
> > > -generator_matrix v \
> > > -dual -end \
> > -end \
> > -with C -do -coding_theoretic_activity \
> > > -export_magma Hamming.magma \
> > -end

```

The command also exports the code to magma by means of the magma file Hamming.magma, shown below:

```

K<w> := GF(2);
V := VectorSpace(K, 7);
C := LinearCode(sub<V |

```

```
[1,1,1,0,0,0,0],
[1,0,0,1,1,0,0],
[0,1,0,1,0,1,0],
[1,1,0,1,0,0,1]>);
```

The next command creates the first order Reed-Muller code in 3 variables. All codewords are created. The codewords and the generator matrix are exported to files.

#### Example 404

```
RM_3_1_and_codewords:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define C -code -field F \
> > > -Reed_Muller 3 \
> > -end \
> > -with C -and F -do -coding_theoretic_activity \
> > > -export_magma RM_3_1.magma \
> > -end \
> > -with C -and F -do -coding_theoretic_activity \
> > > -export_codewords RM_3_1_codewords.csv \
> > -end \
> > -with C -and F -do -coding_theoretic_activity \
> > > -export_genma RM_3_1_genma.csv \
> > -end
```

We can create the code from scratch, using a generator matrix. To do so, we use a makefile variable:

#### Example 405

```
CODE_RM_3_1_GENMA="\
11111111\
01010101\
00110011\
00001111"
```

The following command creates the Reed-Muller code from its generator matrix directly:

#### Example 406

```
RM_3_1_from_generator_matrix:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define genma -vector -format 8 -field F \
> > > -compact $(CODE_RM_3_1_GENMA) \
> > -end \
> > -define C -code -field F \
> > > -generator_matrix genma \
> > -end
> #pdflatex code_n8_k4_q2.tex
```

```
▷ #$(OPEN) code_n8_k4_q2.pdf
```

Let us look at the Hamming code. The following command creates the  $[7, 4]$  Hamming code.

#### Example 407

```
Hamming_code_by_rows:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -basis 7 v \
▷ ▷ -end
▷ #pdflatex code_n7_k4_q2.tex
▷ #$(OPEN) code_n7_k4_q2.pdf
```

A different way to create the same code is using the `long_code` command:

#### Example 408

```
Hamming_code_long:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -long_code 7 4 \
▷ ▷ ▷ "0,5,6" \
▷ ▷ ▷ "1,4,6" \
▷ ▷ ▷ "2,4,5" \
▷ ▷ ▷ "3,4,5,6" \
▷ ▷ -end
```

The following command creates a diagram for the Hamming code inside the Hamming space.

#### Example 409

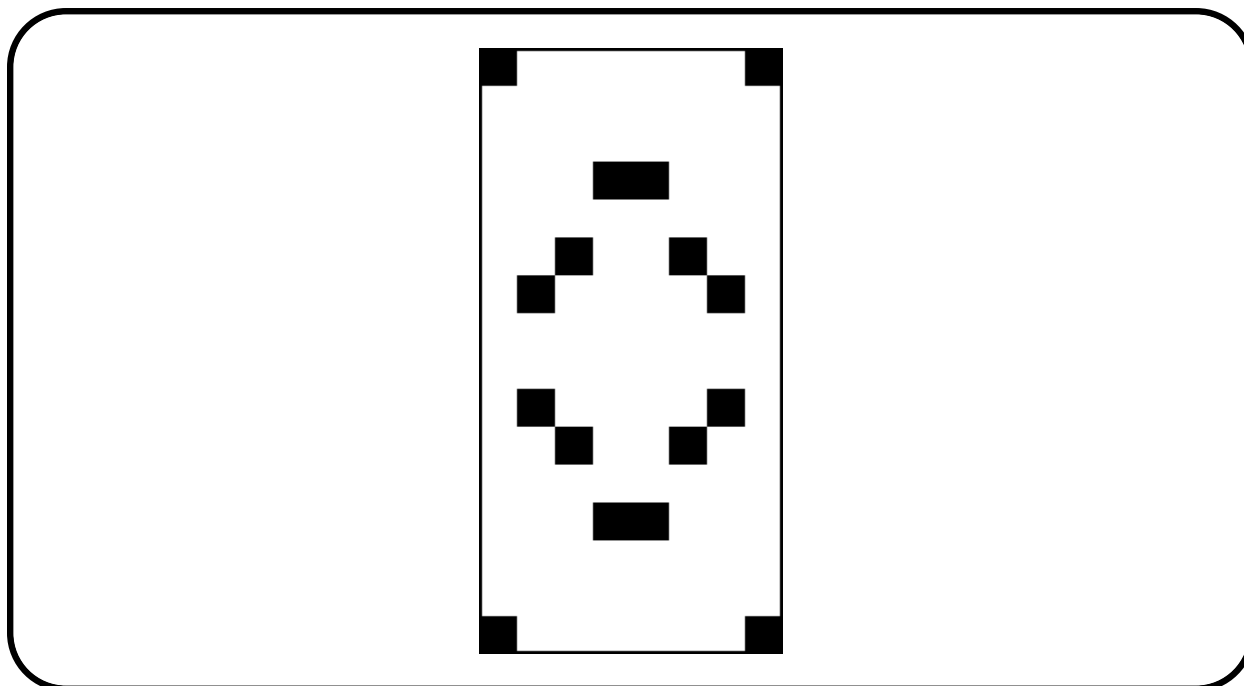
```
Hamming_code_diagram:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -basis 7 v \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -make_diagram \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file by_basis_n7_k4_char_func_7_16.csv \
```

```

▷ ▷ ▷ -box_width 25 -bit_depth 24 \
▷ ▷ ▷ -partition 4 16 8 \
▷ ▷ -end
▷ $(OPEN) by_basis_n7_k4_char_func_7.16_draw.bmp

```

Here is the output:



The next command creates the metric balls of radius one centered at each of the 16 codewords of the Hamming code. At first, we define the numerical values of the codewords of the Hamming code:

#### Example 410

```

HAMMING_CODE_CODEWORDS="0, 67, 37, 102, 22, 85, \
51, 112, 15, 76, 42, 105, 25, 90, 60, 127"

```

Then we create the diagrams of all codewords. We utilize a loop-over command for the codewords and a loop command for the graphical output.

#### Example 411

```

Hamming_code_words:
▷ $(ORBITER) -v 2 \
▷ ▷ -define v -vector -dense $(HAMMING_CODE_CODEWORDS) -end \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -loop_over i v \
▷ ▷ ▷ -with F -do -coding_theoretic_activity \
▷ ▷ ▷ -general_code_binary 7 "Hamming_7.4_word%i" "%i[v]" \
▷ ▷ ▷ -metric_balls 1 \

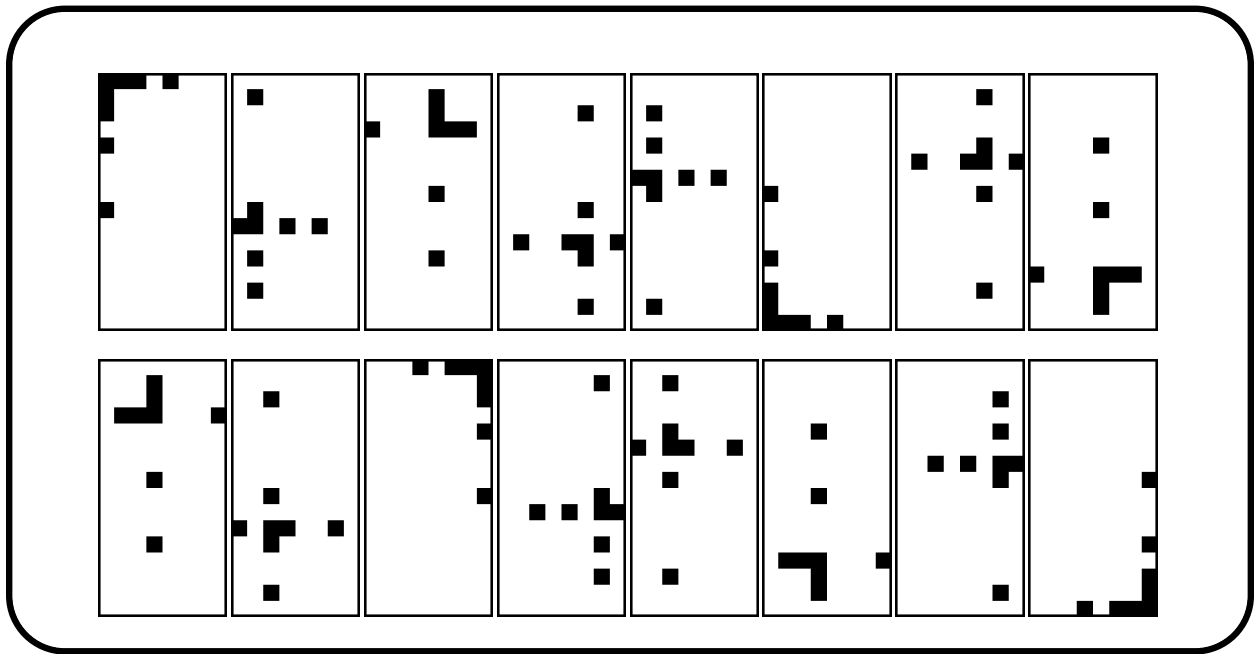
```

```

> > > -end \
> > -end_loop_over i
> $(ORBITER) -v 2 \
> > > -loop i 0 16 1 -draw_matrix \
> > > -input_csv_file Hamming_7_4_word%i_char_func_7_1.csv \
> > > -box_width 25 -bit_depth 8 \
> > > -partition 4 16 8 -end \
> > > -system "convert Hamming_7_4_word%i_char_func_7_1_draw.bmp \
> > > -frame 8 Hamming_7_4_word%i_char_func_7_1_draw.png" \
> > -end_loop i

```

These metric balls form a partition of the Hamming space. The partition can be shown in binary Hamming space:



Orbiter can compute the weight enumerator and the minimum distance of codes. Let us consider the Hamming code, for example. We use a makefile variable for the generator matrix:

#### Example 412

```

HAMMING_CODE_GENERATOR="\
1,0,0,0,0,1,1, \
0,1,0,0,1,0,1, \
0,0,1,0,1,1,0, \
0,0,0,1,1,1,1"

```

The next command computes the weight enumerator:

**Example 413**

```

Hamming_weight_enumerator:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -field F -format 4 \
▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
▷ ▷ -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -generator_matrix v \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -weight_enumerator \
▷ ▷ -end

```

We find that the weight enumerator is

$$(1, 0, 0, 7, 7, 0, 0, 1).$$

The next command computes the minimum distance of the code:

**Example 414**

```

Hamming_minimum_distance:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -field F -format 4 \
▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
▷ ▷ -end \
▷ ▷ -with F -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -minimum_distance v \
▷ ▷ -end

```

The following command computes the minimum distance of the Golay code of length 23:

**Example 415**

```

Golay23_minimum_distance:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -field F -format 12 \
▷ ▷ ▷ -dense $(GOLAY23_CODE_GENERATOR) \
▷ ▷ -end \
▷ ▷ -with F -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -minimum_distance v \
▷ ▷ -end

```

### 10.3 Golay Codes

The Golay code of length 23 is a perfect code of dimension 12 and minimum distance 7. The metric balls of radius three centered around codewords cover the whole Hamming space. We can create the code by listing the columns of a generator matrix in Orbiter ranks of points in  $\text{PG}(11, 2)$ . The following makefile variable does that:

#### Example 416

```
GOLAY_23_COLUMN_RANKS_PROJECTIVELY="0, 1, 2, 3, 4, 5, 6, 7, \
8, 9, 10, 11, 132, 913, 1460, 1750, 1898, 2518, 2787, 2874, \
3320, 3357, 3662"
```

Suppose we want to list the code words. The following command can be used:

#### Example 417

```
Golay23_code_words:
▷ $(ORBITER) -v 2 \
▷ ▷ -define v -vector -dense $(GOLAY_23_COLUMN_RANKS_PROJECTIVELY) -end \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -projective_set 12 v -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma Golay23.magma \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_codewords Golay23.codewords.csv \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_genma Golay23.genma.csv \
▷ ▷ -end
▷ #pdflatex code_n23_k12.q2.tex
▷ #$(OPEN) code_n23_k12.q2.pdf
```



| Defining a CRC-process |           |                                                      |
|------------------------|-----------|------------------------------------------------------|
| Command                | Arguments | Purpose                                              |
| -code                  | label     | Set the a label.                                     |
| -crc_options           | options   | Specify the CRC-code using commands from Table 10.8. |

Table 10.7: Defining a CRC-process

| Options for CRC-Codes        |           |                                                          |
|------------------------------|-----------|----------------------------------------------------------|
| Command                      | Arguments | Purpose                                                  |
| -input                       | fname     | Input file name.                                         |
| -output                      | fname     | Output file name.                                        |
| -crc_type                    | type      | The type of CRC-code.                                    |
| -block_length                | $L$       | Set the block length to $L$ field elements.              |
| -block_based_error_generator |           | Apply block-based error generator.                       |
| -file_based_error_generator  | threshold | Apply file-based error generator.                        |
| -nb_repeats                  | $N$       | Set the number of repeats to $N$ .                       |
| -threshold                   | $t$       | Set probability of error per experiment to $t/1000000$ . |
| -error_log                   | fname     | Set file name for error logging.                         |
| -selected_block              | $i$       | Set block number.                                        |

Table 10.8: Options for CRC-Codes

## 10.4 CRC Codes

A CRC code can be used to detect communication errors. It is a cyclic code, and hence generated by a polynomial over a finite field. The message is encoded as a string, which is then thought of as a polynomial, called the information polynomial. Assume that the check polynomial has degree  $d$ . The information polynomial is then divided by the check polynomial. The remainder is added to the information polynomial multiplied by  $X^d$ . This is the codeword, which is sent.

Table 10.7 lists the commands for defining a CRC-process.

Table 10.8 summarizes options associated with commands for CRC-codes.

Table 10.9 lists the available commands for defining a CRC-code.

The following command performs an exhaustive search over all binary CRC polynomials of degree  $k = 10$  which can detect every error pattern of Hamming weight at most  $t = 3$  in messages of length  $n = 128$ .

| Defining a CRC-Code |           |                                           |
|---------------------|-----------|-------------------------------------------|
| Command             | Arguments | Purpose                                   |
| -type               | type      | Select the type of CRC-code.              |
| -block_length       | length    | Select the symbol length of the CRC-code. |

Table 10.9: Defining a CRC-Code

**Example 418**

```

CRC_3_128_10:
▷ $(ORBITER) -v 1 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -with F -do -coding_theoretic_activity \
▷ ▷ ▷ -find_CRC_polynomials 3 128 10 \
▷ ▷ -end

```

The program finds 244 polynomials in about 1 minute.

Here is a collection of CRC polynomials from various sources:

**Example 419**

```
CRC4="1,4,1,2,1,1,1,0"
```

**Example 420**

```
CRC7="1,7,1,3,1,1,0"
```

**Example 421**

```
CRC8_ATM="1,8,1,2,1,1,1,0"
```

**Example 422**

```
CRC16_CCITT="1,16,1,12,1,5,1,0"
```

**Example 423**

```

CRC32_ETHERNET="1,32,1,26,1,23,1,22,1,16,1,12,1,11,1,10,1,8,1,7,\
1,5,1,4,1,2,1,1,1,0"

```

**Example 424**

```

CRC32_CASTAGNOLI="1,32,1,28,1,27,1,26,1,25,1,23,1,22,1,20,1,19,1,\
18,1,14,1,13,1,11,1,10,1,9,1,8,1,6,1,0"

```

**Example 425**

```

CRC64_ECMA182="1,64,1,62,1,57,1,55,1,54,1,53,1,52,1,47,1,46,1,45,\
1,40,1,39,1,38,1,37,1,35,1,33,1,32,1,31,1,29,1,27,1,24,1,23,1,22,\

```

```
1,21,1,19,1,17,1,13,1,12,1,10,1,9,1,7,1,4,1,1,1,0"
```

**Example 426**

```
CRC64.ROCKSOFT="1,64,1,63,1,61,1,59,1,58,1,56,1,55,1,52,1,49,1,48,\
1,47,1,46,1,44,1,41,1,37,1,36,1,34,1,32,1,31,1,28,1,26,1,23,1,22,1,\
19,1,16,1,13,1,12,1,10,1,9,1,6,1,4,1,3,1,0"
```

We test whether the polynomial `crc32` is irreducible:

**Example 427**

```
crc32.Berlekamp.matrix:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F -sparse 33 $(CRC32.ETHERNET) -end \
> > -with F -do \
> > -finite_field_activity \
> > > -Berlekamp_matrix v \
> > -end
```

Now, we create some new CRC polynomials over the field  $\mathbb{F}_{256}$ . To begin with, we create the 771st roots over  $\mathbb{F}_{256}$ :

**Example 428**

```
CRC_F256_roots_771:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 256 -end \
> > -with F -do -finite_field_activity \
> > > -nth_roots 771 \
> > -end
```

We create a BCH code of length 771 over  $\mathbb{F}_{256}$  with designed distance 2:

**Example 429**

```
CRC_F256_BCH_code_d2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 256 -end \
> > -define C -code -field F \
> > > -BCH 771 2 \
> > -end \
> > -with C -and F -do -coding_theoretic_activity \
> > > -export_magma BCH_lq8_n771_d2.magma \
> > -end
```

```

▷ #pdflatex BCH_codes.q256_n771_d2.tex
▷ #$(OPEN) BCH_codes.q256_n771_d2.pdf

```

The polynomial in dense coding

#### Example 430

```
CRC_POLY_Q256_DEG2_DENSE="214,167,1"
```

We generate C++ source code for the use of this polynomial:

#### Example 431

```

CRC_F256_BCH_write_code_for_division_d2:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 256 -end \
▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
▷ ▷ -define B -vector -field F -dense $(CRC_POLY_Q256_DEG2_DENSE) -end \
▷ ▷ -with F -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -write_code_for_division \
▷ ▷ ▷ alfa A B \
▷ ▷ -end
▷ g++ crc_alfa.cpp -o crc_alfa.out
▷ ./crc_alfa.out

```

We create a BCH code of length 771 over  $\mathbb{F}_{256}$  with designed distance 16:

#### Example 432

```

F256_BCH_code_d16:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 256 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -BCH 771 16 \
▷ ▷ -end
▷ pdflatex BCH_codes.q256_n771_d16.tex
▷ $(OPEN) BCH_codes.q256_n771_d16.pdf

```

The polynomial in sparse coding is:

#### Example 433

```

POLY_Q256_DEG30_SPARSE="1,0,26,1,210,2,24,3,\
138,4,148,5,160,6,58,7,108,8,199,9,95,10,56,\
11,9,12,205,13,194,14,193,15,3,16,248,17,110,\
18,150,19,24,20,169,21,192,22,212,23,112,24,\
144,25,97,26,109,27,174,28,253,29,1,30"

```

The polynomial in dense coding is:

#### Example 434

```
POLY_Q256_DEG30_DENSE="1,26,210,24,138,148,\
160,58,108,199,95,56,9,205,194,193,3,248,110,\
150,24,169,192,212,112,144,97,109,174,253,1"
```

We generate C++ source code for the use of this polynomial:

#### Example 435

```
F256_BCH_write_code_for_division_d16:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 256 -end \
> > -define A -vector -field F -sparse 772 "1,771,1,0" -end \
> > -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
> > -with F -do \
> > -coding_theoretic_activity \
> > > -write_code_for_division \
> > > check_q256_n771_r30 A B \
> > -end
> g++ crc_check_q256_n771_r30.cpp -o crc_check_q256_n771_r30.out
> ./crc_check_q256_n771_r30.out
```

We confirm that the polynomial divides  $X^{771} - 1$  as it should:

#### Example 436

```
F256_BCH_code_d16_division:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 256 -end \
> > -define A -vector -field F -sparse 772 "1,771,1,0" -end \
> > -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_division A B -end
```

The next example introduces three errors. The remainder is not zero, so the errors are detected:

#### Example 437

```
F256_BCH_code_d16_error:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 256 -end \
> > -define A -vector -field F -sparse 771 "2,30,3,31,55,770" -end \
> > -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
> > -with F -do \
> > -finite_field_activity \
> > -polynomial_division A B -end
```

## 10.5 Reed-Muller Codes

The following command creates the Reed Muller code  $RM_3$ . The command exports the code to magma and writes a file of all codewords.

### Example 438

```
RM_3_export:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -Reed_Muller 3 \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma RM_3.magma \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_codewords RM_3_codewords.csv \
▷ ▷ -end
```

The next command creates a diagram of the code in Hamming space, based on the list of codewords:

### Example 439

```
RM_3_Hamming_space_diagram:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -Reed_Muller 3 \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -make_diagram \
▷ ▷ ▷ -metric_balls 1 \
▷ ▷ -end
```

The following command produces a diagram of the characteristic function of the code in the Hamming space  $H(8, 2)$ .

### Example 440

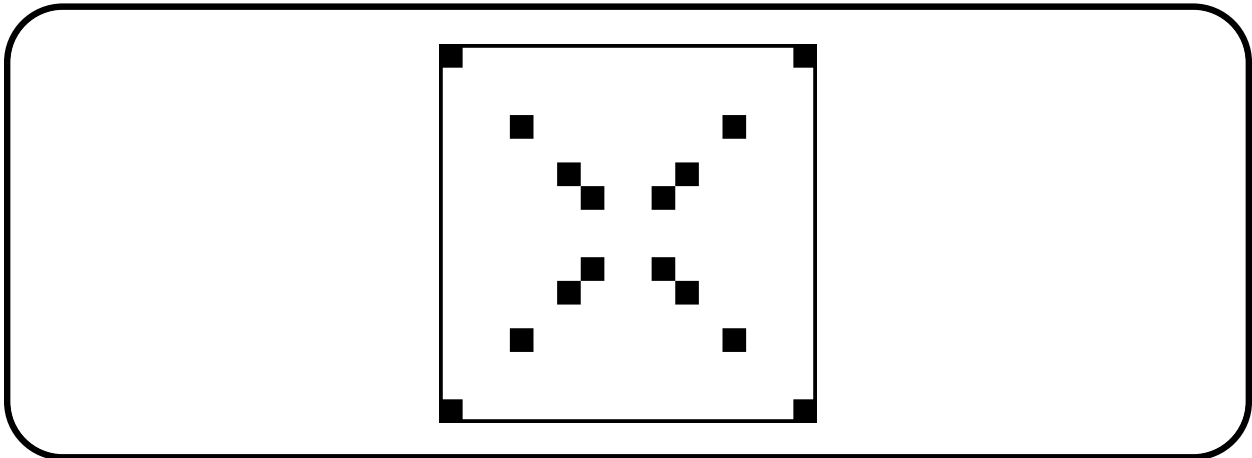
```
RM_3_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file RM_3_distance_H.8.16.csv \
▷ ▷ ▷ -box_width 25 -bit_depth 8 \
▷ ▷ ▷ -partition 4 16 16 \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file RM_3_char_func.8.16.csv \
```

```

▷ ▷ ▷ -box_width 25 -bit_depth 8 \
▷ ▷ ▷ -partition 4 16 16 \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file RM_3_idx_8_16.csv \
▷ ▷ ▷ -box_width 25 -bit_depth 8 \
▷ ▷ ▷ -partition 4 16 16 \
▷ ▷ -end
▷ convert RM_3_distance_H_8_16_draw.bmp RM_3_distance_H_8_16_draw.png
▷ convert RM_3_char_func_8_16_draw.bmp RM_3_char_func_8_16_draw.png
▷ convert RM_3_idx_8_16_draw.bmp RM_3_idx_8_16_draw.png
▷ $(OPEN) RM_3_distance_H_8_16_draw.bmp

```

The following boolean function representation of  $RM_3$  in  $H(8, 2)$  is produced:



The next command splits the vectors according to their distance from the code:

#### Example 441

```

RM_3.split:
▷ #$(ORBITER) -split_by_values RM_3_distance_8_16.csv
▷ $(ORBITER) -split_by_values RM_3_distance_H_8_16.csv

```

We produce a diagram of each distance set. We loop over all distances  $\delta = 0, 1, 2$ :

#### Example 442

```

RM_3.distance_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -loop L 0 3 1 \
▷ ▷ ▷ -draw_matrix \
▷ ▷ ▷ ▷ -input_csv_file RM_3_distance_H_8_16_value%L.csv \
▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
▷ ▷ ▷ ▷ -partition 5 16 16 \
▷ ▷ ▷ -end \

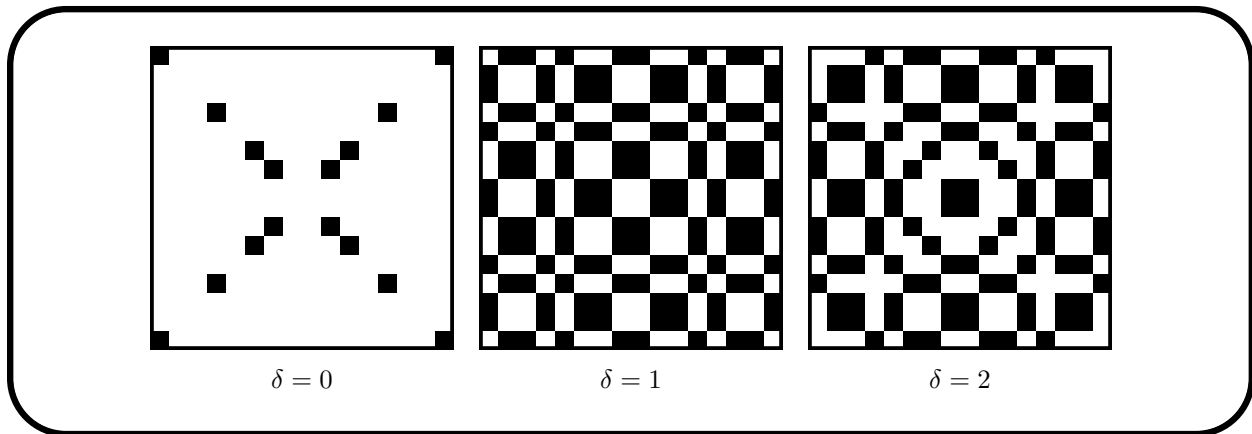
```

```

▷ ▷ -end_loop L
▷ convert RM_3_distance_H_8_16_value0.draw.bmp RM_3_distance_H_8_16_value0.draw.p
ng
▷ convert RM_3_distance_H_8_16_value1.draw.bmp RM_3_distance_H_8_16_value1.draw.p
ng
▷ convert RM_3_distance_H_8_16_value2.draw.bmp RM_3_distance_H_8_16_value2.draw.p
ng

```

The distance sets are shown below



For each distance, we compute the algebraic normal form of the characteristic function of the distance set:

#### Example 443

```

RM_3_algebraic_normal_form:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -loop L 0 3 1 \
▷ ▷ ▷ -define v%L -vector -field F \
▷ ▷ ▷ ▷ -file RM_3_distance_8_16_value%L.csv \
▷ ▷ ▷ -end \
▷ ▷ ▷ -with F -do -finite_field_activity \
▷ ▷ ▷ ▷ -algebraic_normal_form 8 v%L \
▷ ▷ ▷ -end \
▷ ▷ -end_loop L

```

In the example, we will compute a graphical representation of the codewords of the Reed-Muller code of order 6. First, we use makefile variables to define the rows of the generator matrix:

#### Example 444

```

RM_6_GENERATOR_1="0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,\
22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,\
46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63"

```



**Example 445**

```
RM_6.GENERATOR_2="1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,\
41,43,45,47,49,51,53,55,57,59,61,63"
```

**Example 446**

```
RM_6.GENERATOR_3="2,3,6,7,18,19,22,23,10,11,14,15,26,27,30,31,34,35,38,\
39,42,43,46,47,50,51,54,55,58,59,62,63"
```

**Example 447**

```
RM_6.GENERATOR_4="4,6,12,14,36,38,52,54,5,7,13,15,37,39,53,55,20,22,28,\
30,44,46,60,62,21,23,29,31,45,47,61,63"
```

**Example 448**

```
RM_6.GENERATOR_5="8,9,12,13,24,25,28,29,10,11,14,15,26,27,30,31,40,41,\
44,45,56,57,60,61,42,43,46,47,58,59,62,63"
```

**Example 449**

```
RM_6.GENERATOR_6="16,18,24,26,48,50,56,58,17,19,25,27,49,51,57,59,20,22,\
28,30,52,54,60,62,21,23,29,31,53,55,61,63"
```

**Example 450**

```
RM_6.GENERATOR_7="32,34,48,50,33,35,49,51,36,38,52,54,37,39,53,55,40,42,\
56,58,41,43,57,59,44,46,60,62,45,47,61,63"
```

The following command creates the generator matrix, and exports the generator matrix and the set of codewords to two csv files.

**Example 451**

```
RM_6:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define C -code -field F \
> > > -long_code 64 7 \
> > > $(RM_6.GENERATOR_1) \
> > > $(RM_6.GENERATOR_2) \
> > > $(RM_6.GENERATOR_3) \
> > > $(RM_6.GENERATOR_4) \
```

```

▷ ▷ ▷ $(RM_6.GENERATOR_5) \
▷ ▷ ▷ $(RM_6.GENERATOR_6) \
▷ ▷ ▷ $(RM_6.GENERATOR_7) \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_genma RM6_genma.csv \
▷ ▷ -end \
▷ ▷ -with C -and F -do -coding_theoretic_activity \
▷ ▷ ▷ -export_codewords_long RM6_codewords.csv \
▷ ▷ -end \

```

The next command produces a drawing of the generator matrix:

#### Example 452

```

RM_6_genma_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file RM6_genma.csv \
▷ ▷ ▷ -box_width 25 -bit_depth 8 \
▷ ▷ ▷ -partition 3 7 64 \
▷ ▷ -end
▷ convert RM6_genma_draw.bmp RM6_genma_draw.png
▷ $(OPEN) RM6_genma_draw.png

```

The following command produces a drawing of the matrix of codewords:

#### Example 453

```

RM_6_codewords_matrix_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file RM6_codewords.csv \
▷ ▷ ▷ -box_width 25 -bit_depth 8 \
▷ ▷ ▷ -partition 3 128 64 \
▷ ▷ -end
▷ convert RM6_codewords_draw.bmp RM6_codewords_draw.png
▷ $(OPEN) RM6_codewords_draw.png

```

The next command produces individual pictures for each of the 128 codewords. The command utilizes the `-loop` command to automatize the processing. Afterwards, the codeword pictures are composed in one poster, using the `convert` tool from ImageMagix.

#### Example 454

```

RM_6_codewords_draw:
▷ $(ORBITER) -v 4 \
▷ ▷ -loop i 0 128 1 \
▷ ▷ ▷ -csv_file_select_rows RM6_codewords.csv %i \

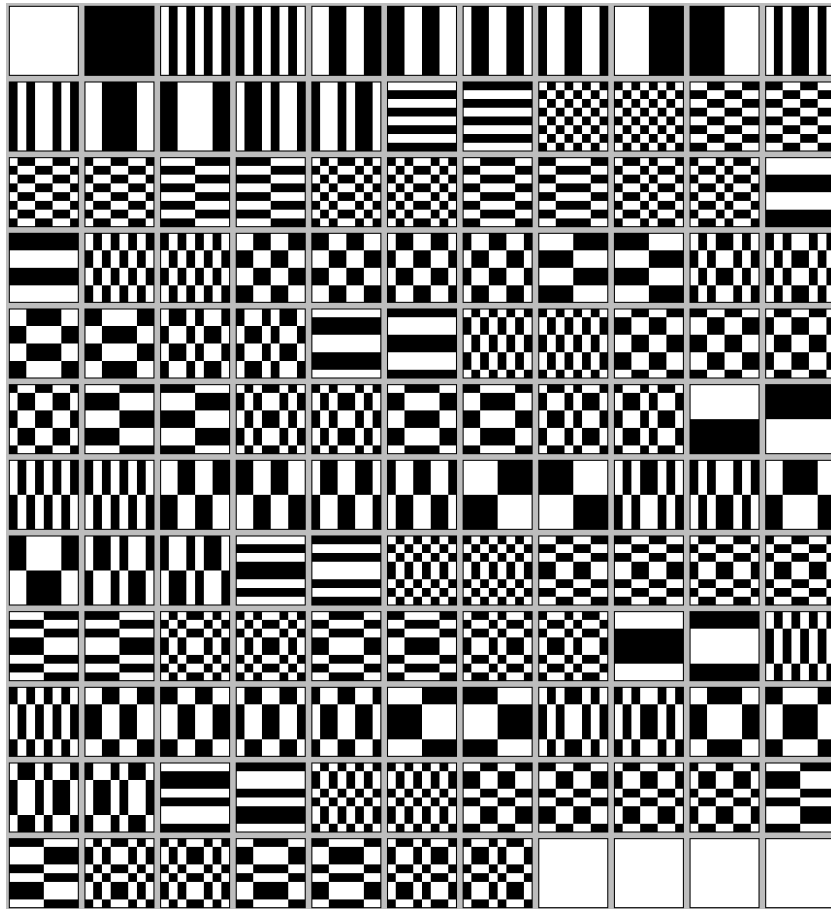
```

```

> > > -reformat RM6_codewords_select_%i.csv RM6_codewords_ref_%i.csv 8 \
> > > -draw_matrix \
> > > > -input_csv_file RM6_codewords_ref_%i.csv \
> > > > -box_width 25 -bit_depth 8 \
> > > > -partition 3 8 8 \
> > > -end \
> > > -system "convert RM6_codewords_ref_%i_draw.bmp -frame 8 %i.png" \
> > -end_loop i
> convert 0.png 1.png 2.png 3.png 4.png 5.png \
> > 6.png 7.png 8.png 9.png 10.png +append a0
> convert 11.png 12.png 13.png 14.png 15.png \
> > 16.png 17.png 18.png 19.png 20.png 21.png +append a1
> convert 22.png 23.png 24.png 25.png 26.png \
> > 27.png 28.png 29.png 30.png 31.png 32.png +append a2
> convert 33.png 34.png 35.png 36.png 37.png \
> > 38.png 39.png 40.png 41.png 42.png 43.png +append a3
> convert 44.png 45.png 46.png 47.png 48.png \
> > 49.png 50.png 51.png 52.png 53.png 54.png +append a4
> convert 55.png 56.png 57.png 58.png 59.png \
> > 60.png 61.png 62.png 63.png 64.png 65.png +append a5
> convert 66.png 67.png 68.png 69.png 70.png \
> > 71.png 72.png 73.png 74.png 75.png 76.png +append a6
> convert 77.png 78.png 79.png 80.png 81.png \
> > 82.png 83.png 84.png 85.png 86.png 87.png +append a7
> convert 88.png 89.png 90.png 91.png 92.png \
> > 93.png 94.png 95.png 96.png 97.png 98.png +append a8
> convert 99.png 100.png 101.png 102.png 103.png \
> > 104.png 105.png 106.png 107.png 108.png 109.png +append a9
> convert 110.png 111.png 112.png 113.png 114.png \
> > 115.png 116.png 117.png 118.png 119.png 120.png +append a10
> convert 121.png 122.png 123.png 124.png 125.png \
> > 126.png 127.png 0.png 0.png 0.png 0.png +append a11
> convert a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 \
> > a11 -append poster_RM_1_6.png

```

The codewords of  $RM_6$  as boolean function in Hamming space are shown below:



| Options for Drawing Modulo N Objects |           |                                                            |
|--------------------------------------|-----------|------------------------------------------------------------|
| Command                              | Arguments | Purpose                                                    |
| -n                                   | n         | Set the number of dots.                                    |
| -mod_s                               | s         |                                                            |
| -divide_out_by                       | k         | Divide by k.                                               |
| -file                                | fname     | Read from file.                                            |
| -inverse                             |           | Connect $i$ with $i^{-1} \bmod n$ .                        |
| -additive_inverse                    |           | Connect $i$ with $-i \bmod n$ .                            |
| -power_cycle                         | a         | Draw a cycle connecting the powers of $a \bmod n$ .        |
| -cyclotomic_sets                     | q list    | Draw the $q$ -cyclotomic sets of all elements in the list. |
| -cyclotomic_sets_thickness           |           | Set the thickness for the drawing of cyclotomic sets.      |

Table 10.10: Options for Drawing Modulo N Objects

## 10.6 BCH Codes

Let  $\beta$  be an  $n$ -th root of unity over  $\mathbb{F}_q$ . The minimum polynomial of  $\beta$  over  $\mathbb{F}_q$  is denoted as  $m_{\beta, \mathbb{F}_q}$ . The BCH code of length  $n$  and designed distance  $d$  is the cyclic code with generator polynomial

$$\text{lcm}\left(m_{\beta^1, \mathbb{F}_q}, m_{\beta^2, \mathbb{F}_q}, \dots, m_{\beta^{d-1}, \mathbb{F}_q}\right).$$

To create the polynomial  $m_{\beta^a, \mathbb{F}_q}$ , we consider the  $q$ -cyclotomic set of  $a$  modulo  $n$ , which is

$$\{aq^i \bmod n \mid i \in \mathbb{Z}\}.$$

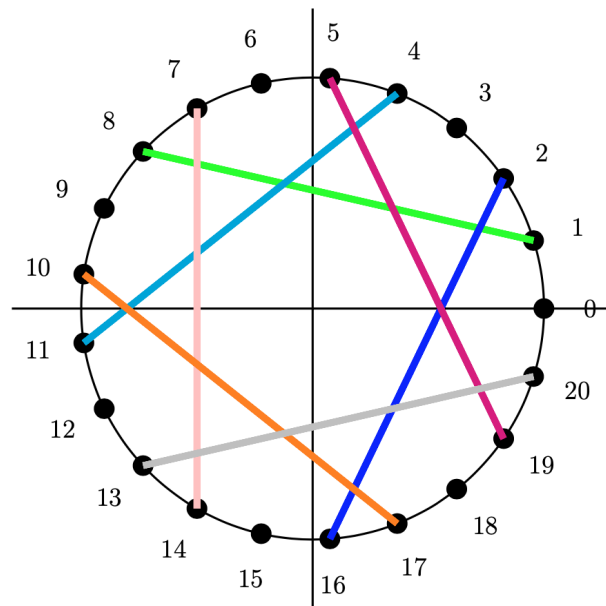
Orbiter can illustrate cyclic sets using the commands of Table 10.10.

Suppose we want to make a BCH-code of length 21 over  $\mathbb{F}_8$ . In Section 3.2, we considered the  $q$ -cyclotomic sets modulo 21 for  $q = 8$ . Let us produce a pictorial representation. Omitting the singletons, a transversal is given by the sets containing 1, 2, 4, 5, 7, 10, 13. For this reason, we issue the command

### Example 455

```
draw_cyclotomic_mod_21_q8:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_options \
▷ ▷ ▷ -radius 100 \
▷ ▷ ▷ -line_width 1.0 -embedded \
▷ ▷ -end \
▷ ▷ -draw_mod_n \
▷ ▷ ▷ -n 21 \
▷ ▷ ▷ -file mod_21_cyclotomic \
▷ ▷ ▷ -cyclotomic_sets 8 "1,2,4,5,7,10,13" \
▷ ▷ -end
▷ pdflatex mod_21_cyclotomic_draw.tex
▷ $(OPEN) mod_21_cyclotomic_draw.pdf
```

The output is shown below:



We will try BCH-codes with minimum distances 3, 5 and 7. Here is distance 3:

#### Example 456

```
F_8_BCH_code_d3:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 8 -override_polynomial 11 -end \
> > -define C -code -field F \
> > > -BCH 21 3 \
> > -end
> pdflatex BCH_codes_q8_n21_d3.tex
> $(OPEN) BCH_codes_q8_n21_d3.pdf
```

The code construction is shown in a latex report:

BCH-code:

$n = 21$ ,  $k = 17$ ,  $d_0 = 3$ ,  $q = 8$ ,

$g(x) = m_1 m_2 = X^4 + 4X^3 + 4X^2 + 3X + 4$

Chosen cyclotomic sets:

$\{ 1, 8 \}$

$\{ 2, 16 \}$

The generator polynomial has degree 4

-dense "4,3,4,4,1"

-sparse "4,0,3,1,4,2,4,3,1,4"

The generator matrix is:

$$\begin{bmatrix} 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 \end{bmatrix}$$

Let us stick to the same parameters for the field and the length but increase the minimum distance to  $d = 5$ :

#### Example 457

F\_8\_BCH\_code\_d5:

```
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 8 -override_polynomial 11 -end \
> > -define C -code -field F \
> > > -BCH 21 5 \
> > -end
> pdflatex BCH_codes.q8_n21_d5.tex
> $(OPEN) BCH_codes.q8_n21_d5.pdf
```

The output file is:

BCH-code:

$n = 21, k = 14, d_0 = 5, q = 8,$

$g(x) = m_1 m_2 m_3 m_4 = X^7 + 3X^6 + 3X^5 + 2X^4 + X^3 + 2X^2 + X + 2$

Chosen cyclotomic sets:

$\{ 1, 8 \}$

$\{ 2, 16 \}$

$\{ 3 \}$

$\{ 4, 11 \}$

The generator polynomial has degree 7

-dense "2,1,2,1,2,3,3,1"

```
-sparse "2,0,1,1,2,2,1,3,2,4,3,5,3,6,1,7"
```

The generator matrix is:

$$\begin{bmatrix} 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 \end{bmatrix}$$

We compute the minimum distance:

#### Example 458

```
F_8_BCH_code_d5_minimum_distance:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 8 -override_polynomial 11 -end \
> > -define v -vector -format 14 -field F \
> > > -compact $(CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11) \
> > -end \
> > -with F -do \
> > -coding_theoretic_activity \
> > > -minimum_distance v \
> > -end
# important: use the same polynomial as when creating the code.
#
# d=5
```

The minimum distance turns out to be  $d = 5$ .

Finally, we create the BCH code with minimum distance  $d = 7$ :

#### Example 459

```
F_8_BCH_code_d7:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 8 -override_polynomial 11 -end \
> > -define C -code -field F \
> > > -BCH 21 7 \
> > -end
```



The output file is:

```
BCH-code:
n = 21, k = 11, d0 = 7, q = 8,
g(x) = m1m2m3m4m5m6 = X10 + X9 + 2X8 + 5X7 + 2X6 + 4X4 + 6X3 + 5X2 + 6X + 6
Chosen cyclotomic sets:
{ 1, 8 }
{ 2, 16 }
{ 3 }
{ 4, 11 }
{ 5, 19 }
{ 6 }
The generator polynomial has degree 10
```

```
-dense "6,6,5,6,4,0,2,5,2,1,1"
```

```
-sparse "6,0,6,1,5,2,6,3,4,4,2,6,5,7,2,8,1,9,1,10"
```

The generator matrix is:

$$\begin{bmatrix} 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 \end{bmatrix}$$

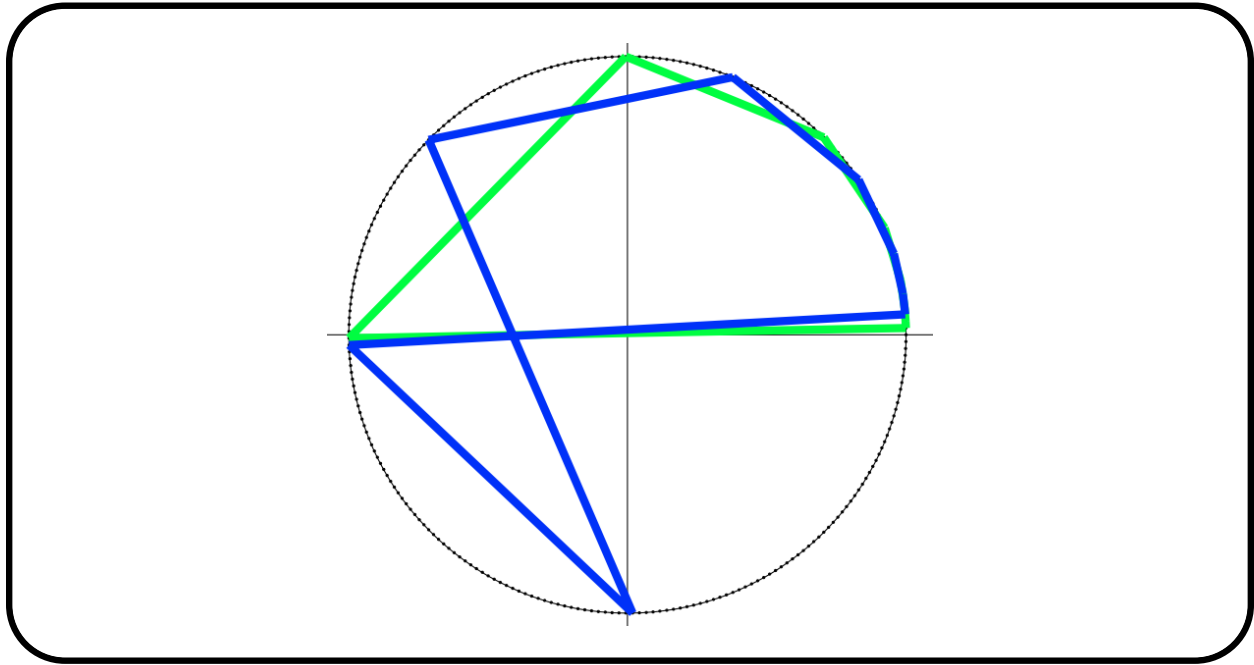
As a larger example, let us consider the 2-cyclotomic sets of 2 and 3 modulo 255. The following command produces a graphical representation on a circle (similar to the unit circle in complex analysis). The 255-th roots of unity are placed in the appropriate position.

#### Example 460

```
draw_mod_255_cyclotomic_1_and_3:
> $(ORBITER) -v 2 \
> > -draw_options -nodes_empty -radius 10 \
> > > -line_width 0.4 -embedded -end \
> > -draw_mod_n \
> > > -n 255 \
> > > -file mod_255_cyclotomic_1_and_3 \
> > > -cyclotomic_sets 2 "1,3" \
> > -end
> pdflatex mod_255_cyclotomic_1_and_3_draw.tex
```

```
▷ $(OPEN) mod_255_cyclotomic_1_and_3_draw.pdf
```

The drawing is shown below



Suppose we want to make a BCH-code over  $\mathbb{F}_{256}$ . In order to keep the degree of the generator polynomial low, we try a quadratic field extension. This way, each cyclotomic set has size either 1 or 2. Since

$$256^2 - 1 = (256 + 1)(256 - 1) = 257 \cdot 3 \cdot 5 \cdot 17,$$

we can consider a code of length  $n = 771 = 257 \cdot 3$ . The following command computes the 256-cyclotomic cosets modulo 771:

#### Example 461

```
BCH_F256_roots_771:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 256 -end \
▷ ▷ -with F -do -finite_field_activity \
▷ ▷ ▷ -nth_roots 771 \
▷ ▷ -end
```

The next command creates a BCH-code of length 771 over  $\mathbb{F}_{256}$  with minimum distance at least 16:

#### Example 462

```
BCH_F256_BCH_code_d16:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 256 -end \
▷ ▷ -define C -code -field F \
```

```

> > > -BCH 771 16 \
> > -end
> pdflatex BCH_codes.q256.n771.d16.tex
> $(OPEN) BCH_codes.q256.n771.d16.pdf

```

We compute the minimum distance:

#### Example 463

```

F_8_BCH_code_d5_minimum_distance:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 8 -override_polynomial 11 -end \
> > -define v -vector -format 14 -field F \
> > > -compact $(CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11) \
> > -end \
> > -with F -do \
> > -coding_theoretic_activity \
> > > -minimum_distance v \
> > -end
# important: use the same polynomial as when creating the code.
#
# d=5

```

The minimum distance turns out to be  $d = 5$ .

## 10.7 Reed-Solomon Codes

Reed-Solomon codes are BCH-codes where the length  $n$  divides  $q - 1$ . In particular, they are cyclic codes. They are almost never binary.

To create a Reed-Solomon code over  $\mathbb{F}_7$  with minimum distance at least 3. The command

### Example 464

```
RS_6_3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define C -code -field F -Reed_Solomon 6 3 -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma RS_6_3.magma \
▷ ▷ -end
▷ pdflatex RS_codes_q7_n6_d3.tex
▷ $(OPEN) RS_codes_q7_n6_d3.pdf
```

creates this code. The following report is produced:

RS-code:

$$n = 6, k = 4, d_0 = 3, q = 7,$$

$$g(x) = m_1 m_2 = X^2 + 2X + 6$$

The generator polynomial has degree 2

-dense "6,2,1"

-sparse "6,0,2,1,1,2"

The generator matrix is:

$$\begin{bmatrix} 6 & 2 & 1 & 0 & 0 & 0 \\ 0 & 6 & 2 & 1 & 0 & 0 \\ 0 & 0 & 6 & 2 & 1 & 0 \\ 0 & 0 & 0 & 6 & 2 & 1 \end{bmatrix}$$

As the report shows, the code is cyclic, generated by

$$(X - \alpha)(X - \alpha^2) = (X - 3)(X - 2) = X^2 + 2X + 6.$$

Let us investigate this code. We start with the weight enumerator. The command

### Example 465

```
RS_6_3_weight_enumerator:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define C -code -field F -Reed_Solomon 6 3 -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma RS_6_3.magma \
```

```

▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -weight_enumerator \
▷ ▷ -end

```

computes the weight enumerator, which turns out to be

$$(1, 0, 0, 120, 360, 972, 948).$$

In polynomial form, this is

$$1y^6 + 120x^3y^3 + 360x^4y^2 + 972x^5y + 948x^6.$$

This confirms that the minimum distance is three.

Let us consider an example of a Reed-Solomon code in characteristic two. We will create a Reed Solomon code of designed distance 3 over  $\mathbb{F}_8$ . The command

#### Example 466

```

RS_7_3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma RS_7_3.magma \
▷ ▷ -end
▷ pdflatex RS_codes_q8_n7_d3.tex
▷ $(OPEN) RS_codes_q8_n7_d3.pdf

```

creates this code. The following report is produced:

RS-code:

$$n = 7, k = 5, d_0 = 3, q = 8,$$

$$g(x) = m_1m_2 = X^2 + 6X + 5$$

The generator polynomial has degree 2

-dense "5,6,1"

-sparse "5,0,6,1,1,2"

The generator matrix is:

$$\begin{bmatrix} 5 & 6 & 1 & 0 & 0 & 0 & 0 \\ 0 & 5 & 6 & 1 & 0 & 0 & 0 \\ 0 & 0 & 5 & 6 & 1 & 0 & 0 \\ 0 & 0 & 0 & 5 & 6 & 1 & 0 \\ 0 & 0 & 0 & 0 & 5 & 6 & 1 \end{bmatrix}$$

As we can see, the code is a cyclic code generated by the polynomial

$$(X - \alpha)(X - \alpha^2) = X^2 + 6X + 5.$$

What is the minimum weight of the code? The following command

#### Example 467

```
RS_7_3_weight_enumerator:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
▷ ▷ -with C -do -coding_theoretic_activity \
▷ ▷ ▷ -export_magma RS_7_3.magma \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -weight_enumerator \
▷ ▷ -end
▷ pdflatex RS_codes_q8_n7_d3.tex
▷ $(OPEN) RS_codes_q8_n7_d3.pdf
```

computes the weight enumerator, which turns out to be

$$y^7 + 245x^3y^4 + 1225x^4y^3 + 5586x^5y^2 + 12838x^6y + 12873x^7.$$

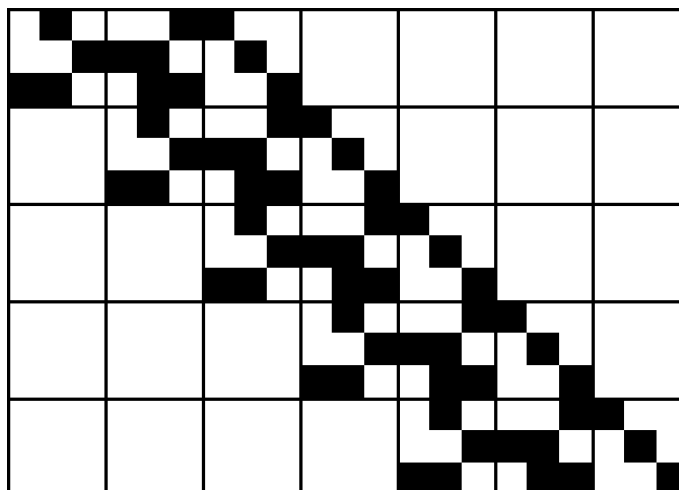
This shows that the minimum distance is 3, as expected. Computing the automorphism group of the code is computationally infeasible.

The next command performs field reduction of the code.

#### Example 468

```
RS_7_3_field_reduction:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
▷ ▷ -with F -do \
▷ ▷ -finite_field_activity \
▷ ▷ -field_reduction "RS_8_red_2" \
▷ ▷ ▷ 2 5 7 $(CODE_RS_F8_N7_K5_D3_GENMA) \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix -input_csv_file RS_8_red_2.csv \
▷ ▷ -box_width 40 -bit_depth 24 \
▷ ▷ -partition 4 "3,3,3,3,3" "3,3,3,3,3,3,3" -end
▷ pdflatex field_reduction_Q8_q2_5_7.tex
▷ $(OPEN) field_reduction_Q8_q2_5_7.pdf
```

This produces a  $[21, 15]_2$  code. The reduced matrix is below



We store the generator matrix in a makefile variable:

#### Example 469

```
RS_8_reduced="\
0100011000000000000000\
0011100100000000000000\
1100110010000000000000\
0000100011000000000000\
0000011100100000000000\
0001100110010000000000\
0000000100011000000000\
0000000011100100000000\
0000000110011001000000\
0000000000100011000000\
000000000001110010000\
00000000000011001000\
000000000000010001100\
000000000000001110010\
000000000000000110011001"
```

Let us compute the weight enumerator of the reduced code. The command

#### Example 470

```
RS_7_3_reduced.weight_enumerator:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -format 15 -field F \
▷ ▷ ▷ -compact $(RS_8_reduced) \
▷ ▷ -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -generator_matrix v \
▷ ▷ -end \
```

```

▷ ▷ -with C -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -weight_enumerator \
▷ ▷ -end

```

computes the weight enumerator of the binary code. It is

$$\begin{aligned}
 &1y^{21} + 28x^3y^{18} + 84x^4y^{17} + 273x^5y^{16} + 924x^6y^{15} + 1956x^7y^{14} + 2982x^8y^{13} + \\
 &4340x^9y^{12} + 5796x^{10}y^{11} + 5796x^{11}y^{10} + 4340x^{12}y^9 + 2982x^{13}y^8 + 1956x^{14}y^7 + \\
 &924x^{15}y^6 + 273x^{16}y^5 + 84x^{17}y^4 + 28x^{18}y^3 + 1x^{21}
 \end{aligned}$$

In particular, the field reduced Reed-Solomon code has minimum distance three. Since there are other codes of minimum distance 4, the field-reduced code is not optimal.



## 10.8 Twisted Tensor Product Codes

The command

### Example 471

```
TTP_A_4:
> $(ORBITER) -v 2 \
> > -define F4 -finite_field -q 4 -end \
> > -define F16 -finite_field -q 16 -end \
> > -define C -code -field F4 -ttpA F16 -end \
> > -with C -do -coding_theoretic_activity \
> > > -export_magma TTP_A_q4.magma \
> > -end \
> > -with C -do -coding_theoretic_activity \
> > > -report \
> > -end
> pdflatex code_n18_k9_q4.tex
> $(OPEN) code_n18_k9_q4.pdf
```

creates the twisted tensor product code arising from the hyperoval of PG(2, 16). The command also produces a report of the code, as shown below.

The generator matrix is:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 3 & 1 & 3 & 1 & 2 & 3 & 0 & 3 & 2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 2 & 1 & 2 & 3 & 0 & 2 & 3 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 3 & 1 & 0 & 3 & 2 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 3 & 3 & 1 & 1 & 2 & 2 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 3 & 2 & 0 & 1 & 0 & 0 & 3 & 2 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 2 & 3 & 1 & 0 & 0 & 0 & 2 & 3 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 2 & 2 & 1 & 2 & 1 & 3 & 0 & 2 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 2 & 0 & 3 & 3 & 0 & 3 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The command also exports the code to magma. The magma file is shown below:

```
K<w> := GF(4);
V := VectorSpace(K, 18);
C := LinearCode(sub<V |
[1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0],
[1,w^2,1,w^2,1,w^1,w^2,0,w^2,w^1,1,0,0,0,0,0,0,0],
[0,w^1,0,w^1,1,w^1,w^2,0,w^1,w^2,0,1,0,0,0,0,0,0],
[1,w^1,w^2,1,0,w^2,w^1,0,1,0,0,0,1,0,0,0,0,0],
[0,w^2,w^2,1,1,w^1,w^1,0,0,1,0,0,0,1,0,0,0,0],
[1,1,w^2,w^1,0,1,0,0,w^2,w^1,0,0,0,0,1,0,0,0],
[1,1,w^1,w^2,1,0,0,0,w^1,w^2,0,0,0,0,0,1,0,0],
```

```
[1,w^1,w^1,1,w^1,1,w^2,0,w^1,w^1,0,0,0,0,0,0,1,0],  
[1,w^1,0,w^2,w^2,0,w^2,0,1,1,0,0,0,0,0,0,0,1]>);
```

## 10.9 Bounds

In coding theory, one main question is to determine the best value of  $d_{\max}$  for a fixed  $n$ ,  $k$  and  $q$  such that a linear  $[n, k, d]_q$  code exists. There are many bounds, both upper and lower bounds. An upper bound tells us that no code with  $d \geq d_{\max}$  exists. A lower bound tells us that a code with  $d \geq d_{\max}$  exists. The command

### Example 472

```
bounds_for_d_given_n15_k6_q2:
> $(ORBITER) -v 2 \
> > -make_bounds_for_d_given_n_and_k_and_q 15 6 2
```

gives upper and lower bounds on the optimal minimum distance  $d_{\max}$  of a  $[15, 6]_2$  code. The values of the Gilbert-Varshamov lower bound and the Singleton, Hamming, Plotkin and Griesmer upper bounds are computed. The output is:

```
n = 15 k=6 q=2
d_GV = 5
d_singleton = 10
d_hamming = 6
d_plotkin = 7
d_griesmer = 6
```

This shows that  $5 \leq d_{\max} \leq 6$ . The command

### Example 473

```
coding_theory_bounds_q2:
> $(ORBITER) -v 2 -table_of_bounds 12 2
> $(ORBITER) -v 2 \
> > -csv_file_latex 1 table_of_bounds_n12_q2.csv
> pdflatex table_of_bounds_n12_q2.tex
> $(OPEN) table_of_bounds_n12_q2.pdf
```

produces a table of bounds for binary codes with  $n, k \leq 12$ . The second command produces the latex report below:

| n | k | q | GV | S | H  | P | G  | n  | k  | q | GV | S  | H  | P  | G  |
|---|---|---|----|---|----|---|----|----|----|---|----|----|----|----|----|
| 2 | 1 | 2 | 2  | 2 | 2  | 2 | 3  | 9  | 6  | 2 | 2  | 4  | 2  | 4  | 3  |
| 2 | 2 | 2 | 1  | 1 | 2  | 1 | 1  | 9  | 7  | 2 | 2  | 3  | 2  | 4  | 2  |
| 3 | 1 | 2 | 3  | 3 | 4  | 3 | 4  | 9  | 8  | 2 | 2  | 2  | 2  | 4  | 2  |
| 3 | 2 | 2 | 2  | 2 | 2  | 2 | 2  | 9  | 9  | 2 | 1  | 1  | 2  | 4  | 1  |
| 3 | 3 | 2 | 1  | 1 | 2  | 1 | 1  | 10 | 1  | 2 | 10 | 10 | 10 | 10 | 11 |
| 4 | 1 | 2 | 4  | 4 | 4  | 4 | 5  | 10 | 2  | 2 | 5  | 9  | 8  | 6  | 6  |
| 4 | 2 | 2 | 2  | 3 | 2  | 2 | 2  | 10 | 3  | 2 | 4  | 8  | 6  | 5  | 5  |
| 4 | 3 | 2 | 2  | 2 | 2  | 2 | 2  | 10 | 4  | 2 | 4  | 7  | 6  | 5  | 4  |
| 4 | 4 | 2 | 1  | 1 | 2  | 2 | 1  | 10 | 5  | 2 | 3  | 6  | 4  | 5  | 4  |
| 5 | 1 | 2 | 5  | 5 | 6  | 5 | 6  | 10 | 6  | 2 | 3  | 5  | 4  | 5  | 4  |
| 5 | 2 | 2 | 3  | 4 | 4  | 3 | 3  | 10 | 7  | 2 | 2  | 4  | 2  | 5  | 3  |
| 5 | 3 | 2 | 2  | 3 | 2  | 2 | 2  | 10 | 8  | 2 | 2  | 3  | 2  | 5  | 2  |
| 5 | 4 | 2 | 2  | 2 | 2  | 2 | 2  | 10 | 9  | 2 | 2  | 2  | 2  | 5  | 2  |
| 5 | 5 | 2 | 1  | 1 | 2  | 2 | 1  | 10 | 10 | 2 | 1  | 1  | 2  | 5  | 1  |
| 6 | 1 | 2 | 6  | 6 | 6  | 6 | 7  | 11 | 1  | 2 | 11 | 11 | 12 | 11 | 12 |
| 6 | 2 | 2 | 3  | 5 | 4  | 4 | 4  | 11 | 2  | 2 | 6  | 10 | 8  | 7  | 7  |
| 6 | 3 | 2 | 3  | 4 | 4  | 3 | 3  | 11 | 3  | 2 | 5  | 9  | 8  | 6  | 6  |
| 6 | 4 | 2 | 2  | 3 | 2  | 3 | 2  | 11 | 4  | 2 | 4  | 8  | 6  | 5  | 5  |
| 6 | 5 | 2 | 2  | 2 | 2  | 3 | 2  | 11 | 5  | 2 | 4  | 7  | 4  | 5  | 4  |
| 6 | 6 | 2 | 1  | 1 | 2  | 3 | 1  | 11 | 6  | 2 | 3  | 6  | 4  | 5  | 4  |
| 7 | 1 | 2 | 7  | 7 | 8  | 7 | 8  | 11 | 7  | 2 | 3  | 5  | 4  | 5  | 4  |
| 7 | 2 | 2 | 4  | 6 | 6  | 4 | 4  | 11 | 8  | 2 | 2  | 4  | 2  | 5  | 3  |
| 7 | 3 | 2 | 3  | 5 | 4  | 4 | 4  | 11 | 9  | 2 | 2  | 3  | 2  | 5  | 2  |
| 7 | 4 | 2 | 3  | 4 | 4  | 3 | 3  | 11 | 10 | 2 | 2  | 2  | 2  | 5  | 2  |
| 7 | 5 | 2 | 2  | 3 | 2  | 3 | 2  | 11 | 11 | 2 | 1  | 1  | 2  | 5  | 1  |
| 7 | 6 | 2 | 2  | 2 | 2  | 3 | 2  | 12 | 1  | 2 | 12 | 12 | 12 | 12 | 13 |
| 7 | 7 | 2 | 1  | 1 | 2  | 3 | 1  | 12 | 2  | 2 | 6  | 11 | 10 | 8  | 8  |
| 8 | 1 | 2 | 8  | 8 | 8  | 8 | 9  | 12 | 3  | 2 | 5  | 10 | 8  | 6  | 6  |
| 8 | 2 | 2 | 4  | 7 | 6  | 5 | 5  | 12 | 4  | 2 | 5  | 9  | 6  | 6  | 6  |
| 8 | 3 | 2 | 4  | 6 | 4  | 4 | 4  | 12 | 5  | 2 | 4  | 8  | 6  | 6  | 5  |
| 8 | 4 | 2 | 3  | 5 | 4  | 4 | 4  | 12 | 6  | 2 | 3  | 7  | 4  | 6  | 4  |
| 8 | 5 | 2 | 2  | 4 | 2  | 4 | 3  | 12 | 7  | 2 | 3  | 6  | 4  | 6  | 4  |
| 8 | 6 | 2 | 2  | 3 | 2  | 4 | 2  | 12 | 8  | 2 | 3  | 5  | 4  | 6  | 4  |
| 8 | 7 | 2 | 2  | 2 | 2  | 4 | 2  | 12 | 9  | 2 | 2  | 4  | 2  | 6  | 3  |
| 8 | 8 | 2 | 1  | 1 | 2  | 4 | 1  | 12 | 10 | 2 | 2  | 3  | 2  | 6  | 2  |
| 9 | 1 | 2 | 9  | 9 | 10 | 9 | 10 | 12 | 11 | 2 | 2  | 2  | 2  | 6  | 2  |
| 9 | 2 | 2 | 5  | 8 | 6  | 6 | 6  | 12 | 12 | 2 | 1  | 1  | 2  | 6  | 1  |
| 9 | 3 | 2 | 4  | 7 | 6  | 5 | 4  |    |    |   |    |    |    |    |    |
| 9 | 4 | 2 | 3  | 6 | 4  | 4 | 4  |    |    |   |    |    |    |    |    |
| 9 | 5 | 2 | 3  | 5 | 4  | 4 | 4  |    |    |   |    |    |    |    |    |

The command

#### Example 474

```
GV_n15_k6_d5:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -Gilbert_Varshamov 15 6 5 \
▷ ▷ -end
```

creates a  $[15, 6, d]_2$  with minimum distance  $g \geq 5$  using a greedy algorithm based on the proof of the Gilbert-Varshamov bound. The code that is produced has the following generator matrix:

```
1 1 1 1 1 1 1 1 1 1 0 0 0 0 0
1 1 1 1 1 0 0 0 0 0 1 0 0 0 0
1 1 1 0 0 1 1 0 0 0 0 1 0 0 0
1 1 0 1 0 1 0 1 0 0 0 0 1 0 0
1 0 1 0 1 0 1 1 0 0 0 0 0 1 0
1 0 1 1 0 1 0 0 1 0 0 0 0 0 1
```

To compute the minimum distance of the code, we do:

#### Example 475

```
CODE_GV_N15_K6="\
111111111100000\
111110000010000\
111001100001000\
110101010000100\
101010110000010\
101101001000001"
```

#### Example 476

```
GV_n15_k6_d5_weight_enumerator:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define v -vector -format 6 -field F \
▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
▷ ▷ -end \
▷ ▷ -define C -code -field F \
▷ ▷ ▷ -generator_matrix v \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -coding_theoretic_activity \
▷ ▷ ▷ -weight_enumerator \
▷ ▷ -end
```

The weight enumerator is computed to be:

$$1y^{15} + 27x^6y^9 + 24x^8y^7 + 9x^{10}y^5 + 3x^{12}y^3.$$

From this, we see that the code has minimum distance 6, which is better than predicted.

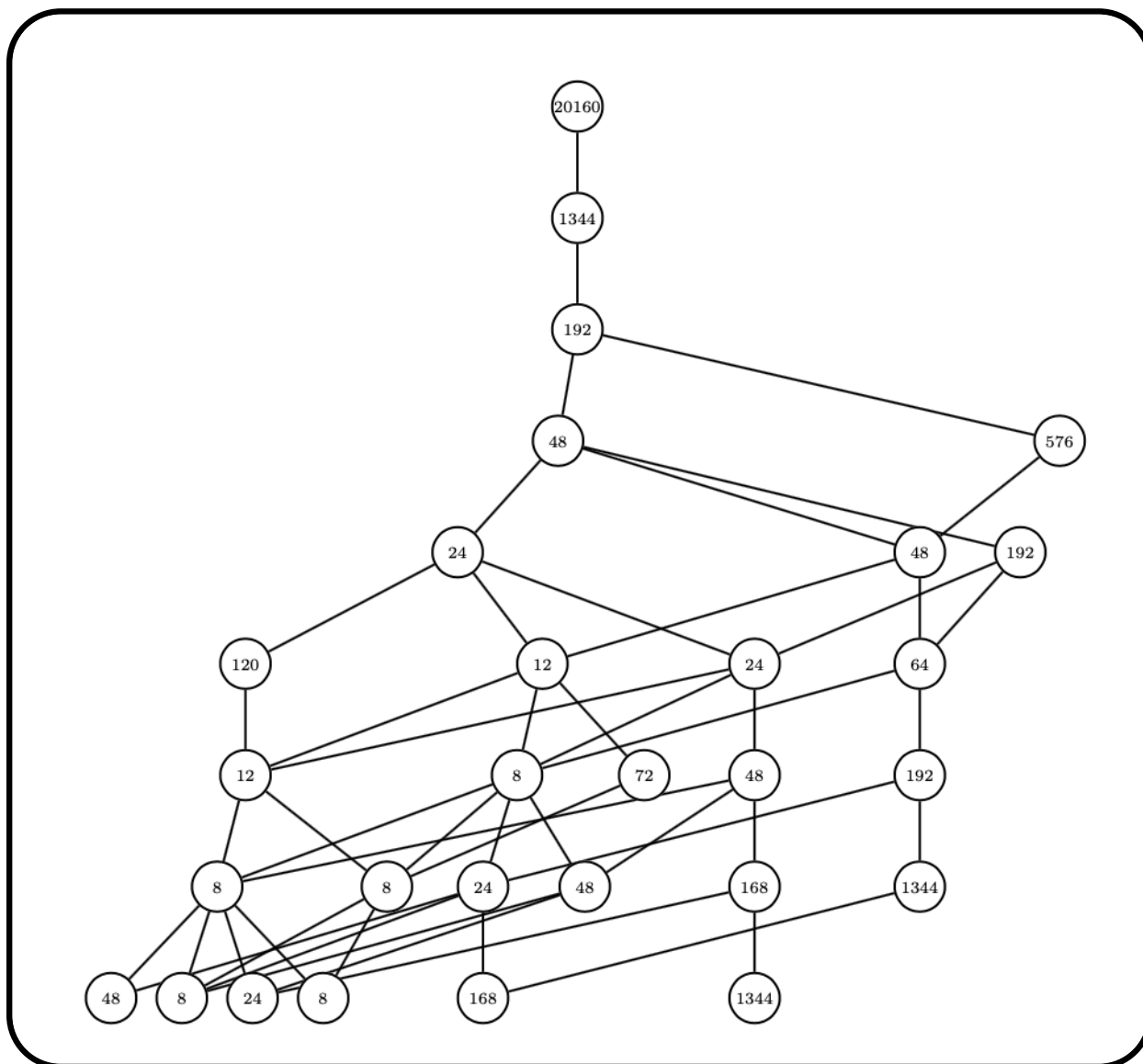
## 10.10 Classification of Optimal Linear Codes

The classification problem of optimal codes in coding theory is the problem of determining the equivalence classes of codes for a given set of values of  $n$  and  $k$  and  $q$  with a lower bound on  $d$ . Orbiter can be used to classify linear codes with given redundancy and bounded minimum distance. The redundancy of a linear  $[n, k]$  code is the parameter  $r = n - k$ . Codes with redundancy  $r$  can be identified with subsets of  $\text{PG}(r-1, q)$ . Under this correspondence, a code with minimum distance at least  $d$  corresponds to a subset such that any  $d-1$  elements are independent. We use the notation  $\Lambda_{r-1,s}(q)$  to denote the poset of subsets of  $\text{PG}(r-1, q)$  for which any  $d-1$ -subset (if any) is independent. Under the correspondence, the action of  $\text{PGL}(r, q)$  on  $\Lambda_{r-1,s}(q)$  corresponds to the orbits of equivalent linear codes. For this reason, we are interested in determining the orbits of  $\text{PGL}(r, q)$  on  $\Lambda_{r-1,s}(q)$ . An orbit of size  $n$  represents an isometry class of  $[n, n-r, d; q]$  codes with  $d \geq s+1$ . The projective stabilizer of the subset is the automorphism group of the code. The Orbiter command

### Example 477

```
codes_8_4_4:
> $(ORBITER) -v 6 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define Control -poset_classification_control \
> > > -problem_label codes_8_4_4 \
> > > -draw_options \
> > > > -embedded -radius 250 \
> > > > -line_width 1.0 -spanning_tree \
> > > -end \
> > -end \
> > -define G \
> > -linear_group -PGL 4 2 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -linear_codes Control 3 8 \
> > -end
> #pdflatex codes_8_4_4_poset_lvl.8.tex
> #$(OPEN) codes_8_4_4_poset_lvl.8.pdf
> #pdflatex codes_8_4_4_poset.tex
> #$(OPEN) codes_8_4_4_poset.pdf
```

classifies linear codes with redundancy 4 and minimum distance at least 4. Orbiter confirms that there is exactly one such code, and it computes the code together with the projective stabilizer. Orbiter creates the action of the group  $\text{PGL}(4, 2)$  on the poset  $\Lambda_{3,3}(2)$ . Using poset classification, Orbiter then produces the poset of orbits shown below



In this diagram, the numbers stand for Orbiter ranks of points in  $\text{PG}(3, 2)$ . All nodes except for the root node have a number attached to it. The nodes represent subsets. In order to determine the set associated to a node, follow the path from the root node to the node and collect the points according to their labels. The root node represents the empty set. The  $[8, 4, 4; 2]$ -code is represented by the set  $\{0, 1, 2, 3, 8, 11, 13, 14\}$ . The fact that there is only one node at level 8 in the poset of orbits tells us that the code is unique up to equivalence. Let us look at the code. The elements of the set  $\{0, 1, 2, 3, 8, 11, 13, 14\}$  are points in  $\text{PG}(3, 2)$ . We write the coordinate vectors in the columns of a matrix  $H$ :

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

This matrix is the parity check matrix  $H$  of the code  $\mathcal{C}$ . This means that the words of the code are the vectors  $\mathbf{c}$  such that  $\mathbf{c} \cdot H^\top = 0$ . Observe that the vectors that we put in the columns of  $H$  all have odd weight. They are in fact the points of the hyperplane  $x + y + z + w = 0$ . This shows that the stabilizer of the code which is the stabilizer of the set is equal to  $\text{AGL}(3, 2)$ , a group of order 1344.



# Chapter 11

## Combinatorics

### 11.1 Introduction

In Tables 11.1 and 11.2, global Orbiter commands for Combinatorics are summarized.

The command

#### Example 478

```
Sym_10_conj_classes:  
▷ $(ORBITER) -v 2 -conjugacy_classes_Sym_n 10  
▷ $(OPEN) classes_Sym_10.csv
```

produces a list of the conjugacy classes of  $\text{Sym}(10)$ . The list is written to a csv file.

The next command computes the character table of the symmetric group  $\text{Sym}(4)$ :

#### Example 479

```
Char_Sym_4:  
▷ $(ORBITER) -v 2 -character_table_symmetric_group 4
```

The command produces the following output:

The character table of  $\text{Sym}(4)$  is the matrix

$$\begin{bmatrix} 1 & -1 & 1 & 1 & -1 \\ 3 & 1 & 0 & -1 & -1 \\ 2 & 0 & -1 & 2 & 0 \\ 3 & -1 & 0 & -1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

Orbiter often uses the lexicographic order to arrange things. Consider the  $k$ -subsets of an  $n$ -set. The following example creates the 3-subsets of a 6-set in lexicographic order. A tree representation is produced. The leaves of the tree represent the subsets:

| Combinatorics (Part 1)         |                 |                                                                                  |
|--------------------------------|-----------------|----------------------------------------------------------------------------------|
| Command                        | Arguments       | Purpose                                                                          |
| -random_permutation            | $n$ fname       | Creates a random permutation in $\text{Sym}(n)$ and stores it in the given file. |
| -create_random_k_subsets       | $n$ $k$ $N$     | Creates $N$ random $k$ -subsets of an $n$ -set.                                  |
| -read_poset_file               | fname           | Reads a poset from the given file.                                               |
| -read_poset_file_with_grouping | fname x-stretch | Reads a poset from the given file and sets stretch factor for orbit grouping.    |
| -list_parameters_of_SRG        | $v_{\max}$      | Performs a sift for putative parameter sets of SRGs.                             |
| -conjugacy_classes_Sym_n       | $n$             | Compute a list of conjugacy classes of $\text{Sym}(n)$ .                         |
| -tree_of_all_k_subsets         | $n$ $k$         | Creates a tree-file for all $k$ -subsets of an $n$ -set.                         |
| -Delandtsheer_Doyen            |                 | See Section 13.4.                                                                |
| -tdo_refinement                |                 | See Section 11.5.                                                                |
| -tdo_print                     |                 | See Section 11.5.                                                                |
| -convert_stack_to_tdo          |                 | See Section 11.5.                                                                |
| -maximal_arc_parameters        |                 | See Section 11.5.                                                                |
| -arc_parameters                |                 | See Section 11.5.                                                                |
| -pentomino_puzzle              |                 |                                                                                  |

Table 11.1: Combinatorics (Part 1)

| Combinatorics (Part 2)               |                                                |                                                                                                                 |
|--------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Command                              | Arguments                                      | Purpose                                                                                                         |
| -make_elementary_symmetric_functions | $n$ $k_{\max}$                                 | Computes the elementary symmetric functions in $n$ variables of degree $1, \dots, k_{\max}$                     |
| -Dedekind_numbers                    | $n_{\min}$ $n_{\max}$<br>$q_{\min}$ $q_{\max}$ | Computes the Dedekind numbers $D_{n,q}$ for $n_{\min} \leq n \leq n_{\max}$ and $q_{\min} \leq q \leq q_{\max}$ |
| -rank_k_subset                       | $n$ $k$ $L$                                    | Computes the ranks of $k$ -subsets chosen from an $n$ -set. $L$ is a list of $k$ -sets taken from an $n$ -set.  |
| -geometry_builder                    |                                                | See Section 11.4.                                                                                               |
| -character_table_symmetric_group     | $n$                                            | Computes the character table of $\text{Sym}(n)$ using the algorithm of Burnside.                                |
| -domino_portrait                     | $D$ $s$ fname                                  | Computes a domino portrait for a graphics file in r/g/b format using double- $D$ domino sets.                   |

Table 11.2: Combinatorics (Part 2)

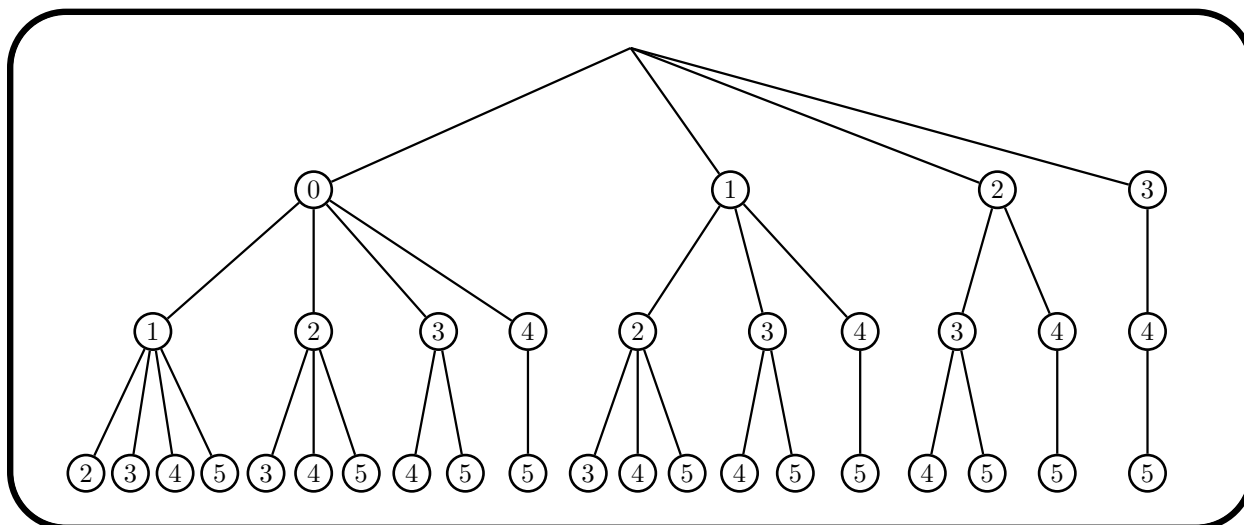
**Example 480**

```

all_subsets_6_3:
> $(ORBITER) -v 2 -tree_of_all_k_subsets 6 3
> $(ORBITER) -v 20 \
> > -draw_options -embedded \
> > > -nodes -radius 80 \
> > > -xin 5000 -yin 5000 \
> > > -xout 1000000 -yout 500000 \
> > > -scale 0.5 -line_width 1.0 \
> > -end \
> > -tree_draw -file all_k_subsets_n6_k3.tree -end
> pdflatex all_k_subsets_n6_k3_draw.tex
> $(OPEN) all_k_subsets_n6_k3_draw.pdf

```

Here is the tree:



The following command illustrates how to create random  $k$ -subsets of a set of size  $n$ . In the example, we create 20 5-subsets of a 10-element set:

**Example 481**

```

random_k_subsets:
> $(ORBITER) -v 4 \
> > -create_random_k_subsets 10 5 20

```

Using the lexicographic order, the  $k$ -subsets of an  $n$ -element set are ranked. The following command computes the ranks of a number of 3-subsets of a 10-element set:

**Example 482**

```

rank_k_subsets_test:
> $(ORBITER) -v 2 \

```

```

> > -define Blocks -vector -format 7 \
> > > -dense "0,1,2,0,3,4,1,3,5,2,4,5,3,6,7,1,6,8,0,6,9" \
> > > -end \
> > -rank_k_subset 10 3 Blocks

```

Orbiter can create the Sylvester type Hadamard matrix of size  $2^n$  (also called the Walsh matrix). The following command creates the matrix of size  $2^4 \times 2^4$  and produces a graphical representation:

#### Example 483

```

Walsh_matrix_4:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 2 -end \
> > -with F -do -finite_field_activity \
> > > -Walsh_matrix 4 -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file Walsh_01_4.csv \
> > -box_width 10 -bit_depth 24 -partition 3 16 16 -end
> #pdflatex GF_2.tex
> #$(OPEN) GF_2.pdf

```

The following command creates the matrix of Dedekind numbers of order at most 10:

#### Example 484

```

Dedekind_10_10:
> $(ORBITER) -v 3 -Dedekind_numbers 2 10 2 10
> $(ORBITER) -v 3 -csv_file_latex 1 Dedekind_2_10_2_10.csv
> pdflatex Dedekind_2_10_2_10.tex
> $(OPEN) Dedekind_2_10_2_10.pdf

```

The following table is produced:

| 2  | 3  | 4    | 5      | 6      | 7       | 8        | 9         | 10        |
|----|----|------|--------|--------|---------|----------|-----------|-----------|
| 2  | 1  | 3    | 6      | 10     | 15      | 21       | 28        | 36        |
| 3  | 2  | 8    | 20     | 40     | 70      | 112      | 168       | 240       |
| 4  | 3  | 18   | 60     | 150    | 315     | 588      | 1008      | 1620      |
| 5  | 6  | 48   | 204    | 624    | 1554    | 3360     | 6552      | 11808     |
| 6  | 9  | 116  | 670    | 2580   | 7735    | 19544    | 43596     | 88440     |
| 7  | 18 | 312  | 2340   | 11160  | 39990   | 117648   | 299592    | 683280    |
| 8  | 30 | 810  | 8160   | 48750  | 209790  | 720300   | 2096640   | 5380020   |
| 9  | 56 | 2184 | 29120  | 217000 | 1119720 | 4483696  | 14913024  | 43046640  |
| 10 | 99 | 5880 | 104754 | 976248 | 6045837 | 28245840 | 107370900 | 348672528 |

The following command creates the elementary symmetric functions in 4 variables.

#### Example 485

```
elementary_symmetric_functions_4:
▷ $(ORBITER) -make_elementary_symmetric_functions 4 4
```

The output is:

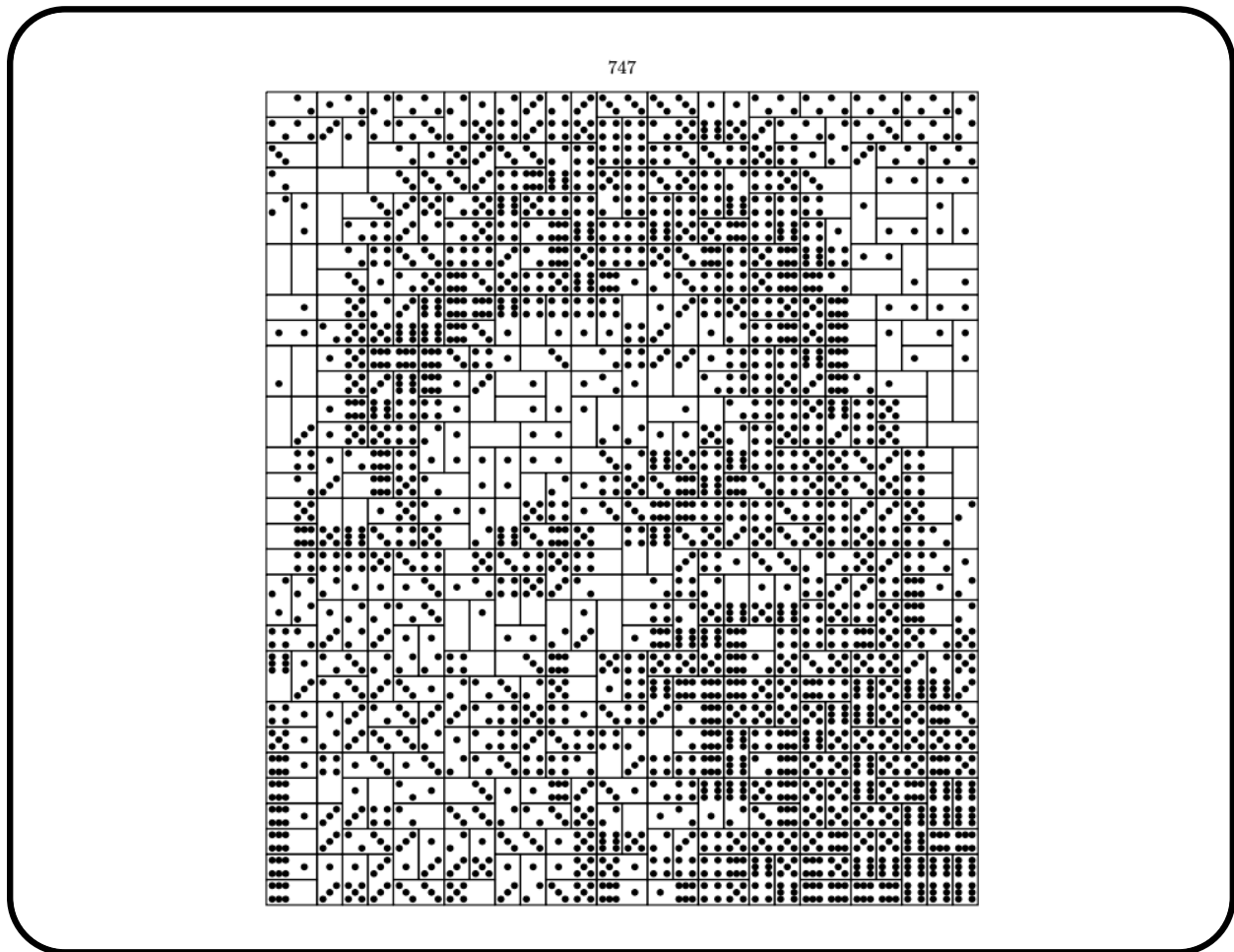
```
k=1 :
x0 + x1 + x2 + x3
k=2 :
x0*x1 + x0*x2 + x0*x3 + x1*x2 + x1*x3 + x2*x3
k=3 :
x0*x1*x2 + x0*x1*x3 + x0*x2*x3 + x1*x2*x3
k=4 :
x0*x1*x2*x3
```

Orbiter can compute Domino portraits. To do so, we need an input file in r/g/b format of size  $(D+1)s \times Ds$ , where  $D = 7$  for double-six dominos.

#### Example 486

```
domino_portrait:
▷ cp $(MY_PATH)/examples/users_guide/anton_28x32.? .
▷ $(ORBITER) -v 3 -domino_portrait 7 4 anton_28x32 -end
```

The portrait is shown below

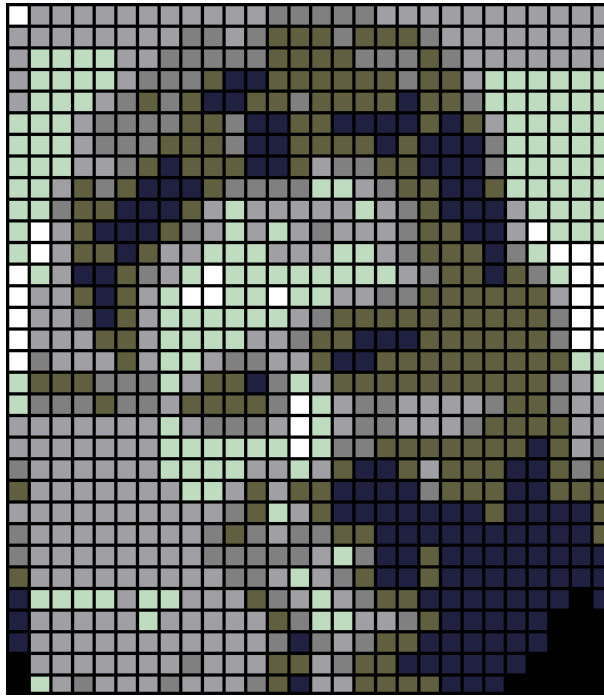


It is possible to compare the domino portrait with a grayscale version of the input image. The following command creates a grayscale image of the input file that was written during the previous command.

#### Example 487

```
domino_portrait_input:
> cp $(MY_PATH)/examples/users_guide/anton_28x32.? .
> $(ORBITER) -v 2 \
> > -define all.one_r -vector -repeat 1 28 -end \
> > -define all.one_c -vector -repeat 1 32 -end \
> > -draw_matrix \
> > > -grayscale \
> > > -invert_colors \
> > > -input_csv_file anton_28x32.m.csv \
> > > -box_width 20 -bit_depth 8 \
> > > -partition 3 \
> > > > all.one_c all.one_r \
> > -end
> $(OPEN) anton_28x32.m_draw.bmp
```

The grayscale version of the input file is



## 11.2 Combinatorial Objects

Combinatorial algorithms often need to process large quantities of combinatorial data. Orbiter facilitates batch processing by means of input streams. An input stream defines a sequence of combinatorial objects, all of the same type. The objects can be defined in files or on the command line, or any combination thereof. Once an input stream is defined, there are commands for activities that can be applied to the objects in the stream.

Table 11.3 shows the commands that are available to define input streams. In Table 11.4, activities for combinatorial objects are listed. Some commands require extra options. Table 11.5 shows options for classification. Table 11.6 shows options for the report which is generated after a classification.

Let us consider some examples. First, we create the Hirschfeld surface in  $PG(3, 4)$ , using point ranks as described in Section 4.2. The 45 points on the surface can be coded as in the following makefile variable:

### Example 488

```
HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS="0,1,2,3,4,5,6,7,8,9,\
10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,\
53,54,59,60,61,62,67,68,69,70,75,76,79,80,81,82"
```

The next command creates a combinatorial object from this set of points:

### Example 489

```
Hirschfeld.q4.from.set:
> $(ORBITER) -v 4 \
> > -define H -set -here \
> > > $(HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS) \
> > -end \
> > -define C -combinatorial_object \
> > > -label Hirschfeld_surface Hirschfeld\_surface \
> > > -set_of_points H \
> > -end
```

The next example creates the two hyperovals in  $PG(2, 16)$ . The hyperovals are stored in two makefile variables:

### Example 490

```
HYPEROVAL_16_144="0, 1, 2, 3, 52, 67, 89, 106, 126, \
141, 159, 176, 184, 199, 220, 235, 245, 262"
```

### Example 491

```
HYPEROVAL_16_16320="0, 1, 2, 3, 52, 70, 83, 109, 127, \
139, 156, 174, 186, 199, 217, 229, 256, 264"
```

The next command creates combinatorial objects for these two hyperovals:



| Defining Combinatorial Objects             |                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                    | Arguments                                 | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| -label                                     | label-txt<br>label-tex                    | A label for the objects.                                                                                                                                                                                                                                                                                                                                                                                                                        |
| -set_of_points                             | $S$                                       | A set $S$ consisting of points.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| -set_of_lines                              | $S$                                       | A set $S$ consisting of lines.                                                                                                                                                                                                                                                                                                                                                                                                                  |
| -set_of_points_and_lines                   | $S_1 S_2$                                 | A set $S_1$ consisting of points and a second set $S_2$ consisting of lines.                                                                                                                                                                                                                                                                                                                                                                    |
| -set_of_packing                            | $S q$                                     | A set $S$ of packings of $\text{PG}(3, q)$ .                                                                                                                                                                                                                                                                                                                                                                                                    |
| -file_of_points                            | fname                                     | A set consisting of points read from file.                                                                                                                                                                                                                                                                                                                                                                                                      |
| -file_of_points_csv                        | fname col-<br>header                      | A csv-file containing sets of points in a specific column. The column header is given.                                                                                                                                                                                                                                                                                                                                                          |
| -file_of_lines                             | fname                                     | A set consisting of lines read from file.                                                                                                                                                                                                                                                                                                                                                                                                       |
| -file_of_packings                          | fname                                     | A set consisting of packings read from file.                                                                                                                                                                                                                                                                                                                                                                                                    |
| -file_of_packings_through_spread_table     | spread-table<br>fname-<br>packings<br>$q$ | A file of packings of $\text{PG}(3, q)$ . This relies on the presence of a spread-table.                                                                                                                                                                                                                                                                                                                                                        |
| -file_of_designs_through_block_orbits      | fname1<br>fname2 $v k$                    | A set of designs (or configurations) built from a set of orbits on $k$ -subsets. The file fname1 is a matrix of zero-one vectors, for instance the solutions of a Diophantine system (see Section 15.3). The file fname2 is a list of orbits of a group on $k$ -subsets created using the -export_something activity from Table 7.2 using the keyword set_orbits. The design will be invariant under the group used to compute the $k$ -orbits. |
| -file_of_point_set                         | fname                                     | A file containing sets of points.                                                                                                                                                                                                                                                                                                                                                                                                               |
| -file_of_designs                           | fname $v b k c$                           | A file containing designs or large sets. Here, $v$ is the number of points in the design, $b$ is the number of blocks, $k$ is the block size, and $c$ is the size of one class of the partition.                                                                                                                                                                                                                                                |
| -file_of_incidence_geometries              | fname $v b f$                             | A file of incidence geometries. The incidence structure is listed as set of flags. Here, $v$ is the number of points, $b$ is the number of blocks and $f$ is the number of flags. If point $i$ and block $j$ are incident, this corresponds to the value $ib + j$ being listed as a flag (recall that $i$ and $j$ are zero-based).                                                                                                              |
| -file_of_incidence_geometries_by_row_ranks | fname $v b r$                             | A file describing incidence geometries defined by their row ranks.                                                                                                                                                                                                                                                                                                                                                                              |
| -incidence_geometry                        | fname $v b f$                             | An incidence geometry defined by a set of flags. Here, $v$ is the number of points, $b$ is the number of blocks and $f$ is the number of flags.                                                                                                                                                                                                                                                                                                 |
| -incidence_geometry_by_row_ranks           | fname $v b r$                             | An incidence geometry defined by row ranks.                                                                                                                                                                                                                                                                                                                                                                                                     |
| -from_parallel_search                      | fname-mask<br>nb_cases<br>cases_fname     |                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

Table 11.3: Defining Combinatorial Objects

| Activities for Combinatorial Objects |            |                                                                     |
|--------------------------------------|------------|---------------------------------------------------------------------|
| Command                              | Arguments  | Purpose                                                             |
| -save                                |            | Save the objects under the name that was assigned automatically.    |
| -save_as                             | fname      | Save the objects under the given name.                              |
| -extract_subset                      | set fname  | Extract a subset of the objects.                                    |
| -line_type_old                       |            | Compute the line type of a set in projective space.                 |
| -line_type                           |            | Compute the line type of a set in projective space.                 |
| -conic_type                          |            | Compute the conic type of a set in projective space.                |
| -non_conical_type                    |            |                                                                     |
| -ideal                               | R          | Compute the ideal of the set, using the ring R.                     |
| -canonical_form_PG                   | P options  | Requires a projective space P. See Tab. 11.5 for options.           |
| -canonical_form                      | options    | Classification by canonical form. See Tab. 11.5 for options.        |
| -report                              | options    | See Tab. 11.6 for options.                                          |
| -draw_incidence_matrices             | prefix     |                                                                     |
| -test_distinguishing_property        | G          | Test the distinguishing property for the graph G.                   |
| -unpack_from_restricted_action       | prefix G   |                                                                     |
| -line_covering_type                  | prefix P L | Computes the line type of the lines in L in the projective space P. |

Table 11.4: Activities for Combinatorial Objects

| Classification of Combinatorial Objects |           |                                                              |
|-----------------------------------------|-----------|--------------------------------------------------------------|
| Command                                 | Arguments | Purpose                                                      |
| -save_classification                    | prefix    | Save classification when finished.                           |
| -max_TDO_depth                          | $d$       | Limit TDO depth to $d$ in the report.                        |
| -save_canonical_labeling                |           | Save the canonical labeling.                                 |
| -save_ago                               |           | Save the automorphism group orders to file.                  |
| -save_transversal                       |           | Save the indices of the elements chosen for the transversal. |

Table 11.5: Classification of Combinatorial Objects

| Options for Reporting After Classification |           |                                                                               |
|--------------------------------------------|-----------|-------------------------------------------------------------------------------|
| Command                                    | Arguments | Purpose                                                                       |
| -export_flag_orbits                        |           | Export flag orbits to csv file.                                               |
| -show_incidence_matrices                   |           | Show incidence matrices in the report.                                        |
| -show_TDO                                  |           | Compute the TDO for the report.                                               |
| -show_TDA                                  |           | Compute the TDA for the report.                                               |
| -export_labels                             |           |                                                                               |
| -export_group_orbiter                      |           |                                                                               |
| -export_group_GAP                          |           |                                                                               |
| -lex_least                                 | geo       | Show lex-least orbit representatives. Here, geo is a geometry builder object. |

Table 11.6: Options for Reporting After Classification

**Example 492**

```

hyperoval_16_create:
▷ $(ORBITER) -v 2 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label hyperoval_q16 hyperoval\_q16 \
▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
▷ ▷ -end \

```

In the next example, we read the points of an elliptic curve from file (see Section 4.2):

**Example 493**

```

EC_read: elliptic_curve_b1_c3.q11.txt
▷ $(ORBITER) -v 4 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label EC EC \
▷ ▷ ▷ -file_of_points elliptic_curve_b1_c3.q11.txt \
▷ ▷ -end

```

In the next example, we read a packing, using a previously defined table of spreads, stored in a csv file.

**Example 494**

```

PG_3.5_assume_31_read:
▷ $(ORBITER) -v 2 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label packings_25 packings_25 \
▷ ▷ ▷ -file_of_packings_through_spread_table \
▷ ▷ ▷ ▷ H31_packings.csv \
▷ ▷ ▷ ▷ SPREAD_TABLES_5_REG/spread_25_spreads.csv \
▷ ▷ ▷ ▷ 5 \
▷ ▷ -end

```

The following command reads a file of large sets of designs:

#### Example 495

```
LS_AG_2_3_read:
▷ $(ORBITER) -v 2 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label designs designs \
▷ ▷ ▷ -file_of_designs \
▷ ▷ ▷ solutions.csv 9 84 3 12 \
▷ ▷ -end
```

The next command reads incidence geometries from a file containing the flags:

#### Example 496

```
geo_7_3_read:
▷ $(ORBITER) -v 10 \
▷ ▷ -draw_incidence_structure_description \
▷ ▷ ▷ -width 60 -width_10 6 -end \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label geo_7_3 geo\7\3 \
▷ ▷ ▷ -file_of_incidence_geometries \
▷ ▷ ▷ 7_3.inc 7 7 21 \
▷ ▷ -end
```

The next command creates incidence geometries from a file containing row-ranks:

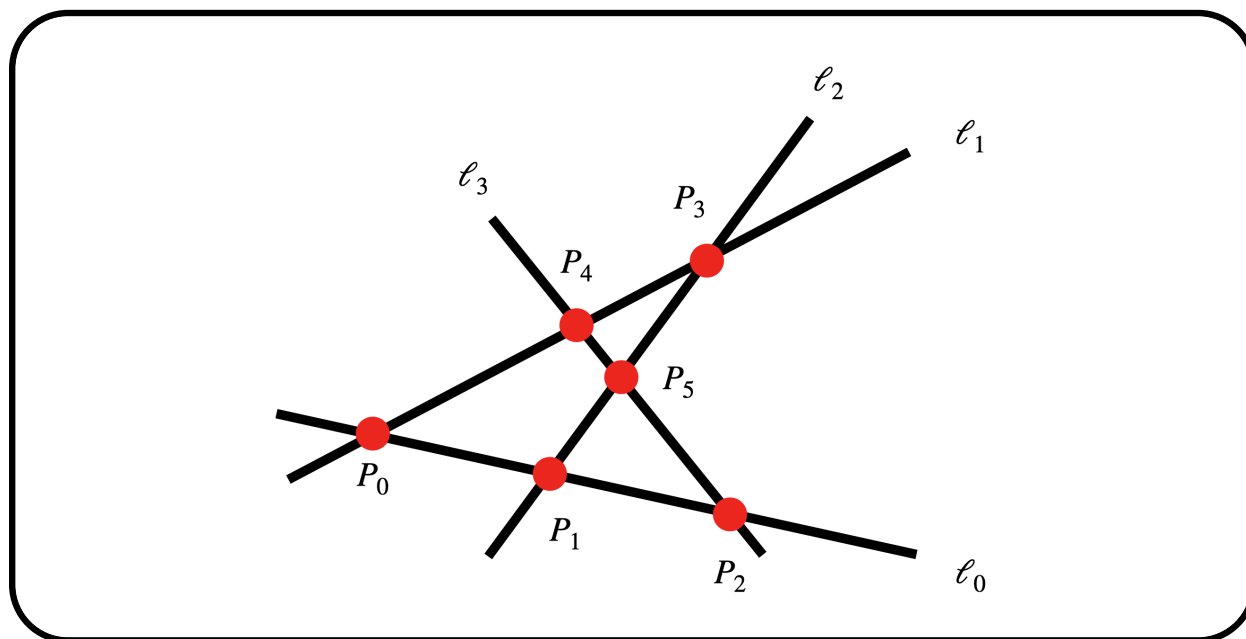
#### Example 497

```
Desargues_path_lex_least_read:
▷ echo $(DESARGUES_PATH_LEX_LEAST) >Desargues_path_lex_least.inc
▷ $(ORBITER) -v 10 \
▷ ▷ -draw_incidence_structure_description \
▷ ▷ ▷ -width 60 -width_10 6 -end \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label Desargues_path_lex_least Desargues\path\lex\_least \
▷ ▷ ▷ -file_of_incidence_geometries_by_row_ranks \
▷ ▷ ▷ Desargues_path_lex_least.inc 10 10 3 \
▷ ▷ -end
```

Combinatorial objects can be stored in text files. There can be any number of objects in one file. The objects themselves are coded. For instance, a set of points in projective space is given as a set of integers, using the Orbiter point ranks. Likewise, a set of lines is coded using Orbiter line ranks. For designs, there are several ways in which the object can be stored. One way is by listing the incidences (flags) in a numerical form. An incidence between point  $i$  and line  $j$  is recorded as  $ib + j$ . Here,  $i$  and  $j$  are the zero based index values of the point and line pair, and  $b$  is the number of lines. Another way is by recording the flags in a row of the incidence matrix as a subset. In this case, the number of incidences per row must be constant, say  $r$ . The flags in a given row thus comprise an  $r$ -subset of a set of size  $b$ , and the ranking of subsets is used to encode the flags in one row. Proceeding in a row-by-row fashion, the incidence matrix is encoded as a vector of ranks of length  $v$ . Yet another way is by listing the columns of the incidence matrix, provided the number

of incidences is constant across columns, say  $k$ . In this case, we encode the flags in one column as a rank of a  $k$ -subset of a set of size  $v$ .

The Pasch configuration is the configuration of 6 points and 4 lines shown below:



In the geometry, we have 4 lines, which we can identify with the index sets of the points as  $\{0, 1, 2\}$ ,  $\{0, 3, 4\}$ ,  $\{1, 3, 5\}$  and  $\{2, 4, 5\}$ . The incidence matrix of the configuration is

|       | $\ell_0$ | $\ell_1$ | $\ell_2$ | $\ell_3$ |
|-------|----------|----------|----------|----------|
| $P_0$ | 1        | 1        | 0        | 0        |
| $P_1$ | 1        | 0        | 1        | 0        |
| $P_2$ | 1        | 0        | 0        | 1        |
| $P_3$ | 0        | 1        | 1        | 0        |
| $P_4$ | 0        | 1        | 0        | 1        |
| $P_5$ | 0        | 0        | 1        | 1        |

The rows correspond to the points, the columns correspond to the lines. The labels of points and lines are shown.

How to encode an incidence structure for computer storage? There are three ways to encode the incidence structure. One way encodes the flags of the geometry. This will be described next. The flag space is the set of all possible flags in the incidence matrix between the given number of points and lines. The space is totally ordered using the row-major ordering

|       | $\ell_0$ | $\ell_1$ | $\ell_2$ | $\ell_3$ |
|-------|----------|----------|----------|----------|
| $P_0$ | 0        | 1        | 2        | 3        |
| $P_1$ | 4        | 5        | 6        | 7        |
| $P_2$ | 8        | 9        | 10       | 11       |
| $P_3$ | 12       | 13       | 14       | 15       |
| $P_4$ | 16       | 17       | 18       | 19       |
| $P_5$ | 20       | 21       | 22       | 23       |

The Pasch configuration can now be coded as

$$\{0, 1, 4, 6, 8, 11, 13, 14, 17, 19, 22, 23\}.$$

The file `pasch.inc` contains:

```
6 4 12
0 1 4 6 8 11 13 14 17 19 22 23
-1 1
24
```

The first line lists the number of rows and columns of the incidence matrix, and the number of incidences. The geometry is encoded on the next line. After that, a marker of -1 shows that this is the only geometry in this file (the file format allows for any number of incidence geometries, all with the same parameters). The final row is the order of the automorphism group of the geometry. This row is optional. In case that there are several geometries in the file, the orders will all be listed. In this case, the possible values will be collected with multiplicities, and listed in decreasing order. The command

#### Example 498

```
geo.pasch.read:
> $(ORBITER) -v 10 \
> > -define C -combinatorial_object \
> > > -label Pasch Pasch \
> > > -file_of_incidence_geometries \
> > > > pasch.inc 6 4 12 \
> > -end
```

reads the incidence geometry from the file `pasch.inc`. It is also possible to enter the incidence geometry directly from the command line. The following example uses the `-incidence_geometry` command to do so:

#### Example 499

```
geo.pasch.given:
> $(ORBITER) -v 10 \
> > -define C -combinatorial_object \
> > > -label Pasch Pasch \
> > > -incidence_geometry \
> > > > "0,1,4,6,8,11,13,14,17,19,22,23" \
> > > > 6 4 12 \
```

▷ ▷ -end

### 11.3 Combinatorial Linear Spaces

A linear space is a pair  $(S, \mathcal{L})$  where  $S$  is a set and  $\mathcal{L}$  is a set of subsets of  $S$  such that each set  $L \in \mathcal{L}$  satisfies  $|L| \geq 2$  and moreover for any two  $a, b \in S$  there is exactly one element  $L \in \mathcal{L}$  such that both  $a$  and  $b$  belong to  $L$ . The usual notions of isomorphism and automorphism apply. For finite linear spaces, a first combinatorial property is the number  $a_i$  which counts the number of sets  $L \in \mathcal{L}$  of size  $i$ . The vector  $(a_2, \dots, a_n)$  is the line type of  $(S, \mathcal{L})$ . The equation

$$\binom{n}{2} = \sum_{j=2}^n a_j \binom{j}{2} \quad (11.1)$$

is satisfied. The equation can be used to generate all possible line types of a putative linear space. Here is an example. For  $|S| = 6$ , (11.1) becomes

$$x_0 \binom{6}{2} + x_1 \binom{5}{2} + x_2 \binom{4}{2} + x_3 \binom{3}{2} + x_4 \binom{2}{2} = \binom{6}{2}.$$

Here,  $(x_0, x_1, \dots, x_4)$  is the line type of a putative linear space on 6 points. That is,  $x_i = a_{6-i}$  is the number of lines of size  $6 - i$ . The extended coefficient matrix of the system is

$$[15 \ 10 \ 6 \ 3 \ 1 \mid 15].$$

The Orbiter command

#### Example 500

```
linsp6:
▷ $(ORBITER) -v 4 \
▷ ▷ -define A -vector -format 1 -dense "15,10,6,3,1" -end \
▷ ▷ -define D -diophant -label linsp6 \
▷ ▷ -coefficient_matrix A \
▷ ▷ -RHS "mult=1,EQ=15" \
▷ ▷ -x_min_global 0 \
▷ ▷ -x_max_global 15 \
▷ ▷ -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -diophant_activity -solve_mckay \
▷ ▷ -end
▷
# Found 15 solutions with 22 backtrack steps
```

solves the system using McKay's program possolve [51]. The program finds 15 solutions, written to the file `linsp6.sol`.

Let us consider a problem from [11]. Suppose we are interested in linear spaces on 30 points and with line type  $(7, 5^{27}, 4^{24})$ . This notation means that we assume one 7-line, 27 5-lines and 24 4-lines. The type of a point  $P$  is the set of integers

$$p_j = \#j\text{-lines through } P.$$

We are trying to precompute the matrix of point types

$$(p_{ij})$$

where  $j = 7, 5, 4$  and  $i$  belongs to an index set of all possible point types. Fixing a point  $P$ , counting points  $Q \neq P$  collinear with  $P$  yields

$$6p_7 + 4p_5 + 3p_4 = 29, \quad p_7 \leq 1, \quad p_5 \leq 27, \quad p_4 \leq 24.$$

Using the Orbiter commands



**Example 501**

```

linsp30_pt.types:
> $(ORBITER) -v 4 \
> > -define A -vector -format 1 -dense "6,4,3" -end \
> > -define D -diophant \
> > > -label linsp30_pt.types \
> > > -coefficient_matrix A \
> > > -RHS "mult=1,EQ=29" -x_bounds "0,1,0,27,0,24" \
> > -end \
> > -with D -do \
> > > -diophant_activity -solve_mckay \
> > -end

```

we determine the possibilities

$$(p_7, p_5, p_4) = \begin{cases} 1 & 5 & 1 \\ 1 & 2 & 5 \\ 0 & 5 & 3 \\ 0 & 2 & 7 \end{cases}$$

The rows in this matrix are called the point types ( $i = 0, 1, 2, 3$ ). Let  $b_i$  be the number of points of type  $i$ . By counting points, incident (point, line) pairs by  $j$ -lines and pairs of intersecting  $j$ -lines, we arrive at the following system:

$$\begin{aligned}
b_0 + b_1 + b_2 + b_3 &= 30 \\
b_0 + b_1 &= 7 \\
5b_0 + 2b_1 + 5b_2 + 2b_3 &= 135 = 27 \cdot 5 \\
b_0 + 5b_1 + 3b_2 + 7b_3 &= 96 = 24 \cdot 4 \\
10b_0 + b_1 + 10b_2 + b_3 &\leq 351 = \binom{27}{2} \\
10b_1 + 3b_2 + 21b_3 &\leq 276 = \binom{24}{2}
\end{aligned}$$

Using the Orbiter commands

**Example 502**

```

linsp30_pt.distribution:
> $(ORBITER) -v 4 \
> > -define A -vector -format 6 -dense \
> > > "1,1,1,1,1,1,0,0,5,2,5,2,1,5,3,7,10,1,10,1,0,10,3,21" \
> > -end \
> > -define D -diophant \
> > > -label linsp30_pt.distribution \
> > > -coefficient_matrix A \
> > > -RHS "mult=1,EQ=30" \
> > > -RHS "mult=1,EQ=7" \
> > > -RHS "mult=1,EQ=135" \
> > > -RHS "mult=1,EQ=96" \
> > > -RHS "mult=1,LE=351" \
> > > -RHS "mult=1,LE=276" \

```

```

▷ ▷ ▷ -x_min_global 0 -x_max_global 30 \
▷ ▷ -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -diophant_activity -solve_mckay \
▷ ▷ -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -diophant_activity -draw_as_bitmap 20 8 \
▷ ▷ -end

```

we determine the possibilities

$$(b_0, b_1, b_2, b_3) = \begin{cases} 2 & 5 & 23 & 0 \\ 3 & 4 & 22 & 1 \\ 4 & 3 & 21 & 2 \\ 5 & 2 & 20 & 3 \\ 6 & 1 & 19 & 4 \\ 7 & 0 & 18 & 5 \end{cases}$$

## 11.4 Classification of Configurations and Geometries

A partial linear space is a set system on a fixed set  $V$ . We write  $\mathcal{L} = (V, \mathcal{B})$ , where  $\mathcal{B}$  is a set of distinct subsets of  $V$ , called lines. The members of  $V \cup \mathcal{B}$  are called elements. For two elements  $x, y$ , we say that  $x$  is incident with  $y$ , written  $xIy$ , if either  $x \in y$  or  $y \in x$ . We require that any line has at least two points and that any two points are contained in at most one line. A *decomposition* of a linear space is a partition  $\Pi = (C_1, \dots, C_n)$  of  $V \cup \mathcal{B}$  such that each  $C_i$  either is a subset of  $V$  or a subset of  $\mathcal{B}$ . A decomposition is called *tactical* if for all  $i$ , the incidence number

$$\iota(C_i, C_j) = \#\{y \in C_j, xIy\}$$

does not depend on the choice of  $x \in C_i$ . Any linear space has a tactical decomposition, as the discrete partition (every element is in its own class) is tactical. Let  $\text{Aut}(\mathcal{L})$  be the automorphism group of the linear space, which is the subgroup of  $\text{Sym}(V)$  which preserves incidence. For  $\alpha \in \text{Aut}(\mathcal{L})$ , we say that the decomposition  $\Pi$  preserves  $\alpha$  if  $\alpha$  fixes every class of  $\Pi$ . For  $A \leq \text{Aut}(\mathcal{L})$ , we say that  $\Pi$  preserves  $A$  if  $\Pi$  preserves every element  $\alpha \in A$ . Mostly, we are interested in those decompositions  $\Pi$  which preserve  $\text{Aut}(\mathcal{L})$ . In light of this, the discrete decomposition is not that interesting.

Any linear space has a coarsest tactical decomposition that preserves its automorphism group: The orbit partition of the automorphism group acting on  $V \cup \mathcal{B}$  will do. Up to ordering of the classes, the coarsest tactical refinement is unique. Computing the orbit decomposition is challenging as it involves computing the automorphism group. Computationally, there are easier ways to get to admissible decompositions. One way is by means of successive refinements. If a class  $C_i$  does not have the property that  $\iota(C_i, C_j)$  is well-defined for all  $x \in C_i$ , then a refinement of  $C_i$  will do. The coarsest refinement of  $C_i$  has the property that if  $C_i$  preserves some group  $A$  then the refinement will do, too. This shows that there is an algorithm to compute a tactical decomposition of any given linear space  $\mathcal{P}$ . Simply start with the decomposition of two classes, one the set of points and one the set of blocks, and refine. The output may or may not be equal to the decomposition arising from the orbit partition of  $\text{Aut}(\mathcal{L})$ .

Let us consider the opposite question. Given a tactical decomposition, can we classify the linear spaces which admit a given tactical decomposition. The `-geometry_builder` option can help. Table 11.7 shows the options for the geometry builder.

The command

### Example 503

```
geo_10_3:
▷ $(ORBITER) -v 2 \
▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
▷ ▷ -define Geo -geometry_builder \
▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
▷ ▷ ▷ -fname_GEO 10_3 \
▷ ▷ ▷ -output_to_inc_file \
▷ ▷ ▷ -output_to_sage_file \
▷ ▷ ▷ -output_to_blocks_file \
▷ ▷ ▷ -test Test_lines \
▷ ▷ -end
```

classifies the configurations  $10_3$ . It uses isomorphism tests after 4, 5, 6, 7, 8, 9 and 10 points. The positions of the tests is defined using a set called `Test_lines`. The set of test lines is defined using a loop command. The command shows that there are exactly 10 configurations of this kind. One of them is the Desargues configuration. Four different output files can be written. Each contains all geometries, but the file format is different.

| Constructing and Classifying Geometries |           |                                                                                                                                  |
|-----------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| Command                                 | Arguments | Purpose                                                                                                                          |
| -V                                      | part      | The initial partition of points (rows).                                                                                          |
| -B                                      | part      | The initial partition of blocks (columns).                                                                                       |
| -TDO                                    | tdo       | The initial row-tactical decomposition scheme.                                                                                   |
| -fuse                                   | fuse      | The partition of row classes.                                                                                                    |
| -girth_test                             | $g$       | Require the girth of the collinearity graph to be at least $g$ .                                                                 |
| -lambda                                 | $\lambda$ | Set $\lambda$ for two-design test. Every pair of points is required to lie in $\lambda$ blocks.                                  |
| -no_square_test                         |           | Disable the test for linear spaces.                                                                                              |
| -simple                                 |           | Construct simple designs only (needs -lambda)                                                                                    |
| -search_tree                            |           | Write a file containing the search tree (at the level of rows of the partial geometry).                                          |
| -search_tree_flags                      |           | Write a file containing the search tree (at the level of flags of the partial geometry). A flag is an incident point-block pair. |
| -orderly                                |           | Apply orderly generation.                                                                                                        |
| -special_test_not_orderly               |           | Apply the special test. This option only applies to generation which is not orderly.                                             |
| -split                                  | $l\ r\ m$ | Split the search tree at line $l$ . After $l$ lines, continue only the cases which are congruent to $r$ modulo $m$ .             |
| -fname_GEO                              | fname     | Set the output file name base (no extension).                                                                                    |
| -output_to_inc_file                     |           | Set output to inc file.                                                                                                          |
| -output_to_sage_file                    |           | Set output to sage file.                                                                                                         |
| -output_to_blocks_file                  |           | Set output to a file containing the blocks in coded form.                                                                        |
| -output_to_blocks_latex_file            |           | Set output to a file containing the blocks in latex.                                                                             |

Table 11.7: Constructing and Classifying Geometries

1. The option `-output_to_inc_file` writes `10_3.inc`. The file contains the incidences in increasing order. The position in the incidence matrix is given. Apart from the header and footer, each row in the file describes exactly one geometry. The incidences are given in numeric form. Each incidence is the numerical position of the point/block pair in the incidence matrix. The position is the numbering of the matrix entries in the incidence matrix in row-major ordering, starting with zero for the top left entry. The index of the incidence in row  $i$  (zero-based) and column  $j$  is  $b \cdot i + j$ , where  $b$  is the number of blocks in the geometry. The header consists of one row, containing the number of points, the number of lines, and the number of incidences. The footer consists of two rows. A single `-1` indicates the beginning of the footer. The second row in the footer summarizes the automorphism group orders of the geometries. Here is an example: The file `10_3.inc` lists the 10 configurations  $10_3$ :

```

10 10 30
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 53 58 62 66 69 74 78 79 85 87 89 96 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 54 58 62 66 69 73 78 79 85 87 89 96 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 54 58 62 67 69 73 76 79 85 88 89 96 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 56 58 62 67 69 73 78 79 84 86 89 95 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 47 48 52 54 57 62 66 68 73 77 79 84 86 89 95 98 99
0 1 2 10 13 14 20 25 26 31 33 35 41 47 48 52 54 57 62 66 68 73 78 79 84 86 89 95 97 99
0 1 2 10 13 14 20 25 26 31 33 35 41 47 48 52 54 57 62 66 69 73 78 79 84 86 88 95 97 99
0 1 2 10 13 14 20 25 26 31 33 37 41 45 48 52 54 57 62 66 68 73 75 79 84 86 89 97 98 99
0 1 2 10 13 14 20 25 26 31 33 37 41 45 48 52 54 57 62 66 68 73 75 79 84 88 89 96 97 99
0 1 2 10 13 14 20 25 26 31 33 37 41 45 48 52 54 57 62 66 68 73 76 79 84 88 89 95 97 99
-1 10
120, 24, 12, 10, 6, 4^2, 3^2, 2

```

2. The option `-output_to_sage_file` writes `10_3.sage`. This file is meant to be read by Sage [63].
3. The option `-output_to_blocks_file` writes `10_3.blocks`. Here is the content of the file `10_3.blocks`:

```

10 10 3
0 15 26 44 51 68 81 109 114 116
0 15 26 46 49 68 81 109 114 116
0 15 26 46 49 68 83 106 115 116
0 15 26 46 52 69 77 106 114 116
0 15 26 46 56 69 80 101 106 119
0 15 26 46 56 69 80 103 104 119
0 15 26 46 56 69 80 103 107 117
0 15 26 46 56 72 80 93 106 119
0 15 26 46 56 72 81 93 105 119
0 15 26 46 56 74 79 93 105 119
-1 10
120, 24, 12, 10, 6, 4^2, 3^2, 2

```

contains the blocks. Each number represents the rank of a 3-subset corresponding to a block in the lexicographic ordering of all 3-subsets.

4. The option `-output_to_blocks_latex_file` writes `10_3.blocks_long`. The file `10_3.blocks_long` contains a list of all blocks written out in a format ready for use in latex.

It is possible to create graphical representations of the search tree. The additional option `-search_tree` can be given, as shown in the following command:

**Example 504**

```

geo_10_3.tree:
> $(ORBITER) -v 20 \
> > -define Test_lines -set -loop 0 11 1 -end \
> > -define GEO -geometry_builder \
> > > -V 10 -B 10 -TDO 3 -fuse 1 \
> > > -fname_GEO 10_3 \
> > > -output_to_inc_file \
> > > -search_tree \
> > > -test Test_lines \
> > -end
> $(ORBITER) -v 20 \
> > -draw_options -embedded -radius 40 \
> > > -paperheight 220 \
> > > -paperwidth 330 \
> > > -xin 10000 -yin 10000 \
> > > -xout 1000000 -yout 500000 \
> > > -scale 2 -line_width 0.3 \
> > > -nodes_empty \
> > -end \
> > -tree_draw \
> > > -file 10_3_tree.txt \
> > -end
> pdflatex 10_3_tree.draw.tex
> $(OPEN) 10_3_tree.draw.pdf

```

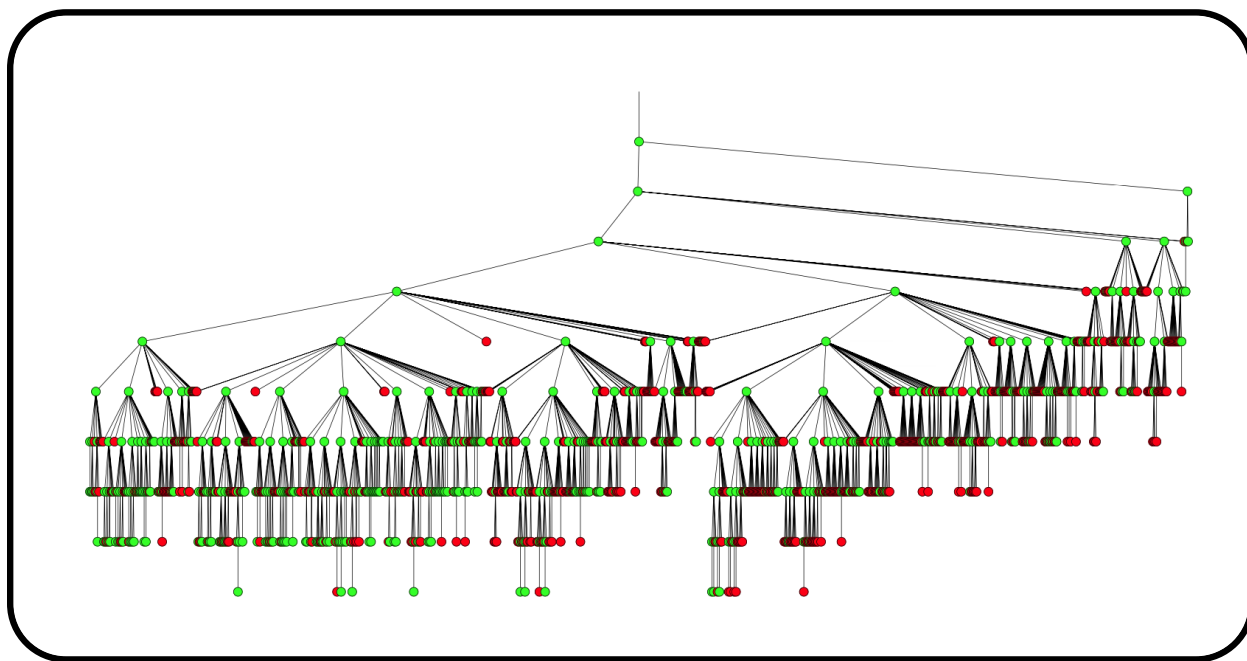
The `-search_tree` option causes Orbiter to create a file containing the search tree. The name of the file is derived from the file name given with the `fname_GEO` option. Here, the `fname_GEO` option sets the output file to `10_3`. The `-search_tree` option then creates the file `10_3_tree.txt`. Once the classification is completed, the `-tree_draw` command can be used to create a drawing of the search tree from the file just created. The vertex color represents the outcome of the isomorphism test. A green node is accepted. A red node is rejected. The search will continue for the green nodes only. A green node at the bottom of the tree corresponds to an isomorphism type of a geometry satisfying all the requirements. In the example of the configurations  $10_3$ , there are 10 green nodes at the very bottom of the search tree. They represent the 10 isomorphism types of configurations  $10_3$ . The size of the tree can be determined by counting the lines of the file `10_3_tree.txt` and subtracting one:

```
wc 10_3_tree.txt
```

The word count command yields the following output:

```
1471 13543 37633 10_3_tree.txt
```

This means that there are 1471 lines in the file. Hence the search tree has 1470 nodes. The resulting tree is shown below



Any incidence structure defines a graph on its underlying set of points. The vertices of the collinearity graph are the points of the incidence structure. Two vertices are adjacent if the incidence structure contains a block which contains the associated points. The distance between two points in the geometry is the distance of the associated vertices in the collinearity graph. The girth is the length of the shortest cycle. Incidence structures with high girth are often interesting. Orbiter allows classifying incidence structures with a given girth. More specifically, Orbiter classifies all incidence structures whose girth is at least a given number, or proves that there are none. This is interesting because the girth can be used as a filter, and classifying with a given girth may be faster than classifying all incidence geometries. For instance, the following example classifies all cubic graphs on 10 vertices with girth at least 5:

#### Example 505

```
geo.petersen:
> $(ORBITER) -v 8 \
> > -define Test_lines -set -loop 3 11 1 -end \
> > -define Geo -geometry_builder \
> > > -V 10 -B 15 -TDO 3 -fuse 1 \
> > > -fname_GEO petersen -girth 5 \
> > > -output_to_inc_file \
> > > -search_tree \
> > > -test Test_lines \
> > -end
```

There is a unique graph with these properties. It is the Petersen graph. Its automorphism group is  $\text{Sym}(5)$  of order 120.

Let us look at the girth of configurations. For instance, while there are 245342 isomorphism classes of configurations  $15_3$ , only one of them has girth 4. This is the Cremona Richmond configuration with automorphism group  $\text{Sym}(6)$  of order 720. It is associated to a cubic surface. Let us compare the classification with and without girth condition. The following command classifies all configurations  $15_3$ :

**Example 506**

```

15_3.inc:
> $(ORBITER) -v 2 \
> > -define Test_lines -set -loop 4 16 1 -end \
> > -define Geo -geometry_builder \
> > > -V 15 -B 15 -TDO 3 \
> > > -fuse 1 -fname_GEO 15_3 \
> > > -output_to_inc_file \
> > > -test Test_lines \
> > > -special_test_not_orderly \
> > -end

```

The next command classifies the configurations  $15_3$  with girth 4.

**Example 507**

```

geo_15_3.g4:
> $(ORBITER) -v 2 \
> > -define Test_lines -set -loop 4 16 1 -end \
> > -define Geo -geometry_builder \
> > > -V 15 -B 15 -TDO 3 \
> > > -fuse 1 -fname_GEO 15_3.g4 \
> > > -output_to_inc_file \
> > > -girth 4 \
> > > -search_tree \
> > > -special_test_not_orderly \
> > > -test Test_lines \
> > -end
> $(ORBITER) -v 2 \
> > -draw_options -embedded -radius 50 \
> > > -nodes_empty \
> > > -scale 0.5 -line_width 0.3 -end \
> > -tree_draw -file 15_3.g4_tree.txt -end
> pdflatex 15_3.g4_tree_draw.tex
> $(OPEN) 15_3.g4_tree_draw.pdf

```

The next command classifies the configurations  $40_4$  with girth 4. Exactly two configuration arise, both with a group of order 51840. Note the extra option `-special_test_not_orderly` to speed up the search.

**Example 508**

```

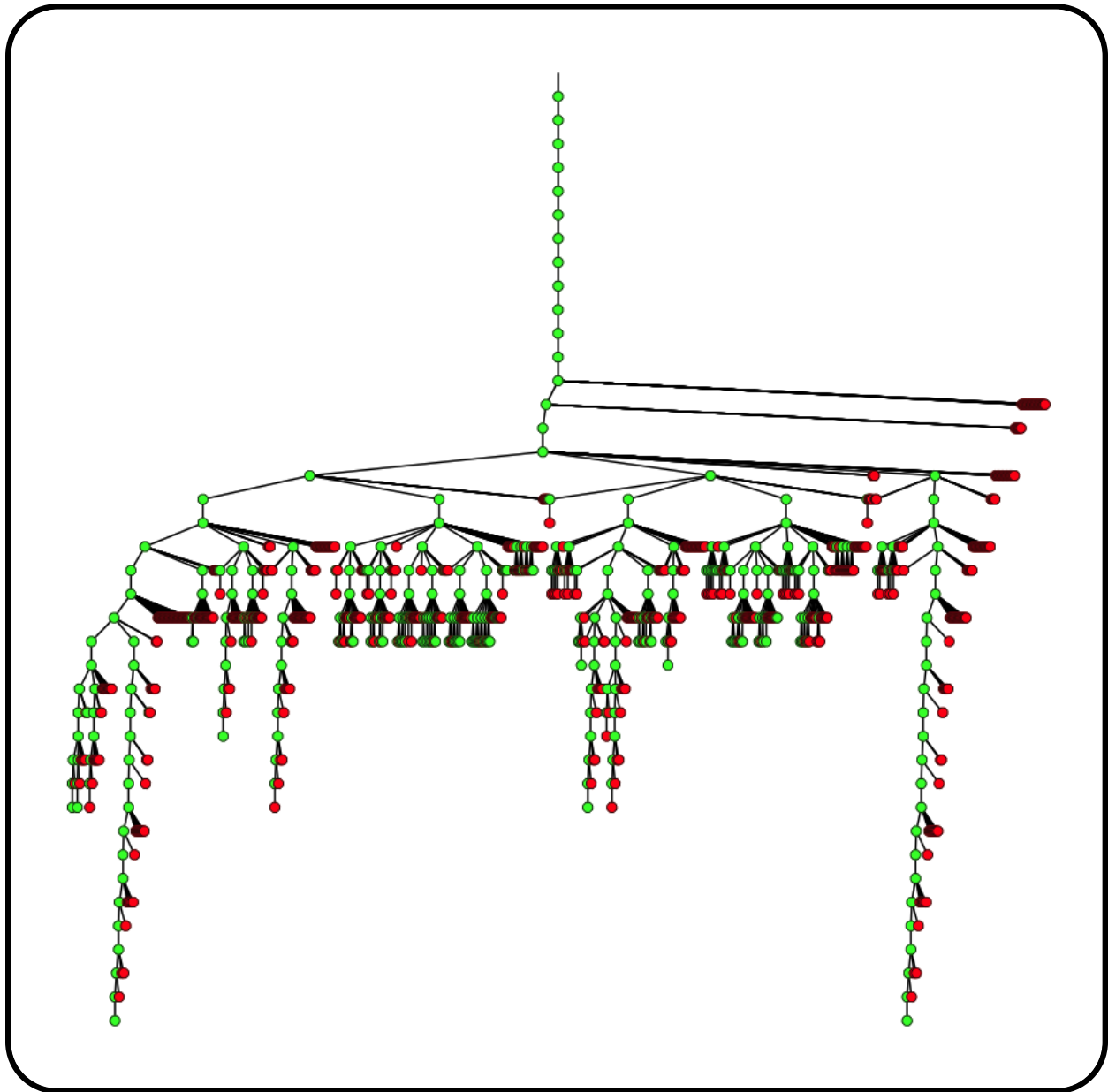
geo_40_4.g4:
> $(ORBITER) -v 5 \
> > -define Test_lines -set -loop 0 41 1 -end \
> > -define Geo -geometry_builder \
> > > -V 40 -B 40 -TDO 4 \
> > > -fuse 1 \
> > > -fname_GEO 40_4.g4 \
> > > -girth 4 \

```



```
▷ ▷ ▷ -search_tree \  
▷ ▷ ▷ -special_test_not_orderly \  
▷ ▷ ▷ -test Test_lines \  
▷ ▷ ▷ -output_to_sage_file \  
▷ ▷ ▷ -output_to_inc_file \  
▷ ▷ -end  
▷ $(ORBITER) -v 2 \  
▷ ▷ -draw_options -embedded -radius 50 \  
▷ ▷ ▷ -xin 10000 -yin 10000 \  
▷ ▷ ▷ -xout 1000000 -yout 1000000 \  
▷ ▷ ▷ -nodes_empty \  
▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \  
▷ ▷ -tree_draw -file 40_4_g4_tree.txt -end  
▷ pdflatex 40_4_g4_tree_draw.tex  
▷ $(OPEN) 40_4_g4_tree_draw.pdf
```

The search tree is shown below



The next command classifies the configurations  $63_3$  with girth 6. Exactly two configuration arise, both with a group of order 12096. Note the extra option `-special_test_not_orderly` to speed up the search.

#### Example 509

```
geo_63_3.g6:
> $(ORBITER) -v 2 \
> > -define Test_lines -set -loop 4 64 1 -end \
> > -define Geo -geometry_builder \
> > > -V 63 -B 63 -TDO 3 \
> > > -fuse 1 -fname_GEO 63_3.g6 \
> > > -girth 6 \
> > > -test Test_lines \
```

```

> > > -special_test_not_orderly \
> > > -output_to_sage_file \
> > > -output_to_inc_file \
> > > -search_tree \
> > -end

```

The configuration is also known as the split Cayley hexagon.

The next command classifies the Latin squares of order 6.

#### Example 510

```

geo.LSQ6.inc:
> $(ORBITER) -v 2 \
> > -define Test_lines -set -loop 1 19 1 -end \
> > -define Geo -geometry_builder \
> > > -V 6,6,6 -B 1,1,1,36 -TDO \
> > > "1,0,0,6, 0,1,0,6, 0,0,1,6" \
> > > -fuse 3 \
> > > -fname_GEO LSQ6 \
> > > -output_to_inc_file \
> > > -test Test_lines \
> > -end

```

Up to isomorphism, there are exactly 12 such objects.

| Refining Tactical Decompositions |                 |                                                                                                                     |
|----------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------|
| Command                          | Arguments       | Purpose                                                                                                             |
| -lambda3                         | $\lambda_3$ $s$ | Refine as 3-design with $\lambda_3$ and with block-size $s$                                                         |
| -solution                        | $i$ fname       | Use solutions to system $i$ from file fname.                                                                        |
| -range                           | $f$ $l$         | Refine cases $i$ with $f \leq i < f + l$ only.                                                                      |
| -select                          | label           | Select the case for refinement by label.                                                                            |
| -o1                              | $s$             | Omit $s$ variables from the first refinement system.                                                                |
| -o2                              | $s$             | Omit $s$ variables from the second refinement system.                                                               |
| -D1_upper_bound_x0               | $b$             | Add the bound $x_0 \leq b$ in the first refinement.                                                                 |
| -reverse                         |                 | Sort the distributions in reverse order.                                                                            |
| -reverse_inverse                 |                 |                                                                                                                     |
| -nopacking                       |                 | Do not use packing inequalities.                                                                                    |
| -dual_is_linear_space            |                 | Assume that the dual incidence structure is a linear space also. This is valid for projective planes, for instance. |
| -geometric_test                  |                 | Subject the distributions to the geometric test.                                                                    |
| -once                            |                 | Find at most one refinement in each case. This can be used to test which cases can be refined.                      |
| -mckay                           |                 | Use McKay's solver instead (by default, a lexicographic solver is used).                                            |
| -input_file                      | fname           | Specify the input TDO-file for refinement.                                                                          |

Table 11.8: Refining Tactical Decompositions

## 11.5 Tactical Decompositions

Table 11.8 lists the Orbiter commands for decomposition refinement.

Suppose we want to study projective planes of order 16. It is a linear space with  $16^2 + 16 + 1 = 273$  points and equally many lines. Each point lies on 17 lines and each line contains 17 points. Any two points lie on exactly one line and any two lines intersect in exactly one point.

We decide to study maximal arcs of degree 4 in this plane (the degree has to divide the order of the plane). A maximal arc of degree  $d$  is a set of points so that each line intersects in either  $d$  or zero points. A line which intersects in  $d$  points is called a secant. A line which intersects in no point is called an external line. The command

### Example 511

```
max_arc_16_4.start:
▷ $(ORBITER) -v 4 -maximal_arc_parameters 16 4
```

creates a decomposition stack for the parameters of the arc and writes the file `max_arc_q16_r4.stack`

```

<HTDO type=pt ptanz=2 btanz=2 fuse=simple>
      221      52
    52      17      0
    221      13      4

    1  1
</HTDO>

```

This is a point-tactical decomposition with 2 point-classes and 2 block-classes. The point classes are associated with the rows. The block-classes are associated with the columns. The first row and column indicates the size of the classes. The entries  $a_{ij}$  count the number of blocks in the column class  $j$  that are incident with a given point in the  $i$ th row class. The fuse information at the bottom (1 1) is a partition of the row classes which indicates the ancestor decomposition which was column tactical. The next step is to convert the stack file to a tdo file. The command

#### Example 512

```

max_arc_16_4.convert_stack.tdo:
> $(ORBITER) -v 4 -convert_stack_to_tdo max_arc_q16_r4.stack

```

does that. It creates the file max\_arc\_q16\_r4.tdo. It also prints the decomposition stack:

```

lambda_scheme at level 2 :
is 1 x 1
      | 273_{ 1}
=====
273_{ 0} |

row_scheme at level 4 :
is 2 x 2
      | 221_{ 1} 52_{ 2}
=====
52_{ 0} |      17      0
221_{ 3} |      13      4

col_scheme at level 3 :
is 1 x 2
      | 221_{ 1} 52_{ 2}
=====
273_{ 0} |      17      17

```

Next, we can compute all coarsest column-tactical refinements of the decomposition. To this end, the command

#### Example 513

```

max_arc_16_4.refine:
> $(ORBITER) -v 4 -tdo_refinement \
> > -input_file max_arc_q16_r4.tdo -dual_is_linear_space -end
>

```

```
max_arc_16_4r_print:
▷ $(ORBITER) -v 4 -tdo_print max_arc.q16_r4r.tdo
```

is used. Because the incidence structure is a projective plane, the dual is a linear space also. Hence the option `-dual_is_linear_space` can be used, which is helpful to reduce possibilities. As it turns out, there is exactly one refinement, and it is tactical. The file `max_arc.q16_r4r.tdo` is produced. Note the added letter `r` at the end of the file name (`r` for refinement). We can use the following command to display the decomposition stack in the file:

#### Example 514

```
max_arc_16_4r_print:
▷ $(ORBITER) -v 4 -tdo_print max_arc.q16_r4r.tdo
```

This produces the following output:

```
decomposition 0.1:
lambda_scheme at level 2 :
is 1 x 1
      | 273_{ 1}
=====
273_{ 0} |

row_scheme at level 4 :
is 2 x 2
      | 221_{ 1} 52_{ 2}
=====
52_{ 0} |      17      0
221_{ 3} |      13      4

col_scheme at level 4 :
is 2 x 2
      | 221_{ 1} 52_{ 2}
=====
52_{ 0} |      4      0
221_{ 3} |      13     17

extra_col_scheme at level 3 :
is 1 x 2
      | 221_{ 1} 52_{ 2}
=====
273_{ 0} |      17     17
```

# Chapter 12

## Graph Theory

### 12.1 Creating Graphs

Tables 12.1-12.2 list Orbiter commands to create graphs.

Let us consider some examples. The command

#### Example 515

```
Cycle_graph_13:
> $(ORBITER) -v 2 \
> > -define Gamma -graph \
> > > -cycle 13 \
> > -end
```

creates the cycle graph of degree 13.

There are many ways to read graphs from file. One way is by means of an adjacency matrix stored as a csv file. Consider an example. The `-load_csv_no_border` command can be used to create a graph from a csv file containing the adjacency matrix. The following command sequence uses a makefile variable to store the adjacency matrix of a graph. The matrix is then copied into a file and the graph is created from the file. Here is the makefile variable containing the adjacency matrix:

#### Example 516

```
TRIANGLE_GRAPH="0,1,1\n1,0,1\n1,1,0\n"
```

The following command writes the adjacency matrix to a csv file and then creates the graph from the csv file:

#### Example 517

```
make_triangle_graph:
> echo $(TRIANGLE_GRAPH) >triangle_graph.csv
> $(ORBITER) -v 6 \
> > -define G -graph \
> > > -load_csv_no_border \
> > > triangle_graph.csv \
```

| Creating Graphs (Part 1)                         |                    |                                                                                                                                                      |
|--------------------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                          | Arguments          | Purpose                                                                                                                                              |
| -load                                            | filename           | Read a graph from file in Orbiter format.                                                                                                            |
| -load_csv_no_border                              | filename           | Read a graph from a csv file. The first row and the first column are part of the adjacency matrix.                                                   |
| -load_adjacency_matrix_from_csv_and_select_value | filename $v$       | Read a graph from a csv file. The first row and the first column are ignored. Entries equal to $v$ define adjacencies. All other values are ignored. |
| -load_dimacs                                     | filename           | Read a graph from file in dimacs format.                                                                                                             |
| -edge_list                                       | $n$ list-of-edges  | Create a graph on $n$ vertices from a list of edges as ranked pairs.                                                                                 |
| -edges_as_pairs                                  | $n$ edges-as-pairs | Create a graph on $n$ vertices from a list of edges as pairs.                                                                                        |
| -cycle                                           | $n$                | Cycle graph on $n$ vertices.                                                                                                                         |
| -inversion_graph                                 | text               | Read inversion graph.                                                                                                                                |
| -Hamming                                         | $n$ $q$            | Hamming graph $H(n, q)$                                                                                                                              |
| -Johnson                                         | $n$ $k$ $s$        | Johnson graph                                                                                                                                        |
| -Paley                                           | $F$                | Paley graph over the field $F = \mathbb{F}_q$ .                                                                                                      |
| -Sarnak                                          | $p$ $q$            | Lubotzky-Phillips-Sarnak graph [47]                                                                                                                  |
| -Schlaefli                                       | $F$                | Schlaefli graph                                                                                                                                      |
| -Shrikhande                                      |                    | Shrikhande graph                                                                                                                                     |
| -Winnie_Li                                       | $F$ $i$            | Winnie-Li graph (see [46]) over the field $F = \mathbb{F}_q$ with subfield index $i$ .                                                               |
| -Grassmann                                       | $n$ $k$ $F$ $r$    | Grassmann graph over $F = \mathbb{F}_q$ .                                                                                                            |
| -coll_orthogonal                                 | $O$ $S$            | Collinearity graph of the set $S$ in the orthogonal geometry $O$ .                                                                                   |
| -triedral_pair_disjointness_graph                |                    | Trihedral pair disjointness graph.                                                                                                                   |
| -non_attacking_queens_graph                      | $n$                | Create the graph for non-attacking queens on a $n \times n$ chess board.                                                                             |

Table 12.1: Creating Graphs (Part 1)



| Creating Graphs (Part 2) |                          |                                                                                                                                                                            |
|--------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                  | Arguments                | Purpose                                                                                                                                                                    |
| -subset                  | label labeltex<br>subset | Define vertex coloring with two colors based on a subset of vertices.                                                                                                      |
| -disjoint_sets_graph     | fname                    | Reads a set of sets from the given file. Define a graph on the sets. Vertices correspond to the given sets. Two vertices are adjacent if the associated sets are disjoint. |
| -orbital_graph           | $G$ $i$                  | Define orbital graph from the $i$ -th orbit of the group $G$ acting on pairs.                                                                                              |
| -collinearity_graph      | inc-matrix               | Collinearity graph of the given incidence matrix.                                                                                                                          |
| -chain_graph             | P1 P2                    | Chain graph with respect to the partitions P1 and P2.                                                                                                                      |
| -Cayley_graph            | $G$ gens                 | Cayley graph with respect to group $G$ and generating set gens.                                                                                                            |

Table 12.2: Creating Graphs (Part 2)

```
▷ ▷ -end
```

This will create the three-cycle graph.

The command

#### Example 518

```
Chain.232:
▷ $(ORBITER) -v 2 \
▷ ▷ -define P1 -vector -dense 2,3,2 -end \
▷ ▷ -define P2 -vector -dense 2,3,2 -end \
▷ ▷ -define Gamma -graph \
▷ ▷ ▷ -chain_graph P1 P2 \
▷ ▷ -end
```

creates the chain graph with respect to the partitions  $(2, 3, 2)$  and  $(2, 3, 2)$ .

The command

#### Example 519

```
Paley.13_graph:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define Gamma -graph -Paley F -end \
```

creates the Paley graph on 13 vertices. The command first creates the field  $\mathbb{F}_{13}$  and then the associated Paley graph.

The command

#### Example 520

```
Winnie_Li_graph_q16_index2:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 16 -end \
> > -define Gamma -graph -Winnie_Li F 2 -end \
```

creates the Winnie Li graph [46] on 16 vertices with index 2. The command first creates the field  $\mathbb{F}_{16}$  and then the associated graph.

The command

#### Example 521

```
Sarnak_graph_29_5:
> $(ORBITER) -v 2 \
> > -define Gamma -graph -Sarnak 29 5 -end \
```

creates the graph  $X_{29,5}$  from [47] on 60 vertices.

The command

#### Example 522

```
trihedral_pair_graph:
> $(ORBITER) -v 2 \
> > -define Gamma \
> > > -graph -trihedral_pair_disjointness_graph \
> > -end
```

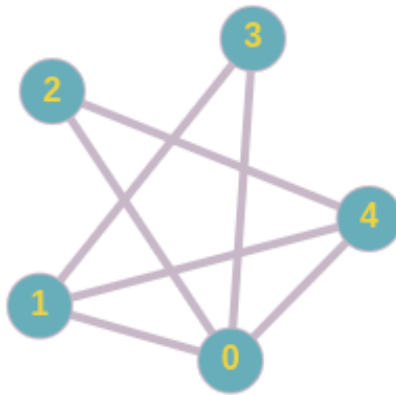
creates the graph of trihedral pairs in a general cubic surface with 27 lines. Two vertices are adjacent if the associated trihedral pairs are line-disjoint.

The command

#### Example 523

```
small_graph:
> $(ORBITER) -v 2 \
> > -define G -graph -edges_as_pairs \
> > > 5 "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
> > -end
```

creates a graph by listing the edges in pairs. The following graph is created:



The command

#### Example 524

```
petersen:
> $(ORBITER) -v 2 \
> > -define G -graph -Johnson 5 2 0 -end
```

creates the Petersen graph.

The command

#### Example 525

```
Johnson_6_2_0:
> $(ORBITER) -v 2 \
> > -define G -graph -Johnson 6 2 0 -end
```

creates the Johnson graph  $J(6, 2, 0)$ .

The command

#### Example 526

```
Hamming_graph_3:
> $(ORBITER) -v 2 \
> > -define G -graph -Hamming 3 2 -end
```

creates the Hamming graph of order 3.

The command

**Example 527**

```
Op_6.2_coll_graph:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 13 -end \
> > -define O -orthogonal_space 1 6 F -end \
> > -define v -vector -loop 0 35 1 -end \
> > -define Gamma -graph -coll_orthogonal O v -end \
> > -with Gamma -do \
> > > -graph_theoretic_activity -save \
> > -end
```

creates the collinearity graph of the orthogonal space  $O^+(6, 2)$ .

There is a graph on 315 vertices that arises from the Cohen-Tits near octagon (see [17]). The graph was first constructed in [20] and has automorphism group equal to  $\text{Aut}(HJ)$ , the automorphism group of the Higman-Sims sporadic simple group. The graph is distance-regular. An incidence matrix can be found in Ascii format on the web site [7]. In the following, we assume that a file `halljanko315.csv` is present, containing the incidence matrix of the graph. The following command creates the graph from the file:

**Example 528**

```
HJ_graph:
> cp $(MY_PATH)/examples/users_guide/halljanko315.csv .
> $(ORBITER) -v 6 \
> > -define G -graph \
> > > -load_csv_no_border \
> > > halljanko315.csv \
> > -end
```

In Section 16.7, we will compute the automorphism group of the graph (of order 1209600). This will create a file `halljanko315_gens.csv` which we use here to create an orbital graph. An orbital graph is a graph whose adjacency matrix corresponds to an orbit of a permutation group in the action on pairs. The group is the automorphism group of the graph. The following command creates the third orbital graph:

**Example 529**

```
Group.Perm315_Orbital_3.colored_graph: halljanko315_gens.csv
> $(ORBITER) -v 2 \
> > -define gens -vector -format 5 -file \
> > > halljanko315_gens.csv -end \
> > -define G -permutation_group \
> > > -bsgs halljanko315 "File\halljanko315" \
> > > 315 1209600 "0,1,2" 5 gens \
> > -end \
> > -define Gamma -graph \
> > > -orbital_graph G 3 \
> > -end \
> > -with Gamma -do \
> > > -graph_theoretic_activity -save \
> > -end
```

| Graph Modifiers |           |                                                                                                            |
|-----------------|-----------|------------------------------------------------------------------------------------------------------------|
| Command         | Arguments | Purpose                                                                                                    |
| -complement     |           | Complementary graph.                                                                                       |
| -distance_2     |           | Distance two graph: Two vertices are adjacent if and only if they have distance two in the original graph. |

Table 12.3: Graph Modifiers

Table 12.3 shows some Orbiter commands to modify graphs. The commands replace the given graph by the graph obtained after applying the specified modification.

For a graph  $\Gamma$ , the distance 2 graph  $\Delta$  has the same vertices as  $\Gamma$ , and two vertices in  $\Delta$  are adjacent if and only if the distance in  $\Gamma$  is two. The following command creates the distance 2 graph of the Cohen-Tits graph.

**Example 530**

```
HJ_d2_graph:
> $(ORBITER) -v 6 \
> > -define G -graph \
> > > -load_csv_no_border \
> > > halljanko315.csv \
> > > -distance_2 \
> > -end
```

Let us look at some examples of Cayley graphs. The first graph has  $G = \mathbb{Z}_{11}$  and connection set all elements congruent 1 mod 3. We create the group as a subgroup of the one-dimensional affine group over  $\mathbb{F}_{11}$ . This means that the map  $x \mapsto ax + b \bmod 11$  is encoded as a pair  $(a, b)$ .

**Example 531**

```
Cayley_Z11_1mod3:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 11 -end \
> > -define S -vector -dense \
> > > "1,1, 1,4, 1,7, 1,10" -end \
> > -define G -linear_group -AGL 1 F \
> > > -subgroup_by_generators "Z11" 11 1 "1,1" \
> > -end \
> > -define Gamma -graph \
> > > -Cayley_graph G S \
> > -end
```

The vertices of the Cayley graph are ordered. The ordering is determined by the stabilizer chain. This is a total ordering.

In the following example, we create a Cayley graph based on the symmetric group on 4 things. We take the Coxeter generators as connection set:

**Example 532**

```

Cayley_Sym4_coxeter:
> $(ORBITER) -v 2 \
> > -define S -vector -dense "1,0,2,3, 0,2,1,3, 0,1,3,2" -end \
> > -define G -permutation_group -symmetric_group 4 \
> > -end \
> > -define Gamma -graph \
> > > -Cayley_graph G S \
> > -end

```

The star graph is another Cayley graph for the symmetric group. The connection set is given by the permutations  $(0, i)$  for  $i = 1, \dots, n - 1$ . The next example creates the star graph on 4 vertices:

**Example 533**

```

Cayley_Sym4_star:
> $(ORBITER) -v 2 \
> > -define S -vector -dense "1,0,2,3, 2,1,0,3, 3,1,2,0" -end \
> > -define G -permutation_group -symmetric_group 4 \
> > -end \
> > -define Gamma -graph \
> > > -Cayley_graph G S \
> > -end

```

| Graph Theoretic Activities |              |                                                                      |
|----------------------------|--------------|----------------------------------------------------------------------|
| Command                    | Arguments    | Purpose                                                              |
| -find_cliques              | options      | Find all cliques. See Section 15.1.                                  |
| -export_magma              |              | Export to Magma [16].                                                |
| -export_maple              |              | Export to Maple [50].                                                |
| -export_csv                |              | Export to csv-file.                                                  |
| -export_graphviz           |              | Export to graphviz-file.                                             |
| -print                     |              | Print the graph.                                                     |
| -sort_by_colors            |              | Sort the vertices by color classes.                                  |
| -split                     | file         | Split the graph into subgraphs.                                      |
| -split_by_starters         | file         | Split the graph into subgraphs according to starters.                |
| -split_by_clique           | label clique | Compute the neighborhood graph of the given clique.                  |
| -save                      |              | Save the graph to file in binary format.                             |
| -automorphism_group        |              | Compute the automorphism group and write a report. See Section 16.7. |
| -properties                |              | Compute properties of the graph.                                     |
| -eigenvalues               |              | Compute the eigenvalues of the graph.                                |
| -draw                      |              | Draw the graph.                                                      |

Table 12.4: Graph Theoretic Activities

## 12.2 Graph Theoretic Activities

Graph theoretic activities allow us to perform tasks on graphs. Table 12.4 shows the commands for graph theoretic activities. These are activities that can be applied to objects of type graph.

Let us continue the example of the three-cycle from Section 12.1. The command

### Example 534

```
triangle_graph_properties:
▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
▷ $(ORBITER) -v 6 \
▷ ▷ -define G -graph \
▷ ▷ ▷ -load_csv_no_border \
▷ ▷ ▷ triangle_graph.csv \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ ▷ -graph_theoretic_activity -properties \
▷ ▷ -end
```

computes the degree type of the graph. This is the distribution of degrees in the degree sequence of the graph. It prints the distribution of degree values in exponential notation. The multiplicities are indicated as exponent. For instance, the graph in this example has three vertices of degree 2, so the degree type is written as  $2^3$ .

The following command exports the adjacency matrix and creates a bitmap drawing of it.

**Example 535**

```

Cycle_13_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -define Gamma -graph -cycle 13 -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -export_csv -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -export_graphviz -end
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file Cycle_13.csv \
▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
▷ dot -Tpng Cycle_13.gv >Cycle_13.png
▷ #twopi -Tpng Cycle_13.gv >Cycle_13.png
▷ #$(OPEN) Cycle_13_draw.bmp
▷ #pdflatex Cycle_13_report.tex
▷ #$(OPEN) Cycle_13_report.pdf

```

The command first creates the cycle graph of order 13, and then exports the adjacency matrix as csv file. It then draws the adjacency matrix as a bitmap graphics.

Suppose we want to compute the eigenvalues of the adjacency matrix of a graph. In the following example, the command `-eigenvalues` is used to compute the eigenvalues (both regular and Laplace) of the 9-cycle:

**Example 536**

```

Cycle_9_eigenvalues:
▷ $(ORBITER) -v 2 \
▷ ▷ -define Gamma -graph \
▷ ▷ ▷ -cycle 9 \
▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -eigenvalues -end
▷ pdflatex Cycle_9_eigenvalues.tex
▷ #$(OPEN) Cycle_9_eigenvalues.pdf

```

The following output is produced:

| $i$ | $\lambda_i$ | $\theta_i$       |
|-----|-------------|------------------|
| 0   | 2           | 3.87939          |
| 1   | 1.53209     | 3.87939          |
| 2   | 1.53209     | 3                |
| 3   | 0.347296    | 3                |
| 4   | 0.347296    | 1.6527           |
| 5   | -1          | 1.6527           |
| 6   | -1          | 0.467911         |
| 7   | -1.87939    | 0.467911         |
| 8   | -1.87939    | $-2.26243e - 16$ |



The energy is 11.5175  
 Eigenvalues:  $\lambda_i$   
 Laplace eigenvalues:  $\theta_i$

The command

#### Example 537

```
Paley_13_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define Gamma -graph -Paley F -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -export_csv -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -export_graphviz -end
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file Paley_13.csv \
▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
▷ dot -Tpng Paley_13.gv >Paley_13.png
▷ $(OPEN) Paley_13_draw.bmp
```

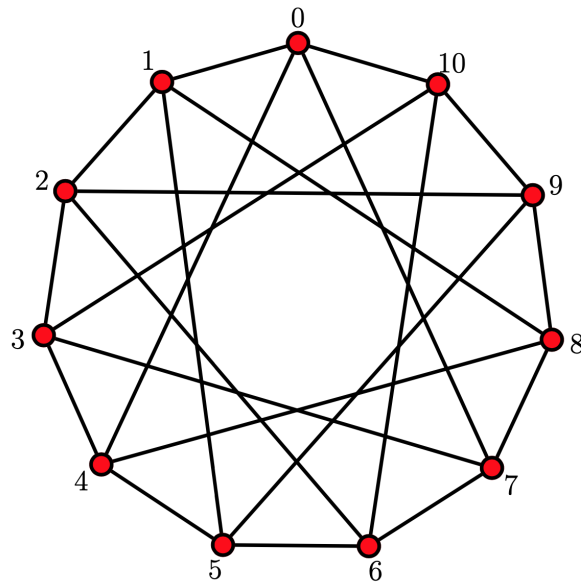
draws the Paley graph of order 13 created in Section 12.1 using the external tool graphviz.

Let us consider the Cayley graphs from Section 12.1. Here is a command that draws the first graph and computes the eigenvalues:

#### Example 538

```
Cayley_Z11_1mod3_eigenvalues_and_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_options -xin 2000000 \
▷ ▷ ▷ -yin 2000000 -embedded -radius 20000 -end \
▷ ▷ -define F -finite_field -q 11 -end \
▷ ▷ -define S -vector -dense \
▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
▷ ▷ -define G -linear_group -AGL 1 F \
▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
▷ ▷ -end \
▷ ▷ -define Gamma -graph \
▷ ▷ ▷ -Cayley_graph G S \
▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -eigenvalues -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -draw -end
▷ pdflatex Cayley_graph_AGL_1.11_draw.tex
▷ $(OPEN) Cayley_graph_AGL_1.11_draw.pdf
```

The drawing is shown below



Let us draw the Cayley graph of  $\text{Sym}(5)$  with respect to the Coxeter generators:

#### Example 539

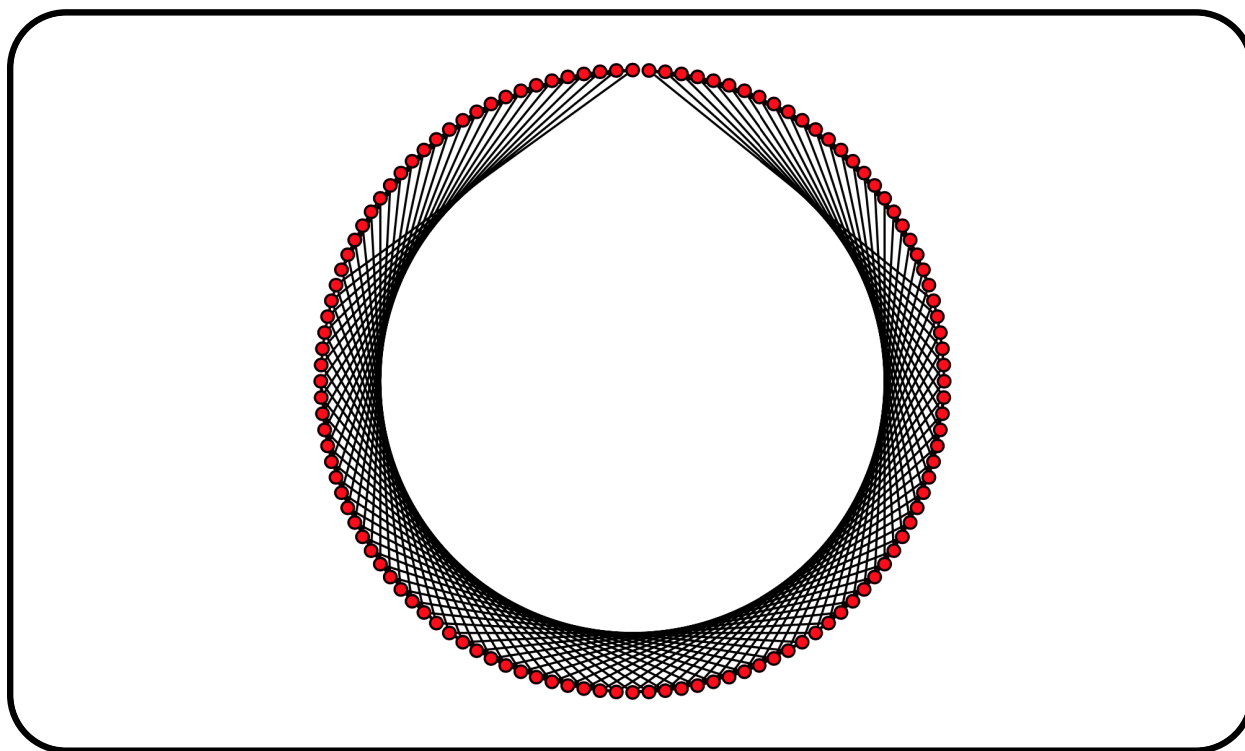
Cayley\_Sym5\_coxeter\_draw:

```

> $(ORBITER) -v 2 \
> > -draw_options -xin 1000000 -yin 1000000 \
> > > -embedded -radius 10000 -nodes_empty -end \
> > -define S -vector -dense \
> > > "1,0,2,3,4, 0,2,1,3,4, 0,1,3,2,4, 0,1,2,4,3" -end \
> > -define G -permutation_group -symmetric_group 5 \
> > -end \
> > -define Gamma -graph \
> > > -Cayley_graph G S \
> > -end \
> > -with Gamma -do \
> > -graph_theoretic_activity -draw -end
> pdflatex Cayley_graph_Sym_5.draw.tex
> $(OPEN) Cayley_graph_Sym_5.draw.pdf

```

The drawing is shown below



It is possible to create the collinearity graph of an incidence structure. The incidence structure must be encoded by means of an incidence matrix. Let us continue an example from Section 4.7, where the incidence matrix of  $Q(4, 2)$  was created. At that time, we wrote the incidence matrix to file. Here, we read the incidence matrix from file and create the collinearity graph of it:

#### Example 540

```
PG0_5_2_collinearity_graph: 0_5_2_incidence_matrix.csv
> $(ORBITER) -v 3 \
> > -define Inc -vector -file 0_5_2_incidence_matrix.csv -end \
> > -define Gamma -graph -collinearity_graph Inc -end \
> > -with Gamma -do \
> > -graph_theoretic_activity \
> > > -properties \
> > -end
```

The command also computes properties of the graph. The graph has 15 vertices of degree 6. This is because in the geometry, each point lies on three lines, and hence is collinear with 6 other points.

| Classifying Graphs |           |                                                       |
|--------------------|-----------|-------------------------------------------------------|
| Command            | Arguments | Purpose                                               |
| -regular           | $r$       | Regular of degree $r$                                 |
| -n                 | $n$       | Number of vertices $n$                                |
| -girth             | $d$       | Girth at least $d$                                    |
| -tournament        |           | Classify tournaments.                                 |
| -no_transmitter    |           | Tournament without transmitter (requires -tournament) |
| -test_multi_edge   |           |                                                       |
| -identify          |           |                                                       |
| -depth             |           |                                                       |

Table 12.5: Classifying Graphs

| Activities after Graph Classification       |           |         |
|---------------------------------------------|-----------|---------|
| Command                                     | Arguments | Purpose |
| -draw_level_graph                           | $k$       |         |
| -draw_graphs                                |           |         |
| -list_graphs_at_level                       | min max   |         |
| -draw_graphs_at_level                       | $k$       |         |
| -draw_options                               |           |         |
| -recognize_graphs_from_adjacency_matrix_csv |           |         |

Table 12.6: Activities after Graph Classification

## 12.3 Classification of Graphs and Tournaments

Table 12.5 lists the Orbiter commands to classify graphs and tournaments. Table 12.6 lists activities after the classification has been completed.

The following command classifies all graphs on 5 vertices:

### Example 541

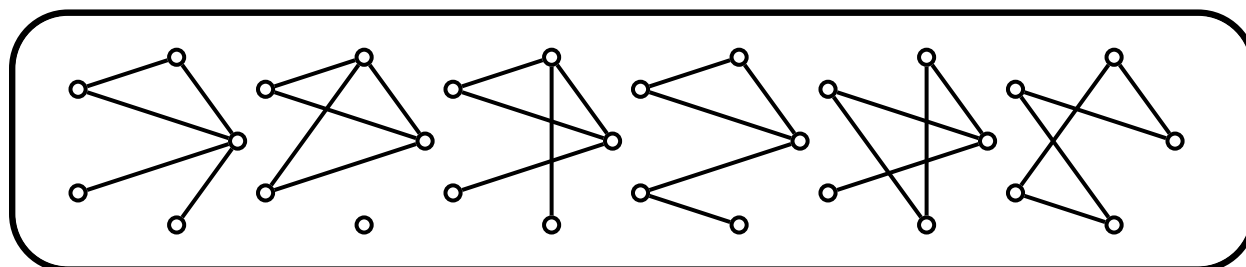
```
graph.classify_5:
> $(ORBITER) -v 2 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define GC -graph_classification \
> > > -n 5 \
> > > -poset_classification_control \
> > > > -problem_label graphs.v5 \
> > > > -depth 10 \
> > > > -draw_options -radius 250 \
> > > > -embedded \
> > > > -end \
> > > -end \
> > -end \
> > -with GC -do \
```

```

▷ ▷ -graph_classification.activity \
▷ ▷ ▷ -list_graphs_at_level 5 5 \
▷ ▷ -end \
▷ ▷ -with GC -do \
▷ ▷ -graph_classification.activity \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 300 -nodes_empty \
▷ ▷ ▷ ▷ -line_width 1.5 \
▷ ▷ ▷ ▷ -scale 0.1 \
▷ ▷ ▷ -end \
▷ ▷ ▷ -draw_graphs_at_level 5 \
▷ ▷ -end \
▷ ▷ -print_symbols
▷ pdflatex graphs_v5_level_5_reps.tex
▷ $(OPEN) graphs_v5_level_5_reps.pdf
▷ #pdflatex graphs_v5_poset.tex
▷ #$(OPEN) graphs_v5_poset.pdf

```

After the classification, the graphs with 5 edges are shown.



The next command classifies all tournaments on 4 vertices:

#### Example 542

```

tournament_classify_4:
▷ $(ORBITER) -v 2 \
▷ ▷ -define GC -graph_classification \
▷ ▷ ▷ -n 4 -tournament \
▷ ▷ ▷ -poset_classification_control \
▷ ▷ ▷ ▷ -problem_label tournament_4 \
▷ ▷ ▷ ▷ -depth 6 \
▷ ▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ ▷ -radius 250 -embedded \
▷ ▷ ▷ ▷ -end \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with GC -do \
▷ ▷ -graph_classification.activity \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 400 \
▷ ▷ ▷ ▷ -line_width 2 -scale 0.10 \
▷ ▷ ▷ -end \

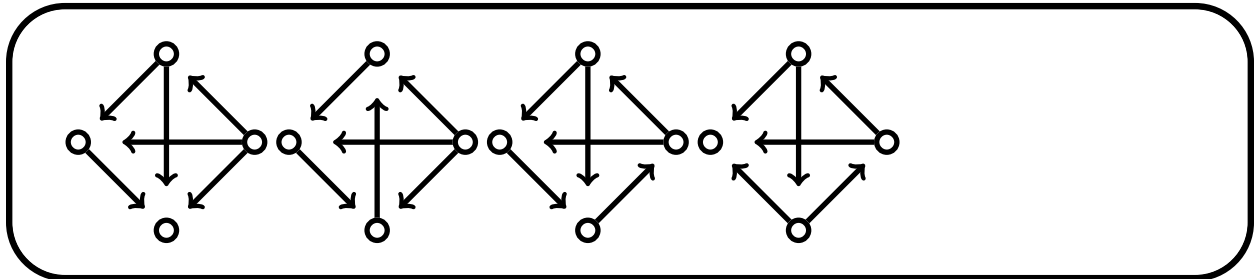
```

```

▷ ▷ ▷ -draw_graphs_at_level 6 \
▷ ▷ -end \
▷ ▷ -print_symbols
▷ pdflatex tournament_4_level_6_reps.tex
▷ $(OPEN) tournament_4_level_6_reps.pdf
▷

```

There are four tournaments, shown below:



The next command classifies all cubic graphs on 8 vertices:

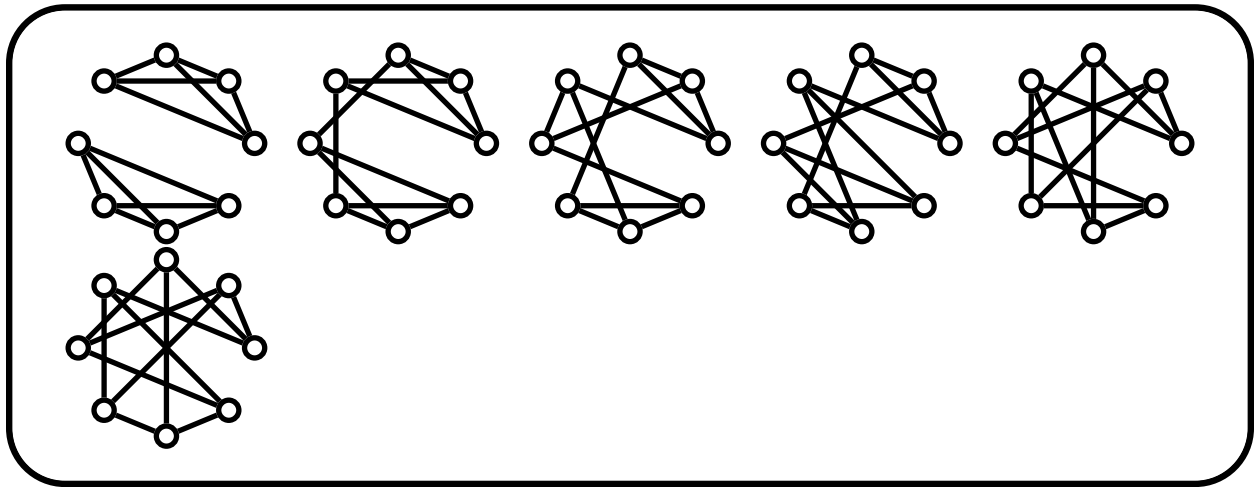
#### Example 543

```

graph_classify_8_r3:
▷ $(ORBITER) -v 3 \
▷ ▷ -define GC -graph_classification \
▷ ▷ ▷ -n 8 -regular 3 \
▷ ▷ ▷ -poset_classification_control \
▷ ▷ ▷ ▷ -problem_label graphs_v8_r3 \
▷ ▷ ▷ ▷ -depth 12 \
▷ ▷ ▷ ▷ -draw_options -radius 250 \
▷ ▷ ▷ ▷ ▷ -line_width 0.2 -embedded \
▷ ▷ ▷ ▷ -end \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with GC -do \
▷ ▷ -graph_classification_activity \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 400 \
▷ ▷ ▷ ▷ -line_width 2 -scale 0.10 \
▷ ▷ ▷ -end \
▷ ▷ ▷ -draw_graphs_at_level 12 \
▷ ▷ -end \
▷ ▷ -print_symbols
▷ #pdflatex graphs_v8_r3_poset_lvl_12.tex
▷ #$(OPEN) graphs_v8_r3_poset_lvl_12.pdf

```

There are six cubic graphs. The graph drawings are shown below:







## Chapter 13

# Design Theory

### 13.1 Creating Designs

A design is an incidence structure of points and blocks. The incidence matrix of a design has rows corresponding to the points and columns corresponding to the blocks. An entry in a certain row and column is one if and only if the point associated with the row is contained in the block associated with the column, zero otherwise. A decomposition of the design is a partition of the points and blocks such that each class consists either exclusively of points or exclusively of blocks.

A decomposition is point-tactical if for all points, the number of incident lines in the  $j$ th block class depends only on the class of the point. If the point belongs to class  $i$ , this number is denoted as  $a_{ij}$ . A decomposition is block-tactical if for all blocks, the number of incident points in the  $i$ th point class depends only on the class of the block. If the block belongs to class  $j$ , this number is denoted as  $b_{ij}$ .

A projective plane of order  $n$  is a design with  $n^2 + n + 1$  points and equally many blocks (also called lines), each of size  $n+1$  such that any two points lie in exactly one block and any two blocks have exactly one point in common. Projective planes are known to exist for all  $n = q$  which are a power of a prime. This follows from a construction which utilizes the projective geometry  $\text{PG}(2, q)$ . Points are the one-dimensional subspaces of  $\mathbb{F}_q^3$ , blocks are the two-dimensional subspaces of  $\mathbb{F}_q^3$ , and incidence is natural (inclusion of subspaces). The automorphism group of this design is the collineation group of the projective space. Projective planes other than these exist, though none are known when  $n$  is not a prime power. The number of lines through a point equals the number of points on a line. The fact that these numbers exist imply that there is a tactical decomposition. Namely, the trivial decomposition with two classes, one containing all points and one containing all lines. The structure constants of the decomposition are the numbers just described.

Table 13.1 lists Orbiter commands to create designs. Table 13.2 lists Orbiter activities for designs. The command

#### Example 544

```
design_PG_2_3:
> $(ORBITER) -v 8 \
> > -define F -finite_field -q 3 -end \
> > -define D -design -field F -family PG_2.q -end \
> > -with D -do \
> > > -design_activity \
> > > > -export_inc \
> > -end \
> > -with D -do \
> > > -design_activity \
```

| Creating Designs                |              |                                                                                                                                                |
|---------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                         | Arguments    | Purpose                                                                                                                                        |
| -field                          | $F$          | Some constructions of designs require a finite field. Set the field to be $F$ .                                                                |
| -catalogue                      | iso          | Recall a design of a given family from the Orbiter catalogue. The parameter iso is the isomorphism type in the catalogue. It is zero-based.    |
| -family                         | family       | Select the family name of the design to be constructed. The following family names are supported: PG_2.q for the projective plane PG(2, $q$ ). |
| -list_of_base_blocks            | G fname col  | Creates a design by spanning the orbits of the base blocks given in column col in file fname under the group G. See Section 13.4.              |
| -list_of_blocks_coded           | $v$ $k$ L    | Create a design on $v$ points and with block size $k$ from the coded list of sets in L.                                                        |
| -list_of_sets_coded             | $v$ label    | Create a design on $v$ points and with block codes in the list in L.                                                                           |
| -list_of_blocks_coded_from_file | $v$ $k$ file | Create a design on $v$ points and with block size $k$ from the coded list of sets read from the given file.                                    |
| -list_of_blocks_from_file       | $v$ fname    | Create a design on $v$ points and with block codes in the given file.                                                                          |
| -wreath_product_designs         | $n$ $k$      | Create a class of designs with wreath product action.                                                                                          |
| -linear_space_from_latin_square | matrix       | Create a linear space associated with a Latin Square given as matrix. The matrix must be an Orbiter object of type vector.                     |
| -no_group                       |              |                                                                                                                                                |

Table 13.1: Creating Designs

| Design Activities               |                                                                            |         |
|---------------------------------|----------------------------------------------------------------------------|---------|
| Command                         | Arguments                                                                  | Purpose |
| -load_table                     | label group<br>H_label<br>H_group order<br>H_gens l                        |         |
| -canonical_form                 | options                                                                    |         |
| -extract_solutions_by_index_csv | label group<br>fname-<br>solutions-in<br>fname-<br>solutions-out<br>prefix |         |
| -extract_solutions_by_index_txt | label group<br>fname-<br>solutions-in<br>fname-<br>solutions-out<br>prefix |         |
| -export_inc                     |                                                                            |         |
| -intersection_matrix            |                                                                            |         |
| -export_blocks                  |                                                                            |         |
| -row_sums                       |                                                                            |         |
| -tactical_decomposition         |                                                                            |         |

Table 13.2: Design Activities

```

> > > > -tactical_decomposition \
> > -end

```

creates the design PG(2, 3).

We have created the following design:

{19, 79, 126, 219, 256, 284, 371, 392, 465, 541, 619, 627, 653}

The stabilizer is generated by:

Strong generators for a group of order 5616:

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \\
 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,2,0,0,0,2,  
 1,0,0,0,2,0,0,0,1,  
 1,0,0,0,1,0,1,0,1,  
 1,0,0,0,1,0,0,1,1,  
 1,0,0,0,0,1,0,1,0,  
 0,1,0,1,0,0,0,0,1,

The blocks of the design are encoded in the lexicographic ordering of  $k$ -subsets (here  $k = 4$ ). The program also displays the tactical decomposition scheme of the design, which is

$$\begin{array}{c|c} \rightarrow & 13_1 \\ \hline 13_0 & 4 \end{array} \quad \begin{array}{c|c} \downarrow & 13_1 \\ \hline 13_0 & 4 \end{array}$$

In Section 16.4, we will show how to compute further properties of the design.

The command

#### Example 545

wreath\_product\_designs\_n4\_k2.inc.txt:

```

> $(ORBITER) -v 8 \
> > -define D -design -wreath_product_designs 4 2 -end \
> > -with D -do \
> > > -design_activity \
> > > > -export_inc \
> > -end

```

creates a design on 8 points invariant under the wreath product  $\text{Sym}(4) \wr \text{Sym}(2)$ . The design has 12 blocks of size 4. The command

**Example 546**

```
wreath_product_designs_n8_k6.inc.txt:
> $(ORBITER) -v 8 \
> > -define D -design -wreath_product_designs 8 6 -end \
> > -with D -do \
> > > -design_activity \
> > > > -export_inc \
> > -end
```

creates a design on 16 points invariant under the wreath product  $\text{Sym}(8) \wr \text{Sym}(2)$ . The design has 3920 blocks of size 6. We will compute the automorphism groups of these two designs in Section 16.3.

A design can be created by listing the blocks in coded fashion. The codes correspond to the ranking of  $k$ -subsets of the  $v$  set which is the underlying set of the design. The following command creates a configuration  $15_4 20_3$  from its set of blocks.

**Example 547**

```
design_20:
> $(ORBITER) -v 8 \
> > -define D -design -list_of_blocks_coded 15 3 \
> > "61, 67, 72, 76, 129, 147, 152, 156, 197, 204, 215, 224, 249, 261, 267, 276,
296, 303, 309, 319" \
> > -end \
> > -with D -do \
> > > -design_activity \
> > > > -export_inc \
> > -end
```

It is also possible to create a design by listing the blocks as sets. Here is an example. At first, we create a makefile variable for a csv file containing the blocks:

**Example 548**

```
GEO_BLOCKS_600="Row,C0,C1,C2\n\
0, 0, 5, 14, \n\
1, 1, 6, 10, \n\
2, 2, 7, 11, \n\
3, 3, 8, 12, \n\
4, 4, 9, 13, \n\
5, 0, 6, 13, \n\
6, 1, 7, 14, \n\
7, 2, 8, 10, \n\
8, 3, 9, 11, \n\
9, 4, 5, 12, \n\
10, 0, 7, 12, \n\
11, 1, 8, 13, \n\
```

```

12, 2, 9, 14, \n\
13, 3, 5, 10, \n\
14, 4, 6, 11, \n\
15, 0, 8, 11, \n\
16, 1, 9, 12, \n\
17, 2, 5, 13, \n\
18, 3, 6, 14, \n\
19, 4, 7, 10, \n\
20, 0, 9, 10, \n\
21, 1, 5, 11, \n\
22, 2, 6, 12, \n\
23, 3, 7, 13, \n\
24, 4, 8, 14, \n\
END\n"

```

The following command creates the csv file from the makefile variable using the unix echo command. The sed command is necessary to remove space characters that are inserted by the echo command. The Orbiter command then reads the list of blocks from the csv file. After that, it creates the design from the list of blocks. As we will see below, the design has an automorphism group of order 600 and is related to the Latin Square of order 5 associated with the cyclic group of order 5.

#### Example 549

```

design_600:
▷ echo $(GEO_BLOCKS_600) | sed 's/ //' >geo.600.blocks.csv
▷ $(ORBITER) -v 8 \
▷ ▷ -define D -design -list_of_blocks_from_file \
▷ ▷ ▷ 15 geo.600.blocks.csv \
▷ ▷ -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -design_activity \
▷ ▷ ▷ ▷ -export_inc \
▷ ▷ -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -design_activity \
▷ ▷ ▷ ▷ -intersection_matrix \
▷ ▷ -end

```

The command also computes the intersection matrix of the design. The design is a transversal design. The configuration graph is three copies of the complete graph  $K_5$ .

How can we convince ourselves that the design created previously is indeed associated to a certain Latin Square? There are exactly two Latin Squares of order 5. One is the Cayley table of the cyclic group of order 5, the other is the multiplication table of the non-associative loop of order 5, see Table 13.3.

We create makefile variables for the two tables:

#### Example 550

```

LSQ_5A_TABLE="0,1,2,3,4,1,2,3,4,0,2,3,4,0,1,3,4,0,1,2,4,0,1,2,3"

```

| Latin Squares of Order 5 |   |   |   |   |
|--------------------------|---|---|---|---|
| Cyclic 5                 |   |   |   |   |
| 0                        | 1 | 2 | 3 | 4 |
| 1                        | 2 | 3 | 4 | 0 |
| 2                        | 3 | 4 | 0 | 1 |
| 3                        | 4 | 0 | 1 | 2 |
| 4                        | 0 | 1 | 2 | 3 |

| Nonassociative Loop |   |   |   |   |
|---------------------|---|---|---|---|
| 0                   | 1 | 2 | 3 | 4 |
| 1                   | 0 | 4 | 2 | 3 |
| 2                   | 4 | 3 | 1 | 0 |
| 3                   | 2 | 0 | 4 | 1 |
| 4                   | 3 | 1 | 0 | 2 |

Table 13.3: Latin Squares of Order 5

**Example 551**

```
LSQ_5B_TABLE="0,1,2,3,4,1,0,4,2,3,2,4,3,1,0,3,2,0,4,1,4,3,1,0,2"
```

The next two commands create the associated linear spaces:

**Example 552**

```
LSQ_5A:
> $(ORBITER) -v 8 \
> > -define LSQ5A -vector -format 5 -dense $(LSQ_5A_TABLE) -end \
> > -define D -design -linear_space_from_latin_square \
> > > LSQ5A \
> > -end \
> > -with D -do \
> > > -design_activity \
> > > > -export_inc \
> > -end
```

**Example 553**

```
LSQ_5B:
> $(ORBITER) -v 8 \
> > -define LSQ5B -vector -format 5 -dense $(LSQ_5B_TABLE) -end \
> > -define D -design -linear_space_from_latin_square \
> > > LSQ5B \
> > -end \
> > -with D -do \
> > > -design_activity \
```

```

> > > > -export_inc \
> > -end

```

Finally, we perform the canonical form algorithm on these two linear spaces, noting that the isomorphisms / automorphisms of the Latin Squares corresponds to the isomorphisms / automorphism of the associated linear spaces.

#### Example 554

```

LSQ_5.c:
> $(ORBITER) -v 3 \
> > -define C -combinatorial_object \
> > > -label LSQ_5 LSQ\5 \
> > > -file_of_incidence_geometries latin_square_order5_LSQ5A.inc.txt 15 28 90 \
> > > -file_of_incidence_geometries latin_square_order5_LSQ5B.inc.txt 15 28 90 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form \
> > > > -save_ago \
> > > > -max_TDO_depth 4 \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_TDO \
> > > > -show_TDA \
> > > > -dont_show_incidence_matrices \
> > > -end \
> > -end
> pdflatex LSQ_5.classification.tex
> $(OPEN) LSQ_5.classification.pdf

```

This establishes the fact that the geometries are not isomorphic, and it also produces the automorphism groups of each geometry. It turns out that the Latin Square of order 5 arising from the cyclic group has a symmetry group of order 600. The second Latin Square arising from the loop has a symmetry group of order 72. Orbiter produces a report listing generators for each group. For more information about the canonical form of designs, see Section 16.4. Since there is only one linear space of Latin Square 5 type with a group of order 600, the object constructed earlier must be that same space which is associated to the Cayley table of the cyclic group of order 5.



## 13.2 Assuming Symmetry

One way to construct designs is by assuming a suitable group of symmetries. Let us consider an example. It is possible to construct  $t$ -( $v, k, \lambda$ ) designs invariant under a permutation group  $G$  acting on a set  $V$  with  $|V| = v$  as follows: Classify the orbits of  $G$  on subsets of size  $k$  and less. Construct a matrix which describes the relationship between the orbits on  $t$ -sets and the orbits on  $k$ -sets. This matrix is often referred to as the Kramer-Mesner matrix (cf. [41]). For each pair of  $t$ -orbit and  $k$ -orbit, for instance with representatives  $T$  and  $K$ , say, we count the number of elements in the orbit of  $K$  which contain  $T$ . The rows of the matrix are in correspondence to the  $t$ -orbits, while the columns are in correspondence to the  $k$ -orbits. The matrix entry  $a_{ij}$  is the number just defined where  $T$  is the representative of the  $i$ -th orbit on  $t$ -sets, and where  $K$  is the representative of the  $j$ -th orbit on  $k$ -sets. Let  $M_{t,k}(G)$  be the Kramer-Mesner matrix for the group  $G \leq \text{Sym}(V)$  defined in this way. The  $t$ -( $v, k, \lambda$ ) designs invariant under  $G$  are in one-to-one correspondence to the solutions of

$$M_{t,k}(G) \cdot \mathbf{x} = \lambda \mathbf{1},$$

where  $\mathbf{x}$  is a column vector of zeros and ones and  $\mathbf{1}$  is the column vector of all ones. The length of  $\mathbf{x}$  is the number of  $k$ -orbits of  $G$  on  $V$ , while the length of  $\mathbf{1}$  is the number of  $t$ -orbits of  $G$  on  $V$ . Any vector  $\mathbf{x}$  satisfying the matrix equation corresponds to a design invariant under  $G$ . Simply take the blocks of the design to be the union of those orbits of  $G$  on  $k$ -subsets whose associated entry in  $\mathbf{x}$  is one. We assume the group  $\text{PTL}(2, 32)$  in the action on points of the projective line  $\text{PG}(1, 32)$  over the field  $\mathbb{F}_{32}$ . The parameters of the design are 7-(33, 8, 10), that is, each 7-subset of  $\text{PG}(1, 32)$  is covered exactly 10 times by the chosen 8-subsets comprising the design. Let us consider the following Orbiter command:

### Example 555

```
KM_PGGL_2_32:
> $(ORBITER) -v 3 \
> > -orbiter_path $(ORBITER_EXE_PATH) \
> > -define Control -poset_classification_control \
> > > -problem_label KM_PGGL_2_32 \
> > > -W -depth 8 \
> > > -draw_options -embedded -sideways -radius 50 \
> > > > -scale 0.5 -line.width 0.3 -end \
> > -end \
> > -define G -linear_group -PGGL 2 32 -end \
> > -define Orb -orbits -group G \
> > > -on_subsets 8 Control \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -Kramer_Mesner_matrix 7 8 \
> > -end \
> > -with Orb -do -orbits_activity \
> > > -report \
> > > -report_options \
> > > > -draw_poset -type_ordinary \
> > > -end \
> > -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file KM_PGGL_2_32_KM_7_8.csv \
> > -box_width 20 -bit_depth 24 \
> > -partition 3 32 97 -end
> pdflatex KM_PGGL_2_32_poset_lvl.8.tex
> $(OPEN) KM_PGGL_2_32_poset_lvl.8.pdf
> $(OPEN) KM_PGGL_2_32_KM_7_8_draw.bmp
```

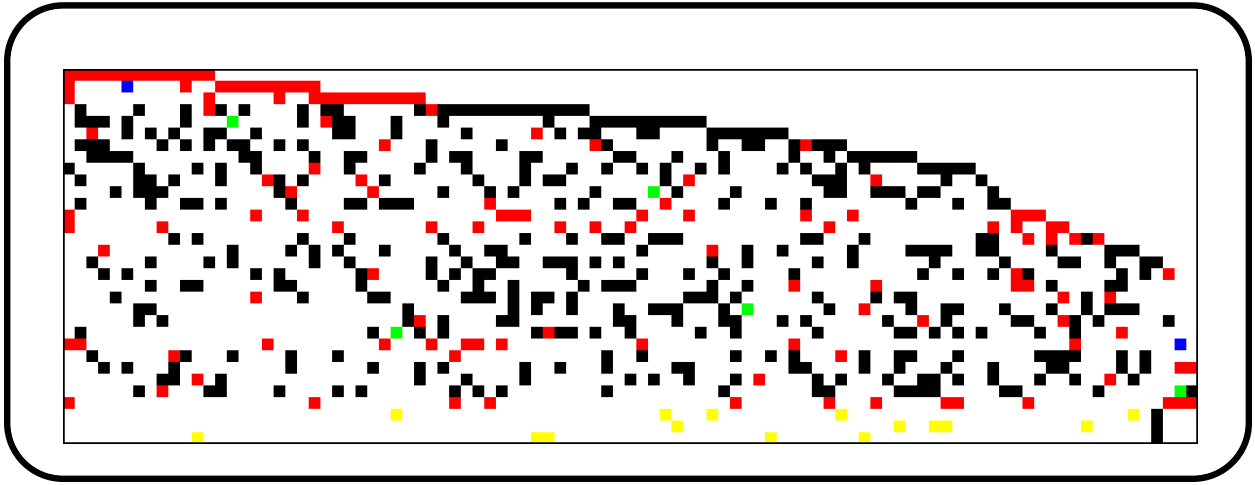
The first command creates the group  $\text{PTL}(2, 32)$  and computes the Kramer-Mesner matrix

$$M_{7,8}(\text{PTL}(2, 32)).$$

The number of 7-orbits is 32. The number of 8-orbits is 97. Correspondingly, the Kramer-Mesner matrix has 32 rows and 97 columns. The matrix is stored in the csv-file

KM\_PGGL\_2\_32\_KM\_7\_8.csv.

The second command produces the graphical representation of the matrix shown below



(different colors represent different values of the entries in the matrix). The command

#### Example 556

```
KM_PGGL_2_32.solve:
> $(ORBITER) -v 4 \
> > -define A -vector -file KM_PGGL_2_32_KM_7_8.csv -end \
> > -define D -diophant \
> > -label "KM_PGGL_2_32_KM_7_8.system" \
> > -coefficient_matrix A \
> > -RHS_constant "EQ=10" \
> > -x_min_global 0 -x_max_global 1 \
> > -end \
> > -with D -do \
> > > -diophant_activity -solve_mckay \
> > -end
```

creates the diophantine system associated with the Kramer-Mesner matrix and solves it. It performs a complete enumeration of all solutions by solving the system and producing the solution vectors  $\mathbf{x}$  corresponding to the designs.

Let us construct configurations with an assumed symmetry group. Suppose we are interested in configurations with  $v = 15$  points, and 25 blocks of size 3. We assume a group of order 5 acting semi regularly. In cycle notation, the generator may be taken as

$$(0, 1, 2, 3, 4)(5, 6, 7, 8, 9)(10, 11, 12, 13, 14).$$

In two row notation, we write

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ 1 & 2 & 3 & 4 & 0 & 6 & 7 & 8 & 9 & 5 & 11 & 12 & 13 & 14 & 10 \end{bmatrix}$$

A makefile variable will hold the generator (the second row in the two-row notation):

#### Example 557

```
GEN_C5="1,2,3,4,0,6,7,8,9,5,11,12,13,14,10"
```

Next, we need to compute the orbits on pairs and triples. We also need the Kramer-Mesner matrices

$$M_{1,3}, \quad M_{2,3}$$

Every point lies on 5 blocks, and every pair orbit is covered at most once. This leads to the system

$$M_{1,3}\mathbf{x} = 5 \cdot \mathbf{1}, \quad M_{2,3}\mathbf{x} \leq \mathbf{1}, \quad \mathbf{0} \leq \mathbf{x} \leq \mathbf{1},$$

where  $\mathbf{0}$  is the all-zero vector and  $\mathbf{1}$  is the all-one vector. The following command computes  $M_{1,3}$ :

#### Example 558

```
C5_KM_1_3:
▷ $(ORBITER) -v 3 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
▷ ▷ -define G -permutation_group -symmetric_group 15 \
▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
▷ ▷ -end \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label C5 -W -depth 3 \
▷ ▷ ▷ -draw_options -embedded -sideways -radius 50 \
▷ ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
▷ ▷ -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_subsets 3 Control \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -Kramer_Mesner_matrix 1 3 \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -export_something set_orbits 3 \
▷ ▷ -end \
```

The matrix has 3 rows and 91 columns. The command also exports the orbits on 3-subsets to a file with the name `C5_set_orbits_level_3.csv`. The following command computes  $M_{2,3}$ :

#### Example 559

```
C5_KM_2_3:
▷ $(ORBITER) -v 3 \
▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
```

```

▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
▷ ▷ -define G -permutation_group -symmetric_group 15 \
▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
▷ ▷ -end \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -problem_label C5 -W -depth 3 \
▷ ▷ ▷ -draw_options -embedded -sideways -radius 50 \
▷ ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
▷ ▷ -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_subsets 3 Control \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -Kramer_Mesner_matrix 2 3 \
▷ ▷ -end

```

The matrix has 21 rows and 91 columns. The next command stacks the two matrices together, one on top of the other:

#### Example 560

```

C5_concatenate:
▷ $(ORBITER) -v 3 \
▷ ▷ -csv_file_concatenate C5_KM_combined.csv 2 C5_KM_1_3.csv C5_KM_2_3.csv

```

Next, we need to solve the system. To this end, we use the following command.

#### Example 561

```

C5_solve:
▷ $(ORBITER) -v 4 \
▷ ▷ -define A -vector -file C5_KM_combined.csv -end \
▷ ▷ -define D -diophant \
▷ ▷ ▷ -label "C5_KM_combined_system" \
▷ ▷ ▷ -coefficient_matrix A \
▷ ▷ ▷ -x_min_global 0 \
▷ ▷ ▷ -x_max_global 1 \
▷ ▷ ▷ -RHS "mult=3,EQ=5" \
▷ ▷ ▷ -RHS "mult=21,LE=1" \
▷ ▷ -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -diophant_activity -solve_mckay \
▷ ▷ -end

```

Orbiter finds 21540 solutions in 61001 backtrack steps. Next, we need to assemble the designs by concatenating the set orbits on 3-subsets that are selected by the solution vector  $\mathbf{x}$ . Once the designs are created, we need to classify them up to isomorphism. We utilize the classification algorithm in Orbiter which is based on canonical forms computed by Nauty:

**Example 562**

```

C5_design.classify:
> $(ORBITER) -v 2 \
> > -define C -combinatorial_object \
> > > -label C5_config C5\_config \
> > > -file_of_designs_through_block_orbits \
> > > > C5_KM_combined_system_sol.csv \
> > > > C5_set_orbits_level_3.csv \
> > > > 15 3 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -show_incidence_matrices \
> > > -end \
> > -end
> pdflatex C5_config_classification.tex
> $(OPEN) C5_config_classification.pdf

```

A latex report is produced. Orbiter finds 42 isomorphism types of designs. The largest automorphism group associated with these designs has order 600.

| Creating Large Sets with Assumed Symmetry |           |                                               |
|-------------------------------------------|-----------|-----------------------------------------------|
| Command                                   | Arguments | Purpose                                       |
| -H                                        | go gens   | Specify the assumed group $H$ .               |
| -N                                        | go gens   | Specify the normalizer $N$ of the group $H$ . |
| -report                                   |           | Create a latex report.                        |
| -prefix                                   | prefix    | Set a prefix for output files.                |
| -selected_orbit_length                    | l         | Select orbit length $l$ .                     |

Table 13.4: Creating Large Sets with Assumed Symmetry

| Large Set Activity                                      |                               |         |
|---------------------------------------------------------|-------------------------------|---------|
| Command                                                 | Arguments                     | Purpose |
| -normalizer_on_orbits_of_<br>a_given_length             | l nb_orbits<br>control        |         |
| -create_graph_on_orbits_of_length                       | fname l                       |         |
| -create_graph_on_orbits_<br>of_length_based_on_N_orbits | fname-mask l<br>nb_orbits r m |         |
| -read_solution_file                                     | l fname                       |         |

Table 13.5: Large Set Activity

### 13.3 Design Theory – Large Sets

Fix a set of size  $v$  and an integer  $k$  with  $1 < k < v$ . Is it possible to partition the set of  $k$ -subsets of  $v$  into designs, all with the same parameters? If so, the resulting set of designs is called a large set (of designs). So, a large set of designs is a set of designs, all of the same types, on a fixed  $v$ -element set whose block sets are pairwise disjoint and partition the set of  $k$ -subsets. Let us see how Orbiter can help construct and classify small large sets.

Tables 13.4 shows Orbiter commands to construct large sets with assumed symmetry group. Table 13.5 lists Orbiter activities for objects of type large-set-with-assumed-symmetry.

Suppose we consider  $AG(2, 3)$ , the affine plane of order 3. It is a configuration with 9 points, 12 lines, 4 lines on each point and 3 points on each line. To see whether or not it is unique, we use the following command:

#### Example 563

```

AG_2.3.inc:
> $(ORBITER) -v 2 \
> > -define Geo -geometry_builder \
> > > -V 9 -B 12 \
> > > -TDO 4 -fuse 1 \
> > > -fname_GEO AG_2.3 \
> > > -test 3,4,5,6,7,8,9 \
> > > -output_to_inc_file \
> > -end

```

The command produces the file `AG_2.3.inc`, which contains the following lines:

```

9 12 36
0 1 2 3 12 16 17 18 24 31 32 33 37 40 43 46 49 53 56 59 62 64 69 71 74 78 80 82 87 89 93 94 99 102 103 107
-1 1
432

```

This shows that the design is unique, and has an automorphism group of order 432. For the following commands, we will treat blocks of the design as sets of ranks of  $k$ -subsets. Next, we create a table of all designs AG(2,3), as orbit under the group Sym(9). The following command does that:

**Example 564**

```
AG_2_3_BLOCKS="0,13,22,27,35,41,47,53,55,59,71,76"
```

**Example 565**

```

LS_AG_2_3_design_table_create:
> $(ORBITER) -v 5 \
> > -define B -vector -dense $(AG_2_3_BLOCKS) -end \
> > -define D -design -list_of_blocks_coded 9 3 B -end \
> > -define Sym9 -permutation_group -symmetric_group 9 -end \
> > -define T -design_table D "AG_2_3" Sym9 -end

```

The number of designs is  $|\text{Sym}(9)|/432 = 362880/432 = 840$ . To find all large sets, we establish the block-disjointness graph on this set of designs. After that, we find all cliques of size 7:

**Example 566**

```

LS_AG_2_3_disjoint_sets_graph_and_cliques:
> $(ORBITER) -v 2 \
> > -define Gamma -graph \
> > > -disjoint_sets_graph \
> > > AG_2_3_design_table.csv \
> > -end \
> > -with Gamma -do \
> > -graph_theoretic_activity \
> > > -save \
> > -end \
> > -with Gamma -do \
> > -graph_theoretic_activity \
> > > -find_cliques -target_size 7 -end \
> > -end \
> > -print_symbols

```

The files AG\_2\_3\_design\_table\_disjoint\_sets\_sol.txt and AG\_2\_3\_design\_table\_disjoint\_sets\_sol.csv are created, each containing the cliques of size 7. There are exactly 15360 cliques of size 7. It remains to classify the resulting 15360 large sets up to isomorphism. To do that, we first need to create the actual large sets from the cliques. The following command does that:

**Example 567**

```
LS_AG_2_3_export_solutions:
> $(ORBITER) -v 20 \
> > -define B -vector -dense $(AG_2_3_BLOCKS) -end \
> > -define D -design -list_of_blocks 9 3 B -end \
> > -define Sym9 -permutation_group -symmetric_group 9 -end \
> > -define T -design_table D "AG_2_3" Sym9 -end \
> > -with D -do \
> > -design_activity \
> > > -extract_solutions_by_index "AG_2_3" Sym9 \
> > > > AG_2_3_design_table_disjoint_sets_sol.csv \
> > > > solutions.csv \
> > > > "" \
> > -end
```

The final step is the classification of these large sets up to isomorphism. This will be discussed in Section 16.4.



## 13.4 Design Theory – Delandtsheer-Doyen

Delandtsheer and Doyen in [24] study line-transitive and point-imprimitive designs and show that they are rare in a certain sense. Orbiter can be used to construct such designs assuming that there is a grid structure on the set of points and assuming that the design is invariant under a chosen group  $G$ . The group  $G$  is assumed to be a subgroup of the group  $\text{AGL}(d_1, q_1) \times \text{AGL}(d_2, q_2)$  acting on a grid of size  $q_1^{d_1} \times q_2^{d_2}$  in product action.

Table 13.6 shows Orbiter commands to search for linear spaces of Delandtsheer/Doyen type. Finite projective planes often arise in this context. However, not all examples are projective planes. Orbiter can help to classify small examples.

Let us consider an example. Suppose we want to classify all designs on 21 points with blocks of size  $k = 5$  invariant under a cyclic group of order 21 preserving a grid of type  $3 \times 7$ . To this end, we consider the group  $\text{AGL}(1, 3) \times \text{AGL}(1, 7)$ . The subgroup is generated by the map

$$(\tau_1, \tau_2), \mathbb{Z}_3 \times \mathbb{Z}_7 \rightarrow \mathbb{Z}_3 \times \mathbb{Z}_7,$$

where

$$\tau_1 : \mathbb{Z}_3 \rightarrow \mathbb{Z}_3, x \mapsto x + 1 \pmod{3}, \quad \tau_2 : \mathbb{Z}_7 \rightarrow \mathbb{Z}_7, y \mapsto y + 1 \pmod{7}.$$

With blocks of size 5, we cover 10 pairs each. The group of order 21 allows to cover each of the  $210 = \binom{21}{2}$  pairs exactly once using a single orbit of a block. Under the group  $G$ , there are 55 orbits on pairs. The question remains to construct all blocks and to classify the resulting designs.

At first, three makefile variables are defined:

### Example 568

```
PP4=-d1 1 -q1 3 -d2 1 -q2 7 -K 5 -problem_label PP4
```

### Example 569

```
PP4_GROUP1=-subgroup "1,1,1,1, " "21" -group_label "cyclic21"
```

### Example 570

```
PP4_MASK1=\
▷ -nb_orbits_on_blocks 1 \
▷ -mask_label "no_mask"
```

The next command performs a classification of the sets of size 5 under the given group and using the given mask conditions:

### Example 571

```
DD_PP4:
▷ $(ORBITER) -v 6 \
▷ ▷ -define pair_search_control -poset_classification_control \
▷ ▷ ▷ -problem_label PP4_pairs -W \
▷ ▷ ▷ -draw_options -end \
▷ ▷ -end \
```

| Creating Linear Spaces of Delandtsheer-Doyen Type |                          |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                           | Arguments                | Purpose                                                                                                                                                                                                                                                                                                                                                                                                             |
| -d1                                               | d1                       | Affine group parameter.                                                                                                                                                                                                                                                                                                                                                                                             |
| -d2                                               | d2                       | Affine group parameter.                                                                                                                                                                                                                                                                                                                                                                                             |
| -q1                                               | q1                       | Affine group parameter.                                                                                                                                                                                                                                                                                                                                                                                             |
| -q2                                               | q2                       | Affine group parameter.                                                                                                                                                                                                                                                                                                                                                                                             |
| -group_label                                      | label                    |                                                                                                                                                                                                                                                                                                                                                                                                                     |
| -mask_label                                       | label                    |                                                                                                                                                                                                                                                                                                                                                                                                                     |
| -problem_label                                    | label                    |                                                                                                                                                                                                                                                                                                                                                                                                                     |
| -DDx                                              | x                        | Deladtsheer-Doyen parameter $x$ .                                                                                                                                                                                                                                                                                                                                                                                   |
| -DDy                                              | y                        | Secondary Deladtsheer-Doyen parameter $y$ .                                                                                                                                                                                                                                                                                                                                                                         |
| -K                                                | k                        | Specify the size of the base-line to generate the linear space under the affine group.                                                                                                                                                                                                                                                                                                                              |
| -pair_search_control                              | label                    | Options for the classification of pair orbits, see Tab. 7.3.                                                                                                                                                                                                                                                                                                                                                        |
| -search_control                                   | label                    | Options for the classification of the base lines by means of subsets, see Tab. 7.3. The depth of the search for starters (partial base-lines) is set here, using the -depth command.                                                                                                                                                                                                                                |
| -R                                                | nb_row_types<br>row_type |                                                                                                                                                                                                                                                                                                                                                                                                                     |
| -C                                                | nb_col_types<br>col_type |                                                                                                                                                                                                                                                                                                                                                                                                                     |
| -nb_orbits_on_blocks                              | n                        |                                                                                                                                                                                                                                                                                                                                                                                                                     |
| -masktest                                         | options                  | Specify conditions on the mask, which is the pattern induced by the base-line when considering the grid structure.                                                                                                                                                                                                                                                                                                  |
| -search_partial_base_lines                        |                          | Search for partial base-lines. The size of the partial base-line is specified in the search_control data structure, using the -depth argument. Once the classification of partial base-lines is finished (of size $s$ , say), the covering matrix of pair orbits is written. For each orbit on partial base-lines of size $s$ , the $\binom{s}{2}$ disjoint pair-orbits covered by that representative are written. |
| -singletons                                       | sz                       | Search for singletons. This requires that a classification of partial base-lines of size sz has been completed before. This command initiates the completion of the partial base-lines.                                                                                                                                                                                                                             |
| -subgroup                                         | gens order               | Specify generators for the subgroup $G$ which acts line-transitive and point-imprimitive on the linear space.                                                                                                                                                                                                                                                                                                       |
| -search_wrt_subgroup                              |                          |                                                                                                                                                                                                                                                                                                                                                                                                                     |

Table 13.6: Creating Linear Spaces of Delandtsheer-Doyen Type

```

▷ ▷ -define search_control -poset_classification_control \
▷ ▷ ▷ -problem_label PP4_search -W \
▷ ▷ ▷ -draw_options -end \
▷ ▷ ▷ -depth 5 \
▷ ▷ -end \
▷ ▷ -Delandtsheer_Doyen \
▷ ▷ ▷ -pair_search_control pair_search_control \
▷ ▷ ▷ -search_control search_control \
▷ ▷ ▷ -search_partial_base_lines \
▷ ▷ ▷ $(PP4) $(PP4_GROUP1) $(PP4_MASK1) \
▷ ▷ ▷ -end \

```

We find exactly one solution. The following command spans the design from the base line by computing the orbit under  $G$ :

#### Example 572

```

DD_PP4_span_design:
▷ $(ORBITER) -v 6 \
▷ ▷ -define D -design -list_of_base_blocks \
▷ ▷ ▷ group \
▷ ▷ ▷ PP4_no.mask.cyclic21.cases.csv
▷ ▷ ▷ Starter \
▷ ▷ -end

```

The next command creates a design object from the base line under the group  $G$ :

#### Example 573

```

DD_PP4_design_orbit:
▷ $(ORBITER) -v 4 \
▷ ▷ -define G1 -linear_group -AGL 1 3 \
▷ ▷ ▷ -end \
▷ ▷ -define G2 -linear_group -AGL 1 7 \
▷ ▷ ▷ -end \
▷ ▷ -define G -modified_group -direct_product "G1,G2" \
▷ ▷ ▷ "21" "1,1,1,1" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -define D -design -list_of_base_blocks \
▷ ▷ ▷ G \
▷ ▷ ▷ PP4_no.mask.cyclic21.cases.csv \
▷ ▷ ▷ Starter \
▷ ▷ -end

```

As  $\text{PG}(2,4)$  admits an action by a cyclic group of order 21 in product action, and we find only one design, the design we found must be  $\text{PG}(2,4)$ . It is the only line-transitive point-imprimitive design with these

parameters.

## Chapter 14

# Finite Geometry

### 14.1 Spreads

A  $t$ -spread of  $\text{PG}(n, q)$  is a set of disjoint  $\text{PG}(t, q)$  that cover all of  $\text{PG}(n, q)$  pointwise.  $t$ -spreads in  $\text{PG}(n, q)$  exist if and only if  $t + 1$  divides  $n + 1$ . Table 14.1 lists the Orbiter commands to create spreads. Table 14.2 lists Orbiter spread activities. Table 14.3 list Orbiter commands to classify spreads. Table 14.4 lists the Orbiter commands that apply to a object of type spread classify. Table 14.5 lists the Orbiter activities for spread tables.

The following two commands create the two spreads of order 9, relying on the Orbiter knowledge base.

#### Example 574

```
create_spread_9a:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -define S -spread -kernel_field F \
▷ ▷ ▷ -group G -k 2 -catalogue 0 \
▷ ▷ ▷ -end
```

#### Example 575

```
create_spread_9b:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -define S -spread -kernel_field F \
▷ ▷ ▷ -group G -k 2 -catalogue 1 \
▷ ▷ ▷ -end
```

The first spread is the Desarguesian spread, with automorphism group of order 5760. The second spread is the Hall spread with automorphism group of order 1920.

Spreads can be defined using spread sets. A spread set is a set of  $q^k$  matrices of size  $k \times k$  over  $\mathbb{F}_q$  such that  $A_i - A_j$  is nonsingular for all  $i \neq j$ . Let us look at an example. The spread due to Rao and Rao [55] can be defined using the following makefile variable.

| Creating a Spread   |                            |                                                                                                                                        |
|---------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Command             | Arguments                  | Purpose                                                                                                                                |
| -kernel_field       | $F$                        | Define the kernel of the spread. $F$ must be an object of type finite field.                                                           |
| -group              | $G$                        | Define the group acting on the spread. Should be $\text{P}\Gamma\text{L}(2k, F)$ .                                                     |
| -group_on_subspaces | $G$                        |                                                                                                                                        |
| -k                  | $k$                        | Set the dimension of the spread.                                                                                                       |
| -catalogue          | $i$                        | Pull spread number $i$ from the catalogue of spreads associated with the given field and the given dimension.                          |
| -family             | $L$                        | Define a spread from a named family $L$ . So far, no family has been provided.                                                         |
| -spread_set         | label-txt<br>label-tex $S$ | Define a spread from the named spreadset $S$ . The spreadset $S$ must be a vector object. It must contain $q^3 k^2$ entries over $F$ . |
| -transform          | elt                        | Apply the transformation given by the group element.                                                                                   |
| -transform_inverse  | elt                        | Apply the inverse transformation given by the group element.                                                                           |

Table 14.1: Creating a Spread

| Spread Activities |           |                        |
|-------------------|-----------|------------------------|
| Command           | Arguments | Purpose                |
| -report           |           | Create a latex report. |

Table 14.2: Spread Activities

| Classification of Spreads     |           |                                           |
|-------------------------------|-----------|-------------------------------------------|
| Command                       | Arguments | Purpose                                   |
| -projective_space             | P         | Select the projective space object P.     |
| -starter_size                 | k         | Choose starter size $k$ .                 |
| -k                            | k         | Choose dimension of the spread elements.  |
| -poset_classification_control | control   | Select poset classification control.      |
| -output_prefix                | prefix    | Set output prefix for file i/o.           |
| -recoordinatize               |           | Turn recoordinatization on (recommended). |

Table 14.3: Classification of Spreads

| Activities for the Classification of Spreads |                         |                                                                                                                          |
|----------------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Command                                      | Arguments               | Purpose                                                                                                                  |
| -compute_starter                             |                         | Perform classification of partial spreads.                                                                               |
| -prepare_lifting_single_case                 | k                       | Prepare to lift single case $k$ . Here (and below) case means isomorphism type in the classification of partial spreads. |
| -prepare_lifting_all_cases                   |                         | Prepare to lift all cases of partial spreads.                                                                            |
| -split                                       | r m                     | Lift only those cases $k$ with $k \equiv r \pmod{m}$ .                                                                   |
| -isomorph                                    | prefix1 prefix2 options | Perform isomorphism testing after lifting is complete.                                                                   |

Table 14.4: Activities for the Classification of Spreads

| Activities for Spread Tables      |             |                                                     |
|-----------------------------------|-------------|-----------------------------------------------------|
| Command                           | Arguments   | Purpose                                             |
| -find_spread                      | spread      | Find a given spread in the table.                   |
| -find_spread_and_dualize          | spread      | Find a given spread in the table and dualize.       |
| -dualize_packing                  | packing     | Dualize a packing.                                  |
| -print_spreads                    | spreads     | Print the given spreads.                            |
| -export_spreads_to_csv            | fname index | Export the spread whose index is given to csv-file. |
| -find_spreads_containing_one_line | index       | Find all spreads which contain the given line.      |

Table 14.5: Activities for Spread Tables

**Example 576**

```

SPREAD_SET_27_RAO_RAO="\
0,0,0,0,0,0,0,0,0, \
1,1,0,2,1,1,0,0,2, \
1,0,1,1,2,2,0,1,0, \
1,2,2,1,2,0,2,2,2, \
0,0,2,2,2,0,1,2,0, \
1,1,2,0,2,1,2,1,0, \
0,1,0,1,0,1,0,2,1, \
2,0,2,0,0,2,1,1,0, \
2,2,2,0,1,1,0,1,2, \
2,0,0,1,0,2,1,2,1, \
0,2,2,2,2,2,2,0,2, \
2,1,2,0,2,0,2,0,1, \
0,1,2,2,0,1,0,1,1, \
1,0,0,0,1,0,0,0,1, \
2,1,0,1,2,1,0,2,0, \
0,2,0,0,2,2,1,1,2, \
0,0,1,0,1,2,2,2,1, \
2,0,1,2,2,1,1,0,1, \
0,1,1,1,1,0,1,2,2, \
2,2,0,2,0,0,0,2,2, \
2,1,1,1,1,2,2,1,2, \
2,2,1,2,1,0,2,0,0, \
1,2,0,2,0,2,1,0,0, \
1,2,1,1,0,0,1,1,1, \
0,2,1,1,1,1,2,2,0, \
1,1,1,0,0,1,1,0,2, \
1,0,2,2,1,2,2,1,1 \
"

```

Each line represents one matrix of the spread set, with matrix entries being listed consecutively. The following command can be used to define the spread:

**Example 577**

```

create_spread_Rao_Rao_27:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define SS -vector -dense $(SPREAD_SET_27_RAO_RAO) -end \
▷ ▷ -define G -linear_group -PGL 6 F -end \
▷ ▷ -define S -spread -kernel_field F \
▷ ▷ ▷ -group G -k 3 -spread_set "Rao_Rao" "Rao\_Rao" SS \
▷ ▷ ▷ -end

```

The following command creates the Desarguesian line-spread in  $\text{PG}(3, 2)$ :



**Example 578**

```

desarguesian_spread_in_PG_3_2:
> $(ORBITER) -v 3 \
> > -define FQ -finite_field -q 4 -end \
> > -define Fq -finite_field -q 2 -end \
> > -with FQ -and Fq -do -finite_field_activity \
> > > -cheat_sheet_desarguesian_spread 2 -end
> pdflatex Desarguesian_Spread_3_2.tex
> $(OPEN) Desarguesian_Spread_3_2.pdf

```

The cheat sheet contains the following spread:

Spread element 0 is  $(1,0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0$

Spread element 1 is  $(0,1) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{34}$

Spread element 2 is  $(1,1) = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_9$

Spread element 3 is  $(2,1) = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_{17}$

Spread element 4 is  $(3,1) = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_{22}$

Spread elements by rank:  $(0, 34, 9, 17, 22)$ .

The following command creates the Desarguesian plane-spread in  $\text{PG}(5, 2)$ :

**Example 579**

```

desarguesian_spread_in_PG_5_2:
> $(ORBITER) -v 3 \
> > -define FQ -finite_field -q 8 -end \
> > -define Fq -finite_field -q 2 -end \
> > -with FQ -and Fq -do -finite_field_activity \
> > > -cheat_sheet_desarguesian_spread 2 -end
> pdflatex Desarguesian_Spread_5_2.tex
> $(OPEN) Desarguesian_Spread_5_2.pdf

```

Spread element 0 is  $(1,0) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}_0$

|                                                                           |                                                                                                         |      |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------|
| Spread element 1 is (0,1) =                                               | $\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$ | 1394 |
| Spread element 2 is (1,1) =                                               | $\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$ | 189  |
| Spread element 3 is (2,1) =                                               | $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$ | 671  |
| Spread element 4 is (3,1) =                                               | $\begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$ | 562  |
| Spread element 5 is (4,1) =                                               | $\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$ | 1040 |
| Spread element 6 is (5,1) =                                               | $\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$ | 792  |
| Spread element 7 is (6,1) =                                               | $\begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$ | 1161 |
| Spread element 8 is (7,1) =                                               | $\begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$ | 373  |
| Spread elements by rank: ( 0, 1394, 189, 671, 562, 1040, 792, 1161, 373 ) |                                                                                                         |      |

Two  $t$ -spreads are isomorphic if there is a collineation which maps one to the other. The classification problem for  $t$ -spreads is the problem of determining a complete set of pairwise non-isomorphic  $t$ -spreads. Orbiter can be used to classify spreads for small parameters. For greater classification power, the method of classification by substructure is used. Let us look at some examples.

At first, we look at an example which is sufficiently small and can be solved using the standard method. Here, the standard method is poset classification algorithm for partial spreads. Suppose we want to classify the line spreads in  $\text{PG}(3, 4)$  under the action of  $\text{PTL}(4, 4)$ . Under the André, Bruck-Bose construction [3, 18], these spreads correspond to translation planes of order 16 with kernel  $\mathbb{F}_4$ . In order to classify the spreads of  $\text{PG}(3, 4)$ , we utilize the poset classification, as shown in the following command

#### Example 580

```

classify_spreads_16_4:
> $(ORBITER) -v 4 \
> > -define Control -poset_classification_control \
> > > -draw_options \
> > > > -radius 20 \
> > > > -nodes_empty \
> > > > -line_width 0.2 \

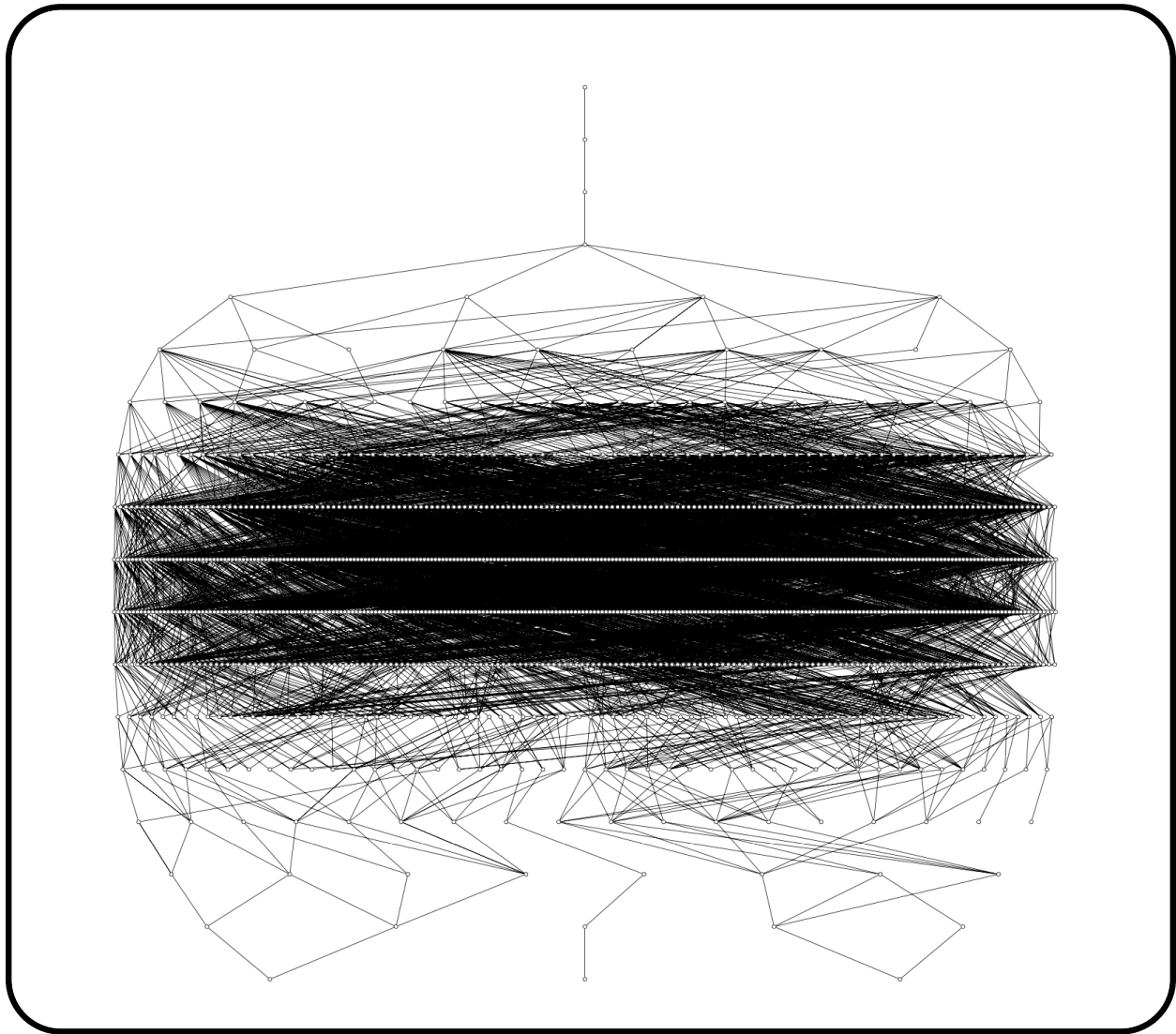
```

```

▷ ▷ ▷ ▷ -embedded \
▷ ▷ ▷ -end \
▷ ▷ ▷ -problem_label spreads_16_4 \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define C -spread_classifier \
▷ ▷ ▷ -projective_space P \
▷ ▷ ▷ -poset_classification_control Control \
▷ ▷ ▷ -k 2 \
▷ ▷ ▷ -starter_size 17 \
▷ ▷ ▷ -output_prefix "." \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ -end
▷ #pdflatex spreads_16_4_poset_lvl_17.tex
▷ #$(OPEN) spreads_16_4_poset_lvl_17.pdf

```

The algorithm computes the poset of orbits for the group  $G = \text{PTL}(4,4)$  acting on the poset of partial spreads in  $\text{PG}(3,4)$ . The poset of orbits is shown below:



Up to isomorphism, there are exactly three line-spreads in  $\text{PG}(3, 4)$  (corresponding to the three nodes at the bottom of the poset of orbits in the figure). These three spreads are the dearguesian spread, the Hall spread, and the semifield spread, respectively. Here is the relevant output taken from the latex report:

There are 3 orbits at level 17.

### Orbit 0 / 3 at Level 17

Node number: 1126

$$\{0, 25, 50, 75, 90, 107, 122, 140, 144, 157, 179, 204, 213, 238, 268, 334, 345\}_{1200}$$

Strong generators for a group of order 1200:

$$\begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 1 & \omega & 0 & 1 \\ \omega^2 & \omega^2 & \omega & 1 \end{bmatrix}_0, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ \omega & \omega^2 & 0 & 0 \\ \omega & \omega & 1 & \omega^2 \\ 0 & 1 & 1 & 0 \end{bmatrix}_1, \begin{bmatrix} \omega & 1 & \omega & \omega \\ \omega & \omega^2 & \omega^2 & 0 \\ \omega & 0 & 0 & 1 \\ 0 & \omega & \omega & 1 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,2,3,0,2,1,1,3,2,0,

1,0,0,0,3,1,0,0,3,3,2,1,0,2,2,0,1,

1,3,1,1,1,2,2,0,1,0,0,3,0,1,1,3,0,

There are 0 extensions

Number of generators 3

### Orbit 1 / 3 at Level 17

Node number: 1127

$$\{0, 25, 50, 75, 90, 107, 140, 157, 179, 204, 213, 238, 265, 282, 299, 316, 356\}_{81600}$$

Strong generators for a group of order 81600:

$$\begin{bmatrix} \omega & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \omega & \omega^2 \\ 0 & 0 & 1 & 1 \end{bmatrix}_0, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 1 & \omega^2 & \omega & \omega^2 \\ 1 & \omega & 1 & 1 \end{bmatrix}_0, \\ \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ \omega & \omega^2 & 0 & 0 \\ 0 & 0 & 0 & \omega^2 \\ 0 & 0 & \omega & 1 \end{bmatrix}_1, \begin{bmatrix} \omega^2 & 0 & 0 & \omega \\ \omega & \omega^2 & 1 & \omega^2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \omega^2 & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & \omega^2 & \omega^2 & 0 \\ 1 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & \omega^2 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \\ \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & \omega & 1 \\ 0 & 1 & \omega & \omega^2 \\ \omega & 1 & 1 & 1 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,3,0,0,0,0,3,0,

1,0,0,0,0,1,0,0,0,0,2,3,0,0,1,1,0,

1,0,0,0,0,1,0,0,2,1,3,1,2,3,2,2,0,

1,0,0,0,3,1,0,0,0,0,0,1,0,0,3,2,1,

1,0,0,3,3,1,2,1,0,0,2,0,0,0,1,2,1,

0,1,1,0,2,0,1,1,0,0,2,1,0,0,0,2,0,

0,0,0,1,0,0,2,1,0,1,2,3,2,1,1,1,0,

There are 0 extensions

Number of generators 7

### Orbit 2 / 3 at Level 17

Node number: 1128

$$\{0, 25, 50, 75, 90, 108, 122, 140, 158, 183, 199, 217, 233, 250, 268, 312, 345\}_{576}$$

| Spreads in the Orbiter Catalogue |       |                     |
|----------------------------------|-------|---------------------|
| OCN                              | Aut   | Name                |
| 0                                | 1200  | Hall spread         |
| 1                                | 81600 | Desarguesian spread |
| 2                                | 576   | Semifield spread    |

Table 14.6: Spreads in PG(3, 4) in the Orbiter Catalogue

Strong generators for a group of order 576:

$$\begin{bmatrix} \omega & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ \omega & 0 & \omega & 1 \\ \omega^2 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ \omega & \omega^2 & 0 & 0 \\ \omega & 0 & 1 & 1 \\ \omega^2 & 0 & \omega^2 & 1 \end{bmatrix}_0, \\
 \begin{bmatrix} \omega & \omega & \omega & \omega \\ \omega^2 & 0 & \omega^2 & 0 \\ \omega^2 & 0 & \omega^2 & \omega \\ 0 & \omega^2 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & \omega^2 & 1 \\ 1 & \omega^2 & 1 & 0 \\ 1 & 0 & \omega & \omega \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & \omega^2 & \omega^2 & 0 \\ 0 & 0 & 0 & \omega^2 \\ 1 & 0 & 1 & \omega^2 \\ \omega & 1 & \omega & 1 \end{bmatrix}_0$$

1,0,0,0,0,2,0,0,0,0,2,0,0,0,3,1,  
 1,0,0,0,0,1,0,0,3,0,3,2,1,0,0,2,0,  
 1,0,0,0,3,1,0,0,3,0,2,2,1,0,1,2,0,  
 1,1,1,1,2,0,2,0,2,0,2,1,0,2,2,3,0,  
 1,0,3,1,1,3,1,0,1,0,2,2,0,0,1,1,  
 0,1,1,0,0,0,0,1,2,0,2,1,3,2,3,2,0,  
 There are 0 extensions  
 Number of generators 6

The three spreads in PG(3, 4) can be distinguished by their stabilizer orders. Table 14.6 lists the line spreads in PG(3, 4) according to their orbiter catalogue number (OCN).

Classification by substructure is a powerful method to classify objects. In this method, we do not classify the whole poset. Instead, we classify the substructures, and then move directly to the objects at the target level, thereby skipping over many intermediate levels. The jump is facilitated by an auxiliary algorithm such as clique finding. Table 14.7 shows the commands available for classification by substructure.

Let us now look at spreads in PG(3, 5), with substructures the partial spreads of size  $s = 5$ . We perform the lifting, which means we will construct for each partial spreads of size  $s$  all spreads of PG(3, 5) containing the chosen partial spread as a substructure. In a final step, we will perform an isomorph classification on the set of liftings. This will furnish the desired classification of spreads of PG(3, 5). From a computational point of view, the lifting process is the bottleneck in this procedure. To enhance the performance of the lifting, we use specialized algorithms from graph theory, based on cliques. A clique in a graph is a complete subgraph, i.e. a subset of the vertices such that any two in the subset are adjacent. A variant of this is rainbow clique finding. Here, we have a coloring of the vertices, so that edges are only present between vertices of different color. The goal of rainbow clique finding is to find a clique whose vertices represent all colors. Many problems in the field of combinatorial designs can be attacked using rainbow cliques.

To illustrate the technique, let us discuss some examples. Regarding the substructure, we choose the parameter  $s = 5$ . This is based on some experimentation. The command

| Isomorph Testing              |           |                                                                             |
|-------------------------------|-----------|-----------------------------------------------------------------------------|
| Command                       | Arguments | Purpose                                                                     |
| -prefix_iso                   | prefix    | Set a prefix for files related to isomorph testing.                         |
| -prefix_with_directory        | prefix    | Set a prefix for files related to isomorph testing.                         |
| -prefix_classify              | prefix    | Set a prefix for files related to classification.                           |
| -solution_prefix              | prefix    | Set a prefix for solution files.                                            |
| -base_fname                   | base      | Set the main file name.                                                     |
| -use_database_for_starter     |           | Use database to store subobjects.                                           |
| -implicit_fusion              |           | Use implicit fusion during the isomorphism testing.                         |
| -build_db                     |           | Initialize the database.                                                    |
| -read_solutions               |           | Read solutions.                                                             |
| -list_of_cases                | fname     | Read a list of cases from the given file.                                   |
| -read_solutions_after_split   | m         | Read the solution files after split modulo m.                               |
| -read_statistics_after_split  | m         | Read the statistics files after split modulo m.                             |
| -recognize                    | label     |                                                                             |
| -compute_orbits               |           | Compute the orbits of the stabilizer. These are the flag orbits.            |
| -isomorph_testing             |           | Apply isomorphism testing to the flag orbits to produce the classification. |
| -classification_graph         |           | Compute the classification class (after classification).                    |
| -event_file                   | fname     | Reuse a previously written event file during the classification.            |
| -print_mod                    | N         | Set print interval to N.                                                    |
| -isomorph_report              |           | Write a report on the classification.                                       |
| -export_source_code           |           | Export the classification as C++ source code.                               |
| -subset_orbits                |           |                                                                             |
| -subset_orbits_file           | fname     |                                                                             |
| -eliminate_graphs_if_possible |           |                                                                             |
| -down_orbits                  |           |                                                                             |

Table 14.7: Isomorph Testing

**Example 581**

```

classify_spreads_25_starter_lift_case_0:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 20 \
▷ ▷ ▷ ▷ -nodes_empty \
▷ ▷ ▷ ▷ -line_width 0.2 \
▷ ▷ ▷ ▷ -embedded \
▷ ▷ ▷ -end \
▷ ▷ ▷ -W \
▷ ▷ ▷ -problem_label spreads_25 \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define C -spread_classifier \
▷ ▷ ▷ -projective_space P \
▷ ▷ ▷ -k 2 \
▷ ▷ ▷ -starter_size 5 \
▷ ▷ ▷ -recoordinatize \
▷ ▷ ▷ -poset_classification_control Control \
▷ ▷ ▷ -output_prefix "" \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -prepare_lifting_single_case 0 \
▷ ▷ -end

```

classifies the partial spreads of size  $s = 5$  and prepares for the lifting of the first case only. In order to prepare for the lifting, a graph is constructed which describes the lines that can be added to the first partial spread. The vertices of the graph are the lines disjoint from the initial set of 5 lines in the partial spread. Two vertices are joined by an edge if the associated lines are disjoint. The vertices of the graph are colored according to the very first basis vector in the generator matrix of the subspace in reduced row echelon form. In order to find the rainbow cliques in the graph, the command

**Example 582**

```

spreads_25_starter_0_cliques:
▷ $(ORBITER) -v 2 \
▷ ▷ ▷ -define G -graph -load spreads_25_graph_0.bin -end \
▷ ▷ ▷ -with G -do \
▷ ▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 -end \
▷ ▷ ▷ -end \

```

can be used.

The command



**Example 583**

```

classify_spreads_25_starter_lift_all_cases:
▷ $(ORBITER) -v 3 \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 20 \
▷ ▷ ▷ ▷ -nodes_empty \
▷ ▷ ▷ ▷ -line_width 0.2 \
▷ ▷ ▷ ▷ -embedded \
▷ ▷ ▷ -end \
▷ ▷ ▷ -W \
▷ ▷ ▷ -problem_label spreads_25 \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define C -spread_classifier \
▷ ▷ ▷ -projective_space P \
▷ ▷ ▷ -k 2 \
▷ ▷ ▷ -starter_size 5 \
▷ ▷ ▷ -recoordinatize \
▷ ▷ ▷ -poset_classification_control Control \
▷ ▷ ▷ -output_prefix "" \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -prepare_lifting_all_cases \
▷ ▷ -end

```

recomputes the partial spreads of size  $s = 5$  and prepares for the lifting of all orbit representatives (there are 28). This leads to 28 graphs, each of which is written to a file. The next command performs the rainbow clique finding in each of the 28 graphs:

**Example 584**

```

spreads_25_starter_cliques:
▷ $(ORBITER) -v 2 \
▷ ▷ -loop L 0 29 1 \
▷ ▷ ▷ -define G -graph -load spreads_25_graph_%L.bin -end \
▷ ▷ ▷ -with G -do \
▷ ▷ ▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 -end \
▷ ▷ ▷ -end \
▷ ▷ -endloop L

```

The resulting cliques are again stored in files. The command

## Example 585

```

classify_spreads_25_isomorph:
▷ $(ORBITER) -v 4 \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -draw_options \
▷ ▷ ▷ ▷ -radius 20 \
▷ ▷ ▷ ▷ -nodes_empty \
▷ ▷ ▷ ▷ -line_width 0.2 \
▷ ▷ ▷ ▷ -embedded \
▷ ▷ ▷ -end \
▷ ▷ ▷ -W \
▷ ▷ ▷ -problem_label spreads_25 \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -define C -spread_classifier \
▷ ▷ ▷ -projective_space P \
▷ ▷ ▷ -k 2 \
▷ ▷ ▷ -starter_size 5 \
▷ ▷ ▷ -recoordinatize \
▷ ▷ ▷ -poset_classification_control Control \
▷ ▷ ▷ -output_prefix "" \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -build_db \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "" \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -read_solutions \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -compute_orbits \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
▷ ▷ ▷ -end \

```

| Spreads in the Orbiter Catalogue |        |      |
|----------------------------------|--------|------|
| OCN                              | Aut    | Name |
| 0                                | 1008   |      |
| 1                                | 1008   |      |
| 2                                | 1728   |      |
| 3                                | 216    |      |
| 4                                | 360    |      |
| 5                                | 288    |      |
| 6                                | 3600   |      |
| 7                                | 244800 |      |

Table 14.8: Spreads in  $PG(7, 2)$  in the Orbiter Catalogue

```

▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -isomorph_testing \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do -spread_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -isomorph_report \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex spreads_25_isomorphism_types.tex
▷ $(OPEN) spreads_25_isomorphism_types.pdf

```

performs the final isomorph rejection on the spreads arising from the rainbow cliques in all cases. It results in a transversal of the isomorphism classes of spreads of  $PG(3, 5)$ . In total, 21 spreads are found. This agrees with the results in the literature, see [23].

Table 14.8 lists the solid spreads in  $PG(7, 2)$  according to their Orbiter catalogue number (OCN).

| Activities for Translation Planes |           |                                                                      |
|-----------------------------------|-----------|----------------------------------------------------------------------|
| Command                           | Arguments | Purpose                                                              |
| -export_incma                     |           | Export the incidence matrix to file.                                 |
| -p_rank                           | p         | Compute the p-rank of the incidence matrix for the given prime $p$ . |
| -report                           |           | Produce a latex report.                                              |

Table 14.9: Activities for Translation Planes

## 14.2 Translation Planes

Orbiter can create translation planes from spreads. The construction of translation planes from spreads is due to André and Bruck, Bose (cf. [3, 18]). In order to perform the construction, we need a field  $F = \mathbb{F}_q$  which is the kernel of the plane, a spread of  $k$ -subspaces, and the groups  $\text{PTL}(2k, F)$  and  $\text{PTL}(2k + 1, F)$ .

Table 14.9 lists Orbiter activities for translation planes.

For instance, the command

### Example 586

```

create_translation_plane_9b:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define G -linear_group -PGL 4 F -end \
▷ ▷ -define G1 -linear_group -PGL 5 F -end \
▷ ▷ -define S -spread -kernel_field F \
▷ ▷ ▷ -group G -k 2 -catalogue 1 \
▷ ▷ ▷ -end \
▷ ▷ -define T -translation_plane S G G1 -end \
▷ ▷ -with T -do -translation_plane_activity \
▷ ▷ ▷ -export_incma \
▷ ▷ -end \
▷ ▷ -with T -do -translation_plane_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -define A -linear_group -import_group_of_plane T -end \
▷ ▷ -define Orb -orbits -group A \
▷ ▷ ▷ -on_points \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -report \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -stabilizer 92 \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -export_trees \
▷ ▷ -end
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file plane_catalogue_q3_k2_1_incma.csv \
▷ ▷ ▷ -box_width 6 -bit_depth 8 \

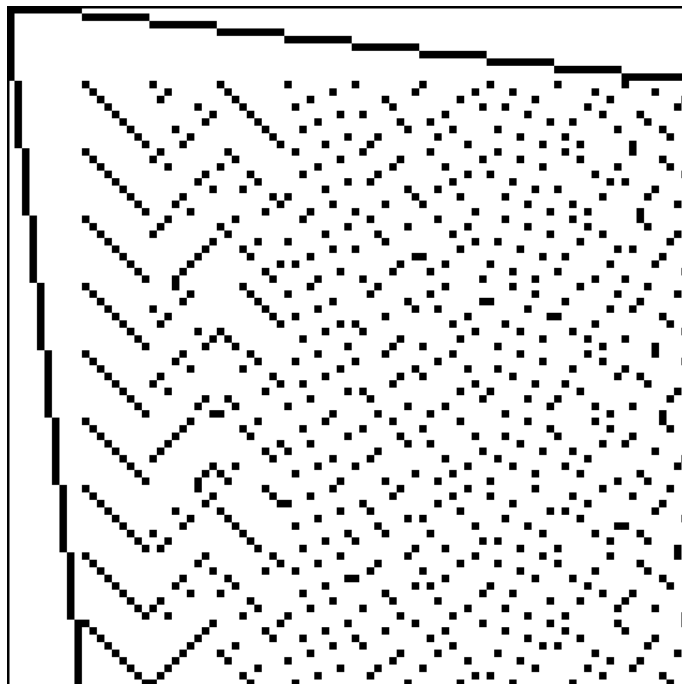
```

```

> > > -partition 2 91 91 \
> > -end
> $(ORBITER) -v 3 \
> > -draw_layered_graph \
> > > orbit_PGL_5_3_on_andre_3.layered_graph \
> > > -radius 250 -spanning_tree -embedded -nodes_empty \
> > > -line_width 1.1 -x_stretch 2.4 -scale 0.15 \
> > -end
> pdflatex orbit_PGL_5_3_on_andre_3_draw.tex
> $(OPEN) orbit_PGL_5_3_on_andre_3_draw.pdf
> pdflatex group_of_plane_plane_catalogue_q3.k2.1.orbits_report.tex
> $(OPEN) group_of_plane_plane_catalogue_q3.k2.1.orbits_report.pdf
> pdflatex group_of_plane_plane_catalogue_q3.k2.1.stab_pt.92.report.tex
> $(OPEN) group_of_plane_plane_catalogue_q3.k2.1.stab_pt.92.report.pdf

```

creates the (projective) Hall plane of order 9 from the Hall spread. In this example, we use the fact that  $\text{PTL}(n, q) = \text{PGL}(n, q)$  if  $q$  is prime. The example also creates a bitmap drawing of the incidence matrix of the plane, shown below:



In the next example, we will create a translation plane of order 16 with kernel of order 4:

#### Example 587

```

create_translation_plane_16_4.0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 4 -end \
> > -define G -linear_group -PGGL 4 F -end \

```

```

> > -define G1 -linear_group -PGGL 5 F -end \
> > -define S -spread -kernel_field F \
> > > -group G -k 2 -catalogue 0 \
> > > -end \
> > -define T -translation_plane S G G1 -end \
> > -with T -do -translation_plane_activity \
> > > -export_incma \
> > -end
> $(ORBITER) -v 2 \
> > -draw_matrix \
> > > -input_csv_file plane_catalogue_q4_k2_0_incma.csv \
> > > -box_width 6 -bit_depth 8 \
> > > -partition 4 273 273 \
> > -end
> $(OPEN) plane_catalogue_q4_k2_0_incma.draw.bmp

```

This plane is the Hall plane, and the spread is the Hall spread. The stabilizer of the spread has order 1200.

In the next example, we will create a translation plane of order 16 with kernel of order 2:

#### Example 588

```

create_translation_plane_16_2_0:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 2 -end \
> > -define G -linear_group -PGL 8 F -end \
> > -define G1 -linear_group -PGL 9 F -end \
> > -define S -spread -kernel_field F \
> > > -group G -k 4 -catalogue 0 \
> > > -end \
> > -define T -translation_plane S G G1 -end \
> > -with T -do -translation_plane_activity \
> > > -export_incma \
> > -end
> $(ORBITER) -v 2 \
> > -draw_matrix \
> > > -input_csv_file plane_catalogue_q2_k4_0_incma.csv \
> > > -box_width 6 -bit_depth 8 \
> > > -partition 4 273 273 \
> > -end
> $(OPEN) plane_catalogue_q2_k4_0_incma.draw.bmp

```

The spread has a stabilizer of order 1008, which means that the associated translation plane has a stabilizer of order  $1008 \cdot 256 = 258045$ . According to [53], there are two planes whose associated spreads have an automorphism group of this order. They can be distinguished by the 2-rank of their incidence matrices. The Johnson-Walker plane has a 2-rank of 100. The Lorimer-Rahilly plane has a 2-rank of 106. Using Orbiter, we compute the 2-rank of the translation plane that we have created:

**Example 589**

```

RREF_plane_16_2_0_rank_of_incma:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define v -vector -field F \
> > > -file plane_catalogue_q2_k4_0_incma.csv \
> > -end \
> > -with F -do -finite_field_activity \
> > -RREF v \
> > -end

```

It turns out that the 2-rank of our plane is 106, so the plane is Lorimer-Rahilly.

Let us investigate the Rao / Rao plane from Section 14.1, which we know is isomorphic to the spread in the Orbiter catalogue with number 0 and projective stabilizer of order 84. The command

**Example 590**

```

create_translation_plane_27_Rao_Rao:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 3 -end \
> > -define SS -vector -dense $(SPREAD_SET_27_RA0_RA0) -end \
> > -define G -linear_group -PGL 6 F -end \
> > -define G1 -linear_group -PGL 7 F -end \
> > -define S -spread -kernel_field F \
> > > -group G -k 3 -spread_set "Rao_Rao" "Rao\_Rao" SS \
> > > -end \
> > -define T -translation_plane S G G1 -end \
> > -with T -do -translation_plane_activity \
> > > -export_incma \
> > -end

```

creates the translation plane from the spread. To compute the 3-rank of the incidence matrix, we issue the following command:

**Example 591**

```

RREF_Rao_Rao_plane_incma_rank:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 3 -end \
> > -define v -vector -field F \
> > > -file plane_Rao_Rao_incma.csv \
> > -end \
> > -with F -do -finite_field_activity \
> > > -RREF v \
> > -end

```

The 3-rank turns out to be 271. According to the Moorhouse tables [54], the plane has Moorhouse number IV.

Suppose we want to study the orbits of the group of a translation plane on points and blocks. The following command creates the translation plane of order 27 with Orbiter number 4. Once done, it creates the action on points and blocks and invokes the Schreier orbit algorithm (see Section 7.1).

**Example 592**

```
translation_plane_27_4_orbits:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 3 -end \
> > -define G -linear_group -PGL 6 F -end \
> > -define G1 -linear_group -PGL 7 F -end \
> > -define S -spread -kernel_field F \
> > > -group G -k 3 -catalogue 4 \
> > > -end \
> > -define T -translation_plane S G G1 -end \
> > -with T -do -translation_plane_activity \
> > > -export_incma \
> > -end \
> > -with T -do -translation_plane_activity \
> > > -report \
> > -end \
> > -define A -linear_group -import_group_of_plane T -end \
> > -define Orb -orbits -group A \
> > > -on_points \
> > -end
```

The group of the plane has order 1592136. The degree of the action is 1514. The orbit lengths are

1, 28, 729, 756.



| Creating Packings with Assumed Symmetry |             |                                                                                                                                                                    |
|-----------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                 | Arguments   | Purpose                                                                                                                                                            |
| -H                                      | description | Specify the assumed group $H$ of symmetries. The orbits of $H$ on the set of spreads are considered. The packings will be constructed as union of orbits.          |
| -N                                      | description | Specify the normalizer of $H$ .                                                                                                                                    |
| -cliques_on_fixpoint_graph              | $s$         | Using poset classification, classify the orbits of $N$ on cliques of size $\leq s$ in the graph on fixed points.                                                   |
| -cliques_on_fixpoint_graph_control      | descr       | Specify poset classification options related to the classification of cliques on the fixed point graph as in Table 7.3.                                            |
| -fixp_clique_types_save_individually    |             | Sort the cliques on fixed points by the type of their spreads and write one csv file for each possible type containing the index of the cliques of the given type. |
| -process_long_orbits                    | descr       | Proceed on to long orbits using Table 14.13.                                                                                                                       |
| -spread_tables_prefix                   | $P$         | Use prefix $P$ to access spread tables.                                                                                                                            |
| -report                                 |             | Create a report of the classification process.                                                                                                                     |
| -regular_packing                        |             | Initialize Klein correspondence and identify (regular) spreads with external lines to the Klein quadric using the polarity of the Klein quadric.                   |

Table 14.10: Creating Packings with Assumed Symmetry

## 14.3 Packings

A packing of  $\text{PG}(3, q)$  is a set of pairwise line-disjoint spreads of  $\text{PG}(3, q)$  of size  $q^2 + q + 1$ . Each spread contains  $q^2 + 1$  lines. A simple counting argument shows that every line is contained in exactly one spread of the packing. The classification problem for packings is the problem of determining a complete set of pairwise non-isomorphic packings. Orbiter can be used to classify packings for small parameters. It is sometimes useful to make a symmetry assumption. This means that only those packings will be found that satisfy the symmetry assumption. The reason for making such an assumption is that the problem becomes easier and hence more tractable. Often, an assumption is made that the packings are invariant under a (nontrivial) group  $H$ . This section describes various ways in which Orbiter can help find and classify packings, with or without symmetry assumption.

Table 14.10 list Orbiter commands related to the construction of packings with assumed symmetry.

Table 14.11 list Orbiter activities for packings.

Table 14.12 list Orbiter activities for packings with assumed symmetry and with fixpoints.

Table 14.13 list Orbiter commands related to the construction of packings with assumed symmetry by picking long orbits.

The following command creates a table of all labeled spreads in  $\text{PG}(3, 4)$ . There are three isomorphism types of spreads in  $\text{PG}(3, 4)$ . The command computes the orbits of each. In total, this gives 5096448 labeled spreads.

| Activities for Packings       |           |                                                         |
|-------------------------------|-----------|---------------------------------------------------------|
| Command                       | Arguments | Purpose                                                 |
| -report                       |           | Produce a latex report                                  |
| -export_reduced_spread_orbits | fname     | Export reduced spread orbits.                           |
| -create_graph_on_mixed_orbits | L         | Create the graph of orbits whose length is listed in L. |

Table 14.11: Activities for Packings

| Activities for Packings with Fixpoints |                  |                                          |
|----------------------------------------|------------------|------------------------------------------|
| Command                                | Arguments        | Purpose                                  |
| -report                                |                  | Produce a latex report                   |
| -print_packing                         | text             | Print the packing.                       |
| -compare_files_of_packings             | fname1<br>fname2 | Compare packings in the two given files. |

Table 14.12: Activities for Packings with Fixpoints

| Creating Packings from Long Orbits |           |                                                                                                                                               |
|------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Command                            | Arguments | Purpose                                                                                                                                       |
| -split                             | $r$ $m$   | Define a subset of cases of fixed point cliques to be worked on. Only those cases whose number is congruent to $r$ modulo $m$ are considered. |
| -orbit_length                      | $l$       | Use orbits of length $l$ .                                                                                                                    |
| -mixed_orbits                      | length    |                                                                                                                                               |
| -list_of_cases_from_file           | fname     | Define a subset of cases of fixed point cliques to be worked on. Only the cases listed the given file are considered.                         |
| -solution_path                     | $P$       | Use $P$ as a prefix for all solution files.                                                                                                   |
| -create_graphs                     |           | For each case, create the graph that describes whether two orbits of length $l$ are compatible.                                               |
| -solve                             |           | Perform clique finding and write solutions to file.                                                                                           |
| -read_solutions                    |           | Read solutions from file.                                                                                                                     |

Table 14.13: Creating Packings from Long Orbits

**Example 593**

```
spread_table_PG_3_4:
> - mkdir SPREAD_TABLES_4
> $(ORBITER) -v 6 \
> > -define Control -poset_classification_control \
> > > -problem_label spreads_PG_3_4 \
> > -end \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -define T -spread_table P 2 "0,1,2" "SPREAD_TABLES_4/" Control
```

There are 21 isomorphism types of spreads in  $\text{PG}(3, 5)$ . The regular spread has Orbiter catalogue number equal to 12. The following command creates a table of all regular spreads in  $\text{PG}(3, 5)$ :

**Example 594**

```
spread_table_PG_3_5_regular:
> - mkdir SPREAD_TABLES_5_REG
> $(ORBITER) -v 6 \
> > -define Control -poset_classification_control \
> > > -problem_label spreads_PG_3_5 \
> > -end \
> > -define F -finite_field -q 5 -end \
> > -define P -projective_space -n 3 -field F -end \
> > -define T -spread_table P 2 "12" "SPREAD_TABLES_5_REG/" Control \
> > -print_symbols
```

There are 155000 regular spreads.

Suppose we are interested in packings invariant under a group of order 31. We pick the matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 4 & 3 \\ 0 & 3 & 3 & 4 \\ 0 & 3 & 2 & 3 \end{bmatrix}$$

of order 124 in  $\text{GL}(4, 5)$ . At first, we store the element in a makefile variable:

**Example 595**

```
PGL_4_5_SUBGROUP_31_ME=-PGL 4 5 \
> -subgroup_by_generators "31" 31 1 \
> "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3"
```

We compute the normalizer of the cyclic subgroup generated by this element inside  $\text{GL}(4, 5)$ :

**Example 596**

```
PG_3.5_element_of_order_31_GL_normalizer:
> $(ORBITER) -v 6 -define G \
> > -linear_group -GL 4 5 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -normalizer_of_cyclic_subgroup "124" \
> > > "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3" \
> > -end
> pdflatex normalizer_of_124_in_GL_4_5.tex
> $(OPEN) normalizer_of_124_in_GL_4_5.pdf
```

We compute the normalizer of the cyclic subgroup generated by this element inside  $\text{PGL}(4, 5)$ :

**Example 597**

```
PG_3.5_element_of_order_31_ME_normalizer:
> $(ORBITER) -v 6 -define G \
> > -linear_group -PGL 4 5 -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -normalizer_of_cyclic_subgroup "31" \
> > > "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3" \
> > -end
> mv normalizer_of_31_in_PGL_4_5.tex normalizer_of_31_ME_in_PGL_4_5.tex
> pdflatex normalizer_of_31_ME_in_PGL_4_5.tex
> $(OPEN) normalizer_of_31_ME_in_PGL_4_5.pdf
```

We store the normalizer in a makefile variable:

**Example 598**

```
PGL_4.5_SUBGROUP_31_ME_NORMALIZER=-PGL 4 5 \
> -subgroup_by_generators "normalizer_31" "372" 4 \
> "1,0,0,0,0,4,0,0,0,0,4,0,0,0,4, \
> 1,0,0,0,0,3,0,0,0,0,3,0,0,0,0,3, \
> 1,0,0,0,0,4,0,0,0,0,2,1,0,3,2,4, \
> 1,0,0,0,0,0,1,0,0,0,0,1,0,1,1,3,"
```

We create packings invariant under the cyclic group of order 31 generated by this element.

**Example 599**

```
PG_3.5_assume_31_graph:
> $(ORBITER) -v 5 \
> > -define Control -poset_classification_control \
> > > -problem_label spreads_PG_3.5 \
> > -end \
> > -define F -finite_field -q 5 -end \
```

```
▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \  
▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \  
▷ ▷ -define T -spread_table P3 2 "12" "SPREAD_TABLES_5_REG/" Control \  
▷ ▷ -define PC -packing_classify P3 P5 T \  
▷ ▷ -define PW -packing_with_symmetry_assumption PC \  
▷ ▷ ▷ ▷ -H "H31" $(PGL_4_5_SUBGROUP_31_ME) -end \  
▷ ▷ ▷ ▷ -N "N31" $(PGL_4_5_SUBGROUP_31_ME_NORMALIZER) -end \  
▷ ▷ ▷ -end \  
▷ ▷ -define PWF -packing_choose_fixed_points PW 0 -end \  
▷ ▷ -define L -packing_long_orbits PWF \  
▷ ▷ ▷ -orbit_length 31 -create_graphs \  
▷ ▷ ▷ -end \  
▷ ▷ -print_symbols
```

| Creating BLT-Sets |           |                                                                                     |
|-------------------|-----------|-------------------------------------------------------------------------------------|
| Command           | Arguments | Purpose                                                                             |
| -catalogue        | $i$       | Create the BLT-set number $i$ from the Orbiter catalogue ( $i$ is zero-based).      |
| -family           | name      | Create a BLT-set from the named family. See Table 14.15 for possibilities for name. |
| -flock            | flock     | Create a BLT-set from a given flock.                                                |
| -space            | space     | Set the space.                                                                      |
| -invariants       |           | Compute isomorphism invariants.                                                     |

Table 14.14: Creating BLT-Sets

| Families of BLT-sets |                            |                                                                       |
|----------------------|----------------------------|-----------------------------------------------------------------------|
| Command              | Condition                  | Purpose                                                               |
| Linear               |                            | Linear BLT-set.                                                       |
| Fisher               |                            | Fisher BLT-set [27].                                                  |
| Mondello             | $q \equiv \pm 1 \pmod{10}$ | Mondello BLT-set due to Penttila [56].                                |
| FTWKB                | $q \equiv \pm 2 \pmod{3}$  | Fisher, Thas, Walker [68], Kantor, Betten [15] BLT-set.               |
| Kantor1              | $q = p^e, e > 1$           | Kantor's first family.                                                |
| Kantor2              | $q \equiv \pm 2 \pmod{5}$  | Kantor's second family.                                               |
| LP_37_72             | $q = 37$                   | BLT-set for $q = 37$ with ago=72 due to Law and Penttila [45].        |
| LP_37_41a            | $q = 37$                   | First BLT-set for $q = 37$ with ago=4, due to Law and Penttila [45].  |
| LP_37_41b            | $q = 37$                   | Second BLT-set for $q = 37$ with ago=4, due to Law and Penttila [45]. |
| LP_71                | $q = 71$                   | BLT-set for $q = 71$ due to Law and Penttila [45].                    |

Table 14.15: Families of BLT-sets

## 14.4 BLT-Sets

A BLT-set of  $Q(4, q)$  is a set of  $q + 1$  point on the quadric such that no point on the quadric is collinear to more than two points of the set. BLT sets are related to spreads of  $PG(3, q)$ , to flocks of the quadratic cone in  $PG(3, q)$ , and other objects in finite geometry. These sets have been introduced in [4] for the purposes of creating new flocks from old. A BLT-set exists if and only if  $q$  is odd. It is an interesting problem to classify BLT-sets of  $Q(4, q)$  under the orthogonal group. Some references are Law [43], Penttila-Royle [57], Penttila-Law [44, 45], Betten [9], AlAzemi-Betten-Chowdhury [1]. For small values of  $q$ , computer search has been used extensively. As we will discuss below, Orbiter is useful to supporting such computations.

Before we go into the problem of classification, let us first discuss how known BLT-sets can be constructed in Orbiter. Orbiter maintains a catalogue of BLT-sets over small fields. Besides that, Orbiter can be used to create instances of known families of BLT-sets.

Table 14.14 shows options to create known BLT-sets.

Table 14.15 shows names for known families of sporadic sets.

| BLT-Set Activity  |           |                                                             |
|-------------------|-----------|-------------------------------------------------------------|
| Command           | Arguments | Purpose                                                     |
| -report           |           | Produce a latex report.                                     |
| -export_gap       | index     | Export to GAP/fining based on the indexed point.            |
| -create_flock     |           | Create a flock.                                             |
| -BLT_test         |           | Test of the set has the BLT-property.                       |
| -export_set_in_PG |           | Export the set in ranks of the underlying projective space. |
| -plane_invariant  |           | Compute the plane invariant.                                |

Table 14.16: BLT-Set Activity

| Classifying BLT-Sets |           |                                                              |
|----------------------|-----------|--------------------------------------------------------------|
| Command              | Arguments | Purpose                                                      |
| -starter_size        | s         | Set the size of partial BLT-sets that is used as subobjects. |

Table 14.17: Classifying BLT-Sets

Table 14.16 shows Orbiter activities for a BLT-set object.

Table 14.17 shows options to prepare for the classification of BLT-sets.

Table 14.18 shows commands for the classification of BLT-sets.

Let us demonstrate some examples. The command

#### Example 600

```
BLT_5.1:
▷ $(ORBITER) -v 10 \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -end \
▷ ▷ -define BLT -BLT_set \
▷ ▷ ▷ -space 0 -catalogue 1 -invariants \
▷ ▷ -end \
```

| Activities for the Classification of BLT-sets |           |                                                                                          |
|-----------------------------------------------|-----------|------------------------------------------------------------------------------------------|
| Command                                       | Arguments | Purpose                                                                                  |
| -compute_starter                              |           | Perform classification of partial spreads.                                               |
| -poset_classification_activity                | label     |                                                                                          |
| -create_graphs                                |           | Prepare to lift all cases of partial BLT-sets by creating the associated colored graphs. |
| -split                                        | r m       | Lift only those cases $k$ with $k \equiv r \pmod{m}$ .                                   |
| -isomorph                                     | options   | Perform isomorphism testing after lifting is complete.                                   |

Table 14.18: Activities for the Classification of BLT-sets

```

▷ ▷ -with BLT -do -blt_set_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex BLT_catalogue_q5_iso1.tex
▷ $(OPEN) BLT_catalogue_q5_iso1.pdf

```

creates the first BLT-set of order 5 from the catalogue. Suppose we want to export the BLT-set to GAP/fining. The following command can do that:

#### Example 601

```

BLT_5_1.export_gap:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 5 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -end \
▷ ▷ -define BLT -BLT_set \
▷ ▷ ▷ -space 0 -catalogue 1 \
▷ ▷ -end \
▷ ▷ -with BLT -do -blt_set_activity \
▷ ▷ ▷ -export_gap \
▷ ▷ -end

```

The command creates the following GAP/fining file:

```

LoadPackage("fining");
# BLT-set catalogue_q5_iso1
# Quadratic form:  $X_0^2 + X_1X_2 + X_3X_4$ 
# Group of order 720
mat1 := [[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^3, 0*Z(5)],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), 0*Z(5), Z(5)^0, 0*Z(5)],
[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^2, Z(5)^0]];
frob1 := FrobeniusAutomorphism(GF(5))^0;
psi1 := ProjectiveSemilinearMap(mat1, frob1, GF(5));
mat2 := [[Z(5)^0, 0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5)],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), 0*Z(5), Z(5)^1, 0*Z(5)],
[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^3, Z(5)^3]];
frob2 := FrobeniusAutomorphism(GF(5))^0;
psi2 := ProjectiveSemilinearMap(mat2, frob2, GF(5));
mat3 := [[Z(5)^3, 0*Z(5), 0*Z(5), Z(5)^1, Z(5)^0],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[Z(5)^3, 0*Z(5), 0*Z(5), Z(5)^2, Z(5)^2],
[Z(5)^0, 0*Z(5), 0*Z(5), Z(5)^0, Z(5)^2]];
frob3 := FrobeniusAutomorphism(GF(5))^0;
psi3 := ProjectiveSemilinearMap(mat3, frob3, GF(5));
mat4 := [[Z(5)^3, 0*Z(5), 0*Z(5), Z(5)^0, Z(5)^1],

```



```

[0*Z(5),Z(5)^2,0*Z(5),0*Z(5),0*Z(5)],
[0*Z(5),0*Z(5),Z(5)^2,0*Z(5),0*Z(5)],
[Z(5)^2,0*Z(5),0*Z(5),Z(5)^0,Z(5)^2],
[Z(5)^1,0*Z(5),0*Z(5),Z(5)^0,Z(5)^0]];
frob4 := FrobeniusAutomorphism(GF(5))^0;
psi4 := ProjectiveSemilinearMap(mat4, frob4,GF(5));
mat5 := [[Z(5)^1,Z(5)^0,0*Z(5),Z(5)^1,Z(5)^0],
[0*Z(5),Z(5)^0,0*Z(5),0*Z(5),0*Z(5)],
[Z(5)^3,Z(5)^3,Z(5)^0,Z(5)^0,Z(5)^3],
[Z(5)^3,Z(5)^3,0*Z(5),Z(5)^0,Z(5)^0],
[Z(5)^0,Z(5)^0,0*Z(5),Z(5)^2,Z(5)^0]];
frob5 := FrobeniusAutomorphism(GF(5))^0;
psi5 := ProjectiveSemilinearMap(mat5, frob5,GF(5));
mat6 := [[Z(5)^1,0*Z(5),0*Z(5),Z(5)^3,Z(5)^2],
[Z(5)^0,Z(5)^0,Z(5)^1,Z(5)^1,Z(5)^0],
[0*Z(5),0*Z(5),Z(5)^0,0*Z(5),0*Z(5)],
[Z(5)^3,0*Z(5),Z(5)^1,Z(5)^3,Z(5)^1],
[Z(5)^0,0*Z(5),Z(5)^2,Z(5)^3,Z(5)^3]];
frob6 := FrobeniusAutomorphism(GF(5))^0;
psi6 := ProjectiveSemilinearMap(mat6, frob6,GF(5));
gens := [psi1, psi2, psi3, psi4, psi5, psi6];
G := Group(gens);
Size(G);
pg := ProjectiveSpace(4,5);
S:=
[0*Z(5),Z(5)^0,0*Z(5),0*Z(5),0*Z(5)],
[0*Z(5),0*Z(5),Z(5)^0,0*Z(5),0*Z(5)],
[0*Z(5),Z(5)^0,Z(5)^1,Z(5)^2,Z(5)^1],
[Z(5)^0,Z(5)^0,Z(5)^1,Z(5)^1,Z(5)^0],
[Z(5)^0,Z(5)^3,Z(5)^0,Z(5)^2,Z(5)^2],
[Z(5)^0,Z(5)^3,Z(5)^0,Z(5)^3,Z(5)^1]
];
S := List(S,x -> VectorSpaceToElement(pg,x));

```

The command

#### Example 602

```

BLT_11_0:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 11 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -end \
▷ ▷ -define BLT -BLT_set \
▷ ▷ ▷ -space 0 -catalogue 0 -invariants \
▷ ▷ -end \
▷ ▷ -with BLT -do -blt_set_activity \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex BLT_catalogue_q11_iso0.tex
▷ $(OPEN) BLT_catalogue_q11_iso0.pdf

```

creates the BLT-set #0 in  $Q(4,11)$ . The command

**Example 603**

```

BLT_11_Mondello:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 11 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -define BLT -BLT_set \
> > > -space 0 -family "Mondello" -invariants \
> > -end \
> > -with BLT -do -blt_set_activity \
> > > -report \
> > -end
> pdflatex BLT_Mondello_q11.tex
> $(OPEN) BLT_Mondello_q11.pdf

```

creates the Mondello BLT-set in  $Q(4, 11)$ . Orbiter creates the following report:

The quadratic form is:

$$X_0^2 + X_1X_2 + X_3X_4 = 0$$

The BLT-set is:

| $i$ | Rank | Point            | $(a, b, c)$  |
|-----|------|------------------|--------------|
| 0   | 846  | (1, 6, 4, 10, 3) | (22, 11, 1)  |
| 1   | 851  | (1, 5, 7, 10, 3) | (22, 110, 1) |
| 2   | 1234 | (1, 5, 1, 7, 7)  | (37, 11, 1)  |
| 3   | 613  | (1, 6, 10, 5, 1) | (73, 110, 1) |
| 4   | 1307 | (1, 1, 3, 8, 5)  | (59, 36, 1)  |
| 5   | 1418 | (1, 3, 9, 6, 10) | (95, 36, 1)  |
| 6   | 1022 | (1, 9, 5, 10, 2) | (99, 96, 1)  |
| 7   | 835  | (1, 2, 6, 3, 3)  | (99, 36, 1)  |
| 8   | 950  | (1, 10, 8, 2, 9) | (95, 96, 1)  |
| 9   | 789  | (1, 8, 2, 4, 4)  | (59, 96, 1)  |
| 10  | 611  | (1, 7, 7, 5, 1)  | (73, 11, 1)  |
| 11  | 1236 | (1, 4, 4, 7, 7)  | (37, 110, 1) |

Plane intersection type is  $4^{18} 3^{148}$

Plane invariant is too big (18 planes)

$$\begin{array}{c|c} \rightarrow & 18_1 \\ \hline 12_0 & 6 \end{array} \quad \begin{array}{c|c} \downarrow & 18_1 \\ \hline 12_0 & 4 \end{array}$$

$$C_0 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}_{12}$$

$$C_1 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17\}_{18}$$

$$\begin{array}{c|c} \rightarrow & 18_1 \\ \hline 12_0 & 6 \end{array}$$

$$\begin{array}{c|c} \downarrow & 18_1 \\ \hline 12_0 & 4 \end{array}$$

$$C_0 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}_{12}$$

$$C_1 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17\}_{18}$$

The classification of BLT-sets is a difficult problem. One approach is by means of the poset of partial BLT-sets. The orbits at level  $q + 1$  in the poset of partial BLT-sets are the isomorphisms classes of BLT-sets. We wish to illustrate this method by an example. The example is quite small, in order to allow us to compare different construction algorithms.

We begin with a classification of the complete poset of partial BLT sets. This method is very limited, as the number of orbits of partial BLT-sets gets out of hand quickly as  $q$  increases.

#### Example 604

```
BLT_13_deep_search:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 13 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -define C -BLT_set_classifier 0 -starter_size 14 -end \
> > -with C -do -BLT_set_classify_activity \
> > > -compute_starter \
> > > > -problem_label BLT_q13 \
> > > > -W -depth 14 \
> > > -end \
> > -end
> #pdflatex BLT_q13_poset.tex
> #$(OPEN) BLT_q13_poset.pdf
```

The technique of classification via substructure can help. Various authors have described different versions of this technique, see [1, 9, 43]. As an example, we classify the BLT-sets of order 13 with the technique of substructures and rainbow cliques.

We consider the classification problem of BLT-sets of order 13. We use the technique of substructures of size 5. So, to begin with, we classify all partial BLT-sets of size 5. For each isomorphism type of partial BLT-set of size 5, we create a colored graph. The vertices in the graph are the points in  $Q(4, 13)$  which can be added to the partial BLT-set. Two vertices are connected if they can both be added. Here is the command:

#### Example 605

```
BLT_13_classify_starter:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 13 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -define C -BLT_set_classifier 0 -starter_size 5 -end \
> > -with C -do -BLT_set_classify_activity \
> > > -compute_starter \
> > > > -problem_label BLT_q13 \
> > > > -W -depth 5 \
> > > -end \
> > -end \
```

```

▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -create_graphs \
▷ ▷ -end

```

In the next step, we compute all rainbow cliques in each of the graphs:

#### Example 606

```

BLT_13_cliques:
▷ $(ORBITER) -v 2 \
▷ ▷ -loop L 0 38 1 \
▷ ▷ ▷ -define G -graph \
▷ ▷ ▷ ▷ -load BLT_q13_graph_5_%L.bin \
▷ ▷ ▷ -end \
▷ ▷ ▷ -with G -do \
▷ ▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 9 -end \
▷ ▷ ▷ -end \
▷ ▷ -end_loop L

```

Next, we create a data structure for isomorphism testing. The first step is to create a database of all partial BLT-sets of order at most 5:

#### Example 607

```

BLT_13_isomorph_read_DB:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -end \
▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ ▷ ▷ -problem_label BLT_q13 \
▷ ▷ ▷ ▷ -W -depth 5 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -build_db \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "" \
▷ ▷ ▷ -end \
▷ ▷ -end

```

The next step is to read the rainbow cliques from the clique finding process:

**Example 608**

```

BLT_13_isomorph_read_solutions:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -end \
▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ ▷ ▷ -problem_label BLT_q13 \
▷ ▷ ▷ ▷ -W -depth 5 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -read_solutions \
▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
▷ ▷ ▷ -end \
▷ ▷ -end

```

Once the cliques have been read from file, we compute the flag orbits in the relation between partial BLT-sets and full BLT-sets. For each orbit representative of a partial BLT-set, we compute the orbits of the stabilizer on the BLT-sets incident with it. The following command can be used.

**Example 609**

```

BLT_13_isomorph_stabilizer_orbits:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define O -orthogonal_space 0 5 F -end \
▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -compute_starter \
▷ ▷ ▷ ▷ -problem_label BLT_q13 \
▷ ▷ ▷ ▷ -W -depth 5 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do -BLT_set_classify_activity \
▷ ▷ ▷ -isomorph \
▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
▷ ▷ ▷ ▷ -use_database_for_starter \
▷ ▷ ▷ ▷ -compute_orbits \
▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
▷ ▷ ▷ ▷ -solution_prefix "" \
▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
▷ ▷ ▷ -end \
▷ ▷ -end

```

Finally, we perform isomorphism testing:

#### Example 610

```
BLT_13_isomorph_testing:
> $(ORBITER) -v 4 \
> > -define F -finite_field -q 13 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -define C -BLT_set_classifier 0 -starter_size 5 -end \
> > -with C -do -BLT_set_classify_activity \
> > > -compute_starter \
> > > > -problem_label BLT_q13 \
> > > > -W -depth 5 \
> > > -end \
> > -end \
> > -with C -do -BLT_set_classify_activity \
> > > -isomorph \
> > > > -prefix_iso "./BLT_q13" \
> > > > -use_database_for_starter \
> > > > -isomorph_testing \
> > > > -solution_prefix "" \
> > > > -base_fname "BLT_q13_graph" \
> > > -end \
> > -end \
> > -with C -do -BLT_set_classify_activity \
> > > -isomorph \
> > > > -prefix_iso "./BLT_q13" \
> > > > -use_database_for_starter \
> > > > -isomorph_report \
> > > > -solution_prefix "" \
> > > > -base_fname "BLT_q13_graph" \
> > > -end \
> > -end
> pdflatex BLT_q13_isomorphism_types.tex
> $(OPEN) BLT_q13_isomorphism_types.pdf
```

This last command results in exactly three isomorphism types of BLT-sets of order 13. Of course, these results match with the results from the classification of the complete poset of partial BLT-sets before. The difference is that the second approach is computationally much less demanding, as the number of orbits to be considered is much smaller. This is because many partial BLT-sets do not complete to BLT-sets. This means that the classification of poset of partial BLT-sets wastes a lot of time classifying things that are irrelevant.

## Chapter 15

# Computer Science Primitives

### 15.1 Clique Finding

A clique in a graph  $\Gamma = (V, E)$  is a subset  $S$  of the vertices such that any two elements of  $S$  are adjacent in  $\Gamma$ . Finding and classifying cliques in graphs is important for many applications of graph theory. Orbiter can help. The command `-find_cliques` command from Table 12.4 can be used to find all cliques in a graph. Additional options for this command are shown in Table 15.1. For instance, the cliques of size 3 in the graph `graph_v5_e7.colored_graph` from Section 12.1 can all be found using the following command:

#### Example 611

```
small_graph_cliques: graph_v5_e7.colored_graph
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -graph -load graph_v5_e7.colored_graph -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -find_cliques -target_size 3 \
▷ ▷ -end
```

It is also possible to classify all cliques under the automorphism group of the graph. This is a multi-

| Finding Cliques            |           |                                                                                                                 |
|----------------------------|-----------|-----------------------------------------------------------------------------------------------------------------|
| Command                    | Arguments | Purpose                                                                                                         |
| <code>-rainbow</code>      |           | Find all rainbow cliques. The size of the cliques is the number of vertex colors.                               |
| <code>-target_size</code>  | $s$       | Find all cliques of size $s$ .                                                                                  |
| <code>-weighted</code>     | $s$       | Find weighted cliques.                                                                                          |
| <code>-Sajeeb</code>       |           | Use the implementation by Sajeeb Chowdhury.                                                                     |
| <code>-output_file</code>  | $fname$   | Write cliques to the named file.                                                                                |
| <code>-restrictions</code> | $l\ r\ m$ | Restricted search: At level $l$ , restrict to all branches congruent to $r$ modulo $m$ . Here, $0 \leq r < m$ . |

Table 15.1: Finding Cliques

step process, though. At first, the automorphism group of the graph has to be computed. Then, poset classification can be invoked to classify the cliques of a certain size. Here is an example. We wish to classify the cliques in the Paley graph of order 13. The first command creates the graph and computes the automorphism group:

#### Example 612

```
Paley_13_aut:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define Gamma -graph -Paley F -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -automorphism_group \
▷ ▷ -end
```

The command writes a file `Paley_13_group.makefile`. The group is given using a base and strong generating set. The base consists of the two points 0,1. Three strong generators with respect to this base are given in a csv file. Using this group, the next command classifies all cliques of size at most 5 in the Paley graph of order 13 under the action of the automorphism group. It turns out that there are no 5-cliques, and that the largest cliques have size 3. The command shows that there is a unique orbit of 3-cliques:

#### Example 613

```
Paley_13_cliques_classify:
▷ $(ORBITER) -v 4 \
▷ ▷ -define Control -poset_classification_control \
▷ ▷ ▷ -W \
▷ ▷ ▷ -problem_label Paley13_cliques \
▷ ▷ ▷ -clique_test Gamma \
▷ ▷ ▷ -depth 5 \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define gens -vector -file Paley_13_gens.csv -end \
▷ ▷ -define G -permutation_group \
▷ ▷ ▷ -bsgs Paley_13 "Paley\13" 13 78 "0,1" 3 gens -end \
▷ ▷ -define Gamma -graph -Paley F -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_subsets 5 Control \
▷ ▷ -end
```

For comparison, the command

#### Example 614

```
Paley_13_cliques_all:
▷ $(ORBITER) -v 10 \
▷ ▷ -define F -finite_field -q 13 -end \
▷ ▷ -define Gamma -graph -Paley F -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity \
```



```

> > > -find_cliques -target_size 3 \
> > -end

```

finds all cliques of size 3. There are exactly 26 of them. Because of the previous command, we know that they are all equivalent under the automorphism group of the graph.

Let us consider the orbital graph created in Section 12.1. We wish to study the 5-cliques. We first determine the number of 5-cliques, and then the number of orbits of 5-cliques under the automorphism group. The following command computes all 5-cliques:

#### Example 615

```

HJ64.cliques5:
> $(ORBITER) -v 6 \
> > -define Gamma -graph \
> > > -load \
> > > Group_Perm315_Orbital_3.colored_graph \
> > -end \
> > -with Gamma -do \
> > -graph_theoretic_activity \
> > > -find_cliques -target_size 5 -end \
> > -end

```

It turns out that there are exactly 1008 5-cliques. Concerning the classification with respect to the automorphism group of the graph, we apply the following command:

#### Example 616

```

HJ64.cliques5.classify:
> $(ORBITER) -v 6 \
> > -define Control -poset_classification_control \
> > > -W \
> > > -problem_label HJ64.cliques \
> > > -clique_test Gamma \
> > > -depth 5 \
> > -end \
> > -define Gamma -graph \
> > > -load \
> > > Group_Perm315_Orbital_3.colored_graph \
> > -end \
> > -define gens -vector \
> > > -file halljanko315_gens.csv \
> > -end \
> > -define G -permutation_group \
> > -bsgs halljanko315 "File\halljanko315" \
> > > 315 1209600 "0,1,42,95" 5 gens -end \
> > -define Orb -orbits -group G \
> > > -on_subsets 5 Control \
> > -end

```

This command shows that all of the 1008 5-cliques lie in one orbit under the group. The orbit representative picked by Orbiter is  $\{0, 8, 31, 110, 283\}$ . These numbers refer to the vertices of the graph. They are zero-based. The stabilizer of the clique has order 1200.

Let us look at the collinearity graph of  $Q(4, 2)$  one more time. The following command computes the cliques of size 3:

**Example 617**

```
PG0_5_2.cliques: 0_5_2.incidence.matrix.csv
> $(ORBITER) -v 3 \
> > -define Inc -vector -file 0_5_2.incidence.matrix.csv -end \
> > -define Gamma -graph -collinearity_graph Inc -end \
> > -with Gamma -do \
> > -graph.theoretic.activity \
> > > -find_cliques -target_size 3 -end \
> > -end
```

There are 15 cliques of size 3. They correspond to the lines in the geometry.

## 15.2 Rainbow Cliques

Consider a graph  $\Gamma = (V, E)$ . A vertex coloring of  $\Gamma$  is a map  $f : V \rightarrow C$  where  $C$  is a finite set whose elements we call colors. Let  $k = |C|$ . A rainbow clique in  $\Gamma$  is a clique  $X$  such that  $|X \cap f^{-1}(c)| = 1$  for all  $c \in C$ . This means that the clique contains exactly one vertex from every color. Rainbow cliques can be helpful in the search for combinatorial objects. This is because the conditions for rainbow cliques are stronger than the conditions for ordinary cliques, which leads to better pruning in a backtrack search algorithm for cliques. Orbiter offers two algorithms for rainbow cliques. Let us consider an example. The classification of BLT-sets can be facilitated using rainbow cliques. The following algorithm was pointed out in [9], building on earlier work of Penttila. At first, the partial BLT-sets of a certain size (usually 5) are computed. The orbit representatives of this classification are called starter sets. Then, for each orbit representative, a certain graph is defined. The graph helps us to find the BLT-sets that contain the given starter set. The graph has a vertex coloring, and the extensions correspond to rainbow cliques. Let us look at an example. We classify the BLT-sets of size 11. The first command classifies the starters of size 5 and creates the graphs. The graphs are  $\Gamma_0, \dots, \Gamma_{s-1}$ , where  $s$  is the number of orbits on partial BLT-sets.

### Example 618

```
BLT_11_classify_starter:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 11 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -define C -BLT_set_classifier 0 -starter_size 5 -end \
> > -with C -do -BLT_set_classify_activity \
> > > -compute_starter \
> > > > -problem_label BLT_q11 \
> > > > -W -depth 5 \
> > > -end \
> > -end \
> > -with C -do -BLT_set_classify_activity \
> > > -create_graphs \
> > -end
```

Consider just one graph, say the first. The following command computes the rainbow cliques in  $\Gamma_0$ :

### Example 619

```
BLT_11_clique_0:
> $(ORBITER) -v 2 \
> > -define G -graph \
> > > -load BLT_q11_graph_5_0.bin \
> > -end \
> > -with G -do \
> > -graph_theoretic_activity \
> > > -find_cliques -rainbow -target_size 12 -end \
> > -end
```

There are three rainbow cliques. The following command computes the rainbow cliques in  $\Gamma_0$  using the second algorithm. It is the same algorithm as before, but the implementation is due to Sajeeb Chowdhury. It also uses multithreading, and hence is often a lot faster than the first algorithm:

**Example 620**

```

BLT_11_clique_0_Sajeeb:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -graph \
▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -find_cliques -rainbow -Sajeeb -end \
▷ ▷ -end

```

Let us look at the graph  $\Gamma_0$ . At first we sort the vertices by color class:

**Example 621**

```

BLT_11_0_sort:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -graph \
▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -sort_by_colors \
▷ ▷ -end

```

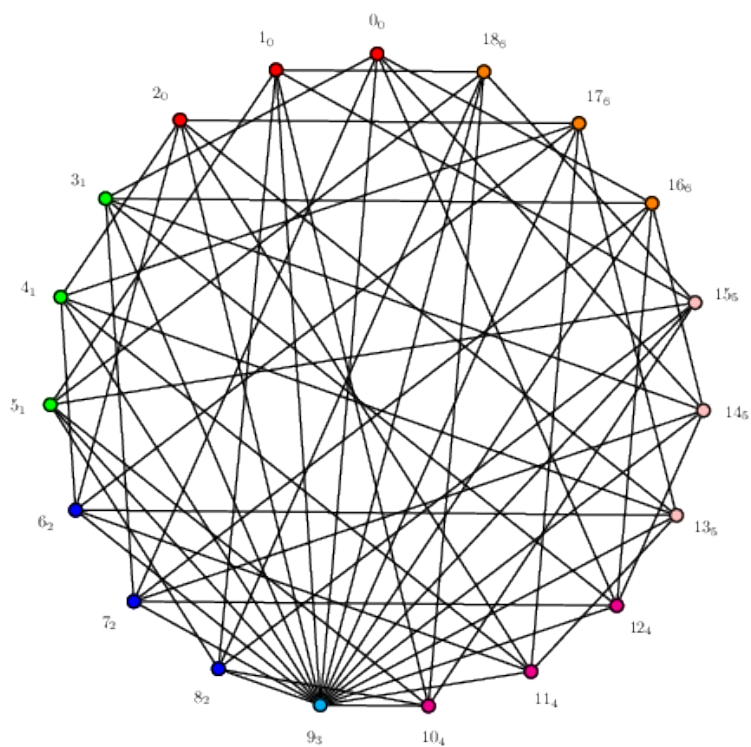
Next, we produce a drawing of the graph. The subscripts at the vertex label indicate the vertex color:

**Example 622**

```

BLT_11_0_sorted_draw:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_options \
▷ ▷ ▷ -radius 100 \
▷ ▷ ▷ -scale 0.4 \
▷ ▷ ▷ -line_width 1.0 -embedded \
▷ ▷ -end \
▷ ▷ -define G -graph \
▷ ▷ ▷ -load BLT_q11_graph_5_0_sorted.bin \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -draw \
▷ ▷ -end
▷ pdflatex BLT_q11_graph_5_0_sorted_draw.tex
▷ $(OPEN) BLT_q11_graph_5_0_sorted_draw.pdf

```



### 15.3 Diophantine Systems

Diophantine systems of equations and inequalities arise frequently in Combinatorics. In Table 15.2, Orbiter commands for creating and solving diophantine systems are shown. In Table 15.3, Orbiter activities for diophantine systems are shown.

Consider the matrix

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Suppose we want to find all column vectors  $\mathbf{x}$  with entries in  $0, 1$  such that

$$A\mathbf{x} = \mathbf{1}$$

where  $\mathbf{1}$  is the all-one column vector. Orbiter offers two algorithms to do this. One is McKay's possolve, the other is Knuth's dancing links (DLX). In order to get started, we need to create a diophant object. In the following example, we use the makefile variable TEST\_SYSTEM for the coefficient matrix.

#### Example 623

```
TEST_SYSTEM="\
0,1,0,1,0,0, \
0,0,1,0,1,0, \
1,0,1,0,0,0, \
0,1,0,1,0,1, \
1,0,0,0,0,1, \
1,0,1,0,0,0, \
0,1,0,0,1,1"
```

The right hand side will be specified using the RHS command below:

#### Example 624

```
solve_test_system:
▷ $(ORBITER) -v 4 \
▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
▷ ▷ -define D -diophant \
▷ ▷ ▷ -label test_system \
▷ ▷ ▷ -coefficient_matrix A \
▷ ▷ ▷ -RHS "mult=7,EQ=1" \
▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
▷ ▷ -end
```

There are two commands to solve a diophantine system: `-solve_mckay` and `-solve_DLX`. The latter is more restrictive, as it allows only  $0, 1$  variables. Here is the McKay solver:

| Creating Diophantine Systems |                        |                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                      | Arguments              | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                    |
| -label                       | label                  | Use the given name as file name.                                                                                                                                                                                                                                                                                                                                                                                           |
| -coefficient_matrix          | A                      | Set the coefficient matrix to the previously created vector with format information.                                                                                                                                                                                                                                                                                                                                       |
| -coefficient_matrix_csv      | fname                  | Read the coefficient matrix from the given csv-file.                                                                                                                                                                                                                                                                                                                                                                       |
| -RHS                         | options                | Set RHS info for a set of $m$ rows. Possible types of conditions are EQ, LE, INT, ZOR. This command can be repeated. Example: mult=2,EQ=1 is condition for 2 rows where the RHS is equal to zero. Example: mult=1,LE=3 is one condition where the RHS is at most three. Example: mult=1,INT=1to2 is one condition where the RHS is between 1 and 2. Example: mult=1,ZOR=2 is one condition where the RHS is either 0 or 2. |
| -RHS_csv                     | fname                  | Read the RHS information from the given csv file.                                                                                                                                                                                                                                                                                                                                                                          |
| -RHS_constant                | options                | Set the RHS as in -RHS (without the mult component).                                                                                                                                                                                                                                                                                                                                                                       |
| -x_max_global                | $a$                    | Set the upper bound for all variables to $a$                                                                                                                                                                                                                                                                                                                                                                               |
| -x_min_global                | $a$                    | Set the lower bound for all variables to $a$                                                                                                                                                                                                                                                                                                                                                                               |
| -x_bounds                    | list-of-values         | Set the lower and upper bounds for all variables.                                                                                                                                                                                                                                                                                                                                                                          |
| -x_bounds_csv                | fname                  | Read the lower and upper bounds for all variables from the given file.                                                                                                                                                                                                                                                                                                                                                     |
| -has_sum                     | $s$                    | Force the sum of the variables to be $s$ .                                                                                                                                                                                                                                                                                                                                                                                 |
| -problem_of_Steiner_type     | N fname                | Create an instance of an exact cover problem based on the matrix in the given file. The underlying set has size $N$ .                                                                                                                                                                                                                                                                                                      |
| -maximal_arc                 | $s$ $d$ secants subset | Create system for a maximal arc of size $s$ and degree $d$ in $PG(2, q)$ . Use the given set of two pencil lines. The subset picks the lines from the given pencils which are external.                                                                                                                                                                                                                                    |
| -field                       | F                      | Use the field F. Related to -maximal_arc.                                                                                                                                                                                                                                                                                                                                                                                  |

Table 15.2: Creating Diophantine Systems

| Diophantine System Activities         |                 |                                                                                                                                                                                     |
|---------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                               | Arguments       | Purpose                                                                                                                                                                             |
| -print                                |                 | Print the system.                                                                                                                                                                   |
| -solve_mckay                          |                 | Solve the system using McKay's possolve.                                                                                                                                            |
| -solve_DLX                            |                 | Solve the system using Knuth's dancing links.                                                                                                                                       |
| -solve_standard                       |                 | Solve the system using the standard solver.                                                                                                                                         |
| -draw                                 |                 | Produce a drawing of the coefficient matrix of the system.                                                                                                                          |
| -draw_as_bitmap                       | $w \ b$         | Produce a bitmap drawing of the coefficient matrix of the system, using boxes of $w$ pixels with. Set the color bit-depth to $b$ ( $b = 8$ or $b = 24$ ). The output is a bmp-file. |
| -perform_column_reductions            |                 | Eliminate variables which must be zero and write a reduced system.                                                                                                                  |
| -test_single_equation                 |                 | For each row of the system, compute the number of solutions of the system restricted to the nonzero coefficients.                                                                   |
| -project_to_single_equation_and_solve | $i \ j$         | Solve the system assuming the $j$ th solution to the restricted system consisting of the $i$ th row.                                                                                |
| -project_to_two_equations_and_solve   | $i \ j \ r \ m$ | Solve the system assuming any solution to the restricted system consisting of the $i$ th and the $j$ -th row whose number is congruent to $r \bmod m$ .                             |

Table 15.3: Diophantine System Activities



**Example 625**

```
McKay_test:
> $(ORBITER) -v 4 \
> > -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
> > -define D -diophant \
> > > -label test_system \
> > > -coefficient_matrix A \
> > > -RHS "mult=7,EQ=1" \
> > > -x_min_global 0 -x_max_global 1 \
> > -end \
> > -with D -do \
> > > -diophant_activity -solve_mckay \
> > -end
```

The solutions are written to the file `DLX_test.sol`. Next, we invoke the dancing links solver:

**Example 626**

```
DLX_test:
> $(ORBITER) -v 4 \
> > -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
> > -define D \
> > -diophant -label test_system \
> > > -coefficient_matrix A \
> > > -RHS "mult=7,EQ=1" \
> > > -x_min_global 0 -x_max_global 1 \
> > -end \
> > -with D -do \
> > > -diophant_activity -solve_DLX \
> > -end
```



## Chapter 16

# Canonical Forms with Nauty

### 16.1 Overview of Canonical Forms

Table 16.1 lists Orbiter commands related to canonical labelings of combinatorial objects, including objects in projective spaces.

The graph theory package Nauty [52] provides a canonical form algorithm for graphs. Problems in projective spaces can be reduced to equivalent problems graphs, so that Nauty can help here as well. For an instance of this technique to the classification of arcs, see [2].

| Classification by Canonical Form |            |                                                                                                                                             |
|----------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Command                          | Arguments  | Purpose                                                                                                                                     |
| -space                           | L          | A projective space is required for some types of combinatorial objects.                                                                     |
| -ring                            | R          | A polynomial ring is required for algebraic varieties.                                                                                      |
| -input_fname_mask                | fname-mask | Input file name mask.                                                                                                                       |
| -nb_files                        | n          | Number of input files.                                                                                                                      |
| -output_fname                    | fname      | Output file name.                                                                                                                           |
| -label_po_go                     | label      | Column label for po-go columns. This is the stabilizer of the orbit representative of the primary orbit.                                    |
| -label_po_index                  | label      | Column label for the subgroup index of the secondary orbit representative inside the stabilizer of the representative of the primary orbit. |
| -label_po                        | label      | Column label for the primary orbit number.                                                                                                  |
| -label_so                        | label      | Column label for the secondary orbit number.                                                                                                |
| -label_equation                  | label      | Column label for the algebraic equation (in case of a variety).                                                                             |
| -label_equation2                 | label      | Column label for the optional second algebraic equation (in case of a variety).                                                             |
| -label_points                    | label      | Column label for the set of points of the object (optional).                                                                                |
| -label_lines                     | label      | Column label for the set of lines of the object (optional).                                                                                 |
| -carry_through                   | label      | Column label of a property that is carried through from the input file to the output file. This option can be repeated.                     |
| -algorithm_nauty                 |            | Select canonical forms computed by Nauty (recommended).                                                                                     |
| -algorithm_substructure          |            | Select canonical forms computed by an algorithm based on substructures (not recommended).                                                   |
| -substructure_size               | s          | Select the size of the substructure (for the substructure based algorithm).                                                                 |
| -skip                            | list       | List of cases to be skipped. Case numbers are assigned consecutively. They are 0-based.                                                     |

Table 16.1: Classification by Canonical Form

## 16.2 Canonical Forms of Objects in Projective Space

Suppose we want to compute the stabilizer of an elliptic curve. In Section 4.1, we have created an elliptic curve over  $\mathbb{F}_{11}$  and stored the set of  $\mathbb{F}_q$ -points in the file

elliptic\_curve\_b1\_c3\_q11.txt.

The following example computes the set stabilizer of the curve. This means it computes the set stabilizer of the points on the curve. In order to do so, an input stream is created which referst to the file containing the Orbiter point ranks of points on the curve.

### Example 627

```
EC_canon: elliptic_curve_b1_c3_q11.txt
> $(ORBITER) -v 3 \
> > -define C -combinatorial_object \
> > > -label EC EC \
> > > -file_of_points elliptic_curve_b1_c3_q11.txt \
> > -end \
> > -define F -finite_field -q 11 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form_PG P \
> > > > -save_ago \
> > > > -save_transversal \
> > > > -max_TDO_depth 400 \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_TDO \
> > > > -show_TDA \
> > > > -dont_show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex EC_classification.tex
> $(OPEN) EC_classification.pdf
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file EC_object0_TDA_flag_orbits.csv \
> > -secondary_input_csv_file EC_object0_TDA.csv \
> > -box_width 20 -bit_depth 24 \
> > -end
> $(OPEN) EC_object0_TDA_flag_orbits_draw.bmp
```

Orbiter shows that the curve has a collineation stabilizer of order 6, generated by

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 8 \\ 5 & 9 & 5 \\ 8 & 1 & 1 \end{bmatrix}.$$

The following example computes the canonical form and the automorphism group of the Hirschfeld surface in  $\text{PG}(3,4)$ . Using indexing of points in  $\text{PG}(3,4)$ , we encode the surface as a set of points using Orbiter ranks. We use a makefile variable to provide these ranks as input for the canonical form procedure.

#### Example 628

```
HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS="0,1,2,3,4,5,6,7,8,9,\
10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,\
53,54,59,60,61,62,67,68,69,70,75,76,79,80,81,82"
```

#### Example 629

```
Hirschfeld.q4-c:
▷ $(ORBITER) -v 6 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label Hirschfeld_surface_q4 Hirschfeld\surface\q4 \
▷ ▷ ▷ -file_of_points Hirschfeld_surface_q4.txt \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 4 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form_PG P \
▷ ▷ ▷ -save_ago \
▷ ▷ ▷ -max_TDO_depth 10 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ -export_labels \
▷ ▷ ▷ -dont_show_incidence_matrices \
▷ ▷ ▷ -export_group_GAP \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex Hirschfeld_surface_q4_classification.tex
▷ $(OPEN) Hirschfeld_surface_q4_classification.pdf
```

**Example 630**

```

Hirschfeld.q4_set.c:
> $(ORBITER) -v 4 \
> > -define H -set -here \
> > > $(HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS) \
> > -end \
> > -define C -combinatorial_object \
> > > -label Hirschfeld_surface.q4 Hirschfeld\surface\_q4 \
> > > -set_of_points H \
> > -end \
> > -define F -finite_field -q 4 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form_PG P \
> > > > -save_ago \
> > > > -max_TDO_depth 4 \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -show_TDO \
> > > > -show_TDA \
> > > -end \
> > -end
> pdflatex Hirschfeld_surface.q4.classification.tex
> $(OPEN) Hirschfeld_surface.q4.classification.pdf

```

In the next example, we compute the canonical form of the two hyperovals in  $PG(2, 16)$ .

**Example 631**

```

hyperoval_16_canonical_form:
> $(ORBITER) -v 2 \
> > -define C -combinatorial_object \
> > > -label hyperoval.q16 hyperoval\_q16 \
> > > -set_of_points $(HYPEROVAL_16.16320) \
> > > -set_of_points $(HYPEROVAL_16.144) \
> > -end \
> > -define F -finite_field -q 16 -end \
> > -define P -projective_space -n 2 -field F -v 0 -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form_PG P \
> > > > -save_ago \
> > > > -save_transversal \
> > > > -max_TDO_depth 10 \
> > > -end \
> > -end \

```

```

> > -with C -do \
> > -combinatorial_object.activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_TDO \
> > > > -show_TDA \
> > > > -dont_show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex hyperoval_q16_classification.tex
> $(OPEN) hyperoval_q16_classification.pdf
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file hyperoval_q16_object0_TDA_flag_orbits.csv \
> > -secondary_input_csv_file hyperoval_q16_object0_TDA.csv \
> > -box_width 4 -bit_depth 24 \
> > -end
> $(OPEN) hyperoval_q16_object0_TDA_flag_orbits_draw.bmp
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file hyperoval_q16_object1_TDA_flag_orbits.csv \
> > -secondary_input_csv_file hyperoval_q16_object1_TDA.csv \
> > -box_width 4 -bit_depth 24 \
> > -end
> $(OPEN) hyperoval_q16_object1_TDA_flag_orbits_draw.bmp

```

In the next example, we compute the set stabilizers of orbits of  $\text{PGL}(4, 2)$  on subsets of  $\text{PG}(3, 2)$ , as computed earlier in Section 7.3, using the command `PG_3_2_subsets`. Concerning the work in Dickson [25] only subsets whose size is odd are relevant, so we restrict to those subsets:

#### Example 632

```

Dickson_sets_stabilizer:
> $(ORBITER) -v 3 \
> > -define C -combinatorial_object \
> > > -label Dickson_sets Dickson\sets \
> > > -set_of_points "0,1,2,5,6" \
> > > -set_of_points "0,1,2,3,6" \
> > > -set_of_points "0,1,2,3,4" \
> > > -set_of_points "0,1,2,3,8" \
> > > -set_of_points "0,1,2,5,6,7,8" \
> > > -set_of_points "0,1,2,3,5,6,7" \
> > > -set_of_points "0,1,2,3,5,6,9" \
> > > -set_of_points "0,1,2,3,5,6,10" \
> > > -set_of_points "0,1,2,3,5,6,4" \
> > > -set_of_points "0,1,2,3,8,11,13" \
> > > -set_of_points "3,6,9,7,10,12,8,11,13,14,4" \
> > > -set_of_points "3,5,6,9,7,10,12,11,13,14,4" \
> > > -set_of_points "0,1,2,3,5,6,9,7,10,12,4" \
> > -end \
> > -define F -finite_field -q 2 -end \

```



```

▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form_PG P \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -max_TDO_depth 4 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex Dickson_sets_classification.tex
▷ $(OPEN) Dickson_sets_classification.pdf

```

There are two ovoids in  $PG(3, 2)$ . The classical ovoid is the elliptic quadric. It was created using the command `elliptic_quadric_ovoid_q8` in Section 4.10. The following command computes the stabilizer of the ovoid:

#### Example 633

```

ovoid_q8_canon: ovoid_q8.txt
▷ $(ORBITER) -v 6 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label ovoid ovoid \
▷ ▷ ▷ -file_of_points ovoid_q8.txt \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form_PG P \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -max_TDO_depth 4 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
▷ ▷ ▷ ▷ -export_group_GAP \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ #pdflatex ovoid_classification.tex
▷ #$(OPEN) ovoid_classification.pdf

```

The other ovoid is the Suzuki Tits ovoid, which was created using the command `ovoid_ST_q8` in Section 4.10. The stabilizer of the Suzuki Tits ovoid is the Suzuki group. The following command computes this group for  $q = 8$ .

**Example 634**

```
ovoid_ST_q8_canon: ovoid_ST_q8.txt
▷ $(ORBITER) -v 6 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label ovoid_ST ovoid\_ST \
▷ ▷ ▷ -file_of_points ovoid_ST_q8.txt \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 8 -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form_PG P \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -max_TDO_depth 4 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
▷ ▷ ▷ ▷ -export_group_GAP \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ #pdflatex ovoid_ST_classification.tex
▷ #$(OPEN) ovoid_ST_classification.pdf
```

We can store the generators in a makefile variable as follows:

**Example 635**

```
SUZUKI_8_GENERATORS="\
1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1, \
1,0,0,0,0,6,0,0,0,0,2,0,0,0,0,3,0, \
1,0,0,0,1,1,1,0,0,0,1,0,1,0,0,1,0, \
1,0,0,0,3,6,2,2,5,0,2,0,3,0,6,3,2, \
0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,2"
# group order 87360 = 3 * 29120
```

We can now recover the Suzuki group using the command:

**Example 636**

```
Suzuki_8:
▷ $(ORBITER) -v 6 \
```

```

> > -define F -finite_field -q 8 -end \
> > -define gens -vector -field F \
> > > -compact $(SUZUKI_8_GENERATORS) -end \
> > -define G -linear_group -PGGL 4 8 \
> > -subgroup_by_generators "Sz8" "87360" 5 gens \
> > > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGGL_4_8_Subgroup_Sz8_87360_report.tex
> $(OPEN) PGGL_4_8_Subgroup_Sz8_87360_report.pdf

```

Let us go back to Section 5.3 and compute the stabilizer of the Veronese variety. The following command can be used. We utilize the list of point ranks computed earlier:

#### Example 637

```

Veronese_1.3_q7_c:
> $(ORBITER) -v 6 \
> > -define C -combinatorial_object \
> > > -label veronese_1.3_q7 veronese\1\3\q7 \
> > > -set_of_points "3, 0, 4, 268, 336, 184, 224, 364" \
> > -end \
> > -define F -finite_field -q 7 -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form_PG P \
> > > > -save_ago \
> > > > -max_TDO_depth 4 \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -show_TDO \
> > > > -show_TDA \
> > > > -dont_show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex veronese_1.3_q7_classification.tex
> $(OPEN) veronese_1.3_q7_classification.pdf

```

The list of points is unpacked and points are printed with their homogeneous coordinates:

| $i$ | Rank | Point        |
|-----|------|--------------|
| 0   | 3    | (0, 0, 0, 1) |
| 1   | 0    | (1, 0, 0, 0) |
| 2   | 4    | (1, 1, 1, 1) |
| 3   | 268  | (1, 2, 4, 1) |
| 4   | 336  | (6, 4, 5, 1) |
| 5   | 184  | (1, 4, 2, 1) |
| 6   | 224  | (6, 2, 3, 1) |
| 7   | 364  | (6, 1, 6, 1) |

The automorphism group is determined to have order 336. Generators are produced:

The stabilizer of order 336 is generated by:

$$g_1 = \begin{bmatrix} 6 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \text{ of order 2 and with 16 fixed points.}$$

$$g_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \text{ of order 3 and with 10 fixed points.}$$

$$g_3 = \begin{bmatrix} 6 & 0 & 0 & 0 \\ 4 & 2 & 0 & 0 \\ 4 & 4 & 3 & 0 \\ 6 & 2 & 3 & 1 \end{bmatrix} \text{ of order 6 and with 4 fixed points.}$$

$$g_4 = \begin{bmatrix} 1 & 5 & 4 & 6 \\ 6 & 1 & 0 & 3 \\ 5 & 0 & 2 & 4 \\ 1 & 1 & 1 & 1 \end{bmatrix} \text{ of order 8 and with 0 fixed points.}$$

1,0,0,0,0,6,0,0,0,0,1,0,0,0,0,6,  
 1,0,0,0,0,2,0,0,0,0,4,0,0,0,0,1,  
 1,0,0,0,3,5,0,0,3,3,4,0,1,5,4,6,  
 1,5,4,6,6,1,0,3,5,0,2,4,1,1,1,1,

The array of numbers at the end is the coding of the generators. This can be used to recreate the group. Finally, a tactical decomposition is produced. This is the extended incidence matrix of  $\text{PG}(3, 7)$ , using combinatorial refinement. One artificial point and one artificial line are introduced to encode the variety as a combinatorial object.

Decomposition by combinatorial refinement:

| $\rightarrow$ | $1_1$ | $28_2$ | $400_5$ | $2422_6$ |
|---------------|-------|--------|---------|----------|
| $8_0$         | 1     | 7      | 50      | 0        |
| $1_4$         | 1     | 0      | 0       | 0        |
| $168_3$       | 0     | 1      | 6       | 50       |
| $224_7$       | 0     | 0      | 8       | 49       |
| $\downarrow$  | $1_1$ | $28_2$ | $400_5$ | $2422_6$ |
| $8_0$         | 8     | 2      | 1       | 0        |
| $1_4$         | 1     | 0      | 0       | 0        |
| $168_3$       | 0     | 6      | 2       | 3        |
| $224_7$       | 0     | 0      | 5       | 5        |

### 16.3 Canonical Forms of Incidence Geometries

Let us consider incidence structures, which are sets of subsets. These subsets are chosen from the same set, which we call the carrier set. The elements of the set are often called points. The distinguished subsets are called blocks or sometimes lines. There are conditions that restrict the way in which the sets can be chosen. There is a notion of isomorphism on such set systems. Two set systems are isomorphic if there is a bijection between the underlying sets which takes one to the other. The incidence matrix is the 0/1 matrix whose rows correspond to the elements of the group set, and whose columns correspond to the chosen subsets. An entry 1 indicates that the corresponding point belongs to the corresponding block. A flag is an incident point-line pair. An anti-flag is a non-incident point-line pair. Two points are said to be collinear if there is a line containing both.

It is interesting to study the action of the automorphism group on the elements of a geometry. Properties of interest are various levels of transitivity on the elements of the geometry. For instance, a geometry is line-transitive if the automorphism group is transitive on lines. Likewise, it is flag transitive if the automorphism group is transitive on flags. The collinearity graph of a geometry is the graph whose vertices correspond to the points, with two vertices adjacent if the associated points are collinear. The girth of the incidence geometry is the girth of the associated collineation graph. A geometry is triangle free if its girth is at least 4.

A configuration  $v_r b_k$  is an incidence geometry on a set of size  $v$  and with  $b$  lines such that each line has size  $k$  and each point is contained in exactly  $r$  lines. In the special case where  $b = v$  and  $k = r$ , the name symmetric configuration  $v_r$  is used (the term symmetric is somewhat misleading because the incidence matrix of a symmetric configuration need not be symmetric). Orbiter can be used to classify incidence geometries. One of the important steps in this process is computing a canonical form of the incidence geometry.

We will also be producing drawings of the incidence matrices of geometries. There are two types of drawings:

1. Bitmap drawings,
2. Latex drawings.

In the bitmap drawings, flags are indicated as heavy squares while anti-flags are drawn as small squares. The coloring will indicate the orbits of the automorphism group on flags and anti-flags. Objects with the same color belong to the same orbit. In a flag-transitive geometry, all incidences have the same color, which is black. Bitmaps can be created using the `-draw_matrix` command. The input is an integer-valued matrix in csv format. The matrix entries are translated into colors. The possible commands after `-draw_matrix` are shown in Table 16.2. Orbiter uses latex drawings for instance in reports. Table 16.3 shows the available commands to customize such drawings.

The following command computes the canonical form and a report of the projective plane  $\text{PG}(2, 2)$ , which is a configuration  $7_3$ .

#### Example 638

```
geo_7.3_c:
> $(ORBITER) -v 10 \
> > -draw_incidence_structure_description \
> > > -width 60 -width_10 6 -end \
> > -define C -combinatorial_object \
> > > -label geo_7.3 geo\7\3 \
> > > -file_of_incidence_geometries 7.3.inc 7 7 21 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
```

| Creating a Bitmap Graphics |             |                                                                                        |
|----------------------------|-------------|----------------------------------------------------------------------------------------|
| Command                    | Arguments   | Purpose                                                                                |
| -input_csv_file            | csv-file    | Specify the input csv file.                                                            |
| -secondary_input_csv_file  | csv-file    | Specify a secondary input csv file (optional).                                         |
| -input_object              | $l$         | Specify input to be Orbiter object $l$ .                                               |
| -partition                 | $w \ R \ C$ | Specify a partition $R$ of rows and $C$ of columns. Use separating lines of with $w$ . |
| -box_width                 | $w$         | Use $w$ pixels per matrix entry.                                                       |
| -bit_depth                 | $d$         | Use color bit depth of $d$ bits ( $d = 8$ or $d = 24$ ).                               |
| -invert_colors             |             | Use an inverted color scheme.                                                          |
| -grayscale                 |             | Use a grayscale color set.                                                             |

Table 16.2: Creating a Bitmap Graphics

| Options for Drawing Incidence Structures |           |                                                                                    |
|------------------------------------------|-----------|------------------------------------------------------------------------------------|
| Command                                  | Arguments | Purpose                                                                            |
| -width                                   | $w$       | Set the width of one box in units of 0.1mm.                                        |
| -width_10                                | $w$       | Set the width of one tenth of a box. Should with the value of width divided by 10. |
| -outline_thin                            |           | Draw a thin outline.                                                               |
| -unit_length                             |           | Default: 0.065mm.                                                                  |
| -thick_lines                             |           | Default: 0.5mm.                                                                    |
| -thin_lines                              |           | Default: 0.15mm.                                                                   |
| -geo_line_width                          |           | Default: 0.25mm.                                                                   |

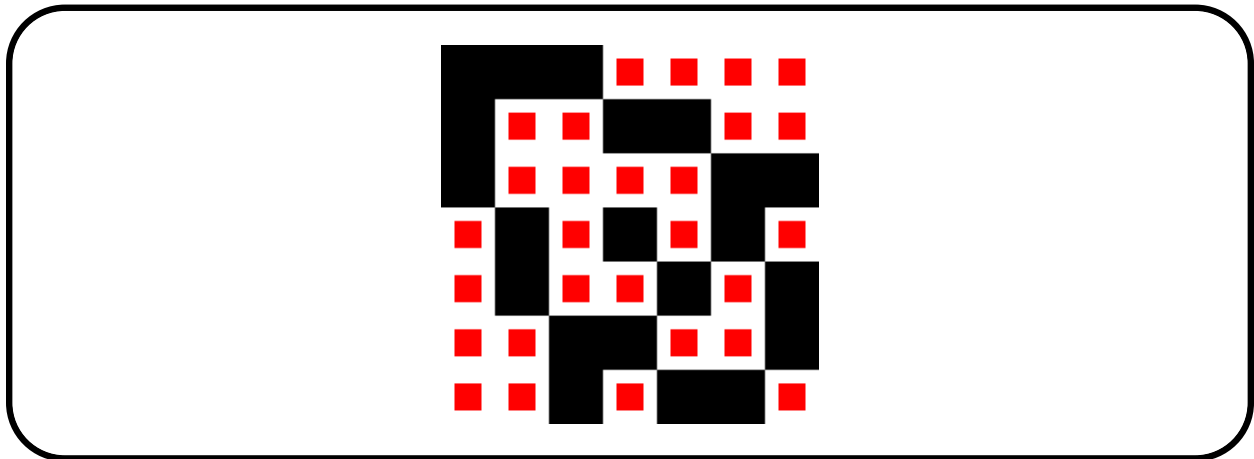
Table 16.3: Options for Drawing Incidence Structures

```

> > > -canonical_form \
> > > > -save_ago \
> > > > -save_transversal \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex 7_3_classification.tex
> $(OPEN) 7_3_classification.pdf
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file 7_3_object0_TDA_flag_orbits.csv \
> > -secondary_input_csv_file 7_3_object0_TDA.csv \
> > -box_width 32 -bit_depth 24 \
> > -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file 7_3_object0_INP_flag_orbits.csv \
> > -secondary_input_csv_file 7_3_object0_INP.csv \
> > -box_width 32 -bit_depth 24 \
> > -end
> $(OPEN) 7_3_object0_INP_flag_orbits_draw.bmp

```

A bitmap drawing is produced, shown below:



The command also produces the following report of the geometry:



## Summary of Orbits

|   | Rep | # | Ago | Objects |
|---|-----|---|-----|---------|
| 0 | 0   | 1 | 168 | 0       |

Ago : 168

### Isomorphism type 0 / 1

Isomorphism type 0 / 1 is original object 0 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {0}  
incidence structure:

( 0, 1, 2, 7, 10, 11, 14, 19, 20, 22, 24, 26, 29, 32, 34, 37, 38, 41, 44, 46, 47 )

Column sets of the encoded object:

{ 0, 1, 2 }

{ 0, 3, 4 }

{ 0, 5, 6 }

{ 1, 3, 5 }

{ 1, 4, 6 }

{ 2, 3, 6 }

{ 2, 4, 5 }

Row sets of the encoded object:

{ 0, 1, 2 } = 0

{ 0, 3, 4 } = 9

{ 0, 5, 6 } = 14

{ 1, 3, 5 } = 20

{ 1, 4, 6 } = 23

{ 2, 3, 6 } = 27

{ 2, 4, 5 } = 28

|                          |                          |                          |                          |                          |                          |                          |
|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |

Generators for the automorphism group:

The stabilizer of order 168 is generated by:

$g_1 = (3, 5)(4, 6)(8, 9)(12, 13)$  of order 2 and with 6 fixed points.

$g_2 = (3, 4)(5, 6)(10, 11)(12, 13)$  of order 2 and with 6 fixed points.

$g_3 = (1, 2)(5, 6)(10, 12)(11, 13)$  of order 2 and with 6 fixed points.

$g_4 = (1, 3)(2, 4)(7, 8)(11, 12)$  of order 2 and with 6 fixed points.

$g_5 = (0, 1)(4, 5)(8, 10)(9, 11)$  of order 2 and with 6 fixed points.

Canonical labeling:

canonical row = 6

canonical orbit number = 0

Flags : ( 0, 1, 2, 7, 10, 11, 14, 19, 20, 22, 24, 26, 30, 31, 34, 36, 39, 41, 44, 46, 47 )

|    | 7                        | 8                        | 9                        | 10                       | 11                       | 12                       | 13                       |
|----|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| *0 | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |                          |                          |                          |                          |
| *1 | <input type="checkbox"/> |                          |                          | <input type="checkbox"/> | <input type="checkbox"/> |                          |                          |
| *2 | <input type="checkbox"/> |                          |                          |                          |                          | <input type="checkbox"/> | <input type="checkbox"/> |
| *3 |                          | <input type="checkbox"/> |                          | <input type="checkbox"/> | <input type="checkbox"/> |                          |                          |
| *5 |                          |                          | <input type="checkbox"/> | <input type="checkbox"/> |                          |                          | <input type="checkbox"/> |
| *4 |                          | <input type="checkbox"/> |                          |                          | <input type="checkbox"/> |                          | <input type="checkbox"/> |
| *6 |                          |                          | <input type="checkbox"/> |                          | <input type="checkbox"/> | <input type="checkbox"/> |                          |

Flag orbits:

orbit length : number of orbits of that length:

21 1

Anti-Flag orbits:

orbit length : number of orbits of that length:

28 1

The following command computes the canonical form and a report of the affine plane  $AG(2, 3)$ , which is a configuration  $9_4 12_3$ .

#### Example 639

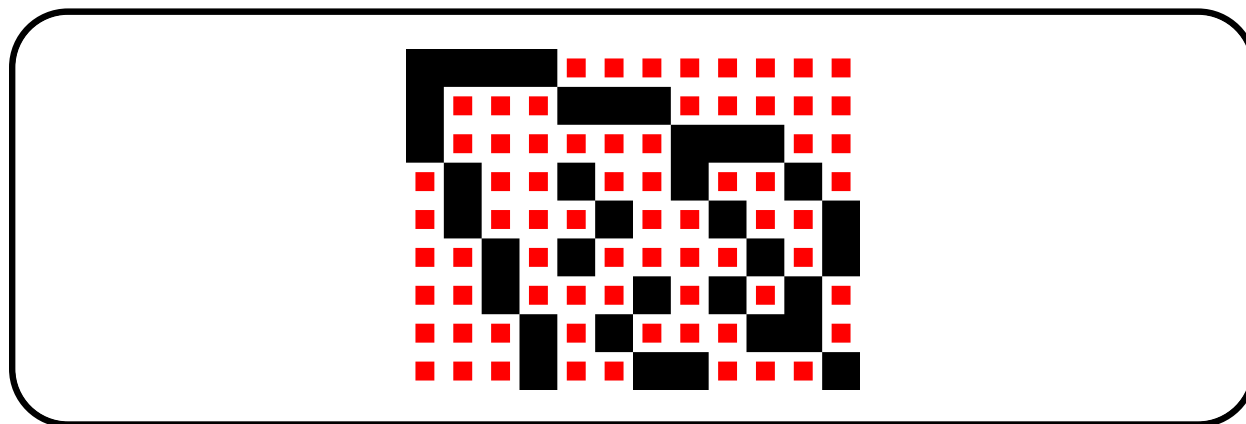
```
AG_2.3_c: AG_2.3.inc
▷ $(ORBITER) -v 2 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label AG_2.3 AG\_2\_3 \
▷ ▷ ▷ -file_of_incidence_geometries \
▷ ▷ ▷ AG_2.3.inc 9 12 36 \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -max.TDO_depth 10 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ ▷ -show_incidence_matrices \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex AG_2.3_classification.tex
▷ $(OPEN) AG_2.3_classification.pdf
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file AG_2.3_object0_INP_flag_orbits.csv \
▷ ▷ -secondary_input_csv_file AG_2.3_object0_INP.csv \
▷ ▷ -box_width 40 -bit_depth 24 \
```

```

▷ ▷ -end
▷ $(OPEN) AG_2_3_object0_INP_flag_orbits_draw.bmp

```

The following bitmap drawing is produced:



Because the geometry is flag transitive, there is only one color being used for the incidence (black). In addition, the geometry is also anti-flag transitive. The smallish boxes all have the same color (red). Orbiter also produces the following report of the geometry:

### Summary of Orbits

|   | Rep | # | Ago | Objects |
|---|-----|---|-----|---------|
| 0 | 0   | 1 | 432 | 0       |

Ago :432

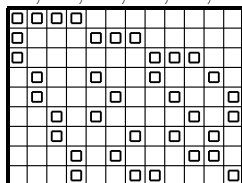
### Isomorphism type 0 / 1

Isomorphism type 0 / 1 is original object 0 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {0}

incidence structure:

( 0, 1, 2, 3, 12, 16, 17, 18, 24, 31, 32, 33, 37, 40, 43, 46, 49, 53, 56, 59, 62, 64, 69, 71, 74, 78, 80, 82, 87, 89, 93, 94, 99, 102, 103, 107 )



Generators for the automorphism group:

The stabilizer of order 432 is generated by:

$g_1 = (3, 4)(5, 7)(6, 8)(11, 12)(13, 14)(16, 17)(19, 20)$  of order 2 and with 7 fixed points.

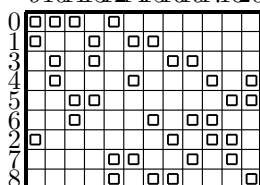
$g_2 = (3, 5)(4, 6)(7, 8)(10, 11)(14, 15)(16, 18)(19, 20)$  of order 2 and with 7 fixed points.

$g_3 = (1, 3)(2, 4)(7, 8)(9, 10)(14, 16)(15, 19)(18, 20)$  of order 2 and with 7 fixed points.

Decomposition by combinatorial refinement:

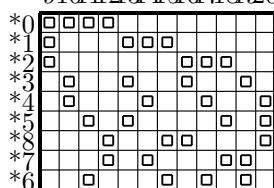
|               |        |
|---------------|--------|
| $\rightarrow$ | $12_1$ |
| $9_0$         | $4$    |
| $\downarrow$  | $12_1$ |
| $9_0$         | $3$    |

9101131245697820



canonical orbit number = 0

910112345678920



orbit length : number of orbits of that length:

36 1

orbit length : number of orbits of that length:

72 1

### Example 640

```
geo_10_3.c:
> $(ORBITER) -v 10 \
> > -draw_incidence_structure_description \
> > > -width 60 -width 10 6 -end \
```

```

> > -define C -combinatorial_object \
> > > -label geo_10_3 geo\_10\_3 \
> > > -file_of_incidence_geometries 10_3.inc 10 10 30 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form \
> > > > -save_ago \
> > > > -save_transversal \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex 10_3_classification.tex
> $(OPEN) 10_3_classification.pdf
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file 10_3_object7_TDA_flag_orbits.csv \
> > -secondary_input_csv_file 10_3_object7_TDA.csv \
> > -box_width 16 -bit_depth 24 \
> > -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file 10_3_object7_INP_flag_orbits.csv \
> > -secondary_input_csv_file 10_3_object7_INP.csv \
> > -box_width 16 -bit_depth 24 \
> > -end

```

The report is shown below. It is truncated for reasons of space. Only the first two geometries are shown. Note that the ordering of geometries in the report may be different from the ordering in the input file. This is because the classification program sorts the geometries according to the canonical form. Also, note that the report includes the incidence geometry in the original form as well as the tactical decomposition induced by the orbits of the automorphism group.

## Summary of Orbits

|   | Rep | # | Ago | Objects |
|---|-----|---|-----|---------|
| 0 | 9   | 1 | 3   | 9       |
| 1 | 1   | 1 | 2   | 1       |
| 2 | 6   | 1 | 10  | 6       |
| 3 | 5   | 1 | 4   | 5       |
| 4 | 4   | 1 | 24  | 4       |
| 5 | 7   | 1 | 4   | 7       |
| 6 | 8   | 1 | 3   | 8       |
| 7 | 3   | 1 | 120 | 3       |
| 8 | 2   | 1 | 12  | 2       |
| 9 | 0   | 1 | 6   | 0       |

Ago :2,  $3^2$ ,  $4^2$ , 6, 10, 12, 24, 120

## Isomorphism type 0 / 10

Isomorphism type 0 / 10 is original object 9 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {9}

incidence structure:

( 0, 1, 2, 10, 13, 14, 20, 25, 26, 31, 33, 35, 41, 44, 47, 52, 53, 58, 62, 66, 69, 74, 78, 79, 85, 87, 89, 96, 97, 98 )

Generators for the automorphism group:

The stabilizer of order 3 is generated by:

$g_1 = (0, 1, 3)(2, 5, 4)(6, 7, 8)(10, 13, 11)(12, 14, 15)(16, 18, 17)$  of order 3 and with 2 fixed points.

Decomposition by automorphism group:

|   |    |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|----|
|   | 10 | 13 | 11 | 12 | 14 | 15 | 16 | 18 | 17 | 19 |
| 0 | □  |    | □  | □  |    |    |    |    |    |    |
| 1 | □  | □  |    |    | □  |    |    |    |    |    |
| 3 |    | □  | □  |    |    | □  |    |    |    |    |
| 2 | □  |    |    |    |    | □  | □  |    |    |    |
| 5 |    | □  |    | □  |    |    |    | □  |    |    |
| 4 |    |    | □  |    | □  |    |    |    | □  |    |
| 6 |    |    |    | □  |    |    | □  |    |    | □  |
| 7 |    |    |    |    | □  |    |    | □  |    | □  |
| 8 |    |    |    |    |    | □  |    |    | □  | □  |
| 9 |    |    |    |    |    |    | □  | □  | □  |    |

Canonical labeling:

canonical row = 5

canonical orbit number = 1

Flags : 0,1,2,16,17,18,25,27,29,34,38,39,40,43,45,51,53,56,62,63,64,70,74,77,82,86,89,91,95,98,

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
|    | 16 | 18 | 17 | 19 | 15 | 12 | 14 | 10 | 13 | 11 |
| 9  | □  | □  | □  |    |    |    |    |    |    |    |
| 1  |    |    |    |    |    |    | □  | □  | □  |    |
| 0  |    |    |    |    |    | □  |    | □  |    | □  |
| 3  |    |    |    | □  |    |    |    |    | □  | □  |
| 6  | □  |    |    | □  |    | □  |    |    |    |    |
| 7  |    | □  |    | □  |    |    | □  |    |    |    |
| 8  |    |    | □  | □  | □  |    |    |    |    |    |
| *2 | □  |    |    |    | □  |    |    | □  |    |    |
| *4 |    | □  |    |    |    |    | □  |    |    | □  |
| *5 |    | □  |    |    |    | □  |    |    | □  |    |

## Isomorphism type 1 / 10

Isomorphism type 1 / 10 is original object 1 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {1}

incidence structure:

( 0, 1, 2, 10, 13, 14, 20, 25, 26, 31, 33, 35, 41, 44, 47, 52, 54, 58, 62, 66, 69, 73, 78, 79, 85, 87, 89, 96, 97, 98 )

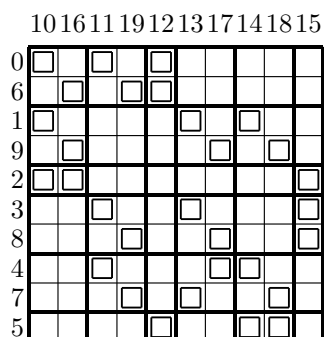
|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| □ | □ | □ |   |   |   |   |   |   |   |   |
| □ |   |   | □ | □ |   |   |   |   |   |   |
| □ |   |   |   |   | □ | □ |   |   |   |   |
|   | □ |   | □ |   | □ |   |   |   |   |   |
|   | □ |   |   |   |   |   | □ |   |   |   |
|   |   | □ |   | □ |   |   |   | □ |   |   |
|   |   |   | □ |   |   |   |   |   | □ |   |
|   |   |   |   |   |   |   |   |   | □ | □ |
|   |   |   |   |   | □ |   |   | □ |   | □ |
|   |   |   |   |   |   |   |   |   | □ | □ |

Generators for the automorphism group:

The stabilizer of order 2 is generated by:

$g_1 = (0, 6)(1, 9)(3, 8)(4, 7)(10, 16)(11, 19)(13, 17)(14, 18)$  of order 2 and with 4 fixed points.

Decomposition by automorphism group:

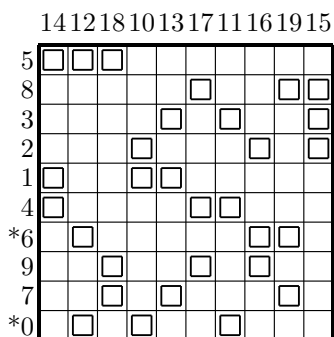


Canonical labeling:

canonical row = 0

canonical orbit number = 0

Flags : 0,1,2,15,18,19,24,26,29,33,37,39,40,43,44,50,55,56,61,67,68,72,75,77,82,84,88,91,93,96,



The following command computes the canonical form for the three triangle free configurations  $24_3$  found by Abdullah Alazemi. These configurations have 24 points, 24 lines, each line consists of 3 points and each point is on 3 lines.

#### Example 641

```
FILE_24.3.TFC.INC="24 24 72\
\n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
258 259 269 272 282 293 300 308 318 325 333 342 352 358 \
367 379 381 392 398 400 417 428 429 442 443 450 466 471 \
479 492 497 502 517 519 521 542 548 551 571 574 575 \
48\
\n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
258 259 269 272 281 293 301 308 318 324 327 342 354 357 \
367 373 378 392 400 403 417 419 430 442 446 447 466 472 \
479 492 500 503 518 525 526 545 549 551 571 572 574 \
48\
\n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
132 146 157 158 171 175 179 195 201 207 220 226 232 244 \
257 258 269 274 277 293 300 307 318 323 329 342 352 356 \
367 374 381 392 397 406 416 423 431 441 450 454 468 476 \
477 494 499 503 519 521 525 544 547 550 570 572 575 \
```

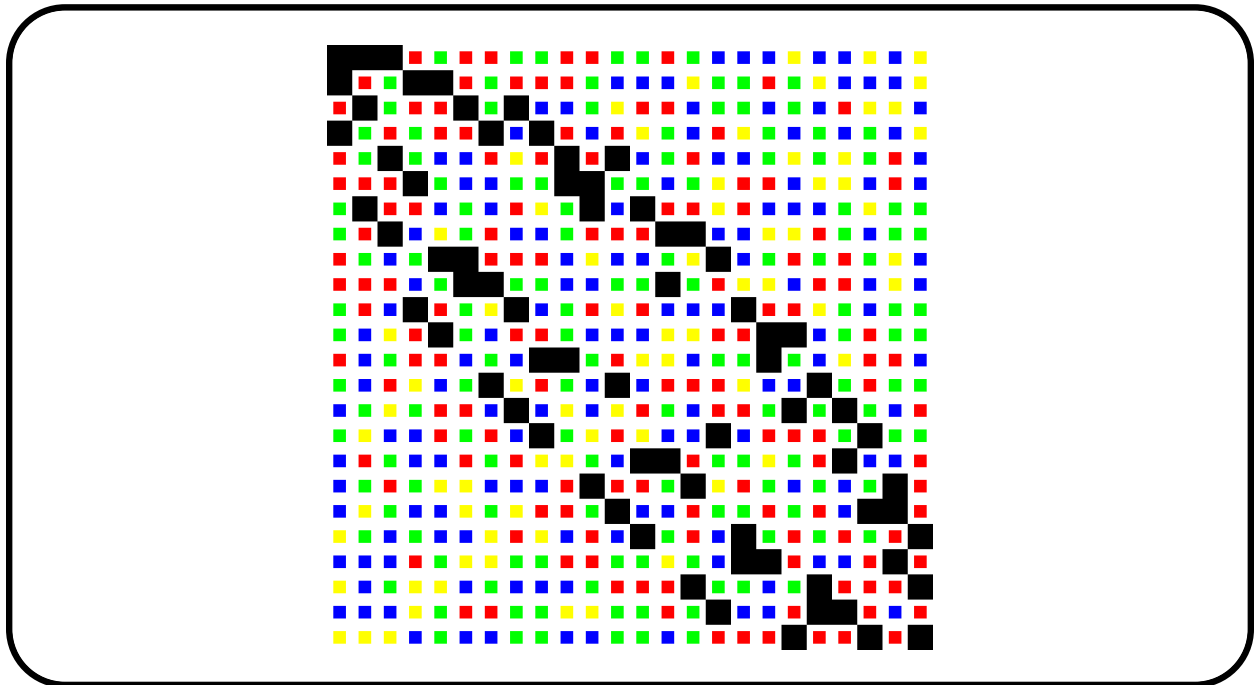


144\  
 \n-1 3"

**Example 642**

```
TFC_24_3.c:
▷ echo $(FILE_24_3.TFC_INC) >24_3.TFC.inc
▷ $(ORBITER) -v 20 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label 24_3.TFC 24\3\TFC \
▷ ▷ ▷ -file_of_incidence_geometries 24_3.TFC.inc 24 24 72 \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form \
▷ ▷ ▷ -save_ago \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ ▷ -show_incidence_matrices \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex 24_3.TFC_classification.tex
▷ $(OPEN) 24_3.TFC_classification.pdf
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file 24_3.TFC.object2.TDA.flag_orbits.csv \
▷ ▷ -secondary_input_csv_file 24_3.TFC.object2.TDA.csv \
▷ ▷ -box_width 40 -bit_depth 24 \
▷ ▷ -end
▷ $(OPEN) 24_3.TFC.object2.TDA.flag_orbits.draw.bmp
```

The command also computes the tactical decomposition induced by the automorphism group. In addition, the command also computes the orbits on flags and on anti-flags. The third of the three geometries is flag transitive. A bitmap drawing is produced, shown below:



The geometry is flag transitive, so all incidences are drawn in the same color (black).

The next command produces a diagram of the flag orbits for each of the twelve Latin squares of order 6.

#### Example 643

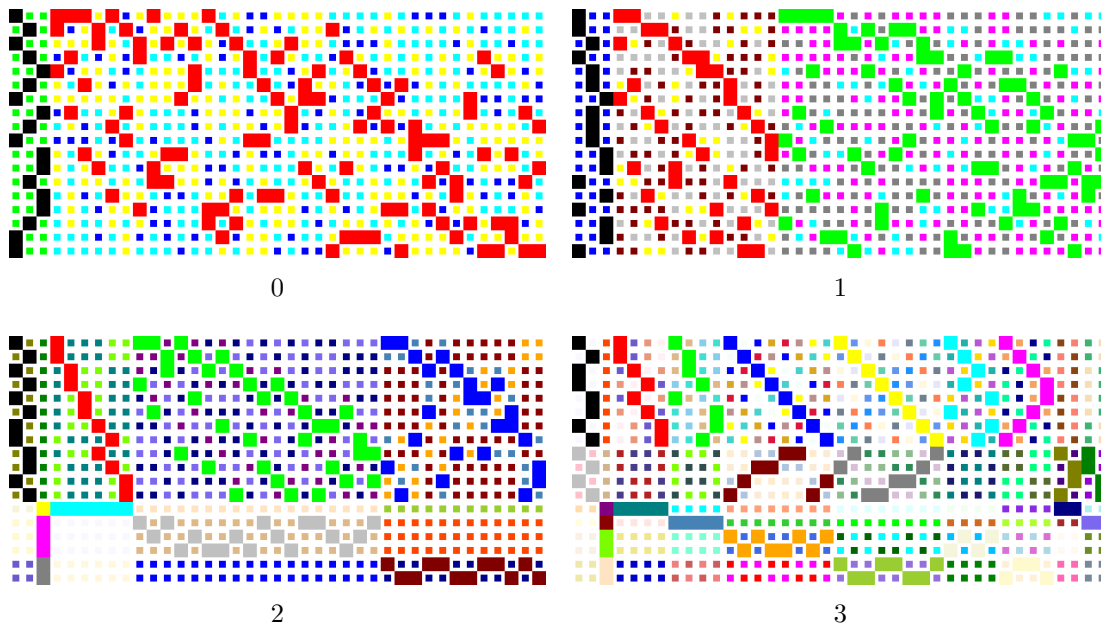
```
geo.LSQ6.c: LSQ6.inc
> $(ORBITER) -v 3 \
> > -draw_incidence_structure_description \
> > > -width 60 -width_10 6 -end \
> > -define C -combinatorial_object \
> > > -label LSQ6 LSQ6 \
> > > -file_of_incidence_geometries \
> > > > LSQ6.inc 18 39 126 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form \
> > > > -save_ago \
> > > > -save_transversal \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex LSQ6_classification.tex
```

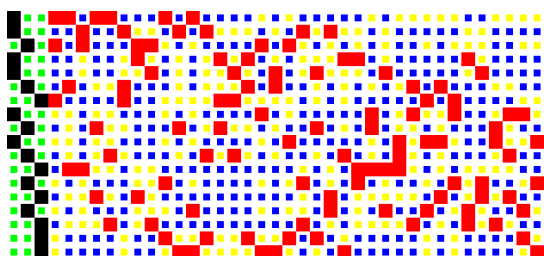
```

▷ $(OPEN) LSQ6_classification.pdf
▷ $(ORBITER) -v 2 \
▷ ▷ -loop L 0 12 1 \
▷ ▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file LSQ6_object%L_TDA_flag_orbits.csv \
▷ ▷ ▷ -secondary_input_csv_file LSQ6_object%L_TDA.csv \
▷ ▷ ▷ -box_width 32 -bit_depth 24 \
▷ ▷ ▷ -end \
▷ ▷ ▷ -system "convert \
▷ ▷ ▷ ▷ LSQ6_object%L_TDA_flag_orbits.draw.bmp \
▷ ▷ ▷ ▷ LSQ6_object%L_TDA_flag_orbits.draw.png" \
▷ ▷ -end_loop L
▷ $(ORBITER) -v 2 \
▷ ▷ -loop L 0 12 1 \
▷ ▷ ▷ -draw_matrix \
▷ ▷ ▷ -input_csv_file LSQ6_object%L_INP_flag_orbits.csv \
▷ ▷ ▷ -secondary_input_csv_file LSQ6_object%L_INP.csv \
▷ ▷ ▷ -box_width 32 -bit_depth 24 \
▷ ▷ ▷ -end \
▷ ▷ -end_loop L

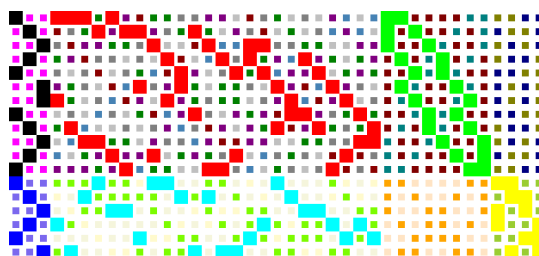
```

The output is shown below





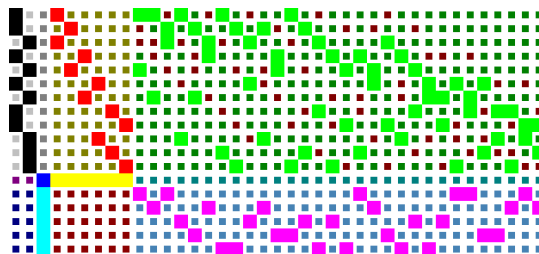
4



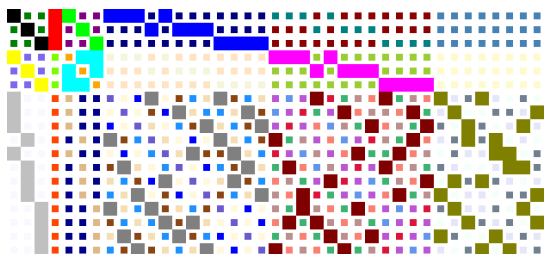
5



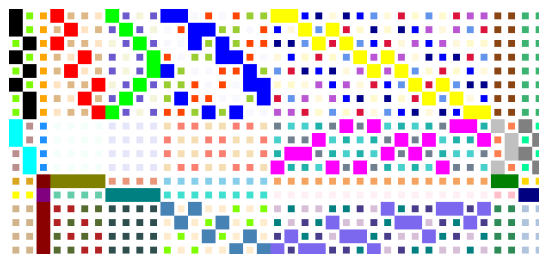
6



7



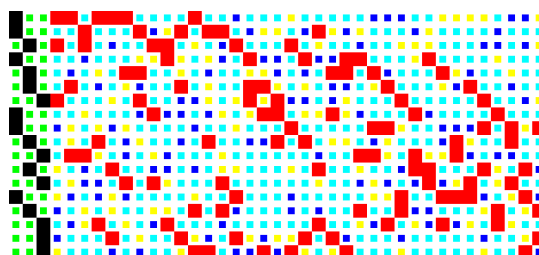
8



9



10



11

## 16.4 Canonical Forms of Objects from Design Theory

In Section 13.1, we discussed how to create designs in Orbiter. In this section, we wish to compute properties of designs related to the automorphism group. The first step is to export the incidence matrix of the design to file. After that, we compute the canonical form of the design, which allows us to determine many properties. The following example computes the properties of  $PG(2, 3)$ :

### Example 644

```
design_PG_2_3_canonical:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 3 -end \
▷ ▷ -define D -design -field F -family PG_2_q -end \
▷ ▷ -with D -do \
▷ ▷ ▷ -design_activity \
▷ ▷ ▷ ▷ -export_inc \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ $(ORBITER) -v 3 \
▷ ▷ -draw_incidence_structure_description \
▷ ▷ ▷ -width 60 -width_10 6 -end \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label PG_2_3 PG\_2\_3 \
▷ ▷ ▷ -file_of_incidence_geometries \
▷ ▷ ▷ ▷ PG_2_3_inc.txt 13 13 52 \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -save_transversal \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ ▷ -show_incidence_matrices \
▷ ▷ ▷ ▷ -export_group_GAP \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex PG_2_3_classification.tex
▷ $(OPEN) PG_2_3_classification.pdf
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file PG_2_3_object0_TDA_flag_orbits.csv \
▷ ▷ -secondary_input_csv_file PG_2_3_object0_TDA.csv \
▷ ▷ -box_width 32 -bit_depth 24 \
▷ ▷ -end
▷ $(OPEN) PG_2_3_object0_TDA_flag_orbits_draw.bmp
▷
```

The command

**Example 645**

```
wreath_product_designs_n4_k2.c: wreath_product_designs_n4_k2_inc.txt
> $(ORBITER) -v 10 \
> > -draw_incidence_structure_description \
> > > -width 60 -width_10 6 -end \
> > -define C -combinatorial_object \
> > > -label wreath_4_2 wreath\_4\_2 \
> > > -file_of_incidence_geometries \
> > > wreath_product_designs_n4_k2_inc.txt \
> > > 8 12 24 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form \
> > > > -save_ago \
> > > > -save_transversal \
> > > -end \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -report \
> > > > -export_flag_orbits \
> > > > -show_incidence_matrices \
> > > > -export_group_GAP \
> > > -end \
> > -end
> pdflatex wreath_4_2_classification.tex
> $(OPEN) wreath_4_2_classification.pdf
```

computes the automorphism group of the design on 8 points created in Section 13.1. The group is  $\text{Sym}(4) \wr \text{Sym}(2)$ . The command

**Example 646**

```
wreath_product_designs_n8_k6.c: wreath_product_designs_n8_k6_inc.txt
> $(ORBITER) -v 10 \
> > -draw_incidence_structure_description \
> > > -width 60 -width_10 6 -end \
> > -define C -combinatorial_object \
> > > -label wreath_8_6 wreath\_8\_6 \
> > > -file_of_incidence_geometries \
> > > wreath_product_designs_n8_k6_inc.txt \
> > > 16 3920 23520 \
> > -end \
> > -with C -do \
> > -combinatorial_object_activity \
> > > -canonical_form \
> > > > -save_ago \
> > > > -save_transversal \
> > > -end \
> > -end \
```

```

▷ ▷ -with C -do \
▷ ▷ -combinatorial_object.activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ ▷ -export_group_GAP \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex wreath_8.6_classification.tex
▷ $(OPEN) wreath_8.6_classification.pdf

```

computes the automorphism group of the design on 16 points created in Section 13.1. The group is  $\text{Sym}(8) \wr \text{Sym}(2)$ .

In Section 13.3, some large sets of  $\text{AG}(2, 3)$  were constructed. The final isomorphism classification is performed using the Nauty interface. A list of combinatorial objects is created, and the `-canonical_form` command is applied as activity. This will produce a list of pairwise non-isomorphic designs. The size of this list is the number of isomorphism types of large sets of  $\text{AG}(2, 3)$ .

#### Example 647

```

LS_AG_2.3_solutions_classify:
▷ $(ORBITER) -v 2 \
▷ ▷ -draw_incidence_structure_description \
▷ ▷ ▷ -width 20 -width_10 2 -end \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label LS_AG_2.3 LS\AG\2\3 \
▷ ▷ ▷ -file_of_designs \
▷ ▷ ▷ solutions.csv 9 84 3 12 \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object.activity \
▷ ▷ ▷ -canonical_form \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -save_transversal \
▷ ▷ ▷ ▷ -max_TDO_depth 10 \
▷ ▷ ▷ -end \
▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object.activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex LS_AG_2.3_classification.tex
▷ $(OPEN) LS_AG_2.3_classification.pdf
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file LS_AG_2.3_object0_INP_flag_orbits.csv \
▷ ▷ -secondary_input_csv_file LS_AG_2.3_object0_INP.csv \
▷ ▷ -box_width 12 -bit_depth 24 \
▷ ▷ -end

```

```
▷ $(OPEN) LS_AG_2_3_object0_INP_flag_orbits_draw.bmp
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file LS_AG_2_3_object1_INP_flag_orbits.csv \
▷ ▷ -secondary_input_csv_file LS_AG_2_3_object1_INP.csv \
▷ ▷ -box_width 12 -bit_depth 24 \
▷ ▷ -end
▷ $(OPEN) LS_AG_2_3_object1_INP_flag_orbits_draw.bmp
```

It turns out that there are exactly two isomorphism types, with automorphism groups of order 54 and 42, respectively.



## 16.5 Canonical Forms of Linear Codes

Using Nauty, Orbiter can compute canonical forms and automorphism groups of codes. For linear codes, the semilinear automorphism group can be computed.

Consider the  $[3, 2, 2]$  code generated by

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Using the following command, the semilinear automorphism group of the code can be computed:

### Example 648

```
code_3_2.aut:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define genma -vector -field F -format 2 \
▷ ▷ ▷ -dense $(CODE_N3_K2_Q2_GENMA) \
▷ ▷ -end \
▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
▷ ▷ -with P -do \
▷ ▷ -projective_space_activity \
▷ ▷ ▷ -canonical_form_of_code \
▷ ▷ ▷ ▷ "3_2" genma -save_ago -label "3_2" \
▷ ▷ ▷ ▷ -classification_prefix "3_2" \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex 3_2_classification.tex
▷ $(OPEN) 3_2_classification.pdf
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file 3_2_object0_TDA_flag_orbits.csv \
▷ ▷ -secondary_input_csv_file 3_2_object0_TDA.csv \
▷ ▷ -box_width 16 -bit_depth 24 \
▷ ▷ -end
▷ $(OPEN) 3_2_object0_TDA_flag_orbits_draw.bmp
```

The code has a semilinear automorphism group of order 6. The following report is written:

### Summary of Orbits

|   | Rep | # | Ago | Objects |
|---|-----|---|-----|---------|
| 0 | 0   | 1 | 6   | 0       |

Ago : 6

### Isomorphism type 0 / 1

Isomorphism type 0 / 1 is original object 0 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {0}

set of points of size 3: ( 0, 1, 2 )

| $i$ | Rank | Point  |
|-----|------|--------|
| 0   | 0    | (1, 0) |
| 1   | 1    | (0, 1) |
| 2   | 2    | (1, 1) |

Column sets of the encoded object:

$\{ 0, 1, 2 \}$

$\{ 0, 1, 2, 3 \}$

Row sets of the encoded object:

$\{ 0, 1 \} = 0$

$\{ 0, 1 \} = 0$

$\{ 0, 1 \} = 0$

$\{ 1 \} = 1$

|   |   |
|---|---|
| 0 | 0 |
| 0 | 0 |
| 0 | 0 |
| 0 | 0 |
| 1 | 0 |

Generators for the automorphism group:

The stabilizer of order 6 is generated by:

$g_1 = (1, 2)$  of order 2 and with 4 fixed points.

$g_2 = (0, 1)$  of order 2 and with 4 fixed points.

Generators for the automorphism group as matrix group:

The stabilizer of order 6 is generated by:

$$g_1 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 10 \\ 11 \end{bmatrix} \text{ of order 2 and with 1 fixed points.}$$

$$g_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 01 \\ 10 \end{bmatrix} \text{ of order 2 and with 1 fixed points.}$$

Decomposition by combinatorial refinement:

$$\begin{array}{c|c} \rightarrow & 2_1 \\ \hline 4_0 & 2 \end{array}$$

$$\begin{array}{c|c} \downarrow & 2_1 \\ \hline 4_0 & 3 \end{array}$$

Decomposition by automorphism group:

|       |
|-------|
| 4 5   |
| 0 0 0 |
| 1 0 0 |
| 2 0 0 |
| 3 0 0 |

Canonical labeling:

canonical row = 3

canonical orbit number = 1

Flags : ( 0, 1, 2, 3, 4, 5, 7 )

|        |
|--------|
| 4 5    |
| 0 0 0  |
| 1 0 0  |
| 2 0 0  |
| *3 0 0 |

Flag orbits:

orbit length : number of orbits of that length:

|   |   |
|---|---|
| 1 | 1 |
| 3 | 2 |

Anti-Flag orbits:

orbit length : number of orbits of that length:

|   |   |
|---|---|
| 1 | 1 |
|---|---|

The command

#### Example 649

```
CODE_RM_3_1_GENMA="\
11111111\
01010101\
00110011\
00001111"
```

#### Example 650

```
RM_3_1_group:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 2 -end \
▷ ▷ -define genma -vector -field F -format 4 \
▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
▷ ▷ -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with P -do \
▷ ▷ -projective_space_activity \
▷ ▷ ▷ -canonical_form_of_code \
▷ ▷ ▷ ▷ "RM_3_1" genma -save_ago -label "RM_3_1" \
▷ ▷ ▷ ▷ -classification_prefix "RM_3_1" \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex RM_3_1_classification.tex
▷ $(OPEN) RM_3_1_classification.pdf
```

computes the automorphism group of the Reed-Muller code. The group is an affine linear group  $AGL(3, 2)$  of order 1344. A report is created, showing the automorphism group and the action on  $PG(3, 2)$ , with the Reed-Muller code distinguished.

The following command creates a drawing of the incidence matrix between points and lines in  $PG(3, 2)$ , with the Reed-Muller code distinguished:

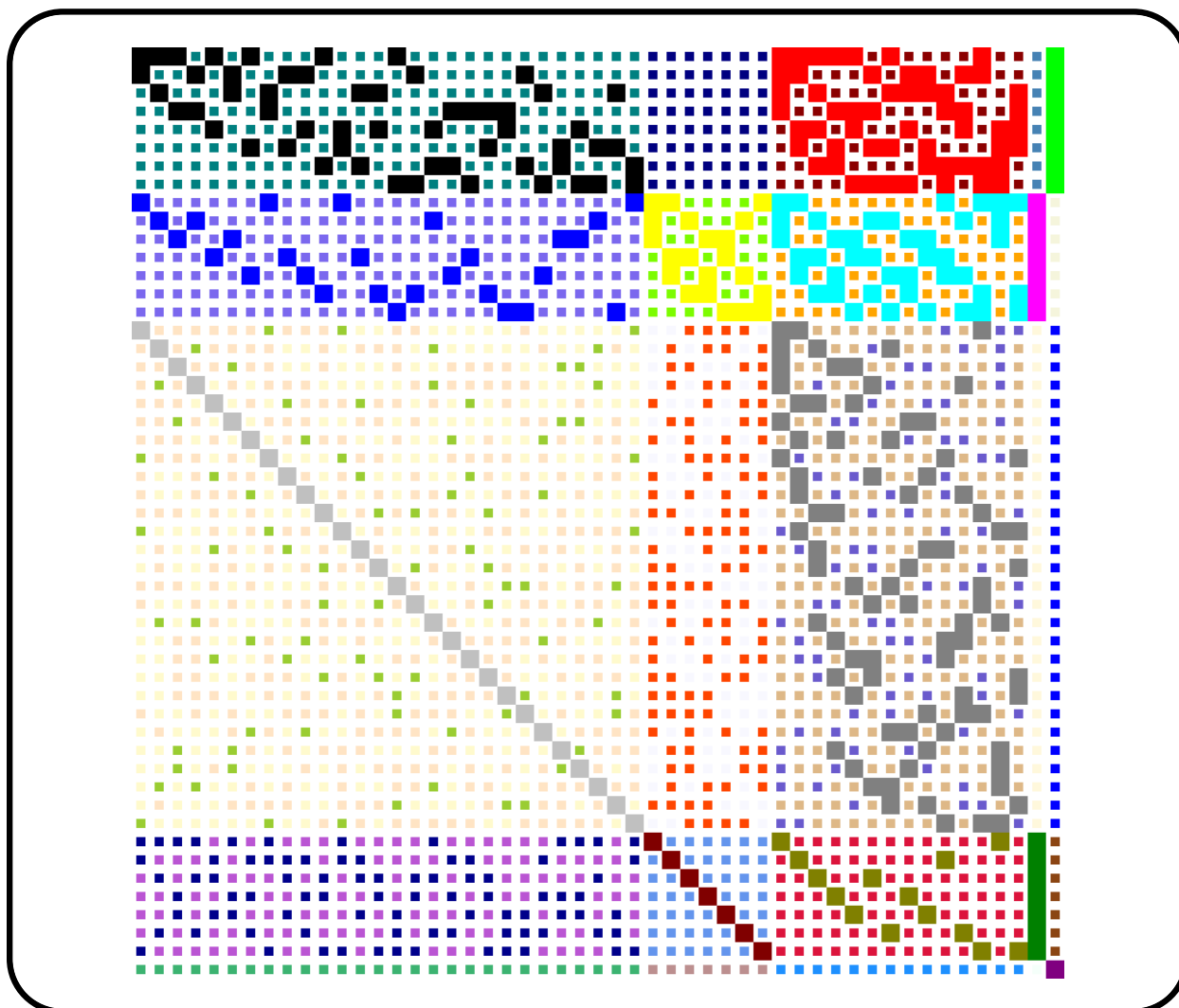
**Example 651**

```

RM_3_1_group_and_diagram:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define genma -vector -field F -format 4 \
> > > -compact $(CODE_RM_3_1_GENMA) \
> > -end \
> > -define P -projective_space -n 3 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -canonical_form_of_code \
> > > > "RM_3_1" genma -save_ago -label "RM_3_1" \
> > > > -classification_prefix "RM_3_1" \
> > > -end \
> > -end
> pdflatex RM_3_1_classification.tex
> $(OPEN) RM_3_1_classification.pdf
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file RM_3_1_object0_INP_flag_orbits.csv \
> > -secondary_input_csv_file RM_3_1_object0_INP.csv \
> > -box_width 16 -bit_depth 24 \
> > -end
> $(ORBITER) -v 2 -draw_matrix \
> > -input_csv_file RM_3_1_object0_TDA_flag_orbits.csv \
> > -secondary_input_csv_file RM_3_1_object0_TDA.csv \
> > -box_width 16 -bit_depth 24 \
> > -end
> $(OPEN) RM_3_1_object0_INP_flag_orbits_draw.bmp
> $(OPEN) RM_3_1_object0_TDA_flag_orbits_draw.bmp

```

The drawing is shown below:



The command

#### Example 652

```
RS_6.4.7_group:
▷ $(ORBITER) -v 2 \
▷ ▷ -define F -finite_field -q 7 -end \
▷ ▷ -define genma -vector -field F -format 4 \
▷ ▷ ▷ -compact $(CODE_RS_6.4.7) \
▷ ▷ -end \
▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
▷ ▷ -with P -do \
▷ ▷ -projective_space_activity \
▷ ▷ ▷ -canonical_form_of_code \
▷ ▷ ▷ ▷ "RS_6" genma -save_ago -label "RS_6" \
▷ ▷ ▷ ▷ -classification_prefix "RS_6" \
▷ ▷ ▷ -end \
▷ ▷ -end
```

shows that the automorphism group has order 12. After some shortening, the output is:

Isomorphism type 0 / 1 is original object 0 and appears 1 times:  
set of points of size 6:  $\{(0, 9, 51, 344, 253, 3)\}$

| $i$ | Rank | Point        |
|-----|------|--------------|
| 0   | 0    | (1, 0, 0, 0) |
| 1   | 9    | (5, 1, 0, 0) |
| 2   | 51   | (6, 5, 1, 0) |
| 3   | 344  | (0, 6, 5, 1) |
| 4   | 253  | (0, 0, 4, 1) |
| 5   | 3    | (0, 0, 0, 1) |

Group order 12

This isomorphism type appears 1 times, namely for the following 1 input objects:  $\{0\}$

Stabilizer:

Strong generators for a group of order 12:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 \\ 5 & 0 & 6 & 0 \\ 5 & 1 & 0 & 0 \end{bmatrix},$$

$$\begin{bmatrix} 0 & 0 & 0 & 3 \\ 4 & 0 & 0 & 6 \\ 0 & 4 & 0 & 1 \\ 0 & 0 & 4 & 1 \end{bmatrix}$$

1,0,0,0,2,0,0,1,5,0,6,0,5,1,0,0,  
0,0,0,1,6,0,0,2,0,6,0,5,0,0,6,5,

$$\begin{array}{c|cc} \rightarrow & 2850_1 & 1_2 \\ \hline 401_0 & 57 & 1 \end{array}$$

The command

#### Example 653

```
GV_n15_k6.d5.group:
> $(ORBITER) -v 2 \
> > -define F -finite_field -q 2 -end \
> > -define genma -vector -field F -format 6 \
> > > -compact $(CODE_GV_N15_K6) \
> > -end \
> > -define P -projective_space -n 5 -field F -v 0 -end \
> > -with P -do \
> > -projective_space_activity \
> > > -canonical_form_of_code \
```

```
▷ ▷ ▷ ▷ "GV_n15_k6_d5" genma -save_ago -label "GV_n15_k6_d5" \  
▷ ▷ ▷ ▷ -classification_prefix "GV_n15_k6_d5" \  
▷ ▷ ▷ -end \  
▷ ▷ -end  
▷ #pdflatex GV_n15_k6_d5_classification.tex  
▷ #$(OPEN) GV_n15_k6_d5_classification.pdf
```

computes the automorphism group of the Gilbert-Varshamov code from Section 10.9. It has order 12.

## 16.6 Canonical Forms of General Codes

The command

### Example 654

```
HAMMING_CODE_CODEWORDS="0, 67, 37, 102, 22, 85, \
51, 112, 15, 76, 42, 105, 25, 90, 60, 127"
```

### Example 655

```
Hamming_graph_7_with_Hamming_code:
▷ $(ORBITER) -v 2 \
▷ ▷ -define G -graph -Hamming 7 2 \
▷ ▷ ▷ -subset "_Hamming_code" "\\_with\\_Hamming\\_code" \
▷ ▷ ▷ $(HAMMING_CODE_CODEWORDS) -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity -export_csv -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity -export_graphviz -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity -save -end \
▷ ▷ -with G -do \
▷ ▷ -graph_theoretic_activity -automorphism_group -end
▷ pdflatex Hamming_7.2_Hamming_code_report.tex
▷ $(OPEN) Hamming_7.2_Hamming_code_report.pdf
```

computes the set stabilizer of the Hamming code inside the automorphism group of the Hamming graph. The group has order  $2688 = 16 \cdot 168$ .



## 16.7 Canonical Forms of Graphs

Orbiter can compute isomorphism and automorphism between graphs. Here are some examples.

Suppose we want to compute the automorphism group of the cycle graph of order 13:

### Example 656

```
Cycle_13_aut:
> $(ORBITER) -v 2 \
> > -define Gamma -graph -cycle 13 -end \
> > -with Gamma -do \
> > > -graph_theoretic_activity -automorphism_group \
> > -end \
```

The output is two files: The first one, `Cycle_13_group.makefile` is a makefile containing an Orbiter command to create the automorphism group: The second file is `Cycle_13_gens.csv`, which contains the permutation representation of the group, and which is needed for the makefile.

The next command computes the automorphism group of the chain graph with respect to the partition  $(2, 3, 2)$ .

### Example 657

```
Chain_232_aut:
> $(ORBITER) -v 2 \
> > -define P1 -vector -dense 2,3,2 -end \
> > -define P2 -vector -dense 2,3,2 -end \
> > -define Gamma -graph \
> > > -chain_graph P1 P2 \
> > -end \
> > -with Gamma -do \
> > > -graph_theoretic_activity -automorphism_group \
> > -end
> pdflatex chain_graph_report.tex
> $(OPEN) chain_graph_report.pdf
```

The following report is written:

The automorphism group of *chain\_graph* has order 1152 and is generated by:  
Strong generators for a group of order 1152:

(12, 13),  
(3, 4),  
(2, 3),  
(10, 11),  
(9, 10),  
(5, 6),

```

(7, 8),
(0, 1),
(0, 12)(1, 13)(2, 9)(3, 10)(4, 11)(5, 7)(6, 8)

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 12, 14,
0, 1, 2, 4, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
0, 1, 3, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 10, 12, 13, 14,
0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 9, 11, 12, 13, 14,
0, 1, 2, 3, 4, 6, 5, 7, 8, 9, 10, 11, 12, 13, 14,
0, 1, 2, 3, 4, 5, 6, 8, 7, 9, 10, 11, 12, 13, 14,
1, 0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
12, 13, 9, 10, 11, 7, 8, 5, 6, 2, 3, 4, 0, 1, 14,

```

Junttila and Kaski maintain a collection of graphs that can be used as test cases. The graphs are stored in Dimacs format and can be read in through the Orbiter `-load_dimacs` command. For instance, the following command computes the automorphism group of the Levi graph of the desarguesian projective plane of order 16:

#### Example 658

```

JK_graph_pp16_1:
> $(ORBITER) -v 2 \
> > -define Gamma -graph -load_dimacs \
> > ../JUNTILA_KASKI/benchmarks/pp/pp16-1 \
> > -end \
> > -with Gamma -do \
> > -graph_theoretic_activity -save -end \
> > -with Gamma -do \
> > -graph_theoretic_activity -automorphism_group -end \

```

The command shows a group of order 34217164800. As a measurement of the complexity, the number of backtrack nodes in Nauty is recorded:

```

nb_backtrack1 = 6
nb_backtrack2 = 134
nb_backtrack3 = 134
nb_backtrack4 = 1

```

Here, `nb_backtrack1` is the number of calls to `firstpathnode`, `nb_backtrack2` is the number of calls to `othernode`, `nb_backtrack3` is the number of calls to `processnode`, `nb_backtrack4` is the number of calls to `firstterminal`. These are the four recursive functions in Nauty.

Unfortunately, the complexity of graph isomorphism is not well-understood. We can see this here. While the first projective plane of order 16 can be handled relatively easily, the second one causes problems. The following command hardly finishes:

**Example 659**

```

JK_graph_pp16_2:
▷ $(ORBITER) -v 2 \
▷ ▷ -define Gamma -graph -load_dimacs \
▷ ▷ ../JUNTTILA_KASKI/benchmarks/pp/pp16-2 \
▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -save -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -automorphism_group -end \

```

The difference between the two planes is that the first plane has a very large automorphism group, while the second one has not. For any  $q$ , the Desarguesian plane  $\text{PG}(2, q)$  has the largest automorphism group of all projective planes of order  $q$ .

The following example considers the block intersection graph of a Steiner triple system (“STS”) of order 13. There are exactly two  $\text{STS}(13)$ . The one we consider here has a group of order 39. The block intersection graph has the same automorphism group.

**Example 660**

```

JK_graph_sts_13:
▷ $(ORBITER) -v 2 \
▷ ▷ -define Gamma -graph -load_dimacs \
▷ ▷ ▷ ../JUNTTILA_KASKI/benchmarks/srg/sts-13 \
▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -save -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity -automorphism_group -end
▷ make ORBITER_PATH=$(ORBITER_EXE_PATH) -f sts-13_group.makefile sts-13

```

The automorphism group has order 39 and is generated by:

```

(
(1, 25, 16)(2, 18, 20)(3, 7, 15)(4, 13, 11)(5, 6, 17)
(9, 12, 19)(10, 14, 24)(21, 23, 22),
(0, 1, 2)(3, 4, 5)(7, 8, 9)(11, 12, 13)(14, 16, 18)
(15, 17, 19)(20, 22, 24)(21, 23, 25))

```

Graphs can be created from groups by means of orbitals. An orbital is an orbit of a permutation group  $G$  on the set of pairs. Here is an example. We start from the Coxeter-Tits graph on 315 vertices, whose automorphism group is the Hall-Janko group  $J_2 : 2$ . We first read the graph from file `halljanko315.csv` and compute the automorphism group using Nauty:

**Example 661**

```

halljanko315_gens.csv:
> $(ORBITER) -v 6 \
> > -define G -graph \
> > > -load_csv_no_border \
> > > halljanko315.csv \
> > -end \
> > -with G -do \
> > > -graph_theoretic_activity -automorphism_group \
> > -end \
> > -with G -do \
> > > -graph_theoretic_activity -properties \
> > -end

```

The next step is to compute the orbits of the automorphism group on pairs, using the following command:

**Example 662**

```

HJ_group_and_orbits:
> $(ORBITER) -v 2 \
> > -define Control -poset_classification_control \
> > > -W \
> > > -problem_label HJ_orbits \
> > > -depth 2 \
> > -end \
> > -define gens -vector -file \
> > > halljanko315_gens.csv -end \
> > -define G -permutation_group \
> > > -bsgs halljanko315 "File\halljanko315" \
> > > 315 1209600 "0,1,2" 6 gens \
> > -end \
> > -define Orb -orbits -group G \
> > > -on_subsets 2 Control \
> > -end

```

There are 4 orbits on pairs. We decide to pick the fourth orbit to create a new graph. Because indexing is zero-based, we give the orbit index of 3:

**Example 663**

```

HJ_orbital_graph_3:
> $(ORBITER) -v 2 \
> > -define gens -vector -file \
> > > halljanko315_gens.csv -end \
> > -define G -permutation_group \
> > > -bsgs halljanko315 "File\halljanko315" \
> > > 315 1209600 "0,1,2" 6 gens \
> > -end \
> > -define Gamma -graph \
> > > -orbital_graph G 3 \

```

```

▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -properties \
▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -save \
▷ ▷ -end

```

The graph is regular of degree 64.

The next command computes the automorphism group of the collinearity graph of the  $Q(4, 2)$  quadric.

#### Example 664

```

PGO_5_2_graph_group: 0.5_2.incidence.matrix.csv
▷ $(ORBITER) -v 3 \
▷ ▷ -define Inc -vector -file 0.5_2.incidence.matrix.csv -end \
▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -automorphism_group \
▷ ▷ -end \
▷ ▷ -with Gamma -do \
▷ ▷ -graph_theoretic_activity \
▷ ▷ ▷ -eigenvalues \
▷ ▷ -end
▷ pdflatex collinearity_graph_eigenvalues.tex
▷ $(OPEN) collinearity_graph_eigenvalues.pdf

```

The group is  $\text{PGO}(5, 2)$  of order 720. The command creates the group as a permutation group on the 15 vertices of the graph. The group is no longer treated as a matrix group.

## 16.8 Canonical Forms of Quartic Curves

We wish to study the automorphism groups of the quartic curves of Edge that have been considered in Section 4.10. We start by creating a cheat sheet of the field  $\mathbb{F}_{17}$

### Example 665

```
F_17_edge:
▷ $(ORBITER) -v 3 \
▷ ▷ -define F -finite_field -q 17 -end \
▷ ▷ -with F -do -finite_field_activity \
▷ ▷ ▷ -cheat_sheet_GF -end
▷ pdflatex GF_17.tex
▷ $(OPEN) GF_17.pdf
```

Next, we compute the canonical form of the Edge quartic. This command also computes generators for the automorphism group of the curve.

### Example 666

```
Edge_curve_17_nauty:
▷ $(ORBITER) -v 3 \
▷ ▷ -define C -combinatorial_object \
▷ ▷ ▷ -label Edge_curve_q17 Edge\_curve\_q17 \
▷ ▷ ▷ -file_of_points Edge_q17.txt \
▷ ▷ -end \
▷ ▷ -define F -finite_field -q 17 -end \
▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -canonical_form_PG P \
▷ ▷ ▷ ▷ -save_ago \
▷ ▷ ▷ ▷ -save_transversal \
▷ ▷ ▷ ▷ -max_TDO_depth 10 \
▷ ▷ ▷ ▷ -end \
▷ ▷ ▷ -end \
▷ ▷ -with C -do \
▷ ▷ -combinatorial_object_activity \
▷ ▷ ▷ -report \
▷ ▷ ▷ ▷ -export_flag_orbits \
▷ ▷ ▷ ▷ -show_TDO \
▷ ▷ ▷ ▷ -show_TDA \
▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
▷ ▷ ▷ ▷ -export_group_GAP \
▷ ▷ ▷ -end \
▷ ▷ -end
▷ pdflatex Edge_curve_q17_classification.tex
▷ $(OPEN) Edge_curve_q17_classification.pdf
▷ $(ORBITER) -v 2 -draw_matrix \
▷ ▷ -input_csv_file Edge_curve_q17_object0_TDA_flag_orbits.csv \
▷ ▷ -secondary_input_csv_file Edge_curve_q17_object0_TDA.csv \
▷ ▷ -box_width 4 -bit_depth 24 \
```

```

▷ ▷ -end
▷ $(OPEN) Edge_curve.q17_object0_TDA_flag_orbits_draw.bmp

```

Using the generators that have just been computed, we can recreate the group of the quartic curve:

#### Example 667

```

Edge_curve_17_group:
▷ $(ORBITER) -v 3 \
▷ ▷ -define G -linear_group -PGL 3 17 \
▷ ▷ -subgroup_by_generators "Stab_Edge" "24" 3 \
▷ ▷ ▷ "1,0,0,0,13,0,0,0,4, \
▷ ▷ ▷ 1,0,0,0,0,16,0,16,0, \
▷ ▷ ▷ 0,1,16,2,4,4,15,4,4" \
▷ ▷ -end \
▷ ▷ -with G -do \
▷ ▷ ▷ -group_theoretic_activity \
▷ ▷ ▷ -print_elements_tex \
▷ ▷ ▷ -report_group_table \
▷ ▷ ▷ -report \
▷ ▷ -end
▷ pdflatex PGL_3_17_Subgroup_Stab_Edge_24_report.tex
▷ $(OPEN) PGL_3_17_Subgroup_Stab_Edge_24_report.pdf

```





# Chapter 17

## Interfaces

### 17.1 Graphical Output

Orbiter can produce graphical output in a variety of formats:

1. TikZ / Latex [66], see Table 16.3,
2. Metapost [34],
3. Bitmap files (.bmp) [69], see Table 16.2,
4. Povray, see Section 17.2,
5. Gnuplot [30], see Section 17.5.

The poset classification algorithm from Sections 7.3 and 7.4 computes partially ordered sets. The posets are created using the `-draw_poset` option in the poset classification control command package, see Table 7.3. The posets are stored in a file with extension `.layered_graph`. These files can be drawn using the `-draw_layered_graph` command. The commands in Table 17.1 and Table 17.2 show ways in which to customize the drawings. Let us consider an example. Suppose we are interested in the Schreier trees of a permutation group represented in a stabilizer chain. We take  $\text{PGL}(4, 2)$  in its action on the wedge product. The command

#### Example 668

```
PGL_6_2.Wedge_4_2.detached.report.tex:
> $(ORBITER) -v 4 \
> > -define G -linear_group -PGL 4 2 \
> > > -wedge_detached \
> > -end \
> > -with G -do \
> > -group_theoretic_activity \
> > > -report \
> > -end
> pdflatex PGL_6_2.Wedge_4_2.detached.report.tex
> $(OPEN) PGL_6_2.Wedge_4_2.detached.report.pdf
```

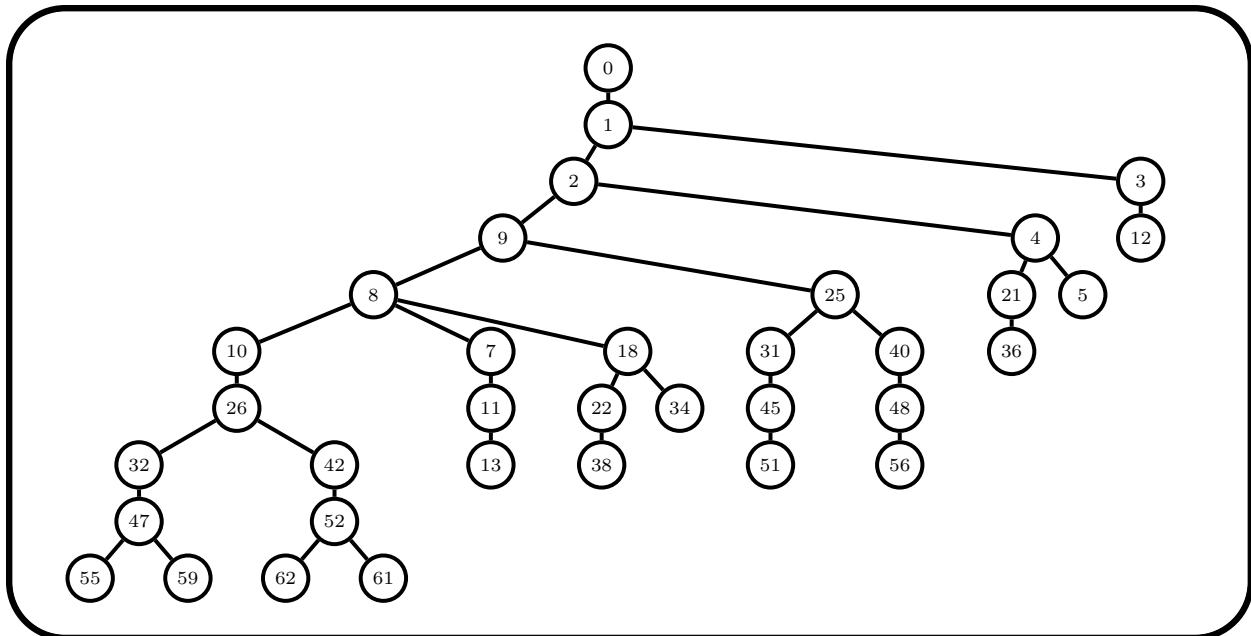
produces a report. The following tree is taken from this report. It represents the first basic orbit in the stabilizer chain of the group in that action.

| Drawing Options for Layered Graph Files (Part 1) |           |                                                                                                                                    |
|--------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------|
| Command                                          | Arguments | Purpose                                                                                                                            |
| -paperheight                                     | $h$       | Set paperheight for the latex document.                                                                                            |
| -paperwidth                                      | $w$       | Set paperwidth for the latex document.                                                                                             |
| -xin                                             | $a$       | Assume input $x$ -coordinates are in the interval $[0, a]$ . Default value: 10000.                                                 |
| -yin                                             | $a$       | Assume input $y$ -coordinates are in the interval $[0, a]$ . Default value: 10000.                                                 |
| -xout                                            | $a$       | Assume output $x$ -coordinates are in the interval $[0, a]$ . Default value: 1000000.                                              |
| -yout                                            | $a$       | Assume output $y$ -coordinates are in the interval $[0, a]$ . Default value: 1000000.                                              |
| -spanning_tree                                   |           | Place nodes according to a spanning tree. Default value: off.                                                                      |
| -circle                                          |           | Circle all nodes. Default value: on.                                                                                               |
| -corners                                         |           | Draw corners at the outside of the drawing. Default value: off.                                                                    |
| -radius                                          | $r$       | Use radius $r$ for drawing circles around nodes. Default value: 200.                                                               |
| -embedded                                        |           | Create latex headers for stand-alone latex files. Default value: off.                                                              |
| -sideways                                        |           | Create latex figure sideways. Default value: off.                                                                                  |
| -label_edges                                     |           | Label the edges in Schreier trees. Default value: off.                                                                             |
| -x_stretch                                       | $s$       | Apply $x$ -axis scaling by a factor of $s$ . Default value: $s = 1.0$ . This option does not affect the drawing of Schreier trees. |
| -y_stretch                                       | $s$       | Apply $y$ -axis scaling by a factor of $s$ . Default value: $s = 1.0$ . This option does not affect the drawing of Schreier trees. |

Table 17.1: Drawing Options for Layered Graph Files (Part 1)

| Drawing Options for Layered Graph Files (Part 2) |                         |                                                                                                                                                                                            |
|--------------------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                          | Arguments               | Purpose                                                                                                                                                                                    |
| -scale                                           | $s$                     | Use Tikz global scale-factor of $s$ . Default value: $s = 0.45$ .                                                                                                                          |
| -line_width                                      | $s$                     | Set Tikz line width to $s$ . Default value: $s = 1.5$ .                                                                                                                                    |
| -rotated                                         |                         | Rotate the output.                                                                                                                                                                         |
| -nodes                                           |                         | Turn on node drawing.                                                                                                                                                                      |
| -nodes_empty                                     |                         | Do not label the nodes. Default value: off.                                                                                                                                                |
| -select_layers                                   | $S$                     | Draw layers whose index is given in the list $S$ only.                                                                                                                                     |
| -paths_in_between                                | $l_1 \ i_1 \ l_2 \ i_2$ | Draw all paths from node $(l_1, i_1)$ to node $(l_2, i_2)$ . Here, $(l, i)$ is the $i$ -th node at layer $l$ (counting from zero). Delete all other edges between layers $l_1$ and $l_2$ . |

Table 17.2: Drawing Options for Layered Graph Files (Part 2)



The command

#### Example 669

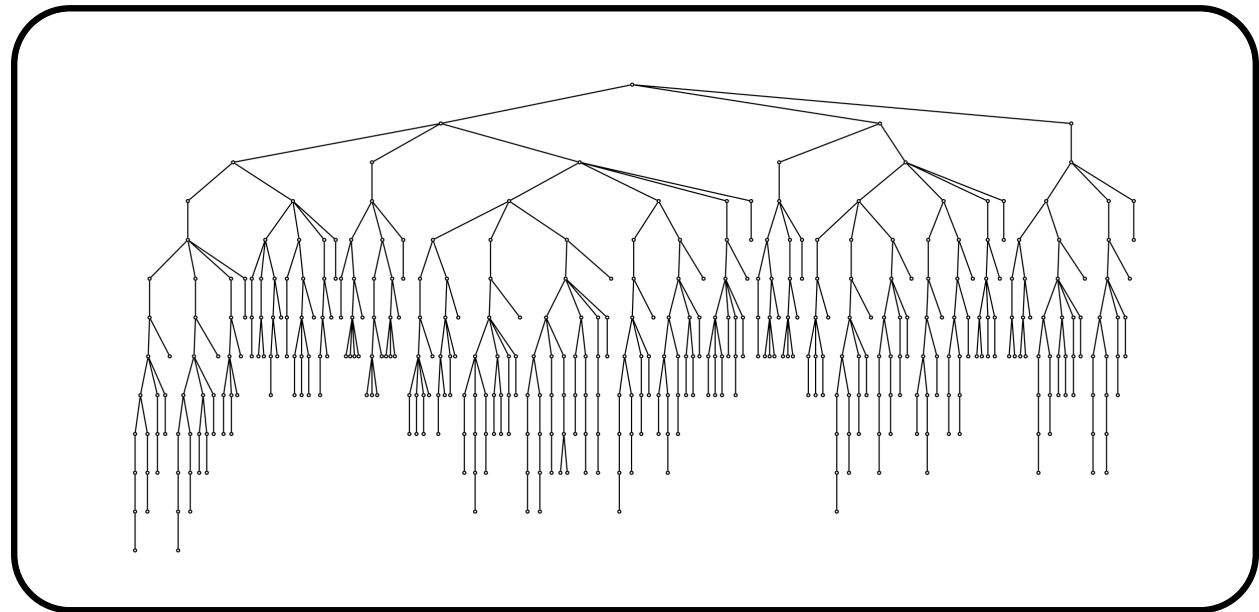
```
schreier_tree_graphical.output:
> $(ORBITER) -v 4 \
> > -draw_options \
> > > -yout 500000 \
> > > -radius 15 -nodes_empty \
> > > -line_width 0.5 -y_stretch 0.25 \
> > > -embedded \
```

```

▷ ▷ -end \
▷ ▷ -define G -linear_group -PGL 4 2 -end \
▷ ▷ -define Orb -orbits -group G \
▷ ▷ ▷ -on_polynomials 3 \
▷ ▷ -end \
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -export_something "orbit" 6 \
▷ ▷ -end
▷ ▷ -with Orb -do -orbits_activity \
▷ ▷ ▷ -draw_tree 6 \
▷ ▷ -end
▷

```

draws the 6th Schreier tree in the classification of orbits of  $\text{PGL}(4, 2)$  on homogeneous polynomials of degree 3 in 4 variables, shown below



The orbit has length 420.

| Povray Interface |             |                                              |
|------------------|-------------|----------------------------------------------|
| Command          | Arguments   | Purpose                                      |
| -povray          | description | Povray job description, see Table 17.4.      |
| -prepare_frames  | n           | Prepare frames description, see Table 17.10. |

Table 17.3: Povray Interface

| Povray Job Options |            |                                               |
|--------------------|------------|-----------------------------------------------|
| Command            | Arguments  | Purpose                                       |
| -video_options     |            | Set options from Tables 17.5 and 17.6.        |
| -round             | n          | Select round $n$ for rendering only.          |
| -rounds            | list       | Select a list of rounds for rendering.        |
| -nb_frames_default | f          | Select the number of frames per round.        |
| -output_mask       | fname-mask | Set mask for output file name generation.     |
| -scene_objects     | options    | Define Scene objects according to Table 17.9. |

Table 17.4: Povray Job Options

## 17.2 The Povray Interface

Orbiter can be used to create raytracing 3D-graphics. Orbiter serves as a front end for the raytracing software Povray [59]. This is a multi step process: A 3D scene is defined through orbiter commands. Next, Orbiter produces Povray files. After that, the povray files are processed through povray, and turned into graphics files (png), called frames. The frames can be turned into a video by using tools like ffmpeg (see Section 17.3). By default, an rotational animation is produced. The main commands are listed in Table 17.3.

To describe a povray job, the commands in Table 17.4 can be used.

The Orbiter Povray interface requires some general information about the animation, the camera position, the boundary box for clipping, the font size for text and others. Tables 17.5- 17.6 list the commands to control the 3D-povray frontend.

A povray graphics starts with a description of the scene. The scene will be rendered to the final frame. A scene is composed of geometric objects. Various types of objects are available: points, lines, planes, faces, algebraic surfaces, reguli, 3D-text, and others. Some complex objects are predefined, for instance the Eckardt surface. Once the objects are defined, output commands can be invoked to draw them in various colors and with various options. At times, there are many objects in one scene. In order to ease drawing, objects can be grouped together. Only objects of the same type may be grouped. A group of object can be drawn at once. Tables 17.7 and 17.8 summarize the Orbiter commands to build objects of a 3D scene. The commands in Table 17.9 are used to build the scene. Each of these commands applies to a group of objects of the same kind. Groups of objects are created using the commands in Table 17.8 which start with `group_of`. Here is a simple example which combines scene building and graphical output. The example creates a cube with vertices, edges and faces:

### Example 670

```
cube:
▷ $(ORBITER) -v 2 -povray \
▷ ▷ -round 0 -nb_frames_default 30 \
▷ ▷ -output_mask cube_%d.%03d.pov \
```

| Povray Interface (Part 1) |                         |                                                                                                                                                                    |
|---------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                   | Arguments               | Purpose                                                                                                                                                            |
| -do_not_rotate            |                         | No rotation. By default, the animation consists of a full rotation around a vertical axis.                                                                         |
| -rotate_about_z_axis      |                         | Rotate around $z$ -axis.                                                                                                                                           |
| -rotate_about_111         |                         | Rotate around $(1, 1, 1)$ -axis (default).                                                                                                                         |
| -rotate_about_custom_axis | axis                    | Rotate around a custom axis. The axis is specified as a vector of length 3.                                                                                        |
| -boundary_none            |                         | Remove the clipping.                                                                                                                                               |
| -boundary_box             |                         | Clip using a box shape.                                                                                                                                            |
| -boundary_sphere          |                         | Clip using a sphere (default).                                                                                                                                     |
| -font_size                | $s$                     | Set font size to $s$ .                                                                                                                                             |
| -stroke_width             | $s$                     | Set text depth to $s$ .                                                                                                                                            |
| -omit_bottom_plane        |                         | Remove the bottom plane.                                                                                                                                           |
| -W                        | $w$                     | Set output dimension to $w$ pixels wide.                                                                                                                           |
| -H                        | $h$                     | Set output dimension to $h$ pixels height.                                                                                                                         |
| -nb_frames                | $n$                     | Set number of frames to $n$ . One revolution around the axis is split into $n$ frames.                                                                             |
| -zoom                     | $r\ a_s\ a_t\ c_s\ c_t$ | Set zoom angle and clipping with in round $r$ to change from $a_s$ to $a_t$ and from $c_s$ to $c_t$ , respectively.                                                |
| -pan                      | $r\ F\ T\ C$            | In round $r$ , pan the camera from location $F$ to location $T$ in a rotational movement with center at $C$ . Each of $F, T, C$ are three dimensional coordinates. |
| -pan_reverse              | $r\ F\ T\ C$            | Same as -pan, but camera movement is in opposite order.                                                                                                            |
| -no_background            |                         | Remove background.                                                                                                                                                 |
| -no_bottom_plane          | $r$                     | Remove bottom plane in round $r$ .                                                                                                                                 |
| -camera                   | $r\ S\ C\ L$            | In round $r$ , set camera location at $C$ , sky at $S$ and pointing towards $L$ . Each of $S, C, L$ are three-dimensional coordinate vectors.                      |

Table 17.5: Povray Interface (Part 1)

| Povray Interface (Part 2) |                                                              |                                                                                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                   | Arguments                                                    | Purpose                                                                                                                                                                                                          |
| -clipping                 | $r\ c$                                                       | In round $r$ , set clipping radius to $c$ .                                                                                                                                                                      |
| -text                     | $r\ a\ \text{text}$                                          | In round $r$ , produce running text $\text{text}$ with sustain value $a$ .                                                                                                                                       |
| -label                    | $r\ s\ a\ g\ \text{text}$                                    | In round $r$ , produce running text $\text{text}$ with start value $s$ , sustain $s$ and gravity $g$ .                                                                                                           |
| -latex                    | $r\ s\ a\ \text{praeamble}\ g\ \text{text}\ l\ \text{fname}$ | In round $r$ , produce running latex text $\text{text}$ with start value $s$ , sustain $s$ and gravity $g$ . Put praeamble in the latex source code. Use $\text{fname}$ for the latex file names (no extension). |
| -global_picture_scale     | $d$                                                          | Set scaling factor to $d$ .                                                                                                                                                                                      |
| -picture                  | $r\ d\ \text{fname}\ \text{options}$                         | In round $r$ , place picture $\text{fname}$ scaled by $d$ using options.                                                                                                                                         |
| -picture                  | $r\ d\ \text{fname}\ \text{options}$                         | In round $r$ , place picture $\text{fname}$ scaled by $d$ using options.                                                                                                                                         |
| -look_at                  | $L$                                                          | Override camera look-at value to $L$ . $L$ is a three-dimensional vector.                                                                                                                                        |
| -default_angle            | $a$                                                          | Set default camera angle to $a$ .                                                                                                                                                                                |
| -clipping_radius          | $f$                                                          | Set default clipping radius to $f$ .                                                                                                                                                                             |
| -scale_factor             | $s$                                                          | Set default scale factor to $s$ .                                                                                                                                                                                |
| -line_radius              | $s$                                                          | Set default line radius to $s$ .                                                                                                                                                                                 |

Table 17.6: Povray Interface (Part 2)

```

▷ ▷ -video_options -W 1024 -H 768 \
▷ ▷ -global_picture_scale 0.5 \
▷ ▷ -default_angle 75 \
▷ ▷ -clipping_radius 2.7 \
▷ ▷ -end \
▷ ▷ -scene_objects \
▷ ▷ ▷ -obj_file cube_centered.obj \
▷ ▷ ▷ -edge "0, 1" \
▷ ▷ ▷ -edge "0, 2" \
▷ ▷ ▷ -edge "0, 4" \
▷ ▷ ▷ -edge "1, 3" \
▷ ▷ ▷ -edge "1, 5" \
▷ ▷ ▷ -edge "2, 3" \
▷ ▷ ▷ -edge "2, 6" \
▷ ▷ ▷ -edge "3, 7" \
▷ ▷ ▷ -edge "4, 5" \
▷ ▷ ▷ -edge "4, 6" \
▷ ▷ ▷ -edge "5, 7" \
▷ ▷ ▷ -edge "6, 7" \
▷ ▷ ▷ -group_of_things_as_interval 0 8 \
▷ ▷ ▷ -spheres 0 0.3 $(POLISHED_CHROME_WHITE) \
▷ ▷ ▷ -group_of_things_as_interval 0 6 \
▷ ▷ ▷ -prisms 1 0.05 $(YELLOW_TRANSPARENT) \

```

| Scene Elements (Part 1)                           |                   |                                                                                                                |
|---------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------|
| Command                                           | Arguments         | Purpose                                                                                                        |
| -cubic_lex                                        | coeffs            | Cubic surface given by 20 coefficients in lexicographic ordering                                               |
| -cubic_orbiter                                    | coeffs            | Cubic surface given by 20 coefficients in Orbiter ordering                                                     |
| -cubic_Goursat                                    | $A \ B \ C$       | Cubic surface with tetrahedral symmetry given by 3 Goursat coefficients as $Axyz + B(x^2 + y^2 + z^2) + C = 0$ |
| -quadric_lex_10                                   | coeffs            | Quadric surface given by 10 coefficients in lexicographic ordering                                             |
| -quartic_lex_35                                   | coeffs            | Quartic surface given by 35 coefficients in lexicographic ordering                                             |
| -octic_lex_165                                    | coeffs            | Octic surface given by 165 coefficients in lexicographic ordering                                              |
| -point                                            | coeffs            | Point given by three coordinates                                                                               |
| -point_list_from_csv_file                         | fname             | List of points with coordinates given in a csv file                                                            |
| -line_through_two_points_recentered_from_csv_file | fname             | List of lines through two points with point coordinates given in a csv file                                    |
| -line_through_two_points_from_csv_file            | fname             | List of lines through two points with point coordinates given in a csv file                                    |
| -point_as_intersection_of_two_lines               | $i_1 \ i_2$       | Create a point from the intersection of two lines $i_1$ and $i_2$                                              |
| -edge                                             | $i_1 \ i_2$       | Create an edge (line segment) between points $i_1$ and $i_2$                                                   |
| -text                                             | $i_1 \ s$         | Create a label $s$ located at the point $i$                                                                    |
| -triangular_face_given_by_three_lines             | $i_1 \ i_2 \ i_3$ | Create a triangular face give by three lines $i_1, i_2, i_3$                                                   |
| -face                                             | pts               | Create a face through the vertices pts, ordered cyclically                                                     |
| -quadric_through_three_skew_lines                 | $i_1 \ i_2 \ i_3$ | Create a quadric through three skew lines                                                                      |
| -plane_defined_by_three_points                    | $i_1 \ i_2 \ i_3$ | Create a plane through three noncollinear points                                                               |
| -line_through_two_points_recentered               | pt-coords         | Create a line through two points given by 6 coordinates, recentered                                            |

Table 17.7: Scene Elements (Part 1)



| Scene Elements (Part 2)                      |                     |                                                                                                                             |
|----------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Command                                      | Arguments           | Purpose                                                                                                                     |
| -line_through_two_points                     | pt-coords           | Create a line through two points given by 6 coordinates                                                                     |
| -line_through_two_existing_points            | $i_1 i_2$           | Create a line through two points                                                                                            |
| -line_through_point_with_direction           | $x y z u_x u_y u_z$ | Create a line through a point $(x, y, z)$ with a given direction $(u_x, u_y, u_z)$                                          |
| -plane_by_dual_coordinates                   | $a b c d$           | Create the plane $ax + by + cz + d = 0$ given in dual coordinates                                                           |
| -dodecahedron                                |                     | Create a Dodecahedron centered at the origin (20 points, 30 edges, 12 faces)                                                |
| -Hilbert_Cohn_Vossen_surface                 |                     | Create the Hilbert, Cohn-Vossen surface (1 cubic surface, 45 tritangent planes, 27 lines)                                   |
| -obj_file                                    | fname               | Read points and faces from the given .obj file                                                                              |
| -group_of_things                             | list                | Create a group of things from the given list                                                                                |
| -group_of_things_with_offset                 | list offset         | Create a group of things from the given list, each value is increase by offset                                              |
| -group_of_things_as_interval                 | $a b$               | Create a group of things indexed by the interval $a, \dots, a + b - 1$                                                      |
| -group_of_things_as_interval_with_exceptions | $a b \text{ ex}$    | Create a group of things indexed by the interval $a, \dots, a + b - 1$ with the exceptional elements in the list ex removed |
| -group_of_all_points                         |                     | Create a group of things from all points currently defined                                                                  |
| -group_of_all_faces                          |                     | Create a group of things from all faces currently defined                                                                   |
| -group_subset_at_random                      | $i f$               | Create a group of things from the existing group $i$ by picking a random subset with probability $f$                        |
| -create_regulus                              | $i N$               | Create a regulus for quadric $i$ with $N$ lines                                                                             |

Table 17.8: Scene Elements (Part 2)

| Scene Object Types |                  |                                                                                                                                   |
|--------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Command            | Arguments        | Purpose                                                                                                                           |
| -spheres           | $i$ $r$ prop     | For each element in point group $i$ , create a sphere of radius $r$ with given Povray properties.                                 |
| -cylinders         | $i$ $r$ prop     | For each element in edge group $i$ , create a cylinder of radius $r$ with given Povray properties.                                |
| -prisms            | $i$ $d$ prop     | For each element in face group $i$ , create a prism of half-thickness $d$ with given Povray properties.                           |
| -planes            | $i$ prop         | For each element in plane group $i$ , create a plane with given Povray properties.                                                |
| -lines             | $i$ $r$ prop     | For each element in line group $i$ , create a line of radius $r$ with given Povray properties.                                    |
| -cubics            | $i$ prop         | For each element in group $i$ of cubics, create a surface with given Povray properties.                                           |
| -quadrics          | $i$ prop         | For each element in group $i$ of quadrics, create a surface with given Povray properties.                                         |
| -quartics          | $i$ prop         | For each element in group $i$ of quartics, create a surface with given Povray properties.                                         |
| -octics            | $i$ prop         | For each element in group $i$ of octics, create a surface with given Povray properties.                                           |
| -texts             | $i$ $d$ $s$ prop | For each element in group $i$ of labels, create a text element with half-thickness $d$ and size $s$ with given Povray properties. |

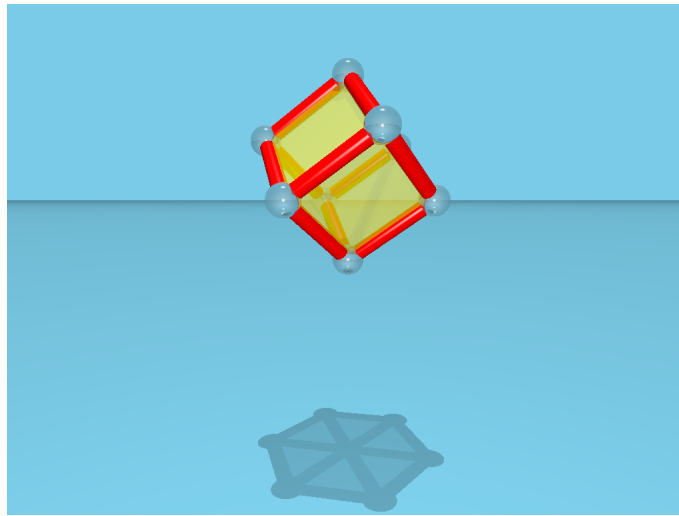
Table 17.9: Scene Object Types

```

> > > -group_of_things_as_interval 0 12 \
> > > -cylinders 2 0.15 $(COLOR_RED) \
> > -scene_objects_end \
> > -povray_end
> - rm -rf POV
> mkdir POV
> mv cube_0_*.pov POV
> mv makefile_animation POV

```

This command instructs Orbiter to create 30 povray files (extension .pov), one for each frame of a rotating scene. The scene contains a cube whose vertices are shown in chrome, whose edges are in red, and whose faces are yellow and transparent. The cube turns around a vertical axis of symmetry. Here is the first frame of the result:



The coordinates of the cube are stored in an object file `cube_centered.obj`. The content of this file is:

```

v -1 -1 -1
v 1 -1 -1
v -1 1 -1
v 1 1 -1
v -1 -1 1
v 1 -1 1
v -1 1 1
v 1 1 1
f 1 2 4 3
f 1 2 6 5
f 1 3 7 5
f 2 4 8 6
f 3 4 8 7
f 5 6 8 7

```

The monkey saddle is a cubic surface, given by the equation

$$z = x^3 - 3xy^2$$

The next example plots the surface known as the monkey saddle. The tangent plane at  $(0,0,0)$  is drawn as well. An animation is created by rotating the scene around the  $z$ -axis.

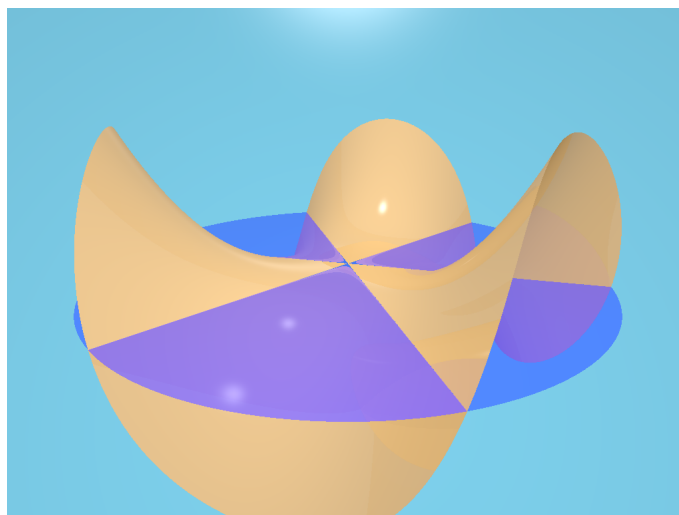
#### Example 671

```
MONKEY_SADDLE_CUBIC="1,0,0,0,-3,0,0,0,0,0,0,0,0,0,0,0,0,-1,0"
```

#### Example 672

```
monkey:
▷ $(ORBITER) -v 2 -povray \
▷ ▷ -round 0 -nb_frames_default 30 \
▷ ▷ -output_mask monkey_%d_%03d.pov \
▷ ▷ -video_options -W 1024 -H 768 \
▷ ▷ -global_picture_scale 0.8 \
▷ ▷ -default_angle 75 \
▷ ▷ -clipping_radius 0.8 \
▷ ▷ -camera 0 "0,0,1" "1,1,0.5" "0,0,0" \
▷ ▷ -rotate_about_z_axis \
▷ ▷ -end \
▷ ▷ -scene_objects \
▷ ▷ ▷ -cubic_lex $(MONKEY_SADDLE_CUBIC) \
▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -cubics 0 $(COLOR_GOLD) \
▷ ▷ ▷ -planes 1 $(COLOR_BLUE) \
▷ ▷ -scene_objects_end \
▷ ▷ -povray_end
▷ - rm -rf POV
▷ mkdir POV
▷ mv monkey_0_*.pov POV
▷ mv makefile_animation POV
```

Here is one of the frames that are created:



The Eckardt surface is given by the equation

$$\frac{5}{2}xyz - (x^2 + y^2 + z^2) + 1 = 0.$$

We use the following code to plot the surface and the lines on it. The Schläfli labeling of the lines is indicated.

#### Example 673

Eckardt:

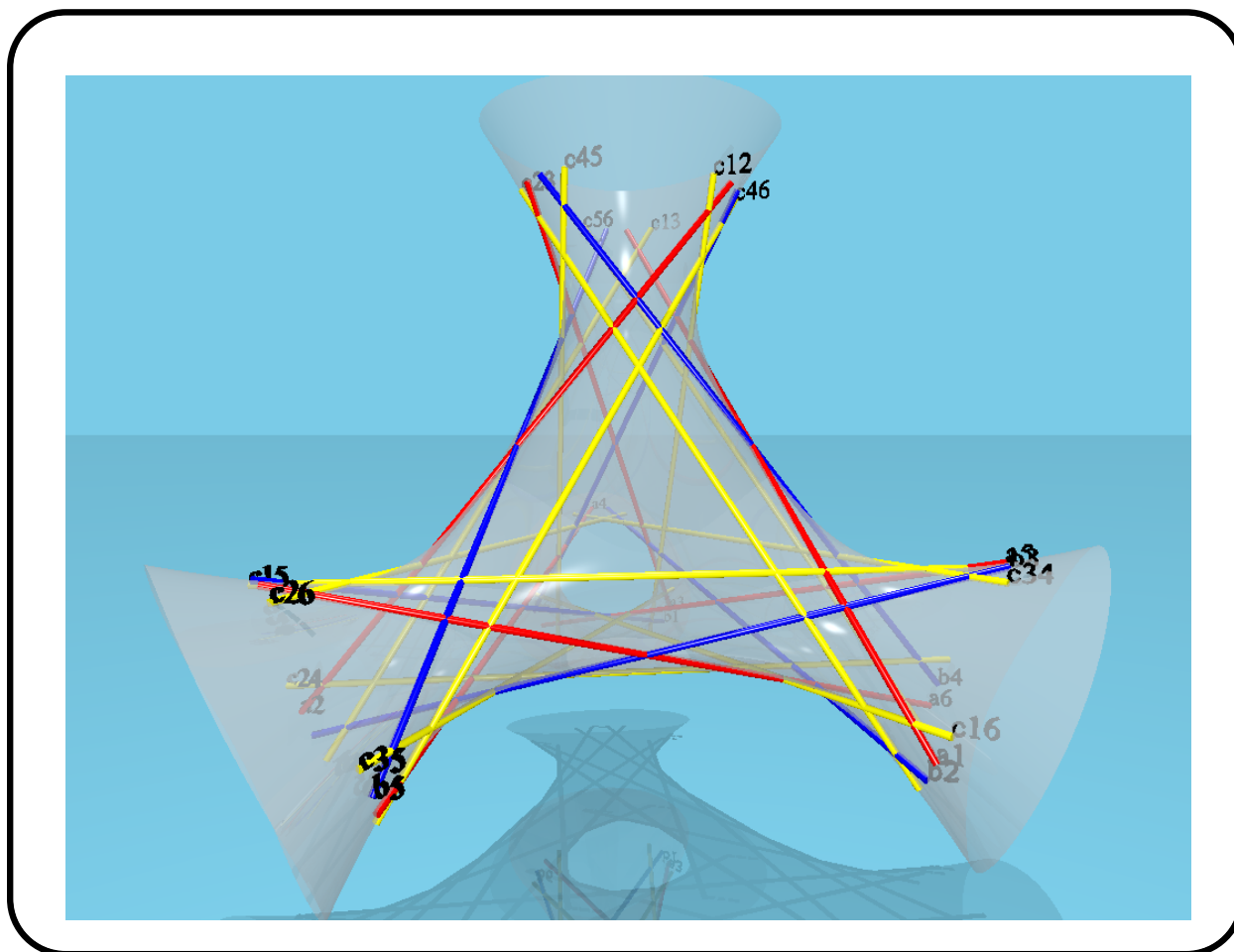
```

> $(ORBITER) -v 2 -povray \
> > -round 0 -nb_frames_default 30 \
> > -output_mask Eckardt_%d_%03d.pov \
> > -video_options -W 1024 -H 768 \
> > -global_picture_scale 0.9 \
> > -default_angle 75 \
> > -clipping_radius 2.4 \
> > -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
> > -end \
> > -scene_objects \
> > > -Hilbert.Cohn.Vossen.surface \
> > > -group_of_things "0" \
> > > -cubics 0 $(SURFACE_COLOR) \
> > > -group_of_things_as_interval 0 6 \
> > > -group_of_things_as_interval 6 6 \
> > > -group_of_things_as_interval_with_exceptions 12 15 \
> > > > "14,19,23" \
> > > -lines 1 0.02 $(COLOR.RED.SHINY) \
> > > -lines 2 0.02 $(COLOR.BLUE.SHINY) \
> > > -lines 3 0.02 $(COLOR.YELLOW.SHINY) \
> > > -label 0 "a1" \
> > > -label 2 "a2" \
> > > -label 4 "a3" \
> > > -label 6 "a4" \
> > > -label 8 "a5" \
> > > -label 10 "a6" \

```

```
▷ ▷ ▷ -label 12 "b1" \  
▷ ▷ ▷ -label 14 "b2" \  
▷ ▷ ▷ -label 16 "b3" \  
▷ ▷ ▷ -label 18 "b4" \  
▷ ▷ ▷ -label 20 "b5" \  
▷ ▷ ▷ -label 22 "b6" \  
▷ ▷ ▷ -label 24 "c12" \  
▷ ▷ ▷ -label 26 "c13" \  
▷ ▷ ▷ -label 30 "c15" \  
▷ ▷ ▷ -label 32 "c16" \  
▷ ▷ ▷ -label 34 "c23" \  
▷ ▷ ▷ -label 36 "c24" \  
▷ ▷ ▷ -label 40 "c26" \  
▷ ▷ ▷ -label 42 "c34" \  
▷ ▷ ▷ -label 44 "c35" \  
▷ ▷ ▷ -label 48 "c45" \  
▷ ▷ ▷ -label 50 "c46" \  
▷ ▷ ▷ -label 52 "c56" \  
▷ ▷ ▷ -group_of_things.as_interval 0 6 \  
▷ ▷ ▷ -texts 4 0.2 0.15 $(COLOR.BLACK.NO.SHADOW) \  
▷ ▷ ▷ -group_of_things.as_interval 6 6 \  
▷ ▷ ▷ -texts 5 0.2 0.15 $(COLOR.BLACK.NO.SHADOW) \  
▷ ▷ ▷ -group_of_things.as_interval 12 12 \  
▷ ▷ ▷ -texts 6 0.2 0.15 $(COLOR.BLACK.NO.SHADOW) \  
▷ ▷ -scene_objects.end \  
▷ ▷ -povray_end  
▷ - rm -rf POV  
▷ mkdir POV  
▷ mv Eckardt_0_*.pov POV  
▷ mv makefile_animation POV
```

The rendering yields:



The Endrass octic [26] is the algebraic surface given by the equation

$$X8 := 64 \left( -w^2 + x^2 \right) \left( -w^2 + y^2 \right) \left( (x+y)^2 - 2w^2 \right) \left( (x-y)^2 - 2w^2 \right) - \left( -4 \left( 1 + \sqrt{2} \right) \left( x^2 + y^2 \right)^2 + \left( 8 \left( 2 + \sqrt{2} \right) z^2 + 2 \left( 2 + 7\sqrt{2} \right) w^2 \right) \left( x^2 + y^2 \right) - 16z^4 + 8 \left( 1 - 2\sqrt{2} \right) z^2 w^2 - \left( 1 + 12\sqrt{2} \right) w^4 \right)^2$$

The following Orbiter command creates a povray graphics of the otic.

### Example 674

```
ENDRASS_OCTIC_LEX_165="-93.2548,0,0,0,-309.019,0,0,527.529,0,395.647,\
0,0,0,0,0,0,0,0,0,-687.529,0,0,1582.59,0,1186.94,0,0,0,0,-1055.06,0,\
-1582.59,0,-593.47,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-309.019,\
0,0,1582.59,0,1186.94,0,0,0,0,-2110.12,0,-3165.17,0,-1186.94,0,0,0,0,0,\
0,874.039,0,1560.63,0,1677.92,0,343.362,0,0,0,0,0,0,0,0,0,0,0,0,0,\
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-93.2548,0,0,527.529,0,395.647,\
0,0,0,0,-1055.06,0,-1582.59,0,-593.47,0,0,0,0,0,0,874.039,0,1560.63,0,\
1677.92,0,343.362,0,0,0,0,0,0,0,0,0,-256,0,-468.077,0,-789.019,0,\
-525.726,0,0.941125"
```

**Example 675**

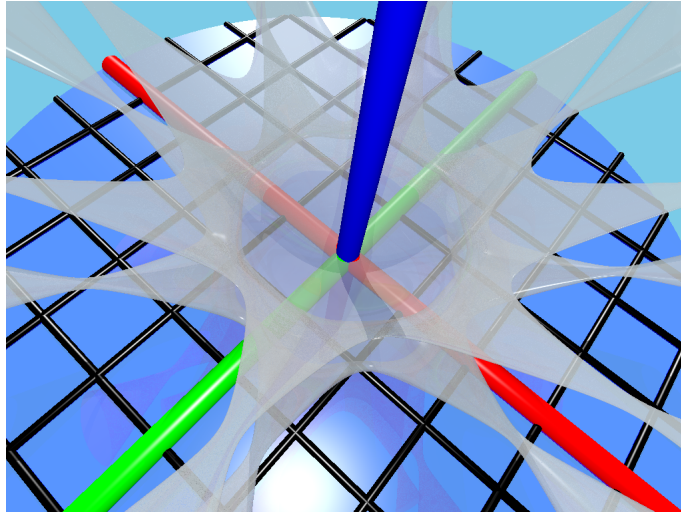
```

endrass8:
▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
▷ $(ORBITER) -v 2 -povray \
▷ ▷ -round 0 -nb_frames_default 30 \
▷ ▷ -output_mask endrass_octic.%d.%03d.pov \
▷ ▷ -video_options -W 1024 -H 768 \
▷ ▷ -global_picture_scale 0.75 \
▷ ▷ -default_angle 75 \
▷ ▷ -clipping_radius 3.7 \
▷ ▷ -no_bottom_plane \
▷ ▷ -camera 0 "1,1,1" "6,6,3" "0,0,0" \
▷ ▷ -rotate_about_111 \
▷ ▷ -end \
▷ ▷ -scene_objects \
▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
▷ ▷ ▷ ▷ coordinate_grid.csv \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -group_of_things "1" \
▷ ▷ ▷ -group_of_things "2" \
▷ ▷ ▷ -group_of_things_as_interval 3 39 \
▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
▷ ▷ ▷ -octic_lex_165 $(ENDRASS_OCTIC_LEX_165) \
▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -octics 4 $(SURFACE_COLOR_SEETHROUGH) \
▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue transmit 0.5 } \
finish { diffuse 0.9 phong 1 }}" \
▷ ▷ -scene_objects_end \
▷ ▷ -povray_end
▷ - rm -rf POV
▷ mkdir POV
▷ mv endrass_octic_0_*.pov POV
▷ mv makefile_animation POV

```

Here is the rendering:





It includes coordinate axes and the  $x, y$ -plane.

| Preparing Frames for an Animation |                       |                                                                                                                                                                                                             |
|-----------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                           | Arguments             | Purpose                                                                                                                                                                                                     |
| <code>-i</code>                   | <code>s l mask</code> | Specify the input file names by running a <code>printf</code> command with the given mask applied to the index $i$ where $i$ loops over all integers from $s$ to $s + l - 1$ . This option can be repeated. |
| <code>-step</code>                | <code>s</code>        | Increment the index in steps of size $s$ .                                                                                                                                                                  |
| <code>-o</code>                   | <code>mask</code>     | Create the output file using the given mask.                                                                                                                                                                |
| <code>-output_starts_at</code>    | <code>i</code>        | Start output file indices at $i$ (default is 0).                                                                                                                                                            |

Table 17.10: Preparing Frames for an Animation

### 17.3 Creating Animations

Orbiter can be used to create animations. This relies on the software `ffmpeg`. In a first step, all frames (i.e. individual graphics files) are created using Orbiter's `povray` interface. After that, the frames are used to create the animation. In order to use `ffmpeg`, the frames should have a uniform file naming scheme, using a consecutive numbering to arrange the files in order. This is achieved by using a `printf` style mask, with `%d` representing the number of the current frame. In order to do so, Orbiter can be used to copy and rename files. A temporary directory can be used to collect the files. The Orbiter command `prepare_frames` can be used. For a list of commands, see Tables 17.10. For instance, the command

#### Example 676

```

monkey_video:
▷ - rm -r FRAMES
▷ - mkdir FRAMES
▷ - rm monkey.mp4
▷ $(ORBITER) \
▷ ▷ -prepare_frames \
▷ ▷ ▷ -i 0 30 monkey_0_%03d.png \
▷ ▷ ▷ -output_starts_at 0 \
▷ ▷ ▷ -o FRAMES/frame%04d.png \
▷ ▷ -end
▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 monkey.mp4

```

creates a video `monkey.mp4` from a set of 30 files. The individual filenames are created using the `printf` format string `monkey_0_%03d.png`, with an integer index that is drawn from the interval  $[0, 29]$ . The part that starts with a percent sign and ends with a “d” character defines the way in which the integer is formatted. The number three before the “d” indicates that three characters will be printed. The zero indicates the use of leading zeros. So, the first file would be `monkey_0_000.png` and the very last file is `monkey_0_029.png`. The description of the `printf` format string can be found in the documentation of the C standard library [40].

## 17.4 Continuous Function Plotter

Orbiter can plot functions using a built-in function tracker. The functions must be continuous apart from a finite number of poles. The function can have multiple components, each described using an expression. Each expression is specified in Reverse Polish Notation (RPN). Consider an example. A Lissajous curve is defined using coordinate functions of the form

$$x = r \sin(at + c), \quad y = r \sin(bt), \quad a, b, c, r \in \mathbb{R}.$$

The terms

$$r \sin(at + c), \quad r \sin(bt)$$

are the expressions of the two coordinate functions. RPN means that the operator is listed after the operands. A stack data structure is used to hold temporary values. Operators are pushed to the top of the stack using the push commands. A binary operator pops the two elements from the stack, performs the operation, and pushes the resulting value back onto the stack. For a unary operator, only one element is popped and replaced by the result. Here are some examples of expressions rewritten in RPN:

$$\begin{aligned} \sin(x) &\mapsto \text{push } x \text{ sin,} \\ a + b &\mapsto \text{push } a \text{ push } b \text{ add,} \\ a \cdot b &\mapsto \text{push } a \text{ push } b \text{ mult.} \end{aligned}$$

The coordinate functions are enclosed between `-code` and `-code_end` commands. Each coordinate function is described in RPN and terminated using a `return` keyword. By the time the `return` keyword is reached, the RPN expression must have exactly one value on the stack which is considered the value of the expression. Constants are declared between the `-const` and `-const_end` keywords. Likewise, variables are declared between the `-var` and `-var_end` keywords. Picking  $a = 3$ ,  $b = 2$ ,  $c = \pi/2$  and  $r = 7$ , the function is computed using

### Example 677

```
lissajous:
▷ $(ORBITER) -v 2 \
▷ ▷ -smooth_curve "lissajous" 0.07 2000 15 0 18.85 \
▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
▷ ▷ -var t -var_end \
▷ ▷ -code \
▷ ▷ ▷ push t push a mult push c add sin push r mult return \
▷ ▷ ▷ push t push b mult sin push r mult return \
▷ ▷ -code_end \
```

The sequence

```
push t push a mult push c add sin push r mult
```

is  $r \sin(at + c)$  expressed in RPN. The constants are defined in the line

```
-const a 3 b 2 c 1.57 r 7 -const_end
```

The input variable is defined using the line

```
-var t -var_end
```

The sequence

```
-smooth_curve "lissajous" 0.07 2000 15 0 18.85
```

defines the name of the output file, the fact that two consecutive points are never further than  $\epsilon = 0.07$  away, the fact that points that are 15 or more away from the origin should be ignored, and the fact that the variable  $t$  loops over the range  $[0, 18.85]$  with a default of 2000 steps. The evaluator automatically reduces the step-size if consecutive image points are more than  $\epsilon$  apart. The code to produce the plot is

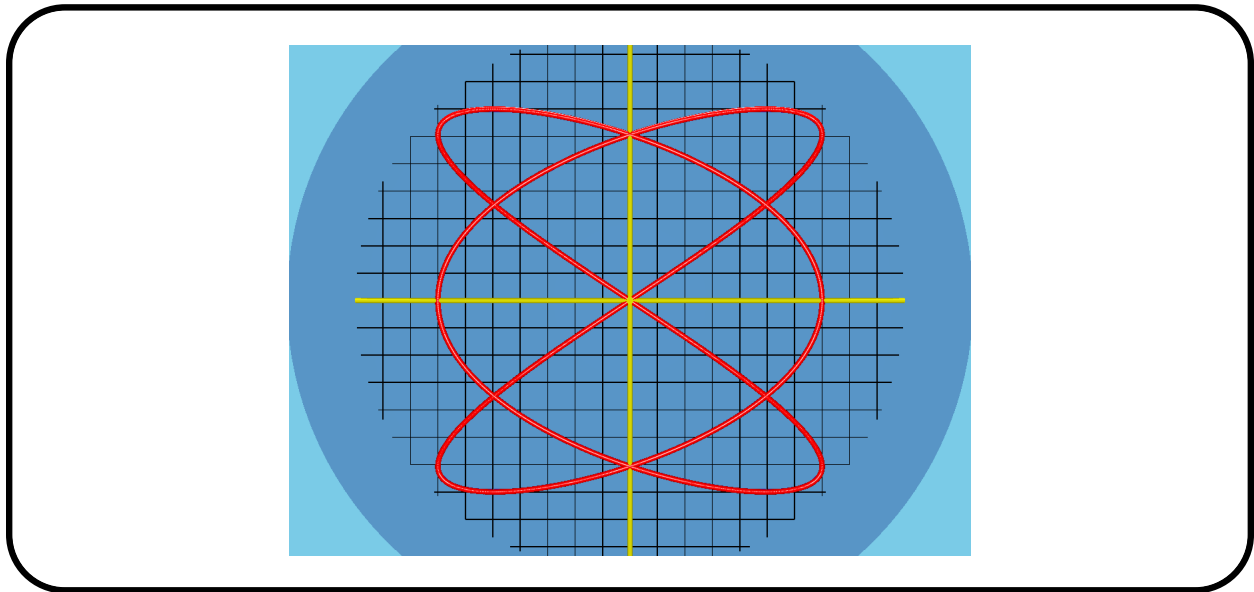
**Example 678**

```

lissajous_plot:
▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
▷ $(ORBITER) -v 2 -povray \
▷ ▷ -round 0 -nb_frames_default 1 \
▷ ▷ -output_mask lissajous.%d.%03d.pov \
▷ ▷ -video_options -W 1024 -H 768 \
▷ ▷ -global_picture_scale 0.40 \
▷ ▷ -default_angle 45 \
▷ ▷ -clipping_radius 5 \
▷ ▷ -omit_bottom_plane \
▷ ▷ -camera 0 "0,-1,0" "0,0,12" "0,0,0" \
▷ ▷ -rotate_about_z_axis \
▷ ▷ -end \
▷ ▷ -scene_objects \
▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
▷ ▷ ▷ coordinate_grid.csv \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -group_of_things "1" \
▷ ▷ ▷ -group_of_things "2" \
▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
▷ ▷ ▷ -group_of_things_as_interval 3 39 \
▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
▷ ▷ ▷ -point_list_from_csv_file \
▷ ▷ ▷ function_lissajous.N2000.points.csv \
▷ ▷ ▷ -group_of_things_as_interval 0 6524\
▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
finish { diffuse 0.9 phong 1}}" \
▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
▷ ▷ ▷ -group_of_things "0" \
▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue*0.5 \
transmit 0.5 } }" \
▷ ▷ -scene_objects_end \
▷ ▷ -povray_end
▷ - rm -rf POV
▷ mkdir POV
▷ mv lissajous_0_*.pov POV
▷ mv makefile_animation POV

```

The plot is shown below



We can turn it into a 3D plot by using the  $t$  value for the  $z$  coordinate. The function is computed using the command

#### Example 679

```
lissajous_3d:
▷ $(ORBITER) -v 2 \
▷ ▷ -smooth_curve "lissajous_3d" 0.07 2000 50 0 18.85 \
▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
▷ ▷ -var t -var_end \
▷ ▷ -code \
▷ ▷ ▷ push t push a mult push c add sin push r mult return \
▷ ▷ ▷ push t push b mult sin push r mult return \
▷ ▷ ▷ push t return \
▷ ▷ -code_end \
```

The code to produce the 3D plot is

#### Example 680

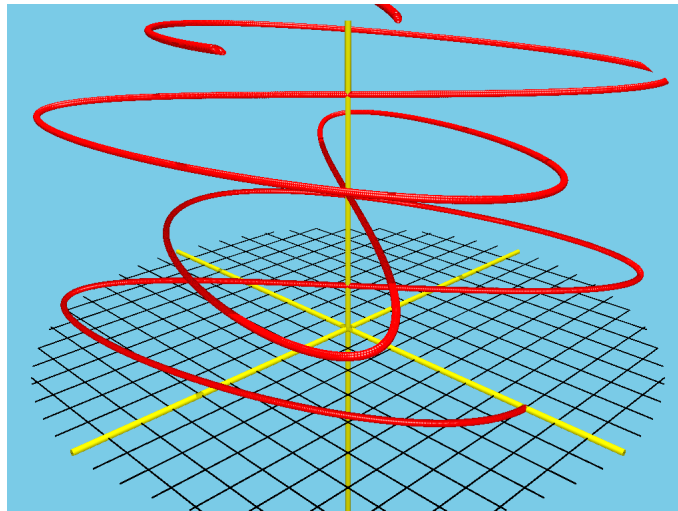
```
lissajous_3d_plot:
▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
▷ $(ORBITER) -v 2 -povray \
▷ ▷ -round 0 -nb_frames_default 30 \
▷ ▷ -output_mask lissajous_3d_%d%03d.pov \
▷ ▷ -video_options -W 1024 -H 768 \
▷ ▷ -global_picture_scale 0.40 \
▷ ▷ -default_angle 45 \
▷ ▷ -clipping_radius 5 \
▷ ▷ -omit_bottom_plane \
▷ ▷ -camera 0 "0,0,1" "7,7,5" "0,0,1" \
▷ ▷ -rotate_about_z_axis \
```

```

> > -end \
> > -scene_objects \
> > > -line_through_two_points_recentered_from_csv_file \
> > > coordinate_grid.csv \
> > > -group_of_things "0" \
> > > -group_of_things "1" \
> > > -group_of_things "2" \
> > > -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
> > > -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
> > > -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
> > > -group_of_things_as_interval 3 39 \
> > > -lines 3 0.02 "texture{ pigment{ color Black } }" \
> > > -point_list_from_csv_file \
> > > function_lissajous_3d_N2000_points.csv \
> > > -group_of_things_as_interval 0 6538\
> > > -spheres 4 0.1 "texture{ pigment{ color Red } }" \
finish { diffuse 0.9 phong 1}}" \
> > > -plane_by_dual_coordinates "0,0,1,0" \
> > > -group_of_things "0" \
> > -scene_objects_end \
> > -povray_end
> - rm -rf POV
> mkdir POV
> mv lissajous_3d_0_*.pov POV
> mv makefile_animation POV

```

The 3D curve is shown below



## 17.5 The Gnuplot Interface

Orbiter can interface with gnuplot [30] to draw diagrams. Here is an example. Suppose we have data in a csv file like this:

```
Row, Curve-1, Curve-2
1, 2, 4
2, 6, 8
3, 9, 10
4, 13, 17
5, 17, 20
6, 19, 25
7, 23, 30
END
```

The following command creates this csv file from a makefile variable

### Example 681

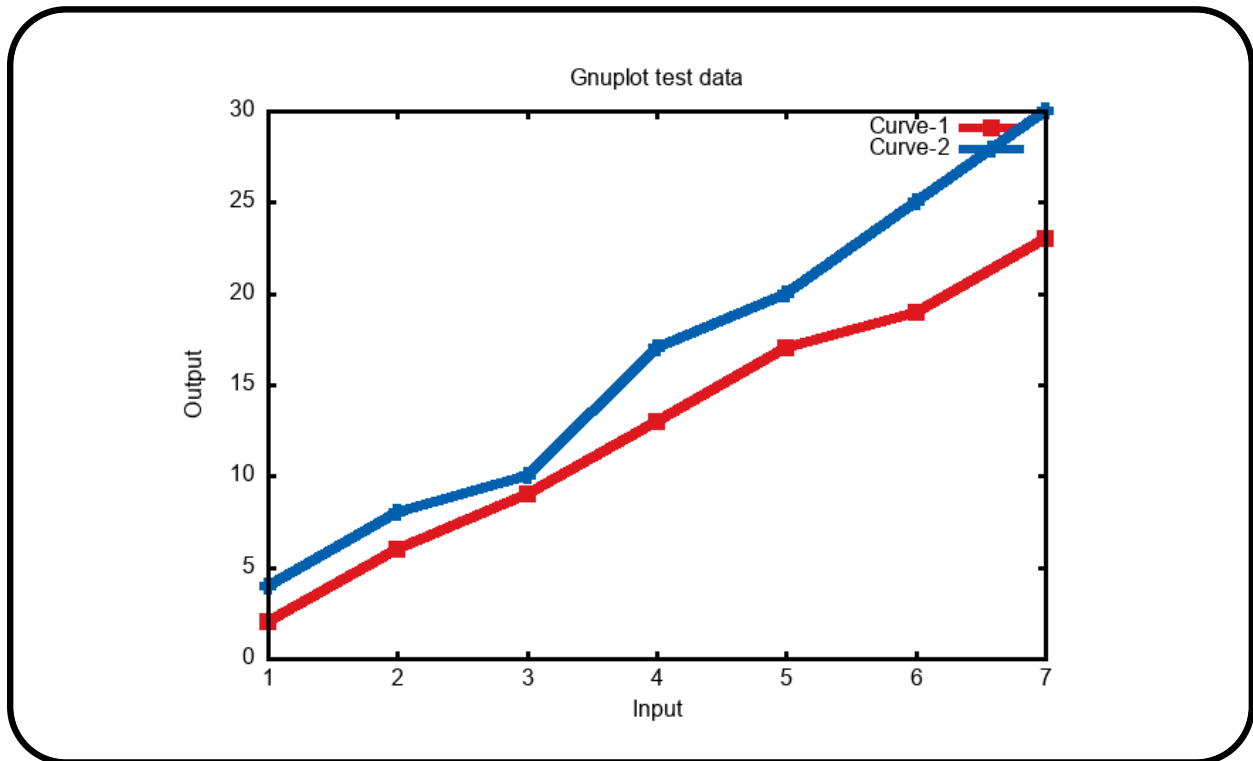
```
GNUPLOT_TEST_INPUT="Row, Curve-1, Curve-2\n1, 2, 4\n2, 6, 8\n3, 9, 10\n4, 13, 17\n5, 17, 20\n6, 19, 25\n7, 23, 30\nEND\n"
```

The next command invokes the Orbiter gnuplot interface:

### Example 682

```
gnuplot_test_data:
▷ echo $(GNUPLOT_TEST_INPUT) >gnuplot_test_data.csv
▷ $(ORBITER) -v 3 \
▷ ▷ -gnuplot gnuplot_test_data.csv \
▷ ▷ "Gnuplot test data" \
▷ ▷ "Input" \
▷ ▷ "Output"
▷ $(OPEN) gnuplot_test_data_gnuplot.png
```

The resulting plot created by gnuplot is:



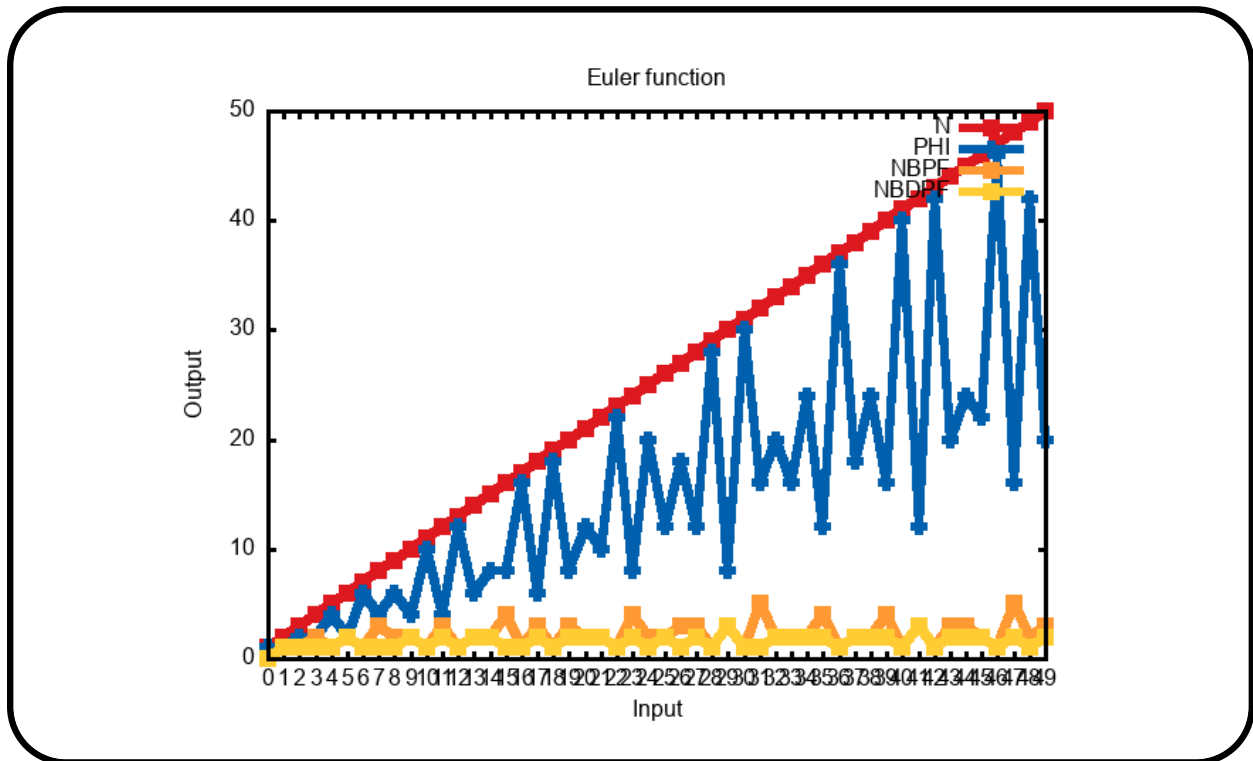
The next command plots the Eulerfunction, the number of prime divisors and the number of distinct prime divisors of all integers  $n$  between 1 and 50:

#### Example 683

```
gnuplot_Eulerfunction_50:
> $(ORBITER) -v 1 -eulerfunction_interval 1 50
> $(ORBITER) -v 3 \
> > -gnuplot table_eulerfunction_1_50.csv \
> > "Euler function" \
> > "Input" \
> > "Output"
```

The resulting plot is:







## Chapter 18

# Mathematical Data in Orbiter

### 18.1 Cubic Surfaces

Orbiter contains a knowledge base of mathematical data. This is like a database, except for the way that the data is stored. The data does not reside in files, like in a database. Rather, the data is compiled into the Orbiter executable. This means that the data is immediately available, without any delay. We will now discuss what data exists in Orbiter and how to access it.

In most cases, the data that Orbiter stores includes the automorphism group of the objects. Individual objects can be accessed by creating them from the catalogue. Other functions exist that can write reports about all objects of a certain type.

The command

#### Example 684

```
make_table_of_surfaces:  
▷ $(ORBITER) -v 3 \  
▷ ▷ -make_table_of_surfaces  
▷ pdflatex surfaces_report.tex  
▷ $(OPEN) surfaces_report.pdf
```

creates a table of all cubic surfaces contained in the Orbiter knowledge base. The output is shown in Appendix A.1.

The command

#### Example 685

```
make_table_of_quartic_curves:  
▷ $(ORBITER) -v 3 \  
▷ ▷ -make_table_of_quartic_curves  
▷ pdflatex quartic_curves_report.tex  
▷ $(OPEN) quartic_curves_report.pdf
```

creates a table of all quartic curves contained in the Orbiter knowledge base.

It is possible to export data on cubic surfaces and quartic curves. Two file types are supported: csv and sql. For instance, the command

**Example 686**

```

cubic_surfaces_table.q17:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 17 -end \
> > -define P -projective_space \
> > > -n 3 -field F -v 0 \
> > -end \
> > -with P -do \
> > > -projective_space_activity \
> > > > -table_of_cubic_surfaces \
> > -end

```

exports the cubic surfaces over the field  $\mathbb{F}_{17}$  contained in the Orbiter knowledge base. The files `table_of_cubic_surfaces_q17_info.csv` and `table_of_cubic_surfaces_q17_data.sql` are created. For each isomorphism class of cubic surfaces, one line is generated in the file. The command

**Example 687**

```

quartic_curves_table.q17:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 17 -end \
> > -define P -projective_space \
> > > -n 2 -field F -v 0 \
> > -end \
> > -with P -do \
> > > -projective_space_activity \
> > > > -table_of_quartic_curves \
> > -end

```

exports all quartic curves over the field  $\mathbb{F}_{17}$

## 18.2 BLT-Sets

Orbiter contains a library of BLT-sets. It is possible to export data on BLT-sets to csv-files. For instance, the command

### Example 688

```
blt_set_table_q13:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 13 -end \
> > -define O -orthogonal_space 0 5 F -end \
> > -with O -do \
> > > -orthogonal_space_activity \
> > > > -table_of_blt_sets \
> > -end
```

exports the BLT-sets over the field  $\mathbb{F}_{13}$  contained in the Orbiter knowledge base.

The command

### Example 689

```
blt_set_table:
> $(ORBITER) -v 30 \
> > -define Q -vector -dense "3,5,7,9,11,13,17,19,23,25,27,29,31" -end \
> > -define NB -vector -dense "1,2,2,3,4,3,6,5,9,6,6,9,8" -end \
> > -loop_over i Q \
> > > -define F%i -finite_field -q "%i[Q]" -end \
> > > -define O%i -orthogonal_space 0 5 F%i -end \
> > > -with O%i -do \
> > > > -orthogonal_space_activity \
> > > > > -table_of_blt_sets \
> > > > -end \
> > > -end \
> > -end_loop_over i \
> > -print_symbols
```

creates tables of BLT-sets for all orders up to 31.

The command

### Example 690

```
BLT_ORDER_Q="3,5,7,9,11,13,17,19,23,25,27,\
29,31,37,41,43,47,49,53,59,61,67,71,73"
```

### Example 691

```
BLT_NUMBER_ISO="1,2,2,3,4,3,6,5,9,6,6,\
9,8,7,10,6,10,8,8,9,5,6,8,5"
```

**Example 692**

```

blt_set_export_gap:
▷ $(ORBITER) -v 30 \
▷ ▷ -define Q -vector -dense \
▷ ▷ ▷ $(BLT_ORDER_Q) \
▷ ▷ -end \
▷ ▷ -define NB -vector -dense \
▷ ▷ ▷ $(BLT_NUMBER_ISO) \
▷ ▷ -end \
▷ ▷ -loop_over i Q \
▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -end \
▷ ▷ ▷ -loop j 0 %i[NB] 1 \
▷ ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
▷ ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
▷ ▷ ▷ ▷ -end \
▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set_activity \
▷ ▷ ▷ ▷ ▷ -export_gap \
▷ ▷ ▷ ▷ -end \
▷ ▷ ▷ -end_loop j \
▷ ▷ -end_loop_over i \
▷ ▷ -print_symbols

```

exports all BLT-sets of order at most 73 to GAP.

## Chapter 19

# Miscellaneous

### 19.1 Miscellaneous

Tables 19.1-19.2 list miscellaneous Orbiter commands.

The command `-csv_file_select_rows` can be used to select rows from a csv file. The command `-csv_file_select_cols` can be used to select columns from a csv file. The command `-csv_file_select_rows_and_cols` selects rows and columns. Here is an example. We create the multiplication table of the finite field  $\mathbb{F}_7$ , ordered according to the powers of a primitive element:

$$\alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5.$$

The even powers of  $\alpha$  create a multiplicative subgroup. We select the rows and columns corresponding to even powers  $\alpha^0, \alpha^2, \alpha^4$ .

#### Example 693

```
misc_select:
> $(ORBITER) -v 3 \
> > -define F -finite_field -q 7 -end \
> > -with F -do -finite_field_activity -cheat_sheet_GF -end
> $(ORBITER) -v 4 -csv_file_select_rows_and_cols \
> > GF_q7_multiplication_table_reordered.csv \
> > "0,2,4" "0,2,4"
```

Here is the file `GF_q7_multiplication_table_reordered.csv`

```
Row,C0,C1,C2,C3,C4,C5
0,1,3,2,6,4,5
1,3,2,6,4,5,1
2,2,6,4,5,1,3
3,6,4,5,1,3,2
4,4,5,1,3,2,6
5,5,1,3,2,6,4
END
```

and next the file that is created by selecting rows and columns 0, 2, 4:

| Miscellaneous Orbiter Commands (Part 1) |                            |                                                                                                                                   |
|-----------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Command                                 | Arguments                  | Purpose                                                                                                                           |
| -create_files                           | descr                      | Create text files.                                                                                                                |
| -save_matrix_csv                        | label                      | Save matrix to a csv file.                                                                                                        |
| -csv_file_tally                         | fname                      | Tally a csv file.                                                                                                                 |
| -csv_file_select_rows                   | fname $R$                  | Selects rows listed in $R$ from the csv-file fname.                                                                               |
| -csv_file_split_rows_modulo             | fname $n$                  | Splits the rows from the csv-file fname modulo $n$ .                                                                              |
| -csv_file_select_cols                   | fname $R$                  | Selects columns listed in $R$ from the csv-file fname.                                                                            |
| -csv_file_select_rows_and_cols          | fname $R$ $C$              | Selects rows listed in $R$ and columns listed in $C$ from the csv-file fname.                                                     |
| -csv_file_sort_each_row                 | fname                      | Sorts each individual row in the given csv file, thinking of the entries as a set.                                                |
| -csv_file_sort_rows                     | fname                      | Sorts the rows in the given csv file.                                                                                             |
| -csv_file_join                          | fname col-label            | Joins csv file fname according to column with label col-label. This option is given once for each file that should be joined.     |
| -csv_file_concatenate                   | fname-out f1 f2 ...        | Join csv files f1, f2, ... The output is written to fname-out.                                                                    |
| -csv_file_concatenate_from_mask         | $N$ fname-mask fname-out   | Join csv files. The input files arise from fname-mask with all values $i = 0, \dots, N - 1$ . The output is written to fname-out. |
| -csv_file_extract_column_to_txt         | fname col-label            |                                                                                                                                   |
| -csv_file_latex                         | header fname               | Produces a latex table from the given csv-file. The binary variable header determines whether a header is produced.               |
| -draw_matrix                            | descr                      | Produce a bitmap graphic, see Table 16.2.                                                                                         |
| -reformat                               | fname-in fname-out nb-cols | Reformat the data from fname-in in nb-cols columns. Write the output to fname-out.                                                |
| -split_by_values                        | fname-in                   | Split the file fname-in by values. For each value, a separate file is written.                                                    |

Table 19.1: Miscellaneous Orbiter Commands (Part 1)



| Miscellaneous Orbiter Commands (Part 2) |                                            |                                                                                                                                                                                                                                              |
|-----------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command                                 | Arguments                                  | Purpose                                                                                                                                                                                                                                      |
| -change_values                          | fname-in<br>fname-out<br>map-in map-out    | Change the values of the file in fname-in according to a mapping. The output is written to fname-out. The mapping is defined by two corresponding sequences. The input values are in map-in, the corresponding output values are in map-out. |
| -store_as_csv_file                      | fname $m$ $n$ $L$                          | Stores the data in $L$ to a csv file. The data is an $m \times n$ matrix in row-major ordering.                                                                                                                                              |
| -mv                                     | fname-in<br>fname-out                      | Rename file from fname-in to fname-out.                                                                                                                                                                                                      |
| -system                                 | command                                    | Execute the given system command.                                                                                                                                                                                                            |
| -loop                                   | variable<br>val_from<br>val_to<br>val_step | Create a loop with the given loop variable, whose values range from loop_from to val_to minus one in steps of val_step.                                                                                                                      |
| -loop_over                              | variable<br>domain                         | Create a loop with the given loop variable, whose values range over the given domain.                                                                                                                                                        |
| -plot_function                          | function-name                              |                                                                                                                                                                                                                                              |
| -draw_projective_curve                  | options                                    |                                                                                                                                                                                                                                              |
| -tree_draw                              | options                                    | Draw a tree from a description. For options, see Table 19.3.                                                                                                                                                                                 |
| -extract_from_file                      | fname-in key<br>fname-out                  | Extract lines from a text file, starting with the given key and continuing until the next empty row.                                                                                                                                         |
| -extract_from_file_with_tail            | fname-in key tail<br>fname-out             | Extract lines from a text file, starting with the given key and continuing until the next empty row.                                                                                                                                         |
| -serialize_file_names                   | fname<br>output-mask                       |                                                                                                                                                                                                                                              |
| -save_4_bit_data_file                   | fname vector-<br>data                      |                                                                                                                                                                                                                                              |
| -gnuplot                                | fname title label_x label_y                | Produce an diagram of data series using gnuplot [30].                                                                                                                                                                                        |
| -compare_columns                        | fname col1<br>col2                         |                                                                                                                                                                                                                                              |
| -gcd_worksheet                          | nb_problems<br>N key                       |                                                                                                                                                                                                                                              |
| -draw_layered_graph                     | options                                    |                                                                                                                                                                                                                                              |

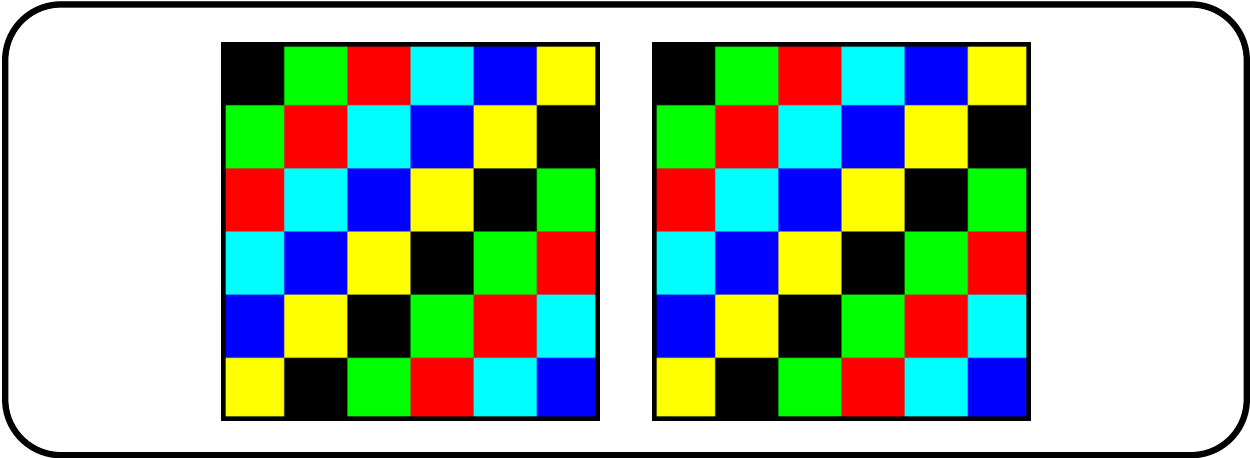
Table 19.2: Miscellaneous Orbiter Commands (Part 2)

| Options for Drawing Trees |           |                      |
|---------------------------|-----------|----------------------|
| Command                   | Arguments | Purpose              |
| -file                     | fname     | File in .tree format |
| -restrict                 | color     |                      |
| -select_path              | path      |                      |

Table 19.3: Options for Drawing Trees

```
Row, "C0", "C2", "C4"  
0, "1", "2", "4"  
1, "2", "4", "1"  
2, "4", "1", "2"  
END
```

A graphical output is shown below:



## 19.2 Limitations

Several limitations exist in Orbiter. Here is a list:

1. Field elements are encoded as `int`. This limits the size of fields that can be handled to  $2^{8s-1}$  where  $s = \text{sizeof}(\text{int})$ .
2. The ranks of elements in the permutation domain are encoded as `long int`. This limits the size of permutation domains that can be handled. The degree of a permutation group must be less than  $2^{8s-1}$  where  $s = \text{sizeof}(\text{long int})$ .
3. The finite field class builds tables for the addition and multiplication of field elements. This restricts the size of the fields that can be created.
4. The projective geometry class tries to build a bitmatrix for the adjacency matrix if the number of lines is less than `MAX_NUMBER_OF_LINES_FOR_INCIDENCE_MATRIX` which is defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of lines is too big, the table is not created. In this case, the projective geometry class may behave slower.
5. The projective geometry class tries to build a table for the lines if the number of points is less than `MAX_NUMBER_OF_POINTS_FOR_POINT_TABLE` and the number of lines is less than `MAX_NUMBER_OF_LINES_FOR_LINE_TABLE`, both of which are defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of points is too big, the table is not created. In this case, the projective geometry class may behave slow.
6. The projective geometry class tries to build a table for the lines through any two points if the number of points is less than `MAX_NB_POINTS_FOR_LINE_THROUGH_TWO_POINTS_TABLE` which is defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of points is too big, the table is not created. In this case, the projective geometry class may behave slow.
7. The projective geometry class tries to build a table for the intersection points of pairs of lines if the number of points is less than `MAX_NB_POINTS_FOR_LINE_INTERSECTION_TABLE` which is defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of points or lines is too big, the table is not created. In this case, the projective geometry class may behave slow.
8. For Windows users: Cygwin by default uses 32 bit integers for both `int` and `long int`. Using Cygwin 64 to compile Orbiter recommended.
9. A limited list of primitive polynomials are hard-coded in Orbiter. For large fields, the user must provide their own primitive polynomial. The polynomials encoded in orbiter are not guaranteed to be compatible with the subfield relationship.

The command

### Example 694

```
F_15bit:
▷ $(ORBITER) -v 10 \
▷ ▷ -define F -finite_field -q 32749 -without_tables -end \
▷ ▷ -define v -vector -field F -allow_negatives \
▷ ▷ ▷ -dense "-1,-1" -end \
▷ ▷ -with F -do -finite_field_activity -product_of v -end
▷
```

creates the field  $\mathbb{F}_{32749}$ . Tables are suppressed for speed. Note that

$$2^{15} - 19 = 32749$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). The command creates two instances of  $-1$  in the field and multiplies them.

The command

#### Example 695

```
F_31bit:
▷ $(ORBITER) -v 10 \
▷ ▷ -define F -finite_field -q 2147483647 -without_tables -end \
▷ ▷ -define v -vector -field F -allow_negatives \
▷ ▷ ▷ -dense "-1,-1" -end \
▷ ▷ -with F -do -finite_field_activity -product_of v -end
```

creates the field  $\mathbb{F}_{2147483647}$ . Tables are suppressed for speed. Note that

$$2^{31} - 1 = 2147483647 = p$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). The command creates two instances of  $-1$  in the field  $\mathbb{Z}_p$  and multiplies them. The fact that this is done correctly is remarkable, because the intermediate value exceeds the size of 32 bit signed integers. Recall that  $-1$  is stored as  $p - 1$ , so

$$-1 \equiv 2147483646.$$

But

$$-1 \cdot -1 \equiv 2147483646 \cdot 2147483646 = 4611686009837453316,$$

which exceeds the range of 32 bit signed integers. The reason why this command works is that the multiplication of integers is performed using long integer arithmetic. Long integer arithmetic treats integers as text strings, and hence is not bound by the size of a machine word. Unfortunately, long integer arithmetic is considerably slower than arithmetic in machine words. So, there is a performance drawback with large finite fields in Orbiter.

The command

#### Example 696

```
F_32bit:
▷ $(ORBITER) -v 10 \
▷ ▷ -define F -finite_field -q 4294967291 -without_tables -end \
```

creates the field  $\mathbb{F}_{4294967291}$ . Note that

$$2^{32} - 5 = 4294967291$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). The command will likely not run, and terminate with an error message. The reason is hardware dependent. The command tests the size of a machine word. Most compilers map the C++ data type *signed int* to 32 bit machine words, so this command will fail in such an environment. This is true even with 64 bit hardware. There may be some compiler options to change this.

Machines for which the previous command works correctly, the following command should also be fine:

**Example 697**

```

F_63bit:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 9223372036854775783 -without_tables -end \
> > -define v -vector -field F -allow_negatives \
> > > -dense "-1,-1" -end \
> > -with F -do -finite_field_activity -product_of v -end

```

Note that

$$2^{63} - 25 = 9223372036854775783$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). However, the next command will almost surely fail:

**Example 698**

```

F_64bit:
> $(ORBITER) -v 10 \
> > -define F -finite_field -q 18446744073709551557 -without_tables -end \
> > -define v -vector -field F -allow_negatives \
> > > -dense "-1,-1" -end \
> > -with F -do -finite_field_activity -product_of v -end

```

This is because we are exceeding the range that can be stored in a 64 bit unsigned integer machine word. Note that

$$2^{64} - 59 = 18446744073709551557$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>).



## Chapter 20

# Orbiter on Windows

### 20.1 Using Windows Subsystem Linux

The following quote from <https://docs.microsoft.com/en-us/windows/wsl/> summarizes the function of the Windows Subsystem for Linux:

Windows Subsystem for Linux (WSL) lets developers run a GNU/Linux environment – including most command-line tools, utilities, and applications – directly on Windows, unmodified, without the overhead of a traditional virtual machine or dual-boot setup. You can:

1. Choose your favorite GNU/Linux distributions from the Microsoft Store.
2. Run common command-line tools such as grep, sed, awk, or other ELF-64 binaries.
3. Run Bash shell scripts and GNU/Linux command-line applications including:
4. Tools: vim, emacs, tmux
5. Languages: NodeJS, Javascript, Python, Ruby, C/C++, C# & F#, Rust, Go, etc.
6. Services: SSHD, MySQL, Apache, lighttpd, MongoDB, PostgreSQL.
7. Install additional software using your own GNU/Linux distribution package manager.
8. Invoke Windows applications using a Unix-like command-line shell.
9. Invoke GNU/Linux applications on Windows.

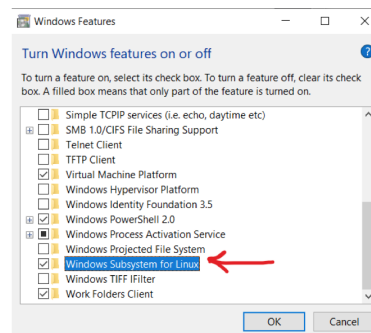
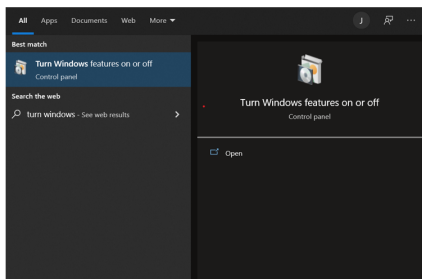
The following set of slides will illustrate the installation of Orbiter under WSL.

## Resources

- Many of the steps will be taken from the following sources:
  - [https://okunhardt.github.io/documents/Installing\\_WSL.pdf](https://okunhardt.github.io/documents/Installing_WSL.pdf)
  - <https://docs.microsoft.com/en-us/windows/wsl/basic-commands>
- Consult the two links for further help and suggestions.

## Installing WSL

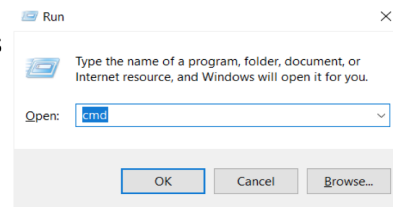
- Search “Turn Windows features on or off” in the Windows search bar
- Search for “Windows Subsystem for Linux”, the box must be checked
- Restart the computer





## Update

- The Windows Subsystem for Linux kernel does not automatically update due to system settings
- Updates must be done manually
- To update, first you need to command prompt as admin
  - Press Windows + R to open the “Run” box
  - Type “cmd” into the box
  - Press Ctrl + Shift + Enter
  - When the window prompt opens, click “Yes”
  - Command prompt will now open as admin
  - In command prompt
    - Type `wsl --update`
    - Type `wsl --shutdown`



```
Administrator: C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19044.1415]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>wsl --update
Checking for updates...
Downloading updates...
Installing updates...
This change will take effect on the next full restart of WSL. To force a restart, please run 'wsl --shutdown'.
Kernel version: 5.10.60.1
```

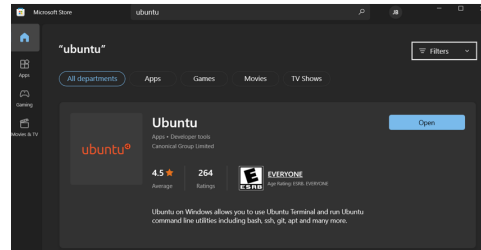
## WSL1, WSL2

- When using WSL, you can adjust the configurations according to the Linux distribution that you are using
- To run Ubuntu distribution, we need the WSL1 configuration
- To check the status, in the command prompt enter
  - `wsl --status`
- To change WSL configuration type
  - `wsl --set-default-version 1`
  - `wsl --shutdown`

```
C:\Users\Joel\CPP_Workspace>wsl --status
Default Distribution: Ubuntu
Default Version: 1
```

## Ubuntu - installation

- Generally, the Ubuntu distribution is installed by default when WSL is installed
  - `wsl --status`
    - Displays the default distribution
- If you find that Ubuntu was not installed, you can find it in the Microsoft store
- Launch Ubuntu after installation



## Ubuntu - launching

- After launching Ubuntu, allow the installation to be initiated
- If you receive an error, this could be a result of the configuration
  - Set configuration to WSL1
    - `wsl --set-default-version 1`
  - Make sure to terminate Ubuntu and reboot
    - `wsl --terminate Ubuntu`
  - Start Ubuntu again
- Once Ubuntu starts correctly
  - Create Username & Password to complete installation
  - Note: the password will not appear when you type it

```
joebarr@LAPTOP-KQU42VS8: ~  
Installing, this may take a few minutes...  
Please create a default UNIX user account. The username does not need to match your Windows username.  
For more information visit: https://aka.ms/wslusers  
Enter new UNIX username: joebarr  
New password:
```

## Ubuntu - update

- Ubuntu does not update automatically, to update run the command
  - `sudo apt update && sudo apt upgrade`
- You will be prompted to enter your password
- When update are ready to be installed the message will appear
  - Do you want to continue? [Y/n]
    - Y + enter

```

joebarr@LAPTOP-KQU42VS8: ~
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 4.4.0-19041-Microsoft x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue Jan 11 22:39:50 MST 2022

System load:          0.52
Usage of /home:       unknown
Memory usage:         53%
Swap usage:           2%
Processes:            7
Users logged in:      0
IPv4 address for wifi0: 192.168.7.65
IPv6 address for wifi0: fd45:1ae2:adef:1:6cc6:baae:e12c:92d7
IPv6 address for wifi0: fd45:1ae2:adef:1:54fa:5f35:b84:2f87
IPv6 address for wifi0: fd45:1ae2:adef:1:700e:4c46:55cb:ef48

updates can be applied immediately.

This message is shown once a day. To disable it please create the
/home/joebarr/.hushlogin file.
joebarr@LAPTOP-KQU42VS8: ~$ sudo apt update && sudo apt upgrade

```

## Ubuntu – g++ and make

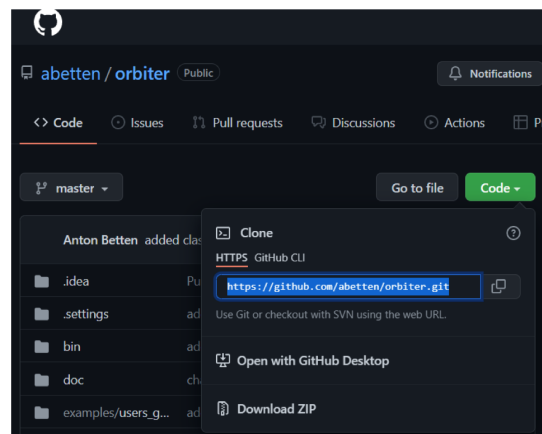
- At this point, you have successfully installed and setup WSL, and now you can use the terminal as you would on Ubuntu
- Terminate and reboot Ubuntu
- Run the command in Ubuntu
  - `sudo apt install g++`
  - You can now compile C++ in WSL
- Run the command in Ubuntu
  - `sudo apt install make`
  - You can now use `makefiles` in WSL

## Orbiter - installation

- The easiest way to run make is through the command prompt, not Ubuntu
- To run WSL commands in command prompt, use either
  - `wsl <command>`
  - `wsl.exe <command>`
- Open command prompt
- Change directory to Users\username
  - `cd C:\Users\your username`

## Orbiter - installation

- In web, go to <https://github.com/abetten/orbiter>
- Click on the green icon “Code” that opens a drop-down menu
- You want to copy HTTPS URL



## Orbiter - installation

- In command prompt, once you are in C:\Users\Joel type the command
  - wsl.exe git clone <https://github.com/abetten/orbiter.git>
  - Hit enter
- Now, orbiter will begin the cloning process

```
C:\Users\Joel>wsl.exe git clone https://github.com/abetten/orbiter.git
Cloning into 'orbiter'...
remote: Enumerating objects: 87873, done.
remote: Counting objects: 100% (7029/7029), done.
remote: Compressing objects: 100% (5007/5007), done.
remote: Total 87873 (delta 5245), reused 3408 (delta 1713), pack-reused 80844
Receiving objects: 100% (87873/87873), 1.55 GiB | 18.87 MiB/s, done.
Resolving deltas: 100% (47649/47649), done.
Updating files: 100% (1154/1154), done.
```

## Orbiter - compile

- After cloning orbiter, run the command
  - `dir`
- You will find a new directory created called “orbiter”
- Change directory to “orbiter”
  - `cd orbiter`

```
C:\Users\Joel>dir
Volume in drive C is Windows
Volume Serial Number is 6C2E-AA7C

Directory of C:\Users\Joel

01/11/2022  12:00 AM  <DIR>      .
01/11/2022  12:00 AM  <DIR>      ..
12/23/2020  02:51 PM  <DIR>      3D Objects
12/23/2020  02:51 PM  <DIR>      Contacts
12/23/2020  02:51 PM  <DIR>      Desktop
01/09/2022  06:11 PM  <DIR>      Documents
12/23/2020  02:51 PM  <DIR>      Downloads
02/09/2018  03:27 PM  <DIR>      Dropbox
12/23/2020  02:51 PM  <DIR>      Favorites
07/10/2020  10:27 AM  <DIR>      HP
08/09/2019  07:25 AM  <DIR>      Intel
12/23/2020  02:51 PM  <DIR>      Links
12/23/2020  02:51 PM  <DIR>      Music
08/04/2021  09:20 AM  <DIR>      OneDrive
01/11/2022  12:02 AM  <DIR>      orbiter
01/09/2022  09:19 PM  <DIR>      Pictures
05/27/2018  03:38 PM  <DIR>      Roaming
12/23/2020  02:51 PM  <DIR>      Saved Games
12/23/2020  02:51 PM  <DIR>      Searches
01/09/2022  04:36 PM  <DIR>      source
12/23/2020  02:51 PM  <DIR>      Videos
               0 File(s)              0 bytes
               21 Dir(s)  70,471,704,576 bytes free
```

## Orbiter - compile

- Now that you are in C:\Users\“your username”\orbiter, run the command
  - wsl.exe make
- The orbiter library will now be compiled, give it some time

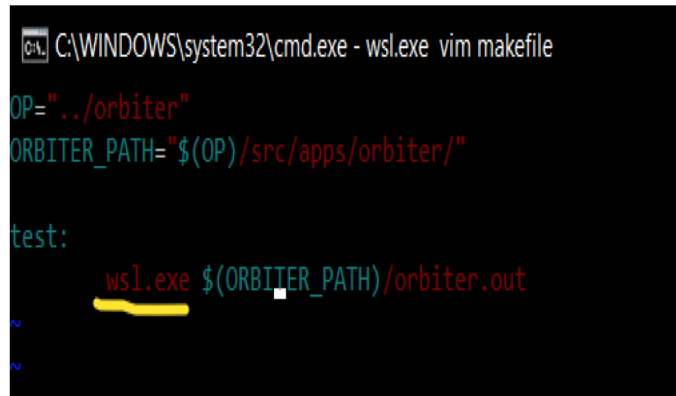
```
C:\Users\Joel\orbiter>wsl.exe make ←
cd src; make all
make[1]: Entering directory '/mnt/c/Users/Joel/orbiter/src'
cd lib; make all
make[2]: Entering directory '/mnt/c/Users/Joel/orbiter/src/lib'
cd foundations; make
```

## Makefile

- Now that orbiter has been successfully compiled, in the directory C:\Users\“your username”\orbiter
  - Change directory to C:\Users\“your username” and create a new directory
  - Ex: `mkdir CPP_Workspace`
- Change directory into `CPP_Workspace`
  - `cd CPP_Workspace`
- In C:\Users\“your username”\“new directory”, run the command
  - `wsl.exe vim makefile`
- Vim (an IDE) will create the file “`makefile`”
- For Vim commands, go to <https://vim.rtorr.com/>
- Remember: all Ubuntu commands must begin with either
  - `wsl` or `wsl.exe`

## Makefile

- To edit file in vim, click “i”
- You will see --insert-- in the lower left-hand corner
- The example to the right demonstrates a simple test to assure that orbiter is running correctly
- Assuming that orbiter directory is located in C:\Users\“your username” then the variable OP and ORBITER\_PATH should work just fine
- Note where wsl.exe is inserted
- Makefile contains Ubuntu commands not windows commands



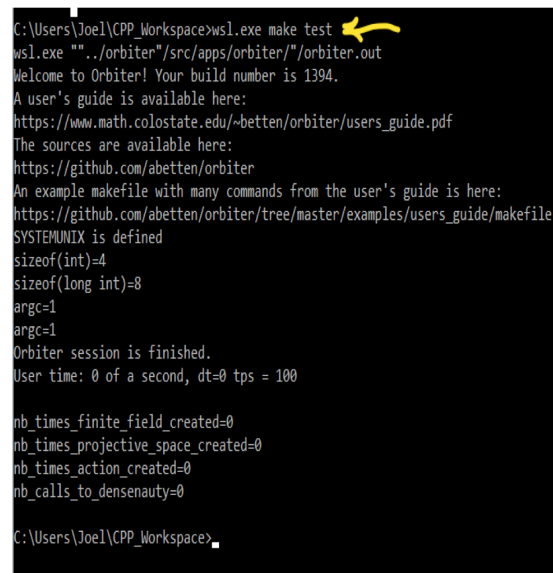
```
C:\WINDOWS\system32\cmd.exe - wsl.exe vim makefile

OP="../orbiter"
ORBITER_PATH="${OP}/src/apps/orbiter/"

test:
    wsl.exe ${ORBITER_PATH}/orbiter.out
```

## Running makefile

- Now that you have created the makefile,
  - Click “esc” to finish editing in vim
  - Run the command
    - :wq + enter
  - This saves & closes the makefile in vim
- You will be returned to
  - C:\Users\“your username”\“new directory”
- In the directory run,
  - wsl.exe make test
  - Hit “enter”
- If everything runs correctly, you will see



```
C:\Users\Joel\CPP_Workspace>wsl.exe make test
wsl.exe "../orbiter/src/apps/orbiter"/orbiter.out
Welcome to Orbiter! Your build number is 1394.
A user's guide is available here:
https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf
The sources are available here:
https://github.com/abetten/orbiter
An example makefile with many commands from the user's guide is here:
https://github.com/abetten/orbiter/tree/master/examples/users_guide/makefile
SYSTEMUNIX is defined
sizeof(int)=4
sizeof(long int)=8
argc=1
argc=1
Orbiter session is finished.
User time: 0 of a second, dt=0 tps = 100

nb_times_finite_field_created=0
nb_times_projective_space_created=0
nb_times_action_created=0
nb_calls_to_densnauty=0
C:\Users\Joel\CPP_Workspace>
```

## Orbiter - notes

- Now that everything runs correctly, visit [https://www.math.colostate.edu/~betten/orbiter/users\\_guide.pdf](https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf)
- This is the Orbiter User's guide
- Remember that you must use "wsl.exe make <target>" or "wsl make <target>" to run make correctly on linux distribution
- Also, note how "wsl.exe" is used inside of the makefile
- Ubuntu commands are used in makefile

## Orbiter - update

- To update orbiter, change directories to
  - C:\Users\"your username"\orbiter
- Run the commands
  - wsl.exe make clean ; wsl.exe make
- Good luck!



## Chapter 21

# Templates

### 21.1 Templates

| $h$ | mon      | vector       |  |  |  |  |  |
|-----|----------|--------------|--|--|--|--|--|
| 0   | $X_0^2$  | (2, 0, 0, 0) |  |  |  |  |  |
| 1   | $X_1^2$  | (0, 2, 0, 0) |  |  |  |  |  |
| 2   | $X_2^2$  | (0, 0, 2, 0) |  |  |  |  |  |
| 3   | $X_3^2$  | (0, 0, 0, 2) |  |  |  |  |  |
| 4   | $X_0X_1$ | (1, 1, 0, 0) |  |  |  |  |  |
| 5   | $X_0X_2$ | (1, 0, 1, 0) |  |  |  |  |  |
| 6   | $X_0X_3$ | (1, 0, 0, 1) |  |  |  |  |  |
| 7   | $X_1X_2$ | (0, 1, 1, 0) |  |  |  |  |  |
| 8   | $X_1X_3$ | (0, 1, 0, 1) |  |  |  |  |  |
| 9   | $X_2X_3$ | (0, 0, 1, 1) |  |  |  |  |  |

| $h$ | mon           | vector       |  |  |  |  |  |
|-----|---------------|--------------|--|--|--|--|--|
| 0   | $X_0^3$       | (3, 0, 0, 0) |  |  |  |  |  |
| 1   | $X_1^3$       | (0, 3, 0, 0) |  |  |  |  |  |
| 2   | $X_2^3$       | (0, 0, 3, 0) |  |  |  |  |  |
| 3   | $X_3^3$       | (0, 0, 0, 3) |  |  |  |  |  |
| 4   | $X_0^2 X_1$   | (2, 1, 0, 0) |  |  |  |  |  |
| 5   | $X_0^2 X_2$   | (2, 0, 1, 0) |  |  |  |  |  |
| 6   | $X_0^2 X_3$   | (2, 0, 0, 1) |  |  |  |  |  |
| 7   | $X_0 X_1^2$   | (1, 2, 0, 0) |  |  |  |  |  |
| 8   | $X_1^2 X_2$   | (0, 2, 1, 0) |  |  |  |  |  |
| 9   | $X_1^2 X_3$   | (0, 2, 0, 1) |  |  |  |  |  |
| 10  | $X_0 X_2^2$   | (1, 0, 2, 0) |  |  |  |  |  |
| 11  | $X_1 X_2^2$   | (0, 1, 2, 0) |  |  |  |  |  |
| 12  | $X_2^2 X_3$   | (0, 0, 2, 1) |  |  |  |  |  |
| 13  | $X_0 X_3^2$   | (1, 0, 0, 2) |  |  |  |  |  |
| 14  | $X_1 X_3^2$   | (0, 1, 0, 2) |  |  |  |  |  |
| 15  | $X_2 X_3^2$   | (0, 0, 1, 2) |  |  |  |  |  |
| 16  | $X_0 X_1 X_2$ | (1, 1, 1, 0) |  |  |  |  |  |
| 17  | $X_0 X_1 X_3$ | (1, 1, 0, 1) |  |  |  |  |  |
| 18  | $X_0 X_2 X_3$ | (1, 0, 1, 1) |  |  |  |  |  |
| 19  | $X_1 X_2 X_3$ | (0, 1, 1, 1) |  |  |  |  |  |



# Appendix A

## Data on Cubic Surfaces

### A.1 Data on Cubic Surfaces

#### Cubic Surfaces by Field Order and Number of Eckardt Points

| $q$ | total | 0  | 1  | 2  | 3  | 4 | 5 | 6  | 9 | 10 | 13 | 18 | 45 |
|-----|-------|----|----|----|----|---|---|----|---|----|----|----|----|
| 4   | 1     | 0  | 0  | 0  | 0  | 0 | 0 | 0  | 0 | 0  | 0  | 0  | 1  |
| 7   | 1     | 0  | 0  | 0  | 0  | 0 | 0 | 0  | 0 | 0  | 0  | 1  | 0  |
| 8   | 1     | 0  | 0  | 0  | 0  | 0 | 0 | 0  | 0 | 0  | 1  | 0  | 0  |
| 9   | 2     | 0  | 0  | 0  | 0  | 0 | 0 | 0  | 1 | 1  | 0  | 0  | 0  |
| 11  | 2     | 0  | 0  | 0  | 0  | 0 | 0 | 1  | 0 | 1  | 0  | 0  | 0  |
| 13  | 4     | 0  | 0  | 0  | 0  | 1 | 0 | 1  | 1 | 0  | 0  | 1  | 0  |
| 16  | 5     | 0  | 0  | 0  | 1  | 0 | 1 | 0  | 1 | 0  | 1  | 0  | 1  |
| 17  | 7     | 0  | 1  | 0  | 1  | 2 | 0 | 3  | 0 | 0  | 0  | 0  | 0  |
| 19  | 10    | 0  | 0  | 2  | 2  | 1 | 0 | 2  | 1 | 1  | 0  | 1  | 0  |
| 23  | 16    | 0  | 2  | 2  | 4  | 3 | 0 | 5  | 0 | 0  | 0  | 0  | 0  |
| 25  | 18    | 0  | 4  | 3  | 3  | 2 | 0 | 3  | 2 | 0  | 0  | 1  | 0  |
| 27  | 11    | 0  | 2  | 2  | 2  | 2 | 0 | 2  | 1 | 0  | 0  | 0  | 0  |
| 29  | 34    | 1  | 6  | 7  | 11 | 3 | 0 | 5  | 0 | 1  | 0  | 0  | 0  |
| 31  | 43    | 1  | 10 | 9  | 11 | 3 | 0 | 5  | 2 | 1  | 0  | 1  | 0  |
| 32  | 11    | 1  | 3  | 0  | 4  | 0 | 2 | 0  | 0 | 0  | 1  | 0  | 0  |
| 37  | 77    | 4  | 25 | 14 | 18 | 5 | 0 | 7  | 3 | 0  | 0  | 1  | 0  |
| 41  | 107   | 9  | 37 | 19 | 28 | 5 | 0 | 8  | 0 | 1  | 0  | 0  | 0  |
| 43  | 126   | 11 | 48 | 21 | 27 | 6 | 0 | 9  | 3 | 0  | 0  | 1  | 0  |
| 47  | 169   | 20 | 67 | 26 | 38 | 7 | 0 | 11 | 0 | 0  | 0  | 0  | 0  |
| 49  | 121   | 16 | 46 | 19 | 25 | 4 | 0 | 6  | 3 | 1  | 0  | 1  | 0  |

| $q$ | total | 0    | 1    | 2   | 3   | 4  | 5  | 6  | 9 | 10 | 13 | 18 | 45 |
|-----|-------|------|------|-----|-----|----|----|----|---|----|----|----|----|
| 53  | 258   | 40   | 110  | 36  | 52  | 8  | 0  | 12 | 0 | 0  | 0  | 0  | 0  |
| 59  | 376   | 72   | 166  | 48  | 68  | 8  | 0  | 13 | 0 | 1  | 0  | 0  | 0  |
| 61  | 427   | 85   | 193  | 53  | 69  | 8  | 0  | 12 | 5 | 1  | 0  | 1  | 0  |
| 64  | 101   | 20   | 51   | 0   | 17  | 0  | 7  | 0  | 2 | 0  | 3  | 0  | 1  |
| 67  | 595   | 139  | 275  | 65  | 85  | 10 | 0  | 15 | 5 | 0  | 0  | 1  | 0  |
| 71  | 731   | 189  | 335  | 75  | 105 | 10 | 0  | 16 | 0 | 1  | 0  | 0  | 0  |
| 73  | 813   | 216  | 378  | 80  | 105 | 11 | 0  | 16 | 6 | 0  | 0  | 1  | 0  |
| 79  | 1081  | 321  | 500  | 97  | 127 | 11 | 0  | 17 | 6 | 1  | 0  | 1  | 0  |
| 81  | 331   | 100  | 149  | 30  | 38  | 4  | 0  | 6  | 3 | 1  | 0  | 0  | 0  |
| 83  | 1292  | 411  | 592  | 107 | 149 | 13 | 0  | 20 | 0 | 0  | 0  | 0  | 0  |
| 89  | 1673  | 577  | 759  | 127 | 176 | 13 | 0  | 20 | 0 | 1  | 0  | 0  | 0  |
| 97  | 2304  | 868  | 1033 | 154 | 203 | 15 | 0  | 22 | 8 | 0  | 0  | 1  | 0  |
| 101 | 2673  | 1053 | 1179 | 169 | 233 | 15 | 0  | 23 | 0 | 1  | 0  | 0  | 0  |
| 103 | 2877  | 1152 | 1268 | 176 | 232 | 16 | 0  | 24 | 8 | 0  | 0  | 1  | 0  |
| 107 | 3313  | 1379 | 1437 | 191 | 263 | 17 | 0  | 26 | 0 | 0  | 0  | 0  | 0  |
| 109 | 3557  | 1501 | 1539 | 201 | 265 | 16 | 0  | 24 | 9 | 1  | 0  | 1  | 0  |
| 113 | 4067  | 1776 | 1733 | 216 | 297 | 18 | 0  | 27 | 0 | 0  | 0  | 0  | 0  |
| 121 | 2753  | 1262 | 1135 | 140 | 179 | 11 | 0  | 17 | 7 | 1  | 0  | 1  | 0  |
| 128 | 929   | 443  | 405  | 0   | 60  | 0  | 18 | 0  | 0 | 0  | 3  | 0  | 0  |

Total: 30917

**0 Eckardt Points**

| $q$ | total | Ago                   |
|-----|-------|-----------------------|
| 29  | 1     | 1                     |
| 31  | 1     | 1                     |
| 32  | 1     | 5                     |
| 37  | 4     | $1^4$                 |
| 41  | 9     | $1^9$                 |
| 43  | 11    | $1^{11}$              |
| 47  | 20    | $1^{20}$              |
| 49  | 16    | $2^7, 1^9$            |
| 53  | 40    | $1^{40}$              |
| 59  | 72    | $1^{72}$              |
| 61  | 85    | $1^{85}$              |
| 64  | 20    | $2^6, 1^{14}$         |
| 67  | 139   | $1^{139}$             |
| 71  | 189   | $1^{189}$             |
| 73  | 216   | $1^{216}$             |
| 79  | 321   | $1^{321}$             |
| 81  | 100   | $4^2, 2^{15}, 1^{83}$ |
| 83  | 411   | $1^{411}$             |
| 89  | 577   | $1^{577}$             |
| 97  | 868   | $1^{868}$             |
| 101 | 1053  | $1^{1053}$            |
| 103 | 1152  | $1^{1152}$            |
| 107 | 1379  | $1^{1379}$            |
| 109 | 1501  | $1^{1501}$            |
| 113 | 1776  | $1^{1776}$            |
| 121 | 1262  | $2^{95}, 1^{1167}$    |
| 128 | 443   | $1^{443}$             |

Total: 11667

## 1 Eckardt Points

| $q$ | total | Ago                         |
|-----|-------|-----------------------------|
| 17  | 1     | 8                           |
| 23  | 2     | $2^2$                       |
| 25  | 4     | $16, 4^3$                   |
| 27  | 2     | $2^2$                       |
| 29  | 6     | $4, 2^5$                    |
| 31  | 10    | $2^{10}$                    |
| 32  | 3     | $2^3$                       |
| 37  | 25    | $4, 2^{24}$                 |
| 41  | 37    | $8, 4, 2^{35}$              |
| 43  | 48    | $2^{48}$                    |
| 47  | 67    | $2^{67}$                    |
| 49  | 46    | $16, 8, 4^8, 2^{36}$        |
| 53  | 110   | $4^2, 2^{108}$              |
| 59  | 166   | $2^{166}$                   |
| 61  | 193   | $4^2, 2^{191}$              |
| 64  | 51    | $12^3, 4^7, 2^{41}$         |
| 67  | 275   | $2^{275}$                   |
| 71  | 335   | $2^{335}$                   |
| 73  | 378   | $8, 4^2, 2^{375}$           |
| 79  | 500   | $2^{500}$                   |
| 81  | 149   | $16, 8^6, 4^{15}, 2^{127}$  |
| 83  | 592   | $2^{592}$                   |
| 89  | 759   | $8, 4^3, 2^{755}$           |
| 97  | 1033  | $8, 4^3, 2^{1029}$          |
| 101 | 1179  | $4^4, 2^{1175}$             |
| 103 | 1268  | $2^{1268}$                  |
| 107 | 1437  | $2^{1437}$                  |
| 109 | 1539  | $4^4, 2^{1535}$             |
| 113 | 1733  | $8, 4^4, 2^{1728}$          |
| 121 | 1135  | $16, 8^2, 4^{77}, 2^{1055}$ |
| 128 | 405   | $2^{405}$                   |

Total: 13488



## 2 Eckardt Points

| $q$ | total | Ago                 |
|-----|-------|---------------------|
| 19  | 2     | $4^2$               |
| 23  | 2     | $4^2$               |
| 25  | 3     | $8^2, 4$            |
| 27  | 2     | $12, 4$             |
| 29  | 7     | $4^7$               |
| 31  | 9     | $4^9$               |
| 37  | 14    | $4^{14}$            |
| 41  | 19    | $4^{19}$            |
| 43  | 21    | $4^{21}$            |
| 47  | 26    | $4^{26}$            |
| 49  | 19    | $8^7, 4^{12}$       |
| 53  | 36    | $4^{36}$            |
| 59  | 48    | $4^{48}$            |
| 61  | 53    | $4^{53}$            |
| 67  | 65    | $4^{65}$            |
| 71  | 75    | $4^{75}$            |
| 73  | 80    | $4^{80}$            |
| 79  | 97    | $4^{97}$            |
| 81  | 30    | $16^2, 8^6, 4^{22}$ |
| 83  | 107   | $4^{107}$           |
| 89  | 127   | $4^{127}$           |
| 97  | 154   | $4^{154}$           |
| 101 | 169   | $4^{169}$           |
| 103 | 176   | $4^{176}$           |
| 107 | 191   | $4^{191}$           |
| 109 | 201   | $4^{201}$           |
| 113 | 216   | $4^{216}$           |
| 121 | 140   | $8^{27}, 4^{113}$   |

Total: 2089

**3 Eckardt Points**

| $q$ | total | Ago                   |
|-----|-------|-----------------------|
| 16  | 1     | 12                    |
| 17  | 1     | 6                     |
| 19  | 2     | $6^2$                 |
| 23  | 4     | $6^4$                 |
| 25  | 3     | 12, $6^2$             |
| 27  | 2     | $6^2$                 |
| 29  | 11    | $6^{11}$              |
| 31  | 11    | $6^{11}$              |
| 32  | 4     | $6^4$                 |
| 37  | 18    | $6^{18}$              |
| 41  | 28    | $6^{28}$              |
| 43  | 27    | $6^{27}$              |
| 47  | 38    | $6^{38}$              |
| 49  | 25    | $12^{10}$ , $6^{15}$  |
| 53  | 52    | $6^{52}$              |
| 59  | 68    | $6^{68}$              |
| 61  | 69    | $6^{69}$              |
| 64  | 17    | $12^4$ , $6^{13}$     |
| 67  | 85    | $6^{85}$              |
| 71  | 105   | $6^{105}$             |
| 73  | 105   | $6^{105}$             |
| 79  | 127   | $6^{127}$             |
| 81  | 38    | $12^7$ , $6^{31}$     |
| 83  | 149   | $6^{149}$             |
| 89  | 176   | $6^{176}$             |
| 97  | 203   | $6^{203}$             |
| 101 | 233   | $6^{233}$             |
| 103 | 232   | $6^{232}$             |
| 107 | 263   | $6^{263}$             |
| 109 | 265   | $6^{265}$             |
| 113 | 297   | $6^{297}$             |
| 121 | 179   | $12^{24}$ , $6^{155}$ |
| 128 | 60    | $6^{60}$              |

Total: 2898

**4 Eckardt Points**

| $q$ | total | Ago             |
|-----|-------|-----------------|
| 13  | 1     | 12              |
| 17  | 2     | $12^2$          |
| 19  | 1     | 12              |
| 23  | 3     | $12^3$          |
| 25  | 2     | 24, 12          |
| 27  | 2     | 36, 12          |
| 29  | 3     | $12^3$          |
| 31  | 3     | $12^3$          |
| 37  | 5     | $12^5$          |
| 41  | 5     | $12^5$          |
| 43  | 6     | $12^6$          |
| 47  | 7     | $12^7$          |
| 49  | 4     | $24^2$ , $12^2$ |
| 53  | 8     | $12^8$          |
| 59  | 8     | $12^8$          |
| 61  | 8     | $12^8$          |
| 67  | 10    | $12^{10}$       |
| 71  | 10    | $12^{10}$       |
| 73  | 11    | $12^{11}$       |
| 79  | 11    | $12^{11}$       |
| 81  | 4     | $24^2$ , $12^2$ |
| 83  | 13    | $12^{13}$       |
| 89  | 13    | $12^{13}$       |
| 97  | 15    | $12^{15}$       |
| 101 | 15    | $12^{15}$       |
| 103 | 16    | $12^{16}$       |
| 107 | 17    | $12^{17}$       |
| 109 | 16    | $12^{16}$       |
| 113 | 18    | $12^{18}$       |
| 121 | 11    | $24^4$ , $12^7$ |

Total: 248

**5 Eckardt Points**

| $q$ | total | Ago                 |
|-----|-------|---------------------|
| 16  | 1     | 64                  |
| 32  | 2     | 80, 16              |
| 64  | 7     | 48, $32^3$ , $16^3$ |
| 128 | 18    | $16^{18}$           |

Total: 28

## 6 Eckardt Points

| $q$ | total | Ago             |
|-----|-------|-----------------|
| 11  | 1     | $24$            |
| 13  | 1     | $24$            |
| 17  | 3     | $24^3$          |
| 19  | 2     | $24^2$          |
| 23  | 5     | $24^5$          |
| 25  | 3     | $48^2, 24$      |
| 27  | 2     | $24^2$          |
| 29  | 5     | $24^5$          |
| 31  | 5     | $24^5$          |
| 37  | 7     | $24^7$          |
| 41  | 8     | $24^8$          |
| 43  | 9     | $24^9$          |
| 47  | 11    | $24^{11}$       |
| 49  | 6     | $48^3, 24^3$    |
| 53  | 12    | $24^{12}$       |
| 59  | 13    | $24^{13}$       |
| 61  | 12    | $24^{12}$       |
| 67  | 15    | $24^{15}$       |
| 71  | 16    | $24^{16}$       |
| 73  | 16    | $24^{16}$       |
| 79  | 17    | $24^{17}$       |
| 81  | 6     | $48^3, 24^3$    |
| 83  | 20    | $24^{20}$       |
| 89  | 20    | $24^{20}$       |
| 97  | 22    | $24^{22}$       |
| 101 | 23    | $24^{23}$       |
| 103 | 24    | $24^{24}$       |
| 107 | 26    | $24^{26}$       |
| 109 | 24    | $24^{24}$       |
| 113 | 27    | $24^{27}$       |
| 121 | 17    | $48^7, 24^{10}$ |

Total: 378

**9 Eckardt Points**

| $q$ | total | Ago                   |
|-----|-------|-----------------------|
| 9   | 1     | 432                   |
| 13  | 1     | 108                   |
| 16  | 1     | 216                   |
| 19  | 1     | 54                    |
| 25  | 2     | 216, 108              |
| 27  | 1     | 162                   |
| 31  | 2     | $54^2$                |
| 37  | 3     | 108, $54^2$           |
| 43  | 3     | $54^3$                |
| 49  | 3     | 216, 108, 54          |
| 61  | 5     | 108, $54^4$           |
| 64  | 2     | 162, 108              |
| 67  | 5     | $54^5$                |
| 73  | 6     | 108, $54^5$           |
| 79  | 6     | $54^6$                |
| 81  | 3     | 864, 216, 108         |
| 97  | 8     | 108, $54^7$           |
| 103 | 8     | $54^8$                |
| 109 | 9     | 108, $54^8$           |
| 121 | 7     | 216, $108^3$ , $54^3$ |

Total: 77

**10 Eckardt Points**

| $q$ | total | Ago |
|-----|-------|-----|
| 9   | 1     | 240 |
| 11  | 1     | 120 |
| 19  | 1     | 120 |
| 29  | 1     | 120 |
| 31  | 1     | 120 |
| 41  | 1     | 120 |
| 49  | 1     | 240 |
| 59  | 1     | 120 |
| 61  | 1     | 120 |
| 71  | 1     | 120 |
| 79  | 1     | 120 |
| 81  | 1     | 480 |
| 89  | 1     | 120 |
| 101 | 1     | 120 |
| 109 | 1     | 120 |
| 121 | 1     | 240 |

Total: 16

**13 Eckardt Points**

| $q$ | total | Ago            |
|-----|-------|----------------|
| 8   | 1     | 576            |
| 16  | 1     | 384            |
| 32  | 1     | 192            |
| 64  | 3     | 1152, 384, 192 |
| 128 | 3     | $192^3$        |

Total: 9

**18 Eckardt Points**

| $q$ | total | Ago  |
|-----|-------|------|
| 7   | 1     | 648  |
| 13  | 1     | 648  |
| 19  | 1     | 648  |
| 25  | 1     | 1296 |
| 31  | 1     | 648  |
| 37  | 1     | 648  |
| 43  | 1     | 648  |
| 49  | 1     | 1296 |
| 61  | 1     | 648  |
| 67  | 1     | 648  |
| 73  | 1     | 648  |
| 79  | 1     | 648  |
| 97  | 1     | 648  |
| 103 | 1     | 648  |
| 109 | 1     | 648  |
| 121 | 1     | 1296 |

Total: 16

**45 Eckardt Points**

| $q$ | total | Ago    |
|-----|-------|--------|
| 4   | 1     | 51840  |
| 16  | 1     | 103680 |
| 64  | 1     | 155520 |

Total: 3

**Even Characteristic**

| $q$ | total | 0   | 1   | 3  | 5  | 9 | 13 | 45 |
|-----|-------|-----|-----|----|----|---|----|----|
| 4   | 1     | 0   | 0   | 0  | 0  | 0 | 0  | 1  |
| 8   | 1     | 0   | 0   | 0  | 0  | 0 | 1  | 0  |
| 16  | 5     | 0   | 0   | 1  | 1  | 1 | 1  | 1  |
| 32  | 11    | 1   | 3   | 4  | 2  | 0 | 1  | 0  |
| 64  | 101   | 20  | 51  | 17 | 7  | 2 | 3  | 1  |
| 128 | 929   | 443 | 405 | 60 | 18 | 0 | 3  | 0  |

Total: 1048

**0 Eckardt Points**

| $q$ | total | Ago           |
|-----|-------|---------------|
| 32  | 1     | 5             |
| 64  | 20    | $2^6, 1^{14}$ |
| 128 | 443   | $1^{443}$     |

Total: 464

**1 Eckardt Points**

| $q$ | total | Ago                 |
|-----|-------|---------------------|
| 32  | 3     | $2^3$               |
| 64  | 51    | $12^3, 4^7, 2^{41}$ |
| 128 | 405   | $2^{405}$           |

Total: 459

**3 Eckardt Points**

| $q$ | total | Ago            |
|-----|-------|----------------|
| 16  | 1     | 12             |
| 32  | 4     | $6^4$          |
| 64  | 17    | $12^4, 6^{13}$ |
| 128 | 60    | $6^{60}$       |

Total: 82

**5 Eckardt Points**

| $q$ | total | Ago                 |
|-----|-------|---------------------|
| 16  | 1     | 64                  |
| 32  | 2     | 80, 16              |
| 64  | 7     | 48, $32^3$ , $16^3$ |
| 128 | 18    | $16^{18}$           |

Total: 28

**9 Eckardt Points**

| $q$ | total | Ago      |
|-----|-------|----------|
| 16  | 1     | 216      |
| 64  | 2     | 162, 108 |

Total: 3

**13 Eckardt Points**

| $q$ | total | Ago            |
|-----|-------|----------------|
| 8   | 1     | 576            |
| 16  | 1     | 384            |
| 32  | 1     | 192            |
| 64  | 3     | 1152, 384, 192 |
| 128 | 3     | $192^3$        |

Total: 9

**45 Eckardt Points**

| $q$ | total | Ago    |
|-----|-------|--------|
| 4   | 1     | 51840  |
| 16  | 1     | 103680 |
| 64  | 1     | 155520 |

Total: 3



## Odd Characteristic

| $q$ | total | 0    | 1    | 2   | 3   | 4  | 6  | 9 | 10 | 18 |
|-----|-------|------|------|-----|-----|----|----|---|----|----|
| 7   | 1     | 0    | 0    | 0   | 0   | 0  | 0  | 0 | 0  | 1  |
| 9   | 2     | 0    | 0    | 0   | 0   | 0  | 0  | 1 | 1  | 0  |
| 11  | 2     | 0    | 0    | 0   | 0   | 0  | 1  | 0 | 1  | 0  |
| 13  | 4     | 0    | 0    | 0   | 0   | 1  | 1  | 1 | 0  | 1  |
| 17  | 7     | 0    | 1    | 0   | 1   | 2  | 3  | 0 | 0  | 0  |
| 19  | 10    | 0    | 0    | 2   | 2   | 1  | 2  | 1 | 1  | 1  |
| 23  | 16    | 0    | 2    | 2   | 4   | 3  | 5  | 0 | 0  | 0  |
| 25  | 18    | 0    | 4    | 3   | 3   | 2  | 3  | 2 | 0  | 1  |
| 27  | 11    | 0    | 2    | 2   | 2   | 2  | 2  | 1 | 0  | 0  |
| 29  | 34    | 1    | 6    | 7   | 11  | 3  | 5  | 0 | 1  | 0  |
| 31  | 43    | 1    | 10   | 9   | 11  | 3  | 5  | 2 | 1  | 1  |
| 37  | 77    | 4    | 25   | 14  | 18  | 5  | 7  | 3 | 0  | 1  |
| 41  | 107   | 9    | 37   | 19  | 28  | 5  | 8  | 0 | 1  | 0  |
| 43  | 126   | 11   | 48   | 21  | 27  | 6  | 9  | 3 | 0  | 1  |
| 47  | 169   | 20   | 67   | 26  | 38  | 7  | 11 | 0 | 0  | 0  |
| 49  | 121   | 16   | 46   | 19  | 25  | 4  | 6  | 3 | 1  | 1  |
| 53  | 258   | 40   | 110  | 36  | 52  | 8  | 12 | 0 | 0  | 0  |
| 59  | 376   | 72   | 166  | 48  | 68  | 8  | 13 | 0 | 1  | 0  |
| 61  | 427   | 85   | 193  | 53  | 69  | 8  | 12 | 5 | 1  | 1  |
| 67  | 595   | 139  | 275  | 65  | 85  | 10 | 15 | 5 | 0  | 1  |
| 71  | 731   | 189  | 335  | 75  | 105 | 10 | 16 | 0 | 1  | 0  |
| 73  | 813   | 216  | 378  | 80  | 105 | 11 | 16 | 6 | 0  | 1  |
| 79  | 1081  | 321  | 500  | 97  | 127 | 11 | 17 | 6 | 1  | 1  |
| 81  | 331   | 100  | 149  | 30  | 38  | 4  | 6  | 3 | 1  | 0  |
| 83  | 1292  | 411  | 592  | 107 | 149 | 13 | 20 | 0 | 0  | 0  |
| 89  | 1673  | 577  | 759  | 127 | 176 | 13 | 20 | 0 | 1  | 0  |
| 97  | 2304  | 868  | 1033 | 154 | 203 | 15 | 22 | 8 | 0  | 1  |
| 101 | 2673  | 1053 | 1179 | 169 | 233 | 15 | 23 | 0 | 1  | 0  |
| 103 | 2877  | 1152 | 1268 | 176 | 232 | 16 | 24 | 8 | 0  | 1  |
| 107 | 3313  | 1379 | 1437 | 191 | 263 | 17 | 26 | 0 | 0  | 0  |
| 109 | 3557  | 1501 | 1539 | 201 | 265 | 16 | 24 | 9 | 1  | 1  |
| 113 | 4067  | 1776 | 1733 | 216 | 297 | 18 | 27 | 0 | 0  | 0  |
| 121 | 2753  | 1262 | 1135 | 140 | 179 | 11 | 17 | 7 | 1  | 1  |

Total: 29869

**0 Eckardt Points**

| $q$ | total | Ago                   |
|-----|-------|-----------------------|
| 29  | 1     | 1                     |
| 31  | 1     | 1                     |
| 37  | 4     | $1^4$                 |
| 41  | 9     | $1^9$                 |
| 43  | 11    | $1^{11}$              |
| 47  | 20    | $1^{20}$              |
| 49  | 16    | $2^7, 1^9$            |
| 53  | 40    | $1^{40}$              |
| 59  | 72    | $1^{72}$              |
| 61  | 85    | $1^{85}$              |
| 67  | 139   | $1^{139}$             |
| 71  | 189   | $1^{189}$             |
| 73  | 216   | $1^{216}$             |
| 79  | 321   | $1^{321}$             |
| 81  | 100   | $4^2, 2^{15}, 1^{83}$ |
| 83  | 411   | $1^{411}$             |
| 89  | 577   | $1^{577}$             |
| 97  | 868   | $1^{868}$             |
| 101 | 1053  | $1^{1053}$            |
| 103 | 1152  | $1^{1152}$            |
| 107 | 1379  | $1^{1379}$            |
| 109 | 1501  | $1^{1501}$            |
| 113 | 1776  | $1^{1776}$            |
| 121 | 1262  | $2^{95}, 1^{1167}$    |

Total: 11203

## 1 Eckardt Points

| $q$ | total | Ago                         |
|-----|-------|-----------------------------|
| 17  | 1     | 8                           |
| 23  | 2     | $2^2$                       |
| 25  | 4     | $16, 4^3$                   |
| 27  | 2     | $2^2$                       |
| 29  | 6     | $4, 2^5$                    |
| 31  | 10    | $2^{10}$                    |
| 37  | 25    | $4, 2^{24}$                 |
| 41  | 37    | $8, 4, 2^{35}$              |
| 43  | 48    | $2^{48}$                    |
| 47  | 67    | $2^{67}$                    |
| 49  | 46    | $16, 8, 4^8, 2^{36}$        |
| 53  | 110   | $4^2, 2^{108}$              |
| 59  | 166   | $2^{166}$                   |
| 61  | 193   | $4^2, 2^{191}$              |
| 67  | 275   | $2^{275}$                   |
| 71  | 335   | $2^{335}$                   |
| 73  | 378   | $8, 4^2, 2^{375}$           |
| 79  | 500   | $2^{500}$                   |
| 81  | 149   | $16, 8^6, 4^{15}, 2^{127}$  |
| 83  | 592   | $2^{592}$                   |
| 89  | 759   | $8, 4^3, 2^{755}$           |
| 97  | 1033  | $8, 4^3, 2^{1029}$          |
| 101 | 1179  | $4^4, 2^{1175}$             |
| 103 | 1268  | $2^{1268}$                  |
| 107 | 1437  | $2^{1437}$                  |
| 109 | 1539  | $4^4, 2^{1535}$             |
| 113 | 1733  | $8, 4^4, 2^{1728}$          |
| 121 | 1135  | $16, 8^2, 4^{77}, 2^{1055}$ |

Total: 13029

**2 Eckardt Points**

| $q$ | total | Ago                 |
|-----|-------|---------------------|
| 19  | 2     | $4^2$               |
| 23  | 2     | $4^2$               |
| 25  | 3     | $8^2, 4$            |
| 27  | 2     | $12, 4$             |
| 29  | 7     | $4^7$               |
| 31  | 9     | $4^9$               |
| 37  | 14    | $4^{14}$            |
| 41  | 19    | $4^{19}$            |
| 43  | 21    | $4^{21}$            |
| 47  | 26    | $4^{26}$            |
| 49  | 19    | $8^7, 4^{12}$       |
| 53  | 36    | $4^{36}$            |
| 59  | 48    | $4^{48}$            |
| 61  | 53    | $4^{53}$            |
| 67  | 65    | $4^{65}$            |
| 71  | 75    | $4^{75}$            |
| 73  | 80    | $4^{80}$            |
| 79  | 97    | $4^{97}$            |
| 81  | 30    | $16^2, 8^6, 4^{22}$ |
| 83  | 107   | $4^{107}$           |
| 89  | 127   | $4^{127}$           |
| 97  | 154   | $4^{154}$           |
| 101 | 169   | $4^{169}$           |
| 103 | 176   | $4^{176}$           |
| 107 | 191   | $4^{191}$           |
| 109 | 201   | $4^{201}$           |
| 113 | 216   | $4^{216}$           |
| 121 | 140   | $8^{27}, 4^{113}$   |

Total: 2089

## 3 Eckardt Points

| $q$ | total | Ago                |
|-----|-------|--------------------|
| 17  | 1     | 6                  |
| 19  | 2     | $6^2$              |
| 23  | 4     | $6^4$              |
| 25  | 3     | $12, 6^2$          |
| 27  | 2     | $6^2$              |
| 29  | 11    | $6^{11}$           |
| 31  | 11    | $6^{11}$           |
| 37  | 18    | $6^{18}$           |
| 41  | 28    | $6^{28}$           |
| 43  | 27    | $6^{27}$           |
| 47  | 38    | $6^{38}$           |
| 49  | 25    | $12^{10}, 6^{15}$  |
| 53  | 52    | $6^{52}$           |
| 59  | 68    | $6^{68}$           |
| 61  | 69    | $6^{69}$           |
| 67  | 85    | $6^{85}$           |
| 71  | 105   | $6^{105}$          |
| 73  | 105   | $6^{105}$          |
| 79  | 127   | $6^{127}$          |
| 81  | 38    | $12^7, 6^{31}$     |
| 83  | 149   | $6^{149}$          |
| 89  | 176   | $6^{176}$          |
| 97  | 203   | $6^{203}$          |
| 101 | 233   | $6^{233}$          |
| 103 | 232   | $6^{232}$          |
| 107 | 263   | $6^{263}$          |
| 109 | 265   | $6^{265}$          |
| 113 | 297   | $6^{297}$          |
| 121 | 179   | $12^{24}, 6^{155}$ |

Total: 2816

## 4 Eckardt Points

| $q$ | total | Ago             |
|-----|-------|-----------------|
| 13  | 1     | 12              |
| 17  | 2     | $12^2$          |
| 19  | 1     | 12              |
| 23  | 3     | $12^3$          |
| 25  | 2     | 24, 12          |
| 27  | 2     | 36, 12          |
| 29  | 3     | $12^3$          |
| 31  | 3     | $12^3$          |
| 37  | 5     | $12^5$          |
| 41  | 5     | $12^5$          |
| 43  | 6     | $12^6$          |
| 47  | 7     | $12^7$          |
| 49  | 4     | $24^2$ , $12^2$ |
| 53  | 8     | $12^8$          |
| 59  | 8     | $12^8$          |
| 61  | 8     | $12^8$          |
| 67  | 10    | $12^{10}$       |
| 71  | 10    | $12^{10}$       |
| 73  | 11    | $12^{11}$       |
| 79  | 11    | $12^{11}$       |
| 81  | 4     | $24^2$ , $12^2$ |
| 83  | 13    | $12^{13}$       |
| 89  | 13    | $12^{13}$       |
| 97  | 15    | $12^{15}$       |
| 101 | 15    | $12^{15}$       |
| 103 | 16    | $12^{16}$       |
| 107 | 17    | $12^{17}$       |
| 109 | 16    | $12^{16}$       |
| 113 | 18    | $12^{18}$       |
| 121 | 11    | $24^4$ , $12^7$ |

Total: 248

## 6 Eckardt Points

| $q$ | total | Ago             |
|-----|-------|-----------------|
| 11  | 1     | $24$            |
| 13  | 1     | $24$            |
| 17  | 3     | $24^3$          |
| 19  | 2     | $24^2$          |
| 23  | 5     | $24^5$          |
| 25  | 3     | $48^2, 24$      |
| 27  | 2     | $24^2$          |
| 29  | 5     | $24^5$          |
| 31  | 5     | $24^5$          |
| 37  | 7     | $24^7$          |
| 41  | 8     | $24^8$          |
| 43  | 9     | $24^9$          |
| 47  | 11    | $24^{11}$       |
| 49  | 6     | $48^3, 24^3$    |
| 53  | 12    | $24^{12}$       |
| 59  | 13    | $24^{13}$       |
| 61  | 12    | $24^{12}$       |
| 67  | 15    | $24^{15}$       |
| 71  | 16    | $24^{16}$       |
| 73  | 16    | $24^{16}$       |
| 79  | 17    | $24^{17}$       |
| 81  | 6     | $48^3, 24^3$    |
| 83  | 20    | $24^{20}$       |
| 89  | 20    | $24^{20}$       |
| 97  | 22    | $24^{22}$       |
| 101 | 23    | $24^{23}$       |
| 103 | 24    | $24^{24}$       |
| 107 | 26    | $24^{26}$       |
| 109 | 24    | $24^{24}$       |
| 113 | 27    | $24^{27}$       |
| 121 | 17    | $48^7, 24^{10}$ |

Total: 378

**9 Eckardt Points**

| $q$ | total | Ago                   |
|-----|-------|-----------------------|
| 9   | 1     | 432                   |
| 13  | 1     | 108                   |
| 19  | 1     | 54                    |
| 25  | 2     | 216, 108              |
| 27  | 1     | 162                   |
| 31  | 2     | $54^2$                |
| 37  | 3     | 108, $54^2$           |
| 43  | 3     | $54^3$                |
| 49  | 3     | 216, 108, 54          |
| 61  | 5     | 108, $54^4$           |
| 67  | 5     | $54^5$                |
| 73  | 6     | 108, $54^5$           |
| 79  | 6     | $54^6$                |
| 81  | 3     | 864, 216, 108         |
| 97  | 8     | 108, $54^7$           |
| 103 | 8     | $54^8$                |
| 109 | 9     | 108, $54^8$           |
| 121 | 7     | 216, $108^3$ , $54^3$ |

Total: 74

**10 Eckardt Points**

| $q$ | total | Ago |
|-----|-------|-----|
| 9   | 1     | 240 |
| 11  | 1     | 120 |
| 19  | 1     | 120 |
| 29  | 1     | 120 |
| 31  | 1     | 120 |
| 41  | 1     | 120 |
| 49  | 1     | 240 |
| 59  | 1     | 120 |
| 61  | 1     | 120 |
| 71  | 1     | 120 |
| 79  | 1     | 120 |
| 81  | 1     | 480 |
| 89  | 1     | 120 |
| 101 | 1     | 120 |
| 109 | 1     | 120 |
| 121 | 1     | 240 |

Total: 16



**18 Eckardt Points**

| $q$ | total | Ago  |
|-----|-------|------|
| 7   | 1     | 648  |
| 13  | 1     | 648  |
| 19  | 1     | 648  |
| 25  | 1     | 1296 |
| 31  | 1     | 648  |
| 37  | 1     | 648  |
| 43  | 1     | 648  |
| 49  | 1     | 1296 |
| 61  | 1     | 648  |
| 67  | 1     | 648  |
| 73  | 1     | 648  |
| 79  | 1     | 648  |
| 97  | 1     | 648  |
| 103 | 1     | 648  |
| 109 | 1     | 648  |
| 121 | 1     | 1296 |

Total: 16



# Appendix B

## The Class Library

### B.1 Classes and Namespaces

|                   |   |          |                     |                                |                                                 |
|-------------------|---|----------|---------------------|--------------------------------|-------------------------------------------------|
| 0                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>a_domain</b>                                 |
| 1                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>algebra_global</b>                           |
| 2                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>generators_symplectic_group</b>              |
| 3                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>group_generators_domain</b>                  |
| 4                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>heisenberg</b>                               |
| 5                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>matrix_group</b>                             |
| 6                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>matrix_group_element</b>                     |
| 7                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>module</b>                                   |
| 8                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>null_polarity_generator</b>                  |
| 9                 | : | orbiter, | layer1_foundations, | algebra,                       | <b>rank_checker</b>                             |
| 10                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>algebraic_geometry_global</b>                |
| 11                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>arc_lifting_with_two_lines</b>               |
| 12                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>clebsch_map</b>                              |
| 13                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>cubic_curve</b>                              |
| 14                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>del_pezzo_surface_of_degree_two_domain</b>   |
| 15                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>del_pezzo_surface_of_degree_two_object</b>   |
| 16                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>eckardt_point</b>                            |
| 17                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>eckardt_point_info</b>                       |
| 18                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>kovalevski_points</b>                        |
| 19                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>quartic_curve_domain</b>                     |
| 20                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>quartic_curve_object</b>                     |
| 21                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>quartic_curve_object_properties</b>          |
| 22                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>schlaefli</b>                                |
| 23                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>schlaefli_double_six</b>                     |
| 24                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>schlaefli_labels</b>                         |
| 25                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>schlaefli_triangular_pairs</b>               |
| 26                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>schlaefli_tritangent_planes</b>              |
| 27                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>seventytwo_cases</b>                         |
| 28                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>smooth_surface_object_properties</b>         |
| 29                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>surface_domain</b>                           |
| 30                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>surface_object</b>                           |
| 31                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>surface_object_properties</b>                |
| 32                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>surface_polynomial_domains</b>               |
| 33                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>variety_object</b>                           |
| 34                | : | orbiter, | layer1_foundations, | algebraic_geometry,            | <b>web_of_cubic_curves</b>                      |
| 35                | : | orbiter, | layer1_foundations, | canonical_form_classification, | <b>classification_of_objects</b>                |
| 36                | : | orbiter, | layer1_foundations, | canonical_form_classification, | <b>classification_of_objects_description</b>    |
| <b>Table 11.5</b> |   |          |                     |                                |                                                 |
| 37                | : | orbiter, | layer1_foundations, | canonical_form_classification, | <b>classification_of_objects_report_options</b> |
| <b>Table 11.6</b> |   |          |                     |                                |                                                 |

38 : orbiter, layer1\_foundations, canonical\_form\_classification, **classify\_bitvectors**  
 39 : orbiter, layer1\_foundations, canonical\_form\_classification, **classify\_using\_canonical\_forms**  
 40 : orbiter, layer1\_foundations, canonical\_form\_classification, **data\_input\_stream**  
 41 : orbiter, layer1\_foundations, canonical\_form\_classification, **data\_input\_stream\_description**

**Table 11.3**

42 : orbiter, layer1\_foundations, canonical\_form\_classification, **data\_input\_stream\_description\_element**  
 43 : orbiter, layer1\_foundations, canonical\_form\_classification, **encoded\_combinatorial\_object**  
 44 : orbiter, layer1\_foundations, canonical\_form\_classification, **object\_with\_canonical\_form**  
 45 : orbiter, layer1\_foundations, coding\_theory, **code\_diagram**  
 46 : orbiter, layer1\_foundations, coding\_theory, **coding\_theory\_domain**  
 47 : orbiter, layer1\_foundations, coding\_theory, **crc\_code\_description** Table 10.9  
 48 : orbiter, layer1\_foundations, coding\_theory, **crc\_codes**  
 49 : orbiter, layer1\_foundations, coding\_theory, **crc\_object**  
 50 : orbiter, layer1\_foundations, coding\_theory, **crc\_options\_description** Table 10.8  
 51 : orbiter, layer1\_foundations, coding\_theory, **create\_BCH\_code**  
 52 : orbiter, layer1\_foundations, coding\_theory, **create\_RS\_code**  
 53 : orbiter, layer1\_foundations, coding\_theory, **cyclic\_codes**  
 54 : orbiter, layer1\_foundations, coding\_theory, **error\_pattern\_generator**  
 55 : orbiter, layer1\_foundations, coding\_theory, **error\_repository**  
 56 : orbiter, layer1\_foundations, coding\_theory, **ttp\_codes**  
 57 : orbiter, layer1\_foundations, combinatorics, **apn\_functions**  
 58 : orbiter, layer1\_foundations, combinatorics, **boolean\_function\_domain**  
 59 : orbiter, layer1\_foundations, combinatorics, **brick\_domain**  
 60 : orbiter, layer1\_foundations, combinatorics, **combinatorics\_domain**  
 61 : orbiter, layer1\_foundations, combinatorics, **decomposition**  
 62 : orbiter, layer1\_foundations, combinatorics, **decomposition\_scheme**  
 63 : orbiter, layer1\_foundations, combinatorics, **domino\_assignment**  
 64 : orbiter, layer1\_foundations, combinatorics, **domino\_change**  
 65 : orbiter, layer1\_foundations, combinatorics, **geo\_parameter**  
 66 : orbiter, layer1\_foundations, combinatorics, **pentomino\_puzzle**  
 67 : orbiter, layer1\_foundations, combinatorics, **polynomial\_function\_domain**  
 68 : orbiter, layer1\_foundations, combinatorics, **row\_and\_col\_partition**  
 69 : orbiter, layer1\_foundations, combinatorics, **tdo\_data**  
 70 : orbiter, layer1\_foundations, combinatorics, **tdo\_refinement**  
 71 : orbiter, layer1\_foundations, combinatorics, **tdo\_refinement\_description** Table 11.8  
 72 : orbiter, layer1\_foundations, combinatorics, **tdo\_scheme\_compute**  
 73 : orbiter, layer1\_foundations, combinatorics, **tdo\_scheme\_synthetic**  
 74 : orbiter, layer1\_foundations, cryptography, **cryptography\_domain**  
 75 : orbiter, layer1\_foundations, data\_structures, **algorithms**  
 76 : orbiter, layer1\_foundations, data\_structures, **ancestry\_family**  
 77 : orbiter, layer1\_foundations, data\_structures, **ancestry\_indi**  
 78 : orbiter, layer1\_foundations, data\_structures, **ancestry\_tree**  
 79 : orbiter, layer1\_foundations, data\_structures, **bitmatrix**  
 80 : orbiter, layer1\_foundations, data\_structures, **bitvector**  
 81 : orbiter, layer1\_foundations, data\_structures, **data\_file**  
 82 : orbiter, layer1\_foundations, data\_structures, **data\_structures\_global**  
 83 : orbiter, layer1\_foundations, data\_structures, **fancy\_set**  
 84 : orbiter, layer1\_foundations, data\_structures, **int\_matrix**  
 85 : orbiter, layer1\_foundations, data\_structures, **int\_vec**  
 86 : orbiter, layer1\_foundations, data\_structures, **int\_vector**  
 87 : orbiter, layer1\_foundations, data\_structures, **lint\_vec**  
 88 : orbiter, layer1\_foundations, data\_structures, **page\_storage**  
 89 : orbiter, layer1\_foundations, data\_structures, **partitionstack**  
 90 : orbiter, layer1\_foundations, data\_structures, **set\_builder**  
 91 : orbiter, layer1\_foundations, data\_structures, **set\_builder\_description** Table 2.7  
 92 : orbiter, layer1\_foundations, data\_structures, **set\_of\_sets**  
 93 : orbiter, layer1\_foundations, data\_structures, **set\_of\_sets\_lint**  
 94 : orbiter, layer1\_foundations, data\_structures, **sorting**  
 95 : orbiter, layer1\_foundations, data\_structures, **spreadsheet**

96 : orbiter, layer1\_foundations, data\_structures, string\_tools  
 97 : orbiter, layer1\_foundations, data\_structures, tally  
 98 : orbiter, layer1\_foundations, data\_structures, tally\_lint  
 99 : orbiter, layer1\_foundations, data\_structures, tally\_vector\_data  
 100 : orbiter, layer1\_foundations, data\_structures, vector\_builder  
 101 : orbiter, layer1\_foundations, data\_structures, vector\_builder\_description Table 2.8  
 102 : orbiter, layer1\_foundations, data\_structures, vector\_hashing  
 103 : orbiter, layer1\_foundations, expression\_parser, formula  
 104 : orbiter, layer1\_foundations, expression\_parser, formula\_vector  
 105 : orbiter, layer1\_foundations, expression\_parser, symbolic\_object\_activity  
 106 : orbiter, layer1\_foundations, expression\_parser, symbolic\_object\_activity\_description Table 2.12  
 107 : orbiter, layer1\_foundations, expression\_parser, symbolic\_object\_builder  
 108 : orbiter, layer1\_foundations, expression\_parser, symbolic\_object\_builder\_description Table 2.10,  
 Table 2.11  
 109 : orbiter, layer1\_foundations, expression\_parser, syntax\_tree  
 110 : orbiter, layer1\_foundations, expression\_parser, syntax\_tree\_latex  
 111 : orbiter, layer1\_foundations, expression\_parser, syntax\_tree\_node  
 112 : orbiter, layer1\_foundations, expression\_parser, syntax\_tree\_node\_terminal  
 113 : orbiter, layer1\_foundations, field\_theory, finite\_field  
 114 : orbiter, layer1\_foundations, field\_theory, finite\_field\_activity  
 115 : orbiter, layer1\_foundations, field\_theory, finite\_field\_activity\_description Table 3.2, Table 3.3,  
 Table 3.5, Table 5.1, Table 3.6, Table 3.7  
 116 : orbiter, layer1\_foundations, field\_theory, finite\_field\_description Table 3.1  
 117 : orbiter, layer1\_foundations, field\_theory, finite\_field\_implementation\_by\_tables  
 118 : orbiter, layer1\_foundations, field\_theory, finite\_field\_implementation\_wo\_tables  
 119 : orbiter, layer1\_foundations, field\_theory, finite\_field\_io  
 120 : orbiter, layer1\_foundations, field\_theory, minimum\_polynomial  
 121 : orbiter, layer1\_foundations, field\_theory, norm\_tables  
 122 : orbiter, layer1\_foundations, field\_theory, normal\_basis  
 123 : orbiter, layer1\_foundations, field\_theory, nth\_roots  
 124 : orbiter, layer1\_foundations, field\_theory, related\_fields  
 125 : orbiter, layer1\_foundations, field\_theory, square\_nonsquare  
 126 : orbiter, layer1\_foundations, field\_theory, subfield\_structure  
 127 : orbiter, layer1\_foundations, geometry, W3q  
 128 : orbiter, layer1\_foundations, geometry, andre\_construction  
 129 : orbiter, layer1\_foundations, geometry, andre\_construction\_line\_element  
 130 : orbiter, layer1\_foundations, geometry, andre\_construction\_point\_element  
 131 : orbiter, layer1\_foundations, geometry, arc\_basic  
 132 : orbiter, layer1\_foundations, geometry, arc\_in\_projective\_space  
 133 : orbiter, layer1\_foundations, geometry, buenhout\_metz  
 134 : orbiter, layer1\_foundations, geometry, desarguesian\_spread  
 135 : orbiter, layer1\_foundations, geometry, flag  
 136 : orbiter, layer1\_foundations, geometry, geometric\_object\_create  
 137 : orbiter, layer1\_foundations, geometry, geometric\_object\_description Table 4.12, Table 4.13  
 138 : orbiter, layer1\_foundations, geometry, geometry\_global  
 139 : orbiter, layer1\_foundations, geometry, grassmann  
 140 : orbiter, layer1\_foundations, geometry, grassmann\_embedded  
 141 : orbiter, layer1\_foundations, geometry, hermitian  
 142 : orbiter, layer1\_foundations, geometry, hjelmslev  
 143 : orbiter, layer1\_foundations, geometry, incidence\_structure  
 144 : orbiter, layer1\_foundations, geometry, intersection\_type  
 145 : orbiter, layer1\_foundations, geometry, klein\_correspondence  
 146 : orbiter, layer1\_foundations, geometry, knarr  
 147 : orbiter, layer1\_foundations, geometry, point\_line  
 148 : orbiter, layer1\_foundations, geometry, points\_and\_lines  
 149 : orbiter, layer1\_foundations, geometry, polarity  
 150 : orbiter, layer1\_foundations, geometry, projective\_space  
 151 : orbiter, layer1\_foundations, geometry, projective\_space\_basic  
 152 : orbiter, layer1\_foundations, geometry, projective\_space\_implementation

153 : orbiter, layer1\_foundations, geometry, **projective\_space\_of\_dimension\_three**  
 154 : orbiter, layer1\_foundations, geometry, **projective\_space\_plane**  
 155 : orbiter, layer1\_foundations, geometry, **projective\_space\_reporting**  
 156 : orbiter, layer1\_foundations, geometry, **projective\_space\_subspaces**  
 157 : orbiter, layer1\_foundations, geometry, **spread\_domain**  
 158 : orbiter, layer1\_foundations, geometry, **spread\_tables**  
 159 : orbiter, layer1\_foundations, geometry, **three\_skew\_subspaces**  
 160 : orbiter, layer1\_foundations, geometry\_builder, **cperm**  
 161 : orbiter, layer1\_foundations, geometry\_builder, **decomposition\_with\_fuse**  
 162 : orbiter, layer1\_foundations, geometry\_builder, **gen\_geo**  
 163 : orbiter, layer1\_foundations, geometry\_builder, **gen\_geo\_conf**  
 164 : orbiter, layer1\_foundations, geometry\_builder, **geometric\_backtrack\_search**  
 165 : orbiter, layer1\_foundations, geometry\_builder, **geometry\_builder**  
 166 : orbiter, layer1\_foundations, geometry\_builder, **geometry\_builder\_description** Table 11.7  
 167 : orbiter, layer1\_foundations, geometry\_builder, **girth\_test**  
 168 : orbiter, layer1\_foundations, geometry\_builder, **inc\_encoding**  
 169 : orbiter, layer1\_foundations, geometry\_builder, **incidence**  
 170 : orbiter, layer1\_foundations, geometry\_builder, **iso\_type**  
 171 : orbiter, layer1\_foundations, geometry\_builder, **test\_semicanonical**  
 172 : orbiter, layer1\_foundations, graph\_theory, **clique\_finder**  
 173 : orbiter, layer1\_foundations, graph\_theory, **clique\_finder\_control** Table 15.1  
 174 : orbiter, layer1\_foundations, graph\_theory, **colored\_graph**  
 175 : orbiter, layer1\_foundations, graph\_theory, **graph\_layer**  
 176 : orbiter, layer1\_foundations, graph\_theory, **graph\_node**  
 177 : orbiter, layer1\_foundations, graph\_theory, **graph\_theory\_domain**  
 178 : orbiter, layer1\_foundations, graph\_theory, **layered\_graph**  
 179 : orbiter, layer1\_foundations, graph\_theory, **rainbow\_cliques**  
 180 : orbiter, layer1\_foundations, graphics, **animate**  
 181 : orbiter, layer1\_foundations, graphics, **draw\_bitmap\_control** Table 16.2  
 182 : orbiter, layer1\_foundations, graphics, **draw\_incidence\_structure\_description** Table 16.3  
 183 : orbiter, layer1\_foundations, graphics, **draw\_mod\_n\_description**  
 184 : orbiter, layer1\_foundations, graphics, **draw\_projective\_curve\_description**  
 185 : orbiter, layer1\_foundations, graphics, **drawable\_set\_of\_objects**  
 186 : orbiter, layer1\_foundations, graphics, **graphical\_output**  
 187 : orbiter, layer1\_foundations, graphics, **layered\_graph\_draw\_options** Table 17.1, Table 17.2  
 188 : orbiter, layer1\_foundations, graphics, **mp\_graphics**  
 189 : orbiter, layer1\_foundations, graphics, **parametric\_curve**  
 190 : orbiter, layer1\_foundations, graphics, **parametric\_curve\_point**  
 191 : orbiter, layer1\_foundations, graphics, **plot\_tools**  
 192 : orbiter, layer1\_foundations, graphics, **povray\_job\_description** Table 17.4  
 193 : orbiter, layer1\_foundations, graphics, **scene** Table 17.9, Table 17.7, Table 17.8  
 194 : orbiter, layer1\_foundations, graphics, **scene\_element\_of\_type\_edge**  
 195 : orbiter, layer1\_foundations, graphics, **scene\_element\_of\_type\_face**  
 196 : orbiter, layer1\_foundations, graphics, **scene\_element\_of\_type\_line**  
 197 : orbiter, layer1\_foundations, graphics, **scene\_element\_of\_type\_plane**  
 198 : orbiter, layer1\_foundations, graphics, **scene\_element\_of\_type\_point**  
 199 : orbiter, layer1\_foundations, graphics, **scene\_element\_of\_type\_surface**  
 200 : orbiter, layer1\_foundations, graphics, **tree**  
 201 : orbiter, layer1\_foundations, graphics, **tree\_draw\_options** Table 19.3  
 202 : orbiter, layer1\_foundations, graphics, **tree\_node**  
 203 : orbiter, layer1\_foundations, graphics, **video\_draw\_options** Table 17.5, Table 17.6  
 204 : orbiter, layer1\_foundations, knowledge\_base, **knowledge\_base**  
 205 : orbiter, layer1\_foundations, l1\_interfaces, **easy\_BMP\_interface**  
 206 : orbiter, layer1\_foundations, l1\_interfaces, **expression\_parser\_sajeeb**  
 207 : orbiter, layer1\_foundations, l1\_interfaces, **expression\_parser\_sajeeb\_private\_data**  
 208 : orbiter, layer1\_foundations, l1\_interfaces, **interface\_gap\_low**  
 209 : orbiter, layer1\_foundations, l1\_interfaces, **interface\_magma\_low**  
 210 : orbiter, layer1\_foundations, l1\_interfaces, **latex\_interface**  
 211 : orbiter, layer1\_foundations, l1\_interfaces, **nauty\_interface**

```

212 : orbiter, layer1_foundations, l1_interfaces, nauty_output
213 : orbiter, layer1_foundations, l1_interfaces, povray_interface
214 : orbiter, layer1_foundations, linear_algebra, gl_class_rep
215 : orbiter, layer1_foundations, linear_algebra, gl_classes
216 : orbiter, layer1_foundations, linear_algebra, linear_algebra
217 : orbiter, layer1_foundations, linear_algebra, linear_algebra_global
218 : orbiter, layer1_foundations, linear_algebra, matrix_block_data
219 : orbiter, layer1_foundations, linear_algebra, representation_theory_domain
220 : orbiter, layer1_foundations, linear_algebra, vector_space
221 : orbiter, layer1_foundations, number_theory, cyclotomic_sets
222 : orbiter, layer1_foundations, number_theory, elliptic_curve
223 : orbiter, layer1_foundations, number_theory, number_theoretic_transform
224 : orbiter, layer1_foundations, number_theory, number_theory_domain
225 : orbiter, layer1_foundations, orbiter_kernel_system, create_file_description
226 : orbiter, layer1_foundations, orbiter_kernel_system, csv_file_support
227 : orbiter, layer1_foundations, orbiter_kernel_system, file_io
228 : orbiter, layer1_foundations, orbiter_kernel_system, file_output
229 : orbiter, layer1_foundations, orbiter_kernel_system, mem_object_registry
230 : orbiter, layer1_foundations, orbiter_kernel_system, mem_object_registry_entry
231 : orbiter, layer1_foundations, orbiter_kernel_system, memory_object
232 : orbiter, layer1_foundations, orbiter_kernel_system, numerics
233 : orbiter, layer1_foundations, orbiter_kernel_system, orbiter_data_file
234 : orbiter, layer1_foundations, orbiter_kernel_system, orbiter_session Table 2.1
235 : orbiter, layer1_foundations, orbiter_kernel_system, orbiter_symbol_table
236 : orbiter, layer1_foundations, orbiter_kernel_system, orbiter_symbol_table_entry
237 : orbiter, layer1_foundations, orbiter_kernel_system, os_interface
238 : orbiter, layer1_foundations, orbiter_kernel_system, override_double
239 : orbiter, layer1_foundations, orbiter_kernel_system, prepare_frames Table 17.10
240 : orbiter, layer1_foundations, orthogonal_geometry, blt_set_domain
241 : orbiter, layer1_foundations, orthogonal_geometry, blt_set_invariants
242 : orbiter, layer1_foundations, orthogonal_geometry, hyperbolic_pair
243 : orbiter, layer1_foundations, orthogonal_geometry, linear_complex
244 : orbiter, layer1_foundations, orthogonal_geometry, orthogonal
245 : orbiter, layer1_foundations, orthogonal_geometry, orthogonal_global
246 : orbiter, layer1_foundations, orthogonal_geometry, orthogonal_group
247 : orbiter, layer1_foundations, orthogonal_geometry, orthogonal_indexing
248 : orbiter, layer1_foundations, orthogonal_geometry, orthogonal_plane_invariant
249 : orbiter, layer1_foundations, orthogonal_geometry, quadratic_form
250 : orbiter, layer1_foundations, orthogonal_geometry, quadratic_form_list_coding
251 : orbiter, layer1_foundations, orthogonal_geometry, unusual_model
252 : orbiter, layer1_foundations, polish, function_command
253 : orbiter, layer1_foundations, polish, function_polish
254 : orbiter, layer1_foundations, polish, function_polish_description
255 : orbiter, layer1_foundations, ring_theory, finite_ring
256 : orbiter, layer1_foundations, ring_theory, homogeneous_polynomial_domain
257 : orbiter, layer1_foundations, ring_theory, longinteger_domain
258 : orbiter, layer1_foundations, ring_theory, longinteger_object
259 : orbiter, layer1_foundations, ring_theory, partial_derivative
260 : orbiter, layer1_foundations, ring_theory, polynomial_double
261 : orbiter, layer1_foundations, ring_theory, polynomial_double_domain
262 : orbiter, layer1_foundations, ring_theory, polynomial_ring_activity_description Table 5.3
263 : orbiter, layer1_foundations, ring_theory, polynomial_ring_description Table 5.2
264 : orbiter, layer1_foundations, ring_theory, ring_theory_global
265 : orbiter, layer1_foundations, ring_theory, table_of_irreducible_polynomials
266 : orbiter, layer1_foundations, ring_theory, unipoly_domain
267 : orbiter, layer1_foundations, solvers, diophant
268 : orbiter, layer1_foundations, solvers, diophant_activity
269 : orbiter, layer1_foundations, solvers, diophant_activity_description Table 15.3
270 : orbiter, layer1_foundations, solvers, diophant_create

```

271 : orbiter, layer1\_foundations, solvers, **diophant\_description** Table 15.2  
 272 : orbiter, layer1\_foundations, solvers, **dlx\_problem\_description**  
 273 : orbiter, layer1\_foundations, solvers, **dlx\_solver**  
 274 : orbiter, layer1\_foundations, solvers, mckay, **tMCKAY**  
 275 : orbiter, layer2\_discreta, typed\_objects, **Vector**  
 276 : orbiter, layer2\_discreta, typed\_objects, **bt\_key**  
 277 : orbiter, layer2\_discreta, typed\_objects, **btree**  
 278 : orbiter, layer2\_discreta, typed\_objects, **database**  
 279 : orbiter, layer2\_discreta, typed\_objects, **design\_parameter**  
 280 : orbiter, layer2\_discreta, typed\_objects, **design\_parameter\_source**  
 281 : orbiter, layer2\_discreta, typed\_objects, **discreta\_base**  
 282 : orbiter, layer2\_discreta, typed\_objects, **discreta\_matrix**  
 283 : orbiter, layer2\_discreta, typed\_objects, **domain**  
 284 : orbiter, layer2\_discreta, typed\_objects, **hollerith**  
 285 : orbiter, layer2\_discreta, typed\_objects, **integer**  
 286 : orbiter, layer2\_discreta, typed\_objects, **longinteger**  
 287 : orbiter, layer2\_discreta, typed\_objects, **matrix\_access**  
 288 : orbiter, layer2\_discreta, typed\_objects, **memory**  
 289 : orbiter, layer2\_discreta, typed\_objects, **number\_partition**  
 290 : orbiter, layer2\_discreta, typed\_objects, **page\_table**  
 291 : orbiter, layer2\_discreta, typed\_objects, **permutation**  
 292 : orbiter, layer2\_discreta, typed\_objects, **printing\_mode**  
 293 : orbiter, layer2\_discreta, typed\_objects, **unipoly**  
 294 : orbiter, layer2\_discreta, typed\_objects, **with**  
 295 : orbiter, layer3\_group\_actions, actions, **action**  
 296 : orbiter, layer3\_group\_actions, actions, **action\_global**  
 297 : orbiter, layer3\_group\_actions, actions, **action\_pointer\_table**  
 298 : orbiter, layer3\_group\_actions, actions, **group\_element**  
 299 : orbiter, layer3\_group\_actions, actions, **induced\_action**  
 300 : orbiter, layer3\_group\_actions, actions, **known\_groups**  
 301 : orbiter, layer3\_group\_actions, actions, **stabilizer\_chain\_base\_data**  
 302 : orbiter, layer3\_group\_actions, combinatorics\_with\_groups, **combinatorics\_with\_action**  
 303 : orbiter, layer3\_group\_actions, combinatorics\_with\_groups, **flag\_orbits\_incidence\_structure**  
 304 : orbiter, layer3\_group\_actions, combinatorics\_with\_groups, **group\_action\_on\_combinatorial\_object**  
 305 : orbiter, layer3\_group\_actions, combinatorics\_with\_groups, **incidence\_structure\_with\_group**  
 306 : orbiter, layer3\_group\_actions, combinatorics\_with\_groups, **orbit\_type\_repository**  
 307 : orbiter, layer3\_group\_actions, combinatorics\_with\_groups, **translation\_plane\_via\_andre\_model**  
 308 : orbiter, layer3\_group\_actions, data\_structures\_groups, **group\_container**  
 309 : orbiter, layer3\_group\_actions, data\_structures\_groups, **orbit\_rep**  
 310 : orbiter, layer3\_group\_actions, data\_structures\_groups, **orbit\_transversal**  
 311 : orbiter, layer3\_group\_actions, data\_structures\_groups, **schreier\_vector**  
 312 : orbiter, layer3\_group\_actions, data\_structures\_groups, **schreier\_vector\_handler**  
 313 : orbiter, layer3\_group\_actions, data\_structures\_groups, **set\_and\_stabilizer**  
 314 : orbiter, layer3\_group\_actions, data\_structures\_groups, **union\_find**  
 315 : orbiter, layer3\_group\_actions, data\_structures\_groups, **union\_find\_on\_k\_subsets**  
 316 : orbiter, layer3\_group\_actions, data\_structures\_groups, **vector\_ge**  
 317 : orbiter, layer3\_group\_actions, data\_structures\_groups, **vector\_ge\_description** Table 2.9  
 318 : orbiter, layer3\_group\_actions, group\_constructions, **direct\_product**  
 319 : orbiter, layer3\_group\_actions, group\_constructions, **linear\_group**  
 320 : orbiter, layer3\_group\_actions, group\_constructions, **linear\_group\_description** Table 6.2, Table 6.3,  
**Table 6.7, Table 6.5**  
 321 : orbiter, layer3\_group\_actions, group\_constructions, **permutation\_group\_create**  
 322 : orbiter, layer3\_group\_actions, group\_constructions, **permutation\_group\_description** Table 6.1  
 323 : orbiter, layer3\_group\_actions, group\_constructions, **permutation\_representation**  
 324 : orbiter, layer3\_group\_actions, group\_constructions, **permutation\_representation\_domain**  
 325 : orbiter, layer3\_group\_actions, group\_constructions, **polarity\_extension**  
 326 : orbiter, layer3\_group\_actions, group\_constructions, **wreath\_product**  
 327 : orbiter, layer3\_group\_actions, groups, **conjugacy\_class\_of\_elements**  
 328 : orbiter, layer3\_group\_actions, groups, **exceptional\_isomorphism\_O4**



```

329 : orbiter, layer3.group.actions, groups, orbits_on_something
330 : orbiter, layer3.group.actions, groups, schreier
331 : orbiter, layer3.group.actions, groups, schreier_sims
332 : orbiter, layer3.group.actions, groups, sims
333 : orbiter, layer3.group.actions, groups, strong_generators
334 : orbiter, layer3.group.actions, groups, subgroup
335 : orbiter, layer3.group.actions, groups, sylow_structure
336 : orbiter, layer3.group.actions, induced_actions, action_by_conjugation
337 : orbiter, layer3.group.actions, induced_actions, action_by_representation
338 : orbiter, layer3.group.actions, induced_actions, action_by_restriction
339 : orbiter, layer3.group.actions, induced_actions, action_by_right_multiplication
340 : orbiter, layer3.group.actions, induced_actions, action_by_subfield_structure
341 : orbiter, layer3.group.actions, induced_actions, action_on_andre
342 : orbiter, layer3.group.actions, induced_actions, action_on_bricks
343 : orbiter, layer3.group.actions, induced_actions, action_on_cosets
344 : orbiter, layer3.group.actions, induced_actions, action_on_determinant
345 : orbiter, layer3.group.actions, induced_actions, action_on_factor_space
346 : orbiter, layer3.group.actions, induced_actions, action_on_flags
347 : orbiter, layer3.group.actions, induced_actions, action_on_galois_group
348 : orbiter, layer3.group.actions, induced_actions, action_on_grassmannian
349 : orbiter, layer3.group.actions, induced_actions, action_on_homogeneous_polynomials
350 : orbiter, layer3.group.actions, induced_actions, action_on_interior_direct_product
351 : orbiter, layer3.group.actions, induced_actions, action_on_k_subsets
352 : orbiter, layer3.group.actions, induced_actions, action_on_module
353 : orbiter, layer3.group.actions, induced_actions, action_on_orbits
354 : orbiter, layer3.group.actions, induced_actions, action_on_orthogonal
355 : orbiter, layer3.group.actions, induced_actions, action_on_set_partitions
356 : orbiter, layer3.group.actions, induced_actions, action_on_sets
357 : orbiter, layer3.group.actions, induced_actions, action_on_sign
358 : orbiter, layer3.group.actions, induced_actions, action_on_spread_set
359 : orbiter, layer3.group.actions, induced_actions, action_on_subgroups
360 : orbiter, layer3.group.actions, induced_actions, action_on_wedge_product
361 : orbiter, layer3.group.actions, induced_actions, product_action
362 : orbiter, layer3.group.actions, interfaces, conjugacy_classes_and_normalizers
363 : orbiter, layer3.group.actions, interfaces, l3_interface_gap
364 : orbiter, layer3.group.actions, interfaces, magma_interface
365 : orbiter, layer3.group.actions, interfaces, nauty_interface_with_group
366 : orbiter, layer4.classification, invariant_relations, classification_step
367 : orbiter, layer4.classification, invariant_relations, flag_orbit_node
368 : orbiter, layer4.classification, invariant_relations, flag_orbits
369 : orbiter, layer4.classification, invariant_relations, orbit_node
370 : orbiter, layer4.classification, isomorph, flag_orbit_folding
371 : orbiter, layer4.classification, isomorph, isomorph
372 : orbiter, layer4.classification, isomorph, isomorph_arguments Table 14.7
373 : orbiter, layer4.classification, isomorph, isomorph_global
374 : orbiter, layer4.classification, isomorph, isomorph_worker
375 : orbiter, layer4.classification, isomorph, representatives
376 : orbiter, layer4.classification, isomorph, substructure_classification
377 : orbiter, layer4.classification, isomorph, substructure_lifting_data
378 : orbiter, layer4.classification, orbits_schreier, orbit_of_equations
379 : orbiter, layer4.classification, orbits_schreier, orbit_of_sets
380 : orbiter, layer4.classification, orbits_schreier, orbit_of_subspaces
381 : orbiter, layer4.classification, poset_classification, classification_base_case
382 : orbiter, layer4.classification, poset_classification, extension
383 : orbiter, layer4.classification, poset_classification, orbit_based_testing
384 : orbiter, layer4.classification, poset_classification, orbit_tracer
385 : orbiter, layer4.classification, poset_classification, poset_classification
386 : orbiter, layer4.classification, poset_classification, poset_classification_activity
387 : orbiter, layer4.classification, poset_classification, poset_classification_activity_description

```

Table 7.4

|     |   |                                                                                                                                                                                   |           |
|-----|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 388 | : | orbiter, layer4.classification, poset.classification, <b>poset.classification_control</b>                                                                                         | Table 7.3 |
| 389 | : | orbiter, layer4.classification, poset.classification, <b>poset.classification_report_options</b>                                                                                  | Table 7.5 |
| 390 | : | orbiter, layer4.classification, poset.classification, <b>poset_of_orbits</b>                                                                                                      |           |
| 391 | : | orbiter, layer4.classification, poset.classification, <b>poset_orbit_node</b>                                                                                                     |           |
| 392 | : | orbiter, layer4.classification, poset.classification, <b>poset_with_group_action</b>                                                                                              |           |
| 393 | : | orbiter, layer4.classification, poset.classification, <b>upstep_work</b>                                                                                                          |           |
| 394 | : | orbiter, layer4.classification, set_stabilizer, <b>compute_stabilizer</b>                                                                                                         |           |
| 395 | : | orbiter, layer4.classification, set_stabilizer, <b>stabilizer_orbits_and_types</b>                                                                                                |           |
| 396 | : | orbiter, layer4.classification, set_stabilizer, <b>substructure_classifier</b>                                                                                                    |           |
| 397 | : | orbiter, layer4.classification, set_stabilizer, <b>substructure_stats_and_selection</b>                                                                                           |           |
| 398 | : | orbiter, layer4.classification, solvers.package, <b>exact_cover</b>                                                                                                               |           |
| 399 | : | orbiter, layer4.classification, solvers.package, <b>exact_cover_arguments</b>                                                                                                     |           |
| 400 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>arc_lifting</b>                                                                     |           |
| 401 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>arc_orbits_on_pairs</b>                                                             |           |
| 402 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>arc_partition</b>                                                                   |           |
| 403 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>classify_triangular_pairs</b>                                                       |           |
| 404 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>six_arcs_not_on_a_conic</b>                                                         |           |
| 405 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>surface_classify_using_arc</b>                                                      |           |
| 406 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>surface_create_by_arc_lifting</b>                                                   |           |
| 407 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>surfaces_arc_lifting</b>                                                            |           |
| 408 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>surfaces_arc_lifting_definition_node</b>                                            |           |
| 409 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>surfaces_arc_lifting_trace</b>                                                      |           |
| 410 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>surfaces_arc_lifting_upstep</b>                                                     |           |
| 411 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.arcs, <b>triangular_pair_with_action</b>                                                     |           |
| 412 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>classification_of_cubic_surfaces_with_double_sixes_activity</b>             |           |
| 413 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>classification_of_cubic_surfaces_with_double_sixes_activity_description</b> | Table 8.6 |
| 414 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>classify_double_sixes</b>                                                   |           |
| 415 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>classify_five_plus_one</b>                                                  |           |
| 416 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>identify_cubic_surface</b>                                                  |           |
| 417 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>surface_classify_wedge</b>                                                  |           |
| 418 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.and.double.sixes, <b>surface_repository</b>                                                      |           |
| 419 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.in.general, <b>cubic_surface_activity</b>                                                        |           |
| 420 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.in.general, <b>cubic_surface_activity_description</b>                                            | Table 8.4 |
| 421 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.in.general, <b>surface_clebsch_map</b>                                                           |           |
| 422 | : | orbiter, layer5.applications, applications.in.algebraic.geometry, cubic_surfaces.in.general, <b>surface_create</b>                                                                |           |

423 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, cubic\_surfaces\_in\_general, **surface\_create\_description** Table 8.1, Table 8.2, Table 8.3

424 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, cubic\_surfaces\_in\_general, **surface\_domain\_high\_level**

425 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, cubic\_surfaces\_in\_general, **surface\_object\_with\_group**

426 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, cubic\_surfaces\_in\_general, **surface\_study**

427 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, cubic\_surfaces\_in\_general, **surface\_with\_action**

428 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, cubic\_surfaces\_in\_general, **table\_of\_surfaces**

429 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_activity**

430 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_activity\_description** Table 8.9

431 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_create**

432 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_create\_description** Table 8.8

433 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_domain\_with\_action**

434 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_from\_surface**

435 : orbiter, layer5.applications, applications\_in\_algebraic\_geometry, quartic\_curves, **quartic\_curve\_object\_with\_group**

436 : orbiter, layer5.applications, apps\_algebra, **action\_on\_forms**

437 : orbiter, layer5.applications, apps\_algebra, **action\_on\_forms\_activity**

438 : orbiter, layer5.applications, apps\_algebra, **action\_on\_forms\_activity\_description**

439 : orbiter, layer5.applications, apps\_algebra, **action\_on\_forms\_description**

440 : orbiter, layer5.applications, apps\_algebra, **algebra\_global\_with\_action**

441 : orbiter, layer5.applications, apps\_algebra, **any\_group**

442 : orbiter, layer5.applications, apps\_algebra, **character\_table\_burnside**

443 : orbiter, layer5.applications, apps\_algebra, **element\_processing\_description** Table 6.11

444 : orbiter, layer5.applications, apps\_algebra, **group\_modification\_description** Table 6.6

445 : orbiter, layer5.applications, apps\_algebra, **group\_theoretic\_activity**

446 : orbiter, layer5.applications, apps\_algebra, **group\_theoretic\_activity\_description** Table 6.8, Table 6.9, Table 6.10, Table 6.13, Table 6.12, Table 9.4

447 : orbiter, layer5.applications, apps\_algebra, **modified\_group\_create**

448 : orbiter, layer5.applications, apps\_algebra, **polynomial\_ring\_activity**

449 : orbiter, layer5.applications, apps\_algebra, **vector\_ge\_builder**

450 : orbiter, layer5.applications, apps\_algebra, **young**

451 : orbiter, layer5.applications, apps\_coding\_theory, **code\_modification\_description** Table 10.6

452 : orbiter, layer5.applications, apps\_coding\_theory, **coding\_theoretic\_activity**

453 : orbiter, layer5.applications, apps\_coding\_theory, **coding\_theoretic\_activity\_description** Table 10.3, Table 10.4, Table 10.5

454 : orbiter, layer5.applications, apps\_coding\_theory, **crc\_process**

455 : orbiter, layer5.applications, apps\_coding\_theory, **crc\_process\_description** Table 10.7

456 : orbiter, layer5.applications, apps\_coding\_theory, **create\_code**

457 : orbiter, layer5.applications, apps\_coding\_theory, **create\_code\_description** Table 10.2

458 : orbiter, layer5.applications, apps\_combinatorics, **boolean\_function\_classify**

459 : orbiter, layer5.applications, apps\_combinatorics, **combinatorial\_object**

460 : orbiter, layer5.applications, apps\_combinatorics, **combinatorial\_object\_activity**

461 : orbiter, layer5.applications, apps\_combinatorics, **combinatorial\_object\_activity\_description** Table 11.4

462 : orbiter, layer5.applications, apps\_combinatorics, **combinatorics\_global**

463 : orbiter, layer5.applications, apps\_combinatorics, **delandtsheer\_doyen**

464 : orbiter, layer5.applications, apps\_combinatorics, **delandtsheer\_doyen\_description** Table 13.6

465 : orbiter, layer5.applications, apps\_combinatorics, **design\_activity**

466 : orbiter, layer5\_applications, apps\_combinatorics, **design\_activity\_description** Table 13.2  
 467 : orbiter, layer5\_applications, apps\_combinatorics, **design\_create**  
 468 : orbiter, layer5\_applications, apps\_combinatorics, **design\_create\_description** Table 13.1  
 469 : orbiter, layer5\_applications, apps\_combinatorics, **design\_tables**  
 470 : orbiter, layer5\_applications, apps\_combinatorics, **difference\_set\_in\_heisenberg\_group**  
 471 : orbiter, layer5\_applications, apps\_combinatorics, **hadamard\_classify**  
 472 : orbiter, layer5\_applications, apps\_combinatorics, **hall\_system\_classify**  
 473 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_activity**  
 474 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_activity\_description**  
 475 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_classify**  
 476 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_was**  
 477 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_was\_activity**  
 478 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_was\_activity\_description** Table 13.5  
 479 : orbiter, layer5\_applications, apps\_combinatorics, **large\_set\_was\_description**  
 480 : orbiter, layer5\_applications, apps\_combinatorics, **regular\_linear\_space\_description**  
 481 : orbiter, layer5\_applications, apps\_combinatorics, **regular\_ls\_classify**  
 482 : orbiter, layer5\_applications, apps\_combinatorics, **tactical\_decomposition**  
 483 : orbiter, layer5\_applications, apps\_geometry, **arc\_generator**  
 484 : orbiter, layer5\_applications, apps\_geometry, **arc\_generator\_description** Table 7.6  
 485 : orbiter, layer5\_applications, apps\_geometry, **arc\_lifting\_simeon**  
 486 : orbiter, layer5\_applications, apps\_geometry, **choose\_points\_or\_lines**  
 487 : orbiter, layer5\_applications, apps\_geometry, **classify\_cubic\_curves**  
 488 : orbiter, layer5\_applications, apps\_geometry, **cubic\_curve\_with\_action**  
 489 : orbiter, layer5\_applications, apps\_geometry, **hermitian\_spreads\_classify**  
 490 : orbiter, layer5\_applications, apps\_geometry, **linear\_set\_classify**  
 491 : orbiter, layer5\_applications, apps\_geometry, **mapping**  
 492 : orbiter, layer5\_applications, apps\_geometry, **mapping\_description** Table 5.6  
 493 : orbiter, layer5\_applications, apps\_geometry, **ovoid\_classify**  
 494 : orbiter, layer5\_applications, apps\_geometry, **ovoid\_classify\_description**  
 495 : orbiter, layer5\_applications, apps\_geometry, **polar**  
 496 : orbiter, layer5\_applications, apps\_geometry, **search\_blocking\_set**  
 497 : orbiter, layer5\_applications, apps\_geometry, **singer\_cycle**  
 498 : orbiter, layer5\_applications, apps\_geometry, **tensor\_classify**  
 499 : orbiter, layer5\_applications, apps\_geometry, **top\_level\_geometry\_global**  
 500 : orbiter, layer5\_applications, apps\_graph\_theory, **cayley\_graph\_search**  
 501 : orbiter, layer5\_applications, apps\_graph\_theory, **create\_graph**  
 502 : orbiter, layer5\_applications, apps\_graph\_theory, **create\_graph\_description** Table 12.1, Table 12.2  
 503 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_classification\_activity**  
 504 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_classification\_activity\_description**  
**Table 12.6**  
 505 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_classify**  
 506 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_classify\_description** Table 12.5  
 507 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_modification\_description** Table 12.3  
 508 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_theoretic\_activity**  
 509 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_theoretic\_activity\_description** Table 12.4  
 510 : orbiter, layer5\_applications, apps\_graph\_theory, **graph\_theory\_apps**  
 511 : orbiter, layer5\_applications, canonical\_form, **canonical\_form\_classifier**  
 512 : orbiter, layer5\_applications, canonical\_form, **canonical\_form\_classifier\_description** Table 16.1  
 513 : orbiter, layer5\_applications, canonical\_form, **canonical\_form\_nauty**  
 514 : orbiter, layer5\_applications, canonical\_form, **canonical\_form\_of\_variety**  
 515 : orbiter, layer5\_applications, canonical\_form, **canonical\_form\_substructure**  
 516 : orbiter, layer5\_applications, canonical\_form, **classification\_of\_combinatorial\_objects**  
 517 : orbiter, layer5\_applications, canonical\_form, **classification\_of\_varieties**  
 518 : orbiter, layer5\_applications, canonical\_form, **input\_objects\_of\_type\_variety**  
 519 : orbiter, layer5\_applications, canonical\_form, **object\_in\_projective\_space\_with\_action**  
 520 : orbiter, layer5\_applications, canonical\_form, **object\_with\_properties**  
 521 : orbiter, layer5\_applications, canonical\_form, **quartic\_curve\_object\_with\_action**  
 522 : orbiter, layer5\_applications, canonical\_form, **variety\_object\_with\_action**  
 523 : orbiter, layer5\_applications, orbits, **orbit\_cascade**

524 : orbiter, layer5.applications, orbits, **orbits\_activity**  
 525 : orbiter, layer5.applications, orbits, **orbits\_activity\_description** Table 7.2  
 526 : orbiter, layer5.applications, orbits, **orbits\_create**  
 527 : orbiter, layer5.applications, orbits, **orbits\_create\_description** Table 7.1  
 528 : orbiter, layer5.applications, orbits, **orbits\_global**  
 529 : orbiter, layer5.applications, orbits, **orbits\_on\_polynomials**  
 530 : orbiter, layer5.applications, orbits, **orbits\_on\_subspaces**  
 531 : orbiter, layer5.applications, orthogonal\_geometry.applications, **BLT\_set\_create**  
 532 : orbiter, layer5.applications, orthogonal\_geometry.applications, **BLT\_set\_create\_description**  
**Table 14.14**  
 533 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_activity**  
 534 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_activity\_description**  
**Table 14.16**  
 535 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_classify**  
 536 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_classify\_activity**  
 537 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_classify\_activity\_description** Table 14.18  
 538 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_classify\_description**  
**Table 14.17**  
 539 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_domain\_with\_action**  
 540 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_group\_properties**  
 541 : orbiter, layer5.applications, orthogonal\_geometry.applications, **blt\_set\_with\_action**  
 542 : orbiter, layer5.applications, orthogonal\_geometry.applications, **flock\_from\_blt\_set**  
 543 : orbiter, layer5.applications, orthogonal\_geometry.applications, **orthogonal\_space\_activity**  
 544 : orbiter, layer5.applications, orthogonal\_geometry.applications, **orthogonal\_space\_activity\_description** Table 4.6  
 545 : orbiter, layer5.applications, orthogonal\_geometry.applications, **orthogonal\_space\_with\_action**  
 546 : orbiter, layer5.applications, orthogonal\_geometry.applications, **orthogonal\_space\_with\_action\_description** Table 4.5  
 547 : orbiter, layer5.applications, orthogonal\_geometry.applications, **table\_of\_blt\_sets**  
 548 : orbiter, layer5.applications, packings, **invariants\_packing**  
 549 : orbiter, layer5.applications, packings, **packing\_classify**  
 550 : orbiter, layer5.applications, packings, **packing\_invariants**  
 551 : orbiter, layer5.applications, packings, **packing\_long\_orbits**  
 552 : orbiter, layer5.applications, packings, **packing\_long\_orbits\_description** Table 14.13  
 553 : orbiter, layer5.applications, packings, **packing\_was**  
 554 : orbiter, layer5.applications, packings, **packing\_was\_activity**  
 555 : orbiter, layer5.applications, packings, **packing\_was\_activity\_description** Table 14.11  
 556 : orbiter, layer5.applications, packings, **packing\_was\_description** Table 14.10  
 557 : orbiter, layer5.applications, packings, **packing\_was\_fixpoints**  
 558 : orbiter, layer5.applications, packings, **packing\_was\_fixpoints\_activity**  
 559 : orbiter, layer5.applications, packings, **packing\_was\_fixpoints\_activity\_description** Table 14.12  
 560 : orbiter, layer5.applications, packings, **packings\_global**  
 561 : orbiter, layer5.applications, packings, **regular\_packing**  
 562 : orbiter, layer5.applications, projective\_geometry, **projective\_space\_activity**  
 563 : orbiter, layer5.applications, projective\_geometry, **projective\_space\_activity\_description**  
**Table 4.7, Table 4.8, Table 4.9, Table 4.10**  
 564 : orbiter, layer5.applications, projective\_geometry, **projective\_space\_global**  
 565 : orbiter, layer5.applications, projective\_geometry, **projective\_space\_with\_action**  
 566 : orbiter, layer5.applications, projective\_geometry, **projective\_space\_with\_action\_description**  
**Table 4.1**  
 567 : orbiter, layer5.applications, projective\_geometry, **summary\_of\_properties\_of\_objects**  
 568 : orbiter, layer5.applications, semifields, **semifield\_classify**  
 569 : orbiter, layer5.applications, semifields, **semifield\_classify\_description**  
 570 : orbiter, layer5.applications, semifields, **semifield\_classify\_with\_substructure**  
 571 : orbiter, layer5.applications, semifields, **semifield\_downstep\_node**  
 572 : orbiter, layer5.applications, semifields, **semifield\_flag\_orbit\_node**  
 573 : orbiter, layer5.applications, semifields, **semifield\_level\_two**  
 574 : orbiter, layer5.applications, semifields, **semifield\_lifting**

575 : orbiter, layer5.applications, semifields, **semifield\_substructure**  
 576 : orbiter, layer5.applications, semifields, **semifield\_trace**  
 577 : orbiter, layer5.applications, semifields, **trace\_record**  
 578 : orbiter, layer5.applications, spreads, **recoordinatize**  
 579 : orbiter, layer5.applications, spreads, **spread\_activity**  
 580 : orbiter, layer5.applications, spreads, **spread\_activity\_description** Table 14.2  
 581 : orbiter, layer5.applications, spreads, **spread\_classify**  
 582 : orbiter, layer5.applications, spreads, **spread\_classify\_activity**  
 583 : orbiter, layer5.applications, spreads, **spread\_classify\_activity\_description** Table 14.4  
 584 : orbiter, layer5.applications, spreads, **spread\_classify\_description** Table 14.3  
 585 : orbiter, layer5.applications, spreads, **spread\_create**  
 586 : orbiter, layer5.applications, spreads, **spread\_create\_description** Table 14.1  
 587 : orbiter, layer5.applications, spreads, **spread\_lifting**  
 588 : orbiter, layer5.applications, spreads, **spread\_table\_activity**  
 589 : orbiter, layer5.applications, spreads, **spread\_table\_activity\_description** Table 14.5  
 590 : orbiter, layer5.applications, spreads, **spread\_table\_with\_selection**  
 591 : orbiter, layer5.applications, spreads, **translation\_plane\_activity**  
 592 : orbiter, layer5.applications, spreads, **translation\_plane\_activity\_description** Table 14.9  
 593 : orbiter, layer5.applications, user\_interface, **activity\_description** Table 2.4, Table 2.5  
 594 : orbiter, layer5.applications, user\_interface, **interface\_algebra** Table 9.1, Table 9.2  
 595 : orbiter, layer5.applications, user\_interface, **interface\_coding\_theory** Table 10.1  
 596 : orbiter, layer5.applications, user\_interface, **interface\_combinatorics** Table 11.1, Table 11.2  
 597 : orbiter, layer5.applications, user\_interface, **interface\_cryptography** Table 9.5, Table 9.6,  
**Table 9.3**  
 598 : orbiter, layer5.applications, user\_interface, **interface\_povray** Table 17.3  
 599 : orbiter, layer5.applications, user\_interface, **interface\_projective**  
 600 : orbiter, layer5.applications, user\_interface, **interface\_symbol\_table**  
 601 : orbiter, layer5.applications, user\_interface, **interface\_toolkit** Table 19.1, Table 19.2  
 602 : orbiter, layer5.applications, user\_interface, **orbiter\_command**  
 603 : orbiter, layer5.applications, user\_interface, **orbiter\_top\_level\_session**  
 604 : orbiter, layer5.applications, user\_interface, **symbol\_definition**

# Appendix C

## The Makefile

### C.1 The Makefile

```
1
2
3   ORBITER_PATH=~/orbiter
4
5   #ORBITER_PATH=/s/hipft-logger/b/nobackup/abetten/orbiter
6   #ORBITER_PATH=/scratch/betten/COMPILE/orbiter
7   #ORBITER_PATH=/data/accounts/fac/betten/orbiter
8
9
10  # uncomment exactly one of the following two lines.
11  # uncomment the first if you want to run orbiter through docker.
12  # uncomment the second if you have an installed copy of orbiter and you want to r
   un it directly
13  #ORBITER_PATH=docker run -it --volume ${PWD}:/mnt -w /mnt abetten/orbiter
14
15  ORBITER_EXE_PATH=$(ORBITER_PATH)/src/apps/orbiter/
16
17  ORBITER=$(ORBITER_EXE_PATH)orbiter.out
18
19  SANDBOX=$(MY_PATH)/src/apps/sandbox/sandbox.out
20
21  #####
22  # additional configurations for when you want to
23  # compile automatically generated code
24  #####
25
26  SRC=$(ORBITER_PATH)/src
27  MY_CPP = g++
28  MY_CC = gcc
29  CPPFLAGS = -Wall -I../DEV.23/orbiter/src/lib -std=c++14
30  LIB = $(SRC)/lib/liborbiter.a -lpthread
31  LFLAGS = -lm -Wl,-rpath -Wl,/usr/local/gcc-8.2.0/lib64
32
33
34  #MAGMA_PATH=/usr/local/magma
35  MAGMA_PATH=""
36
37
38  #OPEN=open
39  OPEN=echo
```

```

40
41
42
43 #####
44 # End of configuration part
45 #####
46
47 GINAC_PATH=$(ORBITER_PATH)/src/apps/ginac
48 SANDBOX_PATH=$(ORBITER_PATH)/src/apps/sandbox
49
50 update:
51   ▷ cd $(ORBITER_PATH); make clean;
52   ▷ cd $(MY_PATH); make cleana; git pull; make
53
54
55 update_all:
56   ▷ cd $(MY_PATH); make clean; git pull; make
57
58
59 sandbox:
60   ▷ $(SANDBOX_PATH)/sandbox.out
61
62
63 #####
64 # testing:
65 #####
66
67 test:
68   ▷ make test_2
69   ▷ make test_3
70   ▷ make test_4
71   ▷ make test_5
72   ▷ make test_6
73   ▷ make test_7
74   ▷ make test_8
75   ▷ make test_9
76   ▷ make test_10
77   ▷ make test_11
78   ▷ make test_12
79   ▷ make test_13
80   ▷ make test_14
81   ▷ make test_15
82   ▷ make test_16
83   ▷ make test_17
84   ▷ make test_18
85   ▷ make test_19
86
87
88
89 test_problem:
90
91
92
93 #####
94 # Makefile Variables
95
96
97 # Section 2.7 vectors
98

```



```

99
100
101 # Section 2.8 vector of group elements
102
103
104 HIRSCHFELD_STAB_GENERATORS="\
105 1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1, \
106 1,0,0,0,0,2,0,0,0,0,2,0,0,0,1,0, \
107 1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0, \
108 1,0,0,0,0,1,0,0,1,1,1,0,0,1,0,1,1, \
109 1,0,0,0,0,0,1,0,0,1,0,0,0,0,0,1,0, \
110 0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0"
111
112
113
114
115 CLASS_2A_REP="1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1"
116
117 FILE_NAME_CLASS_2A="PGGL_4_4_Subgroup.Hirschfeld.Stab.51840_class_of_2A.csv"
118
119
120
121 # Section 2.9 Symbolic Objects
122
123 TEST_FORMULA="(-a+b*b)*x*x+a*b*x"
124
125 BCH_POLYNOMIAL_ALFA_F256="X^2 + 167*X + 214"
126
127 BCH_POLYNOMIAL_BRAVO_F256="X^4+213*X^3+27*X^2+23*X+1"
128
129 VANDERMONDE_4X4_FORMULA="1,x0,x0^2,x0^3,1,x1,x1^2,x1^3,\
130 1,x2,x2^2,x2^3,1,x3,x3^2,x3^3"
131
132 ECKARDT_SURFACE_EQN="a*X3^3-a*b^2*(X0^2+X1^2+X2^2)*X3+b^3*(a^2+1)*X0*X1*X2"
133
134 # general normal form for cubic surfaces with 27 rational lines:
135
136 F_abcd_eqn="-(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(b - d)*X0^2*X2 \
137 + (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(a + b - c - d)*X0*X1*X2 \
138 + (a^2*c - a^2*d - a*c^2 + b*c^2 + a*d - b*c)*(b - d)*X0*X1*X3 \
139 - (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X0*X2^2 \
140 - (a^2*c*d - a*b*c^2 - a^2*d + a*b*d + b*c^2 - b*c*d)*(b - d)*X0*X2*X3 \
141 - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X2 \
142 - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X3 \
143 + (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1*X2^2 \
144 + ((1+1)*a^2*b*c*d - a^2*b*d^2 - (1+1)*a^2*c*d^2 \
145 - (1+1)*a*b^2*c^2 + a*b^2*c*d + (1+1)*a*b*c^2*d + a*b*c*d^2 \
146 - b^2*c^2*d - a^2*b*c + a^2*c*d + a^2*d^2 + a*b^2*c + a*b*c^2 \
147 - (1+1+1)*a*b*c*d - a*c^2*d + a*c*d^2 + b^2*c^2)*X1*X2*X3 \
148 + c*a*(a*d - b*c - a + b + c - d)*(b - d)*X1*X3^2"
149
150
151 # Chapter 3 Basic algebra
152
153
154
155 # Section 3.4 Linear algebra over finite fields
156
157

```

```

158
159
160 V7_VANDERMONDE_EXTENDED="\
161 1,0,0,0,0,0,0,1,0,0,0,0,0,0, \
162 1,1,1,1,1,1,1,0,1,0,0,0,0,0, \
163 1,2,4,1,2,4,1,0,0,1,0,0,0,0, \
164 1,3,2,6,4,5,1,0,0,0,1,0,0,0, \
165 1,4,2,1,4,2,1,0,0,0,0,1,0,0, \
166 1,5,4,6,2,3,1,0,0,0,0,0,1,0, \
167 1,6,1,6,1,6,1,0,0,0,0,0,0,1"
168
169
170
171 CODE_RS_6_4_7="\
172 621000 \
173 062100 \
174 006210 \
175 000621"
176
177
178
179 # Chapter 4 Geometry
180
181
182 # Section 4.5 Algebraic sets
183
184
185 SURFACE_F9_1_EQN="4*X0*X1^2+4*X1^2*X2+8*X0*X2^2+\
186 3*X1*X2^2+3*X0*X1*X2+X0^2*X3+6*X0*X3^2+\
187 5*X0*X1*X3+X0*X2*X3+5*X1*X2*X3"
188
189
190 EC_11_EQUATION="1,0,3,0,0,0,10,1,0,0"
191
192
193
194 HIRSCHFELD_SURFACE_EQUATION="0,0,0,0,0,0,1,0,1,0, 0,1,0,1,0,0,0,0,0"
195
196 HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM="X0^2*X3 + X1^2*X2 + X1*X2^2 + X0*X3^2
197 "
198 #X_0^2*X_3 + X_1^2*X_2 + X_1*X_2^2 + X_0*X_3^2
199 #X^2*W+X*W^2+Y^2*Z+Y*Z^2
200
201
202
203 # Section 4.9 Advanced topics
204
205 H1_LABEL="H1"
206
207 H1_GENS=-PGGL 4 4 \
208 > -subgroup_by_generators "H1" 4 2 \
209 > "1,0,0,0,1,1,0,0,0,0,1,0,0,0,1,1,0, \
210 > 1,0,0,0,0,1,0,0,1,0,1,0,0,1,0,1,0" \
211 > -end
212
213
214
215

```

```

216 # Section 4.10 Geometric objects
217
218
219 DOILY="Row,C0,C1,C2\
220 \n0,0,12,13\
221 \n1,1,12,14\
222 \n2,8,9,12\
223 \n3,4,6,8\
224 \n4,6,10,14\
225 \n5,3,7,8\
226 \n6,7,10,13\
227 \n7,4,11,13\
228 \n8,3,11,14\
229 \n9,0,5,6\
230 \n10,1,5,7\
231 \n11,5,9,11\
232 \n12,0,2,3\
233 \n13,1,2,4\
234 \n14,2,9,10\
235 \nEND"
236
237
238 # q=17:
239 # 3 is p.e. mod 17.
240 # so we pick f=3.
241 # then,  $2f^2=18=1$ 
242 #  $4f = 12$ 
243
244 #  $X^4 - Y^4 - Z^4 + 2f^2YZ^2 + 4fX^2YZ$ 
245
246 #(1,-1,-1,0,0,0,0,0,0,0,0,2f^2,4f,0,0)
247
248 EDGE_CURVE_Q17_EQUATION="1,16,16,0,0,0,0,0,0,0,0,1,12,0,0"
249
250 EDGE_CURVE_Q17_AS_POINTS="4, 7, 16, 19, 20, 23, 32, 35, 89, 100, 244, 251"
251
252
253 FILE_Q17="orbit,curve,pts_on_curve,bitangents,go\
254 \n0,\"$(EDGE_CURVE_Q17_EQUATION)\",\"$(EDGE_CURVE_Q17_AS_POINTS)\",\"\",-1\
255 \nEND"
256
257
258 EDGE_CURVE_Q23_AS_POINTS="4, 25, 26, 47, 48, 71, 92, 95, 114, 119, \
259 136, 143, 158, 167, 180, 191, 202, 215, 224, 239, 246, 263, 268, \
260 287, 290, 311, 312, 334, 335, 356, 359, 378, 383, 400, 407, 422, \
261 431, 444, 455, 466, 479, 488, 503, 510, 527, 530, 532, 551"
262
263
264
265
266
267 # Chapter 5 Ring Theory
268
269 # Section 5.1 polynomials over finite fields
270
271
272
273 CRC32_SPARSE="1,32,1,26,1,23,1,22,1,16,1,12,1,11,\
274 1,10,1,8,1,7,1,5,1,4,1,2,1,1,1,0"

```

```

275
276
277 TWO_TO_THE_32_MINUS_2=4294967294
278
279 INVERSE_SPARSE="1,31,1,25,1,22,1,21,1,15,\
280 1,11,1,10,1,9,1,7,1,6,1,4,1,3,1,1,1,0"
281
282
283
284 # Section 5.2 multivariate polynomial rings
285
286
287
288 CREMONA_MAP_Y0="3*y0^5*y2+4*y0^3*y1^2*y2\
289 +2*y0^3*y2^3+y0*y1^4*y2\
290 +6*y0*y1^2*y2^3+9*y0*y2^5"
291
292 CREMONA_MAP_Y1="y0^5*y1+5*y0^3*y1^3\
293 +12*y0^3*y1*y2^2+3*y0*y1^5\
294 +5*y0*y1^3*y2^2+y0*y1*y2^4"
295
296 CREMONA_MAP_Y2="10*y0^6+11*y0^4*y1^2\
297 +11*y0^4*y2^2+4*y0^2*y1^4\
298 +9*y0^2*y1^2*y2^2+4*y0^2*y2^4"
299
300 CREMONA_MAP_Y3="0"
301
302
303
304
305
306 PTS_OF_SURFACE_ORBIT211_Q3_L9_E4="\
307 0,1,2,5,7,8,10,14,9,12, \
308 15,3,16,37,31,34,20,19,17,32,36,33"
309
310 SURFACE_F_9="x0*x0*x1 - x0*x1*x1 -x0*x1*x3 -x2*x2*x3 - x2*x3*x3"
311
312
313 ENDRASS_SPARSE="\
314 6,0,4,4,2,7,5,9,6,20,6,23,1,25,3,30,1,32,3,34,4,56,6,59,1,61,6,66, \
315 2,68,6,70,3,77,2,79,6,83,6,120,2,123,5,125,3,130,1,132,3,134,3,141, \
316 2,143,6,147,3,156"
317
318
319
320
321 # Section 5.3 Algebraic geometry
322
323 FILE_HERMITIAN_CURVE="Line,equation\n0,\"X^4+Y^4+Z^4\"\\nEND\\n\\n"
324
325 FILE_HERMITIAN_SURFACE="Line,equation\n0,\"X^4+Y^4+Z^4+W^4\"\\nEND\\n\\n"
326
327
328 # Chapter 6 Group Theory
329
330
331
332 # Section 6.1 generic permutation groups
333

```

```

334
335
336 GENERATORS_M12="3,1,9,0,10,11,6,7,8,2,4,5, 7,2,3,1,11,9,5,8,0,6,4,10"
337
338 # Section 6.2 Matrix groups
339
340
341 # Section 6.3 Subgroups
342
343 GEN_C13="1,2,3,4,5,6,7,8,9,10,11,12,0"
344 # (0,1,2,3,4,5,6,7,8,9,10,11,12)
345
346
347
348
349
350
351 GENERATORS_HESSE_GROUP="\
352 3000300030 \
353 2000201230 \
354 1000100111 \
355 1000220200 \
356 1002312010 \
357 0331003211 \
358 2200011331"
359
360
361
362
363 GENERATORS_WEYL_GROUP_E8="\
364 -1,-1,-1,-1,0,0,0,0, \
365 0,0,0,1,0,0,0,0, \
366 1,0,0,0,0,0,0,0, \
367 0,0,1,0,0,0,0,0, \
368 0,1,0,1,1,0,0,0, \
369 0,0,0,0,0,1,0,0, \
370 0,0,0,0,0,0,1,0, \
371 0,0,0,0,0,0,0,1, \
372 -1,0,-1,-1,-1,-1,-1, \
373 0,1,0,1,1,1,1,1, \
374 1,0,0,0,0,0,0,0, \
375 0,0,1,0,0,0,0,0, \
376 0,0,0,1,0,0,0,0, \
377 0,0,0,0,1,0,0,0, \
378 0,0,0,0,0,1,0,0, \
379 0,0,0,0,0,0,1,0"
380
381
382
383 PGOp_6_2_GENS="\
384 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1,0, \
385 1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,1,1,0,0,1,0,0,1,0,0,1,0, \
386 1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,0,0,0,1,0,0,0,0,1,0, \
387 1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,1,0,0,0,0,0,1,0,0,1,0,0,1,0,0,0,1,0, \
388 1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,1,0,1,0,0,0,0,1,0,0,0,0,1,0, \
389 1,0,0,0,0,0,0,1,1,1,1,1,1,0,0,0,1,0,1,0,0,0,0,1,1,0,1,0,0,0,1,0,0, \
390 1,0,0,0,0,0,0,1,0,1,0,0,1,0,1,0,0,0,1,0,0,0,0,0,1,1,0,0,0,0,0,0,1, \
391 0,1,0,1,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,1,0,1,0,0,0,0,0,1,0,0,1,1,0,0,1, \
392 0,0,1,1,1,1,1,0,1,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,1,0,1,0,1,0,1,1,0, \

```

```

393 0,0,0,0,1,0,1,0,1,1,1,1,1,0,1,0,1,0,1,0,0,1,1,0,1,0,0,0,0,1,1,1,1,0,"
394
395
396
397
398
399
400 # Section 6.4 New groups from old
401
402
403 SURFACE_Q13_STAB_GEN="1,0,0,0,9,12,0,0,10,0,12,0,9,0,0,12, \
404 1,0,0,0,0,12,12,6,6,0,0,7,1,0,2,0, \
405 0,1,1,7,3,9,9,11,2,10,10,3,9,9,1,11"
406
407
408
409
410 # Section 6.6 Group theoretic activities based on Magma
411
412 # large sets of PG(2,3):
413
414 GENERATORS_H5="1,2,3,4,0,6,7,8,9,5,10,11,12"
415 # (0, 1, 2, 3, 4)(5, 6, 7, 8, 9)
416
417 GENERATORS_N5="\
418 0,1,2,3,4,9,5,6,7,8,10,11,12, \
419 0,1,2,3,4,5,6,7,8,9,10,12,11, \
420 0,4,3,2,1,5,9,8,7,6,10,11,12, \
421 0,2,4,1,3,5,7,9,6,8,10,11,12, \
422 0,1,2,3,4,5,6,7,8,9,11,10,12, \
423 1,2,3,4,0,6,7,8,9,5,10,11,12, \
424 5,9,8,7,6,0,4,3,2,1,10,11,12"
425
426
427 PGOM_6_2_CLASS_2A_REP="1,0,0,0,0,0,0,1,0,0,\
428 0,0,0,0,1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0"
429
430 FILE_NAME_PGOM_6_2_CLASS_2A="PGOm_6_2_class_of_2A.csv"
431
432 PGOM_6_2_CLASS_2A_REP_CLEAN="1,0,0,0,0,0,0,1,0,0,\
433 0,0,0,0,0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1"
434
435 PGOM_6_2_CLASS_2A_CENTRALIZER_GEN="\
436 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,1, \
437 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1, \
438 1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1, \
439 1,0,0,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0,0,1,1,0,0,0,0,1, \
440 1,0,0,0,0,0,1,1,0,0,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,1,0,0,0,1,1, \
441 1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1, \
442 0,1,1,1,1,0,1,0,1,1,1,0,1,1,0,1,1,0,1,1,1,0,0,0,0,0,1,0,1,1,1,1,1, \
443 1,1,0,0,0,1,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,1,0,0,1,1,0,0,0,0,1, \
444 0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1"
445
446
447
448 # Section 6.7 Group theoretic activities based on GAP
449
450
451

```

```

452 # Section 6.8 Linear groups, advanced topics
453
454
455
456
457 #Co3 from Conway et al., 1985 (ATLAS)
458 #order = 49576656000
459 #Co3 from the paper by Suleiman and Wilson 1997
460
461
462 CONWAY_GEN1="\
463 1101110001000001010000\
464 1111010111110100001011\
465 0000001000000100010101\
466 1111100110110001001110\
467 0101010000000010011101\
468 0000010000000100010101\
469 001000000000100010101\
470 0001000011000000111111\
471 1110100100110100010011\
472 0000000000000110010101\
473 0000000000100100010101\
474 0110111111010011101111\
475 0000000000001100010101\
476 0000000000000100000101\
477 0000000001000100010101\
478 0000000000000100011101\
479 0001000110000010011010\
480 00000000000000000010101\
481 0000000000000101010101\
482 0000000000000100010100\
483 0000000000000100010111\
484 0000000000000100010001"
485
486 CONWAY_GEN2="\
487 0101000010111010111111\
488 0110010100011110110000\
489 0011010000111111010111\
490 0001101110001011010011\
491 1010010000100001011110\
492 1101000000001010100011\
493 1100101010001111010101\
494 1000110100110101010101\
495 0100110001010000000111\
496 1100000010100101010010\
497 0101110110011100000101\
498 0101111101010011111001\
499 1000010101010101010001\
500 0001010000111100100111\
501 0011010010111011001111\
502 0100110010110011111010\
503 1101011001111101100011\
504 0100101001001000100001\
505 1100101100001001110011\
506 0101110110010100000001\
507 0000001101111000101110\
508 1101101010101110000101"
509
510

```

```

511
512
513
514 Ree_gen1="21,5,1,6,17,1,1, 3,13,5,21,6,6,18, 21,3,21,21,22,6,14, \
515 14,18,1,5,13,6,7, 3,3,2,1,24,16,3, 17,3,22,10,16,24,26, \
516 21,21,6,18,20,2,5"
517
518 Ree_gen2="16,3,11,5,16,22,20, 24,6,18,24,7,1,26, 9,23,17,18,23,20,13, \
519 9,7,2,15,17,5,11, 3,3,6,21,4,24,16, 25,8,6,24,21,12,7, \
520 24,15,2,13,11,14,24"
521
522
523
524 # Chapter 7 Orbit Algorithms
525
526
527 # Section 7.7
528
529 SURFACE_F9_1.EQN.RESTRICTED="4*X0*X1^2+4*X1^2*X2+8*X0*X2^2+3*X1*X2^2+3*X0*X1*X2"
530
531
532 # Chapter 8 Cubic Surfaces
533
534 # Section 8.2 Generalities
535
536
537 # Section 8.2 Creation
538
539 # Section 8.3 Activities
540
541 # points:
542 #0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,53,
543 54,59,60,61,62,67,68,69,70,75,76,79,80,81,82
544
545 # points off:
546 #15,16,17,18,19,20,21,22,24,25,28,29,32,33,36,37,40,41,43,44,45,46,49,50,55,56,57
547 ,58,63,64,65,66,71,72,73,74,77,78,83,84
548
549 HIRSCHFELD_SURFACE_POINTS.OFF="15,16,17,18,19,20,21,22,24,\
550 25,28,29,32,33,36,37,40,41,43,44,45,46,49,50,55,56,57,58,\
551 63,64,65,66,71,72,73,74,77,78,83,84"
552
553 #Hesse planes:
554 #7,8,11,13,14,16,17,19,28,29,32,34,35,37,38,40,42,43,44,45,47,48,52,54,56,57,60,6
555 1,63,64,65,66,68,69,73,75,77,78,81,82
556
557 HIRSCHFELD_SURFACE_HESSIE_PLANES="7,8,11,13,14,16,17,19,28,\
558 29,32,34,35,37,38,40,42,43,44,45,47,48,52,54,56,57,60,61,\
559 63,64,65,66,68,69,73,75,77,78,81,82"
560
561
562 # Section 8.4 Classification
563
564 # Section 8.5 Quartic Curves
565
566

```



```

567
568 NB_CUBIC_SURFACES_Q7=1
569
570
571 NB_CUBIC_SURFACES_Q13=4
572
573
574 NB_CUBIC_SURFACES_Q19=10
575
576 NB_QUARTIC_CURVES_Q19=14
577
578
579
580 # Section 8.6 Interface to GAP
581
582
583
584 # Chapter 9 Applications
585
586
587 # Section 9.1 Number Theory
588
589 # Section 9.2 Representation Theory
590
591
592 # Section 9.3 Cryptography
593
594
595 NTRU_N=7
596
597 NTRU_P=3
598
599 NTRU_Q=41
600
601 NTRU_D=2
602
603 NTRU_XN1="-1,0,0,0,0,0,0,1,"
604
605 # D + 1 plus ones and D minus ones
606
607 ALICE_PRIVATE_F="-1,0,1,1,-1,0,1"
608
609 # D plus ones and D minus ones
610
611 ALICE_PRIVATE_G="0,-1,-1,0,1,0,1"
612
613 #F_q(x) = 8X^{6} + 26X^{5} + 31X^{4} + 21X^{3} + 40X^{2} + 2X + 37
614
615 ALICE_PRIVATE_FQ="37,2,40,21,31,26,8"
616
617
618 #F_p(x) = X^{6} + 2X^{5} + X^{3} + X^{2} + X + 1
619 ALICE_PRIVATE_FP="1,1,1,1,0,2,1"
620
621 #C(X)=20X^{6} + 40X^{5} + 2X^{4} + 38X^{3} + 8X^{2} + 26X + 30
622 ALICE_PUBLIC_KEY="30,26,8,38,2,40,20"
623
624 BOB_MESSAGE="1,-1,1,1,0,-1"
625

```

```

626 BOB.ONE_TIME_KEY="-1,1,0,0,0,-1,1"
627
628 #  $E(X) = 31X^6 + 19X^5 + 4X^4 + 2X^3 + 40X^2 + 3X + 25$ 
629 BOB.ENCRYPT= "25,3,40,2,4,19,31"
630
631 #  $C(X)=X^6 + 10X^5 + 33X^4 + 40X^3 + 40X^2 + X + 40$ 
632 ALICE.C1="40,1,40,40,33,10,1"
633
634 #  $A(X)=X^6 + 10X^5 - 8X^4 - X^3 - X^2 + X - 1$ 
635 ALICE.C2="-1,1,-1,-1,-8,10,1"
636
637 #  $A(X)=X^6 + X^5 + X^4 + 2X^3 + 2X^2 + X + 2$ 
638 ALICE.C3="2,1,2,2,1,1,1"
639
640
641 #  $C(X)=2X^5 + X^3 + X^2 + 2X + 1$ 
642 ALICE.C4="1,2,1,1,0,2"
643
644
645
646 # Chapter 10 Coding Theory
647
648 # Section 10.1 Introduction
649
650 CODE_5_2_3.CODEWORDS="0,7,25,30"
651
652
653 # Section 10.2 Linear codes
654
655
656 SIMPLEX.CODE_GENERATOR="\
657 1,0,1,0,1,0,1, \
658 0,1,1,0,0,1,1, \
659 0,0,0,1,1,1,1"
660
661
662
663 CODE_RM_3_1.GENMA="\
664 11111111\
665 01010101\
666 00110011\
667 00001111"
668
669 CODE_RM_4_1.GENMA="\
670 1111111111111111\
671 0101010101010101\
672 0011001100110011\
673 0000111100001111\
674 0000000011111111"
675
676
677 HAMMING.CODE_ROWS_IN.BINARY_RANKS="67,37,22,15"
678
679 HAMMING.CODE.CODEWORDS="0, 67, 37, 102, 22, 85, \
680 51, 112, 15, 76, 42, 105, 25, 90, 60, 127"
681
682
683 HAMMING.CODE_GENERATOR="\
684 1,0,0,0,0,1,1, \

```

[illegible]



```

758 M24_2A_PERM="10, 1, 21, 4, 3, 5, 22, 13, 11, 9, 0, 8, \
759 16, 7, 23, 15, 12, 17, 18, 19, 20, 2, 6, 14"
760
761
762 M24_2B_PERM="2, 5, 0, 8, 11, 1, 16, 14, 3, 19, 21, 4, \
763 22, 23, 7, 20, 6, 18, 17, 9, 15, 10, 12, 13"
764
765 GOLAY24_2A_FIX_SUBCODE="1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, \
766 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, \
767 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, \
768 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, \
769 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, \
770 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, \
771 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, \
772 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, \
773 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, \
774 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, \
775 1, 1, 0, 0, 0, 1, 1, 0, 0, 1"
776
777 GOLAY24_2B_FIX_SUBCODE="1, 0, 1, 0, 0, 0, 0, 0, 0, 1, \
778 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, \
779 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, \
780 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, \
781 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, \
782 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, \
783 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, \
784 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1"
785
786
787
788
789 # Section 10.4 CRC codes
790
791
792 CRC4="1,4,1,2,1,1,1,0"
793
794 CRC7="1,7,1,3,1,0"
795
796 CRC8_ATM="1,8,1,2,1,1,1,0"
797
798 CRC16_CCITT="1,16,1,12,1,5,1,0"
799
800 CRC32_ETHERNET="1,32,1,26,1,23,1,22,1,16,1,12,1,11,1,10,1,8,1,7,\
801 1,5,1,4,1,2,1,1,1,0"
802
803 CRC32_ETHERNET_POLY="X^32+X^26+X^23+X^22+X^16+\
804 X^12+X^11+X^10+X^8+X^7+X^5+X^4+X^2+X^1+1"
805
806 CRC32_CASTAGNOLI="1,32,1,28,1,27,1,26,1,25,1,23,1,22,1,20,1,19,1,\
807 18,1,14,1,13,1,11,1,10,1,9,1,8,1,6,1,0"
808
809 CRC64_ECMA182="1,64,1,62,1,57,1,55,1,54,1,53,1,52,1,47,1,46,1,45,\
810 1,40,1,39,1,38,1,37,1,35,1,33,1,32,1,31,1,29,1,27,1,24,1,23,1,22,\
811 1,21,1,19,1,17,1,13,1,12,1,10,1,9,1,7,1,4,1,1,1,0"
812
813 CRC64_ROCKSOFT="1,64,1,63,1,61,1,59,1,58,1,56,1,55,1,52,1,49,1,48,\
814 1,47,1,46,1,44,1,41,1,37,1,36,1,34,1,32,1,31,1,28,1,26,1,23,1,22,1,\
815 19,1,16,1,13,1,12,1,10,1,9,1,6,1,4,1,3,1,0"
816

```

```

817
818
819 CRC_POLY_Q256_DEG2_DENSE="214,167,1"
820
821
822
823 CRC_POLY_BRAVO_DENSE="1,23,27,213,1"
824
825
826 CRC_POLY_CHARLIE_DENSE="1,126,25,1,196,209,244,3,121,126,35,65,1"
827
828
829 POLY_Q256_DEG30_SPARSE="1,0,26,1,210,2,24,3,\
830 138,4,148,5,160,6,58,7,108,8,199,9,95,10,56,\
831 11,9,12,205,13,194,14,193,15,3,16,248,17,110,\
832 18,150,19,24,20,169,21,192,22,212,23,112,24,\
833 144,25,97,26,109,27,174,28,253,29,1,30"
834
835 POLY_Q256_DEG30_DENSE="1,26,210,24,138,148,\
836 160,58,108,199,95,56,9,205,194,193,3,248,110,\
837 150,24,169,192,212,112,144,97,109,174,253,1"
838
839 CRC_FILE=allan.Gates
840
841 CRC_FILE_EXTENSION=bmp
842
843 TEN_MILLION=10000000
844
845
846 CRC_BCH_Q16_N51_D3_PROJECTIVE_SET="1900, 2703, 2347, 1653, 232, 3681, 752, 511, 3
916, 1222, 632, 2014, 3951, 4006, 2198, 3258, 2804, 3771, 3120, 2132, 3754, 2640,
1995, 943, 3319, 1748, 36, 546, 1198, 4296, 4013, 2118, 2234, 3034, 2441, 4085,
4326, 318, 1180, 2488, 4229, 765, 1470, 2306, 666, 3454, 1763, 0, 1, 2, 3 "
847
848 CRC_POLY_DELTA_DENSE="1, 2, 13, 5, 1"
849 #create_BCH_code::init Q=X^{4} + 5X^{3} + 13X^{2} + 2X + 1
850 #create_BCH_code::init BCH_POLYNOMIAL_F16="X^4+5*X^3+13*X^2+2*X+1"
851
852 ERROR_POLY_BRAVO_COEFF="737, 417, 171, 634, 158, 30, 480, 688, 94, 739, 147, 694,
724, 741, 51, 411, 753, 622, 592, 667, 25, 709, 55, 450, 255, 744, 28, 718, 82,
740, 372, 144, 435, 505, 478, 233, 325, 231, 27, 518, 492, 350, 621, 468, 297, 14
1, 735, 338, 227, 434, 54, 180, 390, 245, 301, 617, 423, 408, 521, 42, 697, 719,
668, 618, 107, 746, 441, 376, 705, 353, 368, 254, 393, 291, 762, 188, 506, 580, 4
19, 525, 500, 652, 433, 130, 52, 212, 511, 191, 182, 100, 676, 706, 248, 268, 365
, 626, 303, 5, 490, 524, 237, 210, 326, 619, 485, 767, 432, 721, 391, 650, 88, 62
7, 177, 700, 354, 682, 38, 360, 70, 636, 609, 766, 203, 7, 369, 584, 475, 84, 87,
251, 680, 479, 256, 121, 473, 604, 614, 265, 585, 40, 458, 551, 163, 661, 358, 6
10, 349, 748, 371, 310, 31, 136, 153, 649, 662, 547, 120, 324, 162, 671, 62, 725,
603, 645, 670, 554, 364, 127, 312, 708, 386, 286, 532, 58, 658, 366, 74, 442, 69
9, 707, 164, 102, 593, 657, 293, 638, 370, 716, 454, 126, 204, 232, 637, 409, 491
, 666, 402, 209, 639, 560, 578, 467, 24, 629, 93, 453, 352, 425, 406, 37, 684, 18
9, 648, 186, 345, 691, 673, 193, 264, 247, 714, 267, 540, 215, 134, 729, 588, 295
, 294, 241, 577, 398, 722, 46, 214, 60, 640, 770, 477, 106, 97, 381, 280, 675, 41
8, 29, 385, 269, 151, 665, 109, 514, 305, 377, 95, 544, 570, 552, 128, 152, 599,
397, 415, 149, 451, 738, 743, 302, 140, 273, 103, 284, 396, 726, 327, 221, 288, 2
00, 344, 285, 112, 542, 22, 313, 263, 218, 61, 157, 362, 559, 507, 243, 664, 206,
550, 242, 244, 230, 145, 761, 413, 679, 392, 447, 17, 330, 156, 279, 752, 487, 7
31, 651, 388, 535, 154, 502, 108, 452, 363, 240, 569, 564, 713, 704, 495, 589, 83
, 15, 483, 261, 537, 595, 219, 573, 516, 252, 199, 337, 616, 339, 486, 555, 653,

```

728, 137, 216, 56, 211, 734, 449, 44, 26, 348, 426, 91, 139, 238, 539, 444, 754, 672, 695, 543, 659, 601, 654, 742, 300, 69, 115, 756, 122, 582, 374, 236, 656, 53, 755, 512, 594, 404, 65, 611, 316, 703, 361, 318, 80, 92, 12, 720, 378, 763, 228, 429, 623, 608, 712, 33, 612, 185, 747, 11, 620, 78, 234, 202, 165, 498, 173, 160, 494, 768, 75, 306, 482, 641, 53, 222, 379, 460, 258, 567, 50, 531, 548, 207, 424, 320, 383, 124, 757, 249, 10, 405, 575, 597, 587, 341, 328, 336, 513, 317, 172, 681, 169, 187, 430, 351, 523, 81, 553, 389, 496, 689, 461, 607, 359, 436, 105, 229, 8, 101, 176, 663, 43, 701, 356, 630, 562, 678, 272, 224, 387, 271, 311, 456, 530, 401, 278, 239, 520, 529, 287, 343, 59, 367, 342, 21, 503, 420, 466, 322, 146, 504, 563, 196, 545, 558, 161, 394, 257, 49, 99, 598, 6, 77, 143, 68, 118, 133, 690, 410, 235, 536, 48, 205, 270, 445, 283, 198, 660, 90, 47, 57, 727, 41, 463, 76, 581, 576, 192, 250, 686, 509, 34, 175, 246, 400, 168, 111, 334, 403, 517, 677, 373, 497, 431, 470, 501, 174, 225"

853

854 ERROR.POLY\_BRAVO\_PLACE="100, 202, 178, 171, 102, 243, 43, 238, 132, 200, 126, 107, 61, 252, 150, 5, 18, 51, 161, 248, 230, 166, 207, 48, 138, 132, 221, 248, 228, 42, 126, 117, 253, 85, 133, 165, 56, 125, 224, 92, 223, 129, 108, 179, 197, 224, 163, 241, 196, 207, 53, 73, 183, 98, 172, 7, 65, 90, 35, 15, 59, 219, 95, 41, 93, 52, 51, 19, 205, 78, 66, 6, 218, 47, 109, 228, 45, 134, 141, 173, 225, 93, 194, 5, 164, 100, 199, 10, 118, 30, 163, 110, 152, 247, 212, 214, 146, 104, 164, 7, 117, 87, 129, 159, 235, 129, 233, 224, 72, 250, 167, 128, 208, 220, 150, 195, 40, 168, 73, 21, 98, 185, 128, 27, 30, 74, 251, 134, 186, 40, 109, 138, 181, 202, 137, 86, 72, 230, 117, 18, 123, 180, 10, 23, 122, 47, 241, 16, 193, 71, 189, 203, 212, 68, 241, 143, 191, 62, 228, 255, 82, 12, 178, 134, 161, 178, 251, 119, 66, 209, 33, 32, 17, 124, 47, 96, 191, 20, 10, 4, 82, 36, 65, 36, 164, 113, 230, 30, 33, 176, 75, 54, 130, 25, 171, 227, 1, 41, 182, 164, 87, 232, 210, 74, 161, 84, 24, 187, 106, 106, 52, 153, 247, 152, 252, 245, 182, 253, 213, 203, 45, 168, 193, 190, 42, 105, 201, 1, 211, 224, 153, 189, 89, 187, 59, 238, 53, 95, 105, 103, 24, 2, 62, 250, 57, 70, 68, 110, 45, 45, 226, 254, 194, 103, 186, 140, 209, 1, 37, 87, 15, 200, 26, 22, 216, 251, 175, 137, 33, 228, 132, 152, 86, 179, 106, 68, 69, 49, 202, 146, 67, 168, 130, 223, 55, 234, 177, 88, 146, 195, 100, 176, 210, 177, 61, 161, 1, 58, 5, 191, 30, 49, 183, 83, 125, 13, 170, 211, 93, 110, 219, 93, 246, 120, 169, 97, 206, 86, 241, 208, 195, 199, 53, 31, 133, 253, 181, 247, 192, 209, 19, 159, 175, 39, 247, 154, 12, 225, 43, 231, 198, 83, 125, 33, 175, 79, 242, 224, 150, 251, 213, 125, 155, 230, 61, 109, 32, 239, 227, 40, 8, 133, 186, 118, 232, 157, 218, 41, 24, 168, 32, 205, 172, 216, 36, 94, 179, 37, 212, 13, 35, 171, 16, 171, 39, 69, 241, 165, 62, 91, 136, 4, 227, 168, 53, 6, 226, 251, 58, 251, 38, 234, 245, 255, 168, 119, 169, 47, 179, 223, 28, 32, 17, 185, 49, 44, 196, 122, 89, 116, 157, 223, 252, 204, 40, 119, 89, 8, 7, 178, 235, 52, 12, 51, 217, 60, 104, 100, 59, 111, 14, 244, 49, 223, 225, 245, 187, 61, 242, 174, 29, 145, 172, 177, 131, 188, 157, 76, 104, 244, 94, 39, 96, 246, 131, 77, 255, 70, 211, 201, 166, 105, 68, 129, 4, 80, 20, 89, 58, 222, 159, 208, 82, 255, 245, 121, 210, 149, 202, 105, 212, 42, 35, 188, 3, 121, 23, 139, 50, 24, 224, 20, 140, 119, 55, 118, 53, 155, 19, 70, 138, 178, 202, 153, 78, 44, 90, 109, 32, 253, 64, 130, 238, 243, 87, 12, 204, 63, 116, 163, 82, 25, 14, 31, 197, 193, 203, 235, 79, 86, 118, 118, 88, 190, 26, 55, 179, 156, 153, 89, 227, 221"

855

856 CRC.ERROR.POLY\_BRAVO="104\*X^5+118\*X^6+27\*X^7+39\*X^8+51\*X^10+245\*X^11+91\*X^12+247\*X^15+125\*X^17+202\*X^21+130\*X^22+210\*X^24+230\*X^25+125\*X^26+224\*X^27+221\*X^28+70\*X^29+243\*X^30+189\*X^31+58\*X^33+193\*X^34+106\*X^37+40\*X^38+18\*X^40+204\*X^41+15\*X^42+77\*X^43+213\*X^44+187\*X^46+243\*X^47+44\*X^48+140\*X^49+204\*X^50+150\*X^51+164\*X^52+122\*X^53+53\*X^54+207\*X^55+242\*X^56+87\*X^57+124\*X^58+121\*X^59+238\*X^60+177\*X^61+82\*X^62+171\*X^65+19\*X^68+24\*X^69+73\*X^70+191\*X^74+185\*X^75+116\*X^76+53\*X^77+168\*X^78+165\*X^80+29\*X^81+228\*X^82+181\*X^83+134\*X^84+186\*X^87+167\*X^88+238\*X^90+61\*X^91+62\*X^92+161\*X^93+132\*X^94+186\*X^95+24\*X^97+119\*X^99+30\*X^100+96\*X^101+36\*X^102+132\*X^103+244\*X^105+103\*X^106+93\*X^107+206\*X^108+226\*X^109+118\*X^111+67\*X^112+168\*X^115+70\*X^118+191\*X^120+202\*X^121+205\*X^122+235\*X^124+176\*X^126+119\*X^127+37\*X^128+5\*X^130+138\*X^133+42\*X^134+203\*X^136+175\*X^137+109\*X^139+33\*X^140+224\*X^141+155\*X^143+117\*X^144+5\*X^145+188\*X^146+126\*X^147+22\*X^149+45\*X^151+87\*X^152+212\*X^153+1

```

69*X^154+170*X^156+88*X^157+102*X^158+28*X^160+24*X^161+228*X^162+10*X^163+82*X^1
64+47*X^165+86*X^168+245*X^169+178*X^171+223*X^172+223*X^173+227*X^174+203*X^175+
246*X^176+208*X^177+73*X^180+118*X^182+38*X^185+152*X^186+187*X^187+228*X^188+153
*X^189+10*X^191+25*X^192+253*X^193+23*X^196+64*X^198+12*X^199+49*X^200+169*X^202+
128*X^203+75*X^204+90*X^205+177*X^206+89*X^207+41*X^209+87*X^210+224*X^211+100*X^
212+59*X^214+190*X^215+79*X^216+234*X^218+175*X^219+68*X^221+89*X^222+68*X^224+22
1*X^225+196*X^227+168*X^228+94*X^229+58*X^230+125*X^231+54*X^232+165*X^233+119*X^
234+153*X^235+36*X^236+117*X^237+32*X^238+159*X^239+208*X^240+224*X^241+161*X^242
+176*X^243+1*X^244+98*X^245+235*X^246+203*X^247+152*X^248+12*X^249+14*X^250+40*X^
251+154*X^252+6*X^254+138*X^255+181*X^256+20*X^257+223*X^258+209*X^261+55*X^263+2
13*X^264+230*X^265+168*X^267+247*X^268+110*X^269+109*X^270+4*X^271+105*X^272+228*
X^273+222*X^278+211*X^279+62*X^280+253*X^283+152*X^284+146*X^285+32*X^286+255*X^2
87+69*X^288+47*X^291+164*X^293+211*X^294+1*X^295+197*X^297+41*X^300+172*X^301+137
*X^302+146*X^303+194*X^305+49*X^306+71*X^310+80*X^311+66*X^312+223*X^313+171*X^31
6+49*X^317+241*X^318+7*X^320+35*X^322+62*X^324+56*X^325+129*X^326+106*X^327+111*X
^328+13*X^330+118*X^334+14*X^336+225*X^337+241*X^338+231*X^339+59*X^341+149*X^342
+245*X^343+202*X^344+252*X^345+155*X^348+241*X^349+129*X^350+242*X^351+24*X^352+7
8*X^353+150*X^354+70*X^356+122*X^358+76*X^359+168*X^360+69*X^361+146*X^362+241*X^
363+251*X^364+212*X^365+96*X^366+210*X^367+66*X^368+30*X^369+230*X^370+193*X^371+
126*X^372+55*X^373+216*X^374+19*X^376+103*X^377+4*X^378+116*X^379+2*X^381+178*X^3
83+68*X^385+33*X^386+129*X^387+246*X^388+172*X^389+183*X^390+72*X^391+183*X^392+2
18*X^393+224*X^394+86*X^396+200*X^397+189*X^398+79*X^400+58*X^401+1*X^402+88*X^40
3+35*X^404+217*X^405+106*X^406+90*X^408+25*X^409+202*X^410+5*X^411+30*X^413+26*X^
415+202*X^417+57*X^418+141*X^419+212*X^420+65*X^423+8*X^424+187*X^425+230*X^426+5
3*X^429+61*X^430+156*X^431+233*X^432+194*X^433+207*X^434+253*X^435+104*X^436+51*X
^441+20*X^442+227*X^444+32*X^445+83*X^447+251*X^449+48*X^450+216*X^451+86*X^452+8
4*X^453+33*X^454+20*X^456+123*X^458+157*X^460+188*X^461+63*X^463+42*X^466+232*X^4
67+179*X^468+153*X^470+137*X^473+251*X^475+105*X^477+133*X^478+138*X^479+43*X^480
+44*X^482+192*X^483+235*X^485+198*X^486+110*X^487+164*X^490+171*X^491+223*X^492+3
2*X^494+133*X^495+177*X^496+179*X^497+179*X^498+225*X^500+89*X^501+97*X^502+105*X
^503+3*X^504+85*X^505+45*X^506+100*X^507+197*X^509+199*X^511+212*X^512+244*X^513+
254*X^514+247*X^516+190*X^517+92*X^518+208*X^520+35*X^521+174*X^523+7*X^524+173*X
^525+82*X^529+89*X^530+40*X^531+17*X^532+179*X^533+120*X^535+78*X^536+19*X^537+23
9*X^539+193*X^540+168*X^542+186*X^543+140*X^544+139*X^545+143*X^547+119*X^548+61*
X^550+180*X^551+1*X^552+145*X^553+178*X^554+83*X^555+50*X^558+195*X^559+164*X^560
+201*X^562+121*X^563+199*X^564+252*X^567+195*X^569+209*X^570+39*X^573+60*X^575+82
*X^576+153*X^577+87*X^578+134*X^580+163*X^581+172*X^582+74*X^584+117*X^585+100*X^
587+201*X^588+253*X^589+161*X^592+65*X^593+13*X^594+159*X^595+104*X^597+55*X^598+
15*X^599+232*X^601+178*X^603+86*X^604+157*X^607+226*X^608+98*X^609+47*X^610+16*X^
611+251*X^612+72*X^614+43*X^616+7*X^617+41*X^618+159*X^619+255*X^620+108*X^621+51
*X^622+6*X^623+214*X^626+128*X^627+74*X^629+211*X^630+171*X^634+21*X^636+130*X^63
7+113*X^638+182*X^639+53*X^640+196*X^641+134*X^645+247*X^648+68*X^649+250*X^650+9
3*X^651+93*X^652+125*X^653+157*X^654+94*X^656+36*X^657+47*X^658+118*X^659+130*X^6
60+23*X^661+241*X^662+131*X^663+210*X^664+45*X^665+227*X^666+248*X^667+95*X^668+1
61*X^670+255*X^671+8*X^672+182*X^673+250*X^675+163*X^676+26*X^677+166*X^678+49*X^
679+109*X^680+225*X^681+195*X^682+52*X^684+31*X^686+238*X^688+131*X^689+178*X^690
+245*X^691+107*X^694+133*X^695+59*X^697+10*X^699+220*X^700+255*X^701+39*X^703+31*
X^704+205*X^705+110*X^706+4*X^707+209*X^708+166*X^709+251*X^712+53*X^713+45*X^714
+30*X^716+248*X^718+219*X^719+136*X^720+224*X^721+89*X^722+61*X^724+12*X^725+179*
X^726+12*X^727+33*X^728+105*X^729+219*X^731+150*X^734+163*X^735+100*X^737+251*X^7
38+200*X^739+42*X^740+252*X^741+218*X^742+175*X^743+132*X^744+52*X^746+234*X^747+
16*X^748+93*X^752+18*X^753+40*X^754+37*X^755+32*X^756+52*X^757+191*X^761+109*X^76
2+227*X^763+185*X^766+129*X^767+17*X^768+95*X^770"

```

857

858

859

860 # Section 10.5 Reed-Muller codes

861

862



```

863
864
865 RM_6_GENERATOR_1="0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,\
866 22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,\
867 46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63"
868
869 RM_6_GENERATOR_2="1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,\
870 41,43,45,47,49,51,53,55,57,59,61,63"
871
872 RM_6_GENERATOR_3="2,3,6,7,18,19,22,23,10,11,14,15,26,27,30,31,34,35,38,\
873 39,42,43,46,47,50,51,54,55,58,59,62,63"
874
875 RM_6_GENERATOR_4="4,6,12,14,36,38,52,54,5,7,13,15,37,39,53,55,20,22,28,\
876 30,44,46,60,62,21,23,29,31,45,47,61,63"
877
878 RM_6_GENERATOR_5="8,9,12,13,24,25,28,29,10,11,14,15,26,27,30,31,40,41,\
879 44,45,56,57,60,61,42,43,46,47,58,59,62,63"
880
881 RM_6_GENERATOR_6="16,18,24,26,48,50,56,58,17,19,25,27,49,51,57,59,20,22,\
882 28,30,52,54,60,62,21,23,29,31,53,55,61,63"
883
884 RM_6_GENERATOR_7="32,34,48,50,33,35,49,51,36,38,52,54,37,39,53,55,40,42,\
885 56,58,41,43,57,59,44,46,60,62,45,47,61,63"
886
887
888
889
890 # Section 10.6 BCH-codes
891
892
893 #CODE_BCH_F8_N21_D5_GENMA
894
895 CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11="\
896 2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,0,0,0,\
897 0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,0,0,\
898 0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,0,\
899 0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,\
900 0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,\
901 0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,\
902 0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,\
903 0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,\
904 0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,\
905 0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,\
906 0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,\
907 0,0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,\
908 0,0,0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,\
909 0,0,0,0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1"
910
911 CODE_BCH_F8_N21_K14_D5_GENMA="\
912 6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,0,0,0,\
913 0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,0,0,\
914 0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,0,\
915 0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,\
916 0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,\
917 0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,\
918 0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,\
919 0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,\
920 0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,\
921 0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,\

```

```
922    0,0,0,0,0,0,0,0,0,6,1,5,5,4,3,1,0,0,0,\
923    0,0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,\
924    0,0,0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,\
925    0,0,0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1"
926
927 CODE_BCH_F8_N63_K9_D43_GENMA="\
928   4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0,0,6,6,6,4,7
     ,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,0,0,0,0,0,0,\
929   0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0,0,6,6,6,4
     ,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,0,0,0,0,0,\
930   0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0,0,6,6,6
     ,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,0,0,0,0,\
931   0,0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0,0,6,6
     ,6,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,0,0,0,\
932   0,0,0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0,0,6
     ,6,6,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,0,0,\
933   0,0,0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0,0
     ,6,6,6,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,0,\
934   0,0,0,0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6,0
     ,0,6,6,6,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,0,\
935   0,0,0,0,0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1,6
     ,0,0,6,6,6,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1,0,\
936   0,0,0,0,0,0,0,4,5,2,2,4,5,6,2,4,2,6,0,0,7,1,5,3,7,1,0,0,5,6,4,7,6,7,7,1,2,6,3,1
     ,6,0,0,6,6,6,4,7,7,0,0,3,4,7,5,6,2,5,1,4,4,1"
937
938 BCH_21_15_GENERATOR_MATRIX="1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
     0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
     0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1,
     1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0,
     0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
     0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
     0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
     1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0,
     0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
     0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
     0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
     1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1"
940
941
942 BCH_21_6_GENERATOR_MATRIX=" 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0,
     0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1,
     0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
     1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1,
     1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1
"
943
944 # Section 10.7 Reed-Solomon codes
945
946
947
948
949 CODE_RS_F8_N7_K5_D3_GENMA="\
950   5,6,1,0,0,0,0,\
951   0,5,6,1,0,0,0,\
952   0,0,5,6,1,0,0,\
953   0,0,0,5,6,1,0,\
954   0,0,0,0,5,6,1"
```

```

955
956
957 RS_8_reduced="\
958 01000110000000000000\
959 00111001000000000000\
960 11001100100000000000\
961 00001000110000000000\
962 00000111001000000000\
963 00011001100100000000\
964 00000001000110000000\
965 00000000111001000000\
966 00000011001100100000\
967 00000000001000110000\
968 00000000000111001000\
969 00000000011001100100\
970 00000000000010001100\
971 00000000000000111001\
972 000000000000110011001"
973
974
975 CODE_21_15_4="\
976 11100010000000000000 \
977 11010001000000000000 \
978 10110000100000000000 \
979 01110000010000000000 \
980 11001000001000000000 \
981 10101000000100000000 \
982 01101000000010000000 \
983 10011000000001000000 \
984 01011000000000100000 \
985 00111000000000010000 \
986 11111000000000001000 \
987 11000100000000001000 \
988 101001000000000000100 \
989 011001000000000000010 \
990 100101000000000000001"
991
992
993 # there are 5 [15,6,6]
994
995 # ago=12
996 CODE_15_6_A="\
997 111111111100000 \
998 111110000010000 \
999 111001100001000 \
1000 110101010000100 \
1001 101010110000010 \
1002 101101001000001"
1003
1004 # ago=12
1005 CODE_15_6_B="\
1006 111111111100000 \
1007 111110000010000 \
1008 111001100001000 \
1009 110101010000100 \
1010 101010110000010 \
1011 011011001000001"
1012
1013 #ago=720:

```

```

1014 CODE_15_6_6_C="\
1015 111111111100000 \
1016 111110000010000 \
1017 111001100001000 \
1018 110101010000100 \
1019 101101001000010 \
1020 100010111000001"
1021
1022 #ago=96:
1023 CODE_15_6_6_D="\
1024 111111111100000 \
1025 111110000010000 \
1026 111001100001000 \
1027 110101010000100 \
1028 101011001000010 \
1029 011001011000001"
1030
1031 #ago=360
1032 CODE_15_6_6_E="\
1033 111111111100000 \
1034 111110000010000 \
1035 111001100001000 \
1036 100111010000100 \
1037 010101110000010 \
1038 010110101000001"
1039
1040
1041
1042 # Section 10.9 Bounds
1043
1044
1045 CODE_GV_N15_K6="\
1046 111111111100000\
1047 111110000010000\
1048 111001100001000\
1049 110101010000100\
1050 101010110000010\
1051 101101001000001"
1052
1053
1054 CODE_GV_N15_K6_CHECK="\
1055 100000000111111\
1056 010000000111100\
1057 001000000111011\
1058 000100000110101\
1059 000010000110010\
1060 000001000101101\
1061 000000100101010\
1062 000000010100110\
1063 000000001100001"
1064
1065
1066
1067 # Chapter 11 Combinatorics
1068
1069
1070
1071 # Section 11.2 Combinatorial Objects
1072

```

```

1073 HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS="0,1,2,3,4,5,6,7,8,9,\
1074 10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,\
1075 53,54,59,60,61,62,67,68,69,70,75,76,79,80,81,82"
1076
1077
1078 HYPEROVAL_16_144="0, 1, 2, 3, 52, 67, 89, 106, 126, \
1079 141, 159, 176, 184, 199, 220, 235, 245, 262"
1080
1081 HYPEROVAL_16_16320="0, 1, 2, 3, 52, 70, 83, 109, 127, \
1082 139, 156, 174, 186, 199, 217, 229, 256, 264"
1083
1084
1085 # Section 11.4 Classification of Configurations and Geometries
1086
1087
1088 DESARGUES_PATH_CANONICAL_ANCESTOR="10 10 3\n0\n1 0\n2 112 119\n3 89 112 119\n4 11
1089 4 118 89 82\n5 106 114 69 107 111\n6 85 105 112 99 83 61\n7 94 105 113 85 35 83 6
1090 0\n8 26 119 55 105 92 79 74 48\n9 119 93 106 15 26 79 55 73 47\n10 0 119 93 106 1
1091 5 26 79 55 73 47\n-1\n"
1092
1093 GEO_BLOCKS_600="Row,C0,C1,C2\n\
1094 0, 0, 5, 14, \n\
1095 1, 1, 6, 10, \n\
1096 2, 2, 7, 11, \n\
1097 3, 3, 8, 12, \n\
1098 4, 4, 9, 13, \n\
1099 5, 0, 6, 13, \n\
1100 6, 1, 7, 14, \n\
1101 7, 2, 8, 10, \n\
1102 8, 3, 9, 11, \n\
1103 9, 4, 5, 12, \n\
1104 10, 0, 7, 12, \n\
1105 11, 1, 8, 13, \n\
1106 12, 2, 9, 14, \n\
1107 13, 3, 5, 10, \n\
1108 14, 4, 6, 11, \n\
1109 15, 0, 8, 11, \n\
1110 16, 1, 9, 12, \n\
1111 17, 2, 5, 13, \n\
1112 18, 3, 6, 14, \n\
1113 19, 4, 7, 10, \n\
1114 20, 0, 9, 10, \n\
1115 21, 1, 5, 11, \n\
1116 22, 2, 6, 12, \n\
1117 23, 3, 7, 13, \n\
1118 24, 4, 8, 14, \n\
1119 END\n"
1120
1121 LSQ_5A.TABLE="0,1,2,3,4,1,2,3,4,0,2,3,4,0,1,3,4,0,1,2,4,0,1,2,3"
1122
1123 LSQ_5B.TABLE="0,1,2,3,4,1,0,4,2,3,2,4,3,1,0,3,2,0,4,1,4,3,1,0,2"
1124
1125
1126 # Chapter 12 Graph theory
1127
1128 # Section 12.1 Creating graphs

```

```

1129
1130 TRIANGLE_GRAPH="0,1,1\n1,0,1\n1,1,0\n"
1131
1132
1133
1134
1135
1136
1137 # Chapter 13 Design theory
1138
1139
1140
1141
1142 # Section 13.2 Assumed symmetry
1143
1144
1145 GEN_C5="1,2,3,4,0,6,7,8,9,5,11,12,13,14,10"
1146
1147
1148
1149 # Section 13.3 Design Theory - Large sets
1150
1151
1152
1153
1154 AG_2_3_BLOCKS="0,13,22,27,35,41,47,53,55,59,71,76"
1155
1156 LARGE_SET_AG_2_3_NEIGHBOR_SET="129,130,133,134,136,\
1157 139,141,142,153,154,156,160,165,166,178,179,183,184,\
1158 185,190,192,194,197,203,204,206,218,221,222,225,227,\
1159 231,248,251,252,255,256,259,261,262,272,277,279,283,\
1160 285,287,299,301,303,305,306,309,313,315,319,320,323,\
1161 325,341,342,343,344,345,349,368,371,375,378,381,383,\
1162 392,393,397,402,403,405,416,419,421,422,425,426,429,\
1163 430,440,443,447,449,453,454,464,467,468,473,474,479,\
1164 490,493,494,497,500,503,513,517,518,520,523,527,536,\
1165 539,541,542,544,547,548,551,563,566,567,571,572,573,\
1166 585,589,590,593,595,596,600,601,603,611,614,615,625,\
1167 629,631,635,637,638,657,659,661,667,668,671,681,683,\
1168 686,689,691,693,705,706,709,710,712,715,717,718,720,\
1169 723,724,729,733,735,747,748,750,752,754,757,777,780,\
1170 781,784,790,791,802,804,807,808,811,814,824,827,828,\
1171 831,832,835,837,838"
1172
1173
1174
1175 # Section 13.4: Design Theory - Delandtsheer Doyen
1176
1177
1178
1179 PP4=-d1 1 -q1 3 -d2 1 -q2 7 -K 5 -problem_label PP4
1180
1181 PP4_GROUP1=-subgroup "1,1,1,1, " "21" -group_label "cyclic21"
1182
1183 PP4_MASK1=\
1184 ▷ -nb_orbits_on_blocks 1 \
1185 ▷ -mask_label "no_mask"
1186
1187

```

```

1188
1189 DELANDTSHEER_DOYEN_PROBLEM_COLBOURN_COLBOURN_7_13= -d1 1 -q1 7 -d2 1 -q2 13 -K 6
    -search_control -W -end -problem_label DD_CC_7_13
1190
1191 DELANDTSHEER_DOYEN_PROBLEM_COLBOURN_COLBOURN_7_13.GROUP1=-subgroup "1,1,1,1, " "9
    1" -group_label "cyclic91"
1192
1193 DELANDTSHEER_DOYEN_PROBLEM_COLBOURN_COLBOURN_7_13.MASK1=\
1194 ▷ -nb_orbits_on_blocks 3 \
1195 ▷ -depth 6 \
1196 ▷ -mask_label "no_mask"
1197
1198
1199
1200 DELANDTSHEER_DOYEN_PROBLEM_27_53= -d1 1 -q1 27 -d2 1 -q2 53 -K 11 -DDx 2 -DDy 1 -
    search_control -W -end
1201
1202 DELANDTSHEER_DOYEN_PROBLEM_27_53.GROUP1=-subgroup \
1203 ▷ "1,1,1,0, 1,3,1,0, 1,9,1,0, 1,0,1,1, -2,0,-4,0" "18603" -group_label "group1"
1204 # mask 1:
1205 # XX.
1206 # X.X+
1207
1208 DELANDTSHEER_DOYEN_PROBLEM_27_53.MASK1=\
1209 ▷ -masktest 3 x ge 1 \
1210 ▷ -masktest 4 x+y ge 3 \
1211 ▷ -depth 4 \
1212 ▷ -mask_label "mask1"
1213
1214 DELANDTSHEER_DOYEN_PROBLEM_3_7= -d1 1 -q1 3 -d2 1 -q2 7 -K 5 -DDx 3 -DDy 1 -searc
    h_control -W -end
1215
1216 DELANDTSHEER_DOYEN_PROBLEM_3_7.GROUP1=-subgroup \
1217 ▷ "1,1,1,0, 1,0,1,1 " "21" -group_label "group-cyclic"
1218
1219 DELANDTSHEER_DOYEN_PROBLEM_3_7.MASK1= -mask_label "mask1" -depth 5
1220
1221
1222
1223
1224
1225
1226 # Chapter 14 Finite geometry
1227
1228 # Section 14.1 Spreads
1229
1230
1231 SPREAD_SET_27_RAO_RAO="\
1232 0,0,0,0,0,0,0,0,0, \
1233 1,1,0,2,1,1,0,0,2, \
1234 1,0,1,1,2,2,0,1,0, \
1235 1,2,2,1,2,0,2,2,2, \
1236 0,0,2,2,2,0,1,2,0, \
1237 1,1,2,0,2,1,2,1,0, \
1238 0,1,0,1,0,1,0,2,1, \
1239 2,0,2,0,0,2,1,1,0, \
1240 2,2,2,0,1,1,0,1,2, \
1241 2,0,0,1,0,2,1,2,1, \
1242 0,2,2,2,2,2,2,0,2, \

```

```

1243 2,1,2,0,2,0,2,0,1, \
1244 0,1,2,2,0,1,0,1,1, \
1245 1,0,0,0,1,0,0,0,1, \
1246 2,1,0,1,2,1,0,2,0, \
1247 0,2,0,0,2,2,1,1,2, \
1248 0,0,1,0,1,2,2,2,1, \
1249 2,0,1,2,2,1,1,0,1, \
1250 0,1,1,1,1,0,1,2,2, \
1251 2,2,0,2,0,0,0,2,2, \
1252 2,1,1,1,1,2,2,1,2, \
1253 2,2,1,2,1,0,2,0,0, \
1254 1,2,0,2,0,2,1,0,0, \
1255 1,2,1,1,0,0,1,1,1, \
1256 0,2,1,1,1,1,2,2,0, \
1257 1,1,1,0,0,1,1,0,2, \
1258 1,0,2,2,1,2,2,1,1 \
1259 "
1260
1261 SPREAD_S27_RAO_RAO="\
1262 0, 33879, 5418, 13103, 30556, 22107, 27225, 4045, 24924, 31961, \
1263 3196, 30100, 28081, 25862, 1339, 6696, 8242, 11747, 14000, 14705, \
1264 9784, 17843, 20772, 9271, 19413, 18678, 16109, 23924"
1265
1266
1267
1268
1269
1270 SPREADS_27_ISO_0="\
1271 0, 33879, 1339, 2678, 3994, 7671, 10180, 5862, 9524, 6852, 22243, \
1272 12745, 24295, 11062, 13615, 23894, 15056, 29367, 16429, 31521, 17726, \
1273 31103, 18887, 26333, 19566, 28400, 21531, 27228"
1274
1275 SPREADS_27_ISO_1="\
1276 0, 33879, 1339, 2678, 3994, 7671, 10182, 5761, 6796, 9327, 15339, \
1277 31914, 24415, 12713, 22748, 11666, 13353, 23555, 30103, 16395, 17827, \
1278 30790, 18254, 26422, 20046, 28112, 20900, 26801"
1279
1280 SPREADS_27_ISO_2="\
1281 0, 33879, 1339, 2678, 3994, 7671, 10182, 5817, 6796, 9276, 23891, \
1282 15368, 11666, 22124, 12713, 24415, 13353, 29619, 15910, 31914, 17030, \
1283 30931, 19213, 26422, 19905, 28112, 21217, 27545"
1284
1285 SPREADS_27_ISO_3="\
1286 0, 33879, 1339, 2678, 3994, 7671, 10625, 6590, 9476, 5576, 24688, \
1287 23043, 10996, 22124, 12723, 13522, 15421, 29894, 16532, 32442, 17997, \
1288 31015, 18311, 26109, 19807, 28113, 21220, 27195"
1289
1290 SPREADS_27_ISO_4="\
1291 0, 33879, 1339, 2678, 3994, 7674, 7051, 10666, 9327, 5419, 19806, \
1292 21332, 22124, 13353, 24415, 12401, 11062, 23717, 15056, 29660, 16395, \
1293 31950, 17873, 31153, 19212, 26221, 28515, 26708"
1294
1295 SPREADS_27_ISO_5="\
1296 0, 33879, 1339, 2678, 3994, 7988, 5333, 6672, 10666, 9327, 11062, \
1297 22124, 13353, 15056, 16395, 17347, 31055, 27061, 24415, 12401, 23076, \
1298 30103, 32394, 19041, 26109, 20380, 28400, 21332"
1299
1300 SPREADS_27_ISO_6="\
1301 0, 33879, 1339, 2679, 3992, 7672, 31140, 6913, 13513, 23167, 5653, \

```



```

1302 10607, 9131, 11225, 22548, 13074, 24645, 15124, 29345, 16226, 31506, \
1303 17684, 18732, 26116, 19458, 28361, 21571, 27680"
1304
1305
1306
1307 # Section 14.3 Packings
1308
1309
1310
1311
1312
1313
1314 # nice generators, from Michael Epstein:
1315
1316 PGL_4_5.SUBGROUP_3B.ME--PGL 4 5 \
1317 ▷ -subgroup_by_generators "3B" 3 1 \
1318 ▷ "1,0,0,0, 0,1,0,0, 0,0,2,2, 0,0,4,2"
1319
1320 PGL_4_5.SUBGROUP_3B.ME.NORMALIZER--PGL 4 5 \
1321 ▷ -subgroup_by_generators "normalizer_3B" "5760" 8 \
1322 ▷ "1,0,0,0,0,4,0,0,0,0,1,0,0,0,0,1, \
1323 ▷ 1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,4, \
1324 ▷ 1,0,0,0,0,4,0,0,0,0,4,0,0,0,0,4, \
1325 ▷ 1,0,0,0,0,1,0,0,0,0,3,0,0,0,0,3, \
1326 ▷ 1,0,0,0,0,3,0,0,0,0,1,0,0,0,0,1, \
1327 ▷ 1,0,0,0,0,1,0,0,0,0,2,4,0,0,2,3, \
1328 ▷ 1,0,0,0,4,4,0,0,0,0,1,0,0,0,0,1, \
1329 ▷ 0,1,0,0,1,0,0,0,0,0,4,0,0,0,0,4,"
1330
1331 PGL_4_5.SUBGROUP_31.ME--PGL 4 5 \
1332 ▷ -subgroup_by_generators "31" 31 1 \
1333 ▷ "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3"
1334
1335 PGL_4_5.SUBGROUP_31.ME.NORMALIZER--PGL 4 5 \
1336 ▷ -subgroup_by_generators "normalizer_31" "372" 4 \
1337 ▷ "1,0,0,0,0,4,0,0,0,0,4,0,0,0,0,4, \
1338 ▷ 1,0,0,0,0,3,0,0,0,0,3,0,0,0,0,3, \
1339 ▷ 1,0,0,0,0,4,0,0,0,0,2,1,0,3,2,4, \
1340 ▷ 1,0,0,0,0,0,1,0,0,0,0,1,0,1,1,3,"
1341
1342
1343
1344
1345
1346 # Section 14.4 BLT-Sets
1347
1348
1349 BLT.ORDER_Q="3,5,7,9,11,13,17,19,23,25,27,\
1350 29,31,37,41,43,47,49,53,59,61,67,71,73"
1351
1352 BLT.NUMBER_ISO="1,2,2,3,4,3,6,5,9,6,6,\
1353 9,8,7,10,6,10,8,8,9,5,6,8,5"
1354
1355
1356
1357 # Chapter 15 Computer Science Primitives
1358
1359 # Section 15.2 Diophantine systems
1360

```

```

1361
1362 TEST_SYSTEM="\
1363 0,1,0,1,0,0, \
1364 0,0,1,0,1,0, \
1365 1,0,1,0,0,0, \
1366 0,1,0,1,0,1, \
1367 1,0,0,0,0,1, \
1368 1,0,1,0,0,0, \
1369 0,1,0,0,1,1"
1370
1371
1372
1373
1374 # Chapter 16 Canonical Forms with Nauty
1375
1376 # Section 16.2 Canonical forms of objects in projective space
1377
1378
1379
1380
1381 SUZUKI_8_GENERATORS="\
1382 1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1, \
1383 1,0,0,0,0,0,6,0,0,0,0,2,0,0,0,0,3,0, \
1384 1,0,0,0,1,1,1,0,0,0,1,0,1,0,0,1,0, \
1385 1,0,0,0,3,6,2,2,5,0,2,0,3,0,6,3,2, \
1386 0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,2"
1387 # group order 87360 = 3 * 29120
1388
1389
1390
1391 # Section 16.3 Canonical Forms of Incidence Geometries
1392
1393
1394
1395 FILE_24.3.TFC_INC="24 24 72\
1396 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
1397 132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
1398 258 259 269 272 282 293 300 308 318 325 333 342 352 358 \
1399 367 379 381 392 398 400 417 428 429 442 443 450 466 471 \
1400 479 492 497 502 517 519 521 542 548 551 571 574 575 \
1401 48\
1402 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
1403 132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
1404 258 259 269 272 281 293 301 308 318 324 327 342 354 357 \
1405 367 373 378 392 400 403 417 419 430 442 446 447 466 472 \
1406 479 492 500 503 518 525 526 545 549 551 571 572 574 \
1407 48\
1408 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
1409 132 146 157 158 171 175 179 195 201 207 220 226 232 244 \
1410 257 258 269 274 277 293 300 307 318 323 329 342 352 356 \
1411 367 374 381 392 397 406 416 423 431 441 450 454 468 476 \
1412 477 494 499 503 519 521 525 544 547 550 570 572 575 \
1413 144\
1414 \n-1 3"
1415
1416
1417 # Section 16.5 Canonical forms of linear codes
1418
1419 # consider the binary code with generator matrix:

```

```
1420 # 1 0 1
1421 # 0 1 1
1422
1423 CODE_N3_K2_Q2_GENMA="1,0,1, 0,1,1"
1424
1425 CODE_N6_K3_Q2_GENMA="\
1426 111100\
1427 110010\
1428 101001"
1429
1430
1431
1432 # Chapter 17 Interfaces
1433
1434
1435 # Section 17.2 Poyvray interface
1436
1437 # povray colors:
1438 POLISHED_CHROME_WHITE=\
1439 ▷ "texture{ Polished_Chrome pigment{quick_color White} }"
1440
1441 YELLOW_TRANSPARENT=\
1442 ▷ "texture{ pigment{ color Yellow transmit 0.7 } \
1443 ▷ finish {diffuse 0.9 phong 0.6} }"
1444
1445 COLOR_RED=\
1446 ▷ "texture{ pigment{ color Red } \
1447 ▷ finish {diffuse 0.9 phong 0.6} }"
1448
1449 COLOR_RED_SHINY=\
1450 ▷ "texture{ pigment{ color Red } \
1451 ▷ finish { diffuse 0.9 phong 1} }"
1452
1453 COLOR_GREEN_SHINY=\
1454 ▷ "texture{ pigment{ color Green } \
1455 ▷ finish { diffuse 0.9 phong 1} }"
1456
1457 COLOR_BLUE_SHINY=\
1458 ▷ "texture{ pigment{ color Blue } \
1459 ▷ finish { diffuse 0.9 phong 1} }"
1460
1461 COLOR_YELLOW_SHINY=\
1462 ▷ "texture{ pigment{ color Yellow } \
1463 ▷ finish { diffuse 0.9 phong 1} }"
1464
1465 COLOR_BLACK_SHINY=\
1466 ▷ "texture{ pigment{ color Black } \
1467 ▷ finish { diffuse 0.9 phong 1} }"
1468
1469 COLOR_RED_SEE_THROUGH=\
1470 ▷ "texture{ pigment{ color Red transmit 0.5 } \
1471 ▷ finish { diffuse 0.9 phong 1} }"
1472
1473 COLOR_GREEN_SEE_THROUGH=\
1474 ▷ "texture{ pigment{ color Green transmit 0.5 } \
1475 ▷ finish { diffuse 0.9 phong 1} }"
1476
1477 COLOR_BLUE_SEE_THROUGH=\
1478 ▷ "texture{ pigment{ color Blue transmit 0.5 } \
```

```

1479 ▷ finish { diffuse 0.9 phong 1}]"
1480
1481 COLOR_YELLOW_SEE_THROUGH=\
1482 ▷ "texture{ pigment{ color Yellow transmit 0.5 } \
1483 ▷ finish { diffuse 0.9 phong 1}]"
1484
1485 COLOR_YELLOW_THICK=\
1486 ▷ "texture{ pigment{ color Yellow } \
1487 ▷ finish { diffuse 0.9 phong 1}]"
1488
1489 COLOR_BLACK_NO_SHADOW=\
1490 ▷ "texture{ pigment{Black} } no_shadow"
1491
1492 SURFACE_COLOR=\
1493 ▷ "texture{ pigment{ White*0.5 } \
1494 ▷ finish {ambient 0.4 diffuse 0.5 roughness 0.001 \
1495 ▷ reflection 0.1 specular .8} }]"
1496
1497 SURFACE_COLOR_SEETHROUGH=\
1498 ▷ "texture{ pigment{ White*0.5 transmit 0.5 } \
1499 ▷ finish {ambient 0.4 diffuse 0.5 roughness 0.001 \
1500 ▷ reflection 0.1 specular .8} }]"
1501
1502 COLOR_GOLD=\
1503 ▷ "texture{ pigment{ Gold } finish \
1504 ▷ {ambient 0.4 diffuse 0.5 roughness 0.001 \
1505 ▷ reflection 0.1 specular .8} }]"
1506
1507 COLOR_TURQUOISE=\
1508 ▷ "texture{ pigment{Cyan*1.3} \
1509 ▷ finish {ambient 0.4 diffuse 0.6 roughness 0.001 \
1510 ▷ reflection 0 specular .8} }]"
1511
1512 MONKEY_SADDLE_CUBIC="1,0,0,0,-3,0,0,0,0,0,0,0,0,0,0,0,0,-1,0"
1513
1514 ECKARDT_CUBIC_DEFORM1_LEX="0, 10, 0, -8, 10, \
1515 25, 2, 0, -20, -8, -20, -10, -24, 10, -2, 12, \
1516 0, -8, 8, 16"
1517
1518 ECKARDT_CUBIC_DEFORM2_LEX="0, -5, 0, -5, -5, \
1519 10, -1, 0, 10, 4, 10, 5, 3, -5, 1, -6, 0, -5, \
1520 -4, 1"
1521
1522 KUMMER_QUARTIC_LEX_35="-2,0,0,0,2,0,0,2,0,2,0,0,\
1523 0,0,0,0,0,0,0,-2,0,0,2,0,2,0,0,0,0,-2,0,2,0,-2"
1524
1525
1526
1527
1528 BEAUVILLE_QUINTIC_LEX_56="-44, 228, 400, 315, -396, -852, \
1529 -512, -553, -1050, -354, 284, 504, -62, -707, -1390, -1010, \
1530 281, -167, -1644, -1024, -72, -196, 192, 373, 322, 78, 150, \
1531 966, 1540, 348, -475, -492, 1063, 1550, 390, 0, 96, 3, -337, \
1532 -426, -66, 425, 673, -156, -216, -223, -60, 1543, 1998, 618, \
1533 263, -250, -919, 557, 1800, 741"
1534
1535 ENDRASS_OCTIC_LEX_165="-93.2548,0,0,0,-309.019,0,0,527.529,0,395.647,\
1536 0,0,0,0,0,0,0,0,-687.529,0,0,1582.59,0,1186.94,0,0,0,0,-1055.06,0,\
1537 -1582.59,0,-593.47,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-309.019,\

```

```

1538 0,0,1582.59,0,1186.94,0,0,0,0,-2110.12,0,-3165.17,0,-1186.94,0,0,0,0,\
1539 0,874.039,0,1560.63,0,1677.92,0,343.362,0,0,0,0,0,0,0,0,0,0,0,0,\
1540 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-93.2548,0,0,527.529,0,395.647,\
1541 0,0,0,0,-1055.06,0,-1582.59,0,-593.47,0,0,0,0,0,874.039,0,1560.63,0,\
1542 1677.92,0,343.362,0,0,0,0,0,0,0,-256,0,-468.077,0,-789.019,0,\
1543 -525.726,0,0.941125"
1544
1545
1546 # Clebsch map up for surface created using arc lifting
1547 # We take a circle of radius r centered at the origin in the affine real plane
1548 # and map it up on the surface.
1549 # The Clebsch surface has
1550 # a = d = 2.618033988 = (3+sqrt(5))/2
1551 # b = c = 1.618033988 = (1+sqrt(5))/2
1552 #
1553 CLEBSCH.A=2.618033988
1554
1555 CLEBSCH.D=2.618033988
1556
1557 CLEBSCH.B=1.618033988
1558
1559 CLEBSCH.C=1.618033988
1560
1561 TWO.PI=6.283185308
1562
1563
1564 # to go from the arclifting surface to the defining equation:
1565 #Matrix(4, 4, [[-0.44721360215312733, 1.1708204000530853, 1.1708204000530853, -0.
    4472135957999158], [-1.1708204000530853, 0.4472136021531272, 1.4472136021531272,
    0.4472135957999158], [4.2360680044124255, -4.2360680044124255, -4.236068004412425
    5, 0.], [1.6180340022062127, -2.6180340022062127, -1.6180340022062127, 0.]])
1566 #-0.44721360215312733, 1.1708204000530853, 1.1708204000530853, -0.447213595799915
    8
1567 #-1.1708204000530853, 0.4472136021531272, 1.4472136021531272, 0.4472135957999158
1568 #4.2360680044124255, -4.2360680044124255, -4.2360680044124255, 0.
1569 #1.6180340022062127, -2.6180340022062127, -1.6180340022062127, 0.
1570
1571
1572 T00=-0.44721360215312733
1573
1574 T01=1.1708204000530853
1575
1576 T02=1.1708204000530853
1577
1578 T03=-0.4472135957999158
1579
1580 T10=-1.1708204000530853
1581
1582 T11=0.4472136021531272
1583
1584 T12=1.4472136021531272
1585
1586 T13=0.4472135957999158
1587
1588 T20=4.2360680044124255
1589
1590 T21=-4.2360680044124255
1591
1592 T22=-4.2360680044124255

```

```

1593
1594 T23=0.
1595
1596 T30=1.6180340022062127
1597
1598 T31=-2.6180340022062127
1599
1600 T32=-1.6180340022062127
1601
1602 T33=0.
1603
1604
1605 CLEBSCH_CUBICS=\
1606 ▷ push b push b mult push d push c push m mult add mult \
1607 ▷ push b push c push d push d push m mult mult add mult \
1608 ▷ push a push d push d push m add mult mult add add \
1609 ▷ push a push c push m mult add mult \
1610 ▷ store c001 \
1611 ▷ push b push d mult \
1612 ▷ push b push 1 push m push c mult add mult \
1613 ▷ push d push a push 1 push m mult add mult add \
1614 ▷ push m push a mult add push c add \
1615 ▷ push c push m push a mult add \
1616 ▷ mult mult \
1617 ▷ store c002 \
1618 ▷ push b \
1619 ▷ push d push c push a push m mult add mult \
1620 ▷ push c push a push m push 1 mult add mult add mult \
1621 ▷ push a push d mult push c push 1 push m mult add mult \
1622 ▷ push m mult add \
1623 ▷ push a push c push m mult add mult \
1624 ▷ store c011 \
1625 ▷ push b push b push c mult mult \
1626 ▷ push 1 push d push m mult add mult \
1627 ▷ push a push b mult push c push d push d push m mult mult add mult \
1628 ▷ push m mult add \
1629 ▷ push a push d mult push c push d push m mult add mult add \
1630 ▷ push a push c push m mult add mult \
1631 ▷ store c012 \
1632 ▷ push m \
1633 ▷ push b push d push m mult add push c mult \
1634 ▷ push d push b push 1 push m mult add mult push m mult add push a mult \
1635 ▷ push b push c mult push d push 1 push m mult add mult add mult \
1636 ▷ push b push d push m mult add mult \
1637 ▷ store d001 \
1638 ▷ push m \
1639 ▷ push d push c push m mult add push a push a mult mult \
1640 ▷ push c push c mult push d push m mult add push a mult add \
1641 ▷ push m push b push c mult mult push c push 1 push m mult add mult add mult \
1642 ▷ push b push d push m mult add mult \
1643 ▷ store d011 \
1644 ▷ push m \
1645 ▷ push c push d mult push d push m mult add push a push a mult mult \
1646 ▷ push c push c mult push d push m mult add push a push b push m mult mult a
    dd \
1647 ▷ push b push d push c push m mult add mult push c push m mult mult add \
1648 ▷ push b push d push m mult add mult mult \
1649 ▷ store d012 \
1650 ▷ push d push 1 push m mult add push a mult push m push b mult push 1 add push c

```

```

    mult add \
1651 ▷ push b add push m push d mult add \
1652 ▷ push a push c mult mult \
1653 ▷ push b push d push m mult add mult \
1654 ▷ store d112 \
1655 ▷ push m \
1656 ▷ push b push d push m mult add push c mult push d push b push 1 push m mult add
    mult \
1657 ▷ push m mult add push a mult push b push c mult push d push 1 push m mult add mu
    lt add \
1658 ▷ push b push d push m mult add mult mult \
1659 ▷ store m002 \
1660 ▷ push m \
1661 ▷ push d push c push m mult add push a push a mult mult \
1662 ▷ push c push c mult push d push m mult add push a mult add \
1663 ▷ push b push c push m mult mult push c push 1 push m mult add mult add \
1664 ▷ push b push d push m mult add mult mult \
1665 ▷ store m012 \
1666 ▷ push m\
1667 ▷ push c push d mult push d push m mult add push a push a mult mult \
1668 ▷ push m push c push c mult push d push m mult add push a push b mult mult mult a
    dd \
1669 ▷ push m push b push d push c push m mult add push c mult mult mult add \
1670 ▷ push b push d push m mult add mult mult \
1671 ▷ store m022 \
1672 ▷ push d push 1 push m mult add push a mult \
1673 ▷ push m push b mult push 1 add push c mult add \
1674 ▷ push b add push m push d mult add \
1675 ▷ push a push c mult mult \
1676 ▷ push b push d push m mult add mult \
1677 ▷ store m122 \
1678 ▷ push m push a mult push c add push d mult push c push a push 1 push m mult add
    mult add \
1679 ▷ push b mult \
1680 ▷ push m push a push d mult mult push c push 1 push m mult add mult add \
1681 ▷ push b push d push m mult add mult \
1682 ▷ store n002 \
1683 ▷ push m \
1684 ▷ push c push d push m mult add push b mult push m push d push c push 1 push m mu
    lt add mult mult add \
1685 ▷ push a mult \
1686 ▷ push b push c mult push d push 1 push m mult add mult add mult \
1687 ▷ push a push b push c push m mult push d push m mult add add add mult \
1688 ▷ store n012 \
1689 ▷ push c push d push m mult add push b mult \
1690 ▷ push m push d push c push 1 push m mult add mult mult add \
1691 ▷ push a mult \
1692 ▷ push b push c mult push d push 1 push m mult add mult add \
1693 ▷ push a push d mult push m push b push c mult mult add mult \
1694 ▷ store n022 \
1695 ▷ push m \
1696 ▷ push c push d push m mult add push b mult \
1697 ▷ push m push d push c push 1 push m mult add mult mult add \
1698 ▷ push a mult \
1699 ▷ push b push c mult push d push 1 push m mult add mult add \
1700 ▷ push m push a mult push c add mult mult \
1701 ▷ store n112 \
1702 ▷ push m \
1703 ▷ push c push d push m mult add push b mult \

```

```

1704 ▷ push m push d push c push 1 push m mult add mult mult add \
1705 ▷ push a mult \
1706 ▷ push b push c mult push d push 1 push m mult add mult add \
1707 ▷ push a push d mult push m push b push c mult mult add mult mult \
1708 ▷ store n122
1709
1710
1711
1712 # Section 17.5 The Gnuplot interface
1713
1714 GNUPLOT_TEST_INPUT="Row,Curve-1,Curve-2\n1,2,4\n2,6,8\n3,9,10\n4,13,17\n5,17,20\n
1715 6,19,25\n7,23,30\nEND\n"
1716
1717 # Chapter 18 Mathematical Data in Orbiter
1718
1719 # Section 18.1 Mathematical on Cubic Surfaces
1720
1721 # Section 18.2 Mathematical on BLT-sets
1722
1723 BLT_ORDER_Q="3,5,7,9,11,13,17,19,23,25,27,29,31,37,41,43,47,49,53,59,61,67,71,73"
1724
1725 BLT_NUMBER_ISO="1,2,2,3,4,3,6,5,9,6,6,9,8,7,10,6,10,8,8,9,5,6,8,5"
1726
1727
1728
1729
1730 #####
1731 # Chapter 2 - Getting Started
1732 #####
1733
1734 test_2:
1735 ▷ make test_2.2
1736 ▷ make test_2.3
1737 ▷ make test_2.4
1738 ▷ make test_2.5
1739 ▷ make test_2.6
1740 ▷ make test_2.7
1741 ▷ make test_2.8
1742 ▷ make test_2.9
1743
1744
1745
1746 #####
1747 # Section 2.2: Orbiter Session
1748 ▷
1749 ▷
1750 SECTION_ORBITER_SESSION:
1751
1752 test_2.2:
1753 ▷ make test_orbiter_session
1754
1755
1756 test_orbiter_session:
1757 ▷ $(ORBITER)
1758
1759
1760

```



```

1761 #####
1762 # Section 2.3: Makefiles and Shell Scripts
1763
1764 SECTION.MAKEFILES_AND_SHELL_SCRIPTS:
1765
1766 test_2.3:
1767
1768
1769 #####
1770 # Section 2.4: Objects and Activities
1771
1772 SECTION.OBJECTS_AND_ACTIVITIES:
1773
1774 test_2.4:
1775 ▷ make example_set
1776 ▷ make object_F_2
1777 ▷ make object_PG_3_2
1778 ▷ make vector_ex
1779
1780
1781 example_set:
1782 ▷ $(ORBITER) -v 2 \
1783 ▷ ▷ -define S -set -here "2,3,5,7,11,13" -end \
1784 ▷ ▷ -print_symbols
1785
1786
1787
1788 object_F_2:
1789 ▷ $(ORBITER) -v 3 -define F -finite_field -q 2 -end
1790
1791
1792 object_PG_3_2:
1793 ▷ $(ORBITER) \
1794 ▷ ▷ -define F -finite_field -q 2 -end \
1795 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end
1796
1797
1798 vector_ex:
1799 ▷ $(ORBITER) -v 2 \
1800 ▷ ▷ -define F -finite_field -q 5 -end \
1801 ▷ ▷ -define v -vector -field F -dense "0,1,2,3,4" -end \
1802 ▷ ▷ -print_symbols
1803
1804
1805
1806
1807
1808
1809
1810 #####
1811 # Section 2.5: Mathematical Data
1812
1813 SECTION.MATHEMATICAL_DATA:
1814
1815
1816 test_2.5:
1817 ▷ #make create_BLT_5.1 #error in init_group
1818 ▷ make create_surface_4.0
1819

```

```

1820 create_BLT_5_1:
1821 ▷ $(ORBITER) -v 2 \
1822 ▷ ▷ -define F -finite_field -q 5 -end \
1823 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
1824 ▷ ▷ -define BLT -BLT_set \
1825 ▷ ▷ ▷ -space 0 -catalogue 1 \
1826 ▷ ▷ -end \
1827 ▷ ▷ -with BLT -do -blt_set_activity \
1828 ▷ ▷ ▷ -report \
1829 ▷ ▷ -end
1830 ▷ pdflatex BLT_catalogue.q5.iso1.tex
1831 ▷ $(OPEN) BLT_catalogue.q5.iso1.pdf
1832
1833
1834
1835 create_surface_4_0:
1836 ▷ $(ORBITER) -v 3 \
1837 ▷ ▷ -define F -finite_field -q 4 -end \
1838 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
1839 ▷ ▷ -define S4_0 -cubic_surface -space P -catalogue 0 -end \
1840 ▷ ▷ -with S4_0 -do \
1841 ▷ ▷ ▷ -cubic_surface_activity \
1842 ▷ ▷ ▷ -report \
1843 ▷ ▷ -end
1844 ▷ pdflatex surface_catalogue.q4.iso0_report.tex
1845 ▷ $(OPEN) surface_catalogue.q4.iso0_report.pdf
1846
1847
1848
1849 #####
1850 # Section 2.6: Set Builder
1851
1852 SECTION.SET_BUILDER:
1853
1854
1855 test_2.6:
1856 ▷ make set_of_primes
1857 ▷ make set_interval
1858
1859 set_of_primes:
1860 ▷ $(ORBITER) -v 2 \
1861 ▷ ▷ -define S -set -here "2,3,5,7,11,13" -end \
1862 ▷ ▷ -print_symbols
1863
1864 set_interval:
1865 ▷ $(ORBITER) -v 2 -define S -set -loop 0 64 1 -end \
1866 ▷ ▷ -print_symbols
1867
1868
1869
1870
1871 #####
1872 # Section 2.7: Vector Builder
1873
1874 SECTION.VECTOR_BUILDER:
1875
1876
1877 test_2.7:
1878 ▷ make vector_example1

```

```

1879 ▷ make vector_example2
1880 ▷ make vector_example_repeat
1881 ▷ make vector_example_all_one_11
1882 ▷ make vector_loop_8
1883 ▷ make vector_loop_odd_16
1884 ▷ make vector_concatenate
1885 ▷ make matrix_example1
1886 ▷ make vector_example_sparse
1887 ▷ make matrix_example2
1888 ▷ make matrix_example_co_1
1889
1890
1891 vector_example1:
1892 ▷ $(ORBITER) -v 2 \
1893 ▷ ▷ -define F -finite_field -q 5 -end \
1894 ▷ ▷ -define v -vector -field F -dense "0,1,2,3,4" -end \
1895 ▷ ▷ -print_symbols
1896
1897
1898 vector_example2:
1899 ▷ $(ORBITER) -v 2 \
1900 ▷ ▷ -define F -finite_field -q 5 -end \
1901 ▷ ▷ -define v -vector -field F -format 2 -dense "0,1,2,3,4,0" -end \
1902 ▷ ▷ -print_symbols
1903
1904 vector_example_repeat:
1905 ▷ $(ORBITER) -v 2 \
1906 ▷ ▷ -define v -vector -repeat "0,1,2,3" 11 -end \
1907 ▷ ▷ -print_symbols
1908
1909
1910 vector_example_all_one_11:
1911 ▷ $(ORBITER) -v 2 \
1912 ▷ ▷ -define v -vector -repeat 1 11 -end \
1913 ▷ ▷ -print_symbols
1914
1915 vector_loop_8:
1916 ▷ $(ORBITER) -v 2 \
1917 ▷ ▷ -define v -vector -loop 0 8 1 -end \
1918
1919 vector_loop_odd_16:
1920 ▷ $(ORBITER) -v 2 \
1921 ▷ ▷ -define v -vector -loop 1 16 2 -end \
1922
1923
1924 vector_concatenate:
1925 ▷ $(ORBITER) -v 2 \
1926 ▷ ▷ -define a -vector -compact "1,2,3" -end \
1927 ▷ ▷ -define b -vector -compact "4,5,6" -end \
1928 ▷ ▷ -define c -vector -compact "7,8,9" -end \
1929 ▷ ▷ -define abc -vector -concatenate a,b,c -end
1930
1931
1932 matrix_example1:
1933 ▷ $(ORBITER) -v 2 \
1934 ▷ ▷ -define v -vector -format 2 \
1935 ▷ ▷ ▷ -dense "1,2,3,4,5,6" -end \
1936 ▷ ▷ -print_symbols
1937

```

```

1938
1939 vector_example_sparse:
1940 ▷ $(ORBITER) -v 2 \
1941 ▷ ▷ -define F -finite_field -q 5 -end \
1942 ▷ ▷ -define v -vector -field F -format 4 -sparse 20 "1,0,1,19" -end \
1943 ▷ ▷ -print_symbols
1944
1945
1946
1947 matrix_example2:
1948 ▷ $(ORBITER) -v 2 \
1949 ▷ ▷ -define F -finite_field -q 2 -end \
1950 ▷ ▷ -define v -vector -field F -format 4 \
1951 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) -end \
1952 ▷ ▷ -print_symbols
1953
1954
1955
1956
1957 matrix_example_co.1:
1958 ▷ $(ORBITER) -v 2 \
1959 ▷ ▷ -define F -finite_field -q 2 -end \
1960 ▷ ▷ -define v -vector -field F -format 22 \
1961 ▷ ▷ ▷ -compact $(CONWAY_GEN1) -end \
1962 ▷ ▷ -print_symbols
1963
1964
1965
1966
1967
1968 #####
1969 # Section 2.8: Vector of Group Elements Builder
1970
1971 SECTION_VECTOR_OF_GROUP_ELEMENTS_BUILDER:
1972
1973
1974 test_2.8:
1975 ▷ make Hirschfeld_Class_2A
1976 ▷ make surface_4.0_export_action
1977
1978
1979 Hirschfeld_Class_2A:
1980 ▷ $(ORBITER) -v 9 \
1981 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
1982 ▷ ▷ -define G -linear_group -PGGL 4 4 \
1983 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
1984 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
1985 ▷ ▷ ▷ -end \
1986 ▷ ▷ -with G -do \
1987 ▷ ▷ -group_theoretic_activity \
1988 ▷ ▷ ▷ -conjugacy_class_of "2A" $(CLASS_2A_REP) \
1989 ▷ ▷ -end
1990
1991
1992
1993
1994
1995 surface_4.0_export_action:
1996 ▷ $(ORBITER) -v 3 \

```

```

1997 ▷ ▷ -define Class2A -vector -file $(FILE.NAME.CLASS.2A) -end \
1998 ▷ ▷ -define F -finite_field -q 4 -end \
1999 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
2000 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
2001 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
2002 ▷ ▷ -define gens.ge -vector.ge \
2003 ▷ ▷ ▷ -action G \
2004 ▷ ▷ ▷ -vector_data Class2A \
2005 ▷ ▷ -end \
2006 ▷ ▷ -with S -do \
2007 ▷ ▷ -cubic_surface_activity \
2008 ▷ ▷ ▷ -export_something_with_group_element "action_on_tritangent_planes" gens.ge
2009 ▷ ▷ -end \
2010 ▷ ▷ -with S -do \
2011 ▷ ▷ -cubic_surface_activity \
2012 ▷ ▷ ▷ -export_something_with_group_element "action_on_double_sixes" gens.ge \
2013 ▷ ▷ -end \
2014 ▷ ▷ -with S -do \
2015 ▷ ▷ -cubic_surface_activity \
2016 ▷ ▷ ▷ -export_something_with_group_element "action_on_lines" gens.ge \
2017 ▷ ▷ -end \
2018
2019
2020 #####
2021 # Section 2.9: Symbolic Algebra
2022
2023 SECTION.FORMULA.BUILDER:
2024
2025 test_2.9:
2026 ▷ make symbolic_poly
2027 ▷ make symbolic_CRC32
2028 ▷ make symbolic_CRC_alfa
2029 ▷ make symbolic_CRC_alfa.draw_trees
2030 ▷ make symbolic_CRC_bravo
2031 ▷ make symbolic_CRC_bravo.draw_trees
2032 ▷ make symbolic_test_4_expand
2033 ▷ make symbolic_test_4_expand.draw
2034 ▷ make symbolic_test_5_expand
2035 ▷ make test_expand_1
2036 ▷ make test_expand_2
2037 ▷ make test_expand_3
2038 ▷ make test_expand_4
2039 ▷ make symbolic_objects_building_on_earlier_objects
2040 ▷ make symbolic_objects_self_referential_1
2041 ▷ make symbolic_objects_self_referential_2
2042 ▷ make symbolic_test_6
2043 ▷ make symbolic_matrix1
2044 ▷ make symbolic_matrix_2x2
2045 ▷ make symbolic_matrix_3x3
2046 ▷ make symbolic_determinant_2x2
2047 ▷ make symbolic_determinant_3x3
2048 ▷ make symbolic_determinant_vandermonde
2049 ▷ make symbolic_characteristic_polynomial_2x2
2050 ▷ make symbolic_characteristic_polynomial_2x2.draw
2051 ▷ make symbolic_characteristic_polynomial_3x3
2052 ▷ make symbolic_object1
2053 ▷ make symbolic_object1.expand
2054 ▷ make symbolic_object1.draw

```

```

2055 ▷ make symbolic_object1_evaluate
2056 ▷ make symbolic_Eckardt_surface
2057 ▷ make symbolic_Eckardt_surface_q13_a3_b1
2058 ▷ make symbolic_Fabcd
2059 ▷ make symbolic_Fabcd.expand
2060 ▷ make symbolic_Fabcd.q31.E18
2061 ▷ make symbolic_Fabcd.q31.E18.with_implicit_substitution
2062 ▷ make symbolic_Fabcd.q31.E18.with_implicit_substitution.do_not_simplify
2063 ▷ make symbolic_matrix_minor
2064 ▷ make symbolic_matrix_nullspace_1
2065 ▷ make symbolic_matrix_nullspace_2
2066 ▷ make symbolic_Clebsch
2067 ▷ make Clebsch_map_up_Fabcd
2068
2069
2070
2071
2072
2073
2074 symbolic_poly:
2075 ▷ $(ORBITER) -v 3 \
2076 ▷ ▷ -define F -finite_field -q 7 -end \
2077 ▷ ▷ -define M -symbolic_object \
2078 ▷ ▷ ▷ -field F \
2079 ▷ ▷ ▷ -text "-X^2-X+1"\
2080 ▷ ▷ -end \
2081
2082
2083
2084 symbolic_CRC32:
2085 ▷ $(ORBITER) -v 3 \
2086 ▷ ▷ -define F -finite_field -q 2 -end \
2087 ▷ ▷ -define M -symbolic_object \
2088 ▷ ▷ ▷ -field F \
2089 ▷ ▷ ▷ -text $(CRC32_ETHERNET_POLY) \
2090 ▷ ▷ -end \
2091
2092
2093 symbolic_CRC_alfa:
2094 ▷ $(ORBITER) -v 3 \
2095 ▷ ▷ -define F -finite_field -q 256 -end \
2096 ▷ ▷ -define M -symbolic_object \
2097 ▷ ▷ ▷ -field F \
2098 ▷ ▷ ▷ -text $(BCH_POLYNOMIAL_ALFA_F256) \
2099 ▷ ▷ -end \
2100
2101
2102
2103
2104 symbolic_CRC_alfa_draw_trees:
2105 ▷ $(ORBITER) -v 3 \
2106 ▷ ▷ -define F -finite_field -q 256 -end \
2107 ▷ ▷ -define M -symbolic_object \
2108 ▷ ▷ ▷ -field F \
2109 ▷ ▷ ▷ -text $(BCH_POLYNOMIAL_ALFA_F256) \
2110 ▷ ▷ -end \
2111 ▷ ▷ -define Mx -symbolic_object \
2112 ▷ ▷ ▷ -field F \
2113 ▷ ▷ ▷ -expand M \

```

```

2114 ▷ ▷ ▷ -write.trees.during.expand \
2115 ▷ ▷ -end
2116 ▷ ▷ pdflatex Mx_0_efsxsfsfscsc.tex
2117 ▷ ▷ $(OPEN) Mx_0_efsxsfsfscsc.pdf
2118
2119
2120 symbolic_CRC_bravo:
2121 ▷ $(ORBITER) -v 3 \
2122 ▷ ▷ -define F -finite.field -q 256 -end \
2123 ▷ ▷ -define M -symbolic.object \
2124 ▷ ▷ ▷ -field F \
2125 ▷ ▷ ▷ -text $(BCH.POLYNOMIAL.BRAVO.F256) \
2126 ▷ ▷ -end \
2127
2128 symbolic_CRC_bravo_draw_trees:
2129 ▷ $(ORBITER) -v 3 \
2130 ▷ ▷ -define F -finite.field -q 256 -end \
2131 ▷ ▷ -define M -symbolic.object \
2132 ▷ ▷ ▷ -field F \
2133 ▷ ▷ ▷ -text $(BCH.POLYNOMIAL.BRAVO.F256) \
2134 ▷ ▷ -end \
2135 ▷ ▷ -define Mx -symbolic.object \
2136 ▷ ▷ ▷ -field F \
2137 ▷ ▷ ▷ -expand M \
2138 ▷ ▷ ▷ -write.trees.during.expand \
2139 ▷ ▷ -end
2140 ▷ ▷ pdflatex Mx_0_efsxsfsfscsc.tex
2141 ▷ ▷ $(OPEN) Mx_0_efsxsfsfscsc.pdf
2142
2143
2144 symbolic_test_4_expand:
2145 ▷ $(ORBITER) -v 3 \
2146 ▷ ▷ -define F -finite.field -q 13 -end \
2147 ▷ ▷ -define M -symbolic.object \
2148 ▷ ▷ ▷ -field F \
2149 ▷ ▷ ▷ -matrix 1 \
2150 ▷ ▷ ▷ -text "(a+b)^2" \
2151 ▷ ▷ -end \
2152 ▷ ▷ -define M1 -symbolic.object \
2153 ▷ ▷ ▷ -field F \
2154 ▷ ▷ ▷ -expand M \
2155 ▷ ▷ ▷ -write.trees.during.expand \
2156 ▷ ▷ -end
2157
2158 symbolic_test_4_expand_draw:
2159 ▷ pdflatex M.tex
2160 ▷ $(OPEN) M.pdf
2161 ▷ pdflatex M1.tex
2162 ▷ $(OPEN) M1.pdf
2163 ▷ pdflatex M1_0.tex
2164 ▷ $(OPEN) M1_0.pdf
2165 ▷ pdflatex M1_0.e.tex
2166 ▷ $(OPEN) M1_0.e.pdf
2167 ▷ pdflatex M1_0.ef.tex
2168 ▷ $(OPEN) M1_0.ef.pdf
2169 ▷ pdflatex M1_0.efs.tex
2170 ▷ $(OPEN) M1_0.efs.pdf
2171 ▷ pdflatex M1_0.efsx.tex
2172 ▷ $(OPEN) M1_0.efsx.pdf

```

```

2173 ▷ pdflatex M1_0_efsxs.tex
2174 ▷ $(OPEN) M1_0_efsxs.pdf
2175 ▷ pdflatex M1_0_efsxsf.tex
2176 ▷ $(OPEN) M1_0_efsxsf.pdf
2177 ▷ pdflatex M1_0_efsxsfs.tex
2178 ▷ $(OPEN) M1_0_efsxsfs.pdf
2179 ▷ pdflatex M1_0_efsxsfsfsc.tex
2180 ▷ $(OPEN) M1_0_efsxsfsfsc.pdf
2181 ▷ pdflatex M1_0_efsxsfsfscs.tex
2182 ▷ $(OPEN) M1_0_efsxsfsfscs.pdf
2183 ▷ pdflatex M1_0_efsxsfsfscsc.tex
2184 ▷ $(OPEN) M1_0_efsxsfsfscsc.pdf
2185
2186
2187 symbolic_test_5_expand:
2188 ▷ $(ORBITER) -v 3 \
2189 ▷ ▷ -define F -finite_field -q 13 -end \
2190 ▷ ▷ -define M -symbolic_object \
2191 ▷ ▷ ▷ -field F \
2192 ▷ ▷ ▷ -text "(a+b)^3" \
2193 ▷ ▷ -end \
2194 ▷ ▷ -define M1 -symbolic_object \
2195 ▷ ▷ ▷ -field F \
2196 ▷ ▷ ▷ -expand M \
2197 ▷ ▷ -end
2198 ▷ pdflatex M.tex
2199 ▷ $(OPEN) M.pdf
2200 ▷ pdflatex M1.tex
2201 ▷ $(OPEN) M1.pdf
2202
2203
2204
2205 test_expand_1:
2206 ▷ $(ORBITER) -v 3 \
2207 ▷ ▷ -define Fq -finite_field -q 31 -end \
2208 ▷ ▷ -define f -symbolic_object \
2209 ▷ ▷ ▷ -field Fq \
2210 ▷ ▷ ▷ -text "a*(x+y)" \
2211 ▷ ▷ -end \
2212 ▷ ▷ -define fe -symbolic_object \
2213 ▷ ▷ ▷ -field Fq \
2214 ▷ ▷ ▷ -expand f \
2215 ▷ ▷ ▷ -write_trees_during_expand \
2216 ▷ ▷ -end
2217 ▷ pdflatex fe_0_efsxsfsfscsc.tex
2218 ▷ $(OPEN) fe_0_efsxsfsfscsc.pdf
2219
2220 test_expand_2:
2221 ▷ $(ORBITER) -v 3 \
2222 ▷ ▷ -define Fq -finite_field -q 31 -end \
2223 ▷ ▷ -define f -symbolic_object \
2224 ▷ ▷ ▷ -field Fq \
2225 ▷ ▷ ▷ -text "a*(-(x-y))" \
2226 ▷ ▷ -end \
2227 ▷ ▷ -define fe -symbolic_object \
2228 ▷ ▷ ▷ -field Fq \
2229 ▷ ▷ ▷ -expand f \
2230 ▷ ▷ ▷ -write_trees_during_expand \
2231 ▷ ▷ -end

```



```

2232 > pdflatex fe_0_efsxsfsfscsc.tex
2233 > $(OPEN) fe_0_efsxsfsfscsc.pdf
2234
2235 test_expand_3:
2236 > $(ORBITER) -v 3 \
2237 > > -define Fq -finite_field -q 31 -end \
2238 > > -define f -symbolic_object \
2239 > > > -field Fq \
2240 > > > -text "a*(-(x-y)+(x+y))" \
2241 > > -end \
2242 > > -define fe -symbolic_object \
2243 > > > -field Fq \
2244 > > > -expand f \
2245 > > > -write_trees_during_expand \
2246 > > -end
2247 > pdflatex fe_0_efs.tex
2248 > $(OPEN) fe_0_efs.pdf
2249 > pdflatex fe_0_efsx.tex
2250 > $(OPEN) fe_0_efsx.pdf
2251 > pdflatex fe_0_efsxsx.tex
2252 > $(OPEN) fe_0_efsxsx.pdf
2253 > pdflatex fe_0_efsxsfs.tex
2254 > $(OPEN) fe_0_efsxsfs.pdf
2255 > pdflatex fe_0_efsxsfsfsc.tex
2256 > $(OPEN) fe_0_efsxsfsfsc.pdf
2257 > pdflatex fe_0_efsxsfsfscsefsx.tex
2258 > $(OPEN) fe_0_efsxsfsfscsefsx.pdf
2259 > pdflatex fe_0_efsxsfsfscsefsxsx.tex
2260 > $(OPEN) fe_0_efsxsfsfscsefsxsx.pdf
2261 > pdflatex fe_0_efsxsfsfscsefsxsfsfscsc.tex
2262 > $(OPEN) fe_0_efsxsfsfscsefsxsfsfscsc.pdf
2263
2264
2265 test_expand_4:
2266 > $(ORBITER) -v 3 \
2267 > > -define Fq -finite_field -q 31 -end \
2268 > > -define f -symbolic_object \
2269 > > > -field Fq \
2270 > > > -text "x*(a+(x*(a+(x*(a+x*(a+x*(a+x*(a+x)))))))" \
2271 > > -end \
2272 > > -define fx -symbolic_object \
2273 > > > -field Fq \
2274 > > > -expand f \
2275 > > > -write_trees_during_expand \
2276 > > -end
2277 > pdflatex fx_0_efsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsc.tex
2278 > $(OPEN) fx_0_efsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsc.pdf
2279
2280
2281 symbolic_objects_building_on_earlier_objects:
2282 > $(ORBITER) -v 3 \
2283 > > -define F -finite_field -q 2 -end \
2284 > > -define a -symbolic_object \
2285 > > > -field F \
2286 > > > -text "x+y" \
2287 > > -end \
2288 > > -define b -symbolic_object \
2289 > > > -field F \
2290 > > > -text "a+z" \

```

```

2291 ▷ ▷ -end \
2292
2293
2294 symbolic_objects_self_referential_1:
2295 ▷ $(ORBITER) -v 3 \
2296 ▷ ▷ -define F -finite_field -q 2 -end \
2297 ▷ ▷ -define x -symbolic_object \
2298 ▷ ▷ ▷ -field F \
2299 ▷ ▷ ▷ -text "x" \
2300 ▷ ▷ -end \
2301
2302 symbolic_objects_self_referential_2:
2303 ▷ $(ORBITER) -v 3 \
2304 ▷ ▷ -define F -finite_field -q 2 -end \
2305 ▷ ▷ -define x -symbolic_object \
2306 ▷ ▷ ▷ -field F \
2307 ▷ ▷ ▷ -text "x" \
2308 ▷ ▷ -end \
2309 ▷ ▷ -define a -symbolic_object \
2310 ▷ ▷ ▷ -field F \
2311 ▷ ▷ ▷ -text "x+a" \
2312 ▷ ▷ -end \
2313
2314
2315 symbolic_test_6:
2316 ▷ $(ORBITER) -v 3 \
2317 ▷ ▷ -define F -finite_field -q 13 -end \
2318 ▷ ▷ -define a -symbolic_object \
2319 ▷ ▷ ▷ -field F \
2320 ▷ ▷ ▷ -text "x" \
2321 ▷ ▷ -end \
2322 ▷ ▷ -define a -symbolic_object \
2323 ▷ ▷ ▷ -field F \
2324 ▷ ▷ ▷ -text "a^2" \
2325 ▷ ▷ -end \
2326 ▷ ▷ -define a -symbolic_object \
2327 ▷ ▷ ▷ -field F \
2328 ▷ ▷ ▷ -text "a^2" \
2329 ▷ ▷ -end \
2330 ▷ ▷ -define a -symbolic_object \
2331 ▷ ▷ ▷ -field F \
2332 ▷ ▷ ▷ -text "a^2" \
2333 ▷ ▷ -end
2334 ▷ pdflatex a.tex
2335 ▷ $(OPEN) a.pdf
2336
2337
2338
2339 symbolic_test_7:
2340 ▷ $(ORBITER) -v 10 \
2341 ▷ ▷ -define Fq -finite_field -q 13 -end \
2342 ▷ ▷ -define M -symbolic_object \
2343 ▷ ▷ ▷ -field Fq \
2344 ▷ ▷ ▷ -text "a*b^-1" \
2345 ▷ ▷ -end \
2346 ▷ ▷ -define values -symbolic_object \
2347 ▷ ▷ ▷ -field Fq \
2348 ▷ ▷ ▷ -text "1,3" \
2349 ▷ ▷ -end \

```

```

2350 ▷ ▷ -define eval -symbolic_object \
2351 ▷ ▷ ▷ -field Fq \
2352 ▷ ▷ ▷ -substitute "a,b" M values \
2353 ▷ ▷ -end
2354 ▷ #pdflatex M.tex
2355 ▷ #$(OPEN) M.pdf
2356
2357
2358
2359 symbolic_matrix1:
2360 ▷ $(ORBITER) -v 3 \
2361 ▷ ▷ -define F -finite_field -q 13 -end \
2362 ▷ ▷ -define M -symbolic_object \
2363 ▷ ▷ ▷ -field F \
2364 ▷ ▷ ▷ -matrix 4 \
2365 ▷ ▷ ▷ -text $(VANDERMONDE_4X4_FORMULA) \
2366 ▷ ▷ -end
2367
2368
2369 symbolic_matrix_2x2:
2370 ▷ $(ORBITER) -v 3 \
2371 ▷ ▷ -define F -finite_field -q 13 -end \
2372 ▷ ▷ -define M -symbolic_object \
2373 ▷ ▷ ▷ -field F \
2374 ▷ ▷ ▷ -matrix 2 \
2375 ▷ ▷ ▷ -text "a,b,c,d" \
2376 ▷ ▷ -end
2377
2378
2379 symbolic_matrix_3x3:
2380 ▷ $(ORBITER) -v 3 \
2381 ▷ ▷ -define F -finite_field -q 13 -end \
2382 ▷ ▷ -define M -symbolic_object \
2383 ▷ ▷ ▷ -field F \
2384 ▷ ▷ ▷ -matrix 3 \
2385 ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
2386 ▷ ▷ -end
2387
2388
2389 symbolic_determinant_2x2:
2390 ▷ $(ORBITER) -v 3 \
2391 ▷ ▷ -define F -finite_field -q 13 -end \
2392 ▷ ▷ -define M -symbolic_object \
2393 ▷ ▷ ▷ -field F \
2394 ▷ ▷ ▷ -matrix 2 \
2395 ▷ ▷ ▷ -text "a,b,c,d" \
2396 ▷ ▷ -end \
2397 ▷ ▷ -define det -symbolic_object \
2398 ▷ ▷ ▷ -field F \
2399 ▷ ▷ ▷ -determinant M \
2400 ▷ ▷ -end \
2401 ▷ ▷ -define det1 -symbolic_object \
2402 ▷ ▷ ▷ -field F \
2403 ▷ ▷ ▷ -expand det \
2404 ▷ ▷ -end
2405 ▷ pdflatex det1.tex
2406 ▷ $(OPEN) det1.pdf
2407
2408

```

```

2409 symbolic_determinant_3x3:
2410 ▷ $(ORBITER) -v 3 \
2411 ▷ ▷ -define F -finite.field -q 13 -end \
2412 ▷ ▷ -define M -symbolic.object \
2413 ▷ ▷ ▷ -field F \
2414 ▷ ▷ ▷ -matrix 3 \
2415 ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
2416 ▷ ▷ -end \
2417 ▷ ▷ -define det -symbolic.object \
2418 ▷ ▷ ▷ -field F \
2419 ▷ ▷ ▷ -determinant M \
2420 ▷ ▷ -end \
2421 ▷ ▷ -define det1 -symbolic.object \
2422 ▷ ▷ ▷ -field F \
2423 ▷ ▷ ▷ -expand det \
2424 ▷ ▷ -end
2425 ▷ pdflatex det1.tex
2426 ▷ $(OPEN) det1.pdf
2427
2428
2429 symbolic_determinant_vandermonde:
2430 ▷ $(ORBITER) -v 3 \
2431 ▷ ▷ -define F -finite.field -q 13 -end \
2432 ▷ ▷ -define M -symbolic.object \
2433 ▷ ▷ ▷ -field F \
2434 ▷ ▷ ▷ -matrix 4 \
2435 ▷ ▷ ▷ -text $(VANDERMONDE_4X4_FORMULA) \
2436 ▷ ▷ -end \
2437 ▷ ▷ -define det -symbolic.object \
2438 ▷ ▷ ▷ -field F \
2439 ▷ ▷ ▷ -determinant M \
2440 ▷ ▷ -end \
2441 ▷ ▷ -define det1 -symbolic.object \
2442 ▷ ▷ ▷ -field F \
2443 ▷ ▷ ▷ -expand det \
2444 ▷ ▷ -end
2445
2446
2447 symbolic_characteristic_polynomial_2x2:
2448 ▷ $(ORBITER) -v 3 \
2449 ▷ ▷ -define F -finite.field -q 13 -end \
2450 ▷ ▷ -define M -symbolic.object \
2451 ▷ ▷ ▷ -field F \
2452 ▷ ▷ ▷ -matrix 2 \
2453 ▷ ▷ ▷ -text "a,b,c,d" \
2454 ▷ ▷ -end \
2455 ▷ ▷ -define p -symbolic.object \
2456 ▷ ▷ ▷ -field F \
2457 ▷ ▷ ▷ -characteristic.polynomial lambda M \
2458 ▷ ▷ -end \
2459 ▷ ▷ -define p1 -symbolic.object \
2460 ▷ ▷ ▷ -field F \
2461 ▷ ▷ ▷ -expand p \
2462 ▷ ▷ ▷ -write.trees.during.expand \
2463 ▷ ▷ -end \
2464 ▷ ▷ -define coeffs -symbolic.object \
2465 ▷ ▷ ▷ -field F \
2466 ▷ ▷ ▷ -collect p1 lambda \
2467 ▷ ▷ -end

```

```

2468
2469 symbolic_characteristic_polynomial_2x2_draw:
2470 ▷ pdflatex p1.tex
2471 ▷ $(OPEN) p1.pdf
2472 ▷ pdflatex p1_0.tex
2473 ▷ $(OPEN) p1_0.pdf
2474 ▷ pdflatex p1_0_e.tex
2475 ▷ $(OPEN) p1_0_e.pdf
2476 ▷ pdflatex p1_0_ef.tex
2477 ▷ $(OPEN) p1_0_ef.pdf
2478 ▷ pdflatex p1_0_efs.tex
2479 ▷ $(OPEN) p1_0_efs.pdf
2480 ▷ pdflatex p1_0_efsx.tex
2481 ▷ $(OPEN) p1_0_efsx.pdf
2482 ▷ pdflatex p1_0_efsxs.tex
2483 ▷ $(OPEN) p1_0_efsxs.pdf
2484 ▷ pdflatex p1_0_efsxsf.tex
2485 ▷ $(OPEN) p1_0_efsxsf.pdf
2486 ▷ pdflatex p1_0_efsxsfs.tex
2487 ▷ $(OPEN) p1_0_efsxsfs.pdf
2488 ▷ pdflatex p1_0_efsxsfsc.tex
2489 ▷ $(OPEN) p1_0_efsxsfsc.pdf
2490 ▷ pdflatex p1_0_efsxsfsfscs.tex
2491 ▷ $(OPEN) p1_0_efsxsfsfscs.pdf
2492 ▷ pdflatex p1_0_efsxsfsfscsc.tex
2493 ▷ $(OPEN) p1_0_efsxsfsfscsc.pdf
2494
2495 symbolic_characteristic_polynomial_3x3:
2496 ▷ $(ORBITER) -v 3 \
2497 ▷ ▷ -define F -finite_field -q 13 -end \
2498 ▷ ▷ -define M -symbolic_object \
2499 ▷ ▷ ▷ -field F \
2500 ▷ ▷ ▷ -matrix 3 \
2501 ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
2502 ▷ ▷ -end \
2503 ▷ ▷ -define p -symbolic_object \
2504 ▷ ▷ ▷ -field F \
2505 ▷ ▷ ▷ -characteristic_polynomial lambda M \
2506 ▷ ▷ -end \
2507 ▷ ▷ -define p1 -symbolic_object \
2508 ▷ ▷ ▷ -field F \
2509 ▷ ▷ ▷ -expand p \
2510 ▷ ▷ -end \
2511 ▷ ▷ -define coeffs -symbolic_object \
2512 ▷ ▷ ▷ -field F \
2513 ▷ ▷ ▷ -collect p1 lambda \
2514 ▷ ▷ -end
2515 ▷ #pdflatex p1.tex
2516 ▷ #$(OPEN) p1.pdf
2517
2518
2519
2520 symbolic_object1:
2521 ▷ $(ORBITER) -v 3 \
2522 ▷ ▷ -define F -finite_field -q 13 -end \
2523 ▷ ▷ -define M -symbolic_object \
2524 ▷ ▷ ▷ -field F \
2525 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2526 ▷ ▷ -end

```

```

2527
2528
2529 symbolic_object1_expand:
2530 ▷ $(ORBITER) -v 3 \
2531 ▷ ▷ -define F -finite_field -q 13 -end \
2532 ▷ ▷ -define M -symbolic_object \
2533 ▷ ▷ ▷ -field F \
2534 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2535 ▷ ▷ -end \
2536 ▷ ▷ -define M1 -symbolic_object \
2537 ▷ ▷ ▷ -field F \
2538 ▷ ▷ ▷ -expand M \
2539 ▷ ▷ -end
2540 ▷ pdflatex M1.tex
2541 ▷ $(OPEN) M1.pdf
2542
2543
2544
2545 symbolic_object1_draw:
2546 ▷ $(ORBITER) -v 3 \
2547 ▷ ▷ -define F -finite_field -q 13 -end \
2548 ▷ ▷ -define M -symbolic_object \
2549 ▷ ▷ ▷ -field F \
2550 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2551 ▷ ▷ -end
2552 ▷ pdflatex sajeeb_tree0.tex
2553 ▷ $(OPEN) sajeeb_tree0.pdf
2554 ▷ pdflatex M.tex
2555 ▷ $(OPEN) M.pdf
2556
2557
2558
2559 symbolic_object1_evaluate:
2560 ▷ $(ORBITER) -v 3 \
2561 ▷ ▷ -define Fq -finite_field -q 13 -end \
2562 ▷ ▷ -define f -symbolic_object \
2563 ▷ ▷ ▷ -field Fq \
2564 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2565 ▷ ▷ -end \
2566 ▷ ▷ -define assignment -symbolic_object \
2567 ▷ ▷ ▷ -field Fq \
2568 ▷ ▷ ▷ -text "1,2,3,4,5,6" \
2569 ▷ ▷ -end \
2570 ▷ ▷ -define eval -symbolic_object \
2571 ▷ ▷ ▷ -field Fq \
2572 ▷ ▷ ▷ -substitute "a,b,c,d,x,y" f assignment \
2573 ▷ ▷ -end \
2574
2575
2576
2577
2578
2579 symbolic_Eckardt_surface:
2580 ▷ $(ORBITER) -v 3 \
2581 ▷ ▷ -define F -finite_field -q 13 -end \
2582 ▷ ▷ -define M -symbolic_object \
2583 ▷ ▷ ▷ -field F \
2584 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2585 ▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \

```

```

2586 ▷ ▷ -end
2587
2588
2589
2590 symbolic_Eckardt_surface.q13.a3.b1:
2591 ▷ $(ORBITER) -v 3 \
2592 ▷ ▷ -define F -finite_field -q 13 -end \
2593 ▷ ▷ -define M -symbolic_object \
2594 ▷ ▷ ▷ -field F \
2595 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2596 ▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \
2597 ▷ ▷ -end \
2598 ▷ ▷ -define L -symbolic_object \
2599 ▷ ▷ ▷ -field F \
2600 ▷ ▷ ▷ -text "3,1" \
2601 ▷ ▷ -end \
2602 ▷ ▷ -define M1 -symbolic_object \
2603 ▷ ▷ ▷ -field F \
2604 ▷ ▷ ▷ -substitute "a,b" M L \
2605 ▷ ▷ -end \
2606 ▷ ▷ -define M2 -symbolic_object \
2607 ▷ ▷ ▷ -field F \
2608 ▷ ▷ ▷ -expand M1 \
2609 ▷ ▷ ▷ -write_trees_during_expand \
2610 ▷ ▷ -end
2611 ▷ pdflatex M2.tex
2612 ▷ $(OPEN) M2.pdf
2613
2614
2615 symbolic_Fabcd:
2616 ▷ $(ORBITER) -v 3 \
2617 ▷ ▷ -define F -finite_field -q 13 -end \
2618 ▷ ▷ -define M -symbolic_object \
2619 ▷ ▷ ▷ -field F \
2620 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2621 ▷ ▷ ▷ -text $(F_abcd_eqn) \
2622 ▷ ▷ -end
2623
2624
2625 symbolic_Fabcd.expand:
2626 ▷ $(ORBITER) -v 3 \
2627 ▷ ▷ -define Fq -finite_field -q 31 -end \
2628 ▷ ▷ -define Fabcd -symbolic_object \
2629 ▷ ▷ ▷ -field Fq \
2630 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2631 ▷ ▷ ▷ -text $(F_abcd_eqn) \
2632 ▷ ▷ -end \
2633 ▷ ▷ -define Fabcd_e -symbolic_object \
2634 ▷ ▷ ▷ -field Fq \
2635 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2636 ▷ ▷ ▷ -expand Fabcd \
2637 ▷ ▷ ▷ -write_trees_during_expand \
2638 ▷ ▷ -end
2639
2640
2641 # q=31
2642 #E=18 (a,b,c,d) = (25,5,5,25)
2643 #E=10 (a,b,c,d) = (14,13,13,14)
2644 #E=9 (a,b,c,d) = (5,3,12,5)

```

```

2645 #E=6 (a,b,c,d) = (2,30,30,2)
2646 #E=4 (a,b,c,d) = (4,2,2,4)
2647 #E=3 (a,b,c,d) = (6,2,3,6)
2648 #E=2 (a,b,c,d) = (2,3,3,2)
2649 #E=1 (a,b,c,d) = (2,3,5,2)
2650 #E=0 (a,b,c,d) = (2,5,9,7)
2651
2652
2653 symbolic_Fabcd.q31_E18:
2654 ▷ $(ORBITER) -v 3 \
2655 ▷ ▷ -define F -finite_field -q 31 -end \
2656 ▷ ▷ -define M -symbolic_object \
2657 ▷ ▷ ▷ -field F \
2658 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2659 ▷ ▷ ▷ -text $(F_abcd_eqn) \
2660 ▷ ▷ -end \
2661 ▷ ▷ -define L -symbolic_object \
2662 ▷ ▷ ▷ -field F \
2663 ▷ ▷ ▷ -text "25,5,5,25" \
2664 ▷ ▷ -end \
2665 ▷ ▷ -define M1 -symbolic_object \
2666 ▷ ▷ ▷ -field F \
2667 ▷ ▷ ▷ -substitute "a,b,c,d" M L \
2668 ▷ ▷ -end \
2669 ▷ ▷ -define M2 -symbolic_object \
2670 ▷ ▷ ▷ -field F \
2671 ▷ ▷ ▷ -expand M1 \
2672 ▷ ▷ ▷ -write_trees_during_expand \
2673 ▷ ▷ -end
2674 ▷ pdflatex M2.tex
2675 ▷ $(OPEN) M2.pdf
2676
2677
2678 # (13 X1 X2 X3) + (22 X1 X2^{2}) + (22 X1^{2} X3) + (22 X1^{2} X2) + (9 X0 X2^{2})
+ (9 X0^{2} X2) + (22 X1 X3^{2})\\
2679
2680
2681
2682 symbolic_Fabcd.q31_E18_with_implicit_substitution:
2683 ▷ $(ORBITER) -v 3 \
2684 ▷ ▷ -define F -finite_field -q 31 -end \
2685 ▷ ▷ -define a -symbolic_object \
2686 ▷ ▷ ▷ -field F \
2687 ▷ ▷ ▷ -text "25" \
2688 ▷ ▷ -end \
2689 ▷ ▷ -define b -symbolic_object \
2690 ▷ ▷ ▷ -field F \
2691 ▷ ▷ ▷ -text "5" \
2692 ▷ ▷ -end \
2693 ▷ ▷ -define c -symbolic_object \
2694 ▷ ▷ ▷ -field F \
2695 ▷ ▷ ▷ -text "5" \
2696 ▷ ▷ -end \
2697 ▷ ▷ -define d -symbolic_object \
2698 ▷ ▷ ▷ -field F \
2699 ▷ ▷ ▷ -text "25" \
2700 ▷ ▷ -end \
2701 ▷ ▷ -define M -symbolic_object \
2702 ▷ ▷ ▷ -field F \

```



```

2703 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2704 ▷ ▷ ▷ -text $(F_abcd_eqn) \
2705 ▷ ▷ -end \
2706 ▷ ▷ -define M2 -symbolic_object \
2707 ▷ ▷ ▷ -field F \
2708 ▷ ▷ ▷ -expand M \
2709 ▷ ▷ -end
2710 ▷ pdflatex M2.tex
2711 ▷ $(OPEN) M2.pdf
2712
2713
2714
2715 symbolic_Fabcd.q31_E18_with_implicit_substitution.do_not_simplify:
2716 ▷ $(ORBITER) -v 3 \
2717 ▷ ▷ -define F -finite_field -q 31 -end \
2718 ▷ ▷ -define a -symbolic_object \
2719 ▷ ▷ ▷ -field F \
2720 ▷ ▷ ▷ -text "25" \
2721 ▷ ▷ -end \
2722 ▷ ▷ -define b -symbolic_object \
2723 ▷ ▷ ▷ -field F \
2724 ▷ ▷ ▷ -text "5" \
2725 ▷ ▷ -end \
2726 ▷ ▷ -define c -symbolic_object \
2727 ▷ ▷ ▷ -field F \
2728 ▷ ▷ ▷ -text "5" \
2729 ▷ ▷ -end \
2730 ▷ ▷ -define d -symbolic_object \
2731 ▷ ▷ ▷ -field F \
2732 ▷ ▷ ▷ -text "25" \
2733 ▷ ▷ -end \
2734 ▷ ▷ -define M -symbolic_object \
2735 ▷ ▷ ▷ -field F \
2736 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2737 ▷ ▷ ▷ -text $(F_abcd_eqn) \
2738 ▷ ▷ ▷ -do_not_simplify \
2739 ▷ ▷ -end
2740
2741
2742
2743
2744 symbolic_matrix_minor:
2745 ▷ $(ORBITER) -v 3 \
2746 ▷ ▷ -define F -finite_field -q 13 -end \
2747 ▷ ▷ -define A -symbolic_object \
2748 ▷ ▷ ▷ -field F \
2749 ▷ ▷ ▷ -matrix 4 \
2750 ▷ ▷ ▷ -text "1,0,0,0,0,0,0,1,x0,x1,x2,x3,a,b,c,d" \
2751 ▷ ▷ -end \
2752 ▷ ▷ -define M -symbolic_object \
2753 ▷ ▷ ▷ -field F \
2754 ▷ ▷ ▷ -minor A 3 1 \
2755 ▷ ▷ -end \
2756
2757
2758 symbolic_matrix_nullspace_1:
2759 ▷ $(ORBITER) -v 3 \
2760 ▷ ▷ -define F -finite_field -q 13 -end \
2761 ▷ ▷ -define A1 -symbolic_object \

```

```

2762 ▷ ▷ ▷ -field F \
2763 ▷ ▷ ▷ -matrix 3 \
2764 ▷ ▷ ▷ -text "1,0,0,0,0,0,0,1,y0,y1,y2,0" \
2765 ▷ ▷ -end \
2766 ▷ ▷ -define M1 -symbolic.object \
2767 ▷ ▷ ▷ -field F \
2768 ▷ ▷ ▷ -symbolic.nullspace A1 \
2769 ▷ ▷ -end \
2770
2771 #0, ((12) * (y2)), (y1), 0
2772
2773
2774 symbolic_matrix_nullspace_2:
2775 ▷ $(ORBITER) -v 3 \
2776 ▷ ▷ -define F -finite_field -q 13 -end \
2777 ▷ ▷ -define A2 -symbolic.object \
2778 ▷ ▷ ▷ -field F \
2779 ▷ ▷ ▷ -matrix 3 \
2780 ▷ ▷ ▷ -text "0,1,0,0,0,0,-1,1,y0,y1,y2,0" \
2781 ▷ ▷ -end \
2782 ▷ ▷ -define M2 -symbolic.object \
2783 ▷ ▷ ▷ -field F \
2784 ▷ ▷ ▷ -symbolic.nullspace A2 \
2785 ▷ ▷ -end \
2786
2787
2788 #(y2), 0, (12) * (y0), ((12) * (y0))
2789
2790 symbolic_Clebsch:
2791 ▷ $(ORBITER) -v 3 \
2792 ▷ ▷ -define F -finite_field -q 13 -end \
2793 ▷ ▷ -define A1 -symbolic.object \
2794 ▷ ▷ ▷ -field F \
2795 ▷ ▷ ▷ -matrix 3 \
2796 ▷ ▷ ▷ -text "1,0,0,0,0,0,0,1,y0,y1,y2,0" \
2797 ▷ ▷ -end \
2798 ▷ ▷ -define A2 -symbolic.object \
2799 ▷ ▷ ▷ -field F \
2800 ▷ ▷ ▷ -matrix 3 \
2801 ▷ ▷ ▷ -text "0,1,0,0,0,0,-1,1,y0,y1,y2,0" \
2802 ▷ ▷ -end \
2803 ▷ ▷ -define A3 -symbolic.object \
2804 ▷ ▷ ▷ -field F \
2805 ▷ ▷ ▷ -matrix 2 \
2806 ▷ ▷ ▷ -text "0,0,1,0,0,0,0,1" \
2807 ▷ ▷ -end \
2808 ▷ ▷ -define M1 -symbolic.object \
2809 ▷ ▷ ▷ -field F \
2810 ▷ ▷ ▷ -symbolic.nullspace A1 \
2811 ▷ ▷ -end \
2812 ▷ ▷ -define M2 -symbolic.object \
2813 ▷ ▷ ▷ -field F \
2814 ▷ ▷ ▷ -symbolic.nullspace A2 \
2815 ▷ ▷ -end \
2816 ▷ ▷ -define L -symbolic.object \
2817 ▷ ▷ ▷ -field F \
2818 ▷ ▷ ▷ -stack_matrices_vertically M1,M2,A3 \
2819 ▷ ▷ -end \
2820

```

```

2821
2822
2823
2824
2825 Clebsch_map_up_Fabcd:
2826 ▷ $(ORBITER) -v 3 \
2827 ▷ ▷ -define F -finite_field -q 13 -end \
2828 ▷ ▷ -define M -symbolic_object \
2829 ▷ ▷ ▷ -field F \
2830 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
2831 ▷ ▷ ▷ -text $(F_abcd_eqn) \
2832 ▷ ▷ -end \
2833 ▷ ▷ -define L -symbolic_object \
2834 ▷ ▷ ▷ -field F \
2835 ▷ ▷ ▷ -text "y0+t*y0,t*y1,t*y2,y2" \
2836 ▷ ▷ -end \
2837 ▷ ▷ -define C1 -symbolic_object \
2838 ▷ ▷ ▷ -field F \
2839 ▷ ▷ ▷ -managed_variables "y0,y1,y2" \
2840 ▷ ▷ ▷ -substitute "X0,X1,X2,X3" M L \
2841 ▷ ▷ -end \
2842 ▷ ▷ -define M -symbolic_object \
2843 ▷ ▷ ▷ -field F \
2844 ▷ ▷ ▷ -expand C1 \
2845 ▷ ▷ ▷ -managed_variables "y0,y1,y2" \
2846 ▷ ▷ ▷ -write_trees_during_expand \
2847 ▷ ▷ -end \
2848 ▷ ▷ -define coeff_t -symbolic_object \
2849 ▷ ▷ ▷ -field F \
2850 ▷ ▷ ▷ -managed_variables "y0,y1,y2" \
2851 ▷ ▷ ▷ -collect M t \
2852 ▷ ▷ -end
2853
2854
2855
2856
2857
2858 #####
2859 # Chapter 3 - Finite Fields
2860 #####
2861
2862
2863 test_3:
2864 ▷ make test_3.1
2865 ▷ make test_3.2
2866 ▷ make test_3.3
2867 ▷ make test_3.4
2868 ▷ make test_3.5
2869
2870
2871
2872
2873 #####
2874 # Section 3.1: Prime Fields
2875
2876 SECTION_PRIME_FIELDS:
2877
2878
2879 test_3.1:

```

```

2880 ▷ make F_2
2881 ▷ make F_3
2882 ▷ make F_5
2883 ▷ make F_5_add_table
2884 ▷ make F_7
2885 ▷ make F_7_tables
2886 ▷ make F_127
2887 ▷ make F_11_product_of_all_nonzero_elements
2888 ▷ make F_7_vandermonde
2889 ▷ make F_101_wo
2890 ▷ make F_1021_wo
2891
2892
2893
2894 F_2:
2895 ▷ $(ORBITER) -v 3 -list_arguments \
2896 ▷ ▷ -define F -finite_field -q 2 -end \
2897 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2898 ▷ pdflatex GF_2.tex
2899 ▷ $(OPEN) GF_2.pdf
2900
2901
2902 F_3:
2903 ▷ $(ORBITER) -v 3 \
2904 ▷ ▷ -define F -finite_field -q 3 -end \
2905 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2906 ▷ #pdflatex GF_3.tex
2907 ▷ #$(OPEN) GF_3.pdf
2908
2909 F_5:
2910 ▷ $(ORBITER) -v 3 \
2911 ▷ ▷ -define F -finite_field -q 5 -end \
2912 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2913 ▷ pdflatex GF_5.tex
2914 ▷ $(OPEN) GF_5.pdf
2915
2916
2917 F_5_add_table:
2918 ▷ $(ORBITER) -v 3 \
2919 ▷ ▷ -define F -finite_field -q 5 -end \
2920 ▷ ▷ -with F -do -finite_field_activity -export_tables -end \
2921 ▷ ▷ -draw_matrix -input_csv_file GF_q5_table_add.csv \
2922 ▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 5 5 -end \
2923
2924
2925 F_7:
2926 ▷ $(ORBITER) -v 3 \
2927 ▷ ▷ -define F -finite_field -q 7 -end \
2928 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2929 ▷ pdflatex GF_7.tex
2930 ▷ $(OPEN) GF_7.pdf
2931
2932
2933 F_7_tables:
2934 ▷ $(ORBITER) -v 3 \
2935 ▷ ▷ -define F -finite_field -q 7 -end \
2936 ▷ ▷ -with F -do -finite_field_activity -export_tables -end
2937 ▷ $(ORBITER) -v 3 \
2938 ▷ ▷ -define all_one -vector -repeat 1 7 -end \

```

```

2939 ▷ ▷ -draw_matrix -input.csv.file GF_q7_table_add.csv \
2940 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
2941 ▷ ▷ ▷ -partition 3 all_one all_one \
2942 ▷ ▷ -end \
2943 ▷ ▷ -draw_matrix -input.csv.file GF_q7_table_mul.csv \
2944 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
2945 ▷ ▷ ▷ -partition 3 all_one all_one \
2946 ▷ ▷ -end
2947 ▷ convert GF_q7_table_add_draw.bmp GF_q7_table_add_draw.png
2948 ▷ convert GF_q7_table_mul_draw.bmp GF_q7_table_mul_draw.png
2949
2950
2951 F_127:
2952 ▷ $(ORBITER) -v 3 \
2953 ▷ ▷ -define F -finite_field -q 127 -end \
2954 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2955
2956
2957 F_11_product_of_all_nonzero_elements:
2958 ▷ $(ORBITER) -v 3 \
2959 ▷ ▷ -define F -finite_field -q 11 -end \
2960 ▷ ▷ -define S -vector -field F -loop 1 11 1 -end \
2961 ▷ ▷ -with F -do -finite_field_activity -product_of S -end
2962
2963
2964 F_7_vandermonde:
2965 ▷ $(ORBITER) -v 3 \
2966 ▷ ▷ -define F -finite_field -q 7 -end \
2967 ▷ ▷ -with F -do -finite_field_activity \
2968 ▷ ▷ ▷ -Vandermonde_matrix \
2969 ▷ ▷ -end
2970
2971
2972 F_101_wo:
2973 ▷ $(ORBITER) -v 3 \
2974 ▷ ▷ -define F -finite_field -q 101 -without_tables -end \
2975 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2976 ▷ pdflatex GF_101.tex
2977 ▷ $(OPEN) GF_101.pdf
2978
2979
2980 F_1021_wo:
2981 ▷ $(ORBITER) -v 3 \
2982 ▷ ▷ -define F -finite_field -q 1021 -without_tables -end \
2983 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
2984
2985
2986
2987
2988 #####
2989 # Section 3.2: Extension Fields
2990
2991 SECTION_EXTENSION_FIELDS:
2992 ▷
2993
2994 test_3.2:
2995 ▷ make F_4
2996 ▷ make F_4.tables
2997 ▷ make F_8

```

```

2998 ▷ make F_8_tables
2999 ▷ make F_8b
3000 ▷ make F_9
3001 ▷ make F_16
3002 ▷ make F_16_tables
3003 ▷ make F_64
3004 ▷ make nth_roots_63
3005 ▷ make F8_F2_field_reduction
3006 ▷ make F_4096
3007 ▷ #make F_4489
3008 ▷ #make F_2_power_24
3009
3010
3011 F_4:
3012 ▷ $(ORBITER) -v 3 \
3013 ▷ ▷ -define F -finite_field -q 4 -end \
3014 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3015 ▷ pdflatex GF_4.tex
3016 ▷ $(OPEN) GF_4.pdf
3017
3018
3019
3020
3021 F_4_tables:
3022 ▷ $(ORBITER) -v 3 \
3023 ▷ ▷ -define F -finite_field -q 4 -end \
3024 ▷ ▷ -with F -do -finite_field_activity -export_tables -end
3025 ▷ $(ORBITER) -v 3 \
3026 ▷ ▷ -define all_one -vector -repeat 1 4 -end \
3027 ▷ ▷ -draw_matrix -input_csv_file GF_q4_table_add.csv \
3028 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3029 ▷ ▷ ▷ -partition 3 all_one all_one \
3030 ▷ ▷ -end \
3031 ▷ ▷ -draw_matrix -input_csv_file GF_q4_table_mul.csv \
3032 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3033 ▷ ▷ ▷ -partition 3 all_one all_one \
3034 ▷ ▷ -end
3035 ▷ convert GF_q4_table_add_draw.bmp GF_q4_table_add_draw.png
3036 ▷ convert GF_q4_table_mul_draw.bmp GF_q4_table_mul_draw.png
3037
3038
3039
3040 F_8:
3041 ▷ $(ORBITER) -v 3 \
3042 ▷ ▷ -define F -finite_field -q 8 -end \
3043 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3044 ▷ pdflatex GF_8.tex
3045 ▷ $(OPEN) GF_8.pdf
3046
3047
3048 F_8_tables:
3049 ▷ $(ORBITER) -v 3 \
3050 ▷ ▷ -define all_one -vector -repeat 1 8 -end \
3051 ▷ ▷ -define F -finite_field -q 8 -end \
3052 ▷ ▷ -with F -do -finite_field_activity -export_tables -end \
3053 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end \
3054 ▷ ▷ -draw_matrix -input_csv_file GF_q8_table_add.csv \
3055 ▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 all_one all_one -end \
3056 ▷ ▷ -draw_matrix -input_csv_file GF_q8_table_mul.csv \

```

```

3057 ▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 all_one all_one -end
3058
3059
3060 F_8b:
3061 ▷ $(ORBITER) -v 3 \
3062 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
3063 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3064 ▷ pdflatex GF_8.tex
3065 ▷ $(OPEN) GF_8.pdf
3066
3067
3068
3069 F_9:
3070 ▷ $(ORBITER) -v 3 \
3071 ▷ ▷ -define F -finite_field -q 9 -end \
3072 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3073 ▷ pdflatex GF_9.tex
3074 ▷ $(OPEN) GF_9.pdf
3075
3076
3077
3078 F_16:
3079 ▷ $(ORBITER) -v 10 \
3080 ▷ ▷ -define F -finite_field -q 16 -compute_related_fields -end \
3081 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3082 ▷ pdflatex GF_16.tex
3083 ▷ $(OPEN) GF_16.pdf
3084
3085
3086
3087
3088 F_16_tables:
3089 ▷ $(ORBITER) -v 3 \
3090 ▷ ▷ -define F -finite_field -q 16 -end \
3091 ▷ ▷ -with F -do -finite_field_activity -export_tables -end
3092 ▷ $(ORBITER) -v 3 \
3093 ▷ ▷ -define all_one -vector -repeat 1 16 -end \
3094 ▷ ▷ -draw_matrix -input_csv_file GF_q16_table_add.csv \
3095 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3096 ▷ ▷ ▷ -partition 3 all_one all_one \
3097 ▷ ▷ -end \
3098 ▷ ▷ -draw_matrix -input_csv_file GF_q16_table_mul.csv \
3099 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3100 ▷ ▷ ▷ -partition 3 all_one all_one \
3101 ▷ ▷ -end
3102 ▷ convert GF_q16_table_add_draw.bmp GF_q16_table_add_draw.png
3103 ▷ convert GF_q16_table_mul_draw.bmp GF_q16_table_mul_draw.png
3104
3105
3106
3107 F_64:
3108 ▷ $(ORBITER) -v 3 \
3109 ▷ ▷ -define F -finite_field -q 64 -end \
3110 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3111 ▷ pdflatex GF_64.tex
3112 ▷ $(OPEN) GF_64.pdf
3113
3114
3115

```

```

3116 nth_roots_63:
3117 ▷ $(ORBITER) -v 3 \
3118 ▷ ▷ -define F -finite_field -q 2 -end \
3119 ▷ ▷ -with F -do -finite_field_activity \
3120 ▷ ▷ ▷ -nth_roots 63 \
3121 ▷ ▷ -end
3122 ▷ pdflatex Nth_roots_q2_n63.tex
3123 ▷ $(OPEN) Nth_roots_q2_n63.pdf
3124
3125
3126 F8_F2_field_reduction:
3127 ▷ $(ORBITER) -v 2 \
3128 ▷ ▷ -define F -finite_field -q 8 -end \
3129 ▷ ▷ -define v -vector -loop 0 8 1 -end \
3130 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
3131 ▷ ▷ -with F -do \
3132 ▷ ▷ -finite_field_activity \
3133 ▷ ▷ -field_reduction "F8_to_F2" \
3134 ▷ ▷ ▷ 2 1 8 v \
3135 ▷ ▷ -end
3136 ▷ $(ORBITER) -v 2 \
3137 ▷ ▷ -draw_matrix -input_csv_file F8_to_F2.csv \
3138 ▷ ▷ -box_width 40 -bit_depth 24 \
3139 ▷ ▷ -partition 4 "3" "3,3,3,3,3,3,3" -end
3140 ▷ convert F8_to_F2_draw.bmp F8_to_F2_draw.png
3141 ▷ $(OPEN) F8_to_F2_draw.png
3142
3143
3144 F_4096:
3145 ▷ $(ORBITER) -v 4 \
3146 ▷ ▷ -define F -finite_field -q 4096 \
3147 ▷ ▷ ▷ -compute_related_fields \
3148 ▷ ▷ -end \
3149 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3150 ▷ pdflatex GF_4096.tex
3151 ▷ $(OPEN) GF_4096.pdf
3152
3153
3154
3155 # 67^2 = 4489
3156
3157 F_4489:
3158 ▷ $(ORBITER) -v 4 \
3159 ▷ ▷ -define F -finite_field -q 4489 -without_tables -end \
3160
3161
3162 # ToDo error message
3163 #finite_field::alpha_power !f.has_table
3164
3165 F_2_power_24:
3166 ▷ $(ORBITER) -v 4 \
3167 ▷ ▷ -define F -finite_field -q 16777216 \
3168 ▷ ▷ ▷ -compute_related_fields \
3169 ▷ ▷ ▷ -without_tables \
3170 ▷ ▷ -end \
3171 ▷ ▷ -with F -do -finite_field_activity \
3172 ▷ ▷ -cheat_sheet_GF \
3173 ▷ ▷ -end
3174 ▷ pdflatex GF_16777216.tex

```



```

3175 ▷ $(OPEN) GF_16777216.pdf
3176
3177
3178 #####
3179 # Section 3.3: Linear Algebra over Finite Fields
3180
3181
3182 SECTION_LINEAR_ALGEBRA:
3183
3184
3185 test_3.3:
3186 ▷ make RREF
3187 ▷ make RREF_V7
3188 ▷ make nullspace
3189 ▷ make RREF_RS_6.4_7
3190 ▷ make eigenstuff
3191 ▷ make classes_GL_3.2
3192 ▷ make classes_GL_4.2
3193
3194
3195 RREF:
3196 ▷ $(ORBITER) -v 2 \
3197 ▷ ▷ -define F -finite_field -q 2 -end \
3198 ▷ ▷ -define v -vector -field F -format 2 \
3199 ▷ ▷ ▷ -dense "1,1,1,1,0,1,1,0,0,1" \
3200 ▷ ▷ -end \
3201 ▷ ▷ -with F -do -finite_field_activity \
3202 ▷ ▷ ▷ -RREF v \
3203 ▷ ▷ -end
3204 ▷ pdflatex v_rref.tex
3205 ▷ $(OPEN) v_rref.pdf
3206
3207
3208 RREF_V7:
3209 ▷ $(ORBITER) -v 2 \
3210 ▷ ▷ -define F -finite_field -q 7 -end \
3211 ▷ ▷ -define V7 -vector -format 7 \
3212 ▷ ▷ ▷ -dense $(V7_VANDERMONDE_EXTENDED) \
3213 ▷ ▷ -end \
3214 ▷ ▷ -with F -do -finite_field_activity \
3215 ▷ ▷ ▷ -RREF V7 \
3216 ▷ ▷ -end
3217 ▷ pdflatex V7_rref.tex
3218 ▷ $(OPEN) V7_rref.pdf
3219
3220
3221
3222 nullspace:
3223 ▷ $(ORBITER) -v 2 \
3224 ▷ ▷ -define F2 -finite_field -q 2 -end \
3225 ▷ ▷ -define v -vector -field F2 -format 2 \
3226 ▷ ▷ ▷ -dense "1,1,1,1,0,1,1,0,0,1" \
3227 ▷ ▷ -end \
3228 ▷ ▷ -with F2 -do \
3229 ▷ ▷ -finite_field_activity \
3230 ▷ ▷ ▷ -nullspace v \
3231 ▷ ▷ -end
3232 ▷ pdflatex v_nullspace.tex
3233 ▷ $(OPEN) v_nullspace.pdf

```

```

3234
3235 RREF_RS_6.4.7:
3236 ▷ $(ORBITER) -v 2 \
3237 ▷ ▷ -define F -finite_field -q 7 -end \
3238 ▷ ▷ -define v -vector -field F -format 4 \
3239 ▷ ▷ ▷ -compact $(CODE_RS_6.4.7) \
3240 ▷ ▷ -end \
3241 ▷ ▷ -with F -do \
3242 ▷ ▷ -finite_field_activity -RREF v -end
3243 ▷ pdflatex v_rref.tex
3244 ▷ $(OPEN) v_rref.pdf
3245
3246
3247 ## these are global algebra functions:
3248
3249
3250 eigenstuff:
3251 ▷ $(ORBITER) -v 6 \
3252 ▷ ▷ -define F -finite_field -q 5 -end \
3253 ▷ ▷ -eigenstuff F 4 "0,1,0,2,0,1,2,1,4,2,3,1,2,0,4,3"
3254
3255 classes_GL_3.2:
3256 ▷ $(ORBITER) -v 7 \
3257 ▷ ▷ -define F -finite_field -q 2 -end \
3258 ▷ ▷ -all_rational_normal_forms F 3
3259 ▷ pdflatex Class_reps_GL_3.2.tex
3260 ▷ $(OPEN) Class_reps_GL_3.2.pdf
3261
3262
3263 classes_GL_4.2:
3264 ▷ $(ORBITER) -v 7 \
3265 ▷ ▷ -define F -finite_field -q 2 -end \
3266 ▷ ▷ -all_rational_normal_forms F 4
3267 ▷ pdflatex Class_reps_GL_4.2.tex
3268 ▷ $(OPEN) Class_reps_GL_4.2.pdf
3269
3270
3271 #####
3272 # Section 3.4: Advanced Topics in finite fields
3273
3274
3275
3276 SECTION_ADVANCED_TOPICS_IN_FINITE_FIELDS:
3277
3278
3279 test_3.4:
3280 ▷ make normal_basis_2.3
3281 ▷ make normal_basis_2.4
3282 ▷ make normal_basis_2.4.poly19
3283 ▷ make normal_basis_2.6
3284 ▷ make normal_basis_2.8
3285 ▷ make F8_over_F2.field_reduction
3286 ▷ make F_64_over_F8.field_reduction
3287 ▷ make F_64_over_F4.field_reduction
3288 ▷ make F_64_over_F2.field_reduction
3289 ▷ make F_8_Nth_roots_21
3290 ▷ make F_8_vandermonde
3291 ▷ make F_1024_vandermonde
3292 ▷ make NTT_k4.q17.cpp

```

```

3293 ▷ make F_17_NTT_compile
3294
3295
3296
3297 normal_basis_2.3:
3298 ▷ $(ORBITER) -v 2 \
3299 ▷ ▷ -define F -finite_field -q 2 -end \
3300 ▷ ▷ -with F -do -finite_field_activity \
3301 ▷ ▷ -normal_basis 3 -end
3302 ▷ pdflatex normal_basis_q2_d3.tex
3303 ▷ $(OPEN) normal_basis_q2_d3.pdf
3304
3305
3306 normal_basis_2.4:
3307 ▷ $(ORBITER) -v 2 \
3308 ▷ ▷ -define F -finite_field -q 2 -end \
3309 ▷ ▷ -with F -do -finite_field_activity \
3310 ▷ ▷ -normal_basis 4 -end
3311
3312
3313 normal_basis_2.4_poly19:
3314 ▷ $(ORBITER) -v 2 \
3315 ▷ ▷ -define F -finite_field -q 2 -end \
3316 ▷ ▷ -with F -do -finite_field_activity \
3317 ▷ ▷ -normal_basis_with_given_polynomial 19 4 -end
3318 ▷ pdflatex normal_basis_q2_d4_poly19.tex
3319 ▷ $(OPEN) normal_basis_q2_d4_poly19.pdf
3320
3321
3322 normal_basis_2.6:
3323 ▷ $(ORBITER) -v 2 \
3324 ▷ ▷ -define F -finite_field -q 2 -end \
3325 ▷ ▷ -with F -do -finite_field_activity \
3326 ▷ ▷ -normal_basis 6 -end
3327
3328
3329
3330 normal_basis_2.8:
3331 ▷ $(ORBITER) -v 2 \
3332 ▷ ▷ -define F -finite_field -q 2 -end \
3333 ▷ ▷ -with F -do -finite_field_activity \
3334 ▷ ▷ -normal_basis 8 -end
3335 ▷ pdflatex normal_basis_q2_d8.tex
3336 ▷ $(OPEN) normal_basis_q2_d8.pdf
3337
3338
3339
3340 F8_over_F2_field_reduction:
3341 ▷ $(ORBITER) -v 2 \
3342 ▷ ▷ -define F -finite_field -q 8 -end \
3343 ▷ ▷ -loop L 0 8 1 \
3344 ▷ ▷ ▷ -with F -do \
3345 ▷ ▷ ▷ -finite_field_activity \
3346 ▷ ▷ ▷ ▷ -field_reduction "F8_red_%L" 2 1 1 "%L" \
3347 ▷ ▷ ▷ -end \
3348 ▷ ▷ -end_loop L
3349 ▷ $(ORBITER) -v 2 \
3350 ▷ ▷ -loop L 0 8 1 \
3351 ▷ ▷ ▷ -draw_matrix \

```

```

3352 ▷ ▷ ▷ ▷ -input_csv_file F8.red_%L.csv \
3353 ▷ ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3354 ▷ ▷ ▷ ▷ -partition 4 3 3 \
3355 ▷ ▷ ▷ -end \
3356 ▷ ▷ -end_loop L
3357 ▷ #pdflatex field_reduction_Q8.q2_5_7.tex
3358
3359
3360
3361
3362 F_64_over_F8.field_reduction:
3363 ▷ $(ORBITER) -v 2 \
3364 ▷ ▷ -define F -finite_field -q 64 -end \
3365 ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
3366 ▷ ▷ -with F -do \
3367 ▷ ▷ -finite_field_activity -field_reduction "F64_over_F8" 8 8 8 \
3368 ▷ ▷ ▷ elts -end
3369 ▷ $(ORBITER) -v 2 -draw_matrix \
3370 ▷ ▷ -input_csv_file F64_over_F8.csv \
3371 ▷ ▷ -box_width 40 -bit_depth 24 \
3372 ▷ ▷ -partition 4 "2,2,2,2,2,2,2,2" "2,2,2,2,2,2,2,2" -end
3373 ▷ $(OPEN) F64_over_F8.draw.bmp
3374 ▷ #pdflatex field_reduction_Q64.q8_8_8.tex
3375 ▷ #$(OPEN) field_reduction_Q64.q8_8_8.pdf
3376
3377
3378 F_64_over_F4.field_reduction:
3379 ▷ $(ORBITER) -v 2 \
3380 ▷ ▷ -define F -finite_field -q 64 -end \
3381 ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
3382 ▷ ▷ -with F -do \
3383 ▷ ▷ -finite_field_activity \
3384 ▷ ▷ ▷ -field_reduction "F64_over_F4" 4 8 8 elts -end
3385 ▷ $(ORBITER) -v 2 -draw_matrix \
3386 ▷ ▷ -input_csv_file F64_over_F4.csv \
3387 ▷ ▷ -box_width 40 -bit_depth 24 \
3388 ▷ ▷ -partition 4 "3,3,3,3,3,3,3,3" "3,3,3,3,3,3,3,3" -end
3389 ▷ $(OPEN) F64_over_F4.draw.bmp
3390 ▷ #pdflatex field_reduction_Q64.q4_8_8.tex
3391 ▷ #$(OPEN) field_reduction_Q64.q4_8_8.pdf
3392
3393
3394 F_64_over_F2.field_reduction:
3395 ▷ $(ORBITER) -v 2 \
3396 ▷ ▷ -define F -finite_field -q 64 -end \
3397 ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
3398 ▷ ▷ -with F -do \
3399 ▷ ▷ -finite_field_activity \
3400 ▷ ▷ ▷ -field_reduction "F64_over_F2" 2 8 8 elts -end
3401 ▷ $(ORBITER) -v 2 -draw_matrix \
3402 ▷ ▷ -input_csv_file F64_over_F2.csv \
3403 ▷ ▷ -box_width 40 -bit_depth 24 \
3404 ▷ ▷ -partition 4 "6,6,6,6,6,6,6,6" "6,6,6,6,6,6,6,6" -end
3405 ▷ $(OPEN) F64_over_F2.draw.bmp
3406 ▷ #pdflatex field_reduction_Q64.q2_8_8.tex
3407 ▷ #$(OPEN) field_reduction_Q64.q2_8_8.pdf
3408
3409
3410

```

```

3411
3412
3413 F_8_Nth_roots_21:
3414 ▷ $(ORBITER) -v 3 \
3415 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
3416 ▷ ▷ -with F -do -finite_field_activity \
3417 ▷ ▷ ▷ -nth_roots 21 \
3418 ▷ ▷ -end
3419 ▷ pdflatex Nth_roots_q8_n21.tex
3420 ▷ $(OPEN) Nth_roots_q8_n21.pdf
3421
3422
3423
3424
3425 F_8_vandermonde:
3426 ▷ $(ORBITER) -v 3 \
3427 ▷ ▷ -define F -finite_field -q 8 -end \
3428 ▷ ▷ -with F -do -finite_field_activity \
3429 ▷ ▷ ▷ -Vandermonde_matrix \
3430 ▷ ▷ -end
3431
3432
3433
3434 F_1024_vandermonde:
3435 ▷ $(ORBITER) -v 3 \
3436 ▷ ▷ -define F -finite_field -q 1024 -end \
3437 ▷ ▷ -with F -do -finite_field_activity \
3438 ▷ ▷ ▷ -Vandermonde_matrix \
3439 ▷ ▷ -end
3440 ▷ #rm Vandermonde_1024.csv
3441 ▷ #rm Vandermonde_inv_1024.csv
3442
3443 #User time: 0:46
3444
3445
3446
3447
3448 NTT_k4_q17.cpp:
3449 ▷ $(ORBITER) -v 3 \
3450 ▷ ▷ -define F -finite_field -q 17 -end \
3451 ▷ ▷ -with F -do -coding_theoretic_activity \
3452 ▷ ▷ ▷ -NTT 4 17 \
3453 ▷ ▷ -end
3454
3455 F_17_NTT_compile: NTT_k4_q17.cpp
3456 ▷ $(MY_CPP) NTT_k4_q17.cpp $(CPPFLAGS) \
3457 ▷ ▷ $(LIB) $(LFLAGS) -o NTT_k4_q17.out
3458 ▷ ./NTT_k4_q17.out
3459
3460 F64_1:
3461 ▷ $(ORBITER) -v 3 \
3462 ▷ ▷ -define F -finite_field -q 64 -override_polynomial 97 -end \
3463 ▷ ▷ -with F -do -finite_field_activity \
3464 ▷ ▷ ▷ -nth_roots 7 \
3465 ▷ ▷ -end
3466 ▷ pdflatex Nth_roots_q64_n7.tex
3467 ▷ $(OPEN) Nth_roots_q64_n7.pdf
3468
3469

```

```

3470 #####
3471 # Section 3.5: Basic Ring Theory
3472
3473
3474
3475 SECTION.BASIC_RING_THEORY:
3476
3477
3478 test_3.5:
3479 ▷ make Polynomial_ring
3480
3481
3482
3483 Polynomial_ring:
3484 ▷ $(ORBITER) -v 3 \
3485 ▷ ▷ -define F -finite_field -q 4 -end \
3486 ▷ ▷ -define R -polynomial_ring -field F \
3487 ▷ ▷ ▷ -number_of_variables 4 \
3488 ▷ ▷ ▷ -homogeneous_of_degree 3 \
3489 ▷ ▷ ▷ -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
3490 ▷ ▷ -end
3491
3492
3493
3494
3495 #####
3496 # Chapter 4 - Geometry
3497 #####
3498
3499
3500 test_4:
3501 ▷ make test_4.1
3502 ▷ make test_4.2
3503 ▷ make test_4.3
3504 ▷ make test_4.4
3505 ▷ make test_4.5
3506 ▷ make test_4.6
3507 ▷ make test_4.7
3508 ▷ make test_4.8
3509 ▷ make test_4.9
3510 ▷ make test_4.10
3511
3512
3513
3514 #####
3515 # Section 4.1: Finite Projective Spaces
3516
3517 SECTION.FINITE_PROJECTIVE_SPACES:
3518
3519
3520 test_4.1:
3521 ▷ make PG_3_2_easy
3522 ▷ make PG_1_16
3523 ▷ make PG_2_2
3524 ▷ make PG_2_3
3525 ▷ make PG_2_4
3526 ▷ make PG_2_13
3527 ▷ make PG_2_64
3528 ▷ make PG_3_2

```

```

3529 ▷ make PG_3_4
3530 ▷ make PG_3_5
3531 ▷ make PG_3_7
3532 ▷ make PG_3_8
3533 ▷ make PG_3_16
3534 ▷ make PG_3_25
3535 ▷ make PG_4_3
3536 ▷ make PG_8_2
3537
3538
3539 PG_3_2_easy:
3540 ▷ $(ORBITER) -v 2 \
3541 ▷ ▷ -define F -finite_field -q 2 -end \
3542 ▷ ▷ -define P -projective_space -n 3 -field F -end
3543
3544
3545
3546
3547 PG_1_16:
3548 ▷ $(ORBITER) -v 2 \
3549 ▷ ▷ -define F -finite_field -q 16 -end \
3550 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
3551 ▷ ▷ -with P -do -projective_space_activity \
3552 ▷ ▷ ▷ -cheat_sheet \
3553 ▷ ▷ -end
3554 ▷ pdflatex PG_1_16.tex
3555 ▷ $(OPEN) PG_1_16.pdf
3556
3557
3558 PG_2_2:
3559 ▷ $(ORBITER) -v 2 \
3560 ▷ ▷ -define F -finite_field -q 2 -end \
3561 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3562 ▷ ▷ -with P -do -projective_space_activity \
3563 ▷ ▷ ▷ -cheat_sheet \
3564 ▷ ▷ -end
3565 ▷ pdflatex PG_2_2.tex
3566 ▷ $(OPEN) PG_2_2.pdf
3567
3568 PG_2_3:
3569 ▷ $(ORBITER) -v 2 \
3570 ▷ ▷ -define F -finite_field -q 3 -end \
3571 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3572 ▷ ▷ -with P -do -projective_space_activity \
3573 ▷ ▷ ▷ -cheat_sheet \
3574 ▷ ▷ -end
3575 ▷ pdflatex PG_2_3.tex
3576 ▷ $(OPEN) PG_2_3.pdf
3577
3578
3579 PG_2_4:
3580 ▷ $(ORBITER) -v 2 \
3581 ▷ ▷ -define F -finite_field -q 4 -end \
3582 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3583 ▷ ▷ -with P -do -projective_space_activity \
3584 ▷ ▷ ▷ -cheat_sheet \
3585 ▷ ▷ -end
3586 ▷ pdflatex PG_2_4.tex
3587 ▷ $(OPEN) PG_2_4.pdf

```

```

3588
3589
3590
3591
3592 PG_2_13:
3593 ▷ $(ORBITER) -v 2 \
3594 ▷ ▷ -define F -finite_field -q 13 -end \
3595 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3596 ▷ ▷ -with P -do -projective_space_activity \
3597 ▷ ▷ ▷ -cheat_sheet \
3598 ▷ ▷ -end
3599 ▷ pdflatex PG_2_13.tex
3600 ▷ $(OPEN) PG_2_13.pdf
3601
3602
3603
3604
3605 PG_2_64:
3606 ▷ $(ORBITER) -v 2 \
3607 ▷ ▷ -define F -finite_field -q 64 -end \
3608 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3609 ▷ ▷ -with P -do -projective_space_activity \
3610 ▷ ▷ ▷ -cheat_sheet \
3611 ▷ ▷ -end
3612 ▷ pdflatex PG_2_64.tex
3613 ▷ $(OPEN) PG_2_64.pdf
3614
3615
3616
3617 PG_3_2:
3618 ▷ $(ORBITER) -v 2 \
3619 ▷ ▷ -define F -finite_field -q 2 -end \
3620 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
3621 ▷ ▷ -with P -do -projective_space_activity \
3622 ▷ ▷ ▷ -cheat_sheet \
3623 ▷ ▷ -end
3624 ▷ pdflatex PG_3_2.tex
3625 ▷ $(OPEN) PG_3_2.pdf
3626
3627
3628 PG_3_4:
3629 ▷ $(ORBITER) -v 2 \
3630 ▷ ▷ -define F -finite_field -q 4 -end \
3631 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
3632 ▷ ▷ -with P -do -projective_space_activity \
3633 ▷ ▷ ▷ -cheat_sheet \
3634 ▷ ▷ -end
3635 ▷ pdflatex PG_3_4.tex
3636 ▷ $(OPEN) PG_3_4.pdf
3637
3638 PG_3_5:
3639 ▷ $(ORBITER) -v 2 \
3640 ▷ ▷ -define F -finite_field -q 5 -end \
3641 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
3642 ▷ ▷ -with P -do -projective_space_activity \
3643 ▷ ▷ ▷ -cheat_sheet \
3644 ▷ ▷ -end
3645 ▷ pdflatex PG_3_5.tex
3646 ▷ $(OPEN) PG_3_5.pdf

```



```
3647
3648
3649 PG_3_7:
3650 ▷ $(ORBITER) -v 2 \
3651 ▷ ▷ -define F -finite.field -q 7 -end \
3652 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
3653 ▷ ▷ -with P -do -projective.space.activity \
3654 ▷ ▷ ▷ -cheat.sheet \
3655 ▷ ▷ -end
3656 ▷ pdflatex PG_3_7.tex
3657 ▷ $(OPEN) PG_3_7.pdf
3658
3659
3660
3661
3662 PG_3_8:
3663 ▷ $(ORBITER) -v 2 \
3664 ▷ ▷ -define F -finite.field -q 8 -end \
3665 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
3666 ▷ ▷ -with P -do -projective.space.activity \
3667 ▷ ▷ ▷ -cheat.sheet \
3668 ▷ ▷ -end
3669 ▷ pdflatex PG_3_8.tex
3670 ▷ $(OPEN) PG_3_8.pdf
3671
3672
3673 PG_3_16:
3674 ▷ $(ORBITER) -v 2 \
3675 ▷ ▷ -define F -finite.field -q 16 -end \
3676 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
3677 ▷ ▷ -with P -do -projective.space.activity \
3678 ▷ ▷ ▷ -cheat.sheet \
3679 ▷ ▷ -end
3680 ▷ pdflatex PG_3_16.tex
3681 ▷ $(OPEN) PG_3_16.pdf
3682
3683
3684
3685
3686 PG_3_25:
3687 ▷ $(ORBITER) -v 2 \
3688 ▷ ▷ -define F -finite.field -q 25 -end \
3689 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
3690 ▷ ▷ -with P -do -projective.space.activity \
3691 ▷ ▷ ▷ -cheat.sheet \
3692 ▷ ▷ -end
3693 ▷ pdflatex PG_3_25.tex
3694 ▷ $(OPEN) PG_3_25.pdf
3695
3696
3697
3698
3699 PG_4_3:
3700 ▷ $(ORBITER) -v 2 \
3701 ▷ ▷ -define F -finite.field -q 3 -end \
3702 ▷ ▷ -define P -projective.space -n 4 -field F -v 0 -end \
3703 ▷ ▷ -with P -do -projective.space.activity \
3704 ▷ ▷ ▷ -cheat.sheet \
3705 ▷ ▷ -end
```

```

3706 ▷ pdflatex PG_4.3.tex
3707 ▷ $(OPEN) PG_4.3.pdf
3708
3709
3710 PG_8.2:
3711 ▷ $(ORBITER) -v 2 \
3712 ▷ ▷ -define F -finite_field -q 2 -end \
3713 ▷ ▷ -define P -projective_space -n 8 -field F -v 0 -end \
3714 ▷ ▷ -with P -do -projective_space_activity \
3715 ▷ ▷ ▷ -cheat_sheet \
3716 ▷ ▷ -end
3717 ▷ pdflatex PG_8.2.tex
3718 ▷ $(OPEN) PG_8.2.pdf
3719
3720
3721
3722
3723 #####
3724 # Section 4.2: Indexing Points and Lines
3725
3726 SECTION_INDEXING_POINTS_AND_LINES:
3727
3728 test_4.2:
3729 ▷ make PG_2.4.rank_point
3730 ▷ make PG_2.4.unrank_point
3731 ▷ make PG_2.2.rank_lines
3732 ▷ make PG_2.2.unrank_lines
3733 ▷ make PG_3.3.rank_lines
3734 ▷ make PG_3.5.unrank_lines
3735
3736
3737 PG_2.4.rank_point:
3738 ▷ $(ORBITER) -v 2 \
3739 ▷ ▷ -define v -vector -dense "3,3,1" -format 1 -end \
3740 ▷ ▷ -define F -finite_field -q 4 -end \
3741 ▷ ▷ -with F -do -finite_field_activity \
3742 ▷ ▷ ▷ -rank_point_in_PG v -end
3743
3744 # geometry_global::do_rank_point_in_PG coeff: ( 3, 3, 1 ) has rank 20
3745
3746 PG_2.4.unrank_point:
3747 ▷ $(ORBITER) -v 2 \
3748 ▷ ▷ -define v -vector -dense "20" -end \
3749 ▷ ▷ -define F -finite_field -q 4 -end \
3750 ▷ ▷ -with F -do -finite_field_activity \
3751 ▷ ▷ ▷ -unrank_point_in_PG 2 v -end
3752
3753
3754
3755 PG_2.2.rank_lines:
3756 ▷ $(ORBITER) -v 2 \
3757 ▷ ▷ -define v -vector -format 4 \
3758 ▷ ▷ -dense "1,0,0, 0,1,0, 1,0,0, 0,0,1, 1,1,1, 0,1,0, 1,1,1, 0,0,1" \
3759 ▷ ▷ -end \
3760 ▷ ▷ -define F -finite_field -q 2 -end \
3761 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3762 ▷ ▷ -with P -do \
3763 ▷ ▷ -projective_space_activity \
3764 ▷ ▷ ▷ -rank_lines_in_PG v \

```

```

3765 ▷ ▷ -end \
3766
3767
3768 PG_2_2_unrank_lines:
3769 ▷ $(ORBITER) -v 2 \
3770 ▷ ▷ -define v -vector -format 4 \
3771 ▷ ▷ -dense "0,1,2,3,4,5,6" \
3772 ▷ ▷ -end \
3773 ▷ ▷ -define F -finite_field -q 2 -end \
3774 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3775 ▷ ▷ -with P -do \
3776 ▷ ▷ -projective_space_activity \
3777 ▷ ▷ ▷ -unrank_lines_in_PG v \
3778 ▷ ▷ -end \
3779
3780
3781
3782 PG_3_3_rank_lines:
3783 ▷ $(ORBITER) -v 2 \
3784 ▷ ▷ -define v1 -vector -format 3 \
3785 ▷ ▷ -dense "1,0,2,2,0,1,1,2, 1,0,2,0,0,1,1,2, 1,0,2,2,0,1,2,1" \
3786 ▷ ▷ -end \
3787 ▷ ▷ -define v2 -vector -format 3 \
3788 ▷ ▷ -dense "1,0,0,0,0,1,0,0, 1,0,0,0,0,0,0,1, 0,1,0,0,0,0,2,1" \
3789 ▷ ▷ -end \
3790 ▷ ▷ -define F -finite_field -q 3 -end \
3791 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
3792 ▷ ▷ -with P -do \
3793 ▷ ▷ -projective_space_activity \
3794 ▷ ▷ ▷ -rank_lines_in_PG v1 \
3795 ▷ ▷ -end \
3796 ▷ ▷ -with P -do \
3797 ▷ ▷ -projective_space_activity \
3798 ▷ ▷ ▷ -rank_lines_in_PG v2 \
3799 ▷ ▷ -end
3800
3801
3802 PG_3_5_unrank_lines:
3803 ▷ $(ORBITER) -v 2 \
3804 ▷ ▷ -define v -vector \
3805 ▷ ▷ ▷ -dense "0,36,72,108,144,805" \
3806 ▷ ▷ -end \
3807 ▷ ▷ -define F -finite_field -q 5 -end \
3808 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
3809 ▷ ▷ -with P -do \
3810 ▷ ▷ -projective_space_activity \
3811 ▷ ▷ ▷ -unrank_lines_in_PG v \
3812 ▷ ▷ -end
3813
3814
3815
3816
3817
3818
3819 #####
3820 # Section 4.3: Incidence Matrices
3821
3822
3823 SECTION_INCIDENCE_MATRICES:

```

```

3824
3825
3826 test_4_3:
3827 ▷ make PG_2_16
3828 ▷ make PG_2_2.incidence_matrix
3829 ▷ make PG_2_4.incidence_matrix
3830 ▷ make PG_2_8.incidence_matrix
3831 ▷ make PG_2_4.with_decomposition
3832 ▷ make PG_2_4.incma_cyclic
3833 ▷ make PG_2_4.incma.singer_sub_3
3834 ▷ make PG_2_4.incma.singer_sub_7
3835
3836
3837 PG_2_16:
3838 ▷ $(ORBITER) -v 2 \
3839 ▷ ▷ -draw_options -xin 20000 -yin 20000 \
3840 ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty -end \
3841 ▷ ▷ -define F -finite_field -q 16 -end \
3842 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3843 ▷ ▷ -with P -do -projective_space_activity \
3844 ▷ ▷ ▷ -cheat_sheet \
3845 ▷ ▷ -end
3846 ▷ pdflatex PG_2_16.tex
3847 ▷ $(OPEN) PG_2_16.pdf
3848
3849
3850
3851 PG_2_2.incidence_matrix:
3852 ▷ $(ORBITER) -v 2 \
3853 ▷ ▷ -define F -finite_field -q 2 -end \
3854 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3855 ▷ ▷ -with P -do -projective_space_activity \
3856 ▷ ▷ ▷ -export_point_line_incidence_matrix \
3857 ▷ ▷ -end
3858 ▷ $(ORBITER) -v 2 \
3859 ▷ ▷ -define all_one -vector -repeat 1 7 -end \
3860 ▷ ▷ -draw_matrix \
3861 ▷ ▷ ▷ -input_csv_file PG_n2_q2_incidence_matrix.csv \
3862 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
3863 ▷ ▷ ▷ -partition 3 \
3864 ▷ ▷ ▷ ▷ all_one all_one \
3865 ▷ ▷ -end
3866 ▷ $(OPEN) PG_n2_q2_incidence_matrix_draw.bmp
3867
3868
3869 PG_2_4.incidence_matrix:
3870 ▷ $(ORBITER) -v 2 \
3871 ▷ ▷ -define F -finite_field -q 4 -end \
3872 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3873 ▷ ▷ -with P -do -projective_space_activity \
3874 ▷ ▷ ▷ -export_point_line_incidence_matrix \
3875 ▷ ▷ -end
3876 ▷ $(ORBITER) -v 2 \
3877 ▷ ▷ -define all_one -vector -repeat 1 21 -end \
3878 ▷ ▷ -draw_matrix \
3879 ▷ ▷ ▷ -input_csv_file PG_n2_q4_incidence_matrix.csv \
3880 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
3881 ▷ ▷ ▷ -partition 3 \
3882 ▷ ▷ ▷ ▷ all_one all_one \

```

```

3883 ▷ ▷ -end
3884 ▷ $(OPEN) PG_n2_q4_incidence_matrix_draw.bmp
3885
3886 # writes PG_n2_q4_incidence_matrix.csv
3887
3888
3889
3890 PG_2_8_incidence_matrix:
3891 ▷ $(ORBITER) -v 2 \
3892 ▷ ▷ -define F -finite_field -q 8 -end \
3893 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3894 ▷ ▷ -with P -do -projective_space_activity \
3895 ▷ ▷ ▷ -export_point_line_incidence_matrix \
3896 ▷ ▷ -end
3897 ▷ $(ORBITER) -v 2 \
3898 ▷ ▷ -define all_one -vector -repeat 1 73 -end \
3899 ▷ ▷ -draw_matrix \
3900 ▷ ▷ ▷ -input_csv_file PG_n2_q8_incidence_matrix.csv \
3901 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
3902 ▷ ▷ ▷ -partition 3 \
3903 ▷ ▷ ▷ ▷ all_one all_one \
3904 ▷ ▷ -end
3905 ▷ $(OPEN) PG_n2_q8_incidence_matrix_draw.bmp
3906
3907
3908
3909 # ToDo:
3910
3911 PG_2_4_with_decomposition:
3912 ▷ $(ORBITER) -v 2 \
3913 ▷ ▷ -define F -finite_field -q 4 -end \
3914 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3915 ▷ ▷ -with P -do -projective_space_activity \
3916 ▷ ▷ ▷ -decomposition_by_element_PG \
3917 ▷ ▷ ▷ 1 "0,1,0, 0,0,1, 2,1,1, 0" \
3918 ▷ ▷ ▷ "PG_2_4_singer" \
3919 ▷ ▷ -end
3920 ▷ pdflatex PG_2_4_singer.tex
3921 ▷ $(OPEN) PG_2_4_singer.pdf
3922
3923
3924 #PG_2_4_singer_incma_cyclic.csv
3925 #PG_2_4_singer_incma_subgroup_index_3.csv
3926 #PG_2_4_singer_incma_subgroup_index_7.csv
3927
3928
3929
3930 PG_2_4_incma_cyclic:
3931 ▷ $(ORBITER) -v 2 \
3932 ▷ ▷ -list_arguments \
3933 ▷ ▷ -define R -vector -repeat 1 21 -end \
3934 ▷ ▷ -define C -vector -repeat 1 21 -end \
3935 ▷ ▷ -draw_matrix \
3936 ▷ ▷ -input_csv_file PG_2_4_singer_incma_cyclic.csv \
3937 ▷ ▷ -box_width 40 -bit_depth 24 \
3938 ▷ ▷ -partition 3 R C \
3939 ▷ ▷ -end
3940 ▷ $(OPEN) PG_2_4_singer_incma_cyclic_draw.bmp
3941

```

```

3942
3943 PG_2_4.incma.singer.sub.3:
3944 ▷ $(ORBITER) -v 2 \
3945 ▷ ▷ -list_arguments \
3946 ▷ ▷ -define R -vector -repeat 3 7 -end \
3947 ▷ ▷ -define C -vector -repeat 3 7 -end \
3948 ▷ ▷ -draw_matrix \
3949 ▷ ▷ -input_csv_file PG_2_4.singer.incma.subgroup_index.3.csv \
3950 ▷ ▷ -box_width 40 -bit_depth 24 \
3951 ▷ ▷ -partition 3 R C \
3952 ▷ ▷ -end
3953 ▷ $(OPEN) PG_2_4.singer.incma.subgroup_index.3.draw.bmp
3954
3955 PG_2_4.incma.singer.sub.7:
3956 ▷ $(ORBITER) -v 2 \
3957 ▷ ▷ -draw_matrix \
3958 ▷ ▷ -input_csv_file PG_2_4.singer.incma.subgroup_index.7.csv \
3959 ▷ ▷ -box_width 20 -bit_depth 24 \
3960 ▷ ▷ -partition 3 3,3,3,3,3,3 3,3,3,3,3,3 -end
3961 ▷ $(OPEN) PG_2_4.singer.incma.subgroup_index.7.draw.bmp
3962
3963
3964
3965
3966
3967
3968
3969
3970
3971
3972 #####
3973 # Section 4.4: The Grassmannian
3974
3975
3976 SECTION_GRASSMANNIAN:
3977
3978
3979
3980 test_4.4:
3981 ▷ make GR_3_2_2
3982 ▷ make planes_in_pencil
3983
3984
3985
3986
3987
3988 GR_3_2_2:
3989 ▷ $(ORBITER) -v 2 \
3990 ▷ ▷ -define F -finite_field -q 2 -end \
3991 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
3992 ▷ ▷ -with P -do \
3993 ▷ ▷ -projective_space_activity \
3994 ▷ ▷ ▷ -report_Grassmannian 2 \
3995 ▷ ▷ -end
3996 ▷ pdflatex Gr_3_2_2.tex
3997 ▷ $(OPEN) Gr_3_2_2.pdf
3998
3999
4000

```

```

4001
4002
4003
4004 planes_in_pencil:
4005 ▷ $(ORBITER) -v 2 \
4006 ▷ ▷ -define F -finite_field -q 8 -end \
4007 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4008 ▷ ▷ -with P -do \
4009 ▷ ▷ -projective_space_activity \
4010 ▷ ▷ ▷ -planes_through_line 0 \
4011 ▷ ▷ -end
4012
4013
4014
4015 #####
4016 # Section 4.5: Algebraic Sets
4017
4018
4019 SECTION.ALGEBRAIC_SETS:
4020
4021
4022 test_4.5:
4023 ▷ make Fermat_curve_F7
4024 ▷ make Hesse_pencil_F7
4025 ▷ make surface_F9
4026 ▷ make elliptic_curve_b1_c3_q11.txt
4027 ▷ make EC_11.txt
4028 ▷ make Hirschfeld_surface_q4.txt
4029 ▷ make Hirschfeld_surface_q16.txt
4030
4031
4032
4033 Fermat_curve_F7:
4034 ▷ $(ORBITER) -v 6 \
4035 ▷ ▷ -define F -finite_field -q 7 -end \
4036 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4037 ▷ ▷ -define R -polynomial_ring \
4038 ▷ ▷ ▷ -field F \
4039 ▷ ▷ ▷ -number_of_variables 3 \
4040 ▷ ▷ ▷ -homogeneous_of_degree 3 \
4041 ▷ ▷ ▷ -monomial_ordering_lex \
4042 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
4043 ▷ ▷ ▷ -end \
4044 ▷ ▷ -define Fermat -symbolic_object \
4045 ▷ ▷ ▷ -field F \
4046 ▷ ▷ ▷ -text "y0^3+y1^3+y2^3" \
4047 ▷ ▷ ▷ -end \
4048 ▷ ▷ -with P -do \
4049 ▷ ▷ -projective_space_activity \
4050 ▷ ▷ ▷ -projective_variety R Fermat "" \
4051 ▷ ▷ -end
4052
4053
4054 Hesse_pencil_F7:
4055 ▷ $(ORBITER) -v 6 \
4056 ▷ ▷ -define F -finite_field -q 7 -end \
4057 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4058 ▷ ▷ -define R -polynomial_ring \
4059 ▷ ▷ ▷ -field F \

```

```

4060 ▷ ▷ ▷ -number_of_variables 3 \
4061 ▷ ▷ ▷ -homogeneous_of_degree 3 \
4062 ▷ ▷ ▷ -monomial_ordering_lex \
4063 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
4064 ▷ ▷ ▷ -end \
4065 ▷ ▷ -define Hesse_pencil -symbolic_object \
4066 ▷ ▷ ▷ -field F \
4067 ▷ ▷ ▷ -text "y0^3+y1^3+y2^3+y0*y1*y2" \
4068 ▷ ▷ ▷ -end \
4069 ▷ ▷ -with P -do \
4070 ▷ ▷ -projective_space_activity \
4071 ▷ ▷ ▷ -projective_variety R Hesse_pencil "" \
4072 ▷ ▷ -end
4073
4074
4075 surface_F9:
4076 ▷ $(ORBITER) -v 6 \
4077 ▷ ▷ -define F -finite_field -q 9 -print_numerically -end \
4078 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4079 ▷ ▷ -define R -polynomial_ring \
4080 ▷ ▷ ▷ -field F \
4081 ▷ ▷ ▷ -number_of_variables 4 \
4082 ▷ ▷ ▷ -homogeneous_of_degree 3 \
4083 ▷ ▷ ▷ -monomial_ordering_lex \
4084 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
4085 ▷ ▷ ▷ -end \
4086 ▷ ▷ -define surface_F9 -symbolic_object \
4087 ▷ ▷ ▷ -field F \
4088 ▷ ▷ ▷ -text $(SURFACE_F9_1.EQN) \
4089 ▷ ▷ ▷ -end \
4090 ▷ ▷ -with P -do \
4091 ▷ ▷ -projective_space_activity \
4092 ▷ ▷ ▷ -projective_variety R surface_F9 "" \
4093 ▷ ▷ -end
4094
4095
4096 elliptic_curve_b1_c3.q11.txt:
4097 ▷ $(ORBITER) -v 2 \
4098 ▷ ▷ -define F -finite_field -q 11 -end \
4099 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4100 ▷ ▷ -define EC -geometric_object P \
4101 ▷ ▷ ▷ -elliptic_curve 1 3 \
4102 ▷ ▷ -end \
4103 ▷ ▷ -with EC -do -combinatorial_object_activity -save \
4104 ▷ ▷ -end
4105
4106
4107 EC_11.txt:
4108 ▷ $(ORBITER) -v 2 \
4109 ▷ ▷ -define F -finite_field -q 11 -end \
4110 ▷ ▷ -define R -polynomial_ring -field F \
4111 ▷ ▷ ▷ -number_of_variables 3 \
4112 ▷ ▷ ▷ -homogeneous_of_degree 3 \
4113 ▷ ▷ ▷ -end \
4114 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4115 ▷ ▷ -define EC -geometric_object P \
4116 ▷ ▷ ▷ -projective_variety R \
4117 ▷ ▷ ▷ "EC_11" "EC_11" \
4118 ▷ ▷ ▷ $(EC_11.EQUATION) \

```



```

4119 ▷ ▷ ▷ -end \
4120 ▷ ▷ -with EC -do -combinatorial_object_activity -save \
4121 ▷ ▷ -end
4122
4123
4124
4125
4126 Hirschfeld_surface.q4.txt:
4127 ▷ $(ORBITER) -v 2 \
4128 ▷ ▷ -define F -finite_field -q 4 -end \
4129 ▷ ▷ -define R -polynomial_ring -field F \
4130 ▷ ▷ ▷ -number_of_variables 4 \
4131 ▷ ▷ ▷ -homogeneous_of_degree 3 \
4132 ▷ ▷ ▷ -end \
4133 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4134 ▷ ▷ -define H4 -geometric_object P \
4135 ▷ ▷ ▷ -projective_variety R \
4136 ▷ ▷ ▷ ▷ "Hirschfeld_surface.q4" \
4137 ▷ ▷ ▷ ▷ "Hirschfeld\_surface\_q4" \
4138 ▷ ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_EQUATION) \
4139 ▷ ▷ ▷ -end \
4140 ▷ ▷ -with H4 -do -combinatorial_object_activity -save \
4141 ▷ ▷ -end
4142
4143 # creates Hirschfeld_surface.q4.txt
4144
4145 Hirschfeld_surface.q16.txt:
4146 ▷ $(ORBITER) -v 2 \
4147 ▷ ▷ -define F -finite_field -q 16 -end \
4148 ▷ ▷ -define R -polynomial_ring -field F \
4149 ▷ ▷ ▷ -number_of_variables 4 \
4150 ▷ ▷ ▷ -homogeneous_of_degree 3 \
4151 ▷ ▷ ▷ -end \
4152 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4153 ▷ ▷ -define H16 -geometric_object P \
4154 ▷ ▷ ▷ -projective_variety R \
4155 ▷ ▷ ▷ ▷ "Hirschfeld_surface.q16" \
4156 ▷ ▷ ▷ ▷ "Hirschfeld\_surface\_q16" \
4157 ▷ ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_EQUATION) \
4158 ▷ ▷ ▷ -end \
4159 ▷ ▷ -with H16 -do -combinatorial_object_activity -save \
4160 ▷ ▷ -end
4161
4162
4163 # the coefficient vector is given as a list of pairs.
4164 # 165 = binomial(11,3)
4165
4166
4167
4168 #####
4169 # Section 4.6: The Klein Quadric and Pluecker coordinates
4170
4171
4172 SECTION.KLEIN_QUADRIC_AND_PLUECKER_COORDINATES:
4173
4174
4175 test_4.6:
4176 ▷ make GR.4.2.2
4177

```

```

4178
4179 GR_4_2_2:
4180 ▷ $(ORBITER) -v 2 \
4181 ▷ ▷ -define F -finite_field -q 2 -end \
4182 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4183 ▷ ▷ -with P -do \
4184 ▷ ▷ -projective_space_activity \
4185 ▷ ▷ ▷ -report_Grassmannian 2 \
4186 ▷ ▷ -end
4187 ▷ pdflatex Gr_4_2_2.tex
4188 ▷ $(OPEN) Gr_4_2_2.pdf
4189
4190
4191 #####
4192 # Section 4.7: Orthogonal spaces
4193
4194
4195 SECTION.ORTHOGONAL.SPACES:
4196
4197
4198 test_4_7:
4199 ▷ make Op_4_2
4200 ▷ make 0_5_2_incidence_matrix.csv
4201 ▷ make Op_6_2
4202 ▷ make Op_6_2_incidence_matrix.csv
4203 ▷ make Op_6_2_with_group
4204 ▷ make Op_6_8
4205 ▷ make Op_8_2
4206 ▷ make Op_6_64
4207 ▷ #make Op_6_64_line_rank_problem
4208 ▷ make Op_6_64_line_rank
4209 ▷ make Op_6_64_report
4210 ▷ #make elliptic_quadric_subspace fails
4211 ▷ make BLT_database_7_1
4212 ▷ make BLT_database_7_1_print
4213 ▷ make Doily_W_2
4214
4215
4216
4217 Op_4_2:
4218 ▷ $(ORBITER) -v 2 \
4219 ▷ ▷ -define F -finite_field -q 2 -end \
4220 ▷ ▷ -define O -orthogonal_space 1 4 F -without_group -end \
4221 ▷ ▷ -with O -do -orthogonal_space_activity \
4222 ▷ ▷ ▷ -cheat_sheet_orthogonal -end
4223 ▷ pdflatex 0_1_4_2_report.tex
4224 ▷ $(OPEN) 0_1_4_2_report.pdf
4225
4226
4227 0_5_2_incidence_matrix.csv:
4228 ▷ $(ORBITER) -v 2 \
4229 ▷ ▷ -define F -finite_field -q 2 -end \
4230 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4231 ▷ ▷ -with O -do -orthogonal_space_activity \
4232 ▷ ▷ ▷ -export_point_line_incidence_matrix \
4233 ▷ ▷ -end
4234 ▷ $(ORBITER) -v 2 \
4235 ▷ ▷ -define all_one_r -vector -repeat 1 15 -end \
4236 ▷ ▷ -define all_one_c -vector -repeat 1 15 -end \

```

```

4237 ▷ ▷ -draw_matrix \
4238 ▷ ▷ ▷ -input_csv_file 0_5_2.incidence_matrix.csv \
4239 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
4240 ▷ ▷ ▷ -partition 2 \
4241 ▷ ▷ ▷ ▷ all_one_r all_one_c \
4242 ▷ ▷ -end
4243 ▷ $(OPEN) 0_5_2.incidence_matrix_draw.bmp
4244
4245 #0(5,2) projectively = Q(4,2) = (dual of) W(3,2) = W(3,2)
4246 # recall that W(3,2) and Q(4,q) are self dual if q is even
4247
4248
4249 Op_6_2:
4250 ▷ $(ORBITER) -v 2 \
4251 ▷ ▷ -define F -finite_field -q 2 -end \
4252 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
4253 ▷ ▷ -with O -do -orthogonal_space_activity \
4254 ▷ ▷ ▷ -cheat_sheet_orthogonal -end
4255 ▷ pdflatex 0_1_6_2.report.tex
4256 ▷ $(OPEN) 0_1_6_2.report.pdf
4257
4258
4259 Op_6_2.incidence_matrix.csv:
4260 ▷ $(ORBITER) -v 2 \
4261 ▷ ▷ -define F -finite_field -q 2 -end \
4262 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
4263 ▷ ▷ -with O -do -orthogonal_space_activity \
4264 ▷ ▷ ▷ -export_point_line_incidence_matrix \
4265 ▷ ▷ -end
4266 ▷ $(ORBITER) -v 2 \
4267 ▷ ▷ -define all_one_r -vector -repeat 1 35 -end \
4268 ▷ ▷ -define all_one_c -vector -repeat 1 105 -end \
4269 ▷ ▷ -draw_matrix \
4270 ▷ ▷ ▷ -input_csv_file Op_6_2.incidence_matrix.csv \
4271 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
4272 ▷ ▷ ▷ -partition 2 \
4273 ▷ ▷ ▷ ▷ all_one_r all_one_c \
4274 ▷ ▷ -end
4275 ▷ $(OPEN) Op_6_2.incidence_matrix_draw.bmp
4276
4277
4278 Op_6_2.with_group:
4279 ▷ $(ORBITER) -v 2 \
4280 ▷ ▷ -define F -finite_field -q 2 -end \
4281 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
4282 ▷ ▷ -with O -do -orthogonal_space_activity \
4283 ▷ ▷ ▷ -cheat_sheet_orthogonal -end
4284 ▷ pdflatex 0_1_6_2.report.tex
4285 ▷ $(OPEN) 0_1_6_2.report.pdf
4286
4287 # problem:
4288 # error message:
4289 #stabilizer_chain_base_data::allocate_base_data degree is too large
4290
4291
4292 Op_6_8:
4293 ▷ $(ORBITER) -v 2 \
4294 ▷ ▷ -define F -finite_field -q 8 -end \
4295 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \

```

```

4296 ▷ ▷ -with 0 -do -orthogonal_space_activity \
4297 ▷ ▷ ▷ -cheat_sheet_orthogonal \
4298 ▷ ▷ -end
4299 ▷ pdflatex 0_1.6.8_report.tex
4300 ▷ $(OPEN) 0_1.6.8_report.pdf
4301
4302
4303 Op_8_2:
4304 ▷ $(ORBITER) -v 2 \
4305 ▷ ▷ -define F -finite_field -q 2 -end \
4306 ▷ ▷ -define 0 -orthogonal_space 1 8 F -without_group -end \
4307 ▷ ▷ -with 0 -do -orthogonal_space_activity \
4308 ▷ ▷ ▷ -cheat_sheet_orthogonal -end
4309 ▷ pdflatex 0_1.8.2_report.tex
4310 ▷ $(OPEN) 0_1.8.2_report.pdf
4311
4312
4313 Op_6_64:
4314 ▷ $(ORBITER) -v 2 \
4315 ▷ ▷ -define F -finite_field -q 64 -end \
4316 ▷ ▷ -define 0 -orthogonal_space 1 6 F -without_group -end \
4317 ▷ ▷ -with 0 -do -orthogonal_space_activity \
4318 ▷ ▷ ▷ -cheat_sheet_orthogonal -end
4319 ▷ pdflatex 0_1.6.64_report.tex
4320 ▷ $(OPEN) 0_1.6.64_report.pdf
4321
4322
4323 # problem, because we are trying to create PGL(6,64):
4324
4325 Op_6_64_line_rank_problem:
4326 ▷ $(ORBITER) -v 4 \
4327 ▷ ▷ -define F -finite_field -q 64 -end \
4328 ▷ ▷ -define 0 -orthogonal_space 1 6 F -end \
4329 ▷ ▷ -with 0 -do -orthogonal_space_activity \
4330 ▷ ▷ ▷ -unrank_line_through_two_points 15447347 15225451 \
4331 ▷ ▷ -end
4332
4333 # use option -without_group to skip the group. This will work:
4334
4335 Op_6_64_line_rank:
4336 ▷ $(ORBITER) -v 4 \
4337 ▷ ▷ -define F -finite_field -q 64 -end \
4338 ▷ ▷ -define 0 -orthogonal_space 1 6 F -without_group -end \
4339 ▷ ▷ -with 0 -do -orthogonal_space_activity \
4340 ▷ ▷ ▷ -unrank_line_through_two_points 15447347 15225451 \
4341 ▷ ▷ -end
4342
4343 # this will create a basic report without the group:
4344
4345 Op_6_64_report:
4346 ▷ $(ORBITER) -v 4 \
4347 ▷ ▷ -define F -finite_field -q 64 -end \
4348 ▷ ▷ -define 0 -orthogonal_space 1 6 F -without_group -end \
4349 ▷ ▷ -with 0 -do -orthogonal_space_activity \
4350 ▷ ▷ ▷ -cheat_sheet_orthogonal \
4351 ▷ ▷ -end
4352 ▷ pdflatex 0_1.6.64_report.tex
4353 ▷ $(OPEN) 0_1.6.64_report.pdf
4354

```

```

4355
4356
4357 # ToDo: this command fails:
4358
4359 elliptic_quadric_subspace:
4360 ▷ $(ORBITER) -v 3 \
4361 ▷ ▷ -define F -finite_field -q 5 -end \
4362 ▷ ▷ -define v -vector -format 4 \
4363 ▷ ▷ ▷ -dense "1,3,0,0,0,0, 0,0,1,0,0,0, 0,0,0,1,0,0, 0,0,0,0,1,1" \
4364 ▷ ▷ -end \
4365 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
4366 ▷ ▷ -with O -do -orthogonal_space_activity \
4367 ▷ ▷ ▷ -intersect_with_subspace v \
4368 ▷ ▷ -end
4369
4370 #We found that the intersection contains 26 points:
4371 #( 2, 3, 1572, 100, 98, 116, 132, 1570, 1588, 1604, 1631, 1688, 1704, 1706, 1747,
    1745, 1847, 1845, 1856, 1888, 1904, 1931, 1988, 2004, 2022, 2020 )
4372
4373
4374
4375 BLT_database_7_1:
4376 ▷ $(ORBITER) -v 2 \
4377 ▷ ▷ -define F -finite_field -q 7 -end \
4378 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4379 ▷ ▷ -define S -geometric_object P \
4380 ▷ ▷ ▷ -BLT_database 1 \
4381 ▷ ▷ -end \
4382 ▷ ▷ -with S -do -combinatorial_object_activity -save \
4383 ▷ ▷ -end
4384
4385 # writes BLT_7_1.txt
4386
4387 BLT_database_7_1.print:
4388 ▷ $(ORBITER) -v 2 \
4389 ▷ ▷ -define F -finite_field -q 7 -end \
4390 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4391 ▷ ▷ -define S -set -file_orbiter_format BLT_7_1.txt -end \
4392 ▷ ▷ -with O -do -orthogonal_space_activity \
4393 ▷ ▷ ▷ -print_points S -end
4394 ▷ pdflatex S_set_report.tex
4395 ▷ $(OPEN) S_set_report.pdf
4396
4397 Doily_W_2:
4398 ▷ $(ORBITER) -v 2 \
4399 ▷ ▷ -define F -finite_field -q 2 -end \
4400 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4401 ▷ ▷ -define W2_points -set -loop 0 15 1 -end \
4402 ▷ ▷ -define W2_lines -set -loop 0 15 1 -end \
4403 ▷ ▷ -with O -do \
4404 ▷ ▷ -orthogonal_space_activity \
4405 ▷ ▷ ▷ -print_points W2_points \
4406 ▷ ▷ -end \
4407 ▷ ▷ -with O -do \
4408 ▷ ▷ -orthogonal_space_activity \
4409 ▷ ▷ ▷ -print_lines W2_lines \
4410 ▷ ▷ -end
4411 ▷ pdflatex W2_points_set_report.tex
4412 ▷ $(OPEN) W2_points_set_report.pdf

```

```

4413 ▷ pdflatex W2_lines_set_of_lines_report.tex
4414 ▷ $(OPEN) W2_lines_set_of_lines_report.pdf
4415
4416
4417 # the output defines doily.csv
4418
4419
4420
4421
4422
4423 #####
4424 # Section 4.8: Hermitian varieties
4425
4426
4427 SECTION.HERMITIAN.VARIETIES:
4428
4429 test_4.8:
4430 ▷ make H_2.4
4431 ▷ make H_2.9
4432 ▷ make H_3.4
4433
4434
4435 H_2.4:
4436 ▷ $(ORBITER) -v 2 \
4437 ▷ ▷ -define F -finite_field -q 4 -end \
4438 ▷ ▷ -with F -do -finite_field_activity \
4439 ▷ ▷ ▷ -cheat_sheet_hermitian 2 -end
4440 ▷ pdflatex H_2.4.tex
4441 ▷ $(OPEN) H_2.4.pdf
4442
4443
4444 H_2.9:
4445 ▷ $(ORBITER) -v 2 \
4446 ▷ ▷ -define F -finite_field -q 9 -end \
4447 ▷ ▷ -with F -do -finite_field_activity \
4448 ▷ ▷ ▷ -cheat_sheet_hermitian 2 -end
4449 ▷ pdflatex H_2.9.tex
4450 ▷ $(OPEN) H_2.9.pdf
4451
4452
4453 # 28 points: 6, 11, 9, 7, 14, 19, 17, 15, 80, 75, 78, 74, 35, 30, 33, 29, 62, 57,
      60, 56, 26, 21, 24, 3, 37, 82, 64, 46
4454
4455
4456 H_3.4:
4457 ▷ $(ORBITER) -v 2 \
4458 ▷ ▷ -define F -finite_field -q 4 -end \
4459 ▷ ▷ -with F -do -finite_field_activity \
4460 ▷ ▷ ▷ -cheat_sheet_hermitian 3 -end
4461 ▷ pdflatex H_3.4.tex
4462 ▷ $(OPEN) H_3.4.pdf
4463
4464
4465 # H_3.4 = the Hirschfeld surface
4466
4467
4468 #####
4469 # Section 4.9: Projective Space Advanced Topics
4470

```

```

4471
4472 SECTION_PROJECTIVE_SPACE_ADVANCED_TOPICS:
4473
4474
4475 test_4.9:
4476 ▷ make fixed_objects_2A
4477 ▷ make fixed_objects_2B
4478 ▷ make fixed_objects_2C
4479 ▷ make fix_structure_G1
4480 ▷ make trans
4481 ▷ make PG_2_3_plane_pencil
4482
4483
4484
4485 fixed_objects_2A:
4486 ▷ $(ORBITER) -v 2 \
4487 ▷ ▷ -define F -finite_field -q 4 -end \
4488 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4489 ▷ ▷ -with P -do \
4490 ▷ ▷ -projective_space_activity \
4491 ▷ ▷ ▷ -report_fixed_objects \
4492 ▷ ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1, 1" \
4493 ▷ ▷ ▷ ▷ fix_structure_2A.tex \
4494 ▷ ▷ -end
4495 ▷ pdflatex fix_structure_2A.tex
4496 ▷ $(OPEN) fix_structure_2A.pdf
4497
4498
4499 fixed_objects_2B:
4500 ▷ $(ORBITER) -v 2 \
4501 ▷ ▷ -define F -finite_field -q 4 -end \
4502 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4503 ▷ ▷ -with P -do \
4504 ▷ ▷ -projective_space_activity \
4505 ▷ ▷ ▷ -report_fixed_objects \
4506 ▷ ▷ ▷ ▷ "1,0,0,0, 1,1,0,0, 0,0,1,0, 0,0,0,1, 0" \
4507 ▷ ▷ ▷ ▷ fix_structure_2B.tex \
4508 ▷ ▷ -end
4509 ▷ pdflatex fix_structure_2B.tex
4510 ▷ $(OPEN) fix_structure_2B.pdf
4511
4512
4513 fixed_objects_2C:
4514 ▷ $(ORBITER) -v 2 \
4515 ▷ ▷ -define F -finite_field -q 4 -end \
4516 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4517 ▷ ▷ -with P -do \
4518 ▷ ▷ -projective_space_activity \
4519 ▷ ▷ ▷ -report_fixed_objects \
4520 ▷ ▷ ▷ ▷ "1,0,0,0, 1,1,0,0, 0,0,1,0, 0,0,1,1, 0" \
4521 ▷ ▷ ▷ ▷ fix_structure_2C.tex \
4522 ▷ ▷ -end
4523 ▷ pdflatex fix_structure_2C.tex
4524 ▷ $(OPEN) fix_structure_2C.pdf
4525
4526
4527
4528
4529 fix_structure_G1:

```

```

4530 ▷ $(ORBITER) -v 2 \
4531 ▷ ▷ -define F -finite_field -q 4 -end \
4532 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4533 ▷ ▷ -with P -do \
4534 ▷ ▷ -projective_space_activity \
4535 ▷ ▷ ▷ -decomposition_by_subgroup \
4536 ▷ ▷ ▷ ▷ $(H1_LABEL) $(H1_GENS) \
4537 ▷ ▷ -end
4538 ▷ pdflatex PGGL_4_4_Subgroup_$(H1_LABEL)_4_decomp.tex
4539 ▷ $(OPEN) PGGL_4_4_Subgroup_$(H1_LABEL)_4_decomp.pdf
4540
4541
4542
4543
4544 trans:
4545 ▷ $(ORBITER) -v 5 \
4546 ▷ ▷ -define F -finite_field -q 16 -end \
4547 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4548 ▷ ▷ -with P -do \
4549 ▷ ▷ -projective_space_activity \
4550 ▷ ▷ ▷ -move_two_lines_in_hyperplane_stabilizer_text \
4551 ▷ ▷ ▷ ▷ "1,0,0,0, 0,0,0,1" "1,1,0,2, 0,0,1,0" \
4552 ▷ ▷ ▷ ▷ "1,0,0,0, 0,0,0,1" "0,1,0,1, 0,0,1,0" \
4553 ▷ ▷ -end
4554
4555
4556 PG_2_3_plane_pencil:
4557 ▷ $(ORBITER) -v 2 \
4558 ▷ ▷ -define F -finite_field -q 2 -end \
4559 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4560 ▷ ▷ -with P -do \
4561 ▷ ▷ -projective_space_activity \
4562 ▷ ▷ ▷ -planes_through_line 0 \
4563 ▷ ▷ -end \
4564
4565
4566
4567
4568 #####
4569 # Section 4.10: Geometric Objects
4570
4571
4572 SECTION_GEOMETRIC_OBJECTS:
4573
4574
4575
4576 test_4.10:
4577 ▷ make elliptic_quadric_ovoid_q8
4578 ▷ make ovoid_ST_q8
4579 ▷ make Baer_PG_2_4
4580 ▷ make Baer_PG_3_4
4581 ▷ make BLT_database_5_0
4582 ▷ make BLT_database_7_0
4583 ▷ make BLT_database_7_0_print
4584 ▷ make BLT_database_67_4
4585 ▷ make Doily_disjoint_sets_graph_cliques_3
4586 ▷ make Doily_disjoint_sets_graph_cliques_5
4587 ▷ make PG_3_2_test
4588 ▷ make Edge_curve_17

```



```

4589 ▷ make Edge_curve.17.line_type
4590 ▷ make Edge_curve.q23.line_type
4591
4592
4593
4594 elliptic_quadric_ovoid_q8:
4595 ▷ $(ORBITER) -v 2 \
4596 ▷ ▷ -define F -finite_field -q 8 -end \
4597 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4598 ▷ ▷ -define O -geometric_object P \
4599 ▷ ▷ ▷ -elliptic_quadric_ovoid \
4600 ▷ ▷ -end \
4601 ▷ ▷ -with 0 -do -combinatorial_object_activity -save \
4602 ▷ ▷ -end
4603
4604 #ovoid_q8.txt
4605 # 65 points
4606
4607 ovoid_ST_q8:
4608 ▷ $(ORBITER) -v 2 \
4609 ▷ ▷ -define F -finite_field -q 8 -end \
4610 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4611 ▷ ▷ -define O -geometric_object P \
4612 ▷ ▷ ▷ -ovoid_ST \
4613 ▷ ▷ -end \
4614 ▷ ▷ -with 0 -do -combinatorial_object_activity -save \
4615 ▷ ▷ -end
4616
4617 #ovoid_ST_q8.txt
4618
4619
4620 Baer_PG_2.4:
4621 ▷ $(ORBITER) -v 2 \
4622 ▷ ▷ -define F -finite_field -q 4 -end \
4623 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4624 ▷ ▷ -define O -geometric_object P \
4625 ▷ ▷ ▷ -Baer_substructure \
4626 ▷ ▷ -end \
4627 ▷ ▷ -with 0 -do -combinatorial_object_activity -save \
4628 ▷ ▷ -end
4629
4630 Baer_PG_3.4:
4631 ▷ $(ORBITER) -v 2 \
4632 ▷ ▷ -define F -finite_field -q 4 -end \
4633 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4634 ▷ ▷ -define O -geometric_object P \
4635 ▷ ▷ ▷ -Baer_substructure \
4636 ▷ ▷ -end \
4637 ▷ ▷ -with 0 -do -combinatorial_object_activity -save \
4638 ▷ ▷ -end
4639
4640 BLT_database_5_0:
4641 ▷ $(ORBITER) -v 2 \
4642 ▷ ▷ -define F -finite_field -q 5 -end \
4643 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4644 ▷ ▷ -define O -geometric_object P \
4645 ▷ ▷ ▷ -BLT_database 0 \
4646 ▷ ▷ -end \
4647 ▷ ▷ -with 0 -do -combinatorial_object_activity -save \

```

```

4648 ▷ ▷ -end
4649
4650 # writes BLT_5_0.txt
4651
4652
4653 BLT_database_7_0:
4654 ▷ $(ORBITER) -v 2 \
4655 ▷ ▷ -define F -finite_field -q 7 -end \
4656 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4657 ▷ ▷ -define O -geometric_object P \
4658 ▷ ▷ ▷ -BLT_database 0 \
4659 ▷ ▷ -end \
4660 ▷ ▷ -with O -do -combinatorial_object_activity -save \
4661 ▷ ▷ -end
4662
4663 # writes BLT_7_0.txt
4664
4665
4666 BLT_database_7_0.print:
4667 ▷ $(ORBITER) -v 2 \
4668 ▷ ▷ -define F -finite_field -q 7 -end \
4669 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4670 ▷ ▷ -define Obj -geometric_object P \
4671 ▷ ▷ ▷ -BLT_database 0 \
4672 ▷ ▷ -end \
4673 ▷ ▷ -with Obj -do -combinatorial_object_activity -save \
4674 ▷ ▷ -end \
4675 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4676 ▷ ▷ -define BLT_7_0 -set -file_orbiter_format BLT_7_0.txt -end \
4677 ▷ ▷ -with O -do -orthogonal_space_activity \
4678 ▷ ▷ ▷ -print_points BLT_7_0 -end
4679 ▷ pdflatex BLT_7_0_set_report.tex
4680 ▷ $(OPEN) BLT_7_0_set_report.pdf
4681
4682
4683 BLT_database_67_4:
4684 ▷ $(ORBITER) -v 2 \
4685 ▷ ▷ -define F -finite_field -q 67 -end \
4686 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4687 ▷ ▷ -define Obj -geometric_object P \
4688 ▷ ▷ ▷ -BLT_database 4 \
4689 ▷ ▷ -end \
4690 ▷ ▷ -with Obj -do -combinatorial_object_activity -save \
4691 ▷ ▷ -end \
4692 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4693 ▷ ▷ -define BLT_67_4 -set -file_orbiter_format BLT_67_4.txt -end \
4694 ▷ ▷ -with O -do -orthogonal_space_activity \
4695 ▷ ▷ ▷ -print_points BLT_67_4 -end
4696 ▷ pdflatex BLT_67_4_set_report.tex
4697 ▷ $(OPEN) BLT_67_4_set_report.pdf
4698
4699
4700
4701
4702
4703
4704
4705
4706 Doily_disjoint_sets_graph_cliques_3:

```

```

4707 ▷ echo $(DOILY) >doily.csv
4708 ▷ $(ORBITER) -v 2 \
4709 ▷ ▷ -define G -graph -disjoint_sets_graph \
4710 ▷ ▷ ▷ doily.csv \
4711 ▷ ▷ -end \
4712 ▷ ▷ -with G -do \
4713 ▷ ▷ ▷ -graph.theoretic.activity \
4714 ▷ ▷ ▷ -find_cliques \
4715 ▷ ▷ ▷ ▷ -target.size 3 \
4716 ▷ ▷ ▷ ▷ -output.file doily_cliques \
4717 ▷ ▷ ▷ -end \
4718 ▷ ▷ -end \
4719 ▷ ▷ -print_symbols
4720 ▷ $(ORBITER) -v 2 \
4721 ▷ ▷ -union doily.csv doily_cliques.txt doily_cliques_union.csv
4722
4723 # 80 cliques
4724
4725 Doily_disjoint_sets_graph_cliques_5:
4726 ▷ echo $(DOILY) >doily.csv
4727 ▷ $(ORBITER) -v 2 \
4728 ▷ ▷ -define G -graph -disjoint_sets_graph \
4729 ▷ ▷ ▷ doily.csv \
4730 ▷ ▷ -end \
4731 ▷ ▷ -with G -do \
4732 ▷ ▷ ▷ -graph.theoretic.activity \
4733 ▷ ▷ ▷ -find_cliques \
4734 ▷ ▷ ▷ ▷ -target.size 5 \
4735 ▷ ▷ ▷ ▷ -output.file doily_cliques_5 \
4736 ▷ ▷ ▷ -end \
4737 ▷ ▷ -end \
4738 ▷ ▷ -print_symbols
4739
4740 # 6 cliques
4741 # doily_cliques_5.csv
4742
4743
4744 PG_3_2_test:
4745 ▷ $(ORBITER) -v 2 \
4746 ▷ ▷ -define F -finite_field -q 2 -end \
4747 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4748 ▷ ▷ -with P -do -projective_space_activity \
4749 ▷ ▷ ▷ -cheat_sheet \
4750 ▷ ▷ -end
4751 ▷ pdflatex PG_3_2.tex
4752 ▷ $(OPEN) PG_3_2.pdf
4753
4754
4755 Edge_curve_17:
4756 ▷ $(ORBITER) -v 2 \
4757 ▷ ▷ -define F -finite_field -q 17 -end \
4758 ▷ ▷ -define R -polynomial_ring -field F \
4759 ▷ ▷ ▷ -number_of_variables 3 \
4760 ▷ ▷ ▷ -homogeneous_of_degree 4 \
4761 ▷ ▷ ▷ -end \
4762 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4763 ▷ ▷ -define C -geometric_object P \
4764 ▷ ▷ ▷ -projective_variety R \
4765 ▷ ▷ ▷ ▷ "Edge_q17" "Edge\q17" \

```

```

4766 ▷ ▷ ▷ ▷ $(EDGE_CURVE_Q17.EQUATION) \
4767 ▷ ▷ -end \
4768 ▷ ▷ -with C -do -combinatorial_object_activity -save \
4769 ▷ ▷ -end
4770
4771 #Edge_q17.txt
4772 #combinatorial_object_create::init created a set of size 12
4773
4774
4775
4776
4777
4778
4779
4780
4781 Edge_curve_17_line_type:
4782 ▷ echo $(FILE_Q17) >edge_q17.csv
4783 ▷ $(ORBITER) -v 2 \
4784 ▷ ▷ -define F -finite_field -q 17 -end \
4785 ▷ ▷ -define R -polynomial_ring -field F \
4786 ▷ ▷ ▷ -number_of_variables 3 \
4787 ▷ ▷ ▷ -homogeneous_of_degree 4 \
4788 ▷ ▷ ▷ -end \
4789 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4790 ▷ ▷ -define C -geometric_object P \
4791 ▷ ▷ ▷ -projective_variety R \
4792 ▷ ▷ ▷ ▷ "Edge_q17" "Edge\_q17" \
4793 ▷ ▷ ▷ ▷ $(EDGE_CURVE_Q17.EQUATION) \
4794 ▷ ▷ ▷ -end \
4795 ▷ ▷ -with C -do \
4796 ▷ ▷ -combinatorial_object_activity \
4797 ▷ ▷ ▷ -line_type_old \
4798 ▷ ▷ -end \
4799 ▷ ▷ -print_symbols
4800
4801 #( 4^6, 2^30, 1^132, 0^139 )
4802
4803
4804 Edge_curve_q23_line_type:
4805 ▷ $(ORBITER) -v 2 \
4806 ▷ ▷ -define F -finite_field -q 23 -end \
4807 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4808 ▷ ▷ -define E -geometric_object P \
4809 ▷ ▷ ▷ -set "edge_curve_q23" "edge\_curve\_q23" \
4810 ▷ ▷ ▷ $(EDGE_CURVE_Q23.AS.POINTS) \
4811 ▷ ▷ -end \
4812 ▷ ▷ -with E -do \
4813 ▷ ▷ -combinatorial_object_activity \
4814 ▷ ▷ ▷ -save \
4815 ▷ ▷ -end \
4816 ▷ ▷ -with E -do \
4817 ▷ ▷ -combinatorial_object_activity \
4818 ▷ ▷ ▷ -line_type_old \
4819 ▷ ▷ -end \
4820 ▷ ▷ -print_symbols
4821
4822
4823
4824

```

```

4825
4826
4827 #####
4828 # Chapter 5 - Ring Theory
4829 #####
4830
4831
4832 test_5:
4833 ▷ make test_5.1
4834 ▷ make test_5.2
4835 ▷ make test_5.3
4836
4837
4838
4839
4840 #####
4841 # Section 5.1: Polynomials over Finite Fields
4842
4843 SECTION.POLYNOMIALS:
4844
4845
4846 test_5.1:
4847 ▷ make sift_polynomials_deg3.q2
4848 ▷ make sift_polynomials_deg4.q2
4849 ▷ make poly_division
4850 ▷ make poly_division2
4851 ▷ make poly_gcd
4852 ▷ make poly_mult_mod1
4853 ▷ make poly_mult_mod2
4854 ▷ make poly_mult_mod_F4
4855 ▷ make mult_polynomials_2.5.7
4856 ▷ make polynomial_division_ranked_2.27.13
4857 ▷ make mult_polynomials_2.8.15
4858 ▷ make polynomial_division_ranked_2.120.25
4859 ▷ make mult_polynomials_2.7.7
4860 ▷ make mult_polynomials_2.4.6
4861 ▷ make polynomial_division_ranked_2.24.13
4862 ▷ make mult_polynomials_1024.999.997
4863 ▷ make polynomial_division_ranked_2.349147.1033
4864 ▷ make mult_polynomials_17.12
4865 ▷ make polynomial_division_ranked_2.204.37
4866 ▷ make Berlekamp_matrix_crc32
4867 ▷ make power_mod_inverse
4868 ▷ make mult_mod_to_get_one
4869 ▷ make Berlekamp_matrix_2.3
4870 ▷ make Berlekamp_matrix_2.4
4871 ▷ make Berlekamp_matrix_4.3a
4872 ▷ make Berlekamp_matrix_4.3b
4873 ▷ make find_roots_a
4874 ▷ make find_roots_b
4875 ▷ make find_roots_c
4876 ▷ make find_roots_d
4877 ▷ make find_roots_e
4878 ▷ make roots_over_F2
4879 ▷ make roots_over_F8
4880 ▷ make irred_3.2
4881 ▷ make irred_4.2
4882 ▷ make irred_5.2
4883 ▷ make irred_6.2

```

```

4884 ▷ make irred_7.2
4885 ▷ make irred_8.2
4886 ▷ make irred_9.2
4887 ▷ make irred_10.2
4888 ▷ make irred_2.4
4889 ▷ make irred_3.4
4890 ▷ make F2_get_primitive_poly_of_deg_10
4891 ▷ make F2_get_primitive_poly_range_2_10
4892
4893
4894 # check which polynomials are irreducible and which are primitive:
4895
4896 sift_polynomials_deg3.q2:
4897 ▷ $(ORBITER) -v 2 \
4898 ▷ ▷ -define F -finite_field -q 2 -end \
4899 ▷ ▷ -with F -do \
4900 ▷ ▷ -finite_field_activity -sift_polynomials 8 16 -end
4901
4902
4903
4904 sift_polynomials_deg4.q2:
4905 ▷ $(ORBITER) -v 2 \
4906 ▷ ▷ -define F -finite_field -q 2 -end \
4907 ▷ ▷ -with F -do \
4908 ▷ ▷ -finite_field_activity -sift_polynomials 16 32 -end
4909
4910
4911
4912
4913 poly_division:
4914 ▷ $(ORBITER) -v 2 \
4915 ▷ ▷ -define F -finite_field -q 2 -end \
4916 ▷ ▷ -with F -do \
4917 ▷ ▷ -finite_field_activity \
4918 ▷ ▷ -polynomial_division "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" -end
4919
4920 poly_division2:
4921 ▷ $(ORBITER) -v 2 \
4922 ▷ ▷ -define F -finite_field -q 2 -end \
4923 ▷ ▷ -define A -vector -field F -sparse 11 "1,0,1,10" -end \
4924 ▷ ▷ -define B -vector -field F -dense "1,0,1,1" -end \
4925 ▷ ▷ -with F -do \
4926 ▷ ▷ -finite_field_activity \
4927 ▷ ▷ -polynomial_division A B -end
4928
4929
4930 poly_gcd:
4931 ▷ $(ORBITER) -v 2 \
4932 ▷ ▷ -define F -finite_field -q 2 -end \
4933 ▷ ▷ -with F -do \
4934 ▷ ▷ -finite_field_activity \
4935 ▷ ▷ -extended_gcd_for_polynomials "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" -end
4936
4937 poly_mult_mod1:
4938 ▷ $(ORBITER) -v 2 \
4939 ▷ ▷ -define F -finite_field -q 7 -end \
4940 ▷ ▷ -with F -do \
4941 ▷ ▷ -finite_field_activity \
4942 ▷ ▷ -polynomial_mult_mod "1,2,3" "3,4,5" "6,0,0,1" -end

```

```

4943
4944 poly_mult_mod2:
4945 ▷ $(ORBITER) -v 2 \
4946 ▷ ▷ -define F -finite_field -q 7 -end \
4947 ▷ ▷ -with F -do \
4948 ▷ ▷ -finite_field_activity \
4949 ▷ ▷ -polynomial_mult_mod "3,1,2" "5,3,4" "6,0,0,1" -end
4950
4951
4952
4953 poly_mult_mod.F4:
4954 ▷ $(ORBITER) -v 2 \
4955 ▷ ▷ -define F -finite_field -q 2 -end \
4956 ▷ ▷ -with F -do \
4957 ▷ ▷ -finite_field_activity \
4958 ▷ ▷ -polynomial_mult_mod "1,1" "1,1" "1,1,1" -end
4959 ▷ $(ORBITER) -v 2 \
4960 ▷ ▷ -define F -finite_field -q 2 -end \
4961 ▷ ▷ -with F -do \
4962 ▷ ▷ -finite_field_activity \
4963 ▷ ▷ -polynomial_mult_mod "0,1" "1,1" "1,1,1" -end
4964 ▷ $(ORBITER) -v 2 \
4965 ▷ ▷ -define F -finite_field -q 2 -end \
4966 ▷ ▷ -with F -do \
4967 ▷ ▷ -finite_field_activity \
4968 ▷ ▷ -polynomial_mult_mod "0,1" "0,1" "1,1,1" -end
4969
4970
4971
4972
4973
4974 mult_polynomials_2.5.7:
4975 ▷ $(ORBITER) -v 2 \
4976 ▷ ▷ -define F -finite_field -q 2 -end \
4977 ▷ ▷ -with F -do \
4978 ▷ ▷ -finite_field_activity -mult_polynomials 5 7 -end
4979 ▷ pdflatex polynomial_mult_5.7.tex
4980 ▷ $(OPEN) polynomial_mult_5.7.pdf
4981
4982
4983 polynomial_division_ranked_2.27.13:
4984 ▷ $(ORBITER) -v 2 \
4985 ▷ ▷ -define F -finite_field -q 2 -end \
4986 ▷ ▷ -with F -do \
4987 ▷ ▷ -finite_field_activity \
4988 ▷ ▷ ▷ -polynomial_division_ranked 27 13 \
4989 ▷ ▷ -end
4990 ▷ pdflatex polynomial_division_27.13.tex
4991 ▷ $(OPEN) polynomial_division_27.13.pdf
4992
4993
4994
4995
4996 mult_polynomials_2.8.15:
4997 ▷ $(ORBITER) -v 2 \
4998 ▷ ▷ -define F -finite_field -q 2 -end \
4999 ▷ ▷ -with F -do \
5000 ▷ ▷ -finite_field_activity -mult_polynomials 8 15 -end
5001 ▷ pdflatex polynomial_mult_8.15.tex

```

```

5002 ▷ $(OPEN) polynomial_mult.8.15.pdf
5003
5004 polynomial_division_ranked.2.120.25:
5005 ▷ $(ORBITER) -v 2 \
5006 ▷ ▷ -define F -finite_field -q 2 -end \
5007 ▷ ▷ -with F -do \
5008 ▷ ▷ -finite_field_activity \
5009 ▷ ▷ ▷ -polynomial_division_ranked 120 25 \
5010 ▷ ▷ -end
5011 ▷ pdflatex polynomial_division_120.25.tex
5012 ▷ $(OPEN) polynomial_division_120.25.pdf
5013
5014 # the answer is 5
5015
5016
5017 mult_polynomials.2.7.7:
5018 ▷ $(ORBITER) -v 2 \
5019 ▷ ▷ -define F -finite_field -q 2 -end \
5020 ▷ ▷ -with F -do \
5021 ▷ ▷ -finite_field_activity \
5022 ▷ ▷ -mult_polynomials 7 7 -end
5023 ▷ pdflatex polynomial_mult_7.7.tex
5024 ▷ $(OPEN) polynomial_mult_7.7.pdf
5025
5026
5027
5028 mult_polynomials.2.4.6:
5029 ▷ $(ORBITER) -v 2 \
5030 ▷ ▷ -define F -finite_field -q 2 -end \
5031 ▷ ▷ -with F -do \
5032 ▷ ▷ -finite_field_activity \
5033 ▷ ▷ -mult_polynomials 4 6 -end
5034 ▷ pdflatex polynomial_mult_4.6.tex
5035 ▷ $(OPEN) polynomial_mult_4.6.pdf
5036
5037 polynomial_division_ranked.2.24.13:
5038 ▷ $(ORBITER) -v 2 \
5039 ▷ ▷ -define F -finite_field -q 2 -end \
5040 ▷ ▷ -with F -do \
5041 ▷ ▷ -finite_field_activity \
5042 ▷ ▷ ▷ -polynomial_division_ranked 24 13 \
5043 ▷ ▷ -end
5044 ▷ pdflatex polynomial_division_24.13.tex
5045 ▷ $(OPEN) polynomial_division_24.13.pdf
5046
5047
5048
5049 mult_polynomials_1024.999.997:
5050 ▷ $(ORBITER) -v 2 \
5051 ▷ ▷ -define F -finite_field -q 2 -end \
5052 ▷ ▷ -with F -do \
5053 ▷ ▷ -finite_field_activity \
5054 ▷ ▷ ▷ -mult_polynomials 999 997 \
5055 ▷ ▷ -end
5056 ▷ pdflatex polynomial_mult_999.997.tex
5057 ▷ $(OPEN) polynomial_mult_999.997.pdf
5058
5059
5060 polynomial_division_ranked.2.349147.1033:

```



```

5061 ▷ $(ORBITER) -v 2 \
5062 ▷ ▷ -define F -finite_field -q 2 -end \
5063 ▷ ▷ -with F -do \
5064 ▷ ▷ -finite_field_activity \
5065 ▷ ▷ -polynomial_division_ranked 349147 1033 \
5066 ▷ ▷ -end
5067 ▷ pdflatex polynomial_division_349147_1033.tex
5068 ▷ $(OPEN) polynomial_division_349147_1033.pdf
5069
5070
5071
5072
5073
5074 mult_polynomials_17_12:
5075 ▷ $(ORBITER) -v 2 \
5076 ▷ ▷ -define F -finite_field -q 2 -end \
5077 ▷ ▷ -with F -do \
5078 ▷ ▷ -finite_field_activity \
5079 ▷ ▷ -mult_polynomials 17 12 -end
5080 ▷ pdflatex polynomial_mult_17_12.tex
5081 ▷ $(OPEN) polynomial_mult_17_12.pdf
5082 ▷
5083 # gives 204
5084
5085 polynomial_division_ranked_2_204_37:
5086 ▷ $(ORBITER) -v 2 \
5087 ▷ ▷ -define F -finite_field -q 2 -end \
5088 ▷ ▷ -with F -do \
5089 ▷ ▷ -finite_field_activity \
5090 ▷ ▷ ▷ -polynomial_division_ranked 204 37 \
5091 ▷ ▷ -end
5092 ▷ pdflatex polynomial_division_204_37.tex
5093 ▷ $(OPEN) polynomial_division_204_37.pdf
5094 ▷
5095 # answer is 18
5096 ▷
5097 ▷
5098
5099
5100
5101
5102 #ToDo:
5103
5104 #test_crc32:
5105 #▷ $(ORBITER) -v 3 \
5106 #▷ ▷ -crc32 "123456789"
5107
5108
5109 Berlekamp_matrix_crc32:
5110 ▷ $(ORBITER) -v 2 \
5111 ▷ ▷ -define F -finite_field -q 2 -end \
5112 ▷ ▷ -define v -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
5113 ▷ ▷ -with F -do \
5114 ▷ ▷ -finite_field_activity \
5115 ▷ ▷ -Berlekamp_matrix v -end
5116
5117
5118 # N = 2^32-1 = 3 * 5 * 17 * 257 * 65537
5119 # N / 3 = 1431655765

```

```

5120 # N / 5 = 858993459
5121 # N / 17 = 252645135
5122 # N / 257 = 16711935
5123 # N / 65537 = 65535
5124
5125
5126
5127 power_mod_inverse:
5128 ▷ $(ORBITER) -v 2 \
5129 ▷ ▷ -define F -finite_field -q 2 -end \
5130 ▷ ▷ -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
5131 ▷ ▷ -define A -vector -field F -sparse 2 "1,1" -end \
5132 ▷ ▷ -with F -do \
5133 ▷ ▷ -finite_field_activity \
5134 ▷ ▷ -polynomial_power_mod A $(TWO_TO_THE_32_MINUS_2) M \
5135 ▷ ▷ -end
5136
5137
5138 #A(X)=X^{31} + X^{25} + X^{22} + X^{21} + X^{15}
5139 #+ X^{11} + X^{10} + X^9 + X^7 + X^6 + X^4 + X^3 + X + 1
5140
5141 mult_mod_to_get_one:
5142 ▷ $(ORBITER) -v 2 \
5143 ▷ ▷ -define F -finite_field -q 2 -end \
5144 ▷ ▷ -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
5145 ▷ ▷ -define A -vector -field F -sparse 2 "1,1" -end \
5146 ▷ ▷ -define B -vector -field F -sparse 33 $(INVERSE_SPARSE) -end \
5147 ▷ ▷ -with F -do \
5148 ▷ ▷ -finite_field_activity \
5149 ▷ ▷ ▷ -polynomial_mult_mod A B M \
5150 ▷ ▷ -end
5151
5152 #C(X)=1
5153
5154
5155
5156
5157
5158
5159
5160 Berlekamp_matrix_2.3:
5161 ▷ $(ORBITER) -v 2 \
5162 ▷ ▷ -define F -finite_field -q 2 -end \
5163 ▷ ▷ -define v -vector -field F -dense "1,1,0,1" -end \
5164 ▷ ▷ -with F -do \
5165 ▷ ▷ -finite_field_activity \
5166 ▷ ▷ -Berlekamp_matrix v -end
5167 ▷ pdflatex Berlekamp_matrix_q2_d3.tex
5168 ▷ $(OPEN) Berlekamp_matrix_q2_d3.pdf
5169
5170 # the polynomial X^3+X+1 is irreducible over GF(2) because the rank of the Berlek
    amp matrix is 2.
5171
5172 Berlekamp_matrix_2.4:
5173 ▷ $(ORBITER) -v 2 \
5174 ▷ ▷ -define F -finite_field -q 2 -end \
5175 ▷ ▷ -define v -vector -field F -dense "1,1,0,0,1" -end \
5176 ▷ ▷ -with F -do \
5177 ▷ ▷ -finite_field_activity \

```

```

5178 ▷ ▷ -Berlekamp_matrix v -end
5179
5180 # the polynomial  $X^4+X+1$  is irreducible over GF(2) because the rank of the Berlek
    amp matrix is 3.
5181
5182
5183 Berlekamp_matrix_4.3a:
5184 ▷ $(ORBITER) -v 2 \
5185 ▷ ▷ -define F -finite_field -q 4 -end \
5186 ▷ ▷ -define v -vector -field F -dense "1,3,0,1" -end \
5187 ▷ ▷ -with F -do \
5188 ▷ ▷ -finite_field_activity \
5189 ▷ ▷ -Berlekamp_matrix v -end
5190
5191 Berlekamp_matrix_4.3b:
5192 ▷ $(ORBITER) -v 2 \
5193 ▷ ▷ -define F -finite_field -q 4 -end \
5194 ▷ ▷ -define v -vector -field F -dense "1,3,1,1" -end \
5195 ▷ ▷ -with F -do \
5196 ▷ ▷ -finite_field_activity \
5197 ▷ ▷ -Berlekamp_matrix v -end
5198
5199
5200
5201
5202 find_roots.a:
5203 ▷ $(ORBITER) -v 2 \
5204 ▷ ▷ -define F -finite_field -q 19 -end \
5205 ▷ ▷ -define v -vector -field F -dense "18,1,1" -end \
5206 ▷ ▷ -with F -do \
5207 ▷ ▷ -finite_field_activity \
5208 ▷ ▷ -polynomial_find_roots v -end
5209
5210 find_roots.b:
5211 ▷ $(ORBITER) -v 2 \
5212 ▷ ▷ -define F -finite_field -q 19 -end \
5213 ▷ ▷ -define v -vector -field F -dense "1,3,1" -end \
5214 ▷ ▷ -with F -do \
5215 ▷ ▷ -finite_field_activity \
5216 ▷ ▷ -polynomial_find_roots v -end
5217
5218 find_roots.c:
5219 ▷ $(ORBITER) -v 2 \
5220 ▷ ▷ -define F -finite_field -q 19 -end \
5221 ▷ ▷ -define v -vector -field F -dense "1,16,1" -end \
5222 ▷ ▷ -with F -do \
5223 ▷ ▷ -finite_field_activity \
5224 ▷ ▷ -polynomial_find_roots v -end
5225
5226 find_roots.d:
5227 ▷ $(ORBITER) -v 2 \
5228 ▷ ▷ -define F -finite_field -q 19 -end \
5229 ▷ ▷ -define v -vector -field F -dense "1,18,1" -end \
5230 ▷ ▷ -with F -do \
5231 ▷ ▷ -finite_field_activity \
5232 ▷ ▷ -polynomial_find_roots v -end
5233
5234 find_roots.e:
5235 ▷ $(ORBITER) -v 2 \

```

```

5236 ▷ ▷ -define F -finite_field -q 19 -end \
5237 ▷ ▷ -define v -vector -field F -dense "1,16,3" -end \
5238 ▷ ▷ -with F -do \
5239 ▷ ▷ -finite_field_activity \
5240 ▷ ▷ -polynomial_find.roots v -end
5241
5242
5243 roots_over_F2:
5244 ▷ $(ORBITER) -v 2 \
5245 ▷ ▷ -define F -finite_field -q 2 -end \
5246 ▷ ▷ -define v -vector -field F -dense "0,1,0,1,1,1" -end \
5247 ▷ ▷ -with F -do \
5248 ▷ ▷ -finite_field_activity \
5249 ▷ ▷ -polynomial_find.roots v -end
5250
5251
5252
5253 roots_over_F8:
5254 ▷ $(ORBITER) -v 2 \
5255 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
5256 ▷ ▷ -define v -vector -field F -dense "0,1,0,1,1,1" -end \
5257 ▷ ▷ -with F -do \
5258 ▷ ▷ -finite_field_activity \
5259 ▷ ▷ -polynomial_find.roots v -end
5260
5261
5262
5263
5264 # degree and then order of the field of coefficients:
5265
5266 irred_3.2:
5267 ▷ $(ORBITER) -v 3 \
5268 ▷ ▷ -define F -finite_field -q 2 -end \
5269 ▷ ▷ -with F -do \
5270 ▷ ▷ -finite_field_activity \
5271 ▷ ▷ -make_table_of_irreducible_polynomials 3 -end
5272 ▷ pdflatex Irred.q2_d3.tex
5273 ▷ $(OPEN) Irred.q2_d3.pdf
5274
5275
5276 irred_4.2:
5277 ▷ $(ORBITER) -v 3 \
5278 ▷ ▷ -define F -finite_field -q 2 -end \
5279 ▷ ▷ -with F -do \
5280 ▷ ▷ -finite_field_activity \
5281 ▷ ▷ -make_table_of_irreducible_polynomials 4 -end
5282 ▷ pdflatex Irred.q2_d4.tex
5283 ▷ $(OPEN) Irred.q2_d4.pdf
5284
5285 # 3 polys
5286
5287 irred_5.2:
5288 ▷ $(ORBITER) -v 3 \
5289 ▷ ▷ -define F -finite_field -q 2 -end \
5290 ▷ ▷ -with F -do \
5291 ▷ ▷ -finite_field_activity \
5292 ▷ ▷ -make_table_of_irreducible_polynomials 5 -end
5293 ▷ pdflatex Irred.q2_d5.tex
5294 ▷ $(OPEN) Irred.q2_d5.pdf

```

```

5295
5296 # 6 polys
5297
5298 irred_6_2:
5299 ▷ $(ORBITER) -v 3 \
5300 ▷ ▷ -define F -finite_field -q 2 -end \
5301 ▷ ▷ -with F -do \
5302 ▷ ▷ -finite_field_activity \
5303 ▷ ▷ -make_table_of_irreducible_polynomials 6 -end
5304 ▷ pdflatex Irred_q2_d6.tex
5305 ▷ $(OPEN) Irred_q2_d6.pdf
5306
5307 # 9 polys: 97, 73, 67, 115, 117, 109, 91, 87, 103.
5308
5309 irred_7_2:
5310 ▷ $(ORBITER) -v 3 \
5311 ▷ ▷ -define F -finite_field -q 2 -end \
5312 ▷ ▷ -with F -do \
5313 ▷ ▷ -finite_field_activity \
5314 ▷ ▷ -make_table_of_irreducible_polynomials 7 -end
5315 ▷ pdflatex Irred_q2_d7.tex
5316 ▷ $(OPEN) Irred_q2_d7.pdf
5317
5318 # 18 polys
5319
5320 irred_8_2:
5321 ▷ $(ORBITER) -v 3 \
5322 ▷ ▷ -define F -finite_field -q 2 -end \
5323 ▷ ▷ -with F -do \
5324 ▷ ▷ -finite_field_activity \
5325 ▷ ▷ -make_table_of_irreducible_polynomials 8 -end
5326 ▷ pdflatex Irred_q2_d8.tex
5327 ▷ $(OPEN) Irred_q2_d8.pdf
5328
5329 # 30 polys
5330
5331 irred_9_2:
5332 ▷ $(ORBITER) -v 3 \
5333 ▷ ▷ -define F -finite_field -q 2 -end \
5334 ▷ ▷ -with F -do \
5335 ▷ ▷ -finite_field_activity \
5336 ▷ ▷ -make_table_of_irreducible_polynomials 9 -end
5337 ▷ pdflatex Irred_q2_d9.tex
5338 ▷ $(OPEN) Irred_q2_d9.pdf
5339
5340 # 56 polys
5341
5342 irred_10_2:
5343 ▷ $(ORBITER) -v 3 \
5344 ▷ ▷ -define F -finite_field -q 2 -end \
5345 ▷ ▷ -with F -do \
5346 ▷ ▷ -finite_field_activity \
5347 ▷ ▷ -make_table_of_irreducible_polynomials 10 -end
5348 ▷ pdflatex Irred_q2_d10.tex
5349 ▷ $(OPEN) Irred_q2_d10.pdf
5350
5351 # 99 polys
5352
5353 irred_2_4:

```

```

5354 ▷ $(ORBITER) -v 3 \
5355 ▷ ▷ -define F -finite_field -q 4 -end \
5356 ▷ ▷ -with F -do \
5357 ▷ ▷ -finite_field_activity \
5358 ▷ ▷ -make_table_of_irreducible_polynomials 2 -end
5359 ▷ pdflatex Irred.q4.d2.tex
5360 ▷ $(OPEN) Irred.q4.d2.pdf
5361
5362 # 6 polys
5363
5364
5365 irred_3.4:
5366 ▷ $(ORBITER) -v 6 \
5367 ▷ ▷ -define F -finite_field -q 4 -end \
5368 ▷ ▷ -with F -do \
5369 ▷ ▷ -finite_field_activity \
5370 ▷ ▷ -make_table_of_irreducible_polynomials 3 -end
5371 ▷ pdflatex Irred.q4.d3.tex
5372 ▷ $(OPEN) Irred.q4.d3.pdf
5373
5374 # 20 polys
5375
5376
5377 F2.get_primitive_poly_of_deg_10:
5378 ▷ $(ORBITER) -v 6 \
5379 ▷ ▷ -define F -finite_field -q 2 -end \
5380 ▷ ▷ -with F -do \
5381 ▷ ▷ -finite_field_activity \
5382 ▷ ▷ -get_primitive_polynomial 10 -end
5383
5384
5385 F2.get_primitive_poly_range_2_10:
5386 ▷ $(ORBITER) -v 2 \
5387 ▷ ▷ -define F -finite_field -q 2 -end \
5388 ▷ ▷ -with F -do \
5389 ▷ ▷ -finite_field_activity \
5390 ▷ ▷ -get_primitive_polynomial_in_range 2 10 -end
5391
5392 #| grep //
5393 # for degree 61, it gets stuck in factoring 2^61-1 (which is prime)
5394
5395
5396
5397
5398
5399
5400
5401
5402 #####
5403 # Section 5.2: Multivariate Polynomials
5404
5405
5406
5407 SECTION.MULTIVARIATE.POLYNOMIALS:
5408
5409
5410 test_5.2:
5411 ▷ #make parse_algebraic_formula error message
5412 ▷ make Fermat_cubic_F9

```

```

5413 ▷ make Fermat_cubic_F9_c
5414 ▷ make Cremona_map
5415 ▷ make arcs_5_2_q11
5416 ▷ make arcs_5_2_q11_ideal
5417 ▷ make surface_9lines_4E_ideal
5418 ▷ make F_9_q7
5419 ▷ make random_k_subsets_PG_2_11
5420 ▷ #make line_type_in_PG_2_11
5421 ▷ make random_arc_5_2_q11_ideal
5422 ▷ make Endrass_F7.txt
5423 ▷ make octic_prepare
5424 ▷ make affine_map_F5_0
5425 ▷ make affine_map_F5_1
5426 ▷ make affine_map_F5_2
5427 ▷ make affine_map_F5_3
5428 ▷ make affine_map_F5_4
5429 ▷ make permutation_polynomial_F5
5430
5431
5432
5433 parse_algebraic_formula:
5434 ▷ $(ORBITER) -v 6 \
5435 ▷ ▷ -define F -finite_field -q 13 -end \
5436 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5437 ▷ ▷ -define R -polynomial_ring \
5438 ▷ ▷ ▷ -field F \
5439 ▷ ▷ ▷ -number_of_variables 3 \
5440 ▷ ▷ ▷ -homogeneous_of_degree 3 \
5441 ▷ ▷ ▷ -monomial_ordering_partition \
5442 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
5443 ▷ ▷ -end \
5444 ▷ ▷ -with R -do \
5445 ▷ ▷ ▷ -ring_theoretic_activity \
5446 ▷ ▷ ▷ -parse_equation "eqn" "eqn" "y0^3+y1^3+y2^3" "" "" \
5447 ▷ ▷ -end
5448
5449 Fermat_cubic_F9:
5450 ▷ $(ORBITER) -v 6 \
5451 ▷ ▷ -define F -finite_field -q 9 -end \
5452 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5453 ▷ ▷ -define R -polynomial_ring \
5454 ▷ ▷ ▷ -field F \
5455 ▷ ▷ ▷ -number_of_variables 3 \
5456 ▷ ▷ ▷ -homogeneous_of_degree 3 \
5457 ▷ ▷ ▷ -monomial_ordering_partition \
5458 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
5459 ▷ ▷ ▷ -end \
5460 ▷ ▷ -define Fermat_cubic -symbolic_object \
5461 ▷ ▷ ▷ -field F \
5462 ▷ ▷ ▷ -text "y0^3+y1^3+y2^3" \
5463 ▷ ▷ ▷ -end \
5464 ▷ ▷ -with P -do \
5465 ▷ ▷ -projective_space_activity \
5466 ▷ ▷ ▷ -projective_variety R Fermat_cubic "" \
5467 ▷ ▷ -end
5468
5469 Fermat_cubic_F9_c:
5470 ▷ $(ORBITER) -v 6 \
5471 ▷ ▷ -define C -combinatorial_object \

```

```

5472 ▷ ▷ ▷ -label Fermat_cubic_F9 Fermat\_cubic\_F9 \
5473 ▷ ▷ ▷ -set_of_points "3, 5, 13, 28, 45, 53, 61, 69, 77, 85" \
5474 ▷ ▷ -end \
5475 ▷ ▷ -define F -finite_field -q 9 -end \
5476 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5477 ▷ ▷ -with C -do \
5478 ▷ ▷ -combinatorial_object_activity \
5479 ▷ ▷ ▷ -canonical_form_PG P \
5480 ▷ ▷ ▷ ▷ -save_ago \
5481 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
5482 ▷ ▷ ▷ -end \
5483 ▷ ▷ -end \
5484 ▷ ▷ -with C -do \
5485 ▷ ▷ -combinatorial_object_activity \
5486 ▷ ▷ ▷ -report \
5487 ▷ ▷ ▷ ▷ -export_flag_orbits \
5488 ▷ ▷ ▷ ▷ -show_TDO \
5489 ▷ ▷ ▷ ▷ -show_TDA \
5490 ▷ ▷ ▷ ▷ -export_labels \
5491 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
5492 ▷ ▷ ▷ ▷ -export_group_GAP \
5493 ▷ ▷ ▷ -end \
5494 ▷ ▷ -end
5495 ▷ pdflatex Fermat_cubic_F9_classification.tex
5496 ▷ $(OPEN) Fermat_cubic_F9_classification.pdf
5497
5498
5499 Cremona_map:
5500 ▷ $(ORBITER) -v 6 \
5501 ▷ ▷ -define F -finite_field -q 13 -end \
5502 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5503 ▷ ▷ -define R -polynomial_ring \
5504 ▷ ▷ ▷ -field F \
5505 ▷ ▷ ▷ -number_of_variables 3 \
5506 ▷ ▷ ▷ -homogeneous_of_degree 6 \
5507 ▷ ▷ ▷ -monomial_ordering_lex \
5508 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
5509 ▷ ▷ ▷ -end \
5510 ▷ ▷ -define Y0 -symbolic_object \
5511 ▷ ▷ ▷ -field F \
5512 ▷ ▷ ▷ -text $(CREMONA_MAP_Y0) \
5513 ▷ ▷ ▷ -end \
5514 ▷ ▷ -define Y1 -symbolic_object \
5515 ▷ ▷ ▷ -field F \
5516 ▷ ▷ ▷ -text $(CREMONA_MAP_Y1) \
5517 ▷ ▷ ▷ -end \
5518 ▷ ▷ -define Y2 -symbolic_object \
5519 ▷ ▷ ▷ -field F \
5520 ▷ ▷ ▷ -text $(CREMONA_MAP_Y2) \
5521 ▷ ▷ ▷ -end \
5522 ▷ ▷ -define Cremona -symbolic_object \
5523 ▷ ▷ ▷ -field F \
5524 ▷ ▷ ▷ -text "Y0,Y1,Y2" \
5525 ▷ ▷ ▷ -end \
5526 ▷ ▷ -with P -do \
5527 ▷ ▷ -projective_space_activity \
5528 ▷ ▷ ▷ -map R Cremona "" \
5529 ▷ ▷ -end
5530

```



```

5531
5532
5533
5534 arcs_5.2.q11:
5535 ▷ $(ORBITER) -v 4 \
5536 ▷ ▷ -define F -finite_field -q 11 -end \
5537 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5538 ▷ ▷ -define Control -poset_classification_control \
5539 ▷ ▷ ▷ -problem_label arcs_5.2.q11 \
5540 ▷ ▷ ▷ -W -depth 5 \
5541 ▷ ▷ -end \
5542 ▷ ▷ -with P -do \
5543 ▷ ▷ -projective_space_activity \
5544 ▷ ▷ ▷ -classify_arcs \
5545 ▷ ▷ ▷ ▷ -control Control \
5546 ▷ ▷ ▷ ▷ -target_size 5 \
5547 ▷ ▷ ▷ ▷ -d 2 \
5548 ▷ ▷ ▷ -end \
5549 ▷ ▷ -end
5550 ▷ #pdflatex arcs_5.2.q11.poset.tex
5551 ▷ #$(OPEN) arcs_5.2.q11.poset.pdf
5552
5553
5554 # 2 orbits:
5555 # 0 1 2 3 37
5556 # 0 1 2 3 49
5557
5558
5559 arcs_5.2.q11.ideal:
5560 ▷ $(ORBITER) -v 2 \
5561 ▷ ▷ -define F -finite_field -q 11 -end \
5562 ▷ ▷ -define R -polynomial_ring \
5563 ▷ ▷ ▷ -field F \
5564 ▷ ▷ ▷ -number_of_variables 3 \
5565 ▷ ▷ ▷ -homogeneous_of_degree 2 \
5566 ▷ ▷ ▷ -monomial_ordering_lex \
5567 ▷ ▷ ▷ -variables "x0,x1,x2" "x_0,x_1,x_2" \
5568 ▷ ▷ ▷ -end \
5569 ▷ ▷ -define C -combinatorial_object \
5570 ▷ ▷ ▷ -label arcs_5.2.q11.lvl.5 arcs\5\2\q11\lvl\5 \
5571 ▷ ▷ ▷ -file_of_points arcs_5.2.q11.lvl.5 \
5572 ▷ ▷ -end \
5573 ▷ ▷ -with C -do \
5574 ▷ ▷ -combinatorial_object_activity \
5575 ▷ ▷ ▷ -ideal R \
5576 ▷ ▷ -end
5577
5578 #( 0, 1, 2, 3, 37 )
5579 #generator 0 / 1 is 7*x0*x1 + 5*x0*x2 + 10*x1*x2
5580 #We found 12 points on the generator of the ideal
5581 #They are : ( 0, 1, 2, 3, 37, 54, 74, 80, 93, 105, 121, 128 )
5582
5583 #( 0, 1, 2, 3, 49 )
5584 #generator 0 / 1 is 4*x0*x1 + 8*x0*x2 + 10*x1*x2
5585 #looping over all generators of the ideal:
5586 #generator 0 / 1 is ( 0, 4, 8, 0, 10, 0 ) :
5587 #We found 12 points on the generator of the ideal
5588 #They are : ( 0, 1, 2, 3, 41, 49, 58, 77, 83, 95, 109, 130 )
5589

```

```

5590
5591
5592 surface_9lines_4E_ideal:
5593 ▷ $(ORBITER) -v 2 \
5594 ▷ ▷ -define Pts -vector -dense \
5595 ▷ ▷ ▷ $(PTS_OF_SURFACE_ORBIT211_Q3_L9_E4) \
5596 ▷ ▷ ▷ -end \
5597 ▷ ▷ -define F -finite_field -q 3 -end \
5598 ▷ ▷ -define R -polynomial_ring \
5599 ▷ ▷ ▷ -field F \
5600 ▷ ▷ ▷ -number_of_variables 4 \
5601 ▷ ▷ ▷ -homogeneous_of_degree 3 \
5602 ▷ ▷ ▷ -monomial_ordering_lex \
5603 ▷ ▷ ▷ -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
5604 ▷ ▷ ▷ -end \
5605 ▷ ▷ -with R -do \
5606 ▷ ▷ ▷ -ring_theoretic_activity \
5607 ▷ ▷ ▷ -ideal "surf_eqn" "surf\_eqn" Pts \
5608 ▷ ▷ ▷ -end
5609
5610
5611 #The ideal has dimension 2
5612 #generators for the ideal:
5613 #0 1 0 0 2 0 2 0 0 0 0 0 0 0 0 0 0 0
5614 #0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 2 0
5615 #x0*x0*x1 + 2*x0*x1*x1 + 2*x0*x1*x3
5616 #2*x2*x2*x3 + 2*x2*x3*x3
5617
5618
5619
5620 F_9_q7:
5621 ▷ $(ORBITER) -v 10 \
5622 ▷ ▷ -define F -finite_field -q 7 -end \
5623 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5624 ▷ ▷ -define R -polynomial_ring \
5625 ▷ ▷ ▷ -field F \
5626 ▷ ▷ ▷ -number_of_variables 4 \
5627 ▷ ▷ ▷ -homogeneous_of_degree 3 \
5628 ▷ ▷ ▷ -monomial_ordering_partition \
5629 ▷ ▷ ▷ -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
5630 ▷ ▷ ▷ -end \
5631 ▷ ▷ -define F_9 -cubic_surface -space P \
5632 ▷ ▷ ▷ -by_equation R "F_9" \
5633 ▷ ▷ ▷ "\DF_9\D" "x0,x1,x2,x3" \
5634 ▷ ▷ ▷ $(SURFACE_F_9) \
5635 ▷ ▷ ▷ "" \
5636 ▷ ▷ ▷ "\Dno parameters\D" "" \
5637 ▷ ▷ -end \
5638 ▷ ▷ -with F_9 -do \
5639 ▷ ▷ -cubic_surface_activity \
5640 ▷ ▷ ▷ -report \
5641 ▷ ▷ -end
5642 ▷ pdflatex surface.equation_F_9.q7.report.tex
5643 ▷ $(OPEN) surface.equation_F_9.q7.report.pdf
5644
5645
5646
5647
5648 # we create 20 5-subsets of PG(2,11) at random. Note that PG(2,11) has 133 points

```

```

5649 .
5650 random_k_subsets_PG_2_11:
5651 ▷ $(ORBITER) -v 4 \
5652 ▷ ▷ -create_random_k_subsets 133 5 20
5653
5654
5655 #random_k_subsets_n133_k5_nb20.csv
5656
5657
5658 # toDo
5659
5660 line_type_in_PG_2_11:
5661 ▷ $(ORBITER) -v 3 \
5662 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
5663 ▷ ▷ -define F -finite_field -q 11 -end \
5664 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5665 ▷ ▷ -define C -combinatorial_object \
5666 ▷ ▷ ▷ -label random_sets random_sets \
5667 ▷ ▷ ▷ -file_of_points random_k_subsets_n133_k5_nb20.csv \
5668 ▷ ▷ -end \
5669 ▷ ▷ -with C -do \
5670 ▷ ▷ -combinatorial_object_activity \
5671 ▷ ▷ ▷ line_type_old \
5672 ▷ ▷ -end
5673
5674
5675 # the second one is an arc: 3,33,40,83,102
5676
5677 # we compute the ideal:
5678
5679
5680 random_arc_5_2_q11_ideal:
5681 ▷ $(ORBITER) -v 2 \
5682 ▷ ▷ -define F -finite_field -q 11 -end \
5683 ▷ ▷ -define R -polynomial_ring \
5684 ▷ ▷ ▷ -field F \
5685 ▷ ▷ ▷ -number_of_variables 3 \
5686 ▷ ▷ ▷ -homogeneous_of_degree 2 \
5687 ▷ ▷ ▷ -monomial_ordering_lex \
5688 ▷ ▷ ▷ -variables "x0,x1,x2" "x_0,x_1,x_2" \
5689 ▷ ▷ ▷ -end \
5690 ▷ ▷ -define C -combinatorial_object \
5691 ▷ ▷ ▷ -label random_arc random\_arc \
5692 ▷ ▷ ▷ -set_of_points "3,33,40,83,102" \
5693 ▷ ▷ -end \
5694 ▷ ▷ -with C -do \
5695 ▷ ▷ -combinatorial_object_activity \
5696 ▷ ▷ ▷ -ideal R \
5697 ▷ ▷ -end
5698
5699 #generator 0 / 1 is 10*x0*x0 + 3*x0*x1 + 8*x0*x2 + 2*x1*x1 + 10*x2*x2
5700 #We found 12 points on the generator of the ideal
5701 #They are : ( 3, 15, 19, 33, 40, 42, 46, 50, 83, 88, 102, 108 )
5702
5703
5704
5705
5706 Endrass.F7.txt:

```

```

5707 ▷ $(ORBITER) -v 2 \
5708 ▷ ▷ -define F -finite_field -q 7 -end \
5709 ▷ ▷ -define R -polynomial_ring -field F \
5710 ▷ ▷ ▷ -number_of_variables 4 \
5711 ▷ ▷ ▷ -homogeneous_of_degree 8 \
5712 ▷ ▷ ▷ -end \
5713 ▷ ▷ -define eqn -vector -field F -sparse 165 \
5714 ▷ ▷ ▷ $(ENDRASS_SPARSE) -end \
5715 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5716 ▷ ▷ -define Endrass_F7 -geometric_object P \
5717 ▷ ▷ ▷ -projective_variety R \
5718 ▷ ▷ ▷ ▷ "Endrass_F7" \
5719 ▷ ▷ ▷ ▷ "Endrass\F7" \
5720 ▷ ▷ ▷ ▷ eqn \
5721 ▷ ▷ -end \
5722 ▷ ▷ -with Endrass_F7 -do \
5723 ▷ ▷ -combinatorial_object_activity -save \
5724 ▷ ▷ -end
5725
5726
5727
5728 # we created a set of 33 points, called Endrass_F7.txt
5729
5730
5731
5732 octic_prepare:
5733 ▷ $(ORBITER) -v 4 \
5734 ▷ ▷ -define A -vector -format 1 -dense "1,1,1,1" -end \
5735 ▷ ▷ -define D -diophant \
5736 ▷ ▷ ▷ -label octic_monomials \
5737 ▷ ▷ ▷ -coefficient_matrix A \
5738 ▷ ▷ ▷ -RHS "mult=1,EQ=8" \
5739 ▷ ▷ ▷ -x_min_global 0 -x_max_global 8 \
5740 ▷ ▷ -end \
5741 ▷ ▷ -with D -do \
5742 ▷ ▷ ▷ -diophant_activity -solve_mckay \
5743 ▷ ▷ -end
5744 ▷ sort -r octic_monomials_sol.csv >octic_monomials_sorted.txt
5745
5746 #Found 165 solutions with 210 backtrack steps
5747 # 165=binomial(11,3)
5748
5749
5750
5751 affine_map_F5_0:
5752 ▷ $(ORBITER) -v 8 \
5753 ▷ ▷ -define F -finite_field -q 5 -end \
5754 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5755 ▷ ▷ -define R -polynomial_ring \
5756 ▷ ▷ ▷ -field F \
5757 ▷ ▷ ▷ -number_of_variables 2 \
5758 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5759 ▷ ▷ ▷ -monomial_ordering_lex \
5760 ▷ ▷ ▷ -variables "x,y" "x,y" \
5761 ▷ ▷ ▷ -end \
5762 ▷ ▷ -define Y0 -symbolic_object \
5763 ▷ ▷ ▷ -field F \
5764 ▷ ▷ ▷ -text "1-x^4" \
5765 ▷ ▷ ▷ -end \

```

```

5766 ▷ ▷ -define Y -symbolic.object \
5767 ▷ ▷ ▷ -field F \
5768 ▷ ▷ ▷ -text "Y0" \
5769 ▷ ▷ ▷ -end \
5770 ▷ ▷ -with P -do \
5771 ▷ ▷ -projective_space_activity \
5772 ▷ ▷ ▷ -affine_map R Y "" \
5773 ▷ ▷ -end
5774
5775 #( 1, 0, 0, 0, 0 )
5776
5777 affine_map_F5.1:
5778 ▷ $(ORBITER) -v 6 \
5779 ▷ ▷ -define F -finite_field -q 5 -end \
5780 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5781 ▷ ▷ -define R -polynomial_ring \
5782 ▷ ▷ ▷ -field F \
5783 ▷ ▷ ▷ -number_of_variables 2 \
5784 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5785 ▷ ▷ ▷ -monomial_ordering_lex \
5786 ▷ ▷ ▷ -variables "x,y" "x,y" \
5787 ▷ ▷ ▷ -end \
5788 ▷ ▷ -define Y0 -symbolic.object \
5789 ▷ ▷ ▷ -field F \
5790 ▷ ▷ ▷ -text "1-(x-1)^4" \
5791 ▷ ▷ ▷ -end \
5792 ▷ ▷ -define Y -symbolic.object \
5793 ▷ ▷ ▷ -field F \
5794 ▷ ▷ ▷ -text "Y0" \
5795 ▷ ▷ ▷ -end \
5796 ▷ ▷ -with P -do \
5797 ▷ ▷ -projective_space_activity \
5798 ▷ ▷ ▷ -affine_map R Y "" \
5799 ▷ ▷ -end
5800
5801 #( 0, 1, 0, 0, 0 )
5802
5803 affine_map_F5.2:
5804 ▷ $(ORBITER) -v 6 \
5805 ▷ ▷ -define F -finite_field -q 5 -end \
5806 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5807 ▷ ▷ -define R -polynomial_ring \
5808 ▷ ▷ ▷ -field F \
5809 ▷ ▷ ▷ -number_of_variables 2 \
5810 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5811 ▷ ▷ ▷ -monomial_ordering_lex \
5812 ▷ ▷ ▷ -variables "x,y" "x,y" \
5813 ▷ ▷ ▷ -end \
5814 ▷ ▷ -define Y0 -symbolic.object \
5815 ▷ ▷ ▷ -field F \
5816 ▷ ▷ ▷ -text "1-(x-2)^4" \
5817 ▷ ▷ ▷ -end \
5818 ▷ ▷ -define Y -symbolic.object \
5819 ▷ ▷ ▷ -field F \
5820 ▷ ▷ ▷ -text "Y0" \
5821 ▷ ▷ ▷ -end \
5822 ▷ ▷ -with P -do \
5823 ▷ ▷ -projective_space_activity \
5824 ▷ ▷ ▷ -affine_map R Y "" \

```

```

5825 ▷ ▷ -end
5826
5827 #( 0, 0, 1, 0, 0 )
5828
5829 affine_map_F5.3:
5830 ▷ $(ORBITER) -v 6 \
5831 ▷ ▷ -define F -finite_field -q 5 -end \
5832 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5833 ▷ ▷ -define R -polynomial_ring \
5834 ▷ ▷ ▷ -field F \
5835 ▷ ▷ ▷ -number_of_variables 2 \
5836 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5837 ▷ ▷ ▷ -monomial_ordering_lex \
5838 ▷ ▷ ▷ -variables "x,y" "x,y" \
5839 ▷ ▷ ▷ -end \
5840 ▷ ▷ -define Y0 -symbolic_object \
5841 ▷ ▷ ▷ -field F \
5842 ▷ ▷ ▷ -text "1-(x-3)^4" \
5843 ▷ ▷ ▷ -end \
5844 ▷ ▷ -define Y -symbolic_object \
5845 ▷ ▷ ▷ -field F \
5846 ▷ ▷ ▷ -text "Y0" \
5847 ▷ ▷ ▷ -end \
5848 ▷ ▷ -with P -do \
5849 ▷ ▷ -projective_space_activity \
5850 ▷ ▷ ▷ -affine_map R Y "" \
5851 ▷ ▷ -end
5852
5853
5854 #( 0, 0, 0, 1, 0 )
5855
5856
5857
5858 affine_map_F5.4:
5859 ▷ $(ORBITER) -v 6 \
5860 ▷ ▷ -define F -finite_field -q 5 -end \
5861 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5862 ▷ ▷ -define R -polynomial_ring \
5863 ▷ ▷ ▷ -field F \
5864 ▷ ▷ ▷ -number_of_variables 2 \
5865 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5866 ▷ ▷ ▷ -monomial_ordering_lex \
5867 ▷ ▷ ▷ -variables "x,y" "x,y" \
5868 ▷ ▷ ▷ -end \
5869 ▷ ▷ -define Y0 -symbolic_object \
5870 ▷ ▷ ▷ -field F \
5871 ▷ ▷ ▷ -text "1-(x-4)^4" \
5872 ▷ ▷ ▷ -end \
5873 ▷ ▷ -define Y -symbolic_object \
5874 ▷ ▷ ▷ -field F \
5875 ▷ ▷ ▷ -text "Y0" \
5876 ▷ ▷ ▷ -end \
5877 ▷ ▷ -with P -do \
5878 ▷ ▷ -projective_space_activity \
5879 ▷ ▷ ▷ -affine_map R Y "" \
5880 ▷ ▷ -end
5881
5882
5883 #( 0, 0, 0, 0, 1 )

```

```

5884
5885
5886
5887 permutation_polynomial_F5:
5888 ▷ $(ORBITER) -v 6 \
5889 ▷ ▷ -define F -finite_field -q 5 -end \
5890 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5891 ▷ ▷ -define R -polynomial_ring \
5892 ▷ ▷ ▷ -field F \
5893 ▷ ▷ ▷ -number_of_variables 2 \
5894 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5895 ▷ ▷ ▷ -monomial_ordering_lex \
5896 ▷ ▷ ▷ -variables "x,y" "x,y" \
5897 ▷ ▷ ▷ -end \
5898 ▷ ▷ -define Y0 -symbolic_object \
5899 ▷ ▷ ▷ -field F \
5900 ▷ ▷ ▷ -text "3*(1-x^4)+4*(1-(x-2)^4)+1*(1-(x-3)^4)+2*(1-(x-4)^4)" \
5901 ▷ ▷ ▷ -end \
5902 ▷ ▷ -define Y -symbolic_object \
5903 ▷ ▷ ▷ -field F \
5904 ▷ ▷ ▷ -text "Y0" \
5905 ▷ ▷ ▷ -end \
5906 ▷ ▷ -with P -do \
5907 ▷ ▷ -projective_space_activity \
5908 ▷ ▷ ▷ -affine_map R Y "" \
5909 ▷ ▷ -end \
5910 ▷ ▷ -define Yx -symbolic_object \
5911 ▷ ▷ ▷ -field F \
5912 ▷ ▷ ▷ -expand Y \
5913 ▷ ▷ ▷ -end \
5914
5915
5916
5917
5918
5919
5920 #####
5921 # Section 5.3: Algebraic Geometry
5922
5923 test_5.3:
5924 ▷ make Veronese_1.3.q7
5925 ▷ make variety_hermitian_curve
5926 ▷ make variety_hermitian_surface
5927
5928 Veronese_1.3.q7:
5929 ▷ $(ORBITER) -v 5 \
5930 ▷ ▷ -define Fq -finite_field -q 7 -end \
5931 ▷ ▷ -define C -symbolic_object \
5932 ▷ ▷ ▷ -field Fq \
5933 ▷ ▷ ▷ -managed_variables "y0,y1" \
5934 ▷ ▷ ▷ -text "y1^3,y0*y1^2,y0^2*y1,y0^3" \
5935 ▷ ▷ -end \
5936 ▷ ▷ -define P1 -projective_space -n 1 -field Fq -v 0 -end \
5937 ▷ ▷ -define P3 -projective_space -n 3 -field Fq -v 0 -end \
5938 ▷ ▷ -define R2 -polynomial_ring \
5939 ▷ ▷ ▷ -field Fq \
5940 ▷ ▷ ▷ -number_of_variables 2 \
5941 ▷ ▷ ▷ -homogeneous_of_degree 3 \
5942 ▷ ▷ ▷ -monomial_ordering_partition \

```

```

5943 ▷ ▷ ▷ -variables "y0,y1" "y_0,y_1" \
5944 ▷ ▷ ▷ -end \
5945 ▷ ▷ -define map -mapping \
5946 ▷ ▷ ▷ -domain P1 \
5947 ▷ ▷ ▷ -codomain P3 \
5948 ▷ ▷ ▷ -ring R2 \
5949 ▷ ▷ ▷ -formula C \
5950 ▷ ▷ ▷ -substitute "" \
5951 ▷ ▷ -end
5952
5953
5954
5955 variety_hermitian_curve:
5956 ▷ echo $(FILE_HERMITIAN_CURVE) >input.csv
5957 ▷ $(ORBITER) -v 8 \
5958 ▷ ▷ -define F -finite_field -q 9 -end \
5959 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5960 ▷ ▷ -define R -polynomial_ring \
5961 ▷ ▷ ▷ -field F \
5962 ▷ ▷ ▷ -number_of_variables 3 \
5963 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5964 ▷ ▷ ▷ -monomial_ordering_partition \
5965 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
5966 ▷ ▷ -end \
5967 ▷ ▷ -define O -orbits -classification_by_canonical_form \
5968 ▷ ▷ ▷ -space P \
5969 ▷ ▷ ▷ -ring R \
5970 ▷ ▷ ▷ -input_fname_mask input.csv \
5971 ▷ ▷ ▷ -nb_files 1 \
5972 ▷ ▷ ▷ -output_fname hermitian_curve_q9 \
5973 ▷ ▷ ▷ -label_equation "equation" \
5974 ▷ ▷ ▷ -algorithm_nauty \
5975 ▷ ▷ ▷ -end \
5976 ▷ ▷ -end \
5977 ▷ -with O -do -orbits_activity \
5978 ▷ ▷ -report \
5979 ▷ ▷ -report_options \
5980 ▷ ▷ ▷ -fname hermitian_curve_q9 \
5981 ▷ ▷ -end \
5982 ▷ -end
5983 ▷ pdflatex hermitian_curve_q9_orbits.tex
5984 ▷ $(OPEN) hermitian_curve_q9_orbits.pdf
5985
5986
5987
5988 variety_hermitian_surface:
5989 ▷ echo $(FILE_HERMITIAN_SURFACE) >input.csv
5990 ▷ $(ORBITER) -v 8 \
5991 ▷ ▷ -define F -finite_field -q 9 -end \
5992 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5993 ▷ ▷ -define R -polynomial_ring \
5994 ▷ ▷ ▷ -field F \
5995 ▷ ▷ ▷ -number_of_variables 4 \
5996 ▷ ▷ ▷ -homogeneous_of_degree 4 \
5997 ▷ ▷ ▷ -monomial_ordering_partition \
5998 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
5999 ▷ ▷ -end \
6000 ▷ ▷ -define O -orbits -classification_by_canonical_form \
6001 ▷ ▷ ▷ -space P \

```



```

6002 ▷ ▷ ▷ -ring R \
6003 ▷ ▷ ▷ -input_fname_mask input.csv \
6004 ▷ ▷ ▷ -nb_files 1 \
6005 ▷ ▷ ▷ -output_fname hermitian_surface_q9 \
6006 ▷ ▷ ▷ -label.equation "equation" \
6007 ▷ ▷ ▷ -algorithm.nauty \
6008 ▷ ▷ ▷ -end \
6009 ▷ ▷ -end \
6010 ▷ -with 0 -do -orbits.activity \
6011 ▷ ▷ -report \
6012 ▷ ▷ -report_options \
6013 ▷ ▷ ▷ -fname hermitian_surface_q9 \
6014 ▷ ▷ -end \
6015 ▷ -end
6016 ▷ pdflatex hermitian_surface_q9_orbits.tex
6017 ▷ $(OPEN) hermitian_surface_q9_orbits.pdf
6018
6019
6020
6021
6022
6023
6024
6025 #####
6026 # Chapter 6 - Group Theory
6027 #####
6028
6029
6030 test_6:
6031 ▷ make test_6.1
6032 ▷ make test_6.2
6033 ▷ make test_6.3
6034 ▷ make test_6.4
6035 ▷ make test_6.5
6036 ▷ make test_6.6
6037 ▷ #make test_6.7 # magma stuff
6038 ▷ make test_6.8
6039
6040
6041 #####
6042 # Section 6.1: Permutation groups
6043
6044
6045 SECTION_PERMUTATION_GROUPS:
6046
6047 test_6.1:
6048 ▷ make Cyclic_6
6049 ▷ make Cyclic_6_group_table
6050 ▷ make Symmetric_3
6051 ▷ make Symmetric_3_group_table
6052 ▷ make Symmetric_3_elements
6053 ▷ make Symmetric_3_long
6054 ▷ make Symmetric_4
6055 ▷ make Symmetric_4_group_table
6056 ▷ make Symmetric_4_long
6057 ▷ make Group_8.1
6058 ▷ make Group_8.2
6059 ▷ make Group_8.3
6060 ▷ make Group_8.4

```

```

6061 ▷ make Group_8.5
6062 ▷ make Group_M12
6063
6064
6065 Cyclic_6:
6066 ▷ $(ORBITER) -v 3 \
6067 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
6068 ▷ ▷ -with G -do \
6069 ▷ ▷ ▷ -group_theoretic_activity \
6070 ▷ ▷ ▷ -report \
6071 ▷ ▷ -end
6072 ▷ pdflatex C_6.report.tex
6073 ▷ $(OPEN) C_6.report.pdf
6074
6075
6076 Cyclic_6_group_table:
6077 ▷ $(ORBITER) -v 3 \
6078 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
6079 ▷ ▷ -with G -do \
6080 ▷ ▷ ▷ -group_theoretic_activity \
6081 ▷ ▷ ▷ -export_group_table \
6082 ▷ ▷ -end
6083 ▷ $(ORBITER) -v 2 \
6084 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
6085 ▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
6086 ▷ ▷ -draw_matrix \
6087 ▷ ▷ ▷ -input_csv_file C_6.group_table.csv \
6088 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6089 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6090 ▷ ▷ -end
6091 ▷ $(OPEN) C_6.group_table_draw.bmp
6092
6093
6094 Symmetric_3:
6095 ▷ $(ORBITER) -v 3 \
6096 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
6097 ▷ ▷ -with G -do \
6098 ▷ ▷ ▷ -group_theoretic_activity \
6099 ▷ ▷ ▷ -report \
6100 ▷ ▷ -end
6101 ▷ pdflatex Sym_3.report.tex
6102 ▷ $(OPEN) Sym_3.report.pdf
6103
6104
6105 Symmetric_3_group_table:
6106 ▷ $(ORBITER) -v 3 \
6107 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
6108 ▷ ▷ -with G -do \
6109 ▷ ▷ ▷ -group_theoretic_activity \
6110 ▷ ▷ ▷ -export_group_table \
6111 ▷ ▷ -end
6112 ▷ $(ORBITER) -v 2 \
6113 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
6114 ▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
6115 ▷ ▷ -draw_matrix \
6116 ▷ ▷ ▷ -input_csv_file Sym_3.group_table.csv \
6117 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6118 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6119 ▷ ▷ -end

```

```

6120 ▷ $(OPEN) Sym_3_group_table.draw.bmp
6121
6122 Symmetric_3_elements:
6123 ▷ $(ORBITER) -v 3 \
6124 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
6125 ▷ ▷ -with G -do \
6126 ▷ ▷ -group_theoretic_activity \
6127 ▷ ▷ ▷ -save_elements_csv "Symmetric3_elts.csv" \
6128 ▷ ▷ -end
6129 ▷ $(ORBITER) -v 2 \
6130 ▷ ▷ -define Sym3_elts -vector -load_csv_data_column \
6131 ▷ ▷ ▷ Symmetric3_elts.csv 1 -end \
6132 ▷ ▷ -save_matrix_csv Sym3_elts
6133 ▷ $(ORBITER) -v 2 \
6134 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
6135 ▷ ▷ -define all_one_c -vector -repeat 1 3 -end \
6136 ▷ ▷ -draw_matrix \
6137 ▷ ▷ ▷ -input_csv_file Sym3_elts_matrix.csv \
6138 ▷ ▷ ▷ -box_width 50 -bit_depth 8 \
6139 ▷ ▷ ▷ -partition 3 \
6140 ▷ ▷ ▷ all_one_r all_one_c \
6141 ▷ ▷ -end
6142 ▷ $(OPEN) Sym3_elts_matrix.draw.bmp
6143
6144 Symmetric_3_long:
6145 ▷ $(ORBITER) -v 3 \
6146 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
6147 ▷ ▷ -with G -do \
6148 ▷ ▷ -group_theoretic_activity \
6149 ▷ ▷ ▷ -export_orbiter \
6150 ▷ ▷ -end \
6151 ▷ ▷ -with G -do \
6152 ▷ ▷ -group_theoretic_activity \
6153 ▷ ▷ ▷ -print_elements_tex \
6154 ▷ ▷ -end \
6155 ▷ ▷ -with G -do \
6156 ▷ ▷ -group_theoretic_activity \
6157 ▷ ▷ ▷ -report \
6158 ▷ ▷ -end \
6159 ▷ ▷ -with G -do \
6160 ▷ ▷ -group_theoretic_activity \
6161 ▷ ▷ ▷ -save_elements_csv "Symmetric3_elts.csv" \
6162 ▷ ▷ -end
6163 ▷ $(ORBITER) -v 3 \
6164 ▷ ▷ -draw_options \
6165 ▷ ▷ ▷ -nodes \
6166 ▷ ▷ ▷ -embedded -radius 250 \
6167 ▷ ▷ ▷ -xin 10000 -yin 10000 \
6168 ▷ ▷ ▷ -xout 1000000 -yout 600000 \
6169 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
6170 ▷ ▷ -end \
6171 ▷ ▷ -tree_draw -file Sym_3_elements_tree.txt -end
6172 ▷ $(ORBITER) -v 2 \
6173 ▷ ▷ -define M -vector -load_csv_data_column \
6174 ▷ ▷ ▷ Symmetric3_elts.csv 1 -end \
6175 ▷ ▷ -save_matrix_csv M
6176 ▷ $(ORBITER) -v 2 \
6177 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
6178 ▷ ▷ -define all_one_c -vector -repeat 1 3 -end \

```

```

6179 ▷ ▷ -draw_matrix \
6180 ▷ ▷ ▷ -input_csv_file M_matrix.csv \
6181 ▷ ▷ ▷ -box_width 50 -bit_depth 8 \
6182 ▷ ▷ ▷ -partition 3 \
6183 ▷ ▷ ▷ ▷ all_one_r all_one_c \
6184 ▷ ▷ -end
6185 ▷ pdflatex Sym_3_elements_tree_draw.tex
6186 ▷ $(OPEN) Sym_3_elements_tree_draw.pdf
6187 ▷ #pdflatex Perm3_report.tex
6188 ▷ #$(OPEN) Perm3_report.pdf
6189
6190
6191 Symmetric_4:
6192 ▷ $(ORBITER) -v 3 \
6193 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
6194 ▷ ▷ -with G -do \
6195 ▷ ▷ -group_theoretic_activity \
6196 ▷ ▷ ▷ -report \
6197 ▷ ▷ -end
6198 ▷ pdflatex Sym_4_report.tex
6199 ▷ $(OPEN) Sym_4_report.pdf
6200
6201
6202 Symmetric_4_group_table:
6203 ▷ $(ORBITER) -v 3 \
6204 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
6205 ▷ ▷ -with G -do \
6206 ▷ ▷ -group_theoretic_activity \
6207 ▷ ▷ ▷ -export_group_table \
6208 ▷ ▷ -end
6209 ▷ $(ORBITER) -v 2 \
6210 ▷ ▷ -define all_one_r -vector -repeat 1 24 -end \
6211 ▷ ▷ -define all_one_c -vector -repeat 1 24 -end \
6212 ▷ ▷ -draw_matrix \
6213 ▷ ▷ ▷ -input_csv_file Sym_4_group_table.csv \
6214 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6215 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6216 ▷ ▷ -end
6217 ▷ $(OPEN) Sym_4_group_table_draw.bmp
6218
6219
6220
6221 Symmetric_4_long:
6222 ▷ $(ORBITER) -v 3 \
6223 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
6224 ▷ ▷ -with G -do \
6225 ▷ ▷ -group_theoretic_activity \
6226 ▷ ▷ ▷ -export_orbiter \
6227 ▷ ▷ -end \
6228 ▷ ▷ -with G -do \
6229 ▷ ▷ -group_theoretic_activity \
6230 ▷ ▷ ▷ -export_group_table \
6231 ▷ ▷ -end \
6232 ▷ ▷ -with G -do \
6233 ▷ ▷ -group_theoretic_activity \
6234 ▷ ▷ ▷ -print_elements_tex \
6235 ▷ ▷ -end \
6236 ▷ ▷ -with G -do \
6237 ▷ ▷ -group_theoretic_activity \

```

```

6238 ▷ ▷ ▷ -report \
6239 ▷ ▷ -end \
6240 ▷ ▷ -with G -do \
6241 ▷ ▷ -group_theoretic_activity \
6242 ▷ ▷ ▷ -save_elements_csv "Sym_4_elts.csv" \
6243 ▷ ▷ -end \
6244 ▷ ▷ -with G -do \
6245 ▷ ▷ -group_theoretic_activity \
6246 ▷ ▷ ▷ -export_inversion_graphs "Sym_4_inversion_graphs.csv" \
6247 ▷ ▷ -end
6248 ▷ $(ORBITER) -v 2 \
6249 ▷ ▷ -draw_options \
6250 ▷ ▷ ▷ -nodes \
6251 ▷ ▷ ▷ -embedded -radius 175 \
6252 ▷ ▷ ▷ -xin 10000 -yin 10000 \
6253 ▷ ▷ ▷ -xout 1500000 -yout 600000 \
6254 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
6255 ▷ ▷ -end \
6256 ▷ ▷ -tree_draw -file Sym_4_elements_tree.txt -end
6257 ▷ $(ORBITER) -v 2 -draw_matrix \
6258 ▷ ▷ -input_csv_file Sym_4_group_table.csv \
6259 ▷ ▷ -box_width 50 -bit_depth 24 -end
6260 ▷ $(ORBITER) -v 2 \
6261 ▷ ▷ -define M -vector -load_csv_data_column \
6262 ▷ ▷ ▷ Sym_4_elts.csv 1 -end \
6263 ▷ ▷ -save_matrix_csv M
6264 ▷ $(ORBITER) -v 2 \
6265 ▷ ▷ -define all_one_r -vector -repeat 1 24 -end \
6266 ▷ ▷ -define all_one_c -vector -repeat 1 4 -end \
6267 ▷ ▷ -draw_matrix \
6268 ▷ ▷ ▷ -input_csv_file M_matrix.csv \
6269 ▷ ▷ ▷ -box_width 50 -bit_depth 8 \
6270 ▷ ▷ ▷ -partition 3 \
6271 ▷ ▷ ▷ ▷ all_one_r all_one_c \
6272 ▷ ▷ -end
6273 ▷ pdflatex Sym_4_elements_tree_draw.tex
6274 ▷ #$(OPEN) Sym_4_elements_tree_draw.pdf
6275
6276
6277
6278 # C8:
6279
6280 Group_8_1:
6281 ▷ $(ORBITER) -v 2 \
6282 ▷ ▷ -define gens -vector -dense \
6283 ▷ ▷ ▷ "1,2,3,4,5,6,7,0" \
6284 ▷ ▷ -end \
6285 ▷ ▷ -define G -permutation_group -symmetric_group 8 \
6286 ▷ ▷ ▷ -subgroup_by_generators G1 8 1 gens -end \
6287 ▷ ▷ -with G -do \
6288 ▷ ▷ -group_theoretic_activity \
6289 ▷ ▷ ▷ -export_group_table \
6290 ▷ ▷ -end
6291 ▷ $(ORBITER) -v 2 \
6292 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
6293 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
6294 ▷ ▷ -draw_matrix \
6295 ▷ ▷ ▷ -input_csv_file \
6296 ▷ ▷ ▷ ▷ Sym_8_Subgroup_G1_8_group_table.csv \

```

```

6297 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6298 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6299 ▷ ▷ -end
6300 ▷ convert Sym_8_Subgroup_G1_8_group_table_draw.bmp \
6301 ▷ ▷ Sym_8_Subgroup_G1_8_group_table_draw.png
6302 ▷ $(OPEN) Sym_8_Subgroup_G1_8_group_table_draw.png
6303
6304
6305
6306 # C4C2:
6307
6308 Group_8.2:
6309 ▷ $(ORBITER) -v 2 \
6310 ▷ ▷ -define gens -vector -dense \
6311 ▷ ▷ ▷ "1,2,3,0,4,5, 0,1,2,3,5,4" \
6312 ▷ ▷ -end \
6313 ▷ ▷ -define G -permutation_group -symmetric_group 6 \
6314 ▷ ▷ ▷ -subgroup_by_generators G2 8 2 gens -end \
6315 ▷ ▷ -with G -do \
6316 ▷ ▷ -group_theoretic_activity \
6317 ▷ ▷ ▷ -export_group_table \
6318 ▷ ▷ -end
6319 ▷ $(ORBITER) -v 2 \
6320 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
6321 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
6322 ▷ ▷ -draw_matrix \
6323 ▷ ▷ ▷ -input_csv_file \
6324 ▷ ▷ ▷ ▷ Sym_6_Subgroup_G2_8_group_table.csv \
6325 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6326 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6327 ▷ ▷ -end
6328 ▷ convert Sym_6_Subgroup_G2_8_group_table_draw.bmp \
6329 ▷ ▷ Sym_6_Subgroup_G2_8_group_table_draw.png
6330 ▷ $(OPEN) Sym_6_Subgroup_G2_8_group_table_draw.png
6331
6332
6333 # Group C2C2C2:
6334
6335 Group_8.3:
6336 ▷ $(ORBITER) -v 2 \
6337 ▷ ▷ -define gens -vector -dense \
6338 ▷ ▷ ▷ "1,0,2,3,4,5, 0,1,3,2,4,5, 0,1,2,3,5,4" \
6339 ▷ ▷ -end \
6340 ▷ ▷ -define G -permutation_group -symmetric_group 6 \
6341 ▷ ▷ ▷ -subgroup_by_generators G3 8 3 gens -end \
6342 ▷ ▷ -with G -do \
6343 ▷ ▷ -group_theoretic_activity \
6344 ▷ ▷ ▷ -export_group_table \
6345 ▷ ▷ -end
6346 ▷ $(ORBITER) -v 2 \
6347 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
6348 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
6349 ▷ ▷ -draw_matrix \
6350 ▷ ▷ ▷ -input_csv_file \
6351 ▷ ▷ ▷ ▷ Sym_6_Subgroup_G3_8_group_table.csv \
6352 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6353 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6354 ▷ ▷ -end
6355 ▷ convert Sym_6_Subgroup_G3_8_group_table_draw.bmp \

```

```

6356 ▷ ▷ Sym_6_Subgroup.G3.8.group.table.draw.png
6357 ▷ $(OPEN) Sym_6_Subgroup.G3.8.group.table.draw.png
6358
6359
6360 # Dihedral group D4 of order 8:
6361
6362 Group_8_4:
6363 ▷ $(ORBITER) -v 2 \
6364 ▷ ▷ -define gens -vector -dense \
6365 ▷ ▷ ▷ "1,2,3,0, 0,3,2,1" \
6366 ▷ ▷ -end \
6367 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
6368 ▷ ▷ ▷ -subgroup_by_generators G4 8 2 gens -end \
6369 ▷ ▷ -with G -do \
6370 ▷ ▷ -group_theoretic_activity \
6371 ▷ ▷ ▷ -export_group_table \
6372 ▷ ▷ -end
6373 ▷ $(ORBITER) -v 2 \
6374 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
6375 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
6376 ▷ ▷ -draw_matrix \
6377 ▷ ▷ ▷ -input_csv_file \
6378 ▷ ▷ ▷ ▷ Sym_4_Subgroup.G4.8.group.table.csv \
6379 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6380 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6381 ▷ ▷ -end
6382 ▷ convert Sym_4_Subgroup.G4.8.group.table.draw.bmp \
6383 ▷ ▷ Sym_4_Subgroup.G4.8.group.table.draw.png
6384 ▷ $(OPEN) Sym_4_Subgroup.G4.8.group.table.draw.png
6385
6386 # Quaternion group:
6387
6388 Group_8_5:
6389 ▷ $(ORBITER) -v 2 \
6390 ▷ ▷ -define gens -vector -dense \
6391 ▷ ▷ ▷ "2,3,1,0,6,7,5,4, 4,5,7,6,1,0,2,3" \
6392 ▷ ▷ -end \
6393 ▷ ▷ -define G -permutation_group -symmetric_group 8 \
6394 ▷ ▷ ▷ -subgroup_by_generators G5 8 2 gens -end \
6395 ▷ ▷ -with G -do \
6396 ▷ ▷ -group_theoretic_activity \
6397 ▷ ▷ ▷ -export_group_table \
6398 ▷ ▷ -end
6399 ▷ $(ORBITER) -v 2 \
6400 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
6401 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
6402 ▷ ▷ -draw_matrix \
6403 ▷ ▷ ▷ -input_csv_file \
6404 ▷ ▷ ▷ ▷ Sym_8_Subgroup.G5.8.group.table.csv \
6405 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
6406 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6407 ▷ ▷ -end
6408 ▷ convert Sym_8_Subgroup.G5.8.group.table.draw.bmp \
6409 ▷ ▷ Sym_8_Subgroup.G5.8.group.table.draw.png
6410 ▷ $(OPEN) Sym_8_Subgroup.G5.8.group.table.draw.png
6411
6412
6413
6414 # M12 of order 95040:

```

```

6415 #
6416 #(1,4)(3,10)(5,11)(6,12)
6417 #(1,8,9)(2,3,4)(5,12,11)(6,10,7)
6418 #[3,1,9,0,10,11,6,7,8,2,4,5]
6419 #[7,2,3,1,11,9,5,8,0,6,4,10]
6420
6421
6422
6423
6424 Group_M12:
6425 ▷ $(ORBITER) -v 2 \
6426 ▷ ▷ -define gens -vector -dense $(GENERATORS.M12) -end \
6427 ▷ ▷ -define G -permutation_group -symmetric_group 12 \
6428 ▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \
6429 ▷ ▷ -with G -do \
6430 ▷ ▷ -group_theoretic_activity \
6431 ▷ ▷ ▷ -report \
6432 ▷ ▷ -end
6433 ▷ pdflatex Sym_12.Subgroup_M12.95040_report.tex
6434 ▷ $(OPEN) Sym_12.Subgroup_M12.95040_report.pdf
6435
6436
6437
6438 #####
6439 # Section 6.2: Linear Groups
6440
6441
6442 SECTION.LINEAR.GROUPS:
6443
6444 test_6.2:
6445 ▷ make PGL_3.2
6446 ▷ make PGL_4.2
6447 ▷ make PGL_4.2_multiply_all_elements
6448 ▷ make PGL_4.2_export
6449 ▷ make PGL_4.2_export_GAP
6450 ▷ make PGL_4.2_generated
6451 ▷ make PGL_5.2
6452 ▷ #make PGL_5.2_multiply_all_elements too slow
6453 ▷ make L_5.3
6454 ▷ make L_4.5
6455 ▷ make PGL_4.5
6456 ▷ make PGGL_3.4
6457 ▷ make PGGL_3.8
6458 ▷ make AGL_1.27
6459 ▷ make AGGL_2.27
6460 ▷ make SP_4.2
6461 ▷ make PSP_4.4
6462 ▷ make PGOp_4.2
6463 ▷ make PGO_5.2
6464 ▷ make PGGO_5.4
6465 ▷ make PGOp_6.2
6466 ▷ make PGOm_6.2
6467 ▷ make PSP_6.2
6468 ▷ make PGO_7.2
6469 ▷ make NullPolarity_6.2
6470
6471
6472
6473 PGL_3.2:

```



```

6474 ▷ $(ORBITER) -v 2 \
6475 ▷ ▷ -define F -finite_field -q 2 -end \
6476 ▷ ▷ -define G -linear_group -PGL 3 F -end \
6477 ▷ ▷ -with G -do \
6478 ▷ ▷ -group_theoretic_activity \
6479 ▷ ▷ ▷ -report \
6480 ▷ ▷ -end
6481 ▷ pdflatex PGL_3_2_report.tex
6482 ▷ $(OPEN) PGL_3_2_report.pdf
6483
6484
6485 PGL_4_2:
6486 ▷ $(ORBITER) -v 2 \
6487 ▷ ▷ -define F -finite_field -q 2 -end \
6488 ▷ ▷ -define G -linear_group -PGL 4 F -lex_least_base -end \
6489 ▷ ▷ -with G -do \
6490 ▷ ▷ -group_theoretic_activity \
6491 ▷ ▷ ▷ -report \
6492 ▷ ▷ -end
6493 ▷ pdflatex PGL_4_2_report.tex
6494 ▷ $(OPEN) PGL_4_2_report.pdf
6495
6496
6497 PGL_4_2_multiply_all_elements:
6498 ▷ $(ORBITER) -v 5 \
6499 ▷ ▷ -define F -finite_field -q 2 -end \
6500 ▷ ▷ -define G -linear_group -PGL 4 F -lex_least_base -end \
6501 ▷ ▷ -with G -do \
6502 ▷ ▷ -group_theoretic_activity \
6503 ▷ ▷ ▷ -multiply_all_elements_in_lex_order \
6504 ▷ ▷ -end
6505
6506 # 0.37 of a second, dt=37 tps = 100
6507
6508
6509 PGL_4_2_export:
6510 ▷ $(ORBITER) -v 2 \
6511 ▷ ▷ -define F -finite_field -q 2 -end \
6512 ▷ ▷ -define G -linear_group -PGL 4 F -end \
6513 ▷ ▷ -with G -do \
6514 ▷ ▷ -group_theoretic_activity \
6515 ▷ ▷ ▷ -report \
6516 ▷ ▷ -end \
6517 ▷ ▷ -with G -do \
6518 ▷ ▷ -group_theoretic_activity \
6519 ▷ ▷ ▷ -export_orbiter \
6520 ▷ ▷ -end
6521 ▷ pdflatex PGL_4_2_report.tex
6522 ▷ $(OPEN) PGL_4_2_report.pdf
6523
6524 # created by PGL_4_2_export
6525
6526
6527 PGL_4_2_export_GAP:
6528 ▷ $(ORBITER) -v 2 \
6529 ▷ ▷ -define F -finite_field -q 2 -end \
6530 ▷ ▷ -define G -linear_group -PGL 4 F -end \
6531 ▷ ▷ -with G -do \
6532 ▷ ▷ -group_theoretic_activity \

```

```

6533 ▷ ▷ ▷ -report \
6534 ▷ ▷ -end \
6535 ▷ ▷ -with G -do \
6536 ▷ ▷ -group_theoretic_activity \
6537 ▷ ▷ ▷ -export_gap \
6538 ▷ ▷ -end
6539 ▷ #pdflatex PGL_4.2_report.tex
6540 ▷ #$(OPEN) PGL_4.2_report.pdf
6541
6542
6543 PGL_4.2_generated:
6544 ▷ $(ORBITER) -v 2 \
6545 ▷ ▷ -define gens -vector -file PGL_4.2_gens.csv -end \
6546 ▷ ▷ -define G -permutation_group \
6547 ▷ ▷ -bsgs PGL_4.2 "{\rm PGL}(4,2)" 15 20160 "0,1,2,3" 6 gens -end \
6548
6549 PGL_5.2:
6550 ▷ $(ORBITER) -v 2 \
6551 ▷ ▷ -define F -finite_field -q 2 -end \
6552 ▷ ▷ -define G -linear_group -PGL 5 F -lex_least_base -end \
6553 ▷ ▷ -with G -do \
6554 ▷ ▷ -group_theoretic_activity \
6555 ▷ ▷ ▷ -report \
6556 ▷ ▷ -end
6557 ▷ pdflatex PGL_5.2_report.tex
6558 ▷ $(OPEN) PGL_5.2_report.pdf
6559
6560 #The order of the group PGL(5, 2) is 9999360
6561
6562 PGL_5.2_multiply_all_elements:
6563 ▷ $(ORBITER) -v 5 \
6564 ▷ ▷ -define F -finite_field -q 2 -end \
6565 ▷ ▷ -define G -linear_group -PGL 5 F -lex_least_base -end \
6566 ▷ ▷ -with G -do \
6567 ▷ ▷ -group_theoretic_activity \
6568 ▷ ▷ ▷ -multiply_all_elements_in_lex_order \
6569 ▷ ▷ -end
6570
6571 # time: 9 min :28 sec on Macbook
6572
6573
6574 L_5.3:
6575 ▷ $(ORBITER) -v 2 \
6576 ▷ ▷ -define F -finite_field -q 3 -end \
6577 ▷ ▷ -define G -linear_group -PSL 5 F -end \
6578 ▷ ▷ -with G -do \
6579 ▷ ▷ -group_theoretic_activity \
6580 ▷ ▷ ▷ -report \
6581 ▷ ▷ -end
6582 ▷ pdflatex PSL_5.3_report.tex
6583 ▷ $(OPEN) PSL_5.3_report.pdf
6584
6585 #PSL(5,3): Order 237783237120 = 121 * 120 * 117 * 108 * 81 * 16
6586
6587
6588
6589 L_4.5:
6590 ▷ $(ORBITER) -v 2 \
6591 ▷ ▷ -define F -finite_field -q 5 -end \

```

```

6592 ▷ ▷ -define G -linear_group -PSL 4 F -end \
6593 ▷ ▷ -with G -do \
6594 ▷ ▷ -group_theoretic_activity \
6595 ▷ ▷ ▷ -report \
6596 ▷ ▷ -end
6597 ▷ pdflatex PSL_4.5_report.tex
6598 ▷ $(OPEN) PSL_4.5_report.pdf
6599
6600 #PSL(4,5): Order 7254000000
6601
6602
6603
6604
6605
6606
6607
6608
6609
6610 PGL_4.5:
6611 ▷ $(ORBITER) -v 2 \
6612 ▷ ▷ -define F -finite_field -q 5 -end \
6613 ▷ ▷ -define G -linear_group -PGL 4 F -end \
6614 ▷ ▷ -with G -do \
6615 ▷ ▷ -group_theoretic_activity \
6616 ▷ ▷ ▷ -report \
6617 ▷ ▷ -end
6618 ▷ pdflatex PGL_4.5_report.tex
6619 ▷ $(OPEN) PGL_4.5_report.pdf
6620
6621
6622
6623 PGGL_3.4:
6624 ▷ $(ORBITER) -v 2 \
6625 ▷ ▷ -define G -linear_group -PGGL 3 4 -end \
6626 ▷ ▷ -with G -do \
6627 ▷ ▷ -group_theoretic_activity \
6628 ▷ ▷ ▷ -report \
6629 ▷ ▷ ▷ -report_sylow \
6630 ▷ ▷ ▷ -report_classes \
6631 ▷ ▷ -end
6632 ▷ pdflatex PGGL_3.4_report.tex
6633 ▷ $(OPEN) PGGL_3.4_report.pdf
6634
6635
6636
6637 PGGL_3.8:
6638 ▷ $(ORBITER) -v 2 \
6639 ▷ ▷ -define G -linear_group -PGGL 3 8 -end \
6640 ▷ ▷ -with G -do \
6641 ▷ ▷ -group_theoretic_activity \
6642 ▷ ▷ ▷ -report \
6643 ▷ ▷ ▷ -report_sylow \
6644 ▷ ▷ ▷ -report_classes \
6645 ▷ ▷ -end
6646 ▷ pdflatex PGGL_3.8_report.tex
6647 ▷ $(OPEN) PGGL_3.8_report.pdf
6648
6649
6650

```

```

6651
6652 AGL_1.27:
6653 ▷ $(ORBITER) -v 2 \
6654 ▷ ▷ -define F -finite_field -q 27 -end \
6655 ▷ ▷ -define G -linear_group -AGL 1 F -end \
6656 ▷ ▷ -with G -do \
6657 ▷ ▷ -group_theoretic_activity \
6658 ▷ ▷ ▷ -report \
6659 ▷ ▷ -end
6660 ▷ pdflatex AGL_1.27_report.tex
6661 ▷ $(OPEN) AGL_1.27_report.pdf
6662
6663
6664 AGGL_2.27:
6665 ▷ $(ORBITER) -v 2 \
6666 ▷ ▷ -define F -finite_field -q 27 -end \
6667 ▷ ▷ -define G -linear_group -AGGL 2 F -end \
6668 ▷ ▷ -with G -do \
6669 ▷ ▷ -group_theoretic_activity \
6670 ▷ ▷ ▷ -report \
6671 ▷ ▷ -end
6672 ▷ pdflatex AGGL_2.27_report.tex
6673 ▷ $(OPEN) AGGL_2.27_report.pdf
6674
6675 #The order of the group AGGL(2, 27) is 1117679472
6676
6677
6678 SP_4.2:
6679 ▷ $(ORBITER) -v 2 \
6680 ▷ ▷ -define F -finite_field -q 2 -end \
6681 ▷ ▷ -define G -linear_group -GL 4 F \
6682 ▷ ▷ ▷ -symplectic_group \
6683 ▷ ▷ -end \
6684 ▷ ▷ -with G -do \
6685 ▷ ▷ -group_theoretic_activity \
6686 ▷ ▷ ▷ -report \
6687 ▷ ▷ -end
6688 ▷ pdflatex GL_4.2_Sp_4.2_report.tex
6689 ▷ $(OPEN) GL_4.2_Sp_4.2_report.pdf
6690
6691 # order 720
6692
6693
6694 PSP_4.4:
6695 ▷ $(ORBITER) -v 5 \
6696 ▷ ▷ -define F -finite_field -q 4 -end \
6697 ▷ ▷ -define G -linear_group -PGL 4 F \
6698 ▷ ▷ ▷ -symplectic_group \
6699 ▷ ▷ -end \
6700 ▷ ▷ -with G -do \
6701 ▷ ▷ -group_theoretic_activity \
6702 ▷ ▷ ▷ -report \
6703 ▷ ▷ -end
6704 ▷ pdflatex PGL_4.4_Sp_4.4_report.tex
6705 ▷ $(OPEN) PGL_4.4_Sp_4.4_report.pdf
6706
6707 #order 979200
6708
6709

```

```

6710 PGOp_4.2:
6711 ▷ $(ORBITER) -v 2 \
6712 ▷ ▷ -define F -finite_field -q 2 -end \
6713 ▷ ▷ -define G -linear_group -PGOp 4 F -end \
6714 ▷ ▷ -with G -do \
6715 ▷ ▷ -group_theoretic_activity \
6716 ▷ ▷ ▷ -report \
6717 ▷ ▷ -end
6718 ▷ pdflatex PGOp_4.2_report.tex
6719 ▷ $(OPEN) PGOp_4.2_report.pdf
6720
6721 # order 72
6722
6723
6724 PGO_5.2:
6725 ▷ $(ORBITER) -v 2 \
6726 ▷ ▷ -define F -finite_field -q 2 -end \
6727 ▷ ▷ -define G -linear_group -PGO 5 F -end \
6728 ▷ ▷ -with G -do \
6729 ▷ ▷ -group_theoretic_activity \
6730 ▷ ▷ ▷ -report \
6731 ▷ ▷ -end
6732 ▷ pdflatex PGO_5.2_report.tex
6733 ▷ $(OPEN) PGO_5.2_report.pdf
6734
6735 # order 720
6736
6737
6738 PGG0_5.4:
6739 ▷ $(ORBITER) -v 2 \
6740 ▷ ▷ -define F4 -finite_field -q 4 -end \
6741 ▷ ▷ -define G -linear_group -PGG0 5 F4 -end \
6742 ▷ ▷ -with G -do \
6743 ▷ ▷ -group_theoretic_activity \
6744 ▷ ▷ ▷ -report \
6745 ▷ ▷ -end
6746 ▷ pdflatex PGG0_5.4_report.tex
6747 ▷ $(OPEN) PGG0_5.4_report.pdf
6748
6749 # order 1958400
6750
6751 PGG0_5.4_export_gap:
6752 ▷ $(ORBITER) -v 2 \
6753 ▷ ▷ -define F4 -finite_field -q 4 -end \
6754 ▷ ▷ -define G -linear_group -PGG0 5 F4 -end \
6755 ▷ ▷ -with G -do \
6756 ▷ ▷ -group_theoretic_activity \
6757 ▷ ▷ ▷ -export_gap \
6758 ▷ ▷ -end
6759
6760
6761 PGG0_5.4_multiply_all_elements:
6762 ▷ $(ORBITER) -v 5 \
6763 ▷ ▷ -define F4 -finite_field -q 4 -end \
6764 ▷ ▷ -define G -linear_group -PGG0 5 F4 -end \
6765 ▷ ▷ -with G -do \
6766 ▷ ▷ -group_theoretic_activity \
6767 ▷ ▷ ▷ -multiply_all_elements_in_lex_order \
6768 ▷ ▷ -end

```

```

6769
6770
6771 PGOp_6.2:
6772 ▷ $(ORBITER) -v 2 \
6773 ▷ ▷ -define F -finite_field -q 2 -end \
6774 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
6775 ▷ ▷ -with G -do \
6776 ▷ ▷ -group_theoretic_activity \
6777 ▷ ▷ ▷ -report \
6778 ▷ ▷ -end
6779 ▷ pdflatex PGOp_6.2.report.tex
6780 ▷ $(OPEN) PGOp_6.2.report.pdf
6781
6782 PGOm_6.2:
6783 ▷ $(ORBITER) -v 2 \
6784 ▷ ▷ -define F -finite_field -q 2 -end \
6785 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
6786 ▷ ▷ -with G -do \
6787 ▷ ▷ -group_theoretic_activity \
6788 ▷ ▷ ▷ -report \
6789 ▷ ▷ -end
6790 ▷ pdflatex PGOm_6.2.report.tex
6791 ▷ $(OPEN) PGOm_6.2.report.pdf
6792
6793
6794
6795
6796
6797 # the following two groups are isomorphic:
6798
6799 PSP_6.2:
6800 ▷ $(ORBITER) -v 2 \
6801 ▷ ▷ -define F -finite_field -q 2 -end \
6802 ▷ ▷ -define G -linear_group -PGL 6 F \
6803 ▷ ▷ ▷ -symplectic_group \
6804 ▷ ▷ -end \
6805 ▷ ▷ -with G -do \
6806 ▷ ▷ -group_theoretic_activity \
6807 ▷ ▷ ▷ -report \
6808 ▷ ▷ -end
6809 ▷ pdflatex PGL_6.2_Sp_6.2.report.tex
6810 ▷ $(OPEN) PGL_6.2_Sp_6.2.report.pdf
6811
6812 # group order 1451520, acting on 63 points
6813
6814
6815 PGO_7.2:
6816 ▷ $(ORBITER) -v 2 \
6817 ▷ ▷ -define F -finite_field -q 2 -end \
6818 ▷ ▷ -define G -linear_group -PGO 7 F -end \
6819 ▷ ▷ -with G -do \
6820 ▷ ▷ -group_theoretic_activity \
6821 ▷ ▷ ▷ -report \
6822 ▷ ▷ -end
6823 ▷ pdflatex PGO_7.2.report.tex
6824 ▷ $(OPEN) PGO_7.2.report.pdf
6825
6826
6827 # group order 1451520, acting on 63 points

```

```

6828
6829
6830
6831
6832
6833 NullPolarity_6.2:
6834 ▷ $(ORBITER) -v 2 \
6835 ▷ ▷ -define F -finite_field -q 2 -end \
6836 ▷ ▷ -define G -linear_group -PGL 6 F \
6837 ▷ ▷ ▷ -null_polarity_group \
6838 ▷ ▷ -end \
6839 ▷ ▷ -with G -do \
6840 ▷ ▷ -group_theoretic_activity \
6841 ▷ ▷ ▷ -report \
6842 ▷ ▷ -end
6843 ▷ pdflatex PGL_6.2_NullPolarity_6.2_report.tex
6844 ▷ $(OPEN) PGL_6.2_NullPolarity_6.2_report.pdf
6845
6846 # group order 23040, acting on 63 points
6847
6848
6849
6850 #####
6851 # Section 6.3: Subgroups
6852
6853 SECTION.SUBGROUPS:
6854
6855
6856 test_6.3:
6857 ▷ make C13
6858 ▷ make C13.generated
6859 ▷ make C13.as_subgroup
6860 ▷ make J1
6861 ▷ make PGL_3.11.singer
6862 ▷ make PGL_3.11.singer_and_frobenius
6863 ▷ make PG_2.4.order_21
6864 ▷ make quaternion
6865 ▷ make cube_group
6866 ▷ make tetra_group
6867 ▷ make Hesse_group
6868 ▷ make Weyl_E8
6869 ▷ make test_subgroup
6870 ▷ make coset_reps
6871 ▷ make coset_reps_read
6872 ▷ make SP_6.2.point_stab_subgroup
6873 ▷ make PGOp_6.2.report
6874 ▷ make PGOm_6.2.report
6875 ▷ make PGOm_6.2.export_magma
6876 ▷ make PGOp_6.2.point_stab_subgroup
6877 ▷ make PGOp_6.2.linear
6878 ▷ make PGOp_6.2.linear_stab_6
6879
6880
6881
6882
6883 C13:
6884 ▷ $(ORBITER) -v 2 \
6885 ▷ ▷ -define gens -vector -dense $(GEN.C13) -end \
6886 ▷ ▷ -define G -permutation_group \

```

```

6887 ▷ ▷ ▷ -bsgs C13 C_{13} 13 13 0 1 \
6888 ▷ ▷ ▷ ▷ gens \
6889 ▷ ▷ ▷ -end \
6890 ▷ ▷ -with G -do \
6891 ▷ ▷ -group_theoretic.activity \
6892 ▷ ▷ ▷ -export_orbiter \
6893 ▷ ▷ -end \
6894 ▷ ▷ -with G -do \
6895 ▷ ▷ -group_theoretic.activity \
6896 ▷ ▷ ▷ -export_group_table \
6897 ▷ ▷ -end \
6898 ▷ ▷ -with G -do \
6899 ▷ ▷ -group_theoretic.activity \
6900 ▷ ▷ ▷ -report \
6901 ▷ ▷ -end \
6902 ▷ ▷ -with G -do \
6903 ▷ ▷ -group_theoretic.activity \
6904 ▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
6905 ▷ ▷ -end
6906 ▷ pdflatex C13_report.tex
6907 ▷ $(OPEN) C13_report.pdf
6908
6909
6910 C13_generated:
6911 ▷ $(ORBITER) -v 2 \
6912 ▷ ▷ -define gens -vector -file C13_gens.csv -end \
6913 ▷ ▷ -define G -permutation_group \
6914 ▷ ▷ -bsgs C13 "C_{13}" 13 13 "0" 1 gens -end \
6915
6916
6917 C13_as_subgroup:
6918 ▷ $(ORBITER) -v 2 \
6919 ▷ ▷ -define G -permutation_group -symmetric_group 13 \
6920 ▷ ▷ ▷ -subgroup_by_generators C13 13 1 $(GEN_C13) -end \
6921 ▷ ▷ -with G -do \
6922 ▷ ▷ -group_theoretic.activity \
6923 ▷ ▷ ▷ -export_orbiter \
6924 ▷ ▷ -end \
6925 ▷ ▷ -with G -do \
6926 ▷ ▷ -group_theoretic.activity \
6927 ▷ ▷ ▷ -report \
6928 ▷ ▷ -end \
6929 ▷ ▷ -with G -do \
6930 ▷ ▷ -group_theoretic.activity \
6931 ▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
6932 ▷ ▷ -end
6933 ▷ #pdflatex Perm13.Subgroup_C13_13_report.tex
6934 ▷ #$(OPEN) Perm13.Subgroup_C13_13_report.pdf
6935
6936
6937
6938
6939 J1:
6940 ▷ $(ORBITER) -v 2 \
6941 ▷ ▷ -define G -linear_group -PGL 7 11 -Janko1 -end \
6942 ▷ ▷ -with G -do \
6943 ▷ ▷ -group_theoretic.activity \
6944 ▷ ▷ ▷ -report \
6945 ▷ ▷ -end

```



```

6946 ▷ pdflatex Janko1_report.tex
6947 ▷ $(OPEN) Janko1_report.pdf
6948
6949
6950 PGL_3_11_singer:
6951 ▷ $(ORBITER) -v 2 \
6952 ▷ ▷ -define G -linear_group -PGL 3 11 -singer 19 -end \
6953 ▷ ▷ -with G -do \
6954 ▷ ▷ -group_theoretic_activity \
6955 ▷ ▷ ▷ -report \
6956 ▷ ▷ -end
6957 ▷ pdflatex PGL_3_11_Singer_3_11_19_report.tex
6958 ▷ $(OPEN) PGL_3_11_Singer_3_11_19_report.pdf
6959
6960
6961 PGL_3_11_singer_and_frobenius:
6962 ▷ $(ORBITER) -v 2 \
6963 ▷ ▷ -define G -linear_group -PGL 3 11 -singer_and_frobenius 19 -end \
6964 ▷ ▷ -with G -do \
6965 ▷ ▷ -group_theoretic_activity \
6966 ▷ ▷ ▷ -report \
6967 ▷ ▷ -end
6968 ▷ ▷ pdflatex PGL_3_11_Singer_and_Frob3_11_19_report.tex
6969 ▷ ▷ $(OPEN) PGL_3_11_Singer_and_Frob3_11_19_report.pdf
6970
6971 PG_2_4_order_21:
6972 ▷ $(ORBITER) -v 2 \
6973 ▷ ▷ -define G -linear_group -PGL 3 4 -end \
6974 ▷ ▷ -with G -do \
6975 ▷ ▷ -group_theoretic_activity \
6976 ▷ ▷ ▷ -search_element_of_order 21 \
6977 ▷ ▷ -end
6978
6979
6980
6981
6982 quaternion:
6983 ▷ $(ORBITER) -v 2 \
6984 ▷ ▷ -define G -linear_group -SL 2 3 \
6985 ▷ ▷ -subgroup_by_generators "quaternion" "8" 3 \
6986 ▷ ▷ ▷ "1,1,1,2, 2,1,1,1, 0,2,1,0" \
6987 ▷ ▷ -end \
6988 ▷ ▷ -with G -do \
6989 ▷ ▷ -group_theoretic_activity \
6990 ▷ ▷ ▷ -print_elements_tex \
6991 ▷ ▷ ▷ -report_group_table \
6992 ▷ ▷ ▷ -report \
6993 ▷ ▷ -end
6994 ▷ pdflatex SL_2_3_Subgroup_quaternion_8_report.tex
6995 ▷ $(OPEN) SL_2_3_Subgroup_quaternion_8_report.pdf
6996
6997
6998 cube_group:
6999 ▷ $(ORBITER) -v 2 \
7000 ▷ ▷ -define gens -vector -dense \
7001 ▷ ▷ ▷ "0,1,0,2,0,0,0,0,1, \
7002 ▷ ▷ ▷ 0,0,1,0,1,0,2,0,0, \
7003 ▷ ▷ ▷ 2,0,0,0,1,0,0,0,1" \
7004 ▷ ▷ -end \

```

```

7005 ▷ ▷ -define G -linear_group -GL 3 3 \
7006 ▷ ▷ -subgroup.by_generators "cube" "48" 3 \
7007 ▷ ▷ ▷ gens \
7008 ▷ ▷ -end \
7009 ▷ ▷ -with G -do \
7010 ▷ ▷ -group_theoretic_activity \
7011 ▷ ▷ ▷ -print_elements_tex \
7012 ▷ ▷ ▷ -report \
7013 ▷ ▷ -end
7014 ▷ pdflatex GL_3_3.Subgroup_cube_48_report.tex
7015 ▷ $(OPEN) GL_3_3.Subgroup_cube_48_report.pdf
7016
7017
7018 tetra_group:
7019 ▷ $(ORBITER) -v 3 \
7020 ▷ ▷ -define G -linear_group -GL 3 3 \
7021 ▷ ▷ -subgroup.by_generators "tetra" "12" 2 \
7022 ▷ ▷ ▷ "0,1,0,0,0,1,1,0,0, 0,0,1,2,0,0,0,2,0" \
7023 ▷ ▷ -end \
7024 ▷ ▷ -with G -do \
7025 ▷ ▷ -group_theoretic_activity \
7026 ▷ ▷ ▷ -print_elements_tex \
7027 ▷ ▷ ▷ -report \
7028 ▷ ▷ -end
7029 ▷ pdflatex GL_3_3.Subgroup_tetra_12_report.tex
7030 ▷ $(OPEN) GL_3_3.Subgroup_tetra_12_report.pdf
7031
7032
7033
7034 Hesse_group:
7035 ▷ $(ORBITER) -v 3 \
7036 ▷ ▷ -define gens -vector -compact \
7037 ▷ ▷ ▷ $(GENERATORS_HESSÉ_GROUP) \
7038 ▷ ▷ -end \
7039 ▷ ▷ -define G -linear_group -PGGL 3 4 \
7040 ▷ ▷ -subgroup.by_generators "Hesse" "432" 7 gens \
7041 ▷ ▷ -end \
7042 ▷ ▷ -with G -do \
7043 ▷ ▷ -group_theoretic_activity \
7044 ▷ ▷ ▷ -print_elements_tex \
7045 ▷ ▷ ▷ -report \
7046 ▷ ▷ -end
7047 ▷ pdflatex PGGL_3_4.Subgroup_Hesse_432_report.tex
7048 ▷ $(OPEN) PGGL_3_4.Subgroup_Hesse_432_report.pdf
7049
7050
7051
7052 Weyl_E8:
7053 ▷ $(ORBITER) -v 3 \
7054 ▷ ▷ -define gens -vector -dense \
7055 ▷ ▷ ▷ $(GENERATORS_WEYL_GROUP_E8) \
7056 ▷ ▷ -end \
7057 ▷ ▷ -define G -linear_group -GL 8 3 \
7058 ▷ ▷ -subgroup.by_generators \
7059 ▷ ▷ ▷ "Weyl_E8" "696729600" 2 gens \
7060 ▷ ▷ -end \
7061 ▷ ▷ -with G -do \
7062 ▷ ▷ -group_theoretic_activity \
7063 ▷ ▷ ▷ -report \

```

```

7064 ▷ ▷ -end
7065 ▷ pdflatex GL_8_3.Subgroup.Weyl.E8.696729600_report.tex
7066 ▷ $(OPEN) GL_8_3.Subgroup.Weyl.E8.696729600_report.pdf
7067
7068 # group generators from http://www.math.rwth-aachen.de/~Gabriele.Nebe/LATTICES/E8
    .b.html
7069
7070
7071
7072 test_subgroup:
7073 ▷ $(ORBITER) -v 2 \
7074 ▷ ▷ -define F -finite_field -q 2 -end \
7075 ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
7076 ▷ ▷ -define G2 -linear_group -PGL 6 F \
7077 ▷ ▷ ▷ -symplectic_group \
7078 ▷ ▷ -end \
7079 ▷ ▷ -with G1 -and G2 -do \
7080 ▷ ▷ -group_theoretic_activity \
7081 ▷ ▷ ▷ -is_subgroup_of \
7082 ▷ ▷ -end
7083
7084
7085 coset_reps:
7086 ▷ $(ORBITER) -v 2 \
7087 ▷ ▷ -define F -finite_field -q 2 -end \
7088 ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
7089 ▷ ▷ -define G2 -linear_group -PGL 6 F \
7090 ▷ ▷ ▷ -symplectic_group \
7091 ▷ ▷ -end \
7092 ▷ ▷ -with G1 -and G2 -do \
7093 ▷ ▷ -group_theoretic_activity \
7094 ▷ ▷ ▷ -coset_reps \
7095 ▷ ▷ -end
7096 ▷ pdflatex PGOp_6_2.coset_reps.tex
7097 ▷ $(OPEN) PGOp_6_2.coset_reps.pdf
7098
7099 coset_reps_read:
7100 ▷ $(ORBITER) -v 2 \
7101 ▷ ▷ -define F -finite_field -q 2 -end \
7102 ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
7103 ▷ ▷ -define G2 -linear_group -PGL 6 F \
7104 ▷ ▷ ▷ -symplectic_group \
7105 ▷ ▷ -end \
7106 ▷ ▷ -define CR -vector_ge -action G2 \
7107 ▷ ▷ ▷ -read_csv \
7108 ▷ ▷ ▷ PGOp_6_2.coset_reps.csv Element \
7109 ▷ ▷ -end
7110
7111 SP_6_2.point_stab_subgroup:
7112 ▷ $(ORBITER) -v 2 \
7113 ▷ ▷ -define F -finite_field -q 2 -end \
7114 ▷ ▷ -define G -linear_group -PGL 6 F \
7115 ▷ ▷ ▷ -symplectic_group \
7116 ▷ ▷ -end \
7117 ▷ ▷ -define G0 -modified_group -from G \
7118 ▷ ▷ ▷ -point_stabilizer 0 \
7119 ▷ ▷ -end \
7120 ▷ ▷ -with G0 -do \
7121 ▷ ▷ -group_theoretic_activity \

```

```

7122 ▷ ▷ ▷ -report \
7123 ▷ ▷ -end
7124 ▷ pdflatex PGL_6.2_Sp_6.2_Stab0_report.tex
7125 ▷ $(OPEN) PGL_6.2_Sp_6.2_Stab0_report.pdf
7126
7127
7128 # group of order 23040
7129
7130
7131 PGOp_6.2_report:
7132 ▷ $(ORBITER) -v 2 \
7133 ▷ ▷ -define F -finite_field -q 2 -end \
7134 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
7135 ▷ ▷ -with G -do \
7136 ▷ ▷ -group_theoretic_activity \
7137 ▷ ▷ ▷ -report \
7138 ▷ ▷ -end
7139 ▷ pdflatex PGOp_6.2_report.tex
7140 ▷ $(OPEN) PGOp_6.2_report.pdf
7141
7142 # group order 40320
7143
7144
7145 PGOm_6.2_report:
7146 ▷ $(ORBITER) -v 2 \
7147 ▷ ▷ -define F -finite_field -q 2 -end \
7148 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
7149 ▷ ▷ -with G -do \
7150 ▷ ▷ -group_theoretic_activity \
7151 ▷ ▷ ▷ -report \
7152 ▷ ▷ -end
7153 ▷ pdflatex PGOm_6.2_report.tex
7154 ▷ $(OPEN) PGOm_6.2_report.pdf
7155
7156
7157
7158 PGOm_6.2_export_magma:
7159 ▷ $(ORBITER) -v 2 \
7160 ▷ ▷ -define F -finite_field -q 2 -end \
7161 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
7162 ▷ ▷ -with G -do \
7163 ▷ ▷ -group_theoretic_activity \
7164 ▷ ▷ ▷ -export_magma \
7165 ▷ ▷ -end
7166
7167
7168
7169 PGOp_6.2_point_stab_subgroup:
7170 ▷ $(ORBITER) -v 2 \
7171 ▷ ▷ -define F -finite_field -q 2 -end \
7172 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
7173 ▷ ▷ -define G0 -modified_group -from G \
7174 ▷ ▷ ▷ -point_stabilizer 0 \
7175 ▷ ▷ -end \
7176 ▷ ▷ -with G0 -do \
7177 ▷ ▷ -group_theoretic_activity \
7178 ▷ ▷ ▷ -report \
7179 ▷ ▷ -end
7180 ▷ pdflatex PGOp_6.2_report.tex

```

```

7181 ▷ $(OPEN) PGOp_6.2_report.pdf
7182
7183
7184 # group of order 1152
7185
7186
7187 PGOp_6.2_linear:
7188 ▷ $(ORBITER) -v 2 \
7189 ▷ ▷ -define F -finite_field -q 2 -end \
7190 ▷ ▷ -define G -linear_group -PGL 6 F \
7191 ▷ ▷ ▷ -subgroup_by_generators PGOp_6.2 \
7192 ▷ ▷ ▷ ▷ 40320 10 $(PGOp_6.2_GENS) \
7193 ▷ ▷ -end \
7194 ▷ ▷ -with G -do \
7195 ▷ ▷ -group_theoretic_activity \
7196 ▷ ▷ ▷ -report \
7197 ▷ ▷ -end
7198 ▷ pdflatex PGL_6.2_Subgroup_PGOp_6.2_40320_report.tex
7199 ▷ $(OPEN) PGL_6.2_Subgroup_PGOp_6.2_40320_report.pdf
7200
7201
7202
7203 PGOp_6.2_linear_stab.6:
7204 ▷ $(ORBITER) -v 2 \
7205 ▷ ▷ -define F -finite_field -q 2 -end \
7206 ▷ ▷ -define G -linear_group -PGL 6 F \
7207 ▷ ▷ ▷ -subgroup_by_generators PGOp_6.2 \
7208 ▷ ▷ ▷ ▷ 40320 10 $(PGOp_6.2_GENS) \
7209 ▷ ▷ -end \
7210 ▷ ▷ -define G6 -modified_group -from G \
7211 ▷ ▷ ▷ -point_stabilizer 6 \
7212 ▷ ▷ -end \
7213 ▷ ▷ -with G6 -do \
7214 ▷ ▷ -group_theoretic_activity \
7215 ▷ ▷ ▷ -report \
7216 ▷ ▷ -end
7217 ▷ pdflatex PGL_6.2_Subgroup_PGOp_6.2_40320_Stab6_report.tex
7218 ▷ $(OPEN) PGL_6.2_Subgroup_PGOp_6.2_40320_Stab6_report.pdf
7219
7220
7221 # group of order 1440, of index 28 in PGOp(6,2)
7222
7223
7224
7225
7226 #####
7227 # Section 6.4: New Actions from Old
7228
7229
7230 SECTION_NEW_ACTIONS_FROM_OLD:
7231
7232
7233 test_6.4:
7234 ▷ make Symmetric_4_on_pairs
7235 ▷ make direct_product_AGL
7236 ▷ make PGGLstar_3.4
7237 ▷ make Group_M11
7238 ▷ make surface.q13.Eckardt
7239 ▷ make surface.q13.Eckardt_on_tritangent_planes

```

```

7240 ▷ make Hirschfeld_stab_projectivity_group
7241 ▷ make on_planes
7242 ▷ make on_planes_restricted
7243 ▷ make T3_on_tensors
7244 ▷ make T3r1
7245 ▷ make T4_on_tensors
7246 ▷ make T4r1
7247 ▷ #make PGGL_2_8_on_conic
7248 ▷ make PGL_4_2_wedge
7249 ▷ make PGL_5_3_wedge
7250 ▷ make PGL_4_2_wd
7251 ▷ make PGL_5_3_wd
7252 ▷ make PGL_4_2_wd_reverse
7253
7254
7255
7256
7257 Symmetric_4_on_pairs:
7258 ▷ $(ORBITER) -v 3 \
7259 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
7260 ▷ ▷ -define G_on_2subsets -modified_group -from G \
7261 ▷ ▷ ▷ -on_k_subsets 2 \
7262 ▷ ▷ -end \
7263 ▷ ▷ -with G_on_2subsets -do \
7264 ▷ ▷ -group_theoretic_activity \
7265 ▷ ▷ ▷ -report \
7266 ▷ ▷ -end
7267 ▷ pdflatex Sym_4_OnSubsets_2_report.tex
7268 ▷ $(OPEN) Sym_4_OnSubsets_2_report.pdf
7269
7270
7271
7272 direct_product_AGL:
7273 ▷ $(ORBITER) -v 4 \
7274 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
7275 ▷ ▷ -define G1 -linear_group -AGL 1 3 \
7276 ▷ ▷ ▷ -end \
7277 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
7278 ▷ ▷ ▷ -end \
7279 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
7280 ▷ ▷ ▷ "21" "1,1,1,1" \
7281 ▷ ▷ -end \
7282 ▷ ▷ -with G -do \
7283 ▷ ▷ -group_theoretic_activity \
7284 ▷ ▷ ▷ -report \
7285 ▷ ▷ -end
7286 ▷ pdflatex product_AGL_1_3_AGL_1_7_report.tex
7287 ▷ $(OPEN) product_AGL_1_3_AGL_1_7_report.pdf
7288
7289 PGGLstar_3_4:
7290 ▷ $(ORBITER) -v 6 \
7291 ▷ ▷ -define F -finite_field -q 4 -end \
7292 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
7293 ▷ ▷ -define G0 -linear_group -PGGL 3 F -end \
7294 ▷ ▷ -define G -modified_group -polarity_extension \
7295 ▷ ▷ ▷ G0 P \
7296 ▷ ▷ -end \
7297 ▷ ▷ -with G -do \
7298 ▷ ▷ -group_theoretic_activity \

```

```

7299 ▷ ▷ ▷ -report \
7300 ▷ ▷ -end
7301 ▷ pdflatex PGGL_3.4_polarity_ext_report.tex
7302 ▷ $(OPEN) PGGL_3.4_polarity_ext_report.pdf
7303
7304
7305
7306 Group_M11:
7307 ▷ $(ORBITER) -v 2 \
7308 ▷ ▷ -define gens -vector -dense $(GENERATORS.M12) -end \
7309 ▷ ▷ -define G -permutation_group -symmetric_group 12 \
7310 ▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \
7311 ▷ ▷ -define G0 -modified_group -from G \
7312 ▷ ▷ ▷ -point_stabilizer 0 \
7313 ▷ ▷ -end \
7314 ▷ ▷ -with G0 -do \
7315 ▷ ▷ -group_theoretic_activity \
7316 ▷ ▷ ▷ -report \
7317 ▷ ▷ -end
7318 ▷ pdflatex Sym_12.Subgroup_M12.95040_Stab0_report.tex
7319 ▷ $(OPEN) Sym_12.Subgroup_M12.95040_Stab0_report.pdf
7320
7321
7322
7323 surface_q13_Eckardt:
7324 ▷ $(ORBITER) -v 3 \
7325 ▷ ▷ -define F -finite_field -q 13 -end \
7326 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
7327 ▷ ▷ -define S -cubic_surface -space P -arc_lifting "0,1,2,3,43,113" -end \
7328 ▷ ▷ -with S -do \
7329 ▷ ▷ -cubic_surface_activity \
7330 ▷ ▷ ▷ -report \
7331 ▷ ▷ -end \
7332 ▷ ▷ -with S -do \
7333 ▷ ▷ -cubic_surface_activity \
7334 ▷ ▷ ▷ -export_something tritangent_planes \
7335 ▷ ▷ -end
7336 ▷ pdflatex surface_arc_lifting_trihedral_q13_arc0.1.2.3.43.113_report.tex
7337 ▷ $(OPEN) surface_arc_lifting_trihedral_q13_arc0.1.2.3.43.113_report.pdf
7338
7339 #surface_equation_F_Hirschfeld_q4_tritangent_planes.csv
7340
7341
7342 surface_q13_Eckardt_on_tritangent_planes:
7343 ▷ $(ORBITER) -v 2 \
7344 ▷ ▷ -orbiter_path $(ORBITER.EXE.PATH) \
7345 ▷ ▷ -draw_options -embedded -end \
7346 ▷ ▷ -define F -finite_field -q 13 -end \
7347 ▷ ▷ -define gens -vector -field F \
7348 ▷ ▷ ▷ -dense $(SURFACE_Q13_STAB_GENS) \
7349 ▷ ▷ -end \
7350 ▷ ▷ -define TriP -set -file \
7351 ▷ ▷ ▷ surface_arc_lifting_trihedral_q13_arc0.1.2.3.43.113_tritangent_planes.csv
7352 ▷ ▷ -end \
7353 ▷ ▷ -define G -linear_group -PGL 4 F \
7354 ▷ ▷ -subgroup_by_generators "stab" \
7355 ▷ ▷ ▷ 24 3 gens \
7356 ▷ ▷ ▷ -end \

```

```

7357 ▷ ▷ -define G_on_planes -modified_group -from G \
7358 ▷ ▷ ▷ -on_k_subspaces 3 \
7359 ▷ ▷ -end \
7360 ▷ ▷ -define Gr -modified_group -from G_on_planes \
7361 ▷ ▷ ▷ -restricted_action TriP TriP \
7362 ▷ ▷ -end \
7363 ▷ ▷ -with Gr -do \
7364 ▷ ▷ -group_theoretic_activity \
7365 ▷ ▷ ▷ -report \
7366 ▷ ▷ -end \
7367 ▷ ▷ -define Orb -orbits -group Gr \
7368 ▷ ▷ ▷ -on_points \
7369 ▷ ▷ -end \
7370 ▷ ▷ -with Orb -do -orbits_activity \
7371 ▷ ▷ ▷ -report \
7372 ▷ ▷ -end \
7373 ▷ ▷ -with Orb -do -orbits_activity \
7374 ▷ ▷ ▷ -draw_tree 0 \
7375 ▷ ▷ -end \
7376 ▷ ▷ -with Orb -do -orbits_activity \
7377 ▷ ▷ ▷ -draw_tree 1 \
7378 ▷ ▷ -end \
7379 ▷ ▷ -with Orb -do -orbits_activity \
7380 ▷ ▷ ▷ -stabilizer 36 \
7381 ▷ ▷ -end
7382 ▷ pdflatex PGL_4_13_Gr_4_3_res_TriP_orbits_report.tex
7383 ▷ $(OPEN) PGL_4_13_Gr_4_3_res_TriP_orbits_report.pdf
7384
7385
7386 Hirschfeld_stab_projectivity_group:
7387 ▷ $(ORBITER) -v 4 \
7388 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
7389 ▷ ▷ -define G -linear_group -PGGL 4 4 \
7390 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
7391 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7392 ▷ ▷ ▷ -end \
7393 ▷ ▷ -define G0 -modified_group -from G \
7394 ▷ ▷ ▷ -projectivity_subgroup \
7395 ▷ ▷ -end \
7396 ▷ ▷ -with G0 -do \
7397 ▷ ▷ -group_theoretic_activity \
7398 ▷ ▷ ▷ -report \
7399 ▷ ▷ -end
7400 ▷ pdflatex PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_ProjectivitySubgroup_report.te
x
7401 ▷ $(OPEN) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_ProjectivitySubgroup_report.pdf

7402
7403
7404
7405
7406
7407
7408 # related to planes.in.pencil:
7409 # we are computing the action on the planes through the line 0.
7410
7411
7412
7413 on_planes:

```



```

7414 ▷ $(ORBITER) -v 2 \
7415 ▷ ▷ -define F -finite_field -q 8 -end \
7416 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
7417 ▷ ▷ -define G -linear_group -PGL 4 F -end \
7418 ▷ ▷ -define G.on_planes -modified_group -from G \
7419 ▷ ▷ ▷ -on_k.subspaces 3 \
7420 ▷ ▷ -end \
7421 ▷ ▷ -with G.on_planes -do \
7422 ▷ ▷ -group_theoretic_activity \
7423 ▷ ▷ -apply "0,8,1,6,4,3,7,2,5" \
7424 ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
7425 ▷ ▷ -end
7426 ▷ pdflatex PGL_4.8_Gr_4.3_apply.tex
7427 ▷ $(OPEN) PGL_4.8_Gr_4.3_apply.pdf
7428
7429
7430 on_planes_restricted:
7431 ▷ $(ORBITER) -v 2 \
7432 ▷ ▷ -define planes -vector -dense "0,8,1,6,4,3,7,2,5" -end \
7433 ▷ ▷ -define F -finite_field -q 8 -end \
7434 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
7435 ▷ ▷ -define G -linear_group -PGL 4 F \
7436 ▷ ▷ ▷ -subgroup_by_generators "cyclic" \
7437 ▷ ▷ ▷ 9 1 "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
7438 ▷ ▷ ▷ -end \
7439 ▷ ▷ -define G.on_planes -modified_group -from G \
7440 ▷ ▷ ▷ -on_k.subspaces 3 \
7441 ▷ ▷ -end \
7442 ▷ ▷ -define Gr -modified_group -from G.on_planes \
7443 ▷ ▷ ▷ -restricted_action planes planes \
7444 ▷ ▷ -end \
7445 ▷ ▷ -with Gr -do \
7446 ▷ ▷ -group_theoretic_activity \
7447 ▷ ▷ ▷ -report \
7448 ▷ ▷ -end
7449 ▷ pdflatex PGL_4.8_Gr_4.3_res_planes_report.tex
7450 ▷ $(OPEN) PGL_4.8_Gr_4.3_res_planes_report.pdf
7451
7452
7453 T3_on_tensors:
7454 ▷ $(ORBITER) -v 2 \
7455 ▷ ▷ -define G \
7456 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
7457 ▷ ▷ ▷ -on_tensors -end \
7458 ▷ ▷ -with G -do \
7459 ▷ ▷ -group_theoretic_activity \
7460 ▷ ▷ ▷ -report \
7461 ▷ ▷ -end
7462 ▷ pdflatex GL_2.2_wreath_Sym3_report.tex
7463 ▷ $(OPEN) GL_2.2_wreath_Sym3_report.pdf
7464 ▷
7465
7466
7467 T3r1:
7468 ▷ $(ORBITER) -v 4 \
7469 ▷ ▷ -define G \
7470 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
7471 ▷ ▷ ▷ -on_rank_one_tensors -end \
7472 ▷ ▷ -with G -do \

```

```

7473 ▷ ▷ -group_theoretic.activity \
7474 ▷ ▷ ▷ -report \
7475 ▷ ▷ -end
7476 ▷ pdflatex GL_2_2_wreath_Sym3_report.tex
7477 ▷ $(OPEN) GL_2_2_wreath_Sym3_report.pdf
7478
7479
7480
7481
7482
7483 T4_on_tensors:
7484 ▷ $(ORBITER) -v 4 \
7485 ▷ ▷ -define G \
7486 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
7487 ▷ ▷ ▷ -on_tensors -end \
7488 ▷ ▷ -with G -do \
7489 ▷ ▷ -group_theoretic.activity \
7490 ▷ ▷ ▷ -report \
7491 ▷ ▷ -end
7492 ▷ pdflatex GL_2_2_wreath_Sym4_report.tex
7493 ▷ $(OPEN) GL_2_2_wreath_Sym4_report.pdf
7494 ▷
7495 ▷
7496 T4r1:
7497 ▷ $(ORBITER) -v 4 \
7498 ▷ ▷ -define G \
7499 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
7500 ▷ ▷ ▷ -on_rank_one_tensors -end \
7501 ▷ ▷ -with G -do \
7502 ▷ ▷ -group_theoretic.activity \
7503 ▷ ▷ ▷ -report \
7504 ▷ ▷ -end
7505 ▷ pdflatex GL_2_2_wreath_Sym4_report.tex
7506 ▷ $(OPEN) GL_2_2_wreath_Sym4_report.pdf
7507
7508
7509 # ToDo
7510
7511 PGGL_2_8_on_conic:
7512 ▷ $(ORBITER) -v 4 \
7513 ▷ ▷ -define G -linear_group \
7514 ▷ ▷ ▷ -PGGL 2 8 -PGL2OnConic \
7515 ▷ ▷ -end \
7516 ▷ ▷ -with G -do \
7517 ▷ ▷ -group_theoretic.activity \
7518 ▷ ▷ ▷ -report \
7519 ▷ ▷ -end
7520 ▷ #pdflatex PGGL_2_8.OnConic_2.8_report.tex
7521 ▷ #$(OPEN) PGGL_2_8.OnConic_2.8_report.pdf
7522
7523 PGL_4_2_wedge:
7524 ▷ $(ORBITER) -v 3 \
7525 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7526 ▷ ▷ -define G2 -modified_group -from G \
7527 ▷ ▷ ▷ -on_wedge_product \
7528 ▷ ▷ -end \
7529 ▷ ▷ -with G -do \
7530 ▷ ▷ -group_theoretic.activity \
7531 ▷ ▷ ▷ -report \

```

```

7532 ▷ ▷ -end
7533 ▷ pdflatex PGL_4.2_report.tex
7534 ▷ $(OPEN) PGL_4.2_report.pdf
7535
7536
7537
7538
7539 PGL_5.3_wedge:
7540 ▷ $(ORBITER) -v 3 \
7541 ▷ ▷ -define G -linear_group -PGL 5 3 -end \
7542 ▷ ▷ -define G2 -modified_group -from G \
7543 ▷ ▷ ▷ -on_wedge_product \
7544 ▷ ▷ -end \
7545 ▷ ▷ -with G -do \
7546 ▷ ▷ -group_theoretic_activity \
7547 ▷ ▷ ▷ -report \
7548 ▷ ▷ -end
7549 ▷ pdflatex PGL_5.3_report.tex
7550 ▷ $(OPEN) PGL_5.3_report.pdf
7551
7552
7553
7554
7555
7556 PGL_4.2_wd:
7557 ▷ $(ORBITER) -v 3 \
7558 ▷ ▷ -define G -linear_group -PGL 4 2 -wedge_detached -end \
7559 ▷ ▷ -with G -do \
7560 ▷ ▷ -group_theoretic_activity \
7561 ▷ ▷ ▷ -report \
7562 ▷ ▷ -end
7563 ▷ pdflatex PGL_6.2_Wedge_4.2_detached_report.tex
7564 ▷ $(OPEN) PGL_6.2_Wedge_4.2_detached_report.pdf
7565
7566
7567
7568 PGL_5.3_wd:
7569 ▷ $(ORBITER) -v 3 \
7570 ▷ ▷ -define G -linear_group -PGL 5 3 -wedge_detached -end \
7571 ▷ ▷ -with G -do \
7572 ▷ ▷ -group_theoretic_activity \
7573 ▷ ▷ ▷ -report \
7574 ▷ ▷ -end
7575 ▷ pdflatex PGL_10.3_Wedge_5.3_detached_report.tex
7576 ▷ $(OPEN) PGL_10.3_Wedge_5.3_detached_report.pdf
7577
7578
7579 PGL_4.2_wd_reverse:
7580 ▷ $(ORBITER) -v 3 \
7581 ▷ ▷ -define G -linear_group -PGL 4 2 -wedge_detached -end \
7582 ▷ ▷ -with G -do \
7583 ▷ ▷ -group_theoretic_activity \
7584 ▷ ▷ ▷ -reverse_isomorphism_exterior_square \
7585 ▷ ▷ -end
7586
7587
7588
7589
7590

```

```

7591
7592 #####
7593 # Section 6.5: Group Theoretic Activities
7594
7595
7596 SECTION.GROUP.THEORETIC.ACTIVITIES:
7597
7598
7599 test_6.5:
7600 ▷ make PGL_3.2.elements
7601 ▷ make Sym_3.elements
7602 ▷ make Cycle_13.power
7603 ▷ make Cycle_12.power
7604 ▷ make PGL_3.4.singer
7605 ▷ make GL_2.8.multiply
7606 ▷ make GL_2.7.multiply
7607 ▷ make GL_2.7.inv
7608 ▷ make GL_2.7.power
7609 ▷ make PGL_3.2.classes
7610 ▷ make PGL_4.2.classes.based.on.normal.form
7611 ▷ make PGL_10.2.classes.based.on.normal.form
7612 ▷ make normal_forms_PGL_4.4
7613 ▷ make PGL_4.4.2A.rank
7614 ▷ make PGL_4.4.2A.unrank
7615 ▷ make PGL_4.5.3B.rank
7616 ▷ make PGL_4.5.3B.unrank
7617 ▷ make normal_forms_PGL_4.5
7618 ▷ make on.planes
7619 ▷ make PGL_4.2.random.element
7620
7621
7622
7623 PGL_3.2.elements:
7624 ▷ $(ORBITER) -v 5 \
7625 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
7626 ▷ ▷ -with G -do \
7627 ▷ ▷ -group_theoretic.activity \
7628 ▷ ▷ ▷ -save_elements_csv "PGL_3.2.elements.csv" \
7629 ▷ ▷ -end
7630
7631 # creates PGL_3.2.elements.csv
7632
7633
7634 Sym_3.elements:
7635 ▷ $(ORBITER) -v 3 \
7636 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
7637 ▷ ▷ -with G -do \
7638 ▷ ▷ -group_theoretic.activity \
7639 ▷ ▷ ▷ -print_elements_tex \
7640 ▷ ▷ -end
7641 ▷ $(ORBITER) -v 2 \
7642 ▷ ▷ -draw_options \
7643 ▷ ▷ ▷ -nodes \
7644 ▷ ▷ ▷ -embedded -radius 250 \
7645 ▷ ▷ ▷ -xin 10000 -yin 10000 \
7646 ▷ ▷ ▷ -xout 1000000 -yout 600000 \
7647 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
7648 ▷ ▷ -end \
7649 ▷ ▷ -tree_draw -file Sym_3.elements_tree.txt -end

```

```

7650 ▷ pdflatex Sym_3.elements_tree.draw.tex
7651 ▷ $(OPEN) Sym_3.elements_tree.draw.pdf
7652
7653
7654
7655 Cycle_13.power:
7656 ▷ $(ORBITER) -v 5 \
7657 ▷ ▷ -define G -permutation_group -symmetric_group 13 -end \
7658 ▷ ▷ -with G -do \
7659 ▷ ▷ -group_theoretic_activity \
7660 ▷ ▷ ▷ -consecutive_powers \
7661 ▷ ▷ ▷ "1,2,3,4,5,6,7,8,9,10,11,12,0" 13 \
7662 ▷ ▷ -end
7663 ▷ pdflatex Sym_13.all.powers.tex
7664 ▷ $(OPEN) Sym_13.all.powers.pdf
7665
7666
7667 Cycle_12.power:
7668 ▷ $(ORBITER) -v 5 \
7669 ▷ ▷ -define G -permutation_group -symmetric_group 12 -end \
7670 ▷ ▷ -with G -do \
7671 ▷ ▷ -group_theoretic_activity \
7672 ▷ ▷ ▷ -consecutive_powers \
7673 ▷ ▷ ▷ "1,2,3,4,5,6,7,8,9,10,11,0" 12 \
7674 ▷ ▷ -end
7675 ▷ pdflatex Sym_12.all.powers.tex
7676 ▷ $(OPEN) Sym_12.all.powers.pdf
7677 ▷
7678
7679
7680
7681
7682 PGL_3_4.singer:
7683 ▷ $(ORBITER) -v 5 \
7684 ▷ ▷ -define G -linear_group -PGL 3 4 -end \
7685 ▷ ▷ -with G -do \
7686 ▷ ▷ -group_theoretic_activity \
7687 ▷ ▷ ▷ -find_singer_cycle \
7688 ▷ ▷ -end
7689
7690
7691 GL_2_8.multiply:
7692 ▷ $(ORBITER) -v 5 \
7693 ▷ ▷ -define G -linear_group -GL 2 8 -end \
7694 ▷ ▷ -with G -do \
7695 ▷ ▷ -group_theoretic_activity \
7696 ▷ ▷ ▷ -multiply "0,1,2,3" "4,5,6,7" \
7697 ▷ ▷ -end
7698 ▷ pdflatex GL_2_8.mult.tex
7699 ▷ $(OPEN) GL_2_8.mult.pdf
7700
7701
7702 GL_2_7.multiply:
7703 ▷ $(ORBITER) -v 5 \
7704 ▷ ▷ -define G -linear_group -GL 2 7 -end \
7705 ▷ ▷ -with G -do \
7706 ▷ ▷ -group_theoretic_activity \
7707 ▷ ▷ ▷ -multiply "0,1,2,3" "4,5,6,0" \
7708 ▷ ▷ -end

```

```

7709 ▷ pdflatex GL_2_7_mult.tex
7710 ▷ $(OPEN) GL_2_7_mult.pdf
7711
7712
7713 GL_2_7_inv:
7714 ▷ $(ORBITER) -v 5 \
7715 ▷ ▷ -define G -linear_group -GL 2 7 -end \
7716 ▷ ▷ -with G -do \
7717 ▷ ▷ -group_theoretic_activity \
7718 ▷ ▷ ▷ -inverse "0,1,2,3" \
7719 ▷ ▷ -end
7720 ▷ pdflatex GL_2_7_inv.tex
7721 ▷ $(OPEN) GL_2_7_inv.pdf
7722
7723 GL_2_7_power:
7724 ▷ $(ORBITER) -v 5 \
7725 ▷ ▷ -define G -linear_group -GL 2 7 -end \
7726 ▷ ▷ -with G -do \
7727 ▷ ▷ -group_theoretic_activity \
7728 ▷ ▷ ▷ -raise_to_the_power "0,1,2,3" 2 \
7729 ▷ ▷ -end
7730 ▷ pdflatex GL_2_7_power.tex
7731 ▷ $(OPEN) GL_2_7_power.pdf
7732
7733
7734 PGL_3_2_classes:
7735 ▷ $(ORBITER) -v 3 \
7736 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
7737 ▷ ▷ -with G -do \
7738 ▷ ▷ -group_theoretic_activity \
7739 ▷ ▷ ▷ -classes_based_on_normal_form \
7740 ▷ ▷ -end
7741 ▷ pdflatex PGL_3_2_classes_based_on_normal_forms_3_2.tex
7742 ▷ $(OPEN) PGL_3_2_classes_based_on_normal_forms_3_2.pdf
7743 ▷ #pdflatex PGL_3_2_classes_out.tex
7744 ▷ #$(OPEN) PGL_3_2_classes_out.pdf
7745
7746 #▷ ▷ -classes \
7747
7748
7749 PGL_4_2_classes_based_on_normal_form:
7750 ▷ $(ORBITER) -v 3 \
7751 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7752 ▷ ▷ -with G -do \
7753 ▷ ▷ -group_theoretic_activity \
7754 ▷ ▷ ▷ -classes_based_on_normal_form \
7755 ▷ ▷ -end
7756 ▷ pdflatex PGL_4_2_classes_based_on_normal_forms_4_2.tex
7757 ▷ $(OPEN) PGL_4_2_classes_based_on_normal_forms_4_2.pdf
7758
7759
7760 PGL_10_2_classes_based_on_normal_form:
7761 ▷ $(ORBITER) -v 3 \
7762 ▷ ▷ -define G -linear_group -PGL 10 2 -end \
7763 ▷ ▷ -with G -do \
7764 ▷ ▷ -group_theoretic_activity \
7765 ▷ ▷ ▷ -classes_based_on_normal_form \
7766 ▷ ▷ -end
7767 ▷ pdflatex PGL_10_2_classes_based_on_normal_forms_10_2.tex

```

```

7768 ▷ $(OPEN) PGL_10.2_classes_based_on_normal_forms_10.2.pdf
7769
7770
7771
7772 normal_forms_PGL_4.4:
7773 ▷ $(ORBITER) -v 7 \
7774 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
7775 ▷ ▷ -with G -do \
7776 ▷ ▷ -group_theoretic_activity \
7777 ▷ ▷ ▷ -classes_based_on_normal_form \
7778 ▷ ▷ -end
7779 ▷ pdflatex PGGL_4.4_classes_based_on_normal_forms_4.4.tex
7780 ▷ $(OPEN) PGGL_4.4_classes_based_on_normal_forms_4.4.pdf
7781
7782
7783
7784
7785 PGL_4.4.2A_rank:
7786 ▷ $(ORBITER) -v 6 \
7787 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
7788 ▷ ▷ -with G -do \
7789 ▷ ▷ -group_theoretic_activity \
7790 ▷ ▷ ▷ -element_rank \
7791 ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1, 1" \
7792 ▷ ▷ -end
7793
7794
7795 PGL_4.4.2A_unrank:
7796 ▷ $(ORBITER) -v 6 \
7797 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
7798 ▷ ▷ -with G -do \
7799 ▷ ▷ -group_theoretic_activity \
7800 ▷ ▷ ▷ -element_unrank "1" \
7801 ▷ ▷ -end
7802 ▷ ▷
7803
7804
7805
7806
7807
7808
7809 PGL_4.5.3B_rank:
7810 ▷ $(ORBITER) -v 6 \
7811 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
7812 ▷ ▷ -with G -do \
7813 ▷ ▷ -group_theoretic_activity \
7814 ▷ ▷ ▷ -element_rank "0,0,0,1, 2,3,0,1, 0,3,4,4, 0,1,2,1" \
7815 ▷ ▷ -end
7816
7817
7818 PGL_4.5.3B_unrank:
7819 ▷ $(ORBITER) -v 6 \
7820 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
7821 ▷ ▷ -with G -do \
7822 ▷ ▷ -group_theoretic_activity \
7823 ▷ ▷ ▷ -element_unrank "701459351" \
7824 ▷ ▷ -end
7825 ▷ ▷
7826

```

```

7827
7828
7829 normal_forms_PGL_4_5:
7830 ▷ $(ORBITER) -v 7 \
7831 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
7832 ▷ ▷ -with G -do \
7833 ▷ ▷ -group_theoretic_activity \
7834 ▷ ▷ ▷ -classes_based_on_normal_form \
7835 ▷ ▷ -end
7836 ▷ pdflatex PGL_4_5_classes_based_on_normal_forms_4.5.tex
7837 ▷ $(OPEN) PGL_4_5_classes_based_on_normal_forms_4.5.pdf
7838
7839
7840
7841 PGL_4_2_random_element:
7842 ▷ $(ORBITER) -v 2 \
7843 ▷ ▷ -define F -finite_field -q 2 -end \
7844 ▷ ▷ -define G -linear_group -PGL 4 F -end \
7845 ▷ ▷ -with G -do \
7846 ▷ ▷ -group_theoretic_activity \
7847 ▷ ▷ ▷ -random_element r1 \
7848 ▷ ▷ -end \
7849 ▷ -print_symbols
7850
7851
7852 #group order H = 20160
7853 #Element :
7854 #1 0 1 1
7855 #0 1 1 0
7856 #0 1 0 1
7857 #1 0 1 0
7858
7859
7860 #coded: ( 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0 )
7861 #Element as permutation:
7862 #(0, 13, 1, 7, 12, 4, 2, 10, 5, 11, 14, 9, 3, 6, 8)
7863 #In list notation:
7864 #( 13, 7, 10, 6, 2, 11, 8, 12, 0, 3, 5, 14, 4, 1, 9 )
7865 #The rank of the element is 9883
7866
7867 #\{0,1,2,3,4\} -> \{13,7,10,6,2\} = \{2,6,7,10,13\}
7868
7869
7870 #####
7871 # Section 6.6: Group Theoretic Activities Based on Magma
7872
7873
7874 SECTION_GROUP_THEORETIC_ACTIVITIES_BASED_ON_MAGMA:
7875
7876
7877 test_6.6:
7878 ▷ make PSP_4_4_export_magma
7879 ▷ make PGGL_2_4_export_magma
7880 ▷ make Hirschfeld_stab_export_magma
7881 ▷ make PGGL_2_4_classes
7882 ▷ make PGL_7_2_classes
7883 ▷ make PGL_8_2_classes
7884 ▷ make Hirschfeld_stab_classes
7885 ▷ make PGGL_2_4_cent_2A

```



```

7886 ▷ make Normalizer_of_H5
7887 ▷ make PGGL_3.4.classes
7888 ▷ make classes_PGGL_4.4
7889 ▷ make subgroups_PGL_4.5
7890 ▷ make classes_PGL_4.5
7891 ▷ #make PGL_4.5.3B.class.again
7892 ▷ make search_primitive.poly.q5.deg3
7893 ▷ make GL_3.5.singer.power
7894 ▷ make PGL_4.5.norm.31
7895 ▷ make Normalizer_of_Z22.in.PGL_2.9
7896 ▷ make PG0m_6.2.classes
7897 ▷ make PG0m_6.2.Class.2A
7898 ▷ make PG0m_6.2.Class.2A.print
7899 ▷ make PG0m_6.2.Class.2A.centralizer
7900 ▷ make PG0m_6.2.Class.2A.centralizer.recover
7901
7902
7903
7904
7905 PSP_4.4.export.magma:
7906 ▷ $(ORBITER) -v 5 \
7907 ▷ ▷ -define F -finite_field -q 4 -end \
7908 ▷ ▷ -define G -linear_group -PGL 4 F \
7909 ▷ ▷ ▷ -symplectic_group \
7910 ▷ ▷ -end \
7911 ▷ ▷ -with G -do \
7912 ▷ ▷ -group_theoretic_activity \
7913 ▷ ▷ ▷ -export_magma \
7914 ▷ ▷ -end
7915
7916 # creates PGL_4.4.Sp_4.4.generators.magma
7917
7918
7919 PGGL_2.4.export.magma:
7920 ▷ $(ORBITER) -v 20 \
7921 ▷ ▷ -define F -finite_field -q 4 \
7922 ▷ ▷ ▷ -compute_related_fields \
7923 ▷ ▷ -end \
7924 ▷ ▷ -define G -linear_group -PGGL 2 F \
7925 ▷ ▷ -end \
7926 ▷ ▷ -with G -do \
7927 ▷ ▷ -group_theoretic_activity \
7928 ▷ ▷ ▷ -report \
7929 ▷ ▷ -end \
7930 ▷ ▷ -with G -do \
7931 ▷ ▷ -group_theoretic_activity \
7932 ▷ ▷ ▷ -export_magma \
7933 ▷ ▷ -end
7934 ▷ pdflatex PGGL_2.4.report.tex
7935 ▷ $(OPEN) PGGL_2.4.report.pdf
7936
7937
7938
7939
7940 Hirschfeld_stab.export.magma:
7941 ▷ $(ORBITER) -v 9 \
7942 ▷ ▷ -orbiter_path $(ORBITER.EXE_PATH) \
7943 ▷ ▷ -define F -finite_field -q 4 \
7944 ▷ ▷ ▷ -compute_related_fields \

```

```

7945 ▷ ▷ -end \
7946 ▷ ▷ -define G -linear_group -PGL 4 F \
7947 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
7948 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7949 ▷ ▷ ▷ -end \
7950 ▷ ▷ -with G -do \
7951 ▷ ▷ -group_theoretic_activity \
7952 ▷ ▷ ▷ -export_magma \
7953 ▷ ▷ -end
7954
7955
7956 PGL_2.4_classes:
7957 ▷ $(ORBITER) -v 3 \
7958 ▷ ▷ -define G \
7959 ▷ ▷ -linear_group -PGL 2 4 \
7960 ▷ ▷ -end \
7961 ▷ ▷ -with G -do \
7962 ▷ ▷ -group_theoretic_activity \
7963 ▷ ▷ ▷ -classes \
7964 ▷ ▷ -end
7965 ▷ $(MAGMA_PATH)magma PGL_2.4_classes.magma
7966 ▷ $(ORBITER) -v 3 \
7967 ▷ ▷ -define G \
7968 ▷ ▷ -linear_group -PGL 2 4 \
7969 ▷ ▷ -end \
7970 ▷ ▷ -with G -do \
7971 ▷ ▷ -group_theoretic_activity \
7972 ▷ ▷ ▷ -classes \
7973 ▷ ▷ -end
7974 ▷ pdflatex PGL_2.4_classes.out.tex
7975 ▷ $(OPEN) PGL_2.4_classes.out.pdf
7976 ▷ $(OPEN) PGL_2.4_classes.out.csv
7977
7978
7979
7980 PGL_7.2_classes:
7981 ▷ $(ORBITER) -v 3 \
7982 ▷ ▷ -define G \
7983 ▷ ▷ -linear_group -PGL 7 2 \
7984 ▷ ▷ -end \
7985 ▷ ▷ -with G -do \
7986 ▷ ▷ -group_theoretic_activity \
7987 ▷ ▷ ▷ -classes \
7988 ▷ ▷ -end
7989 ▷ $(MAGMA_PATH)magma PGL_7.2_classes.magma
7990 ▷ $(ORBITER) -v 3 \
7991 ▷ ▷ -define G \
7992 ▷ ▷ -linear_group -PGL 7 2 \
7993 ▷ ▷ -end \
7994 ▷ ▷ -with G -do \
7995 ▷ ▷ -group_theoretic_activity \
7996 ▷ ▷ ▷ -classes \
7997 ▷ ▷ -end
7998
7999
8000 PGL_8.2_classes:
8001 ▷ $(ORBITER) -v 3 \
8002 ▷ ▷ -define G \
8003 ▷ ▷ -linear_group -PGL 8 2 \

```

```

8004 ▷ ▷ -end \
8005 ▷ ▷ -with G -do \
8006 ▷ ▷ -group_theoretic_activity \
8007 ▷ ▷ ▷ -classes \
8008 ▷ ▷ -end
8009 ▷ $(MAGMA_PATH)magma PGL_8_2_classes.magma
8010 ▷ $(ORBITER) -v 3 \
8011 ▷ ▷ -define G \
8012 ▷ ▷ -linear_group -PGL 8 2 \
8013 ▷ ▷ -end \
8014 ▷ ▷ -with G -do \
8015 ▷ ▷ -group_theoretic_activity \
8016 ▷ ▷ ▷ -classes \
8017 ▷ ▷ -end
8018
8019
8020
8021
8022
8023 Hirschfeld_stab_classes:
8024 ▷ $(ORBITER) -v 9 \
8025 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8026 ▷ ▷ -define G -linear_group -PGGL 4 4 \
8027 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
8028 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
8029 ▷ ▷ ▷ -end \
8030 ▷ ▷ -with G -do \
8031 ▷ ▷ -group_theoretic_activity \
8032 ▷ ▷ ▷ -classes \
8033 ▷ ▷ -end
8034 ▷ $(ORBITER) -v 9 \
8035 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8036 ▷ ▷ -define G -linear_group -PGGL 4 4 \
8037 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
8038 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
8039 ▷ ▷ ▷ -end \
8040 ▷ ▷ -with G -do \
8041 ▷ ▷ -group_theoretic_activity \
8042 ▷ ▷ ▷ -classes \
8043 ▷ ▷ -end
8044 ▷ pdflatex PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_classes_out.tex
8045 ▷ $(OPEN) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_classes_out.pdf
8046
8047
8048
8049 PGGL_2_4_cent_2A:
8050 ▷ $(ORBITER) -v 3 \
8051 ▷ ▷ -define G \
8052 ▷ ▷ -linear_group -PGGL 2 4 -end \
8053 ▷ ▷ -with G -do \
8054 ▷ ▷ -group_theoretic_activity \
8055 ▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
8056 ▷ ▷ ▷ -report \
8057 ▷ ▷ -end
8058 ▷ $(MAGMA_PATH)magma element_2A_centralizer.magma
8059 ▷ $(ORBITER) -v 6 \
8060 ▷ ▷ -define G \
8061 ▷ ▷ -linear_group -PGGL 2 4 -end \
8062 ▷ ▷ -with G -do \

```

```

8063 ▷ ▷ -group_theoretic_activity \
8064 ▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
8065 ▷ ▷ ▷ -report \
8066 ▷ ▷ -end
8067 ▷ pdflatex PGGL_2.4_elt_2A.centralizer.tex
8068 ▷ $(OPEN) PGGL_2.4_elt_2A.centralizer.pdf
8069 ▷
8070
8071
8072 Normalizer_of_H5:
8073 ▷ $(ORBITER) -v 2 \
8074 ▷ ▷ -define G -permutation_group -symmetric_group 13 \
8075 ▷ ▷ ▷ -subgroup_by_generators H5 5 1 \
8076 ▷ ▷ ▷ ▷ $(GENERATORS_H5) -end \
8077 ▷ ▷ -with G -do \
8078 ▷ ▷ -group_theoretic_activity \
8079 ▷ ▷ ▷ -normalizer \
8080 ▷ ▷ -end
8081 ▷ pdflatex Sym_13.Subgroup_H5_5.normalizer.tex
8082 ▷ $(OPEN) Sym_13.Subgroup_H5_5.normalizer.pdf
8083
8084
8085
8086
8087 PGGL_3.4_classes:
8088 ▷ $(ORBITER) -v 3 \
8089 ▷ ▷ -define G \
8090 ▷ ▷ -linear_group -PGGL 3 4 \
8091 ▷ ▷ -end \
8092 ▷ ▷ -with G -do \
8093 ▷ ▷ -group_theoretic_activity \
8094 ▷ ▷ ▷ -classes \
8095 ▷ ▷ -end
8096 ▷ pdflatex PGGL_3.4_classes.out.tex
8097 ▷ $(OPEN) PGGL_3.4_classes.out.pdf
8098
8099
8100
8101
8102
8103
8104
8105 classes_PGGL_4.4:
8106 ▷ $(ORBITER) -v 3 \
8107 ▷ ▷ -magma_path $(MAGMA_PATH) \
8108 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
8109 ▷ ▷ -with G -do \
8110 ▷ ▷ -group_theoretic_activity \
8111 ▷ ▷ ▷ -classes \
8112 ▷ ▷ -end
8113
8114 # group order 1974067200 = 2^13 * 3^4 * 5^2 * 7 * 17
8115
8116
8117
8118
8119
8120 # the -find_subgroup command is too specialized
8121

```

```

8122 subgroups_PGL_4_5:
8123 ▷ $(ORBITER) -v 6 \
8124 ▷ ▷ -define G \
8125 ▷ ▷ -linear_group -PGL 4 5 -end \
8126 ▷ ▷ -with G -do \
8127 ▷ ▷ -group_theoretic_activity \
8128 ▷ ▷ ▷ -find_subgroup 3 \
8129 ▷ ▷ -end
8130 ▷ pdflatex PGL_4_5_report.tex
8131 ▷ $(OPEN) PGL_4_5_report.pdf
8132
8133
8134 classes_PGL_4_5:
8135 ▷ $(ORBITER) -v 6 \
8136 ▷ ▷ -define G \
8137 ▷ ▷ -linear_group -PGL 4 5 -end \
8138 ▷ ▷ -with G -do \
8139 ▷ ▷ -group_theoretic_activity \
8140 ▷ ▷ ▷ -classes \
8141 ▷ ▷ -end
8142 ▷ pdflatex PGL_4_5_classes_out.tex
8143 ▷ $(OPEN) PGL_4_5_classes_out.pdf
8144 ▷
8145 # 163 classes
8146
8147
8148 # two classes of elements of order 3
8149 #Order of element = 3 Class size = 31000 Centralizer order = 93600 Normalizer or
    der = 187200
8150 #of order 3 and with 0 fixed points.
8151 #0,1,0,2,0,1,2,1,4,2,3,1,2,0,4,3,
8152
8153 #Class size = 10075000 Centralizer order = 2880 Normalizer order = 5760
8154 #of order 3 and with 6 fixed points.
8155 #0,0,0,1,2,3,0,1,0,3,4,4,0,1,2,1,
8156
8157
8158 PGL_4_5_3B_class_again:
8159 ▷ $(ORBITER) -v 6 \
8160 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
8161 ▷ ▷ -with G -do \
8162 ▷ ▷ -group_theoretic_activity \
8163 ▷ ▷ ▷ -conjugacy_class_of \
8164 ▷ ▷ ▷ "3B" "0,0,0,1, 2,3,0,1, 0,3,4,4, 0,1,2,1" \
8165 ▷ ▷ -end
8166
8167 # very slow
8168
8169
8170 search_primitive_poly_q5_deg3:
8171 ▷ $(ORBITER) -v 6 \
8172 ▷ ▷ -define F -finite_field -q 5 -end \
8173 ▷ ▷ -with F -do \
8174 ▷ ▷ -finite_field_activity \
8175 ▷ ▷ -get_primitive_polynomial 3 -end
8176
8177
8178 #OK, we found an irreducible and primitive polynomial  $X^3 + X^2 + 2$ 
8179

```

```

8180 GL_3_5_singer_power:
8181 ▷ $(ORBITER) -v 6 -define G \
8182 ▷ ▷ -linear_group -GL 3 5 -end \
8183 ▷ ▷ -with G -do \
8184 ▷ ▷ -group_theoretic_activity \
8185 ▷ ▷ ▷ -raise_to_the_power \
8186 ▷ ▷ ▷ "0,1,0, 0,0,1, 3,0,4" 31 \
8187 ▷ ▷ -end
8188 ▷ pdflatex GL_3_5_power.tex
8189 ▷ $(OPEN) GL_3_5_power.pdf
8190
8191
8192
8193 PGL_4_5_norm_31:
8194 ▷ $(ORBITER) -v 6 -define G \
8195 ▷ ▷ -linear_group -PGL 4 5 -end \
8196 ▷ ▷ -with G -do \
8197 ▷ ▷ -group_theoretic_activity \
8198 ▷ ▷ ▷ -normalizer_of_cyclic_subgroup "31" \
8199 ▷ ▷ ▷ "2,0,0,0, 0,0,1,0, 0,0,0,1, 0,3,0,4"
8200 ▷ ▷ -end
8201 ▷ pdflatex normalizer_of_31_in_PGL_4_5.tex
8202 ▷ $(OPEN) normalizer_of_31_in_PGL_4_5.pdf
8203
8204
8205
8206 Normalizer_of_Z22_in_PGL_2_9:
8207 ▷ $(ORBITER) -v 2 \
8208 ▷ ▷ -define G -linear_group -PGL 2 9 \
8209 ▷ ▷ -subgroup_by_generators Z22 4 2 \
8210 ▷ ▷ ▷ "2,0,0,1, 0,1,1,0" -end \
8211 ▷ ▷ -with G -do \
8212 ▷ ▷ -group_theoretic_activity \
8213 ▷ ▷ ▷ -normalizer \
8214 ▷ ▷ -end
8215 ▷ pdflatex PGL_2_9_Subgroup_Z22_4_normalizer.tex
8216 ▷ $(OPEN) PGL_2_9_Subgroup_Z22_4_normalizer.pdf
8217
8218
8219
8220
8221 PG0m_6_2_classes:
8222 ▷ $(ORBITER) -v 2 \
8223 ▷ ▷ -define F -finite_field -q 2 -end \
8224 ▷ ▷ -define G -linear_group -PG0m 6 F -end \
8225 ▷ ▷ -with G -do \
8226 ▷ ▷ -group_theoretic_activity \
8227 ▷ ▷ ▷ -classes \
8228 ▷ ▷ -end
8229 ▷ pdflatex PG0m_6_2_classes.out.tex
8230 ▷ $(OPEN) PG0m_6_2_classes.out.pdf
8231
8232
8233 PG0m_6_2_Class_2A:
8234 ▷ $(ORBITER) -v 9 \
8235 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8236 ▷ ▷ -define F -finite_field -q 2 -end \
8237 ▷ ▷ -define G -linear_group -PG0m 6 F -end \
8238 ▷ ▷ -with G -do \

```

```

8239 ▷ ▷ -group_theoretic_activity \
8240 ▷ ▷ ▷ -conjugacy_class_of "2A" $(PGOM_6_2_CLASS_2A.REP) \
8241 ▷ ▷ -end
8242
8243
8244
8245 #PGOm_6_2_class_of_2A.csv
8246
8247
8248 PGOm_6_2_Class_2A_print:
8249 ▷ $(ORBITER) -v 9 \
8250 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8251 ▷ ▷ -define PGOm_6_2_Class2A -vector \
8252 ▷ ▷ ▷ -file $(FILE_NAME_PGOm_6_2_CLASS_2A) -end \
8253 ▷ ▷ -define F -finite_field -q 2 -end \
8254 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8255 ▷ ▷ -with G -do \
8256 ▷ ▷ -group_theoretic_activity \
8257 ▷ ▷ ▷ -element_processing \
8258 ▷ ▷ ▷ ▷ -input PGOm_6_2_Class2A \
8259 ▷ ▷ ▷ ▷ -print \
8260 ▷ ▷ ▷ -end \
8261 ▷ ▷ -end
8262 ▷ pdflatex PGOm_6_2_Class2A_elements.tex
8263 ▷ $(OPEN) PGOm_6_2_Class2A_elements.pdf
8264
8265
8266
8267
8268 PGOm_6_2_Class_2A_centralizer:
8269 ▷ $(ORBITER) -v 9 \
8270 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8271 ▷ ▷ -define F -finite_field -q 2 -end \
8272 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8273 ▷ ▷ -with G -do \
8274 ▷ ▷ -group_theoretic_activity \
8275 ▷ ▷ ▷ -centralizer_of_element \
8276 ▷ ▷ ▷ ▷ "2Arep" $(PGOM_6_2_CLASS_2A.REP_CLEAN) \
8277 ▷ ▷ -end
8278 ▷ pdflatex PGOm_6_2_elt_2Arep_centralizer.tex
8279 ▷ $(OPEN) PGOm_6_2_elt_2Arep_centralizer.pdf
8280
8281
8282
8283 PGOm_6_2_Class_2A_centralizer_recover:
8284 ▷ $(ORBITER) -v 2 \
8285 ▷ ▷ -define F -finite_field -q 2 -end \
8286 ▷ ▷ -define Class2A_cent_gens -vector \
8287 ▷ ▷ ▷ -dense $(PGOM_6_2_CLASS_2A_CENTRALIZER_GENS) -end \
8288 ▷ ▷ -define G -linear_group -PGL 6 2 \
8289 ▷ ▷ ▷ -subgroup_by_generators "2ACent" \
8290 ▷ ▷ ▷ 1440 9 Class2A_cent_gens \
8291 ▷ ▷ ▷ -end \
8292 ▷ ▷ -with G -do \
8293 ▷ ▷ -group_theoretic_activity \
8294 ▷ ▷ ▷ -report \
8295 ▷ ▷ -end
8296 ▷ pdflatex PGL_6_2_Subgroup_2ACent_1440_report.tex
8297 ▷ $(OPEN) PGL_6_2_Subgroup_2ACent_1440_report.pdf

```

```

8298
8299
8300 #####
8301 # Section 6.7: The GAP Interface
8302
8303
8304 SECTION_GAP_INTERFACE:
8305
8306 test_6.7:
8307   ▷ make Hirschfeld_stab_export_gap
8308   ▷ make Cyclic_6_canonical_image
8309   ▷ make Edge_curve_q17_canonical_image
8310
8311
8312 Hirschfeld_stab_export_gap:
8313   ▷ $(ORBITER) -v 9 \
8314   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8315   ▷ ▷ -define G -linear_group -PGGL 4 4 \
8316   ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
8317   ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
8318   ▷ ▷ ▷ -end \
8319   ▷ ▷ -with G -do \
8320   ▷ ▷ -group_theoretic_activity \
8321   ▷ ▷ ▷ -export_gap \
8322   ▷ ▷ -end
8323
8324
8325
8326 Cyclic_6_canonical_image:
8327   ▷ $(ORBITER) -v 3 \
8328   ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
8329   ▷ ▷ -with G -do -group_theoretic_activity \
8330   ▷ ▷ ▷ -canonical_image_GAP 3,5 \
8331   ▷ ▷ -end
8332
8333
8334
8335 Edge_curve_q17_canonical_image:
8336   ▷ $(ORBITER) -v 3 \
8337   ▷ ▷ -define G -linear_group -PGL 3 17 -end \
8338   ▷ ▷ -with G -do -group_theoretic_activity \
8339   ▷ ▷ ▷ -canonical_image_GAP $(EDGE_CURVE_Q17_AS_POINTS) \
8340   ▷ ▷ -end
8341
8342 #PGL_3_17_canonical_image.gap
8343
8344
8345
8346 #####
8347 # Section 6.8: Linear Groups, Advanced Topics
8348
8349
8350 SECTION_LINEAR_GROUPS_ADVANCED_TOPICS:
8351
8352 test_6.8:
8353   ▷ make U_3_3
8354   ▷ make PGL_2_3
8355   ▷ make Co3
8356   ▷ #make Ree_27

```



```

8357
8358
8359 U_3_3:
8360 ▷ $(ORBITER) -v 3 \
8361 ▷ ▷ -define F -finite_field -q 9 \
8362 ▷ ▷ ▷ -override_polynomial "17" \
8363 ▷ ▷ -end \
8364 ▷ ▷ -define G -linear_group -PGL 3 F \
8365 ▷ ▷ ▷ -subgroup_by_generators \
8366 ▷ ▷ ▷ "U_3_3" "6048" 2 \
8367 ▷ ▷ ▷ "1,6,4, 5,0,6, 8,5,1, \
8368 ▷ ▷ ▷ 6,2,1, 7,8,4, 0,6,6" \
8369 ▷ ▷ ▷ -end \
8370 ▷ ▷ -with G -do \
8371 ▷ ▷ -group_theoretic_activity \
8372 ▷ ▷ ▷ -report \
8373 ▷ ▷ -end
8374 ▷ pdflatex PGL_3_9-Subgroup.U_3_3.6048_report.tex
8375 ▷ #$(OPEN) PGL_3_9-Subgroup.U_3_3.6048_report.pdf
8376
8377
8378
8379
8380
8381 PGL_2_3:
8382 ▷ $(ORBITER) -v 3 \
8383 ▷ ▷ -define G -linear_group -PGL 2 3 -end \
8384 ▷ ▷ -with G -do \
8385 ▷ ▷ -group_theoretic_activity \
8386 ▷ ▷ ▷ -report \
8387 ▷ ▷ ▷ -report_group_table \
8388 ▷ ▷ -end
8389 ▷ pdflatex PGL_2_3_report.tex
8390 ▷ #$(OPEN) PGL_2_3_report.pdf
8391 ▷ #pdflatex PGL_2_3_group_table_order.24.tex
8392 ▷ #$(OPEN) PGL_2_3_group_table_order.24.pdf
8393 ▷ #$(OPEN) PGL_2_3_report.pdf
8394
8395
8396
8397
8398
8399
8400 #Co3 from Conway et al., 1985 (ATLAS)
8401 #order = 495766656000
8402 #Co3 from the paper by Suleiman and Wilson 1997
8403
8404
8405 Co3:
8406 ▷ $(ORBITER) -v 2 \
8407 ▷ ▷ -define F -finite_field -q 2 -end \
8408 ▷ ▷ -define g1 -vector -field F -format 22 -compact $(CONWAY_GEN1) -end \
8409 ▷ ▷ -define g2 -vector -field F -format 22 -compact $(CONWAY_GEN2) -end \
8410 ▷ ▷ -define gens -vector -concatenate g1,g2 -end \
8411 ▷ ▷ -define G -linear_group -PGL 22 2 \
8412 ▷ ▷ ▷ -subgroup_by_generators "Co3" "495766656000" 2 gens \
8413 ▷ ▷ ▷ -end \
8414 ▷ ▷ -with G -do \
8415 ▷ ▷ -group_theoretic_activity \

```

```

8416 ▷ ▷ ▷ -report \
8417 ▷ ▷ -end
8418 ▷ pdflatex PGL_22_2_Subgroup_Co3_495766656000_report.tex
8419 ▷ #$(OPEN) PGL_22_2_Subgroup_Co3_495766656000_report.pdf
8420
8421 # needs a lot of memory to run!
8422
8423 Ree_27:
8424 ▷ $(ORBITER) -v 2 \
8425 ▷ ▷ -define F -finite_field -q 27 \
8426 ▷ ▷ ▷ -override_polynomial "34" \
8427 ▷ ▷ -end \
8428 ▷ ▷ -define g1 -vector -field F -format 7 -dense $(Ree_gen1) -end \
8429 ▷ ▷ -define g2 -vector -field F -format 7 -dense $(Ree_gen2) -end \
8430 ▷ ▷ -define gens -vector -concatenate g1,g2 -end \
8431 ▷ ▷ -define G -linear_group -PGL 7 F \
8432 ▷ ▷ ▷ -subgroup_by_generators "Ree_27" "10073444472" 2 gens \
8433 ▷ ▷ ▷ -end \
8434 ▷ ▷ -with G -do \
8435 ▷ ▷ -group_theoretic_activity \
8436 ▷ ▷ ▷ -report \
8437 ▷ ▷ -end
8438
8439 # needs a lot of memory to run!
8440
8441
8442
8443
8444
8445 #####
8446 # Chapter 7 - Orbit Algorithms
8447 #####
8448
8449
8450 test_7:
8451 ▷ make test_7.1
8452 ▷ make test_7.2
8453 ▷ make test_7.3
8454 ▷ make test_7.4
8455 ▷ make test_7.5
8456 ▷ make test_7.6
8457 ▷ make test_7.7
8458
8459
8460 #####
8461 # Section 7.1: Orbit Algorithms
8462
8463
8464 SECTION_ORBIT_ALGORITHMS_SCHREIER_TREES:
8465
8466
8467 test_7.1:
8468 ▷ make orbits_PGL_4_2_on_points_draw_tree
8469 ▷ make orbits_PGL_4_2_on_points_export_trees
8470 ▷ make DD_PP4_orbit
8471 ▷ make T3r1_orbits
8472 ▷ #make T3r1_orbits_draw
8473 ▷ make 2C_orbit_under_PGGL_4_4_elements_coded.csv
8474 ▷ make orbits_on_conics_q13

```

```

8475 ▷ make orbits_cubic_curves.q2
8476 ▷ make orbits_cubic_curves.q2_with_draw_tree
8477 ▷ make poly_orbits_d3_n3.q2.csv
8478 ▷ make poly_orbits_d3_n3.q2_get_ranks
8479 ▷ make T4.orbits
8480 ▷ make T4r1.orbits
8481 ▷ make T4r1.orbits_draw
8482 ▷ make T4r1.orbits_4
8483 ▷ make PGL_4.2.on_conic.orbits
8484 ▷ make PGL_7.8.orbits
8485
8486
8487
8488 orbits_PGL_4.2.on_points_draw_tree:
8489 ▷ $(ORBITER) -v 4 \
8490 ▷ ▷ -draw_options -embedded -end \
8491 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
8492 ▷ ▷ -define Orb -orbits -group G \
8493 ▷ ▷ ▷ -on_points \
8494 ▷ ▷ -end \
8495 ▷ ▷ -with Orb -do -orbits_activity \
8496 ▷ ▷ ▷ -report \
8497 ▷ ▷ -end \
8498 ▷ ▷ -with Orb -do -orbits_activity \
8499 ▷ ▷ ▷ -export_something "orbit" 0 \
8500 ▷ ▷ -end
8501 ▷ ▷ -with Orb -do -orbits_activity \
8502 ▷ ▷ ▷ -draw_tree 0 \
8503 ▷ ▷ -end
8504 ▷ pdflatex PGL_4.2.orbits_report.tex
8505 ▷ $(OPEN) PGL_4.2.orbits_report.pdf
8506
8507
8508 orbits_PGL_4.2.on_points_export_trees:
8509 ▷ $(ORBITER) -v 4 \
8510 ▷ ▷ -draw_options -embedded -end \
8511 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
8512 ▷ ▷ -define Orb -orbits -group G \
8513 ▷ ▷ ▷ -on_points \
8514 ▷ ▷ -end \
8515 ▷ ▷ -with Orb -do -orbits_activity \
8516 ▷ ▷ ▷ -report \
8517 ▷ ▷ -end \
8518 ▷ ▷ -with Orb -do -orbits_activity \
8519 ▷ ▷ ▷ -export_trees \
8520 ▷ ▷ -end
8521 ▷ $(ORBITER) -v 3 \
8522 ▷ ▷ -draw_layered_graph \
8523 ▷ ▷ ▷ orbit_PGL_4.2.0.layered_graph \
8524 ▷ ▷ -radius 500 -spanning_tree -embedded \
8525 ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
8526 ▷ ▷ -end
8527 ▷ pdflatex orbit_PGL_4.2.0.draw.tex
8528 ▷ $(OPEN) orbit_PGL_4.2.0.draw.pdf
8529
8530
8531 DD_PP4_orbit:
8532 ▷ $(ORBITER) -v 4 \
8533 ▷ ▷ -define G1 -linear_group -AGL 1 3 \

```

```

8534 ▷ ▷ ▷ -end \
8535 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
8536 ▷ ▷ ▷ -end \
8537 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
8538 ▷ ▷ ▷ "21" "1,1,1,1" \
8539 ▷ ▷ -end \
8540 ▷ ▷ -with G -do \
8541 ▷ ▷ -group_theoretic_activity \
8542 ▷ ▷ ▷ -report \
8543 ▷ ▷ -end \
8544 ▷ ▷ -define S -vector -dense "0,1,3,13,20" -end \
8545 ▷ ▷ -define Orb -orbits -group G \
8546 ▷ ▷ ▷ -of_one_subset S \
8547 ▷ ▷ -end
8548
8549
8550 T3r1_orbits:
8551 ▷ $(ORBITER) -v 4 \
8552 ▷ ▷ -draw_options -embedded -end \
8553 ▷ ▷ -define G \
8554 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
8555 ▷ ▷ ▷ -on_rank_one_tensors -end \
8556 ▷ ▷ -define Orb -orbits -group G \
8557 ▷ ▷ ▷ -on_points \
8558 ▷ ▷ -end \
8559 ▷ ▷ -with Orb -do -orbits_activity \
8560 ▷ ▷ ▷ -report \
8561 ▷ ▷ -end \
8562 ▷ ▷ -with Orb -do -orbits_activity \
8563 ▷ ▷ ▷ -draw_tree 0 \
8564 ▷ ▷ -end
8565 ▷ pdflatex GL_2.2.wreath_Sym3_orbit_0.tree.tex
8566 ▷ $(OPEN) GL_2.2.wreath_Sym3_orbit_0.tree.pdf
8567
8568
8569 # ToDo: layered_graph file is not written
8570
8571 T3r1_orbits_draw:
8572 ▷ $(ORBITER) -v 3 \
8573 ▷ ▷ -draw_layered_graph \
8574 ▷ ▷ ▷ GL_2.2.wreath_Sym3_res27_0.layered_graph \
8575 ▷ ▷ -radius 500 -spanning_tree -embedded \
8576 ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
8577 ▷ ▷ -end
8578 ▷ #pdflatex GL_2.2.wreath_Sym3_report.tex
8579 ▷ #$(OPEN) GL_2.2.wreath_Sym3_report.pdf
8580 ▷ pdflatex GL_2.2.wreath_Sym3_res27_0.draw.tex
8581 ▷ $(OPEN) GL_2.2.wreath_Sym3_res27_0.draw.pdf
8582
8583
8584 # write GL_2.2.wreath_Sym3_res27_0.layered_graph
8585
8586
8587
8588 2C_orbit_under_PGGL_4.4.elements_coded.csv:
8589 ▷ $(ORBITER) -v 6 \
8590 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
8591 ▷ ▷ -with G -do \
8592 ▷ ▷ -group_theoretic_activity \

```

```

8593 ▷ ▷ ▷ -conjugacy_class_of \
8594 ▷ ▷ ▷ "2C" "1,0,0,0, 1,1,0,0, 0,0,1,0, 0,0,1,1, 0" \
8595 ▷ ▷ -end
8596
8597 # class of size 64260
8598 # creates:
8599 # PGGL_4_4.class_of_2C.csv
8600 # 1:33 on Mac
8601 #User time: 2:59 on Mac
8602 # now 1:02 on new Mac
8603
8604
8605
8606
8607 orbits_on_conics_q13:
8608 ▷ $(ORBITER) -v 4 \
8609 ▷ ▷ -define F -finite_field -q 13 -end \
8610 ▷ ▷ -define G -linear_group -PGL 3 F -end \
8611 ▷ ▷ -define R -polynomial_ring \
8612 ▷ ▷ ▷ -field F \
8613 ▷ ▷ ▷ -number_of_variables 3 \
8614 ▷ ▷ ▷ -homogeneous_of_degree 2 \
8615 ▷ ▷ ▷ -monomial_ordering_partition \
8616 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
8617 ▷ ▷ -end \
8618 ▷ ▷ -define Orb -orbits -group G \
8619 ▷ ▷ ▷ -on_polynomials R \
8620 ▷ ▷ -end
8621 ▷ #pdflatex poly_orbits_d2_n2_q13.tex
8622 ▷ #$(OPEN) poly_orbits_d2_n2_q13.pdf
8623
8624
8625 orbits_cubic_curves_q2:
8626 ▷ $(ORBITER) -v 4 \
8627 ▷ ▷ -define F -finite_field -q 2 -end \
8628 ▷ ▷ -define G -linear_group -PGL 3 F -end \
8629 ▷ ▷ -define R -polynomial_ring \
8630 ▷ ▷ ▷ -field F \
8631 ▷ ▷ ▷ -number_of_variables 3 \
8632 ▷ ▷ ▷ -homogeneous_of_degree 3 \
8633 ▷ ▷ ▷ -monomial_ordering_partition \
8634 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
8635 ▷ ▷ -end \
8636 ▷ ▷ -define Orb -orbits -group G \
8637 ▷ ▷ ▷ -on_polynomials R \
8638 ▷ ▷ -end
8639 ▷ #pdflatex poly_orbits_d3_n3_q2.tex
8640 ▷ #$(OPEN) poly_orbits_d3_n3_q2.pdf
8641
8642
8643 orbits_cubic_curves_q2_with_draw_tree:
8644 ▷ $(ORBITER) -v 4 \
8645 ▷ ▷ -draw_options -yout 500000 -radius 150 -nodes_empty \
8646 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
8647 ▷ ▷ -define F -finite_field -q 2 -end \
8648 ▷ ▷ -define G -linear_group -PGL 3 F -end \
8649 ▷ ▷ -define R -polynomial_ring \
8650 ▷ ▷ ▷ -field F \
8651 ▷ ▷ ▷ -number_of_variables 3 \

```

```

8652 ▷ ▷ ▷ -homogeneous_of_degree 3 \
8653 ▷ ▷ ▷ -monomial_ordering_partition \
8654 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
8655 ▷ ▷ -end \
8656 ▷ ▷ -define Orb -orbits -group G \
8657 ▷ ▷ ▷ -on_polynomials R \
8658 ▷ ▷ -end \
8659 ▷ ▷ -with Orb -do -orbits_activity \
8660 ▷ ▷ ▷ -draw_tree 6 \
8661 ▷ ▷ -end
8662 ▷ pdflatex PGL_3.2_orbit_6_tree.tex
8663 ▷ $(OPEN) PGL_3.2_orbit_6_tree.pdf
8664
8665
8666
8667
8668 poly_orbits_d3_n3_q2.csv:
8669 ▷ $(ORBITER) -v 4 \
8670 ▷ ▷ -draw_options -yout 500000 -radius 15 -nodes_empty \
8671 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
8672 ▷ ▷ -define F -finite_field -q 2 -end \
8673 ▷ ▷ -define G -linear_group -PGL 4 F -end \
8674 ▷ ▷ -define R -polynomial_ring \
8675 ▷ ▷ ▷ -field F \
8676 ▷ ▷ ▷ -number_of_variables 4 \
8677 ▷ ▷ ▷ -homogeneous_of_degree 3 \
8678 ▷ ▷ ▷ -monomial_ordering_partition \
8679 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
8680 ▷ ▷ -end \
8681 ▷ ▷ -define Orb -orbits -group G \
8682 ▷ ▷ ▷ -on_polynomials R \
8683 ▷ ▷ -end \
8684 ▷ ▷ -with Orb -do -orbits_activity \
8685 ▷ ▷ ▷ -report \
8686 ▷ ▷ -end \
8687 ▷ ▷ -with Orb -do -orbits_activity \
8688 ▷ ▷ ▷ -draw_tree 6 \
8689 ▷ ▷ -end
8690 ▷ pdflatex PGL_4.2_orbit_6_tree.tex
8691 ▷ $(OPEN) PGL_4.2_orbit_6_tree.pdf
8692
8693 #written file PGL_4.2_orbit_6_tree.tex of size 79891
8694
8695
8696 poly_orbits_d3_n3_q2_get_ranks:
8697 ▷ $(ORBITER) -v 4 \
8698 ▷ ▷ -define cols -vector -dense 0 -end \
8699 ▷ ▷ -csv_file_select_cols poly_orbits_d3_n3_q2.csv "_reps" cols
8700 ▷ #pdflatex poly_orbits_d3_n3_q2.tex
8701 ▷ #$(OPEN) poly_orbits_d3_n3_q2.pdf
8702
8703
8704
8705 T4_orbits:
8706 ▷ $(ORBITER) -v 4 \
8707 ▷ ▷ -define G \
8708 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
8709 ▷ ▷ ▷ -on_tensors -end \
8710 ▷ ▷ -define Orb -orbits -group G \

```

```

8711 ▷ ▷ ▷ -on_points \
8712 ▷ ▷ -end
8713 ▷ #pdflatex GL_2.2_wreath_Sym4_res65535_orbits.tex
8714 ▷ #$(OPEN) GL_2.2_wreath_Sym4_res65535_orbits.pdf
8715 ▷ #pdflatex GL_2.2_wreath_Sym4_report.tex
8716 ▷ #$(OPEN) GL_2.2_wreath_Sym4_report.pdf
8717
8718
8719
8720
8721 T4r1_orbits:
8722 ▷ $(ORBITER) -v 4 \
8723 ▷ ▷ -define G -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
8724 ▷ ▷ ▷ -on_rank_one_tensors -end \
8725 ▷ ▷ -define Orb -orbits -group G \
8726 ▷ ▷ ▷ -on_points \
8727 ▷ ▷ -end \
8728 ▷ ▷ -with Orb -do -orbits.activity \
8729 ▷ ▷ ▷ -export_trees \
8730 ▷ ▷ -end
8731
8732
8733 # problem in export_trees because of f_load_save = true in Algebra.orbits.on_poin
    ts
8734 # solution set f_load_save = false
8735
8736 #orbit_GL_2.2_wreath_Sym4_res.on_rank_one_tensors_0.layered_graph
8737
8738
8739 T4r1_orbits_draw:
8740 ▷ $(ORBITER) -v 3 \
8741 ▷ ▷ -draw_layered_graph \
8742 ▷ ▷ ▷ orbit_GL_2.2_wreath_Sym4_res.on_rank_one_tensors_0.layered_graph \
8743 ▷ ▷ -radius 400 -spanning_tree -embedded \
8744 ▷ ▷ ▷ -line_width 1.1 -x_stretch 2.5 -scale 0.15 \
8745 ▷ ▷ -end
8746 ▷ pdflatex orbit_GL_2.2_wreath_Sym4_res.on_rank_one_tensors_0_draw.tex
8747 ▷ $(OPEN) orbit_GL_2.2_wreath_Sym4_res.on_rank_one_tensors_0_draw.pdf
8748
8749
8750 T4r1_orbits_4:
8751 ▷ $(ORBITER) -v 4 \
8752 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8753 ▷ ▷ -define Control -poset_classification_control \
8754 ▷ ▷ ▷ -problem_label T4r1 -W \
8755 ▷ ▷ ▷ -draw_options -end \
8756 ▷ ▷ -end \
8757 ▷ ▷ -define G -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
8758 ▷ ▷ ▷ -on_rank_one_tensors -end \
8759 ▷ ▷ -define Orb -orbits -group G \
8760 ▷ ▷ ▷ -on_subsets 4 Control \
8761 ▷ ▷ -end \
8762 ▷ ▷ -with Orb -do -orbits.activity \
8763 ▷ ▷ ▷ -report \
8764 ▷ ▷ ▷ -report_options \
8765 ▷ ▷ ▷ -draw_poset -type_ordinary \
8766 ▷ ▷ ▷ -end \
8767 ▷ ▷ -end
8768 ▷ pdflatex T4r1_poset.tex

```

```

8769 ▷ $(OPEN) T4r1_poset.pdf
8770
8771
8772
8773 # ToDo
8774
8775 PGGL_2.8_on_conic_orbits:
8776 ▷ $(ORBITER) -v 4 \
8777 ▷ ▷ -define G \
8778 ▷ ▷ -linear_group -PGGL 2 8 -PGL2OnConic -end \
8779 ▷ ▷ -define Orb -orbits -group G \
8780 ▷ ▷ ▷ -on_points \
8781 ▷ ▷ -end
8782
8783 #any_group::init_linear_group !LG->f_has_strong_generators
8784
8785
8786 # example from the Fining manual, page 107:
8787
8788 PGGL_7.8_orbits:
8789 ▷ $(ORBITER) -v 4 \
8790 ▷ ▷ -define G \
8791 ▷ ▷ -linear_group -PGGL 7 8 -end \
8792 ▷ ▷ -define Orb -orbits -group G \
8793 ▷ ▷ ▷ -on_points \
8794 ▷ ▷ -end
8795
8796 # 0:42
8797
8798
8799
8800
8801
8802
8803 #####
8804 # Section 7.2: Poset Classification
8805
8806
8807 SECTION_POSET_CLASSIFICATION:
8808
8809 test_7.2:
8810 ▷ make poset_of_4subsets
8811 ▷ make poset_of_4subsets.draw
8812 ▷ make poset_of_5subsets
8813 ▷ make poset_of_5subsets.draw
8814 ▷ make Symmetric_4_on_pairs_poset
8815 ▷ make V_3.2_trivial
8816 ▷ make V_4.2_trivial
8817
8818
8819 poset_of_4subsets:
8820 ▷ $(ORBITER) -v 3 \
8821 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8822 ▷ ▷ -define Control -poset_classification_control \
8823 ▷ ▷ ▷ -problem_label poset_4 \
8824 ▷ ▷ ▷ -W -depth 4 \
8825 ▷ ▷ -end \
8826 ▷ ▷ -define G -permutation_group -identity_group 4 -end \
8827 ▷ ▷ -define Orb -orbits -group G \

```



```

8828 ▷ ▷ ▷ -on_subsets 4 Control \
8829 ▷ ▷ -end \
8830 ▷ ▷ -with Orb -do -orbits_activity \
8831 ▷ ▷ ▷ -report \
8832 ▷ ▷ ▷ -report_options \
8833 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
8834 ▷ ▷ ▷ -end \
8835 ▷ ▷ -end
8836 ▷ pdflatex poset_4_poset.tex
8837 ▷ $(OPEN) poset_4_poset.pdf
8838
8839
8840
8841 poset_of_4subsets.draw:
8842 ▷ $(ORBITER) -v 3 \
8843 ▷ ▷ -draw_layered_graph \
8844 ▷ ▷ ▷ poset_4_poset_lvl_4.layered_graph \
8845 ▷ ▷ ▷ -radius 300 -embedded -line_width 1.1 \
8846 ▷ ▷ ▷ -y_stretch 0.9 -scale 0.25 \
8847 ▷ ▷ -end
8848 ▷ pdflatex poset_4_poset_lvl_4.draw.tex
8849 ▷ $(OPEN) poset_4_poset_lvl_4.draw.pdf
8850
8851
8852 poset_of_5subsets:
8853 ▷ $(ORBITER) -v 3 \
8854 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8855 ▷ ▷ -define Control -poset_classification_control \
8856 ▷ ▷ ▷ -problem_label poset_5 \
8857 ▷ ▷ ▷ -W -depth 5 \
8858 ▷ ▷ ▷ -draw_options -radius 150 -end \
8859 ▷ ▷ -end \
8860 ▷ ▷ -define G -permutation_group -identity_group 5 -end \
8861 ▷ ▷ -define Orb -orbits -group G \
8862 ▷ ▷ ▷ -on_subsets 5 Control \
8863 ▷ ▷ -end \
8864 ▷ ▷ -with Orb -do -orbits_activity \
8865 ▷ ▷ ▷ -report \
8866 ▷ ▷ ▷ -report_options \
8867 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
8868 ▷ ▷ ▷ -end \
8869 ▷ ▷ -end
8870 ▷ pdflatex poset_5_poset.tex
8871 ▷ $(OPEN) poset_5_poset.pdf
8872
8873
8874 poset_of_5subsets.draw:
8875 ▷ $(ORBITER) -v 3 \
8876 ▷ ▷ -draw_layered_graph \
8877 ▷ ▷ ▷ poset_5_poset_lvl_5.layered_graph \
8878 ▷ ▷ ▷ -radius 300 -embedded \
8879 ▷ ▷ ▷ -line_width 1.1 -y_stretch 0.9 \
8880 ▷ ▷ ▷ -scale 0.25 \
8881 ▷ ▷ -end
8882 ▷ pdflatex poset_5_poset_lvl_5.draw.tex
8883 ▷ $(OPEN) poset_5_poset_lvl_5.draw.pdf
8884
8885
8886

```

```

8887 Symmetric_4.on.pairs.poset:
8888 ▷ $(ORBITER) -v 3 \
8889 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8890 ▷ ▷ -define Control -poset_classification_control \
8891 ▷ ▷ ▷ -problem_label Sym4.on2 \
8892 ▷ ▷ ▷ -W -depth 6 \
8893 ▷ ▷ ▷ -draw_options -radius 150 -end \
8894 ▷ ▷ -end \
8895 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
8896 ▷ ▷ -define G.on.2subsets -modified_group -from G \
8897 ▷ ▷ ▷ -on_k_subsets 2 \
8898 ▷ ▷ -end \
8899 ▷ ▷ -define Orb -orbits -group G.on.2subsets \
8900 ▷ ▷ ▷ -on_subsets 6 Control \
8901 ▷ ▷ -end \
8902 ▷ ▷ -with Orb -do -orbits_activity \
8903 ▷ ▷ ▷ -report \
8904 ▷ ▷ ▷ -report_options \
8905 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
8906 ▷ ▷ ▷ -end \
8907 ▷ ▷ -end
8908 ▷ pdflatex Sym4.on2.poset.tex
8909 ▷ $(OPEN) Sym4.on2.poset.pdf
8910
8911
8912 V_3_2.trivial:
8913 ▷ $(ORBITER) -v 5 \
8914 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8915 ▷ ▷ -define Control -poset_classification_control \
8916 ▷ ▷ ▷ -problem_label V_3_2.trivial \
8917 ▷ ▷ ▷ -W -depth 3 \
8918 ▷ ▷ ▷ -draw_options \
8919 ▷ ▷ ▷ ▷ -radius 200 -embedded \
8920 ▷ ▷ ▷ -end \
8921 ▷ ▷ -end \
8922 ▷ ▷ -define G -linear_group -PGL 3 2 -identity_group -end \
8923 ▷ ▷ -define Orb -orbits -group G \
8924 ▷ ▷ ▷ -on_subspaces 3 Control \
8925 ▷ ▷ -end \
8926 ▷ ▷ -with Orb -do -orbits_activity \
8927 ▷ ▷ ▷ -report \
8928 ▷ ▷ ▷ -report_options \
8929 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
8930 ▷ ▷ ▷ -end \
8931 ▷ ▷ -end
8932 ▷ pdflatex PGL_3_2.Identity_3_2.poset.tex
8933 ▷ $(OPEN) PGL_3_2.Identity_3_2.poset.pdf
8934
8935
8936
8937 V_4_2.trivial:
8938 ▷ $(ORBITER) -v 5 \
8939 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8940 ▷ ▷ -define Control -poset_classification_control \
8941 ▷ ▷ ▷ -problem_label V_4_2.trivial \
8942 ▷ ▷ ▷ -W -depth 3 \
8943 ▷ ▷ ▷ -draw_options \
8944 ▷ ▷ ▷ ▷ -radius 200 -embedded \
8945 ▷ ▷ ▷ -end \

```

```

8946 ▷ ▷ -end \
8947 ▷ ▷ -define G -linear_group -PGL 4 2 -identity_group -end \
8948 ▷ ▷ -define Orb -orbits -group G \
8949 ▷ ▷ ▷ -on_subspaces 4 Control \
8950 ▷ ▷ -end \
8951 ▷ ▷ -with Orb -do -orbits_activity \
8952 ▷ ▷ ▷ -report \
8953 ▷ ▷ ▷ -report_options \
8954 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
8955 ▷ ▷ ▷ -end \
8956 ▷ ▷ -end
8957
8958
8959
8960 #####
8961 # Section 7.3: Orbits on Subsets
8962
8963
8964 SECTION_ORBITS_ON_SUBSETS:
8965
8966
8967 test_7_3:
8968 ▷ make PG_2_2_subsets
8969 ▷ make PG_3_2_subsets
8970 ▷ make PGL_3_2_singer
8971 ▷ make PGL_3_2_on_lines
8972 ▷ make PGL_2_5_on_subsets
8973 ▷ make PGL_2_7_on_subsets
8974 ▷ make PGGL_2_8_on_subsets
8975 ▷ make PGGL_2_9_on_subsets
8976 ▷ make PGL_2_11_on_subsets
8977 ▷ make PGGL_2_16_on_subsets
8978 ▷ make PGGL_2_32_on_subsets
8979 ▷ make PG_3_4_subsets
8980 ▷ make PGGL_2_9_orbits
8981 ▷ make PG0_5_2_on_subsets
8982
8983
8984
8985 PG_2_2_subsets:
8986 ▷ $(ORBITER) -v 3 \
8987 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8988 ▷ ▷ -define Control -poset_classification_control \
8989 ▷ ▷ ▷ -problem_label PGL_3_2 \
8990 ▷ ▷ ▷ -depth 7 \
8991 ▷ ▷ ▷ -draw_options \
8992 ▷ ▷ ▷ ▷ -radius 200 -embedded \
8993 ▷ ▷ ▷ -end \
8994 ▷ ▷ -end \
8995 ▷ ▷ -define F -finite_field -q 2 -end \
8996 ▷ ▷ -define G -linear_group -PGL 3 F -end \
8997 ▷ ▷ -define Orb -orbits -group G \
8998 ▷ ▷ ▷ -on_subsets 7 Control \
8999 ▷ ▷ -end \
9000 ▷ ▷ -with Orb -do -orbits_activity \
9001 ▷ ▷ ▷ -report \
9002 ▷ ▷ ▷ -report_options \
9003 ▷ ▷ ▷ ▷ -draw_poset -type_aux \
9004 ▷ ▷ ▷ -end \

```

```

9005 ▷ ▷ -end \
9006 ▷ ▷ -with G -do \
9007 ▷ ▷ -group_theoretic_activity \
9008 ▷ ▷ ▷ -stats "PG_2_2_subsets" \
9009 ▷ ▷ -end
9010 ▷ pdfflatex PGL_3_2_poset_lvl_7.tex
9011 ▷ $(OPEN) PGL_3_2_poset_lvl_7.pdf
9012 ▷ pdfflatex PGL_3_2_poset.tex
9013 ▷ $(OPEN) PGL_3_2_poset.pdf
9014
9015
9016 # PG(3,2) has  $2^3+2^2+2^1+1 = 15$  points:
9017 # PG(3,3) has  $3^3+3^2+3^1+1 = 27 + 9 + 3 + 1 = 40$  points.
9018
9019
9020
9021
9022
9023 PG_3_2_subsets:
9024 ▷ $(ORBITER) -v 3 \
9025 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9026 ▷ ▷ -define Control -poset_classification_control \
9027 ▷ ▷ ▷ -problem_label PGL_4_2 \
9028 ▷ ▷ ▷ -depth 15 \
9029 ▷ ▷ ▷ -draw_options \
9030 ▷ ▷ ▷ ▷ -radius 200 -embedded \
9031 ▷ ▷ ▷ -end \
9032 ▷ ▷ -end \
9033 ▷ ▷ -define F -finite_field -q 2 -end \
9034 ▷ ▷ -define G -linear_group -PGL 4 F -end \
9035 ▷ ▷ -define Orb -orbits -group G \
9036 ▷ ▷ ▷ -on_subsets 15 Control \
9037 ▷ ▷ -end \
9038 ▷ ▷ -with Orb -do -orbits_activity \
9039 ▷ ▷ ▷ -report \
9040 ▷ ▷ ▷ -report_options \
9041 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9042 ▷ ▷ ▷ -end \
9043 ▷ ▷ -end
9044 ▷ pdfflatex PGL_4_2_poset.tex
9045 ▷ $(OPEN) PGL_4_2_poset.pdf
9046
9047 PGL_3_2_singer:
9048 ▷ $(ORBITER) -v 3 \
9049 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9050 ▷ ▷ -define Control -poset_classification_control \
9051 ▷ ▷ ▷ -problem_label PGL_3_2_singer_1 \
9052 ▷ ▷ ▷ -W -depth 7 \
9053 ▷ ▷ ▷ -draw_options \
9054 ▷ ▷ ▷ ▷ -radius 200 -embedded \
9055 ▷ ▷ ▷ -end \
9056 ▷ ▷ -end \
9057 ▷ ▷ -define G -linear_group -PGL 3 2 -singer 1 -end \
9058 ▷ ▷ -define Orb -orbits -group G \
9059 ▷ ▷ ▷ -on_subsets 7 Control \
9060 ▷ ▷ -end \
9061 ▷ ▷ -with Orb -do -orbits_activity \
9062 ▷ ▷ ▷ -report \
9063 ▷ ▷ ▷ -report_options \

```

```

9064 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9065 ▷ ▷ ▷ -end \
9066 ▷ ▷ -end
9067 ▷ pdflatex PGL_3.2_singer_1_poset.tex
9068 ▷ $(OPEN) PGL_3.2_singer_1_poset.pdf
9069
9070
9071
9072 PGL_3.2_on_lines:
9073 ▷ $(ORBITER) -v 3 \
9074 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9075 ▷ ▷ -define Control -poset_classification_control \
9076 ▷ ▷ ▷ -problem_label PGL_3.2_lines \
9077 ▷ ▷ ▷ -W -depth 7 \
9078 ▷ ▷ -end \
9079 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
9080 ▷ ▷ -define G_on_lines -modified_group -from G \
9081 ▷ ▷ ▷ -on_k_subspaces 2 \
9082 ▷ ▷ -end \
9083 ▷ ▷ -define Orb -orbits -group G_on_lines \
9084 ▷ ▷ ▷ -on_subsets 7 Control \
9085 ▷ ▷ -end \
9086 ▷ ▷ -with Orb -do -orbits_activity \
9087 ▷ ▷ ▷ -report \
9088 ▷ ▷ ▷ -report_options \
9089 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9090 ▷ ▷ ▷ -end \
9091 ▷ ▷ -end
9092 ▷ pdflatex PGL_3.2_lines_poset.tex
9093 ▷ $(OPEN) PGL_3.2_lines_poset.pdf
9094
9095
9096
9097 PGL_2.5_on_subsets:
9098 ▷ $(ORBITER) -v 5 \
9099 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9100 ▷ ▷ -define Control -poset_classification_control \
9101 ▷ ▷ ▷ -problem_label PGL_2.5 \
9102 ▷ ▷ ▷ -W -depth 6 \
9103 ▷ ▷ -end \
9104 ▷ ▷ -define G -linear_group -PGL 2 5 -end \
9105 ▷ ▷ -define Orb -orbits -group G \
9106 ▷ ▷ ▷ -on_subsets 6 Control \
9107 ▷ ▷ -end \
9108 ▷ ▷ -with Orb -do -orbits_activity \
9109 ▷ ▷ ▷ -report \
9110 ▷ ▷ ▷ -report_options \
9111 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9112 ▷ ▷ ▷ -end \
9113 ▷ ▷ -end
9114 ▷ pdflatex PGL_2.5_poset.tex
9115 ▷ $(OPEN) PGL_2.5_poset.pdf
9116
9117 PGL_2.7_on_subsets:
9118 ▷ $(ORBITER) -v 10 \
9119 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9120 ▷ ▷ -define Control -poset_classification_control \
9121 ▷ ▷ ▷ -problem_label PGL_2.7 \
9122 ▷ ▷ ▷ -W -depth 8 \

```

```

9123 ▷ ▷ -end \
9124 ▷ ▷ -define G -linear_group -PGL 2 7 -end \
9125 ▷ ▷ -define Orb -orbits -group G \
9126 ▷ ▷ ▷ -on_subsets 8 Control \
9127 ▷ ▷ -end \
9128 ▷ ▷ -with Orb -do -orbits_activity \
9129 ▷ ▷ ▷ -report \
9130 ▷ ▷ ▷ -report_options \
9131 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9132 ▷ ▷ ▷ -end \
9133 ▷ ▷ -end
9134 ▷ pdflatex PGL_2.7_poset.tex
9135 ▷ $(OPEN) PGL_2.7_poset.pdf
9136
9137 PGGL_2.8_on_subsets:
9138 ▷ $(ORBITER) -v 10 \
9139 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9140 ▷ ▷ -define Control -poset_classification_control \
9141 ▷ ▷ ▷ -problem_label PGGL_2.8 \
9142 ▷ ▷ ▷ -W -depth 9 \
9143 ▷ ▷ -end \
9144 ▷ ▷ -define G -linear_group -PGGL 2 8 -end \
9145 ▷ ▷ -define Orb -orbits -group G \
9146 ▷ ▷ ▷ -on_subsets 9 Control \
9147 ▷ ▷ -end \
9148 ▷ ▷ -with Orb -do -orbits_activity \
9149 ▷ ▷ ▷ -report \
9150 ▷ ▷ ▷ -report_options \
9151 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9152 ▷ ▷ ▷ -end \
9153 ▷ ▷ -end
9154 ▷ pdflatex PGGL_2.8_poset.tex
9155 ▷ $(OPEN) PGGL_2.8_poset.pdf
9156
9157
9158 PGGL_2.9_on_subsets:
9159 ▷ $(ORBITER) -v 10 \
9160 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9161 ▷ ▷ -define Control -poset_classification_control \
9162 ▷ ▷ ▷ -problem_label PGGL_2.9 \
9163 ▷ ▷ ▷ -W -depth 10 \
9164 ▷ ▷ -end \
9165 ▷ ▷ -define G -linear_group -PGGL 2 9 -end \
9166 ▷ ▷ -define Orb -orbits -group G \
9167 ▷ ▷ ▷ -on_subsets 10 Control \
9168 ▷ ▷ -end \
9169 ▷ ▷ -with Orb -do -orbits_activity \
9170 ▷ ▷ ▷ -report \
9171 ▷ ▷ ▷ -report_options \
9172 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9173 ▷ ▷ ▷ -end \
9174 ▷ ▷ -end
9175 ▷ pdflatex PGGL_2.9_poset.tex
9176 ▷ $(OPEN) PGGL_2.9_poset.pdf
9177
9178
9179 PGL_2.11_on_subsets:
9180 ▷ $(ORBITER) -v 10 \
9181 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \

```

```

9182 ▷ ▷ -define Control -poset_classification_control \
9183 ▷ ▷ ▷ -problem_label PGL_2.11 \
9184 ▷ ▷ ▷ -W -depth 12 \
9185 ▷ ▷ -end \
9186 ▷ ▷ -define G -linear_group -PGL 2 11 -end \
9187 ▷ ▷ -define Orb -orbits -group G \
9188 ▷ ▷ ▷ -on_subsets 12 Control \
9189 ▷ ▷ -end \
9190 ▷ ▷ -with Orb -do -orbits_activity \
9191 ▷ ▷ ▷ -report \
9192 ▷ ▷ ▷ -report_options \
9193 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9194 ▷ ▷ ▷ -end \
9195 ▷ ▷ -end
9196 ▷ pdflatex PGL_2.11_poset.tex
9197 ▷ $(OPEN) PGL_2.11_poset.pdf
9198
9199
9200
9201 PGGL_2.16.on_subsets:
9202 ▷ $(ORBITER) -v 3 \
9203 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9204 ▷ ▷ -define Control -poset_classification_control \
9205 ▷ ▷ ▷ -problem_label PGGL_2.16 \
9206 ▷ ▷ ▷ -W -depth 10 \
9207 ▷ ▷ -end \
9208 ▷ ▷ -define G -linear_group -PGGL 2 16 -end \
9209 ▷ ▷ -define Orb -orbits -group G \
9210 ▷ ▷ ▷ -on_subsets 10 Control \
9211 ▷ ▷ -end \
9212 ▷ ▷ -with Orb -do -orbits_activity \
9213 ▷ ▷ ▷ -report \
9214 ▷ ▷ ▷ -report_options \
9215 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9216 ▷ ▷ ▷ -end \
9217 ▷ ▷ -end
9218 ▷ pdflatex PGGL_2.16_poset.tex
9219 ▷ $(OPEN) PGGL_2.16_poset.pdf
9220
9221
9222 PGGL_2.32.on_subsets:
9223 ▷ $(ORBITER) -v 3 \
9224 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9225 ▷ ▷ -define Control -poset_classification_control \
9226 ▷ ▷ ▷ -problem_label PGGL_2.32 \
9227 ▷ ▷ ▷ -W -depth 8 \
9228 ▷ ▷ -end \
9229 ▷ ▷ -define G -linear_group -PGGL 2 32 -end \
9230 ▷ ▷ -define Orb -orbits -group G \
9231 ▷ ▷ ▷ -on_subsets 8 Control \
9232 ▷ ▷ -end \
9233 ▷ ▷ -with Orb -do -orbits_activity \
9234 ▷ ▷ ▷ -report \
9235 ▷ ▷ ▷ -report_options \
9236 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9237 ▷ ▷ ▷ -end \
9238 ▷ ▷ -end
9239 ▷ pdflatex PGGL_2.32_poset.tex
9240 ▷ $(OPEN) PGGL_2.32_poset.pdf

```

```

9241
9242
9243 PG_3_4.subsets:
9244 ▷ $(ORBITER) -v 3 \
9245 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9246 ▷ ▷ -define Control -poset_classification_control \
9247 ▷ ▷ ▷ -problem_label PGGL_4.4 \
9248 ▷ ▷ ▷ -depth 5 \
9249 ▷ ▷ ▷ -draw_options \
9250 ▷ ▷ ▷ ▷ -radius 200 \
9251 ▷ ▷ ▷ -end \
9252 ▷ ▷ -end \
9253 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
9254 ▷ ▷ -define Orb -orbits -group G \
9255 ▷ ▷ ▷ -on_subsets 5 Control \
9256 ▷ ▷ -end \
9257 ▷ ▷ -with Orb -do -orbits_activity \
9258 ▷ ▷ ▷ -report \
9259 ▷ ▷ ▷ -report_options \
9260 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9261 ▷ ▷ ▷ -end \
9262 ▷ ▷ -end
9263 ▷ pdflatex PGGL_4.4_poset.tex
9264 ▷ $(OPEN) PGGL_4.4_poset.pdf
9265
9266
9267 PGGL_2_9.orbits:
9268 ▷ $(ORBITER) -v 3 \
9269 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9270 ▷ ▷ -define Control -poset_classification_control \
9271 ▷ ▷ ▷ -problem_label PGGL_2_9 -W -depth 5 \
9272 ▷ ▷ ▷ -draw_options -radius 200 -end \
9273 ▷ ▷ -end \
9274 ▷ ▷ -define G -linear_group -PGGL 2 9 -end \
9275 ▷ ▷ -define Orb -orbits -group G \
9276 ▷ ▷ ▷ -on_subsets 5 Control \
9277 ▷ ▷ -end \
9278 ▷ ▷ -with Orb -do -orbits_activity \
9279 ▷ ▷ ▷ -report \
9280 ▷ ▷ ▷ -report_options \
9281 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9282 ▷ ▷ ▷ -end \
9283 ▷ ▷ -end
9284 ▷ pdflatex PGGL_2_9_poset.tex
9285 ▷ $(OPEN) PGGL_2_9_poset.pdf
9286
9287
9288
9289
9290 PGO_5_2.on_subsets:
9291 ▷ $(ORBITER) -v 3 \
9292 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9293 ▷ ▷ -define Control -poset_classification_control \
9294 ▷ ▷ ▷ -problem_label PGO_5.2 \
9295 ▷ ▷ ▷ -depth 15 \
9296 ▷ ▷ ▷ -w \
9297 ▷ ▷ -end \
9298 ▷ ▷ -define F -finite_field -q 2 -end \
9299 ▷ ▷ -define G -linear_group -PGO 5 F -end \

```



```

9300 ▷ ▷ -define Orb -orbits -group G \
9301 ▷ ▷ ▷ -on_subsets 15 Control \
9302 ▷ ▷ -end \
9303 ▷ ▷ -with Orb -do -orbits_activity \
9304 ▷ ▷ ▷ -report \
9305 ▷ ▷ ▷ -report_options \
9306 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9307 ▷ ▷ ▷ -end \
9308 ▷ ▷ -end
9309 ▷ pdflatex PGO.5.2_poset.tex
9310 ▷ $(OPEN) PGO.5.2_poset.pdf
9311
9312
9313
9314
9315 #####
9316 # Section 7.4: Orbits on Subspaces
9317
9318
9319 SECTION_ORBITS_ON_SUBSPACES:
9320
9321
9322 test_7.4:
9323 ▷ make PGL_5.3_wedge_subspace_orbits
9324 ▷ make subspaces_Op_4.2
9325 ▷ make PGL_4.2_on_subspaces
9326 ▷ make PGL_4.2_singer_on_subspaces
9327 ▷ make PGL_8.2_singer_on_subspaces
9328 ▷ make Op_6.2_orbits_on_subspaces
9329 ▷ make Op_6.3_orbits_on_subspaces
9330 ▷ make Op_6.11_orbits_on_subspaces
9331 ▷ make Op_8.2_orbits_on_subspaces
9332 ▷ make PGO_7.2_on_subspaces
9333
9334
9335
9336 PGL_5.3_wedge_subspace_orbits:
9337 ▷ $(ORBITER) -v 3 \
9338 ▷ ▷ -draw_options -radius 75 -embedded -nodes_empty -end \
9339 ▷ ▷ -define G -linear_group -PGL 5 3 -end \
9340 ▷ ▷ -define G2 -modified_group -from G \
9341 ▷ ▷ ▷ -on_wedge_product \
9342 ▷ ▷ -end \
9343 ▷ ▷ -define Control -poset_classification_control \
9344 ▷ ▷ ▷ -draw_options -radius 200 -embedded -end \
9345 ▷ ▷ ▷ -problem_label PGL_5.3_wedge -W -depth 5 \
9346 ▷ ▷ -end \
9347 ▷ ▷ -define Orb -orbits -group G2 \
9348 ▷ ▷ ▷ -on_subspaces 5 Control \
9349 ▷ ▷ -end \
9350 ▷ ▷ -with Orb -do -orbits_activity \
9351 ▷ ▷ ▷ -draw_tree 0 \
9352 ▷ ▷ -end
9353 ▷ pdflatex PGL_5.3_OnWedge_tree_lvl1.5.tex
9354 ▷ $(OPEN) PGL_5.3_OnWedge_tree_lvl1.5.pdf
9355
9356
9357
9358

```

```

9359 subspaces_Op_4.2:
9360 ▷ $(ORBITER) -v 5 \
9361 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9362 ▷ ▷ -define Control -poset_classification_control \
9363 ▷ ▷ ▷ -draw_options -radius 200 -end \
9364 ▷ ▷ ▷ -problem_label Op_4.2 -W -depth 4 \
9365 ▷ ▷ -end \
9366 ▷ ▷ -define G -linear_group -PGL 4 2 -orthogonal 1 -end \
9367 ▷ ▷ -define Orb -orbits -group G \
9368 ▷ ▷ ▷ -on_subspaces 4 Control \
9369 ▷ ▷ -end \
9370 ▷ ▷ -with Orb -do -orbits_activity \
9371 ▷ ▷ ▷ -report \
9372 ▷ ▷ ▷ -report_options \
9373 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9374 ▷ ▷ ▷ -end \
9375 ▷ ▷ -end
9376 ▷ pdflatex PGL_4.2_Orthogonal_plus_4.2_poset.tex
9377 ▷ $(OPEN) PGL_4.2_Orthogonal_plus_4.2_poset.pdf
9378
9379
9380
9381
9382
9383 PGL_4.2_on_subspaces:
9384 ▷ $(ORBITER) -v 5 \
9385 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9386 ▷ ▷ -define Control -poset_classification_control \
9387 ▷ ▷ ▷ -problem_label PGL_4.2 \
9388 ▷ ▷ ▷ -W -depth 4 \
9389 ▷ ▷ -end \
9390 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
9391 ▷ ▷ -define Orb -orbits -group G \
9392 ▷ ▷ ▷ -on_subspaces 4 Control \
9393 ▷ ▷ -end \
9394 ▷ ▷ -with Orb -do -orbits_activity \
9395 ▷ ▷ ▷ -report \
9396 ▷ ▷ ▷ -report_options \
9397 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9398 ▷ ▷ ▷ -end \
9399 ▷ ▷ -end
9400 ▷ pdflatex PGL_4.2_poset.tex
9401 ▷ $(OPEN) PGL_4.2_poset.pdf
9402
9403
9404
9405
9406 PGL_4.2_singer_on_subspaces:
9407 ▷ $(ORBITER) -v 5 \
9408 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9409 ▷ ▷ -define Control -poset_classification_control \
9410 ▷ ▷ ▷ -draw_options -end \
9411 ▷ ▷ ▷ -problem_label PGL_4.2_singer \
9412 ▷ ▷ ▷ -W -depth 4 \
9413 ▷ ▷ -end \
9414 ▷ ▷ -define G -linear_group -PGL 4 2 -singer 1 -end \
9415 ▷ ▷ -define Orb -orbits -group G \
9416 ▷ ▷ ▷ -on_subspaces 4 Control \
9417 ▷ ▷ -end \

```

```

9418 ▷ ▷ -with Orb -do -orbits.activity \
9419 ▷ ▷ ▷ -report \
9420 ▷ ▷ ▷ -report_options \
9421 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9422 ▷ ▷ ▷ -end \
9423 ▷ ▷ -end
9424 ▷ pdflatex PGL_4.2.Singer_4.2.1.poset.tex
9425 ▷ $(OPEN) PGL_4.2.Singer_4.2.1.poset.pdf
9426
9427
9428
9429
9430 PGL_8.2.singer_on_subspaces:
9431 ▷ $(ORBITER) -v 5 \
9432 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9433 ▷ ▷ -define Control -poset_classification_control \
9434 ▷ ▷ ▷ -draw_options -radius 150 -end \
9435 ▷ ▷ ▷ -problem_label PGL_8.2.singer \
9436 ▷ ▷ ▷ -W -depth 8 \
9437 ▷ ▷ -end \
9438 ▷ ▷ -define G -linear_group -PGL 8 2 -singer 1 -end \
9439 ▷ ▷ -define Orb -orbits -group G \
9440 ▷ ▷ ▷ -on_subspaces 8 Control \
9441 ▷ ▷ -end \
9442 ▷ ▷ -with Orb -do -orbits.activity \
9443 ▷ ▷ ▷ -report \
9444 ▷ ▷ ▷ -report_options \
9445 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9446 ▷ ▷ ▷ -end \
9447 ▷ ▷ -end
9448
9449 # May 7, 2020: 16 sec on Mac
9450 # 1643 orbits in total
9451
9452 Op_6.2.orbits_on_subspaces:
9453 ▷ $(ORBITER) -v 5 \
9454 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9455 ▷ ▷ -define Control -poset_classification_control \
9456 ▷ ▷ ▷ -draw_options -radius 200 -end \
9457 ▷ ▷ ▷ -problem_label Op_6.2 -W \
9458 ▷ ▷ ▷ -depth 6 \
9459 ▷ ▷ -end \
9460 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
9461 ▷ ▷ -define Orb -orbits -group G \
9462 ▷ ▷ ▷ -on_subspaces 6 Control \
9463 ▷ ▷ -end
9464
9465
9466
9467 Op_6.3.orbits_on_subspaces:
9468 ▷ $(ORBITER) -v 5 \
9469 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9470 ▷ ▷ -define Control -poset_classification_control \
9471 ▷ ▷ ▷ -draw_options -radius 200 -end \
9472 ▷ ▷ ▷ -problem_label Op_6.3 -W \
9473 ▷ ▷ ▷ -depth 6 \
9474 ▷ ▷ -end \
9475 ▷ ▷ -define G -linear_group -PGL 6 3 -orthogonal 1 -end \
9476 ▷ ▷ -define Orb -orbits -group G \

```

```

9477 ▷ ▷ ▷ -on_subspaces 6 Control \
9478 ▷ ▷ -end
9479
9480 # June 3, 2020 on Mac: 0 sec
9481
9482
9483
9484 Op_6_11_orbits_on_subspaces:
9485 ▷ $(ORBITER) -v 5 \
9486 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9487 ▷ ▷ -define Control -poset_classification_control \
9488 ▷ ▷ ▷ -draw_options -radius 200 -end \
9489 ▷ ▷ ▷ -problem_label Op_6_11 -W \
9490 ▷ ▷ ▷ -depth 6 \
9491 ▷ ▷ -end \
9492 ▷ ▷ -define G -linear_group -PGL 6 11 -orthogonal 1 -end \
9493 ▷ ▷ -define Orb -orbits -group G \
9494 ▷ ▷ ▷ -on_subspaces 6 Control \
9495 ▷ ▷ -end
9496
9497
9498 # June 3, 2020 on Mac: 12 sec
9499
9500
9501 Op_8_2_orbits_on_subspaces:
9502 ▷ $(ORBITER) -v 5 \
9503 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9504 ▷ ▷ -define Control -poset_classification_control \
9505 ▷ ▷ ▷ -draw_options -radius 200 -end \
9506 ▷ ▷ ▷ -problem_label Op_8_2 -W -depth 8 \
9507 ▷ ▷ -end \
9508 ▷ ▷ -define G -linear_group -PGL 8 2 -orthogonal 1 -end \
9509 ▷ ▷ -define Orb -orbits -group G \
9510 ▷ ▷ ▷ -on_subspaces 8 Control \
9511 ▷ ▷ -end \
9512 ▷ ▷ -with Orb -do -orbits_activity \
9513 ▷ ▷ ▷ -report \
9514 ▷ ▷ ▷ -report_options \
9515 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
9516 ▷ ▷ ▷ -end \
9517 ▷ ▷ -end
9518 ▷ pdflatex PGL_8_2_Orthogonal_plus_8_2_poset.tex
9519 ▷ $(OPEN) PGL_8_2_Orthogonal_plus_8_2_poset.pdf
9520
9521
9522
9523 PG0_7_2_on_subspaces:
9524 ▷ $(ORBITER) -v 6 \
9525 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9526 ▷ ▷ -define Control -poset_classification_control \
9527 ▷ ▷ ▷ -draw_options -radius 200 -end \
9528 ▷ ▷ ▷ -problem_label 0_7_2 \
9529 ▷ ▷ ▷ -W -depth 7 \
9530 ▷ ▷ -end \
9531 ▷ ▷ -define F -finite_field -q 2 -end \
9532 ▷ ▷ -define G -linear_group -PGL 7 F -orthogonal 0 -end \
9533 ▷ ▷ -define Orb -orbits -group G \
9534 ▷ ▷ ▷ -on_subspaces 7 Control \
9535 ▷ ▷ -end

```

```

9536
9537
9538
9539 #####
9540 # Section 7.5: Orbits on set partitions
9541
9542
9543 SECTION_ORBITS_ON_SET_PARTITIONS:
9544
9545
9546
9547 test_7.5:
9548   ▷ make C6_on_partition
9549   ▷ make PGL_2_17_on_partition
9550
9551
9552 C6_on_partition:
9553   ▷ $(ORBITER) -v 5 \
9554   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9555   ▷ ▷ -define Control -poset_classification_control \
9556   ▷ ▷ ▷ -problem_label C6 \
9557   ▷ ▷ ▷ -depth 2 \
9558   ▷ ▷ ▷ -W \
9559   ▷ ▷ ▷ -draw_options \
9560   ▷ ▷ ▷ ▷ -radius 200 -embedded \
9561   ▷ ▷ ▷ -end \
9562   ▷ ▷ -end \
9563   ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
9564   ▷ ▷ -define Orb -orbits -group G \
9565   ▷ ▷ ▷ -on_partition 2 Control \
9566   ▷ ▷ -end
9567
9568
9569 PGL_2_17_on_partition:
9570   ▷ $(ORBITER) -v 5 \
9571   ▷ ▷ -define Control -poset_classification_control \
9572   ▷ ▷ ▷ -problem_label PGL_2_17 \
9573   ▷ ▷ ▷ -depth 6 \
9574   ▷ ▷ ▷ -W \
9575   ▷ ▷ -end \
9576   ▷ ▷ -define G -linear_group -PGL 2 17 -end \
9577   ▷ ▷ -define Orb -orbits -group G \
9578   ▷ ▷ ▷ -on_partition 6 Control \
9579   ▷ ▷ -end
9580
9581
9582
9583 #####
9584 # Section 7.6: Arcs and Caps in Projective Spaces
9585
9586
9587 SECTION_ARCS_AND_CAPS_IN_PROJECTIVE_SPACES:
9588
9589
9590
9591 test_7.6:
9592   ▷ make PGL_3_27
9593   ▷ make hyperoval_4_classify
9594   ▷ make hyperoval_8_classify

```

```

9595 ▷ make hyperoval_16_classify
9596 ▷ make hyperoval_16_1_conic_type
9597 ▷ make hyperoval_16_1_nonconical_type
9598 ▷ make hyperoval_16_2_nonconical_type
9599 ▷ #make hyperoval_16_stab_0_disjoint_sets_graph
9600 ▷ #make nc_arcs_16
9601 ▷ make five_arcs_q13
9602
9603
9604
9605
9606 PGL_3_27:
9607 ▷ $(ORBITER) -v 5 \
9608 ▷ ▷ -define G \
9609 ▷ ▷ -linear_group -PGL 3 27 -end \
9610 ▷ ▷ -with G -do \
9611 ▷ ▷ -group_theoretic_activity \
9612 ▷ ▷ ▷ -report \
9613 ▷ ▷ -end
9614 ▷ pdflatex PGL_3_27_report.tex
9615 ▷ $(OPEN) PGL_3_27_report.pdf
9616 ▷
9617
9618
9619
9620
9621 hyperoval_4_classify:
9622 ▷ $(ORBITER) -v 4 \
9623 ▷ ▷ -define F -finite_field -q 4 -end \
9624 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9625 ▷ ▷ -define Control -poset_classification_control \
9626 ▷ ▷ ▷ -problem_label hyperoval_q4 \
9627 ▷ ▷ ▷ -W -depth 6 \
9628 ▷ ▷ -end \
9629 ▷ ▷ -with P -do \
9630 ▷ ▷ -projective_space_activity \
9631 ▷ ▷ ▷ -classify_arcs \
9632 ▷ ▷ ▷ ▷ -control Control \
9633 ▷ ▷ ▷ ▷ -target_size 6 \
9634 ▷ ▷ ▷ ▷ -d 2 \
9635 ▷ ▷ ▷ -end \
9636 ▷ ▷ -end
9637 ▷ #pdflatex hyperoval_q4_poset.tex
9638 ▷ #$(OPEN) hyperoval_q4_poset.pdf
9639
9640
9641
9642
9643 hyperoval_8_classify:
9644 ▷ $(ORBITER) -v 4 \
9645 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9646 ▷ ▷ -define F -finite_field -q 8 -end \
9647 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9648 ▷ ▷ -define Control -poset_classification_control \
9649 ▷ ▷ ▷ -problem_label hyperoval_q8 \
9650 ▷ ▷ ▷ -W -depth 10 \
9651 ▷ ▷ ▷ -draw_options \
9652 ▷ ▷ ▷ ▷ -radius 200 \
9653 ▷ ▷ ▷ -end \

```

```

9654 ▷ ▷ -end \
9655 ▷ ▷ -with P -do \
9656 ▷ ▷ -projective_space_activity \
9657 ▷ ▷ ▷ -classify_arcs \
9658 ▷ ▷ ▷ ▷ -control Control \
9659 ▷ ▷ ▷ ▷ -target_size 10 \
9660 ▷ ▷ ▷ ▷ -d 2 \
9661 ▷ ▷ ▷ -end \
9662 ▷ ▷ -end
9663 ▷ #pdflatex hyperoval.q8_poset.tex
9664 ▷ #$(OPEN) hyperoval.q8_poset.pdf
9665
9666
9667
9668
9669
9670
9671 hyperoval_16_classify:
9672 ▷ $(ORBITER) -v 4 \
9673 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9674 ▷ ▷ -define F -finite_field -q 16 -end \
9675 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9676 ▷ ▷ -define Control -poset_classification_control \
9677 ▷ ▷ ▷ -problem_label hyperoval.q16 \
9678 ▷ ▷ ▷ -W -depth 18 \
9679 ▷ ▷ ▷ -draw_options \
9680 ▷ ▷ ▷ ▷ -radius 200 \
9681 ▷ ▷ ▷ -end \
9682 ▷ ▷ -end \
9683 ▷ ▷ -with P -do \
9684 ▷ ▷ -projective_space_activity \
9685 ▷ ▷ ▷ -classify_arcs \
9686 ▷ ▷ ▷ ▷ -control Control \
9687 ▷ ▷ ▷ ▷ -target_size 18 \
9688 ▷ ▷ ▷ ▷ -d 2 \
9689 ▷ ▷ ▷ -end \
9690 ▷ ▷ -end
9691
9692
9693 ▷ #pdflatex hyperoval.q16_poset.tex
9694 ▷ #$(OPEN) hyperoval.q16_poset.pdf
9695
9696
9697
9698
9699
9700 hyperoval_16_1_conic_type:
9701 ▷ $(ORBITER) -v 2 \
9702 ▷ ▷ -define F -finite_field -q 16 -end \
9703 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9704 ▷ ▷ -define H_16_1 -geometric_object P \
9705 ▷ ▷ ▷ -set "hyperoval_16_144" "hyperoval\_16\_144" \
9706 ▷ ▷ ▷ $(HYPEROVAL_16_144) \
9707 ▷ ▷ -end \
9708 ▷ ▷ -with H_16_1 -do \
9709 ▷ ▷ -combinatorial_object_activity \
9710 ▷ ▷ ▷ -save \
9711 ▷ ▷ -end \
9712 ▷ ▷ -with H_16_1 -do \

```

```

9713 ▷ ▷ -combinatorial_object_activity \
9714 ▷ ▷ ▷ -conic_type 6 \
9715 ▷ ▷ -end \
9716 ▷ ▷ -print_symbols
9717
9718 hyperoval_16_1_nonconical_type:
9719 ▷ $(ORBITER) -v 2 \
9720 ▷ ▷ -define F -finite_field -q 16 -end \
9721 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9722 ▷ ▷ -define H_16_1 -geometric_object P \
9723 ▷ ▷ ▷ -set "hyperoval_16_144" "hyperoval\_16\_144" \
9724 ▷ ▷ ▷ $(HYPEROVAL_16_144) \
9725 ▷ ▷ -end \
9726 ▷ ▷ -with H_16_1 -do \
9727 ▷ ▷ -combinatorial_object_activity \
9728 ▷ ▷ ▷ -save \
9729 ▷ ▷ -end \
9730 ▷ ▷ -with H_16_1 -do \
9731 ▷ ▷ -combinatorial_object_activity \
9732 ▷ ▷ ▷ -non_conical_type \
9733 ▷ ▷ -end \
9734 ▷ ▷ -print_symbols
9735
9736
9737 #We found 17028 non-conical 6 subsets
9738 #Eckardt point number distribution : $13^{252},\,$ $9^{720},\,$ $5^{2304},\,$ $3^{\{13752\}}$
9739
9740
9741 hyperoval_16_2_nonconical_type:
9742 ▷ $(ORBITER) -v 2 \
9743 ▷ ▷ -define F -finite_field -q 16 -end \
9744 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9745 ▷ ▷ -define H_16_2 -geometric_object P \
9746 ▷ ▷ ▷ -set "hyperoval_16_16320" "hyperoval\_16\_16320" \
9747 ▷ ▷ ▷ $(HYPEROVAL_16_16320) \
9748 ▷ ▷ -end \
9749 ▷ ▷ -with H_16_2 -do \
9750 ▷ ▷ -combinatorial_object_activity \
9751 ▷ ▷ ▷ -save \
9752 ▷ ▷ -end \
9753 ▷ ▷ -with H_16_2 -do \
9754 ▷ ▷ -combinatorial_object_activity \
9755 ▷ ▷ ▷ -non_conical_type \
9756 ▷ ▷ -end \
9757 ▷ ▷ -print_symbols
9758
9759 #We found 6188 = {17 choose 5} non-conical 6 subsets
9760 #Eckardt point number distribution : $45^{68},\,$ $13^{2040},\,$ $5^{4080}$
9761
9762 #neighbors_of_0_with_4_removed.csv
9763 #Row,C0,C1,C2,C3
9764 #0,2,3,9,10
9765 #1,1,3,7,8
9766 #2,10,12,13,15
9767 #3,1,5,10,11
9768 #4,3,5,6,13
9769 #5,8,9,11,12
9770 #6,7,11,13,17

```



```

9771 #7,7,10,14,16
9772 #8,1,9,13,16
9773 #9,2,8,13,14
9774 #10,1,2,15,17
9775 #11,6,8,10,17
9776 #12,6,7,9,15
9777 #13,2,6,11,16
9778 #14,5,9,14,17
9779 #15,5,8,15,16
9780 #16,1,6,12,14
9781 #17,2,5,7,12
9782 #18,3,12,16,17
9783 #19,3,11,14,15
9784 #END
9785
9786
9787 # ToDo neighbors_of_0_with_4_removed.csv is missing
9788
9789 hyperoval_16_stab_0_disjoint_sets_graph:
9790 ▷ $(ORBITER) -v 2 \
9791 ▷ ▷ -define G -graph -disjoint_sets_graph \
9792 ▷ ▷ ▷ neighbors_of_0_with_4_removed.csv \
9793 ▷ ▷ -end \
9794 ▷ ▷ -with G -do \
9795 ▷ ▷ ▷ -graph.theoretic.activity \
9796 ▷ ▷ ▷ -find_cliques \
9797 ▷ ▷ ▷ ▷ -target_size 4 \
9798 ▷ ▷ ▷ -end \
9799 ▷ ▷ -end \
9800 ▷ ▷ -print_symbols
9801
9802
9803 # 5 cliques of size 4
9804 #ROW,C0,C1,C2,C3
9805 #0,0,6,15,16
9806 #1,1,2,13,14
9807 #2,3,9,12,18
9808 #3,4,5,7,10
9809 #4,8,11,17,19
9810 #END
9811
9812 #clique 0:
9813 #0,2,3,9,10
9814 #6,7,11,13,17
9815 #15,5,8,15,16
9816 #16,1,6,12,14
9817 # partition: (1,6,12,14|2,3,9,10|5,8,15,16|7,11,13,17)
9818 # 4 is missing, it is the nucleus
9819 # 0 is missing is the chosen point
9820
9821
9822
9823
9824
9825
9826
9827 # nonconical 6-arcs are used for classifying cubic surfaces:
9828
9829

```

```

9830
9831
9832
9833 nc_arcs_16:
9834 ▷ $(ORBITER) -v 4 \
9835 ▷ ▷ -define F -finite_field -q 16 -end \
9836 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9837 ▷ ▷ -define Control -poset_classification_control \
9838 ▷ ▷ ▷ -problem_label nc_arcs_q16_d2 \
9839 ▷ ▷ ▷ -W -depth 6 \
9840 ▷ ▷ -end \
9841 ▷ ▷ -with P -do \
9842 ▷ ▷ -projective_space_activity \
9843 ▷ ▷ ▷ -classify_arcs \
9844 ▷ ▷ ▷ ▷ -control Control \
9845 ▷ ▷ ▷ ▷ -target_size 6 \
9846 ▷ ▷ ▷ ▷ -d 2 \
9847 ▷ ▷ ▷ ▷ -conic_test \
9848 ▷ ▷ ▷ -end \
9849 ▷ ▷ -end
9850 ▷ #pdflatex nc_arcs_q16_d2_poset.tex
9851 ▷ #$(OPEN) nc_arcs_q16_d2_poset.pdf
9852
9853
9854 #User time: 0:00
9855
9856
9857
9858 five_arcs_q13:
9859 ▷ $(ORBITER) -v 4 \
9860 ▷ ▷ -define F -finite_field -q 13 -end \
9861 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9862 ▷ ▷ -define Control -poset_classification_control \
9863 ▷ ▷ ▷ -problem_label five_arcs_q13 -W -depth 5 \
9864 ▷ ▷ -end \
9865 ▷ ▷ -with P -do \
9866 ▷ ▷ -projective_space_activity \
9867 ▷ ▷ ▷ -classify_arcs \
9868 ▷ ▷ ▷ ▷ -control Control \
9869 ▷ ▷ ▷ ▷ -target_size 5 \
9870 ▷ ▷ ▷ ▷ -d 2 \
9871 ▷ ▷ ▷ -end \
9872 ▷ ▷ -end
9873 ▷ #pdflatex five_arcs_q13_poset.tex
9874 ▷ #$(OPEN) five_arcs_q13_poset.pdf
9875
9876
9877
9878
9879
9880
9881
9882 #####
9883 # Section 7.7: Cubic Curves
9884
9885 SECTION_CUBIC_CURVES:
9886
9887
9888

```

```

9889 test_7.7:
9890 ▷ make cubic_curves_PG_2.4
9891 ▷ make cubic_curves_PG_2.4.draw
9892 ▷ make poly_orbits_q4
9893 ▷ make cubic_curves_PG_2.9
9894 ▷ make cubic_curves_PG_2.9.draw
9895 ▷ make curve_orbit_mini
9896 ▷ make curve_orbit
9897
9898
9899
9900 cubic_curves_PG_2.4:
9901 ▷ $(ORBITER) -v 3 \
9902 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9903 ▷ ▷ -draw_options -radius 200 -embedded -end \
9904 ▷ ▷ -define F -finite_field -q 4 -end \
9905 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9906 ▷ ▷ -define Control -poset_classification_control \
9907 ▷ ▷ ▷ -problem_label cc_4 -W -depth 9 \
9908 ▷ ▷ ▷ -draw_options -radius 200 -embedded -end \
9909 ▷ ▷ -end \
9910 ▷ ▷ -define Arc_control -arc_generator_control \
9911 ▷ ▷ ▷ -projective_space P \
9912 ▷ ▷ ▷ -control Control \
9913 ▷ ▷ ▷ -target_size 9 \
9914 ▷ ▷ ▷ -d 3 \
9915 ▷ ▷ -end \
9916 ▷ ▷ -define Orb -orbits \
9917 ▷ ▷ ▷ -on_cubic_curves Arc_control \
9918 ▷ ▷ -end \
9919 ▷ ▷ -with Orb -do -orbits_activity \
9920 ▷ ▷ ▷ -report \
9921 ▷ ▷ -end
9922 ▷ ▷ -with Orb -do -orbits_activity \
9923 ▷ ▷ ▷ -draw_tree 0 \
9924 ▷ ▷ -end
9925 ▷ pdflatex cc_4_poset_lvl_9.tex
9926 ▷ $(OPEN) cc_4_poset_lvl_9.pdf
9927 ▷ pdflatex Cubic_curves_q4.tex
9928 ▷ $(OPEN) Cubic_curves_q4.pdf
9929
9930
9931 cubic_curves_PG_2.4.draw:
9932 ▷ $(ORBITER) -v 3 \
9933 ▷ ▷ -draw_layered_graph cc_4_poset_lvl_9.layered_graph \
9934 ▷ ▷ ▷ -radius 300 -embedded -line_width 1.1 \
9935 ▷ ▷ ▷ -y_stretch 0.9 -scale 0.25 \
9936 ▷ ▷ -end
9937 ▷ pdflatex cc_4_poset_lvl_9.draw.tex
9938 ▷ $(OPEN) cc_4_poset_lvl_9.draw.pdf
9939
9940
9941
9942 poly_orbits_q4:
9943 ▷ $(ORBITER) -v 4 \
9944 ▷ ▷ -draw_options -yout 500000 -radius 15 -nodes_empty \
9945 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
9946 ▷ ▷ -define F -finite_field -q 4 -print_numerically -end \
9947 ▷ ▷ -define R -polynomial_ring \

```

```

9948 ▷ ▷ ▷ -field F \
9949 ▷ ▷ ▷ -number_of_variables 3 \
9950 ▷ ▷ ▷ -homogeneous_of_degree 3 \
9951 ▷ ▷ ▷ -monomial_ordering_partition \
9952 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
9953 ▷ ▷ ▷ -end \
9954 ▷ ▷ -define G -linear_group -PGGL 3 F -end \
9955 ▷ ▷ -define Orb -orbits -group G \
9956 ▷ ▷ ▷ -on_polynomials R \
9957 ▷ ▷ -end \
9958 ▷ ▷ -with Orb -do -orbits_activity \
9959 ▷ ▷ ▷ -report \
9960 ▷ ▷ -end \
9961 ▷ ▷ -with Orb -do -orbits_activity \
9962 ▷ ▷ ▷ -export_something "orbit" 0 \
9963 ▷ ▷ -end
9964 ▷ pdflatex poly_orbits_d3_n2_q4.tex
9965 ▷ $(OPEN) poly_orbits_d3_n2_q4.pdf
9966
9967 #orbits_PGGL_3_4_orbit_0.csv
9968
9969
9970 # ToDo: error in cubic_curve::compute_inflexion_points
9971
9972 cubic_curves_PG_2_9:
9973 ▷ $(ORBITER) -v 3 \
9974 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
9975 ▷ ▷ -draw_options -radius 200 -embedded -end \
9976 ▷ ▷ -define F -finite_field -q 9 -end \
9977 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9978 ▷ ▷ -define Control -poset_classification_control \
9979 ▷ ▷ ▷ -problem_label cc_9 -W -depth 9 \
9980 ▷ ▷ ▷ -draw_options -radius 200 -embedded -end \
9981 ▷ ▷ -end \
9982 ▷ ▷ -define Arc_control -arc_generator_control \
9983 ▷ ▷ ▷ -projective_space P \
9984 ▷ ▷ ▷ -control Control \
9985 ▷ ▷ ▷ -target_size 9 \
9986 ▷ ▷ ▷ -d 3 \
9987 ▷ ▷ -end \
9988 ▷ ▷ -define Orb -orbits \
9989 ▷ ▷ ▷ -on_cubic_curves Arc_control \
9990 ▷ ▷ -end \
9991 ▷ ▷ -with Orb -do -orbits_activity \
9992 ▷ ▷ ▷ -report \
9993 ▷ ▷ -end
9994 ▷ ▷ -with Orb -do -orbits_activity \
9995 ▷ ▷ ▷ -draw_tree 0 \
9996 ▷ ▷ -end
9997 ▷ #pdflatex cc_9_poset_lv1_9.tex
9998 ▷ #$(OPEN) cc_9_poset_lv1_9.pdf
9999
10000
10001 # too big to latex:
10002
10003 cubic_curves_PG_2_9.draw:
10004 ▷ $(ORBITER) -v 3 \
10005 ▷ ▷ -draw_layered_graph cc_9_poset_lv1_9.layered_graph \
10006 ▷ ▷ ▷ -radius 300 -embedded -line_width 1.1 \

```

```

10007 ▷ ▷ ▷ -y_stretch 0.9 -scale 0.25 \
10008 ▷ ▷ -end
10009 ▷ #pdflatex cc_9_poset_lvl_9_draw.tex
10010 ▷ #$(OPEN) cc_9_poset_lvl_9_draw.pdf
10011
10012
10013 curve_orbit_mini:
10014 ▷ $(ORBITER) -v 4 \
10015 ▷ ▷ -draw_options -yout 500000 -radius 15 -nodes_empty \
10016 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
10017 ▷ ▷ -define F -finite_field -q 3 -print_numerically -end \
10018 ▷ ▷ -define R -polynomial_ring \
10019 ▷ ▷ ▷ -field F \
10020 ▷ ▷ ▷ -number_of_variables 3 \
10021 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10022 ▷ ▷ ▷ -monomial_ordering_partition \
10023 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10024 ▷ ▷ ▷ -end \
10025 ▷ ▷ -define cubic -symbolic_object \
10026 ▷ ▷ ▷ -field F \
10027 ▷ ▷ ▷ -text "X0^3+X1^3+X2^3" \
10028 ▷ ▷ ▷ -end \
10029 ▷ ▷ -define G -linear_group -PGL 3 F -end \
10030 ▷ ▷ -define Orb -orbits -group G \
10031 ▷ ▷ ▷ -of_one_polynomial R cubic \
10032 ▷ ▷ -end \
10033 ▷ ▷ -with Orb -do -orbits_activity \
10034 ▷ ▷ ▷ -export_something "orbit" 0 \
10035 ▷ ▷ -end
10036
10037
10038 curve_orbit:
10039 ▷ $(ORBITER) -v 4 \
10040 ▷ ▷ -draw_options -yout 500000 -radius 15 -nodes_empty \
10041 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
10042 ▷ ▷ -define F -finite_field -q 9 -print_numerically -end \
10043 ▷ ▷ -define R -polynomial_ring \
10044 ▷ ▷ ▷ -field F \
10045 ▷ ▷ ▷ -number_of_variables 3 \
10046 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10047 ▷ ▷ ▷ -monomial_ordering_partition \
10048 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10049 ▷ ▷ ▷ -end \
10050 ▷ ▷ -define cubic -symbolic_object \
10051 ▷ ▷ ▷ -field F \
10052 ▷ ▷ ▷ -text $(SURFACE_F9_1_EQN_RESTRICTED) \
10053 ▷ ▷ ▷ -end \
10054 ▷ ▷ -define G -linear_group -PGL 3 F -end \
10055 ▷ ▷ -define Orb -orbits -group G \
10056 ▷ ▷ ▷ -of_one_polynomial R cubic \
10057 ▷ ▷ -end \
10058 ▷ ▷ -with Orb -do -orbits_activity \
10059 ▷ ▷ ▷ -export_something "orbit" 0 \
10060 ▷ ▷ -end
10061
10062
10063 #orbits.on.polynomials::orbit.of.one.polynomial found an orbit of length 589680
10064 #7:15
10065

```

```

10066
10067
10068
10069 #####
10070 # Chapter 8 - Cubic Surfaces
10071 #####
10072
10073
10074 test_8:
10075 ▷ make test_8.1
10076 ▷ make test_8.2
10077 ▷ make test_8.3
10078 ▷ make test_8.4
10079 ▷ make test_8.5
10080 ▷ make test_8.6
10081 ▷ make test_8.7
10082
10083
10084
10085
10086
10087 #####
10088 # Section 8.1: Global Commands for Cubic Surfaces
10089
10090
10091 SECTION_CUBIC_SURFACES_GLOBAL:
10092
10093
10094
10095 test_8.1:
10096 ▷ make surface_4.0_lines_vs_lines
10097 ▷ make surface_4.0_lines_vs_tritangent_planes
10098 ▷ make surface_4.0_dot_products_columns
10099 ▷ make surface_4.0_export_double_sixes
10100 ▷ make surface_4.0_multiply_half_double_six_vs_lines
10101 ▷ make surface_4.0_multiply_half_double_six_vs_incma
10102 ▷ make surface_4.0_dot_products_rows
10103 ▷ make surface_4.0_dot_products_rowspan
10104 ▷ make surface_4.0_single_six_times_lines_vs_lines_rowspan
10105
10106
10107 surface_4.0_lines_vs_lines:
10108 ▷ $(ORBITER) -v 3 \
10109 ▷ ▷ -define F -finite_field -q 4 -end \
10110 ▷ ▷ -define P -projective_space -n 3 -field F -end \
10111 ▷ ▷ -with P -do \
10112 ▷ ▷ -projective_space_activity \
10113 ▷ ▷ ▷ -export_cubic_surface_line_vs_line_incidence_matrix \
10114 ▷ ▷ -end
10115 ▷ $(ORBITER) -v 2 -draw_matrix \
10116 ▷ ▷ -input_csv_file PG_3.4_lines_vs_lines_incma.csv \
10117 ▷ ▷ -box_width 32 -bit_depth 24 \
10118 ▷ ▷ -partition 3 27 27 \
10119 ▷ ▷ -end
10120 ▷ convert PG_3.4_lines_vs_lines_incma.draw.bmp PG_3.4_lines_vs_lines_incma.draw.p
ng
10121
10122
10123 # PG_3.4_lines_vs_lines_incma.csv

```

```

10124
10125
10126 surface_4_0_lines_vs_tritangent_planes:
10127 ▷ $(ORBITER) -v 3 \
10128 ▷ ▷ -define F -finite_field -q 4 -end \
10129 ▷ ▷ -define P -projective_space -n 3 -field F -end \
10130 ▷ ▷ -with P -do \
10131 ▷ ▷ -projective_space_activity \
10132 ▷ ▷ ▷ -export_cubic_surface_line_tritangent_plane_incidence_matrix \
10133 ▷ ▷ -end
10134 ▷ $(ORBITER) -v 2 -draw_matrix \
10135 ▷ ▷ -input_csv_file PG_3_4_lines_tritplanes_incma.csv \
10136 ▷ ▷ -box_width 32 -bit_depth 24 \
10137 ▷ ▷ -partition 3 27 45 \
10138 ▷ ▷ -end
10139 ▷ convert PG_3_4_lines_tritplanes_incma_draw.bmp PG_3_4_lines_tritplanes_incma_dr
    aw.png
10140
10141
10142
10143
10144 surface_4_0_dot_products_columns:
10145 ▷ $(ORBITER) -v 3 \
10146 ▷ ▷ -define line_vs_trit -vector \
10147 ▷ ▷ ▷ -file PG_3_4_lines_tritplanes_incma.csv \
10148 ▷ ▷ -end \
10149 ▷ ▷ -dot_product_of_columns line_vs_trit
10150 ▷ $(ORBITER) -v 2 -draw_matrix \
10151 ▷ ▷ -input_csv_file line_vs_trit_dot_products_columns.csv \
10152 ▷ ▷ -box_width 32 -bit_depth 24 \
10153 ▷ ▷ -partition 3 45 45 \
10154 ▷ ▷ -end
10155 ▷ convert line_vs_trit_dot_products_columns_draw.bmp line_vs_trit_dot_products_co
    lumns_draw.png
10156
10157
10158
10159 surface_4_0_export_double_sixes:
10160 ▷ $(ORBITER) -v 3 \
10161 ▷ ▷ -define F -finite_field -q 4 -end \
10162 ▷ ▷ -define P -projective_space -n 3 -field F -end \
10163 ▷ ▷ -with P -do \
10164 ▷ ▷ -projective_space_activity \
10165 ▷ ▷ ▷ -export_double_sixes \
10166 ▷ ▷ -end \
10167
10168
10169 surface_4_0_multiply_half_double_six_vs_lines:
10170 ▷ $(ORBITER) -v 3 \
10171 ▷ ▷ -define lines_vs_lines -vector \
10172 ▷ ▷ ▷ -file PG_3_4_lines_vs_lines_incma.csv \
10173 ▷ ▷ -end \
10174 ▷ ▷ -define single_six -vector \
10175 ▷ ▷ ▷ -file PG_3_4_single_sixes_char_vec.csv \
10176 ▷ ▷ -end \
10177 ▷ ▷ -matrix_multiply_over_Z single_six lines_vs_lines
10178
10179 # single_six_times_lines_vs_lines.csv
10180

```

```

10181 surface_4_0_multiply_half_double_six_vs_incma:
10182 ▷ $(ORBITER) -v 3 \
10183 ▷ ▷ -define line_vs_trit -vector \
10184 ▷ ▷ ▷ -file PG_3_4_lines_tritplanes_incma.csv \
10185 ▷ ▷ -end \
10186 ▷ ▷ -define single_six -vector \
10187 ▷ ▷ ▷ -file PG_3_4_single_sixes_char_vec.csv \
10188 ▷ ▷ -end \
10189 ▷ ▷ -matrix_multiply_over_Z single_six line_vs_trit
10190
10191 # 72 x 45:
10192 #single_six_times_line_vs_trit.csv
10193
10194 surface_4_0_dot_products_rows:
10195 ▷ $(ORBITER) -v 3 \
10196 ▷ ▷ -define single_six_vs_trit_planes -vector \
10197 ▷ ▷ ▷ -file single_six_times_line_vs_trit.csv \
10198 ▷ ▷ -end \
10199 ▷ ▷ -dot_product_of_rows single_six_vs_trit_planes
10200
10201 # 72 x 72:
10202 #single_six_vs_trit_planes_dot_products_rows.csv
10203
10204 surface_4_0_dot_products_rowspan:
10205 ▷ $(ORBITER) -v 3 \
10206 ▷ ▷ -define single_six_vs_trit_planes -vector \
10207 ▷ ▷ ▷ -file single_six_times_line_vs_trit.csv \
10208 ▷ ▷ -end \
10209 ▷ ▷ -rowspan_over_R single_six_vs_trit_planes
10210
10211 surface_4_0_single_six_times_lines_vs_lines_rowspan:
10212 ▷ $(ORBITER) -v 3 \
10213 ▷ ▷ -define M -vector \
10214 ▷ ▷ ▷ -file single_six_times_lines_vs_lines.csv \
10215 ▷ ▷ -end \
10216 ▷ ▷ -rowspan_over_R M
10217
10218
10219
10220
10221 #####
10222 # Section 8.2: Cubic Surfaces Creation
10223
10224
10225 SECTION_CUBIC_SURFACES_CREATION:
10226
10227
10228
10229 test_8_2:
10230 ▷ make surface_4_0
10231 ▷ make Family_general_F7
10232 ▷ make surface_eckardt_13_4_12
10233 ▷ make Eckardt_13
10234 ▷ make surface_11_random
10235 ▷ make surface_F_abcd_Eckardt_q31_by_equation
10236 ▷ make surface_q13_Eckardt_by_arc_lifting
10237
10238
10239

```



```

10240 surface_4.0:
10241 ▷ $(ORBITER) -v 3 \
10242 ▷ ▷ -define F -finite_field -q 4 -end \
10243 ▷ ▷ -define P -projective_space -n 3 -field F -end \
10244 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
10245
10246
10247 Family_general_F7:
10248 ▷ $(ORBITER) -v 3 \
10249 ▷ ▷ -define F -finite_field -q 7 -end \
10250 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10251 ▷ ▷ -define S7_abcd_2_3_3_4 -cubic_surface \
10252 ▷ ▷ ▷ -space P -family_general_abcd 2 3 3 4 \
10253 ▷ ▷ -end
10254
10255
10256 surface_eckardt_13_4_12:
10257 ▷ $(ORBITER) -v 6 \
10258 ▷ ▷ -define F -finite_field -q 13 -end \
10259 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10260 ▷ ▷ -define Eckardt_4_12 -cubic_surface \
10261 ▷ ▷ ▷ -space P -family_Eckardt 4 12 \
10262 ▷ ▷ -end
10263
10264
10265 Eckardt_13:
10266 ▷ $(ORBITER) -v 3 \
10267 ▷ ▷ -define F -finite_field -q 13 -end \
10268 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10269 ▷ ▷ -define Eckardt_3_1 -cubic_surface \
10270 ▷ ▷ ▷ -space P -family_Eckardt 3 1 \
10271 ▷ ▷ -end
10272
10273
10274 surface_11_random:
10275 ▷ $(ORBITER) -v 10 \
10276 ▷ ▷ -define F -finite_field -q 11 -end \
10277 ▷ ▷ -define P -projective_space -n 3 -field F -v 6 -end \
10278 ▷ ▷ -define S -cubic_surface -space P -random -end
10279
10280
10281
10282 surface_F_abcd_Eckardt_q31_by_equation:
10283 ▷ $(ORBITER) -v 10 \
10284 ▷ ▷ -define F -finite_field -q 31 -end \
10285 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10286 ▷ ▷ -define R -polynomial_ring \
10287 ▷ ▷ ▷ -field F \
10288 ▷ ▷ ▷ -number_of_variables 4 \
10289 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10290 ▷ ▷ ▷ -monomial_ordering_partition \
10291 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
10292 ▷ ▷ ▷ -end \
10293 ▷ ▷ -define F_abcd -cubic_surface -space P \
10294 ▷ ▷ ▷ -by_equation \
10295 ▷ ▷ ▷ R \
10296 ▷ ▷ ▷ "F_abcd" \
10297 ▷ ▷ ▷ "\DF_{a,b,c,d}\D" "X0,X1,X2,X3" \
10298 ▷ ▷ ▷ $(F_abcd_eqn) \

```

```

10299 ▷ ▷ ▷ "a,b,c,d" \
10300 ▷ ▷ ▷ "\D(a,b,c,d)=(2,30,30,2)\D" \
10301 ▷ ▷ ▷ "2,30,30,2" \
10302 ▷ ▷ -end
10303
10304
10305 surface.q13.Eckardt_by_arc_lifting:
10306 ▷ $(ORBITER) -v 3 \
10307 ▷ ▷ -define F -finite_field -q 13 -end \
10308 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10309 ▷ ▷ -define S -cubic_surface -space P -arc_lifting "0,1,2,3,43,113" -end \
10310 ▷ ▷ -with S -do \
10311 ▷ ▷ -cubic_surface_activity \
10312 ▷ ▷ ▷ -report \
10313 ▷ ▷ -end
10314
10315
10316 #####
10317 # Section 8.3: Cubic Surface Activities
10318
10319
10320 SECTION_CUBIC_SURFACE_ACTIVITIES:
10321
10322
10323
10324 test_8.3:
10325 ▷ make surface_4.0_report
10326 ▷ make surface_4.0_export
10327 ▷ make Hirschfeld_surface.get_incidence_matrix.40.40
10328 ▷ make Hirschfeld_surface.incma.40.40.c
10329 ▷ make surface_4.0_plane_type_of_lines_on_Klein_quadric
10330 ▷ make surface_7.0
10331 ▷ #make surface_7.0_plane_type_of_lines_on_Klein_quadric # slow
10332 ▷ make Family_general_F7_report
10333 ▷ make surface_8.0
10334 ▷ #make surface_8.0_plane_type_of_lines_on_Klein_quadric # slow
10335 ▷ make surface_8.0_catalogue
10336 ▷ make surface_8.0_clean
10337 ▷ make surface_8.0b
10338 ▷ make surface_9.0
10339 ▷ #make surface_9.0_plane_type_of_lines_on_Klein_quadric # slow
10340 ▷ make surface_9.1
10341 ▷ #make surface_9.1_plane_type_of_lines_on_Klein_quadric # slow
10342 ▷ make surface_eckardt_13.4.12_report
10343 ▷ make Eckardt_13_report
10344 ▷ make surface_13.0
10345 ▷ make surface_16.0
10346 ▷ make G13.8
10347 ▷ make F13.8
10348 ▷ make F13.16
10349 ▷ make F13.32
10350 ▷ #make F13.64a
10351 ▷ #make F13.64b
10352 ▷ #make Colorado1
10353 ▷ #make Colorado2
10354 ▷ #make Colorado3
10355 ▷ #make F13_128a
10356 ▷ #make F13_128b
10357 ▷ #make F13_128c

```

```

10358 ▷ make move_two_lines
10359 ▷ make surface_F_abcd_Eckardt_q31
10360 ▷ make surface_F_abcd_Eckardt_q31_by_equation_report
10361 ▷ #make surface_F_abcd_sweep_4_27_q7
10362 ▷ make surface_11_random_report
10363 ▷ make surface_11_1
10364
10365
10366
10367
10368
10369 surface_4_0_report:
10370 ▷ $(ORBITER) -v 3 \
10371 ▷ ▷ -define F -finite_field -q 4 -end \
10372 ▷ ▷ -define P -projective_space -n 3 -field F -end \
10373 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
10374 ▷ ▷ -with S -do \
10375 ▷ ▷ -cubic_surface_activity \
10376 ▷ ▷ ▷ -report \
10377 ▷ ▷ -end
10378 ▷ pdflatex surface_catalogue_q4_iso0_report.tex
10379 ▷ $(OPEN) surface_catalogue_q4_iso0_report.pdf
10380
10381
10382 surface_4_0_export:
10383 ▷ $(ORBITER) -v 3 \
10384 ▷ ▷ -define F -finite_field -q 4 -end \
10385 ▷ ▷ -define P -projective_space -n 3 -field F -end \
10386 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
10387 ▷ ▷ -with S -do \
10388 ▷ ▷ -cubic_surface_activity \
10389 ▷ ▷ ▷ -export_something "points" \
10390 ▷ ▷ -end \
10391 ▷ ▷ -with S -do \
10392 ▷ ▷ -cubic_surface_activity \
10393 ▷ ▷ ▷ -export_something "points.off" \
10394 ▷ ▷ -end \
10395 ▷ ▷ -with S -do \
10396 ▷ ▷ -cubic_surface_activity \
10397 ▷ ▷ ▷ -export_something "lines" \
10398 ▷ ▷ -end \
10399 ▷ ▷ -with S -do \
10400 ▷ ▷ -cubic_surface_activity \
10401 ▷ ▷ ▷ -export_something "Hesse_planes" \
10402 ▷ ▷ -end
10403
10404
10405 Hirschfeld_surface_get_incidence_matrix_40_40:
10406 ▷ $(ORBITER) -v 3 \
10407 ▷ ▷ -define points -vector -dense $(HIRSCHFELD_SURFACE_POINTS_OFF) -end \
10408 ▷ ▷ -define planes -vector -dense $(HIRSCHFELD_SURFACE_HESSE_PLANES) -end \
10409 ▷ ▷ -define F -finite_field -q 4 -end \
10410 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10411 ▷ ▷ -with P -do \
10412 ▷ ▷ -projective_space_activity \
10413 ▷ ▷ ▷ -restricted_incidence_matrix 1 3 points planes "H-incma_40_40" \
10414 ▷ ▷ -end
10415
10416

```

```

10417
10418 Hirschfeld_surface_incma_40_40_c:
10419 ▷ $(ORBITER) -v 5 \
10420 ▷ ▷ -draw_incidence_structure_description \
10421 ▷ ▷ ▷ -width 60 -width_10 6 -end \
10422 ▷ ▷ -define C -combinatorial_object \
10423 ▷ ▷ ▷ -label H_incma_40_40 H_incma_40_40 \
10424 ▷ ▷ ▷ -file_of_incidence_geometries H_incma_40_40.inc 40 40 480 \
10425 ▷ ▷ -end \
10426 ▷ ▷ -with C -do \
10427 ▷ ▷ -combinatorial_object_activity \
10428 ▷ ▷ ▷ -canonical_form \
10429 ▷ ▷ ▷ ▷ -save_ago \
10430 ▷ ▷ ▷ ▷ -save_transversal \
10431 ▷ ▷ ▷ -end \
10432 ▷ ▷ -end \
10433 ▷ ▷ -with C -do \
10434 ▷ ▷ -combinatorial_object_activity \
10435 ▷ ▷ ▷ -report \
10436 ▷ ▷ ▷ ▷ -export_flag_orbits \
10437 ▷ ▷ ▷ ▷ -show_incidence_matrices \
10438 ▷ ▷ ▷ ▷ -export_group_GAP \
10439 ▷ ▷ ▷ -end \
10440 ▷ ▷ -end
10441 ▷ $(ORBITER) -v 2 -draw_matrix \
10442 ▷ ▷ -input_csv_file H_incma_40_40.object0.TDA.flag.orbits.csv \
10443 ▷ ▷ -secondary_input_csv_file H_incma_40_40.object0.TDA.csv \
10444 ▷ ▷ -box_width 32 -bit_depth 24 \
10445 ▷ ▷ -end
10446 ▷ pdflatex H_incma_40_40.classification.tex
10447 ▷ $(OPEN) H_incma_40_40.classification.pdf
10448
10449 surface_4_0_plane_type_of_lines_on_Klein_quadric:
10450 ▷ $(ORBITER) -v 6 \
10451 ▷ ▷ -define L -vector -file surface_catalogue_q4_iso0.lines.csv -end \
10452 ▷ ▷ -define F -finite_field -q 4 -end \
10453 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10454 ▷ ▷ -with P -do \
10455 ▷ ▷ -projective_space_activity \
10456 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L \
10457 ▷ ▷ -end
10458
10459 #The plane intersection type is (0^244496, 1^112914, 2^15120, 3^4005, 5^270)
10460 #The plane intersection type is (5^270)
10461 #L_highest_weight_objects.csv
10462
10463
10464
10465
10466 surface_7_0:
10467 ▷ $(ORBITER) -v 3 \
10468 ▷ ▷ -define F -finite_field -q 7 -end \
10469 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10470 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
10471 ▷ ▷ -with S -do \
10472 ▷ ▷ -cubic_surface_activity \
10473 ▷ ▷ ▷ -report \
10474 ▷ ▷ -end \
10475 ▷ ▷ -with S -do \

```

```

10476 ▷ ▷ -cubic_surface_activity \
10477 ▷ ▷ ▷ -export_something "lines" \
10478 ▷ ▷ -end
10479 ▷ pdflatex surface_catalogue_q7_iso0_report.tex
10480 ▷ $(OPEN) surface_catalogue_q7_iso0_report.pdf
10481
10482
10483
10484 surface_7_0_plane_type_of_lines_on_Klein_quadric:
10485 ▷ $(ORBITER) -v 6 \
10486 ▷ ▷ -define L_7_0 -vector \
10487 ▷ ▷ ▷ -file surface_catalogue_q7_iso0_lines.csv -end \
10488 ▷ ▷ -define F -finite_field -q 7 -end \
10489 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10490 ▷ ▷ -with P -do \
10491 ▷ ▷ -projective_space_activity \
10492 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_7_0 \
10493 ▷ ▷ -end
10494
10495 # Time: 4:50
10496 #The plane intersection type is (0^44526575, 1^3529170, 2^112563, 3^8541, 4^324,
10497 #5^27)
10497 #The plane intersection type is (4^324, 5^27)
10498 #L_7_0_highest_weight_objects.csv
10499
10500
10501 Family_general_F7_report:
10502 ▷ $(ORBITER) -v 3 \
10503 ▷ ▷ -define F -finite_field -q 7 -end \
10504 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10505 ▷ ▷ -define S7_abcd_2_3_3_4 -cubic_surface \
10506 ▷ ▷ ▷ -space P -family_general_abcd 2 3 3 4 \
10507 ▷ ▷ -end \
10508 ▷ ▷ -with S7_abcd_2_3_3_4 -do \
10509 ▷ ▷ -cubic_surface_activity \
10510 ▷ ▷ ▷ -report \
10511 ▷ ▷ -end
10512 ▷ pdflatex surface_family_general_abcd_q7_a2_b3_c3_d4_report.tex
10513 ▷ $(OPEN) surface_family_general_abcd_q7_a2_b3_c3_d4_report.pdf
10514
10515
10516 # Fermat with 18 Eckardt points
10517 # no automorphism group
10518
10519
10520 surface_8_0:
10521 ▷ $(ORBITER) -v 3 \
10522 ▷ ▷ -define F -finite_field -q 8 -end \
10523 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10524 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
10525 ▷ ▷ -with S -do \
10526 ▷ ▷ -cubic_surface_activity \
10527 ▷ ▷ ▷ -report \
10528 ▷ ▷ ▷ -all_quartic_curves \
10529 ▷ ▷ -end \
10530 ▷ ▷ -with S -do \
10531 ▷ ▷ -cubic_surface_activity \
10532 ▷ ▷ ▷ -export_something "lines" \
10533 ▷ ▷ -end \

```

```

10534
10535 surface_8_0_plane_type_of_lines_on_Klein_quadric:
10536 ▷ $(ORBITER) -v 6 \
10537 ▷ ▷ -define L_8_0 -vector -file surface_catalogue_q8_iso0_lines.csv -end \
10538 ▷ ▷ -define F -finite_field -q 8 -end \
10539 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10540 ▷ ▷ -with P -do \
10541 ▷ ▷ -projective_space_activity \
10542 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_8_0 \
10543 ▷ ▷ -end
10544
10545 #L_8_0_highest_weight_objects.csv
10546 #intersection numbers: ( 0, 0, 0, 0, 192, 30 )
10547 # time: 15:48
10548
10549
10550
10551
10552
10553
10554 surface_8_0_catalogue:
10555 ▷ $(ORBITER) -v 3 \
10556 ▷ ▷ -define F -finite_field -q 8 -end \
10557 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10558 ▷ ▷ -define S8_0 -cubic_surface -space P -catalogue 0 -end \
10559 ▷ ▷ -with S8_0 -do \
10560 ▷ ▷ -cubic_surface_activity \
10561 ▷ ▷ ▷ -report \
10562 ▷ ▷ -end
10563 ▷ pdflatex surface_catalogue_q8_iso0_report.tex
10564 ▷ $(OPEN) surface_catalogue_q8_iso0_report.pdf
10565
10566
10567
10568 surface_8_0_clean:
10569 ▷ $(ORBITER) -v 3 \
10570 ▷ ▷ -define F -finite_field -q 8 -end \
10571 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10572 ▷ ▷ -define S8_0 -cubic_surface -space P -catalogue 0 \
10573 ▷ ▷ ▷ -select_double_six "15,11,22,19,24,5,16,10,23,20,25,4" \
10574 ▷ ▷ ▷ -select_double_six "3,2,1,0,5,4,9,8,7,6,11,10" \
10575 ▷ ▷ ▷ -transform_inverse "1,4,4,0,6,0,0,0,6,2,0,1,7,0,4,0,0" \
10576 ▷ ▷ ▷ -transform "4,4,0,0, 0,0,1,0, 1,0,0,0, 0,0,0,1, 0" \
10577 ▷ ▷ ▷ -transform_inverse "2,0,0,0,0,2,0,0,0,0,2,0,1,1,2,3,0" \
10578 ▷ ▷ ▷ -end \
10579 ▷ ▷ -with S8_0 -do \
10580 ▷ ▷ -cubic_surface_activity \
10581 ▷ ▷ ▷ -report \
10582 ▷ ▷ -end
10583 ▷ pdflatex surface_catalogue_q8_iso0_report.tex
10584 ▷ $(OPEN) surface_catalogue_q8_iso0_report.pdf
10585
10586
10587
10588 # clean equation for Tekirdag-1:
10589
10590 surface_8_0b:
10591 ▷ $(ORBITER) -v 3 \
10592 ▷ ▷ -define F -finite_field -q 8 -end \

```

```

10593 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10594 ▷ ▷ -define S8_0 -cubic_surface -space P -catalogue 0 \
10595 ▷ ▷ ▷ -select_double_six "15,11,22,19,24,5,16,10,23,20,25,4" \
10596 ▷ ▷ ▷ -select_double_six "3,2,1,0,5,4,9,8,7,6,11,10" \
10597 ▷ ▷ ▷ -transform "1,0,0,0,0,1,0,6,0,0,1,6,0,0,0,1,0" \
10598 ▷ ▷ ▷ -transform_inverse "3,1,1,0,0,1,0,0,0,0,1,0,0,0,0,1,0" \
10599 ▷ ▷ ▷ -transform_inverse "2,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0" \
10600 ▷ ▷ -end \
10601 ▷ ▷ -with S8_0 -do \
10602 ▷ ▷ -cubic_surface_activity \
10603 ▷ ▷ ▷ -report \
10604 ▷ ▷ -end
10605
10606
10607
10608
10609
10610
10611 surface_9_0:
10612 ▷ ▷ $(ORBITER) -v 3 \
10613 ▷ ▷ -define F -finite_field -q 9 -end \
10614 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10615 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
10616 ▷ ▷ -with S -do \
10617 ▷ ▷ -cubic_surface_activity \
10618 ▷ ▷ ▷ -report \
10619 ▷ ▷ ▷ -all_quartic_curves \
10620 ▷ ▷ -end \
10621 ▷ ▷ -with S -do \
10622 ▷ ▷ -cubic_surface_activity \
10623 ▷ ▷ ▷ -export_something "lines" \
10624 ▷ ▷ -end
10625
10626 surface_9_0_plane_type_of_lines_on_Klein_quadric:
10627 ▷ ▷ $(ORBITER) -v 6 \
10628 ▷ ▷ -define L_9_0 -vector -file surface_catalogue_q9_iso0_lines.csv -end \
10629 ▷ ▷ -define F -finite_field -q 9 -end \
10630 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10631 ▷ ▷ -with P -do \
10632 ▷ ▷ -projective_space_activity \
10633 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_9_0 \
10634 ▷ ▷ -end
10635
10636
10637 #intersection numbers: ( 0, 0, 0, 0, 180, 15 )
10638 #time: 44:42
10639
10640
10641 surface_9_1:
10642 ▷ ▷ $(ORBITER) -v 3 \
10643 ▷ ▷ -define F -finite_field -q 9 -end \
10644 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10645 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
10646 ▷ ▷ -with S -do \
10647 ▷ ▷ -cubic_surface_activity \
10648 ▷ ▷ ▷ -report \
10649 ▷ ▷ ▷ -all_quartic_curves \
10650 ▷ ▷ -end \
10651 ▷ ▷ -with S -do \

```

```

10652 ▷ ▷ -cubic_surface_activity \
10653 ▷ ▷ ▷ -export_something "lines" \
10654 ▷ ▷ -end
10655 ▷ pdflatex surface_catalogue_q9_iso1_report.tex
10656 ▷ $(OPEN) surface_catalogue_q9_iso1_report.pdf
10657
10658
10659 surface_9_1_plane_type_of_lines_on_Klein_quadric:
10660 ▷ $(ORBITER) -v 6 \
10661 ▷ ▷ -define L_9_1 -vector -file surface_catalogue_q9_iso1_lines.csv -end \
10662 ▷ ▷ -define F -finite_field -q 9 -end \
10663 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10664 ▷ ▷ -with P -do \
10665 ▷ ▷ -projective_space_activity \
10666 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_9_1 \
10667 ▷ ▷ -end
10668
10669 #intersection numbers: ( 0, 0, 0, 0, 216 )
10670
10671 #44:36
10672
10673
10674
10675 surface_eckardt_13_4_12_report:
10676 ▷ $(ORBITER) -v 6 \
10677 ▷ ▷ -define F -finite_field -q 13 -end \
10678 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10679 ▷ ▷ -define Eckardt_4_12 -cubic_surface \
10680 ▷ ▷ ▷ -space P -family_Eckardt 4 12 \
10681 ▷ ▷ -end \
10682 ▷ ▷ -with Eckardt_4_12 -do \
10683 ▷ ▷ -cubic_surface_activity \
10684 ▷ ▷ ▷ -report \
10685 ▷ ▷ -end
10686 ▷ pdflatex surface_family_Eckardt_q13_a4_b12_report.tex
10687 ▷ $(OPEN) surface_family_Eckardt_q13_a4_b12_report.pdf
10688
10689
10690
10691
10692
10693
10694 # 13_0 has 4 Eckardt points
10695 # 13_1 has 6 Eckardt points
10696 # 13_2 has 9 Eckardt points
10697 # 13_3 has 18 Eckardt points
10698 #
10699
10700
10701
10702
10703 Eckardt_13_report:
10704 ▷ $(ORBITER) -v 3 \
10705 ▷ ▷ -define F -finite_field -q 13 -end \
10706 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10707 ▷ ▷ -define Eckardt_3_1 -cubic_surface \
10708 ▷ ▷ ▷ -space P -family_Eckardt 3 1 \
10709 ▷ ▷ -end \
10710 ▷ ▷ -with Eckardt_3_1 -do \

```



```

10711 ▷ ▷ -cubic_surface_activity \
10712 ▷ ▷ ▷ -report \
10713 ▷ ▷ -end
10714
10715
10716
10717 surface_13.0:
10718 ▷ $(ORBITER) -v 3 \
10719 ▷ ▷ -define F -finite_field -q 13 -end \
10720 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10721 ▷ ▷ -define S13_0 -cubic_surface -space P -catalogue 0 -end \
10722 ▷ ▷ -with S13_0 -do \
10723 ▷ ▷ -cubic_surface_activity \
10724 ▷ ▷ ▷ -report \
10725 ▷ ▷ -end
10726
10727
10728 # clean equation for Tekirdag-2:
10729
10730
10731 surface_16.0:
10732 ▷ $(ORBITER) -v 3 \
10733 ▷ ▷ -define F -finite_field -q 16 -end \
10734 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10735 ▷ ▷ -define S16_0 -cubic_surface -space P -catalogue 0 \
10736 ▷ ▷ ▷ -transform "1,0,0,0,0,1,0,12,0,0,1,12,0,0,0,1,0" \
10737 ▷ ▷ ▷ -transform "15,11,4,0,0,0,12,0,0,12,0,0,0,0,0,1,3" \
10738 ▷ ▷ -end \
10739 ▷ ▷ -with S16_0 -do \
10740 ▷ ▷ -cubic_surface_activity \
10741 ▷ ▷ ▷ -report \
10742 ▷ ▷ -end
10743
10744
10745 #▷ ▷ -transform_inverse "3,0,0,0,0,1,1,0,0,0,1,0,0,0,0,1,0" \
10746 #▷ ▷ -transform_inverse "13,12,1,0,12,13,1,0,0,0,1,0,0,0,0,1,0" \
10747 #▷ ▷ -transform_inverse "1,0,0,0,0,1,0,0,12,12,1,0,0,0,0,1,0" \
10748 #▷ ▷ -transform_inverse "12,0,0,0,0,12,0,0,0,0,1,0,0,0,0,1,0"
10749
10750 # rank of lines ( 66591, 26737, 4093, 69904, 28376, 26470, 70160, 69855, 26208, 5
      847, 369, 32230, 529, 30293, 70068, 2178, 261, 28666, 8575, 105, 31694, 0, 51784,
      25209, 22193, 49862, 274 )
10751 # Rank of points on Klein quadric: ( 29181, 4677, 29950, 33, 62496, 429, 1, 9205,
      37, 29964, 29364, 21501, 4656, 54735, 5425, 30105, 754, 6680, 13354, 758, 30106,
      0, 29209, 48736, 25595, 33780, 4657 )
10752
10753
10754 # ai: 29181, 4677, 29950, 33, 62496, 429
10755 # bi: 1, 9205, 37, 29964, 29364, 21501
10756
10757 # Tekirdag-1:
10758
10759 G13.8:
10760 ▷ $(ORBITER) -v 3 \
10761 ▷ ▷ -define F -finite_field -q 8 -end \
10762 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10763 ▷ ▷ -define T1 -cubic_surface -space P -family_G13 2 -end \
10764 ▷ ▷ -with T1 -do \
10765 ▷ ▷ -cubic_surface_activity \

```

```

10766 ▷ ▷ ▷ -report \
10767 ▷ ▷ -end
10768
10769
10770 F13.8:
10771 ▷ $(ORBITER) -v 3 \
10772 ▷ ▷ -define F -finite_field -q 8 -end \
10773 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10774 ▷ ▷ -define T1 -cubic_surface -space P -family_F13 2 -end \
10775 ▷ ▷ -with T1 -do \
10776 ▷ ▷ -cubic_surface_activity \
10777 ▷ ▷ ▷ -report \
10778 ▷ ▷ -end
10779
10780
10781
10782
10783 # Tekirdag-2:
10784
10785 F13.16:
10786 ▷ $(ORBITER) -v 3 \
10787 ▷ ▷ -define F -finite_field -q 16 -end \
10788 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10789 ▷ ▷ -define T2 -cubic_surface -space P -family_F13 2 -end \
10790 ▷ ▷ -with T2 -do \
10791 ▷ ▷ -cubic_surface_activity \
10792 ▷ ▷ ▷ -report \
10793 ▷ ▷ -end
10794
10795
10796 # Tekirdag-3:
10797
10798 F13.32:
10799 ▷ $(ORBITER) -v 3 \
10800 ▷ ▷ -define F -finite_field -q 32 -end \
10801 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10802 ▷ ▷ -define T3 -cubic_surface -space P -family_F13 2 -end \
10803 ▷ ▷ -with T3 -do \
10804 ▷ ▷ -cubic_surface_activity \
10805 ▷ ▷ ▷ -report \
10806 ▷ ▷ -end
10807
10808
10809 # Kapadokya-1:
10810
10811 F13.64a:
10812 ▷ $(ORBITER) -v 3 \
10813 ▷ ▷ -define F -finite_field -q 64 -end \
10814 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10815 ▷ ▷ -define K1 -cubic_surface -space P -family_F13 2 -end \
10816 ▷ ▷ -with K1 -do \
10817 ▷ ▷ -cubic_surface_activity \
10818 ▷ ▷ ▷ -report \
10819 ▷ ▷ -end
10820
10821
10822 # Kapadokya-2:
10823
10824 F13.64b:

```

```

10825 ▷ $(ORBITER) -v 3 \
10826 ▷ ▷ -define F -finite_field -q 64 -end \
10827 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10828 ▷ ▷ -define K2 -cubic_surface -space P -family_F13 18 -end \
10829 ▷ ▷ -with K2 -do \
10830 ▷ ▷ -cubic_surface_activity \
10831 ▷ ▷ ▷ -report \
10832 ▷ ▷ -end
10833
10834
10835 Colorado1:
10836 ▷ $(ORBITER) -v 3 \
10837 ▷ ▷ -define F -finite_field -q 128 -end \
10838 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10839 ▷ ▷ -define C0-1 -cubic_surface -space P -catalogue 0 \
10840 ▷ ▷ ▷ -transform_inverse "1,0,0,0,0,1,0,96,0,0,1,96,0,0,0,1,0" \
10841 ▷ ▷ -end \
10842 ▷ ▷ -with C0-1 -do \
10843 ▷ ▷ -cubic_surface_activity \
10844 ▷ ▷ ▷ -report \
10845 ▷ ▷ -end
10846
10847
10848 # recognize the arcs from Colorado-1,2,3:
10849
10850
10851
10852 Colorado2:
10853 ▷ $(ORBITER) -v 3 \
10854 ▷ ▷ -define F -finite_field -q 128 -end \
10855 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10856 ▷ ▷ -define C0-2 -cubic_surface -space P -catalogue 926 \
10857 ▷ ▷ ▷ -transform_inverse "1,0,0,0,0,1,0,32,0,0,1,32,0,0,0,1,0" \
10858 ▷ ▷ -end \
10859 ▷ ▷ -with C0-2 -do \
10860 ▷ ▷ -cubic_surface_activity \
10861 ▷ ▷ ▷ -report \
10862 ▷ ▷ -end
10863
10864 Colorado3:
10865 ▷ $(ORBITER) -v 3 \
10866 ▷ ▷ -define F -finite_field -q 128 -end \
10867 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10868 ▷ ▷ -define C0-3 -cubic_surface -space P -catalogue 928 \
10869 ▷ ▷ ▷ -transform_inverse "1,0,0,0,0,1,0,59,0,0,1,59,0,0,0,1,0" \
10870 ▷ ▷ -end \
10871 ▷ ▷ -with C0-3 -do \
10872 ▷ ▷ -cubic_surface_activity \
10873 ▷ ▷ ▷ -report \
10874 ▷ ▷ -end
10875
10876
10877 # Colorado-1:
10878
10879 F13_128a:
10880 ▷ $(ORBITER) -v 3 \
10881 ▷ ▷ -define F -finite_field -q 128 -end \
10882 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10883 ▷ ▷ -define C0-1 -cubic_surface -space P -family_F13 2 -end \

```

```

10884 ▷ ▷ -with C0-1 -do \
10885 ▷ ▷ -cubic_surface_activity \
10886 ▷ ▷ ▷ -report \
10887 ▷ ▷ -end
10888
10889 # Colorado-2:
10890
10891 F13_128b:
10892 ▷ $(ORBITER) -v 3 \
10893 ▷ ▷ -define F -finite_field -q 128 -end \
10894 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10895 ▷ ▷ -define C0-2 -cubic_surface -space P -family_F13 6 -end \
10896 ▷ ▷ -with C0-2 -do \
10897 ▷ ▷ -cubic_surface_activity \
10898 ▷ ▷ ▷ -report \
10899 ▷ ▷ -end
10900
10901 # Colorado-3:
10902
10903 F13_128c:
10904 ▷ $(ORBITER) -v 3 \
10905 ▷ ▷ -define F -finite_field -q 128 -end \
10906 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10907 ▷ ▷ -define C0-3 -cubic_surface -space P -family_F13 14 -end \
10908 ▷ ▷ -with C0-3 -do \
10909 ▷ ▷ -cubic_surface_activity \
10910 ▷ ▷ ▷ -report \
10911 ▷ ▷ -end
10912
10913
10914 move_two_lines:
10915 ▷ $(ORBITER) -v 5 \
10916 ▷ ▷ -define F -finite_field -q 8 -end \
10917 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10918 ▷ ▷ -with P -do -projective_space_activity \
10919 ▷ ▷ ▷ -move_two_lines_in_hyperplane_stabilizer \
10920 ▷ ▷ ▷ 65 4680 72 657 \
10921 ▷ ▷ -end
10922
10923
10924
10925
10926
10927 surface_F_abcd_Eckardt_q31:
10928 ▷ $(ORBITER) -v 10 \
10929 ▷ ▷ -define F -finite_field -q 31 -end \
10930 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10931 ▷ ▷ -define R -polynomial_ring \
10932 ▷ ▷ ▷ -field F \
10933 ▷ ▷ ▷ -number_of_variables 4 \
10934 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10935 ▷ ▷ ▷ -monomial_ordering_partition \
10936 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
10937 ▷ ▷ ▷ -end \
10938 ▷ ▷ -define F_abcd -symbolic_object \
10939 ▷ ▷ ▷ -field F \
10940 ▷ ▷ ▷ -text $(F_abcd_eqn) \
10941 ▷ ▷ ▷ -end \
10942 ▷ ▷ -define abcd_values -symbolic_object \

```

```

10943 ▷ ▷ ▷ -field F \
10944 ▷ ▷ ▷ -text "2,30,30,2" \
10945 ▷ ▷ ▷ -end \
10946 ▷ ▷ -define F_abcd_sub -symbolic_object \
10947 ▷ ▷ ▷ -field F \
10948 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
10949 ▷ ▷ ▷ -substitute "a,b,c,d" F_abcd abcd.values \
10950 ▷ ▷ -end \
10951 ▷ ▷ -define S -cubic_surface -space P \
10952 ▷ ▷ ▷ -by_symbolic_object \
10953 ▷ ▷ ▷ R \
10954 ▷ ▷ ▷ F_abcd_sub \
10955 ▷ ▷ -end \
10956 ▷ ▷ -with S -do \
10957 ▷ ▷ -cubic_surface_activity \
10958 ▷ ▷ ▷ -report \
10959 ▷ ▷ -end
10960 ▷ pdflatex surface.equation.F_abcd_sub.q31.report.tex
10961 ▷ $(OPEN) surface.equation.F_abcd_sub.q31.report.pdf
10962
10963
10964
10965 surface.F_abcd.Eckardt.q31.by.equation.report:
10966 ▷ $(ORBITER) -v 10 \
10967 ▷ ▷ -define F -finite_field -q 31 -end \
10968 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10969 ▷ ▷ -define R -polynomial_ring \
10970 ▷ ▷ ▷ -field F \
10971 ▷ ▷ ▷ -number_of_variables 4 \
10972 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10973 ▷ ▷ ▷ -monomial_ordering_partition \
10974 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
10975 ▷ ▷ ▷ -end \
10976 ▷ ▷ -define F_abcd -cubic_surface -space P \
10977 ▷ ▷ ▷ -by_equation \
10978 ▷ ▷ ▷ R \
10979 ▷ ▷ ▷ "F_abcd" \
10980 ▷ ▷ ▷ "\DF_{a,b,c,d}\D" "X0,X1,X2,X3" \
10981 ▷ ▷ ▷ $(F_abcd_eqn) \
10982 ▷ ▷ ▷ "a,b,c,d" \
10983 ▷ ▷ ▷ "\D(a,b,c,d)=(2,30,30,2)\D" \
10984 ▷ ▷ ▷ "2,30,30,2" \
10985 ▷ ▷ -end \
10986 ▷ ▷ -with F_abcd -do \
10987 ▷ ▷ -cubic_surface_activity \
10988 ▷ ▷ ▷ -report \
10989 ▷ ▷ -end
10990 ▷ pdflatex surface.equation.F_abcd.q31.report.tex
10991 ▷ $(OPEN) surface.equation.F_abcd.q31.report.pdf
10992
10993
10994 # E = 6
10995
10996
10997
10998
10999
11000 surface.F_abcd.sweep.4.27.q7:
11001 ▷ $(ORBITER) -v 3 \

```

```

11002 ▷ ▷ -define F -finite_field -q 7 -end \
11003 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11004 ▷ ▷ -define R -polynomial_ring \
11005 ▷ ▷ ▷ -field F \
11006 ▷ ▷ ▷ -number_of_variables 4 \
11007 ▷ ▷ ▷ -homogeneous_of_degree 3 \
11008 ▷ ▷ ▷ -monomial_ordering_partition \
11009 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
11010 ▷ ▷ ▷ -end \
11011 ▷ ▷ -with P -do \
11012 ▷ ▷ -projective_space_activity \
11013 ▷ ▷ -sweep_4.27 sweep_4.27_q7 \
11014 ▷ ▷ ▷ -space P \
11015 ▷ ▷ ▷ -by_equation R "F_abcd" \
11016 ▷ ▷ ▷ "\DF_{a,b,c,d}\D" "X0,X1,X2,X3" \
11017 ▷ ▷ ▷ $(F_abcd_eqn_no_exponents) \
11018 ▷ ▷ ▷ "a=2,b=3,c=4,d=5" \
11019 ▷ ▷ ▷ "\Da=2,b=3,c=4,d=5\D" \
11020 ▷ ▷ -end \
11021 ▷ ▷ -end \
11022
11023
11024
11025 surface_11_random_report:
11026 ▷ $(ORBITER) -v 10 \
11027 ▷ ▷ -define F -finite_field -q 11 -end \
11028 ▷ ▷ -define P -projective_space -n 3 -field F -v 6 -end \
11029 ▷ ▷ -define S -cubic_surface -space P -random -end \
11030 ▷ ▷ -with S -do \
11031 ▷ ▷ -cubic_surface_activity \
11032 ▷ ▷ ▷ -report \
11033 ▷ ▷ -end
11034 ▷ pdflatex surface_random_q11_report.tex
11035 ▷ $(OPEN) surface_random_q11_report.pdf
11036
11037 surface_11_1:
11038 ▷ $(ORBITER) -v 3 \
11039 ▷ ▷ -define F -finite_field -q 11 -end \
11040 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11041 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
11042 ▷ ▷ -with S -do \
11043 ▷ ▷ -cubic_surface_activity \
11044 ▷ ▷ ▷ -report \
11045 ▷ ▷ -end
11046 ▷ pdflatex surface_catalogue_q11_iso1_report.tex
11047 ▷ $(OPEN) surface_catalogue_q11_iso1_report.pdf
11048
11049 #####
11050 # Section 8.4: Classification of Cubic Surfaces with 27 lines
11051
11052
11053 SECTION_CLASSIFICATION_OF_CUBIC_SURFACES_WITH_27_LINES:
11054
11055
11056 test_8.4:
11057 ▷ make surface_classify_q4
11058 ▷ make surface_classify_q4_arc_lifting_two_lines
11059 ▷ make surface_classify_q7
11060 ▷ make surface_classify_q9

```

```

11061 ▷ make surface_classify_q11
11062 ▷ make surface_classify_q13
11063 ▷ make Family_general_F31_25_5_5_25_recognize
11064 ▷ make surface_F_abcd_identify_q11
11065 ▷ make Family_general_F11_iso_test
11066
11067
11068
11069 surface_classify_q4:
11070 ▷ $(ORBITER) -v 5 \
11071 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11072 ▷ ▷ -define F -finite_field -q 4 -end \
11073 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11074 ▷ ▷ -define Control -poset_classification_control -W \
11075 ▷ ▷ ▷ -problem_label "surf_q4" \
11076 ▷ ▷ -end \
11077 ▷ ▷ -define report_options -poset_classification_report_options \
11078 ▷ ▷ -end \
11079 ▷ ▷ -define Orb -orbits \
11080 ▷ ▷ ▷ -on_cubic_surfaces P Control \
11081 ▷ ▷ -end \
11082 ▷ ▷ -with Orb -do -orbits_activity \
11083 ▷ ▷ ▷ -report \
11084 ▷ ▷ -end
11085 ▷ pdflatex Surfaces_q4.tex
11086 ▷ $(OPEN) Surfaces_q4.pdf
11087
11088 #▷ ▷ ▷ -draw_poset \
11089
11090
11091 surface_classify_q4_old:
11092 ▷ $(ORBITER) -v 5 \
11093 ▷ ▷ -define F -finite_field -q 4 -end \
11094 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11095 ▷ ▷ -define Control -poset_classification_control -W \
11096 ▷ ▷ ▷ -problem_label "surf_q4" \
11097 ▷ ▷ -end \
11098 ▷ ▷ -with P -do \
11099 ▷ ▷ -projective_space_activity \
11100 ▷ ▷ ▷ -classify_surfaces_with_double_sixes Surf27 Control \
11101 ▷ ▷ -end \
11102 ▷ ▷ -with Surf27 -do \
11103 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11104 ▷ ▷ ▷ -report -end \
11105 ▷ ▷ -end \
11106 ▷ ▷ -with Surf27 -do \
11107 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11108 ▷ ▷ ▷ -create_source_code \
11109 ▷ ▷ -end \
11110 ▷ ▷ -with Surf27 -do \
11111 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11112 ▷ ▷ ▷ -stats surf_q4 \
11113 ▷ ▷ -end \
11114 ▷ ▷ -print_symbols
11115 ▷ pdflatex Surfaces_q4.tex
11116 ▷ $(OPEN) Surfaces_q4.pdf
11117
11118 # time: 0:00
11119

```

```

11120
11121
11122
11123 surface_classify_q4_arc_lifting_two_lines:
11124 ▷ $(ORBITER) -v 10 \
11125 ▷ ▷ -define F -finite_field -q 4 -end \
11126 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11127 ▷ ▷ -define Control -poset_classification_control \
11128 ▷ ▷ ▷ -problem_label sixarcs_q4 -W \
11129 ▷ ▷ -end \
11130 ▷ ▷ -with P -do \
11131 ▷ ▷ -projective_space_activity \
11132 ▷ ▷ ▷ -control_six_arcs Control \
11133 ▷ ▷ ▷ -classify_surfaces_through_arcs_and_two_lines \
11134 ▷ ▷ -end
11135 ▷ pdflatex surfaces_arc_lifting_4.tex
11136 ▷ $(OPEN) surfaces_arc_lifting_4.pdf
11137
11138
11139
11140 surface_classify_q7:
11141 ▷ $(ORBITER) -v 5 \
11142 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11143 ▷ ▷ -define F -finite_field -q 7 -end \
11144 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11145 ▷ ▷ -define Control -poset_classification_control -W \
11146 ▷ ▷ ▷ -problem_label "surf_q7" \
11147 ▷ ▷ ▷ -draw_options \
11148 ▷ ▷ ▷ ▷ -xin 400000 \
11149 ▷ ▷ ▷ ▷ -xout 3000000 \
11150 ▷ ▷ ▷ ▷ -yout 3000000 \
11151 ▷ ▷ ▷ ▷ -y_stretch 0.5 \
11152 ▷ ▷ ▷ -end \
11153 ▷ ▷ -end \
11154 ▷ ▷ -define report_options -poset_classification_report_options \
11155 ▷ ▷ ▷ -draw_poset -type_aux \
11156 ▷ ▷ -end \
11157 ▷ ▷ -define Orb -orbits \
11158 ▷ ▷ ▷ -on_cubic_surfaces P Control \
11159 ▷ ▷ -end \
11160 ▷ ▷ -with Orb -do -orbits_activity \
11161 ▷ ▷ ▷ -report \
11162 ▷ ▷ -end
11163 ▷ pdflatex Surfaces_q7.tex
11164 ▷ $(OPEN) Surfaces_q7.pdf
11165
11166
11167
11168 surface_classify_q7_old:
11169 ▷ $(ORBITER) -v 5 \
11170 ▷ ▷ -define F -finite_field -q 7 -end \
11171 ▷ ▷ -define P -projective_space -n 3 -field F -end \
11172 ▷ ▷ -define Control -poset_classification_control -W -end \
11173 ▷ ▷ -with P -do \
11174 ▷ ▷ -projective_space_activity \
11175 ▷ ▷ ▷ -classify_surfaces_with_double_sixes Surf27 Control \
11176 ▷ ▷ -end \
11177 ▷ ▷ -with Surf27 -do \
11178 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \

```



```

11179 ▷ ▷ ▷ -report -end \
11180 ▷ ▷ -end \
11181 ▷ ▷ -print_symbols
11182 ▷ pdflatex Surfaces_q7.tex
11183 ▷ $(OPEN) Surfaces_q7.pdf
11184
11185 #neighbors_7.csv
11186
11187
11188 surface_classify_q9:
11189 ▷ $(ORBITER) -v 5 \
11190 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11191 ▷ ▷ -define F -finite_field -q 9 -end \
11192 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11193 ▷ ▷ -define Control -poset_classification_control -W \
11194 ▷ ▷ ▷ -problem_label "surf_q9" \
11195 ▷ ▷ -end \
11196 ▷ ▷ -define report_options -poset_classification_report_options \
11197 ▷ ▷ -end \
11198 ▷ ▷ -define Orb -orbits \
11199 ▷ ▷ ▷ -on_cubic_surfaces P Control \
11200 ▷ ▷ -end \
11201 ▷ ▷ -with Orb -do -orbits_activity \
11202 ▷ ▷ ▷ -report \
11203 ▷ ▷ -end
11204 ▷ pdflatex Surfaces_q9.tex
11205 ▷ $(OPEN) Surfaces_q9.pdf
11206
11207
11208 surface_classify_q9_old:
11209 ▷ $(ORBITER) -v 5 \
11210 ▷ ▷ -define F -finite_field -q 9 -end \
11211 ▷ ▷ -define P -projective_space -n 3 -field F -end \
11212 ▷ ▷ -define Control -poset_classification_control -W -end \
11213 ▷ ▷ -with P -do \
11214 ▷ ▷ -projective_space_activity \
11215 ▷ ▷ ▷ -classify_surfaces_with_double_sixes Surf27 Control \
11216 ▷ ▷ -end \
11217 ▷ ▷ -with Surf27 -do \
11218 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11219 ▷ ▷ ▷ -report -end \
11220 ▷ ▷ -end \
11221 ▷ ▷ -print_symbols
11222 ▷ pdflatex Surfaces_q9.tex
11223 ▷ $(OPEN) Surfaces_q9.pdf
11224
11225
11226
11227 surface_classify_q11:
11228 ▷ $(ORBITER) -v 5 \
11229 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11230 ▷ ▷ -define F -finite_field -q 11 -end \
11231 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11232 ▷ ▷ -define Control -poset_classification_control -W \
11233 ▷ ▷ ▷ -problem_label "surf_q11" \
11234 ▷ ▷ -end \
11235 ▷ ▷ -define report_options -poset_classification_report_options \
11236 ▷ ▷ -end \
11237 ▷ ▷ -define Orb -orbits \

```

```

11238 ▷ ▷ ▷ -on_cubic_surfaces P Control \
11239 ▷ ▷ -end \
11240 ▷ ▷ -with Orb -do -orbits_activity \
11241 ▷ ▷ ▷ -report \
11242 ▷ ▷ -end
11243 ▷ pdflatex Surfaces.q11.tex
11244 ▷ $(OPEN) Surfaces.q11.pdf
11245
11246
11247 surface_classify.q13:
11248 ▷ $(ORBITER) -v 5 \
11249 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11250 ▷ ▷ -define F -finite_field -q 13 -end \
11251 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11252 ▷ ▷ -define Control -poset_classification_control -W \
11253 ▷ ▷ ▷ -problem_label "surf_q13" \
11254 ▷ ▷ -end \
11255 ▷ ▷ -define report_options -poset_classification_report_options \
11256 ▷ ▷ -end \
11257 ▷ ▷ -define Orb -orbits \
11258 ▷ ▷ ▷ -on_cubic_surfaces P Control \
11259 ▷ ▷ -end \
11260 ▷ ▷ -with Orb -do -orbits_activity \
11261 ▷ ▷ ▷ -report \
11262 ▷ ▷ -end
11263 ▷ pdflatex Surfaces.q13.tex
11264 ▷ $(OPEN) Surfaces.q13.pdf
11265
11266
11267
11268
11269 Family_general.F31_25_5_5_25_recognize:
11270 ▷ $(ORBITER) -v 3 \
11271 ▷ ▷ -define F -finite_field -q 31 -end \
11272 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11273 ▷ ▷ -define Surf -cubic_surface \
11274 ▷ ▷ ▷ -space P -family_general_abcd 25 5 5 25 \
11275 ▷ ▷ -end \
11276 ▷ ▷ -with Surf -do \
11277 ▷ ▷ -cubic_surface_activity \
11278 ▷ ▷ ▷ -report \
11279 ▷ ▷ -end \
11280 ▷ ▷ -define Control -poset_classification_control -W \
11281 ▷ ▷ ▷ -problem_label "surf_q31" \
11282 ▷ ▷ -end \
11283 ▷ ▷ -with P -do \
11284 ▷ ▷ -projective_space_activity \
11285 ▷ ▷ ▷ -classify_surfaces_with_double_sixes AllSurf Control \
11286 ▷ ▷ -end \
11287 ▷ ▷ -with AllSurf -do \
11288 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11289 ▷ ▷ ▷ -recognize Surf \
11290 ▷ ▷ -end
11291
11292 surface_F_abcd_identify.q11:
11293 ▷ $(ORBITER) -v 3 \
11294 ▷ ▷ -define F -finite_field -q 11 -end \
11295 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11296 ▷ ▷ -define Control -poset_classification_control -W \

```

```

11297 ▷ ▷ ▷ -problem_label "surf.q11" \
11298 ▷ ▷ -end \
11299 ▷ ▷ -with P -do \
11300 ▷ ▷ -projective_space_activity \
11301 ▷ ▷ ▷ -classify_surfaces_with_double_sixes AllSurf Control \
11302 ▷ ▷ -end \
11303 ▷ ▷ -with AllSurf -do \
11304 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11305 ▷ ▷ ▷ -identify_general_abcd \
11306 ▷ ▷ -end
11307
11308 Family_general_F11_iso_test:
11309 ▷ $(ORBITER) -v 3 \
11310 ▷ ▷ -define F -finite_field -q 11 -end \
11311 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11312 ▷ ▷ -define Surf1 -cubic_surface \
11313 ▷ ▷ ▷ -space P -family_general_abcd 2 4 4 2 \
11314 ▷ ▷ -end \
11315 ▷ ▷ -define Surf2 -cubic_surface \
11316 ▷ ▷ ▷ -space P -family_general_abcd 5 10 7 5 \
11317 ▷ ▷ -end \
11318 ▷ ▷ -define Control -poset_classification_control -W \
11319 ▷ ▷ ▷ -problem_label "surf.q11" \
11320 ▷ ▷ -end \
11321 ▷ ▷ -with P -do \
11322 ▷ ▷ -projective_space_activity \
11323 ▷ ▷ ▷ -classify_surfaces_with_double_sixes AllSurf Control \
11324 ▷ ▷ -end \
11325 ▷ ▷ -with AllSurf -do \
11326 ▷ ▷ -classification_of_cubic_surfaces_with_double_sixes_activity \
11327 ▷ ▷ ▷ -isomorphism_testing Surf1 Surf2 \
11328 ▷ ▷ -end
11329
11330
11331
11332
11333 #####
11334 # Section 8.5: Quartic Curves Creation
11335
11336
11337 SECTION_QUARTIC_CURVES_CREATION:
11338
11339
11340 test_8.5:
11341 ▷ make quartic_curve_9_0
11342 ▷ make quartic_curve_edge_q9
11343 ▷ make surface_7_0_make_quartic_curves
11344 ▷ make surface_9_1_make_quartic_curves
11345 ▷ make quartic_curve_9_0_report
11346 ▷ make quartic_curve_13_0_report
11347 ▷ make PG_2_13_rank_lines
11348 ▷ make PG_2_13_orbits_on_lines
11349 ▷ make quartic_curve_13_1_report
11350 ▷ make quartic_curves_19_report
11351 ▷ make quartic_curve_q9_1
11352
11353
11354 quartic_curve_9_0:
11355 ▷ $(ORBITER) -v 3 \

```

```

11356 ▷ ▷ -define F -finite_field -q 9 -end \
11357 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11358 ▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end
11359
11360
11361
11362 # the coefficient 1 is actually 4 mod 3:
11363
11364
11365 quartic_curve_edge_q9:
11366 ▷ $(ORBITER) -v 3 \
11367 ▷ ▷ -define F -finite_field -q 9 -end \
11368 ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
11369 ▷ ▷ -define R -polynomial_ring \
11370 ▷ ▷ ▷ -field F \
11371 ▷ ▷ ▷ -number_of_variables 3 \
11372 ▷ ▷ ▷ -homogeneous_of_degree 4 \
11373 ▷ ▷ ▷ -monomial_ordering_partition \
11374 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
11375 ▷ ▷ -end \
11376 ▷ ▷ -define Edge -symbolic_object \
11377 ▷ ▷ ▷ -field F \
11378 ▷ ▷ ▷ -managed_variables "X,Y,Z" \
11379 ▷ ▷ ▷ -text "X^4-Y^4-Z^4+2*f^2*Y^2*Z^2+1*f*X^2*Y*Z" \
11380 ▷ ▷ -end \
11381 ▷ ▷ -define f -symbolic_object \
11382 ▷ ▷ ▷ -field F \
11383 ▷ ▷ ▷ -text "3" \
11384 ▷ ▷ -end \
11385 ▷ ▷ -define Edge_f3 -symbolic_object \
11386 ▷ ▷ ▷ -field F \
11387 ▷ ▷ ▷ -managed_variables "X,Y,Z" \
11388 ▷ ▷ ▷ -substitute "f" Edge f \
11389 ▷ ▷ -end \
11390 ▷ ▷ -define C -quartic_curve \
11391 ▷ ▷ ▷ -space P2 \
11392 ▷ ▷ ▷ -by_symbolic_object R Edge_f3 -end \
11393 ▷ ▷ -with C -do \
11394 ▷ ▷ ▷ -quartic_curve_activity \
11395 ▷ ▷ ▷ -report \
11396 ▷ ▷ -end
11397 ▷ pdflatex quartic_curve_equation_Edge_f3_q9_report.tex
11398 ▷ $(OPEN) quartic_curve_equation_Edge_f3_q9_report.pdf
11399
11400
11401
11402 surface_7_0_make_quartic_curves:
11403 ▷ $(ORBITER) -v 3 \
11404 ▷ ▷ -define F -finite_field -q 7 -end \
11405 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11406 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
11407 ▷ ▷ -with S -do \
11408 ▷ ▷ -cubic_surface_activity \
11409 ▷ ▷ ▷ -report \
11410 ▷ ▷ ▷ -all_quartic_curves \
11411 ▷ ▷ -end \
11412 ▷ #pdflatex surface_catalogue_q7_iso0_report.tex
11413 ▷ #$(OPEN) surface_catalogue_q7_iso0_report.pdf
11414

```

```

11415
11416
11417
11418 surface_9_1_make_quartic_curves:
11419 ▷ $(ORBITER) -v 3 \
11420 ▷ ▷ -define F -finite_field -q 9 -end \
11421 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11422 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
11423 ▷ ▷ -with S -do \
11424 ▷ ▷ -cubic_surface_activity \
11425 ▷ ▷ ▷ -report \
11426 ▷ ▷ ▷ -all_quartic_curves \
11427 ▷ ▷ -end \
11428 ▷ #pdflatex surface_catalogue.q9_iso1_report.tex
11429 ▷ #$(OPEN) surface_catalogue.q9_iso1_report.pdf
11430
11431
11432 quartic_curve_9_0_report:
11433 ▷ $(ORBITER) -v 3 \
11434 ▷ ▷ -define F -finite_field -q 9 -end \
11435 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11436 ▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end \
11437 ▷ ▷ -with C -do \
11438 ▷ ▷ ▷ -quartic_curve_activity \
11439 ▷ ▷ ▷ -report \
11440 ▷ ▷ ▷ -end
11441 ▷ pdflatex quartic_curve_catalogue.q9_iso0_report.tex
11442 ▷ $(OPEN) quartic_curve_catalogue.q9_iso0_report.pdf
11443
11444
11445
11446
11447 quartic_curve_13_0_report:
11448 ▷ $(ORBITER) -v 3 \
11449 ▷ ▷ -define F -finite_field -q 13 -end \
11450 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11451 ▷ ▷ -define C -quartic_curve -space P -catalogue 0 \
11452 ▷ ▷ ▷ -transform "10,4,1,11,5,11,4,1,1" \
11453 ▷ ▷ ▷ -transform_inverse "9,1,0,12,9,0,2,10,11" \
11454 ▷ ▷ -end \
11455 ▷ ▷ -with C -do \
11456 ▷ ▷ ▷ -quartic_curve_activity \
11457 ▷ ▷ ▷ -report \
11458 ▷ ▷ ▷ -end \
11459 ▷ ▷ -with C -do \
11460 ▷ ▷ ▷ -quartic_curve_activity \
11461 ▷ ▷ ▷ -extract_orbit_on_bitangents_by_length 4 \
11462 ▷ ▷ ▷ -end
11463 ▷ #pdflatex quartic_curve_catalogue.q13_iso0_report.tex
11464 ▷ #$(OPEN) quartic_curve_catalogue.q13_iso0_report.pdf
11465
11466 # 170, 111, 140, 2
11467
11468
11469 PG_2_13_rank_lines:
11470 ▷ $(ORBITER) -v 2 \
11471 ▷ ▷ -define v -vector -format 4 \
11472 ▷ ▷ -dense "1,0,0, 0,1,0, 1,0,0, 0,0,1, 1,1,1, 0,1,0, 1,1,1, 0,0,1" \
11473 ▷ ▷ -end \

```

```

11474 ▷ ▷ -define F -finite_field -q 23 -end \
11475 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11476 ▷ ▷ -with P -do \
11477 ▷ ▷ -projective_space_activity \
11478 ▷ ▷ ▷ -rank_lines_in_PG v \
11479 ▷ ▷ -end \
11480
11481
11482 PG_2_13_orbits_on_lines:
11483 ▷ $(ORBITER) -v 5 \
11484 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11485 ▷ ▷ -define Control -poset_classification_control \
11486 ▷ ▷ ▷ -problem_label PGL_3_13 \
11487 ▷ ▷ ▷ -depth 4 \
11488 ▷ ▷ ▷ -draw_options -radius 200 -end \
11489 ▷ ▷ -end \
11490 ▷ ▷ -define G -linear_group -PGL 3 13 -end \
11491 ▷ ▷ -define G.on_lines -modified_group -from G \
11492 ▷ ▷ ▷ -on_k_subspaces 2 \
11493 ▷ ▷ -end \
11494 ▷ ▷ -define Orb -orbits -group G.on_lines \
11495 ▷ ▷ ▷ -on_subsets 4 Control \
11496 ▷ ▷ -end \
11497 ▷ ▷ -with Orb -do -orbits_activity \
11498 ▷ ▷ ▷ -recognize "170, 111, 140, 2" \
11499 ▷ ▷ ▷ -recognize "0,23,24,47" \
11500 ▷ ▷ -end
11501 ▷ #pdflatex PGL_3_13_poset.tex
11502 ▷ #$(OPEN) PGL_3_13_poset.pdf
11503
11504 # stabilizer of {0,23,24,47}
11505 #1,0,0,7,9,0,9,5,3,
11506 #1,3,0,1,12,0,10,9,2,
11507 #1,1,11,7,9,6,10,5,2,
11508 #1,4,11,12,1,8,10,4,2,
11509
11510 quartic_curve_13_1_report:
11511 ▷ $(ORBITER) -v 3 \
11512 ▷ ▷ -define F -finite_field -q 13 -end \
11513 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11514 ▷ ▷ -define C -quartic_curve -space P -catalogue 1 -end \
11515 ▷ ▷ -with C -do \
11516 ▷ ▷ ▷ -quartic_curve_activity \
11517 ▷ ▷ ▷ -report \
11518 ▷ ▷ -end
11519 ▷ pdflatex quartic_curve_catalogue_q13_iso1_report.tex
11520 ▷ $(OPEN) quartic_curve_catalogue_q13_iso1_report.pdf
11521
11522
11523
11524 quartic_curves_19_report:
11525 ▷ $(ORBITER) -v 3 \
11526 ▷ ▷ -define F -finite_field -q 19 -end \
11527 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11528 ▷ ▷ -loop L 0 $(NB_QUARTIC_CURVES_Q19) 1 \
11529 ▷ ▷ -define C -quartic_curve -space P -catalogue %L -end \
11530 ▷ ▷ -with C -do \
11531 ▷ ▷ ▷ -quartic_curve_activity \
11532 ▷ ▷ ▷ -report \

```

```

11533 ▷ ▷ ▷ -end \
11534 ▷ ▷ -end_loop L
11535 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso0_report.tex
11536 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso1_report.tex
11537 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso2_report.tex
11538 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso3_report.tex
11539 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso4_report.tex
11540 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso5_report.tex
11541 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso6_report.tex
11542 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso7_report.tex
11543 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso8_report.tex
11544 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso9_report.tex
11545 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso10_report.tex
11546 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso11_report.tex
11547 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso12_report.tex
11548 ▷ ▷ pdflatex quartic_curve_catalogue_q19_iso13_report.tex
11549
11550
11551
11552 quartic_curve_q9_1:
11553 ▷ $(ORBITER) -v 3 \
11554 ▷ ▷ -define F -finite_field -q 9 -end \
11555 ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
11556 ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
11557 ▷ ▷ -define S9_1 -cubic_surface -space P3 -catalogue 1 -end \
11558 ▷ ▷ -define C -quartic_curve -space P2 -from_cubic_surface S9_1 0 -end \
11559 ▷ ▷ -with C -do \
11560 ▷ ▷ ▷ -quartic_curve_activity \
11561 ▷ ▷ ▷ ▷ -report \
11562 ▷ ▷ ▷ -end
11563 ▷ pdflatex quartic_curve_surface_surface_pt_orb_0_report.tex
11564 ▷ $(OPEN) quartic_curve_surface_surface_pt_orb_0_report.pdf
11565
11566
11567 #The points by rank are: ( 5, 18, 25, 36, 39, 40, 42, 43, 47, 50, 51, 54, 55, 58,
    59, 62, 68, 69, 70, 71, 76, 77, 79, 81, 85, 87, 89, 90 )
11568
11569 #eqn15:
11570 #(1,1,1,1,5,1,7,7,5,0,0,0,0,0,0)
11571
11572
11573 # the 27 single points (of the surface) are:
11574 #6, 17, 81, 118, 146, 192, 229, 265, 332, 351, 362, 386, 412, 432, 488, 505, 531,
    544, 576, 666, 715, 717, 723, 739, 760, 762, 805
11575
11576
11577
11578
11579
11580 #####
11581 # Section 8.6: Quartic Curve Activities
11582
11583
11584 SECTION_QUARTIC_CURVE_ACTIVITIES:
11585
11586
11587 test_8.6:
11588 ▷ make quartic_curves.q7
11589 ▷ make quartic_curves.q13

```

```

11590 ▷ make quartic_curves.q13.combine
11591 ▷ make quartic_curves.q13.classify
11592 ▷ make quartic_curves.q19
11593 ▷ make quartic_curves.q19.combine
11594 ▷ make quartic_curves.q19.classify
11595
11596
11597
11598
11599
11600 quartic_curves.q7:
11601 ▷ $(ORBITER) -v 3 \
11602 ▷ ▷ -define F -finite_field -q 7 -end \
11603 ▷ ▷ -define P -projective_space -n 3 -field F -end \
11604 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
11605 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
11606 ▷ ▷ -end_loop L \
11607 ▷ ▷ -print_symbols \
11608 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
11609 ▷ ▷ ▷ -with S_%L -do \
11610 ▷ ▷ ▷ -cubic_surface_activity \
11611 ▷ ▷ ▷ -export_all_quartic_curves \
11612 ▷ ▷ ▷ -end \
11613 ▷ ▷ -end_loop L \
11614 ▷ ▷ -print_symbols
11615
11616 quartic_curves.q13:
11617 ▷ $(ORBITER) -v 3 \
11618 ▷ ▷ -define F -finite_field -q 13 -end \
11619 ▷ ▷ -define P -projective_space -n 3 -field F -end \
11620 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
11621 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
11622 ▷ ▷ -end_loop L \
11623 ▷ ▷ -print_symbols \
11624 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
11625 ▷ ▷ ▷ -with S_%L -do \
11626 ▷ ▷ ▷ -cubic_surface_activity \
11627 ▷ ▷ ▷ -export_all_quartic_curves \
11628 ▷ ▷ ▷ -end \
11629 ▷ ▷ -end_loop L \
11630 ▷ ▷ -print_symbols
11631
11632 quartic_curves.q13.combine:
11633 ▷ $(ORBITER) -v 3 \
11634 ▷ ▷ -csv_file_concatenate_from_mask $(NB_CUBIC_SURFACES_Q13) \
11635 ▷ ▷ ▷ surface_catalogue.q13_iso%ld_quartics.csv \
11636 ▷ ▷ ▷ quartics.q13.csv
11637
11638
11639 quartic_curves.q13.classify:
11640 ▷ $(ORBITER) -v 30 \
11641 ▷ ▷ -define F -finite_field -q 13 -end \
11642 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11643 ▷ ▷ -define R -polynomial_ring \
11644 ▷ ▷ ▷ -field F \
11645 ▷ ▷ ▷ -number_of_variables 3 \
11646 ▷ ▷ ▷ -homogeneous_of_degree 4 \
11647 ▷ ▷ ▷ -monomial_ordering_partition \
11648 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \

```



```

11649 ▷ ▷ -end \
11650 ▷ ▷ -define O -orbits -classification_by_canonical_form \
11651 ▷ ▷ ▷ -space P \
11652 ▷ ▷ ▷ -ring R \
11653 ▷ ▷ ▷ -input_fname_mask quartics.q13.csv \
11654 ▷ ▷ ▷ -nb_files 1 \
11655 ▷ ▷ ▷ -output_fname quartic_curves.q13.classified \
11656 ▷ ▷ ▷ -label_po_go "PO_GO" \
11657 ▷ ▷ ▷ -label_po_index "PO_INDEX" \
11658 ▷ ▷ ▷ -label_po "PO" \
11659 ▷ ▷ ▷ -label_so "orbit" \
11660 ▷ ▷ ▷ -label_equation "curve" \
11661 ▷ ▷ ▷ -label_points "pts_on_curve" \
11662 ▷ ▷ ▷ -label_lines "bitangents" \
11663 ▷ ▷ ▷ -carry_through "NB_E" \
11664 ▷ ▷ ▷ -carry_through "NB_DOUBLE" \
11665 ▷ ▷ ▷ -carry_through "NB_SINGLE" \
11666 ▷ ▷ ▷ -carry_through "NB_ZERO" \
11667 ▷ ▷ ▷ -algorithm_substructure \
11668 ▷ ▷ ▷ -substructure_size 4 \
11669 ▷ ▷ ▷ -end \
11670 ▷ ▷ -end \
11671 ▷ -with 0 -do -orbits_activity \
11672 ▷ ▷ -report \
11673 ▷ ▷ -report_options \
11674 ▷ ▷ ▷ -fname quartics.q13 \
11675 ▷ ▷ -end \
11676 ▷ -end
11677 ▷ pdflatex quartics.q13.orbits.tex
11678 ▷ $(OPEN) quartics.q13.orbits.pdf
11679
11680
11681
11682 quartic_curves.q19:
11683 ▷ $(ORBITER) -v 3 \
11684 ▷ ▷ -define F -finite_field -q 19 -end \
11685 ▷ ▷ -define P -projective_space -n 3 -field F -end \
11686 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q19) 1 \
11687 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
11688 ▷ ▷ -end_loop L \
11689 ▷ ▷ -print_symbols \
11690 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q19) 1 \
11691 ▷ ▷ ▷ -with S_%L -do \
11692 ▷ ▷ ▷ -cubic_surface_activity \
11693 ▷ ▷ ▷ -export_all_quartic_curves \
11694 ▷ ▷ ▷ -end \
11695 ▷ ▷ -end_loop L \
11696 ▷ ▷ -print_symbols
11697
11698
11699 quartic_curves.q19.combine:
11700 ▷ $(ORBITER) -v 3 \
11701 ▷ ▷ -csv_file_concatenate_from_mask $(NB_CUBIC_SURFACES_Q19) \
11702 ▷ ▷ ▷ surface_catalogue.q19_iso%ld.quartics.csv \
11703 ▷ ▷ ▷ quartics.q19.csv
11704
11705
11706
11707 quartic_curves.q19.classify:

```

```

11708 ▷ $(ORBITER) -v 3 \
11709 ▷ ▷ -define F -finite_field -q 19 -end \
11710 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11711 ▷ ▷ -define R -polynomial_ring \
11712 ▷ ▷ ▷ -field F \
11713 ▷ ▷ ▷ -number_of_variables 3 \
11714 ▷ ▷ ▷ -homogeneous_of_degree 4 \
11715 ▷ ▷ ▷ -monomial_ordering_partition \
11716 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
11717 ▷ ▷ -end \
11718 ▷ ▷ -define O -orbits -classification_by_canonical_form \
11719 ▷ ▷ ▷ -space P \
11720 ▷ ▷ ▷ -ring R \
11721 ▷ ▷ ▷ -input_fname_mask quartics_q19.csv \
11722 ▷ ▷ ▷ -nb_files 1 \
11723 ▷ ▷ ▷ -output_fname quartic_curves_q19_classified \
11724 ▷ ▷ ▷ -label_po_go "P0.GO" \
11725 ▷ ▷ ▷ -label_po_index "P0.INDEX" \
11726 ▷ ▷ ▷ -label_po "P0" \
11727 ▷ ▷ ▷ -label_so "orbit" \
11728 ▷ ▷ ▷ -label_equation "curve" \
11729 ▷ ▷ ▷ -label_points "pts_on_curve" \
11730 ▷ ▷ ▷ -label_lines "bitangents" \
11731 ▷ ▷ ▷ -carry_through "NB.E" \
11732 ▷ ▷ ▷ -carry_through "NB.DOUBLE" \
11733 ▷ ▷ ▷ -carry_through "NB.SINGLE" \
11734 ▷ ▷ ▷ -carry_through "NB.ZERO" \
11735 ▷ ▷ ▷ -algorithm_substructure \
11736 ▷ ▷ ▷ -substructure_size 4 \
11737 ▷ ▷ ▷ -end \
11738 ▷ ▷ -end \
11739 ▷ -with 0 -do -orbits_activity \
11740 ▷ ▷ -report \
11741 ▷ ▷ -report_options \
11742 ▷ ▷ ▷ -fname quartics_q19 \
11743 ▷ ▷ -end \
11744 ▷ -end
11745 ▷ pdflatex quartics_q19_orbits.tex
11746 ▷ $(OPEN) quartics_q19_orbits.pdf
11747
11748
11749
11750
11751
11752
11753 #####
11754 # Section 8.7: Interfaces
11755
11756 SECTION_CUBIC_SURFACES_INTERFACES:
11757
11758
11759 test_8.7:
11760 ▷ make surface_4.0_export_gap
11761
11762 surface_4.0_export_gap:
11763 ▷ $(ORBITER) -v 3 \
11764 ▷ ▷ -define F -finite_field -q 4 -end \
11765 ▷ ▷ -define P -projective_space -n 3 -field F -end \
11766 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \

```

```

11767 ▷ ▷ -with S -do \
11768 ▷ ▷ -cubic_surface_activity \
11769 ▷ ▷ ▷ -export_gap \
11770 ▷ ▷ -end
11771
11772
11773
11774
11775 #####
11776 # Chapter 9 - Applications
11777 #####
11778
11779
11780 test_9:
11781 ▷ make test_9.1
11782 ▷ make test_9.2
11783 ▷ make test_9.3
11784
11785
11786
11787 #####
11788 # Section 9.1: Number Theory
11789
11790
11791 SECTION_NUMBER_THEORY:
11792
11793
11794 test_9.1:
11795 ▷ make PR101
11796 ▷ make PR101_smallest
11797 ▷ make PR29
11798 ▷ make PR31
11799 ▷ make PR37
11800 ▷ make PR_2_100
11801 ▷ make PR_2_1000
11802 ▷ make PE_number_2_1000
11803 ▷ make number_of_primitive_roots_10000
11804 ▷ make IM_3_19
11805 ▷ make IM_723
11806 ▷ make IM
11807 ▷ make IM_gcd
11808 ▷ make PM3a
11809 ▷ make PR_915839
11810 ▷ make PR_915839_check
11811 ▷ make DL_915839
11812 ▷ make sqrt_mod
11813 ▷ make sqrt_5_mod_11
11814 ▷ make sqrt_5_mod_19
11815 ▷ make sqrt_mod_20_31
11816 ▷ make order_of_2_mod_n
11817 ▷ make Eulerfunction_150
11818 ▷ make Eulerfunction_900
11819 ▷ make Eulerfunction_10000
11820 ▷ make power_function_2_mod_11
11821 ▷ make draw_mod_13
11822 ▷ make inverse_mod_a
11823 ▷ make smith_normal_form_1
11824 ▷ make smith_normal_form_2
11825 ▷ make jacobi_35_41

```

```

11826 ▷ make jacobi_33_41
11827 ▷ make jacobi_a
11828 ▷ make jacobi_5_19
11829 ▷ make sqrt_mod_7817
11830 ▷ make Chinese_remainders_A
11831 ▷ make Chinese_remainders_B
11832 ▷ make Chinese_remainders_C2
11833 ▷ make Chinese_remainders_C3
11834
11835
11836
11837 PR101:
11838 ▷ $(ORBITER) -v 5 -primitive_root 101
11839
11840
11841 PR101_smallest:
11842 ▷ $(ORBITER) -v 5 -smallest_primitive_root 101
11843
11844
11845 PR29:
11846 ▷ $(ORBITER) -v 1 -smallest_primitive_root 29
11847
11848 PR31:
11849 ▷ $(ORBITER) -v 1 -smallest_primitive_root 31
11850
11851 PR37:
11852 ▷ $(ORBITER) -v 1 -smallest_primitive_root 37
11853
11854 PR_2_100:
11855 ▷ $(ORBITER) -v 1 -smallest_primitive_root_interval 2 100
11856
11857
11858
11859
11860 PR_2_1000:
11861 ▷ $(ORBITER) -v 1 -smallest_primitive_root_interval 2 1000
11862 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
11863 ▷ primitive_element_table_2_1000.csv
11864 ▷ pdflatex primitive_element_table_2_1000.tex
11865 ▷ $(OPEN) primitive_element_table_2_1000.pdf
11866
11867 PE_number_2_1000:
11868 ▷ $(ORBITER) -v 1 -number_of_primitive_roots_interval 2 1000
11869 ▷ $(ORBITER) -v 1 -csv_file_latex 1 table_number_of_pe_2_1000.csv
11870 ▷ pdflatex table_number_of_pe_2_1000.tex
11871 ▷ $(OPEN) table_number_of_pe_2_1000.pdf
11872
11873 number_of_primitive_roots_10000:
11874 ▷ $(ORBITER) -v 1 -number_of_primitive_roots_interval 10000 10001
11875
11876
11877
11878 IM_3_19:
11879 ▷ $(ORBITER) -v 5 -inverse_mod 3 19
11880
11881
11882 IM_723:
11883 ▷ $(ORBITER) -v 5 -inverse_mod 723 4060
11884

```

```

11885
11886
11887 IM:
11888 ▷ $(ORBITER) -v 5 -inverse_mod 1865025205 2147483647
11889
11890
11891 IM_gcd:
11892 ▷ $(ORBITER) -v 5 -extended_gcd 1865025205 2147483647
11893
11894
11895 PM3a:
11896 ▷ $(ORBITER) -v 5 -power_mod 16807 1073741823 2147483647
11897
11898 # randomized algo:
11899
11900 PR_915839:
11901 ▷ $(ORBITER) -v 5 -primitive_root 915839
11902
11903 #a primitive root modulo 915839 is 43085
11904
11905 PR_915839_check:
11906 ▷ $(ORBITER) -v 5 -power_mod 43085 49842 915839
11907
11908 #the power of 43085 to the 49842 mod 915839 is 487320
11909
11910 DL_915839:
11911 ▷ $(ORBITER) -v 5 -discrete_log 487320 43085 915839
11912
11913
11914 #The discrete log is 49842 since  $487320 = 43085^{49842} \bmod 915839$ , time: 0:22
11915
11916 sqrt_mod:
11917 ▷ $(ORBITER) -v 2 -square_root_mod 33 41
11918
11919
11920 sqrt_5_mod_11:
11921 ▷ $(ORBITER) -v 2 -square_root_mod 5 11
11922
11923
11924 sqrt_5_mod_19:
11925 ▷ $(ORBITER) -v 2 -square_root_mod 5 19
11926
11927
11928 sqrt_mod_20_31:
11929 ▷ $(ORBITER) -v 2 -square_root_mod 20 31
11930
11931 order_of_2_mod_n:
11932 ▷ $(ORBITER) -v 3 -order_of_q_mod_n 2 3 51
11933 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
11934 ▷   order_of_q_mod_n_q2_3_51.csv
11935 ▷ pdflatex order_of_q_mod_n_q2_3_51.tex
11936 ▷ $(OPEN) order_of_q_mod_n_q2_3_51.pdf
11937
11938
11939
11940
11941
11942
11943 Eulerfunction_150:

```

```

11944 ▷ $(ORBITER) -v 1 -eulerfunction_interval 1 150
11945 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
11946 ▷ ▷ table_eulerfunction_1_150.csv
11947 ▷ pdflatex table_eulerfunction_1_150.tex
11948 ▷ $(OPEN) table_eulerfunction_1_150.pdf
11949
11950 Eulerfunction_900:
11951 ▷ $(ORBITER) -v 1 -eulerfunction_interval 900 1150
11952 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
11953 ▷ ▷ table_eulerfunction_900_1150.csv
11954 ▷ pdflatex table_eulerfunction_900_1150.tex
11955 ▷ $(OPEN) table_eulerfunction_900_1150.pdf
11956
11957 Eulerfunction_10000:
11958 ▷ $(ORBITER) -v 1 -eulerfunction_interval 10000 10150
11959 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
11960 ▷ ▷ table_eulerfunction_10000_10150.csv
11961 ▷ pdflatex table_eulerfunction_10000_10150.tex
11962 ▷ $(OPEN) table_eulerfunction_10000_10150.pdf
11963
11964
11965
11966 power_function_2_mod_11:
11967 ▷ $(ORBITER) -v 5 -power_function_mod_n 2 11
11968 ▷ $(ORBITER) -v 1 -csv_file_latex 1 power_function_k2_n11.csv
11969 ▷ pdflatex power_function_k2_n11.tex
11970 ▷ $(OPEN) power_function_k2_n11.pdf
11971
11972
11973 draw_mod_13:
11974 ▷ $(ORBITER) -v 2 \
11975 ▷ ▷ -draw_options -embedded -end \
11976 ▷ ▷ -draw_mod_n \
11977 ▷ ▷ ▷ -n 13 \
11978 ▷ ▷ ▷ -file mod_13 \
11979 ▷ ▷ ▷ -power_cycle 2 \
11980 ▷ ▷ -end
11981 ▷ pdflatex mod_13.draw.tex
11982 ▷ $(OPEN) mod_13.draw.pdf
11983
11984
11985
11986
11987 inverse_mod_26_99:
11988 ▷ $(ORBITER) -v 2 -inverse_mod 26 99
11989
11990
11991 inverse_mod_a:
11992 ▷ $(ORBITER) -v 2 -inverse_mod 18059241 58014043
11993
11994
11995
11996 smith_normal_form_1:
11997 ▷ $(ORBITER) -v 3 \
11998 ▷ ▷ -define M -vector \
11999 ▷ ▷ ▷ -dense "1,2,3,4,5,6,7,8,9" \
12000 ▷ ▷ ▷ -format 3 \
12001 ▷ ▷ -end \
12002 ▷ ▷ -smith_normal_form M

```

```

12003
12004
12005 smith_normal_form_2:
12006 ▷ $(ORBITER) -v 3 \
12007 ▷ ▷ -define M -vector \
12008 ▷ ▷ ▷ -dense "9,8,7,6,5,4,3,2,1" \
12009 ▷ ▷ ▷ -format 3 \
12010 ▷ ▷ -end \
12011 ▷ ▷ -smith_normal_form M
12012
12013
12014
12015 jacobi_35_41:
12016 ▷ $(ORBITER) -v 5 -jacobi 35 41
12017 ▷ pdflatex jacobi_35_41.tex
12018 ▷ $(OPEN) jacobi_35_41.pdf
12019
12020
12021 jacobi_33_41:
12022 ▷ $(ORBITER) -v 5 -jacobi 33 41
12023 ▷ pdflatex jacobi_33_41.tex
12024 ▷ $(OPEN) jacobi_33_41.pdf
12025
12026
12027 jacobi_a:
12028 ▷ $(ORBITER) -v 5 -jacobi 2221 7817
12029
12030 jacobi_5_19:
12031 ▷ $(ORBITER) -v 5 -jacobi 5 19
12032
12033 sqrt_mod_7817:
12034 ▷ ▷ $(ORBITER) -v 2 -square_root_mod 2221 7817
12035
12036
12037
12038
12039
12040 Chinese_remainders_A:
12041 ▷ $(ORBITER) -v 2 \
12042 ▷ ▷ -define R -vector -dense "2,2,5" -end \
12043 ▷ ▷ -define M -vector -dense "5,6,7" -end \
12044 ▷ ▷ -Chinese_remainders R M
12045
12046 Chinese_remainders_B:
12047 ▷ $(ORBITER) -v 2 \
12048 ▷ ▷ -define R -vector -dense "38,2" -end \
12049 ▷ ▷ -define M -vector -dense "74,27" -end \
12050 ▷ ▷ -Chinese_remainders R M
12051
12052 Chinese_remainders_C2:
12053 ▷ $(ORBITER) -v 2 \
12054 ▷ ▷ -define R -vector -dense "2,3" -end \
12055 ▷ ▷ -define M -vector -dense "2147483647,5915587277" -end \
12056 ▷ ▷ -Chinese_remainders R M
12057
12058
12059 #The solution is 5684294357108828365 modulo 12703626939758759219 (computed in lon
integer)
12060

```

```

12061 # checking with Maple:
12062 #5684294357108828365 mod 2147483647;
12063 # 2
12064
12065 #5684294357108828365 mod 5915587277;
12066 # 3
12067
12068
12069
12070 Chinese_remainders_C3:
12071 ▷ $(ORBITER) -v 2 \
12072 ▷ ▷ -define R -vector -dense "2,3,4" -end \
12073 ▷ ▷ -define M -vector -dense "2147483647,5915587277,3267000013" -end \
12074 ▷ ▷ -Chinese_remainders R M
12075
12076
12077 #The solution is 31431541759324477327451572539 modulo 415027493773390165853368698
12078 47 (computed in longinteger)
12079
12079 # checking with Maple:
12080 #31431541759324477327451572539 mod 2147483647;
12081 # 2
12082
12083 #31431541759324477327451572539 mod 5915587277;
12084 # 3
12085
12086 #31431541759324477327451572539 mod 3267000013;
12087 # 4
12088
12089
12090
12091
12092
12093
12094
12095
12096 #####
12097 # Section 9.2: Representation Theory
12098
12099 SECTION_REPRESENTATION_THEORY:
12100
12101
12102 test_9.2:
12103 ▷ make representation_on_polynomials_of_degree_3
12104 ▷ make representation_tetrahedral_group_on_polynomials_of_degree_3
12105
12106
12107 representation_on_polynomials_of_degree_3:
12108 ▷ $(ORBITER) -v 4 \
12109 ▷ ▷ -define F -finite_field -q 3 -end \
12110 ▷ ▷ -define G -linear_group -PGL 4 F -end \
12111 ▷ ▷ -define R -polynomial_ring \
12112 ▷ ▷ ▷ -field F \
12113 ▷ ▷ ▷ -number_of_variables 4 \
12114 ▷ ▷ ▷ -homogeneous_of_degree 3 \
12115 ▷ ▷ ▷ -monomial_ordering_partition \
12116 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
12117 ▷ ▷ -end \
12118 ▷ ▷ -with G -do \

```



```

12119 ▷ ▷ -group_theoretic_activity \
12120 ▷ ▷ ▷ -representation_on_polynomials R \
12121 ▷ ▷ -end
12122 ▷ $(ORBITER) -v 2 \
12123 ▷ ▷ -loop L 0 9 1 -draw_matrix \
12124 ▷ ▷ ▷ -input_csv_file PGL_4_3_rep_3_%L.csv \
12125 ▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 20 20 -end \
12126 ▷ ▷ -end_loop L
12127
12128
12129
12130
12131 representation_tetrahedral_group_on_polynomials_of_degree_3:
12132 ▷ $(ORBITER) -v 4 \
12133 ▷ ▷ -define F -finite_field -q 3 -end \
12134 ▷ ▷ -define G -linear_group -GL 3 F \
12135 ▷ ▷ -subgroup_by_generators "tetra" "12" 2 \
12136 ▷ ▷ "0,1,0,0,0,1,1,0,0, 0,0,1,2,0,0,0,2,0" \
12137 ▷ ▷ -end \
12138 ▷ ▷ -define R -polynomial_ring \
12139 ▷ ▷ ▷ -field F \
12140 ▷ ▷ ▷ -number_of_variables 3 \
12141 ▷ ▷ ▷ -homogeneous_of_degree 3 \
12142 ▷ ▷ ▷ -monomial_ordering_partition \
12143 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
12144 ▷ ▷ -end \
12145 ▷ ▷ -with G -do \
12146 ▷ ▷ -group_theoretic_activity \
12147 ▷ ▷ ▷ -representation_on_polynomials R \
12148 ▷ ▷ -end
12149 ▷ $(ORBITER) -v 2 -loop L 0 2 1 -draw_matrix \
12150 ▷ ▷ -input_csv_file GL_3_3_Subgroup_tetra_12_rep_3_%L.csv \
12151 ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 10 10 -end -end_loop L
12152 ▷ $(OPEN) GL_3_3_Subgroup_tetra_12_rep_3_0.draw.bmp
12153 ▷ $(OPEN) GL_3_3_Subgroup_tetra_12_rep_3_1.draw.bmp
12154
12155 # write GL_3_3_Subgroup_tetra_12_rep_3_0.csv
12156
12157
12158 #####
12159 # Section 9.3: Cryptography
12160
12161
12162 SECTION_CRYPTOGRAPHY:
12163
12164
12165
12166 test_9_3:
12167 ▷ make EC_add
12168 ▷ make EC_cyclic_subgroup
12169 ▷ make EC_points_13
12170 ▷ make EC_points_199
12171 ▷ make EC_Koblitz_encoding
12172 ▷ make EC_bsgs
12173 ▷ make EC_bsgs_decode
12174 ▷ make NTRU_Alice1
12175 ▷ make NTRU_Alice2
12176 ▷ make NTRU_Alice_public_key
12177 ▷ make NTRU_encrypt

```

```

12178 ▷ make NTRU_decrypt1
12179 ▷ make NTRU_decrypt2
12180 ▷ make NTRU_decrypt3
12181 ▷ make NTRU_decrypt4
12182 ▷ make NTRU_decrypt5
12183 ▷ make inv_59_mod
12184 ▷ make RSA_e
12185 ▷ make RSA_d
12186 ▷ make im1
12187 ▷ make RSA_e1
12188 ▷ make RSA_d1
12189 ▷ make im1061
12190 ▷ make RSA_e2
12191 ▷ make RSA_d2
12192 ▷ make im3
12193 ▷ make RSA_e3
12194 ▷ make RSA_d3
12195 ▷ make im4
12196 ▷ make RSA_e4
12197 ▷ make RSA_d4
12198 ▷ make im5
12199 ▷ make RSA_e5
12200 ▷ make RSA_d5
12201 ▷ make RSA_d6
12202 ▷ make smooth
12203 ▷ make im7
12204 ▷ make RSA_e7
12205 ▷ make im8
12206 ▷ make RSA_e8
12207 ▷ make sqrt_big
12208 ▷ make sqrt_mod_33_41
12209 ▷ #make quadratic_sieve
12210 ▷ make pseudoprime3
12211 ▷ make pseudoprime10
12212 ▷ make PR10_test1
12213 ▷ make pseudoprime11
12214 ▷ make pseudoprime20
12215 ▷ make PR10
12216 ▷ make pseudoprime50
12217 ▷ make pseudoprime51
12218 ▷ make pseudoprime30
12219 ▷ make pseudoprime31
12220 ▷ make pseudoprime33
12221 ▷ make pseudoprime34
12222 ▷ make pseudoprime35
12223 ▷ make pseudoprime36
12224 ▷ #make MATH360_hw2
12225 ▷ make F_256_Rijndahl
12226 ▷ make all_square_roots_mod_n_1549411
12227 ▷ make power_mod_211
12228
12229
12230
12231 EC.add:
12232 ▷ $(ORBITER) -v 2 \
12233 ▷ ▷ -define F -finite_field -q 11 -end \
12234 ▷ ▷ -with F -do \
12235 ▷ ▷ -finite_field.activity \
12236 ▷ ▷ -EC.add 1 3 "1,4" "1,4" -end

```

```

12237
12238 EC_cyclic_subgroup:
12239 ▷ $(ORBITER) -v 2 \
12240 ▷ ▷ -define F -finite_field -q 11 -end \
12241 ▷ ▷ -with F -do \
12242 ▷ ▷ -finite_field_activity \
12243 ▷ ▷ -EC_cyclic_subgroup 1 3 "1,4" -end
12244
12245
12246 EC_points_13:
12247 ▷ $(ORBITER) -v 2 \
12248 ▷ ▷ -define F -finite_field -q 13 -end \
12249 ▷ ▷ -with F -do \
12250 ▷ ▷ -finite_field_activity \
12251 ▷ ▷ -EC_points "EC_2_5_q13" 2 5 -end
12252 ▷ $(ORBITER) -v 2 -draw_matrix \
12253 ▷ ▷ -input_csv_file EC_2_5_q13_points_xy.csv \
12254 ▷ ▷ -box_width 20 -bit_depth 24 \
12255 ▷ ▷ -partition 2 "1,1,1,1,1,1,1,1,1,1,1,1" "1,1,1,1,1,1,1,1,1,1,1,1" -end
12256
12257
12258
12259
12260 EC_points_199:
12261 ▷ $(ORBITER) -v 2 \
12262 ▷ ▷ -define F -finite_field -q 199 -end \
12263 ▷ ▷ -with F -do \
12264 ▷ ▷ -finite_field_activity \
12265 ▷ ▷ -EC_points "EC_5_7_q199" 5 7 -end
12266 ▷ $(ORBITER) -v 2 \
12267 ▷ ▷ -draw_matrix -input_csv_file EC_5_7_q199_points_xy.csv \
12268 ▷ ▷ -box_width 10 -bit_depth 24 \
12269 ▷ ▷ -partition 2 199 199 -end
12270
12271 EC_Koblitz_encoding:
12272 ▷ $(ORBITER) -v 6 -seed 17 \
12273 ▷ ▷ -define F -finite_field -q 199 -end \
12274 ▷ ▷ -with F -do \
12275 ▷ ▷ -finite_field_activity \
12276 ▷ ▷ -EC_Koblitz_encoding 5 7 67 "147,164" "DEADBEEF" \
12277 ▷ ▷ -end
12278
12279 EC_bsgs:
12280 ▷ $(ORBITER) -v 2 \
12281 ▷ ▷ -define F -finite_field -q 199 -end \
12282 ▷ ▷ -with F -do \
12283 ▷ ▷ -finite_field_activity \
12284 ▷ ▷ -EC_bsgs 5 7 "147,164" 212 \
12285 ▷ ▷ "172,158,45,195,50,22,10,103,55,33,50,22,\
12286 ▷ ▷ 145,105,31,74,73,155,67,60,25,6" \
12287 ▷ ▷ -end
12288
12289 EC_bsgs_decode:
12290 ▷ $(ORBITER) -v 2 \
12291 ▷ ▷ -define F -finite_field -q 199 -end \
12292 ▷ ▷ -with F -do \
12293 ▷ ▷ -finite_field_activity \
12294 ▷ ▷ -EC_bsgs_decode 5 7 "129,176" 212 \
12295 ▷ ▷ "127,188,51,141,85,29,106,90,41,105,179,71,\

```

```

12296 ▷ ▷ 171,2,16,197,183,72,27,129,37,10" \
12297 ▷ ▷ "50,179,169,13,153,169,115,116,188,110,176" \
12298 ▷ ▷ -end
12299
12300
12301
12302 NTRU_Alice1:
12303 ▷ $(ORBITER) -v 2 \
12304 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
12305 ▷ ▷ -with F -do \
12306 ▷ ▷ -finite_field_activity \
12307 ▷ ▷ -extended_gcd_for_polynomials \
12308 ▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
12309 ▷ ▷ -end
12310
12311
12312
12313
12314 NTRU_Alice2:
12315 ▷ $(ORBITER) -v 2 \
12316 ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
12317 ▷ ▷ -with F -do \
12318 ▷ ▷ -finite_field_activity \
12319 ▷ ▷ -extended_gcd_for_polynomials \
12320 ▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
12321 ▷ ▷ -end
12322
12323
12324 NTRU_Alice_public_key:
12325 ▷ $(ORBITER) -v 2 \
12326 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
12327 ▷ ▷ -with F -do \
12328 ▷ ▷ -finite_field_activity \
12329 ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
12330 ▷ ▷ ▷ $(ALICE_PRIVATE_G) $(NTRU_XN1) \
12331 ▷ ▷ -end
12332
12333
12334 NTRU_encrypt:
12335 ▷ $(ORBITER) -v 2 \
12336 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
12337 ▷ ▷ -with F -do \
12338 ▷ ▷ -finite_field_activity \
12339 ▷ ▷ -NTRU_encrypt \
12340 ▷ ▷ $(NTRU_N) $(NTRU_P) $(ALICE_PUBLIC_KEY) \
12341 ▷ ▷ $(BOB_ONE_TIME_KEY) $(BOB_MESSAGE) \
12342 ▷ ▷ -end
12343
12344
12345
12346 NTRU_decrypt1:
12347 ▷ $(ORBITER) -v 2 \
12348 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
12349 ▷ ▷ -with F -do \
12350 ▷ ▷ -finite_field_activity \
12351 ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
12352 ▷ ▷ ▷ $(BOB_ENCRYPT) $(NTRU_XN1) \
12353 ▷ ▷ -end
12354

```

```

12355
12356
12357 NTRU_decrypt2:
12358 ▷ $(ORBITER) -v 2 \
12359 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
12360 ▷ ▷ -with F -do \
12361 ▷ ▷ -finite_field_activity \
12362 ▷ ▷ -polynomial_center_lift $(ALICE_C1) -end
12363
12364 NTRU_decrypt3:
12365 ▷ $(ORBITER) -v 2 \
12366 ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
12367 ▷ ▷ -with F -do \
12368 ▷ ▷ -finite_field_activity \
12369 ▷ ▷ -polynomial_reduce_mod_p $(ALICE_C2) -end
12370
12371 NTRU_decrypt4:
12372 ▷ $(ORBITER) -v 2 \
12373 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
12374 ▷ ▷ -with F -do \
12375 ▷ ▷ -finite_field_activity \
12376 ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_FP) \
12377 ▷ ▷ ▷ $(ALICE_C3) $(NTRU_XN1) \
12378 ▷ ▷ -end
12379
12380
12381
12382 NTRU_decrypt5:
12383 ▷ $(ORBITER) -v 2 \
12384 ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
12385 ▷ ▷ -with F -do \
12386 ▷ ▷ -finite_field_activity \
12387 ▷ ▷ -polynomial_center_lift $(ALICE_C4) -end
12388
12389 #A(X)= - X^{5} + X^{3} + X^{2} - X + 1
12390 #plaintext BOB.MESSAGE
12391
12392
12393
12394 #####
12395
12396
12397 inv_59_mod:
12398 ▷ $(ORBITER) -v 2 -inverse_mod 59 10200
12399
12400 # the inverse of 59 mod 10200 is 2939
12401
12402
12403
12404 RSA_e:
12405 ▷ $(ORBITER) -v 2 \
12406 ▷ ▷ -RSA 59 10403 2 "1921,1605,1804,2116,0518"
12407
12408
12409 RSA_d:
12410 ▷ $(ORBITER) -v 2 \
12411 ▷ ▷ -RSA 2939 10403 2 "902,3509,9833,3548,5181"
12412
12413

```

```

12414 im1:
12415 ▷ $(ORBITER) -v 2 -inverse_mod 869 1843488
12416
12417 #the inverse of 869 mod 1843488 is 386093
12418
12419
12420 # FUNFACTOR:
12421
12422 RSA.e1:
12423 ▷ $(ORBITER) -v 2 \
12424 ▷ ▷ -RSA 386093 1846303 3 "62114,60103,201518"
12425
12426
12427 RSA.d1:
12428 ▷ $(ORBITER) -v 2 \
12429 ▷ ▷ -RSA 869 1846303 3 "1248407,345776,317846"
12430
12431
12432
12433
12434
12435 #####
12436 # 5503*4603 = 25330309
12437 # 5502*4602 = 25320204
12438
12439
12440 im1061:
12441 ▷ $(ORBITER) -v 2 \
12442 ▷ ▷ -inverse_mod 1061 25320204
12443 ▷
12444 # the inverse of 1061 mod 25320204 is 2076209
12445
12446
12447
12448 RSA.e2:
12449 ▷ $(ORBITER) -v 2 \
12450 ▷ ▷ -RSA_encrypt_text 2076209 25330309 3 creamcheese
12451
12452 #-RSA_encrypt_text 386093 1846303 creamcheese
12453 #408918,1735142,239809,654636
12454
12455
12456 RSA.d2:
12457 ▷ $(ORBITER) -v 2 \
12458 ▷ ▷ -RSA 1061 25330309 3 "19019931,1619805,740498,2671344"
12459
12460
12461 #####
12462 # 7253*8171 = 59264263
12463 # 7252*8170 = 59248840
12464
12465
12466 im3:
12467 ▷ $(ORBITER) -v 2 \
12468 ▷ ▷ -inverse_mod 2909 59248840
12469 ▷
12470 #the inverse of 2909 mod 59248840 is 4358629
12471
12472 RSA.e3:

```

```

12473 ▷ $(ORBITER) -v 2 \
12474 ▷ ▷ -RSA_encrypt_text 2909 59264263 3 encrypted
12475
12476 RSA_d3:
12477 ▷ $(ORBITER) -v 2 \
12478 ▷ ▷ -RSA 4358629 59264263 3 "35270141,9642524,49091707"
12479
12480 #51403,182516,200504 = encrypted
12481
12482
12483 ####
12484 # 7879 * 7901 = 62251979
12485 # 7878 * 7900 = 62236200
12486
12487 # e =
12488
12489 im4:
12490 ▷ $(ORBITER) -v 2 -inverse_mod 583 62236200
12491
12492 # the inverse of 583 mod 62236200 is 32559247
12493
12494 RSA_e4:
12495 ▷ $(ORBITER) -v 2 \
12496 ▷ ▷ -RSA_encrypt_text 583 62251979 3 venividivici
12497
12498 #-RSA_encrypt_text 583 62251979 venividivici
12499 #40513610,53979973,56449676,35068535
12500
12501 RSA_d4:
12502 ▷ $(ORBITER) -v 2 \
12503 ▷ ▷ -RSA 32559247 62251979 "40513610,53979973,56449676,35068535"
12504
12505
12506
12507 #####
12508 # 7369 * 7127 = 52518863
12509 # 7368 * 7126 = 52504368
12510
12511
12512 im5:
12513 ▷ $(ORBITER) -v 2 -inverse_mod 173 52504368
12514
12515 #the inverse of 173 mod 52504368 is 38543669
12516 ▷
12517 RSA_e5:
12518 ▷ $(ORBITER) -v 2 \
12519 ▷ ▷ -RSA_encrypt_text 38543669 52518863 3 fascinating
12520
12521 #-RSA_encrypt_text 38543669 52518863 fascinating
12522 #31526751,8962078,51045732,51894467
12523 ▷ ▷
12524
12525 RSA_d5:
12526 ▷ $(ORBITER) -v 2 \
12527 ▷ ▷ -RSA 173 52518863 "31526751,8962078,51045732,51894467"
12528
12529
12530 RSA_d6:
12531 ▷ $(ORBITER) -v 2 \

```

```

12532 ▷ ▷ -RSA 47177497 55040413 "28702119,48926559"
12533
12534
12535 smooth:
12536 ▷ $(ORBITER) -v 2 \
12537 ▷ ▷ -sift_smooth 100000 100 "2,3,5,7,11,13,17,19"
12538
12539
12540 ▷ ▷
12541 #####
12542 # 1999 * 7907 = 15806093
12543 # 1998 * 7906 = 15796188
12544
12545 im7:
12546 ▷ $(ORBITER) -v 2 -inverse_mod 3221 15796188
12547
12548 #the inverse of 3221 mod 15796188 is 10048553
12549
12550
12551 RSA_e7:
12552 ▷ $(ORBITER) -v 2 \
12553 ▷ ▷ -RSA_encrypt_text 10048553 15806093 3 beachandfun
12554 ▷ ▷
12555 #
12556 # 7853 * 7673 = 60256069
12557 # 7852 * 7672 = 60240544
12558
12559 im8:
12560 ▷ $(ORBITER) -v 2 -inverse_mod 9017 60240544
12561
12562
12563 #the inverse of 9017 mod 60240544 is 14430473
12564
12565 RSA_e8:
12566 ▷ $(ORBITER) -v 2 \
12567 ▷ ▷ -RSA_encrypt_text 9017 60256069 3 strawberry
12568
12569
12570 sqrt_big:
12571 ▷ $(ORBITER) -v 2 -square_root 1002001
12572 ▷
12573 sqrt_mod_33_41:
12574 ▷ $(ORBITER) -v 2 -square_root_mod 33 41
12575 ▷
12576 quadratic_sieve:
12577 ▷ $(ORBITER) -v 5 -quadratic_sieve 31 500 1
12578
12579
12580
12581 #####
12582
12583 pseudoprime3:
12584 ▷ $(ORBITER) -v 5 \
12585 ▷ ▷ -seed 2531011 -find_pseudoprime 3 5 0 0
12586 ▷ pdflatex pseudoprime_3.tex
12587 ▷ $(OPEN) pseudoprime_3.pdf
12588
12589 pseudoprime10:
12590 ▷ $(ORBITER) -v 5 \

```



```

12591 ▷ ▷ -seed 2531011 -find_pseudoprime 10 5 5 5
12592 ▷ pdflatex pseudoprime_10.tex
12593 ▷ $(OPEN) pseudoprime_10.pdf
12594
12595 # 4460190157
12596
12597
12598 PR10_test1:
12599 ▷ $(ORBITER) -v 5 -power_mod 1293 2230095078 4460190157
12600 ▷ $(ORBITER) -v 5 -power_mod 9865 2230095078 4460190157
12601 ▷ $(ORBITER) -v 5 -power_mod 19645 2230095078 4460190157
12602 ▷ $(ORBITER) -v 5 -power_mod 974586571 2230095078 4460190157
12603 ▷ $(ORBITER) -v 5 -power_mod 974586571 1486730052 4460190157
12604 ▷ $(ORBITER) -v 5 -power_mod 974586571 15222492 4460190157
12605 ▷ $(ORBITER) -v 5 -power_mod 974586571 284796 4460190157
12606
12607
12608
12609 pseudoprime11:
12610 ▷ $(ORBITER) -v 5 \
12611 ▷ ▷ -seed 2531011 -find_pseudoprime 11 5 5 5
12612 ▷ pdflatex pseudoprime_11.tex
12613 ▷ $(OPEN) pseudoprime_11.pdf
12614
12615 # 63814633367
12616
12617 # product is 284625399616057168619
12618
12619
12620 pseudoprime20:
12621 ▷ $(ORBITER) -v 5 \
12622 ▷ ▷ -seed 2531011 -find_pseudoprime 20 5 5 5
12623 ▷ pdflatex pseudoprime_20.tex
12624 ▷ $(OPEN) pseudoprime_20.pdf
12625
12626
12627 PR10:
12628 ▷ $(ORBITER) -v 5 -primitive_root 4460190157
12629
12630
12631 # mistake! long integer overflow
12632 # a primitive root modulo 165222861 is 1293
12633
12634
12635
12636
12637 pseudoprime50:
12638 ▷ $(ORBITER) -v 5 \
12639 ▷ ▷ -seed 2531011 -find_pseudoprime 50 5 0 0
12640 ▷ pdflatex pseudoprime_50.tex
12641 ▷ $(OPEN) pseudoprime_50.pdf
12642
12643
12644 #91322792878581218181431392170986926262336688354473
12645
12646 pseudoprime51:
12647 ▷ $(ORBITER) -v 5 \
12648 ▷ ▷ -seed 2531011 -find_pseudoprime 51 5 5 5
12649 ▷ pdflatex pseudoprime_51.tex

```

```

12650 ▷ $(OPEN) pseudoprime_51.pdf
12651
12652 #754600727746834470214089702490004944659715367045417
12653
12654 # product 68912245966050819606199994423264315732335295324400658436661744403244049
    572914094379904326661586100241
12655
12656 pseudoprime30:
12657 ▷ $(ORBITER) -v 5 \
12658 ▷ ▷ -seed 2531011 -find_pseudoprime 30 5 5 5
12659 ▷ pdflatex pseudoprime_30.tex
12660 ▷ $(OPEN) pseudoprime_30.pdf
12661
12662 # 286525565474504516914595596387
12663
12664 pseudoprime31:
12665 ▷ $(ORBITER) -v 5 \
12666 ▷ ▷ -seed 2531011 -find_pseudoprime 31 5 5 5
12667 ▷ pdflatex pseudoprime_31.tex
12668 ▷ $(OPEN) pseudoprime_31.pdf
12669 #8777266765422645523724129853331
12670
12671 # 2514911323283298698837184692002835573476743643265896783515097
12672
12673 # maybe 2 seconds
12674
12675
12676 pseudoprime33:
12677 ▷ $(ORBITER) -v 5 \
12678 ▷ ▷ -seed 2531011 -find_pseudoprime 33 5 5 5
12679 ▷ pdflatex pseudoprime_33.tex
12680 ▷ $(OPEN) pseudoprime_33.pdf
12681
12682
12683 #371674199498295345543363004459891
12684
12685
12686 pseudoprime34:
12687 ▷ $(ORBITER) -v 5 \
12688 ▷ ▷ -seed 2531011 -find_pseudoprime 34 5 5 5
12689 ▷ pdflatex pseudoprime_34.tex
12690 ▷ $(OPEN) pseudoprime_34.pdf
12691
12692 #9309708224110488378214945245346817
12693
12694 # 3460178351758962531912872979731874528849142238619677890786061016947
12695 # 18 sec
12696
12697
12698
12699 pseudoprime35:
12700 ▷ $(ORBITER) -v 5 \
12701 ▷ ▷ -seed 2531011 -find_pseudoprime 35 5 5 5
12702 ▷ pdflatex pseudoprime_35.tex
12703 ▷ $(OPEN) pseudoprime_35.pdf
12704
12705 #81329557792505271120435930267680203
12706
12707 pseudoprime36:

```

```

12708 ▷ $(ORBITER) -v 5 \
12709 ▷ ▷ -seed 2531011 -find_pseudoprime 36 5 5 5
12710 ▷ pdflatex pseudoprime_36.tex
12711 ▷ $(OPEN) pseudoprime_36.pdf
12712
12713 #16262468089199340433363207561599139
12714
12715 # 13226193383093105242537919350220354135219441323641636665484262532145217
12716 # factoring takes 46 seconds
12717
12718
12719 #ToDo:
12720
12721 MATH360.hw2:
12722 ▷ $(ORBITER) -v 3 \
12723 ▷ ▷ -define F -finite_field -q 16 -end \
12724 ▷ ▷ -with F -do -finite_field_activity \
12725 ▷ ▷ -parse_and_evaluate "test" "" "a+b" "a=8,b=14" -end
12726 ▷ $(ORBITER) -v 3 \
12727 ▷ ▷ -define F -finite_field -q 16 -end \
12728 ▷ ▷ -with F -do -finite_field_activity \
12729 ▷ ▷ -parse_and_evaluate "test" "" "a*b" "a=9,b=13" -end
12730 ▷ $(ORBITER) -v 3 \
12731 ▷ ▷ -define F -finite_field -q 16 -end \
12732 ▷ ▷ -with F -do -finite_field_activity \
12733 ▷ ▷ -parse_and_evaluate "test" "" "a*a*a*a*a" "a=9" -end
12734 ▷ $(ORBITER) -v 3 \
12735 ▷ ▷ -define F -finite_field -q 16 -end \
12736 ▷ ▷ -with F -do -finite_field_activity \
12737 ▷ ▷ -parse_and_evaluate "test" "" "(a+b)*(a+b)" "a=5,b=7" -end
12738 ▷ $(ORBITER) -v 3 \
12739 ▷ ▷ -define F -finite_field -q 16 -end \
12740 ▷ ▷ -with F -do -finite_field_activity \
12741 ▷ ▷ -parse_and_evaluate "test" "" "a*a+b*b" "a=5,b=7" -end
12742
12743
12744 F_256_Rijndahl:
12745 ▷ $(ORBITER) -v 3 \
12746 ▷ ▷ -define F -finite_field -q 256 -override_polynomial 283 -end \
12747 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
12748
12749
12750
12751
12752
12753
12754
12755 all_square_roots_mod_n_1549411:
12756 ▷ $(ORBITER) -v 3 -all_square_roots_mod_n 1075922 1549411
12757 ▷
12758
12759
12760 power_mod_211:
12761 ▷ $(ORBITER) -v 3 -power_function_mod_n 2 211
12762 ▷ $(ORBITER) -v 3 \
12763 ▷ ▷ -plot_function power_function_k2_n211.csv
12764 ▷ $(ORBITER) -v 2 -draw_matrix \
12765 ▷ ▷ -input_csv_file power_function_k2_n211_graph.csv \
12766 ▷ ▷ -box_width 10 -bit_depth 8 \

```

```

12767 ▷ ▷ -partition 3 211 211 -end
12768 ▷ $(OPEN) power_function_k2_n211_graph.draw.bmp
12769
12770
12771
12772 #####
12773 # Chapter 10 - Coding Theory
12774 #####
12775
12776
12777 test_10:
12778 ▷ make test_10.1
12779 ▷ make test_10.2
12780 ▷ make test_10.3
12781 ▷ make test_10.4
12782 ▷ make test_10.5
12783 ▷ make test_10.6
12784 ▷ make test_10.7
12785 ▷ make test_10.8
12786 ▷ make test_10.9
12787 ▷ make test_10.10
12788
12789
12790
12791
12792 #####
12793 # Section 10.1: Coding Theory
12794
12795 SECTION.CODING.THEORY.INTRODUCTION:
12796
12797 test_10.1:
12798 ▷ make Allen_Gates_noise_1_percent
12799 ▷ make Hamming_space_4.2_distance_matrix
12800 ▷ make Hamming_space_4.2_distance_matrix_draw
12801 ▷ make Hamming_code_macwilliams
12802 ▷ make code_5.2.3_diagram
12803 ▷ make Hamming_5.2_graph
12804 ▷ make Hamming_5.2_with_5.2.3_code
12805 ▷ make Sylvester_Hadamard_code_8
12806
12807
12808 Allen_Gates_noise_1_percent:
12809 ▷ cp $(MY_PATH)/examples/users_guide/allen_Gates.bmp .
12810 ▷ $(ORBITER) -v 3 \
12811 ▷ ▷ -random_noise_in_bitmap_file \
12812 ▷ ▷ allen_Gates.bmp \
12813 ▷ ▷ allen_Gates_1.bmp \
12814 ▷ ▷ 1 100
12815 ▷ $(OPEN) allen_Gates_1.bmp
12816
12817
12818 Hamming_space_4.2_distance_matrix:
12819 ▷ $(ORBITER) -v 2 \
12820 ▷ ▷ -define F -finite_field -q 2 -end \
12821 ▷ ▷ -with F -do -coding_theoretic_activity \
12822 ▷ ▷ ▷ -Hamming_space_distance_matrix 4 \
12823 ▷ ▷ -end
12824
12825

```

```

12826 Hamming_space_4_2.distance_matrix.draw:
12827 ▷ $(ORBITER) -v 2 \
12828 ▷ ▷ -draw_matrix \
12829 ▷ ▷ ▷ -input_csv_file Hamming_n4_q2.csv \
12830 ▷ ▷ ▷ -box_width 20 -bit_depth 24 \
12831 ▷ ▷ ▷ -partition 4 16 16 \
12832 ▷ ▷ -end
12833 ▷ $(OPEN) Hamming_n4_q2.draw.bmp
12834
12835
12836
12837 Hamming_code_macwilliams:
12838 ▷ $(ORBITER) -v 2 \
12839 ▷ ▷ -make_macwilliams_system 7 4 2
12840 ▷ pdflatex MacWilliams_n7_k4_q2.tex
12841 ▷ $(OPEN) MacWilliams_n7_k4_q2.pdf
12842
12843
12844
12845
12846
12847 code_5_2_3.diagram:
12848 ▷ $(ORBITER) -v 2 \
12849 ▷ ▷ -define F -finite_field -q 2 -end \
12850 ▷ ▷ -with F -do -coding_theoretic_activity \
12851 ▷ ▷ ▷ -general_code_binary 5 "code_5_2_3" \
12852 ▷ ▷ ▷ $(CODE_5_2_3.CODEWORDS) \
12853 ▷ ▷ ▷ -metric_balls 1 \
12854 ▷ ▷ -end
12855 ▷ $(ORBITER) -v 2 \
12856 ▷ ▷ -draw_matrix \
12857 ▷ ▷ ▷ -input_csv_file code_5_2_3.char_func_5.4.csv \
12858 ▷ ▷ ▷ -box_width 25 -bit_depth 24 \
12859 ▷ ▷ ▷ -partition 4 8 4 \
12860 ▷ ▷ -end
12861
12862
12863
12864 Hamming_5_2.graph:
12865 ▷ $(ORBITER) -v 2 \
12866 ▷ ▷ -define G -graph -Hamming 5 2 -end \
12867 ▷ ▷ -with G -do \
12868 ▷ ▷ -graph_theoretic_activity -export_csv -end \
12869 ▷ ▷ -with G -do \
12870 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
12871 ▷ ▷ -with G -do \
12872 ▷ ▷ -graph_theoretic_activity -save -end
12873 ▷ $(ORBITER) -v 2 -draw_matrix \
12874 ▷ ▷ -input_csv_file Hamming_5_2.csv \
12875 ▷ ▷ -box_width 8 -bit_depth 24 -partition 4 32 32 -end
12876 ▷ dot -Tpng Hamming_5_2.gv >Hamming_5_2.png
12877
12878
12879 Hamming_5_2.with_5_2_3.code:
12880 ▷ $(ORBITER) -v 2 \
12881 ▷ ▷ -define G -graph -Hamming 5 2 \
12882 ▷ ▷ ▷ -subset "_code_5_2_3" "\\_code\\_5\\_2\\_3" \
12883 ▷ ▷ ▷ $(CODE_5_2_3.CODEWORDS) -end \
12884 ▷ ▷ -with G -do \

```

```

12885 ▷ ▷ -graph_theoretic_activity -export_csv -end \
12886 ▷ ▷ -with G -do \
12887 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
12888 ▷ ▷ -with G -do \
12889 ▷ ▷ -graph_theoretic_activity -save -end \
12890 ▷ ▷ -with G -do \
12891 ▷ ▷ -graph_theoretic_activity -automorphism_group -end
12892 ▷ pdflatex Hamming_5.2.code.5.2.3.report.tex
12893 ▷ $(OPEN) Hamming_5.2.code.5.2.3.report.pdf
12894
12895 # group has order 32
12896
12897
12898 # n must be a power of 2, and at least 4:
12899 # minus one is represented as 2.
12900
12901 Sylvester_Hadamard_code_8:
12902 ▷ $(ORBITER) -v 2 \
12903 ▷ ▷ -define F -finite_field -q 3 -end \
12904 ▷ ▷ -with F -do -coding_theoretic_activity \
12905 ▷ ▷ ▷ -Sylvester_Hadamard_code 8 \
12906 ▷ ▷ -end
12907
12908
12909
12910
12911 #####
12912 # Section 10.2: Linear codes
12913
12914 SECTION_CODING_THEORY_LINEAR_CODES:
12915
12916
12917
12918
12919 test_10.2:
12920 ▷ make RM_3_1
12921 ▷ make code_6
12922 ▷ make Hamming_generator
12923 ▷ make simplex_code
12924 ▷ make Hamming_generator
12925 ▷ make Hamming_code
12926 ▷ make RM_3_1_and_codewords
12927 ▷ make RM_3_1_from_generator_matrix
12928 ▷ make RM_4_1_and_codewords
12929 ▷ make RM_5_1_and_codewords
12930 ▷ make Hamming_code_by_rows
12931 ▷ make Hamming_code_long
12932 ▷ make Hamming_code_diagram
12933 ▷ make Hamming_code_words
12934 ▷ make Hamming_weight_enumerator
12935 ▷ make Hamming_minimum_distance
12936 ▷ make Golay23_minimum_distance
12937 ▷ make code_Hamming_systematic
12938 ▷ make Hamming_RREF
12939 ▷ make Hamming_nullspace
12940
12941
12942
12943 RM_3_1:

```

```

12944 ▷ $(ORBITER) -v 2 \
12945 ▷ ▷ -define F -finite_field -q 2 -end \
12946 ▷ ▷ -define C -code -field F \
12947 ▷ ▷ ▷ -Reed_Muller 3 \
12948 ▷ ▷ -end \
12949 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
12950 ▷ ▷ ▷ -export_magma RM_3_1.magma \
12951 ▷ ▷ -end \
12952 ▷ ▷ -with C -do -coding_theoretic_activity \
12953 ▷ ▷ ▷ -report \
12954 ▷ ▷ -end
12955 ▷ pdflatex RM_3_code_n8_k4.q2.tex
12956 ▷ $(OPEN) RM_3_code_n8_k4.q2.pdf
12957
12958
12959 code_6:
12960 ▷ $(ORBITER) -v 2 \
12961 ▷ ▷ -define F -finite_field -q 2 -end \
12962 ▷ ▷ -define C -code -field F \
12963 ▷ ▷ ▷ -basis 6 "60,50,41" \
12964 ▷ ▷ -end \
12965 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
12966 ▷ ▷ ▷ -make_diagram \
12967 ▷ ▷ -end \
12968 ▷ ▷ -with C -do -coding_theoretic_activity \
12969 ▷ ▷ ▷ -report \
12970 ▷ ▷ -end
12971 ▷ $(ORBITER) -v 2 -draw_matrix \
12972 ▷ ▷ -input_csv_file by_basis_n6_k3_char_func_6.8.csv \
12973 ▷ ▷ -box_width 20 -bit_depth 24 \
12974 ▷ ▷ -partition 2 "1,1,1,1,1,1,1,1" "1,1,1,1,1,1,1,1" \
12975 ▷ ▷ -end
12976 ▷ pdflatex by_basis_n6_k3_code_n6_k3.q2.tex
12977 ▷ $(OPEN) by_basis_n6_k3_code_n6_k3.q2.pdf
12978 ▷ $(OPEN) by_basis_n6_k3_char_func_6.8_draw.bmp
12979
12980 # linear code with generator matrix
12981 # 111100 = 60
12982 # 110010 = 50
12983 # 101001 = 41
12984
12985
12986
12987
12988 simplex_code:
12989 ▷ $(ORBITER) -v 2 \
12990 ▷ ▷ -define F -finite_field -q 2 -end \
12991 ▷ ▷ -define v -vector -field F -format 3 \
12992 ▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
12993 ▷ ▷ -end \
12994 ▷ ▷ -define C -code -field F \
12995 ▷ ▷ ▷ -generator_matrix v \
12996 ▷ ▷ -end \
12997 ▷ ▷ -with C -do -coding_theoretic_activity \
12998 ▷ ▷ ▷ -report \
12999 ▷ ▷ -end
13000 ▷ pdflatex by_genma_n7_k3_code_n7_k3.q2.tex
13001 ▷ $(OPEN) by_genma_n7_k3_code_n7_k3.q2.pdf
13002

```

```

13003
13004
13005 Hamming_generator:
13006 ▷ $(ORBITER) -v 2 \
13007 ▷ ▷ -define F -finite_field -q 2 -end \
13008 ▷ ▷ -define v -vector -field F -format 3 \
13009 ▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
13010 ▷ ▷ -end \
13011 ▷ ▷ -with F -do \
13012 ▷ ▷ -finite_field_activity \
13013 ▷ ▷ ▷ -nullspace v \
13014 ▷ ▷ -end
13015 ▷ pdflatex nullspace_3.7.tex
13016 ▷ $(OPEN) nullspace_3.7.pdf
13017
13018 # basis in binary:
13019 # 67,37,22,15
13020
13021
13022
13023
13024 Hamming_code:
13025 ▷ $(ORBITER) -v 2 \
13026 ▷ ▷ -define F -finite_field -q 2 -end \
13027 ▷ ▷ -define v -vector -field F -format 3 \
13028 ▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
13029 ▷ ▷ -end \
13030 ▷ ▷ -define C -code -field F \
13031 ▷ ▷ ▷ -generator_matrix v \
13032 ▷ ▷ ▷ -dual -end \
13033 ▷ ▷ -end \
13034 ▷ ▷ -with C -do -coding_theoretic_activity \
13035 ▷ ▷ ▷ -export_magma Hamming.magma \
13036 ▷ ▷ -end
13037
13038 # writes Hamming.magma
13039
13040
13041
13042 RM_3_1_and_codewords:
13043 ▷ $(ORBITER) -v 2 \
13044 ▷ ▷ -define F -finite_field -q 2 -end \
13045 ▷ ▷ -define C -code -field F \
13046 ▷ ▷ ▷ -Reed_Muller 3 \
13047 ▷ ▷ -end \
13048 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13049 ▷ ▷ ▷ -export_magma RM_3_1.magma \
13050 ▷ ▷ -end \
13051 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13052 ▷ ▷ ▷ -export_codewords RM_3_1.codewords.csv \
13053 ▷ ▷ -end \
13054 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13055 ▷ ▷ ▷ -export_genma RM_3_1.genma.csv \
13056 ▷ ▷ -end
13057
13058
13059 RM_3_1_from_generator_matrix:
13060 ▷ $(ORBITER) -v 2 \
13061 ▷ ▷ -define F -finite_field -q 2 -end \

```



```

13062 ▷ ▷ -define genma -vector -format 8 -field F \
13063 ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
13064 ▷ ▷ -end \
13065 ▷ ▷ -define C -code -field F \
13066 ▷ ▷ ▷ -generator_matrix genma \
13067 ▷ ▷ -end
13068 ▷ #pdflatex code_n8_k4_q2.tex
13069 ▷ #$(OPEN) code_n8_k4_q2.pdf
13070
13071 #Codewords: (0,255,170,85,204,51,102,153,240,15,90,165,60,195,150,105)
13072
13073
13074 RM_4_1_and_codewords:
13075 ▷ $(ORBITER) -v 2 \
13076 ▷ ▷ -define F -finite_field -q 2 -end \
13077 ▷ ▷ -define C -code -field F -Reed_Muller 4 -end \
13078 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13079 ▷ ▷ ▷ -export_magma RM_4_1.magma \
13080 ▷ ▷ -end \
13081 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13082 ▷ ▷ ▷ -export_codewords RM_4_1.codewords.csv \
13083 ▷ ▷ -end \
13084 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13085 ▷ ▷ ▷ -export_genma RM_4_1.genma.csv \
13086 ▷ ▷ -end
13087
13088 RM_5_1_and_codewords:
13089 ▷ $(ORBITER) -v 2 \
13090 ▷ ▷ -define F -finite_field -q 2 -end \
13091 ▷ ▷ -define C -code -field F -Reed_Muller 5 -end \
13092 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13093 ▷ ▷ ▷ -export_magma RM_5_1.magma \
13094 ▷ ▷ -end \
13095 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13096 ▷ ▷ ▷ -export_codewords RM_5_1.codewords.csv \
13097 ▷ ▷ -end \
13098 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13099 ▷ ▷ ▷ -export_genma RM_5_1.genma.csv \
13100 ▷ ▷ -end
13101
13102
13103
13104
13105 Hamming_code_by_rows:
13106 ▷ $(ORBITER) -v 2 \
13107 ▷ ▷ -define F -finite_field -q 2 -end \
13108 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
13109 ▷ ▷ -define C -code -field F \
13110 ▷ ▷ ▷ -basis 7 v \
13111 ▷ ▷ -end
13112 ▷ #pdflatex code_n7_k4_q2.tex
13113 ▷ #$(OPEN) code_n7_k4_q2.pdf
13114
13115
13116
13117 Hamming_code_long:
13118 ▷ $(ORBITER) -v 2 \
13119 ▷ ▷ -define F -finite_field -q 2 -end \
13120 ▷ ▷ -define C -code -field F \

```

```

13121 ▷ ▷ ▷ -long_code 7 4 \
13122 ▷ ▷ ▷ "0,5,6" \
13123 ▷ ▷ ▷ "1,4,6" \
13124 ▷ ▷ ▷ "2,4,5" \
13125 ▷ ▷ ▷ "3,4,5,6" \
13126 ▷ ▷ -end
13127
13128
13129 Hamming_code_diagram:
13130 ▷ $(ORBITER) -v 2 \
13131 ▷ ▷ -define F -finite_field -q 2 -end \
13132 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
13133 ▷ ▷ -define C -code -field F \
13134 ▷ ▷ ▷ -basis 7 v \
13135 ▷ ▷ -end \
13136 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13137 ▷ ▷ ▷ -make_diagram \
13138 ▷ ▷ -end
13139 ▷ $(ORBITER) -v 2 \
13140 ▷ ▷ -draw_matrix \
13141 ▷ ▷ ▷ -input_csv_file by_basis_n7_k4_char_func_7_16.csv \
13142 ▷ ▷ ▷ -box_width 25 -bit_depth 24 \
13143 ▷ ▷ ▷ -partition 4 16 8 \
13144 ▷ ▷ -end
13145 ▷ $(OPEN) by_basis_n7_k4_char_func_7_16_draw.bmp
13146
13147
13148 Hamming_code_words:
13149 ▷ $(ORBITER) -v 2 \
13150 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_CODEWORDS) -end \
13151 ▷ ▷ -define F -finite_field -q 2 -end \
13152 ▷ ▷ -loop_over i v \
13153 ▷ ▷ ▷ -with F -do -coding_theoretic_activity \
13154 ▷ ▷ ▷ -general_code_binary 7 "Hamming_7_4_word_%i" "%i[v]" \
13155 ▷ ▷ ▷ -metric_balls 1 \
13156 ▷ ▷ ▷ -end \
13157 ▷ ▷ -end_loop_over i
13158 ▷ $(ORBITER) -v 2 \
13159 ▷ ▷ -loop i 0 16 1 -draw_matrix \
13160 ▷ ▷ ▷ -input_csv_file Hamming_7_4_word_%i_char_func_7_1.csv \
13161 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
13162 ▷ ▷ ▷ -partition 4 16 8 -end \
13163 ▷ ▷ ▷ -system "convert Hamming_7_4_word_%i_char_func_7_1_draw.bmp \
13164 ▷ ▷ ▷ -frame 8 Hamming_7_4_word_%i_char_func_7_1_draw.png" \
13165 ▷ ▷ -end_loop i
13166
13167
13168
13169 Hamming_weight_enumerator:
13170 ▷ $(ORBITER) -v 2 \
13171 ▷ ▷ -define F -finite_field -q 2 -end \
13172 ▷ ▷ -define v -vector -field F -format 4 \
13173 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
13174 ▷ ▷ -end \
13175 ▷ ▷ -define C -code -field F \
13176 ▷ ▷ ▷ -generator_matrix v \
13177 ▷ ▷ -end \
13178 ▷ ▷ -with C -do \
13179 ▷ ▷ -coding_theoretic_activity \

```

```

13180 ▷ ▷ ▷ -weight_enumerator \
13181 ▷ ▷ -end
13182
13183 Hamming_minimum_distance:
13184 ▷ $(ORBITER) -v 2 \
13185 ▷ ▷ -define F -finite_field -q 2 -end \
13186 ▷ ▷ -define v -vector -field F -format 4 \
13187 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
13188 ▷ ▷ -end \
13189 ▷ ▷ -with F -do \
13190 ▷ ▷ -coding_theoretic_activity \
13191 ▷ ▷ ▷ -minimum_distance v \
13192 ▷ ▷ -end
13193
13194
13195 Golay23_minimum_distance:
13196 ▷ $(ORBITER) -v 2 \
13197 ▷ ▷ -define F -finite_field -q 2 -end \
13198 ▷ ▷ -define v -vector -field F -format 12 \
13199 ▷ ▷ ▷ -dense $(GOLAY23_CODE_GENERATOR) \
13200 ▷ ▷ -end \
13201 ▷ ▷ -with F -do \
13202 ▷ ▷ -coding_theoretic_activity \
13203 ▷ ▷ ▷ -minimum_distance v \
13204 ▷ ▷ -end
13205
13206 #d=7 in 0 sec
13207
13208
13209
13210
13211 code_Hamming_systematic:
13212 ▷ $(ORBITER) -v 2 \
13213 ▷ ▷ -define F -finite_field -q 2 -end \
13214 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
13215 ▷ ▷ -define C -code -field F \
13216 ▷ ▷ ▷ -basis 7 v \
13217 ▷ ▷ -end
13218 ▷ #$(ORBITER) -v 2 -draw_matrix \
13219 ▷ #▷ -input_csv_file code_matrix_16_8.csv \
13220 ▷ #▷ -box_width 25 -bit_depth 8 -partition 2 16 8 -end
13221 ▷ #$(OPEN) code_matrix_16_8.draw.bmp
13222 ▷ #pdflatex code_7_16.tex
13223 ▷ #$(OPEN) code_7_16.pdf
13224
13225
13226 #▷ $(DRAW_PATH)/draw_matrix.out -read code_matrix_256_256.csv -box_width 10
13227
13228 # 1 0 0 0 0 1 1 = 67
13229 # 0 1 0 0 1 0 1 = 37
13230 # 0 0 1 0 1 1 0 = 22
13231 # 0 0 0 1 1 1 1 = 15
13232
13233
13234 # ToDo
13235
13236 Hamming_RREF:
13237 ▷ $(ORBITER) -v 2 \
13238 ▷ ▷ -define F -finite_field -q 2 -end \

```

```

13239 ▷ ▷ -define v -vector -format 4 -field F \
13240 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
13241 ▷ ▷ -end \
13242 ▷ ▷ -with F -do \
13243 ▷ ▷ -finite_field_activity \
13244 ▷ ▷ ▷ -RREF v \
13245 ▷ ▷ -end
13246 ▷ pdflatex rref_m4_n7_q2.tex
13247 ▷ $(OPEN) rref_m4_n7_q2.pdf
13248 ▷ #pdflatex RREF_example_q2_4_7.tex
13249 ▷ #gs -sDEVICE=png16 -dFIXEDMEDIA \
13250 ▷ #▷ -dDEVICEWIDTHPOINTS=500 -dDEVICEHEIGHTPOINTS=450 \
13251 ▷ #▷ -r240 -oHamming_dual_page%02d.png \
13252 ▷ #▷ RREF_example_q2_4_7.pdf
13253
13254
13255
13256 Hamming_nullspace:
13257 ▷ $(ORBITER) -v 2 \
13258 ▷ ▷ -define F2 -finite_field -q 2 -end \
13259 ▷ ▷ -define v -vector -format 4 -field F2 \
13260 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
13261 ▷ ▷ -end \
13262 ▷ ▷ -with F2 -do \
13263 ▷ ▷ -finite_field_activity \
13264 ▷ ▷ -nullspace v \
13265 ▷ ▷ -end
13266 ▷ pdflatex nullspace_4_7.tex
13267 ▷ $(OPEN) nullspace_4_7.pdf
13268
13269
13270 #check equations of the Hamming code:
13271 # a4+a5+a6+a7 =1+0+1+0=0 mod2 OK.
13272 # a2+a3+a6+a7 =0+1+1+0=0 mod2 OK.
13273 # a1+a3+a5+a7 =1+1+0+0=0 mod2 OK.
13274
13275 #1010101
13276 #0110011
13277 #0001111
13278
13279
13280
13281
13282
13283 #####
13284 # Section 10.3: Coding Theory - Golay codes
13285
13286 SECTION_CODING_THEORY_GOLAY_CODES:
13287
13288
13289 test_10_3:
13290 ▷ make Golay23_code_words
13291 ▷ make Golay23_code_diagram
13292 ▷ #make Golay23_code_diagram_draw
13293 ▷ make Golay24_make_code
13294 ▷ make Golay24_make_aut_gens
13295 ▷ make Golay24_make_aut_gens_classes
13296 ▷ make M24_elt_2A
13297 ▷ make M24_elt_2B

```

```

13298 ▷ make M24.2A.restricted
13299 ▷ make M24.2B.restricted
13300 ▷ make Golay24.2A.fixed_subcode
13301 ▷ make Golay24.2B.fixed_subcode
13302 ▷ make Golay24.2A.fix_code
13303 ▷ make Golay24.2B.fix_code
13304 ▷ make Golay24.2A.fix_weight12
13305 ▷ make Golay24.2A.fix_weight12_c
13306 ▷ make Golay24.2A.fix_weight12_graph_24
13307
13308
13309
13310
13311 Golay23.code_words:
13312 ▷ $(ORBITER) -v 2 \
13313 ▷ ▷ -define v -vector -dense $(GOLAY_23.COLUMN_RANKS_PROJECTIVELY) -end \
13314 ▷ ▷ -define F -finite_field -q 2 -end \
13315 ▷ ▷ -define C -code -field F \
13316 ▷ ▷ ▷ -projective_set 12 v -end \
13317 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13318 ▷ ▷ ▷ -export_magma Golay23.magma \
13319 ▷ ▷ -end \
13320 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13321 ▷ ▷ ▷ -export_codewords Golay23.codewords.csv \
13322 ▷ ▷ -end \
13323 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13324 ▷ ▷ ▷ -export_genma Golay23.genma.csv \
13325 ▷ ▷ -end
13326 ▷ #pdflatex code_n23_k12.q2.tex
13327 ▷ #$(OPEN) code_n23_k12.q2.pdf
13328
13329
13330 Golay23.code_diagram:
13331 ▷ $(ORBITER) -v 2 \
13332 ▷ ▷ -define F -finite_field -q 2 -end \
13333 ▷ ▷ -define words -vector -file Golay23.codewords.csv -end \
13334 ▷ ▷ -with F -do \
13335 ▷ ▷ -coding_theoretic_activity \
13336 ▷ ▷ ▷ -general_code_binary 23 "Golay_23" \
13337 ▷ ▷ ▷ words \
13338 ▷ ▷ ▷ -embellish 4 \
13339 ▷ ▷ -end
13340
13341
13342 Golay23.code_diagram.draw:
13343 ▷ $(ORBITER) -v 2 \
13344 ▷ ▷ -draw_matrix \
13345 ▷ ▷ ▷ -input_csv_file Golay_23.diagram_01_23_4096.csv \
13346 ▷ ▷ ▷ -box_width 4 -bit_depth 8 \
13347 ▷ ▷ ▷ -partition 20 4096 2048 \
13348 ▷ ▷ -end
13349
13350
13351 Golay24.make_code:
13352 ▷ $(ORBITER) -v 2 \
13353 ▷ ▷ -define F -finite_field -q 2 -end \
13354 ▷ ▷ -define v -vector \
13355 ▷ ▷ ▷ -dense $(GOLAY_24.CODE_PROJECTIVE) \
13356 ▷ ▷ -end \

```

```

13357 ▷ ▷ -define C -code -field F \
13358 ▷ ▷ ▷ -projective_set 12 v \
13359 ▷ ▷ -end
13360
13361
13362 Golay24_make_aut_gens:
13363 ▷ $(ORBITER) -v 2 \
13364 ▷ ▷ -define F -finite_field -q 2 -end \
13365 ▷ ▷ -define G -linear_group -PGL 12 2 \
13366 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
13367 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
13368 ▷ ▷ -end \
13369 ▷ ▷ -with G -do \
13370 ▷ ▷ -group_theoretic_activity \
13371 ▷ ▷ ▷ -report \
13372 ▷ ▷ -end
13373 ▷ pdflatex PGL_12.2.Subgroup_M24_244823040_report.tex
13374 ▷ $(OPEN) PGL_12.2.Subgroup_M24_244823040_report.pdf
13375
13376
13377 Golay24_make_aut_gens_classes:
13378 ▷ $(ORBITER) -v 3 \
13379 ▷ ▷ -define F -finite_field -q 2 -end \
13380 ▷ ▷ -define G -linear_group -PGL 12 2 \
13381 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
13382 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
13383 ▷ ▷ -end \
13384 ▷ ▷ -with G -do \
13385 ▷ ▷ -group_theoretic_activity \
13386 ▷ ▷ ▷ -classes \
13387 ▷ ▷ -end
13388 ▷ $(MAGMA_PATH)magma M24_classes.magma
13389 ▷ $(ORBITER) -v 3 \
13390 ▷ ▷ -define F -finite_field -q 2 -end \
13391 ▷ ▷ -define G -linear_group -PGL 12 2 \
13392 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
13393 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
13394 ▷ ▷ -end \
13395 ▷ ▷ -with G -do \
13396 ▷ ▷ -group_theoretic_activity \
13397 ▷ ▷ ▷ -classes \
13398 ▷ ▷ -end
13399 ▷ pdflatex PGL_12.2.Subgroup_M24_244823040_classes_out.tex
13400 ▷ $(OPEN) PGL_12.2.Subgroup_M24_244823040_classes_out.pdf
13401
13402
13403
13404 M24_elt_2A:
13405 ▷ $(ORBITER) -v 2 \
13406 ▷ ▷ -define F -finite_field -q 2 -end \
13407 ▷ ▷ -define G -linear_group -PGL 12 2 \
13408 ▷ ▷ -subgroup_by_generators "M24_2A" "2" 1 \
13409 ▷ ▷ ▷ $(M24_ELEMENT_2A) \
13410 ▷ ▷ -end \
13411 ▷ ▷ -with G -do \
13412 ▷ ▷ -group_theoretic_activity \
13413 ▷ ▷ ▷ -report \
13414 ▷ ▷ -end
13415 ▷ pdflatex PGL_12.2.Subgroup_M24_2A_2_report.tex

```

```

13416 ▷ $(OPEN) PGL_12.2_Subgroup.M24.2A.2_report.pdf
13417
13418 M24_elt_2B:
13419 ▷ $(ORBITER) -v 2 \
13420 ▷ ▷ -define F -finite_field -q 2 -end \
13421 ▷ ▷ -define G -linear_group -PGL 12 2 \
13422 ▷ ▷ -subgroup_by_generators "M24.2B" "2" 1 \
13423 ▷ ▷ ▷ $(M24_ELEMENT_2B) \
13424 ▷ ▷ -end \
13425 ▷ ▷ -with G -do \
13426 ▷ ▷ -group_theoretic_activity \
13427 ▷ ▷ ▷ -report \
13428 ▷ ▷ -end
13429 ▷ pdflatex PGL_12.2_Subgroup.M24.2B.2_report.tex
13430 ▷ $(OPEN) PGL_12.2_Subgroup.M24.2B.2_report.pdf
13431
13432 M24_2A_restricted:
13433 ▷ $(ORBITER) -v 2 \
13434 ▷ ▷ -define v -vector \
13435 ▷ ▷ ▷ -dense $(GOLAY_24.CODE_PROJECTIVE) \
13436 ▷ ▷ -end \
13437 ▷ ▷ -define F -finite_field -q 2 -end \
13438 ▷ ▷ -define G -linear_group -PGL 12 2 \
13439 ▷ ▷ ▷ -subgroup_by_generators "M24.2A" "2" 1 \
13440 ▷ ▷ ▷ $(M24_ELEMENT_2A) \
13441 ▷ ▷ -end \
13442 ▷ ▷ -define M24_2Ar -modified_group -from G -restricted_action v \\_on\\_code -en
    d \
13443 ▷ ▷ -with M24_2Ar -do \
13444 ▷ ▷ ▷ -group_theoretic_activity \
13445 ▷ ▷ ▷ ▷ -print_elements_tex \
13446 ▷ ▷ -end
13447 ▷ pdflatex PGL_12.2_res.v_elements.tex
13448 ▷ $(OPEN) PGL_12.2_res.v_elements.pdf
13449
13450
13451 # 8 fixed points, 8 cycles of length 2.
13452
13453
13454
13455
13456
13457 M24_2B_restricted:
13458 ▷ $(ORBITER) -v 2 \
13459 ▷ ▷ -define v -vector \
13460 ▷ ▷ ▷ -dense $(GOLAY_24.CODE_PROJECTIVE) \
13461 ▷ ▷ -end \
13462 ▷ ▷ -define F -finite_field -q 2 -end \
13463 ▷ ▷ -define G -linear_group -PGL 12 2 \
13464 ▷ ▷ ▷ -subgroup_by_generators "M24.2B" "2" 1 \
13465 ▷ ▷ ▷ $(M24_ELEMENT_2B) \
13466 ▷ ▷ -end \
13467 ▷ ▷ -define M24_2Br -modified_group \
13468 ▷ ▷ ▷ -from G -restricted_action v v \
13469 ▷ ▷ -end \
13470 ▷ ▷ -with M24_2Br -do \
13471 ▷ ▷ ▷ -group_theoretic_activity \
13472 ▷ ▷ ▷ ▷ -print_elements_tex \
13473 ▷ ▷ -end

```

```

13474 ▷ pdflatex PGL_12.2_res24_elements.tex
13475 ▷ $(OPEN) PGL_12.2_res24_elements.pdf
13476
13477 # 12 cycles of length 2
13478
13479
13480
13481
13482 Golay24_2A.fixed.subcode:
13483 ▷ $(ORBITER) -v 2 \
13484 ▷ ▷ -define perm -vector \
13485 ▷ ▷ ▷ -dense $(M24_2A_PERM) \
13486 ▷ ▷ -end \
13487 ▷ ▷ -define F -finite.field -q 2 -end \
13488 ▷ ▷ -define v -vector \
13489 ▷ ▷ ▷ -dense $(GOLAY_24.CODE.PROJECTIVE) \
13490 ▷ ▷ -end \
13491 ▷ ▷ -define C -code -field F \
13492 ▷ ▷ ▷ -projective_set 12 v \
13493 ▷ ▷ -end \
13494 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13495 ▷ ▷ ▷ -fixed_code perm \
13496 ▷ ▷ -end
13497
13498 # 256 codewords
13499
13500 Golay24_2B.fixed.subcode:
13501 ▷ $(ORBITER) -v 2 \
13502 ▷ ▷ -define perm -vector \
13503 ▷ ▷ ▷ -dense $(M24_2B_PERM) \
13504 ▷ ▷ -end \
13505 ▷ ▷ -define F -finite.field -q 2 -end \
13506 ▷ ▷ -define v -vector \
13507 ▷ ▷ ▷ -dense $(GOLAY_24.CODE.PROJECTIVE) \
13508 ▷ ▷ -end \
13509 ▷ ▷ -define C -code -field F \
13510 ▷ ▷ ▷ -projective_set 12 v \
13511 ▷ ▷ -end \
13512 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13513 ▷ ▷ ▷ -fixed_code perm \
13514 ▷ ▷ -end
13515
13516 # 64 codewords
13517
13518
13519 Golay24_2A.fix.code:
13520 ▷ $(ORBITER) -v 2 \
13521 ▷ ▷ -define F -finite.field -q 2 -end \
13522 ▷ ▷ -define v -vector \
13523 ▷ ▷ ▷ -format 8 \
13524 ▷ ▷ ▷ -dense $(GOLAY24_2A.FIX.SUBCODE) \
13525 ▷ ▷ -end \
13526 ▷ ▷ -define C -code -field F \
13527 ▷ ▷ ▷ -generator_matrix v \
13528 ▷ ▷ -end \
13529 ▷ ▷ -with C -do \
13530 ▷ ▷ -coding_theoretic_activity \
13531 ▷ ▷ ▷ -weight_enumerator \
13532 ▷ ▷ -end \

```



```

13533 ▷ ▷ -with C -do \
13534 ▷ ▷ -coding_theoretic_activity \
13535 ▷ ▷ ▷ -export_codewords_by_weight Golay24_2A_fix \
13536 ▷ ▷ -end
13537
13538 #1y^{24} + 71x^8y^{16} + 112x^{12}y^{12} + 71x^{16}y^8 + 1x^{24}
13539 #weight enumerator:
13540 #( 1, 0, 0, 0, 0, 0, 0, 0, 0, 71, 0, 0, 0, 112, 0, 0, 0, 71, 0, 0, 0, 0, 0, 0, 1
13541 )
13542 # 112 codewords of weight 12:
13543 #Golay24_2A_fix_of_weight_12.csv
13544
13545
13546 Golay24_2B_fix_code:
13547 ▷ $(ORBITER) -v 2 \
13548 ▷ ▷ -define F -finite_field -q 2 -end \
13549 ▷ ▷ -define v -vector \
13550 ▷ ▷ ▷ -format 6 \
13551 ▷ ▷ ▷ -dense $(GOLAY24_2B_FIX_SUBCODE) \
13552 ▷ ▷ -end \
13553 ▷ ▷ -define C -code -field F \
13554 ▷ ▷ ▷ -generator_matrix v \
13555 ▷ ▷ -end \
13556 ▷ ▷ -with C -do \
13557 ▷ ▷ -coding_theoretic_activity \
13558 ▷ ▷ ▷ -weight_enumerator \
13559 ▷ ▷ -end \
13560 ▷ ▷ -with C -do \
13561 ▷ ▷ -coding_theoretic_activity \
13562 ▷ ▷ ▷ -export_codewords_by_weight Golay24_2B_fix \
13563 ▷ ▷ -end
13564
13565 #1y^{24} + 15x^8y^{16} + 32x^{12}y^{12} + 15x^{16}y^8 + 1x^{24}
13566 #weight enumerator:
13567 #( 1, 0, 0, 0, 0, 0, 0, 0, 0, 15, 0, 0, 0, 32, 0, 0, 0, 15, 0, 0, 0, 0, 0, 0, 1 )

13568
13569 Golay24_2A_fix_weight12:
13570 ▷ $(ORBITER) -v 2 \
13571 ▷ ▷ -define F -finite_field -q 2 -end \
13572 ▷ ▷ -define v -vector \
13573 ▷ ▷ ▷ -load_csv_data_column Golay24_2A_fix_of_weight_12.csv 1 \
13574 ▷ ▷ -end \
13575 ▷ ▷ -define D -design -list_of_blocks_coded \
13576 ▷ ▷ ▷ 24 12 v \
13577 ▷ ▷ -end \
13578 ▷ ▷ -with D -do \
13579 ▷ ▷ ▷ -design_activity \
13580 ▷ ▷ ▷ ▷ -export_inc \
13581 ▷ ▷ -end \
13582 ▷ ▷ -with D -do \
13583 ▷ ▷ ▷ -design_activity \
13584 ▷ ▷ ▷ ▷ -intersection_matrix \
13585 ▷ ▷ -end
13586
13587 #sets_v24_inc.txt
13588 #sets_v24_AAt.csv
13589

```

```

13590
13591 Golay24_2A_fix_weight12.c:
13592 ▷ $(ORBITER) -v 3 \
13593 ▷ ▷ -draw_incidence_structure_description \
13594 ▷ ▷ ▷ -width 60 -width_10 6 -end \
13595 ▷ ▷ -define C -combinatorial_object \
13596 ▷ ▷ ▷ -label Golay24_2A_fix_weight12 Golay24\_2A\_fix\_weight12 \
13597 ▷ ▷ ▷ -file_of_incidence_geometries blocks_v24_k12_inc.txt 24 112 1344 \
13598 ▷ ▷ -end \
13599 ▷ ▷ -with C -do \
13600 ▷ ▷ -combinatorial_object_activity \
13601 ▷ ▷ ▷ -canonical_form \
13602 ▷ ▷ ▷ ▷ -save_ago \
13603 ▷ ▷ ▷ ▷ -save_transversal \
13604 ▷ ▷ ▷ -end \
13605 ▷ ▷ -end \
13606 ▷ ▷ -with C -do \
13607 ▷ ▷ -combinatorial_object_activity \
13608 ▷ ▷ ▷ -report \
13609 ▷ ▷ ▷ ▷ -export_flag_orbits \
13610 ▷ ▷ ▷ ▷ -show_incidence_matrices \
13611 ▷ ▷ ▷ ▷ -export_group_GAP \
13612 ▷ ▷ ▷ -end \
13613 ▷ ▷ -end
13614 ▷ pdflatex Golay24_2A_fix_weight12_classification.tex
13615 ▷ $(OPEN) Golay24_2A_fix_weight12_classification.pdf
13616
13617
13618 # stabilizer of order 2752512
13619
13620 Golay24_2A_fix_weight12_graph_24:
13621 ▷ $(ORBITER) -v 3 \
13622 ▷ ▷ -define Gamma -graph \
13623 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
13624 ▷ ▷ ▷ blocks_v24_k12_AAt.csv 24 \
13625 ▷ ▷ -end \
13626 ▷ ▷ -with Gamma -do \
13627 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
13628 ▷ ▷ -end \
13629 ▷ ▷ -with Gamma -do \
13630 ▷ ▷ ▷ -graph_theoretic_activity \
13631 ▷ ▷ ▷ -properties \
13632 ▷ ▷ -end
13633 ▷ #pdflatex _report.tex
13634 ▷ #$(OPEN) _report.pdf
13635
13636 #Degree type: (14^{16},\, , 7^8 )
13637
13638
13639 #####
13640 # Section 10.4: Coding Theory - CRC codes
13641
13642 SECTION_CODING_THEORY_CRC_CODES:
13643
13644
13645 test_10_4:
13646 ▷ make CRC_3_128_10
13647 ▷ make crc32_test
13648 ▷ make crc32_test_hexdata

```

```

13649 ▷ make crc32_Berlekamp_matrix
13650 ▷ make CRC_F256_roots_771
13651 ▷ make CRC_F256_BCH_code_d2
13652 ▷ make CRC_F256_BCH_write_code_for_division_d2
13653 ▷ make CRC_F256_BCH_code_d3
13654 ▷ make CRC_F256_BCH_write_code_for_division_Bravo
13655 ▷ make CRC_F256_BCH_code_d7
13656 ▷ make CRC_F256_BCH_write_code_for_division_Charlie
13657 ▷ make F256_BCH_code_d16
13658 ▷ make F256_BCH_write_code_for_division_d16
13659 ▷ make F256_BCH_code_d16_division
13660 ▷ make F256_BCH_code_d16_error
13661 ▷ make F_256
13662 ▷ make CRC_F_16
13663 ▷ make nth_roots_256
13664 ▷ make CRC_F16_roots_51
13665 ▷ make CRC_F16_BCH_code_d3
13666 ▷ make CRC_F16_BCH_code_d3_mindist_dual
13667 ▷ make CRC_F16_BCH_code_d3_dual_weight_enumerator
13668 ▷ make CRC_F16_BCH_code_d3_projective_set
13669 ▷ make CRC_F16_BCH_code_d3_macwilliams
13670 ▷ make CRC_F16_BCH_write_code_for_division_Delta
13671 ▷ make crc_encode_16
13672 ▷ make crc_encode_32
13673 ▷ make introduce_errors_16_500000
13674 ▷ make introduce_errors_32_100000
13675 ▷ make check_errors_16
13676 ▷ make check_errors_32
13677 ▷ make extract_block
13678 ▷ make crc_test_alfa
13679 ▷ make crc_test_bravo_10M_3
13680 ▷ make error_polynomial_bravo_552
13681 ▷ make crc_test_crc32_10M_3
13682 ▷ make crc_test_bravo_10M_4
13683 ▷ make crc_test_crc32_10M_4
13684 ▷ make crc_test_bravo_10M_5
13685 ▷ make crc_test_crc32_10M_5
13686 ▷ make crc_test_bravo_10M_6
13687 ▷ make crc_test_crc32_10M_6
13688 ▷ make crc_test_bravo_10M_7
13689 ▷ make crc_test_crc32_10M_7
13690 ▷ make crc_test_bravo_10M_8
13691 ▷ make crc_test_crc32_10M_8
13692 ▷ make crc_test_bravo_10_loop
13693 ▷ make crc_test_crc32_10_loop
13694 ▷ make crc_test_crc32_20M_3
13695 ▷ make crc_test_charlie
13696
13697
13698
13699 CRC_3.128.10:
13700 ▷ $(ORBITER) -v 1 \
13701 ▷ ▷ -define F -finite_field -q 2 -end \
13702 ▷ ▷ -with F -do -coding-theoretic-activity \
13703 ▷ ▷ ▷ -find_CRC_polynomials 3 128 10 \
13704 ▷ ▷ -end
13705
13706
13707

```

```

13708
13709 crc32_test:
13710 ▷ $(ORBITER) -v 3 \
13711 ▷ ▷ -define F -finite_field -q 2 -end \
13712 ▷ ▷ -with F -do -coding_theoretic_activity \
13713 ▷ ▷ ▷ -crc32 "123456789" \
13714 ▷ ▷ -end
13715
13716 crc32_test.hexdata:
13717 ▷ $(ORBITER) -v 3 \
13718 ▷ ▷ -define F -finite_field -q 2 -end \
13719 ▷ ▷ -with F -do -coding_theoretic_activity \
13720 ▷ ▷ ▷ -crc32_hexdata "7BD11C4010" \
13721 ▷ ▷ -end
13722
13723 crc32_Berlekamp_matrix:
13724 ▷ $(ORBITER) -v 2 \
13725 ▷ ▷ -define F -finite_field -q 2 -end \
13726 ▷ ▷ -define v -vector -field F -sparse 33 $(CRC32_ETHERNET) -end \
13727 ▷ ▷ -with F -do \
13728 ▷ ▷ -finite_field_activity \
13729 ▷ ▷ ▷ -Berlekamp_matrix v \
13730 ▷ ▷ -end
13731
13732
13733 CRC_F256_roots_771:
13734 ▷ $(ORBITER) -v 3 \
13735 ▷ ▷ -define F -finite_field -q 256 -end \
13736 ▷ ▷ -with F -do -finite_field_activity \
13737 ▷ ▷ ▷ -nth_roots 771 \
13738 ▷ ▷ -end
13739
13740
13741 # alfa:
13742 # 255 = 3 * 5 * 17
13743 # 771 = 3 * 257
13744
13745 CRC_F256_BCH_code_d2:
13746 ▷ $(ORBITER) -v 2 \
13747 ▷ ▷ -define F -finite_field -q 256 -end \
13748 ▷ ▷ -define C -code -field F \
13749 ▷ ▷ ▷ -BCH 771 2 \
13750 ▷ ▷ -end \
13751 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13752 ▷ ▷ ▷ -export_magma BCH_lq8_n771_d2.magma \
13753 ▷ ▷ -end
13754 ▷ #pdflatex BCH_codes.q256_n771_d2.tex
13755 ▷ #$(OPEN) BCH_codes.q256_n771_d2.pdf
13756
13757 #Q=X^{2} + 167X + 214
13758 # degree of polynomial = 3
13759 #-dense "214,167,1"
13760 #-sparse "214,0,167,1,1,2"
13761
13762
13763
13764 CRC_F256_BCH_write_code_for_division_d2:
13765 ▷ $(ORBITER) -v 2 \
13766 ▷ ▷ -define F -finite_field -q 256 -end \

```

```

13767 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
13768 ▷ ▷ -define B -vector -field F -dense $(CRC.POLY.Q256_DEG2.DENSE) -end \
13769 ▷ ▷ -with F -do \
13770 ▷ ▷ -coding_theoretic_activity \
13771 ▷ ▷ ▷ -write_code_for_division \
13772 ▷ ▷ ▷ alfa A B \
13773 ▷ ▷ -end
13774 ▷ g++ crc_alfa.cpp -o crc_alfa.out
13775 ▷ ./crc_alfa.out
13776
13777
13778
13779
13780
13781 # bravo:
13782
13783
13784
13785 # degree of polynomial = 4
13786 #-dense "1,23,27,213,1"
13787 #-sparse "1,0,23,1,27,2,213,3,1,4"
13788
13789 CRC_F256_BCH_code_d3:
13790 ▷ $(ORBITER) -v 2 \
13791 ▷ ▷ -define F -finite_field -q 256 -end \
13792 ▷ ▷ -define C -code -field F \
13793 ▷ ▷ ▷ -BCH 771 3 \
13794 ▷ ▷ -end \
13795 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
13796 ▷ ▷ ▷ -export_magma BCH_lq8_n771_d3.magma \
13797 ▷ ▷ -end
13798 ▷ #pdflatex BCH_codes_q256_n771_d3.tex
13799 ▷ #$(OPEN) BCH_codes_q256_n771_d3.pdf
13800
13801
13802
13803 CRC_F256_BCH_write_code_for_division_Bravo:
13804 ▷ $(ORBITER) -v 2 \
13805 ▷ ▷ -define F -finite_field -q 256 -end \
13806 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
13807 ▷ ▷ -define B -vector -field F -dense $(CRC.POLY.BRAVO.DENSE) -end \
13808 ▷ ▷ -with F -do \
13809 ▷ ▷ -coding_theoretic_activity \
13810 ▷ ▷ ▷ -write_code_for_division \
13811 ▷ ▷ ▷ bravo A B \
13812 ▷ ▷ -end
13813 ▷ g++ crc_bravo.cpp -o crc_bravo.out
13814 ▷ ./crc_bravo.out
13815
13816
13817 # Charlie
13818
13819 CRC_F256_BCH_code_d7:
13820 ▷ $(ORBITER) -v 2 \
13821 ▷ ▷ -define F -finite_field -q 256 -end \
13822 ▷ ▷ -define C -code -field F \
13823 ▷ ▷ ▷ -BCH 771 7 \
13824 ▷ ▷ -end \
13825 ▷ ▷ -with C -and F -do -coding_theoretic_activity \

```

```

13826 ▷ ▷ ▷ -export_magma BCH_lq8_n771_d7.magma \
13827 ▷ ▷ -end
13828 ▷ pdflatex BCH_codes_q256_n771_d7.tex
13829 ▷ $(OPEN) BCH_codes_q256_n771_d7.pdf
13830
13831
13832 # polynomial of degree 12:
13833 #-dense "1,126,25,1,196,209,244,3,121,126,35,65,1"
13834 #-sparse "1,0,126,1,25,2,1,3,196,4,209,5,244,6,3,7,121,8,126,9,35,10,65,11,1,12"
13835
13836
13837 CRC_F256_BCH_write_code_for_division_Charlie:
13838 ▷ $(ORBITER) -v 2 \
13839 ▷ ▷ -define F -finite_field -q 256 -end \
13840 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
13841 ▷ ▷ -define B -vector -field F -dense $(CRC_POLY_CHARLIE_DENSE) -end \
13842 ▷ ▷ -with F -do \
13843 ▷ ▷ -coding_theoretic_activity \
13844 ▷ ▷ ▷ -write_code_for_division \
13845 ▷ ▷ ▷ charlie A B \
13846 ▷ ▷ -end
13847 ▷ g++ crc_charlie.cpp -o crc_charlie.out
13848 ▷ ./crc_charlie.out
13849
13850
13851
13852
13853
13854
13855 F256_BCH_code_d16:
13856 ▷ $(ORBITER) -v 3 \
13857 ▷ ▷ -define F -finite_field -q 256 -end \
13858 ▷ ▷ -define C -code -field F \
13859 ▷ ▷ ▷ -BCH 771 16 \
13860 ▷ ▷ -end
13861 ▷ pdflatex BCH_codes_q256_n771_d16.tex
13862 ▷ $(OPEN) BCH_codes_q256_n771_d16.pdf
13863
13864 #generator polynomial is  $X^{30} + 253X^{29} + 174X^{28} + 109X^{27} + 97X^{26} +$ 
 $144X^{25} + 112X^{24} + 212X^{23} + 192X^{22} + 169X^{21} + 24X^{20} + 150X^{19}$ 
 $+ 110X^{18} + 248X^{17} + 3X^{16} + 193X^{15} + 194X^{14} + 205X^{13} + 9X^{12} +$ 
 $56X^{11} + 95X^{10} + 199X^9 + 108X^8 + 58X^7 + 160X^6 + 148X^5 + 138X^4 +$ 
 $24X^3 + 210X^2 + 26X + 1$ 
13865
13866
13867
13868
13869
13870 F256_BCH_write_code_for_division_d16:
13871 ▷ $(ORBITER) -v 2 \
13872 ▷ ▷ -define F -finite_field -q 256 -end \
13873 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
13874 ▷ ▷ -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
13875 ▷ ▷ -with F -do \
13876 ▷ ▷ -coding_theoretic_activity \
13877 ▷ ▷ ▷ -write_code_for_division \
13878 ▷ ▷ ▷ check_q256_n771_r30 A B \
13879 ▷ ▷ -end
13880 ▷ g++ crc_check_q256_n771_r30.cpp -o crc_check_q256_n771_r30.out

```

```

13881 ▷ ./crc_check.q256_n771_r30.out
13882
13883
13884
13885 F256_BCH_code.d16_division:
13886 ▷ $(ORBITER) -v 2 \
13887 ▷ ▷ -define F -finite_field -q 256 -end \
13888 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
13889 ▷ ▷ -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
13890 ▷ ▷ -with F -do \
13891 ▷ ▷ -finite_field_activity \
13892 ▷ ▷ -polynomial_division A B -end
13893
13894
13895
13896 F256_BCH_code.d16_error:
13897 ▷ $(ORBITER) -v 2 \
13898 ▷ ▷ -define F -finite_field -q 256 -end \
13899 ▷ ▷ -define A -vector -field F -sparse 771 "2,30,3,31,55,770" -end \
13900 ▷ ▷ -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
13901 ▷ ▷ -with F -do \
13902 ▷ ▷ -finite_field_activity \
13903 ▷ ▷ -polynomial_division A B -end
13904
13905
13906
13907
13908 # delta:
13909 # 15 = 3 * 5
13910 # 51 = 3 * 17
13911
13912 F_256:
13913 ▷ $(ORBITER) -v 3 \
13914 ▷ ▷ -define F -finite_field -q 256 -end \
13915 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
13916 ▷ pdflatex GF_256.tex
13917 ▷ $(OPEN) GF_256.pdf
13918
13919
13920 # subfield F16: X^4+X+1
13921
13922
13923 # F16: X^4 + X3 + 1 not compatible with F_256 and x mapsto x^17
13924
13925 CRC_F_16:
13926 ▷ $(ORBITER) -v 3 \
13927 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
13928 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
13929 ▷ pdflatex GF_16_poly19.tex
13930 ▷ $(OPEN) GF_16_poly19.pdf
13931
13932 nth_roots_256:
13933 ▷ $(ORBITER) -v 3 \
13934 ▷ ▷ -define F -finite_field -q 256 -end \
13935 ▷ ▷ -with F -do -finite_field_activity \
13936 ▷ ▷ ▷ -nth_roots 51 \
13937 ▷ ▷ -end
13938 ▷ pdflatex Nth_roots_q2_n51.tex
13939 ▷ $(OPEN) Nth_roots_q2_n51.pdf

```

[illegible]



```

13997 #1y^{51} + 3060x^{44}y^7 + 18360x^{46}y^5 + 25755x^{48}y^3 + 18360x^{50}y
13998 # 1 + 3060 + 18360 + 25755 + 18360 = 65536 = 2^16 = 16^4
13999
14000
14001 CRC_F16_BCH_code.d3.projective.set:
14002 ▷ $(ORBITER) -v 2 \
14003 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
14004 ▷ ▷ -define v -vector -file BCH_q16_n51_d3_check_ranks.csv \
14005 ▷ ▷ ▷ -file BCH_q16_n51_d3_check_ranks.csv -end \
14006
14007
14008
14009
14010 CRC_F16_BCH_code.d3.macwilliams:
14011 ▷ $(ORBITER) -v 2 \
14012 ▷ ▷ -make_macwilliams_system 51 4 16
14013
14014
14015
14016 CRC_F16_BCH_write_code_for_division_Delta:
14017 ▷ $(ORBITER) -v 2 \
14018 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
14019 ▷ ▷ -define A -vector -field F -sparse 52 "1,51,1,0" -end \
14020 ▷ ▷ -define B -vector -field F -dense $(CRC_PLY_DELTA_DENSE) -end \
14021 ▷ ▷ -with F -do \
14022 ▷ ▷ -coding_theoretic_activity \
14023 ▷ ▷ ▷ -write_code_for_division \
14024 ▷ ▷ ▷ delta A B \
14025 ▷ ▷ -end
14026 ▷ g++ crc_delta.cpp -o crc_delta.out
14027 ▷ ./crc_delta.out
14028
14029 #g++ crc_delta.cpp -o crc_delta.out
14030 #./crc_delta.out
14031 #0,0,0,0,0,
14032
14033
14034
14035
14036 crc_encode_16:
14037 ▷ $(ORBITER) -v 3 \
14038 ▷ ▷ -define F -finite_field -q 2 -end \
14039 ▷ ▷ -with F -do \
14040 ▷ ▷ ▷ -coding_theoretic_activity \
14041 ▷ ▷ ▷ ▷ -crc_encode_file_based \
14042 ▷ ▷ ▷ ▷ $(CRC_FILE).$(CRC_FILE_EXTENSION) \
14043 ▷ ▷ ▷ ▷ $(CRC_FILE)_crc16.bin crc16 771 \
14044 ▷ ▷ -end
14045
14046 #-rw-r--r-- 1 betten staff 646576 Aug 24 14:35 allen_Gates_crc16.bin
14047 #-rw-r--r-- 1 betten staff 21656232 Aug 24 15:35 javad-allahyari-Fs1E2JXM3Gc-u
nsplash_crc16.bin
14048
14049
14050
14051 crc_encode_32:
14052 ▷ $(ORBITER) -v 3 \
14053 ▷ ▷ -define F -finite_field -q 2 -end \
14054 ▷ ▷ -with F -do \

```

```

14055 ▷ ▷ ▷ -coding_theoretic_activity \
14056 ▷ ▷ ▷ ▷ -crc_encode_file_based \
14057 ▷ ▷ ▷ ▷ $(CRC_FILE).$(CRC_FILE_EXTENSION) \
14058 ▷ ▷ ▷ ▷ $(CRC_FILE)_crc32.bin crc32 771 \
14059 ▷ ▷ -end
14060
14061
14062 #-rw-r--r-- 1 betten staff 648262 Aug 24 14:34 allen_Gates_crc32.bin
14063
14064
14065
14066 #crc_encode_new:
14067 #▷ $(ORBITER) -v 3 \
14068 #▷ ▷ -define F -finite_field -q 256 -end \
14069 #▷ ▷ -with F -do \
14070 #▷ ▷ ▷ -coding_theoretic_activity \
14071 #▷ ▷ ▷ -crc_new_file_based $(CRC_FILE).$(CRC_FILE_EXTENSION) \
14072 #▷ ▷ -end
14073
14074
14075 introduce_errors_16_500000:
14076 ▷ $(ORBITER) -v 3 \
14077 ▷ ▷ -introduce_errors \
14078 ▷ ▷ ▷ -input $(CRC_FILE)_crc16.bin \
14079 ▷ ▷ ▷ -output $(CRC_FILE)_crc16.e.bin \
14080 ▷ ▷ ▷ -block_based_error_generator \
14081 ▷ ▷ ▷ -block_length 771 \
14082 ▷ ▷ ▷ -threshold 500000 \
14083 ▷ ▷ ▷ -file_based_error_generator 500000 \
14084 ▷ ▷ ▷ -nb_repeats 30 \
14085 ▷ ▷ -end
14086
14087
14088 introduce_errors_32_100000:
14089 ▷ $(ORBITER) -v 3 \
14090 ▷ ▷ -introduce_errors \
14091 ▷ ▷ ▷ -input $(CRC_FILE)_crc32.bin \
14092 ▷ ▷ ▷ -output $(CRC_FILE)_crc32.e.bin \
14093 ▷ ▷ ▷ -block_based_error_generator \
14094 ▷ ▷ ▷ -block_length 771 \
14095 ▷ ▷ ▷ -threshold 100000 \
14096 ▷ ▷ ▷ -file_based_error_generator 100000 \
14097 ▷ ▷ ▷ -nb_repeats 30 \
14098 ▷ ▷ -end
14099
14100
14101 check_errors_16:
14102 ▷ $(ORBITER) -v 3 \
14103 ▷ ▷ -check_errors \
14104 ▷ ▷ ▷ -input $(CRC_FILE)_crc16.e.bin \
14105 ▷ ▷ ▷ -output $(CRC_FILE)_recovered.$(CRC_FILE_EXTENSION) \
14106 ▷ ▷ ▷ -crc_type crc16 \
14107 ▷ ▷ ▷ -error_log $(CRC_FILE)_crc16.e.pattern.csv \
14108 ▷ ▷ ▷ -block_length 771 \
14109 ▷ ▷ -end
14110
14111 check_errors_32:
14112 ▷ $(ORBITER) -v 3 \
14113 ▷ ▷ -check_errors \

```

```

14114 ▷ ▷ ▷ -input $(CRC_FILE)_crc32.e.bin \
14115 ▷ ▷ ▷ -output $(CRC_FILE)_recovered.$(CRC_FILE_EXTENSION) \
14116 ▷ ▷ ▷ -crc_type crc32 \
14117 ▷ ▷ ▷ -error_log $(CRC_FILE)_crc32_e.pattern.csv \
14118 ▷ ▷ ▷ -block_length 771 \
14119 ▷ ▷ -end
14120
14121
14122 extract_block:
14123 ▷ $(ORBITER) -v 3 \
14124 ▷ ▷ -extract_block \
14125 ▷ ▷ ▷ -input $(CRC_FILE)_crc32.e.bin \
14126 ▷ ▷ ▷ -output $(CRC_FILE)_recovered.$(CRC_FILE_EXTENSION) \
14127 ▷ ▷ ▷ -error_log $(CRC_FILE)_crc32_e.pattern.csv \
14128 ▷ ▷ ▷ -block_length 771 \
14129 ▷ ▷ ▷ -selected_block 813731 \
14130 ▷ ▷ -end
14131
14132
14133
14134 # alfa has d = 2, so it can detect any 1 error at the F256 level
14135
14136 crc_test_alfa:
14137 ▷ $(ORBITER) -v 3 \
14138 ▷ ▷ -crc_test "alfa" $(TEN_MILLION) 3
14139
14140
14141 #crc_codes::test_crc_object Nb_test = 10000000 k = 2 nb_undetected=89, time=0:24
14142 #crc_codes::test_crc_object Nb_test = 10000000 k = 3 nb_undetected=153, time=0:26

14143
14144
14145
14146 # bravo has d = 3, so it can detect any 2 errors at the F256 level
14147
14148 crc_test_bravo_10M_3:
14149 ▷ $(ORBITER) -v 3 \
14150 ▷ ▷ -crc_test "bravo" $(TEN_MILLION) 3
14151
14152 #crc_codes::test_crc_object Nb_test = 10000000 k = 3 nb_undetected=0, time=0:27
14153 #23754:crc_codes::test_crc_object Nb_test = 10000000 k = 552 nb_undetected=1, time=11:54
14154 #12596,0,( 737, 417, 171, 634, 158, 30, 480, 688, 94, 739, 147, 694, 724, 741, 51
, 411, 753, 622, 592, 667, 25, 709, 55, 450, 255, 744, 28, 718, 82, 740, 372, 144
, 435, 505, 478, 233, 325, 231, 27, 518, 492, 350, 621, 468, 297, 141, 735, 338,
227, 434, 54, 180, 390, 245, 301, 617, 423, 408, 521, 42, 697, 719, 668, 618, 107
, 746, 441, 376, 705, 353, 368, 254, 393, 291, 762, 188, 506, 580, 419, 525, 500,
652, 433, 130, 52, 212, 511, 191, 182, 100, 676, 706, 248, 268, 365, 626, 303, 5
, 490, 524, 237, 210, 326, 619, 485, 767, 432, 721, 391, 650, 88, 627, 177, 700,
354, 682, 38, 360, 70, 636, 609, 766, 203, 7, 369, 584, 475, 84, 87, 251, 680, 47
9, 256, 121, 473, 604, 614, 265, 585, 40, 458, 551, 163, 661, 358, 610, 349, 748,
371, 310, 31, 136, 153, 649, 662, 547, 120, 324, 162, 671, 62, 725, 603, 645, 67
0, 554, 364, 127, 312, 708, 386, 286, 532, 58, 658, 366, 74, 442, 699, 707, 164,
102, 593, 657, 293, 638, 370, 716, 454, 126, 204, 232, 637, 409, 491, 666, 402, 2
09, 639, 560, 578, 467, 24, 629, 93, 453, 352, 425, 406, 37, 684, 189, 648, 186,
345, 691, 673, 193, 264, 247, 714, 267, 540, 215, 134, 729, 588, 295, 294, 241, 5
77, 398, 722, 46, 214, 60, 640, 770, 477, 106, 97, 381, 280, 675, 418, 29, 385, 2
69, 151, 665, 109, 514, 305, 377, 95, 544, 570, 552, 128, 152, 599, 397, 415, 149
, 451, 738, 743, 302, 140, 273, 103, 284, 396, 726, 327, 221, 288, 200, 344, 285,

```

```

112, 542, 22, 313, 263, 218, 61, 157, 362, 559, 507, 243, 664, 206, 550, 242, 24
4, 230, 145, 761, 413, 679, 392, 447, 17, 330, 156, 279, 752, 487, 731, 651, 388,
535, 154, 502, 108, 452, 363, 240, 569, 564, 713, 704, 495, 589, 83, 15, 483, 26
1, 537, 595, 219, 573, 516, 252, 199, 337, 616, 339, 486, 555, 653, 728, 137, 216
, 56, 211, 734, 449, 44, 26, 348, 426, 91, 139, 238, 539, 444, 754, 672, 695, 543
, 659, 601, 654, 742, 300, 69, 115, 756, 122, 582, 374, 236, 656, 533, 755, 512,
594, 404, 65, 611, 316, 703, 361, 318, 80, 92, 12, 720, 378, 763, 228, 429, 623,
608, 712, 33, 612, 185, 747, 11, 620, 78, 234, 202, 165, 498, 173, 160, 494, 768,
75, 306, 482, 641, 53, 222, 379, 460, 258, 567, 50, 531, 548, 207, 424, 320, 383
, 124, 757, 249, 10, 405, 575, 597, 587, 341, 328, 336, 513, 317, 172, 681, 169,
187, 430, 351, 523, 81, 553, 389, 496, 689, 461, 607, 359, 436, 105, 229, 8, 101,
176, 663, 43, 701, 356, 630, 562, 678, 272, 224, 387, 271, 311, 456, 530, 401, 2
78, 239, 520, 529, 287, 343, 59, 367, 342, 21, 503, 420, 466, 322, 146, 504, 563,
196, 545, 558, 161, 394, 257, 49, 99, 598, 6, 77, 143, 68, 118, 133, 690, 410, 2
35, 536, 48, 205, 270, 445, 283, 198, 660, 90, 47, 57, 727, 41, 463, 76, 581, 576
, 192, 250, 686, 509, 34, 175, 246, 400, 168, 111, 334, 403, 517, 677, 373, 497,
431, 470, 501, 174, 225 ), ( 100, 202, 178, 171, 102, 243, 43, 238, 132, 200, 126,
107, 61, 252, 150, 5, 18, 51, 161, 248, 230, 166, 207, 48, 138, 132, 221, 248, 2
28, 42, 126, 117, 253, 85, 133, 165, 56, 125, 224, 92, 223, 129, 108, 179, 197, 2
24, 163, 241, 196, 207, 53, 73, 183, 98, 172, 7, 65, 90, 35, 15, 59, 219, 95, 41,
93, 52, 51, 19, 205, 78, 66, 6, 218, 47, 109, 228, 45, 134, 141, 173, 225, 93, 1
94, 5, 164, 100, 199, 10, 118, 30, 163, 110, 152, 247, 212, 214, 146, 104, 164, 7
, 117, 87, 129, 159, 235, 129, 233, 224, 72, 250, 167, 128, 208, 220, 150, 195, 4
0, 168, 73, 21, 98, 185, 128, 27, 30, 74, 251, 134, 186, 40, 109, 138, 181, 202,
137, 86, 72, 230, 117, 18, 123, 180, 10, 23, 122, 47, 241, 16, 193, 71, 189, 203,
212, 68, 241, 143, 191, 62, 228, 255, 82, 12, 178, 134, 161, 178, 251, 119, 66,
209, 33, 32, 17, 124, 47, 96, 191, 20, 10, 4, 82, 36, 65, 36, 164, 113, 230, 30,
33, 176, 75, 54, 130, 25, 171, 227, 1, 41, 182, 164, 87, 232, 210, 74, 161, 84, 2
4, 187, 106, 106, 52, 153, 247, 152, 252, 245, 182, 253, 213, 203, 45, 168, 193,
190, 42, 105, 201, 1, 211, 224, 153, 189, 89, 187, 59, 238, 53, 95, 105, 103, 24,
2, 62, 250, 57, 70, 68, 110, 45, 45, 226, 254, 194, 103, 186, 140, 209, 1, 37, 8
7, 15, 200, 26, 22, 216, 251, 175, 137, 33, 228, 132, 152, 86, 179, 106, 68, 69,
49, 202, 146, 67, 168, 130, 223, 55, 234, 177, 88, 146, 195, 100, 176, 210, 177,
61, 161, 1, 58, 5, 191, 30, 49, 183, 83, 125, 13, 170, 211, 93, 110, 219, 93, 246
, 120, 169, 97, 206, 86, 241, 208, 195, 199, 53, 31, 133, 253, 181, 247, 192, 209
, 19, 159, 175, 39, 247, 154, 12, 225, 43, 231, 198, 83, 125, 33, 175, 79, 242, 2
24, 150, 251, 213, 125, 155, 230, 61, 109, 32, 239, 227, 40, 8, 133, 186, 118, 23
2, 157, 218, 41, 24, 168, 32, 205, 172, 216, 36, 94, 179, 37, 212, 13, 35, 171, 1
6, 171, 39, 69, 241, 165, 62, 91, 136, 4, 227, 168, 53, 6, 226, 251, 58, 251, 38,
234, 245, 255, 168, 119, 169, 47, 179, 223, 28, 32, 17, 185, 49, 44, 196, 122, 8
9, 116, 157, 223, 252, 204, 40, 119, 89, 8, 7, 178, 235, 52, 12, 51, 217, 60, 104
, 100, 59, 111, 14, 244, 49, 223, 225, 245, 187, 61, 242, 174, 29, 145, 172, 177,
131, 188, 157, 76, 104, 244, 94, 39, 96, 246, 131, 77, 255, 70, 211, 201, 166, 1
05, 68, 129, 4, 80, 20, 89, 58, 222, 159, 208, 82, 255, 245, 121, 210, 149, 202,
105, 212, 42, 35, 188, 3, 121, 23, 139, 50, 24, 224, 20, 140, 119, 55, 118, 53, 1
55, 19, 70, 138, 178, 202, 153, 78, 44, 90, 109, 32, 253, 64, 130, 238, 243, 87,
12, 204, 63, 116, 163, 82, 25, 14, 31, 197, 193, 203, 235, 79, 86, 118, 118, 88,
190, 26, 55, 179, 156, 153, 89, 227, 221 )
14155 #crc_codes::test_crc_object Nb_test = 1000000 k = 552 nb_undetected=1, time=11:5
4
14156 #crc_codes::test_crc_object done
14157 #interface.coding.theory::worker done
14158
14159
14160 error_polynomial_bravo_552:
14161 ▷ $(ORBITER) -v 3 \
14162 ▷ -define F -finite_field -q 256 -end \
14163 ▷ -define Coeff -vector -dense $(ERROR_POLY_BRAVO_COEFF) -end \
14164 ▷ -define Place -vector -dense $(ERROR_POLY_BRAVO_PLACE) -end \

```

```

14165 ▷ ▷ -with F -do -finite_field.activity \
14166 ▷ ▷ ▷ -assemble_monopoly 771 Coeff Place \
14167 ▷ ▷ -end
14168
14169
14170
14171 crc_test_crc32_10M_3:
14172 ▷ $(ORBITER) -v 3 \
14173 ▷ ▷ -crc_test "crc32" $(TEN_MILLION) 3
14174
14175
14176 #crc_codes::test_crc_object Nb_test = 10000000 k = 3 nb_undetected=0, time=3:31
14177
14178
14179
14180 crc_test_bravo_10M_4:
14181 ▷ $(ORBITER) -v 3 \
14182 ▷ ▷ -crc_test "bravo" $(TEN_MILLION) 4
14183
14184 #crc_codes::test_crc_object Nb_test = 10000000 k = 4 nb_undetected=0, time=0:28
14185
14186 crc_test_crc32_10M_4:
14187 ▷ $(ORBITER) -v 3 \
14188 ▷ ▷ -crc_test "crc32" $(TEN_MILLION) 4
14189
14190 #crc_codes::test_crc_object Nb_test = 10000000 k = 4 nb_undetected=0, time=3:36
14191
14192
14193 crc_test_bravo_10M_5:
14194 ▷ $(ORBITER) -v 3 \
14195 ▷ ▷ -crc_test "bravo" $(TEN_MILLION) 5
14196
14197
14198 #crc_codes::test_crc_object Nb_test = 10000000 k = 5 nb_undetected=0, time=0:30
14199
14200
14201 crc_test_crc32_10M_5:
14202 ▷ $(ORBITER) -v 3 \
14203 ▷ ▷ -crc_test "crc32" $(TEN_MILLION) 5
14204
14205 #crc_codes::test_crc_object Nb_test = 10000000 k = 5 nb_undetected=0, time=3:41
14206
14207
14208
14209
14210 crc_test_bravo_10M_6:
14211 ▷ $(ORBITER) -v 3 \
14212 ▷ ▷ -crc_test "bravo" $(TEN_MILLION) 6
14213
14214
14215 #crc_codes::test_crc_object Nb_test = 10000000 k = 6 nb_undetected=0, time=0:30
14216
14217
14218 crc_test_crc32_10M_6:
14219 ▷ $(ORBITER) -v 3 \
14220 ▷ ▷ -crc_test "crc32" $(TEN_MILLION) 6
14221
14222 #crc_codes::test_crc_object Nb_test = 10000000 k = 6 nb_undetected=0, time=3:45
14223

```

```

14224
14225
14226 crc_test_bravo_10M_7:
14227 ▷ $(ORBITER) -v 3 \
14228 ▷ ▷ -crc_test "bravo" $(TEN_MILLION) 7
14229
14230
14231 #crc_codes::test_crc_object Nb_test = 10000000 k = 7 nb_undetected=0, time=0:31
14232
14233
14234 crc_test_crc32_10M_7:
14235 ▷ $(ORBITER) -v 3 \
14236 ▷ ▷ -crc_test "crc32" $(TEN_MILLION) 7
14237
14238 #crc_codes::test_crc_object Nb_test = 10000000 k = 7 nb_undetected=0, time=3:47
14239
14240
14241 crc_test_bravo_10M_8:
14242 ▷ $(ORBITER) -v 3 \
14243 ▷ ▷ -crc_test "bravo" $(TEN_MILLION) 8
14244
14245
14246 #crc_codes::test_crc_object Nb_test = 10000000 k = 8 nb_undetected=0, time=0:32
14247
14248 crc_test_crc32_10M_8:
14249 ▷ $(ORBITER) -v 3 \
14250 ▷ ▷ -crc_test "crc32" $(TEN_MILLION) 8
14251
14252 #crc_codes::test_crc_object Nb_test = 10000000 k = 8 nb_undetected=0, time=3:50
14253
14254
14255 crc_test_bravo_10_loop:
14256 ▷ $(ORBITER) -v 3 \
14257 ▷ ▷ -loop i 2 770 1 \
14258 ▷ ▷ ▷ -crc_test "bravo" $(TEN_MILLION) %i \
14259 ▷ ▷ -end_loop i
14260
14261
14262 crc_test_crc32_10_loop:
14263 ▷ $(ORBITER) -v 3 \
14264 ▷ ▷ -loop i 2 770 1 \
14265 ▷ ▷ ▷ -crc_test "crc32" $(TEN_MILLION) %i \
14266 ▷ ▷ -end_loop i
14267
14268
14269 ##### 20 Million:
14270
14271
14272
14273 crc_test_crc32_20M_3:
14274 ▷ $(ORBITER) -v 3 \
14275 ▷ ▷ -crc_test "crc32" 20000000 3
14276
14277
14278
14279
14280 # charlie has d = 7, so it can detect any 6 errors at the F256 level
14281
14282

```

```

14283 crc_test_charlie:
14284 ▷ $(ORBITER) -v 3 \
14285 ▷ ▷ -crc_test "charlie" 10000000 7
14286
14287
14288
14289
14290 #####
14291 # Section 10.5: Coding Theory - Reed-Muller codes
14292
14293 SECTION_CODING_THEORY_REED_MULLER_CODES:
14294
14295
14296
14297
14298 test_10_5:
14299 ▷ make RM_3_export
14300 ▷ make RM_3_Hamming_space_diagram
14301 ▷ make RM_3_draw
14302 ▷ make RM_3_split
14303 ▷ make RM_3_distance_draw
14304 ▷ make RM_3_algebraic_normal_form
14305 ▷ make RM_6
14306 ▷ make RM_6_genma_draw
14307 ▷ make RM_6_codewords_matrix_draw
14308 ▷ make RM_6_codewords_draw
14309
14310
14311
14312 RM_3_export:
14313 ▷ $(ORBITER) -v 2 \
14314 ▷ ▷ -define F -finite_field -q 2 -end \
14315 ▷ ▷ -define C -code -field F \
14316 ▷ ▷ ▷ -Reed_Muller 3 \
14317 ▷ ▷ -end \
14318 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14319 ▷ ▷ ▷ -export_magma RM_3.magma \
14320 ▷ ▷ -end \
14321 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14322 ▷ ▷ ▷ -export_codewords RM_3_codewords.csv \
14323 ▷ ▷ -end
14324
14325
14326
14327
14328 RM_3_Hamming_space_diagram:
14329 ▷ $(ORBITER) -v 2 \
14330 ▷ ▷ -define F -finite_field -q 2 -end \
14331 ▷ ▷ -define C -code -field F \
14332 ▷ ▷ ▷ -Reed_Muller 3 \
14333 ▷ ▷ -end \
14334 ▷ ▷ -with C -do \
14335 ▷ ▷ -coding_theoretic_activity \
14336 ▷ ▷ ▷ -make_diagram \
14337 ▷ ▷ ▷ -metric_balls 1 \
14338 ▷ ▷ -end
14339
14340 # creates
14341 # RM_3.8.16.tex

```

```

14342 # RM_3_diagram_8_16.csv
14343 # RM_3_diagram_01_8_16.csv
14344 # RM_3_distance_8_16.csv
14345
14346
14347 RM_3_draw:
14348 ▷ $(ORBITER) -v 2 \
14349 ▷ ▷ -draw_matrix \
14350 ▷ ▷ ▷ -input_csv_file RM_3_distance_H_8_16.csv \
14351 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14352 ▷ ▷ ▷ -partition 4 16 16 \
14353 ▷ ▷ -end
14354 ▷ $(ORBITER) -v 2 \
14355 ▷ ▷ -draw_matrix \
14356 ▷ ▷ ▷ -input_csv_file RM_3_char_func_8_16.csv \
14357 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14358 ▷ ▷ ▷ -partition 4 16 16 \
14359 ▷ ▷ -end
14360 ▷ $(ORBITER) -v 2 \
14361 ▷ ▷ -draw_matrix \
14362 ▷ ▷ ▷ -input_csv_file RM_3_idx_8_16.csv \
14363 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14364 ▷ ▷ ▷ -partition 4 16 16 \
14365 ▷ ▷ -end
14366 ▷ convert RM_3_distance_H_8_16_draw.bmp RM_3_distance_H_8_16_draw.png
14367 ▷ convert RM_3_char_func_8_16_draw.bmp RM_3_char_func_8_16_draw.png
14368 ▷ convert RM_3_idx_8_16_draw.bmp RM_3_idx_8_16_draw.png
14369 ▷ $(OPEN) RM_3_distance_H_8_16_draw.bmp
14370
14371
14372 RM_3_split:
14373 ▷ #$(ORBITER) -split_by_values RM_3_distance_8_16.csv
14374 ▷ $(ORBITER) -split_by_values RM_3_distance_H_8_16.csv
14375
14376 RM_3_distance_draw:
14377 ▷ $(ORBITER) -v 2 \
14378 ▷ ▷ -loop L 0 3 1 \
14379 ▷ ▷ ▷ -draw_matrix \
14380 ▷ ▷ ▷ ▷ -input_csv_file RM_3_distance_H_8_16_value%L.csv \
14381 ▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14382 ▷ ▷ ▷ ▷ -partition 5 16 16 \
14383 ▷ ▷ ▷ -end \
14384 ▷ ▷ -end_loop L
14385 ▷ convert RM_3_distance_H_8_16_value0_draw.bmp RM_3_distance_H_8_16_value0_draw.p
ng
14386 ▷ convert RM_3_distance_H_8_16_value1_draw.bmp RM_3_distance_H_8_16_value1_draw.p
ng
14387 ▷ convert RM_3_distance_H_8_16_value2_draw.bmp RM_3_distance_H_8_16_value2_draw.p
ng
14388
14389 RM_3_algebraic_normal_form:
14390 ▷ $(ORBITER) -v 3 \
14391 ▷ ▷ -define F -finite_field -q 2 -end \
14392 ▷ ▷ -loop L 0 3 1 \
14393 ▷ ▷ ▷ -define v%L -vector -field F \
14394 ▷ ▷ ▷ ▷ -file RM_3_distance_8_16_value%L.csv \
14395 ▷ ▷ ▷ -end \
14396 ▷ ▷ ▷ -with F -do -finite_field_activity \
14397 ▷ ▷ ▷ ▷ -algebraic_normal_form 8 v%L \

```



```

14398 ▷ ▷ ▷ -end \
14399 ▷ ▷ -end_loop L
14400
14401
14402 RM_6:
14403 ▷ $(ORBITER) -v 2 \
14404 ▷ ▷ -define F -finite_field -q 2 -end \
14405 ▷ ▷ -define C -code -field F \
14406 ▷ ▷ ▷ -long_code 64 7 \
14407 ▷ ▷ ▷ $(RM_6_GENERATOR_1) \
14408 ▷ ▷ ▷ $(RM_6_GENERATOR_2) \
14409 ▷ ▷ ▷ $(RM_6_GENERATOR_3) \
14410 ▷ ▷ ▷ $(RM_6_GENERATOR_4) \
14411 ▷ ▷ ▷ $(RM_6_GENERATOR_5) \
14412 ▷ ▷ ▷ $(RM_6_GENERATOR_6) \
14413 ▷ ▷ ▷ $(RM_6_GENERATOR_7) \
14414 ▷ ▷ -end \
14415 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14416 ▷ ▷ ▷ -export_genma RM6_genma.csv \
14417 ▷ ▷ -end \
14418 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14419 ▷ ▷ ▷ -export_codewords_long RM6_codewords.csv \
14420 ▷ ▷ -end \
14421
14422 RM_6_genma_draw:
14423 ▷ $(ORBITER) -v 2 \
14424 ▷ ▷ -draw_matrix \
14425 ▷ ▷ ▷ -input_csv_file RM6_genma.csv \
14426 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14427 ▷ ▷ ▷ -partition 3 7 64 \
14428 ▷ ▷ -end
14429 ▷ convert RM6_genma_draw.bmp RM6_genma_draw.png
14430 ▷ $(OPEN) RM6_genma_draw.png
14431
14432
14433 RM_6_codewords_matrix_draw:
14434 ▷ $(ORBITER) -v 2 \
14435 ▷ ▷ -draw_matrix \
14436 ▷ ▷ ▷ -input_csv_file RM6_codewords.csv \
14437 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14438 ▷ ▷ ▷ -partition 3 128 64 \
14439 ▷ ▷ -end
14440 ▷ convert RM6_codewords_draw.bmp RM6_codewords_draw.png
14441 ▷ $(OPEN) RM6_codewords_draw.png
14442
14443
14444 RM_6_codewords_draw:
14445 ▷ $(ORBITER) -v 4 \
14446 ▷ ▷ -loop i 0 128 1 \
14447 ▷ ▷ ▷ -csv_file_select_rows RM6_codewords.csv %i \
14448 ▷ ▷ ▷ -reformat RM6_codewords_select_%i.csv RM6_codewords_ref_%i.csv 8 \
14449 ▷ ▷ ▷ -draw_matrix \
14450 ▷ ▷ ▷ ▷ -input_csv_file RM6_codewords_ref_%i.csv \
14451 ▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
14452 ▷ ▷ ▷ ▷ -partition 3 8 8 \
14453 ▷ ▷ ▷ -end \
14454 ▷ ▷ -system "convert RM6_codewords_ref_%i_draw.bmp -frame 8 %i.png" \
14455 ▷ ▷ -end_loop i
14456 ▷ convert 0.png 1.png 2.png 3.png 4.png 5.png \

```

```

14457 ▷ ▷ 6.png 7.png 8.png 9.png 10.png +append a0
14458 ▷ convert 11.png 12.png 13.png 14.png 15.png \
14459 ▷ ▷ 16.png 17.png 18.png 19.png 20.png 21.png +append a1
14460 ▷ convert 22.png 23.png 24.png 25.png 26.png \
14461 ▷ ▷ 27.png 28.png 29.png 30.png 31.png 32.png +append a2
14462 ▷ convert 33.png 34.png 35.png 36.png 37.png \
14463 ▷ ▷ 38.png 39.png 40.png 41.png 42.png 43.png +append a3
14464 ▷ convert 44.png 45.png 46.png 47.png 48.png \
14465 ▷ ▷ 49.png 50.png 51.png 52.png 53.png 54.png +append a4
14466 ▷ convert 55.png 56.png 57.png 58.png 59.png \
14467 ▷ ▷ 60.png 61.png 62.png 63.png 64.png 65.png +append a5
14468 ▷ convert 66.png 67.png 68.png 69.png 70.png \
14469 ▷ ▷ 71.png 72.png 73.png 74.png 75.png 76.png +append a6
14470 ▷ convert 77.png 78.png 79.png 80.png 81.png \
14471 ▷ ▷ 82.png 83.png 84.png 85.png 86.png 87.png +append a7
14472 ▷ convert 88.png 89.png 90.png 91.png 92.png \
14473 ▷ ▷ 93.png 94.png 95.png 96.png 97.png 98.png +append a8
14474 ▷ convert 99.png 100.png 101.png 102.png 103.png \
14475 ▷ ▷ 104.png 105.png 106.png 107.png 108.png 109.png +append a9
14476 ▷ convert 110.png 111.png 112.png 113.png 114.png \
14477 ▷ ▷ 115.png 116.png 117.png 118.png 119.png 120.png +append a10
14478 ▷ convert 121.png 122.png 123.png 124.png 125.png \
14479 ▷ ▷ 126.png 127.png 0.png 0.png 0.png 0.png +append a11
14480 ▷ convert a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 \
14481 ▷ ▷ a11 -append poster_RM_1.6.png
14482
14483
14484
14485
14486
14487 #####
14488 # Section 10.6: Coding Theory - BCH Codes
14489
14490 SECTION_CODING_THEORY_BCH_CODES:
14491
14492
14493 test_10.6:
14494 ▷ make draw_cyclotomic_mod_51_q16
14495 ▷ make draw_cyclotomic_mod_21_q8
14496 ▷ make F_8_BCH_code_d3
14497 ▷ make F_8_BCH_code_d4
14498 ▷ make F_8_BCH_code_d5
14499 ▷ make F_8_BCH_code_d5_minimum_distance
14500 ▷ make F_8_BCH_code_d7
14501 ▷ make F8_BCH_code_n63_d43
14502 ▷ #make F8_BCH_code_n63_d43_minimum_distance
14503 ▷ make F2_BCH_code_n21
14504 ▷ make F7_RS_code_n6
14505 ▷ make F_64_again
14506 ▷ make BCH_15_5
14507 ▷ make draw_mod_31
14508 ▷ make PR127
14509 ▷ make draw_mod_127_power
14510 ▷ make draw_mod_251
14511 ▷ make draw_mod_255_cyclotomic_1
14512 ▷ make draw_mod_255_cyclotomic_3
14513 ▷ make draw_mod_255_cyclotomic_1_and_3
14514 ▷ make draw_mod_63_4_cyclotomic_3_6
14515 ▷ make BCH_F_64

```

```

14516 ▷ make BCH_21_poly_mult_mod_F4
14517 ▷ make BCH_21_poly_division_a
14518 ▷ make BCH_21_poly_division_b
14519 ▷ make BCH_21_poly_division_ab
14520 ▷ make BCH_21_generator_matrix
14521 ▷ #make BCH_21_15_weight_enumerator
14522 ▷ make BCH_21_15_dual
14523 ▷ make BCH_21_6_weight_enumerator
14524 ▷ make BCH_21_6_4_macwilliams
14525 ▷ make BCH_21_15_4_field_reduction
14526 ▷ make BCH_21_poly_division_c
14527 ▷ make F16_roots_5
14528 ▷ make F64_roots_21
14529 ▷ make BCH_F256_roots_771
14530 ▷ make BCH_F256_BCH_code_d16
14531
14532
14533 draw_cyclotomic_mod_51_q16:
14534 ▷ $(ORBITER) -v 2 \
14535 ▷ ▷ -draw_options \
14536 ▷ ▷ ▷ -radius 100 \
14537 ▷ ▷ ▷ -line_width 1.0 -embedded \
14538 ▷ ▷ -end \
14539 ▷ ▷ -draw_mod_n \
14540 ▷ ▷ ▷ -n 51 \
14541 ▷ ▷ ▷ -cyclotomic_sets 16 "1,2" \
14542 ▷ ▷ ▷ -file cyclotomic_51 \
14543 ▷ ▷ -end
14544 ▷ pdflatex cyclotomic_51_draw.tex
14545 ▷ $(OPEN) cyclotomic_51_draw.pdf
14546
14547 #▷ ▷ ▷ -cyclotomic_sets 8 "1,2,4,5,7,10,13" \
14548
14549
14550 draw_cyclotomic_mod_21_q8:
14551 ▷ $(ORBITER) -v 2 \
14552 ▷ ▷ -draw_options \
14553 ▷ ▷ ▷ -radius 100 \
14554 ▷ ▷ ▷ -line_width 1.0 -embedded \
14555 ▷ ▷ -end \
14556 ▷ ▷ -draw_mod_n \
14557 ▷ ▷ ▷ -n 21 \
14558 ▷ ▷ ▷ -file mod_21_cyclotomic \
14559 ▷ ▷ ▷ -cyclotomic_sets 8 "1,2,4,5,7,10,13" \
14560 ▷ ▷ -end
14561 ▷ pdflatex mod_21_cyclotomic_draw.tex
14562 ▷ $(OPEN) mod_21_cyclotomic_draw.pdf
14563
14564
14565 F_8_BCH_code_d3:
14566 ▷ $(ORBITER) -v 3 \
14567 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14568 ▷ ▷ -define C -code -field F \
14569 ▷ ▷ ▷ -BCH 21 3 \
14570 ▷ ▷ -end
14571 ▷ pdflatex BCH_codes_q8_n21_d3.tex
14572 ▷ $(OPEN) BCH_codes_q8_n21_d3.pdf
14573
14574 #generator polynomial is  $X^4 + 4X^3 + 4X^2 + 3X + 4$ 

```

```

14575
14576 F_8_BCH_code.d4:
14577 ▷ $(ORBITER) -v 3 \
14578 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14579 ▷ ▷ -define C -code -field F \
14580 ▷ ▷ ▷ -BCH 21 4 \
14581 ▷ ▷ -end
14582
14583 #generator polynomial is  $X^5 + 6X^4 + 7X^3 + 2X + 3$ 
14584
14585
14586 F_8_BCH_code.d5:
14587 ▷ $(ORBITER) -v 3 \
14588 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14589 ▷ ▷ -define C -code -field F \
14590 ▷ ▷ ▷ -BCH 21 5 \
14591 ▷ ▷ -end
14592 ▷ pdflatex BCH.codes.q8.n21.d5.tex
14593 ▷ $(OPEN) BCH.codes.q8.n21.d5.pdf
14594
14595 #-override_polynomial 11
14596 #generator polynomial is  $X^7 + 3X^6 + 3X^5 + 2X^4 + X^3 + 2X^2 + X + 2$ 
14597
14598 F_8_BCH_code.d5.minimum.distance:
14599 ▷ $(ORBITER) -v 2 \
14600 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14601 ▷ ▷ -define v -vector -format 14 -field F \
14602 ▷ ▷ ▷ -compact $(CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11) \
14603 ▷ ▷ -end \
14604 ▷ ▷ -with F -do \
14605 ▷ ▷ -coding_theoretic_activity \
14606 ▷ ▷ ▷ -minimum_distance v \
14607 ▷ ▷ -end
14608 # important: use the same polynomial as when creating the code.
14609 #
14610 # d=5
14611
14612
14613
14614 F_8_BCH_code.d7:
14615 ▷ $(ORBITER) -v 3 \
14616 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14617 ▷ ▷ -define C -code -field F \
14618 ▷ ▷ ▷ -BCH 21 7 \
14619 ▷ ▷ -end
14620
14621
14622
14623
14624 F8_BCH_code.n63.d43:
14625 ▷ $(ORBITER) -v 3 \
14626 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14627 ▷ ▷ -define C -code -field F \
14628 ▷ ▷ ▷ -BCH 63 43 \
14629 ▷ ▷ -end
14630 ▷ pdflatex BCH.codes.q8.n63.d43.tex
14631 ▷ $(OPEN) BCH.codes.q8.n63.d43.pdf
14632

```

```

14633
14634
14635
14636
14637 F8_BCH_code_n63_d43_minimum_distance:
14638 ▷ $(ORBITER) -v 2 \
14639 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
14640 ▷ ▷ -define v -vector -format 9 -field F \
14641 ▷ ▷ ▷ -compact $(CODE_BCH_F8_N63_K9_D43_GENMA) \
14642 ▷ ▷ -end \
14643 ▷ ▷ -with F -do \
14644 ▷ ▷ -coding_theoretic_activity \
14645 ▷ ▷ ▷ -minimum_distance v \
14646 ▷ ▷ -end
14647
14648
14649 #coding_theory_domain::do_minimum_distance The minimum distance is d = 45, comput
      ed in 0 days, 0 hours, 1 minutes, 32 seconds
14650 #1:32
14651
14652
14653
14654
14655
14656
14657 F2_BCH_code_n21:
14658 ▷ $(ORBITER) -v 3 \
14659 ▷ ▷ -define F -finite_field -q 2 -end \
14660 ▷ ▷ -define C -code -field F \
14661 ▷ ▷ ▷ -BCH 21 3 \
14662 ▷ ▷ -end
14663
14664
14665
14666 F7_RS_code_n6:
14667 ▷ $(ORBITER) -v 3 \
14668 ▷ ▷ -define F -finite_field -q 7 -end \
14669 ▷ ▷ -define C -code -field F \
14670 ▷ ▷ ▷ -Reed_Solomon 6 3 \
14671 ▷ ▷ -end
14672
14673
14674 F_64_again:
14675 ▷ $(ORBITER) -v 3 \
14676 ▷ ▷ -define F -finite_field -q 64 -end \
14677 ▷ ▷ -with F -do \
14678 ▷ ▷ ▷ -finite_field_activity \
14679 ▷ ▷ ▷ -cheat_sheet_GF \
14680 ▷ ▷ -end
14681 ▷ pdflatex GF_64.tex
14682 ▷ $(OPEN) GF_64.pdf
14683
14684
14685
14686 BCH_15_5:
14687 ▷ $(ORBITER) \
14688 ▷ ▷ -define F -finite_field -q 2 -end \
14689 ▷ ▷ -define C -code -field F \
14690 ▷ ▷ ▷ -BCH 15 5 \

```

```

14691 ▷ ▷ -end
14692
14693
14694
14695
14696
14697
14698 draw_mod_31:
14699 ▷ $(ORBITER) -v 2 \
14700 ▷ ▷ -draw_options -embedded -end \
14701 ▷ ▷ -draw_mod_n \
14702 ▷ ▷ ▷ -n 31 \
14703 ▷ ▷ ▷ -file mod_31 \
14704 ▷ ▷ ▷ -draw_mod_n_power_cycle 2 \
14705 ▷ ▷ -end
14706 ▷ pdflatex mod_31_draw.tex
14707 ▷ $(OPEN) mod_31_draw.pdf
14708
14709
14710 PR127:
14711 ▷ $(ORBITER) -v 5 -primitive_root 127
14712
14713
14714 draw_mod_127_power:
14715 ▷ $(ORBITER) -v 2 \
14716 ▷ ▷ -draw_options -scale 0.4 -embedded -end \
14717 ▷ ▷ -draw_mod_n \
14718 ▷ ▷ ▷ -n 127 \
14719 ▷ ▷ ▷ -file mod_127 \
14720 ▷ ▷ ▷ -power_cycle 3 \
14721 ▷ ▷ -end
14722 ▷ pdflatex mod_127_draw.tex
14723 ▷ $(OPEN) mod_127_draw.pdf
14724
14725 draw_mod_251:
14726 ▷ $(ORBITER) -v 2 \
14727 ▷ ▷ -draw_options -nodes_empty -radius 10 -embedded -end \
14728 ▷ ▷ -draw_mod_n \
14729 ▷ ▷ ▷ -n 251 \
14730 ▷ ▷ ▷ -file mod_251 \
14731 ▷ ▷ -end
14732 ▷ pdflatex mod_251_draw.tex
14733 ▷ $(OPEN) mod_251_draw.pdf
14734
14735 #-draw_mod_n.inverse
14736
14737
14738 draw_mod_255_cyclotomic_1:
14739 ▷ $(ORBITER) -v 2 \
14740 ▷ ▷ -draw_options -nodes_empty -radius 10 \
14741 ▷ ▷ ▷ -line_width 0.4 -embedded -end \
14742 ▷ ▷ -draw_mod_n \
14743 ▷ ▷ ▷ -n 255 \
14744 ▷ ▷ ▷ -file mod_255_cyclotomic_1 \
14745 ▷ ▷ ▷ -cyclotomic_sets 2 "1" \
14746 ▷ ▷ -end
14747 ▷ pdflatex mod_255_cyclotomic_1_draw.tex
14748 ▷ $(OPEN) mod_255_cyclotomic_1_draw.pdf
14749

```

```

14750 draw_mod_255_cyclotomic_3:
14751 ▷ $(ORBITER) -v 2 \
14752 ▷ ▷ -draw_options -nodes_empty -radius 10 \
14753 ▷ ▷ ▷ -line_width 0.4 -embedded -end \
14754 ▷ ▷ -draw_mod_n \
14755 ▷ ▷ ▷ -n 255 \
14756 ▷ ▷ ▷ -file mod_255_cyclotomic_3 \
14757 ▷ ▷ ▷ -cyclotomic_sets 2 "3" \
14758 ▷ ▷ -end
14759 ▷ pdflatex mod_255_cyclotomic_3.draw.tex
14760 ▷ $(OPEN) mod_255_cyclotomic_3.draw.pdf
14761
14762 draw_mod_255_cyclotomic_1_and_3:
14763 ▷ $(ORBITER) -v 2 \
14764 ▷ ▷ -draw_options -nodes_empty -radius 10 \
14765 ▷ ▷ ▷ -line_width 0.4 -embedded -end \
14766 ▷ ▷ -draw_mod_n \
14767 ▷ ▷ ▷ -n 255 \
14768 ▷ ▷ ▷ -file mod_255_cyclotomic_1_and_3 \
14769 ▷ ▷ ▷ -cyclotomic_sets 2 "1,3" \
14770 ▷ ▷ -end
14771 ▷ pdflatex mod_255_cyclotomic_1_and_3.draw.tex
14772 ▷ $(OPEN) mod_255_cyclotomic_1_and_3.draw.pdf
14773
14774 draw_mod_63_4_cyclotomic_3_6:
14775 ▷ $(ORBITER) -v 2 \
14776 ▷ ▷ -draw_options -radius 20 \
14777 ▷ ▷ ▷ -line_width 0.1 -embedded -end \
14778 ▷ ▷ -draw_mod_n \
14779 ▷ ▷ ▷ -n 63 \
14780 ▷ ▷ ▷ -file mod_63_4_cyclotomic_3_6 \
14781 ▷ ▷ ▷ -cyclotomic_sets 4 "3,6" \
14782 ▷ ▷ ▷ -cyclotomic_sets_thickness 50 \
14783 ▷ ▷ -end
14784 ▷ pdflatex mod_63_4_cyclotomic_3_6.draw.tex
14785 ▷ $(OPEN) mod_63_4_cyclotomic_3_6.draw.pdf
14786
14787 BCH_F_64:
14788 ▷ $(ORBITER) -v 3 \
14789 ▷ ▷ -define F -finite_field -q 64 -end \
14790 ▷ ▷ -with F -do -finite_field_activity \
14791 ▷ ▷ ▷ -cheat_sheet_GF \
14792 ▷ ▷ -end
14793 ▷ pdflatex GF_64.tex
14794
14795
14796
14797
14798 BCH_21_poly_mult_mod_F4:
14799 ▷ $(ORBITER) -v 2 \
14800 ▷ ▷ -define F -finite_field -q 4 -end \
14801 ▷ ▷ -with F -do \
14802 ▷ ▷ -finite_field_activity \
14803 ▷ ▷ ▷ -polynomial_mult_mod "1,0,2,1" "1,0,3,1" \
14804 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
14805 ▷ ▷ -end
14806
14807 #C(X)=X^{6} + X^{5} + X^{4} + X^{2} + 1
14808

```

```

14809 # poly 1,0,1,0,1,1,1
14810
14811
14812 BCH_21_poly_division.a:
14813 ▷ $(ORBITER) -v 2 \
14814 ▷ ▷ -define F -finite_field -q 4 -end \
14815 ▷ ▷ -with F -do \
14816 ▷ ▷ -finite_field_activity \
14817 ▷ ▷ -polynomial_division \
14818 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
14819 ▷ ▷ ▷ "1,0,2,1" \
14820 ▷ ▷ -end
14821
14822 BCH_21_poly_division.b:
14823 ▷ $(ORBITER) -v 2 \
14824 ▷ ▷ -define F -finite_field -q 4 -end \
14825 ▷ ▷ -with F -do \
14826 ▷ ▷ -finite_field_activity \
14827 ▷ ▷ -polynomial_division \
14828 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
14829 ▷ ▷ ▷ "1,0,3,1" \
14830 ▷ ▷ -end
14831
14832
14833 BCH_21_poly_division.ab:
14834 ▷ $(ORBITER) -v 2 \
14835 ▷ ▷ -define F -finite_field -q 4 -end \
14836 ▷ ▷ -with F -do \
14837 ▷ ▷ -finite_field_activity \
14838 ▷ ▷ -polynomial_division \
14839 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
14840 ▷ ▷ ▷ "1,0,1,0,1,1,1" \
14841 ▷ ▷ -end
14842
14843 BCH_21_generator_matrix:
14844 ▷ $(ORBITER) -v 2 \
14845 ▷ ▷ -define F -finite_field -q 4 -end \
14846 ▷ ▷ -with F -do \
14847 ▷ ▷ -coding_theoretic_activity \
14848 ▷ ▷ ▷ -generator_matrix_cyclic_code \
14849 ▷ ▷ ▷ ▷ 21 "1,0,1,0,1,1,1" \
14850 ▷ ▷ -end
14851
14852
14853
14854
14855 BCH_21_15_weight_enumerator:
14856 ▷ $(ORBITER) -v 2 \
14857 ▷ ▷ -define F -finite_field -q 4 -end \
14858 ▷ ▷ -define v -vector -field F -format 15 \
14859 ▷ ▷ ▷ -dense $(BCH_21_15_GENERATOR_MATRIX) \
14860 ▷ ▷ -end \
14861 ▷ ▷ -define C -code -field F \
14862 ▷ ▷ ▷ -generator_matrix v \
14863 ▷ ▷ -end \
14864 ▷ ▷ -with C -do \
14865 ▷ ▷ -coding_theoretic_activity \
14866 ▷ ▷ ▷ -weight_enumerator \
14867 ▷ ▷ -end

```



```

14868
14869 # too slow!
14870
14871 BCH_21_15.dual:
14872 ▷ $(ORBITER) -v 2 \
14873 ▷ ▷ -define F -finite_field -q 4 -end \
14874 ▷ ▷ -define v -vector -field F -format 15 \
14875 ▷ ▷ ▷ -dense $(BCH_21_15_GENERATOR_MATRIX) -end \
14876 ▷ ▷ -with F -do -finite_field_activity \
14877 ▷ ▷ ▷ -nullspace v \
14878 ▷ ▷ -end
14879
14880
14881 BCH_21_6.weight_enumerator:
14882 ▷ $(ORBITER) -v 2 \
14883 ▷ ▷ -define F -finite_field -q 4 -end \
14884 ▷ ▷ -define v -vector -format 6 -field F \
14885 ▷ ▷ ▷ -dense $(BCH_21_6_GENERATOR_MATRIX) \
14886 ▷ ▷ -end \
14887 ▷ ▷ -define C -code -field F \
14888 ▷ ▷ ▷ -generator_matrix v \
14889 ▷ ▷ -end \
14890 ▷ ▷ -with C -do \
14891 ▷ ▷ -coding_theoretic_activity \
14892 ▷ ▷ ▷ -weight_enumerator \
14893 ▷ ▷ -end
14894
14895 #  $1y^{21} + 63x^8y^{13} + 294x^{12}y^9 + 756x^{14}y^7 + 1890x^{16}y^5 + 1092x^{18}$ 
     $y^3$ 
14896
14897 #( 1, 0, 0, 0, 0, 0, 0, 0, 63, 0, 0, 0, 294, 0, 756, 0, 1890, 0, 1092, 0, 0, 0 )
14898
14899
14900
14901 BCH_21_6_4.macwilliams:
14902 ▷ $(ORBITER) -v 2 \
14903 ▷ ▷ -make_macwilliams_system 21 6 4
14904 ▷ pdflatex MacWilliams_n21_k6_q4.tex
14905 ▷ $(OPEN) MacWilliams_n21_k6_q4.pdf
14906
14907
14908
14909 #ww := [1, 0, 0, 84, 252, 1575, 10080, 58032, 319662, 1411116, 5133744, 15282792,
    37951620, 79336530, 135622080, 190615824, 213273081, 188911548, 125744304, 59721
    732, 17767512, 2580255]
14910
14911
14912 BCH_21_15.4.field_reduction:
14913 ▷ $(ORBITER) -v 2 \
14914 ▷ ▷ -define F -finite_field -q 4 -end \
14915 ▷ ▷ -with F -do \
14916 ▷ ▷ -finite_field_activity \
14917 ▷ ▷ ▷ -field_reduction "BCH_21_15.4" 2 15 21 \
14918 ▷ ▷ ▷ ▷ $(BCH_21_15_GENERATOR_MATRIX) \
14919 ▷ ▷ -end
14920 ▷ $(ORBITER) -v 2 \
14921 ▷ ▷ -draw_matrix \
14922 ▷ ▷ ▷ -input_csv_file BCH_21_15.4.csv \
14923 ▷ ▷ ▷ -box_width 20 -bit_depth 24 \

```

[illegible]

```

14982
14983 #generator polynomial is  $X^{30} + 253X^{29} + 174X^{28} + 109X^{27} + 97X^{26} +$ 
       $144X^{25} + 112X^{24} + 212X^{23} + 192X^{22} + 169X^{21} + 24X^{20} + 150X^{19}$ 
       $+ 110X^{18} + 248X^{17} + 3X^{16} + 193X^{15} + 194X^{14} + 205X^{13} + 9X^{12} +$ 
       $56X^{11} + 95X^{10} + 199X^9 + 108X^8 + 58X^7 + 160X^6 + 148X^5 + 138X$ 
       $^4 + 24X^3 + 210X^2 + 26X + 1$ 
14984
14985
14986
14987
14988
14989
14990
14991
14992 #####
14993 # Section 10.7: Coding Theory - Reed Solomon codes
14994
14995 SECTION.CODING.THEORY.REED.SOLOMON.CODES:
14996
14997
14998 test_10_7:
14999 ▷ make RS_6_3
15000 ▷ make RS_6_3.weight.enumerator
15001 ▷ make RREF_RS_6_4_7.weight.enumerator
15002 ▷ make Code_RS_11
15003 ▷ #make Code_RS_11.weight.enumerator
15004 ▷ make RS_7_3
15005 ▷ make RS_7_3.weight.enumerator
15006 ▷ make RS_7_3.field.reduction
15007 ▷ make RS_7_3.reduced.weight.enumerator
15008
15009
15010
15011 RS_6_3:
15012 ▷ $(ORBITER) -v 2 \
15013 ▷ ▷ -define F -finite.field -q 7 -end \
15014 ▷ ▷ -define C -code -field F -ReedSolomon 6 3 -end \
15015 ▷ ▷ -with C -do -coding_theoretic_activity \
15016 ▷ ▷ ▷ -export_magma RS_6_3.magma \
15017 ▷ ▷ -end
15018 ▷ pdflatex RS_codes_q7_n6_d3.tex
15019 ▷ $(OPEN) RS_codes_q7_n6_d3.pdf
15020
15021
15022 RS_6_3.weight.enumerator:
15023 ▷ $(ORBITER) -v 2 \
15024 ▷ ▷ -define F -finite.field -q 7 -end \
15025 ▷ ▷ -define C -code -field F -ReedSolomon 6 3 -end \
15026 ▷ ▷ -with C -do -coding_theoretic_activity \
15027 ▷ ▷ ▷ -export_magma RS_6_3.magma \
15028 ▷ ▷ -end \
15029 ▷ ▷ -with C -do \
15030 ▷ ▷ -coding_theoretic_activity \
15031 ▷ ▷ ▷ -weight.enumerator \
15032 ▷ ▷ -end
15033
15034 #[6,4,3]_7 code.
15035 #( 1, 0, 0, 120, 360, 972, 948 )
15036

```

```

15037 #1y^6 + 90x^4y^2 + 108x^5y + 144x^6
15038
15039 RREF_RS_6_4_7_weight_enumerator:
15040 ▷ $(ORBITER) -v 2 \
15041 ▷ ▷ -define F -finite_field -q 7 -end \
15042 ▷ ▷ -define v -vector -format 4 -field F \
15043 ▷ ▷ ▷ -compact $(CODE_RS_6_4_7) \
15044 ▷ ▷ -end \
15045 ▷ ▷ -define C -code -field F \
15046 ▷ ▷ ▷ -generator_matrix v \
15047 ▷ ▷ -end \
15048 ▷ ▷ -with C -do \
15049 ▷ ▷ -coding_theoretic_activity \
15050 ▷ ▷ ▷ -weight_enumerator \
15051 ▷ ▷ -end
15052
15053
15054 #1y^6 + 120x^3y^3 + 360x^4y^2 + 972x^5y + 948x^6
15055 #weight enumerator:
15056 #( 1, 0, 0, 120, 360, 972, 948 )
15057
15058
15059
15060
15061
15062 Code_RS_11:
15063 ▷ $(ORBITER) -v 2 \
15064 ▷ ▷ -define F -finite_field -q 11 -end \
15065 ▷ ▷ -define C -code -field F -ReedSolomon 10 3 -end \
15066 ▷ ▷ -with C -do -coding_theoretic_activity \
15067 ▷ ▷ ▷ -export_magma RS_11_3.magma \
15068 ▷ ▷ -end
15069
15070
15071 Code_RS_11_weight_enumerator:
15072 ▷ $(ORBITER) -v 2 \
15073 ▷ ▷ -define F -finite_field -q 11 -end \
15074 ▷ ▷ -define C -code -field F -ReedSolomon 10 3 -end \
15075 ▷ ▷ -with C -do -coding_theoretic_activity \
15076 ▷ ▷ ▷ -export_magma RS_11_3.magma \
15077 ▷ ▷ -end \
15078 ▷ ▷ -with C -do \
15079 ▷ ▷ -coding_theoretic_activity \
15080 ▷ ▷ ▷ -weight_enumerator \
15081 ▷ ▷ -end
15082
15083
15084 #1y^(10) + 1200x^3y^7 + 16800x^4y^6 + 209160x^5y^5 + 1734600x^6y^4 + 991
      8000x^7y^3 + 37189800x^8y^2 + 82644700x^9y + 82644620x^(10)
15085
15086
15087
15088 RS_7_3:
15089 ▷ $(ORBITER) -v 2 \
15090 ▷ ▷ -define F -finite_field -q 8 -end \
15091 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
15092 ▷ ▷ -with C -do -coding_theoretic_activity \
15093 ▷ ▷ ▷ -export_magma RS_7_3.magma \
15094 ▷ ▷ -end

```

```

15095 ▷ pdflatex RS_codes_q8_n7_d3.tex
15096 ▷ $(OPEN) RS_codes_q8_n7_d3.pdf
15097
15098
15099 RS_7_3_weight_enumerator:
15100 ▷ $(ORBITER) -v 2 \
15101 ▷ ▷ -define F -finite_field -q 8 -end \
15102 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
15103 ▷ ▷ -with C -do -coding_theoretic_activity \
15104 ▷ ▷ ▷ -export_magma RS_7_3.magma \
15105 ▷ ▷ -end \
15106 ▷ ▷ -with C -do \
15107 ▷ ▷ -coding_theoretic_activity \
15108 ▷ ▷ ▷ -weight_enumerator \
15109 ▷ ▷ -end
15110 ▷ pdflatex RS_codes_q8_n7_d3.tex
15111 ▷ $(OPEN) RS_codes_q8_n7_d3.pdf
15112
15113
15114
15115 RS_7_3_field_reduction:
15116 ▷ $(ORBITER) -v 2 \
15117 ▷ ▷ -define F -finite_field -q 8 -end \
15118 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
15119 ▷ ▷ -with F -do \
15120 ▷ ▷ -finite_field_activity \
15121 ▷ ▷ -field_reduction "RS_8_red_2" \
15122 ▷ ▷ ▷ 2 5 7 $(CODE_RS_F8_N7_K5_D3_GENMA) \
15123 ▷ ▷ -end
15124 ▷ $(ORBITER) -v 2 \
15125 ▷ ▷ -draw_matrix -input_csv_file RS_8_red_2.csv \
15126 ▷ ▷ -box_width 40 -bit_depth 24 \
15127 ▷ ▷ -partition 4 "3,3,3,3,3" "3,3,3,3,3,3,3" -end
15128 ▷ pdflatex field_reduction_Q8_q2_5_7.tex
15129 ▷ $(OPEN) field_reduction_Q8_q2_5_7.pdf
15130
15131
15132 RS_7_3_reduced_weight_enumerator:
15133 ▷ $(ORBITER) -v 2 \
15134 ▷ ▷ -define F -finite_field -q 2 -end \
15135 ▷ ▷ -define v -vector -format 15 -field F \
15136 ▷ ▷ ▷ -compact $(RS_8_reduced) \
15137 ▷ ▷ -end \
15138 ▷ ▷ -define C -code -field F \
15139 ▷ ▷ ▷ -generator_matrix v \
15140 ▷ ▷ -end \
15141 ▷ ▷ -with C -do \
15142 ▷ ▷ -coding_theoretic_activity \
15143 ▷ ▷ ▷ -weight_enumerator \
15144 ▷ ▷ -end
15145
15146
15147
15148
15149 #####
15150 # Section 10.8: Coding Theory - Twisted tensor product codes
15151
15152 SECTION_CODING_THEORY_TWISTED_TENSOR_PRODUCT_CODES:
15153

```

```

15154
15155 test_10_8:
15156 ▷ ▷ make TTP_A_4
15157
15158
15159 TTP_A_4:
15160 ▷ $(ORBITER) -v 2 \
15161 ▷ ▷ -define F4 -finite_field -q 4 -end \
15162 ▷ ▷ -define F16 -finite_field -q 16 -end \
15163 ▷ ▷ -define C -code -field F4 -ttpA F16 -end \
15164 ▷ ▷ -with C -do -coding_theoretic_activity \
15165 ▷ ▷ ▷ -export_magma TTP_A.q4.magma \
15166 ▷ ▷ -end \
15167 ▷ ▷ -with C -do -coding_theoretic_activity \
15168 ▷ ▷ ▷ -report \
15169 ▷ ▷ -end
15170 ▷ pdflatex code_n18_k9_q4.tex
15171 ▷ $(OPEN) code_n18_k9_q4.pdf
15172
15173
15174
15175 #####
15176 # Section 10.9: Coding Theory - Bounds
15177
15178 SECTION_CODING_THEORY_BOUNDS:
15179
15180
15181 test_10_9:
15182 ▷ make bounds_for_d_given_n6_k4_q7
15183 ▷ make bounds_for_d_given_n15_k6_q2
15184 ▷ make coding_theory_bounds_q2
15185 ▷ make coding_theory_bounds_q8
15186 ▷ make GV_n15_k6_d5
15187 ▷ make bounds_for_d_given_n12_k4_q13
15188 ▷ make GV_n15_k6_d5_weight_enumerator
15189 ▷ make code_n15_k6_d6_a_we
15190 ▷ make code_n15_k6_d6_RREF
15191 ▷ make code_n15_k6_d6_check_RREF
15192
15193
15194 bounds_for_d_given_n6_k4_q7:
15195 ▷ $(ORBITER) -v 2 \
15196 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 6 4 7
15197 ▷
15198 bounds_for_d_given_n15_k6_q2:
15199 ▷ $(ORBITER) -v 2 \
15200 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 15 6 2
15201
15202 #n = 15 k=6 q=2
15203 #d_GV = 5
15204 #d_singleton = 10
15205 #d_hamming = 6
15206 #d_plotkin = 7
15207 #d_griesmer = 6
15208
15209
15210 coding_theory_bounds_q2:
15211 ▷ $(ORBITER) -v 2 -table_of_bounds 12 2
15212 ▷ $(ORBITER) -v 2 \

```

```

15213 ▷ ▷ -csv_file_latex 1 table_of_bounds_n12.q2.csv
15214 ▷ pdflatex table_of_bounds_n12.q2.tex
15215 ▷ $(OPEN) table_of_bounds_n12.q2.pdf
15216
15217 # produces table_of_bounds_n20.q2.csv
15218
15219 coding_theory_bounds_q8:
15220 ▷ $(ORBITER) -v 2 -table_of_bounds 20 8
15221
15222
15223 GV_n15_k6.d5:
15224 ▷ $(ORBITER) -v 2 \
15225 ▷ ▷ -define F -finite_field -q 2 -end \
15226 ▷ ▷ -define C -code -field F \
15227 ▷ ▷ ▷ -Gilbert_Varshamov 15 6 5 \
15228 ▷ ▷ -end
15229
15230
15231 # [15,6] code created
15232
15233
15234
15235 bounds_for_d_given_n12.k4.q13:
15236 ▷ $(ORBITER) -v 2 \
15237 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 12 4 13
15238
15239
15240
15241
15242 GV_n15_k6.d5.weight_enumerator:
15243 ▷ $(ORBITER) -v 2 \
15244 ▷ ▷ -define F -finite_field -q 2 -end \
15245 ▷ ▷ -define v -vector -format 6 -field F \
15246 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
15247 ▷ ▷ -end \
15248 ▷ ▷ -define C -code -field F \
15249 ▷ ▷ ▷ -generator_matrix v \
15250 ▷ ▷ -end \
15251 ▷ ▷ -with C -do \
15252 ▷ ▷ -coding_theoretic_activity \
15253 ▷ ▷ ▷ -weight_enumerator \
15254 ▷ ▷ -end
15255
15256 #1y^{15} + 27x^6y^9 + 24x^8y^7 + 9x^{10}y^5 + 3x^{12}y^3
15257 # surprise: d = 6
15258
15259
15260 code_n15_k6.d6.a.we:
15261 ▷ $(ORBITER) -v 2 \
15262 ▷ ▷ -define F -finite_field -q 2 -end \
15263 ▷ ▷ -define v -vector -format 6 -field F \
15264 ▷ ▷ ▷ -compact $(CODE_15_6_6_A) \
15265 ▷ ▷ -end \
15266 ▷ ▷ -define C -code -field F \
15267 ▷ ▷ ▷ -generator_matrix v \
15268 ▷ ▷ -end \
15269 ▷ ▷ -with C -do \
15270 ▷ ▷ -coding_theoretic_activity \
15271 ▷ ▷ ▷ -weight_enumerator \

```

```

15272 ▷ ▷ -end
15273
15274 #1y^{15} + 27x^6y^9 + 24x^8y^7 + 9x^{10}y^5 + 3x^{12}y^3
15275
15276
15277
15278 # weight enumerator
15279 #1y^{15} + 28x^6y^9 + 21x^8y^7 + 12x^{10}y^5 + 2x^{12}y^3
15280
15281
15282 code_n15_k6_d6_RREF:
15283 ▷ $(ORBITER) -v 2 \
15284 ▷ ▷ -define F -finite_field -q 2 -end \
15285 ▷ ▷ -define v -vector -format 6 -field F \
15286 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
15287 ▷ ▷ -end \
15288 ▷ ▷ -with F -do -finite_field_activity \
15289 ▷ ▷ ▷ -RREF v \
15290 ▷ ▷ -end
15291 ▷ #pdflatex RREF_example_q2_6_15.tex
15292 ▷ #$(OPEN) RREF_example_q2_6_15.pdf
15293
15294 code_n15_k6_d6_check_RREF:
15295 ▷ $(ORBITER) -v 2 \
15296 ▷ ▷ -define F -finite_field -q 2 -end \
15297 ▷ ▷ -define v -vector -format 9 -field F \
15298 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6_CHECK) \
15299 ▷ ▷ -end \
15300 ▷ ▷ -with F -do -finite_field_activity \
15301 ▷ ▷ -RREF v \
15302 ▷ ▷ -end
15303 ▷ #pdflatex RREF_example_q2_9_15.tex
15304 ▷ #$(OPEN) RREF_example_q2_9_15.pdf
15305
15306
15307
15308
15309
15310 #####
15311 # Section 10.10: Coding Theory - Classification
15312
15313 SECTION_CODING_THEORY_CLASSIFICATION:
15314
15315 test_10_10:
15316 ▷ make codes_8_4_4
15317 ▷ #make codes_8_4_4_draw
15318 ▷ #make codes_14_4_9_3
15319 ▷ make codes_15_6_6_2
15320 ▷ make codes_d4
15321 ▷ make codes_24_12_8
15322 ▷ #make codes_24_12_8_draw
15323 ▷ make glynn_arc
15324 ▷ make five_points_in_general
15325 ▷ make codes_q13_12_4
15326
15327
15328 # code classification:
15329
15330 codes_8_4_4:

```



```

15331 ▷ $(ORBITER) -v 6 \
15332 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
15333 ▷ ▷ -define Control -poset_classification_control \
15334 ▷ ▷ ▷ -problem_label codes_8_4_4 \
15335 ▷ ▷ ▷ -draw_options \
15336 ▷ ▷ ▷ ▷ -embedded -radius 250 \
15337 ▷ ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
15338 ▷ ▷ ▷ -end \
15339 ▷ ▷ -end \
15340 ▷ ▷ -define G \
15341 ▷ ▷ -linear_group -PGL 4 2 -end \
15342 ▷ ▷ -with G -do \
15343 ▷ ▷ -group_theoretic_activity \
15344 ▷ ▷ ▷ -linear_codes Control 3 8 \
15345 ▷ ▷ -end
15346 ▷ #pdflatex codes_8_4_4_poset_lvl_8.tex
15347 ▷ #$(OPEN) codes_8_4_4_poset_lvl_8.pdf
15348 ▷ #pdflatex codes_8_4_4_poset.tex
15349 ▷ #$(OPEN) codes_8_4_4_poset.pdf
15350
15351
15352 # ToDo: the previous command does not produce the graph
15353
15354 codes_8_4_4_draw:
15355 ▷ $(ORBITER) -v 3 \
15356 ▷ ▷ -draw_layered_graph \
15357 ▷ ▷ ▷ codes_8_4_4_poset_lvl_8_layered_graph \
15358 ▷ ▷ ▷ -radius 250 -embedded -line_width 1.0 \
15359 ▷ ▷ ▷ -y_stretch 1.0 -scale 0.5 \
15360 ▷ ▷ -end
15361 ▷ pdflatex codes_8_4_4_poset_lvl_8_draw.tex
15362 ▷ $(OPEN) codes_8_4_4_poset_lvl_8_draw.pdf
15363
15364
15365
15366 codes_14_4_9_3:
15367 ▷ $(ORBITER) -v 6 \
15368 ▷ ▷ -define Control -poset_classification_control \
15369 ▷ ▷ ▷ -problem_label codes_14_4_9_3 \
15370 ▷ ▷ ▷ -draw_options \
15371 ▷ ▷ ▷ ▷ -embedded -radius 250 \
15372 ▷ ▷ ▷ -end \
15373 ▷ ▷ -end \
15374 ▷ ▷ -define G \
15375 ▷ ▷ -linear_group -PGL 10 3 -end \
15376 ▷ ▷ -with G -do \
15377 ▷ ▷ -group_theoretic_activity \
15378 ▷ ▷ ▷ -linear_codes Control 9 14 \
15379 ▷ ▷ -end
15380 ▷ #pdflatex codes_14_4_9_3_poset_lvl_13.tex
15381 ▷ #$(OPEN) codes_14_4_9_3_poset_lvl_13.pdf
15382
15383 #very slow
15384
15385
15386 # ToDo: report is not generated
15387
15388 codes_15_6_6_2:
15389 ▷ $(ORBITER) -v 6 \

```

```

15390 ▷ ▷ -define Control -poset_classification_control \
15391 ▷ ▷ ▷ -problem_label codes_15_6_6_2 \
15392 ▷ ▷ ▷ -draw_options \
15393 ▷ ▷ ▷ ▷ -embedded -radius 250 \
15394 ▷ ▷ ▷ -end \
15395 ▷ ▷ -end \
15396 ▷ ▷ -define G \
15397 ▷ ▷ -linear_group -PGL 9 2 -end \
15398 ▷ ▷ -with G -do \
15399 ▷ ▷ -group_theoretic_activity \
15400 ▷ ▷ ▷ -linear_codes Control 6 15 \
15401 ▷ ▷ -end
15402 ▷ #pdflatex codes_15_6_6_2_poset_lvl15.tex
15403 ▷ #$(OPEN) codes_15_6_6_2_poset_lvl15.pdf
15404
15405
15406
15407
15408
15409 codes_d4:
15410 ▷ $(ORBITER) -v 3 \
15411 ▷ ▷ -define Control -poset_classification_control \
15412 ▷ ▷ ▷ -W \
15413 ▷ ▷ ▷ -problem_label codes_r4_d4 \
15414 ▷ ▷ ▷ -draw_options -embedded -end \
15415 ▷ ▷ -end \
15416 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
15417 ▷ ▷ -with G -do \
15418 ▷ ▷ -group_theoretic_activity \
15419 ▷ ▷ ▷ -linear_codes Control 4 100 \
15420 ▷ ▷ -end
15421
15422
15423 codes_24_12_8:
15424 ▷ $(ORBITER) -v 6 \
15425 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
15426 ▷ ▷ -define Control -poset_classification_control \
15427 ▷ ▷ ▷ -problem_label codes_24_12_8 \
15428 ▷ ▷ ▷ -draw_options -embedded -radius 250 \
15429 ▷ ▷ ▷ -line_width 1.0 -spanning_tree -end \
15430 ▷ ▷ -end \
15431 ▷ ▷ -define G \
15432 ▷ ▷ -linear_group -PGL 12 2 -end \
15433 ▷ ▷ -with G -do \
15434 ▷ ▷ -group_theoretic_activity \
15435 ▷ ▷ ▷ -linear_codes Control 8 24 \
15436 ▷ ▷ -end
15437 ▷ #pdflatex codes_24_12_8_poset.tex
15438 ▷ #$(OPEN) codes_24_12_8_poset.pdf
15439
15440 #codes_24_12_8_poset_lvl24_layered_graph
15441
15442 codes_24_12_8_draw:
15443 ▷ $(ORBITER) -v 3 \
15444 ▷ ▷ -draw_layered_graph \
15445 ▷ ▷ ▷ codes_24_12_8_poset_lvl24_layered_graph \
15446 ▷ ▷ ▷ -radius 100 -spanning_tree -embedded \
15447 ▷ ▷ ▷ -line_width 0.5 -x_stretch 1.4 \
15448 ▷ ▷ ▷ -scale 0.25 -nodes_empty \

```

```

15449 ▷ ▷ -end
15450 ▷ pdflatex codes_24_12_8_poset_lvl_24.draw.tex
15451 ▷ $(OPEN) codes_24_12_8_poset_lvl_24.draw.pdf
15452
15453 glynn_arc:
15454 ▷ $(ORBITER) -v 5 \
15455 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
15456 ▷ ▷ -define Control -poset_classification_control \
15457 ▷ ▷ ▷ -problem_label glynn_arc \
15458 ▷ ▷ ▷ -draw_options -embedded -radius 250 \
15459 ▷ ▷ ▷ -line_width 1.0 -spanning_tree -end \
15460 ▷ ▷ -end \
15461 ▷ ▷ -define G \
15462 ▷ ▷ -linear_group -PGL 5 9 -end \
15463 ▷ ▷ -with G -do \
15464 ▷ ▷ -group_theoretic_activity \
15465 ▷ ▷ ▷ -linear_codes Control 6 10 \
15466 ▷ ▷ -end
15467 ▷ #pdflatex glynn_arc_poset.tex
15468 ▷ #$(OPEN) glynn_arc_poset.pdf
15469
15470
15471
15472
15473
15474 five_points_in_general:
15475 ▷ $(ORBITER) -v 5 \
15476 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
15477 ▷ ▷ -define Control -poset_classification_control \
15478 ▷ ▷ ▷ -problem_label five_points_in_general \
15479 ▷ ▷ ▷ -draw_options \
15480 ▷ ▷ ▷ -embedded -radius 250 \
15481 ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
15482 ▷ ▷ ▷ -end \
15483 ▷ ▷ -end \
15484 ▷ ▷ -define G \
15485 ▷ ▷ -linear_group -PGL 4 2 -end \
15486 ▷ ▷ -with G -do \
15487 ▷ ▷ -group_theoretic_activity \
15488 ▷ ▷ -linear_codes Control 4 5 \
15489 ▷ ▷ -end
15490 ▷ #pdflatex five_points_in_general_poset.tex
15491 ▷ #$(OPEN) five_points_in_general_poset.pdf
15492
15493
15494
15495
15496 codes_q13_12_4:
15497 ▷ $(ORBITER) -v 6 \
15498 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
15499 ▷ ▷ -define Control -poset_classification_control \
15500 ▷ ▷ ▷ -problem_label codes_q13 \
15501 ▷ ▷ -end \
15502 ▷ ▷ -define G \
15503 ▷ ▷ -linear_group -PGL 4 13 -end \
15504 ▷ ▷ -with G -do \
15505 ▷ ▷ -group_theoretic_activity \
15506 ▷ ▷ -linear_codes Control 6 12 \
15507 ▷ ▷ -end

```

```

15508 ▷ #pdflatex codes_q13_poset.tex
15509 ▷ #$(OPEN) codes_q13_poset.pdf
15510
15511
15512
15513
15514 #####
15515 # Chapter 11 - Combinatorics
15516 #####
15517
15518
15519
15520 test_11:
15521 ▷ make test_11_1
15522 ▷ make test_11_2
15523 ▷ make test_11_3
15524 ▷ make test_11_4 # slow
15525 ▷ make test_11_5 # slow
15526 ▷ make test_11_6
15527 ▷ make test_11_7
15528 ▷ make test_11_8
15529 ▷ make test_11_9
15530 ▷ make test_11_10
15531
15532
15533
15534 #####
15535 # Section 11.1: Combinatorics
15536
15537
15538 SECTION_COMBINATORICS:
15539 ▷
15540
15541
15542 test_11_1:
15543 ▷ make Sym_4_conj_classes
15544 ▷ make Sym_10_conj_classes
15545 ▷ make Sym_15_conj_classes
15546 ▷ make Char_Sym_4
15547 ▷ make Char_Sym_5
15548 ▷ #make Char_Sym_6
15549 ▷ make all_subsets_6_3
15550 ▷ make random_k_subsets
15551 ▷ make rank_k_subsets_test
15552 ▷ make Walsh_matrix_4
15553 ▷ make Dedekind_10_10
15554 ▷ make Dedekind_30_2
15555 ▷ make Dedekind_100_2
15556 ▷ make elementary_symmetric_functions_4
15557 ▷ make elementary_symmetric_functions_8
15558 ▷ make domino_portrait
15559 ▷ make domino_portrait_input
15560
15561
15562
15563 Sym_4_conj_classes:
15564 ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym_n 4
15565
15566 Sym_10_conj_classes:

```

```

15567 ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym.n 10
15568 ▷ $(OPEN) classes_Sym.10.csv
15569
15570 Sym_15_conj_classes:
15571 ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym.n 15
15572
15573 Char_Sym_4:
15574 ▷ $(ORBITER) -v 2 -character_table_symmetric_group 4
15575
15576 Char_Sym_5:
15577 ▷ $(ORBITER) -v 2 -character_table_symmetric_group 5
15578
15579
15580 # too slow:
15581
15582 Char_Sym_6:
15583 ▷ $(ORBITER) -v 2 -character_table_symmetric_group 6
15584
15585 all_subsets_6_3:
15586 ▷ $(ORBITER) -v 2 -tree_of_all_k_subsets 6 3
15587 ▷ $(ORBITER) -v 20 \
15588 ▷ ▷ -draw_options -embedded \
15589 ▷ ▷ ▷ -nodes -radius 80 \
15590 ▷ ▷ ▷ -xin 5000 -yin 5000 \
15591 ▷ ▷ ▷ -xout 1000000 -yout 500000 \
15592 ▷ ▷ ▷ -scale 0.5 -line_width 1.0 \
15593 ▷ ▷ -end \
15594 ▷ ▷ -tree_draw -file all_k_subsets.n6.k3.tree -end
15595 ▷ pdflatex all_k_subsets.n6.k3.draw.tex
15596 ▷ $(OPEN) all_k_subsets.n6.k3.draw.pdf
15597
15598 random_k_subsets:
15599 ▷ $(ORBITER) -v 4 \
15600 ▷ ▷ -create_random_k_subsets 10 5 20
15601
15602
15603 rank_k_subsets_test:
15604 ▷ $(ORBITER) -v 2 \
15605 ▷ ▷ -define Blocks -vector -format 7 \
15606 ▷ ▷ ▷ -dense "0,1,2,0,3,4,1,3,5,2,4,5,3,6,7,1,6,8,0,6,9" \
15607 ▷ ▷ ▷ -end \
15608 ▷ ▷ -rank_k_subset 10 3 Blocks
15609
15610
15611 Walsh_matrix_4:
15612 ▷ $(ORBITER) -v 3 \
15613 ▷ ▷ -define F -finite_field -q 2 -end \
15614 ▷ ▷ -with F -do -finite_field_activity \
15615 ▷ ▷ ▷ -Walsh_matrix 4 -end
15616 ▷ $(ORBITER) -v 2 -draw_matrix \
15617 ▷ ▷ -input_csv_file Walsh_01.4.csv \
15618 ▷ ▷ -box_width 10 -bit_depth 24 -partition 3 16 16 -end
15619 ▷ #pdflatex GF.2.tex
15620 ▷ #$(OPEN) GF.2.pdf
15621
15622
15623 Dedekind_10_10:
15624 ▷ $(ORBITER) -v 3 -Dedekind_numbers 2 10 2 10
15625 ▷ $(ORBITER) -v 3 -csv_file_latex 1 Dedekind.2.10.2.10.csv

```

```

15626 ▷ pdflatex Dedekind.2.10.2.10.tex
15627 ▷ $(OPEN) Dedekind.2.10.2.10.pdf
15628
15629
15630 Dedekind.30.2:
15631 ▷ $(ORBITER) -v 3 -Dedekind-numbers 2 30 2 2
15632
15633
15634 Dedekind.100.2:
15635 ▷ $(ORBITER) -v 3 -Dedekind-numbers 2 100 2 2
15636
15637
15638
15639 elementary_symmetric_functions.4:
15640 ▷ $(ORBITER) -make_elementary_symmetric_functions 4 4
15641
15642 elementary_symmetric_functions.8:
15643 ▷ $(ORBITER) -make_elementary_symmetric_functions 8 8
15644
15645
15646
15647 domino_portrait:
15648 ▷ cp $(MY_PATH)/examples/users_guide/anton.28x32.? .
15649 ▷ $(ORBITER) -v 3 -domino_portrait 7 4 anton.28x32 -end
15650
15651 domino_portrait_input:
15652 ▷ cp $(MY_PATH)/examples/users_guide/anton.28x32.? .
15653 ▷ $(ORBITER) -v 2 \
15654 ▷ ▷ -define all_one_r -vector -repeat 1 28 -end \
15655 ▷ ▷ -define all_one_c -vector -repeat 1 32 -end \
15656 ▷ ▷ -draw_matrix \
15657 ▷ ▷ ▷ -grayscale \
15658 ▷ ▷ ▷ -invert_colors \
15659 ▷ ▷ ▷ -input_csv_file anton.28x32_m.csv \
15660 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
15661 ▷ ▷ ▷ -partition 3 \
15662 ▷ ▷ ▷ ▷ all_one_c all_one_r \
15663 ▷ ▷ -end
15664 ▷ $(OPEN) anton.28x32_m.draw.bmp
15665
15666
15667
15668 #####
15669 # Section 11.2: Combinatorial Objects
15670
15671 SECTION.COMBINATORIAL.OBJECTS:
15672
15673
15674 test.11.2:
15675 ▷ make Hirschfeld.q4_from_set
15676 ▷ make hyperoval.16_create
15677 ▷ make EC_read
15678 ▷ #make PG.3.5_assume.31_read
15679 ▷ #make LS.AG.2.3_read
15680 ▷ make geo.7.3_read
15681 ▷ #make Desargues.path_lex_least_read
15682 ▷ make geo.pasch_read
15683 ▷ make geo.pasch_given
15684

```

```

15685
15686
15687 Hirschfeld_q4_from_set:
15688 ▷ $(ORBITER) -v 4 \
15689 ▷ ▷ -define H -set -here \
15690 ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS) \
15691 ▷ ▷ -end \
15692 ▷ ▷ -define C -combinatorial_object \
15693 ▷ ▷ ▷ -label Hirschfeld_surface Hirschfeld\_surface \
15694 ▷ ▷ ▷ -set_of_points H \
15695 ▷ ▷ -end
15696
15697
15698
15699 hyperoval_16_create:
15700 ▷ $(ORBITER) -v 2 \
15701 ▷ ▷ -define C -combinatorial_object \
15702 ▷ ▷ ▷ -label hyperoval_q16 hyperoval\_q16 \
15703 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
15704 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
15705 ▷ ▷ -end \
15706
15707
15708 EC_read: elliptic_curve_b1_c3_q11.txt
15709 ▷ $(ORBITER) -v 4 \
15710 ▷ ▷ -define C -combinatorial_object \
15711 ▷ ▷ ▷ -label EC EC \
15712 ▷ ▷ ▷ -file_of_points elliptic_curve_b1_c3_q11.txt \
15713 ▷ ▷ -end
15714
15715
15716
15717 PG_3_5_assume_31_read:
15718 ▷ $(ORBITER) -v 2 \
15719 ▷ ▷ -define C -combinatorial_object \
15720 ▷ ▷ ▷ -label packings_25 packings_25 \
15721 ▷ ▷ ▷ -file_of_packings_through_spread_table \
15722 ▷ ▷ ▷ ▷ H31_packings.csv \
15723 ▷ ▷ ▷ ▷ SPREAD_TABLES_5_REG/spread_25_spreads.csv \
15724 ▷ ▷ ▷ ▷ 5 \
15725 ▷ ▷ -end
15726
15727
15728
15729 LS_AG_2_3_read:
15730 ▷ $(ORBITER) -v 2 \
15731 ▷ ▷ -define C -combinatorial_object \
15732 ▷ ▷ ▷ -label designs designs \
15733 ▷ ▷ ▷ -file_of_designs \
15734 ▷ ▷ ▷ solutions.csv 9 84 3 12 \
15735 ▷ ▷ -end
15736
15737
15738
15739 geo_7_3_read:
15740 ▷ $(ORBITER) -v 10 \
15741 ▷ ▷ -draw_incidence_structure_description \
15742 ▷ ▷ ▷ -width 60 -width_10 6 -end \
15743 ▷ ▷ -define C -combinatorial_object \

```

```

15744 ▷ ▷ ▷ -label geo.7.3 geo\_7\_3 \
15745 ▷ ▷ ▷ -file_of_incidence_geometries \
15746 ▷ ▷ ▷ ▷ 7.3.inc 7 7 21 \
15747 ▷ ▷ -end
15748
15749
15750
15751 Desargues_path_lex_least_read:
15752 ▷ echo $(DESARGUES_PATH_LEX_LEAST) >Desargues_path_lex_least.inc
15753 ▷ $(ORBITER) -v 10 \
15754 ▷ ▷ -draw_incidence_structure_description \
15755 ▷ ▷ ▷ -width 60 -width_10 6 -end \
15756 ▷ ▷ -define C -combinatorial_object \
15757 ▷ ▷ ▷ -label Desargues_path_lex_least Desargues\_path\_lex\_least \
15758 ▷ ▷ ▷ -file_of_incidence_geometries_by_row_ranks \
15759 ▷ ▷ ▷ ▷ Desargues_path_lex_least.inc 10 10 3 \
15760 ▷ ▷ -end
15761
15762
15763
15764
15765 geo_pasch_read:
15766 ▷ $(ORBITER) -v 10 \
15767 ▷ ▷ -define C -combinatorial_object \
15768 ▷ ▷ ▷ -label Pasch Pasch \
15769 ▷ ▷ ▷ -file_of_incidence_geometries \
15770 ▷ ▷ ▷ ▷ pasch.inc 6 4 12 \
15771 ▷ ▷ -end
15772
15773 geo_pasch_given:
15774 ▷ $(ORBITER) -v 10 \
15775 ▷ ▷ -define C -combinatorial_object \
15776 ▷ ▷ ▷ -label Pasch Pasch \
15777 ▷ ▷ ▷ -incidence_geometry \
15778 ▷ ▷ ▷ ▷ "0,1,4,6,8,11,13,14,17,19,22,23" \
15779 ▷ ▷ ▷ ▷ 6 4 12 \
15780 ▷ ▷ -end
15781
15782
15783
15784
15785 #####
15786 # Section 11.3: Combinatorial Linear Spaces
15787
15788 SECTION_COMBINATORIAL_LINEAR_SPACES:
15789
15790
15791 test_11_3:
15792 ▷ make linsp6
15793 ▷ make linsp7
15794 ▷ make linsp30_pt.types
15795 ▷ make linsp30_pt.distribution
15796
15797
15798 linsp6:
15799 ▷ $(ORBITER) -v 4 \
15800 ▷ ▷ -define A -vector -format 1 -dense "15,10,6,3,1" -end \
15801 ▷ ▷ -define D -diophant -label linsp6 \
15802 ▷ ▷ -coefficient_matrix A \

```



```

15803 ▷ ▷ -RHS "mult=1,EQ=15" \
15804 ▷ ▷ -x_min_global 0 \
15805 ▷ ▷ -x_max_global 15 \
15806 ▷ ▷ -end \
15807 ▷ ▷ -with D -do \
15808 ▷ ▷ ▷ -diophant_activity -solve_mckay \
15809 ▷ ▷ -end
15810 ▷
15811 # Found 15 solutions with 22 backtrack steps
15812
15813
15814
15815
15816 linsp7:
15817 ▷ $(ORBITER) -v 4 \
15818 ▷ ▷ -define A -vector -format 1 -dense "21,15,10,6,3,1" -end \
15819 ▷ ▷ -define D -diophant -label linsp7 \
15820 ▷ ▷ -coefficient_matrix A \
15821 ▷ ▷ -RHS "mult=1,EQ=21" \
15822 ▷ ▷ -x_min_global 0 \
15823 ▷ ▷ -x_max_global 21 \
15824 ▷ ▷ -end \
15825 ▷ ▷ -with D -do \
15826 ▷ ▷ ▷ -diophant_activity -solve_mckay \
15827 ▷ ▷ -end
15828 ▷
15829
15830 # 32 solutions in 45 backtrack steps
15831
15832
15833
15834
15835
15836
15837
15838 linsp30_pt.types:
15839 ▷ $(ORBITER) -v 4 \
15840 ▷ ▷ -define A -vector -format 1 -dense "6,4,3" -end \
15841 ▷ ▷ -define D -diophant \
15842 ▷ ▷ ▷ -label linsp30_pt.types \
15843 ▷ ▷ ▷ -coefficient_matrix A \
15844 ▷ ▷ ▷ -RHS "mult=1,EQ=29" -x.bounds "0,1,0,27,0,24" \
15845 ▷ ▷ -end \
15846 ▷ ▷ -with D -do \
15847 ▷ ▷ ▷ -diophant_activity -solve_mckay \
15848 ▷ ▷ -end
15849
15850 linsp30_pt.distribution:
15851 ▷ $(ORBITER) -v 4 \
15852 ▷ ▷ -define A -vector -format 6 -dense \
15853 ▷ ▷ ▷ "1,1,1,1,1,1,0,0,5,2,5,2,1,5,3,7,10,1,10,1,0,10,3,21" \
15854 ▷ ▷ -end \
15855 ▷ ▷ -define D -diophant \
15856 ▷ ▷ ▷ -label linsp30_pt.distribution \
15857 ▷ ▷ ▷ -coefficient_matrix A \
15858 ▷ ▷ ▷ -RHS "mult=1,EQ=30" \
15859 ▷ ▷ ▷ -RHS "mult=1,EQ=7" \
15860 ▷ ▷ ▷ -RHS "mult=1,EQ=135" \
15861 ▷ ▷ ▷ -RHS "mult=1,EQ=96" \

```

```

15862 ▷ ▷ ▷ -RHS "mult=1,LE=351" \
15863 ▷ ▷ ▷ -RHS "mult=1,LE=276" \
15864 ▷ ▷ ▷ -x_min_global 0 -x_max_global 30 \
15865 ▷ ▷ -end \
15866 ▷ ▷ -with D -do \
15867 ▷ ▷ ▷ -diophant_activity -solve_mckay \
15868 ▷ ▷ -end \
15869 ▷ ▷ -with D -do \
15870 ▷ ▷ ▷ -diophant_activity -draw_as_bitmap 20 8 \
15871 ▷ ▷ -end
15872
15873
15874
15875 #####
15876 # Section 11.4: Combinatorial Linear Spaces
15877
15878 test_11_4:
15879 ▷ make geo.pasch
15880 ▷ make geo.petersen
15881 ▷ make geo.7.3
15882 ▷ make geo.7.3.no_square_test
15883 ▷ make geo.7.3.no_square_test.draw
15884 ▷ make geo.7.3.orderly
15885 ▷ make geo.7.3.orderly.draw
15886 ▷ make geo.7.3.orderly.mem.debug
15887 ▷ make geo.8.3
15888 ▷ make geo.9.3
15889 ▷ make geo.10.3
15890 ▷ make geo.10.3.inc.draw
15891 ▷ make geo.10.3.orderly
15892 ▷ make geo.10.3.orderly.mem.debug
15893 ▷ make geo.10.3.tree
15894 ▷ make geo.10.3.tree.path
15895 ▷ #make Desargues.path.lex.least.draw
15896 ▷ #make Desargues.path.can.anc.draw
15897 ▷ make geo.11.3
15898 ▷ make geo.12.3
15899 ▷ make geo.12.3.orderly
15900 ▷ make geo.13.3
15901 ▷ make geo.13.3.orderly
15902 ▷ make geo.14.3
15903 ▷ make geo.14.3.orderly
15904 ▷ #make 15.3.inc
15905 ▷ make geo.15.3.g4
15906 ▷ make geo.17.3.g4.orderly
15907 ▷ make geo.18.3.g4
15908 ▷ make geo.19.3.g4
15909 ▷ make geo.20.3.g4
15910 ▷ make geo.21.3.g4
15911 ▷ make geo.15.4
15912 ▷ make geo.16.4.g4
15913 ▷ make geo.16
15914 ▷ make geo.40.4.g4
15915 ▷ make geo.63.3.g6
15916 ▷ make geo.LSQ6.inc
15917
15918
15919 geo.pasch:
15920 ▷ $(ORBITER) -v 8 \

```

```

15921 ▷ ▷ -define Test_lines -set -loop 1 7 1 -end \
15922 ▷ ▷ -define Geo -geometry_builder \
15923 ▷ ▷ ▷ -V 6 -B 4 -TDO 2 -fuse 1 \
15924 ▷ ▷ ▷ -fname_GEO pasch \
15925 ▷ ▷ ▷ -output_to_inc_file \
15926 ▷ ▷ ▷ -test Test_lines \
15927 ▷ ▷ -end
15928
15929
15930 geo.petersen:
15931 ▷ $(ORBITER) -v 8 \
15932 ▷ ▷ -define Test_lines -set -loop 3 11 1 -end \
15933 ▷ ▷ -define Geo -geometry_builder \
15934 ▷ ▷ ▷ -V 10 -B 15 -TDO 3 -fuse 1 \
15935 ▷ ▷ ▷ -fname_GEO petersen -girth 5 \
15936 ▷ ▷ ▷ -output_to_inc_file \
15937 ▷ ▷ ▷ -search_tree \
15938 ▷ ▷ ▷ -test Test_lines \
15939 ▷ ▷ -end
15940
15941 geo.7.3:
15942 ▷ $(ORBITER) -v 2 \
15943 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
15944 ▷ ▷ -define Geo -geometry_builder \
15945 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
15946 ▷ ▷ ▷ -fuse 1 \
15947 ▷ ▷ ▷ -fname_GEO 7.3 \
15948 ▷ ▷ ▷ -output_to_inc_file \
15949 ▷ ▷ ▷ -test Test_lines \
15950 ▷ ▷ -end
15951
15952 geo.7.3.no_square_test:
15953 ▷ $(ORBITER) -v 2 \
15954 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
15955 ▷ ▷ -define Geo -geometry_builder \
15956 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
15957 ▷ ▷ ▷ -fuse 1 \
15958 ▷ ▷ ▷ -fname_GEO 7.3_nst \
15959 ▷ ▷ ▷ -output_to_inc_file \
15960 ▷ ▷ ▷ -test Test_lines \
15961 ▷ ▷ ▷ -no_square_test \
15962 ▷ ▷ -end
15963
15964 geo.7.3.no_square_test.draw:
15965 ▷ $(ORBITER) -v 10 \
15966 ▷ ▷ -draw_incidence_structure_description \
15967 ▷ ▷ ▷ -width 60 -width_10 6 -end \
15968 ▷ ▷ -define C -combinatorial_object \
15969 ▷ ▷ ▷ -label geo.7.3_nst geo.7.3_nst \
15970 ▷ ▷ ▷ -file_of_incidence_geometries 7.3_nst.inc 7 7 21 \
15971 ▷ ▷ -end \
15972 ▷ ▷ -with C -do \
15973 ▷ ▷ -combinatorial_object_activity \
15974 ▷ ▷ ▷ -draw_incidence_matrices \
15975 ▷ ▷ ▷ 7.3_nst \
15976 ▷ ▷ -end
15977 ▷ pdflatex 7.3_nst_incma.tex
15978 ▷ $(OPEN) 7.3_nst_incma.pdf
15979

```

```

15980
15981
15982 geo_7_3_orderly:
15983 ▷ $(ORBITER) -v 200 \
15984 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
15985 ▷ ▷ -define Geo -geometry_builder \
15986 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
15987 ▷ ▷ ▷ -fuse 1 -fname_GEO 7_3 \
15988 ▷ ▷ ▷ -output_to_inc_file \
15989 ▷ ▷ ▷ -test Test_lines \
15990 ▷ ▷ ▷ -search_tree \
15991 ▷ ▷ ▷ -orderly \
15992 ▷ ▷ -end
15993
15994 geo_7_3_orderly_draw:
15995 ▷ $(ORBITER) -v 20 \
15996 ▷ ▷ -draw_options -embedded -radius 50 \
15997 ▷ ▷ ▷ -xin 10000 -yin 10000 \
15998 ▷ ▷ ▷ -xout 1000000 -yout 1000000 \
15999 ▷ ▷ ▷ -nodes_empty \
16000 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
16001 ▷ ▷ -end \
16002 ▷ ▷ -tree_draw -file 7_3_tree.txt -end
16003 ▷ pdflatex 7_3_tree_draw.tex
16004 ▷ $(OPEN) 7_3_tree_draw.pdf
16005
16006 geo_7_3_orderly_mem_debug:
16007 ▷ $(ORBITER) -v 20 \
16008 ▷ ▷ -memory_debug 2 \
16009 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
16010 ▷ ▷ -define Geo -geometry_builder \
16011 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
16012 ▷ ▷ ▷ -fuse 1 -fname_GEO 7_3 \
16013 ▷ ▷ ▷ -output_to_inc_file \
16014 ▷ ▷ ▷ -test Test_lines \
16015 ▷ ▷ ▷ -search_tree \
16016 ▷ ▷ ▷ -orderly \
16017 ▷ ▷ -end
16018
16019 geo_8_3:
16020 ▷ $(ORBITER) -v 2 \
16021 ▷ ▷ -define Test_lines -set -loop 3 9 1 -end \
16022 ▷ ▷ -define Geo -geometry_builder \
16023 ▷ ▷ ▷ -V 8 -B 8 -TDO 3 \
16024 ▷ ▷ ▷ -fuse 1 -fname_GEO 8_3 \
16025 ▷ ▷ ▷ -output_to_inc_file \
16026 ▷ ▷ ▷ -test Test_lines \
16027 ▷ ▷ -end
16028
16029 #-print_at_line 4
16030 # 1 geo: 0 11 18 29 30 38 44 54
16031 # ago=48
16032
16033
16034
16035
16036
16037 geo_9_3:
16038 ▷ $(ORBITER) -v 2 \

```

```

16039 ▷ ▷ -define Test_lines -set -loop 3 10 1 -end \
16040 ▷ ▷ -define Geo -geometry_builder \
16041 ▷ ▷ ▷ -V 9 -B 9 -TDO 3 \
16042 ▷ ▷ ▷ -fuse 1 -fname_GEO 9_3 \
16043 ▷ ▷ ▷ -output_to_inc_file \
16044 ▷ ▷ ▷ -test Test_lines \
16045 ▷ ▷ -end
16046
16047
16048 geo_10_3:
16049 ▷ $(ORBITER) -v 2 \
16050 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
16051 ▷ ▷ -define Geo -geometry_builder \
16052 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
16053 ▷ ▷ ▷ -fname_GEO 10_3 \
16054 ▷ ▷ ▷ -output_to_inc_file \
16055 ▷ ▷ ▷ -output_to_sage_file \
16056 ▷ ▷ ▷ -output_to_blocks_file \
16057 ▷ ▷ ▷ -test Test_lines \
16058 ▷ ▷ -end
16059
16060
16061
16062 # 10 geos
16063 # 8/26/2021: 0 sec on Mac
16064
16065
16066 geo_10_3.inc.draw:
16067 ▷ $(ORBITER) -v 10 \
16068 ▷ ▷ -draw_incidence_structure_description \
16069 ▷ ▷ ▷ -width 60 -width_10 6 -end \
16070 ▷ ▷ -define C -combinatorial_object \
16071 ▷ ▷ ▷ -label geo_10_3 geo\_10\_3 \
16072 ▷ ▷ ▷ -file_of_incidence_geometries \
16073 ▷ ▷ ▷ 10_3.inc 10 10 30 \
16074 ▷ ▷ -end \
16075 ▷ ▷ -with C -do \
16076 ▷ ▷ -combinatorial_object_activity \
16077 ▷ ▷ ▷ -draw_incidence_matrices \
16078 ▷ ▷ ▷ 10_3.inc \
16079 ▷ ▷ -end
16080 ▷ pdflatex 10_3.inc.incma.tex
16081 ▷ $(OPEN) 10_3.inc.incma.pdf
16082
16083
16084 geo_10_3.orderly:
16085 ▷ $(ORBITER) -v 20 \
16086 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
16087 ▷ ▷ -define Geo -geometry_builder \
16088 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
16089 ▷ ▷ ▷ -fname_GEO 10_3 \
16090 ▷ ▷ ▷ -output_to_inc_file \
16091 ▷ ▷ ▷ -test Test_lines \
16092 ▷ ▷ ▷ -orderly \
16093 ▷ ▷ -end
16094
16095 geo_10_3.orderly_mem.debug:
16096 ▷ $(ORBITER) -v 2 \
16097 ▷ ▷ -memory_debug 2 \

```

```

16098 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
16099 ▷ ▷ -define Geo -geometry_builder \
16100 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
16101 ▷ ▷ ▷ -fname_GEO 10_3 \
16102 ▷ ▷ ▷ -output_to_inc_file \
16103 ▷ ▷ ▷ -test Test_lines \
16104 ▷ ▷ ▷ -orderly \
16105 ▷ ▷ -end
16106
16107
16108 geo_10_3_tree:
16109 ▷ $(ORBITER) -v 20 \
16110 ▷ ▷ -define Test_lines -set -loop 0 11 1 -end \
16111 ▷ ▷ -define GEO -geometry_builder \
16112 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
16113 ▷ ▷ ▷ -fname_GEO 10_3 \
16114 ▷ ▷ ▷ -output_to_inc_file \
16115 ▷ ▷ ▷ -search_tree \
16116 ▷ ▷ ▷ -test Test_lines \
16117 ▷ ▷ -end
16118 ▷ $(ORBITER) -v 20 \
16119 ▷ ▷ -draw_options -embedded -radius 40 \
16120 ▷ ▷ ▷ -paperheight 220 \
16121 ▷ ▷ ▷ -paperwidth 330 \
16122 ▷ ▷ ▷ -xin 10000 -yin 10000 \
16123 ▷ ▷ ▷ -xout 1000000 -yout 500000 \
16124 ▷ ▷ ▷ -scale 2 -line_width 0.3 \
16125 ▷ ▷ ▷ -nodes_empty \
16126 ▷ ▷ -end \
16127 ▷ ▷ -tree_draw \
16128 ▷ ▷ ▷ -file 10_3_tree.txt \
16129 ▷ ▷ -end
16130 ▷ pdflatex 10_3_tree_draw.tex
16131 ▷ $(OPEN) 10_3_tree_draw.pdf
16132
16133
16134
16135
16136 geo_10_3_tree_path:
16137 ▷ $(ORBITER) -v 20 \
16138 ▷ ▷ -define Test_lines -set -loop 0 11 1 -end \
16139 ▷ ▷ -define GEO -geometry_builder \
16140 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
16141 ▷ ▷ ▷ -fname_GEO 10_3 \
16142 ▷ ▷ ▷ -output_to_inc_file \
16143 ▷ ▷ ▷ -search_tree \
16144 ▷ ▷ ▷ -test Test_lines \
16145 ▷ ▷ -end
16146 ▷ $(ORBITER) -v 20 \
16147 ▷ ▷ -draw_options -embedded -radius 20 \
16148 ▷ ▷ ▷ -paperheight 220 \
16149 ▷ ▷ ▷ -paperwidth 330 \
16150 ▷ ▷ ▷ -xin 10000 -yin 10000 \
16151 ▷ ▷ ▷ -xout 1000000 -yout 500000 \
16152 ▷ ▷ ▷ -scale 2 -line_width 0.3 \
16153 ▷ ▷ -end \
16154 ▷ ▷ -tree_draw \
16155 ▷ ▷ ▷ -restrict 2 \
16156 ▷ ▷ ▷ -file 10_3_tree.txt \

```

```

16157 ▷ ▷ ▷ -select_path "0,0,15,26,46,56,72,80,93,106,119" \
16158 ▷ ▷ -end
16159 ▷ pdflatex 10_3_tree.draw.tex
16160 ▷ $(OPEN) 10_3_tree.draw.pdf
16161
16162 #▷ ▷ ▷ -nodes_empty \
16163 #-sideways
16164
16165
16166 # ToDo:
16167 #Desargues_path_lex_least.inc is missing
16168
16169 Desargues_path_lex_least.draw:
16170 ▷ echo $(DESARGUES_PATH_LEX_LEAST) >Desargues_path_lex_least.inc
16171 ▷ $(ORBITER) -v 10 \
16172 ▷ ▷ -draw_incidence_structure_description \
16173 ▷ ▷ ▷ -width 60 -width_10 6 -end \
16174 ▷ ▷ -define C -combinatorial_object \
16175 ▷ ▷ ▷ -label geo.10.3 geo\_10\_3 \
16176 ▷ ▷ ▷ -file_of_incidence_geometries_by_row_ranks \
16177 ▷ ▷ ▷ Desargues_path_lex_least.inc 10 10 3 \
16178 ▷ ▷ -end \
16179 ▷ ▷ -with C -do \
16180 ▷ ▷ -combinatorial_object_activity \
16181 ▷ ▷ ▷ -draw_incidence_matrices \
16182 ▷ ▷ ▷ Desargues_path_lex_least \
16183 ▷ ▷ -end
16184 ▷ pdflatex Desargues_path_lex_least.incma.tex
16185 ▷ $(OPEN) Desargues_path_lex_least.incma.pdf
16186
16187
16188 Desargues_path_can_anc.draw:
16189 ▷ echo $(DESARGUES_PATH_CANONICAL_ANCESTOR) >Desargues_path_can_anc.inc
16190 ▷ $(ORBITER) -v 10 \
16191 ▷ ▷ -draw_incidence_structure_description \
16192 ▷ ▷ ▷ -width 60 -width_10 6 -end \
16193 ▷ ▷ -define C -combinatorial_object \
16194 ▷ ▷ ▷ -label geo.10.3 geo\_10\_3 \
16195 ▷ ▷ ▷ -file_of_incidence_geometries_by_row_ranks Desargues_path_can_anc.inc 10 10
16196 ▷ ▷ -end \
16197 ▷ ▷ -with C -do \
16198 ▷ ▷ -combinatorial_object_activity \
16199 ▷ ▷ ▷ -draw_incidence_matrices \
16200 ▷ ▷ ▷ Desargues_path_can_anc \
16201 ▷ ▷ -end
16202 ▷ pdflatex Desargues_path_can_anc.incma.tex
16203 ▷ $(OPEN) Desargues_path_can_anc.incma.pdf
16204
16205
16206
16207 geo.11.3:
16208 ▷ $(ORBITER) -v 2 \
16209 ▷ ▷ -define Test_lines -set -loop 4 12 1 -end \
16210 ▷ ▷ -define Geo -geometry_builder \
16211 ▷ ▷ ▷ -V 11 -B 11 -TDO 3 \
16212 ▷ ▷ ▷ -fuse 1 -fname_GEO 11.3 \
16213 ▷ ▷ ▷ -output_to_inc_file \
16214 ▷ ▷ ▷ -test Test_lines \

```

```

16215 ▷ ▷ -end
16216
16217 # 31 geos
16218 # 8/26/2021: 0 sec on Mac
16219
16220 geo.12.3:
16221 ▷ $(ORBITER) -v 2 \
16222 ▷ ▷ -define Test_lines -set -loop 4 13 1 -end \
16223 ▷ ▷ -define Geo -geometry_builder \
16224 ▷ ▷ ▷ -V 12 -B 12 -TDO 3 \
16225 ▷ ▷ ▷ -fuse 1 -fname_GEO 12.3 \
16226 ▷ ▷ ▷ -output_to_inc_file \
16227 ▷ ▷ ▷ -test Test_lines \
16228 ▷ ▷ ▷ -output_to_sage_file \
16229 ▷ ▷ -end
16230
16231 # 229 geos
16232 #User time: 0.45 of a second, dt=45 tps = 100
16233 #nb.calls.to.densenautey=24586
16234
16235
16236 geo.12.3.orderly:
16237 ▷ $(ORBITER) -v 2 \
16238 ▷ ▷ -define Test_lines -set -loop 4 13 1 -end \
16239 ▷ ▷ -define Geo -geometry_builder \
16240 ▷ ▷ ▷ -V 12 -B 12 -TDO 3 \
16241 ▷ ▷ ▷ -fuse 1 -fname_GEO 12.3 \
16242 ▷ ▷ ▷ -output_to_inc_file \
16243 ▷ ▷ ▷ -test Test_lines \
16244 ▷ ▷ ▷ -orderly \
16245 ▷ ▷ -end
16246
16247
16248
16249 geo.13.3:
16250 ▷ $(ORBITER) -v 2 \
16251 ▷ ▷ -define Test_lines -set -loop 4 14 1 -end \
16252 ▷ ▷ -define Geo -geometry_builder \
16253 ▷ ▷ ▷ -V 13 -B 13 -TDO 3 \
16254 ▷ ▷ ▷ -fuse 1 -fname_GEO 13.3 \
16255 ▷ ▷ ▷ -output_to_inc_file \
16256 ▷ ▷ ▷ -test Test_lines \
16257 ▷ ▷ -end
16258
16259 # 2036 geos, 96, 39, 13, 12^4, 8^3, 6^16, 4^30, 3^20, 2^190, 1^1770
16260 #User time: 0:4
16261 #nb.calls.to.densenautey=216777
16262
16263 geo.13.3.orderly:
16264 ▷ $(ORBITER) -v 2 \
16265 ▷ ▷ -define Test_lines -set -loop 4 14 1 -end \
16266 ▷ ▷ -define Geo -geometry_builder \
16267 ▷ ▷ ▷ -V 13 -B 13 -TDO 3 \
16268 ▷ ▷ ▷ -fuse 1 -fname_GEO 13.3 \
16269 ▷ ▷ ▷ -output_to_inc_file \
16270 ▷ ▷ ▷ -test Test_lines \
16271 ▷ ▷ ▷ -orderly \
16272 ▷ ▷ -end
16273

```



```

16274
16275
16276 geo_14_3:
16277 ▷ $(ORBITER) -v 2 \
16278 ▷ ▷ -define Test_lines -set -loop 4 15 1 -end \
16279 ▷ ▷ -define Geo -geometry_builder \
16280 ▷ ▷ ▷ -V 14 -B 14 -TDO 3 \
16281 ▷ ▷ ▷ -fuse 1 -fname_GEO 14_3 \
16282 ▷ ▷ ▷ -output_to_inc_file \
16283 ▷ ▷ ▷ -test Test_lines \
16284 ▷ ▷ -end
16285
16286 # 21399 geos, 56448, 128, 24^2, 16^3, 14^3, 12^7, 8^15, 7, 6^12, 4^91, 3^19, 2^91
    6, 1^20328
16287 #User time: 0:55
16288 #nb.calls_to_densenaute=2089344
16289
16290
16291 geo_14_3_orderly:
16292 ▷ $(ORBITER) -v 2 \
16293 ▷ ▷ -define Test_lines -set -loop 4 15 1 -end \
16294 ▷ ▷ -define Geo -geometry_builder \
16295 ▷ ▷ ▷ -V 14 -B 14 -TDO 3 \
16296 ▷ ▷ ▷ -fuse 1 -fname_GEO 14_3 \
16297 ▷ ▷ ▷ -output_to_inc_file \
16298 ▷ ▷ ▷ -test Test_lines \
16299 ▷ ▷ ▷ -orderly \
16300 ▷ ▷ -end
16301
16302 #User time: 0:50
16303
16304
16305 15_3.inc:
16306 ▷ $(ORBITER) -v 2 \
16307 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
16308 ▷ ▷ -define Geo -geometry_builder \
16309 ▷ ▷ ▷ -V 15 -B 15 -TDO 3 \
16310 ▷ ▷ ▷ -fuse 1 -fname_GEO 15_3 \
16311 ▷ ▷ ▷ -output_to_inc_file \
16312 ▷ ▷ ▷ -test Test_lines \
16313 ▷ ▷ ▷ -special_test_not_orderly \
16314 ▷ ▷ -end
16315
16316 # 245342 geos, 8064, 720, 192^2, 128, 72, 48^6, 32, 30^2, 24^2, 20^2, 18, 16^10,
    15^2, 12^11, 10^3, 8^34, 6^59, 5^5, 4^180, 3^69, 2^3709, 1^241240
16317 ## 8 min 11 sec on Mac
16318 # running out of memory
16319
16320
16321 geo_15_3.g4:
16322 ▷ $(ORBITER) -v 2 \
16323 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
16324 ▷ ▷ -define Geo -geometry_builder \
16325 ▷ ▷ ▷ -V 15 -B 15 -TDO 3 \
16326 ▷ ▷ ▷ -fuse 1 -fname_GEO 15_3.g4 \
16327 ▷ ▷ ▷ -output_to_inc_file \
16328 ▷ ▷ ▷ -girth 4 \
16329 ▷ ▷ ▷ -search_tree \
16330 ▷ ▷ ▷ -special_test_not_orderly \

```

```

16331 ▷ ▷ ▷ -test Test_lines \
16332 ▷ ▷ -end
16333 ▷ $(ORBITER) -v 2 \
16334 ▷ ▷ -draw_options -embedded -radius 50 \
16335 ▷ ▷ ▷ -nodes.empty \
16336 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
16337 ▷ ▷ -tree_draw -file 15_3_g4_tree.txt -end
16338 ▷ pdflatex 15_3_g4_tree.draw.tex
16339 ▷ $(OPEN) 15_3_g4_tree.draw.pdf
16340
16341 # The unique Cremona Richmond configuration with group of order 720
16342 # cubic surface with 15 real lines and 15 real tritangent planes.
16343 #User time: 0 of a second, dt=0 tps = 100
16344 #nb.calls.to.densenautey=23
16345
16346 #-sideways
16347
16348
16349
16350
16351 geo_17_3_g4_orderly:
16352 ▷ $(ORBITER) -v 2 \
16353 ▷ ▷ -memory_debug 2 \
16354 ▷ ▷ -define Test_lines -set -loop 4 18 1 -end \
16355 ▷ ▷ -define Geo -geometry_builder \
16356 ▷ ▷ ▷ -V 17 -B 17 -TDO 3 \
16357 ▷ ▷ ▷ -fuse 1 -fname_GEO 17_3_g4 \
16358 ▷ ▷ ▷ -output_to_inc_file \
16359 ▷ ▷ ▷ -girth 4 \
16360 ▷ ▷ ▷ -test Test_lines \
16361 ▷ ▷ ▷ -orderly \
16362 ▷ ▷ -end
16363
16364 # 1 sol
16365
16366 geo_18_3_g4:
16367 ▷ $(ORBITER) -v 2 \
16368 ▷ ▷ -define Test_lines -set -loop 4 19 1 -end \
16369 ▷ ▷ -define Geo -geometry_builder \
16370 ▷ ▷ ▷ -V 18 -B 18 -TDO 3 \
16371 ▷ ▷ ▷ -fuse 1 -fname_GEO 18_3_g4 \
16372 ▷ ▷ ▷ -output_to_inc_file \
16373 ▷ ▷ ▷ -girth 4 \
16374 ▷ ▷ ▷ -search_tree \
16375 ▷ ▷ ▷ -special_test_not_orderly \
16376 ▷ ▷ ▷ -test Test_lines \
16377 ▷ ▷ -end
16378
16379 # 4 sol, 1 sec
16380
16381
16382 geo_19_3_g4:
16383 ▷ $(ORBITER) -v 2 \
16384 ▷ ▷ -define Test_lines -set -loop 4 20 1 -end \
16385 ▷ ▷ -define Geo -geometry_builder \
16386 ▷ ▷ ▷ -V 19 -B 19 -TDO 3 \
16387 ▷ ▷ ▷ -fuse 1 -fname_GEO 19_3_g4 \
16388 ▷ ▷ ▷ -output_to_inc_file \
16389 ▷ ▷ ▷ -girth 4 \

```

```

16390 ▷ ▷ ▷ -special_test_not_orderly \
16391 ▷ ▷ ▷ -test Test_lines \
16392 ▷ ▷ -end
16393
16394 # 14 sol, 10 sec on Mac
16395
16396 geo.20.3.g4:
16397 ▷ $(ORBITER) -v 2 \
16398 ▷ ▷ -define Test_lines -set -loop 4 21 1 -end \
16399 ▷ ▷ -define Geo -geometry_builder \
16400 ▷ ▷ ▷ -V 20 -B 20 -TDO 3 \
16401 ▷ ▷ ▷ -fuse 1 -fname_GEO 20.3.g4 \
16402 ▷ ▷ ▷ -output_to_inc_file \
16403 ▷ ▷ ▷ -girth 4 \
16404 ▷ ▷ ▷ -special_test_not_orderly \
16405 ▷ ▷ ▷ -test Test_lines \
16406 ▷ ▷ -end
16407
16408 # 162 sol, User time: 2:5 on Mac
16409
16410 geo.21.3.g4:
16411 ▷ $(ORBITER) -v 2 \
16412 ▷ ▷ -define Test_lines -set -loop 4 22 1 -end \
16413 ▷ ▷ -define Geo -geometry_builder \
16414 ▷ ▷ ▷ -V 21 -B 21 -TDO 3 \
16415 ▷ ▷ ▷ -fuse 1 -fname_GEO 21.3.g4 \
16416 ▷ ▷ ▷ -output_to_inc_file \
16417 ▷ ▷ ▷ -girth 4 \
16418 ▷ ▷ ▷ -special_test_not_orderly \
16419 ▷ ▷ ▷ -test Test_lines \
16420 ▷ ▷ -end
16421
16422
16423 geo.15.4:
16424 ▷ $(ORBITER) -v 2 \
16425 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
16426 ▷ ▷ -define Geo -geometry_builder \
16427 ▷ ▷ ▷ -V 15 -B 15 -TDO 4 \
16428 ▷ ▷ ▷ -fuse 1 -fname_GEO 15.4 \
16429 ▷ ▷ ▷ -output_to_inc_file \
16430 ▷ ▷ ▷ -search_tree \
16431 ▷ ▷ ▷ -special_test_not_orderly \
16432 ▷ ▷ ▷ -test Test_lines \
16433 ▷ ▷ -end
16434 ▷ $(ORBITER) -v 2 \
16435 ▷ ▷ -draw_options -embedded -radius 50 \
16436 ▷ ▷ ▷ -nodes_empty \
16437 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
16438 ▷ ▷ -tree_draw -file 15.4_tree.txt -end
16439 ▷ pdflatex 15.4_tree.draw.tex
16440 ▷ $(OPEN) 15.4_tree.draw.pdf
16441
16442 # 4 objects
16443 # ago=360, 30, 24, 15
16444 #User time: 0.15 of a second, dt=15 tps = 100
16445 #nb_calls_to_densenauly=6767
16446
16447
16448

```

```

16449 geo_16.4.g4:
16450 ▷ $(ORBITER) -v 2 \
16451 ▷ ▷ -define Test_lines -set -loop 4 17 1 -end \
16452 ▷ ▷ -define Geo -geometry_builder \
16453 ▷ ▷ ▷ -V 16 -B 16 -TDO 4 \
16454 ▷ ▷ ▷ -fuse 1 -fname.GEO 16.4.g4 \
16455 ▷ ▷ ▷ -output_to_inc_file \
16456 ▷ ▷ ▷ -special_test_not_orderly \
16457 ▷ ▷ ▷ -girth 4 \
16458 ▷ ▷ ▷ -test Test_lines \
16459 ▷ ▷ -end
16460
16461 # none
16462
16463
16464 geo_15.6:
16465 ▷ $(ORBITER) -v 2 \
16466 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
16467 ▷ ▷ -define Geo -geometry_builder \
16468 ▷ ▷ ▷ -V 15 -B 15 -TDO 6 -fuse 1 \
16469 ▷ ▷ ▷ -fname.GEO 15.6 \
16470 ▷ ▷ ▷ -special_test_not_orderly \
16471 ▷ ▷ ▷ -output_to_inc_file \
16472 ▷ ▷ ▷ -test Test_lines \
16473 ▷ ▷ ▷ -output_to_sage_file \
16474 ▷ ▷ -end
16475
16476
16477
16478
16479 geo_16:
16480 ▷ $(ORBITER) -v 2 \
16481 ▷ ▷ -define Test_lines -set -loop 3 17 1 -end \
16482 ▷ ▷ -define Geo -geometry_builder \
16483 ▷ ▷ ▷ -V 16 -B 20 -TDO 5 \
16484 ▷ ▷ ▷ -fuse 1 -fname.GEO geo_16 \
16485 ▷ ▷ ▷ -special_test_not_orderly \
16486 ▷ ▷ ▷ -output_to_inc_file \
16487 ▷ ▷ ▷ -test Test_lines \
16488 ▷ ▷ -end
16489
16490
16491 geo_40.4.g4:
16492 ▷ $(ORBITER) -v 5 \
16493 ▷ ▷ -define Test_lines -set -loop 0 41 1 -end \
16494 ▷ ▷ -define Geo -geometry_builder \
16495 ▷ ▷ ▷ -V 40 -B 40 -TDO 4 \
16496 ▷ ▷ ▷ -fuse 1 \
16497 ▷ ▷ ▷ -fname.GEO 40.4.g4 \
16498 ▷ ▷ ▷ -girth 4 \
16499 ▷ ▷ ▷ -search_tree \
16500 ▷ ▷ ▷ -special_test_not_orderly \
16501 ▷ ▷ ▷ -test Test_lines \
16502 ▷ ▷ ▷ -output_to_sage_file \
16503 ▷ ▷ ▷ -output_to_inc_file \
16504 ▷ ▷ -end
16505 ▷ $(ORBITER) -v 2 \
16506 ▷ ▷ -draw_options -embedded -radius 50 \
16507 ▷ ▷ ▷ -xin 10000 -yin 10000 \

```

```

16508 ▷ ▷ ▷ -xout 1000000 -yout 1000000 \
16509 ▷ ▷ ▷ -nodes.empty \
16510 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
16511 ▷ ▷ -tree_draw -file 40_4_g4_tree.txt -end
16512 ▷ pdflatex 40_4_g4_tree.draw.tex
16513 ▷ $(OPEN) 40_4_g4_tree.draw.pdf
16514
16515
16516 #-special_test_not_orderly is important for speed purposes
16517 # 2 geos, ago=51840^2
16518 # 40_4_g4.inc
16519
16520
16521 geo.63_3_g6:
16522 ▷ $(ORBITER) -v 2 \
16523 ▷ ▷ -define Test_lines -set -loop 4 64 1 -end \
16524 ▷ ▷ -define Geo -geometry_builder \
16525 ▷ ▷ ▷ -V 63 -B 63 -TDO 3 \
16526 ▷ ▷ ▷ -fuse 1 -fname_GEO 63_3_g6 \
16527 ▷ ▷ ▷ -girth 6 \
16528 ▷ ▷ ▷ -test Test_lines \
16529 ▷ ▷ ▷ -special_test_not_orderly \
16530 ▷ ▷ ▷ -output_to_sage_file \
16531 ▷ ▷ ▷ -output_to_inc_file \
16532 ▷ ▷ ▷ -search_tree \
16533 ▷ ▷ -end
16534
16535 # time 0:38 on Mac Oct 15, 2022
16536 # 2 geos, ago: 12096^2
16537 # 63_3_g6.inc
16538 # split Cayley hexagon H(2) and its dual
16539 # group = G_2(2) = PGU(3,3)
16540
16541
16542
16543 geo.LSQ6.inc:
16544 ▷ $(ORBITER) -v 2 \
16545 ▷ ▷ -define Test_lines -set -loop 1 19 1 -end \
16546 ▷ ▷ -define Geo -geometry_builder \
16547 ▷ ▷ ▷ -V 6,6,6 -B 1,1,1,36 -TDO \
16548 ▷ ▷ ▷ "1,0,0,6, 0,1,0,6, 0,0,1,6" \
16549 ▷ ▷ ▷ -fuse 3 \
16550 ▷ ▷ ▷ -fname_GEO LSQ6 \
16551 ▷ ▷ ▷ -output_to_inc_file \
16552 ▷ ▷ ▷ -test Test_lines \
16553 ▷ ▷ -end
16554
16555 #LSQ6.inc
16556
16557
16558
16559
16560
16561 #####
16562 # Section 11.5: Tactical Decompositions
16563
16564 SECTION_TACTICAL_DECOMPOSITIONS:
16565
16566 test_11_5:

```

```

16567 ▷ make max_arc_16_4.start
16568 ▷ make max_arc_16_4.convert_stack_tdo
16569 ▷ make max_arc_16_4.refine
16570 ▷ make max_arc_16_4r.print
16571
16572
16573 max_arc_16_4.start:
16574 ▷ $(ORBITER) -v 4 -maximal_arc_parameters 16 4
16575
16576
16577 max_arc_16_4.convert_stack_tdo:
16578 ▷ $(ORBITER) -v 4 -convert_stack_to_tdo max_arc.q16_r4.stack
16579
16580 max_arc_16_4.refine:
16581 ▷ $(ORBITER) -v 4 -tdo_refinement \
16582 ▷ -input_file max_arc.q16_r4.tdo -dual_is_linear_space -end
16583 ▷
16584 max_arc_16_4r.print:
16585 ▷ $(ORBITER) -v 4 -tdo_print max_arc.q16_r4r.tdo
16586
16587
16588
16589
16590
16591 #####
16592 # Chapter 12 - Graph Theory
16593 #####
16594
16595
16596 test_12:
16597 ▷ make test_12_1
16598 ▷ make test_12_2
16599 ▷ make test_12_3
16600 ▷ make test_12_4
16601
16602
16603 #####
16604 # Section 12.1: Creating Graphs
16605
16606
16607 SECTION.CREATING_GRAPHS:
16608
16609
16610 test_12_1:
16611 ▷ make Cycle_graph_13
16612 ▷ make make_triangle_graph
16613 ▷ make Chain_232
16614 ▷ make Paley_13_graph
16615 ▷ make Paley_13_graph_complement
16616 ▷ make Winnie_Li_graph_q16_index2
16617 ▷ make Sarnak_graph_29_5
16618 ▷ make tritangent_planes_graph
16619 ▷ make trihedral_pair_graph
16620 ▷ make small_graph
16621 ▷ make petersen
16622 ▷ make Johnson_6_2_0
16623 ▷ make Hamming_graph_3
16624 ▷ make Hamming_graph_7
16625 ▷ make Op_6_2_coll_graph

```

```

16626 ▷ make HJ_graph
16627 ▷ make Group_Perm315_Orbital_3.colored_graph
16628 ▷ make HJ_d2_graph
16629 ▷ make Cayley_Z11_1mod3
16630 ▷ make Cayley_Sym4_coxeter
16631 ▷ make Cayley_Sym4_star
16632 ▷ make Queens_graph
16633 ▷
16634
16635
16636 Cycle_graph_13:
16637 ▷ $(ORBITER) -v 2 \
16638 ▷ ▷ -define Gamma -graph \
16639 ▷ ▷ ▷ -cycle 13 \
16640 ▷ ▷ -end
16641
16642
16643
16644
16645 make_triangle_graph:
16646 ▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
16647 ▷ $(ORBITER) -v 6 \
16648 ▷ ▷ -define G -graph \
16649 ▷ ▷ ▷ -load_csv.no.border \
16650 ▷ ▷ ▷ triangle_graph.csv \
16651 ▷ ▷ -end
16652
16653
16654
16655 Chain_232:
16656 ▷ $(ORBITER) -v 2 \
16657 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
16658 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
16659 ▷ ▷ -define Gamma -graph \
16660 ▷ ▷ ▷ -chain_graph P1 P2 \
16661 ▷ ▷ -end
16662
16663
16664
16665
16666 Paley_13_graph:
16667 ▷ $(ORBITER) -v 2 \
16668 ▷ ▷ -define F -finite_field -q 13 -end \
16669 ▷ ▷ -define Gamma -graph -Paley F -end \
16670
16671
16672 Paley_13_graph_complement:
16673 ▷ $(ORBITER) -v 2 \
16674 ▷ ▷ -define F -finite_field -q 13 -end \
16675 ▷ ▷ -define Gamma -graph -Paley F -complement -end \
16676
16677 Winnie_Li_graph_q16_index2:
16678 ▷ $(ORBITER) -v 2 \
16679 ▷ ▷ -define F -finite_field -q 16 -end \
16680 ▷ ▷ -define Gamma -graph -Winnie_Li F 2 -end \
16681
16682
16683 Sarnak_graph_29_5:
16684 ▷ $(ORBITER) -v 2 \

```

```

16685 ▷ ▷ -define Gamma -graph -Sarnak 29 5 -end \
16686
16687
16688 tritangent_planes_graph:
16689 ▷ $(ORBITER) -v 2 \
16690 ▷ ▷ -define Gamma \
16691 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
16692 ▷ ▷ -end
16693
16694
16695 trihedral_pair_graph:
16696 ▷ $(ORBITER) -v 2 \
16697 ▷ ▷ -define Gamma \
16698 ▷ ▷ ▷ -graph -trihedral_pair_disjointness_graph \
16699 ▷ ▷ -end
16700
16701
16702
16703 small_graph:
16704 ▷ $(ORBITER) -v 2 \
16705 ▷ ▷ -define G -graph -edges_as_pairs \
16706 ▷ ▷ ▷ 5 "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
16707 ▷ ▷ -end
16708
16709
16710
16711
16712 petersen:
16713 ▷ $(ORBITER) -v 2 \
16714 ▷ ▷ -define G -graph -Johnson 5 2 0 -end
16715
16716
16717
16718
16719 Johnson_6_2_0:
16720 ▷ $(ORBITER) -v 2 \
16721 ▷ ▷ -define G -graph -Johnson 6 2 0 -end
16722
16723 Hamming_graph_3:
16724 ▷ $(ORBITER) -v 2 \
16725 ▷ ▷ -define G -graph -Hamming 3 2 -end
16726
16727
16728 Hamming_graph_7:
16729 ▷ $(ORBITER) -v 2 \
16730 ▷ ▷ -define G -graph -Hamming 7 2 -end
16731
16732 # needs halljanko315.csv
16733 # from https://www.win.tue.nl/~aeb/drg/graphs/HJ315.html
16734 #There is a unique distance-regular graph Gamma with intersection array {10,8,8,2
; 1,1,4,5}. It was constructed in Cohen (1981), and uniqueness (given the interse
ction array) was proved in Cohen & Tits (1985).
16735
16736
16737
16738 Op_6_2_coll_graph:
16739 ▷ $(ORBITER) -v 2 \
16740 ▷ ▷ -define F -finite.field -q 13 -end \
16741 ▷ ▷ -define O -orthogonal.space 1 6 F -end \

```



```

16742 ▷ ▷ -define v -vector -loop 0 35 1 -end \
16743 ▷ ▷ -define Gamma -graph -coll_orthogonal 0 v -end \
16744 ▷ ▷ -with Gamma -do \
16745 ▷ ▷ ▷ -graph_theoretic_activity -save \
16746 ▷ ▷ -end
16747
16748
16749 #0_1_6_13_coll_v.colored_graph
16750
16751
16752 HJ_graph:
16753 ▷ cp $(MY_PATH)/examples/users_guide/halljanko315.csv .
16754 ▷ $(ORBITER) -v 6 \
16755 ▷ ▷ -define G -graph \
16756 ▷ ▷ ▷ -load_csv_no_border \
16757 ▷ ▷ ▷ halljanko315.csv \
16758 ▷ ▷ -end
16759
16760
16761 Group_Perm315_Orbital_3.colored_graph: halljanko315.gens.csv
16762 ▷ $(ORBITER) -v 2 \
16763 ▷ ▷ -define gens -vector -format 5 -file \
16764 ▷ ▷ halljanko315.gens.csv -end \
16765 ▷ ▷ -define G -permutation_group \
16766 ▷ ▷ ▷ -bsgs halljanko315 "File\_halljanko315" \
16767 ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
16768 ▷ ▷ -end \
16769 ▷ ▷ -define Gamma -graph \
16770 ▷ ▷ ▷ -orbital_graph G 3 \
16771 ▷ ▷ -end \
16772 ▷ ▷ -with Gamma -do \
16773 ▷ ▷ ▷ -graph_theoretic_activity -save \
16774 ▷ ▷ -end
16775
16776 HJ_d2_graph:
16777 ▷ $(ORBITER) -v 6 \
16778 ▷ ▷ -define G -graph \
16779 ▷ ▷ ▷ -load_csv_no_border \
16780 ▷ ▷ ▷ halljanko315.csv \
16781 ▷ ▷ ▷ -distance_2 \
16782 ▷ ▷ -end
16783
16784
16785
16786 Cayley_Z11_1mod3:
16787 ▷ $(ORBITER) -v 2 \
16788 ▷ ▷ -define F -finite_field -q 11 -end \
16789 ▷ ▷ -define S -vector -dense \
16790 ▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
16791 ▷ ▷ -define G -linear_group -AGL 1 F \
16792 ▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
16793 ▷ ▷ -end \
16794 ▷ ▷ -define Gamma -graph \
16795 ▷ ▷ ▷ -Cayley_graph G S \
16796 ▷ ▷ -end
16797
16798
16799 Cayley_Sym4_coxeter:
16800 ▷ $(ORBITER) -v 2 \

```

```

16801 ▷ ▷ -define S -vector -dense "1,0,2,3, 0,2,1,3, 0,1,3,2" -end \
16802 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
16803 ▷ ▷ -end \
16804 ▷ ▷ -define Gamma -graph \
16805 ▷ ▷ ▷ -Cayley_graph G S \
16806 ▷ ▷ -end
16807
16808
16809 Cayley_Sym4_star:
16810 ▷ $(ORBITER) -v 2 \
16811 ▷ ▷ -define S -vector -dense "1,0,2,3, 2,1,0,3, 3,1,2,0" -end \
16812 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
16813 ▷ ▷ -end \
16814 ▷ ▷ -define Gamma -graph \
16815 ▷ ▷ ▷ -Cayley_graph G S \
16816 ▷ ▷ -end
16817
16818 Queens_graph:
16819 ▷ $(ORBITER) -v 2 \
16820 ▷ ▷ -define G -graph -non_attacking_queens_graph 8 -end \
16821
16822
16823 #####
16824 # Section 12.2: Graphs Theoretic Activities
16825
16826
16827 SECTION_GRAPH_THEORETIC_ACTIVITIES:
16828
16829
16830 test_12.2:
16831 ▷ make triangle_graph_properties
16832 ▷ make Cycle_13_draw
16833 ▷ make Cycle_9_eigenvalues
16834 ▷ make Paley_13_draw
16835 ▷ make Paley_13_eigenvalues
16836 ▷ make tritangent_planes_graph_eigenvalues
16837 ▷ make Cayley_Z11_1mod3_eigenvalues_and_draw
16838 ▷ make Cayley_Sym4_coxeter_draw
16839 ▷ make Cayley_Sym5_coxeter_draw
16840 ▷ make Cayley_Sym4_star_eigenvalues_and_draw
16841 ▷ make graph_v5_e7.colored_graph
16842 ▷ make small_graph_draw
16843 ▷ make petersen_draw
16844 ▷ make Johnson_6_2_0_draw
16845 ▷ make Hamming_graph_3_draw
16846 ▷ make Hamming_graph_7_draw
16847 ▷ make Chain_232_properties
16848 ▷ make Chain_232_eigen
16849 ▷ make HJ_properties
16850 ▷ make HJ_d2_properties
16851 ▷ make PGO_5_2.collinearity_graph
16852 ▷ make trihedral_pair_graph_draw
16853
16854
16855 triangle_graph_properties:
16856 ▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
16857 ▷ $(ORBITER) -v 6 \
16858 ▷ ▷ -define G -graph \
16859 ▷ ▷ ▷ -load_csv.no.border \

```

```

16860 ▷ ▷ ▷ triangle_graph.csv \
16861 ▷ ▷ -end \
16862 ▷ ▷ -with G -do \
16863 ▷ ▷ ▷ -graph_theoretic_activity -properties \
16864 ▷ ▷ -end
16865
16866
16867 Cycle_13_draw:
16868 ▷ $(ORBITER) -v 2 \
16869 ▷ ▷ -define Gamma -graph -cycle 13 -end \
16870 ▷ ▷ -with Gamma -do \
16871 ▷ ▷ -graph_theoretic_activity -export_csv -end \
16872 ▷ ▷ -with Gamma -do \
16873 ▷ ▷ -graph_theoretic_activity -export_graphviz -end
16874 ▷ $(ORBITER) -v 2 -draw_matrix \
16875 ▷ ▷ -input_csv_file Cycle_13.csv \
16876 ▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
16877 ▷ dot -Tpng Cycle_13.gv >Cycle_13.png
16878 ▷ #twopi -Tpng Cycle_13.gv >Cycle_13.png
16879 ▷ #$(OPEN) Cycle_13_draw.bmp
16880 ▷ #pdflatex Cycle_13_report.tex
16881 ▷ #$(OPEN) Cycle_13_report.pdf
16882
16883
16884 Cycle_9_eigenvalues:
16885 ▷ $(ORBITER) -v 2 \
16886 ▷ ▷ -define Gamma -graph \
16887 ▷ ▷ ▷ -cycle 9 \
16888 ▷ ▷ -end \
16889 ▷ ▷ -with Gamma -do \
16890 ▷ ▷ -graph_theoretic_activity -eigenvalues -end
16891 ▷ pdflatex Cycle_9_eigenvalues.tex
16892 ▷ #$(OPEN) Cycle_9_eigenvalues.pdf
16893
16894
16895 Paley_13_draw:
16896 ▷ $(ORBITER) -v 2 \
16897 ▷ ▷ -define F -finite_field -q 13 -end \
16898 ▷ ▷ -define Gamma -graph -Paley F -end \
16899 ▷ ▷ -with Gamma -do \
16900 ▷ ▷ -graph_theoretic_activity -export_csv -end \
16901 ▷ ▷ -with Gamma -do \
16902 ▷ ▷ -graph_theoretic_activity -export_graphviz -end
16903 ▷ $(ORBITER) -v 2 -draw_matrix \
16904 ▷ ▷ -input_csv_file Paley_13.csv \
16905 ▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
16906 ▷ dot -Tpng Paley_13.gv >Paley_13.png
16907 ▷ #$(OPEN) Paley_13_draw.bmp
16908
16909
16910 Paley_13_eigenvalues:
16911 ▷ $(ORBITER) -v 2 \
16912 ▷ ▷ -define F -finite_field -q 13 -end \
16913 ▷ ▷ -define Gamma -graph -Paley F -end \
16914 ▷ ▷ -with Gamma -do \
16915 ▷ ▷ -graph_theoretic_activity -eigenvalues -end
16916 ▷ pdflatex Paley_13_eigenvalues.tex
16917 ▷ #$(OPEN) Paley_13_eigenvalues.pdf
16918

```

```

16919
16920 tritangent_planes_graph_eigenvalues:
16921 ▷ $(ORBITER) -v 2 \
16922 ▷ ▷ -define Gamma \
16923 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
16924 ▷ ▷ -end \
16925 ▷ ▷ -with Gamma -do \
16926 ▷ ▷ -graph_theoretic_activity -eigenvalues -end
16927
16928
16929
16930 Cayley_Z11_1mod3_eigenvalues_and_draw:
16931 ▷ $(ORBITER) -v 2 \
16932 ▷ ▷ -draw_options -xin 2000000 \
16933 ▷ ▷ ▷ -yin 2000000 -embedded -radius 20000 -end \
16934 ▷ ▷ -define F -finite_field -q 11 -end \
16935 ▷ ▷ -define S -vector -dense \
16936 ▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
16937 ▷ ▷ -define G -linear_group -AGL 1 F \
16938 ▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
16939 ▷ ▷ -end \
16940 ▷ ▷ -define Gamma -graph \
16941 ▷ ▷ ▷ -Cayley_graph G S \
16942 ▷ ▷ -end \
16943 ▷ ▷ -with Gamma -do \
16944 ▷ ▷ -graph_theoretic_activity -eigenvalues -end \
16945 ▷ ▷ -with Gamma -do \
16946 ▷ ▷ -graph_theoretic_activity -draw -end
16947 ▷ pdflatex Cayley_graph_AGL_1_11_draw.tex
16948 ▷ $(OPEN) Cayley_graph_AGL_1_11_draw.pdf
16949
16950
16951 Cayley_Sym4_coxeter_draw:
16952 ▷ $(ORBITER) -v 2 \
16953 ▷ ▷ -draw_options -xin 2000000 -yin 2000000 \
16954 ▷ ▷ ▷ -radius 20000 -embedded -nodes_empty -end \
16955 ▷ ▷ -define S -vector -dense \
16956 ▷ ▷ ▷ "1,0,2,3, 0,2,1,3, 0,1,3,2" -end \
16957 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
16958 ▷ ▷ -end \
16959 ▷ ▷ -define Gamma -graph \
16960 ▷ ▷ ▷ -Cayley_graph G S \
16961 ▷ ▷ -end \
16962 ▷ ▷ -with Gamma -do \
16963 ▷ ▷ -graph_theoretic_activity -draw -end
16964 ▷ pdflatex Cayley_graph_Sym_4_draw.tex
16965 ▷ $(OPEN) Cayley_graph_Sym_4_draw.pdf
16966
16967 Cayley_Sym5_coxeter_draw:
16968 ▷ $(ORBITER) -v 2 \
16969 ▷ ▷ -draw_options -xin 1000000 -yin 1000000 \
16970 ▷ ▷ ▷ -embedded -radius 10000 -nodes_empty -end \
16971 ▷ ▷ -define S -vector -dense \
16972 ▷ ▷ ▷ "1,0,2,3,4, 0,2,1,3,4, 0,1,3,2,4, 0,1,2,4,3" -end \
16973 ▷ ▷ -define G -permutation_group -symmetric_group 5 \
16974 ▷ ▷ -end \
16975 ▷ ▷ -define Gamma -graph \
16976 ▷ ▷ ▷ -Cayley_graph G S \
16977 ▷ ▷ -end \

```

```

16978 ▷ ▷ -with Gamma -do \
16979 ▷ ▷ -graph_theoretic.activity -draw -end
16980 ▷ pdflatex Cayley_graph_Sym_5_draw.tex
16981 ▷ $(OPEN) Cayley_graph_Sym_5_draw.pdf
16982
16983
16984 Cayley_Sym4_star_eigenvalues_and_draw:
16985 ▷ $(ORBITER) -v 2 \
16986 ▷ ▷ -draw.options -xin 1000000 -yin 1000000 -embedded -end \
16987 ▷ ▷ -define S -vector -dense "1,0,2,3, 2,1,0,3, 3,1,2,0" -end \
16988 ▷ ▷ -define G -permutation.group -symmetric.group 4 \
16989 ▷ ▷ -end \
16990 ▷ ▷ -define Gamma -graph \
16991 ▷ ▷ ▷ -Cayley_graph G S \
16992 ▷ ▷ -end \
16993 ▷ ▷ -with Gamma -do \
16994 ▷ ▷ -graph_theoretic.activity -eigenvalues -end \
16995 ▷ ▷ -with Gamma -do \
16996 ▷ ▷ -graph_theoretic.activity -draw -end
16997 ▷ pdflatex Cayley_graph_Sym_4_draw.tex
16998 ▷ $(OPEN) Cayley_graph_Sym_4_draw.pdf
16999 ▷ pdflatex Cayley_graph_Sym_4_eigenvalues.tex
17000 ▷ $(OPEN) Cayley_graph_Sym_4_eigenvalues.pdf
17001
17002
17003
17004
17005 graph_v5_e7.colored_graph:
17006 ▷ $(ORBITER) -v 2 \
17007 ▷ ▷ -define G -graph -edges.as.pairs 5 \
17008 ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
17009 ▷ ▷ -end \
17010 ▷ ▷ -with G -do \
17011 ▷ ▷ -graph_theoretic.activity -save -end
17012
17013
17014
17015 small_graph_draw:
17016 ▷ $(ORBITER) -v 2 \
17017 ▷ ▷ -define G -graph -edges.as.pairs 5 \
17018 ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
17019 ▷ ▷ -end \
17020 ▷ ▷ -with G -do \
17021 ▷ ▷ -graph_theoretic.activity -export.csv -end \
17022 ▷ ▷ -with G -do \
17023 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
17024 ▷ ▷ -with G -do \
17025 ▷ ▷ -graph_theoretic.activity -save -end
17026 ▷ $(ORBITER) -v 2 -draw.matrix \
17027 ▷ ▷ -input.csv.file graph_v5_e7.csv \
17028 ▷ ▷ -box.width 40 -bit.depth 24 \
17029 ▷ ▷ -partition 4 "1,1,1,1,1" "1,1,1,1,1" -end
17030 ▷ dot -Tpng graph_v5_e7.gv >graph_v5_e7.png
17031
17032 # creates graph_v5_e7.csv
17033 # creates graph_v5_e7.colored_graph
17034
17035
17036 petersen_draw:

```

```

17037 ▷ $(ORBITER) -v 2 \
17038 ▷ ▷ -define G -graph -Johnson 5 2 0 -end \
17039 ▷ ▷ -with G -do \
17040 ▷ ▷ -graph_theoretic.activity -export_csv -end \
17041 ▷ ▷ -with G -do \
17042 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
17043 ▷ ▷ -with G -do \
17044 ▷ ▷ -graph_theoretic.activity -save -end
17045 ▷ $(ORBITER) -v 2 -draw_matrix \
17046 ▷ ▷ -input_csv_file Johnson_5.2.0.csv \
17047 ▷ ▷ -box_width 40 -bit_depth 24 -partition 4 "10" "10" -end
17048 ▷ dot -Tpng Johnson_5.2.0.gv >Johnson_5.2.0.png
17049
17050
17051 Johnson_6.2.0_draw:
17052 ▷ $(ORBITER) -v 2 \
17053 ▷ ▷ -define G -graph -Johnson 6 2 0 -end \
17054 ▷ ▷ -with G -do \
17055 ▷ ▷ -graph_theoretic.activity -export_csv -end \
17056 ▷ ▷ -with G -do \
17057 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
17058 ▷ ▷ -with G -do \
17059 ▷ ▷ -graph_theoretic.activity -save -end
17060 ▷ $(ORBITER) -v 2 -draw_matrix \
17061 ▷ ▷ -input_csv_file Johnson_6.2.0.csv \
17062 ▷ ▷ -box_width 40 -bit_depth 24 -partition 4 "10" "10" -end
17063 ▷ dot -Tpng Johnson_6.2.0.gv >Johnson_6.2.0.png
17064
17065
17066
17067 Hamming_graph_3_draw:
17068 ▷ $(ORBITER) -v 2 \
17069 ▷ ▷ -define G -graph -Hamming 3 2 -end \
17070 ▷ ▷ -with G -do \
17071 ▷ ▷ -graph_theoretic.activity -export_csv -end \
17072 ▷ ▷ -with G -do \
17073 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
17074 ▷ ▷ -with G -do \
17075 ▷ ▷ -graph_theoretic.activity -save -end
17076 ▷ $(ORBITER) -v 2 -draw_matrix \
17077 ▷ ▷ -input_csv_file Hamming_3.2.csv \
17078 ▷ ▷ -box_width 40 -bit_depth 24 \
17079 ▷ ▷ -partition 4 "1,1,1,1,1,1,1" "1,1,1,1,1,1,1" -end
17080 ▷ dot -Tpng Hamming_3.2.gv >Hamming_3.2.png
17081
17082
17083 Hamming_graph_7_draw:
17084 ▷ $(ORBITER) -v 2 \
17085 ▷ ▷ -define G -graph -Hamming 7 2 -end \
17086 ▷ ▷ -with G -do \
17087 ▷ ▷ -graph_theoretic.activity -export_csv -end \
17088 ▷ ▷ -with G -do \
17089 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
17090 ▷ ▷ -with G -do \
17091 ▷ ▷ -graph_theoretic.activity -save -end
17092 ▷ $(ORBITER) -v 2 -draw_matrix \
17093 ▷ ▷ -input_csv_file Hamming_7.2.csv \
17094 ▷ ▷ -box_width 8 -bit_depth 24 -partition 4 128 128 -end
17095 ▷ dot -Tpng Hamming_7.2.gv >Hamming_7.2.png

```

```

17096
17097
17098
17099
17100
17101
17102 Chain_232_properties:
17103 ▷ $(ORBITER) -v 2 \
17104 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
17105 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
17106 ▷ ▷ -define Gamma -graph \
17107 ▷ ▷ ▷ -chain_graph P1 P2 \
17108 ▷ ▷ -end \
17109 ▷ ▷ -with Gamma -do \
17110 ▷ ▷ ▷ -graph_theoretic_activity -export_csv \
17111 ▷ ▷ -end \
17112 ▷ ▷ -with Gamma -do \
17113 ▷ ▷ ▷ -graph_theoretic_activity -properties \
17114 ▷ ▷ -end
17115
17116 Chain_232_eigen:
17117 ▷ $(ORBITER) -v 2 \
17118 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
17119 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
17120 ▷ ▷ -define Gamma -graph \
17121 ▷ ▷ ▷ -chain_graph P1 P2 \
17122 ▷ ▷ -end \
17123 ▷ ▷ -with Gamma -do \
17124 ▷ ▷ ▷ -graph_theoretic_activity \
17125 ▷ ▷ ▷ -eigenvalues \
17126 ▷ ▷ -end
17127 ▷ pdflatex chain_graph_eigenvalues.tex
17128 ▷ $(OPEN) chain_graph_eigenvalues.pdf
17129
17130
17131
17132 # need the file halljanko315.csv
17133
17134 HJ_properties:
17135 ▷ $(ORBITER) -v 6 \
17136 ▷ ▷ -define G -graph \
17137 ▷ ▷ ▷ -load_csv_no_border \
17138 ▷ ▷ ▷ halljanko315.csv \
17139 ▷ ▷ -end \
17140 ▷ ▷ -with G -do \
17141 ▷ ▷ ▷ -graph_theoretic_activity -properties \
17142 ▷ ▷ -end
17143
17144 #Degree type: (10^{315} )
17145
17146
17147
17148 HJ_d2_properties:
17149 ▷ $(ORBITER) -v 6 \
17150 ▷ ▷ -define G -graph \
17151 ▷ ▷ ▷ -load_csv_no_border \
17152 ▷ ▷ ▷ halljanko315.csv \
17153 ▷ ▷ ▷ -distance_2 \
17154 ▷ ▷ -end \

```

```

17155 ▷ ▷ -with G -do \
17156 ▷ ▷ ▷ -graph.theoretic.activity \
17157 ▷ ▷ ▷ -properties \
17158 ▷ ▷ -end
17159
17160
17161 #Degree type: (80^{315} )
17162
17163
17164
17165
17166 PG0.5.2_collinearity_graph: 0.5.2_incidence_matrix.csv
17167 ▷ $(ORBITER) -v 3 \
17168 ▷ ▷ -define Inc -vector -file 0.5.2_incidence_matrix.csv -end \
17169 ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
17170 ▷ ▷ -with Gamma -do \
17171 ▷ ▷ -graph.theoretic.activity \
17172 ▷ ▷ ▷ -properties \
17173 ▷ ▷ -end
17174
17175
17176 trihedral_pair_graph_draw:
17177 ▷ $(ORBITER) -v 2 -define Gamma \
17178 ▷ ▷ -graph -trihedral_pair_disjointness_graph -end \
17179 ▷ ▷ -with Gamma -do \
17180 ▷ ▷ -graph.theoretic.activity -export_csv -end
17181 ▷ $(ORBITER) -v 2 -draw_matrix \
17182 ▷ ▷ -input_csv_file trihedral_pair_disjointness.csv \
17183 ▷ ▷ -box_width 20 -bit_depth 8 -end
17184 ▷ $(OPEN) trihedral_pair_disjointness_draw.bmp
17185
17186
17187
17188
17189 #####
17190 # Section 12.3: Graph Theory: Classification
17191
17192
17193 SECTION_GRAPH_THEORY_CLASSIFICATION:
17194
17195
17196 test_12.3:
17197 ▷ make graph_classify_5
17198 ▷ make tournament_classify_4
17199 ▷ make graph_classify_8_r3
17200 ▷ make Symmetric_4_inversion_graph_recognize
17201 ▷ make Symmetric_5_inversion_graph_recognize
17202
17203
17204
17205 graph_classify_5:
17206 ▷ $(ORBITER) -v 2 \
17207 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17208 ▷ ▷ -define GC -graph_classification \
17209 ▷ ▷ ▷ -n 5 \
17210 ▷ ▷ ▷ -poset_classification_control \
17211 ▷ ▷ ▷ ▷ -problem_label graphs.v5 \
17212 ▷ ▷ ▷ ▷ -depth 10 \
17213 ▷ ▷ ▷ ▷ -draw_options -radius 250 \

```



```

17214 ▷ ▷ ▷ ▷ -embedded \
17215 ▷ ▷ ▷ ▷ -end \
17216 ▷ ▷ ▷ -end \
17217 ▷ ▷ -end \
17218 ▷ ▷ -with GC -do \
17219 ▷ ▷ -graph_classification_activity \
17220 ▷ ▷ ▷ -list_graphs_at_level 5 5 \
17221 ▷ ▷ -end \
17222 ▷ ▷ -with GC -do \
17223 ▷ ▷ -graph_classification_activity \
17224 ▷ ▷ ▷ -draw_options \
17225 ▷ ▷ ▷ ▷ -radius 300 -nodes_empty \
17226 ▷ ▷ ▷ ▷ -line_width 1.5 \
17227 ▷ ▷ ▷ ▷ -scale 0.1 \
17228 ▷ ▷ ▷ -end \
17229 ▷ ▷ ▷ -draw_graphs_at_level 5 \
17230 ▷ ▷ -end \
17231 ▷ ▷ -print_symbols
17232 ▷ pdflatex graphs_v5_level_5_reps.tex
17233 ▷ $(OPEN) graphs_v5_level_5_reps.pdf
17234 ▷ #pdflatex graphs_v5_poset.tex
17235 ▷ #$(OPEN) graphs_v5_poset.pdf
17236
17237
17238 tournament_classify_4:
17239 ▷ $(ORBITER) -v 2 \
17240 ▷ ▷ -define GC -graph_classification \
17241 ▷ ▷ ▷ -n 4 -tournament \
17242 ▷ ▷ ▷ -poset_classification_control \
17243 ▷ ▷ ▷ ▷ -problem_label tournament_4 \
17244 ▷ ▷ ▷ ▷ -depth 6 \
17245 ▷ ▷ ▷ ▷ -draw_options \
17246 ▷ ▷ ▷ ▷ ▷ -radius 250 -embedded \
17247 ▷ ▷ ▷ ▷ -end \
17248 ▷ ▷ ▷ -end \
17249 ▷ ▷ -end \
17250 ▷ ▷ -with GC -do \
17251 ▷ ▷ -graph_classification_activity \
17252 ▷ ▷ ▷ -draw_options \
17253 ▷ ▷ ▷ ▷ -radius 400 \
17254 ▷ ▷ ▷ ▷ -line_width 2 -scale 0.10 \
17255 ▷ ▷ ▷ -end \
17256 ▷ ▷ ▷ -draw_graphs_at_level 6 \
17257 ▷ ▷ -end \
17258 ▷ ▷ -print_symbols
17259 ▷ pdflatex tournament_4_level_6_reps.tex
17260 ▷ $(OPEN) tournament_4_level_6_reps.pdf
17261 ▷
17262
17263
17264
17265
17266 graph_classify_8_r3:
17267 ▷ $(ORBITER) -v 3 \
17268 ▷ ▷ -define GC -graph_classification \
17269 ▷ ▷ ▷ -n 8 -regular 3 \
17270 ▷ ▷ ▷ -poset_classification_control \
17271 ▷ ▷ ▷ ▷ -problem_label graphs_v8_r3 \
17272 ▷ ▷ ▷ ▷ -depth 12 \

```

```

17273 ▷ ▷ ▷ ▷ -draw_options -radius 250 \
17274 ▷ ▷ ▷ ▷ -line_width 0.2 -embedded \
17275 ▷ ▷ ▷ ▷ -end \
17276 ▷ ▷ ▷ -end \
17277 ▷ ▷ -end \
17278 ▷ ▷ -with GC -do \
17279 ▷ ▷ -graph_classification_activity \
17280 ▷ ▷ ▷ -draw_options \
17281 ▷ ▷ ▷ ▷ -radius 400 \
17282 ▷ ▷ ▷ ▷ -line_width 2 -scale 0.10 \
17283 ▷ ▷ ▷ -end \
17284 ▷ ▷ ▷ -draw_graphs_at_level 12 \
17285 ▷ ▷ -end \
17286 ▷ ▷ -print_symbols
17287 ▷ #pdflatex graphs_v8_r3_poset_lvl12.tex
17288 ▷ #$(OPEN) graphs_v8_r3_poset_lvl12.pdf
17289
17290
17291
17292
17293 Symmetric_4_inversion_graph_recognize:
17294 ▷ $(ORBITER) -v 10 \
17295 ▷ ▷ -define G -permutation-group -symmetric.group 4 -end \
17296 ▷ ▷ -with G -do \
17297 ▷ ▷ -group_theoretic_activity \
17298 ▷ ▷ ▷ -export_inversion_graphs "Symmetric4_inversion_graphs.csv" \
17299 ▷ ▷ -end
17300 ▷ $(ORBITER) -v 2 \
17301 ▷ ▷ -define GC -graph_classification \
17302 ▷ ▷ ▷ -n 4 \
17303 ▷ ▷ ▷ -poset_classification_control \
17304 ▷ ▷ ▷ ▷ -problem_label graphs_v4 -depth 6 \
17305 ▷ ▷ ▷ ▷ -draw_options -radius 250 -embedded -end \
17306 ▷ ▷ ▷ -end \
17307 ▷ ▷ -end \
17308 ▷ ▷ -with GC -do \
17309 ▷ ▷ -graph_classification_activity \
17310 ▷ ▷ ▷ -recognize_graphs_from_adjacency_matrix_csv \
17311 ▷ ▷ ▷ ▷ Symmetric4_inversion_graphs.csv \
17312 ▷ ▷ -end \
17313 ▷ ▷ -print_symbols
17314
17315
17316 Symmetric_5_inversion_graph_recognize:
17317 ▷ $(ORBITER) -v 10 \
17318 ▷ ▷ -define G -permutation-group -symmetric.group 5 -end \
17319 ▷ ▷ -with G -do \
17320 ▷ ▷ -group_theoretic_activity \
17321 ▷ ▷ ▷ -export_inversion_graphs \
17322 ▷ ▷ ▷ ▷ "Symmetric5_inversion_graphs.csv" \
17323 ▷ ▷ -end
17324 ▷ $(ORBITER) -v 2 \
17325 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17326 ▷ ▷ -define GC -graph_classification \
17327 ▷ ▷ ▷ -n 5 \
17328 ▷ ▷ ▷ -poset_classification_control \
17329 ▷ ▷ ▷ ▷ -problem_label graphs_v5 \
17330 ▷ ▷ ▷ ▷ -depth 10 \
17331 ▷ ▷ ▷ ▷ -draw_options \

```

```

17332 ▷ ▷ ▷ ▷ -radius 250 -embedded \
17333 ▷ ▷ ▷ ▷ -end \
17334 ▷ ▷ ▷ -end \
17335 ▷ ▷ -end \
17336 ▷ ▷ -with GC -do \
17337 ▷ ▷ -graph_classification_activity \
17338 ▷ ▷ ▷ -recognize_graphs_from_adjacency_matrix_csv \
17339 ▷ ▷ ▷ Symmetric5_inversion_graphs.csv \
17340 ▷ ▷ -end \
17341 ▷ ▷ -print_symbols
17342 ▷ #pdflatex graphs_v5_poset.tex
17343 ▷ #$(OPEN) graphs_v5_poset.pdf
17344
17345
17346
17347
17348 #####
17349 # Chapter 13 - Design Theory
17350 #####
17351
17352
17353
17354 test_13:
17355 ▷ make test_13.1 # slow
17356 ▷ make test_13.2
17357 ▷ make test_13.3
17358 ▷ make test_13.4
17359 ▷ make test_13.5
17360
17361
17362
17363
17364 #####
17365 # Section 13.1: Design Theory
17366
17367 SECTION.DESIGN.THEORY:
17368
17369
17370 test_11.5:
17371 ▷ make design_PG_2.3
17372 ▷ make design_PG_2.4
17373 ▷ make wreath_product_designs_n4_k2_inc.txt
17374 ▷ make wreath_product_designs_n8_k6_inc.txt
17375 ▷ make design_20
17376 ▷ make design_600
17377 ▷ make LSQ_5A
17378 ▷ make LSQ_5B
17379 ▷ make LSQ_5_c
17380
17381
17382
17383 design_PG_2.3:
17384 ▷ $(ORBITER) -v 8 \
17385 ▷ ▷ -define F -finite_field -q 3 -end \
17386 ▷ ▷ -define D -design -field F -family PG.2.q -end \
17387 ▷ ▷ -with D -do \
17388 ▷ ▷ ▷ -design_activity \
17389 ▷ ▷ ▷ ▷ -export_inc \
17390 ▷ ▷ -end \

```

```

17391 ▷ ▷ -with D -do \
17392 ▷ ▷ ▷ -design_activity \
17393 ▷ ▷ ▷ ▷ -tactical_decomposition \
17394 ▷ ▷ -end
17395
17396 # writes PG.2.q3.inc.txt
17397
17398 design_PG.2.4:
17399 ▷ $(ORBITER) -v 8 \
17400 ▷ ▷ -define F -finite_field -q 4 -end \
17401 ▷ ▷ -define D -design -field F -family PG.2.q -end \
17402 ▷ ▷ -with D -do \
17403 ▷ ▷ ▷ -design_activity \
17404 ▷ ▷ ▷ ▷ -export_inc \
17405 ▷ ▷ -end
17406
17407
17408
17409 wreath_product_designs_n4.k2.inc.txt:
17410 ▷ $(ORBITER) -v 8 \
17411 ▷ ▷ -define D -design -wreath_product_designs 4 2 -end \
17412 ▷ ▷ -with D -do \
17413 ▷ ▷ ▷ -design_activity \
17414 ▷ ▷ ▷ ▷ -export_inc \
17415 ▷ ▷ -end
17416
17417
17418
17419 wreath_product_designs_n8.k6.inc.txt:
17420 ▷ $(ORBITER) -v 8 \
17421 ▷ ▷ -define D -design -wreath_product_designs 8 6 -end \
17422 ▷ ▷ -with D -do \
17423 ▷ ▷ ▷ -design_activity \
17424 ▷ ▷ ▷ ▷ -export_inc \
17425 ▷ ▷ -end
17426
17427
17428 # wreath_product_designs_n8.k6.inc.txt
17429 # The design has 16 points and 3920 blocks of size 6.
17430
17431
17432 design_20:
17433 ▷ $(ORBITER) -v 8 \
17434 ▷ ▷ -define D -design -list_of_blocks_coded 15 3 \
17435 ▷ ▷ "61, 67, 72, 76, 129, 147, 152, 156, 197, 204, 215, 224, 249, 261, 267, 276,
296, 303, 309, 319" \
17436 ▷ ▷ -end \
17437 ▷ ▷ -with D -do \
17438 ▷ ▷ ▷ -design_activity \
17439 ▷ ▷ ▷ ▷ -export_inc \
17440 ▷ ▷ -end
17441
17442
17443 design_600:
17444 ▷ echo $(GEO_BLOCKS_600) | sed 's/ //' >geo.600.blocks.csv
17445 ▷ $(ORBITER) -v 8 \
17446 ▷ ▷ -define D -design -list_of_blocks_from_file \
17447 ▷ ▷ 15 geo.600.blocks.csv \
17448 ▷ ▷ -end \

```

```

17449 ▷ ▷ -with D -do \
17450 ▷ ▷ ▷ -design_activity \
17451 ▷ ▷ ▷ ▷ -export_inc \
17452 ▷ ▷ -end \
17453 ▷ ▷ -with D -do \
17454 ▷ ▷ ▷ -design_activity \
17455 ▷ ▷ ▷ ▷ -intersection_matrix \
17456 ▷ ▷ -end
17457
17458 # see this for why we need to use sed:
17459 #https://stackoverflow.com/questions/2003536/bash-why-is-echo-adding-extra-space
17460
17461 LSQ_5A:
17462 ▷ $(ORBITER) -v 8 \
17463 ▷ ▷ -define LSQ5A -vector -format 5 -dense $(LSQ_5A.TABLE) -end \
17464 ▷ ▷ -define D -design -linear_space_from_latin_square \
17465 ▷ ▷ ▷ LSQ5A \
17466 ▷ ▷ -end \
17467 ▷ ▷ -with D -do \
17468 ▷ ▷ ▷ -design_activity \
17469 ▷ ▷ ▷ ▷ -export_inc \
17470 ▷ ▷ -end
17471
17472 LSQ_5B:
17473 ▷ $(ORBITER) -v 8 \
17474 ▷ ▷ -define LSQ5B -vector -format 5 -dense $(LSQ_5B.TABLE) -end \
17475 ▷ ▷ -define D -design -linear_space_from_latin_square \
17476 ▷ ▷ ▷ LSQ5B \
17477 ▷ ▷ -end \
17478 ▷ ▷ -with D -do \
17479 ▷ ▷ ▷ -design_activity \
17480 ▷ ▷ ▷ ▷ -export_inc \
17481 ▷ ▷ -end
17482
17483
17484 #latin_square_order5_LSQ5B.inc.txt
17485
17486
17487 LSQ_5_c:
17488 ▷ $(ORBITER) -v 3 \
17489 ▷ ▷ -define C -combinatorial_object \
17490 ▷ ▷ ▷ -label LSQ_5 LSQ\5 \
17491 ▷ ▷ ▷ -file_of_incidence_geometries latin_square_order5_LSQ5A.inc.txt 15 28 90 \
17492 ▷ ▷ ▷ -file_of_incidence_geometries latin_square_order5_LSQ5B.inc.txt 15 28 90 \
17493 ▷ ▷ -end \
17494 ▷ ▷ -with C -do \
17495 ▷ ▷ -combinatorial_object_activity \
17496 ▷ ▷ ▷ -canonical_form \
17497 ▷ ▷ ▷ ▷ -save_ago \
17498 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
17499 ▷ ▷ ▷ -end \
17500 ▷ ▷ -end \
17501 ▷ ▷ -with C -do \
17502 ▷ ▷ -combinatorial_object_activity \
17503 ▷ ▷ ▷ -report \
17504 ▷ ▷ ▷ ▷ -export_flag_orbits \
17505 ▷ ▷ ▷ ▷ -show_TDO \
17506 ▷ ▷ ▷ ▷ -show_TDA \
17507 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \

```

```

17508 ▷ ▷ ▷ -end \
17509 ▷ ▷ -end
17510 ▷ pdflatex LSQ_5_classification.tex
17511 ▷ $(OPEN) LSQ_5_classification.pdf
17512
17513
17514
17515
17516
17517 #####
17518 # Section 13.2: Assuming Symmetry
17519
17520 SECTION_ASSUMING_SYMMETRY:
17521
17522 test_11_6:
17523
17524 ▷ make KM_cyclic_7
17525 ▷ make KM_PGGL_2_32
17526 ▷ #make KM_PGGL_2_32_solve
17527 ▷ #make KM_PSL_3_5
17528 ▷ make C5_KM_2_3
17529 ▷ make C5_KM_1_3
17530 ▷ make C5_concatenate
17531 ▷ make C5_solve
17532 ▷ make C5_design_classify
17533 ▷
17534
17535
17536
17537
17538
17539
17540 KM_cyclic_7:
17541 ▷ $(ORBITER) -v 3 \
17542 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17543 ▷ ▷ -define gens -vector -dense "1,2,3,4,5,6,0" -end \
17544 ▷ ▷ -define G -permutation_group -symmetric_group 7 \
17545 ▷ ▷ ▷ -subgroup_by_generators "C7" 7 1 gens \
17546 ▷ ▷ -end \
17547 ▷ ▷ -define Control -poset_classification_control \
17548 ▷ ▷ ▷ -problem_label C7 -W -depth 3 \
17549 ▷ ▷ ▷ -draw_options -embedded -sideways -radius 50 \
17550 ▷ ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
17551 ▷ ▷ -end \
17552 ▷ ▷ -define Orb -orbits -group G \
17553 ▷ ▷ ▷ -on_subsets 3 Control \
17554 ▷ ▷ -end \
17555 ▷ ▷ -with Orb -do -orbits_activity \
17556 ▷ ▷ ▷ -Kramer_Mesner_matrix 2 3 \
17557 ▷ ▷ -end \
17558 ▷ ▷ -with Orb -do -orbits_activity \
17559 ▷ ▷ ▷ -report \
17560 ▷ ▷ ▷ -report_options \
17561 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
17562 ▷ ▷ ▷ -end \
17563 ▷ ▷ -end
17564 ▷ $(ORBITER) -v 4 \
17565 ▷ ▷ -define A -vector -file C7_KM_2.3.csv -end \
17566 ▷ ▷ -define D -diophant \

```

```

17567 ▷ ▷ -label "C7_KM.2.3_system" \
17568 ▷ ▷ -coefficient_matrix A \
17569 ▷ ▷ -RHS_constant "EQ=1" \
17570 ▷ ▷ -x_min_global 0 -x_max_global 1 \
17571 ▷ ▷ -end \
17572 ▷ ▷ -with D -do \
17573 ▷ ▷ ▷ -diophant_activity -solve_mckay \
17574 ▷ ▷ -end
17575
17576
17577 # to create simple 7-designs on 33 points with block size 8 and lambda = 10 invar
      iant under PGGL(2,32):
17578
17579 KM_PGGL_2.32:
17580 ▷ $(ORBITER) -v 3 \
17581 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17582 ▷ ▷ -define Control -poset_classification_control \
17583 ▷ ▷ ▷ -problem_label KM_PGGL_2.32 \
17584 ▷ ▷ ▷ -W -depth 8 \
17585 ▷ ▷ ▷ -draw_options -embedded -sideways -radius 50 \
17586 ▷ ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
17587 ▷ ▷ -end \
17588 ▷ ▷ -define G -linear_group -PGGL 2 32 -end \
17589 ▷ ▷ -define Orb -orbits -group G \
17590 ▷ ▷ ▷ -on_subsets 8 Control \
17591 ▷ ▷ -end \
17592 ▷ ▷ -with Orb -do -orbits_activity \
17593 ▷ ▷ ▷ -Kramer_Mesner_matrix 7 8 \
17594 ▷ ▷ -end \
17595 ▷ ▷ -with Orb -do -orbits_activity \
17596 ▷ ▷ ▷ -report \
17597 ▷ ▷ ▷ -report_options \
17598 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
17599 ▷ ▷ ▷ -end \
17600 ▷ ▷ -end
17601 ▷ $(ORBITER) -v 2 -draw_matrix \
17602 ▷ ▷ -input_csv_file KM_PGGL_2.32_KM.7.8.csv \
17603 ▷ ▷ -box_width 20 -bit_depth 24 \
17604 ▷ ▷ -partition 3 32 97 -end
17605 ▷ pdflatex KM_PGGL_2.32_poset_lvl.8.tex
17606 ▷ $(OPEN) KM_PGGL_2.32_poset_lvl.8.pdf
17607 ▷ $(OPEN) KM_PGGL_2.32_KM.7.8.draw.bmp
17608
17609 KM_PGGL_2.32_solve:
17610 ▷ $(ORBITER) -v 4 \
17611 ▷ ▷ -define A -vector -file KM_PGGL_2.32_KM.7.8.csv -end \
17612 ▷ ▷ -define D -diophant \
17613 ▷ ▷ -label "KM_PGGL_2.32_KM.7.8_system" \
17614 ▷ ▷ -coefficient_matrix A \
17615 ▷ ▷ -RHS_constant "EQ=10" \
17616 ▷ ▷ -x_min_global 0 -x_max_global 1 \
17617 ▷ ▷ -end \
17618 ▷ ▷ -with D -do \
17619 ▷ ▷ ▷ -diophant_activity -solve_mckay \
17620 ▷ ▷ -end
17621
17622
17623
17624

```

```

17625
17626 KM_PSL_3_5:
17627 ▷ $(ORBITER) -v 3 \
17628 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17629 ▷ ▷ -define Control -poset_classification_control \
17630 ▷ ▷ ▷ -problem_label KM_PSL_3_5 \
17631 ▷ ▷ ▷ -W -depth 10 \
17632 ▷ ▷ ▷ -draw_options -embedded -sideways \
17633 ▷ ▷ ▷ ▷ -radius 50 -scale 0.5 -line_width 0.3 -end \
17634 ▷ ▷ -end \
17635 ▷ ▷ -define G -linear_group -PSL 3 5 -end \
17636 ▷ ▷ -define Orb -orbits -group G \
17637 ▷ ▷ ▷ -on_subsets 10 Control \
17638 ▷ ▷ -end \
17639 ▷ ▷ -with Orb -do -orbits_activity \
17640 ▷ ▷ ▷ -Kramer_Mesner_matrix 8 10 \
17641 ▷ ▷ -end \
17642 ▷ ▷ -with Orb -do -orbits_activity \
17643 ▷ ▷ ▷ -report \
17644 ▷ ▷ ▷ -report_options \
17645 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
17646 ▷ ▷ ▷ -end \
17647 ▷ ▷ -end
17648 ▷ $(ORBITER) -v 2 -draw_matrix \
17649 ▷ ▷ -input_csv_file KM_PSL_3_5_KM_8_10.csv \
17650 ▷ ▷ -box_width 10 -bit_depth 8 -partition 3 42 174 -end
17651 ▷ $(ORBITER) -v 4 \
17652 ▷ ▷ -define A -vector -file KM_PSL_3_5_KM_8_10.csv -end \
17653 ▷ ▷ -define D -diophant \
17654 ▷ ▷ ▷ -label "KM_PSL_3_5_KM_8_10_system" \
17655 ▷ ▷ ▷ -coefficient_matrix A \
17656 ▷ ▷ ▷ -RHS_constant "EQ=93" \
17657 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
17658 ▷ ▷ -end \
17659 ▷ ▷ -with D -do \
17660 ▷ ▷ ▷ -diophant_activity -solve_mckay \
17661 ▷ ▷ -end
17662
17663
17664
17665
17666
17667 C5_KM_2_3:
17668 ▷ $(ORBITER) -v 3 \
17669 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17670 ▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
17671 ▷ ▷ -define G -permutation_group -symmetric_group 15 \
17672 ▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
17673 ▷ ▷ -end \
17674 ▷ ▷ -define Control -poset_classification_control \
17675 ▷ ▷ ▷ -problem_label C5 -W -depth 3 \
17676 ▷ ▷ ▷ -draw_options -embedded -sideways -radius 50 \
17677 ▷ ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
17678 ▷ ▷ -end \
17679 ▷ ▷ -define Orb -orbits -group G \
17680 ▷ ▷ ▷ -on_subsets 3 Control \
17681 ▷ ▷ -end \
17682 ▷ ▷ -with Orb -do -orbits_activity \
17683 ▷ ▷ ▷ -Kramer_Mesner_matrix 2 3 \

```



```

17684 ▷ ▷ -end
17685
17686 #C5_KM_2_3.csv
17687
17688
17689 C5_KM_1_3:
17690 ▷ $(ORBITER) -v 3 \
17691 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17692 ▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
17693 ▷ ▷ -define G -permutation_group -symmetric_group 15 \
17694 ▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
17695 ▷ ▷ -end \
17696 ▷ ▷ -define Control -poset_classification_control \
17697 ▷ ▷ ▷ -problem_label C5 -W -depth 3 \
17698 ▷ ▷ ▷ -draw_options -embedded -sideways -radius 50 \
17699 ▷ ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
17700 ▷ ▷ -end \
17701 ▷ ▷ -define Orb -orbits -group G \
17702 ▷ ▷ ▷ -on_subsets 3 Control \
17703 ▷ ▷ -end \
17704 ▷ ▷ -with Orb -do -orbits_activity \
17705 ▷ ▷ ▷ -Kramer_Mesner_matrix 1 3 \
17706 ▷ ▷ -end \
17707 ▷ ▷ -with Orb -do -orbits_activity \
17708 ▷ ▷ ▷ -export_something set_orbits 3 \
17709 ▷ ▷ -end \
17710
17711
17712 # 91 orbits at level 3
17713
17714 #C5_KM_1_3.csv
17715 #C5_set_orbits_level_3.csv
17716
17717
17718 C5_concatenate:
17719 ▷ $(ORBITER) -v 3 \
17720 ▷ ▷ -csv_file_concatenate C5_KM_combined.csv 2 C5_KM_1_3.csv C5_KM_2_3.csv
17721
17722 #C5_KM_combined.csv
17723 # in the combined system, there are three rows
17724 # from the first system and 21 rows from the second system
17725
17726
17727 C5_solve:
17728 ▷ $(ORBITER) -v 4 \
17729 ▷ ▷ -define A -vector -file C5_KM_combined.csv -end \
17730 ▷ ▷ -define D -diophant \
17731 ▷ ▷ ▷ -label "C5_KM_combined_system" \
17732 ▷ ▷ ▷ -coefficient_matrix A \
17733 ▷ ▷ ▷ -x_min_global 0 \
17734 ▷ ▷ ▷ -x_max_global 1 \
17735 ▷ ▷ ▷ -RHS "mult=3,EQ=5" \
17736 ▷ ▷ ▷ -RHS "mult=21,LE=1" \
17737 ▷ ▷ -end \
17738 ▷ ▷ -with D -do \
17739 ▷ ▷ ▷ -diophant_activity -solve_mckay \
17740 ▷ ▷ -end
17741
17742 #diophant::solve_all_mckay found 21540 solutions in 61001 backtrack nodes

```

```

17743 #Found 21540 solutions with 61001 backtrack steps
17744
17745 #C5_KM_combined_system_sol.csv
17746
17747
17748 C5_design_classify:
17749 ▷ $(ORBITER) -v 2 \
17750 ▷ ▷ -define C -combinatorial_object \
17751 ▷ ▷ ▷ -label C5_config C5\_config \
17752 ▷ ▷ ▷ -file_of_designs_through_block_orbits \
17753 ▷ ▷ ▷ ▷ C5_KM_combined_system_sol.csv \
17754 ▷ ▷ ▷ ▷ C5_set_orbits_level_3.csv \
17755 ▷ ▷ ▷ ▷ 15 3 \
17756 ▷ ▷ -end \
17757 ▷ ▷ -with C -do \
17758 ▷ ▷ -combinatorial_object_activity \
17759 ▷ ▷ ▷ -canonical_form \
17760 ▷ ▷ ▷ -end \
17761 ▷ ▷ -end \
17762 ▷ ▷ -with C -do \
17763 ▷ ▷ -combinatorial_object_activity \
17764 ▷ ▷ ▷ -report \
17765 ▷ ▷ ▷ ▷ -show_incidence_matrices \
17766 ▷ ▷ ▷ -end \
17767 ▷ ▷ -end
17768 ▷ pdflatex C5_config_classification.tex
17769 ▷ $(OPEN) C5_config_classification.pdf
17770
17771
17772 # finds 42 isomorphism types
17773
17774 # number 0 has group order 600
17775 # it is flag transitive and anti flag transitive
17776 # number 14 has group order 120
17777 # flag orbits: 15+60
17778 # anti flag orbits: 60+120
17779
17780
17781
17782
17783
17784 #####
17785 # Section 13.3: Design Theory - Large Sets
17786
17787 SECTION_DESIGN_THEORY_LARGE_SETS:
17788
17789 test_11_7:
17790 ▷ make AG_2_3.inc
17791 ▷ make LS_AG_2_3.design_table.create
17792 ▷ make AG_2_3.on_designs
17793 ▷ make AG_2_3.stab_orb_0
17794 ▷ make AG_2_3.stab_orb_0.Perm840_res192
17795 ▷ make LS_AG_2_3.disjoint_sets_graph_and_cliques
17796 ▷ make LS_AG_2_3.disjoint_sets_split
17797 ▷ make LS_AG_2_3.export_solutions
17798 ▷ make design_PG_2_3.table.create
17799 ▷ make design_PG_2_3.group_5
17800 ▷ make design_PG_2_3.group_5_sol_0
17801

```

```

17802
17803
17804 AG_2_3.inc:
17805 ▷ $(ORBITER) -v 2 \
17806 ▷ ▷ -define Geo -geometry_builder \
17807 ▷ ▷ ▷ -V 9 -B 12 \
17808 ▷ ▷ ▷ -TDO 4 -fuse 1 \
17809 ▷ ▷ ▷ -fname_GEO AG_2_3 \
17810 ▷ ▷ ▷ -test 3,4,5,6,7,8,9 \
17811 ▷ ▷ ▷ -output_to_inc_file \
17812 ▷ ▷ -end
17813
17814 #9 12 3
17815 #0 13 22 27 35 41 47 53 55 59 71 76
17816 #-1 1
17817 #432
17818
17819
17820
17821
17822
17823 LS_AG_2_3_design_table.create:
17824 ▷ $(ORBITER) -v 5 \
17825 ▷ ▷ -define B -vector -dense $(AG_2_3_BLOCKS) -end \
17826 ▷ ▷ -define D -design -list_of_blocks_coded 9 3 B -end \
17827 ▷ ▷ -define Sym9 -permutation_group -symmetric_group 9 -end \
17828 ▷ ▷ -define T -design_table D "AG_2_3" Sym9 -end
17829
17830 # creates AG_2_3_design_table.csv
17831 # and AG_2_3.makefile
17832
17833 #0,0,13,22,27,35,41,47,53,55,59,71,76
17834 # is the first design in AG_2_3_design_table.csv
17835
17836 #poset_orbit_node::init_root_node storing strong generators for a group of order
      362880
17837 # stabilizer order 432
17838 # 840 designs
17839
17840 #User time: 0.13 of a second, dt=13 tps = 100
17841
17842
17843 AG_2_3_on_designs:
17844 ▷ $(ORBITER) -v 2 \
17845 ▷ ▷ -define gens -vector -file AG_2_3_gens.csv -end \
17846 ▷ ▷ -define G -permutation_group \
17847 ▷ ▷ -bsgs AG_2_3 "AG_2_3" 840 362880 "0,1,2,3,4,5,6,7" 8 gens -end \
17848 ▷ ▷ -define Orb -orbits -group G \
17849 ▷ ▷ ▷ -on_points \
17850 ▷ ▷ -end \
17851 ▷ ▷ -with Orb -do -orbits_activity \
17852 ▷ ▷ ▷ -stabilizer 0 \
17853 ▷ ▷ -end
17854
17855 #Written file AG_2_3_stab_pt_0.makefile of size 239
17856
17857
17858
17859

```

```

17860
17861 # the stabilizer of the first design:
17862
17863
17864 #ToDo AG_2.3_stab_orb_0_gens.csv does not exist or is empty
17865
17866 AG_2.3_stab_orb_0:
17867 ▷ $(ORBITER) -v 2 \
17868 ▷ ▷ -define gens -vector -file AG_2.3_stab_pt_0_gens.csv -end \
17869 ▷ ▷ -define G -permutation_group \
17870 ▷ ▷ -bsgs AG_2.3_stab_orb_0 "AG_2.3_stab_pt_0" \
17871 ▷ ▷ ▷ 840 432 "0,1,2,3,4,5,6,7,8" 5 gens \
17872 ▷ ▷ -end \
17873 ▷ ▷ -define Gr -modified_group -from G \
17874 ▷ ▷ ▷ -restricted_action $(LARGE_SET_AG_2.3_NEIGHBOR_SET) \\neighbor\\set \
17875 ▷ ▷ -end \
17876 ▷ ▷ -with Gr -do \
17877 ▷ ▷ -group_theoretic_activity \
17878 ▷ ▷ ▷ -export_orbiter \
17879 ▷ ▷ -end
17880 ▷ ▷
17881
17882 AG_2.3_stab_orb_0.Perm840_res192:
17883 ▷ $(ORBITER) -v 2 \
17884 ▷ ▷ -define gens -vector -file Perm840_res192_gens.csv -end \
17885 ▷ ▷ -define G -permutation_group \
17886 ▷ ▷ -bsgs Perm840_res192 "Perm840 {\rm res192}" \
17887 ▷ ▷ ▷ 192 432 "0,1,2,3,4,5,6,7,8" 4 gens \
17888 ▷ ▷ -end \
17889 ▷ ▷ -with G -do \
17890 ▷ ▷ -group_theoretic_activity \
17891 ▷ ▷ ▷ -report \
17892 ▷ ▷ -end
17893 ▷ pdflatex Perm840_res192_report.tex
17894 ▷ $(OPEN) Perm840_res192_report.pdf
17895
17896
17897
17898 LS_AG_2.3_disjoint_sets_graph_and_cliques:
17899 ▷ $(ORBITER) -v 2 \
17900 ▷ ▷ -define Gamma -graph \
17901 ▷ ▷ ▷ -disjoint_sets_graph \
17902 ▷ ▷ ▷ AG_2.3_design_table.csv \
17903 ▷ ▷ -end \
17904 ▷ ▷ -with Gamma -do \
17905 ▷ ▷ -graph_theoretic_activity \
17906 ▷ ▷ ▷ -save \
17907 ▷ ▷ -end \
17908 ▷ ▷ -with Gamma -do \
17909 ▷ ▷ -graph_theoretic_activity \
17910 ▷ ▷ ▷ -find_cliques -target_size 7 -end \
17911 ▷ ▷ -end \
17912 ▷ ▷ -print_symbols
17913
17914
17915 #AG_2.3_design_table_disjoint_sets_colored_graph
17916 #User time: 0.66 of a second, dt=66 tps = 100
17917 #AG_2.3_design_table_disjoint_sets_sol.txt
17918 #AG_2.3_design_table_disjoint_sets_sol.csv

```

```

17919
17920 #15360 solutions
17921
17922 LS_AG_2_3_disjoint_sets_split:
17923 ▷ $(ORBITER) -v 4 \
17924 ▷ ▷ -define Gamma -graph -load \
17925 ▷ ▷ ▷ AG_2_3_design_table_disjoint_sets.colored_graph \
17926 ▷ ▷ -end \
17927 ▷ ▷ -with Gamma -do \
17928 ▷ ▷ -graph_theoretic_activity \
17929 ▷ ▷ ▷ -split_by_clique "0" "0" \
17930 ▷ ▷ -end
17931
17932
17933 #AG_2_3_design_table_disjoint_sets_0.graph
17934 #AG_2_3_design_table_disjoint_sets_0_subset.txt
17935
17936
17937
17938 LS_AG_2_3_export_solutions:
17939 ▷ $(ORBITER) -v 20 \
17940 ▷ ▷ -define B -vector -dense $(AG_2_3_BLOCKS) -end \
17941 ▷ ▷ -define D -design -list_of_blocks 9 3 B -end \
17942 ▷ ▷ -define Sym9 -permutation_group -symmetric_group 9 -end \
17943 ▷ ▷ -define T -design_table D "AG_2_3" Sym9 -end \
17944 ▷ ▷ -with D -do \
17945 ▷ ▷ -design_activity \
17946 ▷ ▷ ▷ -extract_solutions_by_index "AG_2_3" Sym9 \
17947 ▷ ▷ ▷ ▷ AG_2_3_design_table_disjoint_sets_sol.csv \
17948 ▷ ▷ ▷ ▷ solutions.csv \
17949 ▷ ▷ ▷ ▷ "" \
17950 ▷ ▷ -end
17951
17952
17953 #User time: 0.39 of a second, dt=39 tps = 100
17954 # solutions.csv
17955
17956
17957
17958
17959 design_PG_2_3_table_create:
17960 ▷ $(ORBITER) -v 2 \
17961 ▷ ▷ -define F -finite_field -q 3 -end \
17962 ▷ ▷ -define D -design -field F -family PG_2_q -end \
17963 ▷ ▷ -define Sym13 -permutation_group -symmetric_group 13 -end \
17964 ▷ ▷ -define T -design_table D "PG_2_13" Sym13 -end
17965
17966 # written file PG_2_13_design_table.csv
17967 # 1108800 designs
17968 #User time: 7:30
17969
17970 design_PG_2_3_group_5:
17971 ▷ $(ORBITER) -v 2 \
17972 ▷ ▷ -define F -finite_field -q 3 -end \
17973 ▷ ▷ -define D -design -field F -family PG_2_q -end \
17974 ▷ ▷ -define T -design_table D "PG_2_13" Sym13 -end \
17975 ▷ ▷ -define LSW -large_set_with_symmetry_assumption T \
17976 ▷ ▷ ▷ -H "5" $(GENERATORS_H5) \
17977 ▷ ▷ ▷ ▷ -N "1200" $(GENERATORS_N5) \

```

```

17978 ▷ ▷ ▷ ▷ -prefix "H5" \
17979 ▷ ▷ ▷ ▷ -selected_orbit_length 5 \
17980 ▷ ▷ ▷ -end \
17981 ▷ ▷ -with LSW -do \
17982 ▷ ▷ ▷ -large_set_with_symmetry_assumption_activity \
17983 ▷ ▷ ▷ ▷ -normalizer_on_orbits_of_a_given_length 5 \
17984 ▷ ▷ ▷ -end
17985
17986 #H5_N_orbit_reps.csv
17987 # 678 orbits
17988 #User time: 2:39
17989
17990
17991 design_PG_2_3_group_5_sol_0:
17992 ▷ $(ORBITER) -v 2 \
17993 ▷ ▷ -define F -finite_field -q 3 -end \
17994 ▷ ▷ -define D -design -field F -family PG_2.q -end \
17995 ▷ ▷ -define T -design_table D "PG_2.13" Sym13 -end \
17996 ▷ ▷ -define LSW -large_set_with_symmetry_assumption T \
17997 ▷ ▷ ▷ ▷ -H "5" $(GENERATORS_H5) \
17998 ▷ ▷ ▷ ▷ -N "1200" $(GENERATORS_N5) \
17999 ▷ ▷ ▷ ▷ -prefix "H5" \
18000 ▷ ▷ ▷ ▷ -selected_orbit_length 5 \
18001 ▷ ▷ ▷ -end \
18002 ▷ ▷ -with LSW -do \
18003 ▷ ▷ ▷ -large_set_with_symmetry_assumption_activity \
18004 ▷ ▷ ▷ ▷ -read_solution_file 5 case_0_sol.txt \
18005 ▷ ▷ ▷ -end
18006
18007
18008
18009
18010 #####
18011 # Section 13.4: Design Theory - Delandtsheer Doyen
18012
18013 SECTION.DESIGN.THEORY.DELANDTSHEER.DOYEN:
18014
18015 test_11_8:
18016 ▷ make DD_PP4
18017 ▷ make DD_PP4.span_design
18018 ▷ make DD_PP4.design_orbit
18019 ▷ #make PG_2_27_special
18020
18021
18022 DD_PP4:
18023 ▷ $(ORBITER) -v 6 \
18024 ▷ ▷ -define pair_search_control -poset_classification_control \
18025 ▷ ▷ ▷ -problem_label PP4_pairs -W \
18026 ▷ ▷ ▷ -draw_options -end \
18027 ▷ ▷ -end \
18028 ▷ ▷ -define search_control -poset_classification_control \
18029 ▷ ▷ ▷ -problem_label PP4_search -W \
18030 ▷ ▷ ▷ -draw_options -end \
18031 ▷ ▷ ▷ -depth 5 \
18032 ▷ ▷ -end \
18033 ▷ ▷ -Delandtsheer_Doyen \
18034 ▷ ▷ ▷ -pair_search_control pair_search_control \
18035 ▷ ▷ ▷ -search_control search_control \
18036 ▷ ▷ ▷ -search_partial_base_lines \

```

```

18037 ▷ ▷ ▷ $(PP4) $(PP4_GROUP1) $(PP4_MASK1) \
18038 ▷ ▷ ▷ -end \
18039
18040
18041 DD_PP4_span_design:
18042 ▷ $(ORBITER) -v 6 \
18043 ▷ ▷ -define D -design -list_of_base_blocks \
18044 ▷ ▷ ▷ group \
18045 ▷ ▷ ▷ PP4_no_mask_cyclic21_cases.csv
18046 ▷ ▷ ▷ Starter \
18047 ▷ ▷ -end
18048
18049
18050 DD_PP4_design_orbit:
18051 ▷ $(ORBITER) -v 4 \
18052 ▷ ▷ -define G1 -linear_group -AGL 1 3 \
18053 ▷ ▷ ▷ -end \
18054 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
18055 ▷ ▷ ▷ -end \
18056 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
18057 ▷ ▷ ▷ "21" "1,1,1,1" \
18058 ▷ ▷ -end \
18059 ▷ ▷ -with G -do \
18060 ▷ ▷ -group_theoretic_activity \
18061 ▷ ▷ ▷ -report \
18062 ▷ ▷ -end \
18063 ▷ ▷ -define D -design -list_of_base_blocks \
18064 ▷ ▷ ▷ G \
18065 ▷ ▷ ▷ PP4_no_mask_cyclic21_cases.csv \
18066 ▷ ▷ ▷ Starter \
18067 ▷ ▷ -end
18068
18069
18070
18071
18072
18073
18074
18075 PG_2_27_special:
18076 ▷ $(ORBITER) -v 2 \
18077 ▷ ▷ -define F -finite_field -q 27 -override_polynomial 46 -end \
18078 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
18079 ▷ ▷ -with P -do -projective_space_activity \
18080 ▷ ▷ ▷ -cheat_sheet \
18081 ▷ ▷ -end
18082 ▷ pdflatex PG_2_27.tex
18083 ▷ $(OPEN) PG_2_27.pdf
18084
18085
18086
18087
18088
18089
18090 #####
18091 # Chapter 14 - Finite Geometry
18092 #####
18093
18094
18095 test_14:

```

```

18096 ▷ make test_14.1
18097 ▷ make test_14.2
18098 ▷ make test_14.3
18099 ▷ make test_14.4
18100
18101
18102
18103
18104 #####
18105 # Section 14.1: Spreads
18106
18107 SECTION.SPREADS:
18108
18109 test_14.1:
18110 ▷ make create_spread_9a
18111 ▷ make create_spread_9b
18112 ▷ make create_spread_25.7
18113 ▷ make create_spread_Rao.Rao.27
18114 ▷ make desarguesian_spread_in_PG_3.2
18115 ▷ make desarguesian_spread_in_PG_5.2
18116 ▷ make desarguesian_spread_in_PG_3.4
18117 ▷ make desarguesian_spread_in_PG_3.5
18118 ▷ make classify_spreads_4
18119 ▷ make classify_spreads_16.4
18120 ▷ make classify_spreads_25_starter_lift_case_0
18121 ▷ make spreads_25_starter_0_cliques
18122 ▷ make classify_spreads_25_starter_lift_all_cases
18123 ▷ make spreads_25_starter_cliques
18124 ▷ make classify_spreads_25_isomorph
18125 ▷ make classify_spreads_27_3_starter
18126 ▷ make classify_spreads_27_starter_lift_all_cases
18127 ▷ make spreads_27_starter_cliques
18128 ▷ make classify_spreads_27_isomorph_and_recognize
18129 ▷ make create_spread_27.0
18130 ▷ make create_spread_27.1
18131 ▷ make create_spread_27.2
18132 ▷ make create_spread_27.3
18133 ▷ make create_spread_27.4
18134 ▷ make create_spread_27.5
18135 ▷ make create_spread_27.6
18136
18137
18138
18139 create_spread_9a:
18140 ▷ $(ORBITER) -v 3 \
18141 ▷ ▷ -define F -finite_field -q 3 -end \
18142 ▷ ▷ -define G -linear_group -PGL 4 F -end \
18143 ▷ ▷ -define S -spread -kernel_field F \
18144 ▷ ▷ ▷ -group G -k 2 -catalogue 0 \
18145 ▷ ▷ ▷ -end
18146
18147 # desarguesian spread, ago = 5760
18148
18149 create_spread_9b:
18150 ▷ $(ORBITER) -v 3 \
18151 ▷ ▷ -define F -finite_field -q 3 -end \
18152 ▷ ▷ -define G -linear_group -PGL 4 F -end \
18153 ▷ ▷ -define S -spread -kernel_field F \
18154 ▷ ▷ ▷ -group G -k 2 -catalogue 1 \

```



```

18155 ▷ ▷ ▷ -end
18156
18157
18158 # Hall spread, ago = 1920
18159
18160
18161 create_spread_25_7:
18162 ▷ $(ORBITER) -v 3 \
18163 ▷ ▷ -define F -finite_field -q 5 -end \
18164 ▷ ▷ -define G -linear_group -PGL 4 F -end \
18165 ▷ ▷ -define S -spread -kernel_field F \
18166 ▷ ▷ ▷ -group G -k 2 -catalogue 7 \
18167 ▷ ▷ ▷ -end
18168
18169
18170
18171 create_spread_Rao_Rao_27:
18172 ▷ $(ORBITER) -v 3 \
18173 ▷ ▷ -define F -finite_field -q 3 -end \
18174 ▷ ▷ -define SS -vector -dense $(SPREAD_SET_27_RAO_RAO) -end \
18175 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18176 ▷ ▷ -define S -spread -kernel_field F \
18177 ▷ ▷ ▷ -group G -k 3 -spread_set "Rao_Rao" "Rao\_Rao" SS \
18178 ▷ ▷ ▷ -end
18179
18180
18181 desarguesian_spread_in_PG_3_2:
18182 ▷ $(ORBITER) -v 3 \
18183 ▷ ▷ -define FQ -finite_field -q 4 -end \
18184 ▷ ▷ -define Fq -finite_field -q 2 -end \
18185 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
18186 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
18187 ▷ pdflatex Desarguesian_Spread_3.2.tex
18188 ▷ $(OPEN) Desarguesian_Spread_3.2.pdf
18189
18190 desarguesian_spread_in_PG_5_2:
18191 ▷ $(ORBITER) -v 3 \
18192 ▷ ▷ -define FQ -finite_field -q 8 -end \
18193 ▷ ▷ -define Fq -finite_field -q 2 -end \
18194 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
18195 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
18196 ▷ pdflatex Desarguesian_Spread_5.2.tex
18197 ▷ $(OPEN) Desarguesian_Spread_5.2.pdf
18198
18199 desarguesian_spread_in_PG_3_4:
18200 ▷ $(ORBITER) -v 3 \
18201 ▷ ▷ -define FQ -finite_field -q 16 -end \
18202 ▷ ▷ -define Fq -finite_field -q 4 -end \
18203 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
18204 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
18205 ▷ pdflatex Desarguesian_Spread_3.4.tex
18206 ▷ $(OPEN) Desarguesian_Spread_3.4.pdf
18207
18208 desarguesian_spread_in_PG_3_5:
18209 ▷ $(ORBITER) -v 3 \
18210 ▷ ▷ -define FQ -finite_field -q 25 -end \
18211 ▷ ▷ -define Fq -finite_field -q 5 -end \
18212 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
18213 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end

```

```

18214 ▷ pdflatex Desarguesian_Spread_3.5.tex
18215 ▷ $(OPEN) Desarguesian_Spread_3.5.pdf
18216
18217
18218 # ToDo -report
18219
18220 classify_spreads_4:
18221 ▷ $(ORBITER) -v 3 \
18222 ▷ ▷ -define Control -poset_classification_control \
18223 ▷ ▷ ▷ -draw_options \
18224 ▷ ▷ ▷ ▷ -embedded \
18225 ▷ ▷ ▷ -end \
18226 ▷ ▷ ▷ -W \
18227 ▷ ▷ ▷ -problem_label spreads_2.2 \
18228 ▷ ▷ -end \
18229 ▷ ▷ -define F -finite_field -q 2 -end \
18230 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
18231 ▷ ▷ -define C -spread_classifier \
18232 ▷ ▷ ▷ -projective_space P \
18233 ▷ ▷ ▷ -poset_classification_control Control \
18234 ▷ ▷ ▷ -k 2 \
18235 ▷ ▷ ▷ -starter_size 5 \
18236 ▷ ▷ ▷ -output_prefix "." \
18237 ▷ ▷ -end \
18238 ▷ ▷ -with C -do -spread_classify_activity \
18239 ▷ ▷ ▷ -compute_starter \
18240 ▷ ▷ -end
18241 ▷ #pdflatex spreads_2.2_poset_lvl1.5.tex
18242 ▷ #$(OPEN) spreads_2.2_poset_lvl1.5.pdf
18243
18244
18245 classify_spreads_16_4:
18246 ▷ $(ORBITER) -v 4 \
18247 ▷ ▷ -define Control -poset_classification_control \
18248 ▷ ▷ ▷ -draw_options \
18249 ▷ ▷ ▷ ▷ -radius 20 \
18250 ▷ ▷ ▷ ▷ -nodes_empty \
18251 ▷ ▷ ▷ ▷ -line_width 0.2 \
18252 ▷ ▷ ▷ ▷ -embedded \
18253 ▷ ▷ ▷ -end \
18254 ▷ ▷ ▷ -problem_label spreads_16.4 \
18255 ▷ ▷ -end \
18256 ▷ ▷ -define F -finite_field -q 4 -end \
18257 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
18258 ▷ ▷ -define C -spread_classifier \
18259 ▷ ▷ ▷ -projective_space P \
18260 ▷ ▷ ▷ -poset_classification_control Control \
18261 ▷ ▷ ▷ -k 2 \
18262 ▷ ▷ ▷ -starter_size 17 \
18263 ▷ ▷ ▷ -output_prefix "." \
18264 ▷ ▷ -end \
18265 ▷ ▷ -with C -do -spread_classify_activity \
18266 ▷ ▷ ▷ -compute_starter \
18267 ▷ ▷ -end
18268 ▷ #pdflatex spreads_16.4_poset_lvl1.17.tex
18269 ▷ #$(OPEN) spreads_16.4_poset_lvl1.17.pdf
18270
18271
18272

```

```

18273
18274
18275 classify_spreads_25_starter_lift_case_0:
18276 ▷ $(ORBITER) -v 3 \
18277 ▷ ▷ -define Control -poset_classification_control \
18278 ▷ ▷ ▷ -draw_options \
18279 ▷ ▷ ▷ ▷ -radius 20 \
18280 ▷ ▷ ▷ ▷ -nodes_empty \
18281 ▷ ▷ ▷ ▷ -line_width 0.2 \
18282 ▷ ▷ ▷ ▷ -embedded \
18283 ▷ ▷ ▷ -end \
18284 ▷ ▷ ▷ -W \
18285 ▷ ▷ ▷ -problem_label spreads_25 \
18286 ▷ ▷ -end \
18287 ▷ ▷ -define F -finite_field -q 5 -end \
18288 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
18289 ▷ ▷ -define C -spread_classifier \
18290 ▷ ▷ ▷ -projective_space P \
18291 ▷ ▷ ▷ -k 2 \
18292 ▷ ▷ ▷ -starter_size 5 \
18293 ▷ ▷ ▷ -recoordinatize \
18294 ▷ ▷ ▷ -poset_classification_control Control \
18295 ▷ ▷ ▷ -output_prefix "" \
18296 ▷ ▷ -end \
18297 ▷ ▷ -with C -do -spread_classify_activity \
18298 ▷ ▷ ▷ -compute_starter \
18299 ▷ ▷ -end \
18300 ▷ ▷ -with C -do -spread_classify_activity \
18301 ▷ ▷ ▷ -prepare_lifting_single_case 0 \
18302 ▷ ▷ -end
18303
18304
18305 #save_colored_graph fname=spreads_25_graph_0.bin
18306 #save_colored_graph nb_vertices=225
18307 #save_colored_graph nb_colors=21
18308 #save_colored_graph nb_colors_per_vertex=1
18309 #save_colored_graph done
18310 #colored_graph::save done
18311 #Written file spreads_25_graph_0.bin of size 5914
18312
18313
18314 spreads_25_starter_0_cliques:
18315 ▷ $(ORBITER) -v 2 \
18316 ▷ ▷ ▷ -define G -graph -load spreads_25_graph_0.bin -end \
18317 ▷ ▷ ▷ -with G -do \
18318 ▷ ▷ ▷ -graph_theoretic_activity \
18319 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 -end \
18320 ▷ ▷ ▷ -end \
18321
18322 #graph_theoretic_activity::perform_activity Gr->label=spreads_25_graph_0 nb_sol =
      7680
18323
18324
18325 classify_spreads_25_starter_lift_all_cases:
18326 ▷ $(ORBITER) -v 3 \
18327 ▷ ▷ -define Control -poset_classification_control \
18328 ▷ ▷ ▷ -draw_options \
18329 ▷ ▷ ▷ ▷ -radius 20 \
18330 ▷ ▷ ▷ ▷ -nodes_empty \

```

```

18331 ▷ ▷ ▷ ▷ -line_width 0.2 \
18332 ▷ ▷ ▷ ▷ -embedded \
18333 ▷ ▷ ▷ -end \
18334 ▷ ▷ ▷ -W \
18335 ▷ ▷ ▷ -problem_label spreads_25 \
18336 ▷ ▷ -end \
18337 ▷ ▷ -define F -finite_field -q 5 -end \
18338 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
18339 ▷ ▷ -define C -spread_classifier \
18340 ▷ ▷ ▷ -projective_space P \
18341 ▷ ▷ ▷ -k 2 \
18342 ▷ ▷ ▷ -starter_size 5 \
18343 ▷ ▷ ▷ -recoordinatize \
18344 ▷ ▷ ▷ -poset_classification_control Control \
18345 ▷ ▷ ▷ -output_prefix "" \
18346 ▷ ▷ -end \
18347 ▷ ▷ -with C -do -spread_classify_activity \
18348 ▷ ▷ ▷ -compute_starter \
18349 ▷ ▷ -end \
18350 ▷ ▷ -with C -do -spread_classify_activity \
18351 ▷ ▷ ▷ -prepare_lifting_all_cases \
18352 ▷ ▷ -end
18353
18354 spreads_25_starter_cliques:
18355 ▷ $(ORBITER) -v 2 \
18356 ▷ ▷ -loop L 0 29 1 \
18357 ▷ ▷ ▷ -define G -graph -load spreads_25_graph_%L.bin -end \
18358 ▷ ▷ ▷ -with G -do \
18359 ▷ ▷ ▷ -graph_theoretic_activity \
18360 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 -end \
18361 ▷ ▷ ▷ -end \
18362 ▷ ▷ -end_loop L
18363
18364
18365 classify_spreads_25_isomorph:
18366 ▷ $(ORBITER) -v 4 \
18367 ▷ ▷ -define Control -poset_classification_control \
18368 ▷ ▷ ▷ -draw_options \
18369 ▷ ▷ ▷ ▷ -radius 20 \
18370 ▷ ▷ ▷ ▷ -nodes_empty \
18371 ▷ ▷ ▷ ▷ -line_width 0.2 \
18372 ▷ ▷ ▷ ▷ -embedded \
18373 ▷ ▷ ▷ -end \
18374 ▷ ▷ ▷ -W \
18375 ▷ ▷ ▷ -problem_label spreads_25 \
18376 ▷ ▷ -end \
18377 ▷ ▷ -define F -finite_field -q 5 -end \
18378 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
18379 ▷ ▷ -define C -spread_classifier \
18380 ▷ ▷ ▷ -projective_space P \
18381 ▷ ▷ ▷ -k 2 \
18382 ▷ ▷ ▷ -starter_size 5 \
18383 ▷ ▷ ▷ -recoordinatize \
18384 ▷ ▷ ▷ -poset_classification_control Control \
18385 ▷ ▷ ▷ -output_prefix "" \
18386 ▷ ▷ -end \
18387 ▷ ▷ -with C -do -spread_classify_activity \
18388 ▷ ▷ ▷ -compute_starter \
18389 ▷ ▷ -end \

```

```

18390 ▷ ▷ -with C -do -spread_classify_activity \
18391 ▷ ▷ ▷ -isomorph \
18392 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
18393 ▷ ▷ ▷ ▷ -use_database_for_starter \
18394 ▷ ▷ ▷ ▷ -build_db \
18395 ▷ ▷ ▷ ▷ -solution_prefix "" \
18396 ▷ ▷ ▷ ▷ -base_fname "" \
18397 ▷ ▷ ▷ -end \
18398 ▷ ▷ -end \
18399 ▷ ▷ -with C -do -spread_classify_activity \
18400 ▷ ▷ ▷ -isomorph \
18401 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
18402 ▷ ▷ ▷ ▷ -use_database_for_starter \
18403 ▷ ▷ ▷ ▷ -read_solutions \
18404 ▷ ▷ ▷ ▷ -solution_prefix "" \
18405 ▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
18406 ▷ ▷ ▷ -end \
18407 ▷ ▷ -end \
18408 ▷ ▷ -with C -do -spread_classify_activity \
18409 ▷ ▷ ▷ -isomorph \
18410 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
18411 ▷ ▷ ▷ ▷ -use_database_for_starter \
18412 ▷ ▷ ▷ ▷ -compute_orbits \
18413 ▷ ▷ ▷ ▷ -solution_prefix "" \
18414 ▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
18415 ▷ ▷ ▷ -end \
18416 ▷ ▷ -end \
18417 ▷ ▷ -with C -do -spread_classify_activity \
18418 ▷ ▷ ▷ -isomorph \
18419 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
18420 ▷ ▷ ▷ ▷ -use_database_for_starter \
18421 ▷ ▷ ▷ ▷ -isomorph_testing \
18422 ▷ ▷ ▷ ▷ -solution_prefix "" \
18423 ▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
18424 ▷ ▷ ▷ -end \
18425 ▷ ▷ -end \
18426 ▷ ▷ -with C -do -spread_classify_activity \
18427 ▷ ▷ ▷ -isomorph \
18428 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_25" \
18429 ▷ ▷ ▷ ▷ -use_database_for_starter \
18430 ▷ ▷ ▷ ▷ -isomorph_report \
18431 ▷ ▷ ▷ ▷ -solution_prefix "" \
18432 ▷ ▷ ▷ ▷ -base_fname "spreads_25_graph" \
18433 ▷ ▷ ▷ -end \
18434 ▷ ▷ -end
18435 ▷ pdflatex spreads_25_isomorphism_types.tex
18436 ▷ $(OPEN) spreads_25_isomorphism_types.pdf
18437
18438 #We found 21 isomorphism types
18439 #1:33
18440
18441
18442
18443 classify_spreads_27_3_starter:
18444 ▷ $(ORBITER) -v 10 \
18445 ▷ ▷ -define Control -poset_classification_control \
18446 ▷ ▷ ▷ -draw_options \
18447 ▷ ▷ ▷ ▷ -radius 20 \
18448 ▷ ▷ ▷ ▷ -nodes_empty \

```

```

18449 ▷ ▷ ▷ ▷ -line_width 0.2 \
18450 ▷ ▷ ▷ ▷ -embedded \
18451 ▷ ▷ ▷ -end \
18452 ▷ ▷ ▷ -W \
18453 ▷ ▷ ▷ -problem_label spreads_27_3 \
18454 ▷ ▷ -end \
18455 ▷ ▷ -define F -finite_field -q 3 -end \
18456 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
18457 ▷ ▷ -define C -spread_classifier \
18458 ▷ ▷ ▷ -projective_space P \
18459 ▷ ▷ ▷ -poset_classification_control Control \
18460 ▷ ▷ ▷ -k 3 \
18461 ▷ ▷ ▷ -starter_size 5 \
18462 ▷ ▷ ▷ -recoordinatize \
18463 ▷ ▷ ▷ -output_prefix "." \
18464 ▷ ▷ -end \
18465 ▷ ▷ -with C -do -spread_classify_activity \
18466 ▷ ▷ ▷ -compute_starter \
18467 ▷ ▷ -end \
18468 ▷ ▷ -with C -do -spread_classify_activity \
18469 ▷ ▷ ▷ -prepare_lifting_single_case 0 \
18470 ▷ ▷ -end
18471 ▷ #pdflatex spreads_27_3.poset_detailed.lvl5.tex
18472 ▷ #$(OPEN) spreads_27_3.poset_detailed.lvl5.pdf
18473
18474 # 50 orbits at level 5:
18475 #5 : 50 orbits
18476 #total: 60
18477 #(39^2, 26^2, 10, 6^2, 5, 3^9, 2^9, 1^{24}) average is 4 + 26 / 50
18478
18479 # time 4:31
18480
18481
18482
18483 classify_spreads_27_starter_lift_all_cases:
18484 ▷ $(ORBITER) -v 3 \
18485 ▷ ▷ -define Control -poset_classification_control \
18486 ▷ ▷ ▷ -draw_options \
18487 ▷ ▷ ▷ ▷ -radius 20 \
18488 ▷ ▷ ▷ ▷ -nodes_empty \
18489 ▷ ▷ ▷ ▷ -line_width 0.2 \
18490 ▷ ▷ ▷ ▷ -embedded \
18491 ▷ ▷ ▷ -end \
18492 ▷ ▷ ▷ -W \
18493 ▷ ▷ ▷ -problem_label spreads_27_3 \
18494 ▷ ▷ -end \
18495 ▷ ▷ -define F -finite_field -q 3 -end \
18496 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
18497 ▷ ▷ -define C -spread_classifier \
18498 ▷ ▷ ▷ -projective_space P \
18499 ▷ ▷ ▷ -k 3 \
18500 ▷ ▷ ▷ -starter_size 5 \
18501 ▷ ▷ ▷ -recoordinatize \
18502 ▷ ▷ ▷ -poset_classification_control Control \
18503 ▷ ▷ ▷ -output_prefix "" \
18504 ▷ ▷ -end \
18505 ▷ ▷ -with C -do -spread_classify_activity \
18506 ▷ ▷ ▷ -compute_starter \
18507 ▷ ▷ -end \

```

```

18508 ▷ ▷ -with C -do -spread_classify_activity \
18509 ▷ ▷ ▷ -prepare_lifting_all_cases \
18510 ▷ ▷ -end
18511
18512
18513 # 50 graphs
18514
18515
18516
18517 spreads_27_starter_cliques:
18518 ▷ $(ORBITER) -v 2 \
18519 ▷ ▷ -loop L 0 50 1 \
18520 ▷ ▷ ▷ -define G -graph -load spreads_27_3_graph_%L.bin -end \
18521 ▷ ▷ ▷ -with G -do \
18522 ▷ ▷ ▷ -graph_theoretic_activity \
18523 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 23 -end \
18524 ▷ ▷ ▷ -end \
18525 ▷ ▷ -end_loop L
18526
18527 #4:41
18528
18529
18530
18531 classify_spreads_27_isomorph_and_recognize:
18532 ▷ $(ORBITER) -v 3 \
18533 ▷ ▷ -define Control -poset_classification_control \
18534 ▷ ▷ ▷ -draw_options \
18535 ▷ ▷ ▷ ▷ -radius 20 \
18536 ▷ ▷ ▷ ▷ -nodes_empty \
18537 ▷ ▷ ▷ ▷ -line_width 0.2 \
18538 ▷ ▷ ▷ ▷ -embedded \
18539 ▷ ▷ ▷ -end \
18540 ▷ ▷ ▷ -W \
18541 ▷ ▷ ▷ -problem_label spreads_27_3 \
18542 ▷ ▷ -end \
18543 ▷ ▷ -define F -finite_field -q 3 -end \
18544 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
18545 ▷ ▷ -define C -spread_classifier \
18546 ▷ ▷ ▷ -projective_space P \
18547 ▷ ▷ ▷ -k 3 \
18548 ▷ ▷ ▷ -starter_size 5 \
18549 ▷ ▷ ▷ -recoordinatize \
18550 ▷ ▷ ▷ -poset_classification_control Control \
18551 ▷ ▷ ▷ -output_prefix "" \
18552 ▷ ▷ -end \
18553 ▷ ▷ -with C -do -spread_classify_activity \
18554 ▷ ▷ ▷ -compute_starter \
18555 ▷ ▷ -end \
18556 ▷ ▷ -with C -do -spread_classify_activity \
18557 ▷ ▷ ▷ -isomorph \
18558 ▷ ▷ ▷ ▷ -prefix_iso "../spreads_27_3" \
18559 ▷ ▷ ▷ ▷ -use_database_for_starter \
18560 ▷ ▷ ▷ ▷ -build_db \
18561 ▷ ▷ ▷ ▷ -solution_prefix "" \
18562 ▷ ▷ ▷ ▷ -base_fname "" \
18563 ▷ ▷ ▷ -end \
18564 ▷ ▷ -end \
18565 ▷ ▷ -with C -do -spread_classify_activity \
18566 ▷ ▷ ▷ -isomorph \

```

```

18567 ▷ ▷ ▷ ▷ -prefix_iso "./spreads.27.3" \
18568 ▷ ▷ ▷ ▷ -use_database_for_starter \
18569 ▷ ▷ ▷ ▷ -read_solutions \
18570 ▷ ▷ ▷ ▷ -solution_prefix "" \
18571 ▷ ▷ ▷ ▷ -base_fname "spreads.27.3_graph" \
18572 ▷ ▷ ▷ -end \
18573 ▷ ▷ -end \
18574 ▷ ▷ -with C -do -spread_classify_activity \
18575 ▷ ▷ ▷ -isomorph \
18576 ▷ ▷ ▷ ▷ -prefix_iso "./spreads.27.3" \
18577 ▷ ▷ ▷ ▷ -use_database_for_starter \
18578 ▷ ▷ ▷ ▷ -compute_orbits \
18579 ▷ ▷ ▷ ▷ -solution_prefix "" \
18580 ▷ ▷ ▷ ▷ -base_fname "spreads.27.3_graph" \
18581 ▷ ▷ ▷ -end \
18582 ▷ ▷ -end \
18583 ▷ ▷ -with C -do -spread_classify_activity \
18584 ▷ ▷ ▷ -isomorph \
18585 ▷ ▷ ▷ ▷ -prefix_iso "./spreads.27.3" \
18586 ▷ ▷ ▷ ▷ -use_database_for_starter \
18587 ▷ ▷ ▷ ▷ -isomorph_testing \
18588 ▷ ▷ ▷ ▷ -solution_prefix "" \
18589 ▷ ▷ ▷ ▷ -base_fname "spreads.27.3_graph" \
18590 ▷ ▷ ▷ -end \
18591 ▷ ▷ -end \
18592 ▷ ▷ -with C -do -spread_classify_activity \
18593 ▷ ▷ ▷ -isomorph \
18594 ▷ ▷ ▷ ▷ -prefix_iso "./spreads.27.3" \
18595 ▷ ▷ ▷ ▷ -use_database_for_starter \
18596 ▷ ▷ ▷ ▷ -isomorph_report \
18597 ▷ ▷ ▷ ▷ -solution_prefix "" \
18598 ▷ ▷ ▷ ▷ -base_fname "spreads.27.3_graph" \
18599 ▷ ▷ ▷ -end \
18600 ▷ ▷ -end \
18601 ▷ ▷ -with C -do -spread_classify_activity \
18602 ▷ ▷ ▷ -isomorph \
18603 ▷ ▷ ▷ ▷ -prefix_iso "./spreads.27.3" \
18604 ▷ ▷ ▷ ▷ -use_database_for_starter \
18605 ▷ ▷ ▷ ▷ -recognize $(SPREAD_S27_RA0_RA0) \
18606 ▷ ▷ ▷ ▷ -solution_prefix "" \
18607 ▷ ▷ ▷ ▷ -base_fname "spreads.27.3_graph" \
18608 ▷ ▷ ▷ -end \
18609 ▷ ▷ -end
18610 ▷ #pdflatex spreads_27_isomorphism_types.tex
18611 ▷ #$(OPEN) spreads_27_isomorphism_types.pdf
18612 ▷ #pdflatex spreads_27_aut_group.tex
18613 ▷ #$(OPEN) spreads_27_aut_group.pdf
18614
18615
18616 # SPREAD_S27_RA0_RA0 is isomorphic to spread 0 in the list
18617 # (which is different from the ordering of the Orbiter catalogue)
18618 # the stabilizer of the spread has order 84.
18619
18620
18621 #substructure_lifting_data::write_hash_and_datref_file id.to.hash tallied:
18622 #( 1^6076, 2^289, 3^35, 4^5 )
18623 # using 64 bit hash values, based on a modified version of Paul Hsieh's SuperFast
Hash
18624

```



```

18625 #We found 7 isomorphism types
18626 #0:36
18627
18628 #generators for the stabilizer of the Rao/Rao spread:
18629 #1,2,2,2,0,2,1,1,2,2,2,1,2,1,2,1,1,0,2,2,0,2,0,0,1,1,2,2,2,0,0,1,1,2,0,1, #1,0,1,
    0,1,2,0,1,2,2,2,2,1,1,1,0,1,1,2,0,1,1,2,1,1,1,0,2,2,2,0,2,0,1,1,0,
18630
18631
18632
18633 create_spread.27.0:
18634 ▷ $(ORBITER) -v 3 \
18635 ▷ ▷ -define F -finite_field -q 3 -end \
18636 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18637 ▷ ▷ -define S -spread -kernel_field F \
18638 ▷ ▷ ▷ -group G -k 3 -catalogue 0 \
18639 ▷ ▷ ▷ -end \
18640 ▷ ▷ -with S -do -spread_activity \
18641 ▷ ▷ ▷ -report \
18642 ▷ ▷ -end
18643 ▷ pdflatex catalogue_q3_k3_0_report.tex
18644 ▷ $(OPEN) catalogue_q3_k3_0_report.pdf
18645
18646 create_spread.27.1:
18647 ▷ $(ORBITER) -v 3 \
18648 ▷ ▷ -define F -finite_field -q 3 -end \
18649 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18650 ▷ ▷ -define S -spread -kernel_field F \
18651 ▷ ▷ ▷ -group G -k 3 -catalogue 1 \
18652 ▷ ▷ ▷ -end \
18653 ▷ ▷ -with S -do -spread_activity \
18654 ▷ ▷ ▷ -report \
18655 ▷ ▷ -end
18656 ▷ pdflatex catalogue_q3_k3_1_report.tex
18657 ▷ $(OPEN) catalogue_q3_k3_1_report.pdf
18658
18659 create_spread.27.2:
18660 ▷ $(ORBITER) -v 3 \
18661 ▷ ▷ -define F -finite_field -q 3 -end \
18662 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18663 ▷ ▷ -define S -spread -kernel_field F \
18664 ▷ ▷ ▷ -group G -k 3 -catalogue 2 \
18665 ▷ ▷ ▷ -end \
18666 ▷ ▷ -with S -do -spread_activity \
18667 ▷ ▷ ▷ -report \
18668 ▷ ▷ -end
18669 ▷ pdflatex catalogue_q3_k3_2_report.tex
18670 ▷ $(OPEN) catalogue_q3_k3_2_report.pdf
18671
18672 create_spread.27.3:
18673 ▷ $(ORBITER) -v 3 \
18674 ▷ ▷ -define F -finite_field -q 3 -end \
18675 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18676 ▷ ▷ -define S -spread -kernel_field F \
18677 ▷ ▷ ▷ -group G -k 3 -catalogue 3 \
18678 ▷ ▷ ▷ -end \
18679 ▷ ▷ -with S -do -spread_activity \
18680 ▷ ▷ ▷ -report \
18681 ▷ ▷ -end
18682 ▷ pdflatex catalogue_q3_k3_3_report.tex

```

```

18683 ▷ $(OPEN) catalogue_q3_k3_3.report.pdf
18684
18685 create_spread_27.4:
18686 ▷ $(ORBITER) -v 3 \
18687 ▷ ▷ -define F -finite_field -q 3 -end \
18688 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18689 ▷ ▷ -define S -spread -kernel_field F \
18690 ▷ ▷ ▷ -group G -k 3 -catalogue 4 \
18691 ▷ ▷ ▷ -end \
18692 ▷ ▷ -with S -do -spread_activity \
18693 ▷ ▷ ▷ -report \
18694 ▷ ▷ -end
18695 ▷ pdflatex catalogue_q3_k3_4.report.tex
18696 ▷ $(OPEN) catalogue_q3_k3_4.report.pdf
18697
18698 create_spread_27.5:
18699 ▷ $(ORBITER) -v 3 \
18700 ▷ ▷ -define F -finite_field -q 3 -end \
18701 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18702 ▷ ▷ -define S -spread -kernel_field F \
18703 ▷ ▷ ▷ -group G -k 3 -catalogue 5 \
18704 ▷ ▷ ▷ -end \
18705 ▷ ▷ -with S -do -spread_activity \
18706 ▷ ▷ ▷ -report \
18707 ▷ ▷ -end
18708 ▷ pdflatex catalogue_q3_k3_5.report.tex
18709 ▷ $(OPEN) catalogue_q3_k3_5.report.pdf
18710
18711 create_spread_27.6:
18712 ▷ $(ORBITER) -v 3 \
18713 ▷ ▷ -define F -finite_field -q 3 -end \
18714 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18715 ▷ ▷ -define S -spread -kernel_field F \
18716 ▷ ▷ ▷ -group G -k 3 -catalogue 6 \
18717 ▷ ▷ ▷ -end \
18718 ▷ ▷ -with S -do -spread_activity \
18719 ▷ ▷ ▷ -report \
18720 ▷ ▷ -end
18721 ▷ pdflatex catalogue_q3_k3_6.report.tex
18722 ▷ $(OPEN) catalogue_q3_k3_6.report.pdf
18723
18724
18725
18726
18727
18728
18729
18730
18731
18732 #####
18733 # Section 14.2: Translation planes
18734
18735
18736 SECTION_TRANSLATION_PLANES:
18737
18738
18739 test_14.2:
18740 ▷ make create_translation_plane.9b
18741 ▷ make create_translation_plane.16.4.0

```

```

18742 ▷ make create_translation_plane_16_2_0
18743 ▷ make RREF_plane_16_2_0_rank_of_incma
18744 ▷ make create_translation_plane_25_14_rank
18745 ▷ make create_translation_plane_27_Rao_Rao
18746 ▷ make RREF_Rao_Rao_plane_incma_rank
18747 ▷ make create_translation_plane_27_p_rank_of_incidence_matrix
18748 ▷ make create_translation_plane_27_5_Rao_Rao
18749 ▷ make translation_plane_27_4_orbits
18750 ▷ make translation_plane_27_5_orbits
18751 ▷ make translation_plane_27_6_orbits
18752
18753
18754
18755
18756 create_translation_plane_9b:
18757 ▷ $(ORBITER) -v 3 \
18758 ▷ ▷ -define F -finite_field -q 3 -end \
18759 ▷ ▷ -define G -linear_group -PGL 4 F -end \
18760 ▷ ▷ -define G1 -linear_group -PGL 5 F -end \
18761 ▷ ▷ -define S -spread -kernel_field F \
18762 ▷ ▷ ▷ -group G -k 2 -catalogue 1 \
18763 ▷ ▷ ▷ -end \
18764 ▷ ▷ -define T -translation_plane S G G1 -end \
18765 ▷ ▷ -with T -do -translation_plane_activity \
18766 ▷ ▷ ▷ -export_incma \
18767 ▷ ▷ -end \
18768 ▷ ▷ -with T -do -translation_plane_activity \
18769 ▷ ▷ ▷ -report \
18770 ▷ ▷ -end \
18771 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
18772 ▷ ▷ -define Orb -orbits -group A \
18773 ▷ ▷ ▷ -on_points \
18774 ▷ ▷ -end \
18775 ▷ ▷ -with Orb -do -orbits_activity \
18776 ▷ ▷ ▷ -report \
18777 ▷ ▷ -end \
18778 ▷ ▷ -with Orb -do -orbits_activity \
18779 ▷ ▷ ▷ -stabilizer 92 \
18780 ▷ ▷ -end \
18781 ▷ ▷ -with Orb -do -orbits_activity \
18782 ▷ ▷ ▷ -export_trees \
18783 ▷ ▷ -end
18784 ▷ $(ORBITER) -v 2 \
18785 ▷ ▷ -draw_matrix \
18786 ▷ ▷ ▷ -input_csv_file plane_catalogue_q3_k2_1_incma.csv \
18787 ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
18788 ▷ ▷ ▷ -partition 2 91 91 \
18789 ▷ ▷ -end
18790 ▷ $(ORBITER) -v 3 \
18791 ▷ ▷ -draw_layered_graph \
18792 ▷ ▷ ▷ orbit_PGL_5_3_on_andre_3_layered_graph \
18793 ▷ ▷ ▷ -radius 250 -spanning_tree -embedded -nodes_empty \
18794 ▷ ▷ ▷ -line_width 1.1 -x_stretch 2.4 -scale 0.15 \
18795 ▷ ▷ -end
18796 ▷ pdflatex orbit_PGL_5_3_on_andre_3_draw.tex
18797 ▷ $(OPEN) orbit_PGL_5_3_on_andre_3_draw.pdf
18798 ▷ pdflatex group_of_plane_plane_catalogue_q3_k2_1_orbits_report.tex
18799 ▷ $(OPEN) group_of_plane_plane_catalogue_q3_k2_1_orbits_report.pdf
18800 ▷ pdflatex group_of_plane_plane_catalogue_q3_k2_1_stab_pt_92_report.tex

```

```

18801 ▷ $(OPEN) group_of_plane_plane_catalogue_q3_k2_1_stab_pt_92_report.pdf
18802
18803
18804
18805
18806 create_translation_plane_16_4_0:
18807 ▷ $(ORBITER) -v 3 \
18808 ▷ ▷ -define F -finite_field -q 4 -end \
18809 ▷ ▷ -define G -linear_group -PGGL 4 F -end \
18810 ▷ ▷ -define G1 -linear_group -PGGL 5 F -end \
18811 ▷ ▷ -define S -spread -kernel_field F \
18812 ▷ ▷ ▷ -group G -k 2 -catalogue 0 \
18813 ▷ ▷ ▷ -end \
18814 ▷ ▷ -define T -translation_plane S G G1 -end \
18815 ▷ ▷ -with T -do -translation_plane_activity \
18816 ▷ ▷ ▷ -export_incma \
18817 ▷ ▷ -end
18818 ▷ $(ORBITER) -v 2 \
18819 ▷ ▷ -draw_matrix \
18820 ▷ ▷ ▷ -input_csv_file plane_catalogue_q4_k2_0_incma.csv \
18821 ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
18822 ▷ ▷ ▷ -partition 4 273 273 \
18823 ▷ ▷ -end
18824 ▷ $(OPEN) plane_catalogue_q4_k2_0_incma_draw.bmp
18825
18826
18827 #0 : "1200", // Hall spread
18828 #1 : "81600", // Desarguesian spread
18829 #2 : "576", // Semifield spread
18830
18831
18832 create_translation_plane_16_2_0:
18833 ▷ $(ORBITER) -v 3 \
18834 ▷ ▷ -define F -finite_field -q 2 -end \
18835 ▷ ▷ -define G -linear_group -PGL 8 F -end \
18836 ▷ ▷ -define G1 -linear_group -PGL 9 F -end \
18837 ▷ ▷ -define S -spread -kernel_field F \
18838 ▷ ▷ ▷ -group G -k 4 -catalogue 0 \
18839 ▷ ▷ ▷ -end \
18840 ▷ ▷ -define T -translation_plane S G G1 -end \
18841 ▷ ▷ -with T -do -translation_plane_activity \
18842 ▷ ▷ ▷ -export_incma \
18843 ▷ ▷ -end
18844 ▷ $(ORBITER) -v 2 \
18845 ▷ ▷ -draw_matrix \
18846 ▷ ▷ ▷ -input_csv_file plane_catalogue_q2_k4_0_incma.csv \
18847 ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
18848 ▷ ▷ ▷ -partition 4 273 273 \
18849 ▷ ▷ -end
18850 ▷ $(OPEN) plane_catalogue_q2_k4_0_incma_draw.bmp
18851
18852 #0 : "1008",
18853 #1 : "1008",
18854 #2 : "1728",
18855 #3 : "216",
18856 #4 : "360",
18857 #5 : "288",
18858 #6 : "3600",
18859 #7 : "244800",

```

```

18860
18861 RREF_plane.16.2.0_rank.of_incma:
18862 ▷ $(ORBITER) -v 2 \
18863 ▷ ▷ -define F -finite_field -q 2 -end \
18864 ▷ ▷ -define v -vector -field F \
18865 ▷ ▷ ▷ -file plane_catalogue.q2.k4.0_incma.csv \
18866 ▷ ▷ -end \
18867 ▷ ▷ -with F -do -finite_field_activity \
18868 ▷ ▷ -RREF v \
18869 ▷ ▷ -end
18870
18871 # 2-rank is 106, so the plane is Lorimer-Rahilly
18872
18873
18874 create_translation_plane.25.14_rank:
18875 ▷ $(ORBITER) -v 3 \
18876 ▷ ▷ -define F -finite_field -q 5 -end \
18877 ▷ ▷ -define G -linear_group -PGL 4 F -end \
18878 ▷ ▷ -define G1 -linear_group -PGL 5 F -end \
18879 ▷ ▷ -define S -spread -kernel_field F \
18880 ▷ ▷ ▷ -group G -k 2 -catalogue 14 \
18881 ▷ ▷ ▷ -end \
18882 ▷ ▷ -define T -translation_plane S G G1 -end \
18883 ▷ ▷ -with T -do -translation_plane_activity \
18884 ▷ ▷ ▷ -export_incma \
18885 ▷ ▷ -end
18886 ▷ $(ORBITER) -v 2 \
18887 ▷ ▷ -define F -finite_field -q 2 -end \
18888 ▷ ▷ -define v -vector -field F \
18889 ▷ ▷ ▷ -file plane_catalogue.q5.k2.14_incma.csv \
18890 ▷ ▷ -end \
18891 ▷ ▷ -with F -do -finite_field_activity \
18892 ▷ ▷ ▷ -RREF v \
18893 ▷ ▷ -end
18894
18895
18896
18897 create_translation_plane.27_Rao_Rao:
18898 ▷ $(ORBITER) -v 3 \
18899 ▷ ▷ -define F -finite_field -q 3 -end \
18900 ▷ ▷ -define SS -vector -dense $(SPREAD_SET_27_RA0_RA0) -end \
18901 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18902 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
18903 ▷ ▷ -define S -spread -kernel_field F \
18904 ▷ ▷ ▷ -group G -k 3 -spread_set "Rao_Rao" "Rao\_Rao" SS \
18905 ▷ ▷ ▷ -end \
18906 ▷ ▷ -define T -translation_plane S G G1 -end \
18907 ▷ ▷ -with T -do -translation_plane_activity \
18908 ▷ ▷ ▷ -export_incma \
18909 ▷ ▷ -end
18910
18911
18912 # creates plane_Rao_Rao_incma.csv
18913
18914
18915 RREF_Rao_Rao_plane_incma_rank:
18916 ▷ $(ORBITER) -v 2 \
18917 ▷ ▷ -define F -finite_field -q 3 -end \
18918 ▷ ▷ -define v -vector -field F \

```

```

18919 ▷ ▷ ▷ -file plane_Rao_Rao_incma.csv \
18920 ▷ ▷ -end \
18921 ▷ ▷ -with F -do -finite_field_activity \
18922 ▷ ▷ ▷ -RREF v \
18923 ▷ ▷ -end
18924
18925 # 3-rank is 271, so the Rao / Rao plane is Moorhouse IV.
18926
18927
18928 create_translation_plane_27_p_rank_of_incidence_matrix:
18929 ▷ $(ORBITER) -v 3 \
18930 ▷ ▷ -define F -finite_field -q 3 -end \
18931 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18932 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
18933 ▷ ▷ -define S -spread -kernel_field F \
18934 ▷ ▷ ▷ -group G -k 3 -catalogue 6 \
18935 ▷ ▷ ▷ -end \
18936 ▷ ▷ -define T -translation_plane S G G1 -end \
18937 ▷ ▷ -with T -do -translation_plane_activity \
18938 ▷ ▷ ▷ -p_rank 3 \
18939 ▷ ▷ -end
18940
18941 # OCN : 3-rank : spread stab : pt-orb : line-orb : Moorhouse list
18942 # 0 : 217 : 766584 : 757 : 757 : flag trans : desarguesian = Moorhouse I
18943 # 1 : 262 : 2106 : 1,27,729 : 1,27,729 : generalized twisted field = Moorhouse
    II
18944 # 2 : 268 : 1014 : 2,26,729 : 1,54,702 : Andre = Moorhouse VII
18945 # 3 : 273 : 81 : 1,27,729 : 1,27,729 : Sherk = Moorhouse V
18946 # 4 : 274 : 1092 : 28,729 : 1,756 : Hering = Moorhouse III
18947 # 5 : 271 : 84 : 28,729 : 1,756 : flag trans : Moorhouse IV = Rao Rao
18948 # 6 : 265 : 84 : 28,729 : 1,756 : flag trans : Moorhouse VI
18949
18950 # so, Rao / Rao is OCN=5
18951
18952
18953
18954 create_translation_plane_27_5-Rao-Rao:
18955 ▷ $(ORBITER) -v 3 \
18956 ▷ ▷ -define F -finite_field -q 3 -end \
18957 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18958 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
18959 ▷ ▷ -define S -spread -kernel_field F \
18960 ▷ ▷ ▷ -group G -k 3 -catalogue 5 \
18961 ▷ ▷ ▷ -end \
18962 ▷ ▷ -define T -translation_plane S G G1 -end \
18963 ▷ ▷ -with T -do -translation_plane_activity \
18964 ▷ ▷ ▷ -export_incma \
18965 ▷ ▷ -end \
18966 ▷ ▷ -with T -do -translation_plane_activity \
18967 ▷ ▷ ▷ -report \
18968 ▷ ▷ -end \
18969 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
18970 ▷ ▷ -define Orb -orbits -group A \
18971 ▷ ▷ ▷ -on_points \
18972 ▷ ▷ -end
18973 ▷ pdflatex plane_catalogue_q3_k3_5_report.tex
18974 ▷ $(OPEN) plane_catalogue_q3_k3_5_report.pdf
18975
18976

```

```

18977
18978
18979 translation_plane_27.4_orbits:
18980 ▷ $(ORBITER) -v 3 \
18981 ▷ ▷ -define F -finite_field -q 3 -end \
18982 ▷ ▷ -define G -linear_group -PGL 6 F -end \
18983 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
18984 ▷ ▷ -define S -spread -kernel_field F \
18985 ▷ ▷ ▷ -group G -k 3 -catalogue 4 \
18986 ▷ ▷ ▷ -end \
18987 ▷ ▷ -define T -translation_plane S G G1 -end \
18988 ▷ ▷ -with T -do -translation_plane_activity \
18989 ▷ ▷ ▷ -export_incma \
18990 ▷ ▷ -end \
18991 ▷ ▷ -with T -do -translation_plane_activity \
18992 ▷ ▷ ▷ -report \
18993 ▷ ▷ -end \
18994 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
18995 ▷ ▷ -define Orb -orbits -group A \
18996 ▷ ▷ ▷ -on_points \
18997 ▷ ▷ -end
18998
18999
19000
19001 translation_plane_27.5_orbits:
19002 ▷ $(ORBITER) -v 3 \
19003 ▷ ▷ -define F -finite_field -q 3 -end \
19004 ▷ ▷ -define G -linear_group -PGL 6 F -end \
19005 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
19006 ▷ ▷ -define S -spread -kernel_field F \
19007 ▷ ▷ ▷ -group G -k 3 -catalogue 5 \
19008 ▷ ▷ ▷ -end \
19009 ▷ ▷ -define T -translation_plane S G G1 -end \
19010 ▷ ▷ -with T -do -translation_plane_activity \
19011 ▷ ▷ ▷ -export_incma \
19012 ▷ ▷ -end \
19013 ▷ ▷ -with T -do -translation_plane_activity \
19014 ▷ ▷ ▷ -report \
19015 ▷ ▷ -end \
19016 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
19017 ▷ ▷ -define Orb -orbits -group A \
19018 ▷ ▷ ▷ -on_points \
19019 ▷ ▷ -end
19020
19021
19022 translation_plane_27.6_orbits:
19023 ▷ $(ORBITER) -v 3 \
19024 ▷ ▷ -define F -finite_field -q 3 -end \
19025 ▷ ▷ -define G -linear_group -PGL 6 F -end \
19026 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
19027 ▷ ▷ -define S -spread -kernel_field F \
19028 ▷ ▷ ▷ -group G -k 3 -catalogue 6 \
19029 ▷ ▷ ▷ -end \
19030 ▷ ▷ -define T -translation_plane S G G1 -end \
19031 ▷ ▷ -with T -do -translation_plane_activity \
19032 ▷ ▷ ▷ -export_incma \
19033 ▷ ▷ -end \
19034 ▷ ▷ -with T -do -translation_plane_activity \
19035 ▷ ▷ ▷ -report \

```

```

19036 ▷ ▷ -end \
19037 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
19038 ▷ ▷ -define Orb -orbits -group A \
19039 ▷ ▷ ▷ -on_points \
19040 ▷ ▷ -end
19041
19042
19043
19044
19045
19046
19047
19048 #####
19049 # Section 14.3: Packings
19050
19051
19052 SECTION_PACKINGS:
19053
19054
19055 test_14.3:
19056 ▷ #make spread_table_PG_3.4 # slow
19057 ▷ make spread_table_PG_3.5.regular
19058 ▷ make PG_3.5.element_of_order_31.GL.normalizer
19059 ▷ make PG_3.5.element_of_order_31.ME.normalizer
19060 ▷ make PG_3.5.assume_31_graph
19061
19062
19063
19064
19065 spread_table_PG_3.4:
19066 ▷ - mkdir SPREAD_TABLES_4
19067 ▷ $(ORBITER) -v 6 \
19068 ▷ ▷ -define Control -poset_classification_control \
19069 ▷ ▷ ▷ -problem_label spreads_PG_3.4 \
19070 ▷ ▷ -end \
19071 ▷ ▷ -define F -finite_field -q 4 -end \
19072 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
19073 ▷ ▷ -define T -spread_table P 2 "0,1,2" "SPREAD_TABLES_4/" Control
19074
19075
19076 # 5096448 spreads
19077 # 1020 self dual spreads
19078 # User time: 56:38 on Mac
19079 # User time: 17:53 on Mac 1/11/2024
19080
19081 #spread.tables::save writing file SPREAD_TABLES_4/spread_16_spreads.csv
19082 #spread.tables::save written file SPREAD_TABLES_4/spread_16_spreads.csv
19083 #spread.tables::save, writing file SPREAD_TABLES_4/spread_16_spreads_iso.csv
19084 #spread.tables::save, written file SPREAD_TABLES_4/spread_16_spreads_iso.csv
19085 #spread.tables::save, writing file SPREAD_TABLES_4/spread_16_dual_spread_idx.csv
19086 #spread.tables::save, written file SPREAD_TABLES_4/spread_16_dual_spread_idx.csv
19087 #spread.tables::save, writing file SPREAD_TABLES_4/spread_16_self_dual_spreads.cs
19088 #spread.tables::save, written file SPREAD_TABLES_4/spread_16_self_dual_spreads.cs
19089 #spread.tables::save writing file SPREAD_TABLES_4/spread_16_schreier_table.csv
19090 #spread.tables::save written file SPREAD_TABLES_4/spread_16_schreier_table.csv
19091
19092

```



```

19093
19094 spread_table_PG_3_5_regular:
19095 ▷ - mkdir SPREAD_TABLES_5_REG
19096 ▷ $(ORBITER) -v 6 \
19097 ▷ ▷ -define Control -poset_classification_control \
19098 ▷ ▷ ▷ -problem_label spreads_PG_3_5 \
19099 ▷ ▷ -end \
19100 ▷ ▷ -define F -finite_field -q 5 -end \
19101 ▷ ▷ -define P -projective_space -n 3 -field F -end \
19102 ▷ ▷ -define T -spread_table P 2 "12" "SPREAD_TABLES_5_REG/" Control \
19103 ▷ ▷ -print_symbols
19104
19105
19106 # 21 isomorphism types of spreads in PG(3,5)
19107 # 12 is the index of the regular spread in the classification of spreads
19108 # 12/9/2020: 34 sec on Mac
19109 # time: 0:19 1/11/2024 on Mac
19110 # 155000 spreads
19111
19112 #spread_tables::save writing file SPREAD_TABLES_5_REG/spread_25_spreads.csv
19113 #spread_tables::save written file SPREAD_TABLES_5_REG/spread_25_spreads.csv
19114 #spread_tables::save, writing file SPREAD_TABLES_5_REG/spread_25_spreads_iso.csv
19115 #spread_tables::save, written file SPREAD_TABLES_5_REG/spread_25_spreads_iso.csv
19116 #spread_tables::save, writing file SPREAD_TABLES_5_REG/spread_25_dual_spread.idx.
    csv
19117 #spread_tables::save, written file SPREAD_TABLES_5_REG/spread_25_dual_spread.idx.
    csv
19118 #spread_tables::save, writing file SPREAD_TABLES_5_REG/spread_25_self_dual_spread
    s.csv
19119 #spread_tables::save, written file SPREAD_TABLES_5_REG/spread_25_self_dual_spread
    s.csv
19120 #spread_tables::save writing file SPREAD_TABLES_5_REG/spread_25_schreier_table.cs
    v
19121 #spread_tables::save written file SPREAD_TABLES_5_REG/spread_25_schreier_table.cs
    v
19122
19123
19124 PG_3_5_element_of_order_31_GL_normalizer:
19125 ▷ $(ORBITER) -v 6 -define G \
19126 ▷ ▷ -linear_group -GL 4 5 -end \
19127 ▷ ▷ -with G -do \
19128 ▷ ▷ -group_theoretic_activity \
19129 ▷ ▷ ▷ -normalizer_of_cyclic_subgroup "124" \
19130 ▷ ▷ ▷ "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3" \
19131 ▷ ▷ -end
19132 ▷ pdflatex normalizer_of_124_in_GL_4_5.tex
19133 ▷ $(OPEN) normalizer_of_124_in_GL_4_5.pdf
19134
19135 # needs magma
19136 # the group has order 124.
19137 # the normalizer has order 1488
19138 # normalizer has order 1488=4*372=4*4*3*31
19139
19140 PG_3_5_element_of_order_31_ME_normalizer:
19141 ▷ $(ORBITER) -v 6 -define G \
19142 ▷ ▷ -linear_group -PGL 4 5 -end \
19143 ▷ ▷ -with G -do \
19144 ▷ ▷ -group_theoretic_activity \
19145 ▷ ▷ ▷ -normalizer_of_cyclic_subgroup "31" \

```

```

19146 ▷ ▷ ▷ "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3" \
19147 ▷ ▷ -end
19148 ▷ mv normalizer_of_31_in_PGL_4.5.tex normalizer_of_31_ME_in_PGL_4.5.tex
19149 ▷ pdflatex normalizer_of_31_ME_in_PGL_4.5.tex
19150 ▷ $(OPEN) normalizer_of_31_ME_in_PGL_4.5.pdf
19151
19152
19153 # group has order 31
19154 # normalizer has order 372
19155
19156
19157
19158 PG_3_5_assume_31_graph:
19159 ▷ $(ORBITER) -v 5 \
19160 ▷ ▷ -define Control -poset_classification_control \
19161 ▷ ▷ ▷ -problem_label spreads_PG_3_5 \
19162 ▷ ▷ -end \
19163 ▷ ▷ -define F -finite_field -q 5 -end \
19164 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
19165 ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
19166 ▷ ▷ -define T -spread_table P3 2 "12" "SPREAD_TABLES_5_REG/" Control \
19167 ▷ ▷ -define PC -packing_classify P3 P5 T \
19168 ▷ ▷ -define PW -packing_with_symmetry_assumption PC \
19169 ▷ ▷ ▷ -H "H31" $(PGL_4_5_SUBGROUP_31_ME) -end \
19170 ▷ ▷ ▷ -N "N31" $(PGL_4_5_SUBGROUP_31_ME_NORMALIZER) -end \
19171 ▷ ▷ ▷ -end \
19172 ▷ ▷ -define PWF -packing_choose_fixed_points PW 0 -end \
19173 ▷ ▷ -define L -packing_long_orbits PWF \
19174 ▷ ▷ ▷ -orbit_length 31 -create_graphs \
19175 ▷ ▷ ▷ -end \
19176 ▷ ▷ -print_symbols
19177
19178
19179 no:
19180 ▷ pdflatex H31_reduced_spread_orbits_orbits_report.tex
19181 ▷ $(OPEN) H31_reduced_spread_orbits_orbits_report.pdf
19182 ▷ pdflatex H31_line_orbits_orbits_report.tex
19183 ▷ $(OPEN) H31_line_orbits_orbits_report.pdf
19184 ▷ pdflatex H31_line_orbits_orbits_report.tex
19185 ▷ $(OPEN) H31_line_orbits_orbits_report.pdf
19186 ▷ pdflatex N31_line_orbits_orbits_report.tex
19187 ▷ $(OPEN) N31_line_orbits_orbits_report.pdf
19188 ▷ pdflatex H31_point_orbits_orbits_report.tex
19189 ▷ $(OPEN) H31_point_orbits_orbits_report.pdf
19190 ▷ pdflatex N31_point_orbits_orbits_report.tex
19191 ▷ $(OPEN) N31_point_orbits_orbits_report.pdf
19192
19193 ▷ #pdflatex H31_spread_orbits_orbits_report.tex
19194 ▷ #$(OPEN) H31_spread_orbits_orbits_report.pdf
19195 #H31_line_orbits_orbits.bin
19196 #H31_line_orbits_orbits_report.tex
19197 #H_spread_orbits_orbit_types_report.tex
19198 #H31_spread_orbits_orbits.bin
19199 #H31_good_orbits
19200 #H31_spread_types_reduced_orbit_types_report.tex
19201 #H31_reduced_spread_orbits_orbits.bin
19202 #H31_fpc0_lo.graph
19203
19204

```

```

19205 # number of points = 156
19206 # number of lines = 856
19207 #spread.classify::init The order of PGGL(4,5) is 29016000000
19208 #spread orbit 12 has group order 187200 orbit_length = 155000
19209 #spread.tables::init nb_lines=806
19210 #spread.tables::init spread.size=26
19211 #spread.tables::init nb_iso_types_of_spreads=21
19212 #spread.tables::init prefix=SPREAD.TABLES.5_REG/spread_25
19213 #packing.classify::init_P3_and_P5_and_Gr P3->N.points=156
19214 #packing.classify::init_P3_and_P5_and_Gr P3->N.lines=806
19215 #packing.classify::init_P3_and_P5_and_Gr P5->N.points=3906
19216 #packing.classify::init_P3_and_P5_and_Gr P5->N.lines=508431
19217 #-H H31
19218 #-PGL 4 5
19219 #-subgroup_by_generators 31 1 1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3
19220 #-N N31
19221 #-PGL 4 5
19222 #-subgroup_by_generators normalizer_31 4 1,0,0,0,0,4,0,0,0,0,4,0,0,0,0,4, 1,0,0,0
,0,3,0,0,0,0,3,0,0,0,0,3, 1,0,0,0,0,4,0,0,0,0,2,1,0,3,2,4, 1,0,0,0,0,0,1,0,0,0,0,
1,0,1,1,3,
19223 #packing_was::init_H H_goi=31
19224 #packing_was::init_N N_goi=372
19225 #orbits.on.something::init after reading orbits from file H31_point_orbits_orbits
.bin
19226 #orbits.on.something::init orbit length distribution:
19227 #1, 31^5
19228 #orbits.on.something::init after reading orbits from file N31_point_orbits_orbits
.bin
19229 #orbits.on.something::init orbit length distribution:
19230 #1, 31, 124
19231 #orbits.on.something::init after reading orbits from file H31_line_orbits_orbits.
bin
19232 #orbits.on.something::init orbit length distribution:
19233 #31^{26}
19234 #orbits.on.something::init after reading orbits from file H31_spread_orbits_orbit
s.bin
19235 #orbits.on.something::init orbit length distribution:
19236 #31^{5000}
19237 #packing_was::reduce_spreads nb_good_spreads = 248
19238 #packing_was::compute_H_orbits_on_lines after reduced_spread_orbits_under_H->crea
te_latex_report
19239 #orbits.on.something::classify_orbits_by_length
19240 #orbits.on.something::classify_orbits_by_length The distribution of orbit lengths
is: 31^8
19241 #orbits.on.something::init after reading orbits from file N31_line_orbits_orbits.
bin
19242 #orbits.on.something::init orbit length distribution:
19243 #31^2, 372^2
19244
19245
19246
19247 #packing_was::compute_reduced_spread_types_wrt_H
19248 #orbit.type_repository::init nb_sets = 248 goi=31
19249
19250
19251 #orbits.on.something::init after reading orbits from file H31_reduced_spread_orbi
ts_orbits.bin
19252 #orbits.on.something::init orbit length distribution:
19253 #31^8

```

```

19254
19255
19256 #orbits.on.something::init after reading orbits from file N31_line_orbits_orbits.
    bin
19257 #orbits.on.something::init orbit length distribution:
19258 #31^2, 372^2
19259
19260
19261 #####
19262 # Section 14.4: BLT-sets
19263
19264 SECTION.BLT_SETS:
19265
19266
19267 test_14_4:
19268 ▷ make BLT_5.1
19269 ▷ make BLT_5.1_export_gap
19270 ▷ make BLT_5.Linear
19271 ▷ make BLT_9.0
19272 ▷ make BLT_9.Kantor1
19273 ▷ make BLT_11.0
19274 ▷ make BLT_11.Fisher
19275 ▷ make BLT_11.Mondello
19276 ▷ make BLT_11.FTWKB
19277 ▷ make BLT_13.Kantor2
19278 ▷ make BLT_67.4
19279 ▷ make BLT_67.4_export_gap
19280 ▷ make BLT_9.deep_search
19281 ▷ make BLT_11.deep_search
19282 ▷ make BLT_13.deep_search
19283 ▷ make BLT_13.classify_starter
19284 ▷ make BLT_13.cliques
19285 ▷ make BLT_13.isomorph_read.DB
19286 ▷ make BLT_13.isomorph_read.solutions
19287 ▷ make BLT_13.isomorph_stabilizer_orbits
19288 ▷ make BLT_13.isomorph_testing
19289 ▷ make blt_set_export_gap_all
19290
19291
19292
19293 BLT_5.1:
19294 ▷ $(ORBITER) -v 10 \
19295 ▷ ▷ -define F -finite_field -q 5 -end \
19296 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19297 ▷ ▷ -define BLT -BLT_set \
19298 ▷ ▷ ▷ -space 0 -catalogue 1 -invariants \
19299 ▷ ▷ -end \
19300 ▷ ▷ -with BLT -do -blt_set_activity \
19301 ▷ ▷ ▷ -report \
19302 ▷ ▷ -end
19303 ▷ pdflatex BLT_catalogue.q5.iso1.tex
19304 ▷ $(OPEN) BLT_catalogue.q5.iso1.pdf
19305
19306
19307 BLT_5.1_export_gap:
19308 ▷ $(ORBITER) -v 2 \
19309 ▷ ▷ -define F -finite_field -q 5 -end \
19310 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19311 ▷ ▷ -define BLT -BLT_set \

```

```

19312 ▷ ▷ ▷ -space 0 -catalogue 1 \
19313 ▷ ▷ -end \
19314 ▷ ▷ -with BLT -do -blt_set_activity \
19315 ▷ ▷ ▷ -export_gap \
19316 ▷ ▷ -end
19317
19318
19319 BLT_5.Linear:
19320 ▷ $(ORBITER) -v 2 \
19321 ▷ ▷ -define F -finite_field -q 5 -end \
19322 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19323 ▷ ▷ -define BLT -BLT_set \
19324 ▷ ▷ ▷ -space 0 -family "Linear" -invariants \
19325 ▷ ▷ -end \
19326 ▷ ▷ -with BLT -do -blt_set_activity \
19327 ▷ ▷ ▷ -report \
19328 ▷ ▷ -end
19329 ▷ pdflatex BLT.Linear.q5.tex
19330 ▷ $(OPEN) BLT.Linear.q5.pdf
19331
19332
19333 BLT_9.0:
19334 ▷ $(ORBITER) -v 10 \
19335 ▷ ▷ -define F -finite_field -q 9 -end \
19336 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19337 ▷ ▷ -define BLT -BLT_set \
19338 ▷ ▷ ▷ -space 0 -catalogue 0 -invariants \
19339 ▷ ▷ -end \
19340 ▷ ▷ -with BLT -do -blt_set_activity \
19341 ▷ ▷ ▷ -report \
19342 ▷ ▷ -end
19343 ▷ pdflatex BLT.catalogue.q9_iso0.tex
19344 ▷ $(OPEN) BLT.catalogue.q9_iso0.pdf
19345
19346
19347
19348
19349 BLT_9.Kantor1:
19350 ▷ $(ORBITER) -v 2 \
19351 ▷ ▷ -define F -finite_field -q 9 -end \
19352 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19353 ▷ ▷ -define BLT -BLT_set \
19354 ▷ ▷ ▷ -space 0 -family "Kantor1" -invariants \
19355 ▷ ▷ -end \
19356 ▷ ▷ -with BLT -do -blt_set_activity \
19357 ▷ ▷ ▷ -report \
19358 ▷ ▷ -end
19359 ▷ pdflatex BLT.Kantor1.q9.tex
19360 ▷ $(OPEN) BLT.Kantor1.q9.pdf
19361
19362
19363
19364
19365 BLT_11.0:
19366 ▷ $(ORBITER) -v 2 \
19367 ▷ ▷ -define F -finite_field -q 11 -end \
19368 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19369 ▷ ▷ -define BLT -BLT_set \
19370 ▷ ▷ ▷ -space 0 -catalogue 0 -invariants \

```

```

19371 ▷ ▷ -end \
19372 ▷ ▷ -with BLT -do -blt_set_activity \
19373 ▷ ▷ ▷ -report \
19374 ▷ ▷ -end
19375 ▷ pdflatex BLT.catalogue.q11.iso0.tex
19376 ▷ $(OPEN) BLT.catalogue.q11.iso0.pdf
19377
19378
19379 BLT_11.Fisher:
19380 ▷ $(ORBITER) -v 2 \
19381 ▷ ▷ -define F -finite_field -q 11 -end \
19382 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19383 ▷ ▷ -define BLT -BLT_set \
19384 ▷ ▷ ▷ -space 0 -family "Fisher" -invariants \
19385 ▷ ▷ -end \
19386 ▷ ▷ -with BLT -do -blt_set_activity \
19387 ▷ ▷ ▷ -report \
19388 ▷ ▷ -end
19389 ▷ pdflatex BLT.Fisher.q11.tex
19390 ▷ $(OPEN) BLT.Fisher.q11.pdf
19391
19392 BLT_11.Mondello:
19393 ▷ $(ORBITER) -v 2 \
19394 ▷ ▷ -define F -finite_field -q 11 -end \
19395 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19396 ▷ ▷ -define BLT -BLT_set \
19397 ▷ ▷ ▷ -space 0 -family "Mondello" -invariants \
19398 ▷ ▷ -end \
19399 ▷ ▷ -with BLT -do -blt_set_activity \
19400 ▷ ▷ ▷ -report \
19401 ▷ ▷ -end
19402 ▷ pdflatex BLT.Mondello.q11.tex
19403 ▷ $(OPEN) BLT.Mondello.q11.pdf
19404
19405
19406 BLT_11.FTWKB:
19407 ▷ $(ORBITER) -v 2 \
19408 ▷ ▷ -define F -finite_field -q 11 -end \
19409 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19410 ▷ ▷ -define BLT -BLT_set \
19411 ▷ ▷ ▷ -space 0 -family "FTWKB" -invariants \
19412 ▷ ▷ -end \
19413 ▷ ▷ -with BLT -do -blt_set_activity \
19414 ▷ ▷ ▷ -report \
19415 ▷ ▷ -end
19416 ▷ pdflatex BLT.FTWKB.q11.tex
19417 ▷ $(OPEN) BLT.FTWKB.q11.pdf
19418
19419
19420 # for K2, q must be congruent to 2 or 3 mod 5
19421 BLT_13.Kantor2:
19422 ▷ $(ORBITER) -v 2 \
19423 ▷ ▷ -define F -finite_field -q 13 -end \
19424 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19425 ▷ ▷ -define BLT -BLT_set \
19426 ▷ ▷ ▷ -space 0 -family "Kantor2" -invariants \
19427 ▷ ▷ -end \
19428 ▷ ▷ -with BLT -do -blt_set_activity \
19429 ▷ ▷ ▷ -report \

```

```

19430 ▷ ▷ -end
19431 ▷ pdflatex BLT_Kantor2.q13.tex
19432 ▷ $(OPEN) BLT_Kantor2.q13.pdf
19433
19434
19435 BLT_67.4:
19436 ▷ $(ORBITER) -v 4 \
19437 ▷ ▷ -define F -finite_field -q 67 -end \
19438 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19439 ▷ ▷ -define BLT -BLT_set \
19440 ▷ ▷ ▷ -space 0 -catalogue 4 -invariants \
19441 ▷ ▷ -end \
19442 ▷ ▷ -with BLT -do -blt_set_activity \
19443 ▷ ▷ ▷ -report \
19444 ▷ ▷ -end
19445
19446
19447 BLT_67.4_export_gap:
19448 ▷ $(ORBITER) -v 2 \
19449 ▷ ▷ -define F -finite_field -q 67 -end \
19450 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19451 ▷ ▷ -define BLT -BLT_set \
19452 ▷ ▷ ▷ -space 0 -catalogue 4 \
19453 ▷ ▷ -end \
19454 ▷ ▷ -with BLT -do -blt_set_activity \
19455 ▷ ▷ ▷ -export_gap \
19456 ▷ ▷ -end
19457
19458
19459
19460
19461 BLT_9_deep_search:
19462 ▷ $(ORBITER) -v 2 \
19463 ▷ ▷ -define F -finite_field -q 9 -end \
19464 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19465 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 10 -end \
19466 ▷ ▷ -define Activity -poset_classification_activity -report -end -end \
19467 ▷ ▷ -define Activity_export -poset_classification_activity \
19468 ▷ ▷ ▷ -export_level_to_cpp 10 -end \
19469 ▷ ▷ -with C -do -BLT_set_classify_activity \
19470 ▷ ▷ ▷ -compute_starter \
19471 ▷ ▷ ▷ ▷ -problem_label BLT.q9 \
19472 ▷ ▷ ▷ ▷ -W -depth 10 \
19473 ▷ ▷ ▷ -end \
19474 ▷ ▷ -end \
19475 ▷ ▷ -with C -do -BLT_set_classify_activity \
19476 ▷ ▷ ▷ -poset_classification_activity \
19477 ▷ ▷ ▷ ▷ Activity \
19478 ▷ ▷ -end \
19479 ▷ ▷ -with C -do -BLT_set_classify_activity \
19480 ▷ ▷ ▷ -poset_classification_activity \
19481 ▷ ▷ ▷ ▷ Activity_export \
19482 ▷ ▷ -end
19483 ▷ pdflatex BLT.q9.poset.tex
19484 ▷ $(OPEN) BLT.q9.poset.pdf
19485
19486
19487
19488 BLT_11_deep_search:

```

```

19489 ▷ $(ORBITER) -v 2 \
19490 ▷ ▷ -define F -finite.field -q 11 -end \
19491 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
19492 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 12 -end \
19493 ▷ ▷ -with C -do -BLT_set_classify_activity \
19494 ▷ ▷ ▷ -compute_starter \
19495 ▷ ▷ ▷ ▷ -problem_label BLT_q11 \
19496 ▷ ▷ ▷ ▷ -W -depth 12 \
19497 ▷ ▷ ▷ -end \
19498 ▷ ▷ -end
19499 ▷ #pdflatex BLT_q11_poset.tex
19500 ▷ #$(OPEN) BLT_q11_poset.pdf
19501
19502
19503
19504
19505 BLT_13_deep_search:
19506 ▷ $(ORBITER) -v 2 \
19507 ▷ ▷ -define F -finite.field -q 13 -end \
19508 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
19509 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 14 -end \
19510 ▷ ▷ -with C -do -BLT_set_classify_activity \
19511 ▷ ▷ ▷ -compute_starter \
19512 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
19513 ▷ ▷ ▷ ▷ -W -depth 14 \
19514 ▷ ▷ ▷ -end \
19515 ▷ ▷ -end
19516 ▷ #pdflatex BLT_q13_poset.tex
19517 ▷ #$(OPEN) BLT_q13_poset.pdf
19518
19519
19520
19521
19522 BLT_13_classify_starter:
19523 ▷ $(ORBITER) -v 2 \
19524 ▷ ▷ -define F -finite.field -q 13 -end \
19525 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
19526 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
19527 ▷ ▷ -with C -do -BLT_set_classify_activity \
19528 ▷ ▷ ▷ -compute_starter \
19529 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
19530 ▷ ▷ ▷ ▷ -W -depth 5 \
19531 ▷ ▷ ▷ -end \
19532 ▷ ▷ -end \
19533 ▷ ▷ -with C -do -BLT_set_classify_activity \
19534 ▷ ▷ ▷ -create_graphs \
19535 ▷ ▷ -end
19536
19537
19538
19539
19540 BLT_13_cliques:
19541 ▷ $(ORBITER) -v 2 \
19542 ▷ ▷ -loop L 0 38 1 \
19543 ▷ ▷ ▷ -define G -graph \
19544 ▷ ▷ ▷ ▷ -load BLT_q13_graph_5_%L.bin \
19545 ▷ ▷ ▷ -end \
19546 ▷ ▷ ▷ -with G -do \
19547 ▷ ▷ ▷ -graph_theoretic_activity \

```



```

19548 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 9 -end \
19549 ▷ ▷ ▷ -end \
19550 ▷ ▷ -end_loop L
19551
19552
19553 # 3 solutions:
19554 #BLT_q13_graph_5_0_sol.txt
19555 #BLT_q13_graph_5_0_sol.csv
19556
19557
19558
19559 BLT_13_isomorph_read.DB:
19560 ▷ $(ORBITER) -v 2 \
19561 ▷ ▷ -define F -finite_field -q 13 -end \
19562 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19563 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
19564 ▷ ▷ -with C -do -BLT_set_classify_activity \
19565 ▷ ▷ ▷ -compute_starter \
19566 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
19567 ▷ ▷ ▷ ▷ -W -depth 5 \
19568 ▷ ▷ ▷ -end \
19569 ▷ ▷ -end \
19570 ▷ ▷ -with C -do -BLT_set_classify_activity \
19571 ▷ ▷ ▷ -isomorph \
19572 ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
19573 ▷ ▷ ▷ ▷ -use_database_for_starter \
19574 ▷ ▷ ▷ ▷ -build_db \
19575 ▷ ▷ ▷ ▷ -solution_prefix "" \
19576 ▷ ▷ ▷ ▷ -base_fname "" \
19577 ▷ ▷ ▷ -end \
19578 ▷ ▷ -end
19579
19580
19581
19582 BLT_13_isomorph_read.solutions:
19583 ▷ $(ORBITER) -v 2 \
19584 ▷ ▷ -define F -finite_field -q 13 -end \
19585 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19586 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
19587 ▷ ▷ -with C -do -BLT_set_classify_activity \
19588 ▷ ▷ ▷ -compute_starter \
19589 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
19590 ▷ ▷ ▷ ▷ -W -depth 5 \
19591 ▷ ▷ ▷ -end \
19592 ▷ ▷ -end \
19593 ▷ ▷ -with C -do -BLT_set_classify_activity \
19594 ▷ ▷ ▷ -isomorph \
19595 ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
19596 ▷ ▷ ▷ ▷ -use_database_for_starter \
19597 ▷ ▷ ▷ ▷ -read_solutions \
19598 ▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
19599 ▷ ▷ ▷ ▷ -solution_prefix "" \
19600 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
19601 ▷ ▷ ▷ -end \
19602 ▷ ▷ -end
19603
19604
19605 BLT_13_isomorph_stabilizer_orbits:
19606 ▷ $(ORBITER) -v 2 \

```

```

19607 ▷ ▷ -define F -finite.field -q 13 -end \
19608 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
19609 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
19610 ▷ ▷ -with C -do -BLT_set_classify_activity \
19611 ▷ ▷ ▷ -compute_starter \
19612 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
19613 ▷ ▷ ▷ ▷ -W -depth 5 \
19614 ▷ ▷ ▷ -end \
19615 ▷ ▷ -end \
19616 ▷ ▷ -with C -do -BLT_set_classify_activity \
19617 ▷ ▷ ▷ -isomorph \
19618 ▷ ▷ ▷ ▷ -prefix_iso "../BLT_q13" \
19619 ▷ ▷ ▷ ▷ -use_database_for_starter \
19620 ▷ ▷ ▷ ▷ -compute_orbits \
19621 ▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
19622 ▷ ▷ ▷ ▷ -solution_prefix "" \
19623 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
19624 ▷ ▷ ▷ -end \
19625 ▷ ▷ -end
19626
19627 BLT_13_isomorph_testing:
19628 ▷ $(ORBITER) -v 4 \
19629 ▷ ▷ -define F -finite.field -q 13 -end \
19630 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
19631 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
19632 ▷ ▷ -with C -do -BLT_set_classify_activity \
19633 ▷ ▷ ▷ -compute_starter \
19634 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
19635 ▷ ▷ ▷ ▷ -W -depth 5 \
19636 ▷ ▷ ▷ -end \
19637 ▷ ▷ -end \
19638 ▷ ▷ -with C -do -BLT_set_classify_activity \
19639 ▷ ▷ ▷ -isomorph \
19640 ▷ ▷ ▷ ▷ -prefix_iso "../BLT_q13" \
19641 ▷ ▷ ▷ ▷ -use_database_for_starter \
19642 ▷ ▷ ▷ ▷ -isomorph_testing \
19643 ▷ ▷ ▷ ▷ -solution_prefix "" \
19644 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
19645 ▷ ▷ ▷ -end \
19646 ▷ ▷ -end \
19647 ▷ ▷ -with C -do -BLT_set_classify_activity \
19648 ▷ ▷ ▷ -isomorph \
19649 ▷ ▷ ▷ ▷ -prefix_iso "../BLT_q13" \
19650 ▷ ▷ ▷ ▷ -use_database_for_starter \
19651 ▷ ▷ ▷ ▷ -isomorph_report \
19652 ▷ ▷ ▷ ▷ -solution_prefix "" \
19653 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
19654 ▷ ▷ ▷ -end \
19655 ▷ ▷ -end
19656 ▷ pdflatex BLT_q13_isomorphism_types.tex
19657 ▷ $(OPEN) BLT_q13_isomorphism_types.pdf
19658
19659
19660
19661
19662
19663 blt_set_export_gap_all:
19664 ▷ $(ORBITER) -v 3 \
19665 ▷ ▷ -define Q -vector -dense \

```

```

19666 ▷ ▷ ▷ $(BLT_ORDER_Q) \
19667 ▷ ▷ -end \
19668 ▷ ▷ -define NB -vector -dense \
19669 ▷ ▷ ▷ $(BLT_NUMBER_ISO) \
19670 ▷ ▷ -end \
19671 ▷ ▷ -loop_over i Q \
19672 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
19673 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -end \
19674 ▷ ▷ ▷ -loop j 0 %i[NB] 1 \
19675 ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
19676 ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
19677 ▷ ▷ ▷ ▷ -end \
19678 ▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set.activity \
19679 ▷ ▷ ▷ ▷ ▷ -export_gap \
19680 ▷ ▷ ▷ ▷ -end \
19681 ▷ ▷ ▷ -end_loop j \
19682 ▷ ▷ -end_loop_over i \
19683 ▷ ▷ -print_symbols
19684
19685
19686
19687
19688
19689
19690
19691
19692 #####
19693 # Chapter 15 - Computer Science Primitives
19694 #####
19695
19696
19697
19698 test_15:
19699 ▷ make test_15_1
19700 ▷ make test_15_2
19701 ▷ make test_15_3
19702
19703
19704
19705
19706 #####
19707 # Section 15.1: Clique finding
19708
19709
19710 SECTION_GRAPH_THEORY_CLIQUÉ_FINDING:
19711
19712
19713 test_15_1:
19714 ▷ make small_graph_cliques
19715 ▷ make BLT_q13_graph_5_0_cliques_bw
19716 ▷ make BLT_q13_graph_5_0_cliques_rainbow
19717 ▷ make small_graph_cliques_Sajeeb
19718 ▷ make Paley_13_aut
19719 ▷ make Paley_13
19720 ▷ make Paley_13_cliques_classify
19721 ▷ make Paley_13_cliques_all
19722 ▷ make tritangent_planes_graph_aut
19723 ▷ make tritangent_planes_graph_cliques_classify
19724 ▷ make tritangent_planes_graph_cliques_all

```

```

19725 ▷ make PG0.5.2_cliques
19726 ▷ make HJ_d2_c5
19727 ▷ make HJ64_cliques5
19728 ▷ make HJ64_cliques5_classify
19729
19730
19731
19732 small_graph_cliques: graph_v5_e7.colored_graph
19733 ▷ $(ORBITER) -v 2 \
19734 ▷ ▷ -define G -graph -load graph_v5_e7.colored_graph -end \
19735 ▷ ▷ -with G -do \
19736 ▷ ▷ -graph_theoretic.activity \
19737 ▷ ▷ ▷ -find_cliques -target_size 3 \
19738 ▷ ▷ -end
19739
19740 # nb_sol = 3
19741
19742
19743
19744
19745 BLT_q13_graph_5_0_cliques_bw:
19746 ▷ $(ORBITER) -v 2 \
19747 ▷ ▷ -define G -graph -load BLT_q13_graph_5_0.bin -end \
19748 ▷ ▷ -with G -do \
19749 ▷ ▷ -graph_theoretic.activity \
19750 ▷ ▷ ▷ -find_cliques -target_size 9 \
19751 ▷ ▷ -end
19752
19753 # all_cliques_black_and_white
19754
19755
19756 BLT_q13_graph_5_0_cliques_rainbow:
19757 ▷ $(ORBITER) -v 2 \
19758 ▷ ▷ -define G -graph -load BLT_q13_graph_5_0.bin -end \
19759 ▷ ▷ -with G -do \
19760 ▷ ▷ -graph_theoretic.activity \
19761 ▷ ▷ ▷ -find_cliques -rainbow -target_size 9 \
19762 ▷ ▷ -end
19763
19764 # all_rainbow_cliques
19765
19766
19767 small_graph_cliques_Sajeeb:
19768 ▷ $(ORBITER) -v 2 \
19769 ▷ ▷ -define G -graph -load graph_v5_e7.colored_graph -end \
19770 ▷ ▷ -with G -do \
19771 ▷ ▷ -graph_theoretic.activity \
19772 ▷ ▷ ▷ -find_cliques -Sajeeb -target_size 3 \
19773 ▷ ▷ -end
19774
19775 # nb_sol = 3
19776
19777 Paley_13_aut:
19778 ▷ $(ORBITER) -v 2 \
19779 ▷ ▷ -define F -finite_field -q 13 -end \
19780 ▷ ▷ -define Gamma -graph -Paley F -end \
19781 ▷ ▷ -with Gamma -do \
19782 ▷ ▷ -graph_theoretic.activity \
19783 ▷ ▷ ▷ -automorphism_group \

```

```

19784 ▷ ▷ -end
19785
19786 # writes Paley_13.group.makefile
19787 #User time: 0 of a second, dt=0 tps = 100
19788 #nb.calls.to.densenauty=1
19789
19790
19791
19792 Paley_13:
19793 ▷ $(ORBITER) -v 2 \
19794 ▷ ▷ -define gens -vector -file Paley_13.gens.csv -end \
19795 ▷ ▷ -define G -permutation_group \
19796 ▷ ▷ -bsgs Paley_13 "Paley\13" 13 78 "0,1" 3 gens -end \
19797
19798
19799 Paley_13_cliques_classify:
19800 ▷ $(ORBITER) -v 4 \
19801 ▷ ▷ -define Control -poset_classification_control \
19802 ▷ ▷ ▷ -W \
19803 ▷ ▷ ▷ -problem_label Paley13_cliques \
19804 ▷ ▷ ▷ -clique_test Gamma \
19805 ▷ ▷ ▷ -depth 5 \
19806 ▷ ▷ -end \
19807 ▷ ▷ -define F -finite_field -q 13 -end \
19808 ▷ ▷ -define gens -vector -file Paley_13.gens.csv -end \
19809 ▷ ▷ -define G -permutation_group \
19810 ▷ ▷ ▷ -bsgs Paley_13 "Paley\13" 13 78 "0,1" 3 gens -end \
19811 ▷ ▷ -define Gamma -graph -Paley F -end \
19812 ▷ ▷ -define Orb -orbits -group G \
19813 ▷ ▷ ▷ -on_subsets 5 Control \
19814 ▷ ▷ -end
19815
19816 #User time: 0.01 of a second, dt=1 tps = 100
19817
19818
19819 Paley_13_cliques_all:
19820 ▷ $(ORBITER) -v 10 \
19821 ▷ ▷ -define F -finite_field -q 13 -end \
19822 ▷ ▷ -define Gamma -graph -Paley F -end \
19823 ▷ ▷ -with Gamma -do \
19824 ▷ ▷ -graph_theoretic_activity \
19825 ▷ ▷ ▷ -find_cliques -target_size 3 \
19826 ▷ ▷ -end
19827
19828
19829 tritangent_planes_graph_aut:
19830 ▷ $(ORBITER) -v 2 \
19831 ▷ ▷ -define Gamma \
19832 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
19833 ▷ ▷ -end \
19834 ▷ ▷ -with Gamma -do \
19835 ▷ ▷ -graph_theoretic_activity \
19836 ▷ ▷ ▷ -automorphism_group \
19837 ▷ ▷ -end
19838 ▷ pdflatex tritangent_planes_disjointness_report.tex
19839 ▷ $(OPEN) tritangent_planes_disjointness_report.pdf
19840
19841 #tritangent_planes_disjointness_report.tex
19842 #tritangent_planes_disjointness_gens.csv

```

```

19843 #tritangent_planes_disjointness_group.makefile
19844
19845 tritangent_planes_graph_cliques_classify:
19846 ▷ $(ORBITER) -v 4 \
19847 ▷ ▷ -define Control -poset_classification_control \
19848 ▷ ▷ ▷ -W \
19849 ▷ ▷ ▷ -problem_label tri \
19850 ▷ ▷ ▷ -clique_test Gamma \
19851 ▷ ▷ ▷ -depth 9 \
19852 ▷ ▷ -end \
19853 ▷ ▷ -define gens -vector -file \
19854 ▷ ▷ ▷ tritangent_planes_disjointness_gens.csv \
19855 ▷ ▷ -end \
19856 ▷ ▷ -define G -permutation_group \
19857 ▷ ▷ -bsgs tritangent_planes_disjointness \
19858 ▷ ▷ ▷ "tritangent_planes_disjointness" 45 51840 \
19859 ▷ ▷ ▷ "0,3,12,24" 6 gens -end \
19860 ▷ ▷ -define Gamma \
19861 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
19862 ▷ ▷ -end \
19863 ▷ ▷ -define Orb -orbits -group G \
19864 ▷ ▷ ▷ -on_subsets 9 Control \
19865 ▷ ▷ -end \
19866 ▷ ▷ -with Orb -do -orbits_activity \
19867 ▷ ▷ ▷ -recognize "4,9,11,14,21,26,30,35,40" \
19868 ▷ ▷ -end
19869
19870 #tri_reps_lvl_9.csv
19871
19872
19873 tritangent_planes_graph_cliques_all:
19874 ▷ $(ORBITER) -v 2 \
19875 ▷ ▷ -define Gamma \
19876 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
19877 ▷ ▷ -end \
19878 ▷ ▷ -with Gamma -do \
19879 ▷ ▷ -graph_theoretic_activity \
19880 ▷ ▷ ▷ -find_cliques -target_size 9 \
19881 ▷ ▷ -end
19882
19883
19884 #tritangent_planes_disjointness_sol.csv
19885
19886
19887
19888 PG0.5.2_cliques: 0.5.2_incidence_matrix.csv
19889 ▷ $(ORBITER) -v 3 \
19890 ▷ ▷ -define Inc -vector -file 0.5.2_incidence_matrix.csv -end \
19891 ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
19892 ▷ ▷ -with Gamma -do \
19893 ▷ ▷ -graph_theoretic_activity \
19894 ▷ ▷ ▷ -find_cliques -target_size 3 -end \
19895 ▷ ▷ -end
19896
19897
19898 HJ_d2_c5:
19899 ▷ $(ORBITER) -v 6 \
19900 ▷ ▷ -define G -graph \
19901 ▷ ▷ ▷ -load_csv_no_border \

```

```

19902 ▷ ▷ ▷ halljanko315.csv \
19903 ▷ ▷ ▷ -distance_2 \
19904 ▷ ▷ -end \
19905 ▷ ▷ -with G -do \
19906 ▷ ▷ -graph_theoretic_activity \
19907 ▷ ▷ ▷ -find_cliques -target_size 5 -end \
19908 ▷ ▷ -end
19909
19910
19911
19912 #graph_theoretic_activity::perform_activity Gr->label=halljanko315 nb_sol = 26208
19913   0
19914
19915 HJ64_cliques5:
19916 ▷ $(ORBITER) -v 6 \
19917 ▷ ▷ -define Gamma -graph \
19918 ▷ ▷ ▷ -load \
19919 ▷ ▷ ▷ Group_Perm315_Orbital_3.colored_graph \
19920 ▷ ▷ -end \
19921 ▷ ▷ -with Gamma -do \
19922 ▷ ▷ -graph_theoretic_activity \
19923 ▷ ▷ ▷ -find_cliques -target_size 5 -end \
19924 ▷ ▷ -end
19925
19926 #graph_theoretic_activity::perform_activity Gr->label=Group_Perm315_Orbital_3 nb_
19927   sol = 1008
19928 #Group_Perm315_Orbital_3.sol.csv
19929
19930
19931 HJ64_cliques5_classify:
19932 ▷ $(ORBITER) -v 6 \
19933 ▷ ▷ -define Control -poset_classification_control \
19934 ▷ ▷ ▷ -W \
19935 ▷ ▷ ▷ -problem_label HJ64_cliques \
19936 ▷ ▷ ▷ -clique_test Gamma \
19937 ▷ ▷ ▷ -depth 5 \
19938 ▷ ▷ -end \
19939 ▷ ▷ -define Gamma -graph \
19940 ▷ ▷ ▷ -load \
19941 ▷ ▷ ▷ Group_Perm315_Orbital_3.colored_graph \
19942 ▷ ▷ -end \
19943 ▷ ▷ -define gens -vector \
19944 ▷ ▷ ▷ -file halljanko315.gens.csv \
19945 ▷ ▷ -end \
19946 ▷ ▷ -define G -permutation_group \
19947 ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
19948 ▷ ▷ ▷ 315 1209600 "0,1,42,95" 5 gens -end \
19949 ▷ ▷ -define Orb -orbits -group G \
19950 ▷ ▷ ▷ -on_subsets 5 Control \
19951 ▷ ▷ -end
19952
19953 #HJ64_cliques_reps_lvl5.csv
19954
19955 # 1 orbit
19956 #ROW,REP,AGO,OL
19957 #0,"0,8,31,110,283",1200,1008
19958 #END

```

```

19959
19960
19961 #####
19962 # Section 15.2: Rainbow cliques
19963
19964
19965 SECTION_GRAPH_THEORY_RAINBOW_CLIQUES:
19966
19967
19968 test_15_2:
19969 ▷ make BLT_11_classify_starter
19970 ▷ make BLT_11_clique_0
19971 ▷ make BLT_11_clique_0_Sajeeb
19972 ▷ make BLT_11_0_sort
19973 ▷ make BLT_11_0_sorted_draw
19974
19975
19976 BLT_11_classify_starter:
19977 ▷ $(ORBITER) -v 2 \
19978 ▷ ▷ -define F -finite_field -q 11 -end \
19979 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
19980 ▷ ▷ -define C -BLT_set_classifier 0 -starter_size 5 -end \
19981 ▷ ▷ -with C -do -BLT_set_classify_activity \
19982 ▷ ▷ ▷ -compute_starter \
19983 ▷ ▷ ▷ ▷ -problem_label BLT_q11 \
19984 ▷ ▷ ▷ ▷ -W -depth 5 \
19985 ▷ ▷ ▷ -end \
19986 ▷ ▷ -end \
19987 ▷ ▷ -with C -do -BLT_set_classify_activity \
19988 ▷ ▷ ▷ -create_graphs \
19989 ▷ ▷ -end
19990
19991 #NB_CASES_Q11=
19992
19993
19994 BLT_11_clique_0:
19995 ▷ $(ORBITER) -v 2 \
19996 ▷ ▷ -define G -graph \
19997 ▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
19998 ▷ ▷ -end \
19999 ▷ ▷ -with G -do \
20000 ▷ ▷ -graph_theoretic_activity \
20001 ▷ ▷ ▷ -find_cliques -rainbow -target_size 12 -end \
20002 ▷ ▷ -end
20003
20004 # nb_sol = 3
20005
20006 BLT_11_clique_0_Sajeeb:
20007 ▷ $(ORBITER) -v 2 \
20008 ▷ ▷ -define G -graph \
20009 ▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
20010 ▷ ▷ -end \
20011 ▷ ▷ -with G -do \
20012 ▷ ▷ -graph_theoretic_activity \
20013 ▷ ▷ ▷ -find_cliques -rainbow -Sajeeb -end \
20014 ▷ ▷ -end
20015
20016 # nb_sol = 3
20017

```



```

20018
20019 BLT_11_0_sort:
20020 ▷ $(ORBITER) -v 2 \
20021 ▷ ▷ -define G -graph \
20022 ▷ ▷ ▷ -load BLT.q11_graph.5_0.bin \
20023 ▷ ▷ -end \
20024 ▷ ▷ -with G -do \
20025 ▷ ▷ -graph_theoretic.activity \
20026 ▷ ▷ ▷ -sort_by_colors \
20027 ▷ ▷ -end
20028
20029 #BLT.q11_graph.5_0.sorted.bin
20030
20031
20032 BLT_11_0_sorted_draw:
20033 ▷ $(ORBITER) -v 2 \
20034 ▷ ▷ -draw_options \
20035 ▷ ▷ ▷ -radius 100 \
20036 ▷ ▷ ▷ -scale 0.4 \
20037 ▷ ▷ ▷ -line_width 1.0 -embedded \
20038 ▷ ▷ -end \
20039 ▷ ▷ -define G -graph \
20040 ▷ ▷ ▷ -load BLT.q11_graph.5_0.sorted.bin \
20041 ▷ ▷ -end \
20042 ▷ ▷ -with G -do \
20043 ▷ ▷ -graph_theoretic.activity \
20044 ▷ ▷ ▷ -draw \
20045 ▷ ▷ -end
20046 ▷ pdflatex BLT.q11_graph.5_0.sorted_draw.tex
20047 ▷ $(OPEN) BLT.q11_graph.5_0.sorted.draw.pdf
20048
20049
20050
20051
20052 #####
20053 # Section 15.3: Diophantine Systems
20054
20055 SECTION_DIOPHANT:
20056
20057
20058 test_15_3:
20059 ▷ make part10
20060 ▷ make octic_monomials
20061 ▷ make solve_test_system
20062 ▷ make McKay_test
20063 ▷ make DLX_test
20064
20065
20066
20067 part10:
20068 ▷ $(ORBITER) -v 4 \
20069 ▷ ▷ -define A -vector -format 1 -dense "10,9,8,7,6,5,4,3,2,1" -end \
20070 ▷ ▷ -define D -diophant \
20071 ▷ ▷ ▷ -label part10 \
20072 ▷ ▷ ▷ -coefficient_matrix A \
20073 ▷ ▷ ▷ -RHS "mult=1,EQ=10" \
20074 ▷ ▷ ▷ -x_min_global 0 -x_max_global 10 \
20075 ▷ ▷ -end \
20076 ▷ ▷ -with D -do \

```

```

20077 ▷ ▷ ▷ -diophant_activity -solve_mckay \
20078 ▷ ▷ -end
20079 ▷ ▷
20080
20081
20082 ▷
20083 # Finds 42 solutions with 67 backtrack steps
20084
20085
20086 octic_monomials:
20087 ▷ $(ORBITER) -v 4 \
20088 ▷ ▷ -define A -vector -format 1 -dense "1,1,1,1" -end \
20089 ▷ ▷ -define D -diophant \
20090 ▷ ▷ ▷ -label octic_monomials \
20091 ▷ ▷ ▷ -coefficient_matrix A \
20092 ▷ ▷ ▷ -RHS "mult=1,EQ=9" \
20093 ▷ ▷ ▷ -x_min_global 0 -x_max_global 8 \
20094 ▷ ▷ -end \
20095 ▷ ▷ -with D -do \
20096 ▷ ▷ ▷ -diophant_activity -solve_mckay \
20097 ▷ ▷ -end
20098 ▷ sort -r octic_monomials_sol.csv >octic_monomials_sorted.csv
20099
20100 #Found 165 solutions with 210 backtrack steps
20101 # 165=binomial(11,3)
20102
20103
20104 solve_test_system:
20105 ▷ $(ORBITER) -v 4 \
20106 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
20107 ▷ ▷ -define D -diophant \
20108 ▷ ▷ ▷ -label test_system \
20109 ▷ ▷ ▷ -coefficient_matrix A \
20110 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
20111 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
20112 ▷ ▷ -end
20113
20114
20115 McKay_test:
20116 ▷ $(ORBITER) -v 4 \
20117 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
20118 ▷ ▷ -define D -diophant \
20119 ▷ ▷ ▷ -label test_system \
20120 ▷ ▷ ▷ -coefficient_matrix A \
20121 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
20122 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
20123 ▷ ▷ -end \
20124 ▷ ▷ -with D -do \
20125 ▷ ▷ ▷ -diophant_activity -solve_mckay \
20126 ▷ ▷ -end
20127
20128 DLX_test:
20129 ▷ $(ORBITER) -v 4 \
20130 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
20131 ▷ ▷ -define D \
20132 ▷ ▷ -diophant -label test_system \
20133 ▷ ▷ ▷ -coefficient_matrix A \
20134 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
20135 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \

```

```

20136 ▷ ▷ -end \
20137 ▷ ▷ -with D -do \
20138 ▷ ▷ ▷ -diophant_activity -solve_DLX \
20139 ▷ ▷ -end
20140
20141 #DLX_test.sol
20142 # 1 solution in 6 backtrack steps
20143
20144
20145
20146
20147 #####
20148 # Chapter 16 - Canonical Forms with Nauty
20149 #####
20150
20151
20152
20153 test_16:
20154 ▷ make test_16_2
20155 ▷ make test_16_3
20156 ▷ make test_16_4
20157 ▷ make test_16_6
20158 ▷ make test_16_7
20159 ▷ make test_16_8
20160
20161
20162
20163 #####
20164 # Section 16.1: Overview of Canonical Forms
20165
20166
20167 SECTION.OVERVIEW.CANONICAL.FORMS:
20168
20169
20170
20171
20172
20173 #####
20174 # Section 16.2: Objects in projective Space
20175
20176
20177 SECTION.OBJECTS.IN.PROJECTIVE.SPACE:
20178
20179
20180 test_16_2:
20181 ▷ make EC_canon
20182 ▷ make Hirschfeld.q4.c
20183 ▷ make Hirschfeld.stab_group
20184 ▷ make Hirschfeld.stab_subgroup
20185 ▷ make Hirschfeld.stab_orbits
20186 ▷ make Hirschfeld.q4.set.c
20187 ▷ make Dickson.sets.stabilizer
20188 ▷ make Endrass.7c
20189 ▷ make hyperoval_16.canonical.form
20190 ▷ make cubic_curves_PG_2_8.canon
20191 ▷ make ovoid_q8.canon
20192 ▷ make ovoid_ST_q8.canon
20193 ▷ make Suzuki.8
20194 ▷ make quartic_curve_25_0_0.canonical

```

```

20195 ▷ make CRC.BCH.Q16.N51.D3.canonical
20196 ▷ make Veronese_1_3.q7.c
20197
20198
20199 EC.canon: elliptic.curve.b1.c3.q11.txt
20200 ▷ $(ORBITER) -v 3 \
20201 ▷ ▷ -define C -combinatorial.object \
20202 ▷ ▷ ▷ -label EC EC \
20203 ▷ ▷ ▷ -file_of_points elliptic.curve.b1.c3.q11.txt \
20204 ▷ ▷ -end \
20205 ▷ ▷ -define F -finite_field -q 11 -end \
20206 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
20207 ▷ ▷ -with C -do \
20208 ▷ ▷ -combinatorial_object_activity \
20209 ▷ ▷ ▷ -canonical_form_PG P \
20210 ▷ ▷ ▷ ▷ -save_ago \
20211 ▷ ▷ ▷ ▷ -save_transversal \
20212 ▷ ▷ ▷ ▷ -max_TDO_depth 400 \
20213 ▷ ▷ ▷ -end \
20214 ▷ ▷ -end \
20215 ▷ ▷ -with C -do \
20216 ▷ ▷ -combinatorial_object_activity \
20217 ▷ ▷ ▷ -report \
20218 ▷ ▷ ▷ ▷ -export_flag_orbits \
20219 ▷ ▷ ▷ ▷ -show_TDO \
20220 ▷ ▷ ▷ ▷ -show_TDA \
20221 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20222 ▷ ▷ ▷ ▷ -export_group_GAP \
20223 ▷ ▷ ▷ -end \
20224 ▷ ▷ -end
20225 ▷ pdflatex EC.classification.tex
20226 ▷ $(OPEN) EC.classification.pdf
20227 ▷ $(ORBITER) -v 2 -draw_matrix \
20228 ▷ ▷ -input_csv_file EC.object0.TDA.flag_orbits.csv \
20229 ▷ ▷ -secondary_input_csv_file EC.object0.TDA.csv \
20230 ▷ ▷ -box_width 20 -bit_depth 24 \
20231 ▷ ▷ -end
20232 ▷ $(OPEN) EC.object0.TDA.flag_orbits_draw.bmp
20233
20234
20235
20236
20237 Hirschfeld.q4.c:
20238 ▷ $(ORBITER) -v 6 \
20239 ▷ ▷ -define C -combinatorial.object \
20240 ▷ ▷ ▷ -label Hirschfeld.surface.q4 Hirschfeld\surface\q4 \
20241 ▷ ▷ ▷ -file_of_points Hirschfeld.surface.q4.txt \
20242 ▷ ▷ -end \
20243 ▷ ▷ -define F -finite_field -q 4 -end \
20244 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20245 ▷ ▷ -with C -do \
20246 ▷ ▷ -combinatorial_object_activity \
20247 ▷ ▷ ▷ -canonical_form_PG P \
20248 ▷ ▷ ▷ ▷ -save_ago \
20249 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
20250 ▷ ▷ ▷ -end \
20251 ▷ ▷ -end \
20252 ▷ ▷ -with C -do \
20253 ▷ ▷ -combinatorial_object_activity \

```

```

20254 ▷ ▷ ▷ -report \
20255 ▷ ▷ ▷ ▷ -export_flag_orbits \
20256 ▷ ▷ ▷ ▷ -show_TDO \
20257 ▷ ▷ ▷ ▷ -show_TDA \
20258 ▷ ▷ ▷ ▷ -export_labels \
20259 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20260 ▷ ▷ ▷ ▷ -export_group_GAP \
20261 ▷ ▷ ▷ -end \
20262 ▷ ▷ -end
20263 ▷ pdflatex Hirschfeld_surface.q4.classification.tex
20264 ▷ $(OPEN) Hirschfeld_surface.q4.classification.pdf
20265
20266
20267 # group order is 51840
20268
20269
20270 Hirschfeld_stab_group:
20271 ▷ $(ORBITER) -v 9 \
20272 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20273 ▷ ▷ -define G -linear_group -PGGL 4 4 \
20274 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
20275 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
20276 ▷ ▷ ▷ -end \
20277 ▷ ▷ -with G -do \
20278 ▷ ▷ -group_theoretic_activity \
20279 ▷ ▷ ▷ -report \
20280 ▷ ▷ -end
20281 ▷ pdflatex PGGL_4.4.Subgroup_Hirschfeld_Stab.51840_report.tex
20282 ▷ $(OPEN) PGGL_4.4.Subgroup_Hirschfeld_Stab.51840_report.pdf
20283
20284 Hirschfeld_stab_subgroup:
20285 ▷ $(ORBITER) -v 9 \
20286 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20287 ▷ ▷ -define G -linear_group -PGGL 4 4 \
20288 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
20289 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
20290 ▷ ▷ ▷ -end \
20291 ▷ ▷ -define Gsp -modified_group -from G \
20292 ▷ ▷ ▷ -create_special_subgroup \
20293 ▷ ▷ -end \
20294 ▷ ▷ -with Gsp -do \
20295 ▷ ▷ -group_theoretic_activity \
20296 ▷ ▷ ▷ -report \
20297 ▷ ▷ -end \
20298 ▷ ▷ -define Orb -orbits -group Gsp \
20299 ▷ ▷ ▷ -on_points \
20300 ▷ ▷ -end
20301
20302 Hirschfeld_stab_orbits:
20303 ▷ $(ORBITER) -v 9 \
20304 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20305 ▷ ▷ -define G -linear_group -PGGL 4 4 \
20306 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
20307 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
20308 ▷ ▷ ▷ -end \
20309 ▷ ▷ -with G -do \
20310 ▷ ▷ -group_theoretic_activity \
20311 ▷ ▷ ▷ -report \
20312 ▷ ▷ -end \

```

```

20313 ▷ ▷ -define Orb -orbits -group G \
20314 ▷ ▷ ▷ -on_points \
20315 ▷ ▷ -end \
20316 ▷ ▷ -with Orb -do -orbits_activity \
20317 ▷ ▷ ▷ -export_something "orbit" 0 \
20318 ▷ ▷ -end
20319
20320 #orbits_PGGL_4.4.Subgroup_Hirschfeld_Stab.51840_orbit_0.csv
20321
20322
20323 Hirschfeld.q4.set.c:
20324 ▷ $(ORBITER) -v 4 \
20325 ▷ ▷ -define H -set -here \
20326 ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS) \
20327 ▷ ▷ -end \
20328 ▷ ▷ -define C -combinatorial_object \
20329 ▷ ▷ ▷ -label Hirschfeld_surface_q4 Hirschfeld\_surface\_q4 \
20330 ▷ ▷ ▷ -set_of_points H \
20331 ▷ ▷ -end \
20332 ▷ ▷ -define F -finite_field -q 4 -end \
20333 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20334 ▷ ▷ -with C -do \
20335 ▷ ▷ -combinatorial_object_activity \
20336 ▷ ▷ ▷ -canonical_form_PG P \
20337 ▷ ▷ ▷ ▷ -save_ago \
20338 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20339 ▷ ▷ ▷ -end \
20340 ▷ ▷ -end \
20341 ▷ ▷ -with C -do \
20342 ▷ ▷ -combinatorial_object_activity \
20343 ▷ ▷ ▷ -report \
20344 ▷ ▷ ▷ ▷ -show_TDO \
20345 ▷ ▷ ▷ ▷ -show_TDA \
20346 ▷ ▷ ▷ -end \
20347 ▷ ▷ -end
20348 ▷ pdflatex Hirschfeld_surface.q4_classification.tex
20349 ▷ $(OPEN) Hirschfeld_surface.q4_classification.pdf
20350
20351
20352
20353 Dickson.sets.stabilizer:
20354 ▷ $(ORBITER) -v 3 \
20355 ▷ ▷ -define C -combinatorial_object \
20356 ▷ ▷ ▷ -label Dickson.sets Dickson\_sets \
20357 ▷ ▷ ▷ -set_of_points "0,1,2,5,6" \
20358 ▷ ▷ ▷ -set_of_points "0,1,2,3,6" \
20359 ▷ ▷ ▷ -set_of_points "0,1,2,3,4" \
20360 ▷ ▷ ▷ -set_of_points "0,1,2,3,8" \
20361 ▷ ▷ ▷ -set_of_points "0,1,2,5,6,7,8" \
20362 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,7" \
20363 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,9" \
20364 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,10" \
20365 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,4" \
20366 ▷ ▷ ▷ -set_of_points "0,1,2,3,8,11,13" \
20367 ▷ ▷ ▷ -set_of_points "3,6,9,7,10,12,8,11,13,14,4" \
20368 ▷ ▷ ▷ -set_of_points "3,5,6,9,7,10,12,11,13,14,4" \
20369 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,9,7,10,12,4" \
20370 ▷ ▷ -end \
20371 ▷ ▷ -define F -finite_field -q 2 -end \

```

```

20372 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20373 ▷ ▷ -with C -do \
20374 ▷ ▷ -combinatorial_object_activity \
20375 ▷ ▷ ▷ -canonical_form_PG P \
20376 ▷ ▷ ▷ ▷ -save_ago \
20377 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20378 ▷ ▷ ▷ -end \
20379 ▷ ▷ -end \
20380 ▷ ▷ -with C -do \
20381 ▷ ▷ -combinatorial_object_activity \
20382 ▷ ▷ ▷ -report \
20383 ▷ ▷ ▷ ▷ -show_TDO \
20384 ▷ ▷ ▷ ▷ -show_TDA \
20385 ▷ ▷ ▷ -end \
20386 ▷ ▷ -end
20387 ▷ pdflatex Dickson_sets_classification.tex
20388 ▷ $(OPEN) Dickson_sets_classification.pdf
20389
20390
20391
20392 Endrass_7c: Endrass_F7.txt
20393 ▷ $(ORBITER) -v 2 \
20394 ▷ ▷ -define C -combinatorial_object \
20395 ▷ ▷ ▷ -label Endrass_F7 Endrass\F7 \
20396 ▷ ▷ ▷ -file_of_points Endrass_F7.txt \
20397 ▷ ▷ -end \
20398 ▷ ▷ -define F -finite_field -q 7 -end \
20399 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20400 ▷ ▷ -with C -do \
20401 ▷ ▷ -combinatorial_object_activity \
20402 ▷ ▷ ▷ -canonical_form_PG P \
20403 ▷ ▷ ▷ ▷ -save_ago \
20404 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20405 ▷ ▷ ▷ -end \
20406 ▷ ▷ -end \
20407 ▷ ▷ -with C -do \
20408 ▷ ▷ -combinatorial_object_activity \
20409 ▷ ▷ ▷ -report \
20410 ▷ ▷ ▷ ▷ -show_TDO \
20411 ▷ ▷ ▷ ▷ -show_TDA \
20412 ▷ ▷ ▷ -end \
20413 ▷ ▷ -end
20414 ▷ #pdflatex Endrass_F7_classification.tex
20415 ▷ #$(OPEN) Endrass_F7_classification.pdf
20416
20417 # ToDo latex file is too big
20418 # ToDo group order is computed wrong
20419 # should be: group order is 32
20420
20421
20422 hyperoval_16_canonical_form:
20423 ▷ $(ORBITER) -v 2 \
20424 ▷ ▷ -define C -combinatorial_object \
20425 ▷ ▷ ▷ -label hyperoval_q16 hyperoval\q16 \
20426 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
20427 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
20428 ▷ ▷ -end \
20429 ▷ ▷ -define F -finite_field -q 16 -end \
20430 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \

```

```

20431 ▷ ▷ -with C -do \
20432 ▷ ▷ -combinatorial_object_activity \
20433 ▷ ▷ ▷ -canonical_form_PG P \
20434 ▷ ▷ ▷ ▷ -save_ago \
20435 ▷ ▷ ▷ ▷ -save_transversal \
20436 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
20437 ▷ ▷ ▷ -end \
20438 ▷ ▷ -end \
20439 ▷ ▷ -with C -do \
20440 ▷ ▷ -combinatorial_object_activity \
20441 ▷ ▷ ▷ -report \
20442 ▷ ▷ ▷ ▷ -export_flag_orbits \
20443 ▷ ▷ ▷ ▷ -show_TDO \
20444 ▷ ▷ ▷ ▷ -show_TDA \
20445 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20446 ▷ ▷ ▷ ▷ -export_group_GAP \
20447 ▷ ▷ ▷ -end \
20448 ▷ ▷ -end
20449 ▷ pdflatex hyperoval_q16_classification.tex
20450 ▷ $(OPEN) hyperoval_q16_classification.pdf
20451 ▷ $(ORBITER) -v 2 -draw_matrix \
20452 ▷ ▷ -input_csv_file hyperoval_q16_object0_TDA_flag_orbits.csv \
20453 ▷ ▷ -secondary_input_csv_file hyperoval_q16_object0_TDA.csv \
20454 ▷ ▷ -box_width 4 -bit_depth 24 \
20455 ▷ ▷ -end
20456 ▷ $(OPEN) hyperoval_q16_object0_TDA_flag_orbits_draw.bmp
20457 ▷ $(ORBITER) -v 2 -draw_matrix \
20458 ▷ ▷ -input_csv_file hyperoval_q16_object1_TDA_flag_orbits.csv \
20459 ▷ ▷ -secondary_input_csv_file hyperoval_q16_object1_TDA.csv \
20460 ▷ ▷ -box_width 4 -bit_depth 24 \
20461 ▷ ▷ -end
20462 ▷ $(OPEN) hyperoval_q16_object1_TDA_flag_orbits_draw.bmp
20463
20464
20465
20466
20467
20468 cubic_curves_PG_2_8_canon:
20469 ▷ $(ORBITER) -v 6 \
20470 ▷ ▷ -define C -combinatorial_object \
20471 ▷ ▷ ▷ -label cc.8 cc\8 \
20472 ▷ ▷ ▷ -set_of_points "2,3,28,46,51,61,40,71" \
20473 ▷ ▷ -end \
20474 ▷ ▷ -define F -finite_field -q 8 -end \
20475 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
20476 ▷ ▷ -with C -do \
20477 ▷ ▷ -combinatorial_object_activity \
20478 ▷ ▷ ▷ -canonical_form_PG P \
20479 ▷ ▷ ▷ ▷ -save_ago \
20480 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
20481 ▷ ▷ ▷ -end \
20482 ▷ ▷ -end \
20483 ▷ ▷ -with C -do \
20484 ▷ ▷ -combinatorial_object_activity \
20485 ▷ ▷ ▷ -report \
20486 ▷ ▷ ▷ ▷ -export_flag_orbits \
20487 ▷ ▷ ▷ ▷ -show_TDO \
20488 ▷ ▷ ▷ ▷ -show_TDA \
20489 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \

```



```

20490 ▷ ▷ ▷ ▷ -export_group_GAP \
20491 ▷ ▷ ▷ -end \
20492 ▷ ▷ -end
20493 ▷ pdflatex cc_8_classification.tex
20494 ▷ $(OPEN) cc_8_classification.pdf
20495
20496
20497
20498
20499 ovoid_q8_canon: ovoid_q8.txt
20500 ▷ $(ORBITER) -v 6 \
20501 ▷ ▷ -define C -combinatorial_object \
20502 ▷ ▷ ▷ -label ovoid ovoid \
20503 ▷ ▷ ▷ -file_of_points ovoid_q8.txt \
20504 ▷ ▷ -end \
20505 ▷ ▷ -define F -finite_field -q 8 -end \
20506 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20507 ▷ ▷ -with C -do \
20508 ▷ ▷ -combinatorial_object_activity \
20509 ▷ ▷ ▷ -canonical_form_PG P \
20510 ▷ ▷ ▷ ▷ -save_ago \
20511 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20512 ▷ ▷ ▷ -end \
20513 ▷ ▷ -end \
20514 ▷ ▷ -with C -do \
20515 ▷ ▷ -combinatorial_object_activity \
20516 ▷ ▷ ▷ -report \
20517 ▷ ▷ ▷ ▷ -show_TDO \
20518 ▷ ▷ ▷ ▷ -show_TDA \
20519 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20520 ▷ ▷ ▷ ▷ -export_group_GAP \
20521 ▷ ▷ ▷ -end \
20522 ▷ ▷ -end
20523 ▷ #pdflatex ovoid_classification.tex
20524 ▷ #$(OPEN) ovoid_classification.pdf
20525
20526 #150+00u000605540f+0w94 99.1%>
20527
20528
20529
20530
20531 # group order 1572480
20532
20533 ovoid_ST_q8_canon: ovoid_ST_q8.txt
20534 ▷ $(ORBITER) -v 6 \
20535 ▷ ▷ -define C -combinatorial_object \
20536 ▷ ▷ ▷ -label ovoid_ST ovoid\ST \
20537 ▷ ▷ ▷ -file_of_points ovoid_ST_q8.txt \
20538 ▷ ▷ -end \
20539 ▷ ▷ -define F -finite_field -q 8 -end \
20540 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20541 ▷ ▷ -with C -do \
20542 ▷ ▷ -combinatorial_object_activity \
20543 ▷ ▷ ▷ -canonical_form_PG P \
20544 ▷ ▷ ▷ ▷ -save_ago \
20545 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20546 ▷ ▷ ▷ -end \
20547 ▷ ▷ -end \
20548 ▷ ▷ -with C -do \

```

```

20549 ▷ ▷ -combinatorial_object_activity \
20550 ▷ ▷ ▷ -report \
20551 ▷ ▷ ▷ ▷ -show_TDO \
20552 ▷ ▷ ▷ ▷ -show_TDA \
20553 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20554 ▷ ▷ ▷ ▷ -export_group_GAP \
20555 ▷ ▷ ▷ -end \
20556 ▷ ▷ -end
20557 ▷ #pdflatex ovoid.ST_classification.tex
20558 ▷ #$(OPEN) ovoid.ST_classification.pdf
20559
20560 # very slow
20561 #130+0R10+0i0523p2+0W.22 99.8%>
20562
20563
20564
20565 Suzuki.8:
20566 ▷ $(ORBITER) -v 6 \
20567 ▷ ▷ -define F -finite_field -q 8 -end \
20568 ▷ ▷ -define gens -vector -field F \
20569 ▷ ▷ ▷ -compact $(SUZUKI.8_GENERATORS) -end \
20570 ▷ ▷ -define G -linear_group -PGGL 4 8 \
20571 ▷ ▷ -subgroup_by_generators "Sz8" "87360" 5 gens \
20572 ▷ ▷ ▷ -end \
20573 ▷ ▷ -with G -do \
20574 ▷ ▷ -group_theoretic_activity \
20575 ▷ ▷ ▷ -report \
20576 ▷ ▷ -end
20577 ▷ pdflatex PGGL_4.8_Subgroup_Sz8_87360_report.tex
20578 ▷ $(OPEN) PGGL_4.8_Subgroup_Sz8_87360_report.pdf
20579
20580
20581
20582 quartic_curve_25_0_0_canonical:
20583 ▷ $(ORBITER) -v 3 \
20584 ▷ ▷ -define F -finite_field -q 25 -end \
20585 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
20586 ▷ ▷ -define C -combinatorial_object \
20587 ▷ ▷ ▷ -label quartic_25_0_0 quartic\25\0\0 \
20588 ▷ ▷ ▷ -set_of_points "10,11,59,63,124,135,136,170,206,257,275,284,285,367,378,393
,433,619,641,644" \
20589 ▷ ▷ ▷ -set_of_points "9, 24, 62, 67, 77, 84, 87, 89, 125, 130, 158, 172, 197, 219
, 266, 271, 325, 356, 391, 392, 400, 429, 454, 458, 470, 503, 531, 553, 605, 625,
627, 646" \
20590 ▷ ▷ ▷ -set_of_points "9, 24, 55, 75, 79, 88, 93, 94, 112, 142, 186, 213, 230, 249
, 316, 329, 337, 371, 381, 383, 392, 405, 417, 439, 464, 466, 470, 524, 530, 567,
583, 594" \
20591 ▷ ▷ ▷ -set_of_points "10, 23, 34, 51, 65, 83, 117, 126, 144, 146, 147, 159, 172,
181, 185, 197, 198, 207, 240, 281, 283, 293, 300, 301, 305, 346, 357, 384, 409, 4
19, 444, 459, 463, 465, 468, 514, 533, 543, 547, 549, 586, 615, 618, 628" \
20592 ▷ ▷ ▷ -set_of_points "2, 12, 48, 65, 87, 120, 189, 246, 305, 323, 354, 375, 434,
435, 455, 482, 496, 557, 586, 595" \
20593 ▷ ▷ -end \
20594 ▷ ▷ -with C -do \
20595 ▷ ▷ -combinatorial_object_activity \
20596 ▷ ▷ ▷ -canonical_form_PG P \
20597 ▷ ▷ ▷ -save_ago \
20598 ▷ ▷ ▷ -max_TDO_depth 4 \
20599 ▷ ▷ ▷ -end \

```

```

20600 ▷ ▷ -end \
20601 ▷ ▷ -with C -do \
20602 ▷ ▷ -combinatorial_object_activity \
20603 ▷ ▷ ▷ -report \
20604 ▷ ▷ ▷ ▷ -show_TDO \
20605 ▷ ▷ ▷ ▷ -show_TDA \
20606 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20607 ▷ ▷ ▷ ▷ -export_group_GAP \
20608 ▷ ▷ ▷ -end \
20609 ▷ ▷ -end
20610 ▷ pdflatex quartic_25_0_0_classification.tex
20611 ▷ ${OPEN} quartic_25_0_0_classification.pdf
20612
20613
20614
20615 CRC_BCH_Q16_N51_D3_canonical:
20616 ▷ $(ORBITER) -v 12 \
20617 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
20618 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20619 ▷ ▷ -define C -combinatorial_object \
20620 ▷ ▷ ▷ -label BCH_Q16_N51 BCH\_Q16\_N51 \
20621 ▷ ▷ ▷ -set_of_points $(CRC_BCH_Q16_N51_D3_PROJECTIVE_SET) \
20622 ▷ ▷ -end \
20623 ▷ ▷ -with C -do \
20624 ▷ ▷ -combinatorial_object_activity \
20625 ▷ ▷ ▷ -canonical_form_PG P \
20626 ▷ ▷ ▷ ▷ -save_ago \
20627 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20628 ▷ ▷ ▷ -end \
20629 ▷ ▷ -end \
20630 ▷ ▷ -with C -do \
20631 ▷ ▷ -combinatorial_object_activity \
20632 ▷ ▷ ▷ -report \
20633 ▷ ▷ ▷ ▷ -show_TDO \
20634 ▷ ▷ ▷ ▷ -show_TDA \
20635 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20636 ▷ ▷ ▷ ▷ -export_group_GAP \
20637 ▷ ▷ ▷ -end \
20638 ▷ ▷ -end
20639 ▷ pdflatex BCH_Q16_N51_classification.tex
20640 ▷ ${OPEN} BCH_Q16_N51_classification.pdf
20641
20642
20643 # stabilizer of order 408 is generated by:
20644 #1,0,0,0,0,0,1,0,1,5,9,3,12,10,8,10,1, 0,1,0,0,0,0,1,0,0,0,0,1,1,2,13,5,0,
20645
20646
20647 Veronese_1_3_q7_c:
20648 ▷ $(ORBITER) -v 6 \
20649 ▷ ▷ -define C -combinatorial_object \
20650 ▷ ▷ ▷ -label veronese_1_3_q7 veronese\_1\_3\_q7 \
20651 ▷ ▷ ▷ -set_of_points "3, 0, 4, 268, 336, 184, 224, 364" \
20652 ▷ ▷ -end \
20653 ▷ ▷ -define F -finite_field -q 7 -end \
20654 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
20655 ▷ ▷ -with C -do \
20656 ▷ ▷ -combinatorial_object_activity \
20657 ▷ ▷ ▷ -canonical_form_PG P \
20658 ▷ ▷ ▷ ▷ -save_ago \

```

```

20659 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
20660 ▷ ▷ ▷ -end \
20661 ▷ ▷ -end \
20662 ▷ ▷ -with C -do \
20663 ▷ ▷ -combinatorial_object_activity \
20664 ▷ ▷ ▷ -report \
20665 ▷ ▷ ▷ ▷ -show_TDO \
20666 ▷ ▷ ▷ ▷ -show_TDA \
20667 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
20668 ▷ ▷ ▷ ▷ -export_group_GAP \
20669 ▷ ▷ ▷ -end \
20670 ▷ ▷ -end
20671 ▷ pdflatex veronese_1.3_q7_classification.tex
20672 ▷ $(OPEN) veronese_1.3_q7_classification.pdf
20673
20674
20675 #####
20676 # Section 16.3: Incidence Geometries
20677
20678
20679 SECTION_INCIDENCE_GEOMETRIES:
20680
20681
20682
20683 test_16.3:
20684 ▷ make geo_7.3_c
20685 ▷ make geo_10.3_c
20686 ▷ make geo_10.3_c_lex.least
20687 ▷ #make geo_14.3_c
20688 ▷ #make geo_15.3_c
20689 ▷ make TFC_24.3_c
20690 ▷ #make geo_40.4_g4_c
20691 ▷ #make geo_17.3_g4_c
20692 ▷ make AG_2.3_c
20693 ▷ make geo_LSQ6_c
20694
20695
20696 geo_7.3_c:
20697 ▷ $(ORBITER) -v 10 \
20698 ▷ ▷ -draw_incidence_structure_description \
20699 ▷ ▷ ▷ -width 60 -width_10 6 -end \
20700 ▷ ▷ -define C -combinatorial_object \
20701 ▷ ▷ ▷ -label geo_7.3 geo\_7\_3 \
20702 ▷ ▷ ▷ -file_of_incidence_geometries 7.3.inc 7 7 21 \
20703 ▷ ▷ -end \
20704 ▷ ▷ -with C -do \
20705 ▷ ▷ -combinatorial_object_activity \
20706 ▷ ▷ ▷ -canonical_form \
20707 ▷ ▷ ▷ ▷ -save_ago \
20708 ▷ ▷ ▷ ▷ -save_transversal \
20709 ▷ ▷ ▷ -end \
20710 ▷ ▷ -end \
20711 ▷ ▷ -with C -do \
20712 ▷ ▷ -combinatorial_object_activity \
20713 ▷ ▷ ▷ -report \
20714 ▷ ▷ ▷ ▷ -export_flag_orbits \
20715 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20716 ▷ ▷ ▷ ▷ -export_group_GAP \
20717 ▷ ▷ ▷ -end \

```

```

20718 ▷ ▷ -end
20719 ▷ pdflatex 7_3_classification.tex
20720 ▷ $(OPEN) 7_3_classification.pdf
20721 ▷ $(ORBITER) -v 2 -draw_matrix \
20722 ▷ ▷ -input_csv_file 7_3_object0_TDA_flag_orbits.csv \
20723 ▷ ▷ -secondary_input_csv_file 7_3_object0_TDA.csv \
20724 ▷ ▷ -box_width 32 -bit_depth 24 \
20725 ▷ ▷ -end
20726 ▷ $(ORBITER) -v 2 -draw_matrix \
20727 ▷ ▷ -input_csv_file 7_3_object0_INP_flag_orbits.csv \
20728 ▷ ▷ -secondary_input_csv_file 7_3_object0_INP.csv \
20729 ▷ ▷ -box_width 32 -bit_depth 24 \
20730 ▷ ▷ -end
20731 ▷ $(OPEN) 7_3_object0_INP_flag_orbits.draw.bmp
20732
20733
20734
20735 geo.10.3.c:
20736 ▷ $(ORBITER) -v 10 \
20737 ▷ ▷ -draw_incidence_structure_description \
20738 ▷ ▷ ▷ -width 60 -width_10 6 -end \
20739 ▷ ▷ -define C -combinatorial_object \
20740 ▷ ▷ ▷ -label geo.10.3 geo\10\3 \
20741 ▷ ▷ ▷ -file_of_incidence_geometries 10.3.inc 10 10 30 \
20742 ▷ ▷ -end \
20743 ▷ ▷ -with C -do \
20744 ▷ ▷ -combinatorial_object_activity \
20745 ▷ ▷ ▷ -canonical_form \
20746 ▷ ▷ ▷ ▷ -save_ago \
20747 ▷ ▷ ▷ ▷ -save_transversal \
20748 ▷ ▷ ▷ -end \
20749 ▷ ▷ -end \
20750 ▷ ▷ -with C -do \
20751 ▷ ▷ -combinatorial_object_activity \
20752 ▷ ▷ ▷ -report \
20753 ▷ ▷ ▷ ▷ -export_flag_orbits \
20754 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20755 ▷ ▷ ▷ ▷ -export_group_GAP \
20756 ▷ ▷ ▷ -end \
20757 ▷ ▷ -end
20758 ▷ pdflatex 10.3_classification.tex
20759 ▷ $(OPEN) 10.3_classification.pdf
20760 ▷ $(ORBITER) -v 2 -draw_matrix \
20761 ▷ ▷ -input_csv_file 10.3_object7_TDA_flag_orbits.csv \
20762 ▷ ▷ -secondary_input_csv_file 10.3_object7_TDA.csv \
20763 ▷ ▷ -box_width 16 -bit_depth 24 \
20764 ▷ ▷ -end
20765 ▷ $(ORBITER) -v 2 -draw_matrix \
20766 ▷ ▷ -input_csv_file 10.3_object7_INP_flag_orbits.csv \
20767 ▷ ▷ -secondary_input_csv_file 10.3_object7_INP.csv \
20768 ▷ ▷ -box_width 16 -bit_depth 24 \
20769 ▷ ▷ -end
20770
20771
20772
20773
20774
20775 geo.10.3.c_lex.least:
20776 ▷ $(ORBITER) -v 10 \

```

```

20777 ▷ ▷ -draw_incidence_structure_description \
20778 ▷ ▷ ▷ -width 60 -width_10 6 -end \
20779 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
20780 ▷ ▷ -define Geo -geometry_builder \
20781 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
20782 ▷ ▷ ▷ -fname.GEO 10_3 \
20783 ▷ ▷ ▷ -test Test_lines \
20784 ▷ ▷ -end \
20785 ▷ ▷ -define C -combinatorial_object \
20786 ▷ ▷ ▷ -label geo_10_3 geo\_10\_3 \
20787 ▷ ▷ ▷ -file.of_incidence_geometries 10_3.inc 10 10 30 \
20788 ▷ ▷ -end \
20789 ▷ ▷ -with C -do \
20790 ▷ ▷ -combinatorial_object_activity \
20791 ▷ ▷ ▷ -canonical_form \
20792 ▷ ▷ ▷ ▷ -save_ago \
20793 ▷ ▷ ▷ ▷ -save_transversal \
20794 ▷ ▷ ▷ -end \
20795 ▷ ▷ -end \
20796 ▷ ▷ -with C -do \
20797 ▷ ▷ -combinatorial_object_activity \
20798 ▷ ▷ ▷ -report \
20799 ▷ ▷ ▷ ▷ -export_flag_orbits \
20800 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20801 ▷ ▷ ▷ ▷ -export_group_GAP \
20802 ▷ ▷ ▷ ▷ -show_TDO \
20803 ▷ ▷ ▷ ▷ -show_TDA \
20804 ▷ ▷ ▷ ▷ -lex_least Geo \
20805 ▷ ▷ ▷ -end \
20806 ▷ ▷ -end
20807 ▷ pdflatex geo_10_3_classification.tex
20808 ▷ $(OPEN) geo_10_3_classification.pdf
20809 ▷ $(ORBITER) -v 2 -draw_matrix \
20810 ▷ ▷ -input_csv_file geo_10_3_object7_TDA_flag_orbits.csv \
20811 ▷ ▷ -secondary_input_csv_file geo_10_3_object7_TDA.csv \
20812 ▷ ▷ -box_width 16 -bit_depth 24 \
20813 ▷ ▷ -end
20814 ▷ $(ORBITER) -v 2 -draw_matrix \
20815 ▷ ▷ -input_csv_file geo_10_3_object7_INP_flag_orbits.csv \
20816 ▷ ▷ -secondary_input_csv_file geo_10_3_object7_INP.csv \
20817 ▷ ▷ -box_width 16 -bit_depth 24 \
20818 ▷ ▷ -end
20819
20820 #10_3_object7_TDA_flag_orbits.csv
20821
20822 #0 1 2 10 13 14 20 25 26 31 33 35 41 44 46 52 53 56 62 67 68 74 77 79 85 88 89
20823
20824
20825
20826 geo_14_3.c:
20827 ▷ $(ORBITER) -v 2 \
20828 ▷ ▷ -draw_incidence_structure_description \
20829 ▷ ▷ ▷ -width 60 -width_10 6 -end \
20830 ▷ ▷ -define Test_lines -set -loop 4 15 1 -end \
20831 ▷ ▷ -define C -combinatorial_object \
20832 ▷ ▷ ▷ -label geo_14_3 geo\_14\_3 \
20833 ▷ ▷ ▷ -file.of_incidence_geometries 14_3.inc 14 14 42 \
20834 ▷ ▷ -end \
20835 ▷ ▷ -with C -do \

```

```

20836 ▷ ▷ -combinatorial_object_activity \
20837 ▷ ▷ ▷ -canonical_form \
20838 ▷ ▷ ▷ ▷ -save_ago \
20839 ▷ ▷ ▷ ▷ -save_transversal \
20840 ▷ ▷ ▷ -end \
20841 ▷ ▷ -end
20842
20843
20844 geo_15_3.c:
20845 ▷ $(ORBITER) -v 2 \
20846 ▷ ▷ -draw_incidence_structure_description \
20847 ▷ ▷ ▷ -width 50 -width_10 5 -end \
20848 ▷ ▷ -define C -combinatorial_object \
20849 ▷ ▷ ▷ -label geo_15_3 geo\_15\_3 \
20850 ▷ ▷ ▷ -file_of_incidence_geometries 15_3.inc 15 15 45 \
20851 ▷ ▷ -end \
20852 ▷ ▷ -with C -do \
20853 ▷ ▷ -combinatorial_object_activity \
20854 ▷ ▷ ▷ -canonical_form \
20855 ▷ ▷ ▷ -save_ago \
20856 ▷ ▷ -end
20857 ▷ pdflatex 15_3_classification.tex
20858 ▷ $(OPEN) 15_3_classification.pdf
20859
20860 TFC_24_3.c:
20861 ▷ echo $(FILE_24_3_TFC_INC) >24_3_TFC.inc
20862 ▷ $(ORBITER) -v 20 \
20863 ▷ ▷ -define C -combinatorial_object \
20864 ▷ ▷ ▷ -label 24_3_TFC 24\_3\_TFC \
20865 ▷ ▷ ▷ -file_of_incidence_geometries 24_3_TFC.inc 24 24 72 \
20866 ▷ ▷ -end \
20867 ▷ ▷ -with C -do \
20868 ▷ ▷ -combinatorial_object_activity \
20869 ▷ ▷ ▷ -canonical_form \
20870 ▷ ▷ ▷ -save_ago \
20871 ▷ ▷ ▷ -end \
20872 ▷ ▷ -end \
20873 ▷ ▷ -with C -do \
20874 ▷ ▷ -combinatorial_object_activity \
20875 ▷ ▷ ▷ -report \
20876 ▷ ▷ ▷ ▷ -export_flag_orbits \
20877 ▷ ▷ ▷ ▷ -show_TDO \
20878 ▷ ▷ ▷ ▷ -show_TDA \
20879 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20880 ▷ ▷ ▷ -end \
20881 ▷ ▷ -end
20882 ▷ pdflatex 24_3_TFC_classification.tex
20883 ▷ $(OPEN) 24_3_TFC_classification.pdf
20884 ▷ $(ORBITER) -v 2 -draw_matrix \
20885 ▷ ▷ -input_csv_file 24_3_TFC_object2_TDA_flag_orbits.csv \
20886 ▷ ▷ -secondary_input_csv_file 24_3_TFC_object2_TDA.csv \
20887 ▷ ▷ -box_width 40 -bit_depth 24 \
20888 ▷ ▷ -end
20889 ▷ $(OPEN) 24_3_TFC_object2_TDA_flag_orbits_draw.bmp
20890
20891
20892 geo_40_4.g4.c:
20893 ▷ $(ORBITER) -v 2 \
20894 ▷ ▷ -draw_incidence_structure_description \

```

```

20895 ▷ ▷ ▷ -width 50 -width_10 5 -end \
20896 ▷ ▷ -define C -combinatorial_object \
20897 ▷ ▷ ▷ -label 40_4_g4 40\4\g4 \
20898 ▷ ▷ ▷ -file_of_incidence_geometries \
20899 ▷ ▷ ▷ ▷ 40_4_g4.inc 40 40 160 \
20900 ▷ ▷ -end \
20901 ▷ ▷ -with C -do \
20902 ▷ ▷ -combinatorial_object_activity \
20903 ▷ ▷ ▷ -canonical_form \
20904 ▷ ▷ ▷ -save_ago \
20905 ▷ ▷ ▷ -end \
20906 ▷ ▷ -end \
20907 ▷ ▷ -with C -do \
20908 ▷ ▷ -combinatorial_object_activity \
20909 ▷ ▷ ▷ -report \
20910 ▷ ▷ ▷ ▷ -export_flag_orbits \
20911 ▷ ▷ ▷ ▷ -show_TDO \
20912 ▷ ▷ ▷ ▷ -show_TDA \
20913 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20914 ▷ ▷ ▷ -end \
20915 ▷ ▷ -end
20916 ▷ $(ORBITER) -v 2 \
20917 ▷ ▷ -loop L 0 2 1 \
20918 ▷ ▷ ▷ -draw_matrix \
20919 ▷ ▷ ▷ -input_csv_file 40_4_g4.object%L_TDA.flag_orbits.csv \
20920 ▷ ▷ ▷ -secondary_input_csv_file 40_4_g4.object%L_TDA.csv \
20921 ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
20922 ▷ ▷ ▷ -end \
20923 ▷ ▷ ▷ -system "convert \
20924 ▷ ▷ ▷ ▷ 40_4_g4.object%L_TDA.flag_orbits_draw.bmp \
20925 ▷ ▷ ▷ ▷ 40_4_g4.object%L_TDA.flag_orbits_draw.png" \
20926 ▷ ▷ -end_loop L
20927 ▷ pdflatex 40_4_g4_classification.tex
20928 ▷ $(OPEN) 40_4_g4_classification.pdf
20929
20930 geo_17_3_g4.c:
20931 ▷ $(ORBITER) -v 2 \
20932 ▷ ▷ -draw_incidence_structure_description \
20933 ▷ ▷ ▷ -width 50 -width_10 5 -end \
20934 ▷ ▷ -define C -combinatorial_object \
20935 ▷ ▷ ▷ -label 17_3_g4 17\3\g4 \
20936 ▷ ▷ ▷ -file_of_incidence_geometries 17_3_g4.inc 17 17 51 \
20937 ▷ ▷ -end \
20938 ▷ ▷ -with C -do \
20939 ▷ ▷ -combinatorial_object_activity \
20940 ▷ ▷ ▷ -canonical_form \
20941 ▷ ▷ ▷ ▷ -save_ago \
20942 ▷ ▷ ▷ -end \
20943 ▷ ▷ -end \
20944 ▷ ▷ -with C -do \
20945 ▷ ▷ -combinatorial_object_activity \
20946 ▷ ▷ ▷ -report \
20947 ▷ ▷ ▷ ▷ -export_flag_orbits \
20948 ▷ ▷ ▷ ▷ -show_TDO \
20949 ▷ ▷ ▷ ▷ -show_TDA \
20950 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20951 ▷ ▷ ▷ -end \
20952 ▷ ▷ -end
20953 ▷ pdflatex 17_3_g4_classification.tex

```



```

20954 ▷ $(OPEN) 17_3.g4.classification.pdf
20955
20956
20957 AG_2_3.c: AG_2_3.inc
20958 ▷ $(ORBITER) -v 2 \
20959 ▷ ▷ -define C -combinatorial_object \
20960 ▷ ▷ ▷ -label AG_2_3 AG\_2\_3 \
20961 ▷ ▷ ▷ -file_of_incidence_geometries \
20962 ▷ ▷ ▷ AG_2_3.inc 9 12 36 \
20963 ▷ ▷ -end \
20964 ▷ ▷ -with C -do \
20965 ▷ ▷ -combinatorial_object_activity \
20966 ▷ ▷ ▷ -canonical_form \
20967 ▷ ▷ ▷ ▷ -save_ago \
20968 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
20969 ▷ ▷ ▷ -end \
20970 ▷ ▷ -end \
20971 ▷ ▷ -with C -do \
20972 ▷ ▷ -combinatorial_object_activity \
20973 ▷ ▷ ▷ -report \
20974 ▷ ▷ ▷ ▷ -export_flag_orbits \
20975 ▷ ▷ ▷ ▷ -show_TDO \
20976 ▷ ▷ ▷ ▷ -show_TDA \
20977 ▷ ▷ ▷ ▷ -show_incidence_matrices \
20978 ▷ ▷ ▷ -end \
20979 ▷ ▷ -end
20980 ▷ pdflatex AG_2_3.classification.tex
20981 ▷ $(OPEN) AG_2_3.classification.pdf
20982 ▷ $(ORBITER) -v 2 -draw_matrix \
20983 ▷ ▷ -input_csv_file AG_2_3.object0_INP_flag_orbits.csv \
20984 ▷ ▷ -secondary_input_csv_file AG_2_3.object0_INP.csv \
20985 ▷ ▷ -box_width 40 -bit_depth 24 \
20986 ▷ ▷ -end
20987 ▷ $(OPEN) AG_2_3.object0_INP_flag_orbits.draw.bmp
20988
20989
20990
20991
20992 geo.LSQ6.c: LSQ6.inc
20993 ▷ $(ORBITER) -v 3 \
20994 ▷ ▷ -draw_incidence_structure_description \
20995 ▷ ▷ ▷ -width 60 -width_10 6 -end \
20996 ▷ ▷ -define C -combinatorial_object \
20997 ▷ ▷ ▷ -label LSQ6 LSQ6 \
20998 ▷ ▷ ▷ -file_of_incidence_geometries \
20999 ▷ ▷ ▷ LSQ6.inc 18 39 126 \
21000 ▷ ▷ -end \
21001 ▷ ▷ -with C -do \
21002 ▷ ▷ -combinatorial_object_activity \
21003 ▷ ▷ ▷ -canonical_form \
21004 ▷ ▷ ▷ ▷ -save_ago \
21005 ▷ ▷ ▷ ▷ -save_transversal \
21006 ▷ ▷ ▷ -end \
21007 ▷ ▷ -end \
21008 ▷ ▷ -with C -do \
21009 ▷ ▷ -combinatorial_object_activity \
21010 ▷ ▷ ▷ -report \
21011 ▷ ▷ ▷ ▷ -export_flag_orbits \
21012 ▷ ▷ ▷ ▷ -show_incidence_matrices \

```

```

21013 ▷ ▷ ▷ ▷ -export_group_GAP \
21014 ▷ ▷ ▷ -end \
21015 ▷ ▷ -end
21016 ▷ pdflatex LSQ6_classification.tex
21017 ▷ $(OPEN) LSQ6_classification.pdf
21018 ▷ $(ORBITER) -v 2 \
21019 ▷ ▷ -loop L 0 12 1 \
21020 ▷ ▷ ▷ -draw_matrix \
21021 ▷ ▷ ▷ -input_csv_file LSQ6_object%L_TDA_flag_orbits.csv \
21022 ▷ ▷ ▷ -secondary_input_csv_file LSQ6_object%L_TDA.csv \
21023 ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
21024 ▷ ▷ ▷ -end \
21025 ▷ ▷ ▷ -system "convert \
21026 ▷ ▷ ▷ ▷ LSQ6_object%L_TDA_flag_orbits_draw.bmp \
21027 ▷ ▷ ▷ ▷ LSQ6_object%L_TDA_flag_orbits_draw.png" \
21028 ▷ ▷ -end_loop L
21029 ▷ $(ORBITER) -v 2 \
21030 ▷ ▷ -loop L 0 12 1 \
21031 ▷ ▷ ▷ -draw_matrix \
21032 ▷ ▷ ▷ -input_csv_file LSQ6_object%L_INP_flag_orbits.csv \
21033 ▷ ▷ ▷ -secondary_input_csv_file LSQ6_object%L_INP.csv \
21034 ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
21035 ▷ ▷ ▷ -end \
21036 ▷ ▷ -end_loop L
21037
21038
21039
21040
21041
21042
21043
21044
21045
21046
21047
21048
21049
21050 #####
21051 # Section 16.4: Objects from Design Theory
21052
21053
21054 SECTION_OBJECTS_FROM_DESIGN_THEORY:
21055
21056
21057 test_16_4:
21058 ▷ make LS_AG_2_3_solutions_classify
21059 ▷ make design_27c
21060 ▷ make design_PG_2_3_canonical
21061 ▷ make wreath_product_designs_n8_k6_c
21062 ▷ make wreath_product_designs_n4_k2_c
21063
21064
21065
21066 LS_AG_2_3_solutions_classify:
21067 ▷ $(ORBITER) -v 2 \
21068 ▷ ▷ -draw_incidence_structure_description \
21069 ▷ ▷ ▷ -width 20 -width_10 2 -end \
21070 ▷ ▷ -define C -combinatorial_object \
21071 ▷ ▷ ▷ -label LS_AG_2_3 LS\AG\2\3 \

```

```

21072 ▷ ▷ ▷ -file_of_designs \
21073 ▷ ▷ ▷ solutions.csv 9 84 3 12 \
21074 ▷ ▷ -end \
21075 ▷ ▷ -with C -do \
21076 ▷ ▷ -combinatorial_object_activity \
21077 ▷ ▷ ▷ -canonical_form \
21078 ▷ ▷ ▷ ▷ -save_ago \
21079 ▷ ▷ ▷ ▷ -save_transversal \
21080 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
21081 ▷ ▷ ▷ -end \
21082 ▷ ▷ -end \
21083 ▷ ▷ -with C -do \
21084 ▷ ▷ -combinatorial_object_activity \
21085 ▷ ▷ ▷ -report \
21086 ▷ ▷ ▷ ▷ -export_flag_orbits \
21087 ▷ ▷ ▷ ▷ -show_TDO \
21088 ▷ ▷ ▷ -end \
21089 ▷ ▷ -end
21090 ▷ pdflatex LS_AG_2_3_classification.tex
21091 ▷ $(OPEN) LS_AG_2_3_classification.pdf
21092 ▷ $(ORBITER) -v 2 -draw_matrix \
21093 ▷ ▷ -input_csv_file LS_AG_2_3_object0_INP_flag_orbits.csv \
21094 ▷ ▷ -secondary_input_csv_file LS_AG_2_3_object0_INP.csv \
21095 ▷ ▷ -box_width 12 -bit_depth 24 \
21096 ▷ ▷ -end
21097 ▷ $(OPEN) LS_AG_2_3_object0_INP_flag_orbits.draw.bmp
21098 ▷ $(ORBITER) -v 2 -draw_matrix \
21099 ▷ ▷ -input_csv_file LS_AG_2_3_object1_INP_flag_orbits.csv \
21100 ▷ ▷ -secondary_input_csv_file LS_AG_2_3_object1_INP.csv \
21101 ▷ ▷ -box_width 12 -bit_depth 24 \
21102 ▷ ▷ -end
21103 ▷ $(OPEN) LS_AG_2_3_object1_INP_flag_orbits.draw.bmp
21104
21105
21106
21107
21108
21109 design_27c:
21110 ▷ $(ORBITER) -v 4 \
21111 ▷ ▷ -define C -combinatorial_object \
21112 ▷ ▷ ▷ -label design design \
21113 ▷ ▷ ▷ -set_of_points "2,56,30,112,253,90,440,508" \
21114 ▷ ▷ -end \
21115 ▷ ▷ -define F -finite_field -q 27 \
21116 ▷ ▷ ▷ -override_polynomial 46 \
21117 ▷ ▷ -end \
21118 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
21119 ▷ ▷ -with C -do \
21120 ▷ ▷ -combinatorial_object_activity \
21121 ▷ ▷ ▷ -canonical_form_PG P \
21122 ▷ ▷ ▷ -end \
21123 ▷ ▷ -end \
21124 ▷ ▷ -with C -do \
21125 ▷ ▷ -combinatorial_object_activity \
21126 ▷ ▷ ▷ -report \
21127 ▷ ▷ -end
21128 ▷ pdflatex design_classification.tex
21129 ▷ $(OPEN) design_classification.pdf
21130

```

```

21131
21132
21133 design_PG_2_3_canonical:
21134 ▷ $(ORBITER) -v 3 \
21135 ▷ ▷ -define F -finite_field -q 3 -end \
21136 ▷ ▷ -define D -design -field F -family PG_2_q -end \
21137 ▷ ▷ -with D -do \
21138 ▷ ▷ ▷ -design_activity \
21139 ▷ ▷ ▷ ▷ -export_inc \
21140 ▷ ▷ ▷ -end \
21141 ▷ ▷ -end
21142 ▷ $(ORBITER) -v 3 \
21143 ▷ ▷ -draw_incidence_structure_description \
21144 ▷ ▷ ▷ -width 60 -width_10 6 -end \
21145 ▷ ▷ -define C -combinatorial_object \
21146 ▷ ▷ ▷ -label PG_2_3 PG\_2\_3 \
21147 ▷ ▷ ▷ -file_of_incidence_geometries \
21148 ▷ ▷ ▷ ▷ PG_2_3_inc.txt 13 13 52 \
21149 ▷ ▷ -end \
21150 ▷ ▷ -with C -do \
21151 ▷ ▷ -combinatorial_object_activity \
21152 ▷ ▷ ▷ -canonical_form \
21153 ▷ ▷ ▷ ▷ -save_ago \
21154 ▷ ▷ ▷ ▷ -save_transversal \
21155 ▷ ▷ ▷ -end \
21156 ▷ ▷ -end \
21157 ▷ ▷ -with C -do \
21158 ▷ ▷ -combinatorial_object_activity \
21159 ▷ ▷ ▷ -report \
21160 ▷ ▷ ▷ ▷ -export_flag_orbits \
21161 ▷ ▷ ▷ ▷ -show_incidence_matrices \
21162 ▷ ▷ ▷ ▷ -export_group_GAP \
21163 ▷ ▷ ▷ -end \
21164 ▷ ▷ -end
21165 ▷ pdflatex PG_2_3_classification.tex
21166 ▷ $(OPEN) PG_2_3_classification.pdf
21167 ▷ $(ORBITER) -v 2 -draw_matrix \
21168 ▷ ▷ -input_csv_file PG_2_3_object0_TDA_flag_orbits.csv \
21169 ▷ ▷ -secondary_input_csv_file PG_2_3_object0_TDA.csv \
21170 ▷ ▷ -box_width 32 -bit_depth 24 \
21171 ▷ ▷ -end
21172 ▷ $(OPEN) PG_2_3_object0_TDA_flag_orbits.draw.bmp
21173 ▷
21174
21175
21176 wreath_product_designs_n4_k2_c: wreath_product_designs_n4_k2_inc.txt
21177 ▷ $(ORBITER) -v 10 \
21178 ▷ ▷ -draw_incidence_structure_description \
21179 ▷ ▷ ▷ -width 60 -width_10 6 -end \
21180 ▷ ▷ -define C -combinatorial_object \
21181 ▷ ▷ ▷ -label wreath_4_2 wreath\_4\_2 \
21182 ▷ ▷ ▷ -file_of_incidence_geometries \
21183 ▷ ▷ ▷ wreath_product_designs_n4_k2_inc.txt \
21184 ▷ ▷ ▷ 8 12 24 \
21185 ▷ ▷ -end \
21186 ▷ ▷ -with C -do \
21187 ▷ ▷ -combinatorial_object_activity \
21188 ▷ ▷ ▷ -canonical_form \
21189 ▷ ▷ ▷ ▷ -save_ago \

```

```

21190 ▷ ▷ ▷ ▷ -save_transversal \
21191 ▷ ▷ ▷ -end \
21192 ▷ ▷ -end \
21193 ▷ ▷ -with C -do \
21194 ▷ ▷ -combinatorial_object_activity \
21195 ▷ ▷ ▷ -report \
21196 ▷ ▷ ▷ ▷ -export_flag_orbits \
21197 ▷ ▷ ▷ ▷ -show_incidence_matrices \
21198 ▷ ▷ ▷ ▷ -export_group_GAP \
21199 ▷ ▷ ▷ -end \
21200 ▷ ▷ -end
21201 ▷ pdflatex wreath_4_2_classification.tex
21202 ▷ $(OPEN) wreath_4_2_classification.pdf
21203
21204
21205 wreath_product_designs_n8_k6_c: wreath_product_designs_n8_k6_inc.txt
21206 ▷ $(ORBITER) -v 10 \
21207 ▷ ▷ -draw_incidence_structure_description \
21208 ▷ ▷ ▷ -width 60 -width_10 6 -end \
21209 ▷ ▷ -define C -combinatorial_object \
21210 ▷ ▷ ▷ -label wreath_8_6 wreath\_8\_6 \
21211 ▷ ▷ ▷ -file_of_incidence_geometries \
21212 ▷ ▷ ▷ wreath_product_designs_n8_k6_inc.txt \
21213 ▷ ▷ ▷ 16 3920 23520 \
21214 ▷ ▷ -end \
21215 ▷ ▷ -with C -do \
21216 ▷ ▷ -combinatorial_object_activity \
21217 ▷ ▷ ▷ -canonical_form \
21218 ▷ ▷ ▷ ▷ -save_ago \
21219 ▷ ▷ ▷ ▷ -save_transversal \
21220 ▷ ▷ ▷ -end \
21221 ▷ ▷ -end \
21222 ▷ ▷ -with C -do \
21223 ▷ ▷ -combinatorial_object_activity \
21224 ▷ ▷ ▷ -report \
21225 ▷ ▷ ▷ ▷ -export_flag_orbits \
21226 ▷ ▷ ▷ ▷ -export_group_GAP \
21227 ▷ ▷ ▷ -end \
21228 ▷ ▷ -end
21229 ▷ pdflatex wreath_8_6_classification.tex
21230 ▷ $(OPEN) wreath_8_6_classification.pdf
21231
21232
21233
21234
21235
21236
21237 #####
21238 # Section 16.5: Linear Codes
21239
21240
21241 SECTION_CANONICAL_FORMS_OF_LINEAR_CODES:
21242
21243
21244 test_16_5:
21245 ▷ make code_3_2_aut
21246 ▷ make code_6_3_aut
21247 ▷ make RM_3_1_group
21248 ▷ make RM_3_1_group_and_diagram

```

```

21249 ▷ make RM_4_1_group
21250 ▷ make RS_6_4_7_group
21251 ▷ make GV_n15_k6_d5_group
21252 ▷ make code_n15_k6_d6_a_group
21253 ▷ make code_n15_k6_d6_b_group
21254
21255
21256
21257 code_3_2_aut:
21258 ▷ $(ORBITER) -v 2 \
21259 ▷ ▷ -define F -finite_field -q 2 -end \
21260 ▷ ▷ -define genma -vector -field F -format 2 \
21261 ▷ ▷ ▷ -dense $(CODE_N3_K2_Q2_GENMA) \
21262 ▷ ▷ -end \
21263 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
21264 ▷ ▷ -with P -do \
21265 ▷ ▷ -projective_space_activity \
21266 ▷ ▷ ▷ -canonical_form_of_code \
21267 ▷ ▷ ▷ ▷ "3_2" genma -save_ago -label "3_2" \
21268 ▷ ▷ ▷ ▷ -classification_prefix "3_2" \
21269 ▷ ▷ ▷ -end \
21270 ▷ ▷ -end
21271 ▷ pdflatex 3_2_classification.tex
21272 ▷ $(OPEN) 3_2_classification.pdf
21273 ▷ $(ORBITER) -v 2 -draw_matrix \
21274 ▷ ▷ -input_csv_file 3_2_object0_TDA_flag_orbits.csv \
21275 ▷ ▷ -secondary_input_csv_file 3_2_object0_TDA.csv \
21276 ▷ ▷ -box_width 16 -bit_depth 24 \
21277 ▷ ▷ -end
21278 ▷ $(OPEN) 3_2_object0_TDA_flag_orbits.draw.bmp
21279
21280
21281
21282
21283 code_6_3_aut:
21284 ▷ $(ORBITER) -v 2 \
21285 ▷ ▷ -define F -finite_field -q 2 -end \
21286 ▷ ▷ -define genma -vector -field F -format 3 \
21287 ▷ ▷ ▷ -compact $(CODE_N6_K3_Q2_GENMA) \
21288 ▷ ▷ -end \
21289 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
21290 ▷ ▷ -with P -do \
21291 ▷ ▷ -projective_space_activity \
21292 ▷ ▷ ▷ -canonical_form_of_code \
21293 ▷ ▷ ▷ ▷ "6_3" genma -save_ago -label "6_3" \
21294 ▷ ▷ ▷ ▷ -classification_prefix "6_3" \
21295 ▷ ▷ ▷ -end \
21296 ▷ ▷ -end
21297 ▷ pdflatex 6_3_classification.tex
21298 ▷ $(OPEN) 6_3_classification.pdf
21299 ▷ $(ORBITER) -v 2 -draw_matrix \
21300 ▷ ▷ -input_csv_file 6_3_object0_TDA_flag_orbits.csv \
21301 ▷ ▷ -secondary_input_csv_file 6_3_object0_TDA.csv \
21302 ▷ ▷ -box_width 16 -bit_depth 24 \
21303 ▷ ▷ -end
21304 ▷ $(OPEN) 6_3_object0_TDA_flag_orbits.draw.bmp
21305
21306 # group of order 24
21307

```

```

21308
21309 RM_3_1_group:
21310 ▷ $(ORBITER) -v 2 \
21311 ▷ ▷ -define F -finite_field -q 2 -end \
21312 ▷ ▷ -define genma -vector -field F -format 4 \
21313 ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
21314 ▷ ▷ -end \
21315 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
21316 ▷ ▷ -with P -do \
21317 ▷ ▷ -projective_space_activity \
21318 ▷ ▷ ▷ -canonical_form_of_code \
21319 ▷ ▷ ▷ ▷ "RM_3_1" genma -save_ago -label "RM_3_1" \
21320 ▷ ▷ ▷ ▷ -classification_prefix "RM_3_1" \
21321 ▷ ▷ ▷ -end \
21322 ▷ ▷ -end
21323 ▷ pdflatex RM_3_1_classification.tex
21324 ▷ $(OPEN) RM_3_1_classification.pdf
21325
21326 # group order 1344
21327 #RM_3_1_object0_INP_flag_orbits.csv
21328
21329 RM_3_1_group_and_diagram:
21330 ▷ $(ORBITER) -v 2 \
21331 ▷ ▷ -define F -finite_field -q 2 -end \
21332 ▷ ▷ -define genma -vector -field F -format 4 \
21333 ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
21334 ▷ ▷ -end \
21335 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
21336 ▷ ▷ -with P -do \
21337 ▷ ▷ -projective_space_activity \
21338 ▷ ▷ ▷ -canonical_form_of_code \
21339 ▷ ▷ ▷ ▷ "RM_3_1" genma -save_ago -label "RM_3_1" \
21340 ▷ ▷ ▷ ▷ -classification_prefix "RM_3_1" \
21341 ▷ ▷ ▷ -end \
21342 ▷ ▷ -end
21343 ▷ pdflatex RM_3_1_classification.tex
21344 ▷ $(OPEN) RM_3_1_classification.pdf
21345 ▷ $(ORBITER) -v 2 -draw_matrix \
21346 ▷ ▷ -input_csv_file RM_3_1_object0_INP_flag_orbits.csv \
21347 ▷ ▷ -secondary_input_csv_file RM_3_1_object0_INP.csv \
21348 ▷ ▷ -box_width 16 -bit_depth 24 \
21349 ▷ ▷ -end
21350 ▷ $(ORBITER) -v 2 -draw_matrix \
21351 ▷ ▷ -input_csv_file RM_3_1_object0_TDA_flag_orbits.csv \
21352 ▷ ▷ -secondary_input_csv_file RM_3_1_object0_TDA.csv \
21353 ▷ ▷ -box_width 16 -bit_depth 24 \
21354 ▷ ▷ -end
21355 ▷ $(OPEN) RM_3_1_object0_INP_flag_orbits.draw.bmp
21356 ▷ $(OPEN) RM_3_1_object0_TDA_flag_orbits.draw.bmp
21357
21358
21359
21360 RM_4_1_group:
21361 ▷ $(ORBITER) -v 2 \
21362 ▷ ▷ -define F -finite_field -q 2 -end \
21363 ▷ ▷ -define genma -vector -field F -format 5 \
21364 ▷ ▷ ▷ -compact $(CODE_RM_4_1_GENMA) \
21365 ▷ ▷ -end \
21366 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \

```

```

21367 ▷ ▷ -with P -do \
21368 ▷ ▷ -projective_space_activity \
21369 ▷ ▷ ▷ -canonical_form_of_code \
21370 ▷ ▷ ▷ ▷ "RM_4.1" genma -save_ago -label "RM_4.1" \
21371 ▷ ▷ ▷ ▷ -classification_prefix "RM_4.1" \
21372 ▷ ▷ ▷ -end \
21373 ▷ ▷ -end
21374 ▷ pdflatex RM_4.1_classification.tex
21375 ▷ $(OPEN) RM_4.1_classification.pdf
21376 ▷ $(ORBITER) -v 2 -draw_matrix \
21377 ▷ ▷ -input_csv_file RM_4.1_object0_INP_flag_orbits.csv \
21378 ▷ ▷ -secondary_input_csv_file RM_4.1_object0_INP.csv \
21379 ▷ ▷ -box_width 16 -bit_depth 24 \
21380 ▷ ▷ -end
21381 ▷ $(ORBITER) -v 2 -draw_matrix \
21382 ▷ ▷ -input_csv_file RM_4.1_object0_TDA_flag_orbits.csv \
21383 ▷ ▷ -secondary_input_csv_file RM_4.1_object0_TDA.csv \
21384 ▷ ▷ -box_width 16 -bit_depth 24 \
21385 ▷ ▷ -end
21386 ▷ $(OPEN) RM_4.1_object0_INP_flag_orbits.draw.bmp
21387 ▷ $(OPEN) RM_4.1_object0_TDA_flag_orbits.draw.bmp
21388
21389
21390 # group order 322560 = 24*30*28*16
21391
21392
21393 RS_6.4.7.group:
21394 ▷ $(ORBITER) -v 2 \
21395 ▷ ▷ -define F -finite_field -q 7 -end \
21396 ▷ ▷ -define genma -vector -field F -format 4 \
21397 ▷ ▷ ▷ -compact $(CODE_RS_6.4.7) \
21398 ▷ ▷ -end \
21399 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
21400 ▷ ▷ -with P -do \
21401 ▷ ▷ -projective_space_activity \
21402 ▷ ▷ ▷ -canonical_form_of_code \
21403 ▷ ▷ ▷ ▷ "RS_6" genma -save_ago -label "RS_6" \
21404 ▷ ▷ ▷ ▷ -classification_prefix "RS_6" \
21405 ▷ ▷ ▷ -end \
21406 ▷ ▷ -end
21407
21408
21409 GV_n15.k6.d5.group:
21410 ▷ $(ORBITER) -v 2 \
21411 ▷ ▷ -define F -finite_field -q 2 -end \
21412 ▷ ▷ -define genma -vector -field F -format 6 \
21413 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
21414 ▷ ▷ -end \
21415 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
21416 ▷ ▷ -with P -do \
21417 ▷ ▷ -projective_space_activity \
21418 ▷ ▷ ▷ -canonical_form_of_code \
21419 ▷ ▷ ▷ ▷ "GV_n15.k6.d5" genma -save_ago -label "GV_n15.k6.d5" \
21420 ▷ ▷ ▷ ▷ -classification_prefix "GV_n15.k6.d5" \
21421 ▷ ▷ ▷ -end \
21422 ▷ ▷ -end
21423 ▷ #pdflatex GV_n15.k6.d5_classification.tex
21424 ▷ #$(OPEN) GV_n15.k6.d5_classification.pdf
21425

```



```

21426 # ToDo latex file too large
21427 #ago=12
21428
21429
21430
21431 code_n15_k6_d6_a.group:
21432 ▷ $(ORBITER) -v 2 \
21433 ▷ ▷ -define F -finite_field -q 2 -end \
21434 ▷ ▷ -define genma -vector -field F -format 6 \
21435 ▷ ▷ ▷ -compact $(CODE_15_6_6.A) \
21436 ▷ ▷ -end \
21437 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
21438 ▷ ▷ -with P -do \
21439 ▷ ▷ -projective_space_activity \
21440 ▷ ▷ ▷ -canonical_form_of_code \
21441 ▷ ▷ ▷ ▷ "n15_k6_d6_a" genma -save_ago -label "n15_k6_d6_a" \
21442 ▷ ▷ ▷ ▷ -classification_prefix "n15_k6_d6_a" \
21443 ▷ ▷ ▷ -end \
21444 ▷ ▷ -end
21445 ▷ #pdflatex n15_k6_d6_a.classification.tex
21446 ▷ #$(OPEN) n15_k6_d6_a.classification.pdf
21447
21448
21449 code_n15_k6_d6_b.group:
21450 ▷ $(ORBITER) -v 2 \
21451 ▷ ▷ -define F -finite_field -q 2 -end \
21452 ▷ ▷ -define genma -vector -field F -format 6 \
21453 ▷ ▷ ▷ -compact $(CODE_15_6_6.B) \
21454 ▷ ▷ -end \
21455 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
21456 ▷ ▷ -with P -do \
21457 ▷ ▷ -projective_space_activity \
21458 ▷ ▷ ▷ -canonical_form_of_code \
21459 ▷ ▷ ▷ ▷ "n15_k6_d6_b" genma -save_ago -label "n15_k6_d6_b" \
21460 ▷ ▷ ▷ ▷ -classification_prefix "n15_k6_d6_b" \
21461 ▷ ▷ ▷ -end \
21462 ▷ ▷ -end
21463 ▷ #pdflatex n15_k6_d6_b.classification.tex
21464 ▷ #$(OPEN) n15_k6_d6_b.classification.pdf
21465
21466
21467
21468
21469 #####
21470 # Section 16.6: General Codes
21471
21472
21473 SECTION_CANONICAL_FORMS_OF_GENERAL_CODES:
21474
21475 test_16_6:
21476 ▷ make Hamming_graph_7_with_Hamming_code
21477
21478
21479
21480 Hamming_graph_7_with_Hamming_code:
21481 ▷ $(ORBITER) -v 2 \
21482 ▷ ▷ -define G -graph -Hamming 7 2 \
21483 ▷ ▷ ▷ -subset "Hamming_code" "\\_with\\_Hamming\\_code" \
21484 ▷ ▷ ▷ $(HAMMING_CODE_CODEWORDS) -end \

```

```

21485 ▷ ▷ -with G -do \
21486 ▷ ▷ -graph_theoretic.activity -export_csv -end \
21487 ▷ ▷ -with G -do \
21488 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
21489 ▷ ▷ -with G -do \
21490 ▷ ▷ -graph_theoretic.activity -save -end \
21491 ▷ ▷ -with G -do \
21492 ▷ ▷ -graph_theoretic.activity -automorphism_group -end
21493 ▷ pdflatex Hamming_7.2.Hamming_code_report.tex
21494 ▷ $(OPEN) Hamming_7.2.Hamming_code_report.pdf
21495
21496 # group of order 2688 = 16 * 168
21497
21498
21499
21500
21501 #####
21502 # Section 16.7: Graphs
21503
21504
21505 SECTION_CANONICAL_FORMS_OF_GRAPHS:
21506
21507
21508 test_16.7:
21509 ▷ make Cycle_13_aut
21510 ▷ make Queens_graph_aut
21511 ▷ make Queens_graph_aut.orbits.draw
21512 ▷ make inversion_graph
21513 ▷ make Chain_232_aut
21514 ▷ #make JK_graph_pp16_1
21515 ▷ #make JK_graph_pp16_2
21516 ▷ #make JK_graph_pp16_9
21517 ▷ #make JK_graph_grid_3_3
21518 ▷ #make JK_graph_sts_13
21519 ▷ make halljanko315.gens.csv
21520 ▷ make HJ_group_and_orbits
21521 ▷ make HJ_orbital_graph_3
21522 ▷ make PGO_5.2_graph_group
21523
21524
21525 Cycle_13_aut:
21526 ▷ $(ORBITER) -v 2 \
21527 ▷ ▷ -define Gamma -graph -cycle 13 -end \
21528 ▷ ▷ -with Gamma -do \
21529 ▷ ▷ ▷ -graph_theoretic.activity -automorphism_group \
21530 ▷ ▷ -end \
21531
21532
21533 Queens_graph_aut:
21534 ▷ $(ORBITER) -v 2 \
21535 ▷ ▷ -define Gamma -graph -non_attacking_queens_graph 8 -end \
21536 ▷ ▷ -with Gamma -do \
21537 ▷ ▷ ▷ -graph_theoretic.activity -automorphism_group \
21538 ▷ ▷ -end
21539 ▷ pdflatex non_attacking_queens_graph_8_report.tex
21540 ▷ $(OPEN) non_attacking_queens_graph_8_report.pdf
21541
21542 #non_attacking_queens_graph_8_orbits.csv
21543

```

```

21544
21545 Queens_graph_aut_orbits_draw:
21546 ▷ $(ORBITER) -v 2 -reformat \
21547 ▷ ▷ non_attacking_queens_graph_8_orbits.csv \
21548 ▷ ▷ Queens_orbits_on_points_8x8.csv 8
21549 ▷ $(ORBITER) -v 2 \
21550 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
21551 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
21552 ▷ ▷ -draw_matrix \
21553 ▷ ▷ -input_csv_file Queens_orbits_on_points_8x8.csv \
21554 ▷ ▷ -box_width 40 -bit_depth 24 \
21555 ▷ ▷ ▷ -partition 1 \
21556 ▷ ▷ ▷ all_one_r all_one_c \
21557 ▷ ▷ -end
21558 ▷ $(OPEN) Queens_orbits_on_points_8x8_draw.bmp
21559
21560
21561
21562 inversion_graph:
21563 ▷ $(ORBITER) -v 6 \
21564 ▷ ▷ -define G -graph \
21565 ▷ ▷ ▷ -inversion_graph "1,0,2,3" \
21566 ▷ ▷ -end \
21567 ▷ ▷ -with G -do \
21568 ▷ ▷ ▷ -graph_theoretic_activity -properties \
21569 ▷ ▷ -end \
21570 ▷ ▷ -with G -do \
21571 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
21572 ▷ ▷ -end
21573
21574
21575
21576 Chain_232_aut:
21577 ▷ $(ORBITER) -v 2 \
21578 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
21579 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
21580 ▷ ▷ -define Gamma -graph \
21581 ▷ ▷ ▷ -chain_graph P1 P2 \
21582 ▷ ▷ -end \
21583 ▷ ▷ -with Gamma -do \
21584 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
21585 ▷ ▷ -end
21586 ▷ pdflatex chain_graph_report.tex
21587 ▷ $(OPEN) chain_graph_report.pdf
21588
21589
21590
21591
21592 JK_graph_pp16_1:
21593 ▷ $(ORBITER) -v 2 \
21594 ▷ ▷ -define Gamma -graph -load_dimacs \
21595 ▷ ▷ ../JUNTTILA_KASKI/benchmarks/pp/pp16-1 \
21596 ▷ ▷ -end \
21597 ▷ ▷ -with Gamma -do \
21598 ▷ ▷ -graph_theoretic_activity -save -end \
21599 ▷ ▷ -with Gamma -do \
21600 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
21601
21602 # go=34217164800

```

```

21603 #nauty_interface_with_group::create_automorphism_group_of_graph_with_partition_and_labeling: nb.backtrack1 = 6
21604 #nauty_interface_with_group::create_automorphism_group_of_graph_with_partition_and_labeling: nb.backtrack2 = 134
21605
21606 JK_graph_pp16.2:
21607 ▷ $(ORBITER) -v 2 \
21608 ▷ ▷ -define Gamma -graph -load_dimacs \
21609 ▷ ▷ ../JUNTTILA.KASKI/benchmarks/pp/pp16-2 \
21610 ▷ ▷ -end \
21611 ▷ ▷ -with Gamma -do \
21612 ▷ ▷ -graph_theoretic_activity -save -end \
21613 ▷ ▷ -with Gamma -do \
21614 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
21615
21616 # does not finish
21617
21618 JK_graph_pp16.9:
21619 ▷ $(ORBITER) -v 2 \
21620 ▷ ▷ -define Gamma -graph -load_dimacs \
21621 ▷ ▷ ../JUNTTILA.KASKI/benchmarks/pp/pp16-9 \
21622 ▷ ▷ -end \
21623 ▷ ▷ -with Gamma -do \
21624 ▷ ▷ -graph_theoretic_activity -save -end \
21625 ▷ ▷ -with Gamma -do \
21626 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
21627
21628
21629 JK_graph_grid.3.3:
21630 ▷ $(ORBITER) -v 2 \
21631 ▷ ▷ -define Gamma -graph -load_dimacs \
21632 ▷ ▷ ../JUNTTILA.KASKI/benchmarks/grid/grid-w-3-3 \
21633 ▷ ▷ -end \
21634 ▷ ▷ -with Gamma -do \
21635 ▷ ▷ -graph_theoretic_activity -save -end \
21636 ▷ ▷ -with Gamma -do \
21637 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
21638
21639
21640
21641
21642 JK_graph_sts.13:
21643 ▷ $(ORBITER) -v 2 \
21644 ▷ ▷ -define Gamma -graph -load_dimacs \
21645 ▷ ▷ ▷ ../JUNTTILA.KASKI/benchmarks/srg/sts-13 \
21646 ▷ ▷ -end \
21647 ▷ ▷ -with Gamma -do \
21648 ▷ ▷ -graph_theoretic_activity -save -end \
21649 ▷ ▷ -with Gamma -do \
21650 ▷ ▷ -graph_theoretic_activity -automorphism_group -end
21651 ▷ make ORBITER_PATH=$(ORBITER_EXE_PATH) -f sts-13_group.makefile sts-13
21652
21653
21654
21655 halljanko315.gens.csv:
21656 ▷ $(ORBITER) -v 6 \
21657 ▷ ▷ -define G -graph \
21658 ▷ ▷ ▷ -load_csv.no.border \
21659 ▷ ▷ ▷ halljanko315.csv \

```

```

21660 ▷ ▷ -end \
21661 ▷ ▷ -with G -do \
21662 ▷ ▷ ▷ -graph.theoretic.activity -automorphism_group \
21663 ▷ ▷ -end \
21664 ▷ ▷ -with G -do \
21665 ▷ ▷ ▷ -graph.theoretic.activity -properties \
21666 ▷ ▷ -end
21667
21668
21669
21670 HJ.group_and_orbits:
21671 ▷ ▷ $(ORBITER) -v 2 \
21672 ▷ ▷ -define Control -poset_classification_control \
21673 ▷ ▷ ▷ -W \
21674 ▷ ▷ ▷ -problem_label HJ.orbits \
21675 ▷ ▷ ▷ -depth 2 \
21676 ▷ ▷ -end \
21677 ▷ ▷ -define gens -vector -file \
21678 ▷ ▷ ▷ halljanko315_gens.csv -end \
21679 ▷ ▷ -define G -permutation_group \
21680 ▷ ▷ ▷ -bsgs halljanko315 "File\_halljanko315" \
21681 ▷ ▷ ▷ 315 1209600 "0,1,2" 6 gens \
21682 ▷ ▷ -end \
21683 ▷ ▷ -define Orb -orbits -group G \
21684 ▷ ▷ ▷ -on_subsets 2 Control \
21685 ▷ ▷ -end
21686
21687 #ROW,REP,AGO,OL
21688 #0,"0,1",96,12600
21689 #1,"0,2",48,25200
21690 #2,"0,4",768,1575
21691 #3,"0,8",120,10080
21692 #END
21693
21694 HJ.orbital_graph_3:
21695 ▷ ▷ $(ORBITER) -v 2 \
21696 ▷ ▷ -define gens -vector -file \
21697 ▷ ▷ ▷ halljanko315_gens.csv -end \
21698 ▷ ▷ -define G -permutation_group \
21699 ▷ ▷ ▷ -bsgs halljanko315 "File\_halljanko315" \
21700 ▷ ▷ ▷ 315 1209600 "0,1,2" 6 gens \
21701 ▷ ▷ -end \
21702 ▷ ▷ -define Gamma -graph \
21703 ▷ ▷ ▷ -orbital_graph G 3 \
21704 ▷ ▷ -end \
21705 ▷ ▷ -with Gamma -do \
21706 ▷ ▷ ▷ -graph.theoretic.activity \
21707 ▷ ▷ ▷ -properties \
21708 ▷ ▷ -end \
21709 ▷ ▷ -with Gamma -do \
21710 ▷ ▷ ▷ -graph.theoretic.activity \
21711 ▷ ▷ ▷ -save \
21712 ▷ ▷ -end
21713
21714 #Group.Perm315.Orbital_3.colored_graph
21715 #Degree type: (64^{315} )
21716
21717
21718

```

```

21719
21720
21721 PGO_5_2_graph_group: 0_5_2_incidence_matrix.csv
21722 ▷ $(ORBITER) -v 3 \
21723 ▷ ▷ -define Inc -vector -file 0_5_2_incidence_matrix.csv -end \
21724 ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
21725 ▷ ▷ -with Gamma -do \
21726 ▷ ▷ -graph_theoretic_activity \
21727 ▷ ▷ ▷ -automorphism_group \
21728 ▷ ▷ -end \
21729 ▷ ▷ -with Gamma -do \
21730 ▷ ▷ -graph_theoretic_activity \
21731 ▷ ▷ ▷ -eigenvalues \
21732 ▷ ▷ -end
21733 ▷ pdflatex collinearity_graph_eigenvalues.tex
21734 ▷ $(OPEN) collinearity_graph_eigenvalues.pdf
21735
21736
21737
21738
21739 #####
21740 # Section 16.8: Quartic Curves
21741
21742
21743 SECTION_CANONICAL_FORMS_OF_QUARTIC_CURVES:
21744
21745
21746 test_16_8:
21747 ▷ make F_17_edge
21748 ▷ make Edge_curve_17_nauty
21749 ▷ make Edge_curve_17_group
21750
21751
21752
21753 F_17_edge:
21754 ▷ $(ORBITER) -v 3 \
21755 ▷ ▷ -define F -finite_field -q 17 -end \
21756 ▷ ▷ -with F -do -finite_field_activity \
21757 ▷ ▷ ▷ -cheat_sheet_GF -end
21758 ▷ pdflatex GF_17.tex
21759 ▷ $(OPEN) GF_17.pdf
21760
21761
21762
21763
21764
21765 Edge_curve_17_nauty:
21766 ▷ $(ORBITER) -v 3 \
21767 ▷ ▷ -define C -combinatorial_object \
21768 ▷ ▷ ▷ -label Edge_curve_q17 Edge\_curve\_q17 \
21769 ▷ ▷ ▷ -file_of_points Edge_q17.txt \
21770 ▷ ▷ -end \
21771 ▷ ▷ -define F -finite_field -q 17 -end \
21772 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
21773 ▷ ▷ -with C -do \
21774 ▷ ▷ -combinatorial_object_activity \
21775 ▷ ▷ ▷ -canonical_form_PG P \
21776 ▷ ▷ ▷ ▷ -save_ago \
21777 ▷ ▷ ▷ ▷ -save_transversal \

```

```

21778 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
21779 ▷ ▷ ▷ ▷ -end \
21780 ▷ ▷ ▷ -end \
21781 ▷ ▷ -with C -do \
21782 ▷ ▷ -combinatorial_object_activity \
21783 ▷ ▷ ▷ -report \
21784 ▷ ▷ ▷ ▷ -export_flag_orbits \
21785 ▷ ▷ ▷ ▷ -show_TDO \
21786 ▷ ▷ ▷ ▷ -show_TDA \
21787 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
21788 ▷ ▷ ▷ ▷ -export_group_GAP \
21789 ▷ ▷ ▷ -end \
21790 ▷ ▷ -end
21791 ▷ pdflatex Edge_curve.q17_classification.tex
21792 ▷ $(OPEN) Edge_curve.q17_classification.pdf
21793 ▷ $(ORBITER) -v 2 -draw_matrix \
21794 ▷ ▷ -input_csv_file Edge_curve.q17_object0_TDA_flag_orbits.csv \
21795 ▷ ▷ -secondary_input_csv_file Edge_curve.q17_object0_TDA.csv \
21796 ▷ ▷ -box_width 4 -bit_depth 24 \
21797 ▷ ▷ -end
21798 ▷ $(OPEN) Edge_curve.q17_object0_TDA_flag_orbits.draw.bmp
21799
21800 # 9 backtrack nodes total
21801
21802
21803 # aut = 24
21804 # User time: 0.04 of a second, dt=4 tps = 100
21805
21806
21807 # generators for a group of order 24:
21808 #1,0,0,0,13,0,0,0,4,
21809 #1,0,0,0,0,16,0,16,0,
21810 #0,1,16,2,4,4,15,4,4,
21811
21812
21813 Edge_curve.17_group:
21814 ▷ $(ORBITER) -v 3 \
21815 ▷ ▷ -define G -linear_group -PGL 3 17 \
21816 ▷ ▷ -subgroup_by_generators "Stab_Edge" "24" 3 \
21817 ▷ ▷ ▷ "1,0,0,0,13,0,0,0,4, \
21818 ▷ ▷ ▷ 1,0,0,0,0,16,0,16,0, \
21819 ▷ ▷ ▷ 0,1,16,2,4,4,15,4,4" \
21820 ▷ ▷ -end \
21821 ▷ ▷ -with G -do \
21822 ▷ ▷ ▷ -group_theoretic_activity \
21823 ▷ ▷ ▷ ▷ -print_elements.tex \
21824 ▷ ▷ ▷ ▷ -report_group_table \
21825 ▷ ▷ ▷ ▷ -report \
21826 ▷ ▷ -end
21827 ▷ pdflatex PGL_3_17_Subgroup_Stab_Edge_24_report.tex
21828 ▷ $(OPEN) PGL_3_17_Subgroup_Stab_Edge_24_report.pdf
21829
21830
21831
21832
21833
21834
21835 #####
21836 # Chapter 17 - Interfaces

```

```

21837 #####
21838
21839
21840 test_17:
21841 ▷ make test_17_1
21842 ▷ make test_17_2
21843 ▷ make test_17_3
21844 ▷ make test_17_4
21845 ▷ make test_17_5
21846
21847
21848 #####
21849 # Section 17.1: Graphical Output
21850
21851 SECTION_GRAPHICAL_OUTPUT:
21852
21853
21854 test_17_1:
21855 ▷ make PGL_6_2_Wedge_4_2_detached_report.tex
21856 ▷ make schreier_tree_graphical_output
21857 ▷ make Queens_graph_again
21858
21859
21860
21861
21862
21863 PGL_6_2_Wedge_4_2_detached_report.tex:
21864 ▷ $(ORBITER) -v 4 \
21865 ▷ ▷ -define G -linear_group -PGL 4 2 \
21866 ▷ ▷ ▷ -wedge_detached \
21867 ▷ ▷ -end \
21868 ▷ ▷ -with G -do \
21869 ▷ ▷ -group_theoretic_activity \
21870 ▷ ▷ ▷ -report \
21871 ▷ ▷ -end
21872 ▷ pdflatex PGL_6_2_Wedge_4_2_detached_report.tex
21873 ▷ $(OPEN) PGL_6_2_Wedge_4_2_detached_report.pdf
21874
21875
21876 schreier_tree_graphical_output:
21877 ▷ $(ORBITER) -v 4 \
21878 ▷ ▷ -draw_options \
21879 ▷ ▷ ▷ -yout 500000 \
21880 ▷ ▷ ▷ -radius 15 -nodes_empty \
21881 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 \
21882 ▷ ▷ ▷ -embedded \
21883 ▷ ▷ -end \
21884 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
21885 ▷ ▷ -define Orb -orbits -group G \
21886 ▷ ▷ ▷ -on_polynomials 3 \
21887 ▷ ▷ -end \
21888 ▷ ▷ -with Orb -do -orbits_activity \
21889 ▷ ▷ ▷ -export_something "orbit" 6 \
21890 ▷ ▷ -end
21891 ▷ ▷ -with Orb -do -orbits_activity \
21892 ▷ ▷ ▷ -draw_tree 6 \
21893 ▷ ▷ -end
21894 ▷
21895

```



```

21896 Queens_graph_again:
21897 ▷ $(ORBITER) -v 2 \
21898 ▷ ▷ -define G -graph -non_attacking_queens_graph 8 -end \
21899 ▷ ▷ -with G -do \
21900 ▷ ▷ -graph_theoretic_activity -export_csv -end \
21901 ▷ ▷ -with G -do \
21902 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
21903 ▷ ▷ -with G -do \
21904 ▷ ▷ -graph_theoretic_activity -save -end \
21905 ▷ ▷ -with G -do \
21906 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
21907 ▷ ▷ -with G -do \
21908 ▷ ▷ -graph_theoretic_activity -find_cliques \
21909 ▷ ▷ ▷ -target_size 8 -output_file 8queens -end \
21910 ▷ ▷ -end
21911
21912
21913
21914
21915 #####
21916 # Section 17.2: The Povray Interface
21917
21918
21919 SECTION_POVRAY:
21920
21921 test_17_2:
21922 ▷ make cube
21923 ▷ make math261_test
21924 ▷ make plane1
21925 ▷ make plane2
21926 ▷ make analytic_geo_1
21927 ▷ make analytic_geo_1_video
21928 ▷ make monkey
21929 ▷ make Eckardt
21930 ▷ make Eckardt_deform
21931 ▷ make Eckardt_deform_2
21932 ▷ make Clebsch
21933 ▷ make endrass8
21934
21935
21936 cube:
21937 ▷ $(ORBITER) -v 2 -povray \
21938 ▷ ▷ -round 0 -nb_frames_default 30 \
21939 ▷ ▷ -output_mask cube_%d_%03d.pov \
21940 ▷ ▷ -video_options -W 1024 -H 768 \
21941 ▷ ▷ -global_picture_scale 0.5 \
21942 ▷ ▷ -default_angle 75 \
21943 ▷ ▷ -clipping_radius 2.7 \
21944 ▷ ▷ -end \
21945 ▷ ▷ -scene_objects \
21946 ▷ ▷ ▷ -obj_file cube_centered.obj \
21947 ▷ ▷ ▷ -edge "0, 1" \
21948 ▷ ▷ ▷ -edge "0, 2" \
21949 ▷ ▷ ▷ -edge "0, 4" \
21950 ▷ ▷ ▷ -edge "1, 3" \
21951 ▷ ▷ ▷ -edge "1, 5" \
21952 ▷ ▷ ▷ -edge "2, 3" \
21953 ▷ ▷ ▷ -edge "2, 6" \
21954 ▷ ▷ ▷ -edge "3, 7" \

```

```

21955 ▷ ▷ ▷ -edge "4, 5" \
21956 ▷ ▷ ▷ -edge "4, 6" \
21957 ▷ ▷ ▷ -edge "5, 7" \
21958 ▷ ▷ ▷ -edge "6, 7" \
21959 ▷ ▷ ▷ -group_of_things.as.interval 0 8 \
21960 ▷ ▷ ▷ -spheres 0 0.3 $(POLISHED_CHROME_WHITE) \
21961 ▷ ▷ ▷ -group_of_things.as.interval 0 6 \
21962 ▷ ▷ ▷ -prisms 1 0.05 $(YELLOW_TRANSPARENT) \
21963 ▷ ▷ ▷ -group_of_things.as.interval 0 12 \
21964 ▷ ▷ ▷ -cylinders 2 0.15 $(COLOR_RED) \
21965 ▷ ▷ -scene_objects_end \
21966 ▷ ▷ -povray_end
21967 ▷ - rm -rf POV
21968 ▷ mkdir POV
21969 ▷ mv cube_0_*.pov POV
21970 ▷ mv makefile_animation POV
21971
21972
21973 math261_test:
21974 ▷ $(ORBITER) -v 2 -povray \
21975 ▷ ▷ -round 0 -nb_frames_default 30 \
21976 ▷ ▷ -output_mask math261_%d_%03d.pov \
21977 ▷ ▷ -video_options -W 1024 -H 768 \
21978 ▷ ▷ -global_picture_scale 0.1 \
21979 ▷ ▷ -default_angle 75 \
21980 ▷ ▷ -clipping_radius 2.7 \
21981 ▷ ▷ -end \
21982 ▷ ▷ -scene_objects \
21983 ▷ ▷ ▷ -point "0,0,0" \
21984 ▷ ▷ ▷ -point "5,0,0" \
21985 ▷ ▷ ▷ -point "0,5,0" \
21986 ▷ ▷ ▷ -point "0,0,5" \
21987 ▷ ▷ ▷ -point "1,2,3" \
21988 ▷ ▷ ▷ -point "4,5,6" \
21989 ▷ ▷ ▷ -point "5,7,9" \
21990 ▷ ▷ ▷ -edge "0,1" \
21991 ▷ ▷ ▷ -edge "0,2" \
21992 ▷ ▷ ▷ -edge "0,3" \
21993 ▷ ▷ ▷ -edge "0,4" \
21994 ▷ ▷ ▷ -edge "0,5" \
21995 ▷ ▷ ▷ -edge "4,6" \
21996 ▷ ▷ ▷ -edge "5,6" \
21997 ▷ ▷ ▷ -face "0,4,6,5" \
21998 ▷ ▷ ▷ -group_of_things.as.interval 0 7 \
21999 ▷ ▷ ▷ -spheres 0 0.1 $(POLISHED_CHROME_WHITE) \
22000 ▷ ▷ ▷ -group_of_things.as.interval 0 7 \
22001 ▷ ▷ ▷ -cylinders 1 0.05 $(COLOR_RED) \
22002 ▷ ▷ ▷ -prisms 2 0.05 $(YELLOW_TRANSPARENT) \
22003 ▷ ▷ ▷ -group_of_things.as.interval 0 1 \
22004 ▷ ▷ -scene_objects_end \
22005 ▷ ▷ -povray_end
22006 ▷ - rm -rf POV
22007 ▷ mkdir POV
22008 ▷ mv math261_0_*.pov POV
22009 ▷ mv makefile_animation POV
22010
22011
22012
22013 plane1:

```

```

22014 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
22015 ▷ $(ORBITER) -v 2 -povray \
22016 ▷ ▷ -round 0 -nb_frames_default 30 \
22017 ▷ ▷ -output_mask plane1_%d_%03d.pov \
22018 ▷ ▷ -video_options -W 1024 -H 768 \
22019 ▷ ▷ -global_picture_scale 0.40 \
22020 ▷ ▷ -default_angle 75 \
22021 ▷ ▷ -clipping_radius 5 -omit_bottom_plane \
22022 ▷ ▷ -camera 0 "0,0,1" "5,5,3" "0,0,0" \
22023 ▷ ▷ -rotate_about_z_axis \
22024 ▷ ▷ -boundary_box \
22025 ▷ ▷ -end \
22026 ▷ ▷ -scene_objects \
22027 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file coordinate_grid.csv \
22028 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
22029 ▷ ▷ ▷ -plane_by_dual_coordinates "0,1,0,0" \
22030 ▷ ▷ ▷ -plane_by_dual_coordinates "1,0,0,0" \
22031 ▷ ▷ ▷ -point "-2.25,0,0" \
22032 ▷ ▷ ▷ -point "0,-1.8,0" \
22033 ▷ ▷ ▷ -point "0,0,9" \
22034 ▷ ▷ ▷ -face "0,1,2,0" \
22035 ▷ ▷ ▷ -group_of_things "0" \
22036 ▷ ▷ ▷ -group_of_things "1" \
22037 ▷ ▷ ▷ -group_of_things "2" \
22038 ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
22039 ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
22040 ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
22041 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
22042 ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
22043 ▷ ▷ ▷ -group_of_things "0" \
22044 ▷ ▷ ▷ -planes 0 $(COLOR_BLUE_SEE_THROUGH) \
22045 ▷ ▷ ▷ -group_of_things "1" \
22046 ▷ ▷ ▷ -group_of_things "2" \
22047 ▷ ▷ ▷ -group_of_things "0" \
22048 ▷ ▷ ▷ -prisms 0 0.05 $(COLOR_YELLOW_THICK) \
22049 ▷ ▷ -scene_objects_end \
22050 ▷ ▷ -povray_end
22051 ▷ - rm -rf POV
22052 ▷ mkdir POV
22053 ▷ mv plane1_0_*.pov POV
22054 ▷ mv makefile_animation POV
22055
22056
22057
22058 plane2:
22059 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
22060 ▷ $(ORBITER) -v 2 -povray \
22061 ▷ ▷ -round 0 -nb_frames_default 30 \
22062 ▷ ▷ -output_mask plane2_%d_%03d.pov \
22063 ▷ ▷ -video_options -W 2560 -H 1920 \
22064 ▷ ▷ -global_picture_scale 0.40 \
22065 ▷ ▷ -default_angle 75 \
22066 ▷ ▷ -clipping_radius 5 -omit_bottom_plane \
22067 ▷ ▷ ▷ -camera 0 "0,0,1" "6,6,2" "0,0,0" \
22068 ▷ ▷ ▷ -rotate_about_z_axis \
22069 ▷ ▷ ▷ -boundary_box \
22070 ▷ ▷ -end \
22071 ▷ ▷ -scene_objects \
22072 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file coordinate_grid.csv \

```

```

22073 ▷ ▷ ▷ -plane.by_dual_coordinates "0,0,1,0" \
22074 ▷ ▷ ▷ -plane.by_dual_coordinates "0,1,0,0" \
22075 ▷ ▷ ▷ -plane.by_dual_coordinates "1,0,0,0" \
22076 ▷ ▷ ▷ -plane.by_dual_coordinates "4,5,-1,9" \
22077 ▷ ▷ ▷ -group.of_things "0" \
22078 ▷ ▷ ▷ -group.of_things "1" \
22079 ▷ ▷ ▷ -group.of_things "2" \
22080 ▷ ▷ ▷ -group.of_things.as.interval 3 39 \
22081 ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
22082 ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
22083 ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
22084 ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
22085 ▷ ▷ ▷ -group.of_things "0" \
22086 ▷ ▷ ▷ -planes 4 $(COLOR_BLUE_SEE_THROUGH) \
22087 ▷ ▷ ▷ -group.of_things "3" \
22088 ▷ ▷ -scene_objects_end \
22089 ▷ ▷ -povray_end
22090 ▷ - rm -rf POV
22091 ▷ mkdir POV
22092 ▷ mv plane2_0*.pov POV
22093 ▷ mv makefile_animation POV
22094
22095
22096
22097
22098 analytic_geo_1:
22099 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
22100 ▷ $(ORBITER) -v 2 -povray \
22101 ▷ ▷ -round 0 -nb_frames_default 30 \
22102 ▷ ▷ -output_mask analytic_geo_1_%d.%03d.pov \
22103 ▷ ▷ -video_options -W 2560 -H 1920 \
22104 ▷ ▷ -global_picture_scale 0.80 \
22105 ▷ ▷ -default_angle 75 \
22106 ▷ ▷ -clipping_radius 5 -omit_bottom_plane \
22107 ▷ ▷ ▷ -camera 0 "0,0,1" "6,6,2" "0,0,0" \
22108 ▷ ▷ ▷ -rotate_about_z_axis \
22109 ▷ ▷ ▷ -boundary_box \
22110 ▷ ▷ -end \
22111 ▷ ▷ -scene_objects \
22112 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file coordinate_grid.csv \
22113 ▷ ▷ ▷ -plane.by_dual_coordinates "0,0,1,0" \
22114 ▷ ▷ ▷ -plane.by_dual_coordinates "0,1,0,0" \
22115 ▷ ▷ ▷ -plane.by_dual_coordinates "1,0,0,0" \
22116 ▷ ▷ ▷ -group.of_things "0" \
22117 ▷ ▷ ▷ -group.of_things "1" \
22118 ▷ ▷ ▷ -group.of_things "2" \
22119 ▷ ▷ ▷ -group.of_things.as.interval 3 39 \
22120 ▷ ▷ ▷ -lines 0 0.05 $(COLOR_RED_SHINY) \
22121 ▷ ▷ ▷ -lines 1 0.05 $(COLOR_GREEN_SHINY) \
22122 ▷ ▷ ▷ -lines 2 0.05 $(COLOR_BLUE_SHINY) \
22123 ▷ ▷ ▷ -lines 3 0.04 $(COLOR_BLACK_SHINY) \
22124 ▷ ▷ ▷ -group.of_things "0" \
22125 ▷ ▷ ▷ -group.of_things "1" \
22126 ▷ ▷ ▷ -group.of_things "2" \
22127 ▷ ▷ ▷ -planes 4 $(COLOR_BLUE_SEE_THROUGH) \
22128 ▷ ▷ ▷ -planes 5 $(COLOR_GREEN_SEE_THROUGH) \
22129 ▷ ▷ ▷ -planes 6 $(COLOR_RED_SEE_THROUGH) \
22130 ▷ ▷ ▷ -point "0,0,0" \
22131 ▷ ▷ ▷ -point "1,0,0" \

```

```

22132 ▷ ▷ ▷ -point "1,2,0" \
22133 ▷ ▷ ▷ -point "1,2,3" \
22134 ▷ ▷ ▷ -edge "84,85" \
22135 ▷ ▷ ▷ -edge "85,86" \
22136 ▷ ▷ ▷ -edge "86,87" \
22137 ▷ ▷ ▷ -edge "84,87" \
22138 ▷ ▷ ▷ -group_of_things "84,85,86" \
22139 ▷ ▷ ▷ -spheres 7 0.1 $(POLISHED_CHROME_WHITE) \
22140 ▷ ▷ ▷ -group_of_things "87" \
22141 ▷ ▷ ▷ -spheres 8 0.10 $(COLOR_YELLOW_SHINY) \
22142 ▷ ▷ ▷ -group_of_things "0,1,2" \
22143 ▷ ▷ ▷ -cylinders 9 0.075 $(POLISHED_CHROME_WHITE) \
22144 ▷ ▷ ▷ -group_of_things "3" \
22145 ▷ ▷ ▷ -cylinders 10 0.075 $(COLOR_YELLOW_SHINY) \
22146 ▷ ▷ -scene_objects_end \
22147 ▷ ▷ -povray_end
22148 ▷ - rm -rf POV
22149 ▷ mkdir POV
22150 ▷ mv analytic-geo.1_0-*.pov POV
22151 ▷ mv makefile_animation POV
22152
22153
22154 analytic-geo.1_video:
22155 ▷ - rm -r FRAMES
22156 ▷ - mkdir FRAMES
22157 ▷ - rm analytic-geo.1.mp4
22158 ▷ $(ORBITER) \
22159 ▷ ▷ -prepare_frames \
22160 ▷ ▷ ▷ -i 0 30 PNG/ANALYTIC_GEO.1/analytic-geo.1_0-%03d.png \
22161 ▷ ▷ ▷ -output_starts_at 0 \
22162 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22163 ▷ ▷ -end
22164 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22165 ▷ -f mp4 -q:v 0 -vcodec mpeg4 analytic-geo.1.mp4
22166
22167 monkey:
22168 ▷ $(ORBITER) -v 2 -povray \
22169 ▷ ▷ -round 0 -nb_frames_default 30 \
22170 ▷ ▷ -output_mask monkey_%d-%03d.pov \
22171 ▷ ▷ -video_options -W 1024 -H 768 \
22172 ▷ ▷ -global_picture_scale 0.8 \
22173 ▷ ▷ -default_angle 75 \
22174 ▷ ▷ -clipping_radius 0.8 \
22175 ▷ ▷ -camera 0 "0,0,1" "1,1,0.5" "0,0,0" \
22176 ▷ ▷ -rotate_about_z_axis \
22177 ▷ ▷ -end \
22178 ▷ ▷ -scene_objects \
22179 ▷ ▷ ▷ -cubic_lex $(MONKEY_SADDLE_CUBIC) \
22180 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
22181 ▷ ▷ ▷ -group_of_things "0" \
22182 ▷ ▷ ▷ -group_of_things "0" \
22183 ▷ ▷ ▷ -cubics 0 $(COLOR_GOLD) \
22184 ▷ ▷ ▷ -planes 1 $(COLOR_BLUE) \
22185 ▷ ▷ -scene_objects_end \
22186 ▷ ▷ -povray_end
22187 ▷ - rm -rf POV
22188 ▷ mkdir POV
22189 ▷ mv monkey_0-*.pov POV
22190 ▷ mv makefile_animation POV

```

```

22191
22192
22193
22194
22195 Eckardt:
22196 ▷ $(ORBITER) -v 2 -povray \
22197 ▷ ▷ -round 0 -nb_frames_default 30 \
22198 ▷ ▷ -output_mask Eckardt.%d.%03d.pov \
22199 ▷ ▷ -video_options -W 1024 -H 768 \
22200 ▷ ▷ -global_picture_scale 0.9 \
22201 ▷ ▷ -default_angle 75 \
22202 ▷ ▷ -clipping_radius 2.4 \
22203 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22204 ▷ ▷ -end \
22205 ▷ ▷ -scene_objects \
22206 ▷ ▷ ▷ -Hilbert_Cohn_Vossen_surface \
22207 ▷ ▷ ▷ -group_of_things "0" \
22208 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR) \
22209 ▷ ▷ ▷ -group_of_things_as_interval 0 6 \
22210 ▷ ▷ ▷ -group_of_things_as_interval 6 6 \
22211 ▷ ▷ ▷ -group_of_things_as_interval_with_exceptions 12 15 \
22212 ▷ ▷ ▷ ▷ "14,19,23" \
22213 ▷ ▷ ▷ -lines 1 0.02 $(COLOR_RED_SHINY) \
22214 ▷ ▷ ▷ -lines 2 0.02 $(COLOR_BLUE_SHINY) \
22215 ▷ ▷ ▷ -lines 3 0.02 $(COLOR_YELLOW_SHINY) \
22216 ▷ ▷ ▷ -label 0 "a1" \
22217 ▷ ▷ ▷ -label 2 "a2" \
22218 ▷ ▷ ▷ -label 4 "a3" \
22219 ▷ ▷ ▷ -label 6 "a4" \
22220 ▷ ▷ ▷ -label 8 "a5" \
22221 ▷ ▷ ▷ -label 10 "a6" \
22222 ▷ ▷ ▷ -label 12 "b1" \
22223 ▷ ▷ ▷ -label 14 "b2" \
22224 ▷ ▷ ▷ -label 16 "b3" \
22225 ▷ ▷ ▷ -label 18 "b4" \
22226 ▷ ▷ ▷ -label 20 "b5" \
22227 ▷ ▷ ▷ -label 22 "b6" \
22228 ▷ ▷ ▷ -label 24 "c12" \
22229 ▷ ▷ ▷ -label 26 "c13" \
22230 ▷ ▷ ▷ -label 30 "c15" \
22231 ▷ ▷ ▷ -label 32 "c16" \
22232 ▷ ▷ ▷ -label 34 "c23" \
22233 ▷ ▷ ▷ -label 36 "c24" \
22234 ▷ ▷ ▷ -label 40 "c26" \
22235 ▷ ▷ ▷ -label 42 "c34" \
22236 ▷ ▷ ▷ -label 44 "c35" \
22237 ▷ ▷ ▷ -label 48 "c45" \
22238 ▷ ▷ ▷ -label 50 "c46" \
22239 ▷ ▷ ▷ -label 52 "c56" \
22240 ▷ ▷ ▷ -group_of_things_as_interval 0 6 \
22241 ▷ ▷ ▷ -texts 4 0.2 0.15 $(COLOR_BLACK_NO_SHADOW) \
22242 ▷ ▷ ▷ -group_of_things_as_interval 6 6 \
22243 ▷ ▷ ▷ -texts 5 0.2 0.15 $(COLOR_BLACK_NO_SHADOW) \
22244 ▷ ▷ ▷ -group_of_things_as_interval 12 12 \
22245 ▷ ▷ ▷ -texts 6 0.2 0.15 $(COLOR_BLACK_NO_SHADOW) \
22246 ▷ ▷ -scene_objects_end \
22247 ▷ ▷ -povray_end
22248 ▷ - rm -rf POV
22249 ▷ mkdir POV

```

```

22250 ▷ mv Eckardt_0*.pov POV
22251 ▷ mv makefile_animation POV
22252
22253
22254 #"-3,2.333,4" * 1.5 = "-4.5,3.5,6"
22255 #M := Matrix([[ -4.5, 3.5, 6], [1, 1, 1]])
22256 #NullSpace(M)
22257 #=0.186080731891197,-0.781539073943026,0.595458342051830
22258 #▷ ▷ -rotate_about_custom_axis "0.186080731891197,-0.781539073943026,0.5954583420
51830" \
22259 #-W 1024 -H 768
22260 #-W 2560 -H 1920
22261 #-W 4096 -H 3072
22262
22263
22264
22265
22266 Eckardt.deform:
22267 ▷ $(ORBITER) -v 2 -povray \
22268 ▷ ▷ -round 0 -nb_frames_default 93 \
22269 ▷ ▷ -output_mask Eckardt.deform.%d.%03d.pov \
22270 ▷ ▷ -video_options -W 1024 -H 768 \
22271 ▷ ▷ -global_picture_scale 0.9 \
22272 ▷ ▷ -default_angle 75 \
22273 ▷ ▷ -clipping_radius 2.4 \
22274 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22275 ▷ ▷ -end \
22276 ▷ ▷ -scene_objects \
22277 ▷ ▷ ▷ -Hilbert.Cohn.Vossen.surface \
22278 ▷ ▷ ▷ -group_of_things "0" \
22279 ▷ ▷ ▷ -deformation_of_cubic_lex 93 1.107148718 1.570796327 0 \
22280 ▷ ▷ ▷ ▷ $(ECKARDT_CUBIC_DEFORM1_LEX) \
22281 ▷ ▷ ▷ ▷ $(ECKARDT_CUBIC_DEFORM2_LEX) \
22282 ▷ ▷ ▷ -group_of_things.as_interval 0 93 \
22283 ▷ ▷ ▷ -group_is_animated 1 \
22284 ▷ ▷ ▷ -cubics 1 $(SURFACE_COLOR_SEETHROUGH) \
22285 ▷ ▷ -scene_objects_end \
22286 ▷ ▷ -povray_end
22287 ▷ - rm -rf POV
22288 ▷ mkdir POV
22289 ▷ mv Eckardt.deform_0*.pov POV
22290 ▷ mv makefile_animation POV
22291
22292
22293
22294
22295
22296 Eckardt.deform.2:
22297 ▷ $(ORBITER) -v 2 -povray \
22298 ▷ ▷ -round 0 -nb_frames_default 30 \
22299 ▷ ▷ -output_mask Eckardt.deform.%d.%03d.pov \
22300 ▷ ▷ -video_options -W 1024 -H 768 \
22301 ▷ ▷ -global_picture_scale 0.9 \
22302 ▷ ▷ -default_angle 75 \
22303 ▷ ▷ -clipping_radius 2.4 \
22304 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22305 ▷ ▷ -end \
22306 ▷ ▷ -scene_objects \
22307 ▷ ▷ ▷ -Hilbert.Cohn.Vossen.surface \

```

```

22308 ▷ ▷ ▷ -group_of_things "0" \
22309 ▷ ▷ ▷ -deformation_of_cubic_lex 93 1.107148718 1.570796327 0 \
22310 ▷ ▷ ▷ ▷ $(ECKARDT_CUBIC_DEFORM1_LEX) \
22311 ▷ ▷ ▷ ▷ $(ECKARDT_CUBIC_DEFORM2_LEX) \
22312 ▷ ▷ ▷ --group_of_things_as_interval 0 93 \
22313 ▷ ▷ ▷ --group_is_animated 1 \
22314 ▷ ▷ ▷ -group_of_things "0" \
22315 ▷ ▷ ▷ -cubics 1 $(SURFACE_COLOR_SEETHROUGH) \
22316 ▷ ▷ ▷ -group_of_things "24" \
22317 ▷ ▷ ▷ -cubics 2 $(COLOR_RED) \
22318 ▷ ▷ ▷ -group_of_things "70" \
22319 ▷ ▷ ▷ -cubics 3 $(COLOR_BLUE) \
22320 ▷ ▷ -scene_objects_end \
22321 ▷ ▷ -povray_end
22322 ▷ - rm -rf POV
22323 ▷ mkdir POV
22324 ▷ mv Eckardt_deform_0_*.pov POV
22325 ▷ mv makefile_animation POV
22326
22327
22328
22329 Clebsch:
22330 ▷ $(ORBITER) -v 2 -povray \
22331 ▷ ▷ -round 0 -nb_frames_default 30 \
22332 ▷ ▷ -output_mask Clebsch.%d.%03d.pov \
22333 ▷ ▷ -video_options -W 1024 -H 768 \
22334 ▷ ▷ -global_picture_scale 0.9 \
22335 ▷ ▷ -default_angle 80 \
22336 ▷ ▷ -clipping_radius 2.4 \
22337 ▷ ▷ -camera 0 "1,1,1" "-4.5,3.5,6" "0,0,0" \
22338 ▷ ▷ -end \
22339 ▷ ▷ -scene_objects \
22340 ▷ ▷ ▷ -Clebsch_surface \
22341 ▷ ▷ ▷ -group_of_things "0" \
22342 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR) \
22343 ▷ ▷ ▷ -group_of_things_as_interval 0 6 \
22344 ▷ ▷ ▷ -group_of_things_as_interval 6 6 \
22345 ▷ ▷ ▷ -group_of_things_as_interval 12 15 \
22346 ▷ ▷ ▷ -lines 1 0.02 $(COLOR_RED_SHINY) \
22347 ▷ ▷ ▷ -lines 2 0.02 $(COLOR_BLUE_SHINY) \
22348 ▷ ▷ ▷ -lines 3 0.02 $(COLOR_YELLOW_SHINY) \
22349 ▷ ▷ ▷ -group_of_things_as_interval 0 12 \
22350 ▷ ▷ ▷ -spheres 4 0.08 $(COLOR_TURQUOISE) \
22351 ▷ ▷ -scene_objects_end \
22352 ▷ ▷ -povray_end
22353 ▷ - rm -rf POV
22354 ▷ mkdir POV
22355 ▷ mv Clebsch_0_*.pov POV
22356 ▷ mv makefile_animation POV
22357
22358
22359
22360 endrass8:
22361 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
22362 ▷ $(ORBITER) -v 2 -povray \
22363 ▷ ▷ -round 0 -nb_frames_default 30 \
22364 ▷ ▷ -output_mask endrass_octic.%d.%03d.pov \
22365 ▷ ▷ -video_options -W 1024 -H 768 \
22366 ▷ ▷ -global_picture_scale 0.75 \

```



```

22367 ▷ ▷ -default_angle 75 \
22368 ▷ ▷ -clipping_radius 3.7 \
22369 ▷ ▷ -no_bottom_plane \
22370 ▷ ▷ -camera 0 "1,1,1" "6,6,3" "0,0,0" \
22371 ▷ ▷ -rotate_about_111 \
22372 ▷ ▷ -end \
22373 ▷ ▷ -scene_objects \
22374 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
22375 ▷ ▷ ▷ ▷ coordinate_grid.csv \
22376 ▷ ▷ ▷ -group_of_things "0" \
22377 ▷ ▷ ▷ -group_of_things "1" \
22378 ▷ ▷ ▷ -group_of_things "2" \
22379 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
22380 ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
22381 ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
22382 ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
22383 ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
22384 ▷ ▷ ▷ -octic_lex_165 $(ENDRASS_OCTIC_LEX_165) \
22385 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
22386 ▷ ▷ ▷ -group_of_things "0" \
22387 ▷ ▷ ▷ -group_of_things "0" \
22388 ▷ ▷ ▷ -octics 4 $(SURFACE_COLOR_SEETHROUGH) \
22389 ▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue transmit 0.5 } } \
22390 finish { diffuse 0.9 phong 1}]" \
22391 ▷ ▷ -scene_objects_end \
22392 ▷ ▷ -povray_end
22393 ▷ - rm -rf POV
22394 ▷ mkdir POV
22395 ▷ mv endrass.octic_0*.pov POV
22396 ▷ mv makefile_animation POV
22397
22398
22399
22400
22401
22402 #####
22403 # Section 17.3: Creating Animations
22404
22405 SECTION_ANIMATIONS:
22406
22407
22408 test_17_3:
22409 ▷ make dode
22410 ▷ make dode_video
22411 ▷ make monkey_video
22412 ▷ make Eckardt_deform_video
22413 ▷ make Eckardt_surface
22414 ▷ make Kummer_surface
22415 ▷ make Kummer_video
22416 ▷ make Beauville_surface
22417 ▷ make Clebsch_up_create_points
22418 ▷ make Clebsch_surface
22419 ▷ make Clebsch_surface_defining_equation
22420 ▷ make Clebsch_surface_defining_equation_and_curves
22421 ▷ make F7_povray
22422 ▷ make F7_video
22423 ▷ make McKean_povray
22424 ▷ make McKean_video
22425

```

```

22426
22427
22428 dode:
22429 ▷ $(ORBITER) -v 2 \
22430 ▷ ▷ -povray \
22431 ▷ ▷ -round 0 -nb_frames_default 30 \
22432 ▷ ▷ -output_mask dode_%d.%03d.pov \
22433 ▷ ▷ -video_options -W 1024 -H 768 \
22434 ▷ ▷ -global_picture_scale 0.50 \
22435 ▷ ▷ -default_angle 45 \
22436 ▷ ▷ -clipping_radius 5 \
22437 ▷ ▷ -camera 0 "1,1,1" "-2,2,4" "0,0,0" \
22438 ▷ ▷ -rotate_about_111 \
22439 ▷ ▷ -end \
22440 ▷ ▷ -scene_objects \
22441 ▷ ▷ ▷ -dodecahedron \
22442 ▷ ▷ ▷ -group_of_things_as_interval 0 20 \
22443 ▷ ▷ ▷ -spheres 0 0.075 $(POLISHED_CHROME_WHITE) \
22444 ▷ ▷ ▷ -group_of_things_as_interval 0 30 \
22445 ▷ ▷ ▷ -cylinders 1 0.05 $(COLOR_RED_SHINY) \
22446 ▷ ▷ ▷ -group_of_things_as_interval 0 12 \
22447 ▷ ▷ ▷ -prisms 2 0.02 $(YELLOW_TRANSPARENT) \
22448 ▷ ▷ -scene_objects_end \
22449 ▷ ▷ -povray_end
22450 ▷ - rm -rf POV
22451 ▷ mkdir POV
22452 ▷ mv dode_0_*.pov POV
22453 ▷ mv makefile_animation POV
22454
22455
22456 dode_video:
22457 ▷ - rm -r FRAMES
22458 ▷ - mkdir FRAMES
22459 ▷ - rm dode.mp4
22460 ▷ $(ORBITER) \
22461 ▷ ▷ -prepare_frames \
22462 ▷ ▷ ▷ -i 0 30 DODE/dode_0_%03d.png \
22463 ▷ ▷ ▷ -output_starts_at 0 \
22464 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22465 ▷ ▷ -end
22466 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22467 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 dode.mp4
22468
22469
22470 monkey_video:
22471 ▷ - rm -r FRAMES
22472 ▷ - mkdir FRAMES
22473 ▷ - rm monkey.mp4
22474 ▷ $(ORBITER) \
22475 ▷ ▷ -prepare_frames \
22476 ▷ ▷ ▷ -i 0 30 monkey_0_%03d.png \
22477 ▷ ▷ ▷ -output_starts_at 0 \
22478 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22479 ▷ ▷ -end
22480 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22481 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 monkey.mp4
22482
22483 Eckardt_deform_video:
22484 ▷ - rm -r FRAMES

```

```

22485 ▷ - mkdir FRAMES
22486 ▷ - rm Eckardt_deform.mp4
22487 ▷ $(ORBITER) \
22488 ▷ ▷ -prepare_frames \
22489 ▷ ▷ ▷ -i 0 93 Eckardt_deform_0/Eckardt_deform_0_%03d.png \
22490 ▷ ▷ ▷ -output_starts_at 0 \
22491 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22492 ▷ ▷ -end
22493 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22494 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 Eckardt_deform.mp4
22495
22496
22497 Eckardt_surface:
22498 ▷ $(ORBITER) -v 2 -povray \
22499 ▷ ▷ -round 0 -nb_frames_default 30 \
22500 ▷ ▷ -output_mask Eckardt_%d_%03d.pov \
22501 ▷ ▷ -video_options -W 1024 -H 768 \
22502 ▷ ▷ -global_picture_scale 0.9 \
22503 ▷ ▷ -default_angle 75 \
22504 ▷ ▷ -clipping_radius 2.4 \
22505 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22506 ▷ ▷ -end \
22507 ▷ ▷ -scene_objects \
22508 ▷ ▷ ▷ -cubic_Goursat "6,3,-15" \
22509 ▷ ▷ ▷ -group_of_things "0" \
22510 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
22511 ▷ ▷ -scene_objects_end \
22512 ▷ ▷ -povray_end
22513 ▷ - rm -rf POV
22514 ▷ mkdir POV
22515 ▷ mv Eckardt_0*.pov POV
22516 ▷ mv makefile_animation POV
22517
22518
22519
22520 Kummer_surface:
22521 ▷ $(ORBITER) -v 2 -povray \
22522 ▷ ▷ -round 0 -nb_frames_default 30 \
22523 ▷ ▷ -output_mask Kummer_%d_%03d.pov \
22524 ▷ ▷ -video_options -W 1024 -H 768 \
22525 ▷ ▷ -global_picture_scale 0.9 \
22526 ▷ ▷ -default_angle 75 \
22527 ▷ ▷ -clipping_radius 2.4 \
22528 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22529 ▷ ▷ -end \
22530 ▷ ▷ -scene_objects \
22531 ▷ ▷ ▷ -quartic_lex_35 $(KUMMER_QUARTIC_LEX_35) \
22532 ▷ ▷ ▷ -group_of_things "0" \
22533 ▷ ▷ ▷ -quartics 0 $(SURFACE_COLOR_SEETHROUGH) \
22534 ▷ ▷ -scene_objects_end \
22535 ▷ ▷ -povray_end
22536 ▷ - rm -rf POV
22537 ▷ mkdir POV
22538 ▷ mv Kummer_0*.pov POV
22539 ▷ mv makefile_animation POV
22540
22541
22542 # Maple:
22543 #Kummer := expand((x0^2 + x1^2 + x2^2 + x3^2)^2 - 3*(x0^4 + x1^4 + x2^4 + x3^4))

```

```

22544
22545
22546 Kummer_video:
22547 ▷ - rm -r FRAMES
22548 ▷ - mkdir FRAMES
22549 ▷ - rm Kummer.mp4
22550 ▷ $(ORBITER) \
22551 ▷ ▷ -prepare_frames \
22552 ▷ ▷ ▷ -i 0 30 KUMMER/Kummer_0_%03d.png \
22553 ▷ ▷ ▷ -output_starts_at 0 \
22554 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22555 ▷ ▷ -end
22556 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22557 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 Kummer.mp4
22558
22559
22560
22561 Beauville_surface:
22562 ▷ $(ORBITER) -v 2 -povray \
22563 ▷ ▷ -round 0 -nb_frames_default 30 \
22564 ▷ ▷ -output_mask Beauville_%d_%03d.pov \
22565 ▷ ▷ -video_options -W 1024 -H 768 \
22566 ▷ ▷ -global_picture_scale 0.3 \
22567 ▷ ▷ -default_angle 75 \
22568 ▷ ▷ -clipping_radius 2.4 \
22569 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22570 ▷ ▷ -end \
22571 ▷ ▷ -scene_objects \
22572 ▷ ▷ ▷ -quintic_lex.56 $(BEAUVILLE_QUINTIC_LEX.56) \
22573 ▷ ▷ ▷ -group_of_things "0" \
22574 ▷ ▷ ▷ -quintics 0 $(SURFACE_COLOR_SEETHROUGH) \
22575 ▷ ▷ -scene_objects_end \
22576 ▷ ▷ -povray_end
22577 ▷ - rm -rf POV
22578 ▷ mkdir POV
22579 ▷ mv Beauville_0_*.pov POV
22580 ▷ mv makefile_animation POV
22581
22582
22583
22584
22585
22586
22587
22588
22589 Clebsch_up_create_points:
22590 ▷ $(ORBITER) -v 2 \
22591 ▷ ▷ -smooth_curve "Clebsch_map_of_circle_to_defininig_eqn_r2" \
22592 ▷ ▷ ▷ 0.07 1000 5 0 $(TWO-PI) \
22593 ▷ ▷ -const a $(CLEBSCH_A) b $(CLEBSCH_B) c $(CLEBSCH_C) d $(CLEBSCH_D) \
22594 ▷ ▷ ▷ t00 $(T00) t01 $(T01) t02 $(T02) t03 $(T03) \
22595 ▷ ▷ ▷ t10 $(T10) t11 $(T11) t12 $(T12) t13 $(T13) \
22596 ▷ ▷ ▷ t20 $(T20) t21 $(T21) t22 $(T22) t23 $(T23) \
22597 ▷ ▷ ▷ t30 $(T30) t31 $(T31) t32 $(T32) t33 $(T33) \
22598 ▷ ▷ ▷ r 2 one 1 m -1 \
22599 ▷ ▷ -const_end \
22600 ▷ ▷ -var t \
22601 ▷ ▷ ▷ c001 c002 c011 c012 \
22602 ▷ ▷ ▷ d001 d011 d012 d112 \

```

```

22603 ▷ ▷ ▷ m002 m012 m022 m122 \
22604 ▷ ▷ ▷ n002 n012 n112 n022 n122 \
22605 ▷ ▷ ▷ y0 y1 y2 \
22606 ▷ ▷ ▷ y001 y002 y011 y012 y022 y112 y122 \
22607 ▷ ▷ ▷ x0 x1 x2 x3 \
22608 ▷ ▷ ▷ -var_end \
22609 ▷ ▷ ▷ -code \
22610 ▷ ▷ ▷ push t cos push r mult store y0 \
22611 ▷ ▷ ▷ push t sin push r mult store y1 \
22612 ▷ ▷ ▷ push one store y2 \
22613 ▷ ▷ ▷ push y0 push y0 push y1 mult mult store y001 \
22614 ▷ ▷ ▷ push y0 push y0 push y2 mult mult store y002 \
22615 ▷ ▷ ▷ push y0 push y1 push y1 mult mult store y011 \
22616 ▷ ▷ ▷ push y0 push y1 push y2 mult mult store y012 \
22617 ▷ ▷ ▷ push y0 push y2 push y2 mult mult store y022 \
22618 ▷ ▷ ▷ push y1 push y1 push y2 mult mult store y112 \
22619 ▷ ▷ ▷ push y1 push y2 push y2 mult mult store y122 \
22620 ▷ ▷ ▷ $(CLEBSCH.CUBICS) \
22621 ▷ ▷ ▷ push c001 push y001 mult \
22622 ▷ ▷ ▷ push c002 push y002 mult add \
22623 ▷ ▷ ▷ push c011 push y011 mult add \
22624 ▷ ▷ ▷ push c012 push y012 mult add \
22625 ▷ ▷ ▷ store x0 \
22626 ▷ ▷ ▷ push d001 push y001 mult \
22627 ▷ ▷ ▷ push d011 push y011 mult add \
22628 ▷ ▷ ▷ push d012 push y012 mult add \
22629 ▷ ▷ ▷ push d112 push y112 mult add \
22630 ▷ ▷ ▷ store x1 \
22631 ▷ ▷ ▷ push m002 push y002 mult \
22632 ▷ ▷ ▷ push m012 push y012 mult add \
22633 ▷ ▷ ▷ push m022 push y022 mult add \
22634 ▷ ▷ ▷ push m122 push y122 mult add \
22635 ▷ ▷ ▷ store x2 \
22636 ▷ ▷ ▷ push n002 push y002 mult \
22637 ▷ ▷ ▷ push n012 push y012 mult add \
22638 ▷ ▷ ▷ push n022 push y022 mult add \
22639 ▷ ▷ ▷ push n112 push y112 mult add \
22640 ▷ ▷ ▷ push n122 push y122 mult add \
22641 ▷ ▷ ▷ store x3 \
22642 ▷ ▷ ▷ push x0 push t00 mult \
22643 ▷ ▷ ▷ push x1 push t10 mult add \
22644 ▷ ▷ ▷ push x2 push t20 mult add \
22645 ▷ ▷ ▷ push x3 push t30 mult add \
22646 ▷ ▷ ▷ return \
22647 ▷ ▷ ▷ push x0 push t01 mult \
22648 ▷ ▷ ▷ push x1 push t11 mult add \
22649 ▷ ▷ ▷ push x2 push t21 mult add \
22650 ▷ ▷ ▷ push x3 push t31 mult add \
22651 ▷ ▷ ▷ return \
22652 ▷ ▷ ▷ push x0 push t02 mult \
22653 ▷ ▷ ▷ push x1 push t12 mult add \
22654 ▷ ▷ ▷ push x2 push t22 mult add \
22655 ▷ ▷ ▷ push x3 push t32 mult add \
22656 ▷ ▷ ▷ return \
22657 ▷ ▷ ▷ push x0 push t03 mult \
22658 ▷ ▷ ▷ push x1 push t13 mult add \
22659 ▷ ▷ ▷ push x2 push t23 mult add \
22660 ▷ ▷ ▷ push x3 push t33 mult add \
22661 ▷ ▷ ▷ return \

```

```

22662 ▷ ▷ -code_end
22663
22664
22665 Clebsch_surface:
22666 ▷ $(ORBITER) -v 2 -povray \
22667 ▷ ▷ -round 0 -nb_frames_default 30 \
22668 ▷ ▷ -output_mask Clebsch.%d.%03d.pov \
22669 ▷ ▷ -video_options -W 1024 -H 768 \
22670 ▷ ▷ -global_picture_scale 0.9 \
22671 ▷ ▷ -default_angle 75 \
22672 ▷ ▷ -clipping_radius 2.4 \
22673 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
22674 ▷ ▷ -end \
22675 ▷ ▷ -scene_objects \
22676 ▷ ▷ ▷ -cubic_orbiter "0,0,0,0,0,-4.236067972,\
22677 0,0,4.236067972,4.236067972,17.94427188,\
22678 -17.94427188,0,0,- 9.472135941,0,0,5.236067971,\
22679 8.472135938,- 27.41640782" \
22680 ▷ ▷ ▷ -group_of_things "0" \
22681 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
22682 ▷ ▷ ▷ -point_list_from_csv_file \
22683 ▷ ▷ ▷ function_Clebsch_map_of_circle_N1000_points.csv \
22684 ▷ ▷ ▷ -group_of_things_as_interval 0 954 \
22685 ▷ ▷ ▷ -spheres 1 0.07 $(COLOR_RED) \
22686 ▷ ▷ -scene_objects_end \
22687 ▷ ▷ -povray_end
22688 ▷ - rm -rf POV
22689 ▷ mkdir POV
22690 ▷ mv Clebsch_0*.pov POV
22691 ▷ mv makefile_animation POV
22692
22693
22694 Clebsch_surface_defining_equation:
22695 ▷ $(ORBITER) -v 2 -povray \
22696 ▷ ▷ -round 0 -nb_frames_default 30 \
22697 ▷ ▷ -output_mask Clebsch.%d.%03d.pov \
22698 ▷ ▷ -video_options -W 1024 -H 768 \
22699 ▷ ▷ -global_picture_scale 0.6 \
22700 ▷ ▷ -default_angle 75 \
22701 ▷ ▷ -clipping_radius 1.6 \
22702 ▷ ▷ -camera 0 "1,1,1" "-2,0,2" "0,0,0" \
22703 ▷ ▷ -end \
22704 ▷ ▷ -scene_objects \
22705 ▷ ▷ ▷ -cubic_orbiter "0,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,2,2,2,2" \
22706 ▷ ▷ ▷ -group_of_things "0" \
22707 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
22708 ▷ ▷ -scene_objects_end \
22709 ▷ ▷ -povray_end
22710 ▷ - rm -rf POV
22711 ▷ mkdir POV
22712 ▷ mv Clebsch_0*.pov POV
22713 ▷ mv makefile_animation POV
22714
22715
22716
22717 Clebsch_surface_defining_equation_and_curves:
22718 ▷ $(ORBITER) -v 2 -povray \
22719 ▷ ▷ -round 0 -nb_frames_default 30 \
22720 ▷ ▷ -output_mask Clebsch_2curves.%d.%03d.pov \

```

```

22721 ▷ ▷ -video_options -W 1024 -H 768 \
22722 ▷ ▷ -global_picture_scale 0.6 \
22723 ▷ ▷ -default_angle 75 \
22724 ▷ ▷ -clipping_radius 1.6 \
22725 ▷ ▷ -camera 0 "1,1,1" "-2,0,2" "0,0,0" \
22726 ▷ ▷ -end \
22727 ▷ ▷ -scene_objects \
22728 ▷ ▷ ▷ -cubic_orbiter "0,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,2,2,2,2" \
22729 ▷ ▷ ▷ -group_of_things "0" \
22730 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
22731 ▷ ▷ ▷ -point_list_from_csv_file \
22732 ▷ ▷ ▷ function_Clebsch_map_of_circle_to_defininig_eqn.N1000.points.csv \
22733 ▷ ▷ ▷ -group_of_things_as_interval 0 656 \
22734 ▷ ▷ ▷ -spheres 1 0.07 $(COLOR_RED) \
22735 ▷ ▷ ▷ -point_list_from_csv_file \
22736 ▷ ▷ ▷ function_Clebsch_map_of_circle_to_defininig_eqn.r2.N1000.points.csv \
22737 ▷ ▷ ▷ -group_of_things_as_interval 656 1042 \
22738 ▷ ▷ ▷ -spheres 2 0.07 "texture{ pigment{ color Blue } }" \
22739 finish { diffuse 0.9 phong 1} }" \
22740 ▷ ▷ -scene_objects_end \
22741 ▷ ▷ -povray_end
22742 ▷ - rm -rf POV
22743 ▷ mkdir POV
22744 ▷ mv Clebsch_2curves_0*.pov POV
22745 ▷ mv makefile_animation POV
22746
22747 #▷ ▷ ▷ -point_list_from_csv_file function_Clebsch_map_of_circle.N1000.points.csv
\
22748 #▷ ▷ ▷ -group_of_things_as_interval 0 954 \
22749 #▷ ▷ ▷ -spheres 1 0.07 "texture{ pigment{ color Red } finish { diffuse 0.9 phong
1} }" \
22750
22751
22752
22753
22754 F7.povray:
22755 ▷ $(ORBITER) -v 2 -povray \
22756 ▷ ▷ -round 0 -nb_frames_default 30 \
22757 ▷ ▷ -output_mask F7.15.lines.%d.%03d.pov \
22758 ▷ ▷ -video_options -W 1024 -H 768 \
22759 ▷ ▷ -global_picture_scale 1.5 \
22760 ▷ ▷ -default_angle 80 \
22761 ▷ ▷ -clipping_radius 4.4 \
22762 ▷ ▷ -omit_bottom_plane \
22763 ▷ ▷ -camera 0 "1,1,1" "-4.5,3.5,6" "0,0,0" \
22764 ▷ ▷ -end \
22765 ▷ ▷ -scene_objects \
22766 ▷ ▷ ▷ -cubic_lex "0, 0, 6, 0, 0, -13.39014946, -3.341901346, -6.972931640, 5.8271
82718, 0, 0, 7.390149464, 7.390149464, 6.972931640, -1.512349728, -8.485281372, 0
, 0, 0, 0" \
22767 ▷ ▷ ▷ -group_of_things "0" \
22768 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
22769 ▷ ▷ ▷ -line_through_point_with_direction "0, 0, 0, 1, 0, 0" \
22770 ▷ ▷ ▷ -line_through_point_with_direction "0, 0, -1, 0, 1, 0" \
22771 ▷ ▷ ▷ -line_through_point_with_direction "0, 0, 0, 0, 0, -1" \
22772 ▷ ▷ ▷ -line_through_point_with_direction "1, 0, 0, 1, 1, 1" \
22773 ▷ ▷ ▷ -line_through_point_with_direction "-1.414213562, 0, 0, 4.146264370, 1.7320
50808, 1.732050808" \
22774 ▷ ▷ ▷ -line_through_point_with_direction "0, 1.732050808, -1, 2.414213562, -0.317

```

```

837246, 2.414213562" \
22775 ▷ ▷ ▷ -line_through_point_with_direction "-2.133352390, 0, -1, 1.674708020, 1, 0"
\
22776 ▷ ▷ ▷ -line_through_point_with_direction "-2.539058015, 0, 0, 2.211360755, 1, 0"
\
22777 ▷ ▷ ▷ -line_through_point_with_direction "0, 1.148188060, 0, 0, -0.9435440612, 1"
\
22778 ▷ ▷ ▷ -line_through_point_with_direction "-0.9711971171, 0, 0, 1.162155272, 0, 1"
\
22779 ▷ ▷ ▷ -line_through_point_with_direction "2.096037870, 2.096037870, 0, -1.0658519
05, -1.065851905, 1" \
22780 ▷ ▷ ▷ -line_through_point_with_direction "3.921555783, 2.921555781, 0, -1.7224565
85, -1.722456585, 1" \
22781 ▷ ▷ ▷ -group_of_things_as_interval 0 12 \
22782 ▷ ▷ ▷ -lines 1 0.04 $(COLOR_YELLOW) \
22783 ▷ ▷ -scene_objects_end \
22784 ▷ ▷ -povray_end
22785 ▷ - rm -rf POV
22786 ▷ mkdir POV
22787 ▷ mv F7_15_lines_0*.pov POV
22788 ▷ mv makefile_animation POV
22789
22790
22791
22792
22793
22794
22795 F7.video:
22796 ▷ - rm -r FRAMES
22797 ▷ - mkdir FRAMES
22798 ▷ - rm fifteen_with_lines.mp4
22799 ▷ $(ORBITER) \
22800 ▷ ▷ -prepare_frames \
22801 ▷ ▷ ▷ -i 0 30 F7b/F7_15_lines_0_%03d.png \
22802 ▷ ▷ ▷ -output_starts_at 0 \
22803 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22804 ▷ ▷ -end
22805 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22806 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 fifteen_with_lines.mp4
22807
22808
22809 McKean.povray:
22810 ▷ $(ORBITER) -v 2 -povray \
22811 ▷ ▷ -round 0 -nb_frames_default 30 \
22812 ▷ ▷ -output_mask McKean_%d_%03d.pov \
22813 ▷ ▷ -video_options -W 1024 -H 768 \
22814 ▷ ▷ -global_picture_scale 1.5 \
22815 ▷ ▷ -default_angle 80 \
22816 ▷ ▷ -clipping_radius 4.4 \
22817 ▷ ▷ -omit_bottom_plane \
22818 ▷ ▷ -camera 0 "1,1,1" "-4.5,3.5,6" "0,0,0" \
22819 ▷ ▷ -end \
22820 ▷ ▷ -scene_objects \
22821 ▷ ▷ ▷ -cubic_lex "0, 0, 1, 0, 0, -1, -2, 1, \
22822 2, 0, 0, 1, 1,-1, -1, -1, 0, 0, 0" \
22823 ▷ ▷ ▷ -group_of_things "0" \
22824 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
22825 ▷ ▷ -scene_objects_end \
22826 ▷ ▷ -povray_end

```



```

22827 ▷ - rm -rf POV
22828 ▷ mkdir POV
22829 ▷ mv McKean_0_*.pov POV
22830 ▷ mv makefile_animation POV
22831
22832 McKean_video:
22833 ▷ - rm -r FRAMES
22834 ▷ - mkdir FRAMES
22835 ▷ - rm McKean.mp4
22836 ▷ $(ORBITER) \
22837 ▷ -prepare_frames \
22838 ▷ ▷ -i 0 30 MCKEAN/McKean_0_%03d.png \
22839 ▷ ▷ ▷ -output_starts_at 0 \
22840 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
22841 ▷ ▷ -end
22842 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
22843 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 McKean.mp4
22844
22845
22846
22847
22848 #####
22849 # Section 17.4: Continuous Function Plotter
22850
22851
22852
22853 SECTION_CONTINUOUS_FUNCTION_PLOTTER:
22854
22855
22856
22857 test_17_4:
22858 ▷ make lissajous
22859 ▷ make lissajous_plot
22860 ▷ make lissajous_3d
22861 ▷ make lissajous_3d_plot
22862
22863
22864
22865
22866 lissajous:
22867 ▷ $(ORBITER) -v 2 \
22868 ▷ ▷ -smooth_curve "lissajous" 0.07 2000 15 0 18.85 \
22869 ▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
22870 ▷ ▷ -var t -var_end \
22871 ▷ ▷ -code \
22872 ▷ ▷ ▷ push t push a mult push c add sin push r mult return \
22873 ▷ ▷ ▷ push t push b mult sin push r mult return \
22874 ▷ ▷ -code_end \
22875
22876 #function_lissajous_N2000_points.csv
22877
22878 lissajous_plot:
22879 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
22880 ▷ $(ORBITER) -v 2 -povray \
22881 ▷ ▷ -round 0 -nb_frames_default 1 \
22882 ▷ ▷ -output_mask lissajous_%d_%03d.pov \
22883 ▷ ▷ -video_options -W 1024 -H 768 \
22884 ▷ ▷ -global_picture_scale 0.40 \
22885 ▷ ▷ -default_angle 45 \

```

```

22886 ▷ ▷ -clipping_radius 5 \
22887 ▷ ▷ -omit_bottom_plane \
22888 ▷ ▷ -camera 0 "0,-1,0" "0,0,12" "0,0,0" \
22889 ▷ ▷ -rotate_about_z_axis \
22890 ▷ ▷ -end \
22891 ▷ ▷ -scene_objects \
22892 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
22893 ▷ ▷ ▷ coordinate_grid.csv \
22894 ▷ ▷ ▷ -group_of_things "0" \
22895 ▷ ▷ ▷ -group_of_things "1" \
22896 ▷ ▷ ▷ -group_of_things "2" \
22897 ▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
22898 ▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
22899 ▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
22900 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
22901 ▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
22902 ▷ ▷ ▷ -point_list_from_csv_file \
22903 ▷ ▷ ▷ function_lissajous_N2000_points.csv \
22904 ▷ ▷ ▷ -group_of_things_as_interval 0 6524 \
22905 ▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
22906 finish { diffuse 0.9 phong 1 } } \
22907 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
22908 ▷ ▷ ▷ -group_of_things "0" \
22909 ▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue*0.5 \
22910 transmit 0.5 } }" \
22911 ▷ ▷ -scene_objects_end \
22912 ▷ ▷ -povray_end
22913 ▷ - rm -rf POV
22914 ▷ mkdir POV
22915 ▷ mv lissajous_0_*.pov POV
22916 ▷ mv makefile_animation POV
22917
22918 lissajous_3d:
22919 ▷ $(ORBITER) -v 2 \
22920 ▷ ▷ -smooth_curve "lissajous_3d" 0.07 2000 50 0 18.85 \
22921 ▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
22922 ▷ ▷ -var t -var_end \
22923 ▷ ▷ -code \
22924 ▷ ▷ ▷ push t push a mult push c add sin push r mult return \
22925 ▷ ▷ ▷ push t push b mult sin push r mult return \
22926 ▷ ▷ ▷ push t return \
22927 ▷ ▷ -code_end \
22928
22929 lissajous_3d_plot:
22930 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
22931 ▷ $(ORBITER) -v 2 -povray \
22932 ▷ ▷ -round 0 -nb_frames_default 30 \
22933 ▷ ▷ -output_mask lissajous_3d_%d%03d.pov \
22934 ▷ ▷ -video_options -W 1024 -H 768 \
22935 ▷ ▷ -global_picture_scale 0.40 \
22936 ▷ ▷ -default_angle 45 \
22937 ▷ ▷ -clipping_radius 5 \
22938 ▷ ▷ -omit_bottom_plane \
22939 ▷ ▷ -camera 0 "0,0,1" "7,7,5" "0,0,1" \
22940 ▷ ▷ -rotate_about_z_axis \
22941 ▷ ▷ -end \
22942 ▷ ▷ -scene_objects \
22943 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
22944 ▷ ▷ ▷ coordinate_grid.csv \

```

```

22945 ▷ ▷ ▷ -group_of_things "0" \
22946 ▷ ▷ ▷ -group_of_things "1" \
22947 ▷ ▷ ▷ -group_of_things "2" \
22948 ▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
22949 ▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
22950 ▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
22951 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
22952 ▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
22953 ▷ ▷ ▷ -point_list_from_csv_file \
22954 ▷ ▷ ▷ function_lissajous_3d_N2000_points.csv \
22955 ▷ ▷ ▷ -group_of_things_as_interval 0 6538\
22956 ▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
22957 finish { diffuse 0.9 phong 1} }" \
22958 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
22959 ▷ ▷ ▷ -group_of_things "0" \
22960 ▷ ▷ -scene_objects_end \
22961 ▷ ▷ -povray_end
22962 ▷ - rm -rf POV
22963 ▷ mkdir POV
22964 ▷ mv lissajous_3d_0_*.pov POV
22965 ▷ mv makefile_animation POV
22966
22967
22968 #####
22969 # Section 17.5: Gnuplot interface
22970
22971
22972
22973 SECTION.GNUPLOT_INTERFACE:
22974
22975
22976
22977 test_17_5:
22978 ▷ make gnuplot_test_data
22979 ▷ make gnuplot_Eulerfunction_50
22980
22981 gnuplot_test_data:
22982 ▷ echo $(GNUPLOT_TEST_INPUT) >gnuplot_test_data.csv
22983 ▷ $(ORBITER) -v 3 \
22984 ▷ ▷ -gnuplot gnuplot_test_data.csv \
22985 ▷ ▷ "Gnuplot test data" \
22986 ▷ ▷ "Input" \
22987 ▷ ▷ "Output"
22988 ▷ $(OPEN) gnuplot_test_data_gnuplot.png
22989
22990
22991 gnuplot_Eulerfunction_50:
22992 ▷ $(ORBITER) -v 1 -eulerfunction_interval 1 50
22993 ▷ $(ORBITER) -v 3 \
22994 ▷ ▷ -gnuplot table_eulerfunction_1.50.csv \
22995 ▷ ▷ "Euler function" \
22996 ▷ ▷ "Input" \
22997 ▷ ▷ "Output"
22998
22999
23000 #Row,N,PHI,NBPF,NBDPF
23001
23002
23003 #####

```

```

23004 # Chapter 18 - Mathematical Data in Orbiter
23005 #####
23006
23007
23008 test_18:
23009 ▷ make test_18.1
23010 ▷ make test_18.2
23011
23012
23013 #####
23014 # Section 18.1: Mathematical Data on Cubic Surfaces
23015
23016 SECTION.MATHEMATICAL_DATA:
23017
23018
23019 test_18.1:
23020 ▷ make make_table_of_surfaces
23021 ▷ make make_table_of_quartic_curves
23022 ▷ make cubic_surfaces_table.q17
23023 ▷ make quartic_curves_table.q17
23024
23025
23026 make_table_of_surfaces:
23027 ▷ $(ORBITER) -v 3 \
23028 ▷ ▷ -make_table_of_surfaces
23029 ▷ pdflatex surfaces_report.tex
23030 ▷ $(OPEN) surfaces_report.pdf
23031
23032
23033 make_table_of_quartic_curves:
23034 ▷ $(ORBITER) -v 3 \
23035 ▷ ▷ -make_table_of_quartic_curves
23036 ▷ pdflatex quartic_curves_report.tex
23037 ▷ $(OPEN) quartic_curves_report.pdf
23038
23039
23040
23041 cubic_surfaces_table.q17:
23042 ▷ $(ORBITER) -v 3 \
23043 ▷ ▷ -define F -finite_field -q 17 -end \
23044 ▷ ▷ -define P -projective_space \
23045 ▷ ▷ ▷ -n 3 -field F -v 0 \
23046 ▷ ▷ -end \
23047 ▷ ▷ -with P -do \
23048 ▷ ▷ ▷ -projective_space_activity \
23049 ▷ ▷ ▷ ▷ -table_of_cubic_surfaces \
23050 ▷ ▷ -end
23051
23052 quartic_curves_table.q17:
23053 ▷ $(ORBITER) -v 3 \
23054 ▷ ▷ -define F -finite_field -q 17 -end \
23055 ▷ ▷ -define P -projective_space \
23056 ▷ ▷ ▷ -n 2 -field F -v 0 \
23057 ▷ ▷ -end \
23058 ▷ ▷ -with P -do \
23059 ▷ ▷ ▷ -projective_space_activity \
23060 ▷ ▷ ▷ ▷ -table_of_quartic_curves \
23061 ▷ ▷ -end
23062

```

```

23063 # quartic_curves.q17_info.csv
23064
23065
23066
23067 #####
23068 # Section 18.2: Mathematical Data on BLT-Sets
23069
23070 SECTION.MATHEMATICAL_DATA:
23071
23072
23073 test_18_2:
23074 ▷ make blt_set_table.q7
23075 ▷ make blt_set_table.q9
23076 ▷ make blt_set_table.q13
23077 ▷ make blt_set_table
23078 ▷ make blt_set_export_gap
23079
23080
23081
23082 blt_set_table.q7:
23083 ▷ $(ORBITER) -v 3 \
23084 ▷ ▷ -define F -finite_field -q 7 -end \
23085 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
23086 ▷ ▷ -with O -do \
23087 ▷ ▷ ▷ -orthogonal_space_activity \
23088 ▷ ▷ ▷ ▷ -table_of_blt_sets \
23089 ▷ ▷ -end
23090
23091 blt_set_table.q9:
23092 ▷ $(ORBITER) -v 3 \
23093 ▷ ▷ -define F -finite_field -q 9 -end \
23094 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
23095 ▷ ▷ -with O -do \
23096 ▷ ▷ ▷ -orthogonal_space_activity \
23097 ▷ ▷ ▷ ▷ -table_of_blt_sets \
23098 ▷ ▷ -end
23099
23100
23101 blt_set_table.q13:
23102 ▷ $(ORBITER) -v 3 \
23103 ▷ ▷ -define F -finite_field -q 13 -end \
23104 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
23105 ▷ ▷ -with O -do \
23106 ▷ ▷ ▷ -orthogonal_space_activity \
23107 ▷ ▷ ▷ ▷ -table_of_blt_sets \
23108 ▷ ▷ -end
23109
23110
23111 blt_set_table:
23112 ▷ $(ORBITER) -v 30 \
23113 ▷ ▷ -define Q -vector -dense "3,5,7,9,11,13,17,19,23,25,27,29,31" -end \
23114 ▷ ▷ -define NB -vector -dense "1,2,2,3,4,3,6,5,9,6,6,9,8" -end \
23115 ▷ ▷ -loop_over i Q \
23116 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
23117 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -end \
23118 ▷ ▷ ▷ -with O%i -do \
23119 ▷ ▷ ▷ ▷ -orthogonal_space_activity \
23120 ▷ ▷ ▷ ▷ ▷ -table_of_blt_sets \
23121 ▷ ▷ ▷ ▷ -end \

```

```

23122 ▷ ▷ ▷ -end \
23123 ▷ ▷ -end_loop_over i \
23124 ▷ ▷ -print_symbols
23125
23126
23127 blt_set_export_gap:
23128 ▷ $(ORBITER) -v 30 \
23129 ▷ ▷ -define Q -vector -dense \
23130 ▷ ▷ ▷ $(BLT_ORDER_Q) \
23131 ▷ ▷ -end \
23132 ▷ ▷ -define NB -vector -dense \
23133 ▷ ▷ ▷ $(BLT_NUMBER_ISO) \
23134 ▷ ▷ -end \
23135 ▷ ▷ -loop_over i Q \
23136 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
23137 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -end \
23138 ▷ ▷ ▷ -loop j 0 %i[NB] 1 \
23139 ▷ ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
23140 ▷ ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
23141 ▷ ▷ ▷ ▷ -end \
23142 ▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set_activity \
23143 ▷ ▷ ▷ ▷ ▷ -export_gap \
23144 ▷ ▷ ▷ ▷ -end \
23145 ▷ ▷ ▷ -end_loop j \
23146 ▷ ▷ -end_loop_over i \
23147 ▷ ▷ -print_symbols
23148
23149
23150 #####
23151 # Chapter 19 - Miscellaneous
23152 #####
23153
23154
23155 test_19:
23156 ▷ make test_19_1
23157 ▷ make test_19_2
23158
23159
23160
23161 #####
23162 # Section 19.1: Miscellaneous
23163
23164 SECTION_MISCELLANEOUS:
23165
23166
23167
23168 test_19_1:
23169 ▷ make misc_select
23170 ▷ make misc_join
23171 ▷ make table_mod_12
23172
23173
23174
23175 misc_select:
23176 ▷ $(ORBITER) -v 3 \
23177 ▷ ▷ -define F -finite_field -q 7 -end \
23178 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
23179 ▷ $(ORBITER) -v 4 -csv_file_select_rows_and_cols \
23180 ▷ ▷ GF_q7_multiplication_table_reordered.csv \

```

```

23181 ▷ ▷ "0,2,4" "0,2,4"
23182
23183
23184 misc_join:
23185 ▷ $(ORBITER) -v 4 \
23186 ▷ ▷ -csv_file_join 5 \
23187 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F2.csv Orbit_idx \
23188 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F4.csv Orbit_idx \
23189 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F8.csv Orbit_idx \
23190 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F16.csv Orbit_idx \
23191 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F32.csv Orbit_idx
23192
23193
23194 table_mod_12:
23195 ▷ $(ORBITER) -v 2 \
23196 ▷ ▷ -define M -vector -load_csv_no_border clock_mult_excel.csv -end \
23197 ▷ ▷ -define all_one_r -vector -repeat 1 12 -end \
23198 ▷ ▷ -define all_one_c -vector -repeat 1 12 -end \
23199 ▷ ▷ -draw_matrix \
23200 ▷ ▷ -input_object M \
23201 ▷ ▷ -box_width 50 -bit_depth 24 \
23202 ▷ ▷ ▷ -partition 3 \
23203 ▷ ▷ ▷ ▷ all_one_r all_one_c \
23204 ▷ ▷ -end
23205
23206
23207
23208
23209 #####
23210 # Section 19.2: Limitations
23211
23212 SECTION_LIMITATIONS:
23213
23214 test_19_2:
23215 ▷ make F_15bit
23216 ▷ make F_31bit
23217 ▷ #make F_32bit
23218 ▷ #make F_63bit
23219 ▷ #make F_64bit
23220
23221
23222 # 215 - 19 is prime
23223 # https://primes.utm.edu/lists/2small/0bit.html
23224
23225 F_15bit:
23226 ▷ $(ORBITER) -v 10 \
23227 ▷ ▷ -define F -finite_field -q 32749 -without_tables -end \
23228 ▷ ▷ -define v -vector -field F -allow_negatives \
23229 ▷ ▷ ▷ -dense "-1,-1" -end \
23230 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
23231 ▷
23232
23233 F_31bit:
23234 ▷ $(ORBITER) -v 10 \
23235 ▷ ▷ -define F -finite_field -q 2147483647 -without_tables -end \
23236 ▷ ▷ -define v -vector -field F -allow_negatives \
23237 ▷ ▷ ▷ -dense "-1,-1" -end \
23238 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
23239

```

```

23240
23241 # The next command fail on purpose. The goal is to
23242 # show the limits of the integer domain.
23243
23244 F_32bit:
23245 ▷ $(ORBITER) -v 10 \
23246 ▷ ▷ -define F -finite_field -q 4294967291 -without_tables -end \
23247
23248 # we get an error message. The order of the field is too large for 32 bit int:
23249 #  $2^{32} - 5 = 4294967291$ 
23250
23251
23252 F_63bit:
23253 ▷ $(ORBITER) -v 10 \
23254 ▷ ▷ -define F -finite_field -q 9223372036854775783 -without_tables -end \
23255 ▷ ▷ -define v -vector -field F -allow_negatives \
23256 ▷ ▷ ▷ -dense "-1,-1" -end \
23257 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
23258
23259 #  $2^{63} - 25 = 9223372036854775783$  is prime
23260
23261 F_64bit:
23262 ▷ $(ORBITER) -v 10 \
23263 ▷ ▷ -define F -finite_field -q 18446744073709551557 -without_tables -end \
23264 ▷ ▷ -define v -vector -field F -allow_negatives \
23265 ▷ ▷ ▷ -dense "-1,-1" -end \
23266 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
23267
23268 #  $2^{64} - 59 = 18446744073709551557$  is prime
23269
23270
23271
23272
23273 #####
23274
23275
23276 # unclassified:
23277
23278
23279
23280
23281
23282
23283
23284 extract:
23285 ▷ $(ORBITER) -v 3 \
23286 ▷ ▷ -extract_from_file makefile Cremona_map make_Cremona_map.txt
23287 ▷ ~/bin/a2tex.out -numbers -text_width 80 <make_Cremona_map.txt >make_Cremona_map
    .tex
23288
23289 draw_eigenvalue_diag23:
23290 ▷ $(ORBITER) -v 2 \
23291 ▷ ▷ -draw_options \
23292 ▷ ▷ ▷ -radius 10 \
23293 ▷ ▷ ▷ -line_width 1.5 -embedded \
23294 ▷ ▷ -end \
23295 ▷ ▷ -draw_mod_n -n 20 \
23296 ▷ ▷ ▷ -file ev_diag23 \
23297 ▷ ▷ ▷ -eigenvalues 2 0 0 3 \

```



```
23298 ▷ ▷ -end
23299 ▷ pdflatex ev_diag23_draw.tex
23300 ▷ $(OPEN) ev_diag23_draw.pdf
23301
```



# Bibliography

- [1] Abdullah Al-Azemi, Anton Betten, and Sajeeb Roy Chowdhury. A rainbow-clique search algorithm for BLT-sets. In *ICMS 2018—Proceedings of the International Congress on Mathematical Software; James H. Davenport, Manuel Kauers, George Labahn, Josef Urban (ed.)*, pages 71–79. Springer, 2018.
- [2] Awss Al-ogaidi and Anton Betten. Large Arcs in Small Planes, *Congressus Numerantium* 232 (2019), 119–136.
- [3] Johannes André. Über nicht-Desarguessche Ebenen mit transitiver Translationsgruppe. *Math. Z.*, 60:156–186, 1954.
- [4] Laura Bader, Guglielmo Lunardon, and Joseph A. Thas. Derivation of flocks of quadratic cones. *Forum Math.*, 2(2):163–174, 1990.
- [5] John Bamberg, Anton Betten, Philippe Cara, Jan De Beule, Michel Lavrauw, and Max Neunhöffer. *FinInG – Finite Incidence Geometry, Version 1.4.1*, 2018.
- [6] Susan Barwick and Gary Ebert. *Unitals in projective planes*. Springer Monographs in Mathematics. Springer, New York, 2008.
- [7] R. Bayley. <https://www.distanceregular.org>.
- [8] Anton Betten. Twisted tensor product codes. *Des. Codes Cryptogr.*, 47(1-3):191–219, 2008.
- [9] Anton Betten. Rainbow cliques and the classification of small BLT-sets. In *ISSAC 2013—Proceedings of the 38th International Symposium on Symbolic and Algebraic Computation*, pages 53–60. ACM, New York, 2013.
- [10] Anton Betten. The orbiter ecosystem for combinatorial objects. In *ISSAC 2020—Proceedings of the 45th International Symposium on Symbolic and Algebraic Computation*, pages 30–37. ACM, New York, 2020.
- [11] Anton Betten and Dieter Betten. There is no Drake/Larson linear space on 30 points. *J. Combin. Des.*, 18(1):48–70, 2010.
- [12] Anton Betten, Eun Ju Cheon, Seon Jeong Kim, and Tatsuya Maruta. Three families of multiple blocking sets in Desarguesian projective planes of even order. *Des. Codes Cryptogr.*, 68(1-3):49–59, 2013.
- [13] Anton Betten and Fatma Karaoglu. Cubic surfaces over small finite fields. *Des. Codes Cryptogr.*, 87(4):931–953, 2019.
- [14] Anton Betten and Fatma Karaoglu. The Eckardt point configuration of cubic surfaces revisited. *Des. Codes Cryptogr.*, 90(9):2159–2180, 2022.
- [15] Dieter Betten. 4-dimensionale Translationsebenen mit 8-dimensionaler Kollineationsgruppe. *Geometriae Dedicata*, 2:327–339, 1973.
- [16] Wieb Bosma, John Cannon, and Catherine Playoust. The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24(3-4):235–265, 1997. Computational algebra and number theory (London, 1993).

- [17] A. Brouwer. The Cohen-Tits near octagon on 315 points, <https://www.win.tue.nl/~aeb/drg/graphs/HJ315.html>.
- [18] R. H. Bruck and R. C. Bose. The construction of translation planes from projective spaces. *J. Algebra*, 1:85–102, 1964.
- [19] G. Butler. *Fundamental algorithms for permutation groups*, volume 559 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin, 1991.
- [20] Arjeh M. Cohen. Geometries originating from certain distance-regular graphs. In *Finite geometries and designs (Proc. Conf., Chelwood Gate, 1980)*, volume 49 of *London Math. Soc. Lecture Note Ser.*, pages 81–87. Cambridge Univ. Press, Cambridge-New York, 1981.
- [21] Henri Cohen. *A course in computational algebraic number theory*, volume 138 of *Graduate Texts in Mathematics*. Springer-Verlag, Berlin, 1993.
- [22] Jenny Anne Cooley. Cubic Surfaces over Finite Fields, Ph.D. thesis, University of Warwick, 2014.
- [23] Terry Czerwinski and David Oakden. The translation planes of order twenty-five. *J. Combin. Theory Ser. A*, 59(2):193–217, 1992.
- [24] Anne Delandtsheer and Jean Doyen. Most block-transitive  $t$ -designs are point-primitive. *Geom. Dedicata*, 29(3):307–310, 1989.
- [25] L.E. Dickson. Projective classification of cubic surfaces modulo 2, *Ann. of Math.* 16 (1915), 139–157.
- [26] S. Endrass. A projective surface of degree eight with 168 nodes. *J. Algebraic Geom.*, 6(2):325–334, 1997.
- [27] J. Chris Fisher and Joseph A. Thas. Flocks in  $PG(3, q)$ . *Math. Z.*, 169(1):1–11, 1979.
- [28] The GAP Group. *GAP – Groups, Algorithms, and Programming, Version 4.11.1*, 2021. <http://www.gap-system.org>.
- [29] D. G. Glynn and J. W. P. Hirschfeld. On the classification of geometric codes by polynomial functions. *Des. Codes Cryptogr.*, 6(3):189–204, 1995.
- [30] Gnuplot. Gnuplot <http://www.gnuplot.info>.
- [31] R. Hill. On Pellegrino’s 20-caps in  $S_{4,3}$ . In *Combinatorics ’81 (Rome, 1981)*, volume 78 of *North-Holland Math. Stud.*, pages 433–447. North-Holland, Amsterdam, 1983.
- [32] J. W. P. Hirschfeld. del Pezzo surfaces over finite fields. *Tensor (N.S.)*, 37(1):79–84, 1982.
- [33] James W. P. Hirschfeld. The double-six of lines over  $PG(3, 4)$ . *J. Austral. Math. Soc.*, 4:83–89, 1964.
- [34] John D. Hobby. Metapost, a user’s manual. 2010, 2010.
- [35] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. *An introduction to mathematical cryptography*. Undergraduate Texts in Mathematics. Springer, New York, second edition, 2014.
- [36] Derek F. Holt, Bettina Eick, and Eamonn A. O’Brien. *Handbook of computational group theory*. Discrete Mathematics and its Applications (Boca Raton). Chapman & Hall/CRC, Boca Raton, FL, 2005.
- [37] Christopher Jefferson, Markus Pfeiffer, Rebecca Waldecker, and Eliza Jonauskyste. GAP package images. 2019. <https://www.gap-system.org/Packages/images.html>.
- [38] Fatma Karaoglu. The cubic surfaces with twenty-seven lines over finite fields, Ph.D. thesis, University of Sussex, 2018.
- [39] Fatma Karaoglu and Anton Betten. The number of cubic surfaces with 27 lines over a finite field. *J. Algebraic Combin.*, 56(1):43–57, 2022.

- [40] Brian Kernighan and Dennis Ritchie. The C Programming Language. Prentice Hall. 1978 and 1988.
- [41] Earl S. Kramer and Dale M. Mesner.  $t$ -designs on hypergraphs. *Discrete Math.*, 15(3):263–296, 1976.
- [42] Joseph P. S. Kung. The cycle structure of a linear transformation over a finite field. *Linear Algebra Appl.*, 36:141–155, 1981.
- [43] Maska Law. Flocks, generalizes quadrangles and translation planes from BLT-sets, Ph.D. thesis, University of Western Australia, 2003.
- [44] Maska Law and Tim Penttila. Classification of flocks of the quadratic cone over fields of order at most 29. Number suppl., pages S232–S244. 2003. Special issue dedicated to Adriano Barlotti.
- [45] Maska Law and Tim Penttila. Construction of BLT-sets over small fields. *European J. Combin.*, 25(1):1–22, 2004.
- [46] Wen-Ch'ing Winnie Li. Character sums and abelian Ramanujan graphs. *J. Number Theory*, 41(2):199–217, 1992.
- [47] A. Lubotzky, R. Phillips, and P. Sarnak. Ramanujan graphs. *Combinatorica*, 8(3):261–277, 1988.
- [48] Heinz Lüneburg. *On the rational normal form of endomorphisms*. Bibliographisches Institut, Mannheim, 1987. A primer to constructive algebra.
- [49] L. Lunelli and M. Sce. *k-archi completi nei piani proiettivi desarguesiani di rango 8 e 16*. Centro di Calcoli Numerici, Politecnico di Milano, Milan, 1958.
- [50] 2021. Maple 21. Maplesoft, a division of Waterloo Maple Inc., Waterloo, Ontario. 2021.
- [51] Brendan McKay, 1995. Possolve, A program to solve positive integer systems, Australian National University, private communication.
- [52] Brendan McKay, 2020. Nauty and Traces (Version 2.7r1), Australian National University, 2020.
- [53] G. Eric Moorhouse. Projective Planes of Order 16. <http://ericmoorhouse.org/pub/planes16/>.
- [54] G. Eric Moorhouse. Projective Planes of Order 27. <http://ericmoorhouse.org/pub/planes27/>.
- [55] M. L. Narayana Rao and K. Kuppuswamy Rao. A new flag transitive affine plane of order 27. *Proc. Amer. Math. Soc.*, 59(2):337–345, 1976.
- [56] T. Penttila. Regular cyclic BLT-sets. Number 53, pages 167–172. 1998. Combinatorics '98 (Mondello).
- [57] Tim Penttila and Gordon F. Royle. BLT-sets over small fields. *Australas. J. Combin.*, 17:295–307, 1998.
- [58] Wilhelm Plesken. Counting with groups and rings. *J. Reine Angew. Math.*, 334:40–68, 1982.
- [59] 2021. POV-RAY Developers. POV-RAY, Persistence of Vision Raytracer Pty. Ltd. <http://povray.org>, accessed 1/23/2021.
- [60] L. Schläfli, 1858. An attempt to determine the twenty-seven lines upon a surface of the third order and to divide such surfaces into species in reference to the reality of the lines upon the surface, *Quart. J. Math.* **2** (1858), 55–110.
- [61] Bernd Schmalz. Verwendung von Untergruppenleitern zur Bestimmung von Doppelnebenklassen. *Bayreuth. Math. Schr.*, (31):109–143, 1990.
- [62] Ákos Seress. *Permutation group algorithms*, volume 152 of *Cambridge Tracts in Mathematics*. Cambridge University Press, Cambridge, 2003.
- [63] William Stein and David Joyner. SAGE: System for algebra and geometry experimentation. *ACM SIGSAM Bulletin*, 39(2):61–64, 2005. [http://www.sagemath.org/files/sage\\_stein2005.pdf](http://www.sagemath.org/files/sage_stein2005.pdf).

- [64] Ibrahim A. I. Suleiman and Robert A. Wilson. The 2-modular characters of Conway's third group  $\text{Co}_3$ . *J. Symbolic Comput.*, 24(3-4):493–506, 1997.
- [65] Magma system. Magma Calculator <http://magma.maths.usyd.edu.au/calc/>, accessed 11/24/2019.
- [66] Till Tantau. The TikZ and PGF Packages. Manual for Version 3.1.9a – 2022 <https://tikz.dev>.
- [67] D.E. Taylor. *The geometry of the classical groups*, volume 9 of *Sigma Series in Pure Mathematics*. Heldermann Verlag, Berlin, 1992.
- [68] Michael Walker. A class of translation planes. *Geometriae Dedicata*, 5(2):135–146, 1976.
- [69] Wikipedia. Bmp file format. [https://en.wikipedia.org/wiki/BMP\\_file\\_format](https://en.wikipedia.org/wiki/BMP_file_format).
- [70] Robert Wilson, Peter Walsh, Jonathan Tripp, Ibrahim Suleiman, Richard Parker, Simon Norton, Simon Nickerson, Steve Linton, John Bray, and Rachel Abbott, 2019. ATLAS of Finite Group Representations - Version 3, <http://brauer.maths.qmul.ac.uk/Atlas/v3/>, accessed 11/24/2019.